Kalman and Bayesian Filters in Python

Python 卡尔曼滤波与贝叶斯滤波

全书中文整理版

原著：Roger R. Labbe Jr.

14 章正文 · 前言 · 原仓库全部 7 个附录

2026 年 10 月 · 中文参考译文整合与补译

版本、来源与使用说明

本版是非官方中文整理版。正文及附录 E、G 以 pleb631 的中英对照网页为参考，提取中文叙述并对部分术语作统一；前言和附录 A、B、D、H、I 根据所选英文仓库补译。代码、数学公式及原有图表按原书保留，代码注释、变量名、运行输出与图中英文标签保留原文，以便读者复现。参考网页带有自动翻译标记，本版未经过专业出版审校。

“全书”指所选仓库根目录的前言、01—14 章和全部 Appendix 笔记本；辅助实验、动画生成笔记本不作为独立章节重复收录。原仓库没有附录 C、F，本版不虚构这些章节。附录 D、H 仍是作者未完成的草稿；附录 I 只有标题，均如实保留。PDF 为静态阅读版，动画不能像 Jupyter 中那样交互运行。

原作者：Roger R. Labbe Jr.；原作：Kalman and Bayesian Filters in Python。 所选英文仓库：Penghui-Zhang/Kalman-and-Bayesian-Filters-in-Python。 中文参考：pleb631/Kalman-and-Bayesian-Filters-in-Python-chinese（提交 6867aff40a76662af98aa89e20739a9b6a25b3f4）。参考项目及其贡献者的工作在此致谢。本次补译与 PDF 整理使用 AI 辅助完成。

许可说明：原仓库 README 声明书籍为 CC BY 4.0，前言却保留了较早的 CC BY-NC-SA 4.0 声明。本版同时保留来源及历史声明，并以署名—非商业性使用—相同方式共享 4.0 条件提供本次翻译整理。软件代码适用原 MIT 许可证。本版进行了翻译、术语整理及版式调整；不表示原作者或参考译本作者认可这些修改。

阅读建议：先读前言和第 1—4 章，建立预测与更新的直觉，再进入多元、非线性和粒子滤波。使用目录链接可以直接跳转。若发现译文与公式、代码存在冲突，请对照英文原文。

目录

前言

第 1 章：g-h 滤波器

第 2 章：离散贝叶斯滤波

第 3 章：高斯分布

第 4 章：一维卡尔曼滤波器

第 5 章：多元高斯分布

第 6 章：多元卡尔曼滤波器

第 7 章：卡尔曼滤波的数学基础

第 8 章：卡尔曼滤波器的设计

第 9 章：非线性滤波

第 10 章：无迹卡尔曼滤波器

第 11 章：扩展卡尔曼滤波器

第 12 章：粒子滤波

第 13 章：平滑

第 14 章：自适应滤波

附录 A：安装

附录 B：符号与记法

附录 D：H∞ 滤波器

附录 E：集合卡尔曼滤波器

附录 G：非线性卡尔曼滤波器的设计

附录 H：最小二乘滤波器

附录 I：性能的解析评估

前言

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

这是一本介绍卡尔曼滤波与贝叶斯滤波的入门教材。本书使用 Jupyter Notebook 编写，因此你既可以在浏览器中阅读，也可以运行、修改代码，并直接在书中看到结果。还有什么比这更好的学习方式呢?

卡尔曼滤波与贝叶斯滤波

传感器存在噪声。世界上有许多我们希望测量和跟踪的数据与事件，但不能指望传感器提供完美的信息。我的汽车 GPS 会报告海拔；每次经过道路上的同一个位置，读数都会略有不同。用厨房秤给同一个物体称两次重量，也会得到不同的读数。

在简单情况下，解决办法很明显。如果秤的读数只是略有差异，多测几次再取平均值就可以，或者换一台更精确的秤。但如果传感器噪声很大，或者环境让数据采集变得困难，该怎么办？我们可能要跟踪低空飞机，为无人机设计自动驾驶仪，或确保农用拖拉机播种覆盖整片田地。我从事计算机视觉工作，需要跟踪图像中的运动物体，而计算机视觉算法产生的结果往往噪声很大，并不可靠。

本书教你解决这类滤波问题。我会使用许多不同的算法，但它们都建立在*贝叶斯概率*之上。简单地说，贝叶斯概率根据过去的信息判断什么更可能为真。

如果我问你此刻汽车的航向，你大概毫无头绪，只能在 1° 到 360° 之间猜一个数，猜中的概率是 1/360。现在假设我告诉你，两秒前航向是 243°。两秒之内汽车不可能转向太多，因此你能做出准确得多的预测。你正在利用过去的信息，更准确地推断现在或未来的情况。

世界本身也有噪声。这样的预测有助于改善估计，但仍然受噪声影响。我可能刚刚为躲避一只狗而刹车，或者为绕过坑洞而转向。强风和路面结冰也是影响行驶路径的外部因素。在控制领域，我们把它们称为*噪声*，尽管你平时未必这样称呼。

贝叶斯概率远不止这些，但你已经掌握了基本思想：知识是不确定的，我们应根据证据的强弱调整信念。卡尔曼滤波和贝叶斯滤波把关于系统行为的、有噪声且有限的知识，与有噪声且有限的传感器读数结合起来，尽可能准确地估计系统状态。我们的原则是不要丢弃信息。

假设正在跟踪一个物体，传感器报告它突然改变了方向。它真的转向了，还是数据存在噪声？这取决于具体情况。如果目标是喷气式战斗机，我们很可能相信突然机动的报告；如果它是直轨上的货运列车，我们就会降低这一报告的可信度。传感器的精度也会进一步影响判断。我们的信念取决于历史、对被跟踪系统的了解，以及传感器的特性。

鲁道夫·埃米尔·卡尔曼（Rudolf Emil Kálmán）发明了卡尔曼滤波，以数学上最优的方式解决这类问题。它最早用于阿波罗登月任务，此后应用遍及各个领域：飞机、潜艇、巡航导弹，华尔街的市场跟踪，机器人、物联网传感器和实验室仪器；化工厂用它控制和监测反应，医学成像和心脏信号去噪也用得到它。只要涉及传感器或时间序列数据，通常就会涉及卡尔曼滤波或其近亲。

本书的写作动机

我是一名软件工程师，在航空航天领域工作了近二十年，因此总是与卡尔曼滤波打交道，却一直没有亲手实现过。它素以难学著称。理论很优美，但如果没有信号处理、控制理论、概率统计以及制导与控制等方面的扎实训练，就很难掌握。后来我开始用计算机视觉解决跟踪问题，自己实现滤波器的需要变得迫切。

这个领域有许多优秀教材，例如 Grewal 和 Andrews 的《Kalman Filtering》。但缺少必要背景时，坐下来阅读这些书会令人沮丧。通常，开头几章就匆匆扫过本科数年的数学内容，轻描淡写地让你去查伊藤微积分教材，并用短短几段介绍一学期的统计学。它们适合高年级本科生或研究生，也是研究人员和专业人士的重要参考资料，但普通读者读起来十分吃力。符号未作解释就被引入，不同教材为同一概念使用不同术语和变量名，例子和完整解题过程又很少。我常常能够读懂文字、理解定义中的数学，却不知道这些文字和数学试图描述什么现实现象。我反复问自己：“但这到底*意味着什么*？”下面就是曾经困扰我的典型例子：

$$\begin{aligned}\widehat{x}_k &= \Phi_k\widehat{x}_{k-1} + G_k u_{k-1} + K_k\left[z_k - H\Phi_k\widehat{x}_{k-1} - HG_k u_{k-1}\right] \\ \mathbf{P}_{k\mid k} &= \left(I - \mathbf{K}_k\mathbf{H}_k\right)\operatorname{cov}\left(\mathbf{x}_k - \widehat{\mathbf{x}}_{k\mid k-1}\right)\left(I - \mathbf{K}_k\mathbf{H}_k\right)^{\mathrm{T}} + \mathbf{K}_k\operatorname{cov}\left(\mathbf{v}_k\right)\mathbf{K}_k^{\mathrm{T}}\end{aligned}$$

然而，当我终于开始理解卡尔曼滤波时，发现其基本概念其实相当直接。只要掌握几个简单的概率规则，并对如何融合不确定知识有一些直觉，就能够理解它。卡尔曼滤波虽然以困难闻名，但去掉大量形式化术语后，这门学科及其数学之美逐渐显现，我也爱上了这个主题。

理解数学和理论之后，又出现了其他困难。书籍或论文会提出某个结论，再画一幅图作为证明，但我并不清楚结论为何成立，或者无法复现图形。我也可能想问：“R=0 时还成立吗？”有的作者提供过于抽象的伪代码，让人看不出如何实现；有的书提供 Matlab 代码，但我没有这个昂贵软件的许可证。许多书在章末列出很有用的习题，想自己实现卡尔曼滤波就必须掌握，却不提供答案。课堂教学中这或许可以接受，对独立读者却很糟糕。我不喜欢作者为了防止学生“作弊”而保留信息。

这些都会妨碍学习。我想跟踪屏幕上的图像，或为 Arduino 项目编程；想知道书中的图怎样绘制，并尝试作者没有选择的参数；想运行仿真，向信号加入更多噪声，看看滤波器的表现。日常编程有成千上万种使用卡尔曼滤波的机会，但这样一个相当直观的主题，却仿佛成了火箭科学家和学者的专属领域。

我写这本书，就是为了满足这些需要。如果你要设计军用雷达，这本书不能作为唯一教材；你需要在优秀的理工科院校获得硕士或博士层次的训练。本书面向爱好者、好奇的读者，以及需要对数据进行滤波或平滑的工程师。对爱好者来说，它应当提供所需的全部基础；如果希望深入研究卡尔曼滤波，还需要更多知识。我的目标是介绍足够的概念与数学，让你能够读懂专业教材和论文。

本书具有交互性。虽然可以把它当作静态网页阅读，我仍强烈建议按设计的方式使用它。Jupyter Notebook 让我能够把文字、数学、Python 和程序输出放在同一处。本书每幅图、每项数据，都是由笔记本里的 Python 代码生成的。想把某个参数加倍？改掉参数值，按下 CTRL-ENTER，新的图形或输出就会出现。

本书有习题，也有答案。我信任你。如果只是需要答案，就直接阅读；如果想真正掌握知识，请先尝试完成习题。由于本书可交互，你能直接在书中输入并运行自己的解答，不必切换环境，也不必先导入一大堆东西。

本书免费。我在卡尔曼滤波书籍上花了几千美元，很难相信经济困难地区的读者或拮据的学生能够负担这些开销。我从 Python 这样的免费软件，以及 Allen B. Downey 的免费书籍 [1] 中受益良多，现在该回报了。因此，本书免费，托管在 GitHub 的免费服务器上，并且只使用 IPython、MathJax 等自由开放的软件。

在线阅读

GitHub

本书托管在 GitHub，点击章节名称即可阅读。GitHub 会静态渲染 Jupyter Notebook；你可以阅读全部内容，但不能运行或修改代码。

项目地址：

https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python

binder

binder 在线提供可交互笔记本，无需下载本书或安装 Jupyter，就能在浏览器中运行和修改代码。入口：

http://mybinder.org/repo/rlabbe/Kalman-and-Bayesian-Filters-in-Python

nbviewer

nbviewer 能把任意 Notebook 渲染为静态页面。我认为效果略好于 GitHub，但使用稍麻烦。它直接访问 GitHub，因此会渲染我提交到 GitHub 的内容。

本书入口：

http://nbviewer.ipython.org/github/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/table_of_contents.ipynb

PDF 版本

我会定期把笔记本生成为 PDF，可以从以下地址获取：

https://drive.google.com/file/d/0By_SW19c1BfhSVFzNHc0SjduNzg/view?usp=sharing

下载和运行本书

本书的设计目标是交互式使用，我也建议这样阅读。虽然配置稍费功夫，但值得。安装 IPython 及相关库、克隆本书后，你就能运行所有代码，做实验，观察滤波器如何应对不同数据，以及不同滤波器处理相同数据时的差异。我认为这种即时反馈非常重要，也令人振奋。不必只问“如果这样会怎样”，直接试一试！

安装说明见附录 A：安装。

软件安装后，进入本书所在目录，在命令行运行：

```
jupyter notebook
```

浏览器会打开，显示根目录内容。本书按章组织，各章文件名为 *xx-name*.ipynb，其中 *xx* 是章节编号，.ipynb 是 Notebook 扩展名。阅读第 2 章时，点击对应链接并打开章节笔记本。有时还会有用于生成章节动画等内容的辅助笔记本；它们不要求读者阅读，但好奇时可以查看动画的实现方法。

诚然，用这种界面阅读一本书有些笨拙。我沿用了其他一些把 Jupyter Notebook 用来生成整本书的项目的做法。但这种小小的不便换来的收益很大：你无需下载单独的代码库，也不必一边看书一边在 IDE 中运行，所有文字和代码都在同一处。你可以修改代码，立刻观察结果；发现问题后可以修复并向仓库提交改进。更重要的是，你不会遇到传统图书常见的麻烦——书和代码版本不同步，令人困惑不知该信哪一个。

Jupyter

先介绍一下在本书中使用 Jupyter Notebook 的注意事项。本书是交互式的；要运行示例代码，特别是查看动画图形，就需要执行代码单元。我无法在这里讲完 Jupyter 的全部用法，但有几个地方经常让读者困惑。详细文档见 http://jupyter.org/ 。

首先，一定要运行最上方带有 `#format the book` 注释的代码单元。它就在上方。它不仅设置格式，还会加载必要模块，并设置绘图和打印相关的全局选项。因此，除非只是静态阅读，否则请始终运行这个单元。

```
%matplotlib inline
```

这行代码使图形显示在笔记本内部。Matplotlib 是后面会介绍的绘图库。出于我不明白的原因，Jupyter Notebook 默认可能把图形画在外部窗口中。

`%matplotlib` 的百分号表示 IPython 的*魔法命令*：这些命令让内核执行 Python 语言本身之外的操作。还有许多实用的魔法命令，参见：http://ipython.readthedocs.io/en/stable/interactive/magics.html 。

运行一个单元很简单：点击它使其获得焦点（外面会出现边框），然后按 CTRL-Enter。

其次，必须按顺序执行单元。我会把一个问题拆成多个单元；如果直接跳到第十个代码单元运行，几乎肯定不行。如果尚未运行任何内容，从 **Cell** 菜单选择 *Run All Above*，便可方便地保证前面的代码全部执行。

运行过单元之后，有时可以跳转并以不同顺序重新执行，但并非总可以。我正在努力改善这一点，不过需要权衡。例如，第 10 个单元定义了一个变量，第 11、12 个单元修改它；若随后重新执行第 11 个单元，变量已经是第 12 个单元设置的值，而代码预期的却是第 10 个单元的值。因此乱序执行偶尔会产生奇怪结果。我的建议是往前回退一些，再按顺序运行，恢复正确状态。虽然烦人，交互式 Notebook 的优势足以弥补。更好的做法是在 GitHub 提交问题，让我知道并修复！

最后，有读者报告某些浏览器的动画绘图有问题，我尚无法复现。本书部分位置使用 `%matplotlib notebook` 开启交互绘图。若无法显示，可以改成 `%matplotlib inline`。这会失去动画效果，但似乎能在所有平台和浏览器上工作。

SciPy、NumPy 与 Matplotlib

SciPy 是开源数学软件集合。相关生态包括提供数组、线性代数、随机数等功能的 NumPy，以及绘制 NumPy 数组图形的 Matplotlib。SciPy 的模块与 NumPy 有部分功能重叠，并增加了优化、图像处理等能力。

为了让本书的工作量可控，我假定你已经会使用 Python 编程，并且熟悉这些包。不过，我仍会花一点篇幅展示它们的部分功能；更详细的知识需要查阅其他资料。SciPy 主页 https://scipy.org 是很好的起点，之后你可能希望查找相关教程或视频。

NumPy、SciPy 和 Matplotlib 并不随默认 Python 发行版提供。如果尚未安装，请参阅*安装*附录。

本书会大量使用 NumPy 数组，因此现在先来学习。我会介绍足够的入门知识；若要精通，请阅读 NumPy 文档。

`numpy.array` 实现一维或多维数组。它的类型是 `numpy.ndarray`，下文简称 ndarray。可以用任意类似列表的对象创建数组。下面从列表构建一个一维数组：

```
import numpy as np
x = np.array([1, 2, 3])
print(type(x))
x
```

```
<class 'numpy.ndarray'>
```

```
array([1, 2, 3])
```

使用 `import numpy as np` 已经成为业界惯例。

也可以使用元组：

```
x = np.array((4,5,6))
x
```

```
array([4, 5, 6])
```

用嵌套方括号创建多维数组：

```
x = np.array([[1, 2, 3],
              [4, 5, 6]])
print(x)
```

```
[[1 2 3]
 [4 5 6]]
```

也能创建三维或更高维数组，但这里用不到，因此不再展开。

默认情况下，数组沿用列表元素的数据类型；若存在多种类型，就选择能尽量准确表示全部数值的类型。例如，列表同时含有 `int` 和 `float` 时，数组类型会是 `float`。可以用 `dtype` 参数覆盖默认选择。

```
x = np.array([1, 2, 3], dtype=float)
print(x)
```

```
[1. 2. 3.]
```

使用下标访问数组元素：

```
x = np.array([[1, 2, 3],
              [4, 5, 6]])

print(x[1,2])
```

```
6
```

可以用切片获取一列或一行。下标中的冒号 `:` 表示该行或该列的全部数据。因此，`x[:,0]` 返回第一列的全部数据（0 表示第一列）：

```
x[:, 0]
```

```
array([1, 4])
```

获取第二行：

```
x[1, :]
```

```
array([4, 5, 6])
```

获取第二行的最后两个元素：

```
x[1, 1:]
```

```
array([5, 6])
```

与 Python 的 `list` 一样，负索引可从数组末尾计数，-1 表示最后一个位置。因此，获取第二行（也就是最后一行）最后两个元素的另一种方式是：

```
x[-1, -2:]
```

```
array([5, 6])
```

矩阵加法可以使用 + 运算符；矩阵乘法则需要 dot 方法或函数。* 运算符执行逐元素相乘，**不是**线性代数中的矩阵乘法。

```
x = np.array([[1., 2.],
              [3., 4.]])
print('addition:\n', x + x)
print('\nelement-wise multiplication\n', x * x)
print('\nmultiplication\n', np.dot(x, x))
print('\ndot is also a member of np.array\n', x.dot(x))
```

```
addition:
 [[2. 4.]
 [6. 8.]]

element-wise multiplication
 [[ 1.  4.]
 [ 9. 16.]]

multiplication
 [[ 7. 10.]
 [15. 22.]]

dot is also a member of np.array
 [[ 7. 10.]
 [15. 22.]]
```

Python 3.5 引入了用于矩阵乘法的 @ 运算符。

```
x @ x
```

```
array([[ 7., 10.],
       [15., 22.]])
```

这需要 Python 3.5 或更高版本。本书要求 3.6 以上，所以我会尽量使用它。注意，运算符两侧都必须是数组。因此，`x @ 3.` 会引发 ValueError，而 `np.dot(X, 3.)` 可以正常工作。

用 `.T` 获取转置，用 `numpy.linalg.inv` 求逆。SciPy 也提供求逆函数。

```
import scipy.linalg as linalg
print('transpose\n', x.T)
print('\nNumPy ninverse\n', np.linalg.inv(x))
print('\nSciPy inverse\n', linalg.inv(x))
```

```
transpose
 [[1. 3.]
 [2. 4.]]

NumPy ninverse
 [[-2.   1. ]
 [ 1.5 -0.5]]

SciPy inverse
```

```
 [[-2.   1. ]
 [ 1.5 -0.5]]
```

还有一些辅助函数：`zeros` 创建全零矩阵，`ones` 创建全一矩阵，`eye` 创建单位矩阵。如果要创建多维数组，可以用元组指定形状。

```
print('zeros\n', np.zeros(7))
print('\nzeros(3x2)\n', np.zeros((3, 2)))
print('\neye\n', np.eye(3))
```

```
zeros
 [0. 0. 0. 0. 0. 0. 0.]

zeros(3x2)
 [[0. 0.]
 [0. 0.]
 [0. 0.]]

eye
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

也有创建等间隔数据的函数。`arange` 与 Python 的 `range` 很相似，只是返回 NumPy 数组。`linspace` 略有不同，调用方式是 `linspace(start, stop, num)`，其中 `num` 是所需数组的长度。

```
np.arange(0, 2, 0.1)
```

```
array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1. , 1.1,
       1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9])
```

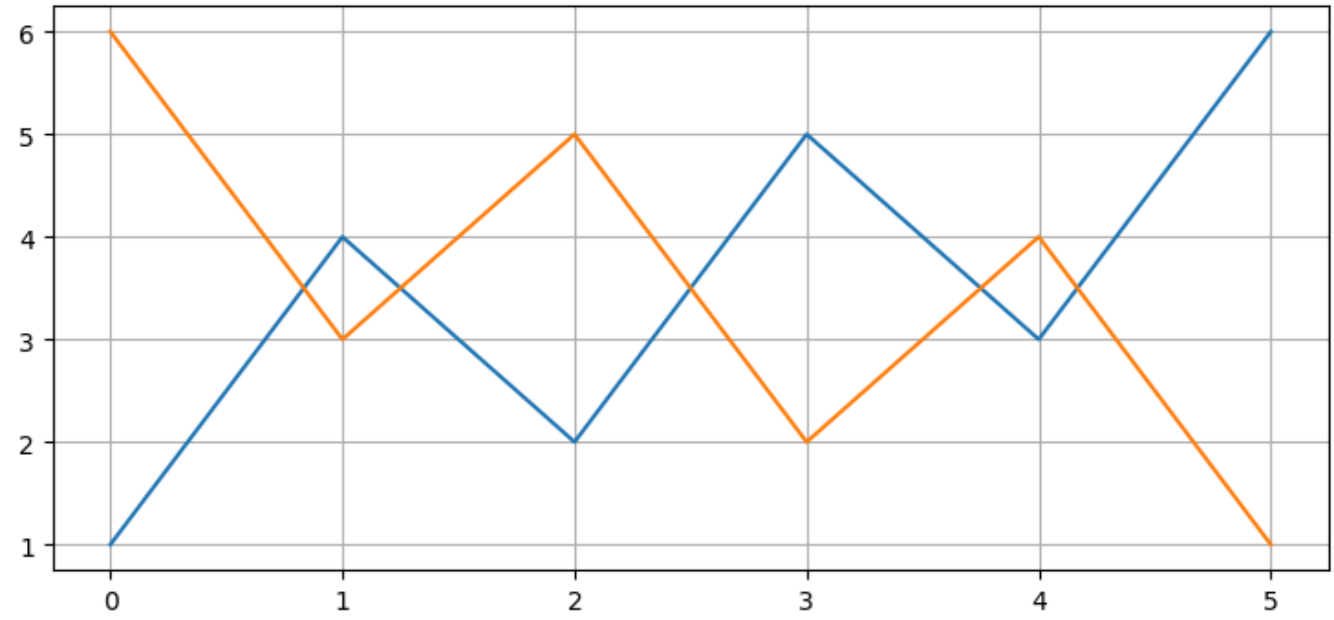
```
np.linspace(0, 2, 20)
```

```
array([0.   , 0.105, 0.211, 0.316, 0.421, 0.526, 0.632, 0.737, 0.842,
       0.947, 1.053, 1.158, 1.263, 1.368, 1.474, 1.579, 1.684, 1.789,
       1.895, 2.   ])
```

现在画一些数据。大多数情况下很简单。Matplotlib 包含名为 `pyplot` 的绘图库，惯例是将它导入为 `plt`。导入后，把数字列表或数组传给 `plt.plot` 即可绘图。多次调用会画出多条数据序列，并使用不同颜色。

```
import matplotlib.pyplot as plt
a = np.array([6, 3, 5, 2, 4, 1])
plt.plot([1, 4, 2, 5, 3, 6])
plt.plot(a)
```

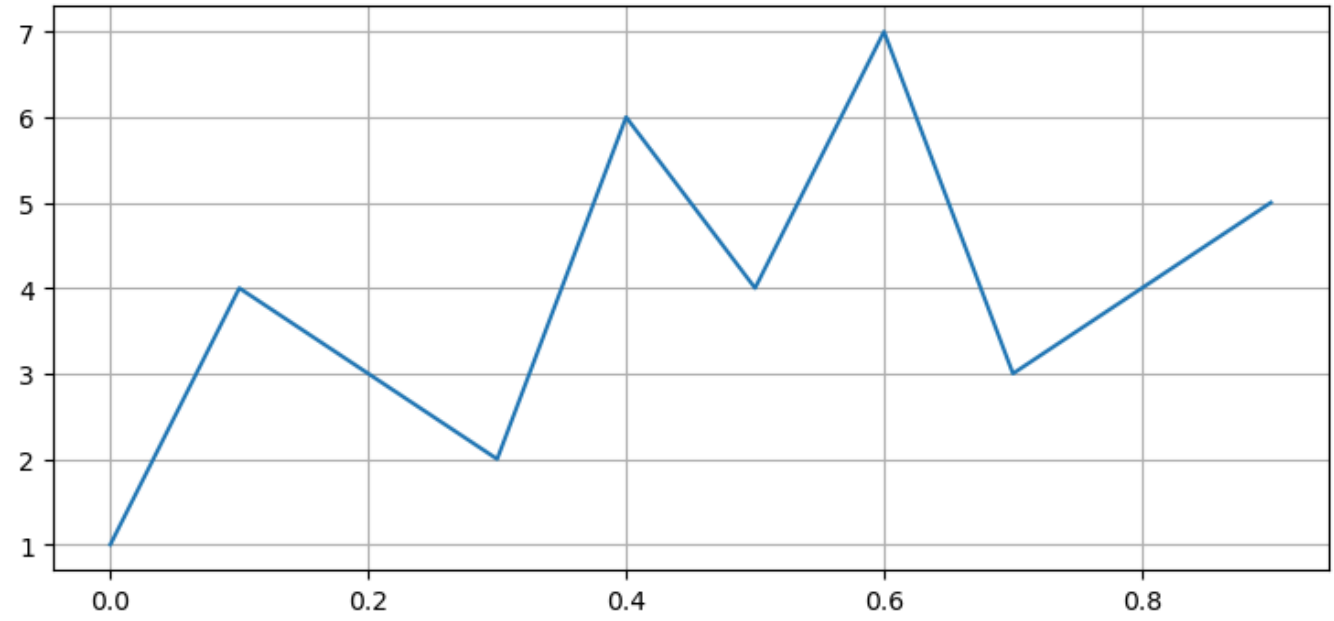
```
[<matplotlib.lines.Line2D at 0x1ecdb844b80>]
```



输出 `[<matplotlib.lines.Line2D at 0x2ba160bed68>]` 是因为 `plt.plot` 返回刚创建的对象。通常不希望显示它，因此我会在最后一条绘图语句后加上分号 `;` 来抑制输出。

默认情况下，`plot` 假定 x 序列每次递增 1。同时传入 x 和 y，就可以使用自己的 x 序列。

```
plt.plot(np.arange(0,1, 0.1), [1,4,3,2,6,4,7,3,4,5]);
```



本书还会使用这些包的许多其他功能。通常我不会逐一解释，而是相信你能够根据上下文理解，或者上网查找说明。与往常一样，如果不确定，就在 Notebook 中新建一个单元，或打开 Python 控制台亲手试验！

练习：创建数组

请创建一个含有 10 个元素的 NumPy 数组，每个元素都等于 1/10。有多种方法，试着实现尽可能多的方案。

```
# your solution
```

解答

下面有三种方法。第一种是我希望你掌握的：用 / 运算符把数组的所有元素除以 10。稍后会用同样的方式把数组中的单位从米换算成千米。

```
print(np.ones(10) / 10.)
print(np.array([.1, .1, .1, .1, .1, .1, .1, .1, .1, .1]))
print(np.array([.1] * 10))
```

```
[0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]
[0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]
[0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]
```

下面还有一种尚未介绍的方法。`numpy.asarray()` 会在参数不是 ndarray 时，将其转换为 ndarray；若已经是 ndarray，则数据不变。这让函数可以方便地同时接收 Python 列表和 ndarray，而当输入本来就是 ndarray 时，无需创建新对象，效率很高。

```
def one_tenth(x):
    x = np.asarray(x)
    return x / 10.

print(one_tenth([1, 2, 3]))            # I work!
print(one_tenth(np.array([4, 5, 6])))  # so do I!
```

```
[0.1 0.2 0.3]
[0.4 0.5 0.6]
```

配套软件

我正在编写一个名为 FilterPy 的开源贝叶斯滤波 Python 库，安装方法见前文。

FilterPy 托管在 GitHub：https://github.com/rlabbe/filterpy ，不过通过 `pip` 安装的版本通常已经足够。

本书专用代码与书放在一起，位于 **/kf_book** 子目录。里面包含书籍格式化代码，以及形如 *xxx*_internal.py 的 Python 文件。这些文件保存对特定章节有用的函数，让我能够把不太值得逐行阅读的代码隐藏起来。例如，生成图表时，我希望你关注图表的内容，而非 Python 绘图的具体步骤。如果对此感兴趣，可以直接阅读源代码。

某些章节引入的函数会在后文继续使用。这些函数最初在 Notebook 中定义，同时也保存在 **/kf_book** 的 Python 文件里，供后续章节按需导入。我会在首次定义时说明这一点，但相关工作仍在推进。我尽量避免这样做，因为目录中的代码可能与书中的代码不同步。不过，Jupyter Notebook 没有提供跨笔记本引用代码单元的方法，这是我目前知道的共享功能的唯一办法。

另有一个没有文档的 **/experiments** 目录，用来编写和测试准备放入书中的代码，其中有一些有趣的内容，可以自由查看。随着本书发展，我计划创建示例和项目，许多材料会移入其中；小型试验则可能删除。如果只是读书，可以放心忽略该目录。

/kf_book 中还有定义本书外观的 CSS 样式文件。IPython Notebook 默认外观较朴素，这方面仍在改进。我借鉴了 Probabilistic Programming and Bayesian Methods for Hackers [2] 等项目，也深受 Lorena Barba 教授优秀工作的影响：CFDPython [3]。本书的外观设计得益于这些项目。

关于 Python 和数学代码的想法

大多数卡尔曼滤波及工程教材由数学家或学者编写。即使提供软件（这并不常见），通常也没有达到生产级质量。以 Paul Zarchan 的《Fundamentals of Kalman Filtering》为例：这是一本值得收藏的好书，也是少数为每个例子和图表提供完整源代码的教材之一。但代码使用 Fortran，除了 `MATMUL` 等函数调用外，几乎没有子程序。全书反复实现卡尔曼滤波器，同一段程序把仿真与滤波混在一起，难以区分。某些章节用略有差异的方式实现同一个滤波器，再用粗体突出改动的几行。如果需要龙格—库塔方法，就把实现直接嵌入代码，甚至没有注释。

有更好的方式。如果我需要龙格—库塔积分，就调用 `ode45`，而不是把实现嵌在自己的代码中。我不想多次实现同一算法并反复调试。若发现错误，只要修复一次，就能确信所有不同项目都得到修复；代码也更易读。大多数时候，我并不关心龙格—库塔算法内部如何实现。

这是一本卡尔曼滤波教材，你也许会说，我们确实关心卡尔曼滤波器的实现。这没错，但真正执行滤波的代码只有大约 10 行，数学部分的实现相当简单。使用卡尔曼滤波器的大部分工作，其实是设计输入数学计算核心的那些矩阵。

潜在的缺点是：滤波方程被封装到函数背后，似乎损害了教学价值。我持相反看法。我希望你学会在真实世界的项目中使用卡尔曼滤波，而不应到处复制粘贴已经成熟的算法。

我使用 Python 类，主要是为了组织滤波器需要的数据，而非实现继承等面向对象特性。例如，`KalmanFilter` 类保存名为 `x`、`P`、`R`、`Q`、`S`、`y`、`K` 的矩阵与向量。我见过过程式的卡尔曼滤波库，要求程序员自己维护这些矩阵。在小程序中也许还好，但实现一组卡尔曼滤波器时，手动管理所有矩阵及相关数据会非常麻烦。我在自己的工作中偶尔也会继承这些类，觉得很方便；不过我知道很多人不喜欢面向对象，因此不想强迫所有人采用它。

许可证

原笔记本本节保留的历史声明：Roger Labbe 的《Kalman Filters and Random Signals in Python》采用 Creative Commons 署名—非商业性使用—相同方式共享 4.0 国际许可协议（CC BY-NC-SA 4.0）。

http://creativecommons.org/licenses/by-nc-sa/4.0/

基于以下项目的工作：

https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python

中文整理说明：当前仓库 README 声明书籍采用 CC BY 4.0，而本节历史文字仍写 CC BY-NC-SA 4.0，两者存在不一致。本版保留该历史声明；为兼顾两者，本中文整理版按署名、非商业性使用、相同方式共享的条件提供，软件代码继续遵循原 MIT 许可证。此说明为本次整理所加，不属于原文。

联系方式

rlabbejr@gmail.com

参考资源

- [1] http://www.greenteapress.com/
- [2] http://nbviewer.ipython.org/github/CamDavidsonPilon/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers/blob/master/Chapter1_Introduction/Chapter1.ipynb
- [3] https://github.com/barbagroup/CFDPython

第 1 章：g-h 滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

在开始之前，请确保您已经了解如何使用 Jupyter Notebooks，并熟悉 SciPy、NumPy 和 Matplotlib 这些软件包。因为本书全程都会用到这些工具。前言部分对这些软件包进行了简要介绍。

通过思维实验培养直觉能力

想象一下，我们生活在一个没有秤的世界里——也就是没有那种用来称体重的仪器。有一天，一位同事跑过来告诉你，她发明了一种“秤”。听完她的介绍后，你立刻站到上面称重。结果显示：“172 磅”。你欣喜若狂——有生以来，你终于知道了自己的体重。更重要的是，一想到可以把这种设备卖给世界各地的减肥中心，你的眼睛里就充满了兴奋的光芒！真是太棒了！

另一位同事听到喧闹声后过来询问发生了什么事。你向他解释了这项发明的原理，然后再次站上体重秤，自豪地宣布了结果：“161 磅。”但随后，你又犹豫了，显得有些困惑。

“几秒钟前还显示 172 磅呢，”你向同事抱怨道。

“我从没说过那是准确的，”她回答道。

传感器的精度并不高。这也是为什么有大量研究致力于解决滤波问题。而解决这个问题，正是本书要探讨的主题。我本可以直接介绍过去半个世纪中人们所提出的各种解决方案，但这些方案都是通过深入探讨我们所知道的事物及其认知方式而得出的。在开始进行数学计算之前，让我们先回顾一下这一探索过程，看看它能否帮助我们更深入地理解滤波的原理。

试试另一个秤吧。

有没有什么办法可以改进这个测量结果呢？最显而易见的办法就是使用精度更高的传感器。不过，你的同事告诉你，她已经制作了 10 台秤，而它们的测量精度大致相同。你让她再拿出一台秤来，你分别用这两台秤称了自己的体重。第一台秤的显示结果是“160 磅”，第二台秤的显示结果是“170 磅”。那么，关于你的体重，我们能得出什么结论呢？

那么，我们有哪些选择呢？

- 我们可以选择只相信 A 的说法，然后将 160 磅作为我们的体重估算值。
- 我们可以选择只相信 B 的说法，那么我们的体重就应该是 170 磅。
- 我们可以选择一个小于 A 和 B 的数字。

- 我们可以选择一个大于 A 和 B 的数。
- 我们可以在 A 和 B 之间选择一个数字。

前两种选择都有一定道理，但我们没有理由偏好其中某一种。我们为什么要选择 A 而不是 B 呢？没有任何理由支持这种选择。第三和第四种选择则是不合理的。虽然这两种测量工具的准确性不高，但完全没有理由选择超出它们所测量范围之外的数值。最后一种选择才是合理的。如果两种测量工具都不准确，那么它们得出的结果很可能高于或低于我的实际体重。在这种情况下，答案很可能介于 A 和 B 之间。

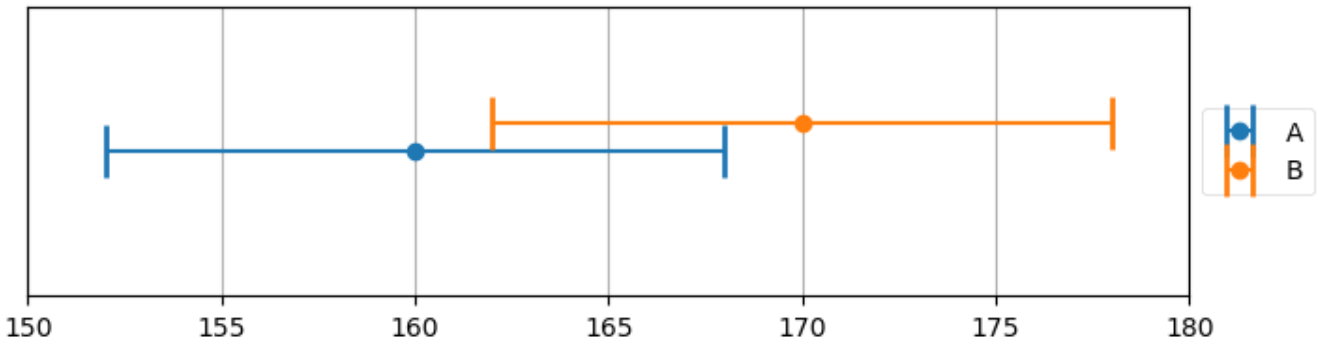
在数学中，这个概念被正式定义为“期望值”，我们稍后会详细探讨。现在，请自己想一想：如果我们进行一百万次测量，通常会出现什么情况呢？有时候，两个秤的读数都会偏低；有时候则都会偏高。而大多数情况下，它们的读数会接近真实重量。如果读数接近真实重量，那么我们当然应该选择 A 和 B 之间的某个数值作为结果。如果读数与真实重量相差太大，我们就无法确定是两者都偏高还是都偏低。不过，只要选择 A 和 B 之间的数值，至少可以减轻最糟糕的测量结果带来的影响。例如，假设我们的真实体重是 180 磅。160 磅的读数显然有较大误差。但如果我们选择 160 磅到 170 磅之间的数值作为结果，那么这个估计值就会比 160 磅更准确。同样的道理也适用于两种秤的读数都高于真实重量的情况。

我们稍后会更正式地探讨这个问题，但目前希望大家都能明白：我们的最佳估算值其实是 A 和 B 的平均值。

$$\frac{160 + 170}{2} = 165$$

我们可以用图表来直观地理解这一点。我根据假设的误差范围 $\pm$ 8 磅，绘制了 A 和 B 的测量值。从图表可以看出，测量值介于 160 磅到 170 磅之间。因此，合理的重量数值必然位于 160 磅到 170 磅的范围内。

```
import kf_book.book_plots as book_plots
from kf_book.book_plots import plot_errorbars
plot_errorbars([(160, 8, 'A'), (170, 8, 'B')], xlims=(150, 180))
```



关于我如何生成这个图表的说明：我从 `kf_book` 子目录中的 book_plots 模块中导入代码来生成图表。生成这个图表需要编写大量的 Python 代码，这些代码相当繁琐，阅读起来也不有趣。在本书中，我经常采用这种方式来生成图表。当相关代码被执行时， `plot_errorbars()` 会被调用，从而将图表插入到书中。

如果您是第一次使用 Jupyter Notebook，那么上面的代码就位于一个单元格中。文本“In [2]::”表示这是一个可以输入代码的单元格。括号中的数字表示该单元格是第二个被执行的单元格。要运行某个单元格，用鼠标点击该单元格使其成为焦点，然后按 CTRL+ENTER 键。在后续操作中，您可以修改单元格中的代码并重新运

行它们。试着把“160”、“170”和“8”这些数值换成其他数字，然后再运行该单元格。根据您输入的数值，输出结果也会相应发生变化。

如果你想查看 plot_errorbars 函数的代码，可以在编辑器中打开该文件，或者新建一个单元格，输入函数名后加上两个问号。按下 Ctrl+Enter 键，浏览器就会打开一个窗口，显示该函数的源代码。这是 Jupyter Notebooks 的独特功能。如果你只想查看该函数的文档，只需在函数名后加上一个问号即可。

```
plot_errorbars??
```

或者

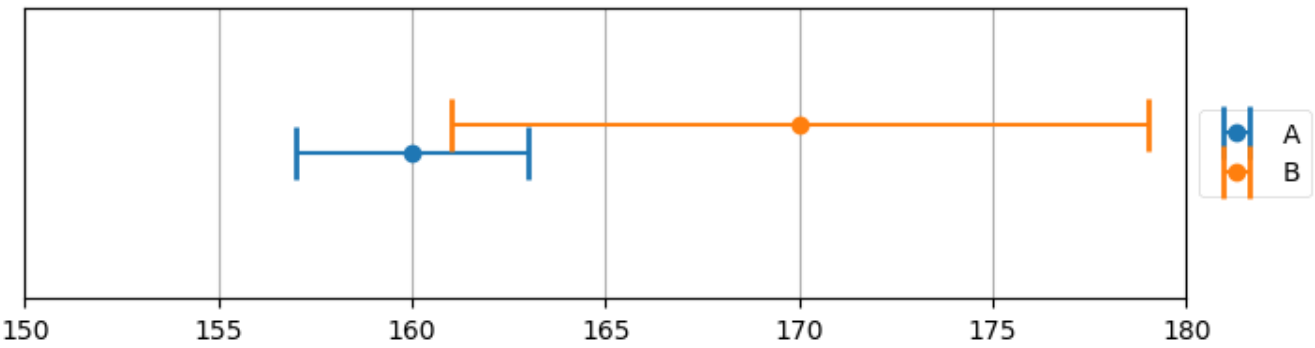
```
plot_errorbars?
```

因此，165 磅似乎是一个合理的估计值。不过，这里还有其他一些信息可以加以利用。只有那些位于 A 和 B 的误差范围之间的数值才是可行的。例如，161 磅是不可能的，因为 B 秤的最大误差为 8 磅，所以它不可能显示出 170 磅的读数。同样，169 磅也是不可能的，因为 A 秤的最大误差为 8 磅，所以它不可能显示出 160 磅的读数。在这个例子中，只有 162 到 168 磅这个范围内的数值才是可行的。

这还不足以让我们得出更准确的数值估计，不过，让我们再假设一下吧。假如现在有人告诉我们，A 的精确度是 B 的三倍，那会怎么样呢？考虑到我们上面列出的 5 种可能性，选择 A 以外的数值仍然没有道理，所以我们就不考虑这些选项了。选择 A 作为估计值似乎更合理——毕竟，我们知道 A 的精确度更高，那为什么不用它来代替 B 呢？B 怎么可能比单独使用 A 能带来更好的结果呢？

答案可能有些违反直觉，但实际上，答案是肯定的，它确实可以。首先，我们再来看一下同样的数据：A=160，B=170。不过，这次 A 的误差为 3 磅，而 B 的误差则是 A 的 3 倍，即 9 磅。

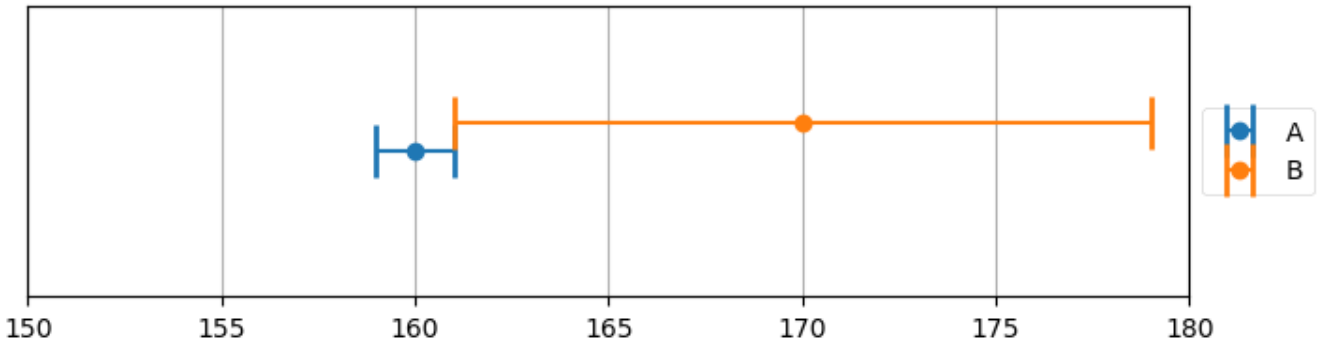
```
plot_errorbars([(160, 3, 'A'), (170, 9, 'B')], xlims=(150, 180))
```



A 和 B 的误差范围之间的重叠区域，才代表了唯一可能的真实重量值。这一重叠区域比 A 单独的误差还要小。更重要的是，从图中可以看出，这一重叠区域并不包括 160 磅或 165 磅这个数值。如果我们只使用 A 的测量结果，因为它比 B 更准确，那么得到的估计值将是 160 磅。而如果我们将 A 和 B 的测量结果取平均值，那么得到的估计值就是 165 磅。但根据我们对秤的准确度的了解，这两种数值都是不可能的。通过同时考虑 A 和 B 的测量结果，我们可以得到 161 磅到 163 磅之间的估计值，这一范围正好是两个误差范围的交集所在。

让我们把这种情况推向极端。假设秤 A 的精确度为 1 磅。也就是说，如果我们真正的体重是 170 磅，那么秤 A 的显示结果可能是 169 磅、170 磅或 171 磅。同时，我们知道秤 B 的精确度为 9 磅。如果我们用这两台秤分别称重，得到的结果是：秤 A 显示 160 磅，秤 B 显示 170 磅。那么，我们应该将自己的体重估计为多少呢？我们可以通过图表来直观地了解这个问题。

```
plot_errorbars([(160, 1, 'A'), (170, 9, 'B')], xlims=(150, 180))
```



从这里可以看出，唯一的合理重量数值就是 161 磅。这是一个非常重要的结果。尽管使用了两个精度不太高的传感器，我们仍然能够得到极其精确的测量结果。

所以，即使两个传感器中有一个的精度较低，但总比只有一个传感器要好。在本书的剩余部分，我都会反复强调这一点。我们绝不能放弃任何信息，无论其质量有多差。我们将开发出相应的数学模型和算法，以便能够利用所有可能的信息来源，从而得出最准确的估算结果。

不过，我们偏离了主题。没有哪个客户会愿意购买多台秤来使用。另外，我们最初假设所有秤的测量精度都差不多。这种“无论精度如何，都把所有测量数据都考虑进来”的思路后来会发挥重要作用，所以请不要忘记这一点。

如果我只有一台秤，但用它来称重很多次会怎么样呢？我们得出结论：如果使用两台精度相同的秤来称重，那么应该将两次测量的结果取平均值。那么，如果我用同一台秤称重 10,000 次呢？我们已经知道，这台秤给出过高或过低的重量数值的概率是相等的。不难证明，多次测量的平均值会非常接近真实的体重。不过，我们现在先来进行一次模拟吧。我会使用 NumPy 来进行计算，NumPy 是 SciPy 生态系统中的数值计算工具。

```
import numpy as np
measurements = np.random.uniform(160, 170, size=10000)
mean = measurements.mean()
print(f'Average of measurements is {mean:.4f}')
```

```
Average of measurements is 164.9353
```

实际打印出来的数字会因随机数生成器的不同而有所差异，但应该会非常接近 165。

这段代码基于一个很可能不成立的假设：当实际重量为 165 磅时，秤的显示值既有可能是 160，也有可能是 165。但实际上，这种情况几乎不会发生。真正的传感器更有可能显示出接近真实值的读数；而当读数与真实值相差越远时，这种可能性就越小。我们将在关于高斯分布的章节中详细讨论这一点。目前，我直接使用

`numpy.random.normal()` 函数即可，该函数会生成更多接近 165 磅的读数，而远离 165 磅的读数则较少。可以认为，该函数的运行方式与真实秤的运作方式类似，即会产生有“噪声”的测量结果。

```
mean = np.random.normal(165, 5, size=10000).mean()
print(f'Average of measurements is {mean:.4f}')
```

```
Average of measurements is 164.9762
```

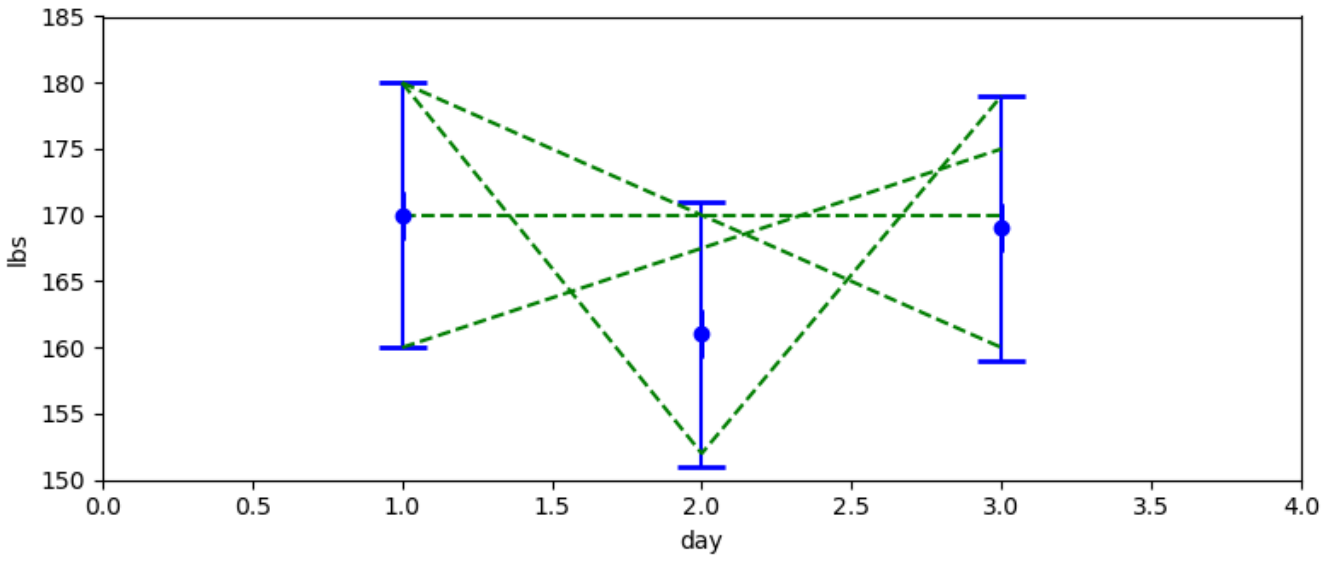
答案仍然非常接近 165。

好吧，太好了，我们找到了解决传感器问题的办法！不过这个办法并不实用。没人有耐心去称一万次，甚至十二万次自己的体重吧。

那么，我们来玩个“假如”的游戏吧。假如你每天测量一次体重，得到的数值分别是 170、161 和 169，那你到底是胖了、瘦了，还是这些数值只是测量上的误差而已呢？

我们真的无法确定。第一次测量结果是 170 磅，最后一次则是 169 磅，这意味着体重减少了 1 磅。不过，如果秤的精确度只有 10 磅的话，这种误差也是可以理解的。实际上，我的体重可能有所增加；也许第一天我的体重是 165 磅，而第三天则上升到了 172 磅。这样的体重变化完全有可能被秤的误差所掩盖。秤显示我的体重在下降，但实际上我的体重却在上升！让我们用图表来仔细看看吧。我把各项测量结果以及误差范围都标了出来，同时用绿色虚线标出了根据这些测量结果可能出现的体重增减情况。

```
import kf_book.gh_internal as gh
gh.plot_hypothesis1()
```

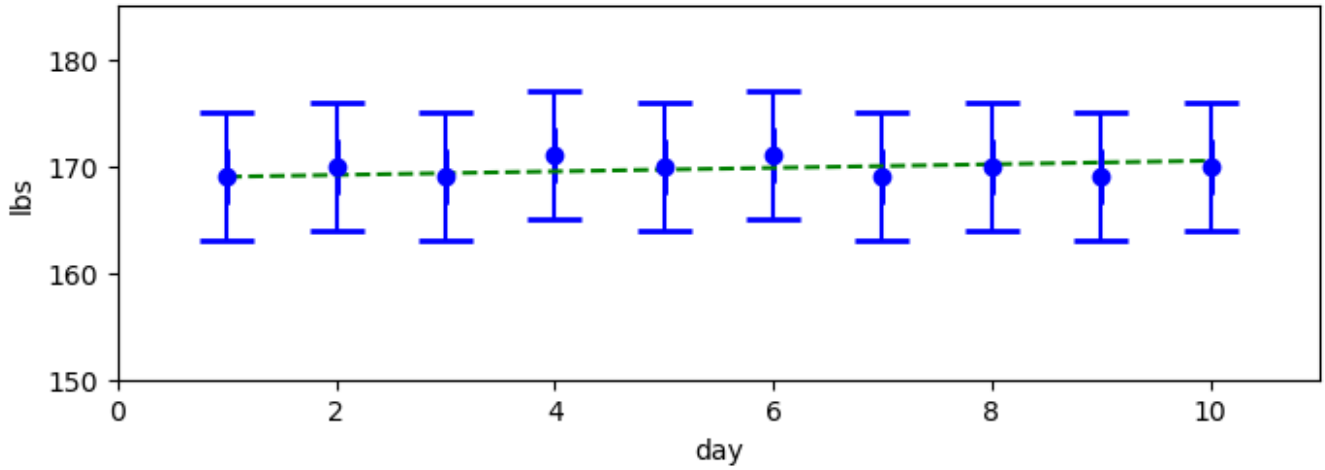


正如我们所看到的，这些测量结果所能解释的体重变化范围其实非常大。实际上，有无数种可能的体重变化情况。那我们是否应该放弃呢？我可不会！请记住，我们这里讨论的是人类的体重。一个人不可能在第一天体重为 180 磅，第三天却只有 160 磅；也不可能一天内减掉 30 磅的体重，第二天又把体重恢复过来（当然，我们假设这个人没有遭受任何肢体截肢或其它伤害）。

我们所测量的物理系统的行为，会影响到我们对测量结果的解读。如果我们每天都在称量同一块石头，那么我们会把所有的波动都视为偶然的误差。但如果我们称量的是一个通过雨水补给、用于家庭用途的蓄水池，那么我们可能会认为这种重量变化是真实存在的。

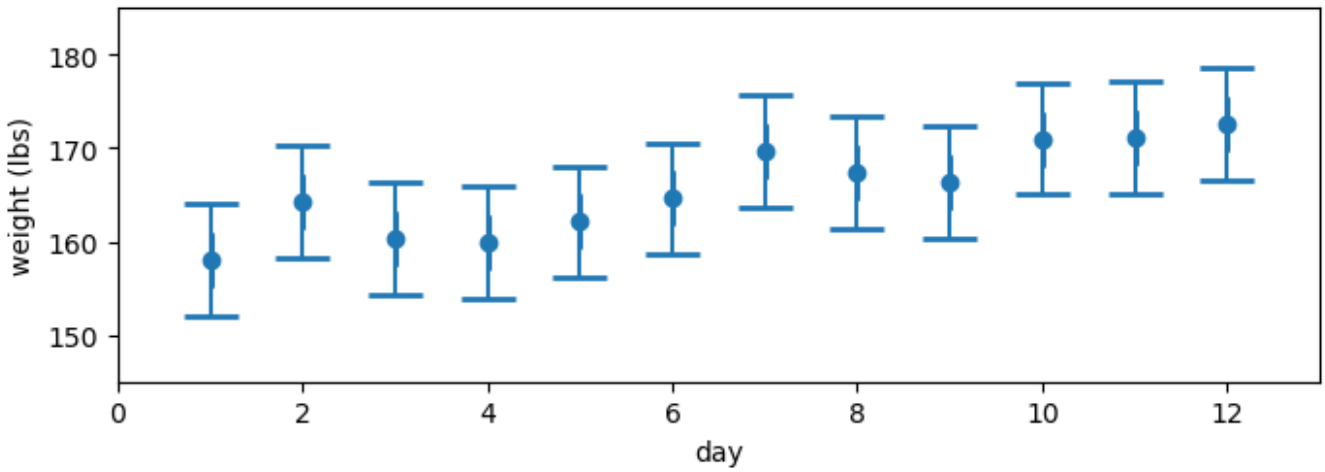
假设我使用不同的测量工具进行测量，得到的数据如下：169、170、169、171、170、171、169、170、169、170。凭直觉来看，这种情况可能意味着什么呢？比如，有可能你每天体重增加 1 磅，但由于测量误差，看起来你的体重没有变化。当然，也有可能你每天体重减少 1 磅，但测量结果仍然显示体重没有变化。不过，这种可能性有多大呢？抛硬币 10 次都正面朝上的概率有多大呢？显然不大。虽然我们无法仅凭这些数据来证明什么，但看起来我的体重很可能保持稳定。在下图中，我用误差条标出了测量结果，同时用虚线绿色线条表示了可能的真实体重。这条虚线并非所谓的“正确答案”，而只是根据测量结果得出的合理推测而已。

```
gh.plot_hypothesis2()
```



再假设一种情况：如果测量结果分别是 158.0、164.2、160.3、159.9、162.1、164.6、169.6、167.4、166.4、171.0 会怎么样呢？让我们先看一下对应的图表，然后再来回答一些问题。

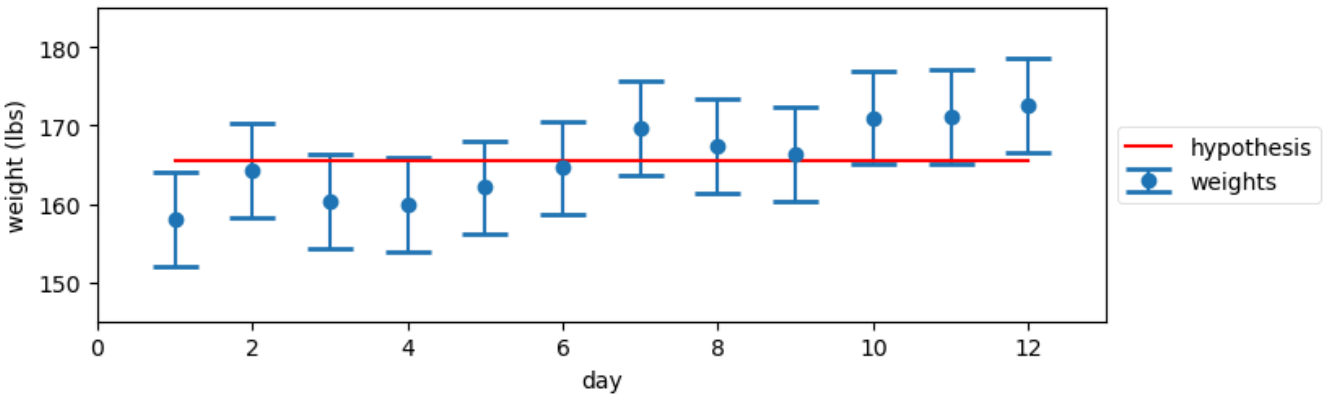
```
gh.plot_hypothesis3()
```



我觉得自己有可能减重了，而这些数据其实只是随机波动而已，这种可能性大吗？其实不大。那我觉得自己的体重可能保持不变，这种可能性呢？同样不大。从数据趋势来看，体重是呈上升趋势的；虽然不是均匀上升，但总体上是上升的。我们无法确定，但看起来像是体重增加了，而且增加的幅度相当可观。让我们用更多的图表来验证这个假设吧。通常来说，通过图表来分析数据比用表格更直观。

那么，让我们来考虑两种假设。首先，假设我们的体重没有发生变化。为了得出这个数值，我们同意应该对各项测量结果进行平均处理。让我们来具体看看吧。

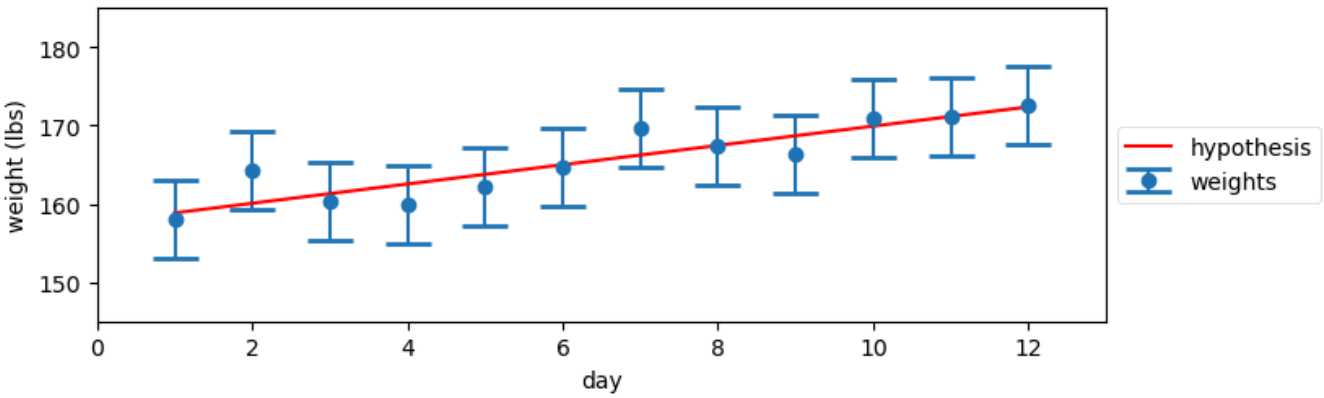
```
gh.plot_hypothesis4()
```



这看起来并不令人信服。事实上，我们可以看到，根本不存在一条能够完全位于所有误差范围之内的水平线。

现在，假设我们的体重增加了。增加了多少呢？我不知道，但 NumPy 知道！我们希望用一条直线来表示这些测量数据，这条直线看起来要“尽可能”符合实际情况。NumPy 有相应的函数可以实现这一点，其原理被称为“最小二乘法”。我们不必太担心具体的计算过程（如果你感兴趣的话，我可以使用 polyfit()函数来处理）。只要把计算结果绘制出来就行了。

```
gh.plot_hypothesis5()
```



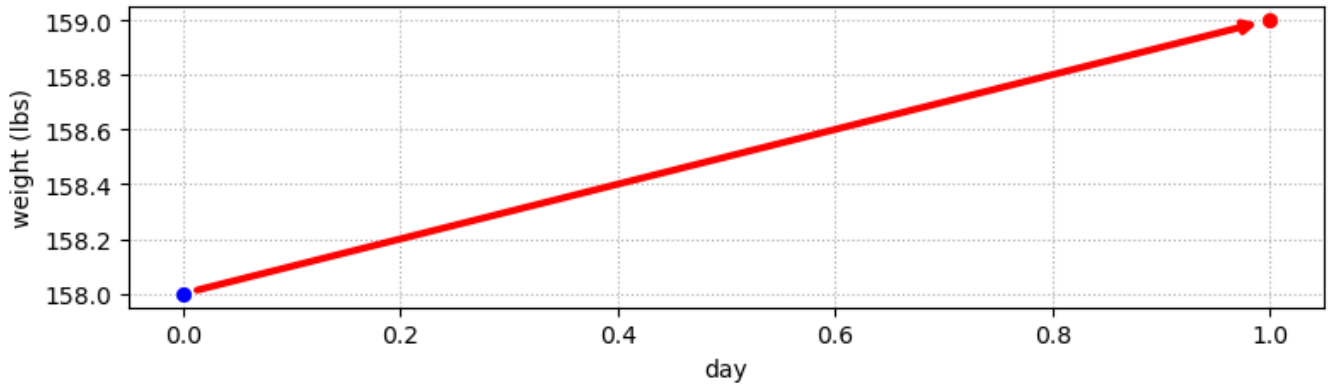
至少在我看来，这样看起来要好得多。现在可以看到，假设值与各个测量值都非常接近，而在之前的图表中，假设值往往与测量值相差甚远。看来，我确实长了体重的可能性要大得多，而不是完全没有长体重。我真的长了 13 磅吗？谁知道呢？这似乎是无法回答的问题。

“但那真的不可能吗？”一位同事插话道。

我们来试试某种“疯狂”的假设吧。假设我知道自己每天大约会增重一磅。现在不必管我是怎么知道这一点的，只要假设这个数值大致正确即可。也许我每天摄入 6000 卡路里的热量，这才导致了这样的体重增加。当然，也可能还有其他方法来估算体重变化。这只是一个思维实验而已，细节并不重要。让我们看看，如果拥有这些信息的话，我们能否加以利用。

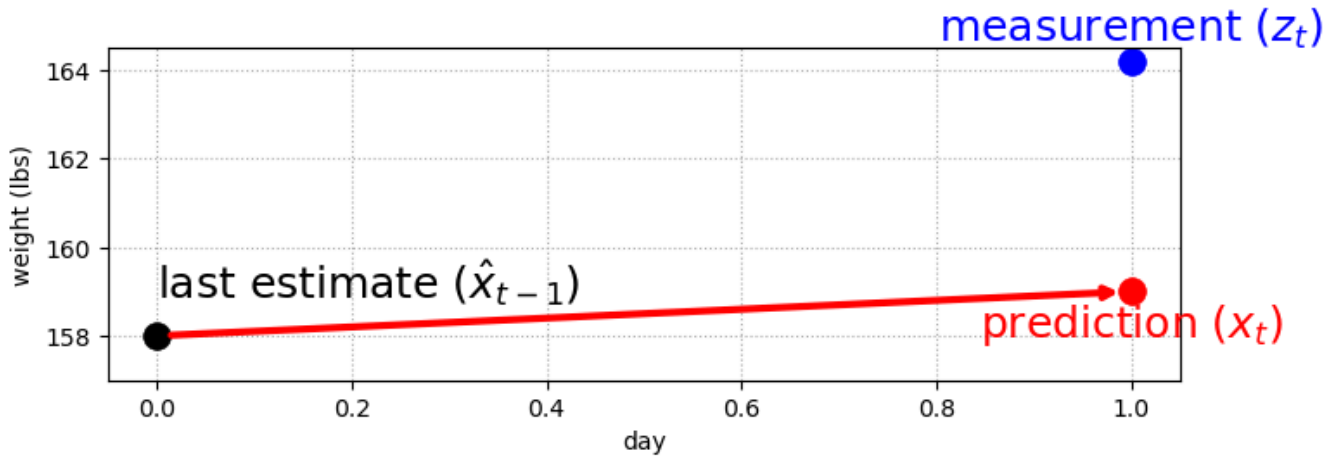
第一次测量得到的数值是 158。我们无法确定还有没有其他可能的数值，所以就以 158 作为我们的估算值吧。如果今天我们的体重是 158，那么明天会是多少呢？我们认为自己的体重每天增加 1 磅，所以预计明天的体重会是 159。具体来说就是：

```
gh.plot_estimate_chart_1()
```



好吧，但这有什么用呢？当然，我们可以假设每天减轻 1 磅这一数据是准确的，从而预测未来 10 天的体重变化。但既然不利用体重秤的测量结果，那还要它做什么呢？接下来，我们再称一次体重。结果显示为 164.2 磅。

```
gh.plot_estimate_chart_2()
```



我们遇到了一个问题：我们的预测结果与实际测量结果并不一致。不过，这其实也是我们预料之中的吧？如果预测结果总是与测量结果完全一致的话，那它就毫无用处了，因为它无法为“滤波器”提供任何有用的信息。当然，既然我们的预测已经完美无缺，那也就没有必要再进行测量了。

> **整本书的关键见解就在下一段中。请仔细阅读！**

那我们该怎么办呢？如果我们仅根据测量结果来做出估计，那么预测结果就不会对最终结果产生影响。如果我们仅根据预测结果来做出估计，那么测量结果就会被忽略。要想让这个方法奏效，我们就需要将预测结果和测量结果结合起来使用（我已经把关键点用粗体标出来了）。

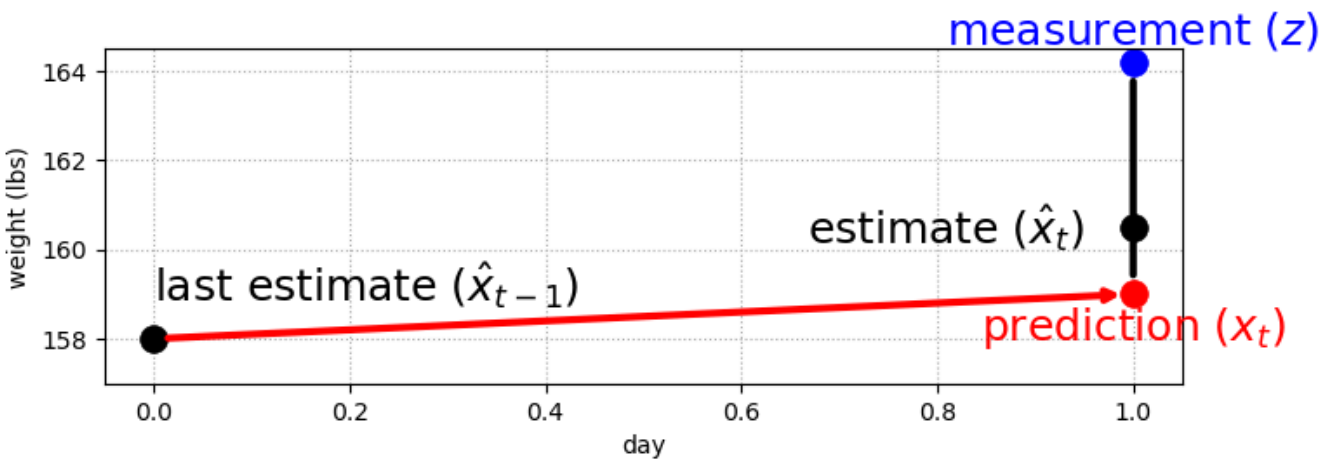
将两个数值进行混合处理——这听起来很像之前提到的“双尺度问题”。运用与之前相同的推理方式，我们可以得出：唯一合理的做法就是选择介于预测值和测量值之间的某个数值。例如，165 或 157 这样的数值都不合理。我们的估算值应该介于 159（预测值）和 164.2（测量值）之间。

再强调一次，这一点非常重要。我们之前已经达成共识：当面对两个有误差的数值时，我们应该取这两个数值之间的中间值作为估算结果。这些数值的来源并不重要。在本章的开头，我们有两个测量结果；但现在，我们只有一个测量结果和一个预测值。无论哪种情况，其推理过程和数学计算方式都是相同的。我们绝不能丢弃任何信息。我是认真的。我见过很多商业软件会丢弃那些有误差的数据。千万不要这么做！虽然我们对体重增加的预测可能不太准确，但只要还有哪怕一点信息，我们都应该加以利用。

我必须坚持要求你停下来，认真思考一下这个问题。我所做的只不过是用基于人体生理学的预测结果来替代了那些不准确的测量结果而已。本质上，这仍然属于数据而已。数学并不关心这些数据是从测量仪器得到的，还是通过预测得出的。我们手头有两组带有某些误差的数据，现在需要把它们结合起来。在本书的后续内容中，我们将运用一些复杂的数学公式来进行这些计算。但数学并不关心数据的来源，它只关心数据的数值及其准确性。

那么，这个估算值应该介于测量值和预测值之间吗？也许吧。不过一般来说，我们似乎能够判断出自己的预测值与测量值相比，到底有多准确。很可能，预测值的准确性会与测量工具的准确性有所不同。回想一下，当测量工具 A 的准确性远高于测量工具 B 时，我们会调整预测值，使其更接近测量工具 A 的数值。我们用图表来具体看看吧。

```
gh.plot_estimate_chart_3()
```



现在，让我们用一个随机选定的数字来调整我们的估算结果：$\frac{4}{10}$。我们的估算结果将是实际测量值的四分之一，其余部分则来自预测值。换句话说，我们在这里表达了一种信念：即预测值比实际测量值更有可能是正确

的。我们是这样计算出来的：

$$estimate = prediction + \frac{4}{10}(measurement - prediction)$$

测量值与预测值之间的差异被称为“残差”，在上面的图表中，该值由黑色的垂直线表示。这一数值日后会非常重要，因为它准确地反映了测量结果与滤波器输出之间的差异。残差越小，说明性能越好。

让我们把代码写出来，然后用上面列出的那些权重值来测试一下，看看结果如何。我们还需要考虑另一个因素：体重的增加是以“磅/时间”为单位来表示的。因此，为了使计算结果更具普遍性，我们需要加入一个时间参数 t，这里我们将其设置为 1 天。

我手动设置了这些体重数据：起始体重为 160 磅，之后每天体重增加 1 磅。也就是说，第一天时，实际体重为 160 磅；第二天时，实际体重为 161 磅，以此类推。

我们需要先对初始重量进行估算。现在讨论初始化策略还为时过早，所以我暂时假设初始重量为 160 磅。

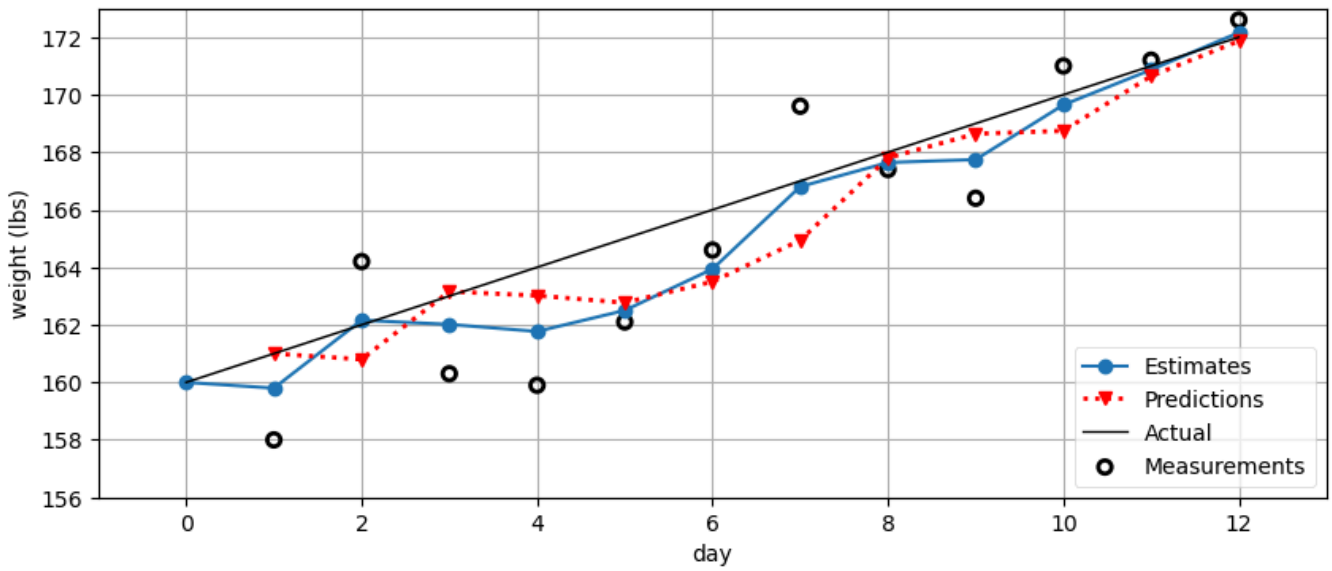
```
from kf_book.book_plots import figsize
import matplotlib.pyplot as plt

weights = [158.0, 164.2, 160.3, 159.9, 162.1, 164.6,
           169.6, 167.4, 166.4, 171.0, 171.2, 172.6]

time_step = 1.0  # day
scale_factor = 4.0/10

def predict_using_gain_guess(estimated_weight, gain_rate, do_print=False):
    # storage for the filtered results
    estimates, predictions = [estimated_weight], []

    # most filter literature uses 'z' for measurements
    for z in weights:
        # predict new position
        predicted_weight = estimated_weight + gain_rate * time_step

        # update filter
        estimated_weight = predicted_weight + scale_factor * (z - predicted_weight)

        # save and log
        estimates.append(estimated_weight)
        predictions.append(predicted_weight)
        if do_print:
            gh.print_results(estimates, predicted_weight, estimated_weight)

    return estimates, predictions

initial_estimate = 160.
estimates, predictions = predict_using_gain_guess(
    estimated_weight=initial_estimate, gain_rate=1, do_print=True)
```

```
previous estimate: 160.00, prediction: 161.00, estimate 159.80
previous estimate: 159.80, prediction: 160.80, estimate 162.16
previous estimate: 162.16, prediction: 163.16, estimate 162.02
previous estimate: 162.02, prediction: 163.02, estimate 161.77
previous estimate: 161.77, prediction: 162.77, estimate 162.50
previous estimate: 162.50, prediction: 163.50, estimate 163.94
previous estimate: 163.94, prediction: 164.94, estimate 166.80
previous estimate: 166.80, prediction: 167.80, estimate 167.64
```

```
previous estimate: 167.64, prediction: 168.64, estimate 167.75
previous estimate: 167.75, prediction: 168.75, estimate 169.65
previous estimate: 169.65, prediction: 170.65, estimate 170.87
previous estimate: 170.87, prediction: 171.87, estimate 172.16
```

```
# plot results
book_plots.set_figsize(10)
gh.plot_gh_results(weights, estimates, predictions, [160, 172])
weights
```

```
[158.0,
 164.2,
 160.3,
 159.9,
 162.1,
 164.6,
 169.6,
 167.4,
 166.4,
 171.0,
 171.2,
 172.6]
```



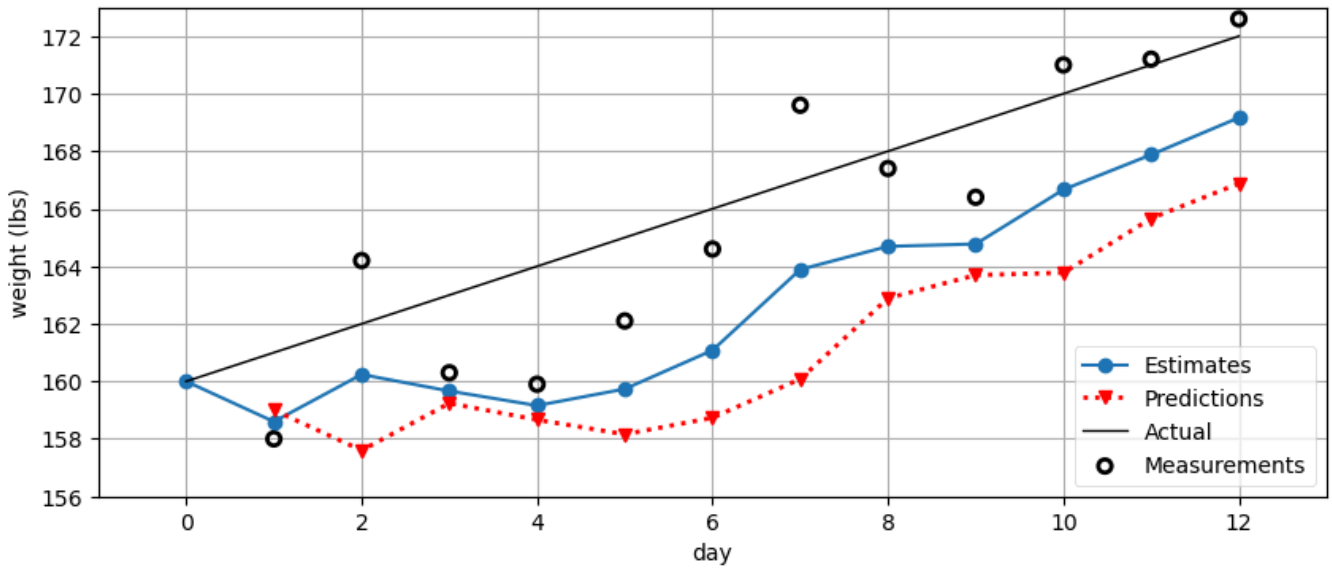
相当不错！这里有大量的数据，那我们来探讨一下如何解读这些数据吧。粗蓝线代表根据某种算法得出的估算值。该线从第 0 天开始，最初的估算值为 160 磅。红线则代表根据前一天의体重所做出的预测值。例如，第 1 天时，前一天の体重为 160 磅，体重增加了 1 磅，因此第 1 天的预测值为 161 磅。第 1 天的实际估算值则介于预测值和实际测量值之间，为 159.8 磅。图表下方列出了每天的实际体重、预测体重以及新的估算值。最后，细黑线代表被测量者实际的体重增加量。

请逐天仔细查看这些数据，确保自己明白在每个步骤中，预测值和估算值是如何得出的。请注意，估算值始终位于测量值和预测值之间。

这些估算值并非呈直线分布，但相比实际测量数据来说，其趋势更为平缓，且与我们所绘制的趋势线较为接近。此外，随着时间的推移，这些估算值似乎还有所改善。

滤波后的结果可能会让你觉得有些荒谬可笑；当然，如果我们假设每天体重增加 1 磅，那么数据看起来自然就“完美”了。那如果我们最初的猜测有误会怎样呢？假设每天体重实际上减少 1 磅，看看滤波后的结果会如何吧。

```
e, p = predict_using_gain_guess(initial_estimate, -1.)
gh.plot_gh_results(weights, e, p, [160, 172])
```



那并没有什么了不起的。各种估算结果与实际测量值相差甚远。显然，那种需要我们准确预测变化率的滤波器没什么用处。即便我们的初始猜测是正确的，但一旦变化率发生改变，该滤波器就会失效。如果我不再暴饮暴食，该滤波器也很难适应这种变化。请注意，它其实是在尝试调整的！尽管我们告诉它我们每天都在减重 1 磅，但估算结果却仍在上升。只是它的调整速度不够快而已。

但是，“要是那样会怎样呢？”如果我们不是简单地把体重增加量定为最初的 1 磅（或其它数值），而是根据现有的测量数据和估算结果来计算呢？第一天时，我们对体重的估算结果是：

$$(160 + 1) + \frac{4}{10}(158 - 161) = 159.8$$

第二天，我们测得的体重为 164.2 磅。这意味着体重增加了 4.4 磅（因为 164.2 - 159.8 = 4.4），而不是 1 磅。我们能利用这些数据吗？看起来是可行的。毕竟，体重测量结果本身就是基于现实中的体重数值，因此具有实际意义。虽然我们对体重增加量的估算可能并不完全准确，但总比随便猜测 1 磅要好。数据总比猜测更可靠，哪怕这些数据存在一定的误差。

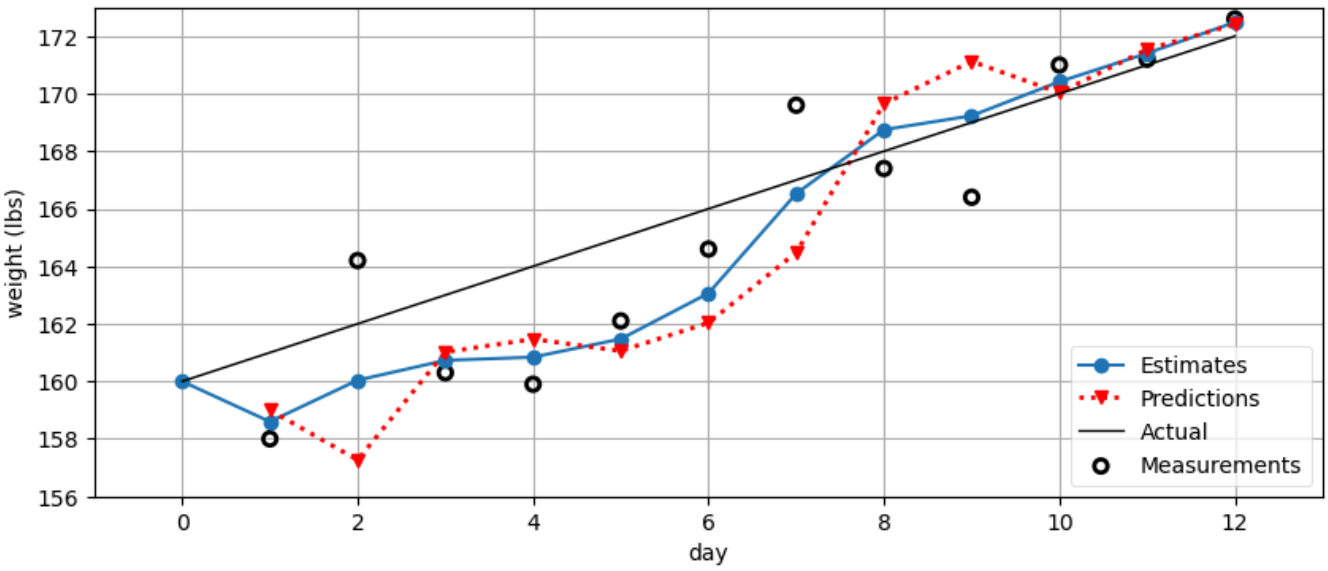
在这一点上，人们往往会犹豫不决，所以一定要确保自己同意这个做法。通过两次称重得到的数值，我们可以估算出体重的增减情况。如果称重结果不准确，那么估算出的数值也会不准确。不过，这种计算方式仍然能提供一些有用的信息。比如，如果我们用精度为 1 磅的秤来称一头牛的体重，结果显示牛的体重增加了 10 磅。实际上，由于测量误差，牛的体重可能增加了 8 磅到 12 磅之间。不过，我们至少可以知道牛的体重确实增加了，而且大致增加了多少。这就是有用的信息。那我们该如何利用这些信息呢？千万不要把它们浪费掉！

回到我的饮食计划上。我们应该把每天的体重增加量定为 4.4 磅吗？昨天我们认为体重增加了 1 磅，今天则认为是 4.4 磅。我们有两个不同的数值，需要想办法把它们结合起来。嗯，看来又遇到了同样的问题。那就用我

们之前用过的那个方法吧——选择两个数值之间的某个中间值。这次，我选择另一个任意数字 $\frac{1}{3}$ 作为中间值。这个计算公式和之前估算体重时用的公式是一样的，只不过这次需要考虑时间因素，因为这实际上是一个增长率（每天增加的体重）。

$$\text{new gain} = \text{old gain} + \frac{1}{3}\frac{\text{measurement - predicted weight}}{1 \text{ day}}$$

```
weight = 160.  # initial guess
gain_rate = -1.0  # initial guess

time_step = 1.
weight_scale = 4./10
gain_scale = 1./3
estimates = [weight]
predictions = []

for z in weights:
    # prediction step
    weight = weight + gain_rate*time_step
    gain_rate = gain_rate
    predictions.append(weight)

    # update step
    residual = z - weight

    gain_rate = gain_rate + gain_scale   * (residual/time_step)
    weight    = weight    + weight_scale * residual

    estimates.append(weight)

gh.plot_gh_results(weights, estimates, predictions, [160, 172])
```



我觉得现在的结果相当不错。由于最初对体重变化的估计值为-1，导致滤波器需要几天时间才能准确预测体重。但一旦能够准确预测后，它就能持续准确地记录体重变化了。我们在确定参数 $\frac{4}{10}$ 和 $\frac{1}{3}$ 的值时并没有采用任何特定的方法（实际上，这些数值并不适合用来解决这个问题）。不过，其余的数学计算都是基于合理的假设得出的。如果你想查看数值变化的详细过程，可以将参数 `time_step` 的值调大，然后重新运行该程序。

在继续之前，还有最后一点需要说明。在预测步骤中，我写了这样一行代码。

```
gain_rate = gain_rate
```

显然，这样做没有任何效果，完全可以忽略不计。我写这段话是为了强调：在预测阶段，需要预测所有变量的下一个数值，包括 `weight` 和 `gain_rate` 。这一点很快就会变得很重要。目前我们假设增益是恒定的，但在将这个算法推广到更一般的情况时，就需要放弃这个假设了。

G-H 滤波器

该算法被称为 g-h 滤波器或 α - β 滤波器。 g 和 h 指的是我们在示例中使用的两个缩放因子。 g 是用于测量的缩放因子（在示例中相当于权重），而 h 则是用于表示随时间变化的测量值的缩放因子（在示例中为磅/天）。 α 和 β 其实只是对这些因子的不同称呼而已。

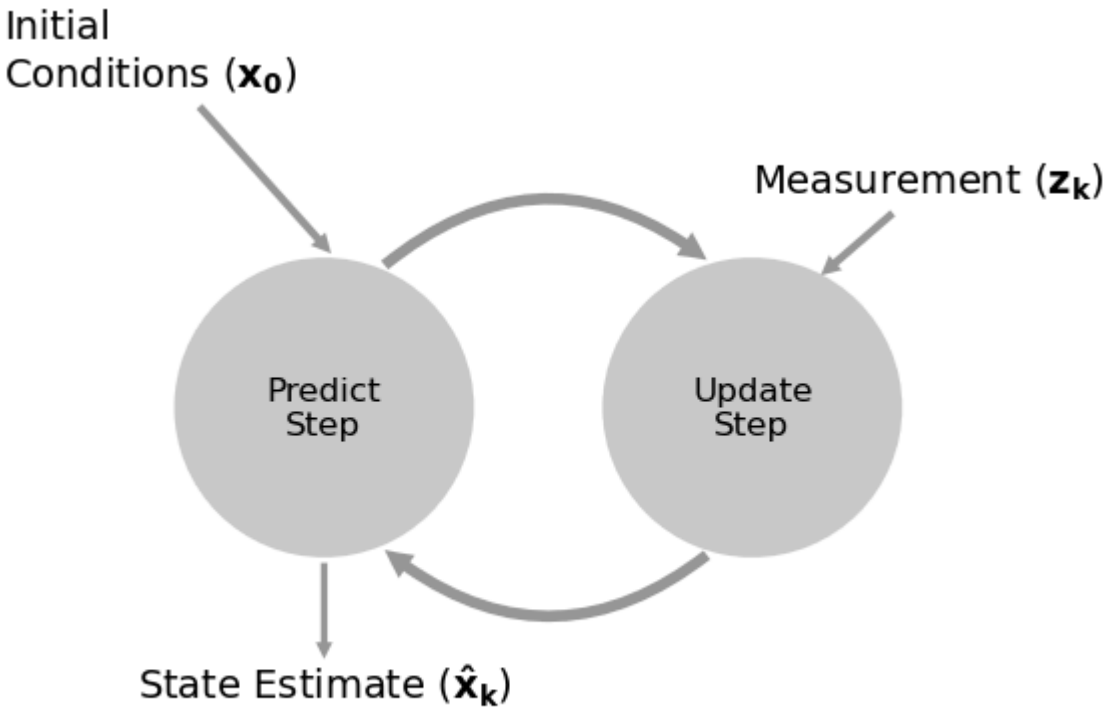
这种滤波器是众多滤波器的基础，包括卡尔曼滤波器。换句话说，卡尔曼滤波器其实是一种特殊的 g-h 滤波器形式，我将在本书后面证明这一点。最小二乘滤波器也是如此，你可能听说过它；而本尼迪克特-博德纳滤波器则可能不太为人所知。每种滤波器在为 g 和 h 赋值的方式上有所不同，但算法本身是相同的。例如，本尼迪克特-博德纳滤波器会为 g 和 h 分配固定值，但这些值必须符合一定的范围限制。而像卡尔曼滤波器这样的滤波器，则会在每个时间步动态地调整 g 和 h 的值。

让我把那些要点再重复一遍，因为它们非常重要。如果你不理解这些要点，那么你就无法理解这本书的其余内容。而如果你理解了这些要点，那么这本书的其余部分就会变得容易理解了——因为那些内容其实都是针对我们关于 g 和 h 的各种“假设”而提出的数学推导而已。虽然数学表达方式可能有所不同，但算法本身是完全相同的。

- 多个数据点总比一个数据点更准确。因此，无论某个数据点的准确性如何，都不要放弃它。
- 为了得到更准确的估算结果，应始终选择位于两个数据点之间的数值作为计算依据。
- 根据当前的估算值，以及我们认为该数值还会发生多大的变化，来预测接下来的测量结果和变化率。
- 新的估算值则是根据预测值和下一次测量结果的准确性来确定的，具体来说，就是将预测值与下一次测量结果按各自的准确性进行加权平均后得到的数值。

让我们来看看该算法的可视化表现形式吧。

```
book_plots.predict_update_chart()
```



让我来介绍一些更正式的术语。所谓“系统”，指的就是我们想要进行评估的对象。在本章中，所谓“系统”指的就是我们试图对其进行衡量的任何事物。有些文献将之称为“被控对象”。这一术语源自控制系统理论。https://en.wikipedia.org/wiki/Plant_(control_theory)

系统的“状态”指的是该系统当前所具有的配置或数值。我们只关心与重量相关的数值。如果我在秤上放置 100 公斤的重物，那么该系统的状态就是 100 公斤。我们根据对自己来说重要的信息来定义系统的状态。秤的颜色对我们来说并不重要，因此不会将其纳入系统的状态中。不过，作为制造商的质量控制工程师，可能会将秤的颜色也纳入系统的状态中，以便于对生产过程进行监控和控制。

该测量值实际上是系统被测得的数值。由于测量可能存在误差，因此该数值可能与系统的真实状态并不完全一致。

状态估计指的是滤波器对系统状态的估算值。例如，对于 100 公斤的重量，由于传感器误差，我们的估算值可能会是 99.327 公斤。这一数值通常简称为“估计值”，我在本章中也是这么处理的。

换句话说，国家可以被看作是该系统的“实际价值”。不过，这个价值通常是我们无法直接察觉的。如果我站在秤上，那么就能得到一个具体的数值。因为这个数值可以直接被观测到，所以我们称之为“可观测量”。相反，我的体重则永远无法被直接观测到，我们只能对其进行测量而已。

这种“隐藏状态”与“可观测信息”的概念非常重要。在所有的估算问题中，都需要通过可观测的数值来推断出隐藏状态。如果你阅读相关文献，就会发现这些术语在描述各种问题时都被经常使用。因此，你必须熟练掌握这些概念。

我们使用流程模型来对系统进行数学描述。在本章中，我们所采用的流程模型是这样的：今天的体重等于昨天的体重加上昨天体重增加的数值。该流程模型并未考虑传感器的因素。另一个例子是汽车的流程模型：距离等

于速度乘以时间。不过，这种模型并不完美，因为汽车的速度会随时间而变化，轮胎也可能会在路面上打滑等等。系统误差或流程误差指的就是这种模型中的偏差。我们永远无法准确知道这个数值；如果能够知道的话，我们就可以改进模型，使其误差为零。有些文献会使用“系统模型”或“系统误差”这样的术语，其实它们都指的是同一种概念。

预测步骤被称为“系统传播”。该过程利用过程模型来生成新的状态估计值。由于存在过程误差，这种估计值并不完美。如果我们是在对随时间变化的数据进行跟踪，那么就可以说我们把系统的状态“传播”到了未来。有些文献将这一过程称为“系统演化”。

更新步骤被称为测量更新。系统传播与测量更新的一次循环被称为一个“时代”。

现在，让我们通过几个不同的例子来更好地理解这个算法。以追踪铁轨上的火车为例。铁轨将火车的位置限制在某个特定的范围内。此外，火车的体积较大且移动速度较慢，因此需要很长时间才能使火车的速度发生显著变化。所以，如果我知道在时间 t 时，火车位于 23 公里处，且行驶速度为 18 公里/小时，那么我完全可以准确预测出 t+1 秒时火车的位置。为什么这一点很重要呢？假设我们只能以 250 米的精度来测量火车的位置。由于火车的行驶速度为 18 公里/小时，即 5 米/秒，那么在 t+1 秒时，火车应该位于 23.005 公里处。但实际上，测量结果可能在 22.755 公里到 23.255 公里之间。因此，如果下一次测量结果显示火车位于 23.4 公里处，那显然这个测量结果是不可靠的。即使在时间 t 时，工程师立刻踩下刹车，火车的速度也仍然会接近 23.005 公里/秒。因为火车不可能在 1 秒钟内大幅减速。如果我们想要为这个问题设计一个滤波器的话（我们稍后会在本章中详细讨论这个问题），那么我们应该设计出一个能够给予预测值比测量值更高权重的滤波器。

现在来考虑追踪被抛出的球的问题。我们知道，当物体处于重力场中时，其在真空中的运动轨迹是抛物线形。但在地球上，由于空气阻力的影响，球的实际运动轨迹并非完美的抛物线。棒球投手正是利用了这一原理来投出曲线球。假设我们使用计算机视觉技术在体育场内追踪球的运动轨迹——这也是我工作中经常做的事情。虽然计算机视觉技术的精确度可能不高，但假设球沿抛物线轨迹运动来预测其未来位置的方法，其精确度也并不高。在这种情况下，我们或许应该设计一种算法，让测量结果和预测结果具有大致相同的权重。

现在，试想一下在飓风中追踪一个氦气气球的情况吧。我们没有任何可靠的模型能够用来预测气球的运动轨迹，除非是在极短的时间范围内（比如，我们知道气球不可能在一秒钟内移动 10 英里）。在这种情况下，我们可以设计一种滤波器，让那些基于实际观测得到的数据比预测结果更受重视。

本书的大部分内容都致力于用数学方式来表达最后三段文字中所表达的观念。这样一来，我们就能找到一个“最优解”（从数学意义上来说）。在本章中，我们将以一种更直观的方式给 g 和 h 赋予不同的数值，这种方式虽然不是最优的，但基本思路是：将那些不太准确的测量结果与那些对系统行为描述不太准确的模型结合起来，从而得到一个比单独使用其中任何一种信息来源都要好的估算结果。

我们可以将其表述为一个算法：

初始化

1. Initialize the state of the filter
2. Initialize our belief in the state

预测

```
1. Use system behavior to predict state at the next time step
2. Adjust belief to account for the uncertainty in prediction
```

更新

```
1. Get a measurement and associated belief about its accuracy
2. Compute residual between estimated state and measurement
3. New estimate is somewhere on the residual line
```

在整本书中，我们都会使用同样的算法，不过会做一些必要的修改。

符号/标记

我将开始介绍文献中使用的各种符号和变量名称。其中一些符号在上面的图表中已经出现过。通常，测量值用 z 来表示，本书中也会采用这种表示方式（不过有些文献使用 y 来表示测量值）。下标 k 表示时间步长，因此 z_k 表示该时间步长对应的数据。粗体字表示向量或矩阵。到目前为止，我们只考虑了只有一个传感器的情况，也就是只有一种测量值。但实际上，我们可以有 n 个传感器，从而有 n 种测量值。 x 表示我们的状态，由于它是一个向量，所以用粗体字表示。以我们的体重变化为例， x 既表示初始体重，也表示初始的体重增加率。

$$x = \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$

在这里，我使用牛顿的记号法——在 x 上加点来表示速度。更准确地说，这个点表示 x 关于时间的导数，而导数自然就是速度了。对于一个重量为 62 公斤、每天增加 0.3 公斤的物体来说，其速度情况如下：

$$x = \begin{bmatrix} 62 \\ 0.3 \end{bmatrix}$$

该算法其实很简单。首先，将状态初始化为初始值 x_0 。接着，进入循环：根据时间点 $k-1$ 的数值来预测时间点 k 的状态。之后，获取实际测量值 z_k ，并在该测量值与预测值之间选择一个中间值，从而得到新的估计值 x_k 。

练习：编写通用算法

在上面的例子中，我特意这样编写代码，为的就是解决我们在本章中一直在讨论的称重问题。例如，各个变量的名称被分别设为“weight_scale”、“gain”等等。我这么做是为了让算法更易于理解——这样就能清楚地看出，我们确实正确地实现了算法的每一步。不过，这段代码只是为了解决某个特定问题而编写的。而实际上，对于任何问题来说，算法都是相同的。因此，让我们重新编写代码，使其具有通用性，从而能够适用于各种问题。请使用如下函数格式来编写代码：

```
def g_h_filter(data, x0, dx, g, h, dt):
    """
    Performs g-h filter on 1 state variable with a fixed g and h.

    'data' contains the data to be filtered.
    'x0' is the initial value for our state variable
    'dx' is the initial change rate for our state variable
```

```
    'g' is the g-h's g scale factor
    'h' is the g-h's h scale factor
    'dt' is the length of the time step
    """
```

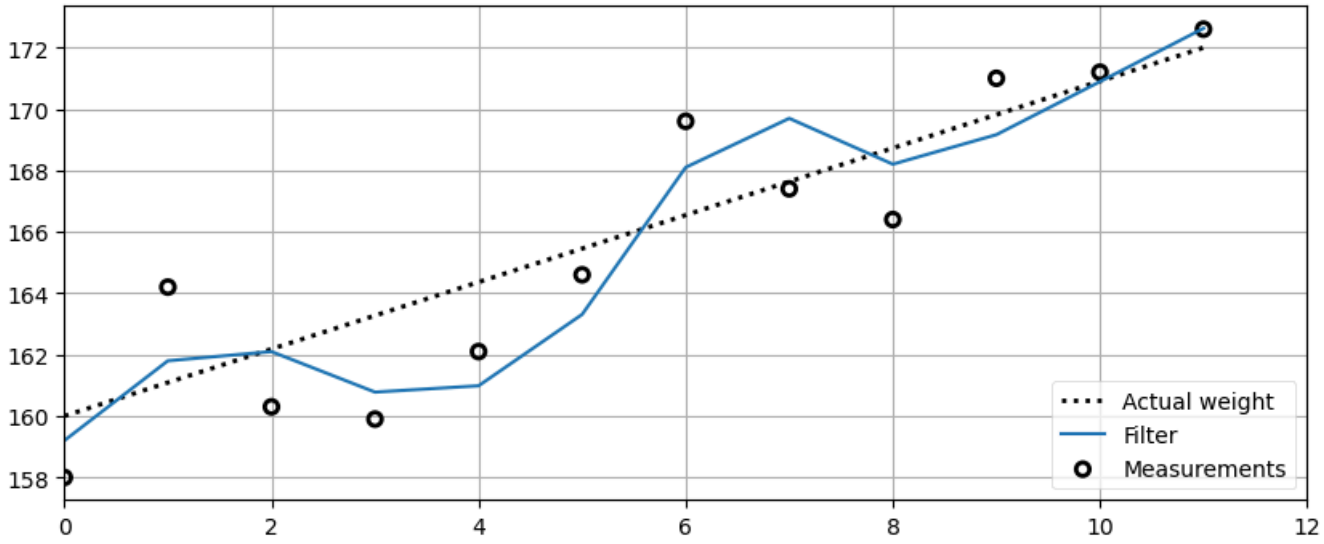
请以 NumPy 数组的形式返回数据，而非列表形式。可以用之前相同的权重数据来进行测试，绘制出结果图，从而确认该功能确实能够正常工作。

```
from kf_book.gh_internal import plot_g_h_results
def g_h_filter(data, x0, dx, g, h, dt):
    pass #  your solution here

# uncomment to run the filter and plot the results
#book_plots.plot_track([0, 11], [160, 172], label='Actual weight')
#data = g_h_filter(data=weights, x0=160., dx=1., g=6./10, h=2./3, dt=1.)
#plot_g_h_results(weights, data)
```

解决方案与讨论

```
import matplotlib.pylab as pylab

def g_h_filter(data, x0, dx, g, h, dt=1.):
    x_est = x0
    results = []
    for z in data:
        # prediction step
        x_pred = x_est + (dx*dt)
        dx = dx

        # update step
        residual = z - x_pred
        dx = dx + h * (residual) / dt
        x_est = x_pred + g * residual
        results.append(x_est)
    return np.array(results)

book_plots.plot_track([0, 11], [160, 172], label='Actual weight')
data = g_h_filter(data=weights, x0=160., dx=1., g=6./10, h=2./3, dt=1.)
plot_g_h_results(weights, data)
print(weights)
print(data)
```

```
[158.0, 164.2, 160.3, 159.9, 162.1, 164.6, 169.6, 167.4, 166.4, 171.0, 171.2, 172.6]
[159.2   161.8   162.1   160.78  160.985 163.311 168.1   169.696
 168.204 169.164 170.892 172.629]
```



这本该是件很简单的事。我只需将权重计算代码中的变量名 `x0` 、 `dx` 等替换掉即可。无需做其他任何更改。

g 和 h 的选择

G-H 滤波器并非单一的滤波器类型，而是一类滤波器的总称。Eli Brookner 在《Tracking and Kalman Filtering Made Easy》一书中列出了 11 种此类滤波器，实际上肯定还有更多。此外，每种滤波器又包含多种子类型。各种滤波器的区别在于如何确定 g 和 h 的值。因此，这里无法给出适用于所有情况的通用建议。有些滤波器将 g 和 h 设置为固定值，而另一些则让这些值动态变化。卡尔曼滤波器就是在每一步都动态调整这些值。有些滤波器允许 g 和 h 在某个范围内取任意值，而另一些则要求这两个值通过某种函数 $f(\cdot), \text{where } g = f(h)$ 相互关联。

本书的主题并非 G-H 滤波器的全部内容；更重要的是，我们感兴趣的是这些滤波器的贝叶斯方面，而这一点我尚未详细阐述。因此，我不会深入探讨 g 和 h 的选择问题。关于这一主题，《Tracking and Kalman Filtering Made Easy》是一本非常好的参考书。如果你觉得我的这种观点有些奇怪，请记住：卡尔曼滤波器的典型实现方式其实根本不涉及 g 和 h 。卡尔曼滤波器可以被看作是一种 G-H 滤波器，因为它在数学上可以转化为这种算法。在设计卡尔曼滤波器时，我们会使用一些设计准则，这些准则在数学上可以转化为 g 和 h ，不过，使用卡尔曼滤波器的形式来思考问题通常更为有效。如果现在还不太明白的话，不必担心，等我们进一步了解卡尔曼滤波器的理论后，就会明白的。

值得研究的是， g 和 h 的不同取值会对结果产生怎样的影响。因此，我们将通过几个例子来进行分析。这将有助于我们深入了解这类滤波器的基本优点和局限性，同时也有助于我们理解更为复杂的卡尔曼滤波器的工作原理。

练习：创建测量函数

现在，让我们来编写一个能够生成带噪声数据的函数吧。在这本书中，我将带噪声的信号定义为原始信号加上白噪声。我们还没有详细探讨统计学知识，因此无法完全理解“白噪声”的定义。简单来说，白噪声就是那种随机地高于或低于原始信号的数值，且没有任何规律可循的信号。可以说，白噪声是一种具有零均值和有限方

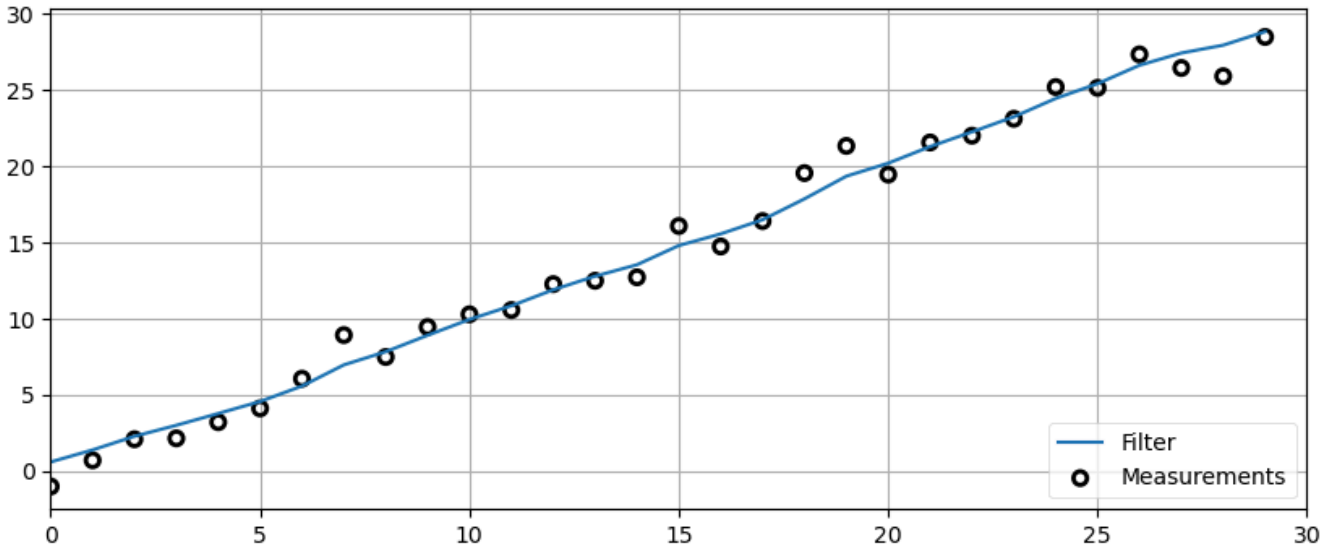
差的随机变量。如果你现在还不理解这些概念，等学到高斯分布的相关内容时就会明白的。如果你完全不懂统计学，那么就直接阅读解答和相关的解释吧。

白噪声可以由 `numpy.random.randn()` 生成。我们需要一个函数，该函数接受起始值、每步的变动幅度、步数以及所需的噪声量作为输入参数。该函数应返回一个包含所有计算结果的列表。可以通过生成 30 个数据点，再利用 `g_h_filter()` 对数据进行滤波，最后用 `plot_g_h_results()` 将结果绘制成图表来测试该函数。

```
# your code here
```

解决方案

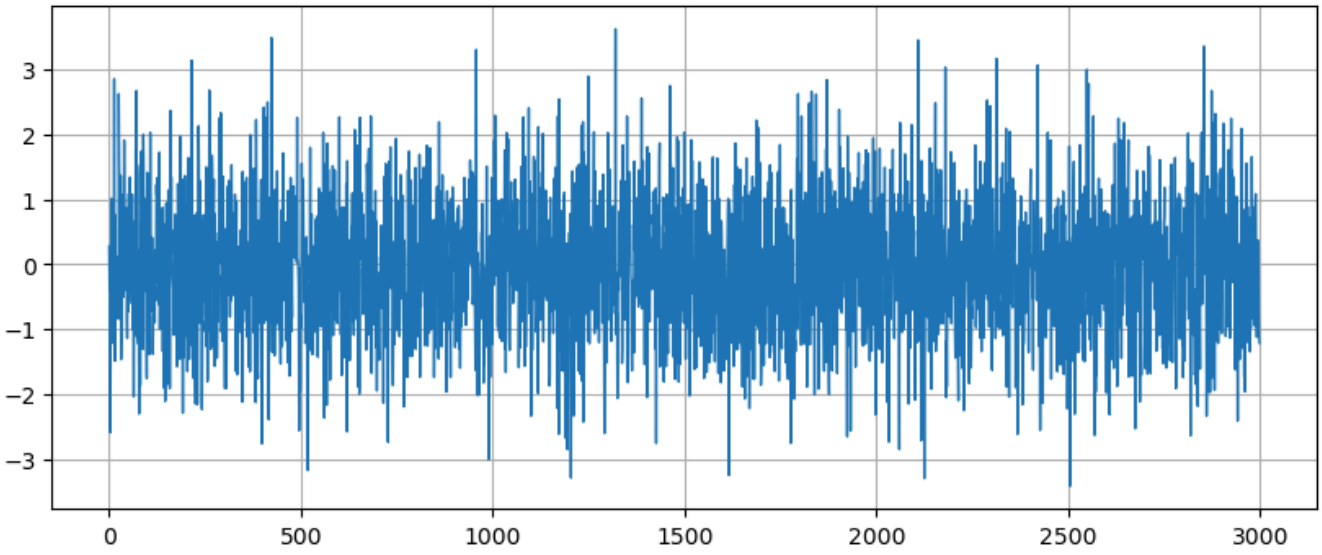
```
from numpy.random import randn
def gen_data(x0, dx, count, noise_factor):
    return [x0 + dx*i + randn()*noise_factor for i in range(count)]

measurements = gen_data(0, 1, 30, 1)
data = g_h_filter(data=measurements, x0=0., dx=1., dt=1., g=.2, h=0.02)
plot_g_h_results(measurements, data)
```



讨论

`randn()` 会生成以 0 为中心的随机数——该数值大于 0 的概率与小于 0 的概率相同。其数值会在 0 的一个标准差范围内波动。如果你不明白“标准差”是什么意思，不必担心。我绘制了 3000 次对 `randn()` 的调用结果：可以看出，这些数值以 0 为中心，大多介于-1 稍低到+1 稍高的范围内，不过偶尔也会出现远高于或低于这个范围的数值。

```
plt.plot([randn() for _ in range(3000)], lw=1);
```



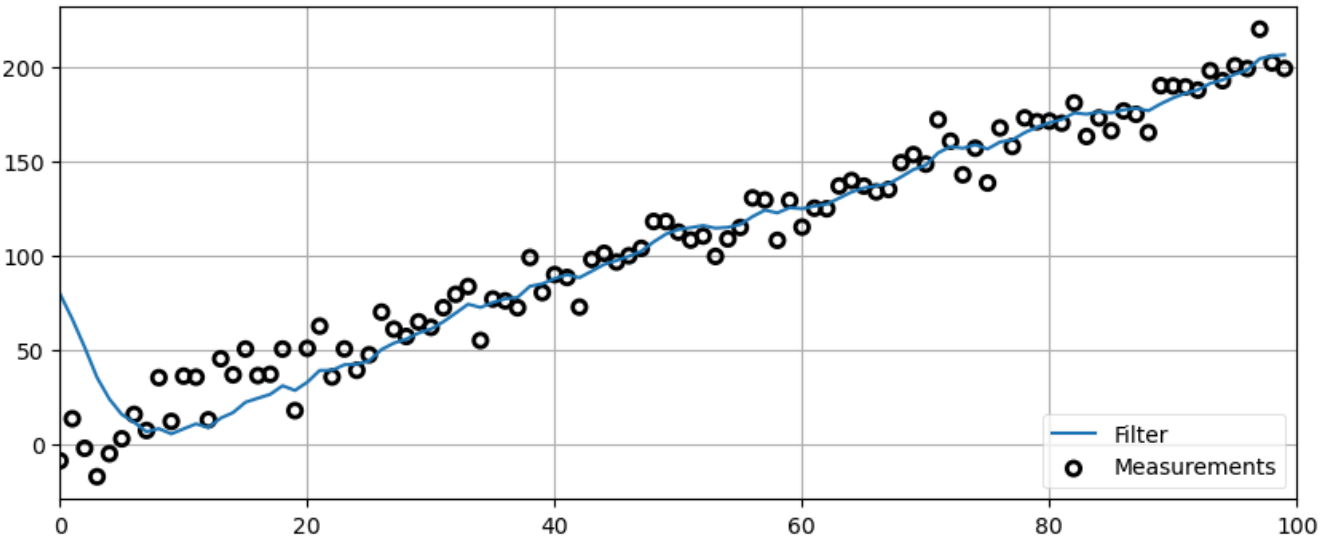
练习：不良的初始条件

现在，请编写代码，利用 `gen_data` 和 `g_h_filter` 来筛选出 100 个数据点。这些数据点的起始值为 5，其导数为 2，噪声缩放因子为 10。同时，g 的值为 0.2，h 的值为 0.02。请将 x 的初始值设为 100。

```
# your code here
```

解决方案与讨论

```
zs = gen_data(x0=5., dx=2., count=100, noise_factor=10)
data = g_h_filter(data=zs, x0=100., dx=2., dt=1., g=0.2, h=0.02)
plot_g_h_results(measurements=zs, filtered_data=data)
```



由于初始值被设定为 100 这个不准确的数值，滤波器在开始时的预测结果与实际测量数据相差甚远。可以看到，滤波器在调整到与实际数据相符的状态之前，会呈现出一种“振荡”现象。所谓“振荡”，指的是信号以正

弦波的形式超出或低于实际数据。这是滤波器中非常常见的现象。在滤波器设计中，人们通常会努力减少这种振荡现象。不过，这个话题我们暂时还不打算深入探讨，我只是想让大家了解这种现象而已。

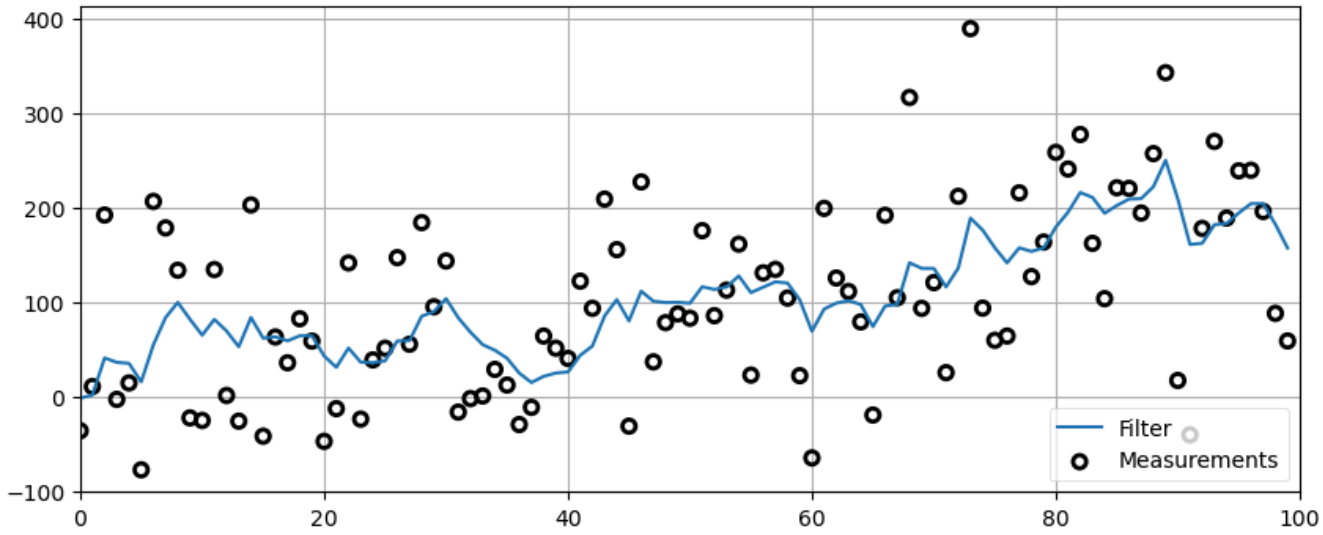
练习：极端噪音

重新进行同样的测试，但这次将噪声系数设置为 100。为了消除初始条件带来的干扰，将初始值从 100 改为 5。

```
# your code here
```

解决方案与讨论

```
zs = gen_data(x0=5., dx=2., count=100, noise_factor=100)
data = g_h_filter(data=zs, x0=5., dx=2., g=0.2, h=0.02)
plot_g_h_results(measurements=zs, filtered_data=data)
```



在我看来，这并不怎么样。可以看出，经过滤波后的信号变化幅度可能小于有噪声的信号，但它远远不是一条直线。如果只展示滤波后的结果，没人能猜出该信号的起始值为 5，并且每个时间点上数值都会增加 2。虽然在该滤波器起作用的情况下，噪声确实有所减少，但在其他地方，信号的数值却出现了超调或欠调的情况。

目前，我们还不足以对这一点做出准确的判断。我们加入了太多不必要的“噪音”；也许，这已经算是滤波效果最好的结果了。不过，既然还有许多章节存在，那就说明我们完全有可能做得更好。

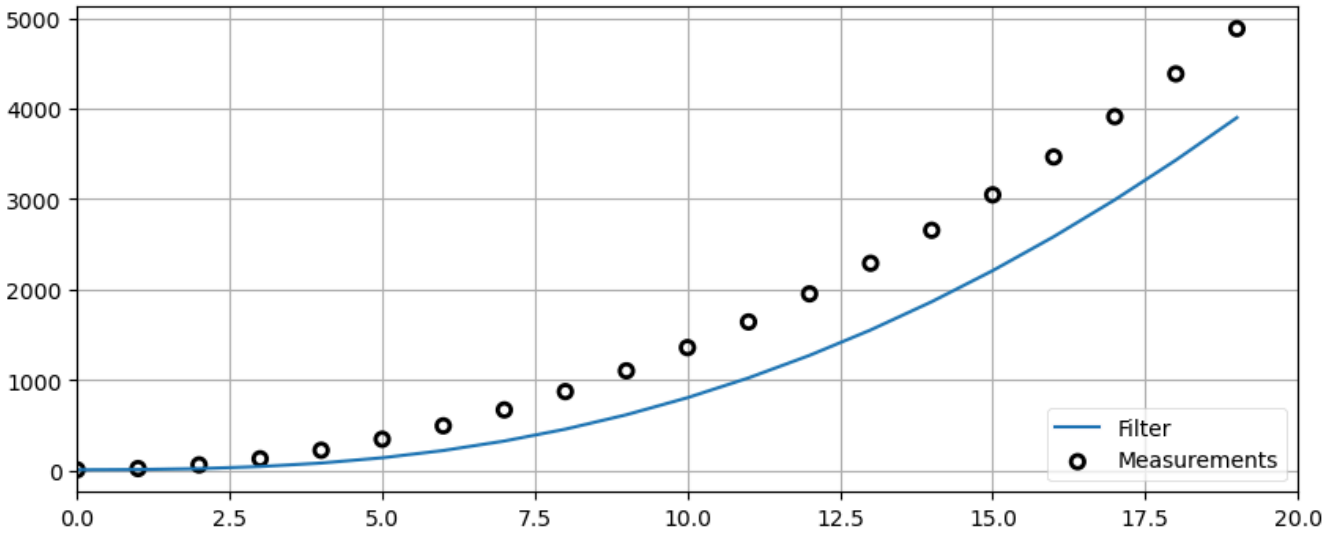
练习：加速度的影响

编写一个新的数据生成函数，该函数会在每个数据点上加上一个恒定的加速度因子。换句话说，在计算每个数据点时，都要让 dx 的值逐渐增大，从而确保速度也持续增加。将噪声值设置为 0，然后使用 `plot_g_h_results` 或你自己的程序来绘制结果。试着尝试不同的加速度和时间步长。请解释你所观察到的现象。

```
# your code here
```

解决方案与讨论

```
def gen_data(x0, dx, count, noise_factor, accel=0.):
    zs = []
    for i in range(count):
        zs.append(x0 + accel * (i**2) / 2 + dx*i + randn()*noise_factor)
        dx += accel
    return zs

predictions = []
zs = gen_data(x0=10., dx=0., count=20, noise_factor=0, accel=9.)
data = g_h_filter(data=zs, x0=10., dx=0., g=0.2, h=0.02)
plot_g_h_results(measurements=zs, filtered_data=data)
```



每一次预测值都会落后于实际数值。仔细想想，这是合乎逻辑的。我们的模型假设速度是恒定的。g-h 滤波器会计算 x 的一阶导数（我们用 $\dot{x}$ 来表示导数），而不计算二阶导数 $\ddot{x}$ 。因此，我们实际上假设 $\ddot{x} = 0$ 为恒定值。在每次预测时，我们将 x 的新值预测为 $x + \dot{x} * t$ 。但由于加速度的存在，预测值必然会落后于实际数值。接着，我们试图重新计算 $\dot{x}$ 的值，但由于 h 的影响，我们只能部分地调整 $\dot{x}$ 以适应新的速度。在下一次迭代中，预测值仍会再次落后于实际数值。

请注意，我们无法对 g 或 h 进行任何调整来纠正这个问题。这被称为系统的滞后误差或系统误差。这是 g-h 滤波器所固有的特性。也许您已经想到了解决这个问题的方法或变通方案。正如您所预期的，许多研究都致力于解决这个问题，我们在本书中将会介绍各种解决方案。

> 归根结底，滤波器的性能好坏，完全取决于用来描述该系统的数学模型的质量如何。

练习：改变 g 的值

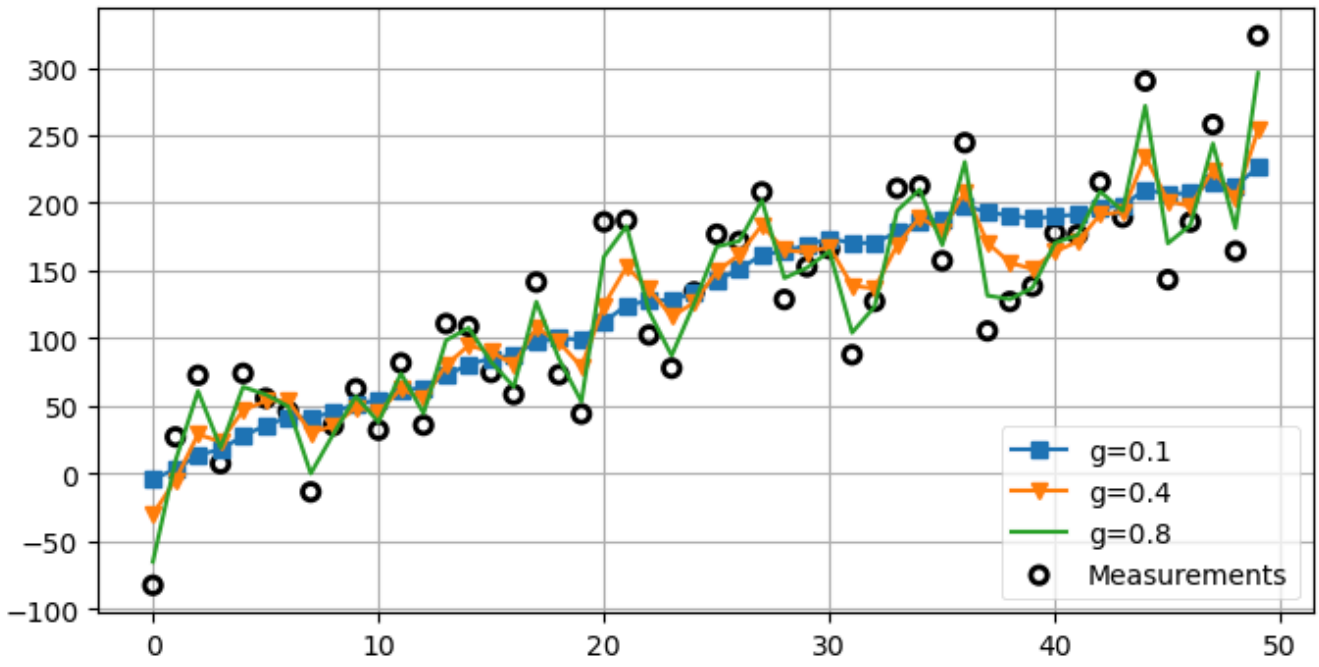
现在，让我们来看看 g 的值发生变化时会产生什么效果。在开始这项练习之前，请先记住： g 是用于决定是采用测量结果还是预测结果的缩放因子。你认为，当 g 的数值较大时，会产生什么效果呢？而当 g 的数值较小时，又会怎样呢?

现在，让 `noise_factor=50` 和 `dx=5` 来处理吧。把 $g = 0.1, 0.4, \text{ and } 0.8$ 的计算结果绘制出来。

```
# your code here
```

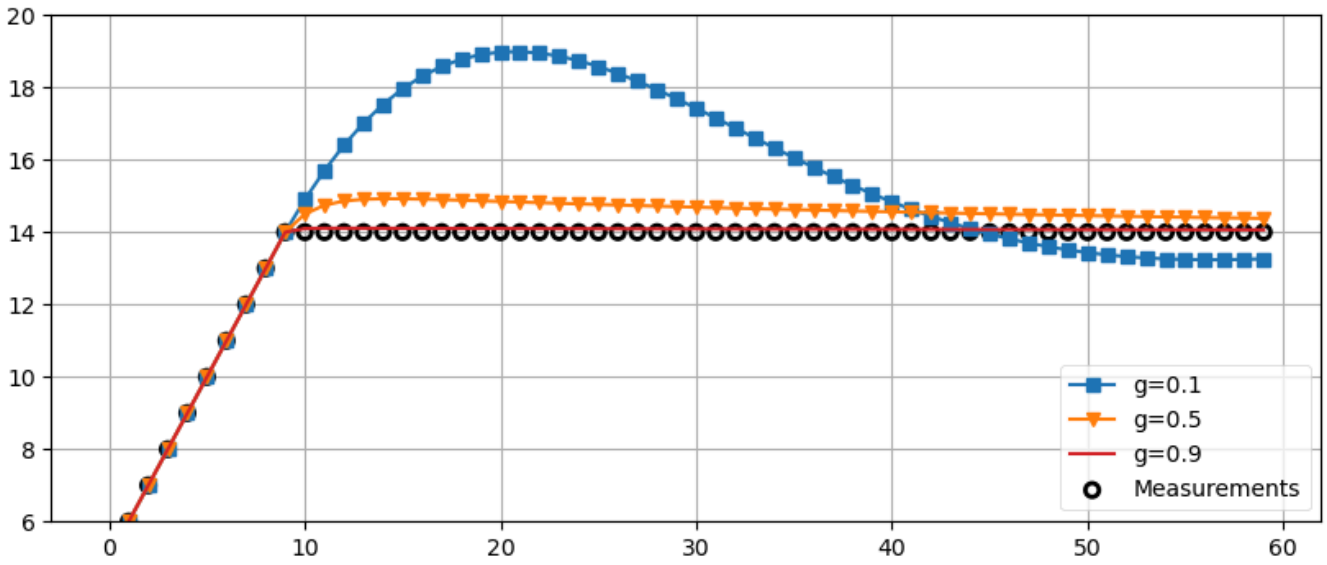
解决方案与讨论

```
np.random.seed(100)
zs = gen_data(x0=5., dx=5., count=50, noise_factor=50)
data1 = g_h_filter(data=zs, x0=0., dx=5., dt=1., g=0.1, h=0.01)
data2 = g_h_filter(data=zs, x0=0., dx=5., dt=1., g=0.4, h=0.01)
data3 = g_h_filter(data=zs, x0=0., dx=5., dt=1., g=0.8, h=0.01)

with book_plots.figsize(y=4):
    book_plots.plot_measurements(zs, color='k')
    book_plots.plot_filter(data1, label='g=0.1', marker='s', c='C0')
    book_plots.plot_filter(data2, label='g=0.4', marker='v', c='C1')
    book_plots.plot_filter(data3, label='g=0.8', c='C2')
    plt.legend(loc=4)
```



显然，当 g 的数值越大时，我们就会更倾向于参考实际测量结果，而非预测值。当 $g = 0.8$ 时，我们几乎完全按照信号来行事，几乎不会忽略任何噪声信号。人们可能会天真地认为，为了最大限度地减少噪声干扰， g 的值应该始终很小。但实际上，这样做的后果就是我们几乎完全忽略了实际测量结果，而只依赖预测值。那么，当信号的变化并非由噪声引起，而是由实际状态的变化所导致时，又会怎样呢？让我们来仔细看看。我会创建这样一些数据：在 9 个步骤内， $\dot{x} = 1$ 的数值保持不变，之后才变为 $\dot{x} = 0$ 。

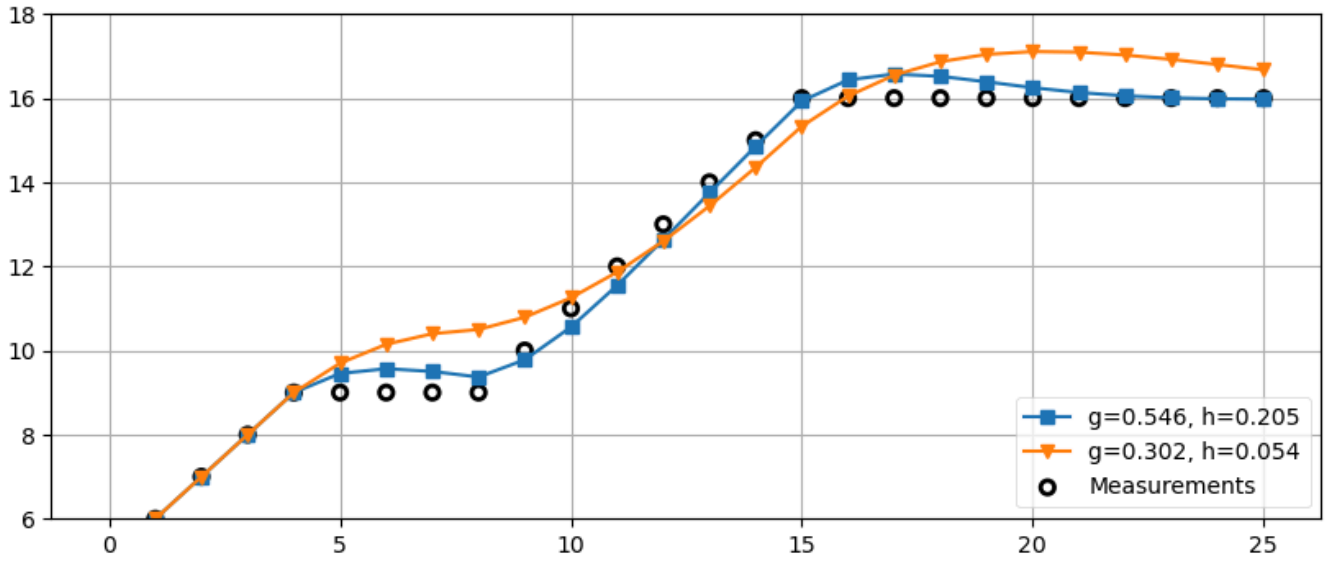
```
zs = [5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
for i in range(50):
    zs.append(14)

data1 = g_h_filter(data=zs, x0=4., dx=1., dt=1., g=0.1, h=0.01)
data2 = g_h_filter(data=zs, x0=4., dx=1., dt=1., g=0.5, h=0.01)
data3 = g_h_filter(data=zs, x0=4., dx=1., dt=1., g=0.9, h=0.01)

book_plots.plot_measurements(zs)
book_plots.plot_filter(data1, label='g=0.1', marker='s', c='C0')
book_plots.plot_filter(data2, label='g=0.5', marker='v', c='C1')
book_plots.plot_filter(data3, label='g=0.9', c='C3')
plt.legend(loc=4)
plt.ylim([6, 20]);
```



在这里，我们可以看到忽略该信号所带来的后果。我们不仅滤除了噪声，同时也把信号中那些正常的、有意义的变动给滤除了。

也许我们需要一种“恰到好处的”滤波机制：让 g 的值既不太大也不太小，而是处于恰到好处的水平。不过，其实并非完全如此。正如前面所提到的，不同的滤波机制会根据问题的数学特性来选择不同的 g 和 h 值。例如，Benedict-Bordner 滤波机制就是为了最小化这种例子中的瞬态误差而设计的，其中 $\dot{x}$ 的值会发生突变。我们在本书中不会详细讨论这种滤波机制，不过这里有两组不同的 g 和 h 组合所对应的曲线图。这种滤波机制能够有效减少 $\dot{x}$ 发生突变时的瞬态误差，但代价是它并不适合处理 $\dot{x}$ 的其他类型的变化。

```
zs = [5,6,7,8,9,9,9,9,9,10,11,12,13,14,
      15,16,16,16,16,16,16,16,16,16,16,16]
data1 = g_h_filter(data=zs, x0=4., dx=1., dt=1., g=.302, h=.054)
data2 = g_h_filter(data=zs, x0=4., dx=1., dt=1., g=.546, h=.205)

book_plots.plot_measurements(zs)
book_plots.plot_filter(data2, label='g=0.546, h=0.205', marker='s', c='C0')
book_plots.plot_filter(data1, label='g=0.302, h=0.054', marker='v', c='C1')
plt.legend(loc=4)
plt.ylim([6, 18]);
```

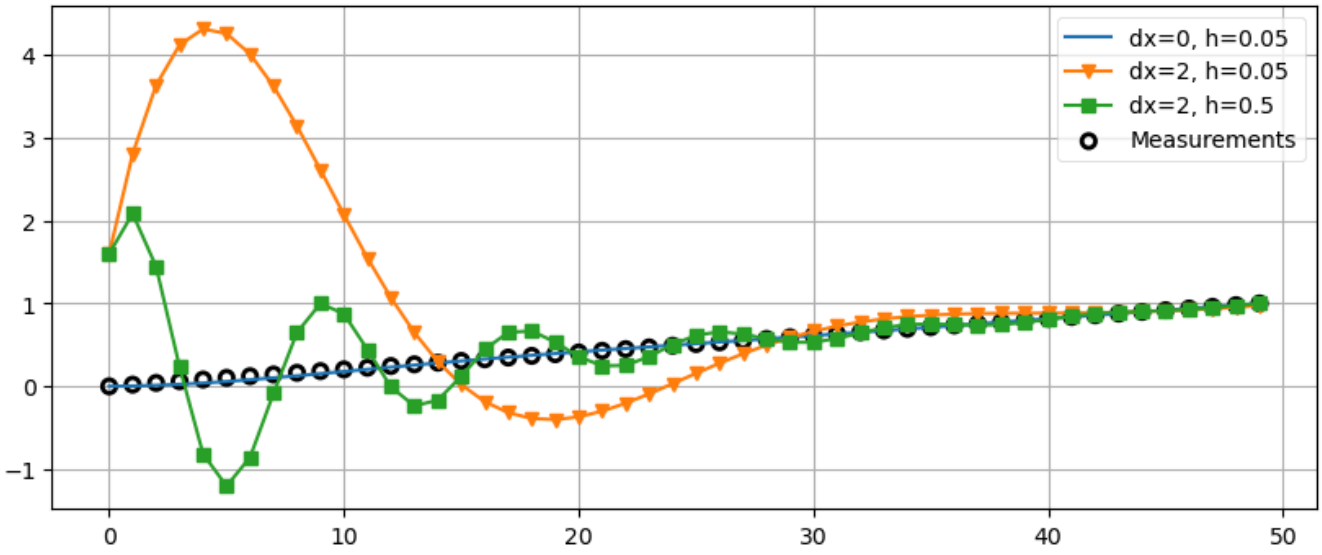


各不相同 h

现在，让我们保持 g 不变，来研究修改 h 所带来的影响。我们知道， h 会影响我们倾向于使用 $\dot{x}$ 的测量结果还是自己的预测结果。但这到底意味着什么呢？如果我们的信号变化很大（相对于滤波器的时间步长来说变化很快），那么较大的 h 值会让我们更快地对那些瞬时变化做出反应。而较小的 h 值则会导致我们反应得更慢。

我们来看三个例子。第一个例子中，测量值从 0 逐渐增加到 1，共分为 50 个步骤。该滤波器使用了几乎正确的初始值来设定 $\dot{x}$ ，同时 h 的数值也很小。从输出结果可以看出，滤波后的信号与原始信号非常接近。第二个例子中， $\dot{x} = 2$ 的初始值被设定得相当不准确。可以看到，滤波器的输出信号会持续波动，直到最终稳定下来并捕捉到原始信号。第三个例子中，虽然各种参数与第二个例子相同，但 $h = 0.5$ 的初始值被重新设定。从波动的幅度来看，第三个例子中的波动幅度明显小于第二个例子，不过频率则更高。此外，第三个例子的滤波器达到稳定状态的速度也稍快一些，但差距不大。

```
zs = np.linspace(0, 1, 50)

data1 = g_h_filter(data=zs, x0=0, dx=0., dt=1., g=.2, h=0.05)
data2 = g_h_filter(data=zs, x0=0, dx=2., dt=1., g=.2, h=0.05)
data3 = g_h_filter(data=zs, x0=0, dx=2., dt=1., g=.2, h=0.5)

book_plots.plot_measurements(zs)
book_plots.plot_filter(data1, label='dx=0, h=0.05', c='C0')
book_plots.plot_filter(data2, label='dx=2, h=0.05', marker='v', c='C1')
book_plots.plot_filter(data3, label='dx=2, h=0.5',  marker='s', c='C2')
plt.legend(loc=1);
```



交互式示例

对于那些在 Jupyter Notebook 中运行该代码的用户，我编写了该滤波器的交互式版本。这样，您就可以实时查看当 x 、 g 和 h 的值发生变化时，会产生什么效果。当您调整 x 、 g 和 h 的数值时，数据会重新被滤波处理，相应的结果也会被绘制出来供您查看。

如果你真的想考验一下自己，那就请阅读下一段文字，试着在移动滑块之前预测出结果吧。

可以尝试以下设置：将 g 和 h 设置为最小值。看看滤波器对数据的追踪效果如何！之所以能实现完美的追踪，是因为我们成功预测了重量的变化情况。将 x 的值调整为大于或小于 5。此时，滤波器会偏离数据点，而无法再次与数据点重合。然后试着逐步增加 g 或 h 的值，观察滤波器是如何重新回到数据点附近的。比较仅添加 g 与仅添加 h 时的效果差异。你能解释这种差异的原因吗？接着，尝试将 g 的值设置为大于 1。你能解释此时的结果吗？将 g 的值恢复为合理的数值（比如 0.1），同时将 h 的值设置得很大。你能解释这些结果吗？最后，将 g 和 h 都设置为最大值。

如果你想进一步探索这个现象，可以将数组 `zs` 的值更改为上述图表中使用的数值，然后重新运行该单元格，查看结果。

```
from ipywidgets import interact

# my FloatSlider returns an ipywidgets.FloatSlider with
# continuous_update=False. Filtering code runs too slowly
# to instantly react to slider changes.
from kf_book.book_plots import FloatSlider

zs1 = gen_data(x0=5, dx=5., count=100, noise_factor=50)

fig = None
def interactive_gh(x, dx, g, h):
    global fig
    if fig is not None: plt.close(fig)
    fig = plt.figure()
    data = g_h_filter(data=zs1, x0=x, dx=dx, g=g, h=h)
    plt.scatter(range(len(zs1)), zs1, edgecolor='k',
                facecolors='none', marker='o', lw=1)
```

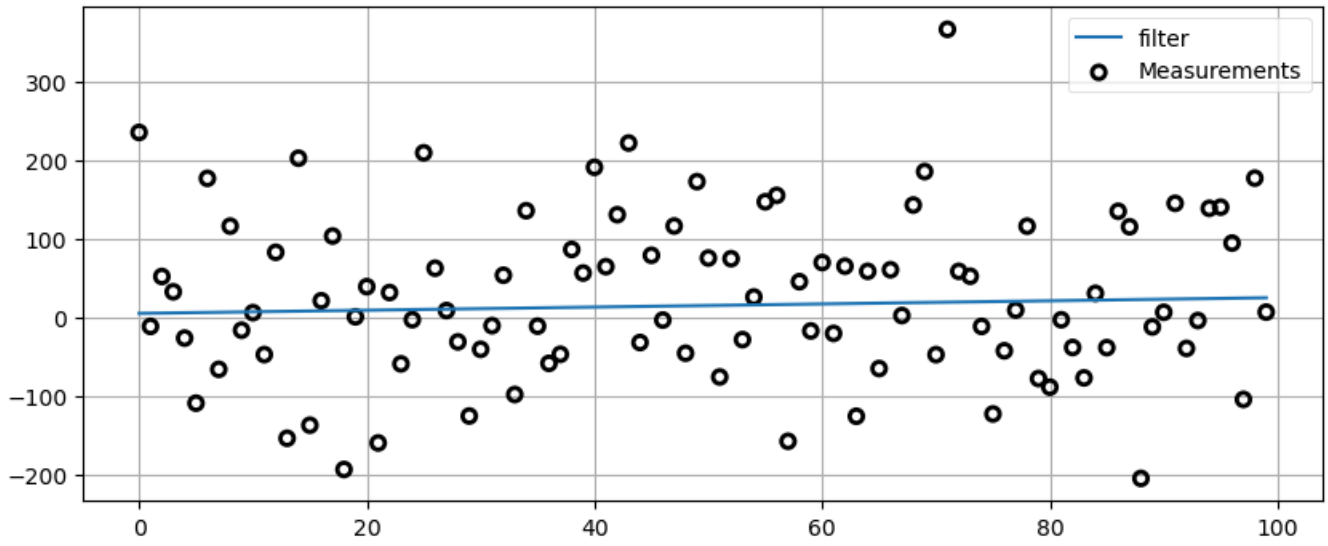
```
    plt.plot(data, color='b')
    plt.show()

interact(interactive_gh,
         x=FloatSlider(value=0, min=-200, max=200),
         dx=FloatSlider(value=5, min=-50, max=50),
         g=FloatSlider(value=.1, min=.01, max=2, step=.02),
         h=FloatSlider(value=.02, min=.0, max=.5, step=.01));
```

```
interactive(children=(FloatSlider(value=0.0, continuous_update=False, description='x', max=200.0,
min=-200.0),…
```

不要对滤波系统说谎

您可以随意将 g 和 h 设置为任意数值。这个滤波器即使在有强烈噪声的情况下也能保持出色的滤波效果。

```
zs = gen_data(x0=5., dx=.2, count=100, noise_factor=100)
data = g_h_filter(data=zs, x0=5., dx=.2, dt=1., g=0., h=0.)

book_plots.plot_measurements(zs)
book_plots.plot_filter(data, label='filter')
plt.legend(loc=1);
```

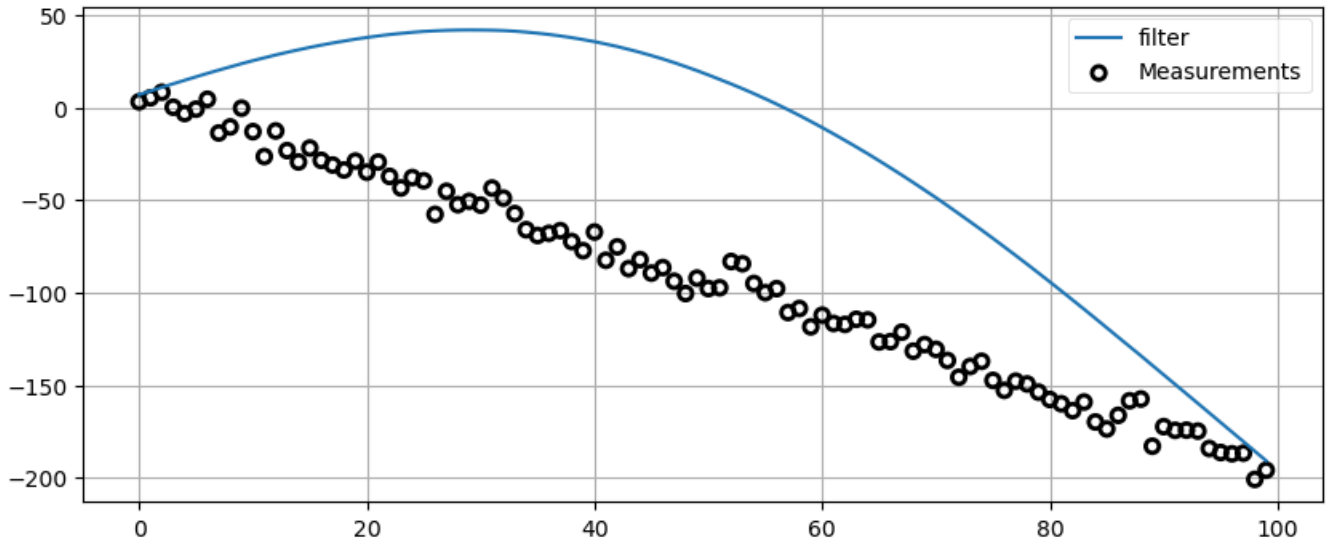


我成功地从充满噪声的数据中提取出了一条直线！也许我现在还不应该试图去争取数学界的菲尔兹奖。我之所以能做到这一点，是因为将 g 和 h 都设为 0。这样做有什么效果呢？它使得滤波器忽略掉所有的测量数据。因此，在每次更新时，系统都会以 $x + \Delta x \Delta t$ 作为新的位置值。当然，如果我们忽略所有的测量数据，最终得到的自然就是一条直线了。

那些忽略实际测量数据的滤波器毫无用处。我知道，你绝不会把 g 和 h 都设为零——因为那样做需要非凡的智慧，而这种智慧只有我具备。但我必须提醒你：如果不小心的话，你很可能会把这两个数值设置得过低。当然，你可以利用测试数据得到看似完美的结果。但当你把同样的滤波器应用到真实数据上时，你一定会失望的，因为你所调整的参数都是针对特定数据集而设定的。 g 和 h 应该反映的是你所要处理的系统的真实行为，而不是某个特定数据集的行为。在后面的章节中，我们会详细学习如何正确设置这些参数。现在我只能再说一

次：请务必小心行事。否则，虽然你用测试数据能得到“完美”的结果，但一旦使用真实数据，结果就会大相径庭了。

```
zs = gen_data(x0=5, dx=-2, count=100, noise_factor=5)
data = g_h_filter(data=zs, x0=5., dx=2., dt=1., g=.005, h=0.001)
book_plots.plot_measurements(zs)
book_plots.plot_filter(data, label='filter')
plt.legend(loc=1);
```



追踪火车的行进轨迹

我们准备通过一个具体的例子来加以说明。在本章的前面部分，我们讨论了如何追踪火车的行踪。火车重量大、速度慢，因此无法快速改变速度。由于火车行驶在铁轨上，它只能通过减速停车后再掉头来改变方向。因此，如果我们知道火车的大致位置和速度，那么我们就可以相当准确地预测它未来的位置。火车在一两秒钟内是无法显著改变其速度的。

那么，让我们来为火车设计一个滤波器吧。火车的位置是通过其相对于某个固定点的距离来表示的，我们把那个固定点视为 0 公里处。也就是说，如果火车的位置为 1 公里，那就意味着火车距离那个固定点 1 公里远。速度则以每秒米来表示。我们每秒测量一次火车的位置，测量误差为 500 米左右。那么，我们应该如何实现这个滤波器呢?

首先，让我们模拟一下没有滤波器的情况。假设火车当前位于 23 公里处，以 15 米/秒的速度行驶。我们可以这样来编码实现这一情况：

```
pos = 23*1000
vel = 15
```

现在，我们可以计算出在未来的某个时刻，列车的位置。当然，这是假设列车的速度保持不变的情况下。

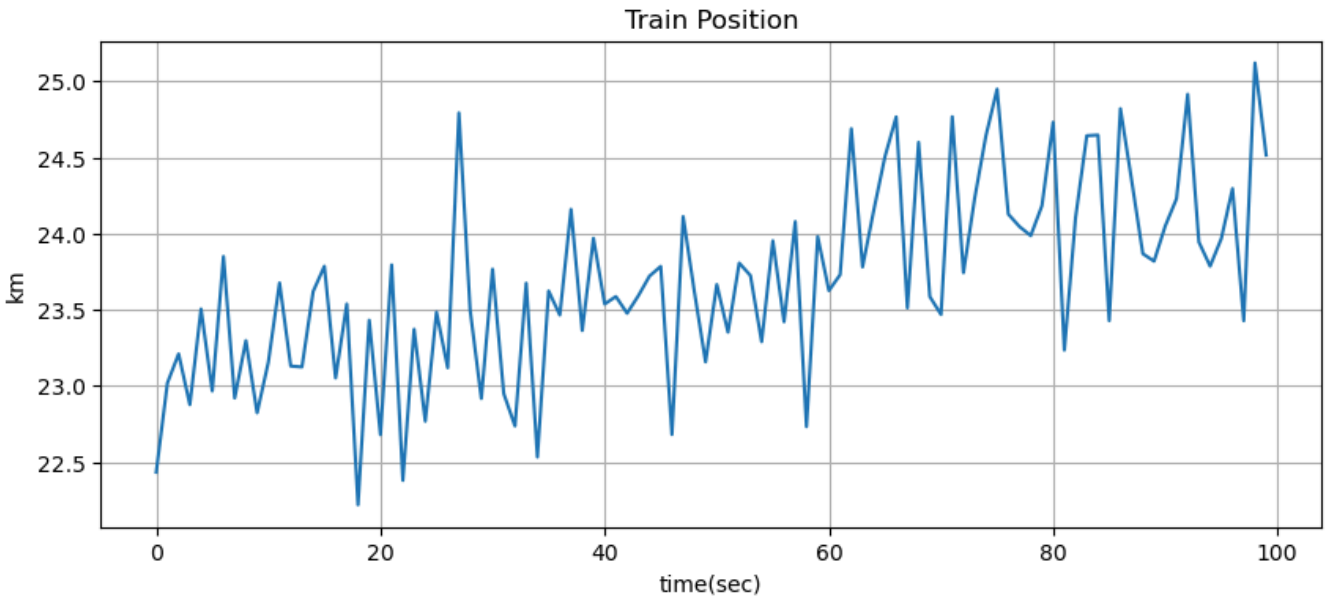
```
def compute_new_position(pos, vel, dt=1):
    return pos + (vel * dt)
```

我们可以通过在位置数据中加入一些随机噪声来模拟测量过程。这里的误差为 500 米，因此代码可能如下所示：

```
def measure_position(pos):
        return pos + random.randn()*500
```

让我们把数据放入一个数组中，然后绘制出 100 秒模拟过程的结果。我会使用 NumPy 的 `asarray` 函数将数据转换为 NumPy 数组。这样，我就可以使用’/’运算符来同时对数组中的所有元素进行除法运算了。

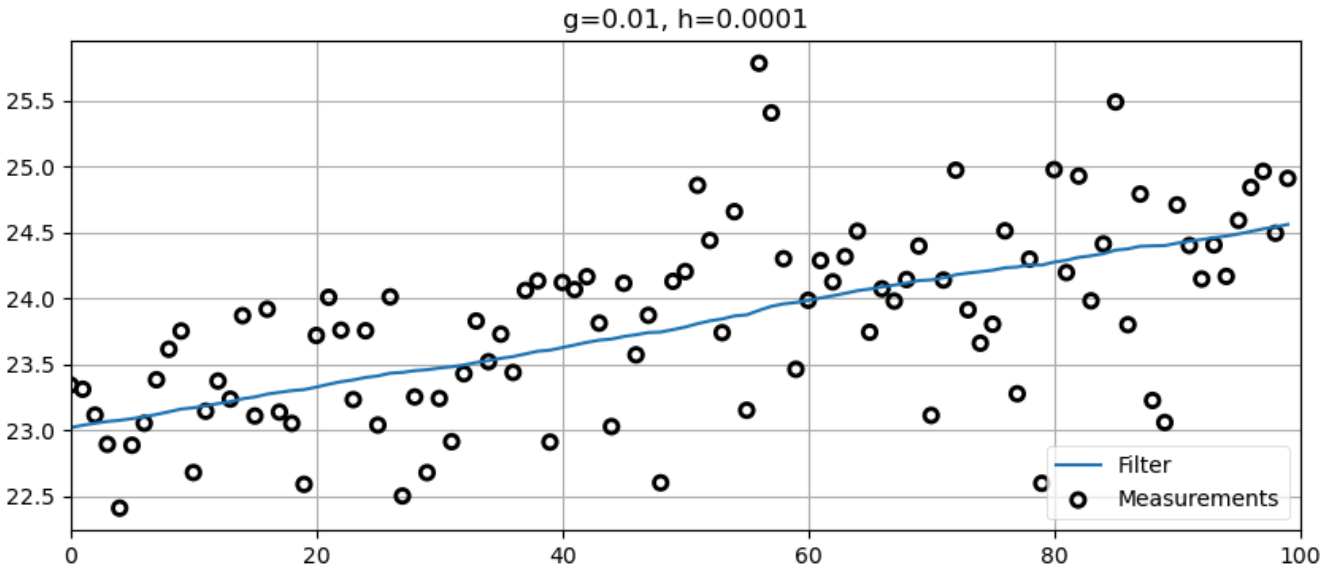
```
from numpy.random import randn

def compute_new_position(pos, vel, dt=1.):
    """ dt is the time delta in seconds."""
    return pos + (vel * dt)

def measure_position(pos):
    return pos + randn()*500

def gen_train_data(pos, vel, count):
    zs = []
    for t in range(count):
        pos = compute_new_position(pos, vel)
        zs.append(measure_position(pos))
    return np.asarray(zs)

pos, vel = 23.*1000, 15.
zs = gen_train_data(pos, vel, 100)

plt.plot(zs / 1000.)  # convert to km
book_plots.set_labels('Train Position', 'time(sec)', 'km')
```



从图表中可以看出，这些测量数据糟糕透顶。没有任何一列真正的火车能够以这样的方式行驶。

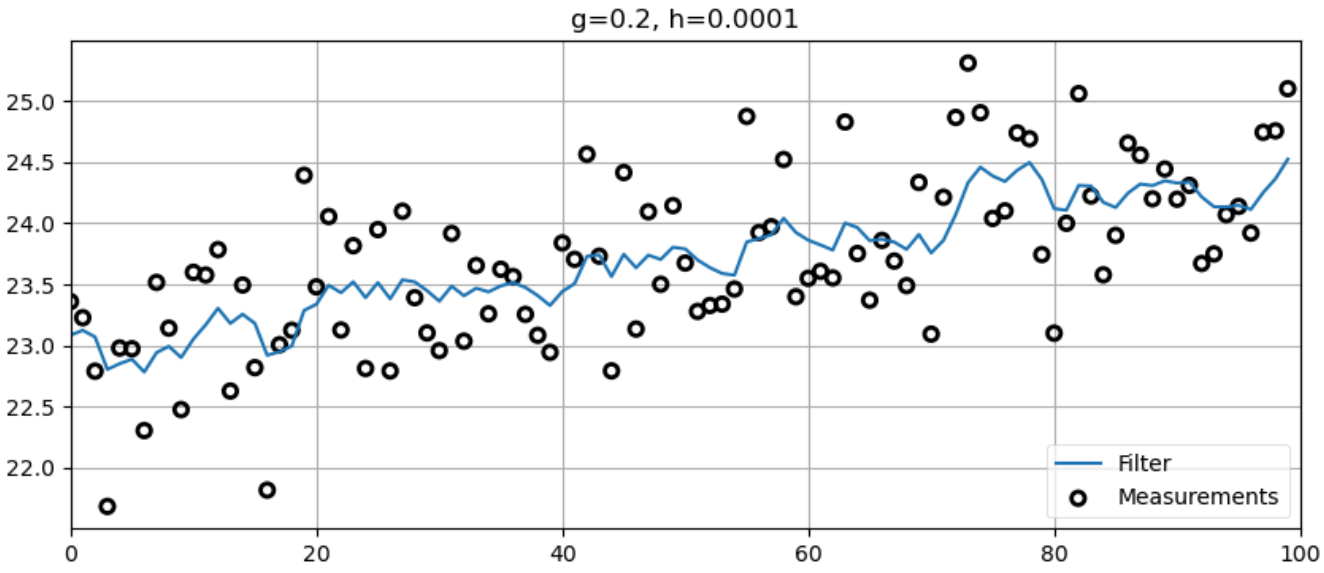
那么，如果我们想滤波这些数据，应该将 g 和 h 设置为多少呢？虽然我们还没有为此制定出具体的理论依据，但我们可以凭直觉来尝试找出一个合理的数值。我们知道，这些测量数据的准确性很低，因此我们不想给它们太大的权重。为此，我们需要将 g 设置得非常小。另外，由于火车无法快速加速或减速，所以 h 也应该设置得非常小。例如：

```
zs = gen_train_data(pos=pos, vel=15., count=100)
data = g_h_filter(data=zs, x0=pos, dx=15., dt=1., g=.01, h=0.0001)
plot_g_h_results(zs/1000., data/1000., 'g=0.01, h=0.0001')
```



作为初步的猜测，这个结果相当不错。让我们把 g 的值调大一些，看看会有什么效果。

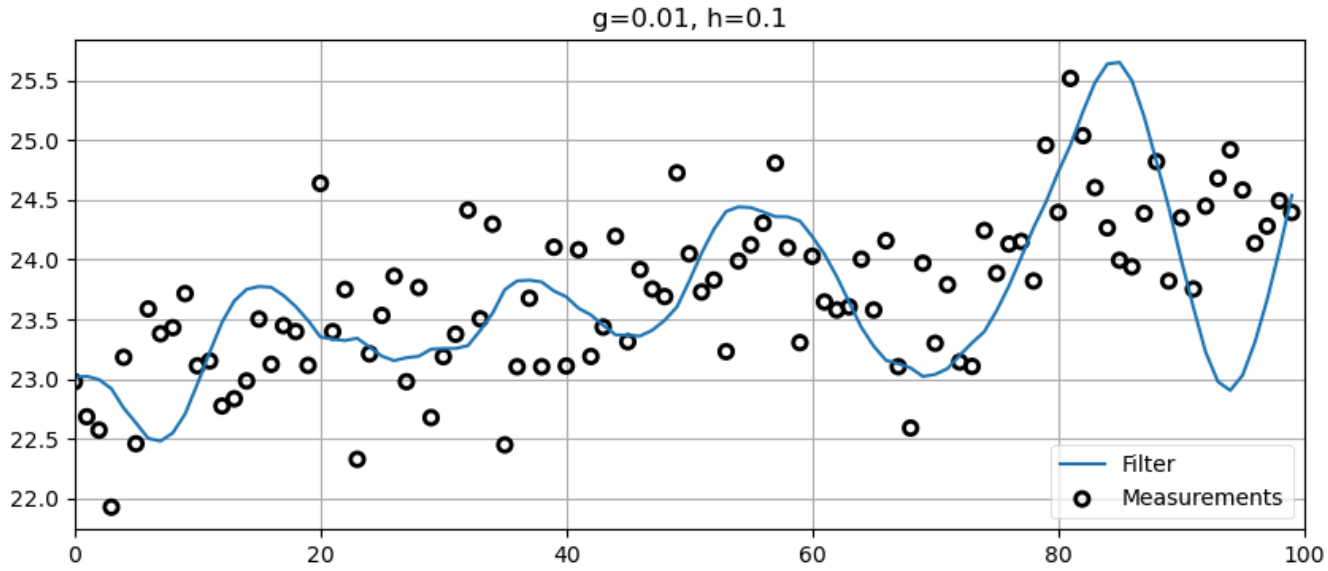
```
zs = gen_train_data(pos=pos, vel=15., count=100)
data = g_h_filter(data=zs, x0=pos, dx=15., dt=1., g=.2, h=0.0001)
plot_g_h_results(zs/1000., data/1000., 'g=0.2, h=0.0001')
```



我们得到了 g=0.2 的结果。可以看出，虽然列车的位置被平滑处理过了，但估计出的位置（进而速度）在极短的時間间隔内会有很大的波动，这种波动幅度远远超过了真实列车的正常波动范围。因此，从实际经验来看，我们更倾向于使用 g<<0.2 。

现在，让我们来看看对 h 做出错误选择会带来怎样的后果吧。

```
zs = gen_train_data(pos=pos, vel=15., count=100)
data = g_h_filter(data=zs, x0=pos, dx=15., dt=1., g=0.01, h=0.1)
plot_g_h_results(zs/1000., data/1000., 'g=0.01, h=0.1')
```

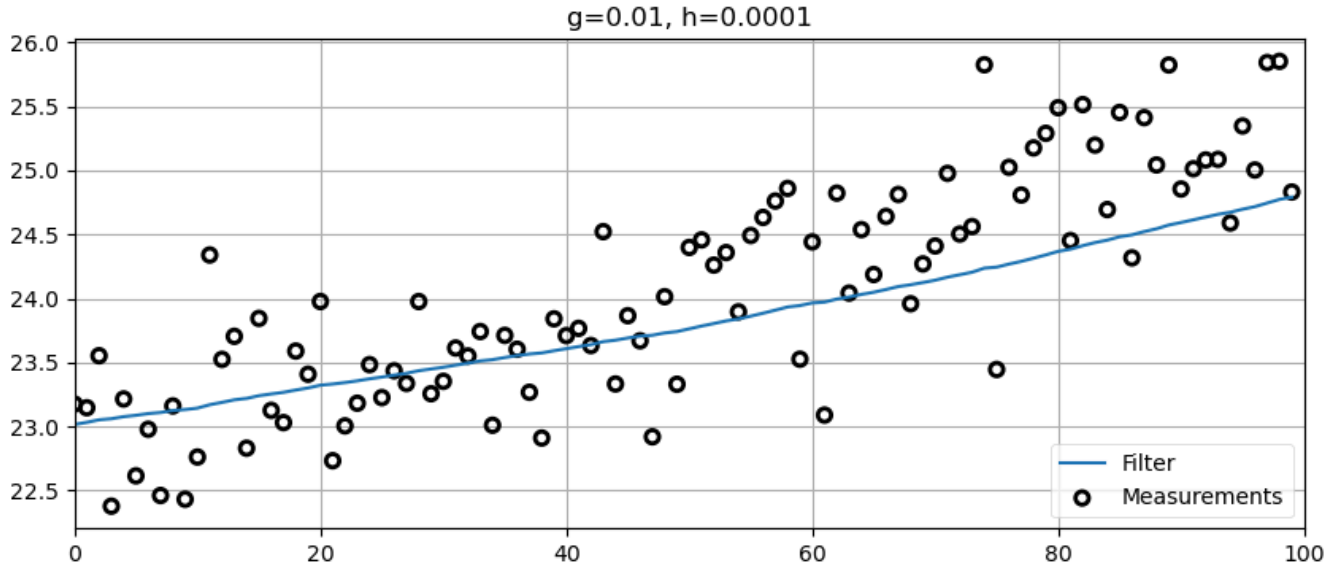


在这里，由于 g 的数值较小，因此位置的变动较为平滑。但 h 的数值较大，导致滤波器对各种测量数据的反应过于敏感。这是因为在几秒钟的时间里，数据的快速变化意味着速度也有很大的变动。而较大的 h 数值则迫使滤波器迅速对这些变化做出反应。不过，火车无法如此快速地改变速度，因此滤波器在处理数据时效果不佳——滤波器反应的速度远远快过了火车的反应速度。

最后，让我们给这列火车加上一些加速度吧。我不知道火车的实际加速度有多大，但假设它的加速度为 0.2 米/秒^{2}吧。

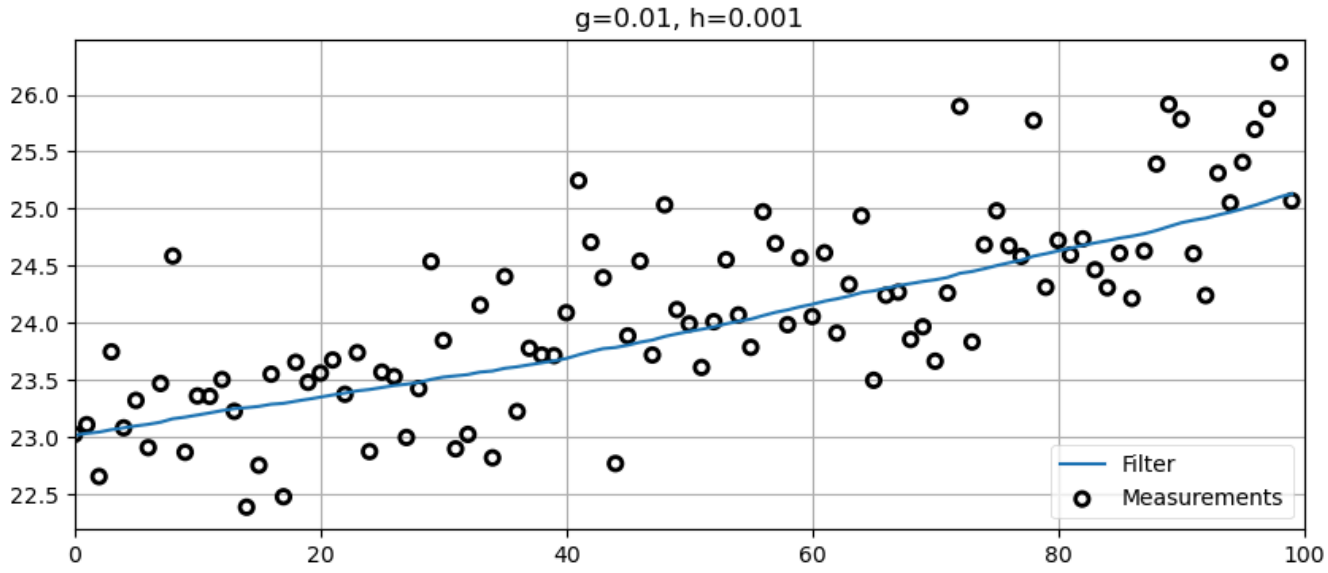
```
def gen_train_data_with_acc(pos, vel, count):
    zs = []
    for t in range(count):
        pos = compute_new_position(pos, vel)
        vel += 0.2
        zs.append(measure_position(pos))
    return np.asarray(zs)
```

```
zs = gen_train_data_with_acc(pos=pos, vel=15., count=100)
data = g_h_filter(data=zs, x0=pos, dx=15., dt=1., g=.01, h=0.0001)
plot_g_h_results(zs/1000., data/1000., 'g=0.01, h=0.0001')
```



可以看到，由于加速度的作用，滤波器已经无法准确跟踪列车的运动了。我们可以通过调整 h 的参数来改善这一情况，不过这样一来，滤波后的结果就会不够平滑。

```
zs = gen_train_data_with_acc(pos=pos, vel=15., count=100)
data = g_h_filter(data=zs, x0=pos, dx=15., dt=1., g=.01, h=0.001)
plot_g_h_results(zs/1000., data/1000., 'g=0.01, h=0.001')
```



这里有两个值得借鉴的教训。首先，可以利用 h 这个参数来应对那些未被纳入模型中的速度变化。但更重要的是，如何在快速、准确地响应行为变化与确保系统在稳定状态时能够输出理想结果之间找到平衡。如果列车的速度始终不变，我们可以将 h 的值设得极小，从而避免滤波后的数值受到测量噪声的过度影响。但在现实情况中，系统的状态几乎总是在变化，我们需要尽快对这些变化做出反应。不过，反应越快，传感器中的噪声带来的影响就越大。

我还可以继续说下去，但我的目的并非要详细阐述 G-H 滤波器的相关理论，而是想深入理解：如何通过结合各种测量数据和预测结果来得到最终的滤波结果。关于如何为这类问题选择合适的参数 g 和 h，已有大量研究文献。为了实现不同的目标，也有相应的最佳选择方法。正如我之前所解释的，使用这样的测试数据进行实验

时，很容易让滤波器“产生错误判断”。在后面的章节中，我们将了解卡尔曼滤波器是如何以类似的方式解决这个问题的，只不过它所使用的数学方法要复杂得多。

使用 FilterPy 实现的 g-h 滤波器

FilterPy 是我自己编写的开源滤波库。它包含了本书中提到的所有滤波功能，此外还有更多其他功能。虽然自己编写 g-h 滤波器相当简单，但随着学习的深入，我们会越来越依赖 FilterPy。作为简要介绍，让我们先来看看 FilterPy 中的 g-h 滤波器吧。

如果您还没有安装 FilterPy，请在命令行中输入以下命令即可。

```
pip install filterpy
```

如需了解如何从 GitHub 安装或下载 FilterPy 的更多详细信息，请参阅附录 A。

要使用 g-h 滤波器，需先将其导入，并通过 `GHFilter` 类来创建该滤波器的实例。

```
from filterpy.gh import GHFilter
f = GHFilter(x=0., dx=0., dt=1., g=.8, h=.2)
```

在创建该对象时，需要指定信号的初始值和变化率（即 `x` 和‘dx’）、更新之间的时间间隔（ `dt` ），以及两个滤波器参数的值（ `g` 和 `h` ）。 `dx` 的单位必须与 `x` / `dt` 的单位相同——如果 `x` 的单位是米， `dt` 的单位是秒，那么 `dx` 的单位就必须是米每秒。

要运行该滤波器，需调用 update 函数，并将测量结果作为参数 `z` 传递进去。需要提醒的是， `z` 在相关文献中是用于表示测量结果的标准名称。

```
f.update(z=1.2)
```

```
(0.96, 0.24)
```

`update()` 会以元组的形式返回 `x` 和 `dx` 的新值，不过你也可以直接通过该对象来访问它们。

```
print(f.x, f.dx)
```

```
0.96 0.24
```

你可以动态地更改 `g` 和 `h` 的值。

```
print(f.update(z=2.1, g=.85, h=.15))
```

```
(1.965, 0.375)
```

您可以批量筛选一系列测量结果。

```
print(f.batch_filter([3., 4., 5.]))
```

```
[[1.965 0.375]
 [2.868 0.507]
 [3.875 0.632]
 [4.901 0.731]]
```

你可以对多个自变量进行筛选。如果你要追踪某架飞机，就需要在三维空间中对其进行跟踪。请使用 NumPy 数组来存储 `x` 、 `dx` 以及各种测量数据。

```
x_0  = np.array([1., 10., 100.])
dx_0 = np.array([10., 12., .2])

f_air = GHFilter(x=x_0, dx=dx_0, dt=1., g=.8, h=.2)
f_air.update(z=np.array((2., 11., 102.)))
print(' x =', f_air.x)
print('dx =', f_air.dx)
```

```
 x = [  3.8   13.2  101.64]
dx = [8.2  9.8  0.56]
```

`GHFilterOrder` 这个类允许你创建 0 阶、1 阶或 2 阶的滤波器。g-h 滤波器属于 1 阶滤波器。至于我们尚未讨论过的 g-h-k 滤波器，它则能够检测加速度信息。这两种滤波器都具备实际应用中所需的功能，比如计算方差降低因子（VRF），不过这一点我们在这章中没有详细讨论。关于 g-h 滤波器的理论及其应用，完全可以写成一本书籍。不过，本书还有其他目标需要实现。如果你感兴趣的话，可以仔细研究 FilterPy 中的相关代码，并进一步阅读相关资料。

FilterPy 的文档地址为：https://filterpy.readthedocs.org/。

摘要/总结

我建议您尝试使用这个滤波器，从而更好地了解它的运作方式。不用试太多次，您就会发现，对 g 和 h 的随意选择往往不会带来好的效果。某个特定的选择在某种情况下可能效果不错，但在另一种情况下则可能适得其反。即便您理解了 g 和 h 的作用，要选择合适的数值也并非易事。事实上，您很难为 g 和 h 选出在特定问题上最理想的数值。滤波器是经过精心设计的，而不是随意挑选出来的。

在某种程度上，我并不想就此结束这一章的讨论。因为关于如何选择 g 和 h ，我们还有很多话可以说。不过，这种形式的 g-h 滤波器并非本书的重点内容。设计卡尔曼滤波器时，需要指定许多参数——这些参数间接地与 g 和 h 的选择有关，但在实际设计卡尔曼滤波器时，我们并不会直接涉及这些参数。此外， g 和 h 的值会在每个时间点上以一种难以预测的方式发生变化。

这些滤波器还有另一个我们几乎没怎么提及的特性——贝叶斯统计。你会注意到，“贝叶斯”这个词出现在了本书的标题中；这绝非巧合！目前，我们暂时先不讨论 g 和 h 这两个概念，因为它们还有待进一步研究。我们将重点探讨一种非常有效的概率推理方法，用于滤波处理。不过，稍后我们会再次用到这种 G-H 滤波算法。这一

次，它将融入一套完善的数学框架，让我们能够利用多个传感器来构建滤波器，准确评估解决方案中的误差，并从而控制机器人。

第 2 章：离散贝叶斯滤波

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

卡尔曼滤波器属于贝叶斯滤波器这一类滤波器。在大多数关于卡尔曼滤波器的教材中，都会介绍贝叶斯公式，也会说明该公式是如何被运用到卡尔曼滤波器方程中的。不过，这些介绍大多停留在非常抽象的层面。

这种方法需要读者对多个数学领域有相当深入的理解。不过，对于如何理解并直观地把握相关情况，大部分工作仍需由读者自己来完成。

我会用另一种方式来探讨这个主题。在这一点上，我深受 Dieter Fox 和 Sebastian Thrun 的启发。其核心思路是通过让物体在走廊中移动来帮助人们理解贝叶斯统计的原理——他们使用机器人来模拟物体移动，而我则用狗来代替。我喜欢狗，不过狗的行为比机器人更难以预测，这就给数据处理带来了挑战。据我所知，关于这一方法的最早研究文献是 Fox 在 1999 年发表的论文[1]，更详细的介绍则见于 Fox 在 2003 年的论文[2]。Sebastian Thrun 也在他那门精彩的 Udacity 课程《人工智能与机器人技术》中采用了这种方法[3]。如果你喜欢观看视频，我强烈建议先暂停阅读本书，先学习该课程的前几节课，然后再回来仔细阅读本书，以便更深入地理解这个主题。

现在，让我们来做一个简单的思维实验，就像我们之前对 g-h 滤波器所做的那样。通过这个实验，我们可以了解如何利用概率来进行数据的滤波和追踪处理。

追踪一只狗

让我们从一个简单的问题开始吧。我们的工作场所允许狗进入，因此人们会带着自己的狗来上班。不过，有时候狗会离开办公室，跑到走廊里去。我们希望能够追踪到这些狗的行踪。在一次黑客马拉松活动中，有人发明了一种声纳传感器，可以安装在狗的项圈上。该传感器会发出信号，然后接收反射回来的回声。根据回声返回的速度，我们就能判断狗是否位于某个敞开的门口前。此外，该传感器还能检测到狗的移动情况，并报告狗的移动方向。该传感器通过 Wi-Fi 与网络相连，每秒会发送一次更新信息。

我想追踪我的狗西蒙的行踪。于是，我把该设备安装在西蒙的项圈上，然后启动 Python 编程语言，准备编写代码来追踪西蒙在楼内的位置。乍看之下，这似乎是不可能的。如果我监听西蒙项圈上的传感器数据，可能会听到“门”、“走廊”之类的信息。那我该如何利用这些信息来确定西蒙的具体位置呢?

为了便于处理，我们会假设走廊上只有 10 个位置，这些位置分别用 0 到 9 来编号，其中 1 号位置位于 0 号位置的右侧。出于稍后会解释的原因，我们还会假设走廊是圆形或矩形的。如果你从 9 号位置向右移动，就会回到 0 号位置。

当我开始监听传感器的信号时，我没有理由认为西蒙一定位于走廊的某个特定位置。在我看来，他有可能处于走廊的任何一个位置。走廊总共有 10 个位置，因此他处于任意一个位置的概率都是 1/10。

让我们用 NumPy 数组来表示我们对他在该位置上的角色的看法吧。虽然也可以使用 Python 列表，但 NumPy 数组具备一些我们很快就会用到的功能。

```
import numpy as np
belief = np.array([1/10]*10)
print(belief)
```

```
[0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]
```

在贝叶斯统计学中，这被称为“先验概率”。所谓先验概率，指的是在获取任何测量数据或其它信息之前的概率值。更准确地说，这就是所谓的先验概率分布。概率分布指的是某一事件所有可能发生的概率的集合。由于总会有某件事情发生，因此概率分布的各项概率之和必定为 1。概率分布列出了所有可能发生的事件以及每种事件的概率。

我相信你以前肯定用过概率这个概念——比如“今天下雨的概率是 30%”。最后一段的内容其实也属于概率的范畴。不过，贝叶斯统计学则代表了概率论领域的一场革命，因为它将概率视为对某一事件的信念或看法。我们来看个例子：我知道，如果我把一枚公平的硬币无限次地抛掷下去，那么正面和反面出现的概率各为 50%。这种统计方法被称为频率主义统计学，以区别于贝叶斯统计学。贝叶斯统计学的计算方式是基于事件发生的频率来进行的。

我再抛一次硬币，然后让它自然落地。我觉得硬币会落在哪一面呢？频率主义概率论对此无能为力；它只会说，50%的抛硬币结果会是正面。从某种意义上说，给硬币的当前状态赋予概率是没有意义的。硬币要么是正面，要么是反面，我们只是不知道到底是哪一面而已。贝叶斯理论则将此视为对某一事件的“信念”——也就是说，我认为这次抛硬币结果是正面的概率是 50%。不过，有些人反对使用“信念”这个词；因为“信念”可能意味着在没有证据的情况下仍然认为某事为真。在这本书中，“信念”始终指的是我们对某事的了解程度。我们会在后续的内容中进一步了解这一点。

贝叶斯统计学会考虑过去的各种信息。比如，我们观察到每 100 天中有 4 天下雨。基于此，我们可以推断出明天下雨的概率为 1/25。不过，天气预报并非如此进行。如果今天正在下雨，且风暴系统没有移动，那么明天很可能会继续下雨。天气预报其实是一种贝叶斯统计方法。

在实践中，统计学家会结合使用频率主义和贝叶斯主义的方法。有时候，确定先验概率很困难或根本不可能，这时就需要采用频率主义方法。而在本书中，我们能够确定先验概率。当我谈论某件事的概率时，我指的是在已知过去事件的情况下，该事件发生的概率。在这种情况下，我采用的是贝叶斯主义方法。

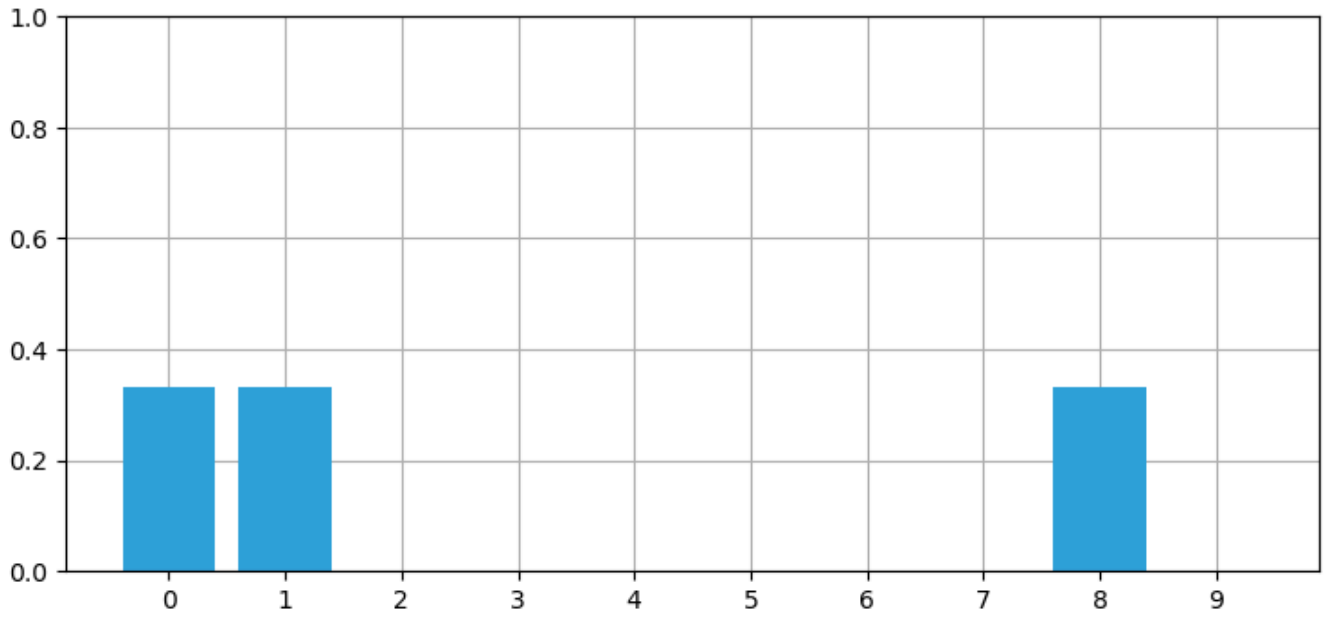
现在，让我们来绘制一下走廊的地图吧。我们将把前两扇门放得靠得很近，然后再放一扇门，距离稍远一些。我们用 1 来表示门，用 0 来表示墙壁：

```
hallway = np.array([1, 1, 0, 0, 0, 0, 0, 0, 1, 0])
```

我开始收听西蒙通过网络发送的信息。从传感器那里得到的第一组数据是“门”。暂时假设传感器总是能给出正确的答案。由此可以推断，他现在正站在某扇门的前面，但到底是哪扇门呢？我没有理由认为他站在第一扇、

第二扇或第三扇门的前面。我能做的就是给每扇门分配一个概率值。由于三扇门的可能性都相等，所以每扇门被分配的概率都是 1/3。

```
import kf_book.book_plots as book_plots
from kf_book.book_plots import figsize, set_figsize
import matplotlib.pyplot as plt

belief = np.array([1/3, 1/3, 0, 0, 0, 0, 0, 0, 1/3, 0])
book_plots.bar_plot(belief)
```



这种分布被称为分类分布，是一种用于描述观察到特定结果的概率的离散分布。由于我们对狗所在的位置有几种不同的猜测，因此这是一种多模态分布。当然，我们并不是说狗同时处于三个不同的位置上，而只是说我们的判断范围被限制在这三个位置之中。根据我的贝叶斯推断，狗在 0 号门的概率为 33.3%，在 1 号门的概率也为 33.3%，在 8 号门的概率同样是 33.3%。

这在两个方面都带来了改进。我之前认为有一些职位根本不可能被填补，现在这些职位都被排除了。而对于那些仍有可能被填补的职位，我的信心也从 10%上升到了 33%。这种情况会一直存在：随着我们了解得越多，各种职位被填补的概率就会越接近 100%。

关于分布的众数，简单来说：给定一组数字，比如{1, 2, 2, 2, 3, 3, 4}，众数就是出现次数最多的那个数字。在这个例子中，众数是 2。一个分布可以有多个众数。例如，{1, 2, 2, 2, 3, 3, 4, 4, 4}这个集合的众数是 2 和 4，因为这两个数字都出现了三次。我们把前者称为单峰分布，后者则称为多峰分布。

这种分布形态也被称为直方图。直方图以图形方式展示了某一组数据的分布情况。上图就是一幅直方图。

我在上面的代码中手动编写了 `belief` 数组的相关处理。那么，我们该如何在代码中实现这一功能呢？我们可以用 1 来表示门，用 0 来表示墙。因此，我们需要将“走廊”对应的数值乘以相应的百分比，具体操作如下：

```
belief = hallway * (1/3)
print(belief)
```

```
[0.333 0.333 0.    0.    0.    0.    0.    0.    0.333 0.   ]
```

从传感器读数中提取信息

先暂且不谈 Python，让我们仔细思考一下这个问题。假设我们从西蒙的传感器中读取到了以下数据：

- 门
- 向右移动
- 门

我们能推断出西蒙的位置吗？当然可以！根据走廊的布局，只有唯一一个位置能够产生这样的传感器读数，那就是走廊的左端。因此，我们可以确定西蒙现在位于第二道门的前方。如果还不明白的话，不妨假设西蒙是从第二道或第三道门开始移动的。如果他那样做的话，他的传感器会检测到“墙壁”信号。但这与实际的传感器读数不符，所以可以排除这种可能性。用同样的逻辑来分析其他所有可能的起始位置后，我们可以得出结论：西蒙现在肯定位于第二道门的前方。我们的结论是：

```
belief = np.array([0., 1., 0., 0., 0., 0., 0., 0., 0.])
```

我设计了走廊的布局以及传感器的检测方式，以便能快速得到准确的答案。不过，现实中的问题往往没那么简单明了。不过，这个例子应该能帮助你理解其中的原理：最初，第一个传感器的检测结果表明西蒙可能位于几个位置，但概率都很低（0.333）。但在经过位置更新和再次检测后，我们就能更准确地确定他的位置了。你可以合理地推测，如果走廊很长，门的数量也很多，那么经过多次检测和位置更新后，我们要么能确定西蒙的具体位置，要么至少能把可能的位置范围缩小到几个选项之内。当所有的传感器检测结果都只与一两个初始位置相符时，这种情况就有可能发生。

我们现在就可以实施这个解决方案，不过，让我们先来考虑一下现实中可能出现的各种复杂情况吧。

噪音较大的传感器

完美的传感器其实很少见。也许，如果西蒙坐在门前挠痒痒的话，传感器就无法检测到那扇门；或者，如果他不是面朝走廊的方向，传感器也会给出错误的检测结果。因此，当我面对某扇门时，不能将 1/3 的概率分配给它。我必须给每扇门分配小于 1/3 的概率，同时也要给每面空墙分配很小的概率。大概就是这么回事吧。

```
[.31, .31, .01, .01, .01, .01, .01, .01, .31, .01]
```

起初，这似乎是无法克服的难题。如果传感器的信号有噪声，那么所有数据都不可信。如果我们始终无法确定数据的准确性，又如何能得出任何结论呢？

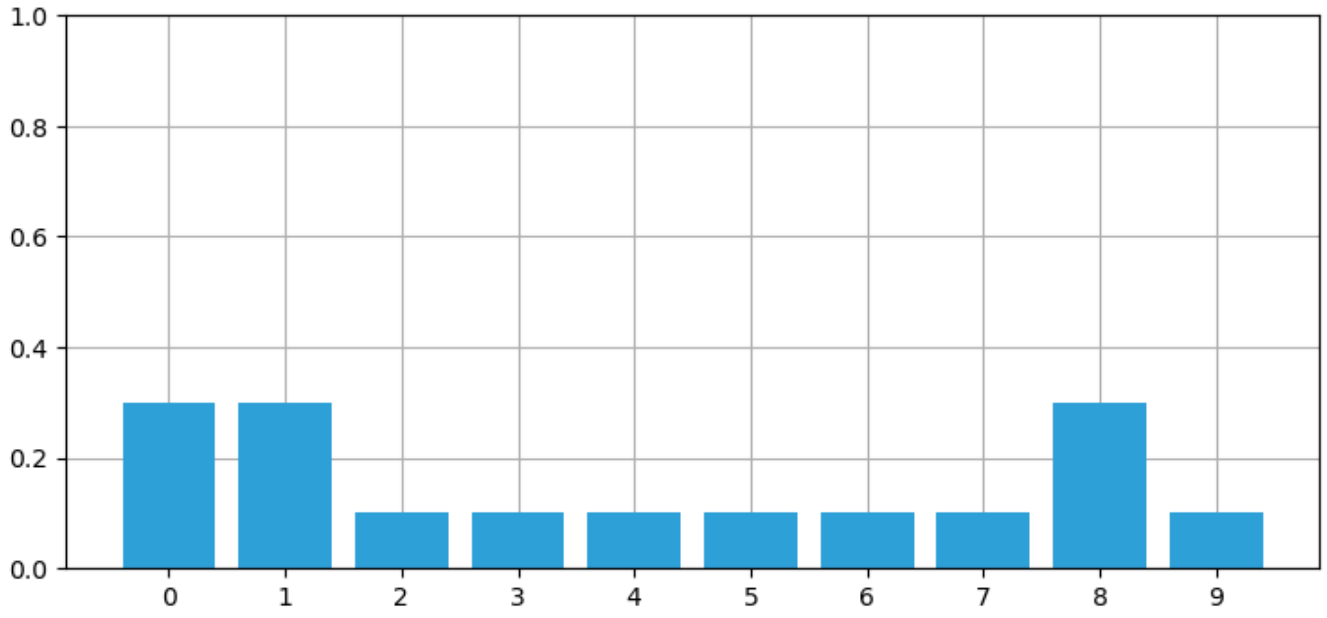
与上述问题一样，答案也与概率有关。我们已经能够为狗的位置赋予相应的概率值；现在，我们需要把传感器噪声带来的额外不确定性也考虑进去。

假设我们得到了关于“有门”的检测结果，且测试表明，该传感器的判断正确的概率是错误的 3 倍。那么，当检测结果为“有门”时，我们应该将概率分布的值乘以 3。不过，这样做之后，得到的就不再是一个真正的概率分布了。不过，我们稍后会知道如何解决这个问题。

让我们用 Python 代码来演示一下。在这里，我使用变量 z 来表示测量结果。在相关文献中， z 或 y 也是常用的变量名。作为程序员，我更喜欢使用有意义的变量名。不过，为了方便您阅读相关文献或其它滤波代码，我现在开始使用这些缩写形式。

```
def update_belief(hall, belief, z, correct_scale):
    for i, val in enumerate(hall):
        if val == z:
            belief[i] *= correct_scale

belief = np.array([0.1] * 10)
reading = 1 # 1 is 'door'
update_belief(hallway, belief, z=reading, correct_scale=3.)
print('belief:', belief)
print('sum =', sum(belief))
plt.figure()
book_plots.bar_plot(belief)
```

```
belief: [0.3 0.3 0.1 0.1 0.1 0.1 0.1 0.1 0.3 0.1]
sum = 1.6000000000000003
```



这并不是一种概率分布，因为它各元素的和并不等于 1.0。不过，代码的实现方式大体上是正确的：各个门的概率值被设置为 0.3，这个数值是墙壁概率值 0.1 的 3 倍。我们所需要做的，就是对结果进行归一化处理，使得所有概率值之和等于 1.0。归一化的方法是将列表中每个元素都除以该列表中所有元素的总和。使用 NumPy 的话，这一操作非常简单。

```
belief / sum(belief)
```

```
array([0.188, 0.188, 0.062, 0.062, 0.062, 0.062, 0.062, 0.062, 0.188,
       0.062])
```

FilterPy 通过 `normalize` 函数来实现这一功能：

```
from filterpy.discrete_bayes import normalize
normalize(belief)
```

说“正确的概率是错误的 3 倍”这种说法有点奇怪。我们处理的是概率问题，所以应该明确指出传感器判断正确的概率，再据此计算出相应的比例系数。相关的计算公式如下：

$$scale = \frac{prob_{correct}}{prob_{incorrect}} = \frac{prob_{correct}}{1 - prob_{correct}}$$

此外， `for` 循环的使用也很麻烦。一般来说，最好避免在 NumPy 代码中使用 `for` 循环。NumPy 是用 C 和 Fortran 语言实现的，因此，如果不用 for 循环，程序的执行速度通常会比使用 for 循环快 100 倍以上。

我们该如何摆脱这个 `for` 循环呢？NumPy 允许使用布尔数组来对数组进行索引操作。我们可以利用逻辑运算符来创建布尔数组。这样，我们就能找到走廊里所有的门了：

```
hallway == 1
```

```
array([ True,  True, False, False, False, False, False, False,  True,
       False])
```

当使用布尔数组作为另一个数组的索引时，该操作只会返回索引值为 `True` 的元素。因此，我们可以用这种方式来替代 `for` 循环。

```
belief[hall==z] *= scale
```

只有那些等于 `z` 的元素才会被 `scale` 乘以。

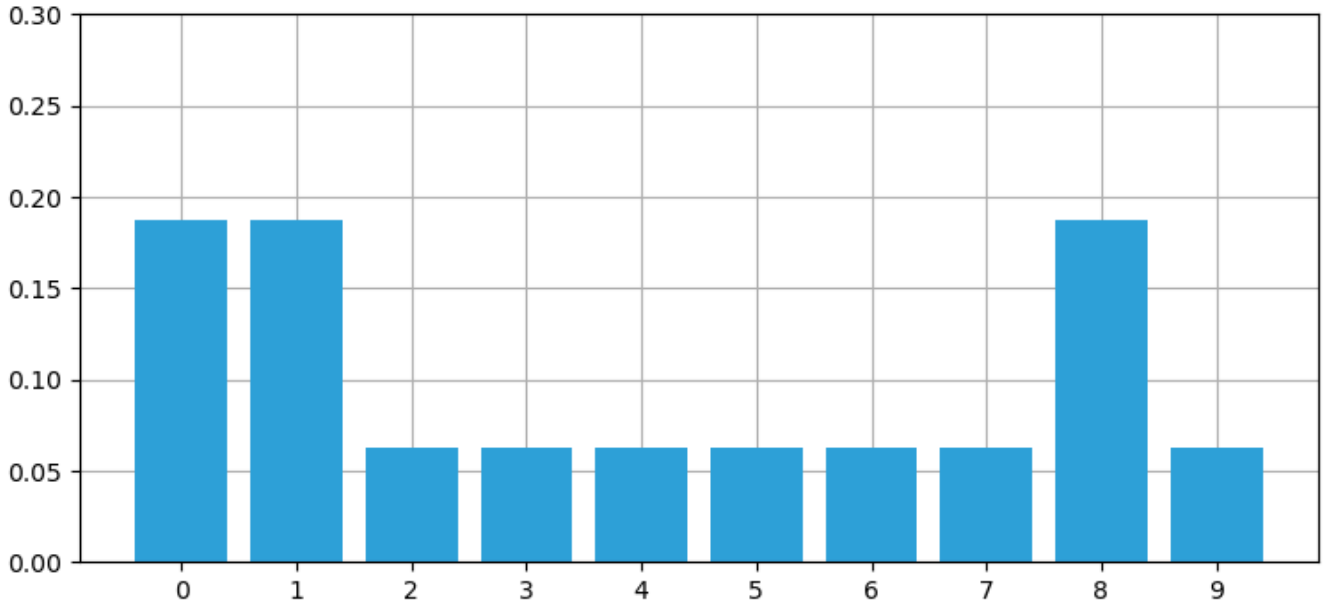
教授 NumPy 的使用方法超出了本书的范畴。我会使用 NumPy 中的常用语法，并在首次介绍时加以说明。如果您是 NumPy 的新手，网上有很多文章和视频介绍如何高效且恰当地使用 NumPy。

这是我们改进后的版本：

```
from filterpy.discrete_bayes import normalize

def scaled_update(hall, belief, z, z_prob):
    scale = z_prob / (1. - z_prob)
    belief[hall==z] *= scale
    normalize(belief)

belief = np.array([0.1] * 10)
scaled_update(hallway, belief, z=1, z_prob=.75)

print('sum =', sum(belief))
print('probability of door =', belief[0])
```

```
print('probability of wall =', belief[2])
book_plots.bar_plot(belief, ylim=(0, .3))
```

```
sum = 1.0
probability of door = 0.1875
probability of wall = 0.06249999999999999
```



从输出结果可以看出，现在的总和为 1.0。同时，门与墙的概率之比仍然是 3:1。这一结果符合我们的直觉：门的概率肯定小于 0.333，而墙的概率则肯定大于 0.0。此外，这也符合我们的另一个直觉：目前我们还没有任何信息可以用来区分不同的门或墙的位置。因此，所有门的概率都应该相同，墙的概率也应当如此。

这个结果被称为“后验概率分布”。所谓“后验概率分布”，指的是在纳入了测量数据之后的概率分布。简单来说，就是把测量数据考虑进去之后的概率分布情况。回顾一下，所谓“先验概率分布”，则是指在还没有纳入测量数据之前的概率分布。

另一个术语是“似然”。当我们计算 `belief[hall==z] *= scale` 时，实际上是在计算在给定测量结果的情况下，每个位置出现的可能性（不是概率）。不过，这种“似然”并不属于概率分布，因为它们的总和并不等于 1。

将这些因素结合起来，就得到了这个方程式。

$$posterior = \frac{likelihood \times prior}{normalization}$$

当我们谈论滤波器的输出时，通常把预测后的状态称为“先验状态”或“预测值”；而把更新后的状态则称为“后验状态”或“估计状态”。

学习并牢记这些术语非常重要，因为大多数文献中都会大量使用它们。

`scaled_update()` 会执行这个计算吗？它会。让我把它重新表述成这样：

```
def scaled_update(hall, belief, z, z_prob):
    scale = z_prob / (1. - z_prob)
```

```
    likelihood = np.ones(len(hall))
    likelihood[hall==z] *= scale
    return normalize(likelihood * belief)
```

这个函数并非具有完全的通用性。它包含了与走廊结构相关的信息，以及如何将各种测量数据与走廊结构相对应的方法。我们始终致力于编写具有通用性的函数。在这里，我们将把似然的计算部分从函数中分离出来，要求调用者自行进行可能性的计算。

以下是该算法的完整实现代码：

```
def update(likelihood, prior):
    return normalize(likelihood * prior)
```

似然的计算方式因具体问题而异。例如，传感器可能不会只返回 1 或 0，而是返回 0 到 1 之间的数值，以此表示物体位于门前的概率。传感器也可能利用计算机视觉技术来识别物体的形状，然后将其与门进行匹配。或者，传感器可以使用声纳来测量距离。无论哪种情况，似然的计算方式都不尽相同。在本书中，我们将看到许多这样的例子，并学习如何进行这些计算。

FilterPy 实现了 `update` 功能。以下是该功能的通用实现方式：

```
from filterpy.discrete_bayes import update

def lh_hallway(hall, z, z_prob):
    """ compute likelihood that a measurement matches
    positions in the hallway."""

    try:
        scale = z_prob / (1. - z_prob)
    except ZeroDivisionError:
        scale = 1e8

    likelihood = np.ones(len(hall))
    likelihood[hall==z] *= scale
    return likelihood

belief = np.array([0.1] * 10)
likelihood = lh_hallway(hallway, z=1, z_prob=.75)
update(likelihood, belief)
```

```
array([0.188, 0.188, 0.062, 0.062, 0.062, 0.062, 0.062, 0.062, 0.188,
       0.062])
```

融入动作/将动作融入其中

回想一下，当我们把各种测量数据和运动信息都考虑进来时，我们能够多快找到精确的解决方案。不过，那是在一个拥有完美传感器的理想化世界中才有可能实现的。那么，如果使用有噪声的传感器，我们还能找到精确的解决方案吗？

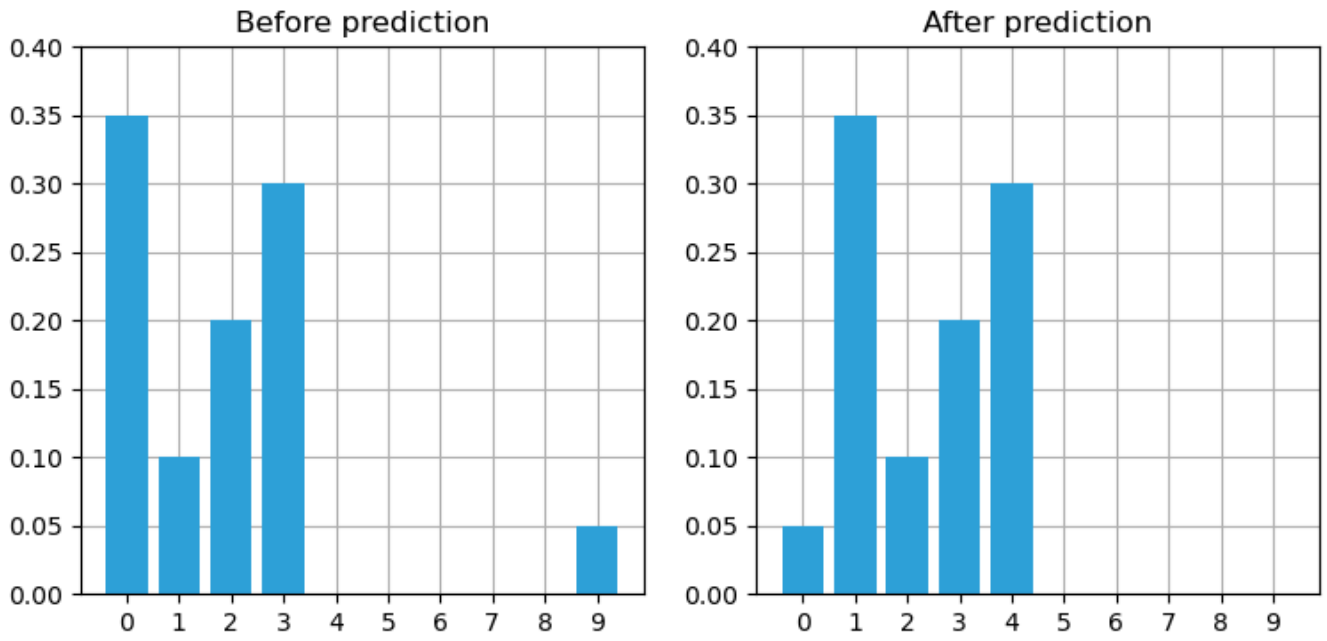
不幸的是，答案是否定的。即便传感器的读数与极其复杂的走廊布局完全吻合，我们也无法 100%确定狗的确切位置——毕竟，所有传感器的读数都可能出错。当然，在正常情况下，大多数传感器的读数都是正确的，我

们可以有接近 100%的把握来判断狗的位置，但永远无法做到 100%确定。这听起来可能很复杂，不过，我们还是来编写相关的数学公式吧。

首先，我们先来处理这个简单的情况：假设运动传感器非常精确，它报告说狗向右移动了一个单位距离。那么，我们应该如何修改 `belief` 数组呢?

希望经过一番思考后，大家能明白：我们应该把所有的数值都向右移动一个位置。如果我们之前认为西蒙位于第 3 个位置的概率是 50%，那么在他向右移动一个位置后，我们应认为他位于第 4 个位置的概率也是 50%。由于走廊是环形的，因此我们需要使用取模运算来进行这种移动。

```
def perfect_predict(belief, move):
    """ move the position by `move` spaces, where positive is
    to the right, and negative is to the left
    """
    n = len(belief)
    result = np.zeros(n)
    for i in range(n):
        result[i] = belief[(i-move) % n]
    return result

belief = np.array([.35, .1, .2, .3, 0, 0, 0, 0, 0, .05])
plt.subplot(121)
book_plots.bar_plot(belief, title='Before prediction', ylim=(0, .4))

belief = perfect_predict(belief, 1)
plt.subplot(122)
book_plots.bar_plot(belief, title='After prediction', ylim=(0, .4))
```



我们可以看到，我们已经正确地将所有数值向右移动了一位。数值的移动是循环进行的，即从数组的末尾移动到开头。

下一个动画会展示这一过程，这样你就能亲眼看到其运作方式了。可以使用滑块来控制时间的推进或回溯。该模拟展示了西蒙在走廊里不断行走的情景。由于目前还没有加入新的测量数据，因此概率分布的形状不会发生

变化，只有位置会改变而已。

```
from ipywidgets import interact, IntSlider

belief = np.array([.35, .1, .2, .3, 0, 0, 0, 0, 0, .05])
perfect_beliefs = []

for _ in range(20):
    # Simon takes one step to the right
    belief = perfect_predict(belief, 1)
    perfect_beliefs.append(belief)

def simulate(time_step):
    book_plots.bar_plot(perfect_beliefs[time_step], ylim=(0, .4))
    plt.show()

interact(simulate, time_step=IntSlider(value=0, max=len(perfect_beliefs)-1));
```

```
interactive(children=(IntSlider(value=0, description='time_step', max=19), Output()), _dom_classes=
('widget-in…
```

术语/专业词汇

让我们暂停一下，回顾一下相关术语。我在上一章中已经介绍了这些术语，但现在再花点时间来巩固一下大家的理解吧。

我们所试图建模或处理的，其实就是这个“系统”。在这里，这个“系统”指的就是我们的狗。而“状态”则指的是狗当前的形态或状况。在本章中，狗的位置就代表了它的状态。我们通常无法知道狗的真实状态，因此可以说，我们的滤波器实际上只是对系统状态的估算而已。在实践中，这种估算结果通常也被视为系统的“状态”。所以，请务必根据上下文来理解相关概念。

一次包含测量数据的预测与更新过程，被称为系统的状态或演化过程，简称为时间演化[7]。另一个相关术语是“系统传播”。它指的是系统的状态如何随时间而发生变化。对于滤波器而言，时间通常是以离散的单位来表示的，比如 1 秒。在我们的狗只追踪系统中，系统的状态就是狗的位置；而状态演化则是指在经过一定时间后，狗的位置发生了怎样的变化。

我们用流程模型来描述系统的行为。在我们的模型中，狗在每个时间步都会移动一个或多个位置。不过，这种模型并不能准确反映狗的真实行为。模型中的误差被称为系统误差或流程误差。

这一预测就构成了我们的新“先验信息”。时间不断向前推进，而我们在做出预测时，并没有那些测量数据作为参考。

我们来看一个例子。目前，那只狗的位置是 17 米处。我们的时间间隔为 2 秒，而那只狗的移动速度为 15 米/秒。那么，我们能预测出两秒后它会在哪里吗?

显然，

$$\begin{aligned}\bar{x} &= 17 + (15*2) \\ &= 47\end{aligned}$$

我用横线来表示变量，以此表明这些变量属于先验值/预测值。该过程模型的方程可以这样表示：

$$\bar{x}_{k+1} = f_x(\bullet) + x_k$$

x_k 代表当前的位置或状态。如果狗位于 17 米处，那么 $x_k = 17$ 就是该位置的信息。

$f_x(\bullet)$ 是 x 的状态传播函数。它描述了 x_k 在一个时间步长内会发生多大的变化。在我们的例子中，该函数执行的是 $15 \cdot 2$ 这样的计算，因此我们可以将其定义为

$$f_x(v_x, t) = v_k t$$

.

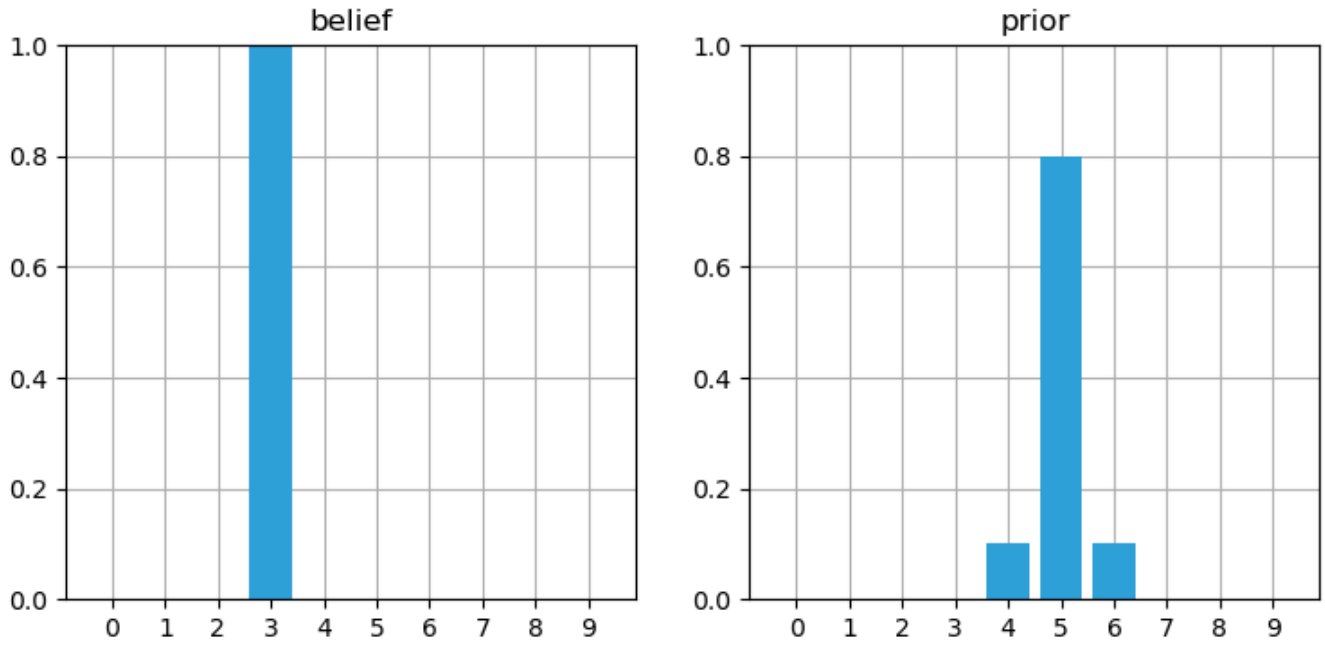
给预测结果增添了不确定性

`perfect_predict()` 假设所有的测量结果都是精确的，但实际上所有传感器都会产生一定的误差。如果传感器显示我们的狗移动了一个单位距离，但实际上它移动了两个单位距离，或者根本没移动呢？这听起来像是个无法解决的问题，但让我们试着建立模型来分析一下吧。

假设该传感器的移动测量结果有 80%的概率是正确的，10%的概率会向右偏移一个单位，10%的概率会向左偏移一个单位。也就是说，如果测量结果显示为 4（即向右移动 4 个单位），那么狗实际上有 80%的概率真的向右移动了 4 个单位，10%的概率向右移动了 3 个单位，10%的概率向右移动了 5 个单位。

现在，数组中的每个结果都需要包含三种不同情况下的概率。例如，以数字 2 所代表的移动情况来说：如果我们确定狗是从位置 3 开始移动的，那么它位于位置 5 的概率为 80%，而位于位置 4 或 6 的概率则各为 10%。让我们试着把这一点用代码体现出来吧：

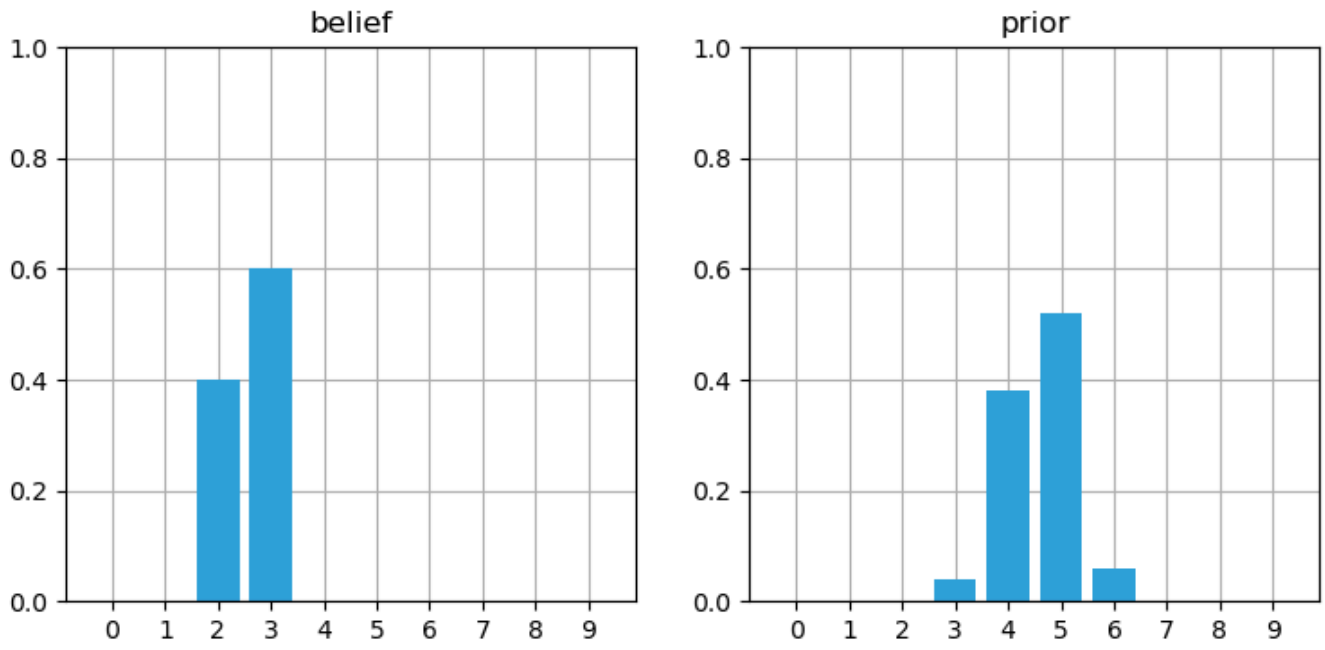
```
def predict_move(belief, move, p_under, p_correct, p_over):
    n = len(belief)
    prior = np.zeros(n)
    for i in range(n):
        prior[i] = (
            belief[(i-move) % n]   * p_correct +
            belief[(i-move-1) % n] * p_over +
            belief[(i-move+1) % n] * p_under)
    return prior

belief = [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.]
prior = predict_move(belief, 2, .1, .8, .1)
book_plots.plot_belief_vs_prior(belief, prior)
```



看起来它运作正常。那么，当我们的判断并非百分之百确定时，又会怎样呢?

```
belief = [0, 0, .4, .6, 0, 0, 0, 0, 0, 0]
prior = predict_move(belief, 2, .1, .8, .1)
book_plots.plot_belief_vs_prior(belief, prior)
prior
```

```
array([0.  , 0.  , 0.  , 0.04, 0.38, 0.52, 0.06, 0.  , 0.  , 0.  ])
```



这里的计算结果比较复杂，但你应该还是能够自行理解。0.04 这个数值是由于 0.4 的预测值可能少了 1 个单位所导致的。0.38 这个数值的由来是：有 80%的概率是我们移动了 2 个单位（即 0.4 + 0.8），还有 10%的概率是我们预测不足（即 0.6 + 0.1）。如果预测过高，则不会产生这种结果，因为如果预测过高，那么 0.4 和 0.6 的数

值都会超过这个数值。我强烈建议多做几道练习题，直到完全理解这一点，因为后续的所有计算都依赖于对这一步的理解。

如果你查看更新后的概率值，可能会感到失望。在上面的例子中，最初两个位置的概率值分别为 0.4 和 0.6；但经过更新后，这些概率值不仅降低了，而且分布也变得十分分散。

这并非巧合，也不是精心挑选的例子所导致的结果——这种规律在所有的预测中都成立。如果传感器存在噪声，那么在每次预测时都会丢失一些信息。假设我们进行无限次次的预测，结果会怎样呢？既然每一步都会丢失信息，最终我们就会得到零信息，此时，所有概率值都会在 `belief` 数组中均匀分布。我们试着进行 100 次迭代来验证这一点。该图表是动态的，可以通过滑块来改变迭代次数。

```
belief = np.array([1.0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
predict_beliefs = []

for i in range(100):
    belief = predict_move(belief, 1, .1, .8, .1)
    predict_beliefs.append(belief)

print('Final Belief:', belief)

# make interactive plot
def show_prior(step):
    book_plots.bar_plot(predict_beliefs[step-1])
    plt.title(f'Step {step}')
    plt.show()

interact(show_prior, step=IntSlider(value=1, max=len(predict_beliefs)));
```

```
Final Belief: [0.104 0.103 0.101 0.099 0.097 0.096 0.097 0.099 0.101 0.103]
```

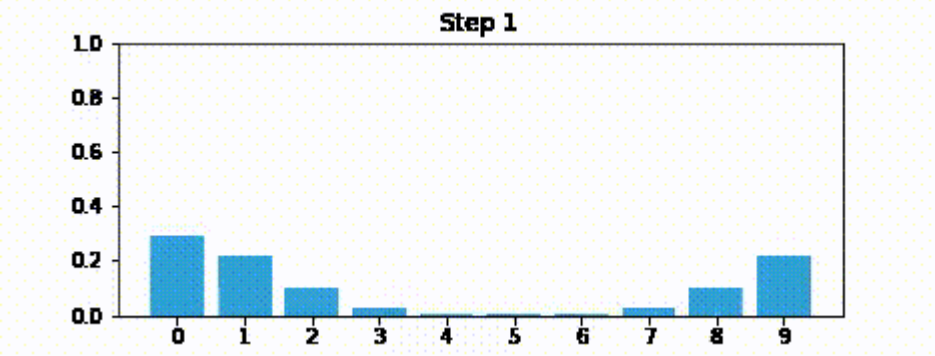
```
interactive(children=(IntSlider(value=1, description='step'), Output()), _dom_classes=('widget-
interact',))
```

```
print('Final Belief:', belief)
```

```
Final Belief: [0.104 0.103 0.101 0.099 0.097 0.096 0.097 0.099 0.101 0.103]
```

经过 100 次迭代后，我们几乎丢失了所有信息，尽管我们确信自己是从位置 0 开始迭代的。你可以试着改变迭代次数，看看不同迭代次数会对结果产生什么影响。例如，经过 100 次迭代后，还剩下少量信息；经过 50 次迭代后，还能保留较多信息；但到了 200 次迭代后，几乎所有信息都消失了。

另外，如果您是在线查看此内容的话，这里还有该结果的动画演示。



在本书的剩余部分中，我不会再单独生成这些动画了。关于如何在网上免费运行本书中的代码，或如何在电脑上安装 IPython，请参阅前言中的说明。这样，您就可以运行所有的代码片段并查看相应的动画效果了。请务必亲自尝试运行这些代码，而不仅仅是被动地阅读文字说明。

通过卷积进行泛化处理

我们假设，移动误差最多为一个位置单位。但实际上，误差也可能为两个、三个或更多位置单位。作为程序员，我们总是希望让代码具有通用性，从而能够适用于所有情况。

这个问题可以通过卷积来轻松解决。卷积就是用一个函数来修改另一个函数。在我们的例子中，我们是用传感器的误差函数来修改概率分布。`predict_move()` 的实现方式其实就是卷积，不过我们并没有这样称呼它。从形式上来说，卷积的定义是……

$$(f*g)(t) = \int_0^t f(\tau)g(t-\tau)\mathrm{d}\tau$$

其中，$f*g$ 表示将 f 与 g 进行卷积运算。这并不意味着相乘。

积分是用于连续函数的，但我们这里处理的是离散函数。因此，我们用求和来代替积分，同时用方括号来代替圆括号。

$$(f*g)[t] = \sum_{\tau=0}^{t} f[\tau]g[t-\tau]$$

比较结果显示，`predict_move()` 正在计算这个公式——它实际上是在计算一系列乘法运算的结果之和。

可汗学院[4]对卷积的概念有很好的讲解，而维基百科上也有一些关于卷积的精彩动画演示[5]。不过，其基本原理其实很容易理解。所谓卷积，就是将一个被称为“核”的数组在另一个数组上滑动，同时将当前位置周围的数值与第二个数组中的相应数值相乘。在上面的例子中，我们将 0.8 设为移动到正确位置的概率，0.1 设为移动位置偏小的概率，0.1 则设为移动位置偏大的概率。我们用数组 `[0.1, 0.8, 0.1]` 来表示这个“核”。我们所需要做的，就是编写一个循环，遍历数组中的每个元素，将其与“核”中的数值相乘，然后把所有结果相加。为了强调“信念”其实是一种概率分布，我将其命名为 `pdf` 。

```
def predict_move_convolution(pdf, offset, kernel):
    N = len(pdf)
    kN = len(kernel)
```

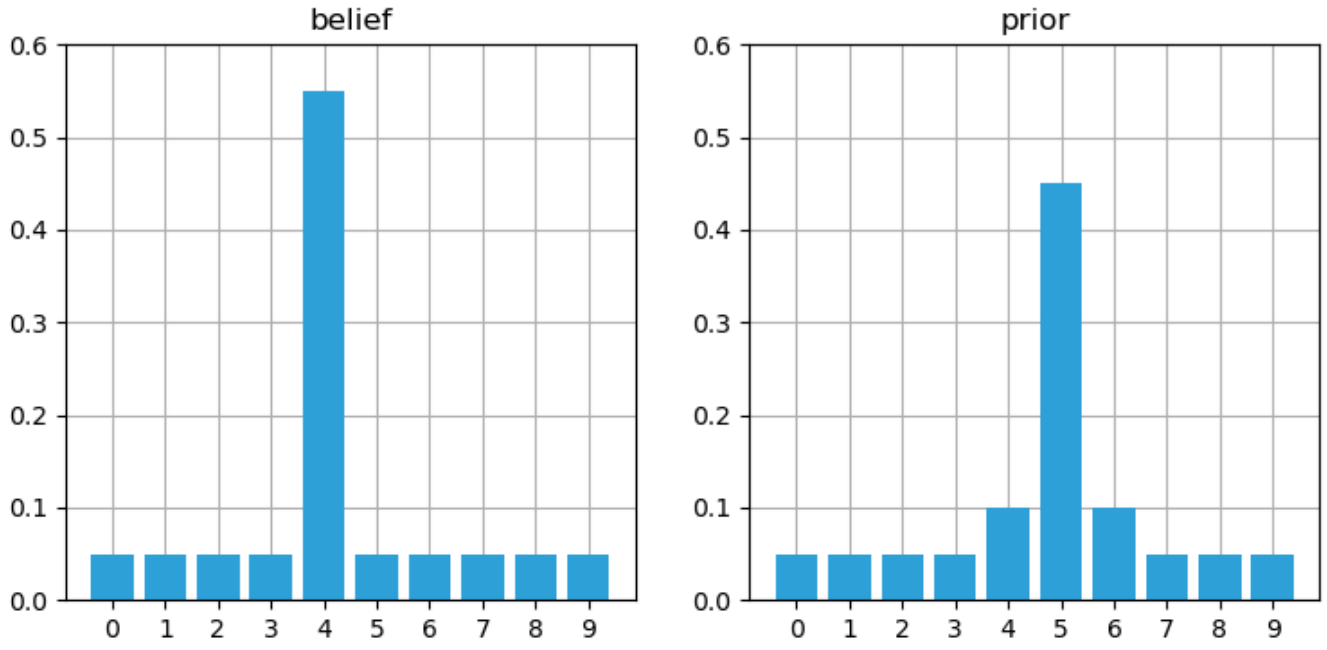
```
    width = int((kN - 1) / 2)

    prior = np.zeros(N)
    for i in range(N):
        for k in range (kN):
            index = (i + (width-k) - offset) % N
            prior[i] += pdf[index] * kernel[k]
    return prior
```

该代码演示了该算法的运作方式，但执行速度相当慢。SciPy 在 `ndimage.filters` 模块中提供了用于卷积操作的函数 `convolve()` 。在进行卷积操作之前，需要将 pdf 函数向右平移 `offset` 个单位； `np.roll()` 函数正好实现了这一功能。整个“移动并预测”算法只需一行代码即可实现：

```
convolve(np.roll(pdf, offset), kernel, mode='wrap')
```

FilterPy 通过 `discrete_bayes` ‘`predict()` 函数来实现这一功能。

```
from filterpy.discrete_bayes import predict

belief = [.05, .05, .05, .05, .55, .05, .05, .05, .05, .05]
prior = predict(belief, offset=1, kernel=[.1, .8, .1])
book_plots.plot_belief_vs_prior(belief, prior, ylim=(0,0.6))
```

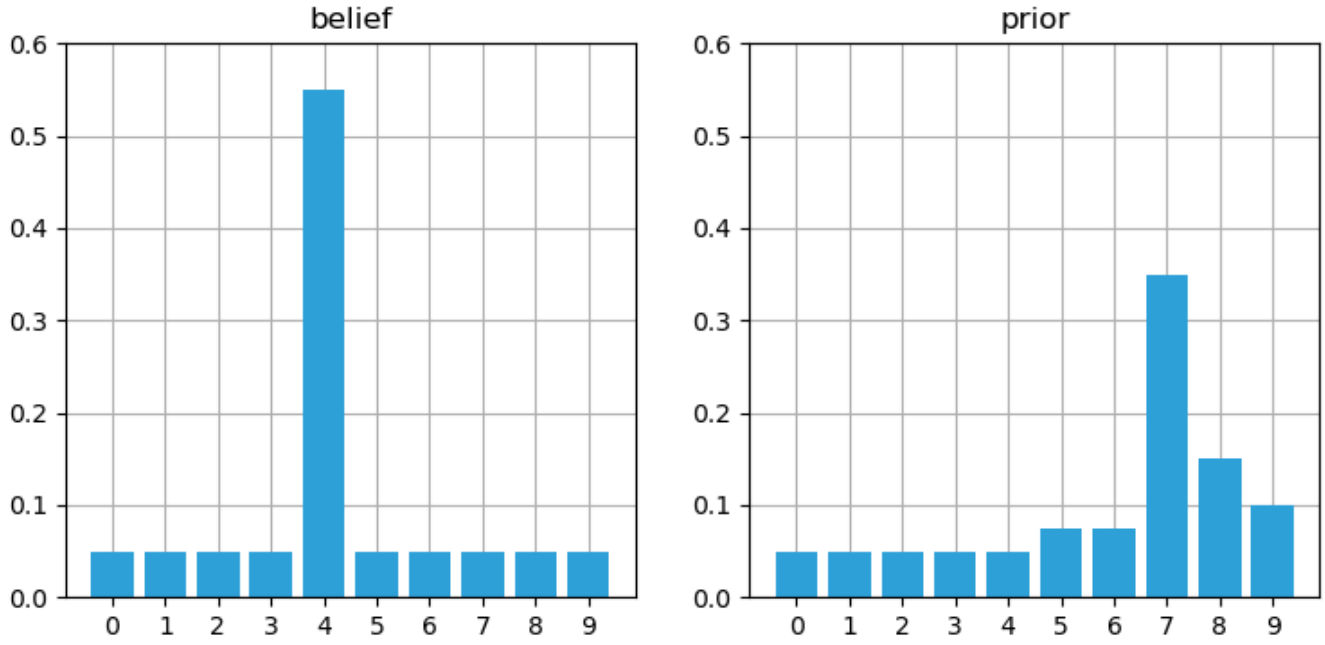


除了中间的元素外，其他所有元素的值都保持不变。第 4 位和第 6 位的数值应该分别是……

第 5 个位置应该填入……

让我们确保：当移动距离大于一时，或者使用非对称的核函数时，该算法能够正确地调整各元素的排列位置。

```
prior = predict(belief, offset=3, kernel=[.05, .05, .6, .2, .1])
book_plots.plot_belief_vs_prior(belief, prior, ylim=(0,0.6))
```



该位置的调整是正确的，调整了 3 个单位。同时，我们更重视“超调”情况的可能性，而非“欠调”情况。因此，这个结果看起来是正确的。

请确保你明白我们在做什么。我们正在预测狗的移动方向，然后通过将各种概率值进行叠加来得到最终的预测结果。

如果我们不使用概率的话，那就用我之前给出的那个公式吧：

$$\bar{x}_{k+1} = x_k + f_x(\bullet)$$

我们之前预测狗的位置时，是将狗移动的距离加上它当前的位置来确定的。比如，如果狗最初位于 10 的位置，然后移动了 5 米，那么它现在就位于 15 米的位置。其实很简单吧。不过，我们是用概率模型来描述这一过程的，所以我们的计算公式是：

$$\bar{x}_{k+1} = x_k * f_x(\bullet)$$

我们将当前的概率位置估计值，与我们对狗的移动距离的概率估计值相乘。原理是一样的，只不过计算方式略有不同。 x 用粗体标出，表示它是一个数字数组。

将测量数据与运动状态更新信息相结合

在预测过程中，信息可能会丢失，这似乎意味着我们的系统会很快变得一无所知。但实际上，每次预测之后都会进行更新，我们将新的测量结果纳入到估算中。这样的更新有助于提升我们的认知水平。更新后的结果会被用于下一次预测。虽然预测会降低我们的确定性，但这一结果又会作为输入用于下一次更新，从而再次提升确定性。

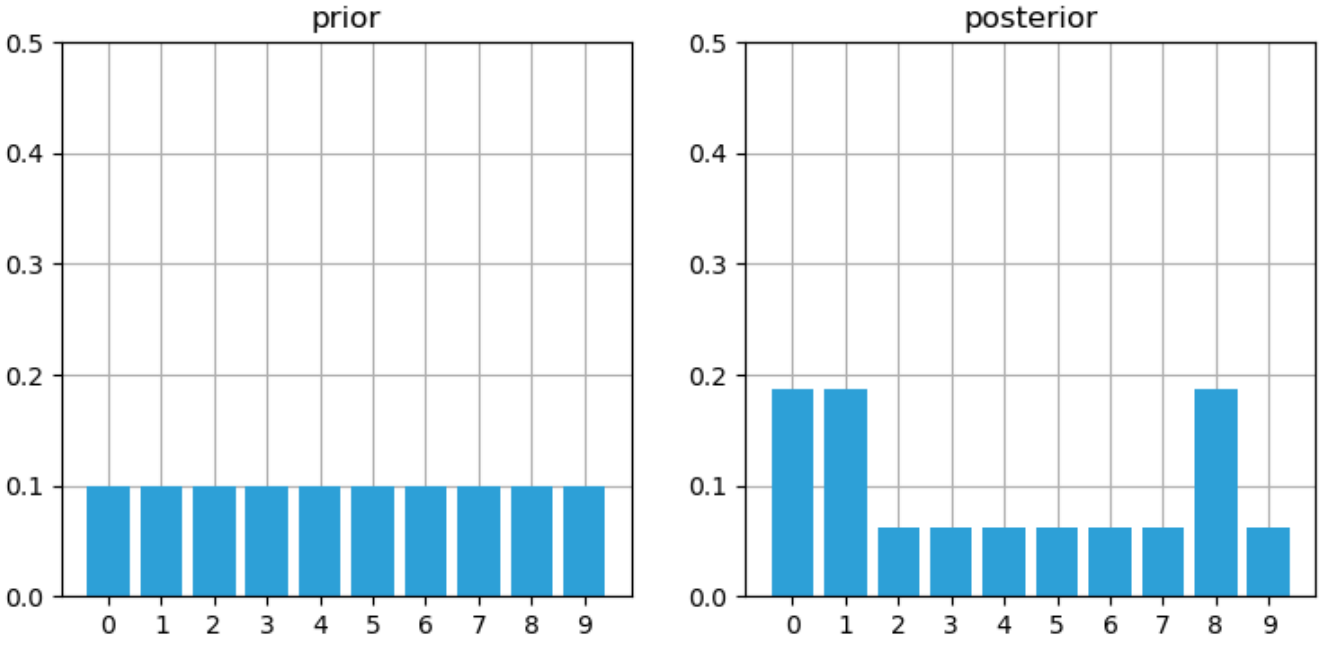
让我们用直观的方式来思考这个问题。想象一个简单的场景：你正在追踪一只静止不动的狗。在每次预测时，你都认为它没有移动。这样一来，你的预测模型很快就能准确判断出狗的位置。但随后，厨房里的微波炉打开了，狗立刻冲了出去。你并不知道这一情况，所以在下一次预测时，你仍然认为狗停留在原来的位置。但实际

上，测量结果却表明狗正在朝厨房方向移动。随着时间的推移，你认为狗静止不动的概率会越来越小，而认为狗正以极快的速度朝厨房移动的概率则越来越大。

直觉就是这么告诉我们的。那么，数学又告诉我们什么呢?

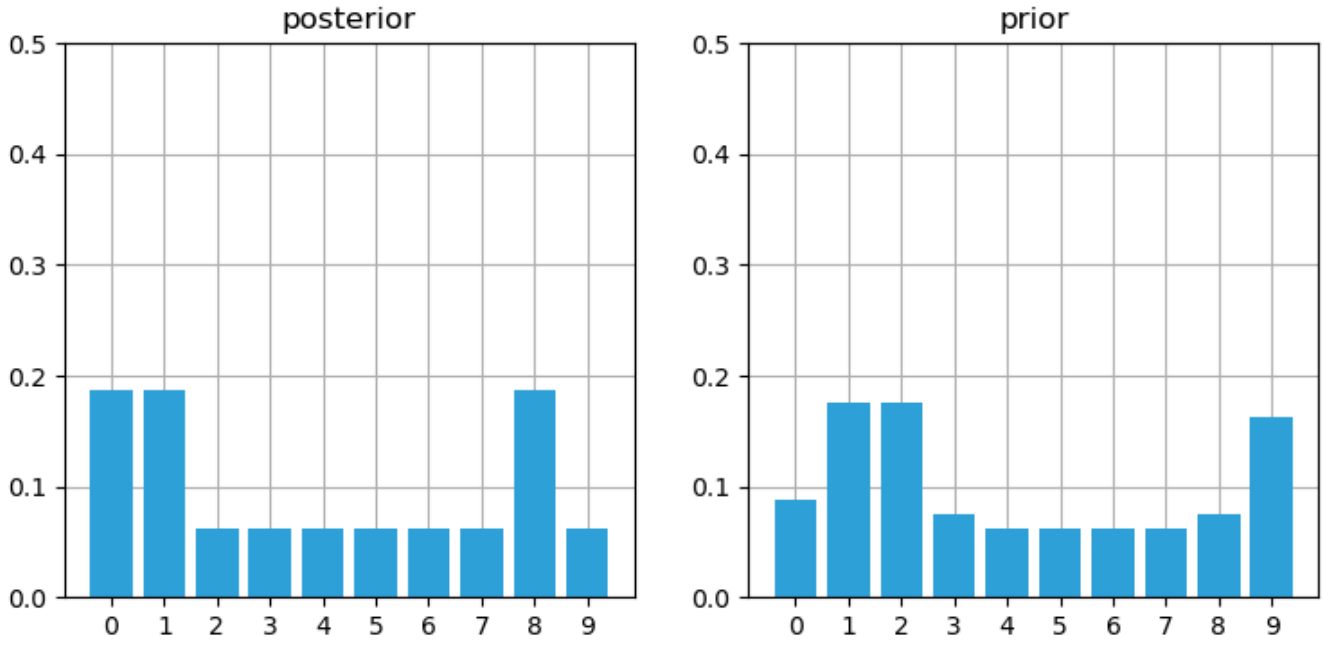
我们已经完成了更新和预测的相关程序编写。现在只需将其中一个步骤的输出结果作为输入传递给另一个步骤即可。这样，我们就成功实现了狗的追踪功能！！！让我们看看它的实际表现如何吧。我们会假设狗从位置 0 开始移动，每个时间单位都向右移动一个单位。与现实世界的情况类似，一开始我们不会知道狗的具体位置，因此会给予所有位置相同的概率。

```
from filterpy.discrete_bayes import update

hallway = np.array([1, 1, 0, 0, 0, 0, 0, 0, 1, 0])
prior = np.array([.1] * 10)
likelihood = lh_hallway(hallway, z=1, z_prob=.75)
posterior = update(likelihood, prior)
book_plots.plot_prior_vs_posterior(prior, posterior, ylim=(0,.5))
```



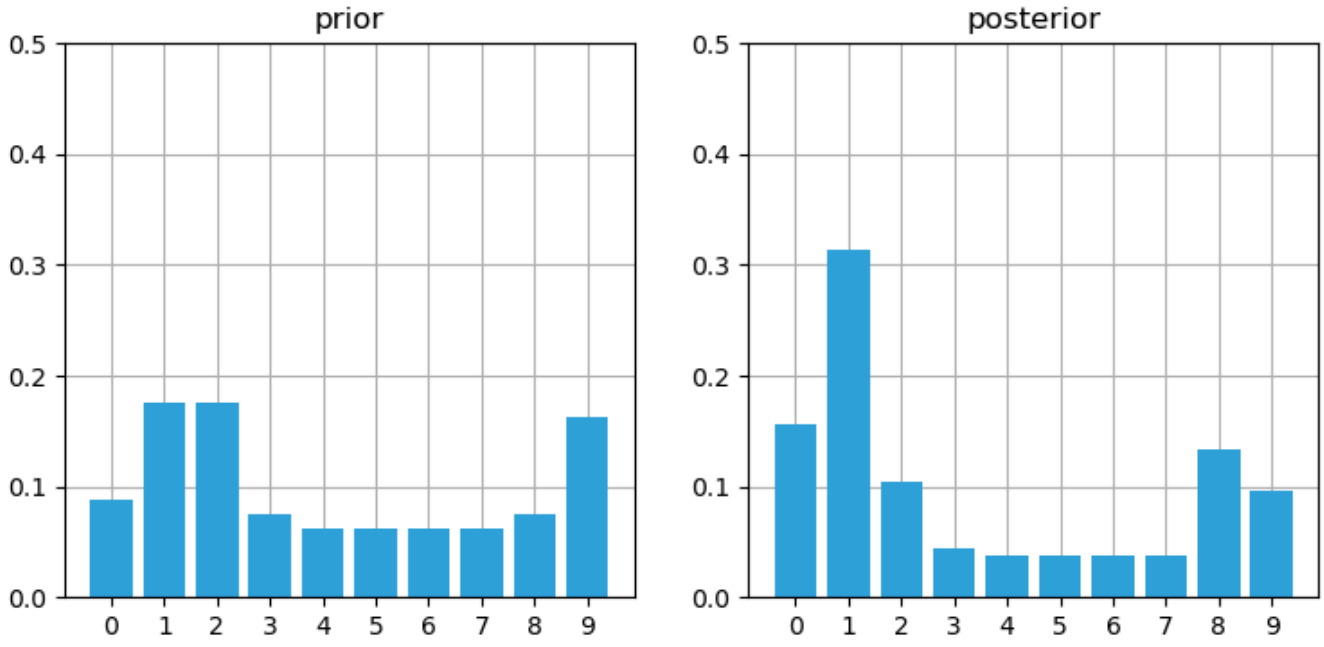
在第一次更新之后，我们给每个门的位置分配了较高的概率，而给每个墙的位置分配了较低的概率。

```
kernel = (.1, .8, .1)
prior = predict(posterior, 1, kernel)
book_plots.plot_prior_vs_posterior(prior, posterior, True, ylim=(0,.5))
```



在预测步骤中，这些概率值向右移动了些，分布也变得稍微模糊了一些。现在，让我们看看在下一个处理步骤中会发生什么。

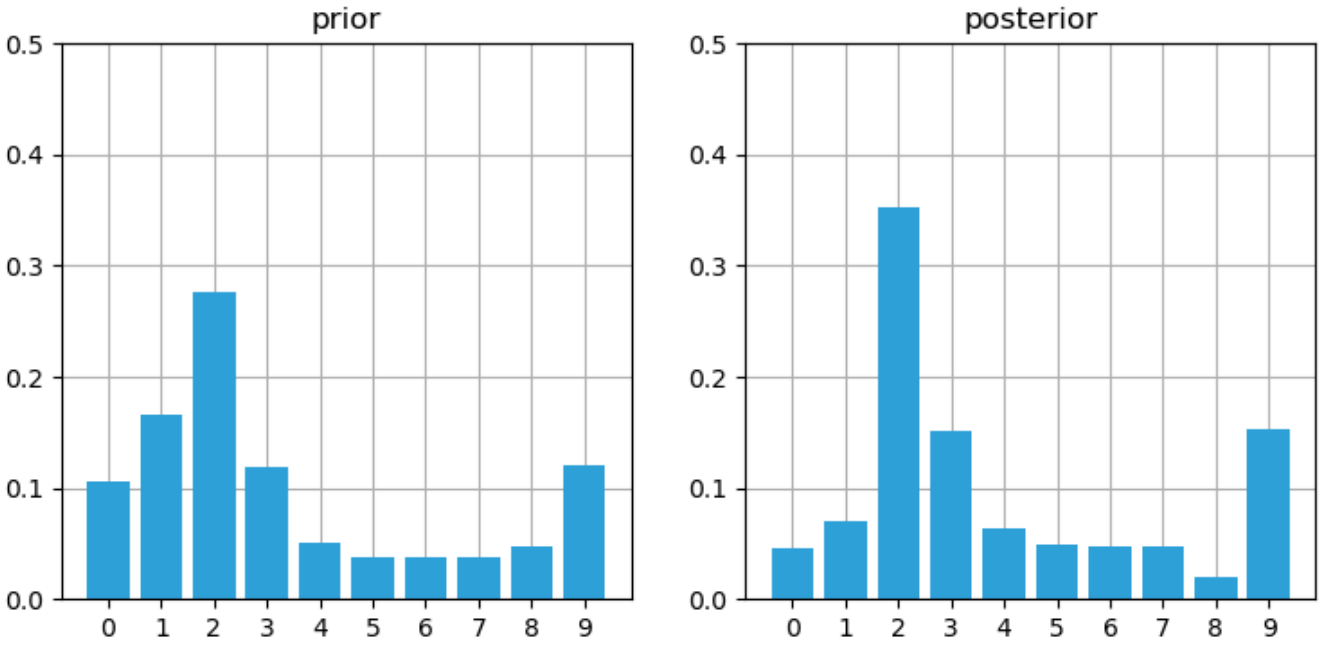
```
likelihood = lh_hallway(hallway, z=1, z_prob=.75)
posterior = update(likelihood, prior)
book_plots.plot_prior_vs_posterior(prior, posterior, ylim=(0,.5))
```



请注意，位置 1 处有个较高的条形图。这种情况对应于从位置 0 开始、检测到一扇门后向右移动 1 个单位、然后再检测到另一扇门的正确流程。没有其他位置能产生与这一系列观察结果相符的结果。现在，我们将进行一次更新，然后再检测墙壁的位置。

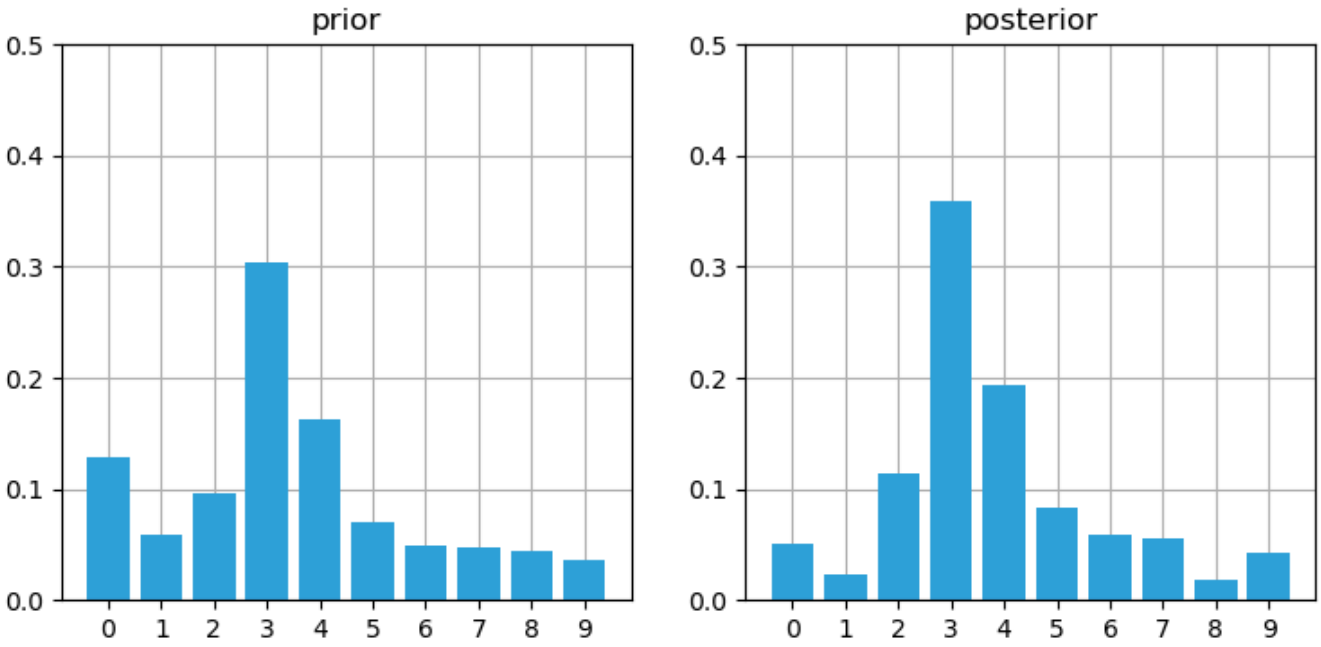
```
prior = predict(posterior, 1, kernel)
likelihood = lh_hallway(hallway, z=0, z_prob=.75)
```

```
posterior = update(likelihood, prior)
book_plots.plot_prior_vs_posterior(prior, posterior, ylim=(0,.5))
```



真是令人兴奋！在位置 2 上，有一个非常突出的条形图，其数值约为 35%。这一数值是图中其他所有条形图数值的两倍多。与上一次的统计结果相比，这一数值还高出约 4%；上一次统计中，数值最高的条形图数值约为 31%。让我们再观察一个周期吧。

```
prior = predict(posterior, 1, kernel)
likelihood = lh_hallway(hallway, z=0, z_prob=.75)
posterior = update(likelihood, prior)
book_plots.plot_prior_vs_posterior(prior, posterior, ylim=(0,.5))
```



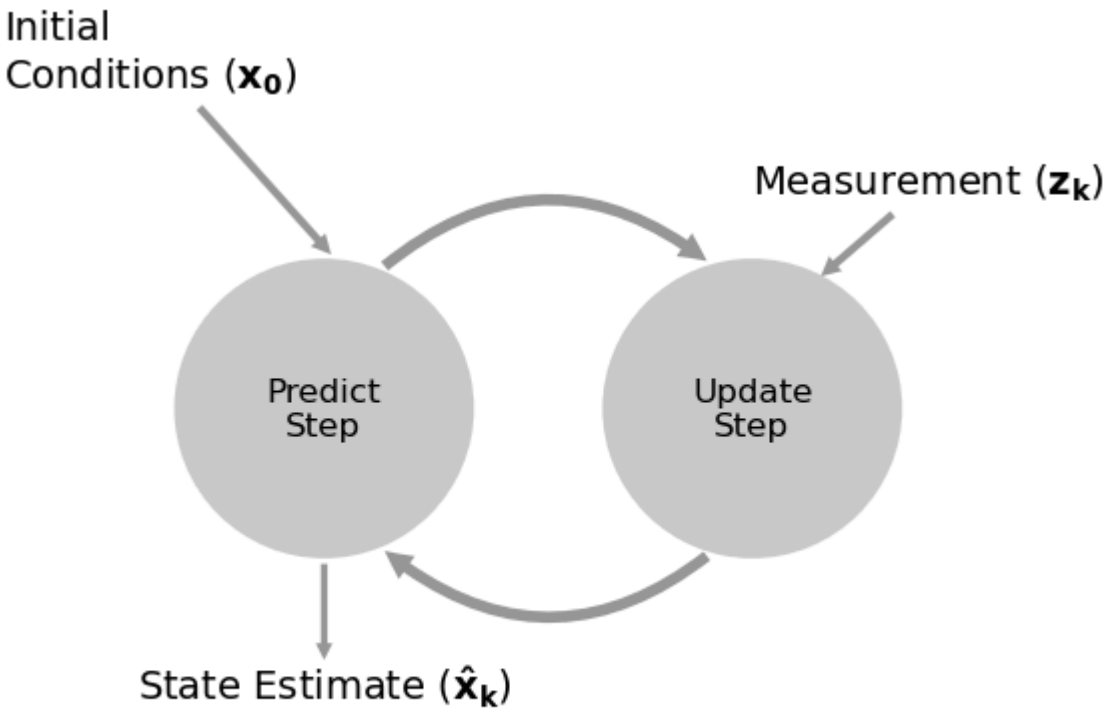
我忽略了一个重要问题。之前，我以为在预测步骤中使用了运动传感器来获取数据；后来，在讨论狗和微波炉的问题时，我又以为你并不知道狗突然开始奔跑这一情况。我提到，随着时间的推移，你会越来越认为狗正在奔跑，但我没有给出相关的代码实现。简而言之，如果我们无法直接测量某个过程，那要如何检测或估算该过程中的变化呢?

目前，我打算先忽略这个问题。在后面的章节中，我们会学习与这种估算方法相关的数学原理。不过，目前来说，学会这个算法就已经足够了。虽然解决这个问题非常重要，但我们目前还缺乏必要的数学工具来处理它。因此，在本章的剩余部分，我们将假设自己拥有能够感知运动的传感器，从而暂时忽略这个问题。

离散贝叶斯算法

该图表展示了该算法的运作方式：

```
book_plots.predict_update_chart()
```



这种滤波器属于 g-h 滤波器的一种。在这里，我们利用误差的百分比来间接计算 g 和 h 这两个参数。虽然可以将离散贝叶斯算法表示为 g-h 滤波器的形式，但那样做会掩盖该滤波器的真正逻辑。

滤波器方程如下：

$$\begin{aligned}\bar{x} &= x * f_x(\bullet) && \text{Predict Step} \\ x &= \|L \cdot \bar{x}\| && \text{Update Step}\end{aligned}$$

L 是表示似然函数的常用方式，所以我使用这种表示法。 $||| 则表示取范数。为了确保 x 是一个总和为一的概率分布，我们必须对似然函数与先验概率的乘积进行归一化处理。

我们可以用伪代码来表示这一点。

初始化

```
1. Initialize our belief in the state
```

预测

```
1. Based on the system behavior, predict state for the next time step
2. Adjust belief to account for the uncertainty in prediction
```

更新

```
1. Get a measurement and associated belief about its accuracy
2. Compute how likely it is the measurement matches each state
3. Update state belief with this likelihood
```

在讲解卡尔曼滤波器时，我们会使用完全相同的算法；只不过在具体的计算细节上会有所不同而已。

这种形式的算法有时被称为“预测修正器”。我们先进行预测，然后再对预测结果进行修正。

让我们来让这个过程可视化吧。首先，我们需要编写一些函数来执行滤波操作，并在各个步骤中绘制出相应的结果。我用黑色标出了各个门的位置。先前的位置用橙色表示，后续的位置则用蓝色表示。我还画了一条粗竖线，用来标明西蒙的实际位置。这条线并非滤波操作的输出结果——我们之所以知道西蒙的位置，是因为我们模拟了他的移动轨迹。

```
def discrete_bayes_sim(prior, kernel, measurements, z_prob, hallway):
    posterior = np.array([.1]*10)
    priors, posteriors = [], []
    for i, z in enumerate(measurements):
        prior = predict(posterior, 1, kernel)
        priors.append(prior)

        likelihood = lh_hallway(hallway, z, z_prob)
        posterior = update(likelihood, prior)
        posteriors.append(posterior)
    return priors, posteriors


def plot_posterior(hallway, posteriors, i):
    plt.title('Posterior')
    book_plots.bar_plot(hallway, c='k')
    book_plots.bar_plot(posteriors[i], ylim=(0, 1.0))
    plt.axvline(i % len(hallway), lw=5)
    plt.show()

def plot_prior(hallway, priors, i):
    plt.title('Prior')
    book_plots.bar_plot(hallway, c='k')
    book_plots.bar_plot(priors[i], ylim=(0, 1.0), c='#ff8015')
    plt.axvline(i % len(hallway), lw=5)
    plt.show()
```

```
def animate_discrete_bayes(hallway, priors, posteriors):
    def animate(step):
        step -= 1
        i = step // 2
        if step % 2 == 0:
            plot_prior(hallway, priors, i)
        else:
            plot_posterior(hallway, posteriors, i)

    return animate
```

让我们运行这个滤波器，并让它以动画形式展示出来吧。

```
# change these numbers to alter the simulation
kernel = (.1, .8, .1)
z_prob = 1.0
hallway = np.array([1, 1, 0, 0, 0, 0, 0, 0, 1, 0])

# measurements with no noise
zs = [hallway[i % len(hallway)] for i in range(50)]

priors, posteriors = discrete_bayes_sim(prior, kernel, zs, z_prob, hallway)
interact(animate_discrete_bayes(hallway, priors, posteriors), step=IntSlider(value=1,
max=len(zs)*2));
```

```
interactive(children=(IntSlider(value=1, description='step'), Output()), _dom_classes=('widget-
interact',))
```

现在我们可以看到结果了。可以看到，先验概率会改变数值的位置并降低其确定性；而后验概率则保持不变，同时由于吸收了测量结果带来的信息，其确定性会提高。我在这里用线段 `z_prob = 1.0` 来表示“完美的测量结果”；我们将在下一节探讨不完美测量所带来的影响。最后……

另一点需要注意的是：当我们处于门的前方时，我们的定位精度会很高；而当我们在走廊的中间时，定位精度则会下降。这一点其实很容易理解。由于门的数量有限，当传感器检测到我们位于某扇门的前方时，我们就能更准确地确定自己的位置。而如果周围没有门的话，我们的定位精度就会下降。

不良传感器数据的影响

你可能会对上述结果产生怀疑，因为我始终将正确的传感器数据传递给了相关函数。不过，我们声称这段代码其实起到了滤波的作用——它应该能够滤除那些有问题的传感器测量数据。那么，它真的做到了这一点吗?

为了便于编程和可视化处理，我会将走廊的布局调整为以交替出现的门和走廊为主。同时，我会使用 6 组正确的测量数据来运行该算法。

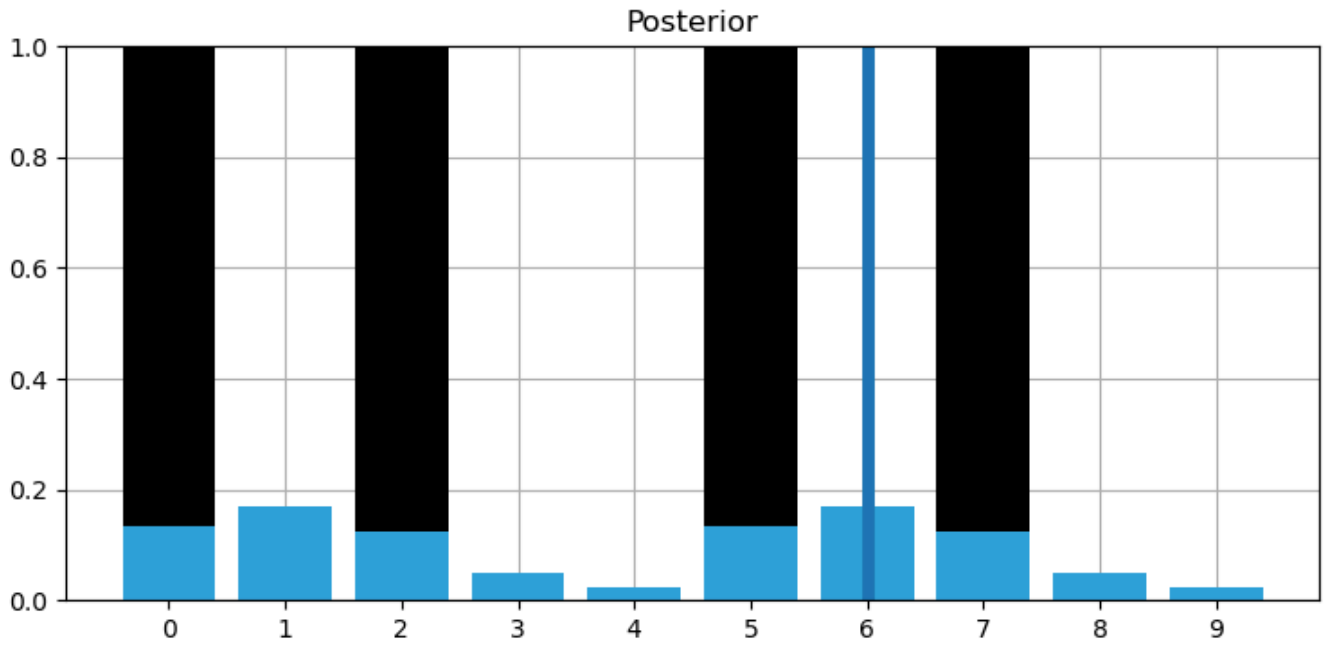
```
hallway = np.array([1, 0, 1, 0, 0]*2)
kernel = (.1, .8, .1)
prior = np.array([.1] * 10)
zs = [1, 0, 1, 0, 0, 1]
z_prob = 0.75
priors, posteriors = discrete_bayes_sim(prior, kernel, zs, z_prob, hallway)
```

```
interact(animate_discrete_bayes(hallway, priors, posteriors), step=IntSlider(value=12,
max=len(zs)*2));
```

```
interactive(children=(IntSlider(value=12, description='step', max=12), Output()), _dom_classes=
('widget-intera…
```

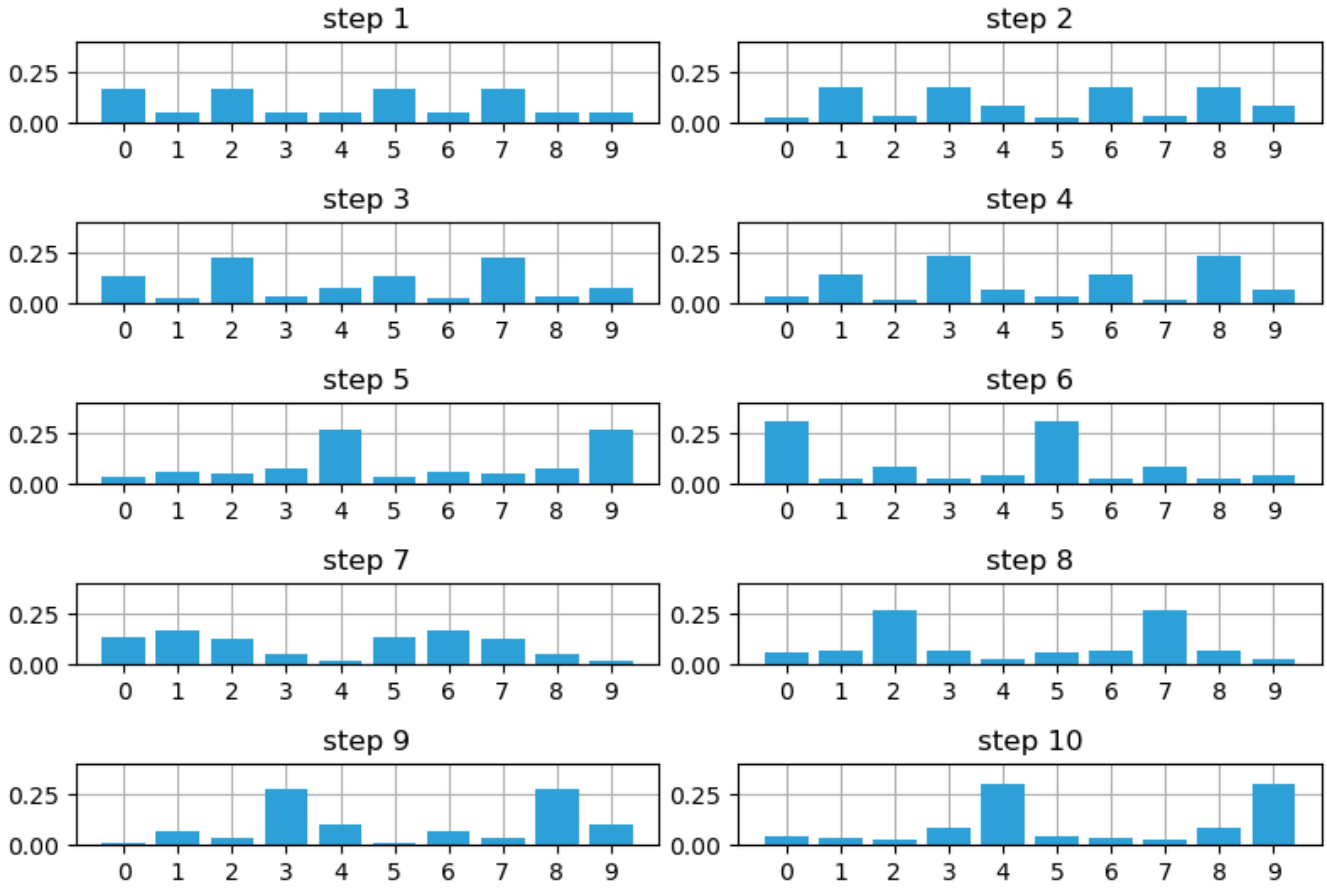
我们已经确定了那些可能从位置 0 或 5 开始的情况。因为我们观察到了这样的门和墙的排列顺序：1,0,1,0,0。现在，我故意输入了一个错误的测量值。按照正常情况，下一次的测量值应该是 0，但实际上却得到了 1。

```
measurements = [1, 0, 1, 0, 0, 1, 1]
priors, posteriors = discrete_bayes_sim(prior, kernel, measurements, z_prob, hallway);
plot_posterior(hallway, posteriors, 6)
```



那一次错误的测量严重影响了我们的研究成果。现在，让我们继续进行一系列正确的测量吧。

```
with figsize(y=5.5):
    measurements = [1, 0, 1, 0, 0, 1, 1, 1, 0, 0]
    for i, m in enumerate(measurements):
        likelihood = lh_hallway(hallway, z=m, z_prob=.75)
        posterior = update(likelihood, prior)
        prior = predict(posterior, 1, kernel)
        plt.subplot(5, 2, i+1)
        book_plots.bar_plot(posterior, ylim=(0, .4), title=f'step {i+1}')
    plt.tight_layout()
```



我们迅速滤除了那些有问题的传感器数据，从而确定了狗狗最可能所在的位置。

缺点与局限性

请不要被我所选的这些简单示例所误导。这其实是一个功能强大且完善的滤波工具，您完全可以将其应用到实际的解决方案中。如果您需要的是一种多模态、离散型的滤波功能，那么这个工具完全能满足您的需求。

不过，这种滤镜并不常被使用，因为它存在不少局限性。克服这些局限性，正是本书后续各章节要探讨的内容。

第一个问题是数据量的处理问题。在解决狗狗定位问题时，我们只使用了一个变量 pos 来表示狗狗的位置。但实际上，大多数情况下，我们需要跟踪更大范围内的多个信息。至少，我们需要知道狗狗的 (x, y) 坐标，可能还需要知道它的速度 $(\dot{x}, \dot{y})$ 。我们还没有考虑多维情况，但实际上，我们可以用多维网格来存储每个位置上的概率值，而不是使用数组。每进行一次 `update()` 或 `predict()` 操作时，都需要更新网格中的所有数值。因此，对于一个包含四个变量的简单问题来说，每个时间步长所需的计算时间就是 $O(n^4)$ 。而实际上，复杂的系统可能需要跟踪 10 个或更多的变量，这将导致极高的计算成本。

第二个问题是，滤波器是离散的，而我们所生活的世界却是连续的。直方图要求将滤波器的输出表现为一系列离散的点。例如，要精确到 1 厘米地描述 100 米长的走廊，就需要用 10,000 个点来表示这条走廊。因此，每次更新或预测时，都需要计算 10,000 个不同的概率值。随着维度的增加，这种情况会呈指数级恶化。例如，对于 100×100 米的庭院来说，若要达到 1 厘米的精度，就需要 1 亿个单元格来表示该区域。

第三个问题是，该滤波器具有多模态特性。在上一个例子中，我们得出的结论是：狗很可能位于第 4 或第 9 个位置。不过，这并不总是个问题。稍后我们会研究的粒子滤波器就具有多模态特性，正因如此，它们才被广泛使用。但是，试想一下：如果汽车中的 GPS 系统告诉你，有 40%的概率你当前位于 D 街，而 30%的概率你位于 Willow 大道上，那会怎么样呢?

第四个问题是，需要测量状态的变化情况。我们必须使用运动传感器来检测狗的移动程度。虽然有办法解决这个问题，但那样会使得本章的阐述变得复杂。鉴于上述问题，我就不再详细讨论了。

也就是说，如果我有那种这种技术能够解决的小问题，我肯定会选择使用它。这种技术的实现、调试和理解都相当简单，这些都是它的优点。

跟踪与控制

我们一直在被动地跟踪那些能够自主移动的物体。不过，再考虑一个类似的问题吧。我正在对仓库进行自动化改造，希望利用机器人来收集客户订单中的所有物品。也许最简单的方法就是让机器人沿着轨道移动。我希望能够为机器人指定目的地，让它自动前往那里。但是，轨道和机器人的电机都不完美。车轮打滑以及电机性能不佳意味着，机器人很难准确到达指定的位置。此外，还有多台机器人需要同时操作，我们必须知道每台机器人的位置，以避免它们相互碰撞。

因此，我们可以添加传感器来辅助定位。也许可以在轨道上每隔几英尺就安装一个磁铁，然后使用霍尔传感器来统计经过了多少个磁铁。如果统计到 10 个磁铁，那么机器人应该位于第 10 个磁铁的位置。当然，也有可能漏掉某个磁铁或重复计数某个磁铁，因此我们必须考虑到一定程度的误差。我们可以利用上一节中的代码来追踪机器人的位置，因为磁铁的计数方式与检测门框的位置很相似。

但我们还没有完成。我们明白了永远不要浪费任何信息。只要掌握了某种信息，就应该利用它来改进自己的预测。我们还遗漏了哪些信息呢？我们知道，在每一时刻，我们都在向机器人的各个部件发送控制指令。比如，假设我们每秒向机器人发送一次移动指令：向左移动 1 个单位、向右移动 1 个单位，或者保持不动。当我发出“向左移动 1 个单位”的指令后，我预计 1 秒钟后，机器人会位于当前位置的左侧 1 个单位处。当然，这是一种简化的处理方式，因为我没有考虑加速度的因素。不过，我并不是想在这里讲解控制理论。车轮和电机本身并不完美，所以机器人实际移动的距离可能会是 0.9 个单位，也有可能是 1.2 个单位。

现在，整个解决方案已经很清晰了。我们假设狗会继续沿原来的方向移动。不过，对于我的狗来说，这种假设并不成立！机器人的行为则容易预测得多。与其基于某种假设来做出不确定的预测，不如直接将我们发给机器人的指令输入进去！换句话说，当我们调用 `predict()` 时，我们会把给机器人的指令以及描述该指令执行可能性的参数一起传递过去。

模拟列车的运行行为

我们需要模拟一辆有缺陷的火车。当我们命令它移动时，它有时会犯小错误；同时，它的传感器也可能会返回错误的数值。

```
class Train(object):

    def __init__(self, track_len, kernel=[1.], sensor_accuracy=.9):
```

```
        self.track_len = track_len
        self.pos = 0
        self.kernel = kernel
        self.sensor_accuracy = sensor_accuracy

    def move(self, distance=1):
        """ move in the specified direction
        with some small chance of error"""

        self.pos += distance
        # insert random movement error according to kernel
        r = random.random()
        s = 0
        offset = -(len(self.kernel) - 1) / 2
        for k in self.kernel:
            s += k
            if r <= s:
                break
            offset += 1
        self.pos = int((self.pos + offset) % self.track_len)
        return self.pos

    def sense(self):
        pos = self.pos
         # insert random sensor error
        if random.random() > self.sensor_accuracy:
            if random.random() > 0.5:
                pos += 1
            else:
                pos -= 1
        return pos
```

现在，我们可以开始编写这个滤波器了。我们会将其放入一个函数中，这样就可以根据不同的假设来运行该函数。我假设机器人总是从轨道的起点开始移动。该轨道被设定为 10 个单位长，但实际上，你可以将其想象成长度为 10,000 个单位的轨道，只不过磁铁图案每隔 10 个单位就会重复出现一次。将轨道长度设为 10 个单位，便于我们进行绘制和检查。

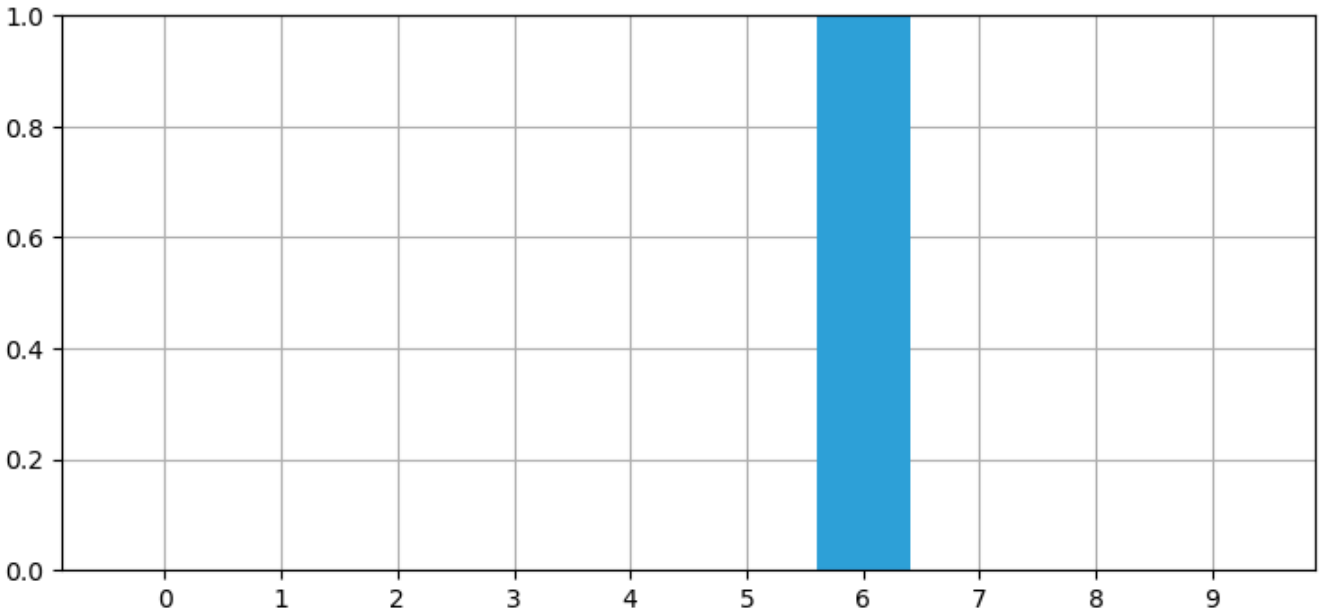
```
def train_filter(iterations, kernel, sensor_accuracy,
             move_distance, do_print=True):
    track = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
    prior = np.array([.9] + [0.01]*9)
    posterior = prior[:]
    normalize(prior)

    robot = Train(len(track), kernel, sensor_accuracy)
    for i in range(iterations):
        # move the robot and
        robot.move(distance=move_distance)

        # peform prediction
        prior = predict(posterior, move_distance, kernel)

        #  and update the filter
        m = robot.sense()
        likelihood = lh_hallway(track, m, sensor_accuracy)
        posterior = update(likelihood, prior)
        index = np.argmax(posterior)

        if do_print:
            print(f'time {i}: pos {robot.pos}, sensed {m}, at position {track[robot.pos]}')
```

```
            conf = posterior[index] * 100
            print(f'        estimated position is {index} with confidence {conf:.4f}%:')

    book_plots.bar_plot(posterior)
    if do_print:
        print()
        print('final position is', robot.pos)
        index = np.argmax(posterior)
        conf = posterior[index]*100
        print(f'Estimated position is {index} with confidence {conf:.4f}')
```

请仔细阅读代码，确保自己完全理解了它的功能。现在，让我们在不使用传感器、也不进行任何移动操作的情况下运行一下程序。如果代码没有错误，那么程序应该能够准确地定位机器人的位置。虽然输出结果看起来有点复杂，但如果你对更新/预测过程的原理不太清楚，一定要仔细阅读相关内容，以便彻底理解其工作原理。

```
import random

random.seed(3)
np.set_printoptions(precision=2, suppress=True, linewidth=60)
train_filter(4, kernel=[1.], sensor_accuracy=.999,
             move_distance=4, do_print=True)
```

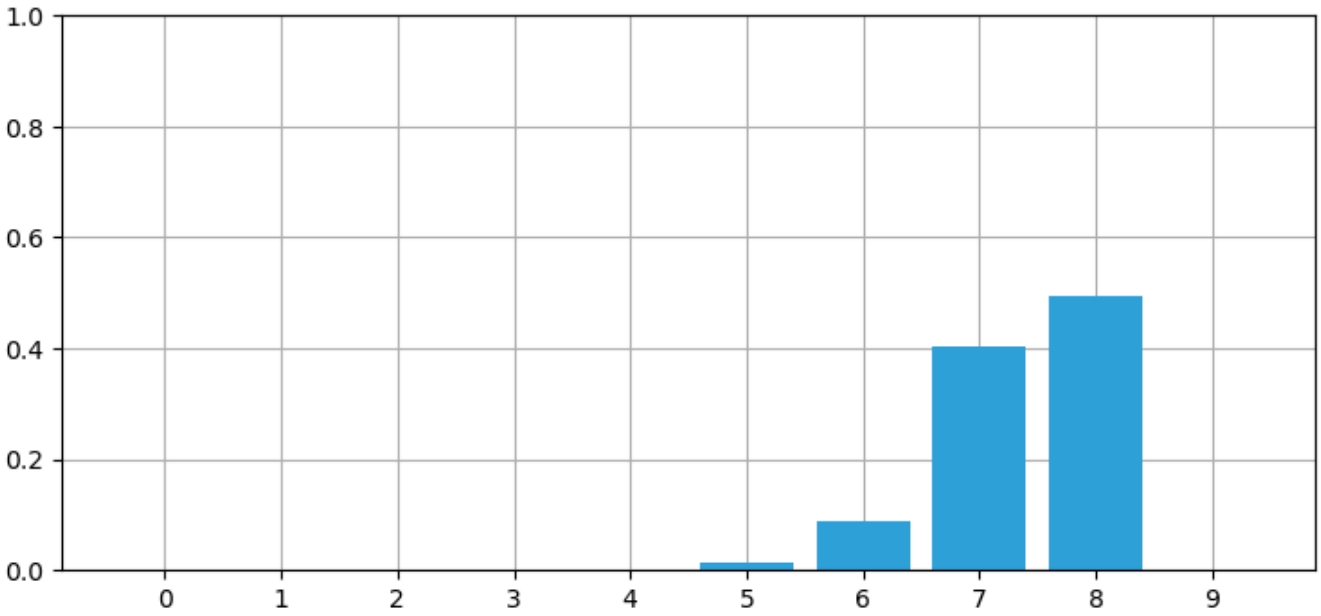
```
time 0: pos 4, sensed 4, at position 4
        estimated position is 4 with confidence 99.9900%:
time 1: pos 8, sensed 8, at position 8
        estimated position is 8 with confidence 100.0000%:
time 2: pos 2, sensed 2, at position 2
        estimated position is 2 with confidence 100.0000%:
time 3: pos 6, sensed 6, at position 6
        estimated position is 6 with confidence 100.0000%:

final position is 6
Estimated position is 6 with confidence 100.0000
```



我们可以看到，该代码能够完美地控制机器人的动作。因此，我们有理由相信该代码是可靠的。现在，让我们来看看该代码在遇到错误时的表现如何。

```
random.seed(5)
train_filter(4, kernel=[.1, .8, .1], sensor_accuracy=.9,
         move_distance=4, do_print=True)
```

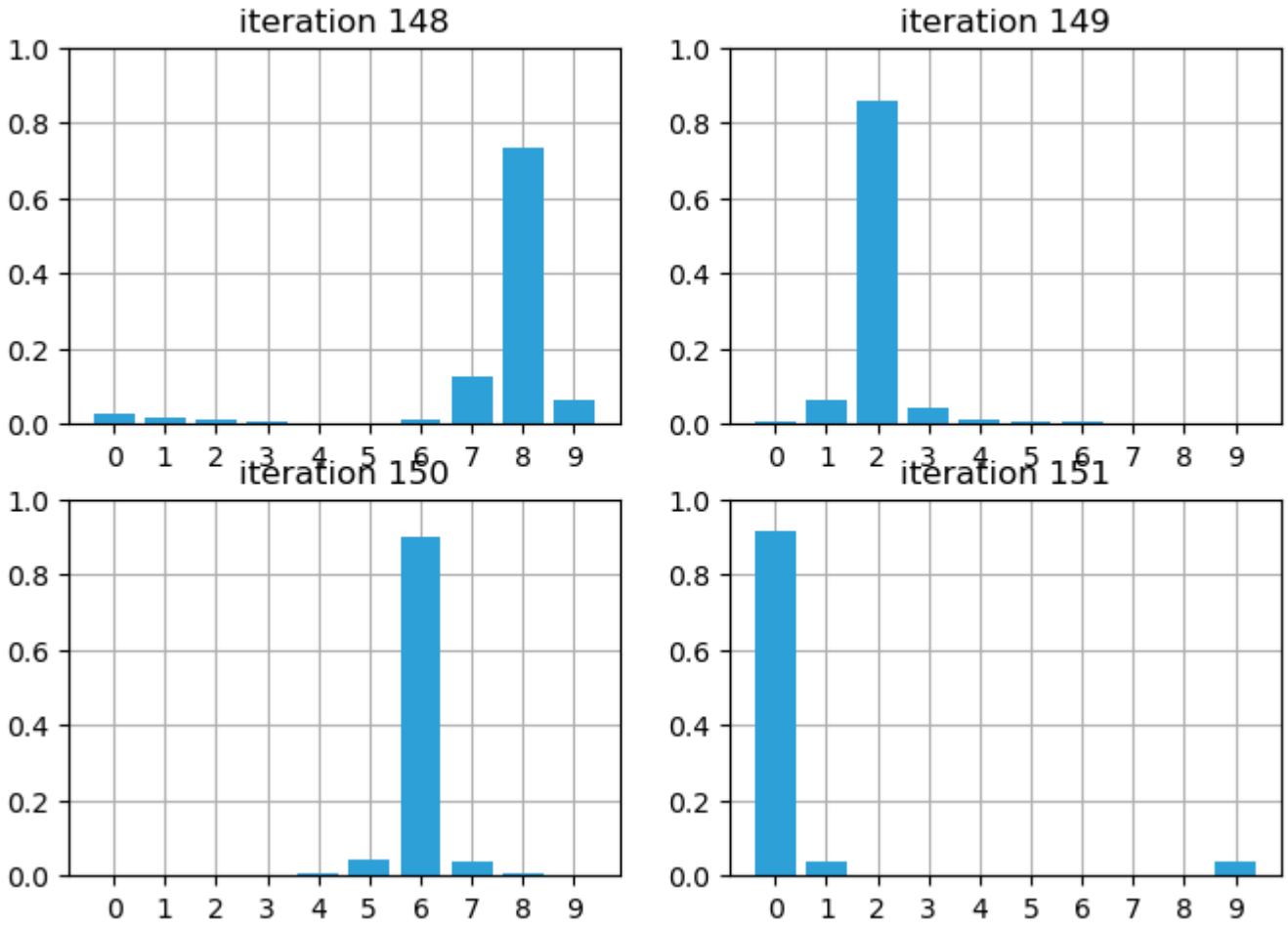
```
time 0: pos 4, sensed 4, at position 4
        estimated position is 4 with confidence 96.0390%:
time 1: pos 8, sensed 9, at position 8
        estimated position is 9 with confidence 52.1180%:
time 2: pos 3, sensed 3, at position 3
        estimated position is 3 with confidence 88.3993%:
time 3: pos 7, sensed 8, at position 7
        estimated position is 8 with confidence 49.3174%:

final position is 7
Estimated position is 8 with confidence 49.3174
```



在时间点 1 时出现了测量误差，不过我们对自己的定位仍然相当有信心。

现在，让我们进行一次长时间的模拟测试，看看该滤波器在面对错误时会有怎样的反应。

```
with figsize(y=5.5):
    for i in range (4):
        random.seed(3)
        plt.subplot(221+i)
        train_filter(148+i, kernel=[.1, .8, .1],
                     sensor_accuracy=.8,
                     move_distance=4, do_print=False)
        plt.title (f'iteration {148 + i}')
```



我们可以看到，在第 149 次迭代时出现了问题，置信度有所下降。不过，经过几次迭代后，滤波器能够自我修正，重新获得对估计位置的信任。

贝叶斯定理与全概率定理

在本章中，我们完全是通过对所掌握的信息进行推理来推导出相关的数学公式。在这个过程中，我们发现了贝叶斯定理和全概率定理。

贝叶斯定理告诉我们，如何根据先前的信息来计算某个事件的概率。

我们通过这种概率计算方式来实现 `update()` 函数：

$$posterior = \frac{likelihood \times prior}{normalization\,factor}$$

我们还没有建立起用于阐述贝叶斯定理的数学体系，但这就是贝叶斯定理。本书中提到的每一个滤波机制，其实都是贝叶斯定理的具体体现。在下一章中，我们会进一步发展相关的数学理论，不过这样做反而会让人忽略这个公式所表达的简单道理。

$$updated knowledge = \|likelihood of new knowledge \times prior knowledge\|$$

其中，$\|\cdot\|$ 表示对该项进行归一化处理。

我们是通过一个简单的例子来理解这个原理的：想象一只狗在走廊里行走。不过，正如我们将看到的，同样的原理也适用于各种滤波问题。在后面的每一章中，我们都会用到这个原理。

同样地， `predict()` 步骤用于计算多个可能事件的总概率。在统计学中，这被称为“总概率定理”。我们将在下一章中详细讨论这一内容，届时会先介绍一些相关的数学知识作为铺垫。

目前，你需要明白：贝叶斯定理其实是一种将新信息纳入现有信息中的公式。

摘要/总结

这段代码虽然很简短，但效果却相当出色！我们实现了一种贝叶斯滤波器。我们学会了如何在没有初始信息的情况下，从有噪声的传感器数据中提取有用的信息。尽管本章中使用的传感器产生的数据噪声很大（例如，大多数传感器的准确率不到 80%），但我们仍然能够快速确定狗的大致位置。我们了解到，虽然每次预测都会使我们的认知有所偏差，但只要再获取一次测量数据，即便该数据也包含噪声，我们的认知水平仍会得到提升，从而更接近真实结果。

这本书主要介绍卡尔曼滤波器的相关内容。虽然其中所使用的数学公式有所不同，但其逻辑原理与本章所阐述的完全一致。卡尔曼滤波器运用贝叶斯推理，结合各种测量数据和过程模型来得出估算结果。

如果你能理解这一章的内容，那么你就能掌握卡尔曼滤波器的使用方法了。这一点非常重要。如果有什么地方不明白，请重新阅读这一章的内容，并尝试自己编写相关代码。本书的后续内容都将以我们在这里所学习的算法为基础。如果你不明白为什么这种滤波器能有效工作，那么你很难理解本书的其余内容。不过，如果你能理解其基本原理——即在测量时将各种概率相乘，在更新时调整这些概率，就能得到一个稳定的解决方案——那么在稍微学习一些数学知识后，你就可以开始实际应用卡尔曼滤波器了。

参考文献

- [1] D. Fox、W. Burgard 和 S. Thrun。“蒙特卡洛定位：一种高效的移动机器人位置估计方法。”载于《人工智能研究杂志》，1999 年。

http://www.cs.cmu.edu/afs/cs/project/jair/pub/volume11/fox99a-html/jair-localize.html

- [2] Dieter Fox 等人。“用于位置估计的贝叶斯滤波器”。载于《IEEE 普适计算》杂志，2003 年 9 月。

http://swarmlab.unimaas.nl/wp-content/uploads/2012/07/fox2003bayesian.pdf

- [3] 塞巴斯蒂安·特伦。《用于机器人的人工智能》。

https://www.udacity.com/course/cs373

- [4] 可汗学院。“卷积入门”

https://www.khanacademy.org/math/differential-equations/laplace-transform/convolution-integral/v/introduction-to-the-convolution

- [5] 维基百科。“卷积”

http://en.wikipedia.org/wiki/Convolution

- [6] 维基百科。“全概率定理”

  http://en.wikipedia.org/wiki/Law_of_total_probability

- [7] Wikipedia. "Time Evolution"

https://en.wikipedia.org/wiki/Time_evolution

- [8] We need to rethink how we teach statistics from the ground up

http://www.statslife.org.uk/opinion/2405-we-need-to-rethink-how-we-teach-statistics-from-the-ground-up

第 3 章：高斯分布

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

最后一章探讨了离散贝叶斯滤波器的一些缺点。在许多跟踪和滤波问题中，我们希望使用的是单峰且连续的滤波器。也就是说，我们希望用浮点数来进行计算（即实现连续性），并且只输出一个最可能的估计值（即单峰性）。例如，我们希望知道某架飞机的位置是(12.34, -95.54, 2389.5)，其中分别代表纬度、经度和高度。我们不希望滤波器给出“飞机可能在(1.65, -78.01, 2100.45)或(34.36, -98.23, 2543.79)”这样的答案。这种结果与我们对物理世界的直观理解不符。此外，如前所述，处理多峰情况会带来极高的计算成本。当然，如果有多个位置估计值，那么导航也就变得不可能了。

我们希望有一种单一的、连续的方式来表示概率，这种方式能够真实反映现实世界的运作方式，同时计算起来也高效。高斯分布正好具备所有这些特性。

均值、方差和标准差

你们中的大多数人应该都接触过统计学相关的内容，不过我还是想再详细讲解一下。即便你们觉得自己已经很熟悉这些内容了，也请务必仔细阅读。我这么做有两个原因：首先，我希望我们能以相同的方式理解各种术语；其次，我希望大家能建立起对统计学的直观理解，这对后面章节的学习会有很大帮助。很多人上完统计学课程后，只记得那些公式和计算方法，却不清楚自己所学内容的实际意义。

随机变量

每次掷骰子时，结果都在 1 到 6 之间。如果我们用一个公平的骰子掷一百万次，那么预计会有 1/6 的概率得到 1 点。因此，我们说得到 1 点的概率是 1/6。同样地，如果我问你下一次掷骰子得到 1 点的概率是多少，你也会回答 1/6。

这些数值及其对应的概率共同构成了一个随机变量。这里所说的“随机”，并不意味着该过程是不可预测的，而只是表示我们无法确切知道最终的结果。掷骰子的结果其实是可确定的，但我们缺乏足够的信息来计算出具体结果。我们只能以概率的方式来了解可能的结果。

在定义各种术语时，数值的范围被称为“样本空间”。对于骰子来说，其样本空间为{1, 2, 3, 4, 5, 6}。对于硬币来说，其样本空间为{正面, 反面}。所谓“空间”，其实是一个具有特定结构的集合。因此，骰子的样本空间实际上就是 1 到 6 这个范围内的自然数集合的一个子集。

另一个随机变量的例子是大学里学生的身高。在这里，样本空间指的是由生物学因素所决定的两个数值范围之间的所有实数。

像抛硬币或掷骰子这样的随机变量属于离散随机变量。这意味着，它们的样本空间由有限个数值或可数无限个数值构成，比如自然数。而人的身高则属于连续随机变量，因为人的身高可以取两个极限值之间的任意实数。

不要将随机变量的测量结果与其实际数值混为一谈。如果我们只能将一个人的身高精确到 0.1 米，那么我们只能记录下 0.1、0.2、0.3……2.7 这些数值，也就是 27 个离散的值。不过，一个人的身高实际上可以介于这些数值之间的任意实数范围内，因此身高属于连续型随机变量。

在统计学中，大写字母通常用来表示随机变量，这些字母一般取自字母表的后面部分。因此，我们可以说 X 表示掷骰子的结果，而 X 则表示大一诗歌班上学生的身高。在后面的章节中，我们将利用线性代数来解决这些问题。在那些章节里，向量会用小写字母表示，矩阵则用大写字母表示。不过，这两种惯例有时会相互冲突，这时就需要根据上下文来判断作者到底使用了哪种表示方式。我个人总是用粗体符号来表示向量和矩阵，这样就能更清楚地区分两者了。

概率分布

概率分布描述了随机变量在样本空间中取各个值的概率。例如，对于一个公平的六面骰子来说：

价值/重要性	概率
1	1/6
2	1/6
3	1/6
4	1/6
5	1/6
6	1/6

我们用小写字母 p 来表示这种分布： $p(x)$ 。用普通的函数表示法来写，就是：

$$P(X = 4) = p(4) = \frac{1}{6}$$

这意味着，骰子掷出 4 的概率为 $\frac{1}{6}$ 。 $P(X = x_k)$ 实际上表示“ X 为 x_k 的概率”。请注意这种细微的符号差异：大写的 P 表示单个事件的概率，而小写的 p 则代表概率分布函数。如果不仔细注意的话，很容易搞混这些概念。有些教材会用 Pr 来代替 P，以避免这种混淆。

另一个例子就是公平的硬币。它的样本空间为{正面, 反面}。由于硬币是公平的，因此出现正面的概率为 50%，出现反面的概率也为 50%。我们可以将其表示为：

$$P(X = H) = 0.5$$
$$P(X = T) = 0.5$$

样本空间并不唯一。对于骰子来说，一个有效的样本空间可以是{1, 2, 3, 4, 5, 6}。另一个有效的样本空间可以是{偶数, 奇数}。再比如，{所有角上有点，所有角上没有点}也是一种有效的样本空间。只要样本空间能够涵盖所有可能性，并且每个事件都只能由样本空间中的一个元素来表示，那么这个样本空间就是有效的。{偶数, 1, 3, 4, 5}不是骰子的有效样本空间，因为数字 4 既属于“偶数”，也属于“4”。

离散随机变量所有可能取值的概率之和，被称为离散概率分布；连续随机变量所有可能取值的概率之和，则被称为连续概率分布。

要成为一个概率分布，那么每个数值对应的概率都必须非负，即必须大于或等于 0。其次，所有数值对应的概率之和必须等于 1。这一点从抛硬币的例子中就能很容易理解：如果正面出现的概率是 70%，那么反面出现的概率就必须是 30%。我们将这一要求正式表述为：

$$\sum_u P(X = u) = 1$$

对于离散分布而言，情况也是如此。

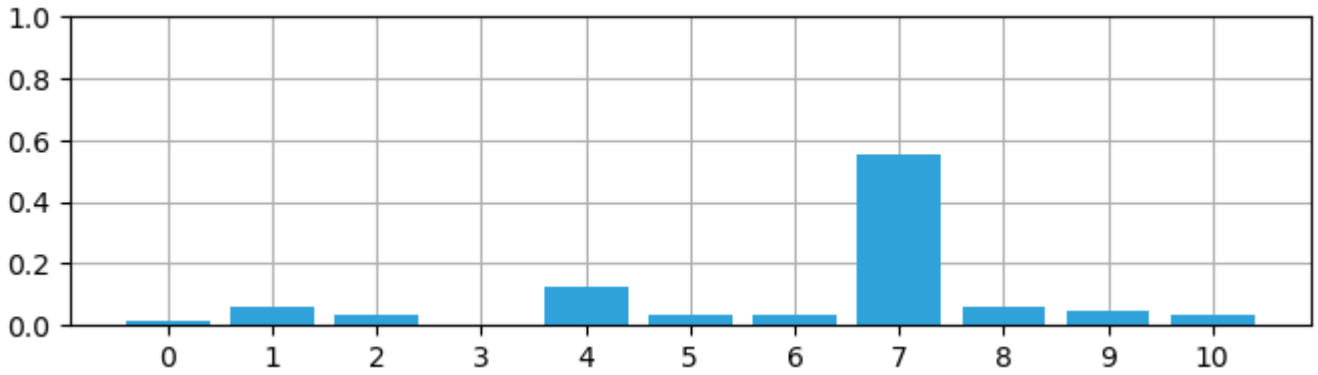
$$\int_u P(X = u)du = 1$$

适用于连续分布的情况。

在上一章中，我们利用概率分布来估算狗在走廊中的位置。例如：

```
import numpy as np
import kf_book.book_plots as book_plots

belief = np.array([1, 4, 2, 0, 8, 2, 2, 35, 4, 3, 2])
belief = belief / np.sum(belief)
with book_plots.figsize(y=2):
    book_plots.bar_plot(belief)
print('sum = ', np.sum(belief))
```

```
sum =  1.0
```



每个位置对应的概率都在 0 到 1 之间，所有概率之和等于 1。因此，这实际上是一种概率分布。由于每个概率都是离散的，我们可以更准确地将其称为离散概率分布。在实际应用中，除非有特殊需要，否则我们通常不会刻意区分“离散”和“连续”这两个概念。

随机变量的均值、中位数和众数

给定一组数据，我们通常希望知道该组的代表性数值或平均值。衡量这一数值的方法有很多，这一概念被称为“集中趋势的度量”。例如，我们可能想知道某个班级里学生的平均身高。我们都知道如何计算一组数据的平均值，不过让我再详细说明一下，以便引入更正式的符号和术语。平均值的另一个说法是“均值”。我们通过将所有数值相加，再除以数值的个数来计算均值。如果以米为单位来表示学生的身高的话……

$$X = \{1.8, 2.0, 1.7, 1.9, 1.6\}$$

我们将平均值计算为

$$\mu = \frac{1.8 + 2.0 + 1.7 + 1.9 + 1.6}{5} = 1.8$$

按照惯例，通常用符号 μ （μ）来表示平均值。

我们可以用方程式来正式表达这一计算过程。

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

NumPy 提供了 `numpy.mean()` 这个函数来计算平均值。

```
x = [1.8, 2.0, 1.7, 1.9, 1.6]
np.mean(x)
```

```
1.8
```

为方便使用，NumPy 数组提供了 `mean()` 这个方法。

```
x = np.array([1.8, 2.0, 1.7, 1.9, 1.6])
x.mean()
```

```
1.8
```

一组数字的众数是指出现次数最多的那个数。如果只有一个数出现次数最多，那么我们称这种数据集为单峰的；如果有两个或两个以上数出现次数相同，那么这种数据集就是多峰的。例如，集合{1, 2, 2, 2, 3, 4, 4, 4}的众数是 2 和 4，因此它是多峰的；而集合{5, 7, 7, 13}的众数是 7，所以它是单峰的。在本书中，我们不会用这种方式来计算众数，但我们会从更一般的意义上使用“单峰”和“多峰”这两个概念。例如，在关于离散贝叶斯的章节中，我们将对狗所在位置的判断视为一种多峰分布，因为我们给狗可能所在的不同位置分配了不同的概率。

最后，一组数字的中位数就是该组数字的中间值。也就是说，有一半的数字小于中位数，另一半数字大于中位数。这里的“大于”和“小于”都是相对于已排序的数字而言的。如果该组数字的个数是偶数，则取中间的两个数字的平均值作为中位数。

Numpy 提供了 `numpy.median()` 这个函数来计算中位数。可以看到，{1.8, 2.0, 1.7, 1.9, 1.6}这个数列的中位数是 1.8，因为在排序后，1.8 是该数列的第三个元素。在这个例子中，中位数等于平均值，但通常情况下并非

如此。

```
np.median(x)
```

```
1.8
```

随机变量的期望值

随机变量的期望值，指的是：如果我们对它进行无限次抽样，然后把所有样本的值加起来再平均，那么得到的平均值就是该随机变量的期望值。假设我们有 $x = [1, 3, 5]$ 个数值，且每个数值出现的概率都相等。那么， x 的平均值应该是多少呢?

当然，这就是 1、3 和 5 的平均值，也就是 3。这很合理：我们预期 1、3 和 5 出现的次数应该相等。所以，$(1 + 3 + 5) / 3 = 3$ 显然就是这一系列无穷多个样本的平均值。换句话说，这里的期望值其实就是样本空间的平均值。

现在，假设每个数值出现的概率各不相同。比如说，数字 1 出现的概率为 80%，数字 3 出现的概率为 15%，而数字 5 出现的概率仅为 5%。在这种情况下，我们通过将每个数值乘以其出现的概率，再将所有结果相加来计算期望值。对于这种情况，我们可以这样计算：

$$E[X] = (1)(0.8) + (3)(0.15) + (5)(0.05) = 1.5$$

在这里，我使用符号 $E[X]$ 来表示 x 的期望值。有些文献则使用 $E(x)$ 来表示这一数值。 x 的值为 1.5，这一结果在直觉上是可以理解的：因为 x 为 1 的概率远远大于为 3 或 5 的概率；而 3 的概率又大于 5 的概率。

我们可以通过以下方式来 formalize 这一点：令 x_i 表示 X 的 i^{th} 值，而 p_i 则表示该值出现的概率。这样我们就得到了所需的表达式。

$$E[X] = \sum_{i=1}^{n} p_i x_i$$

简单的代数运算就能证明：如果所有概率都相等，那么期望值就等于平均值。

$$E[X] = \sum_{i=1}^{n} p_i x_i = \frac{1}{n} \sum_{i=1}^{n} x_i = \mu_x$$

如果 x 是连续函数，那么我们可以将求和替换为积分，具体如下所示。

$$E[X] = \int_a^b x f(x) dx$$

其中， $f(x)$ 表示 x 的概率分布函数。我们暂时还不需要使用这个公式，但会在下一章中用到它。

我们可以用一些 Python 代码来模拟这个过程。在这里，我选取了 1,000,000 个样本，然后计算出我们通过分析方法所得到的概率分布的期望值。

```
total = 0
N = 1000000
```

```
for r in np.random.rand(N):
    if r <= .80: total += 1
    elif r < .95: total += 3
    else: total += 5

total / N
```

```
1.50053
```

可以看出，计算得到的数值与通过分析方法得出的数值非常接近。不过，这两个数值并不完全相同，因为要得到精确的数值，需要无限大的样本量。

锻炼/运动

掷骰子的期望值是多少?

解决方案

每一方的概率都是相等的，因此每方的概率都是 1/6。

锻炼/运动

在均匀连续分布的条件下

$$f(x) = \frac{1}{b-a}$$

计算 $a = 0$ 和 $b = 20$ 的期望值。

解决方案

$$\begin{aligned} E[X] &= \int_0^{20} x\frac{1}{20}dx \\ &= \left[\frac{x^2}{40}\right]_0^{20} \\ &= 10 - 0 \\ &= 10 \end{aligned}$$

随机变量的方差

上述计算结果给出了学生的平均身高，但并未告诉我们所有想要了解的信息。例如，假设我们有三个班级，分别记为 X 、 Y 和 Z ，这些班级学生的身高分别为：

```
X = [1.8, 2.0, 1.7, 1.9, 1.6]
Y = [2.2, 1.5, 2.3, 1.7, 1.3]
Z = [1.8, 1.8, 1.8, 1.8, 1.8]
```

使用 NumPy 进行分析后我们发现，每个类别的平均身高都是相同的。

```
print(np.mean(X), np.mean(Y), np.mean(Z))
```

```
1.8 1.8 1.8
```

每个组别的平均值都是 1.8 米。不过需要注意的是，第二组别中身高的差异远远大于第一组别，而第三组别则完全没有身高上的差异。

平均值能让我们了解一些关于数据的信息，但并不能完全反映实际情况。我们需要知道学生的身高之间存在多大的差异。原因可能有很多。比如，某个学区需要订购 5,000 张课桌，他们当然希望所购买的课桌尺寸能够适应所有学生的身高差异。

统计学将这种衡量差异的概念正式化为“标准差”和“方差”这两个概念。计算方差的公式为：

$$VAR(X) = E[(X - \mu)^2]$$

暂且不考虑平方项，可以看出，方差实际上就是样本空间 X 与均值 μ: 之间的偏差的期望值（即 $X - \mu$）。关于平方项的用途，我稍后会再解释。期望值的计算公式为 $E[X] = \sum_{i=1}^{n} p_i x_i$，将其代入上述公式中，就可以得到最终结果。

$$VAR(X) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

让我们来计算这三个类别的方差，看看会得到什么数值，同时也能更好地理解这个概念。

X 的平均值为 1.8（$\mu_x = 1.8$），因此我们进行如下计算：

$$\begin{aligned} VAR(X) &= \frac{(1.8-1.8)^2 + (2-1.8)^2 + (1.7-1.8)^2 + (1.9-1.8)^2 + (1.6-1.8)^2}{5} \\ &= \frac{0 + 0.04 + 0.01 + 0.01 + 0.04}{5} \\ VAR(X) &= 0.02m^2 \end{aligned}$$

NumPy 提供了 `var()` 函数来计算方差：

```
print(f"{np.var(X):.2f} meters squared")
```

```
0.02 meters squared
```

这可能有点难以理解。高度是以米为单位的，而方差则是以米的平方为单位的。因此，我们使用更常见的度量标准——标准差。标准差的定义是方差的平方根。

$$\sigma = \sqrt{VAR(X)} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$$

通常，用 σ 表示标准差，用 σ^2 表示方差。在本书的大部分内容中，我会使用 σ^2 来表示方差，而不是 $VAR(X)$；其实这两个符号表示的是同一种概念。

在第一节课中，我们用以下方法来计算标准差。

$$\begin{aligned}\sigma_x &= \sqrt{\frac{(1.8-1.8)^2 + (2-1.8)^2 + (1.7-1.8)^2 + (1.9-1.8)^2 + (1.6-1.8)^2}{5}} \\ &= \sqrt{\frac{0 + 0.04 + 0.01 + 0.01 + 0.04}{5}} \\ \sigma_x &= 0.1414\end{aligned}$$

我们可以使用 NumPy 中的相关函数来验证这个计算结果。该函数用于计算标准差。‘std’是标准差的常用缩写形式。

```
print(f"std {np.std(X):.4f}")
print(f"var {np.std(X)**2:.4f}")
```

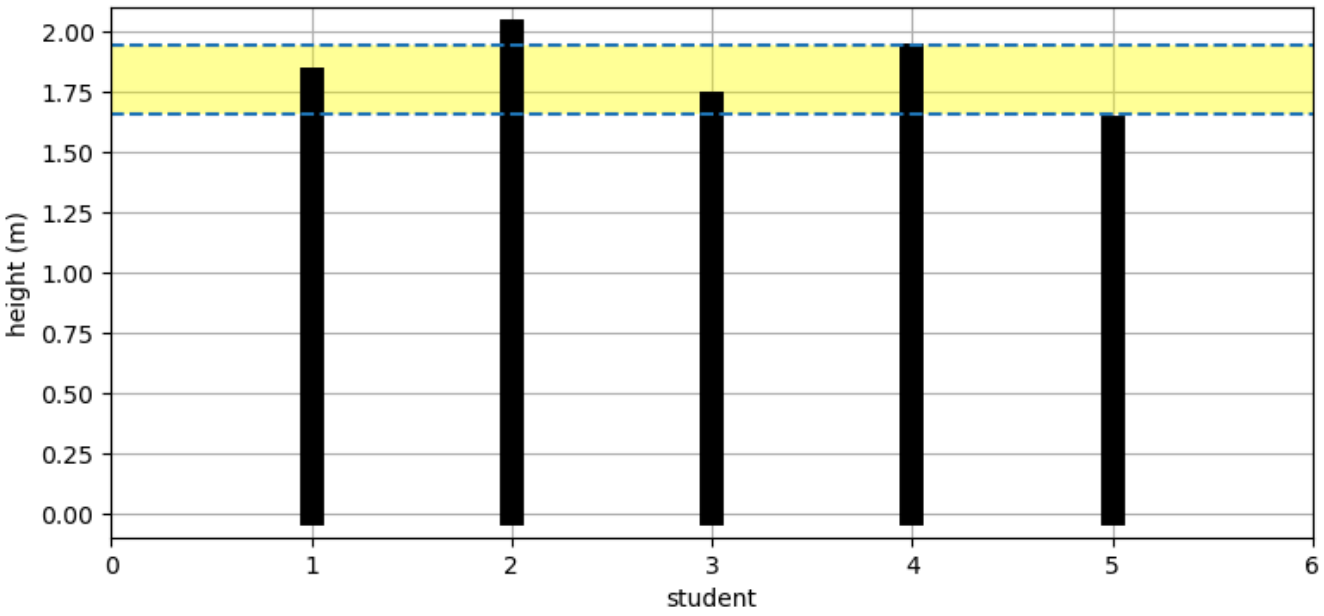
```
std 0.1414
var 0.0200
```

当然， $0.1414^2 = 0.02$ 的结果也与我们之前计算出的方差一致。

标准差到底表示什么意思呢？它告诉我们，各个数值之间的差异有多大。“差异有多大”并不是一个严格的数学概念。在下一节中，当我们引入高斯分布的概念后，就能更精确地定义这一概念了。目前可以这么说：对于大多数情况而言，68%的数值都位于平均值的一个标准差范围内。换句话说，对于某个随机选取的群体来说，68%的学生的身高会介于 1.66 米（1.8 米减去 0.1414 米）到 1.94 米（1.8 米加上 0.1414 米）之间。

我们可以通过图表来直观地了解这一点：

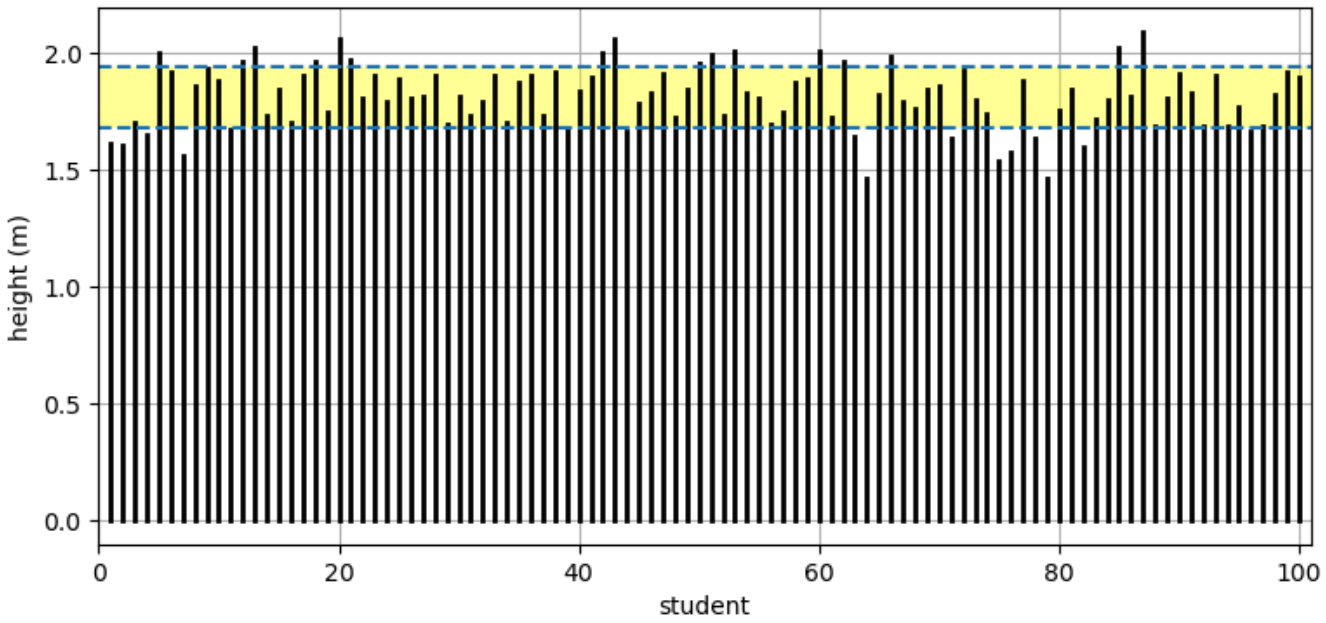
```
from kf_book.gaussian_internal import plot_height_std
import matplotlib.pyplot as plt

plot_height_std(X)
```



如果只有 5 名学生，那么显然不可能让 68%的数据落在 1 个标准差范围内。不过，我们发现 5 名学生中有 3 名的数据落在 60%的范围内。在只有 5 个样本的情况下，60%已经是最接近 68%的数值了。让我们来看看有 100 名学生的班级的情况吧。

> 我们将一个标准差记作 1σ，读作“一个标准差”，而不是“一个西格玛”。两个标准差则记作 2σ，依此类推。

```
from numpy.random import randn
data = 1.8 + randn(100)*.1414
mean, std = data.mean(), data.std()

plot_height_std(data, lw=2)
print(f'mean = {mean:.3f}')
print(f'std  = {std:.3f}')
```




```
mean = 1.808
std  = 0.133
```

粗略估计，大约 68%的数据点的高度都落在平均值 1.8 的±0#范围内。不过，我们也可以通过代码来验证这一点。

```
np.sum((data > mean-std) & (data < mean+std)) / len(data) * 100.
```

```
68.0
```

我们稍后会更深入地探讨这个问题。目前，让我们先来计算一下标准差吧。

$$Y = [2.2, 1.5, 2.3, 1.7, 1.3]$$

Y的平均值为 $\mu = 1.8$ 米。

$$\begin{aligned}\sigma_y &= \sqrt{\frac{(2.2-1.8)^2 + (1.5-1.8)^2 + (2.3-1.8)^2 + (1.7-1.8)^2 + (1.3-1.8)^2}{5}} \\ &= \sqrt{0.152} = 0.39\ m\end{aligned}$$

我们将使用 NumPy 来验证这一点。

```
print(f'std of Y is {np.std(Y):.2f} m')
```

```
std of Y is 0.39 m
```

这与我们的预期相符。 Y的身高差异更大，标准差也更高。

最后，让我们来计算 Z 的标准差。由于各个数值之间没有差异，因此标准差应该为零。

$$\begin{aligned}\sigma_z &= \sqrt{\frac{(1.8-1.8)^2 + (1.8-1.8)^2 + (1.8-1.8)^2 + (1.8-1.8)^2 + (1.8-1.8)^2}{5}} \\ &= \sqrt{\frac{0+0+0+0+0}{5}} \\ \sigma_z &= 0.0\ m\end{aligned}$$

```
print(np.std(Z))
```

```
0.0
```

在继续之前，我需要指出：我暂时忽略了一个事实，那就是通常情况下，男性的身高都高于女性。一般来说，如果一个群体中只有男性或只有女性，那么该群体内的身高差异会小于由男女共同组成的群体。其他因素也是如此。营养良好的儿童比营养不良的儿童身高更高；斯堪的纳维亚人比意大利人更高。在设计实验时，统计学家必须把这些因素考虑进去。

我建议，我们进行这项分析的初衷，可能是为了为一个学区订购课桌。对于每个年龄组来说，很可能会出现两个不同的平均身高值：一个接近女性学生的平均身高，另一个则接近男性学生的平均身高。整个班级的平均身高则介于这两个数值之间。如果我们按照所有学生的平均身高来订购课桌，那么最终得到的课桌很可能既不适合男生，也不适合女生使用！

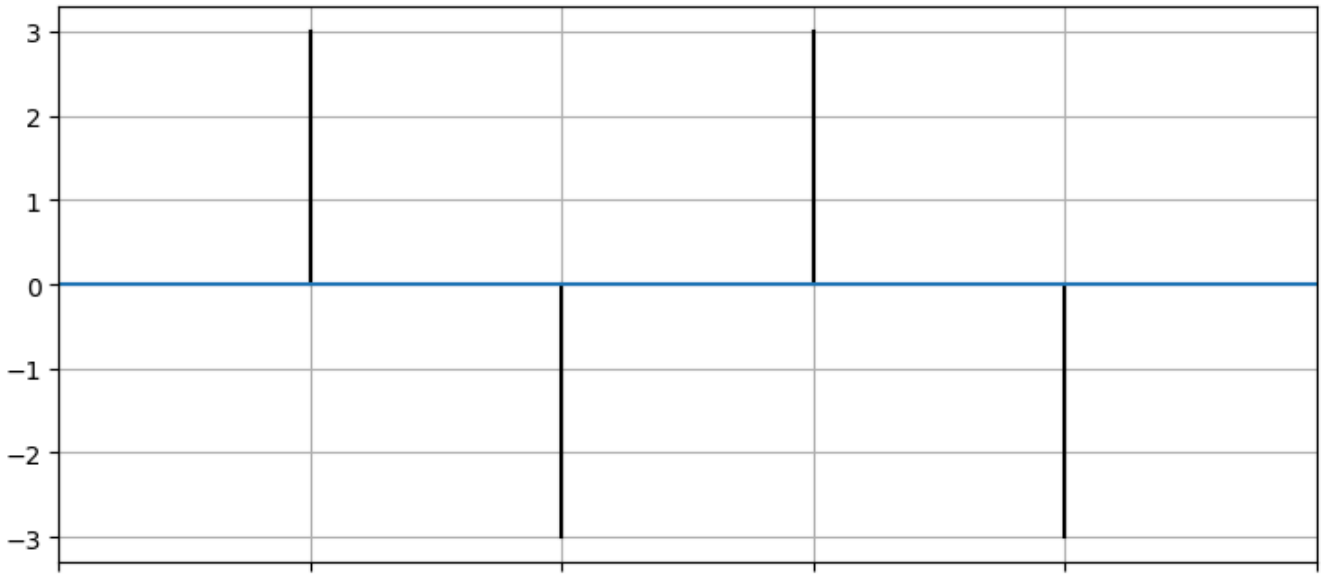
在本书中，我们不会探讨这些问题。如果您需要了解处理这些问题的方法，可以参考任何标准的概率论教材。

为什么是差值的平方呢?

为什么我们要用差值的平方来计算方差呢？其实可以详细解释一下其中的数学原理，但让我们用简单的方式来理解吧。下图展示了 X 的值与 $X = [3, -3, 3, -3]$ 的平均值之间的关系。

```
X = [3, -3, 3, -3]
mean = np.average(X)
for i in range(len(X)):
```

```
    plt.plot([i ,i], [mean, X[i]], color='k')
plt.axhline(mean)
plt.xlim(-1, len(X))
plt.tick_params(axis='x', labelbottom=False)
```



如果我们不计算差值的平方，那么各种正负号就会相互抵消，结果为零：

$$\frac{(3-0)+(-3-0)+(3-0)+(-3-0)}{4}=0$$

这显然是错误的，因为数据中的方差肯定大于 0。

也许我们可以使用绝对值来计算吧？通过观察可以发现，计算结果为 $12 / 4 = 3$，这显然是正确的——因为每个数值都比平均值大 3。但如果我们使用 $Y = [6, -2, -3, 1]$ 呢？这种情况下，计算结果为 $12 / 4 = 3$。显然，Y 的数值分布比 X 更分散，但计算出的方差却相同。如果我们使用平方值来计算的话，Y 的方差为 3.5，这一数值反映了 Y 更大的波动性。

这并不能作为该方法的正确性的证明。事实上，这一方法的发明者卡尔·弗里德里希·高斯也意识到，这种处理方式其实有些任意性。如果数据中存在异常值，那么对差异进行平方处理会使得这些异常值的权重过大。例如，让我们来看看以下情况会怎样：

```
X = [1, -1, 1, -2, -1, 2, 1, 2, -1, 1, -1, 2, 1, -2, 100]
print(f'Variance of X with outlier    = {np.var(X):6.2f}')
print(f'Variance of X without outlier = {np.var(X[:-1]):6.2f}')
```

```
Variance of X with outlier    = 621.45
Variance of X without outlier =   2.03
```

这样“正确”吗？由你来判断吧。如果不把那个值为 100 的异常值考虑在内，我们得到的结果就是 $\sigma^2 = 2.03$，这一结果准确地反映了在没有该异常值的情况下 X 的变动情况。而那个异常值却严重影响了整体数据的方差计算结果。我们是想通过这种方式来确认存在异常值，还是希望能够妥善处理这个异常值，同时得到一个接近于没有异常值时的估计值呢？当然，这取决于你的具体问题。

我不会继续沿着这条道路走了；如果您感兴趣的话，可以参考詹姆斯·伯杰在贝叶斯稳健性领域的相关研究成果，或者阅读彼得·J·胡贝尔关于稳健统计学的优秀著作[4]。在本书中，我们将始终按照高斯所定义的方式来使用方差和标准差。

从这一点可以看出，这些统计摘要永远无法完整地反映数据的真实情况。以高斯所定义的方差为例，它无法告诉我们数据中是否存在某个异常值。不过，统计摘要仍然是一种非常有用的工具，因为它能让我们用寥寥几个数字就能概括整个数据集的情况。如果我们有 10 亿个数据点，我们肯定不想手动去查看图表或逐个查看数字列表；统计摘要则为我们提供了一种有效的方式来描述数据的特征。

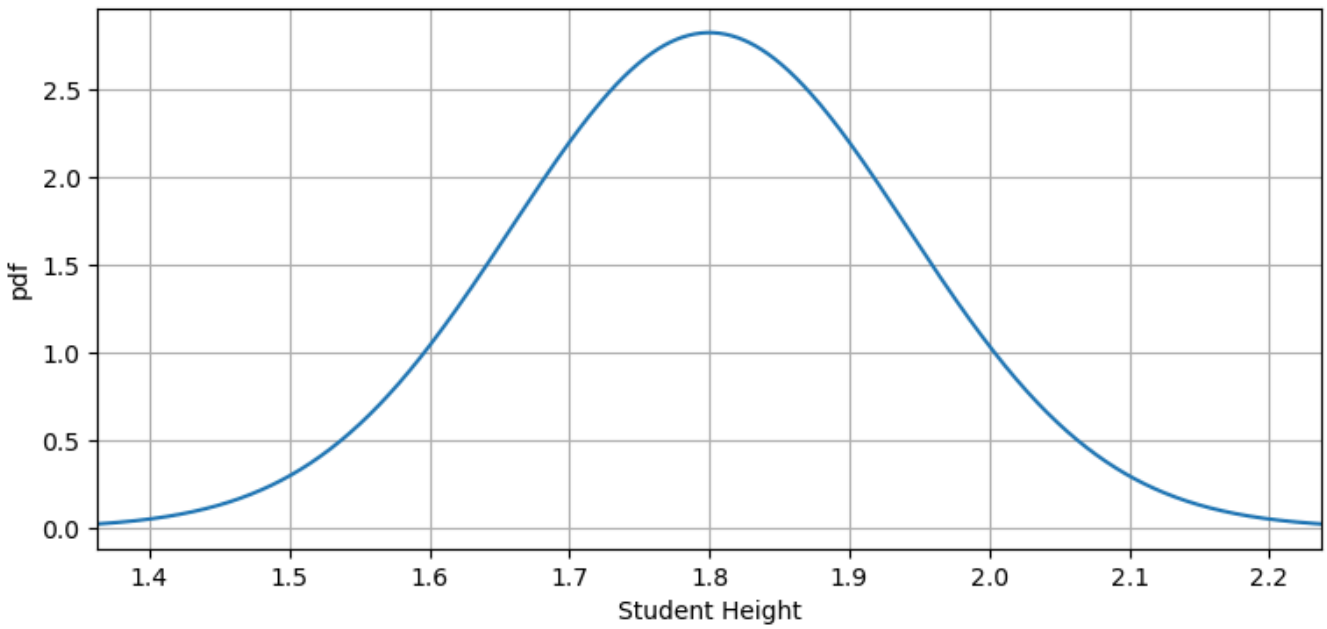
高斯分布/高斯函数

我们现在准备开始学习高斯函数的相关知识了。让我们先回顾一下本章的学习目的吧。

> 我们希望有一种单一的、连续的方式来表示概率，这种方式能够真实反映现实世界的运作方式，同时还能在计算上做到高效。

让我们看一下高斯分布的图表，以便更好地理解我们正在讨论的内容。

```
from filterpy.stats import plot_gaussian_pdf
plot_gaussian_pdf(mean=1.8, variance=0.1414**2,
                  xlabel='Student Height', ylabel='pdf');
```



这条曲线实际上就是概率密度函数。它反映了随机变量取某个特定值的概率大小。从图表中可以看出，学生的身高在 1.8 米左右的概率稍高一些，而身高为 1.9 米的概率则远远高于 1.4 米。换句话说，大多数学生的身高会在 1.8 米左右，而身高为 1.4 米或 2.2 米的学生则很少。另外，可以注意到，这条曲线的中心正好位于 1.8 米这个数值上。

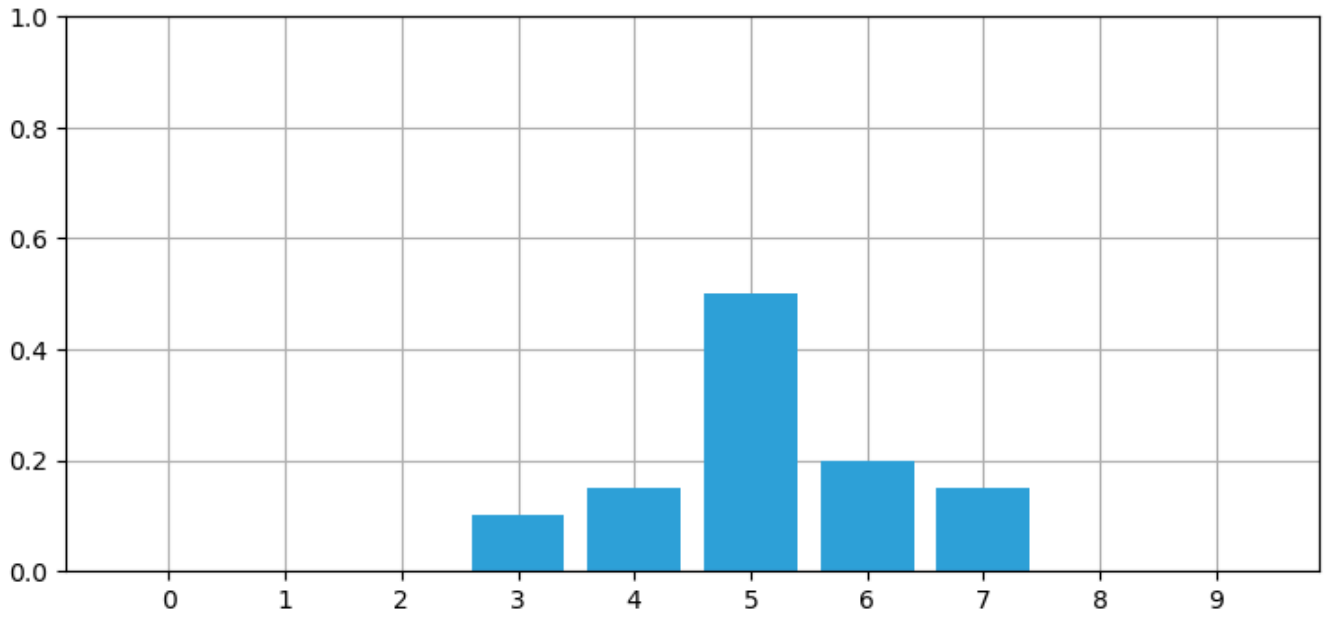
> 我在“Supporting_Notebooks”文件夹中的“Computing_and_Plotting_PDFs”文档里详细介绍了如何绘制高斯曲线图以及更多相关内容。您可以在这里在线阅读该文档[1]。

你可能会认出这就是所谓的“正态分布曲线”。这种曲线随处可见，因为在现实世界中，许多数据的分布都符合这种模式。不过，我不会用“正态分布曲线”来指代高斯分布，因为许多概率分布其实都呈现出类似的曲线形状。对于非数学领域的读者来说，这一概念可能不够精确，因此，当看到有人在没有明确说明的情况下使用这一术语时，需谨慎判断其含义。

这条曲线并非只适用于高度的测量——实际上，大量自然现象都遵循这种分布规律，包括我们在各种滤波处理中使用的传感器所呈现的数据分布。正如我们将看到的，它还具备我们所期望的所有特性：它能够以概率的形式来表示某种“单一的信念或价值”；它是连续的；而且，在计算上也非常高效。我们很快还会发现，它还有其他一些我们可能没有意识到的优点。

为了进一步激励你，不妨回想一下在“离散贝叶斯”那一章中提到的各种概率分布的形状吧：

```
import kf_book.book_plots as book_plots
belief = [0., 0., 0., 0.1, 0.15, 0.5, 0.2, .15, 0, 0]
book_plots.bar_plot(belief)
```



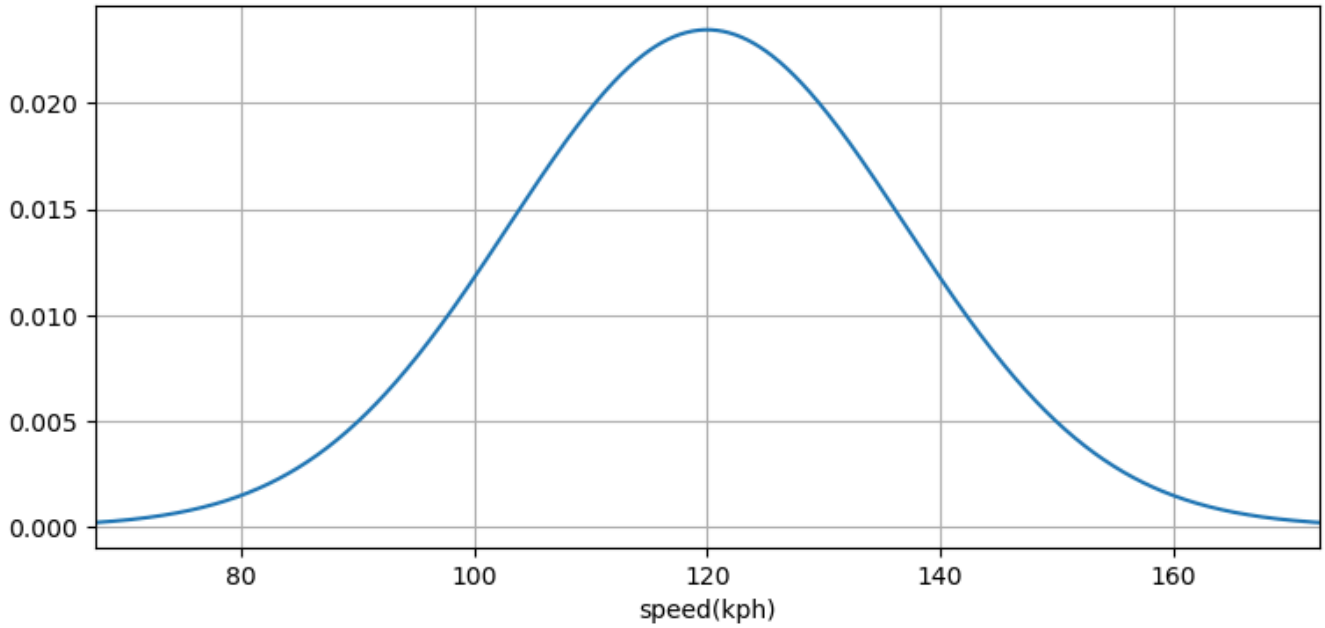
它们并非完美的高斯曲线，但大致类似。在后续的计算中，我们将用高斯曲线来替代那一章中使用的离散概率值！

命名法/术语表

在继续之前，先来解释一下一些术语。这张图表展示了随机变量在某个特定区间内取任意值的概率密度。这到底是什么意思呢？想象一下，我们对某段高速公路上汽车的行驶速度进行了无数次极其精确的测量。然后，我

们可以根据测量结果来绘制图表，显示以不同速度行驶的汽车数量。如果平均速度为 120 公里/小时，那么图表可能会是这样的：

```
plot_gaussian_pdf(mean=120, variance=17**2, xlabel='speed(kph)');
```



y 轴表示概率密度——即，在相应的 x 轴数值所对应的速度下，有多少汽车以该速度行驶。我将在下一节中进一步解释这一点。

高斯模型并不完美。虽然这些图表没有体现出来，但实际上，高斯分布的曲线会无限延伸下去。曲线的两端是数值最低的区域。当然，人的身高或汽车的速度不可能小于零，更不可能达到负无穷大。俗话说“地图无法完全反映现实”，这一点在贝叶斯滤波和统计学中同样适用。上述的高斯分布模型试图描述汽车速度的分布情况，但作为模型，它必然存在缺陷。在各种滤波处理中，模型与现实之间的差异会不断出现。高斯函数被广泛应用于数学的各个领域，不是因为它们能完美地反映现实，而是因为相比其他较为精确的模型，高斯函数更易于使用。不过，即便在本书中，高斯模型也无法完美地描述现实情况，这就迫使我们不得不使用计算成本更高的替代方案。

你会听到，这些分布被称为高斯分布或正态分布。在这个语境中，高斯和正态指的是同一种分布，可以互换使用。在本书中，我会交替使用这两个术语，因为不同的资料可能会使用其中的一个词。我希望你能习惯看到这两种叫法。另外，就像这一段中所说的，人们通常会简称其为高斯分布或正态分布——这两种都是高斯分布的常见简称。

高斯分布

让我们来了解一下高斯函数的工作原理。高斯函数只需两个参数就能完全描述：均值（ μ ）和方差（ σ^2 ）。其定义如下：

$$f(x, \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$$

$\exp[x]$ 其实指的是 e^x 。

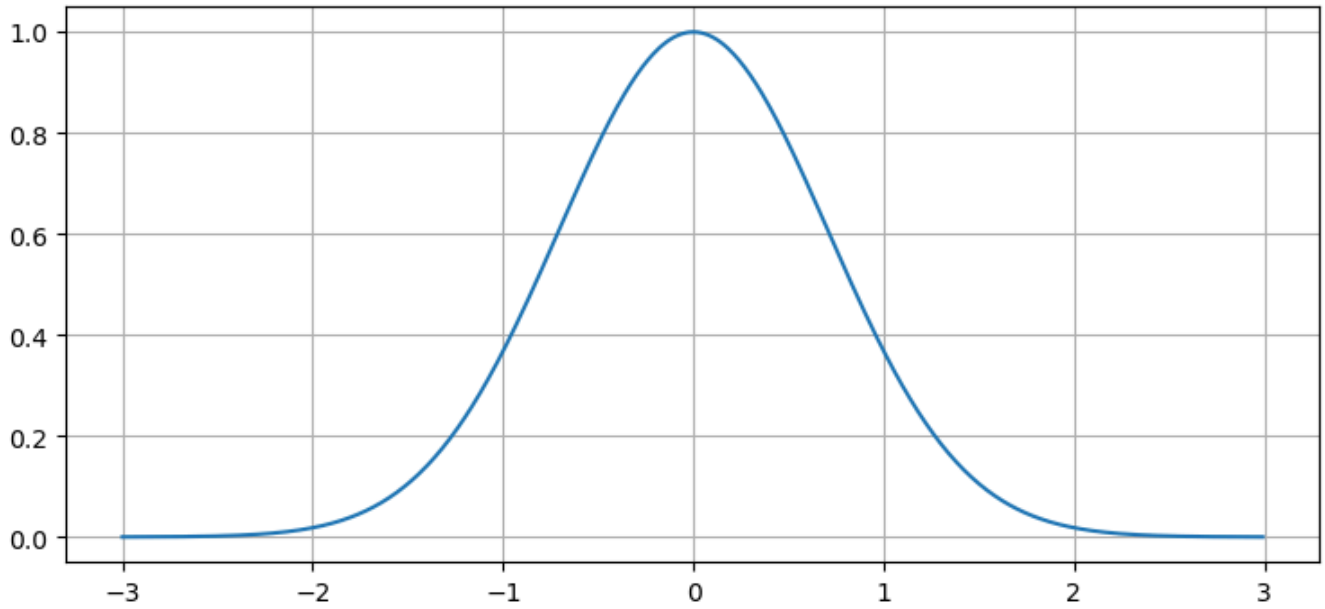
如果你之前没见过这个公式，不必因此而却步；你无需记住它或对其进行复杂的运算。该函数的计算过程已存储在`stats.py`文件中，函数名为`gaussian(x, mean, var, normed=True)`。

去掉那些常数后，就可以看出这其实是一个简单的指数函数而已：

$$f(x) \propto e^{-x^2}$$

呈现出人们所熟悉的钟形曲线形状。

```
x = np.arange(-3, 3, .01)
plt.plot(x, np.exp(-x**2));
```

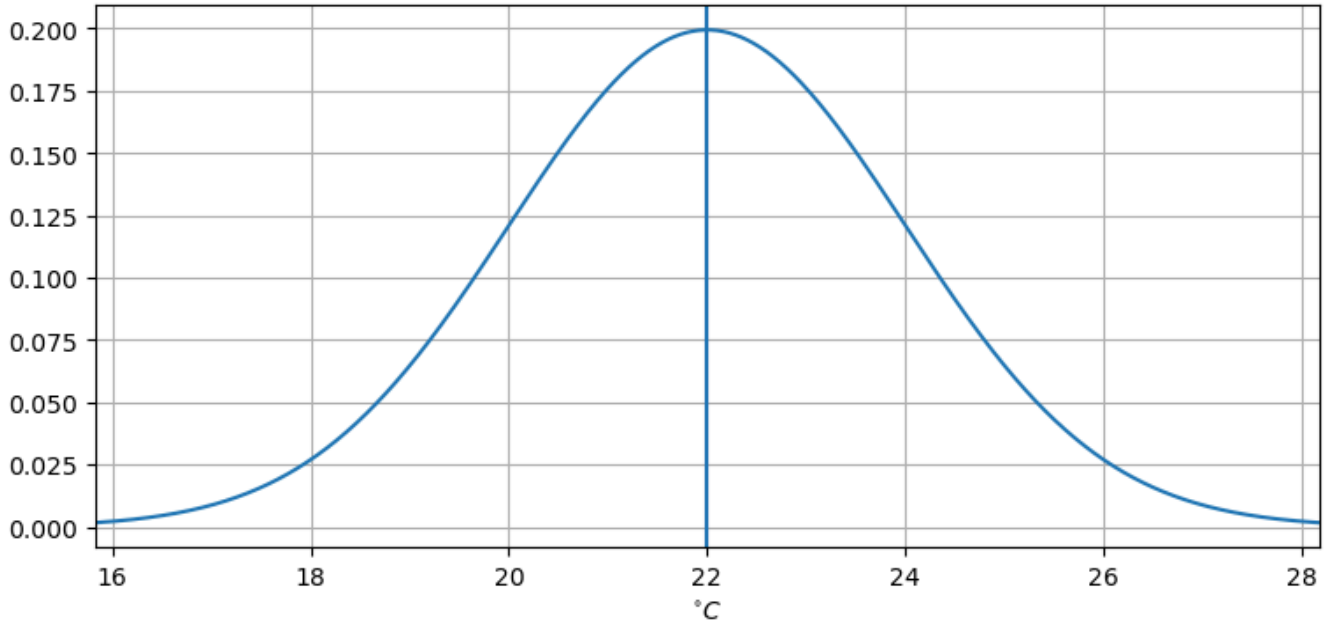


让我们回顾一下如何查看函数的代码。在某个单元格中，输入函数名称，后跟两个问号，然后按 CTRL+ENTER 键。这样就会弹出一个窗口，显示该函数的源代码。请取消下一个单元格中的注释，然后试着运行一下吧。

```
from filterpy.stats import gaussian
#gaussian??
```

让我们绘制一个均值为 22、方差为 4 的高斯分布曲线吧。

```
plot_gaussian_pdf(22, 4, mean_line=True, xlabel='$^{\circ}C$');
```



这条曲线代表什么意思呢？假设我们的温度计显示的温度为 22°C。不过，没有任何温度计能够做到完全精确的测量，因此每次测量得到的数值都会与实际数值略有偏差。但是，根据中心极限定理，如果我们进行多次测量，那么这些测量结果会呈现出正态分布。从这张图表可以看出，该曲线的形状反映了在实际温度为 22°C 的情况下，温度计显示特定数值的概率大小。

请记住，高斯分布是一种连续型分布。想象一下一条无限长的直线：如果你随机选择一个点，那么该点恰好位于 2 这个数值上的概率是多少呢？显然为 0%，因为有无数个可能的数值可以选择。正态分布也是如此；在上图中，温度恰好为 2°C 的概率也是 0%，因为温度数值可以有无数种可能。

这条曲线究竟是什么呢？其实，它就是我们所说的概率密度函数。曲线下面积的大小，代表了相应数值出现的概率。例如，如果计算 20 到 22 之间曲线的面积，那么得到的结果就是温度值位于这两个数值之间的概率。

另一种理解方式是：岩石或海绵的密度究竟是什么？所谓密度，指的是在给定空间内所含物质的质量大小。岩石的密度较高，而海绵的密度则较低。因此，如果你想知道一块岩石的重量，但又没有秤的话，你可以先测量出它的体积，再乘以它的密度，这样就能得到它的质量了。实际上，大多数物体的密度都是不均匀的，所以需要将整个岩石内部的密度值进行积分，才能得到准确的重量。

$$M = \iiint_R p(x, y, z) dV$$

对于概率密度函数，我们也是用同样的方法来处理。如果你想知道温度是否在 20°C 到 21°C 之间，就需要将上述曲线在 20 到 21 这个区间内进行积分。众所周知，曲线的积分值代表了曲线下面积。由于这里涉及的是概率密度函数，因此其积分值就是相应的概率。

温度恰好为 22°C 的概率是多少呢？凭直觉来说，应该是 0。这些都是实数，所以，22°C 与 22.00000000000017°C 之间的差异其实微乎其微。从数学上讲，如果对 22 到 22 之间的区间进行积分，结果也是 0。

再想想那块岩石吧：岩石上某一个单独的点的重量是多少呢？一个微小的点当然没有重量。询问某个单独点的重量是没有意义的；同样，询问连续分布函数取某个特定值的概率也是没有意义的。这两者的答案显然都是

零。

实际上，我们的传感器并没有无限的精确度。因此，当显示为 22°C 时，实际上指的是一个范围，比如 22°C±0.1°C。我们可以通过计算 21.9°C 到 22.1°C 之间的概率来了解这个范围的概率大小。

我们可以从贝叶斯或频率主义的角度来思考这个问题。作为贝叶斯主义者，如果温度计显示的温度恰好为 22°C，那么我们的信念可以用这条曲线来描述：我们认为实际温度接近 22°C 的概率非常高，而认为实际温度接近 18°C 的概率则非常低。作为频率主义者，我们会说：如果我们对某个系统进行 10 亿次测量，每次测量的温度都恰好为 22°C，那么这些测量结果的直方图就会呈现出与这条曲线相似的形状。

如何计算概率或曲线下的面积呢？需要对高斯函数的方程进行积分。

$$\int_{x_0}^{x_1} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}(x-\mu)^2 / \sigma^2} dx$$

这被称为累积概率分布，通常简称为 cdf。

我编写了 `filterpy.stats.norm_cdf`，它可以帮你计算积分。例如，我们可以这样来计算：

```
from filterpy.stats import norm_cdf
print('Cumulative probability of range 21.5 to 22.5 is {:.2f}%'.format(
      norm_cdf((21.5, 22.5), 22,4)*100))
print('Cumulative probability of range 23.5 to 24.5 is {:.2f}%'.format(
      norm_cdf((23.5, 24.5), 22,4)*100))
```

```
Cumulative probability of range 21.5 to 22.5 is 19.74%
Cumulative probability of range 23.5 to 24.5 is 12.10%
```

“平均值”顾名思义，就是所有可能概率的平均值。由于曲线的对称性，该点也是曲线上最高的部分。温度计显示的温度为 22 摄氏度，因此我们就以 22 摄氏度作为平均值。

对于随机变量 X 而言，正态分布的表示方式为 $X \sim N(\mu, \sigma^2)$，其中 $\sim$ 表示“按照某种分布规律来分布”。这意味着，我可以将温度计的测量结果用这种方式来表示。

$$\text{temp} \sim N(22, 4)$$

这是一个极其重要的成果。高斯函数让我只需用两个数值就能描述无数种可能的数值情况！只要知道 $\mu = 22$ 和 $\sigma^2 = 4$ 这两个数值，我就能计算出任意范围内各数值的分布情况。

有些资料使用 $N(\mu, \sigma)$ 来代替 $N(\mu, \sigma^2)$。两种方式都可以，都属于常见的惯例。如果你看到像 $N(22, 4)$ 这样的符号，就需要弄清楚到底使用的是哪种形式。在这本书中，我一直使用 $N(\mu, \sigma^2)$，所以在这个例子中，也使用 $\sigma = 2$ 和 $\sigma^2 = 4$。

方差与信念

由于这是一种概率密度分布，因此曲线下的面积必须始终等于 1。这一点很容易理解——曲线下的面积代表了所有可能的结果。既然某种事件必然会发生，那么其概率自然为 1。因此，所有概率密度的总和必须为 1。我们可以通过编写一些代码来验证这一点。（如果你擅长数学，可以将高斯函数从 $-\infty$ 积分到 ∞ 即可。）

```
print(norm_cdf((-1e8, 1e8), mu=0, var=4))
```

```
1.0
```

这引出了一个重要的结论：如果方差较小，那么曲线就会比较窄。这是因为方差反映了样本与平均值的偏离程度。为了使曲线的面积保持为 1，曲线的高度就必须相应地增加。相反，如果方差较大，曲线就会比较宽，为了使面积仍为 1，曲线的长度就必须相应地缩短。

让我们用图表来直观地看一下吧。我们将使用之前提到的 `filterpy.stats.gaussian`，它既可以接受单个数值，也可以接受数值数组作为输入。

```
from filterpy.stats import gaussian

print(gaussian(x=3.0, mean=2.0, var=1))
print(gaussian(x=[3.0, 2.0], mean=2.0, var=1))
```

```
0.24197072451914337
[0.378 0.622]
```

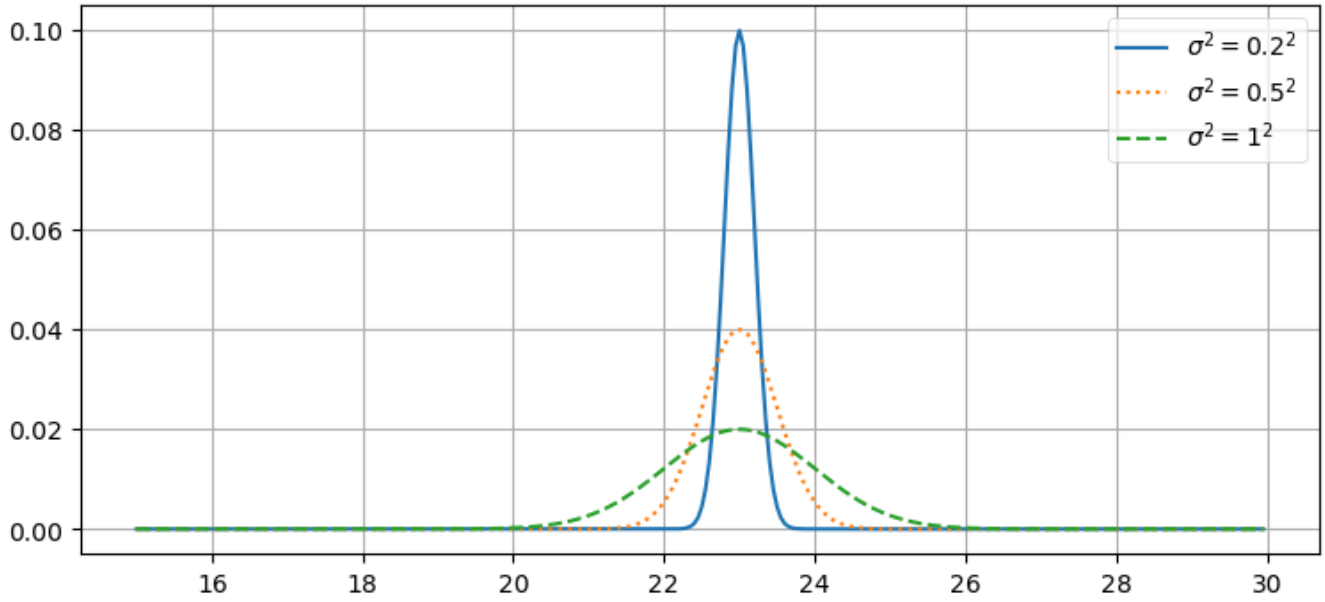
默认情况下， `gaussian` 会将对输出数据进行标准化处理，从而使其重新变为概率分布。可以使用参数 `normed` 来控制这一行为。

```
print(gaussian(x=[3.0, 2.0], mean=2.0, var=1, normed=False))
```

```
[0.242 0.399]
```

如果高斯函数没有经过归一化处理，那么它就被称为高斯函数，而不是高斯分布。

```
xs = np.arange(15, 30, 0.05)
plt.plot(xs, gaussian(xs, 23, 0.2**2), label='$\sigma^2=0.2^2$')
plt.plot(xs, gaussian(xs, 23, .5**2), label='$\sigma^2=0.5^2$', ls=':')
plt.plot(xs, gaussian(xs, 23, 1**2), label='$\sigma^2=1^2$', ls='--')
plt.legend();
```



这说明了什么？当参数为 $\sigma^2 = 0.2^2$ 时，高斯分布的宽度非常窄。这说明我们非常确信 $x = 23$ 这一结论，其置信度达到了 ± 0.2 标准差的水平。相比之下，当参数为 $\sigma^2 = 1^2$ 时，虽然我们也认为 $x = 23$ 是正确的，但我们的置信度要低得多。对于 $x = 23$，我们的置信度更低；因此，对于 x 的可能取值，我们的看法也比较模糊——比如，我们认为 $x = 20$ 或 $x = 26$ 更有可能成为 x 的值。至于 $\sigma^2 = 0.2^2$，它几乎不可能成为 x 的值。

如果我们回想一下温度计的原理，就可以把这三条曲线看作是三支不同温度计的测量结果。$\sigma^2 = 0.2^2$ 对应的曲线代表的是非常精确的温度计的测量结果；而 $\sigma^2 = 1^2$ 对应的曲线则代表的是精度较低的温度计的测量结果。需要注意的是，高斯分布有一个非常重要的优点：我们只需两个数值——均值和方差——就能完整地描述温度计的测量结果以及其误差。

高斯分布的等价表达形式为 $N(\mu, 1/\tau)$，其中 μ 代表均值，τ 代表精度。$1/\tau = \sigma^2$ 则是方差的倒数。虽然我们在本书中不使用这种表达方式，但它有助于我们理解：方差实际上反映了数据的精确程度。方差越小，精度越高——这意味着我们的测量结果非常精确。相反，方差越大，精度越低——这意味着我们的估计结果具有较大的不确定性。你应该习惯用这些等价形式来思考高斯分布。从贝叶斯的角度来看，高斯分布体现了我们对某次测量的置信程度，同时也反映了该次测量的精度以及测量结果中的不确定性。这些其实都是表达同一事实的不同方式而已。

我有点过于超前了，但在接下来的章节中，我们将利用高斯分布来描述我们对各种数值的估计，比如我们所追踪的物体的位置，以及我们所使用的传感器的精确度。

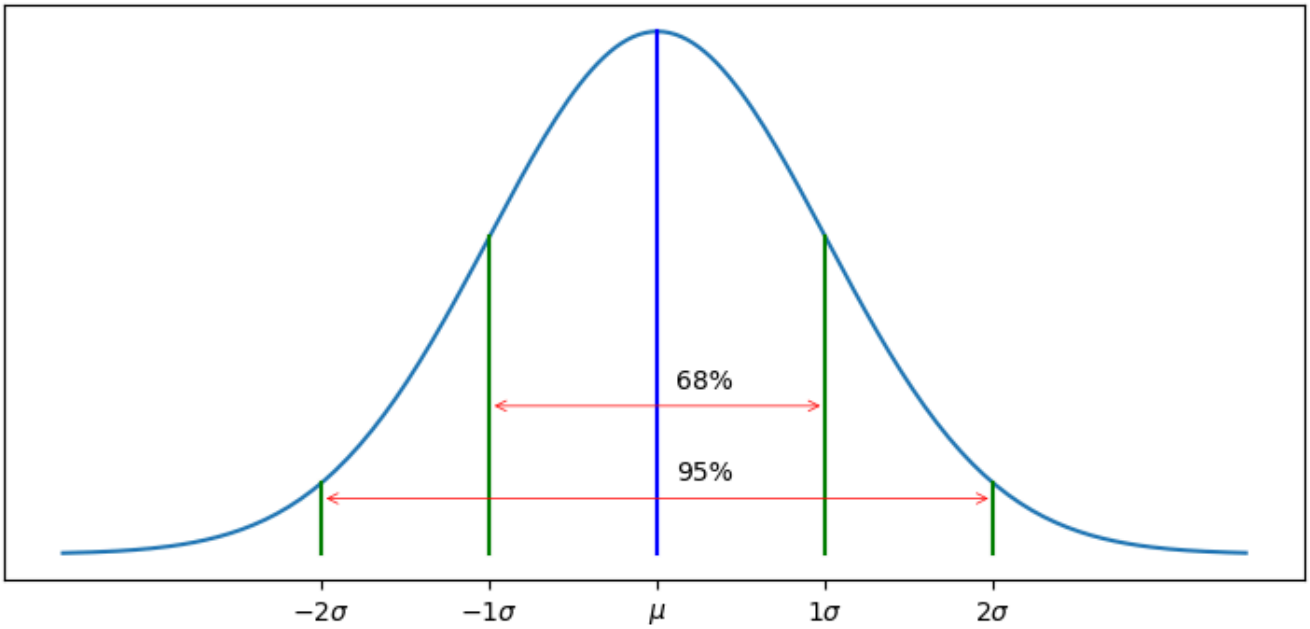
68-95-99.7 法则

现在，值得花些时间来谈谈标准差。标准差是一种衡量数据与平均值偏离程度的指标。对于高斯分布而言，68%的数据位于平均值的一个标准差范围内，95%的数据位于两个标准差范围内，99.7%的数据则位于三个标准差范围内。这一规律通常被称为“68-95-99.7 法则”。如果你得知某个班级的平均分数为 71 分，标准差为 9.4 分，那么可以推断：如果数据呈正态分布的话，95%的学生的分数应该介于 52.2 分到 89.8 分之间。

最后，这些数值并非随意确定的。如果表示位置的高斯分布的参数为 $\mu = 22$ 米，那么标准差的单位也是米。因此， $\sigma = 0.2$ 意味着 68%的测量结果位于 21.8 米到 22.2 米之间。方差则是标准差的平方，所以为 $\sigma^2 = .04$ 米的 ^{2} 倍。正如上一节所提到的，使用 $\sigma^2 = 0.2^2$ 来表示这个数值会更有意义，因为 0.2 这个数值与数据的单位一致。

下图展示了标准差与正态分布之间的关系。

```
from kf_book.gaussian_internal import display_stddev_plot
display_stddev_plot()
```



交互式高斯分布

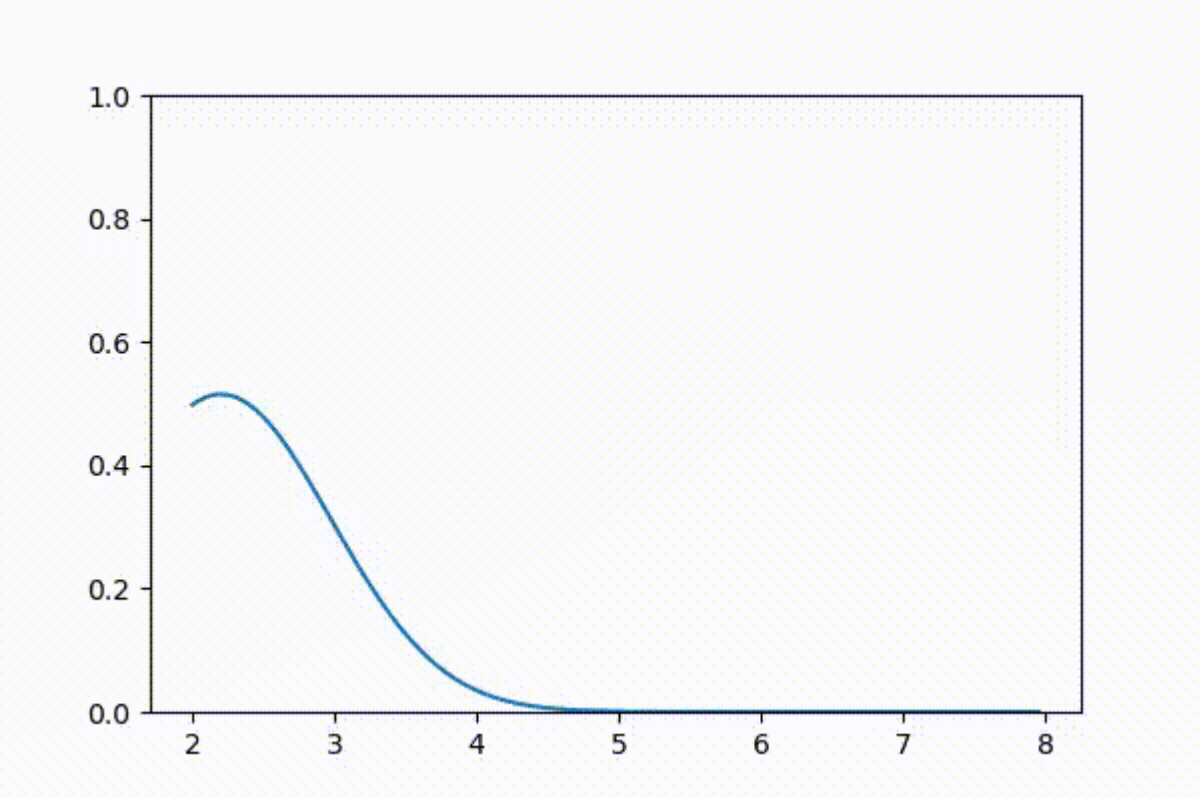
对于那些在 Jupyter Notebook 中阅读此内容的人士，这里有高斯曲线的交互式版本。可以使用滑块来调整 μ 和 σ^2 的值。调整 μ 会改变图形的左右位置，因为这相当于在调整平均值；而调整 σ^2 则会使高斯曲线的形状变得更粗或更细。

```
import math
from ipywidgets import interact, FloatSlider

def plt_g(mu,variance):
    plt.figure()
    xs = np.arange(2, 8, 0.01)
    ys = gaussian(xs, mu, variance)
    plt.plot(xs, ys)
    plt.ylim(0, 0.04)
    plt.show()

interact(plt_g, mu=FloatSlider(value=5, min=3, max=7),
         variance=FloatSlider(value = .03, min=.01, max=1.));
```

```
interactive(children=(FloatSlider(value=5.0, description='mu', max=7.0, min=3.0),
```

```
FloatSlider(value=0.03, desc…
```

最后，如果您是通过在线方式阅读这篇文章的，那么这里有一个高斯分布的动画演示。首先，将均值向右移动；接着，将均值重新置于 $\mu = 5$ 位置，并调整方差值。



正态分布随机变量的计算性质

离散贝叶斯滤波器通过将各种概率随机变量相乘和相加来运作。卡尔曼滤波器则使用高斯随机变量来替代各种随机变量，不过算法的其余部分则保持不变。这意味着我们需要对高斯随机变量进行相乘和相加的操作（高斯随机变量其实就是正态分布随机变量的另一种说法）。

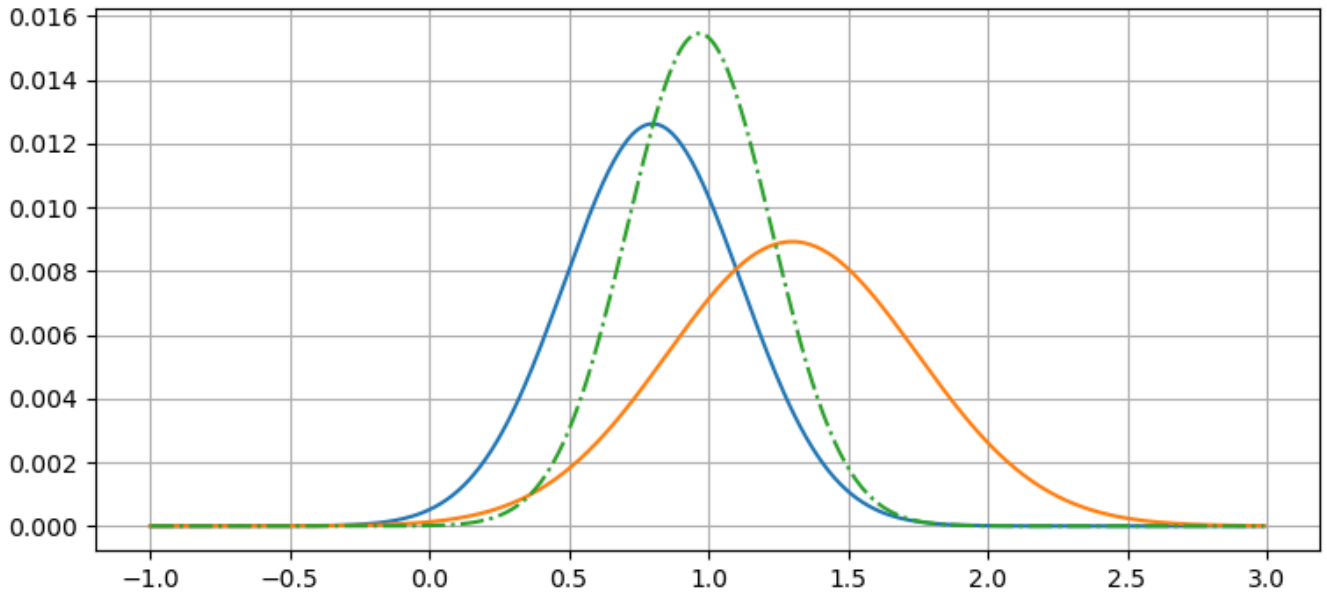
高斯随机变量有一个非常重要的性质：两个独立的高斯随机变量之和仍然服从正态分布！不过，两个高斯随机变量的乘积并不服从高斯分布，而是与高斯分布成正比。可以说，两个高斯分布相乘的结果仍然是一个高斯函数（需要注意的是，这里的“函数”指的是其数值之和为一这一性质并不一定成立）。

维基百科上有关于这个性质的详细介绍，我也在本章末尾对此进行了验证。https://en.wikipedia.org/wiki/Sum_of_normally_distributed_random_variables

在开始计算之前，我们先来从视觉上验证一下吧。

```
x = np.arange(-1, 3, 0.01)
g1 = gaussian(x, mean=0.8, var=.1)
g2 = gaussian(x, mean=1.3, var=.2)
plt.plot(x, g1, x, g2)

g = g1 * g2  # element-wise multiplication
```

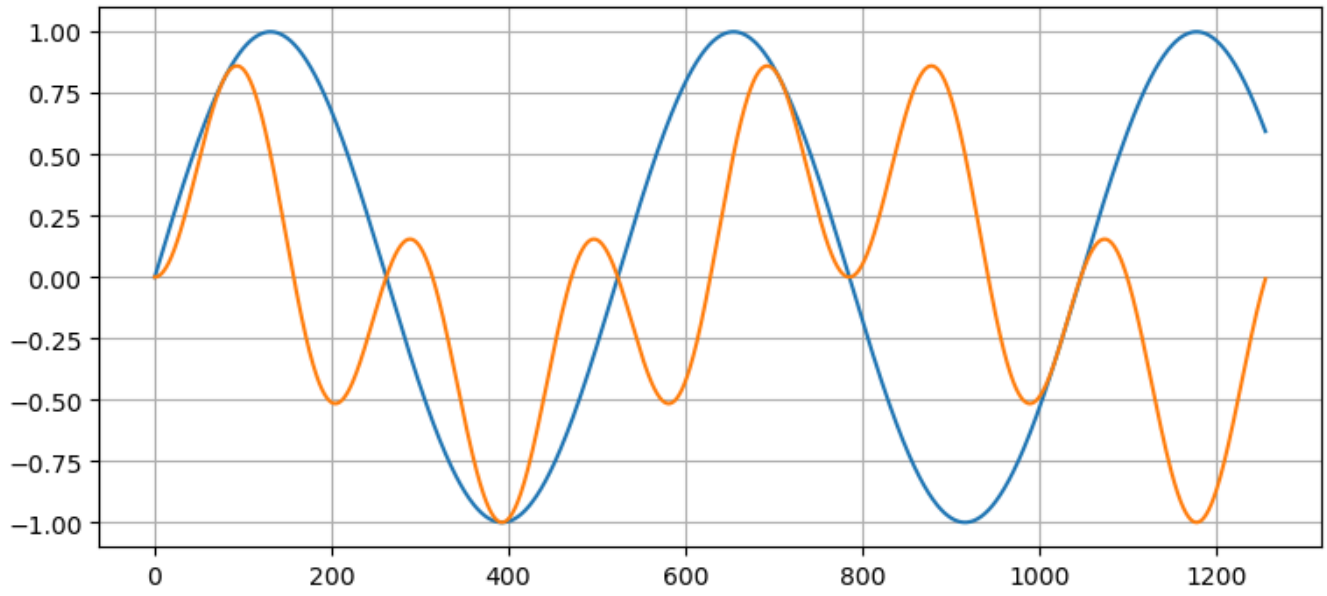
```
g = g / sum(g)  # normalize
plt.plot(x, g, ls='-.');
```



在这里，我创建了两个高斯函数，分别为 g1= $N(0.8, 0.1)$ 和 g2= $N(1.3, 0.2)$ ，然后把它们绘制出来。接着，我将这两个高斯函数相乘，再对结果进行归一化处理。如图所示，最终得到的结果呈现出高斯分布的形态。

高斯函数属于非线性函数。通常情况下，当对非线性方程进行相乘运算时，得到的结果会是另一种类型的函数。例如，两个正弦函数的乘积，其形状与 `sin(x)` 完全不同。

```
x = np.arange(0, 4*np.pi, 0.01)
plt.plot(np.sin(1.2*x))
plt.plot(np.sin(1.2*x) * np.sin(2*x));
```



但是，两个高斯分布相乘的结果仍然是一个高斯函数。这也是卡尔曼滤波器在计算上可行的关键原因。换句话说，卡尔曼滤波器之所以使用高斯分布，是因为高斯分布在计算上更为简便。

两个独立的高斯函数的乘积为：

$$\begin{aligned}\mu &= \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2} \\ \sigma^2 &= \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}\end{aligned}$$

两个高斯随机变量的和可以表示为

$$\begin{aligned}\mu &= \mu_1 + \mu_2 \\ \sigma^2 &= \sigma_1^2 + \sigma_2^2\end{aligned}$$

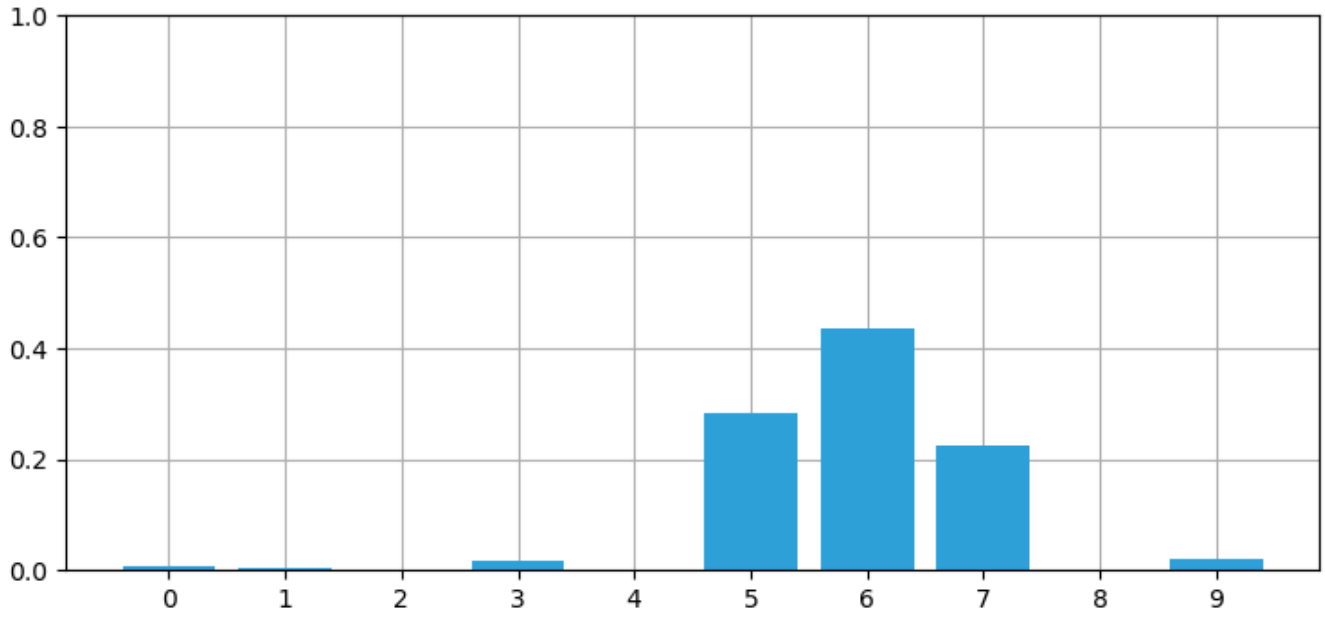
在本章的末尾，我推导出了这些方程式。不过，理解这些推导过程其实并不那么重要。

把所有东西整合在一起

现在，我们准备来探讨如何利用高斯函数来进行滤波处理。在下一章中，我们将实际实现一个基于高斯函数的滤波器。在此，我会先解释一下，为什么我们要使用高斯函数来进行滤波处理。

在上一章中，我们用数组来表示概率分布。为了更新概率分布，我们需要将该分布与另一个表示各点上测量结果可能性的分布进行逐元素相乘，具体操作如下：

```
def normalize(p):
    return p / sum(p)

def update(likelihood, prior):
    return normalize(likelihood * prior)

prior =      normalize(np.array([4, 2, 0, 7, 2, 12, 35, 20, 3, 2]))
likelihood = normalize(np.array([3, 4, 1, 4, 2, 38, 20, 18, 1, 16]))
posterior = update(likelihood, prior)
book_plots.bar_plot(posterior)
```

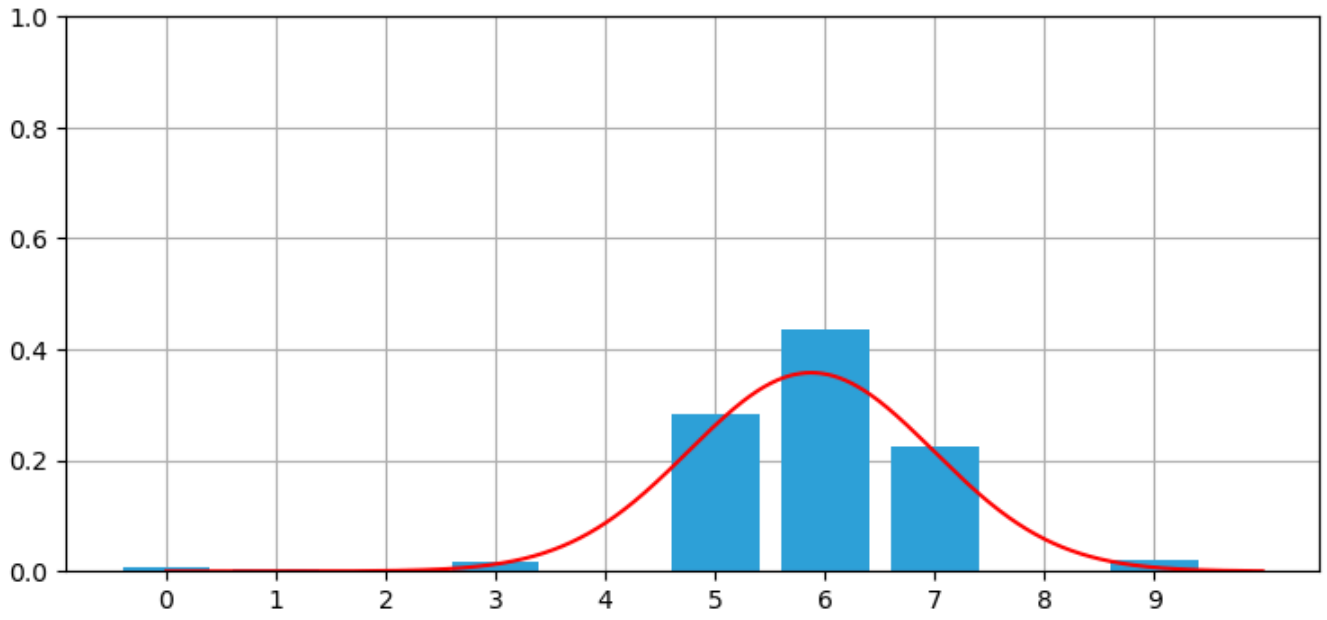


换句话说，为了得到这个结果，我们必须进行 10 次乘法运算。而对于那些具有多维大规模数组的实时滤波器来说，所需的乘法运算次数将达到数十亿次，同时还需要大量的内存来存储数据。

不过，这种分布看起来更像是高斯分布。如果我们用高斯分布来代替原来的数组结构会怎么样呢？我会计算后验分布的均值和方差，然后把结果与条形图一起展示出来。

```
xs = np.arange(0, 10, .01)

def mean_var(p):
    x = np.arange(len(p))
    mean = np.sum(p * x,dtype=float)
    var = np.sum((x - mean)**2 * p)
    return mean, var

mean, var = mean_var(posterior)
book_plots.bar_plot(posterior)
plt.plot(xs, gaussian(xs, mean, var, normed=False), c='r');
print('mean: %.2f' % mean, 'var: %.2f' % var)
```

```
mean: 5.88 var: 1.24
```



这真是令人印象深刻。我们只需两个数字就能描述整个数字分布情况。不过，由于这个例子中只有 10 个数字，因此可能不太具有说服力。但实际上，某个真实问题中可能包含数百万个数字，但我们仍然只需要两个数字来描述它。

接下来，请记住，我们的滤波器是通过以下方式来执行更新功能的：

```
def update(likelihood, prior):
    return normalize(likelihood * prior)
```

如果数组中包含一百万个元素，那就需要进行一百万次乘法运算。不过，如果我们用高斯函数来替代这些数组，那么所需的计算次数就会大大减少。

$$
\begin{aligned}
\mu &= \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2} \\
\sigma^2 &= \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}
\end{aligned}
$$

也就是三次乘法和两次除法。

贝叶斯定理

在上一章中，我们通过分析某一时刻所掌握的信息来构建算法，这些信息被表示为离散概率分布。在这个过程中，我们发现了贝叶斯定理。贝叶斯定理告诉我们，如何根据先验信息来计算某个事件的概率。

我们通过这种概率计算方式来实现 `update()` 函数：

$$
posterior = \frac{likelihood \times prior}{normalization}
$$

原来，这就是贝叶斯定理。稍后我会详细阐述其中的数学原理，但这么做反而会掩盖了这个公式所表达的简单含义。我们可以这样理解这个公式：

$$
updatedknowledge = \|likelihoodofnewknowledge \times priorknowledge\|
$$

其中，$\|\cdot\|$ 表示对该项进行归一化处理。

我们是通过关于“狗在走廊里行走”的简单推理得出了这个结论。不过，正如我们将会看到的，同样的原理也适用于各种滤波问题。在后面的每一章中，我们都会用到这个原理。

总结一下，先验概率是指在考虑测量结果之前，某件事发生的概率；而后验概率则是指在将测量结果所包含的信息纳入考量之后的概率。

贝叶斯定理是

$$
P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}
$$

$P(A \mid B)$ 被称为条件概率。也就是说，它表示在 B 发生的情况下， A 发生的概率。例如，如果昨天下过雨，那么今天下雨的概率就会比平常更高，因为降雨天气通常会持续多天。我们可以将“在昨天下雨的条件下，今天下雨的概率”表示为 P（今天下雨 | 昨天下雨）。

我忽略了一个重要点。在上面的代码中，我们处理的是概率值，而不是单个概率值。我刚才给出的贝叶斯公式中使用的是概率值，而非概率分布。不过，该公式同样也适用于概率分布的情况。在处理概率分布时，我们使用小写的 p 来表示。

$$
p(A \mid B) = \frac{p(B \mid A)p(A)}{p(B)}
$$

在上述公式中， B 代表证据， $p(A)$ 代表先验概率， $p(B \mid A)$ 代表似然值， $p(A \mid B)$ 则代表后验概率。将这些数学符号对应的含义代入后，可以看出贝叶斯定理其实与我们所说的“更新公式”是一致的。让我们用与问题

相关的术语来重新表述这个公式。我们用 x_i 表示某一点的位置，用 z 表示在该点的测量结果。因此，我们想要知道的是 $P(x_i \mid z)$ ，也就是在得到测量结果 z 的条件下，狗位于 x_i 的概率。

那么，让我们把这个值代入方程中，然后求解吧。

$$p(x_i \mid z) = \frac{p(z \mid x_i)p(x_i)}{p(z)}$$

看起来有点复杂，但实际上相当简单。让我们先弄清楚右边各项的含义。首先是 $p(z \mid x_i)$ 。它表示每个单元格 x_i 的测量结果所对应的概率或可能性。 $p(x_i)$ 则代表先验概率——即在考虑各项测量结果之前的初始判断。我们将这两者相乘。这其实就是 `update()` 函数中所进行的那种未经归一化的乘法运算而已。

```
def update(likelihood, prior):
    posterior = prior * likelihood   # p(z|x) * p(x)
    return normalize(posterior)
```

最后一个需要考虑的项是分母 $p(z)$ 。它表示在不考虑位置信息的情况下，得到测量结果 z 的概率。这一数值通常被称为“归一化值”。我们通过计算 x 的总和来得到这个值。这就是我们进行归一化处理的原理！因此，该函数实际上只是在运用贝叶斯定理进行计算而已。

在文献中，这些方程通常以积分的形式呈现。毕竟，积分本质上就是对连续函数进行求和。因此，你可能会看到贝叶斯定理被写成这样的形式：

$$p(A \mid B) = \frac{p(B \mid A)p(A)}{\int p(B \mid A_j)p(A_j)dA_j}.$$

这个分母通常无法通过解析方法来求解；即便能够求解，其数学运算也极其复杂。最近，英国皇家统计学会的一篇评论文章将这个问题称为“极其棘手的难题”[8]。那些采用贝叶斯方法的教材中，到处都是无法用解析方法求解的积分方程。不必被这些方程吓倒，我们只需对后验概率进行归一化处理，就能轻松解决这个问题。在“粒子滤波”这一章中，我们将学习更多处理这类问题的方法。在此之前，请记住：在实际应用中，这些积分只不过是可以通过求和来处理的项而已。我的意思是，当面对满页的积分公式时，把它们看作求和运算即可，这样就能理解其含义了。通常，问题的答案其实都很简单。令人惊讶的是，作者常常忽略提及这种解读。

很可能，你还没有完全理解贝叶斯定理的强大之处。我们想要计算的是 $p(x_i \mid Z)$ 。也就是说，在第 i 步时，根据现有的测量结果，我们处于某种状态的概率是多少。一般来说，这是一个极其困难的问题。不过，贝叶斯定理具有普遍性。我们可能想知道：根据癌症检测的结果，患癌症的概率是多少；或者根据各种传感器的数据，下雨的概率又是多少。乍一看，这些问题似乎无法解决。

但贝叶斯定理允许我们利用其逆运算来计算这个值，而逆运算通常比较容易计算。

$$p(x_i \mid Z) \propto p(Z \mid x_i)p(x_i)$$

也就是说，要想知道在给定特定传感器数据的情况下下雨的概率，我们只需计算在确实下雨的条件下，那些传感器数据出现的概率即可！这显然容易得多。当然，天气预报仍然是个难题，但贝叶斯定理让这个问题变得可解决了。

同样地，正如你在“离散贝叶斯”那一章中所看到的，我们是通过计算：当西蒙位于位置 x 时，各个传感器所检测到的数值出现的概率，从而来确定西蒙处于走廊上某个特定位置的概率。这样一来，原本复杂的问题就变

得简单了。

全概率定理

我们现在已经知道了 `update()` 函数背后的数学原理；那么 `predict()` 函数又是什么呢？ `predict()` 函数实际上实现了全概率定理。让我们回顾一下 `predict()` 函数的计算过程。 `predict()` 函数计算的是，在给定所有可能的移动情况的概率之后，处于某个特定位置的概率。我们可以用公式来表示这一点。在时间 t 时，处于位置 i 的概率可以表示为 $P(X_i^t)$ 。我们通过将时间 $t-1$ 时的先验概率 $P(X_j^{t-1})$ 与从单元格 x_j 移动到 x_i 的概率相乘，从而得到了这个结果。也就是说

$$P(X_i^t) = \sum_j P(X_j^{t-1})P(x_i \mid x_j)$$

那个公式被称为“全概率定理”。引用维基百科[6]的说法：“它描述了可以通过多个不同事件来实现的结果的总概率。”我本可以直接把那个公式告诉你，并演示其计算过程，但你理解该公式为何有效的概率其实很低。作为提醒，以下是用于计算该公式的代码。

```
for i in range(N):
    for k in range (kN):
        index = (i + (width-k) - offset) % N
        result[i] += prob_dist[index] * kernel[k]
```

使用 scipy.stats 计算概率

在这一章中，我使用了 FilterPy 中的代码来计算和绘制高斯分布曲线。这么做是为了让你们有机会了解这些代码的实现方式。不过，正如那句话所说，Python 本身就提供了许多内置的统计功能，这些功能都包含在 `scipy.stats` 模块中。那么，我们就来了解一下如何使用 scipy.stats 来计算各种统计量和概率吧。

`scipy.stats` 模块包含了许多可用于计算各种概率分布特性的函数。该模块的完整文档请参见：http://docs.scipy.org/doc/scipy/reference/stats.html。我们重点关注 norm 这个函数，它用于实现正态分布的计算。下面来看一段使用 `scipy.stats.norm` 来计算高斯分布的代码，并将其结果与 FilterPy 中的 `gaussian()` 函数所返回的结果进行比较。

```
from scipy.stats import norm
import filterpy.stats
print(norm(2, 3).pdf(1.5))
print(filterpy.stats.gaussian(x=1.5, mean=2, var=3*3))
```

```
0.13114657203397997
0.13114657203397995
```

`norm(2, 3)` 这个调用会创建出 scipy 中所说的“固定”分布。它会生成一个均值为 2、标准差为 3 的对象。之后，你可以多次使用这个对象来获取不同数值对应的概率密度，具体操作如下：

```
n23 = norm(2, 3)
print('pdf of 1.5 is       %.4f' % n23.pdf(1.5))
```

```
print('pdf of 2.5 is also  %.4f' % n23.pdf(2.5))
print('pdf of 2 is         %.4f' % n23.pdf(2))
```

```
pdf of 1.5 is       0.1311
pdf of 2.5 is also  0.1311
pdf of 2 is         0.1330
```

[2]中的文档还列出了许多其他功能。例如，我们可以利用该函数从该分布中生成 n 个样本。

```
np.set_printoptions(precision=3, linewidth=50)
print(n23.rvs(size=15))
```

```
[ 2.269  2.485 -0.469  1.991  2.468  1.667  1.231
  4.577  2.56   0.224 -3.851  2.604  3.583 -1.623
  2.348]
```

我们可以得到这个概率：即从该分布中随机抽取的一个数值小于或等于 x 的概率。

```
# probability that a random value is less than the mean 2
print(n23.cdf(2))
```

```
0.5
```

我们可以了解该分布的多种特性：

```
print('variance is', n23.var())
print('standard deviation is', n23.std())
print('mean is', n23.mean())
```

```
variance is 9.0
standard deviation is 3.0
mean is 2.0
```

用高斯函数来模拟世界的局限性

早些时候我提到了中心极限定理。该定理指出，在特定条件下，任何独立随机变量的算术和都呈正态分布，而与这些随机变量的具体分布方式无关。这一点非常重要，因为自然界中有很多随机变量的分布并不遵循正态分布。但是，当我们对大量随机变量应用中心极限定理时，最终得到的结果都是正态分布。

不过，这一证明的关键前提是“在某些条件下”。而这些条件在现实世界中往往不成立。例如，厨房秤的刻度不可能低于零。但如果我们将测量误差视为高斯分布，那么曲线的左侧会延伸到负无穷大，这意味着出现负数值的概率极低。

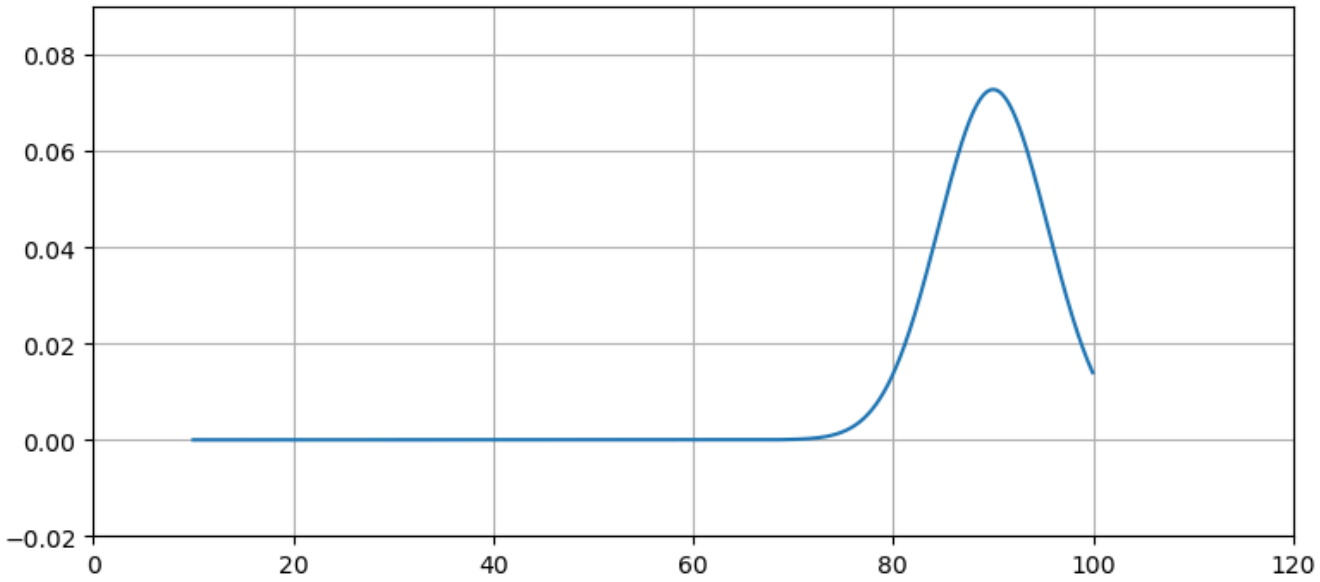
这是一个范围很广的主题，我无法面面俱到地加以阐述。

让我们来看一个简单的例子。我们通常认为，像考试成绩这样的数值是呈正态分布的。如果你曾经有过老师按照某种标准来给成绩打分的经历，那你应该也了解这种假设。但实际上，考试成绩不可能遵循正态分布。因为

正态分布意味着，无论某个数值与平均值的偏离程度如何，该数值被选中的概率都是非零的。例如，假设平均分为 90 分，标准差为 13 分。根据正态分布的假设，有人得到 90 分的概率很大，而得到 40 分的概率则很小。不过，这也意味着，有人得到-10 分或 150 分的概率极低。实际上，得到 0 分或 100 分的概率也微乎其微。高斯分布的“尾部”是无限长的。

但在测试中，情况并非如此。如果不考虑额外加分的话，成绩不可能低于 0 分，也不可能高于 100 分。让我们用正态分布来描绘这个数值范围，从而看看这种模型在描述实际测试成绩分布时的准确性有多差。

```
xs = np.arange(10, 100, 0.05)
ys = [gaussian(x, 90, 30) for x in xs]
plt.plot(xs, ys, label='var=0.2')
plt.xlim(0, 120)
plt.ylim(-0.02, 0.09);
```



曲线下的面积不可能等于 1，因此这并不是一种概率分布。实际上，比正态分布所预测的，有更多的学生取得了接近分数区间的上限的分数。因此，曲线的“尾部”会变得比较宽。此外，该测试很可能无法准确区分学生之间细微的技能差异，所以，位于平均值左侧的分数分布也可能会有些许的聚集现象。

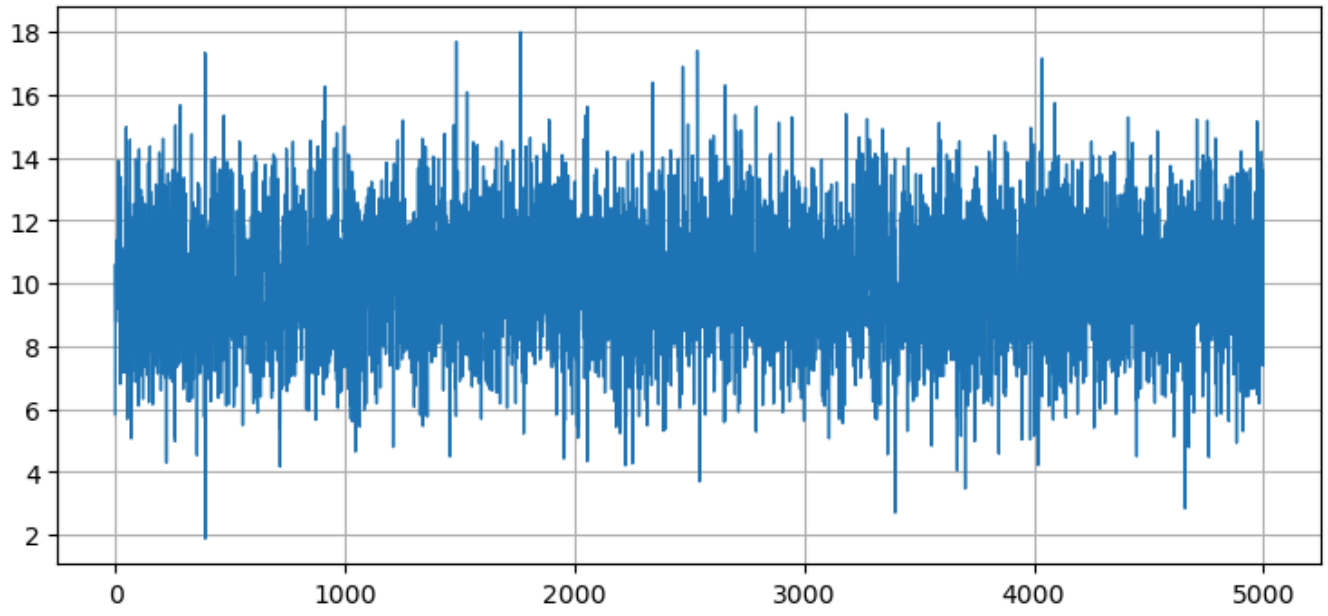
传感器用于测量世界各处的信息。不过，传感器的测量结果往往并不符合高斯分布。现在就讨论这对卡尔曼滤波器的设计者来说意味着什么还为时过早。需要记住的是，卡尔曼滤波器的数学模型其实是基于对世界的理想化描述而建立的。接下来，我会展示一些代码。在本书的后面部分，我会利用这些代码来生成各种分布，从而模拟不同的过程和传感器的行为。这种分布被称为“学生 t 分布”。

假设我想要对一个输出中包含白噪声的传感器进行建模。为简化问题，假设该信号的值为恒定的 10，而噪声的标准差为 2。我们可以使用 `numpy.random.randn()` 函数来生成均值为 0、标准差为 1 的随机数。我可以通过以下方式来模拟这一过程：

```
from numpy.random import randn
def sense():
    return 10 + randn()*2
```

让我们把那个信号画出来，看看它到底是什么样子的。

```
zs = [sense() for i in range(5000)]
plt.plot(zs, lw=1);
```

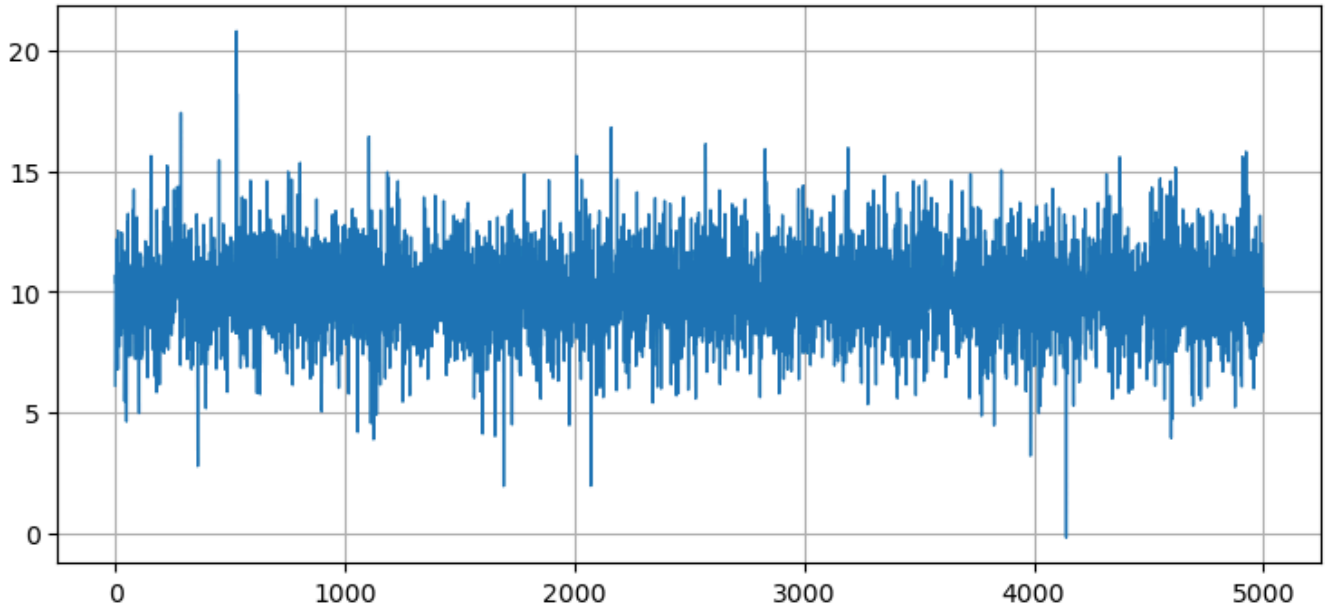


这正是我所预期的结果。信号值大致集中在 10 左右。标准差为 2 意味着，68%的测量值会落在 10 的±2 范围内，99%的测量值会落在 10 的±6 范围内。实际情况确实如此。

现在，让我们来看看用学生 t 分布生成的分布曲线。我不会详细解释其中的数学原理，只会把相关源代码提供给大家，然后利用这些代码来绘制出分布曲线。

```
import random
import math

def rand_student_t(df, mu=0, std=1):
    """return random number distributed by Student's t
    distribution with `df` degrees of freedom with the
    specified mean and standard deviation.
    """
    x = random.gauss(0, std)
    y = 2.0*random.gammavariate(0.5*df, 2.0)
    return x / (math.sqrt(y / df)) + mu
```

```
def sense_t():
    return 10 + rand_student_t(7)*2

zs = [sense_t() for i in range(5000)]
plt.plot(zs, lw=1);
```



从图表中可以看出，虽然其分布形态与正态分布类似，但存在一些异常值，这些异常值的数值远远超出了平均值的 3 个标准差范围（即 7 到 13 之间）。

学生的 t 分布不太可能准确反映你的传感器（比如 GPS 或多普勒传感器）的实际性能。不过，这并非一本关于如何对物理系统进行建模的书籍。不过，当面对现实世界中的噪声时，这种分布方式仍能提供有用的数据，帮助我们测试滤波器的性能。在本书的后续内容中，我们将在各种模拟和测试中继续使用这类分布方式。

这并非毫无根据的担忧。卡尔曼滤波方程假设噪声呈正态分布，如果这一假设不成立，滤波效果就会大打折扣。对于那些对任务成败至关重要的滤波器来说，比如航天器上的滤波器，设计人员必须充分了解有关传感器性能的各种理论和实际数据。例如，我在 NASA 的一个项目报告中看到，虽然从理论上讲，应该使用 3 个标准差来区分噪声和有效测量值，但实际上他们不得不使用 5 到 6 个标准差才能达到理想效果。这是通过实验得出的结论。

rand_student_t 的代码包含在 `filterpy.stats` 中。您可以将其与……一起使用。

```
from filterpy.stats import rand_student_t
```

虽然这里不详细讨论，但统计学中有一些方法可以用来描述概率分布的形态，即概率分布与指数分布的差异。正态分布是以均值为中心的对称形状——就像钟形曲线一样。不过，概率分布也可能在均值周围呈现不对称性。衡量这种不对称性的指标被称为“偏度”。此外，概率分布的尾部可能比指数分布更短、更粗、更细，或者具有完全不同的形状。衡量这种形状的指标则被称为“峰度”。 `scipy.stats` 模块中包含 `describe` 函数，该函数可用于计算这些统计指标。

```
import scipy
scipy.stats.describe(zs)
```

```
DescribeResult(nobs=5000, minmax=(-0.19143741858944452, 20.804030812682086), mean=9.996901102331591,
variance=2.7388116003856884, skewness=0.015276194207011526, kurtosis=1.622567142788295)
```

让我们来考察两个正态分布的总体，一个规模较小，另一个规模较大：

```
print(scipy.stats.describe(np.random.randn(10)))
print()
print(scipy.stats.describe(np.random.randn(300000)))
```

```
DescribeResult(nobs=10, minmax=(-1.2037202953168122, 0.894366248899801), mean=-0.18007426896541692,
variance=0.43204122396032946, skewness=-0.08295117992362264, kurtosis=-0.9197042651911951)

DescribeResult(nobs=300000, minmax=(-4.75206637131407, 4.178833173976851),
mean=-0.00047436810878343096, variance=0.9973760542573228, skewness=-0.0038868831924059035,
kurtosis=-0.005797621180390955)
```

由于样本数量较少，样本数据的分布并不均匀，因此其偏度和峰度都远非零。通过将计算出的均值和方差与理论上的均值 0 和方差 1 进行比较，也可以看出这一点。相比之下，大样本的均值和方差则非常接近理论值，同时其偏度和峰度也接近于零。

高斯函数的乘积（可选）

阅读这一部分并不重要。在这里，我推导了两个高斯函数相乘时的公式。

可以通过将两个高斯函数的方程相乘并合并各项来得到这个结果。不过，这样的代数运算相当复杂。我会用贝叶斯定理来推导这个结果。我们可以这样表述这个问题：设先验概率为 $N(\bar{\mu}, \bar{\sigma}^2)$ ，测量结果为 $z \propto N(z, \sigma_z^2)$ 。那么，在给定测量结果 z 的情况下，后验概率 x 是多少呢?

将后验概率表示为 $p(x \mid z)$ 。现在，我们可以利用贝叶斯定理来进一步推导了。

$$p(x \mid z) = \frac{p(z \mid x)p(x)}{p(z)}$$

$p(z)$ 是一个用于标准化的常数，因此我们可以据此建立相应的比例关系。

$$p(x \mid z) \propto p(z \mid x)p(x)$$

现在，我们将高斯函数的方程代入其中，这些方程如下：

$$p(z \mid x) = \frac{1}{\sqrt{2\pi\sigma_z^2}} \exp\Big[-\frac{(z-x)^2}{2\sigma_z^2}\Big]$$

$$p(x) = \frac{1}{\sqrt{2\pi\bar{\sigma}^2}} \exp\Big[-\frac{(x-\bar{\mu})^2}{2\bar{\sigma}^2}\Big]$$

我们可以忽略那些常数项，这样就能得到：

$$\begin{aligned} p(x \mid z) &\propto \exp\Big[-\frac{(z-x)^2}{2\sigma_z^2}\Big] \exp\Big[-\frac{(x-\bar{\mu})^2}{2\bar{\sigma}^2}\Big] \\ &\propto \exp\Big[-\frac{(z-x)^2}{2\sigma_z^2} - \frac{(x-\bar{\mu})^2}{2\bar{\sigma}^2}\Big] \\ &\propto \exp\Big[-\frac{1}{2\sigma_z^2\bar{\sigma}^2}[\bar{\sigma}^2(z-x)^2 + \sigma_z^2(x-\bar{\mu})^2]\Big] \end{aligned}$$

现在，我们将平方项展开，然后按照后验概率 x 来对各项进行分组。

$$
\begin{aligned}
p(x \mid z) \quad &\propto \exp\Big[-\frac{1}{2\sigma_z^2\bar{\sigma}^2}[\bar{\sigma}^2(z^2-2xz+x^2)+\sigma_z^2(x^2-2x\bar{\mu}+\bar{\mu}^2)]\Big] \\
&\propto \exp\Big[-\frac{1}{2\sigma_z^2\bar{\sigma}^2}[x^2(\bar{\sigma}^2+\sigma_z^2)-2x(\sigma_z^2\bar{\mu}+\bar{\sigma}^2 z)+(\bar{\sigma}^2 z^2+\sigma_z^2\bar{\mu}^2)]\Big]
\end{aligned}
$$

最后一个括号内不包含后验概率 x，因此可以将其视为常数而忽略不计。

$$
p(x \mid z) \propto \exp\Big[-\frac{1}{2}\frac{x^2(\bar{\sigma}^2+\sigma_z^2)-2x(\sigma_z^2\bar{\mu}+\bar{\sigma}^2 z)}{\sigma_z^2\bar{\sigma}^2}\Big]
$$

将分子和分母同时除以 $\bar{\sigma}^2+\sigma_z^2$，即可得到结果。

$$
p(x \mid z) \propto \exp\Big[-\frac{1}{2}\frac{x^2-2x(\frac{\sigma_z^2\bar{\mu}+\bar{\sigma}^2 z}{\bar{\sigma}^2+\sigma_z^2})}{\frac{\sigma_z^2\bar{\sigma}^2}{\bar{\sigma}^2+\sigma_z^2}}\Big]
$$

比例性让我们可以随心所欲地创建或删除各种常量，因此我们可以将这一点纳入考虑范围之内。

$$
p(x \mid z) \propto \exp\Big[-\frac{1}{2}\frac{(x-\frac{\sigma_z^2\bar{\mu}+\bar{\sigma}^2 z}{\bar{\sigma}^2+\sigma_z^2})^2}{\frac{\sigma_z^2\bar{\sigma}^2}{\bar{\sigma}^2+\sigma_z^2}}\Big]
$$

高斯分布指的是……

$$
N(\mu,\sigma^2) \propto \exp\Big[-\frac{1}{2}\frac{(x-\mu)^2}{\sigma^2}\Big]
$$

由此可见， $p(x \mid z)$ 的平均值为

$$
\mu_{posterior} = \frac{\sigma_z^2\bar{\mu}+\bar{\sigma}^2 z}{\bar{\sigma}^2+\sigma_z^2}
$$

以及方差为

我省略了那些常数项，因此得到的结果并非正态分布，而是与 1 成正比。贝叶斯定理通过使用 $p(z)$ 作为除数来实现结果的标准化，从而确保了结果呈正态分布。在滤波器的更新过程中，我们也会进行标准化处理，以确保滤波器的估计值符合高斯分布。

$$
N_1 = \|N_2 \cdot N_3\|
$$

独立高斯随机变量的和（可选）

同样，这一部分也没什么必要仔细阅读。在这里，我推导了两个独立的高斯随机变量之和的公式。

该数值由以下方式得出：

$$
\begin{aligned}
\mu &= \mu_1 + \mu_2 \\
\sigma^2 &= \sigma_1^2 + \sigma_2^2
\end{aligned}
$$

关于这一点，有几种证明方法。由于在上一章中，我们已经在概率直方图的计算中使用了卷积运算，因此在这里我也会继续使用卷积运算来证明这一点。

要找到两个高斯随机变量之和的密度函数，我们需要将这两个随机变量的密度函数相加。由于这些函数都是非线性的连续函数，因此必须通过积分来计算它们的和。如果随机变量 p 和 z 是独立的（比如，先验概率和测量结果），那么我们可以直接计算它们的和。

$$p(x) = \int_{-\infty}^{\infty} f_p(x-z) f_z(z) dx$$

这就是卷积的运算公式。现在，我们只需进行一些数学计算即可：

$$\begin{aligned}
p(x) &= \int_{-\infty}^{\infty} f_2(x-x_1) f_1(x_1) dx \\
&= \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}\,\sigma_z} \exp\left[-\frac{(x-z-\mu_z)^2}{2\sigma_z^2}\right] \frac{1}{\sqrt{2\pi}\,\sigma_p} \exp\left[-\frac{(x-\mu_p)^2}{2\sigma_p^2}\right] dx \\
&= \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}\sqrt{\sigma_p^2+\sigma_z^2}} \exp\left[-\frac{(x-(\mu_p+\mu_z)))^2}{2(\sigma_z^2+\sigma_p^2)}\right] \frac{1}{\sqrt{2\pi}\frac{\sigma_p\sigma_z}{\sqrt{\sigma_p^2+\sigma_z^2}}} \exp\left[-\frac{(x-\frac{\sigma_p^2(x-\mu_z)+\sigma_z^2\mu_p}{\sigma_p^2+\sigma_z^2}))^2}{2\left(\frac{\sigma_p\sigma_x}{\sqrt{\sigma_z^2+\sigma_p^2}}\right)^2}\right] dx \\
&= \frac{1}{\sqrt{2\pi}\sqrt{\sigma_p^2+\sigma_z^2}} \exp\left[-\frac{(x-(\mu_p+\mu_z)))^2}{2(\sigma_z^2+\sigma_p^2)}\right] \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}\frac{\sigma_p\sigma_z}{\sqrt{\sigma_p^2+\sigma_z^2}}} \exp\left[-\frac{(x-\frac{\sigma_p^2(x-\mu_z)+\sigma_z^2\mu_p}{\sigma_p^2+\sigma_z^2}))^2}{2\left(\frac{\sigma_p\sigma_x}{\sqrt{\sigma_z^2+\sigma_p^2}}\right)^2}\right] dx
\end{aligned}$$

积分内的表达式属于正态分布。正态分布的总和为一，因此该积分的值也为一。这就得到了我们想要的结果。

$$p(x) = \frac{1}{\sqrt{2\pi}\sqrt{\sigma_p^2+\sigma_z^2}} \exp\left[-\frac{(x-(\mu_p+\mu_z)))^2}{2(\sigma_z^2+\sigma_p^2)}\right]$$

这表现为一种正常状态，即……

$$\begin{aligned}
\mu_x &= \mu_p + \mu_z \\
\sigma_x^2 &= \sigma_z^2 + \sigma_p^2 \blacksquare
\end{aligned}$$

总结与要点

这一章对统计学的介绍相当不够全面。我所讲述的内容仅仅是为了在本书的后续部分中使用高斯分布而必需的概念而已。如果你打算深入研究卡尔曼滤波的相关知识，那么我所讲的内容远远不够用。如果你是第一次接触统计学，我建议你阅读专门的统计学教材。我个人很喜欢 Schaum 系列的教材，Alan Downey 所著的《Think Stats》[5]也是一本非常好的书，而且可以在网上免费阅读。

在继续之前，您必须先理解以下几点：

- 正态分布体现了连续的概率分布特性。
- 它们完全由两个参数来描述：均值（ μ ）和方差（ σ^2 ）。
- μ 是所有可能数值的平均值。

- 方差 σ^2 反映了我们的测量结果与平均值的偏离程度。
- 标准差（ σ ）是方差（ σ^2 ）的平方根。
- 自然界中的许多现象都近似于正态分布，但这种数学模型并不完美。
- 在滤波问题中，计算 $p(x \mid z)$ 几乎是不可能的，而计算 $p(z \mid x)$ 则相对简单。贝叶斯定理让我们能够利用 $p(z \mid x)$ 来计算 $p(x \mid z)$ 。

在接下来的几章中，我们将利用高斯分布和贝叶斯定理来帮助进行数据滤波。正如上一节所提到的，有时候高斯分布并不能很好地描述现实世界的情况。本书的后半部分将专门介绍那些即使在噪声或系统行为极不符合高斯分布的情况下也能有效工作的滤波算法。

参考文献

[1] https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/Supporting_Notebooks/Computing_and_plotting_PDFs.ipynb

[2] http://docs.scipy.org.cn/doc/scipy/reference/generated/scipy.stats.norm.html

[3] http://docs.scipy.org.cn/doc/scipy/reference/tutorial/stats.html

[4] 休伯，彼得·J.《稳健统计方法》，第二版。工业与应用数学学会，1996 年。

[5] 阿兰·多尼著。《Think Stats》第二版。O’Reilly Media 出版社。

https://github.com/AllenDowney/ThinkStats2

http://greenteapress.com/thinkstats/

有用的维基百科链接

https://en.wikipedia.org/wiki/Probability_distribution

https://en.wikipedia.org/wiki/Random_variable

https://en.wikipedia.org/wiki/Sample_space

https://en.wikipedia.org/wiki/Central_tendency

https://en.wikipedia.org/wiki/Expected_value

https://en.wikipedia.org/wiki/Standard_deviation

https://en.wikipedia.org/wiki/Variance

https://en.wikipedia.org/wiki/Probability_density_function

https://en.wikipedia.org/wiki/Central_limit_theorem

https://en.wikipedia.org/wiki/68%E2%80%9395%E2%80%9399.7_rule

https://en.wikipedia.org/wiki/Cumulative_distribution_function

https://en.wikipedia.org/wiki/Skewness

https://en.wikipedia.org/wiki/Kurtosis

第 4 章：一维卡尔曼滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

既然我们已经了解了离散贝叶斯滤波器和高斯分布，那么现在就可以开始实现卡尔曼滤波器了。我们的实现方式将与处理离散贝叶斯滤波器时完全相同：不会从方程式开始，而是通过逐步分析问题来编写代码。

“一维”的意思是指，该滤波器只跟踪一个状态变量，比如 x 轴上的位置。在后面的章节中，我们将学习更通用的一维滤波器形式，这种滤波器能够同时跟踪多个状态变量，比如位置、速度和加速度。记得在 g-h 滤波器中，我们利用了速度这一参数，从而得到了比仅跟踪位置更准确的估计结果。卡尔曼滤波器也是如此。

那么，为什么不直接采用多维形式的滤波器呢？说实话，相关的数学计算相当复杂，我试图用直观的方法来设计滤波器，但这种方法已经不再适用了。这些复杂的数学公式掩盖了卡尔曼滤波器运作所依据的那些简单原理。

在这一章中，我们学习了如何利用高斯分布来构建贝叶斯滤波器。实际上，卡尔曼滤波器本质上就是一种利用高斯分布的贝叶斯滤波器。在下一章中，我们将把模型扩展到多维形式，届时卡尔曼滤波器的全部优势就能得以充分发挥了！

问题描述

与在“离散贝叶斯滤波器”那一章中类似，我们在这里也要追踪工作场所里走廊中移动的物体。假设在最近的黑客马拉松活动中，有人开发出了一个 RFID 追踪器，能够相当准确地确定狗的位置。该传感器会反馈狗距离走廊左端的距离，单位为米。因此，23.4 米的数值意味着狗距离走廊左端 23.4 米远。

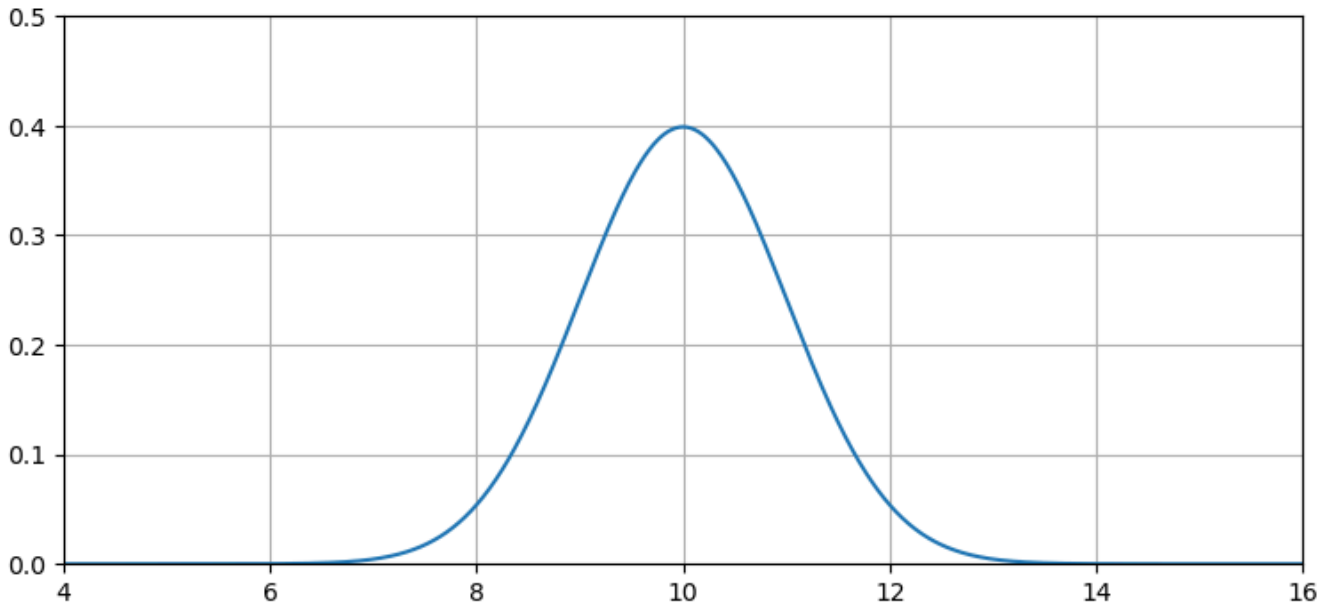
这种传感器并不完美。例如，当测量值为 23.4 时，实际上狗可能位于 23.7 或 23.0 的位置。不过，测量值为 47.6 的可能性则极低。在黑客松期间的测试也证实了这一结果：该传感器的精度“相当不错”。虽然存在一定的误差，但这些误差都很小。此外，这些误差似乎在真实位置的上下范围内均匀分布；也就是说，距离为 23 米的物体，其测量值很可能是 22.9 或 23.1。或许我们可以用高斯函数来描述这种误差分布。

我们预测这只狗正在移动。不过，这种预测并不完美。有时候，我们的预测会偏高；有时候则偏低。通常情况下，预测值会稍微偏高或偏低一点，而不会偏差太大。或许，我们可以用高斯函数来模拟这一现象。

将信念视为高斯分布

我们可以用高斯函数来表示我们对狗狗所在位置的判断。假设我们认为狗狗位于 10 米处，那么这种判断的方差为 1 米。相应的概率密度函数如下图所示：

```
import filterpy.stats as stats
stats.plot_gaussian_pdf(mean=10., variance=1.,
                        xlim=(4, 16), ylim=(0, .5));
```

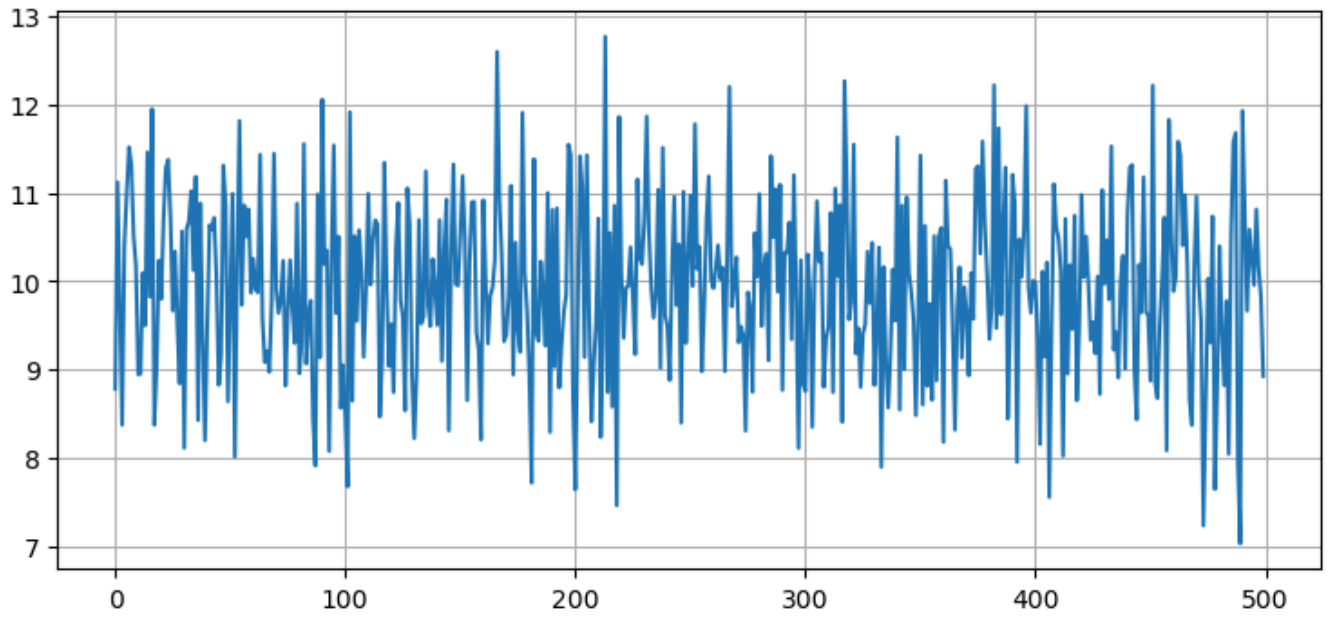


该图表反映了我们对狗所在位置的不确定性。这其实是一种相当不精确的估计。虽然我们认为狗很可能位于 10 米处，但 9 米到 11 米之间的任何位置也都有可能。假设狗静止不动，我们再次询问传感器。这次，传感器显示的距离是 10.2 米。那么，我们能否利用这一额外信息来更准确地估计狗的位置呢?

直觉告诉我们，我们是可以这么做的。试想一下：如果我们读取传感器的数值 500 次，而每次得到的数值都在 8 到 12 之间，且数值都接近 10，那么我们就有充分的理由相信，那只狗确实就在 10 附近。当然，也有可能有其他解释。也许那只狗只是随机地来回移动，其运动轨迹恰好符合正态分布的随机抽样规律。但这种可能性极低——我从未见过狗会有这样的行为。让我们来看看 $N(10,1)$ 的 500 次测量结果吧：

```
import numpy as np
from numpy.random import randn
import matplotlib.pyplot as plt

xs = range(500)
ys = randn(500)*1. + 10.
plt.plot(xs, ys)
print(f'Mean of readings is {np.mean(ys):.3f}')
```

```
Mean of readings is 9.936
```



凭肉眼观察，这一结果与我们的直觉相符——没有狗会以这种方式移动。不过，传感器检测到的数据确实呈现如此形态。各项测量值的算术平均值恰好为 10。假设狗处于静止状态，那么可以说狗位于位置 10，其误差范围为 1。

基于高斯概率的跟踪算法

这种离散贝叶斯滤波器利用概率直方图来追踪狗的位置。直方图中的每个区间代表一个位置，而该区间的数值则代表了狗处于该位置的概率。

跟踪过程是通过一系列的预测和更新来实现的。我们使用了相应的数学公式来进行计算。

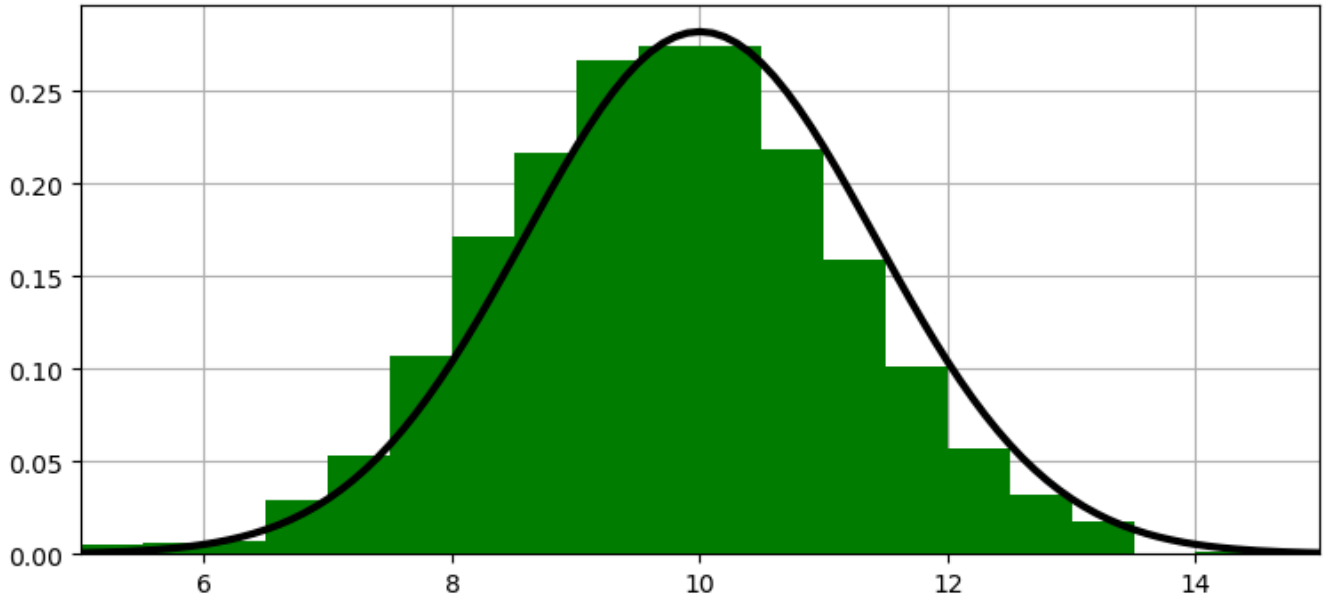
$$\begin{aligned}\bar{x} &= x * f_x(\bullet) && \text{Predict} \\ x &= L \cdot \bar{x} && \text{Update}\end{aligned}$$

为了计算新的概率分布，需要记住： $\bar{x}$ 代表先验概率； L 表示在给定先验概率 $\bar{x}$ 的条件下，某次测量的概率； $f_x(\bullet)$ 代表过程模型； $*$ 表示卷积操作。 x 以粗体标出，表示其实际上是一个数值的直方图或向量。

这种方法确实有效，但生成的直方图表明，那只狗可能同时出现在多个位置上。此外，对于规模较大的问题，这种计算方式的速度极其缓慢。

我们能把直方图 x 替换为高斯分布 $N(x, \sigma^2)$ 吗？当然可以！我们已经知道如何用高斯分布来表示概率。高斯分布其实只是一个数字对 $N(\mu, \sigma^2)$, ，它完全可以替代整个概率直方图。

```
import kf_book.kf_internal as kf_internal
kf_internal.gaussian_vs_histogram()
```



希望你能明白其中的妙处。我们可以用一对数字 $x = N(\mu, \sigma^2)$ 来替代成百上千个数字。

高斯曲线的尾部向两侧无限延伸，因此该曲线能够涵盖直方图中的任意多个数值。如果用这种方式来表示我们对狗在走廊上位置的判断，那么这个高斯曲线就涵盖了整个走廊（以及该轴线上的整个“宇宙”）。我们认为狗很可能位于 10 号位置，但也有可能位于 8 号或 14 号位置；当然，狗位于 10 号位置的概率也是极小的。

在本章中，我们用高斯分布来替代直方图：

discrete Bayes	Gaussian	Step
$\bar{x} = x * f(x)$	$\bar{x}_N = x_N \oplus f_{x_N}(\bullet)$	Predict
$x = \|L\bar{x}\|$	$x_N = L \otimes \bar{x}_N$	Update

其中，$\oplus$ 和 $\otimes$ 表示对高斯函数施加的某种未知运算。在本书的其余部分，我不会再使用这种表示方法。不过，下标表明 x_N 确实是一个高斯函数。

这种离散贝叶斯滤波器在预测过程中使用了卷积运算。我们发现，该滤波器实际上运用了全概率定理，即将各种概率相加来得到最终结果。因此，或许我们可以将高斯函数也相加起来。此外，该滤波器还通过乘法运算将测量结果纳入先验概率中，所以也许我们也可以对高斯函数进行乘法运算。难道真的有那么简单吗？

$$\begin{aligned}\bar{x} &\stackrel{?}{=} x + f_x(\bullet) \\ x &\stackrel{?}{=} L \cdot \bar{x}\end{aligned}$$

只有当两个高斯函数的和与积仍然是高斯函数时，这种算法才有效。否则，在第一个训练周期之后，x 就不再满足高斯函数的性质，整个算法也就无法继续运行了。

高斯模型预测结果

我们利用牛顿运动定律，根据当前速度和先前的位置来计算出当前位置。

$$\begin{aligned}\bar{x}_k &= x_{k-1} + v_k\Delta t \\ &= x_{k-1} + f_x\end{aligned}$$

为了使方程更简洁明了，我不再使用 $f_x(\bullet)$ 这个符号，而改用 f_x 。

如果狗位于 10 米处，其速度为 15 米/秒，且时间间隔为 2 秒，那么我们可以得到：

$$\begin{aligned} f_x &= v\Delta t = 15 \cdot 2 \\ \bar{x}_k &= 10 + (15 \cdot 2) = 40 \end{aligned}$$

我们无法确定他目前的所在位置和速度，因此这种表示方式不可行。我们需要用高斯函数来表示这种不确定性。

定位其实很简单。我们将 x 定义为高斯分布。如果我们认为狗位于 10 米处，而我们的不确定性标准差为 0.2 米，那么就能得出 $x = N(10, 0.2^2)$ 的值。

那我们对他移动路径的不确定性又该如何处理呢？我们将这种不确定性定义为高斯分布。如果狗的速度为 15 米/秒，时间间隔为 1 秒，而不确定性的标准差为 0.7 米/秒，那么我们就能计算出 $f_x = N(15, 0.7^2)$ 的值了。

先验概率的公式为：

$$\bar{x} = x + f_x$$

两个高斯函数的和是多少呢？在上一章中，我已经证明了：

$$\begin{aligned} \mu &= \mu_1 + \mu_2 \\ \sigma^2 &= \sigma_1^2 + \sigma_2^2 \end{aligned}$$

这真是个好消息：两个高斯函数的和仍然是一个高斯函数！

从数学上来说，这个公式是成立的。但是，从直观上来看，这真的合理吗？试着想象一下这个抽象方程在现实世界中的体现吧。我们有……

$$\begin{aligned} x &= N(10, 0.2^2) \\ f_x &= N(15, 0.7^2) \end{aligned}$$

如果把它们加起来，就得到：

$$\begin{aligned} \bar{x} &= \mu_x + \mu_{f_x} = 10 + 15 &= 25 \\ \bar{\sigma}^2 &= \sigma_x^2 + \sigma_{f_x}^2 = 0.2^2 + 0.7^2 &= 0.53 \end{aligned}$$

预测位置等于上一个位置加上移动距离，这很合乎逻辑。那么，方差呢？这一点较难直观理解。不过，请回想一下：在使用离散贝叶斯滤波器的 `predict()` 函数时，我们总会丢失一些信息。我们其实并不知道狗到底移动到了哪里，因此置信度应该会降低（方差则会上升）。 $\sigma_{f_x}^2$ 代表了由于对狗的移动方向预测不准确而导致的不确定性。所以，我们需要把这一不确定性加到现有的不确定性上。

我们可以利用 Python 的 `collection` 模块中的 `namedtuple` 类来创建高斯对象。其实，我们也可以用元组来表示高斯分布，因为在 Python 中， $N(10, 0.04)$ 实际上就是以 `g = (10., 0.04)` 的形式实现的。我们可以通过 `g[0]` 来获取高斯分布的均值，通过 `g[1]` 来获取方差。

`namedtuple` 的用法与元组类似，只不过需要指定数据的类型和各个字段的名称。虽然理解这一点并非必要，但我修改了 `__repr__` 方法，使其能按照本章所介绍的格式来显示其值。

```
from collections import namedtuple
gaussian = namedtuple('Gaussian', ['mean', 'var'])
gaussian.__repr__ = lambda s: f'𝒩(μ={s[0]:.3f}, σ²={s[1]:.3f})'
```

现在，我们可以用以下方式来生成高斯分布的图像了：

```
g1 = gaussian(3.4, 10.1)
g2 = gaussian(mean=4.5, var=0.2**2)
print(g1)
print(g2)
```

```
𝒩(μ=3.400, σ²=10.100)
𝒩(μ=4.500, σ²=0.040)
```

我们可以通过下标或字段名称来获取均值和方差：

```
g1.mean, g1[0], g1[1], g1.var
```

```
(3.4, 3.4, 10.1, 10.1)
```

以下是我们对“predict”函数的实现方式。其中， `pos` 和 `movement` 都是高斯元组，其形式为(μ , σ^2)。

```
def predict(pos, movement):
    return gaussian(pos.mean + movement.mean, pos.var + movement.var)
```

我们来测试一下。如果初始位置服从高斯分布 $N(10, 0.2^2)$ ，而运动过程也服从高斯分布 $N(15, 0.7^2)$ ，那么先验概率是多少呢?

```
pos = gaussian(10., .2**2)
move = gaussian(15., .7**2)
predict(pos, move)
```

```
𝒩(μ=25.000, σ²=0.530)
```

先前的结果显示，那只狗的距离为 25 米，误差为 0.53 米。这一结果与我们手动计算的结果一致。

高斯函数的更新情况

离散贝叶斯滤波器将我们对狗所在位置的判断以概率直方图的形式表现出来。这种概率分布是离散的、多模态的。它意味着我们可以同时认为狗处于两个不同的位置上，而这些位置都是离散的。

我们建议用高斯分布来替代直方图。所使用的离散贝叶斯滤波器正是利用这一公式来计算后验概率的。

```
def update(likelihood, prior):
    posterior = likelihood * prior
    return normalize(posterior)
```

这是该方程式的具体实现方式：

$$x = \|L\bar{x}\|$$

我们刚刚证明了可以用高斯分布来表示先验概率。那么，似然概率呢？似然概率指的是在当前状态下，进行某次测量的概率。我们已经知道如何用高斯分布来表示各种测量结果。例如，假设我们的传感器检测到狗位于 23 米处，标准差为 0.4 米。那么，用似然概率来表示的话，就是 $z = N(23, 0.16)$ 。

概率分布和先验概率都用高斯函数来建模。那么，高斯函数可以相乘吗？两个高斯函数的乘积还是高斯函数吗？

前者当然成立，后者也几乎成立！在上一章中，我已经证明：两个高斯函数的乘积与另一个高斯函数成正比。

$$\begin{aligned}\mu &= \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2}, \\ \sigma^2 &= \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}\end{aligned}$$

我们可以立即得出几个结论。如果对结果进行标准化处理，那么乘积仍然是一个高斯分布。如果其中一个高斯分布代表“似然值”，另一个代表“先验概率”，那么平均值就是先验概率与测量结果的加权之和。方差则是先验概率的方差与测量结果的方差的组合。最后，方差的值完全不受平均值的影响！

我们可以用贝叶斯术语来表达这一点：

$$\begin{aligned}N(\mu, \sigma^2) &= \|prior \cdot likelihood\| \\ &= \|N(\bar{\mu}, \bar{\sigma}^2) \cdot N(\mu_z, \sigma_z^2)\| \\ &= N(\frac{\bar{\sigma}^2\mu_z + \sigma_z^2\bar{\mu}}{\bar{\sigma}^2 + \sigma_z^2}, \frac{\bar{\sigma}^2\sigma_z^2}{\bar{\sigma}^2 + \sigma_z^2})\end{aligned}$$

如果我们在函数 `gaussian_multiply()` 中实现这一点，那么就可以将滤波器的更新步骤也一并实现出来。

```
def gaussian_multiply(g1, g2):
    mean = (g1.var * g2.mean + g2.var * g1.mean) / (g1.var + g2.var)
    variance = (g1.var * g2.var) / (g1.var + g2.var)
    return gaussian(mean, variance)

def update(prior, likelihood):
    posterior = gaussian_multiply(likelihood, prior)
    return posterior

# test the update function
predicted_pos = gaussian(10., .2**2)
measured_pos = gaussian(11., .1**2)
estimated_pos = update(predicted_pos, measured_pos)
estimated_pos
```

```
𝒩(μ=10.800, σ²=0.008)
```

如果我们使用更具体的名称来表述，或许会更容易理解：

```
def update_dog(dog_pos, measurement):
    estimated_pos = gaussian_multiply(measurement, dog_pos)
    return estimated_pos
```

这样虽然不那么抽象了，或许有助于理解，但这并不是良好的编程习惯。我们正在编写的卡尔曼滤波器应该适用于各种问题，而不仅仅是用于跟踪走廊里的狗。因此，我们不会使用包含“dog”这个词的变量名。此外，这种写法也让人看不清：实际上，我们是在将概率与先验概率相乘而已。

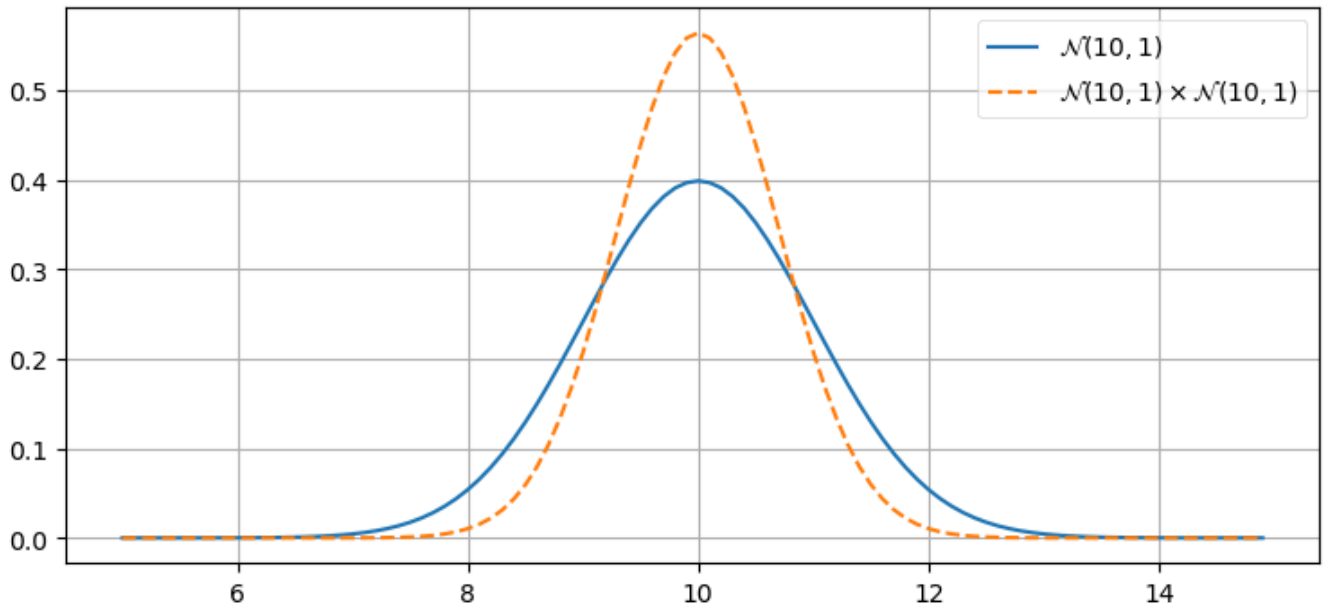
我们已经实现了大部分的滤波功能，但我觉得这一步还是有点复杂。我确认过，高斯函数是可以相乘的，而且这种操作确实能够正确地完成更新步骤。但为什么会是这样呢？让我们花点时间来仔细研究一下高斯函数的相乘运算吧。

理解高斯乘法运算

让我们来绘制 $N(10,1) \times N(10,1)$ 的概率密度函数吧。不用查看结果，你能判断出它的形状吗？新的均值应该是多少呢？这条曲线会比 $N(10,1)$ 的更宽、更窄，还是保持不变呢？

```
z = gaussian(10., 1.)  # Gaussian N(10, 1)

product = gaussian_multiply(z, z)

xs = np.arange(5, 15, 0.1)
ys = [stats.gaussian(x, z.mean, z.var) for x in xs]
plt.plot(xs, ys, label='$\mathcal{N}(10,1)$')

ys = [stats.gaussian(x, product.mean, product.var) for x in xs]
plt.plot(xs, ys, label='$\mathcal{N}(10,1) \\times \mathcal{N}(10,1)$', ls='--')
plt.legend()
print(product)
```

```
𝒩(μ=10.000, σ²=0.500)
```



乘法运算后的结果比原始的高斯分布更细长，但均值保持不变。这一结果符合你的直觉吗?

可以把高斯分布看作是两次测量结果。如果我进行了两次测量，每次得到的数值都是 10 米，那么我应该认为该长度接近 10 米。因此，平均值应该是 10 米。认为该长度实际上是 11 米或 9.5 米是没有道理的。此外，两次测量的结果比一次测量的结果更可靠，所以测量结果的方差应该更小。

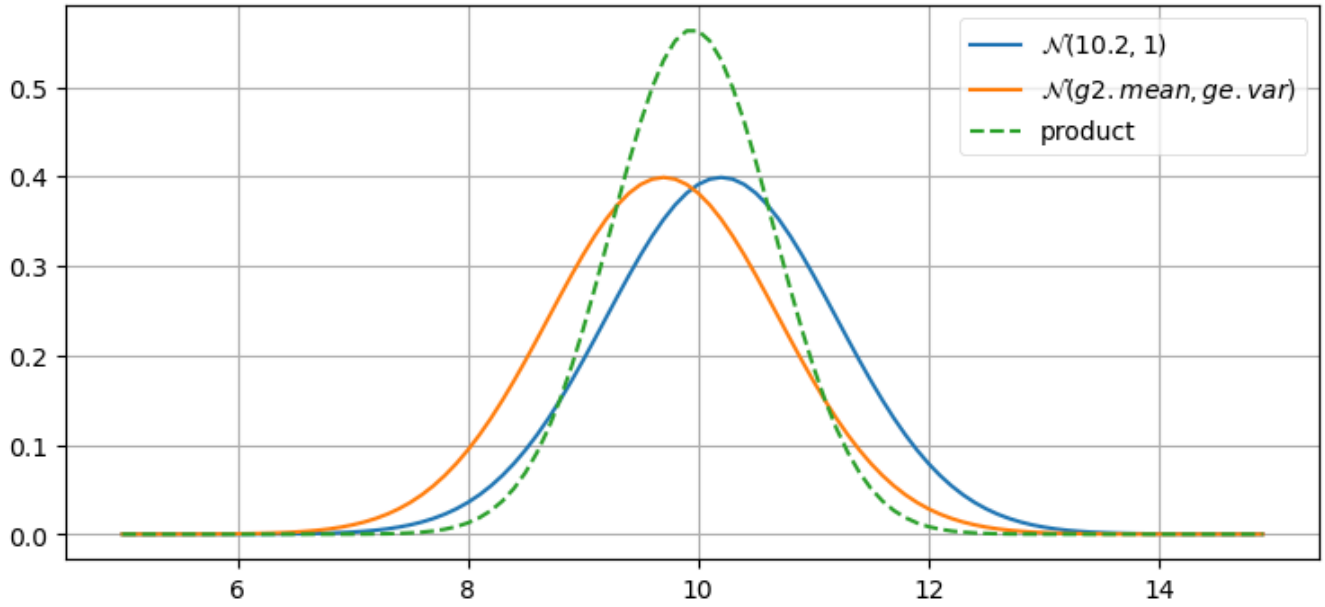
“三思而后行”是一句众所周知的格言。高斯乘法正是这一物理现象的数学体现。

我不太可能连续两次得到相同的测量结果。现在，让我们来绘制 $N(10.2, 1) \times N(9.7, 1)$ 的概率密度函数吧。你认为结果会怎样呢？先好好思考一下，然后再看看图表吧。

```
m = 3.3
f'$({m},{m})$'
```

```
'$(3.3,3.3)$'
```

```
def plot_products(g1, g2):
    plt.figure()
    product = gaussian_multiply(g1, g2)

    xs = np.arange(5, 15, 0.1)
    ys = [stats.gaussian(x, g1.mean, g1.var) for x in xs]
    plt.plot(xs, ys, label='$\mathcal{N}$' + f'$({g1.mean},{g1.var})$')

    ys = [stats.gaussian(x, g2.mean, g2.var) for x in xs]
    plt.plot(xs, ys, label='$\mathcal{N}$' + '$({g2.mean},{ge.var})$')

    ys = [stats.gaussian(x, product.mean, product.var) for x in xs]
    plt.plot(xs, ys, label='product', ls='--')
    plt.legend();
    plt.show()

z1 = gaussian(10.2, 1)
z2 = gaussian(9.7, 1)

plot_products(z1, z2)
```



如果你让两个人来测量桌子到墙的距离，其中一个人测得 10.2 米，另一个人测得 9.7 米，那么如果你对两人的测量能力都相当信任的话，最合理的估计应该是两者的平均值，即 9.95 米。

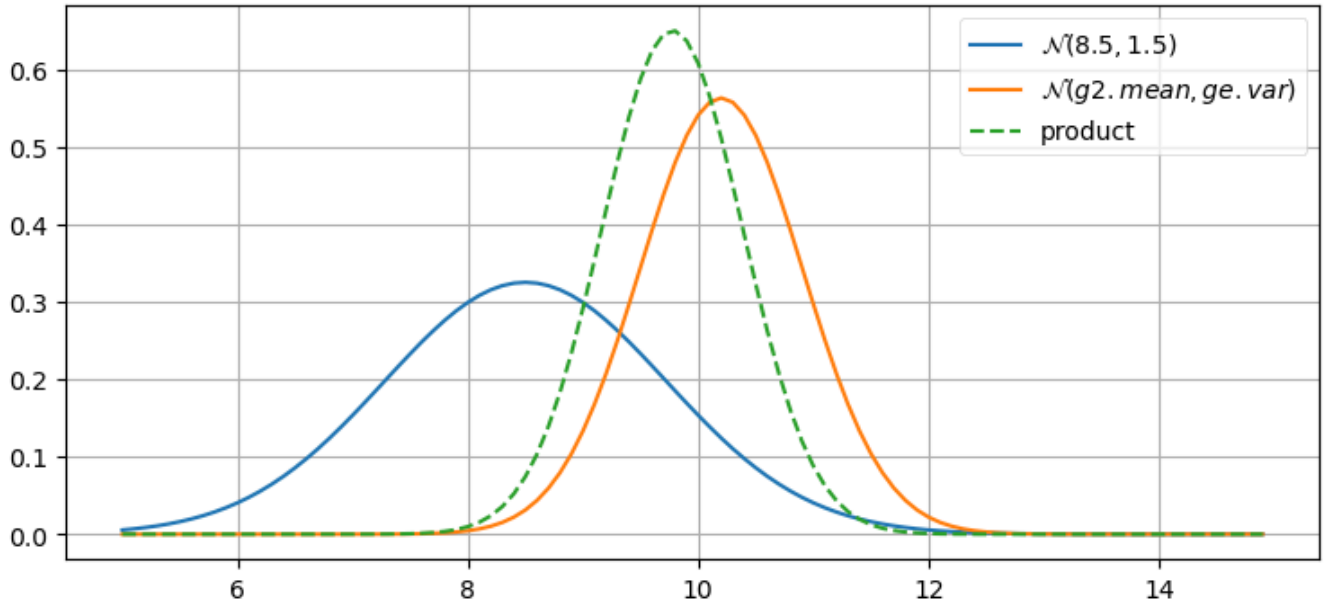
请回想一下 g-h 滤波器的相关原理。我们之前说过，如果我用两台秤来称体重，第一台秤显示 160 磅，第二台秤显示 170 磅，而这两台秤的测量精度都相当准确的话，那么最合理的估计值应该是 165 磅。此外，与 160 磅或 170 磅相比，我更应该相信 165 磅这个数值。因为现在我有两个接近 165 磅的测量结果，这就增加了我对这个数值的信任度，因为可以排除其中某个数值出现严重误差的可能性。

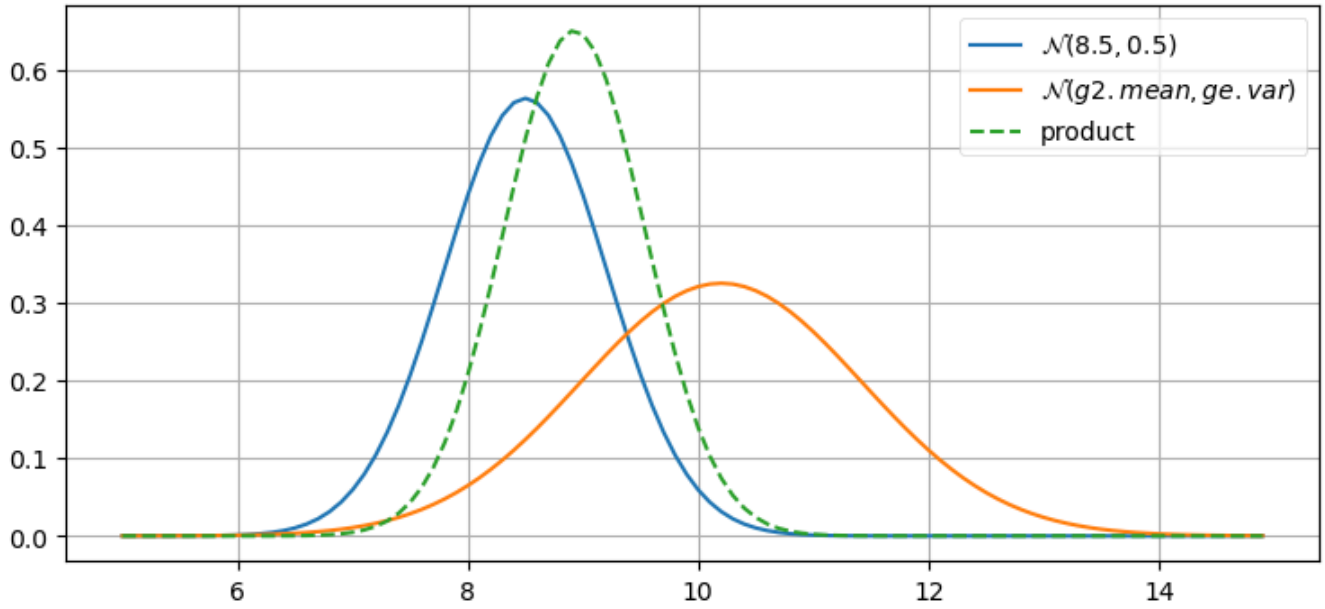
在更复杂的情况下，这种推理方式就显得不合常理了。让我们进一步思考一下。或许，更合理的假设是：其中一个人犯了错误，所以真实的距离应该是 10.2 或 9.7，而不可能是 9.95。当然，这种可能性是存在的。但是，由于我们的测量结果都存在误差，我们没有理由认为其中某个测量结果是完全准确的，或者认为某个人犯了严重的错误，从而可以忽略他的测量结果。根据所有可用的信息，最合理的估计值应该是 9.95。

在卡尔曼滤波器的更新步骤中，我们并非将两次测量结果相加，而是将某次测量结果与先前的估计值相加。对于 g-h 滤波器来说，道理也是一样的。无论我们是结合两次测量结果的信息，还是将某次测量结果与预测值相结合，其数学运算方式都是一样的。

让我们来看看。我会为 $N(8.5, 1.5)$ 设定一个相对不准确的初始值，而为 $N(10.2, 0.5)$. 则设定一个更准确的测量值。所谓“准确”，指的是传感器的误差小于初始值的误差，而不是说我知道那只狗的距离确实更接近 10.2 而不是 8.5。接下来，我会绘制出相反的情况：为 $N(8.5, 0.5)$ 设定一个准确的初始值，而为 $N(10.2, 1.5)$ 则设定一个不准确的测量值。

```
prior, z = gaussian(8.5, 1.5), gaussian(10.2, 0.5)
plot_products(prior, z)

prior, z = gaussian(8.5, 0.5), gaussian(10.2, 1.5)
plot_products(prior, z)
```





最终得到的高斯分布比两个输入值都要高。这是合乎逻辑的——因为我们已经把各种信息都考虑进去了，所以方差应该会减小。另外，可以注意到，最终得到的结果更接近于那个方差较小的输入值。我们对那个值的置信度更高，因此给予它更高的权重是合理的。

看起来这个方法确实有效，但是否真的正确呢？关于这一点还有更多需要探讨的。不过，我现在想先让这个滤波器能够正常运行起来，这样你们就能亲自体验它的效果了。之后，我们再重新探讨高斯乘法的原理，弄清楚为什么这种方法是正确的。

交互式示例

这段交互式代码提供了滑块，可用于调整两个高斯函数的均值和方差。当您移动滑块时，图表会随之重新绘制。请将光标置于代码单元格内，然后按 CTRL+Enter 键来执行该代码。

```
from ipywidgets import interact

def interactive_gaussian(m1, m2, v1, v2):
    g1 = gaussian(m1, v1)
    g2 = gaussian(m2, v2)
    plot_products(g1, g2)

interact(interactive_gaussian,
         m1=(5, 10., .5), m2=(10, 15, .5),
         v1=(.1, 2, .1), v2=(.1, 2, .1));
```

```
interactive(children=(FloatSlider(value=7.5, description='m1', max=10.0, min=5.0, step=0.5),
FloatSlider(value…
```

卡尔曼滤波器初探

让我们回到具体的实现上，来运用卡尔曼滤波器吧。我们已经实现了 `update()` 和 `predict()` 这两个函数。现在只需要编写一些通用代码来模拟狗的行为并生成相应的测量数据即可。我把 `DogSimulation` 类放在了 `kf_internal` 中，这样就不会因为处理这个任务而分心了。

这段样板代码通过定义各种参数、计算方差以及生成模拟的狗的移动轨迹，从而确立了整个问题的框架。

```
import kf_book.kf_internal as kf_internal
from kf_book.kf_internal import DogSimulation

np.random.seed(13)

process_var = 1. # variance in the dog's movement
sensor_var = 2. # variance in the sensor

x = gaussian(0., 20.**2)  # dog's position, N(0, 20**2)
velocity = 1
dt = 1. # time step in seconds
process_model = gaussian(velocity*dt, process_var) # displacement to add to x

# simulate dog and get measurements
dog = DogSimulation(
    x0=x.mean,
    velocity=process_model.mean,
    measurement_var=sensor_var,
    process_var=process_model.var)

# create list of measurements
zs = [dog.move_and_sense() for _ in range(10)]
```

这就是卡尔曼滤波器。

```
print('PREDICT\t\t\tUPDATE')
print('     x      var\t\t  z\t    x      var')

# perform Kalman filter on measurement z
for z in zs:
    prior = predict(x, process_model)
    likelihood = gaussian(z, sensor_var)
    x = update(prior, likelihood)
```

```
    kf_internal.print_gh(prior, x, z)

print()
print(f'final estimate:        {x.mean:10.3f}')
print(f'actual final position: {dog.x:10.3f}')
```

```
PREDICT         UPDATE
     x      var       z     x      var
  1.000  401.000    1.354     1.352   1.990
  2.352    2.990    1.882     2.070   1.198
  3.070    2.198    4.341     3.736   1.047
  4.736    2.047    7.156     5.960   1.012
  6.960    2.012    6.939     6.949   1.003
  7.949    2.003    6.844     7.396   1.001
  8.396    2.001    9.847     9.122   1.000
 10.122    2.000    12.553   11.338   1.000
 12.338    2.000    16.273   14.305   1.000
 15.305    2.000    14.800   15.053   1.000

final estimate:            15.053
actual final position:     14.838
```

以下是该滤波器的动画演示。预测结果以红色三角形表示。在每次预测之后，滤波器会接收下一个测量值，该测量值以黑色圆圈表示。随后，滤波器会在这两个数值之间得出一个估算值。

```
from kf_book import book_plots as book_plots
from ipywidgets.widgets import IntSlider

# save output in these lists for plotting
xs, predictions = [], []

process_model = gaussian(velocity, process_var)

# perform Kalman filter
x = gaussian(0., 20.**2)
for z in zs:
    prior = predict(x, process_model)
    likelihood = gaussian(z, sensor_var)
    x = update(prior, likelihood)

    # save results
    predictions.append(prior.mean)
    xs.append(x.mean)

def plot_filter(step):
    plt.cla()
    step -= 1
    i = step // 3 + 1

    book_plots.plot_predictions(predictions[:i])
    if step % 3 == 0:
        book_plots.plot_measurements(zs[:i-1])
        book_plots.plot_filter(xs[:i-1])
    elif step % 3 == 1:
        book_plots.plot_measurements(zs[:i])
        book_plots.plot_filter(xs[:i-1])
    else:
        book_plots.plot_measurements(zs[:i])
        book_plots.plot_filter(xs[:i])
```

```
    plt.xlim(-1, 10)
    plt.ylim(0, 20)
    plt.legend(loc=2);
    plt.show()

interact(plot_filter, step=IntSlider(value=1, min=1, max=len(predictions)*3));
```

```
interactive(children=(IntSlider(value=1, description='step', max=30, min=1), Output()), _dom_classes=
('widget-…
```

我已经绘制出了先验概率值、测量结果以及滤波器的输出结果。在循环的每次迭代中，我们都会先确定先验概率值，然后进行测量，根据测量结果计算出概率值，最后将这个概率值纳入到先验概率值中。

从图表中可以看出，滤波后的估计值始终位于测量值和预测值之间。回想一下，对于 g-h 滤波器来说，我们曾经指出，估计值必须始终处于测量值和先验值之间。选择超出这两个范围的数值是没有意义的。如果我预测自己处于 10 的位置，但实际测量结果却是 9，那么认为自己应该处于 8 或 11 的位置就显得很荒谬了。

代码详解/代码实现过程

现在让我们来仔细看看这段代码。

```
process_var = 1.
sensor_var = 2.
```

这些分别是过程模型和传感器的方差值。传感器的方差很容易理解：它指的是每次测量结果之间的差异大小。而过程方差则反映了过程模型中的误差程度。我们预测，狗在每个时间点都会向前移动一米。不过，狗的行为往往不会完全按照我们的预期行事，比如有障碍物或听到松鼠的叫声时，狗的行进路径就会发生变化。如果这是一个响应数字指令的机器人，那么其性能会好得多，相应的方差值也可能接近于 0。这些数字并非“神奇的数值”；方差的平方根实际上代表了误差的大小，单位是米。虽然只要输入正确的数值，卡尔曼滤波器就能正常工作，但如果输入的数值与实际情况不符，滤波器的性能就会很差。

```
x = gaussian(0., 20.**2)
```

这就是那只狗的初始位置，用高斯分布来表示。该位置为 0 米，方差为 400 米，相当于标准差为 20 米。可以这样理解：“我有 99.7%的把握认为，该狗的位置在 0 米上下 60 米的范围内。”因为根据高斯分布，99.7%的数值都位于平均值上下 1 个标准差范围内。

```
process_model = gaussian(velocity, process_var)
```

这就是所谓的“过程模型”——也就是我们对狗的移动方式所做出的描述。那我怎么知道狗的速度呢？这有什么神奇的吗？其实可以把这看作是一种预测而已。或者，也许我们还有别的方式来测量速度。如果这是一台机器人，那么这些数据就可以作为机器人的控制输入信号。在后面的章节中，我们将学习如何处理没有速度传感器或相关输入信号的情况。所以，请暂时接受这种简化后的解释吧。

接下来，我们初始化模拟过程，并进行 10 次测量：

```
dog = DogSimulation(
    x0=x.mean,
    velocity=process_model.mean,
    measurement_var=sensor_var,
    process_var=process_model.var)

zs = [dog.move_and_sense() for _ in range(10)]
```

现在，我们进入 `predict() ... update()` 循环。

```
for z in zs:
    prior = predict(x, process_model)
    likelihood = gaussian(z, sensor_var)
    x = update(prior, likelihood)
```

在循环的第一次迭代中， `prior` 的值为 `(1.0, 401.0)` ，这一点可以从打印出的表格中看出。在预测之后，我们认为当前的值应为 1.0，而方差则从 400 上升到了 401。方差有所增加，这在预测过程中是不可避免的，因为预测过程本身就会导致信息的丢失。

然后，我们以 `prior` 作为当前位置，调用更新函数。

因此，我得到的结果是： `pos = (1.352, 1.990), z = 1.354` 。

发生了什么？实际上，狗的位置应该是 1.0，但由于传感器噪声的影响，实际测量值为 1.354。这个数值与预测值 1 相差很大。先验值的方差为 401 米。较大的方差意味着置信度很低，因此滤波器将位置估计为与测量值非常接近的 1.352。

现在来看看方差：1.99 百万 ^{2} 。这一数值相比 401 百万 ^{2} 有了大幅下降。为什么呢？因为 RFID 的方差相对较小，仅为 2.0 百万 ^{2} ，因此我们更信任 RFID 的测量结果。不过，之前的测量结果也包含了一些有用的信息，所以现在的方差略小于 2.0 百万。

现在，该软件会循环执行，依次调用 `predict()` 和 `update()` 。最终，估算出的位置为 15.053，而实际位置为 14.838。偏差已收敛到 1.0 米以内，即 ^{2} 所表示的数值。

现在请看这个图表。带有噪声的测量数据用黑色圆点表示，而经过滤波后的结果则用蓝色实线表示。两者都存在相当大的噪声，但可以明显看出，测量数据的噪声要大得多。我用红色三角形标出了预测值。可以看出，预测值始终位于测量值和先验值之间。这就是第一个卡尔曼滤波器的应用实例，看起来它确实有效！

滤波功能的实现仅需要几行代码而已。大部分代码用于初始化、数据存储、模拟狗的移动以及结果输出。负责执行滤波操作的代码非常简洁：

```
prior = predict(x, process_model)
likelihood = gaussian(z, sensor_var)
x = update(prior, likelihood)
```

如果我们不使用 `predict` 和 `update` 函数，那么代码可能会是这样的：

```
for z in zs:
    # predict
    dx = velocity*dt
```

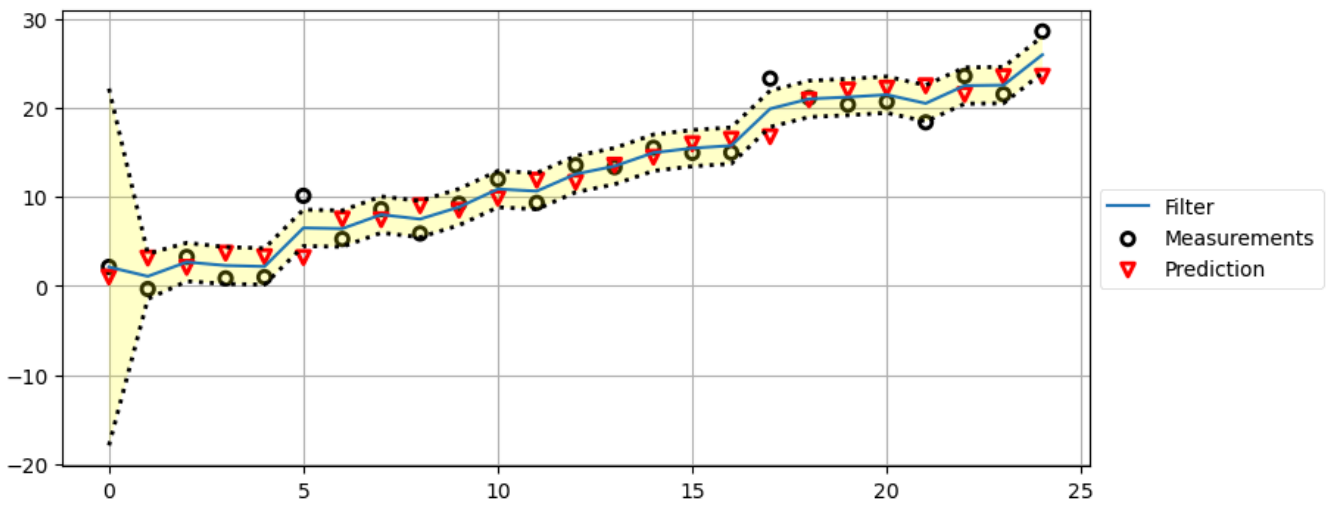
```
    pos = pos + dx
    var = var + process_var

    # update
    pos  = (var*z + sensor_var*pos) / (var + sensor_var)
    var = (var * sensor_var) / (var + sensor_var)
```

只需要 5 行非常简单的数学公式，就能实现整个滤波过程！

在这个例子中，我只绘制了 10 个数据点，这样，打印出来的结果就不会过于复杂难懂。现在，让我们来看看当数据量增加时，该滤波器的性能如何。方差以浅黄色区域的形式显示在虚线之间。为了便于观察，我把过程方差和传感器方差的数值放大了——当然，在实际的卡尔曼滤波器中，这些数值是不会随意变化的。

```
process_var = 2.
sensor_var = 4.5
x = gaussian(0., 400.)
process_model = gaussian(1., process_var)
N = 25

dog = DogSimulation(x.mean, process_model.mean, sensor_var, process_var)
zs = [dog.move_and_sense() for _ in range(N)]

xs, priors = np.zeros((N, 2)), np.zeros((N, 2))
for i, z in enumerate(zs):
    prior = predict(x, process_model)
    x = update(prior, gaussian(z, sensor_var))
    priors[i] = prior

    xs[i] = x

book_plots.plot_measurements(zs)
book_plots.plot_filter(xs[:, 0], var=priors[:, 1])
book_plots.plot_predictions(priors[:, 0])
book_plots.show_legend()
kf_internal.print_variance(xs)
```

```
    4.4502 2.6507 2.2871 2.1955 2.1712
    2.1647 2.1629 2.1625 2.1623 2.1623
    2.1623 2.1623 2.1623 2.1623 2.1623
    2.1623 2.1623 2.1623 2.1623 2.1623
    2.1623 2.1623 2.1623 2.1623 2.1623
```



我们可以看到，经过 9 次迭代后，方差值收敛到了 2.1623。这意味着我们对位置的估计已经相当准确了。该数值相当于 $\sigma = 1.47$ 米。与传感器的 $\sigma = 2.12$ 米相比，我们的估计结果更为精确。由于初始位置的不确定性，最初的几次测量结果并不可靠，但经过滤波处理后，得到的估计值方差明显小于传感器的测量结果！

这段代码完整实现了卡尔曼滤波器。如果你尝试过阅读相关文献，可能会感到惊讶：因为这里的实现方式与那些书中冗长的数学公式完全不同。只要我们专注于使用这些公式，而不用去推导它们，那么这个概念就很容易理解了。此外，我相信你也能感觉到，自己对整个过程的直觉理解相当不错。我们用高斯分布来表示各种概率值，随着时间的推移，这些概率值会越来越准确，因为更多的测量数据意味着我们有更多的信息可以用来进行计算。

练习：修改方差数值

修改 `process_var` 和 `sensor_var` 的值，观察其对滤波效果以及方差的影响。其中，哪一个因素对方差收敛的影响更大呢？也就是说，哪一个方案能使得方差更小呢？

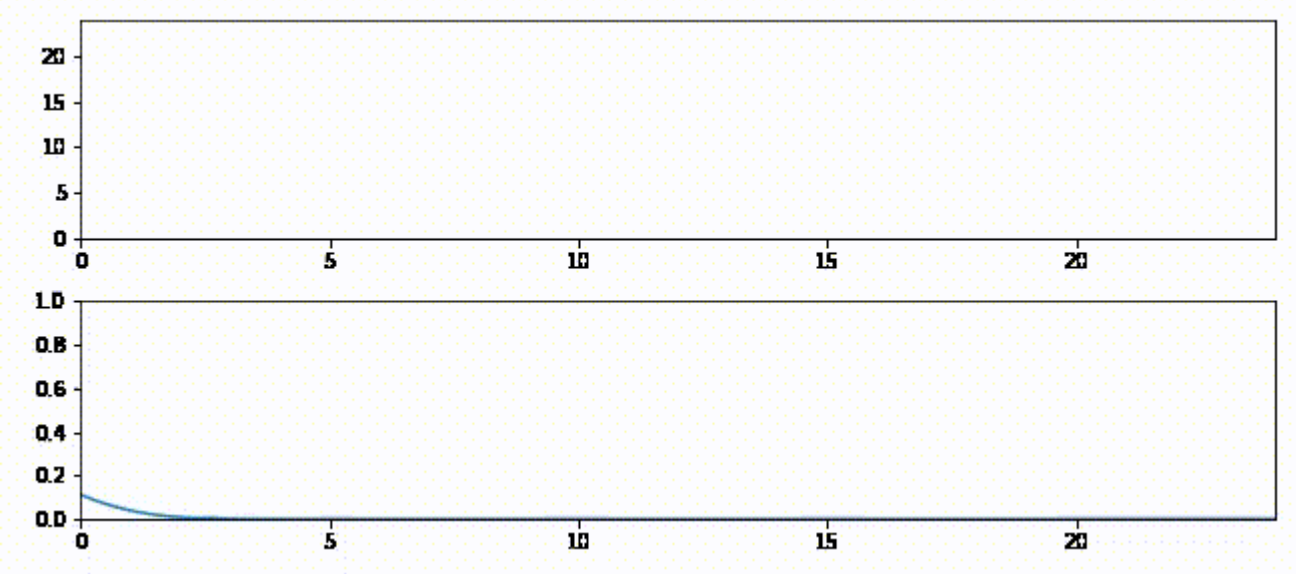
```
process_var = 40
sensor_var = 2
```

或者：

```
process_var = 2
sensor_var = 40
```

KF 动画

如果您是通过浏览器阅读这段文字，那么您会看到一个动画，展示该滤波器是如何跟踪那只狗的。就在这句话的 正 下 方 。



上图以绿色线条显示了滤波器的处理结果，用红色虚线则标出了测量值。下图则展示了每一步处理后的高斯分布情况。

当测量开始时，可以看到实际数值与最初的预测值有很大差异。此时，高斯曲线的幅度较小（曲线较平缓），因此滤波器对自身的预测结果不太信任。相应地，滤波器会大幅调整自己的估算值。随着滤波器不断更新其估算结果，高斯曲线会变得更高，这意味着估算的准确性有所提高，滤波器的输出也越接近平直线。在 `x = 15` 及更高的数值范围内，虽然测量结果中存在大量噪声，但与第一次遇到噪声时的情况相比，滤波器的反应并不明显。

卡尔曼增益

我们看到，该滤波器确实有效。现在，让我们回到数学公式上来，弄清楚其中的具体原理。后验概率 x 是由似然值与先验概率 $L\bar{x}$ 相乘得到的，而这两者都是高斯分布。

因此，后验概率的均值如下：

$$\mu = \frac{\bar{\sigma}^2\mu_z + \sigma_z^2\bar{\mu}}{\bar{\sigma}^2 + \sigma_z^2}$$

我用下标 z 来表示该测量值。我们可以将其重新表示为：

$$\mu = \left(\frac{\bar{\sigma}^2}{\bar{\sigma}^2 + \sigma_z^2}\right)\mu_z + \left(\frac{\sigma_z^2}{\bar{\sigma}^2 + \sigma_z^2}\right)\bar{\mu}$$

从这个形式可以看出，我们是通过权重来调整测量值和先验值的。

$$\mu = W_1\mu_z + W_2\bar{\mu}$$

各权重之和为一，因为分母是一个用于归一化的项。我们引入了一个新项 $K = W_1$，从而得到：

$$\begin{aligned}\mu &= K\mu_z + (1-K)\bar{\mu} \\ &= \bar{\mu} + K(\mu_z - \bar{\mu})\end{aligned}$$

在……的地方/在何处

$$K = \frac{\bar{\sigma}^2}{\bar{\sigma}^2 + \sigma_z^2}$$

K 就是卡尔曼增益。它是卡尔曼滤波器的核心要素。实际上，它是一个用于调整数值的参数，其取值介于 μ_z 和 $\bar{\mu}$ 之间。

我们来看几个例子吧。如果现在的测量精度是之前的九倍，那么 $\bar{\sigma}^2 = 9\sigma_z^2$，而……

$$\begin{aligned}\mu &= \frac{9\sigma_z^2\mu_z + \sigma_z^2\bar{\mu}}{9\sigma_z^2 + \sigma_z^2} \\ &= \left(\frac{9}{10}\right)\mu_z + \left(\frac{1}{10}\right)\bar{\mu}\end{aligned}$$

因此，$K = \frac{9}{10}$。在计算后验概率时，我们取测量值的九成，以及先验概率的十分之一。

如果测量结果和先前的数据都同样准确的话，那么 $\bar{\sigma}^2 = \sigma_z^2$ 和……

$$\mu = \frac{\sigma_z^2(\bar{\mu} + \mu_z)}{2\sigma_z^2}$$
$$= \left(\frac{1}{2}\right)\bar{\mu} + \left(\frac{1}{2}\right)\mu_z$$

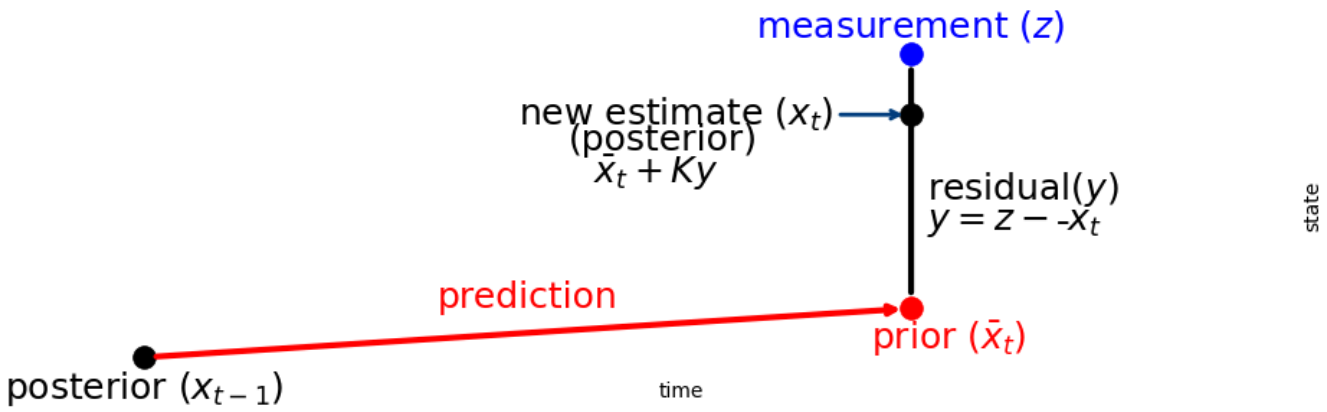
这是两个平均值的平均值。从直觉上来说，对两个同样准确的数值取平均值是合理的。

我们还可以用卡尔曼增益来表示方差：

$$\begin{aligned}\sigma^2 &= \frac{\bar{\sigma}^2\sigma_z^2}{\bar{\sigma}^2 + \sigma_z^2} \\ &= K\sigma_z^2 \\ &= (1-K)\bar{\sigma}^2\end{aligned}$$

我们可以通过查看这张图表来理解这一点：

```
import kf_book.book_plots as book_plots
book_plots.show_residual_chart()
```



卡尔曼增益 K 其实是一个比例因子，用于确定残差值的大小。这样一来， `update()` 和 `predict()` 的实现方式虽然有所不同，但实际上是等效的。

```
def update(prior, measurement):
    x, P = prior        # mean and variance of prior
    z, R = measurement  # mean and variance of measurement

    y = z - x        # residual
    K = P / (P + R)  # Kalman gain

    x = x + K*y      # posterior
    P = (1 - K) * P  # posterior variance
    return gaussian(x, P)

def predict(posterior, movement):
    x, P = posterior # mean and variance of posterior
    dx, Q = movement # mean and variance of movement
    x = x + dx
    P = P + Q
    return gaussian(x, P)
```

我为什么要用这种形式来表达卡尔曼滤波器呢？又为什么要选择这么糟糕的变量名称呢？其实有几个原因。大多数书籍和论文都是以这种形式来介绍卡尔曼滤波器的。虽然我也可以从贝叶斯原理来推导卡尔曼滤波器，但这种推导方式并不常用。其他推导方式也都会得到与这种形式相同的方程。所以，人们必须学会并理解这种表达方式。

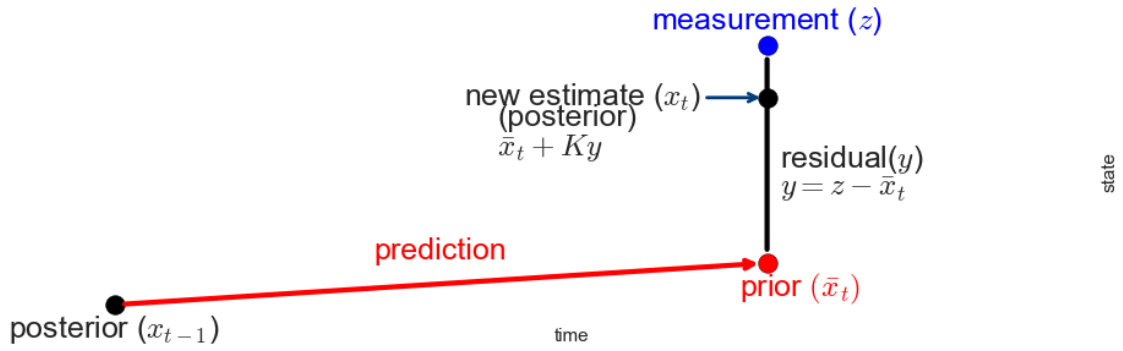
`z`、`P`、`Q` 和 `R` 这些符号是从哪里来的呢？在本书的其余部分中，你还会经常看到这些符号的用法。在相关文献中，R 通常用来表示测量噪声，Q 用来表示过程噪声，而 P 则用来表示状态的方差。虽然并非所有文献都如此，但用 z 来表示测量结果的做法相当普遍。你几乎会在所有书籍和论文中看到这些符号的用法。请尽快熟悉它们吧。

这也是思考滤波问题的一种有效方式。我们正是用这种方式来处理 g-h 滤波器的。其核心思路是：先计算残差 $y = \mu_z - \bar{\mu}$，再以先验信息中的不确定性与测量值 $K = P/(P+R)$ 之比来确定卡尔曼增益，最后通过将 Ky 加到先验信息上来得到后验信息。

在这种表达方式中，贝叶斯方法的本质被忽略了，同样被忽略的还有“我们将概率值与先验概率相乘”这一事实。不过，这两种观点其实是等价的，因为其数学表达方式完全相同。我选择采用贝叶斯方法，是因为我觉得它能让我们更直观、更深入地理解概率推理的原理。而使用 K 的这种表达方式，则有助于我们深入理解所谓的“正交投影法”。卡尔曼博士在发明这种滤波器时，使用的是正交投影法，而非贝叶斯推理。在接下来的几章中，你会对此有更深入的了解。

该算法的完整描述

请回想一下我们用来描述 g-h 滤波器的那个图表：



在这一章中，我们所做的其实都是一样的。卡尔曼滤波器会先进行预测，再获取实际测量值，最后根据这两个数值来得出一个新的估计值。

这一点非常重要：本书中的每一个滤波器都采用了相同的算法，只不过在数学细节上有所区别而已。后面的章节中，相关的数学公式可能会比较复杂，但核心思想其实很容易理解。

重要的是，不要只盯着某个特定滤波器所对应的方程式的细节，而应理解这些方程式究竟在计算什么，以及为何要这样计算。世界上有无数种滤波器，它们虽然都使用不同的数学公式来实现相同的算法，但所选的数学公式会影响到最终的结果质量以及该滤波器能够处理的问题类型。不过，这些数学公式的选择并不会影响滤波器的基本原理。

以下是该算法的通用流程：

初始化

```
1. Initialize the state of the filter
2. Initialize our belief in the state
```

预测

```
1. Use system behavior to predict state at the next time step
2. Adjust belief to account for the uncertainty in prediction
```

更新

```
1. Get a measurement and associated belief about its accuracy
2. Compute residual between estimated state and measurement
3. Compute scaling factor based on whether the measurement
or prediction is more accurate
4. set state between the prediction and measurement based
on scaling factor
5. update belief in the state based on how certain we are
in the measurement
```

很难找到一种不符合这种模式的贝叶斯滤波算法。有些滤波器会忽略某些因素，比如预测误差；而另一些则采用极其复杂的计算方法。但归根结底，所有贝叶斯滤波算法都是如此运作的。

单变量卡尔曼滤波器的方程如下：

预测

Equation	Implementation	Kalman Form
$\bar{x} = x + f_x$	$\bar{\mu} = \mu + \mu_{f_x}$	$\bar{x} = x + dx$
	$\bar{\sigma}^2 = \sigma^2 + \sigma^2_{f_x}$	$\bar{P} = P + Q$

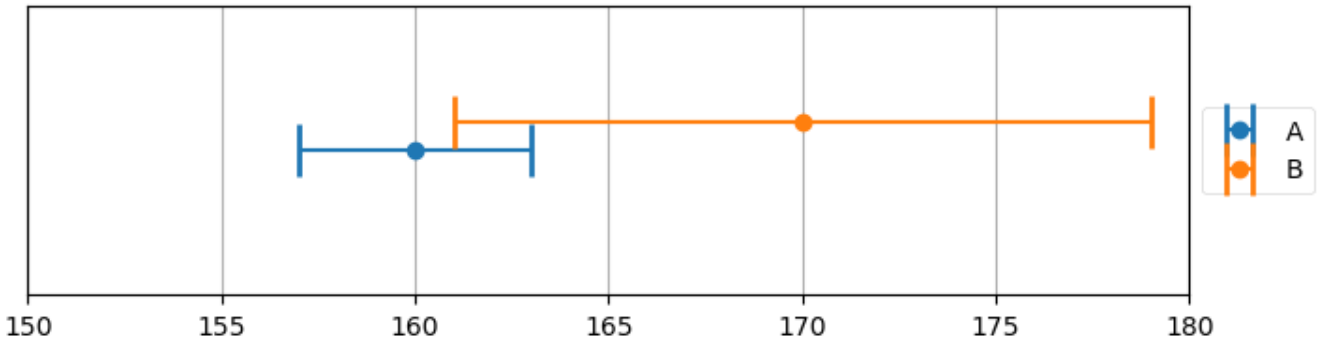
更新

Equation	Implementation	Kalman Form
$x = \|L\bar{x}\|$	$y = z - \bar{\mu}$	$y = z - \bar{x}$
	$K = \frac{\bar{\sigma}^2}{\bar{\sigma}^2 + \sigma_z^2}$	$K = \frac{\bar{P}}{\bar{P} + R}$
	$\mu = \bar{\mu} + Ky$	$x = \bar{x} + Ky$
	$\sigma^2 = \frac{\bar{\sigma}^2\sigma_z^2}{\bar{\sigma}^2 + \sigma_z^2}$	$P = (1 - K)\bar{P}$

与 g-h 滤波器及离散贝叶斯滤波器的比较

现在正是了解这三种滤波器在处理误差方式上的差异的好时机。对于 g-h 滤波器，我们按照下图所示的方式对测量数据进行了处理：

```
book_plots.plot_errorbars([(160, 3, 'A'), (170, 9, 'B')], xlims=(150, 180))
```

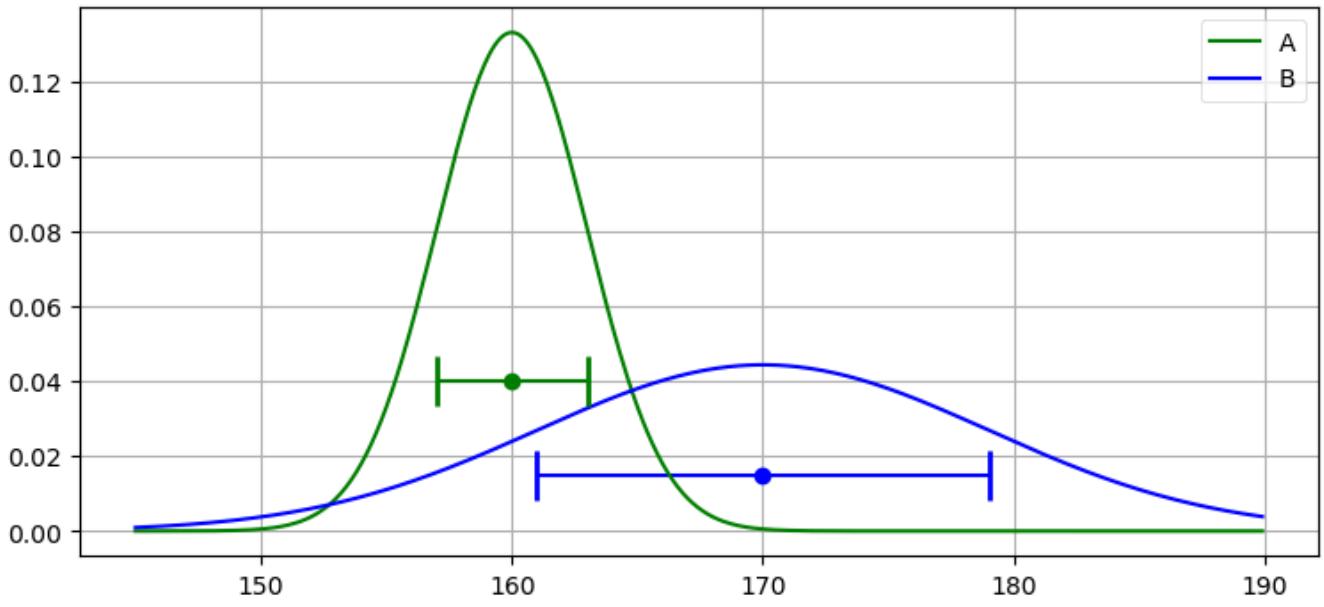


传感器 A 的测量值为 160，传感器 B 的测量值为 170。那些条形图表示的是误差范围——它们反映了测量结果可能存在的误差幅度。因此，传感器 A 的实际测量值应该在 157 到 163 之间，而传感器 B 的实际测量值则应在 161 到 179 之间。

当时我并没有明确给出其定义，但实际上这是一种均匀分布。所谓均匀分布，指的是在该范围内，所有事件被选中的概率都是相等的。根据这一模型，传感器 A 读出 157、160 或 163 的概率都是相同的。而那些超出这个范围的数值，被选中的概率则为 0。

我们可以用高斯函数来模拟这种情况。我用 $N(160, 3^2)$ 表示传感器 A，用 $N(170, 9^2)$ 表示传感器 B。下图中，我还标出了均匀分布的误差范围，以便进行比较。

```
xs = np.arange(145, 190, 0.1)
ys = [stats.gaussian(x, 160, 3**2) for x in xs]
plt.plot(xs, ys, label='A', color='g')

ys = [stats.gaussian(x, 170, 9**2) for x in xs]
plt.plot(xs, ys, label='B', color='b')
plt.legend();
plt.errorbar(160, [0.04], xerr=[3], fmt='o', color='g', capthick=2, capsize=10)
plt.errorbar(170, [0.015], xerr=[9], fmt='o', color='b', capthick=2, capsize=10);
```

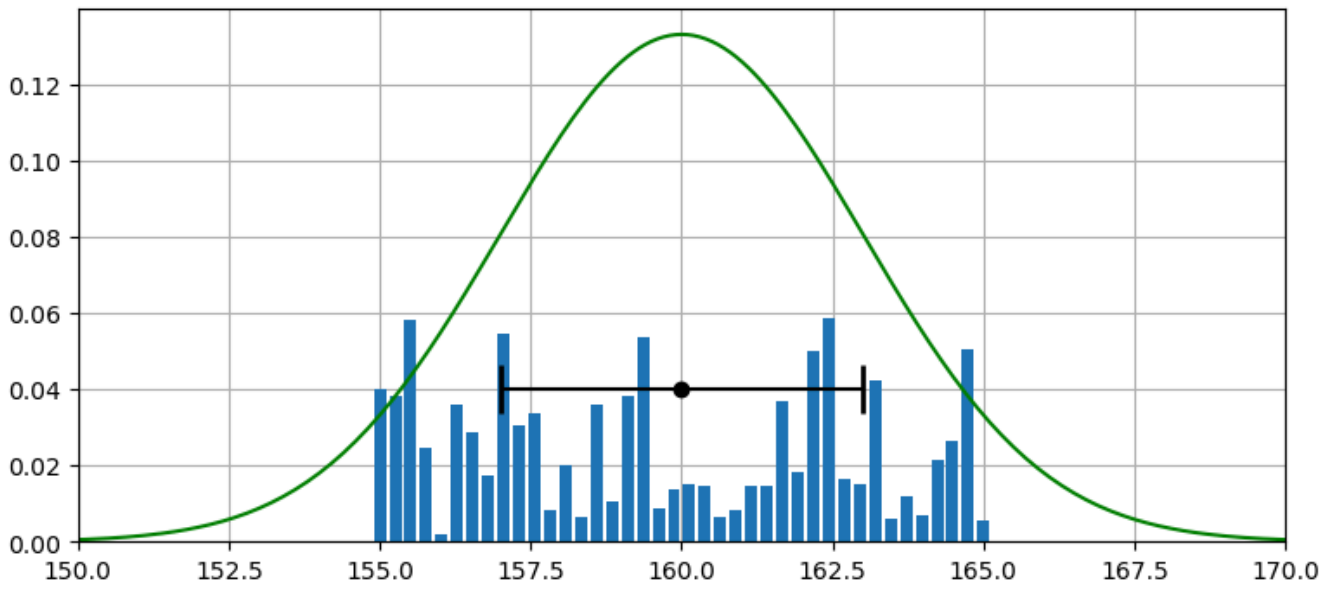


选择使用均匀分布或高斯分布其实是一种建模上的取舍。这两种分布都无法完全反映现实情况。在大多数情况下，高斯分布更符合实际情况。大多数传感器所测得的数值往往接近于被测量的真实值，而很少会测得与真实值相差很大的数值。高斯分布正好能够体现这种趋势。相比之下，均匀分布则假设在某个范围内，任何数值被测得的可能性都是相等的。

现在，让我们来看看在离散贝叶斯滤波器中使用的离散分布。该模型将所有可能的数值范围划分为若干个离散的区间，并为每个区间分配一个概率值。只要这些概率值之和等于 1，这种分配方式完全可以由人为决定。

让我们用均匀分布、高斯分布和离散分布来分别绘制某个传感器的数据曲线吧。

```
from random import random
xs = np.arange(145, 190, 0.1)
ys = [stats.gaussian(x, 160, 3**2) for x in xs]
belief = np.array([random() for _ in range(40)])
belief = belief / sum(belief)

x = np.linspace(155, 165, len(belief))
plt.gca().bar(x, belief, width=0.2)
plt.plot(xs, ys, label='A', color='g')
plt.errorbar(160, [0.04], xerr=[3], fmt='o', color='k', capthick=2, capsize=10)
plt.xlim(150, 170);
```



我使用随机数来构建离散分布，以此说明该模型能够用来模拟任何形式的概率分布。这一特性使得该模型具有极大的实用性。只要拥有足够多的离散区间，我们就能模拟出任何复杂传感器的误差特性。不过，这种强大功能也带来了数学上的复杂性：对高斯分布进行运算时，只需执行两步数学操作，结果依然是高斯分布。这种规律性让我们能够轻松分析滤波器的性能和行为。但对于离散分布来说，情况则完全不同：对其进行运算需要遍历所有数据，而且我们很难准确描述运算结果。基于离散分布来分析滤波器的性能特性，几乎是不可能的。

这里并没有所谓的“正确”选择。在本书的后面部分，我们将介绍使用离散分布的粒子滤波算法。这是一种极其强大的技术，因为它能够处理各种复杂的情形。不过，这种方法的缺点是计算速度较慢，且难以进行数学分析。

目前，我们先忽略这些问题，继续在接下来的几章中使用高斯函数。随着学习的深入，你将会了解在数学模型中使用高斯函数的优点和局限性。

滤波器设计入门

到目前为止，我们已经为位置传感器开发了相应的滤波器。对于这类问题，我们已经很熟悉了；不过，要将其应用到其他类型的问题的处理上，可能就需要使用卡尔曼滤波器了。说实话，目前的描述中还缺少不少信息。接下来的章节会补充这些遗漏的内容。不过，我们不妨先试着为温度计设计并实现一个卡尔曼滤波器，从而了解其原理。温度计的传感器会输出与所测温度相对应的电压值。我们已经查阅了该传感器的制造商规格书，得知该传感器产生的噪声属于白噪声，其标准差为 0.13 伏特。

我们可以利用这个功能来模拟温度传感器的测量结果：

```
def volt(voltage, std):
    return voltage + (randn() * std)
```

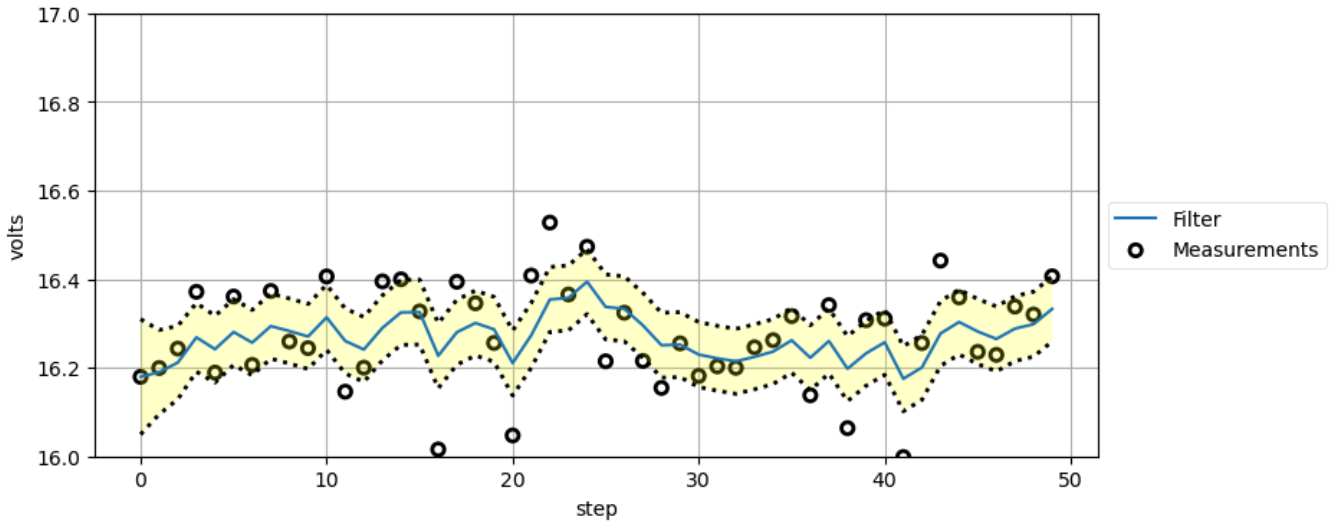
现在，我们需要编写卡尔曼滤波器的处理流程。与之前的问题类似，我们需要进行预测和更新的操作。传感步骤其实很简单：调用 `volt()` 来获取测量结果，然后将结果传递给 `update()` 方法。但是，预测步骤该如何处理呢？我们没有传感器来检测电压中的“变化”，而且，在短时间内，我们预期电压会保持不变。那我们该如何处理这种情况呢？

像往常一样，我们会依赖数学公式来进行预测。由于没有观测到任何温度变化，我们将该数值设为零。不过，这意味着我们预测的是温度将永远保持不变。如果真是如此，那么随着时间的推移，我们对预测结果的信心会越来越高。一旦传感器收集了足够的测量数据，它就能非常准确地预测后续的温度变化，从而忽略那些由实际温度变化所导致的测量结果。这种机制被称为“自满型滤波器”，显然这是需要避免的。因此，我们会在预测过程中加入一定的误差，从而让滤波器不会忽视随时间产生的电压变化。在下面的代码中，我将 `process_var = .05**2` 设置为电压在每个时间步长内的预期变化幅度。我选择这个数值只是为了便于展示在更新和预测过程中，该数值是如何变化的。对于真实的传感器来说，这个数值应设置为实际的电压变化幅度。正如你所预期的那样。例如，如果这是用于测量室内空气温度的温湿度计，那么这个数值会非常小；而如果这是用于测量化学反应室内温度的热电偶，那么数值则会很大。关于如何选择合适的数值，我们将在后面的章节中详细说明。

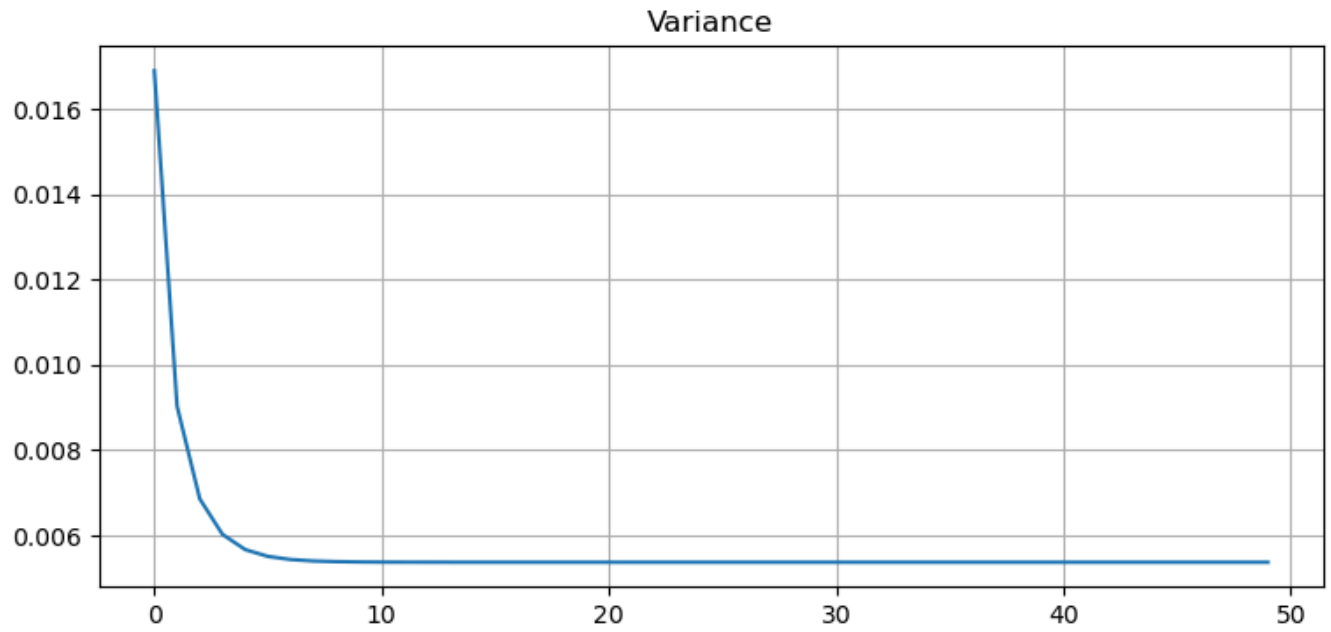
让我们看看会发生什么吧。

```
temp_change = 0
voltage_std = .13
process_var = .05**2
actual_voltage = 16.3

x = gaussian(25., 1000.) # initial state
process_model = gaussian(0., process_var)

N = 50
zs = [volt(actual_voltage, voltage_std) for i in range(N)]
ps = []
estimates = []

for z in zs:
    prior = predict(x, process_model)
```

```
    x = update(prior, gaussian(z, voltage_std**2))

    # save for latter plotting
    estimates.append(x.mean)
    ps.append(x.var)

# plot the filter output and the variance
book_plots.plot_measurements(zs)
book_plots.plot_filter(estimates, var=np.array(ps))
book_plots.show_legend()
plt.ylim(16, 17)
book_plots.set_labels(x='step', y='volts')
plt.show()

plt.plot(ps)
plt.title('Variance')
print(f'Variance converges to {ps[-1]:.3f}')
```




```
Variance converges to 0.005
```



第一张图表展示了各个传感器的测量值与滤波器输出值之间的关系。尽管传感器产生的噪声很大，但我们仍然能够大致确定传感器的电压值。在作者进行测试时，滤波器输出的最后一个电压值为 16.213 ，这一数值与 `volt()` 函数所使用的 16.4 数值非常接近。在其他测试中，得到的数值则有所差异。

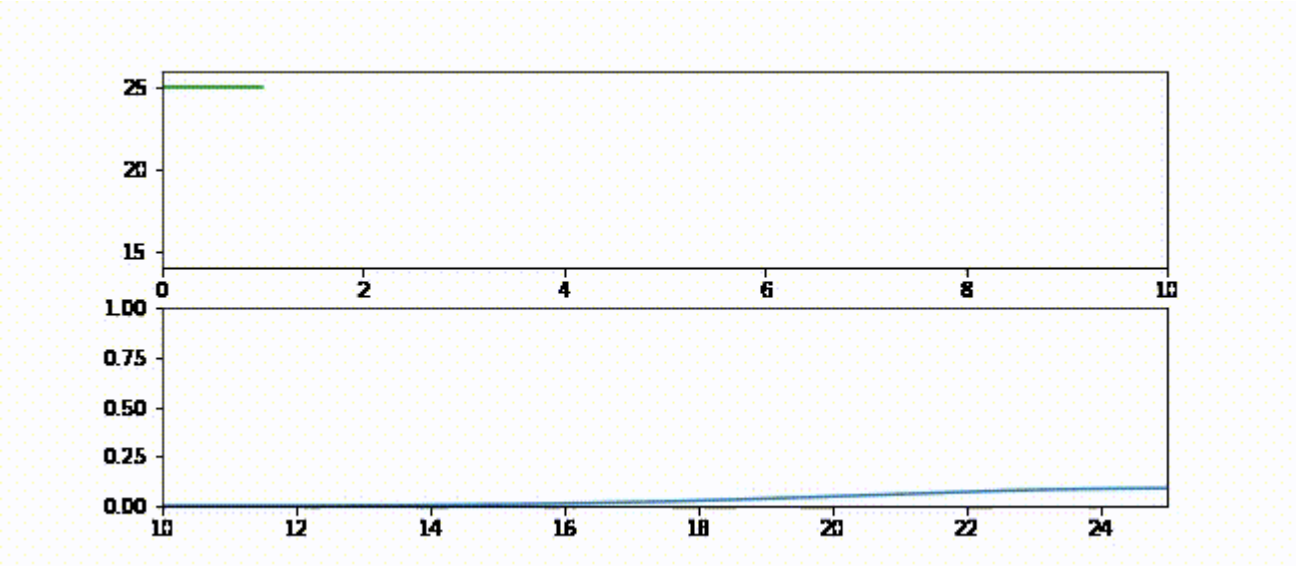
技术规格表顾名思义，就是对产品各项性能的详细说明。由于制造过程中的不可避免的差异，每个传感器的实际性能都会有所不同。所列数值通常为最大值——技术规格表实际上是一种保证，确保产品的性能至少能达到该水平。如果你购买的是昂贵的设备，通常会附有一份详细记录该设备测试结果的表格；这类数据通常相当可靠。不过，如果是一款便宜的传感器，那么在销售之前很可能几乎没有经过任何测试。制造商通常只对少量样品进行测试，以确保其性能符合要求。如果你需要将这款传感器用于关键应用中，就必须仔细阅读技术规格表，弄清楚各项数值的具体含义。这些数值是最大值吗？还是比如 3σ 错误率的数值呢？所有的产品都经过测试了吗？该方差是否服从正态分布，还是属于其他某种分布呢？此外，制造过程并不完美。你收到的产品可能存在缺陷，其性能可能不符合说明书上的描述。

例如，我正在查看一款气流传感器的数据手册。其中有一个名为“重复性”的参数，其数值为 $\pm 0.50\%$ 。这个数值是否服从高斯分布呢？是否存在偏差呢？比如，低温时，重复性可能接近 0.0% ；而高温时则始终接近 $+0.50\%$ 。电气元件的数据手册中通常会有一节名为“典型性能特性”的内容。这些内容用于描述那些无法通过表格轻易呈现的信息。例如，我看到的这张图表展示了 LM555 定时器的输出电压与电流之间的关系。图表中有三条曲线，分别对应不同温度下的性能表现。理想情况下，输出电压应呈线性关系，但实际上这三条曲线都是弯曲的。这说明，该芯片的输出电压误差很可能不服从高斯分布——在高温环境下，输出电压会降低；而当输入电流很大时，输出电压的非线性程度也很高。

正如你可能猜到的那样，对传感器的性能进行建模，是构建一个性能良好的卡尔曼滤波器过程中较为困难的部分之一。

动画

对于那些在浏览器中阅读此内容的人士，这里有一个动画，展示了该滤波器的工作原理。如果您不是通过浏览器来查看，请访问 https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/animations/05_volt_animate.gif 来查看该动画。



在动画中，最上方的图表首先用绿色线条表示预测出的下一电压值，然后用红色“+”号表示实际测量值。接着，用浅红色线条表示误差值，最后用蓝色线条表示滤波器的输出值。可以看出，当滤波器开始进行修正时，修正幅度相当大；但经过几次调整后，即使测量值与预测值相差很大，滤波器对输出值的调整幅度也变得很小了。

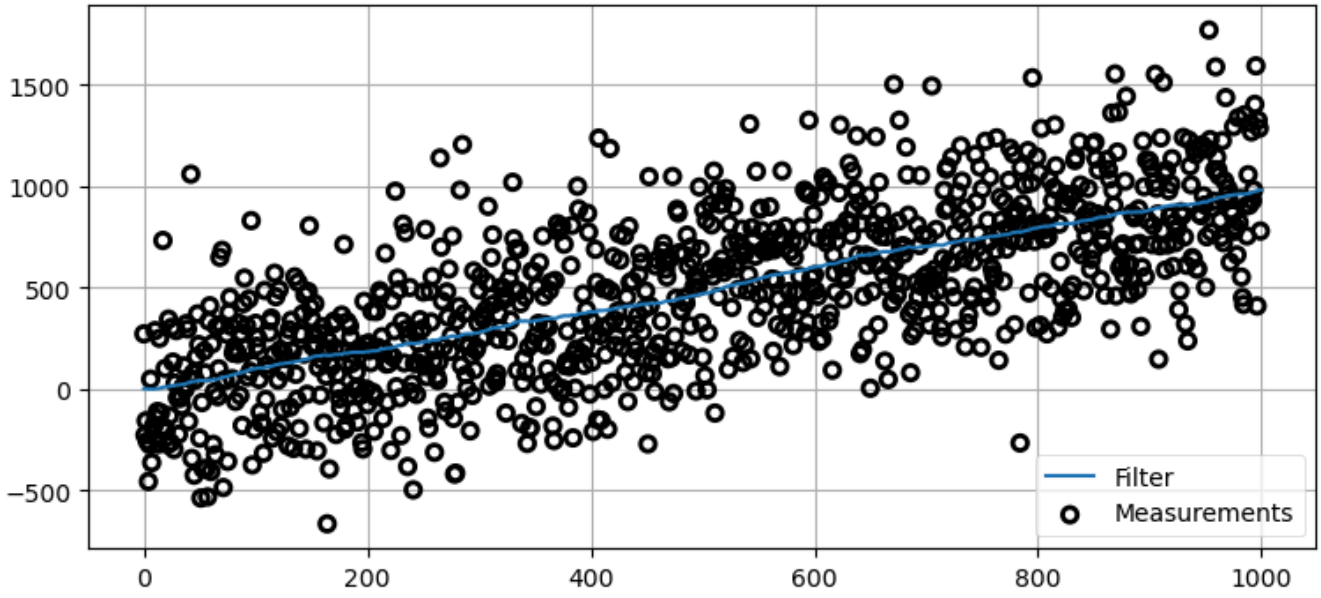
下图展示了随着滤波器不断更新，高斯分布形态的变化情况。在滤波器开始工作时，高斯曲线的中心位于 25 处，这正是我们对电压的初始估计值。由于初始时存在不确定性，高斯曲线既宽又短。但随着滤波器不断更新，高斯曲线的中心迅速移至 16.0 左右，同时曲线也变得更陡峭，这反映出滤波器对电压估计值的信心越来越强。另外，您还会注意到高斯曲线的高度会有些波动。仔细观察会发现，在预测阶段，高斯曲线会变得更短且更分散；而当滤波器再次获取新的测量数据后，高斯曲线又会变陡峭且更窄。

可以将这个动画理解为基于 g-h 滤波器的处理过程。在每个步骤中，g-h 滤波器会先进行预测，然后进行测量，接着计算出预测值与实际测量值之间的差异。之后，根据某个缩放因子来选择残差曲线上的一个点。卡尔曼滤波器其实做的是完全相同的事情，只不过缩放因子会随时间而变化。当滤波器对当前状态的置信度越高时，该缩放因子就会更倾向于采用滤波器的预测结果，而非实际测量值。

示例：极高的噪音水平

在使用了“狗形滤波器”之后，我并没有在信号中加入太多噪声。我推测，那只狗的位置应该为 0。那么，这种滤波器在现实环境中的表现如何呢？我会先在 RFID 传感器中加入更多的噪声，同时保持过程方差为 2 米。我会加入极其大量的噪声——多到几乎淹没真实的测量结果。如果传感器的标准差为 300 米，那么你对这种滤波器的性能有什么直觉上的判断呢？换句话说，实际位置为 1.0 米的数据，可能会被记录为 287.9 米、-589.6 米，或者该范围内的其他数值。请在继续阅读之前先思考一下。

```
sensor_var = 300.**2
process_var = 2.
process_model = gaussian(1., process_var)
pos = gaussian(0., 500.)
N = 1000
dog = DogSimulation(pos.mean, 1., sensor_var, process_var)
zs = [dog.move_and_sense() for _ in range(N)]
ps = []

for i in range(N):
    prior = predict(pos, process_model)
    pos = update(prior, gaussian(zs[i], sensor_var))
    ps.append(pos.mean)

book_plots.plot_measurements(zs, lw=1)
book_plots.plot_filter(ps)
plt.legend(loc=4);
```



在这个例子中，噪声非常大，但滤波器处理后的输出却几乎呈直线状！这一结果真是令人惊讶！你认为造成这种现象的原因可能是什么？

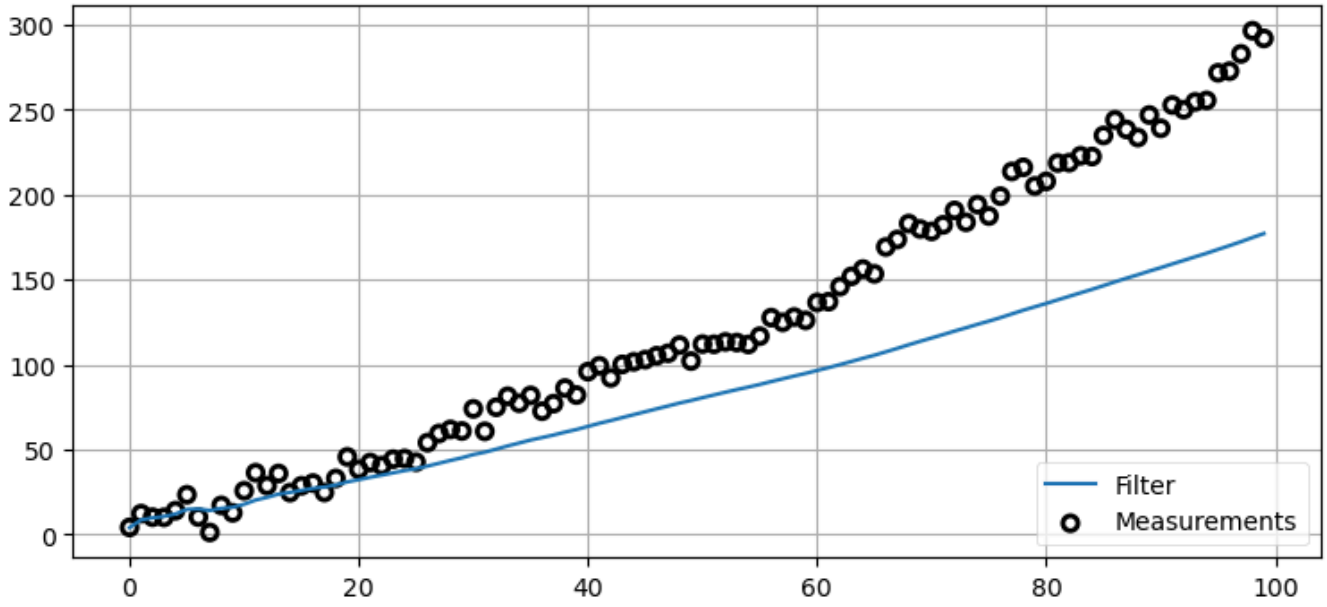
由于我们的工艺误差很小，因此得到的曲线几乎是一条直线。较小的工艺误差意味着预测结果非常可靠，而预测结果本身又呈直线形态，因此滤波器输出的结果也几乎是一条直线。

示例：不正确的过程偏差

最后一个滤镜的效果真是棒极了！我们当然应该把处理过程中的误差降到最低，这样才能确保最终的结果既平滑又完美。

过程方差告诉滤波器：系统随着时间的推移发生了多大的变化。如果你人为地将这个数值设置得过低，从而欺骗了滤波器，那么滤波器就无法对实际发生的变化做出反应。让我们让狗在每个时间步长上都将速度增加一个小数值，然后看看当过程方差为 0.001 米/秒时，滤波器的表现如何。

```
sensor_var = 20.
process_var = .001
process_model = gaussian(1., process_var)
pos = gaussian(0., 500.)
N = 100
dog = DogSimulation(pos.mean, 1, sensor_var, process_var*10000)
zs, ps = [], []
for _ in range(N):
    dog.velocity += 0.04
    zs.append(dog.move_and_sense())

for z in zs:
    prior = predict(pos, process_model)
    pos = update(prior, gaussian(z, sensor_var))
    ps.append(pos.mean)

book_plots.plot_measurements(zs, lw=1)
book_plots.plot_filter(ps)
plt.legend(loc=4);
```

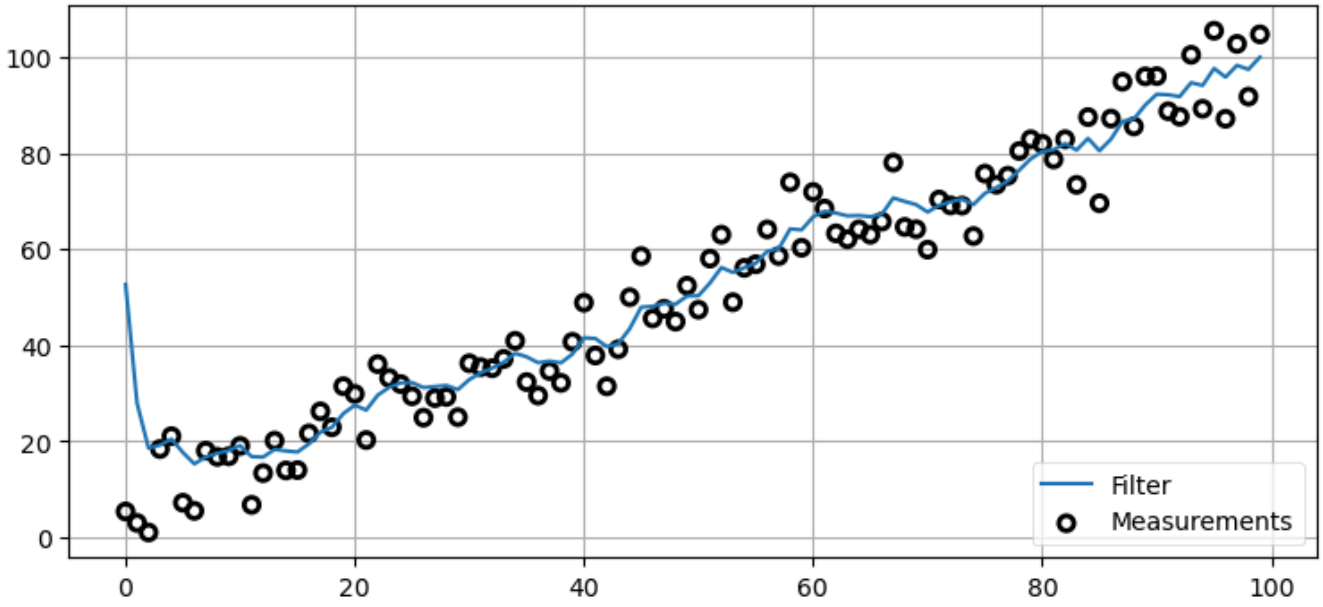


显然，该滤波器并没有正确地响应各项测量数据。测量结果清楚地表明狗的速度正在发生变化，但滤波器却认为自己的预测结果已经非常准确了，因此几乎完全忽略了这些测量数据。我建议您调整一下与狗的运动相关的参数，以减少误差。我们将在后面的章节中进一步探讨这个话题。关键在于要明白：数学模型要求各种方差能够准确反映系统的实际情况。该滤波器没有意识到自己与实际测量结果之间存在偏差，也无法自行进行调整。它根据先前的预测值和测量结果来计算卡尔曼增益，然后选择更准确的数值作为最终估计结果。

示例：不准确的初步估算

现在，让我们来看看当我们对位置的初始估计有误时，会产生怎样的结果。为避免影响结果的准确性，我会将传感器的误差范围设定为 30，同时将初始位置设置为 1000 米。那么，滤波器能否纠正 1000 米的误差呢?

```
sensor_var = 5.**2
process_var = 2.
pos = gaussian(1000., 500.)
process_model = gaussian(1., process_var)
N = 100
dog = DogSimulation(0, 1, sensor_var, process_var)
zs = [dog.move_and_sense() for _ in range(N)]
ps = []

for z in zs:
    prior = predict(pos, process_model)
    pos = update(prior, gaussian(z, sensor_var))
    ps.append(pos.mean)

book_plots.plot_measurements(zs, lw=1)
book_plots.plot_filter(ps)
plt.legend(loc=4);
```

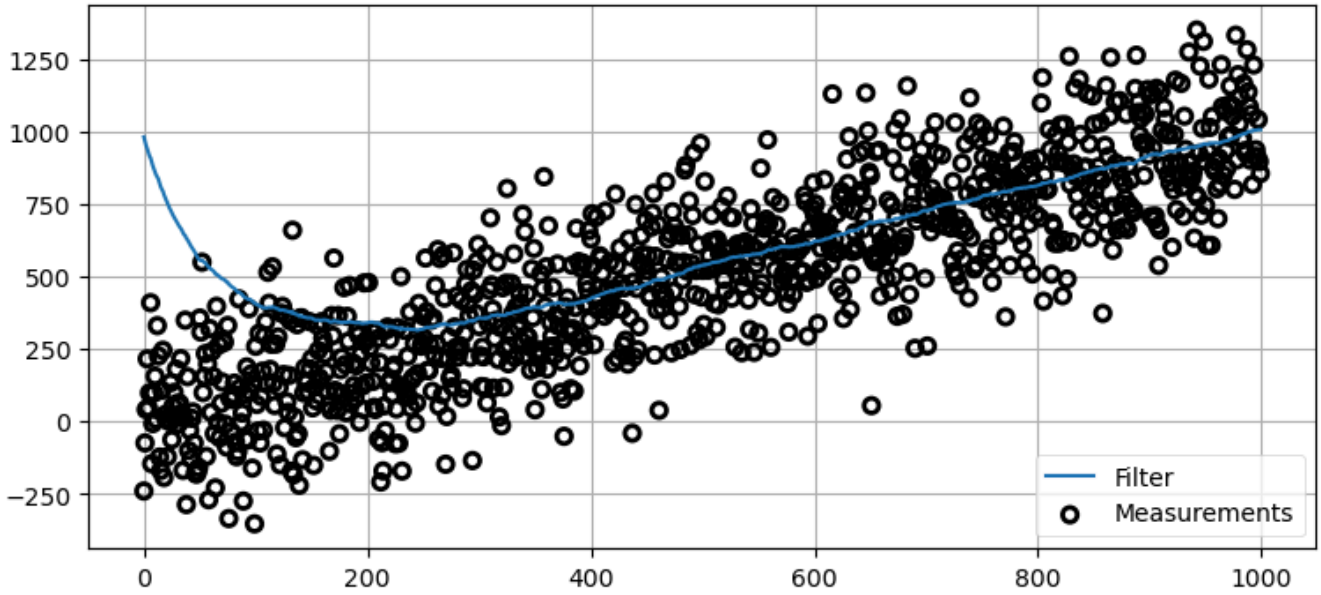


答案依然是肯定的！因为，仅仅在第一步之后，我们对传感器的测量结果就相当有信心了：原本估计的距离为 1000 米，之后则降至了大约 50 米。再经过 5 到 10 次测量后，我们就能得到准确的数值。这样一来，我们就解决了初始值选取的问题。实际上，我们通常会直接将传感器的第一次测量结果作为初始值。不过，即便我们最初给出了完全错误的估计，也没关系——只要卡尔曼滤波器的方差设置得当，以匹配实际的过程和测量误差，滤波器最终仍能得到正确的结果。

示例：巨大的噪声以及不准确的初始估计值

那如果同时出现最糟糕的情况呢：既有巨大的噪音，又有不准确的初始估计值？

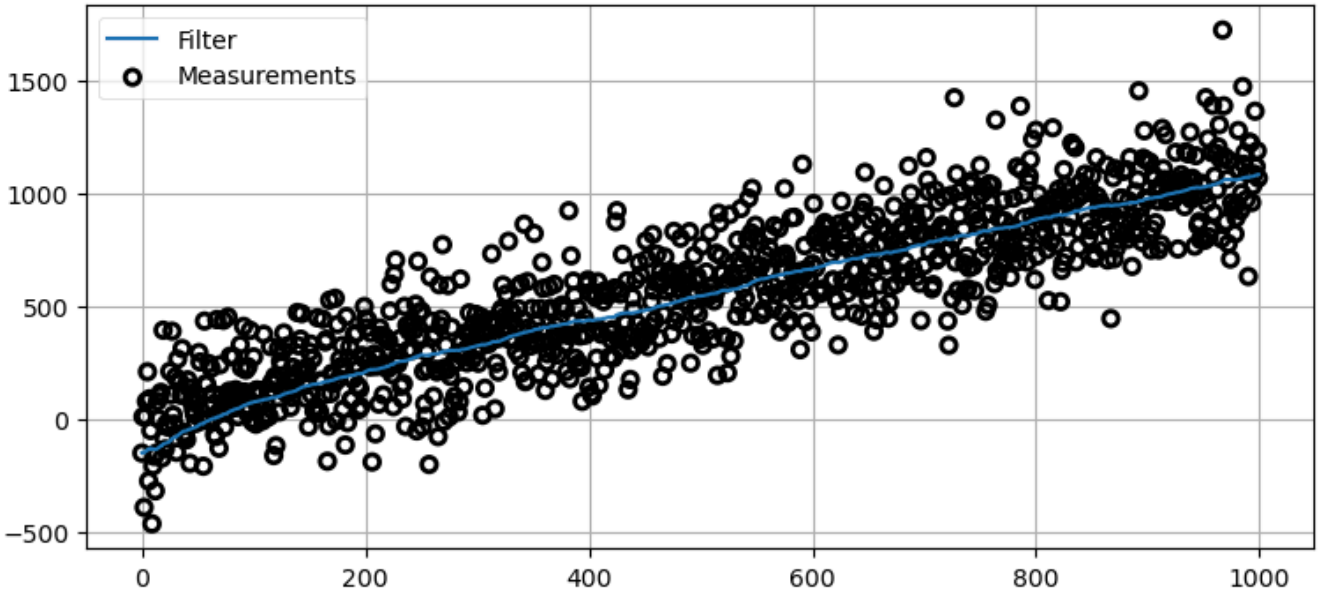
```
sensor_var = 30000.
process_var = 2.
pos = gaussian(1000., 500.)
process_model = gaussian(1., process_var)

N = 1000
dog = DogSimulation(0, 1, sensor_var, process_var)
zs = [dog.move_and_sense() for _ in range(N)]
ps = []

for z in zs:
    prior = predict(pos, process_model)
    pos = update(prior, gaussian(z, sensor_var))
    ps.append(pos.mean)

book_plots.plot_measurements(zs, lw=1)
book_plots.plot_filter(ps)
plt.legend(loc=4);
```



这次，滤波器的处理效果不太理想。请注意，之前的例子中只进行了 100 次更新，而这个例子则进行了 1000 次更新。在我看来，滤波器需要大约 400 次迭代才能达到不错的准确度；不过，要想获得理想的结果，可能需要超过 600 次迭代。卡尔曼滤波器确实不错，但我们不能指望它有奇迹般的效果。如果数据中的噪声非常大，或者初始条件极其糟糕，那这就是目前所能达到的最佳效果了。

最后，让我们按照建议，将第一次测量得到的数值作为初始位置吧。

```
sensor_var = 30000.
process_var = 2.
process_model = gaussian(1., process_var)
N = 1000
dog = DogSimulation(0, 1, sensor_var, process_var)
zs = [dog.move_and_sense() for _ in range(N)]

pos = gaussian(zs[0], 500.)
ps = []
for z in zs:
    prior = predict(pos, process_model)
    pos = update(prior, gaussian(z, sensor_var))
    ps.append(pos.mean)

book_plots.plot_measurements(zs, lw=1)
book_plots.plot_filter(ps)
plt.legend(loc='best');
```



这一简单的改动显著提升了算法的运行效果。在某些情况下，需要大约 200 次迭代才能得到满意的解决方案；而在其他情况下，算法则能很快收敛到正确结果。这一切都取决于第一次测量时存在的噪声程度。如果噪声较大，那么初始估计值就会与狗的实际位置相差甚远。

200 次迭代看起来似乎很多，但实际上，我们所引入的误差幅度实在巨大。在现实世界中，我们使用的传感器有温度计、激光测距仪、GPS 卫星、计算机视觉系统等等。这些传感器的误差远远没有这么大。对于一款便宜的温度计来说，其合理的误差范围应该是 0.2 摄氏度左右，而我们的代码却导致了 30,000 摄氏度的巨大误差。

练习：交互式图表

利用 Jupyter Notebook 的动画功能来实现卡尔曼滤波器。这样，你就可以通过滑块实时调整各种参数值。具体操作方法请参考“高斯分布”章节中的“交互式高斯函数”部分。你需要使用 `interact()` 函数来调用相关的计算和绘图功能。传递给 `interact()` 的每个参数都会自动对应一个滑块。我已经编写好了相关的模板代码，你需要自行填写所需的代码即可。

```
from ipywidgets import interact
from kf_book.book_plots import FloatSlider

def plot_kalman_filter(start_pos,
                       sensor_noise,
                       velocity,
                       process_noise):
    plt.figure();
    # your code goes here

interact(plot_kalman_filter,
         start_pos=(-10, 10),
         sensor_noise=FloatSlider(value=5, min=0, max=100),
         velocity=FloatSlider(value=1, min=-2., max=2.),
         process_noise=FloatSlider(value=5, min=0, max=100.));
```

```
interactive(children=(IntSlider(value=0, description='start_pos', max=10, min=-10),
FloatSlider(value=5.0, con…
```

解决方案

以下是一种可能的解决方案。我们可以使用滑块来调整起始位置、传感器中的噪声水平、每个时间步长内的移动量以及移动过程中的误差大小。其中，“处理噪声”这个参数比较难以理解——它实际上描述了狗在每个时间步长内偏离预定路径的程度。因此，我们需要将这一数值加到狗在每个时间步长后的位置上。我设置了随机数生成器的种子，这样每次重新计算时都会使用相同的随机数，从而便于我们比较在不同滑块设置下得到的结果。

```
from numpy.random import seed
from ipywidgets import interact

def plot_kalman_filter(start_pos,
                       sensor_noise,
                       velocity,
                       process_noise):
    N = 20
    zs, ps = [], []
    seed(303)
    dog = DogSimulation(start_pos, velocity, sensor_noise, process_noise)
    zs = [dog.move_and_sense() for _ in range(N)]
    pos = gaussian(0., 1000.) # mean and variance
    process_model = gaussian(velocity, process_noise)

    for z in zs:
        pos = predict(pos, process_model)
        pos = update(pos, gaussian(z, sensor_noise))
        ps.append(pos.mean)

    plt.figure()
    plt.plot(zs, c='k', marker='o', linestyle='', label='measurement')
    plt.plot(ps, c='#004080', alpha=0.7, label='filter')
    plt.legend(loc=4);
    plt.show()

interact(plot_kalman_filter,
         start_pos=(-10, 10),
         sensor_noise=FloatSlider(value=5, min=0., max=100),
         velocity=FloatSlider(value=1, min=-2., max=2.),
         process_noise=FloatSlider(value=.1, min=0, max=40));
```

```
interactive(children=(IntSlider(value=0, description='start_pos', max=10, min=-10),
FloatSlider(value=5.0, con…
```

练习——非线性系统

卡尔曼滤波器的方程都是线性的：

$$
\begin{aligned}
N(\bar{\mu}, \bar{\sigma}^2) &= N(\mu, \sigma^2) + N(\mu_{move}, \sigma^2_{move}) \\
N(\mu, \sigma^2) &= N(\bar{\mu}, \bar{\sigma}^2) \times N(\mu_z, \sigma^2_z)
\end{aligned}
$$

你认为，这种滤波器在处理非线性系统时，效果如何呢？效果好还是差呢？

实现一个卡尔曼滤波器，该滤波器使用以下公式来生成测量值。

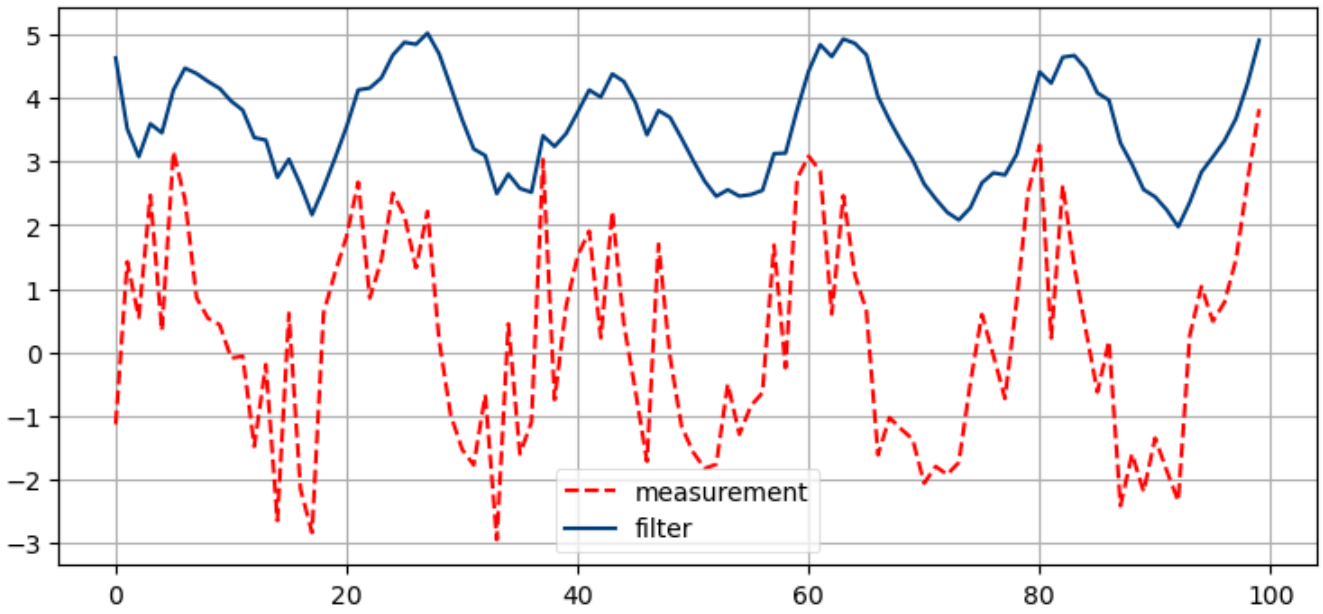
```
for i in range(100):
    z = math.sin(i/3.) * 2
```

调整方差和初始值，看看会产生什么效果。比如，如果初始猜测非常糟糕，会得到怎样的结果呢？

```
#enter your code here.
```

解决方案

```
import math

sensor_var = 30.
process_var = 2.
pos = gaussian(100., 500.)
process_model = gaussian(1., process_var)

zs, ps = [], []

for i in range(100):
    pos = predict(pos, process_model)

    z = math.sin(i/3.)*2 + randn()*1.2
    zs.append(z)

    pos = update(pos, gaussian(z, sensor_var))
    ps.append(pos.mean)

plt.plot(zs, c='r', linestyle='dashed', label='measurement')
plt.plot(ps, c='#004080', label='filter')
plt.legend(loc='best');
```



讨论

这太糟糕了！输出信号根本不像是正弦波，其形态与正弦波相差甚远。在线性系统中，我们可以在信号中加入大量噪声，但仍然能得到相当准确的结果。而在这里，哪怕是少量的噪声也会导致极其糟糕的结果。

如果我们回顾一下 g-h 滤波器的相关内容，就能理解这里发生的情况。g-h 滤波器的结构决定了：其输出值必然介于预测值和测量值之间。像这样的变化信号始终处于加速状态，而我们的过程模型则假设信号始终以恒定速度运动。因此，从数学上来说，滤波器的输出值必然总是滞后于输入信号。

在从业者开始使用卡尔曼滤波器后不久，他们就发现该滤波器在处理非线性系统时效果不佳。于是，他们开始寻求解决这一问题的方法。后面的章节将专门探讨这个问题。

固定增益滤波器

嵌入式计算机的处理器性能通常极为有限。许多嵌入式计算机甚至没有浮点运算功能。这些简单的计算公式可能会给芯片带来很大的负担。不过，随着技术的进步，这种情况有所改善。不过，千万不要低估了这样一个事实：当需要购买数百万个处理器时，每个处理器多节省一美元的成本也是相当可观的。

在上面的例子中，滤波器的方差最终会收敛到一个固定值。只要测量值和过程中的方差保持恒定，这种情况必然会发生。你可以利用这一特性，通过模拟来确定方差最终会收敛到什么数值。然后，你可以将该数值直接写入滤波器中。只要给滤波器一个合理的初始值（我建议使用第一次测量得到的数值作为初始值），滤波器就能发挥出很好的性能。例如，狗的跟踪滤波器就可以简化为这样的形式：

```
def update(x, z):
    K = .13232  # experimentally derived Kalman gain
    y = z - x   # residual
    x = x + K*y # posterior
    return x

def predict(x):
    return x + vel*dt
```

我使用了卡尔曼增益这一更新公式，以此强调：我们完全不必考虑方差的问题。如果各方差都收敛到同一个数值，那么卡尔曼增益也会相应地收敛到那个数值。

FilterPy 的实现方式

FilterPy 实现了 `predict()` 和 `update()` 功能。这些功能不仅适用于本章所讨论的单变量情况，也适用于我们在后续章节中将要学习的多元变量情况。正因如此，它们的接口略有不同。FilterPy 不会将高斯分布参数作为元组来处理，而是将它们视为两个独立的变量来处理。

`predict()` 需要多个参数，但我们只需要使用其中的四个即可：

```
predict(x, P, u, Q)
```

x 代表系统的状态。 P 代表系统的方差。 u 代表由于该过程而产生的变动，而 Q 则代表该过程中的噪声。在调用 predict() 时，必须使用带名称的参数，因为大多数参数都是可选的。 predict() 的第三个参数并非 u 。

这些名称听起来实在糟糕透顶。确实如此！正如我之前所说，这些名称源自控制理论的悠久历史。你读过的每一篇论文或每一本书都会使用这些名称。所以，我们只能习惯它们。如果拒绝记住这些名称，那你将永远无法理解那些学术文献的内容。

让我们试着用状态 $N(10,3)$ 和移动方式 $N(1,4)$ 来试试看。预计最终位置会是 11（10+1），方差则为 7（3+4）。

```
import filterpy.kalman as kf
kf.predict(x=10., P=3., u=1., Q=4.)
```

```
(11.0, 7.0)
```

update 也接受多个参数，但目前你只需关注以下四个参数即可：

```
update(x, P, z, R)
```

与之前一样， x 和 P 分别代表系统的状态和方差。 z 代表测量结果， R 则代表测量结果的方差。让我们先执行最后的预测操作，以得到先验信息，然后再进行更新。

```
x, P = kf.predict(x=10., P=3., u=1., Q=2.**2)
print(f'{x:.3f}')

x, P = kf.update(x=x, P=P, z=12., R=3.5**2)
print(f'{x:.3f} {P:.3f}')
```

```
11.000
11.364 4.455
```

我进行了多次测量，但结果波动很大。因此，最终的估计值仍然接近之前的 11 这个数值。

最后一点。在预测结果的输出上，我没有使用变量名 prior 。在本书的其余部分，我也不会使用这个变量名。卡尔曼滤波方程中使用的变量名是 x 。所谓“先验值”和“后验值”，其实都是对系统状态的估计值：前者是在考虑测量结果之前的估计值，后者则是在考虑了测量结果之后的估计值。

摘要/总结

本章所介绍的卡尔曼滤波器，其实是我们接下来要学习的更通用滤波器的一种特殊形式。大多数教材都不会详细讨论这种一维形式的滤波器。不过，我认为它是一个非常重要的基础。我们在本书的开头学习了 g-h 滤波器，接着又实现了离散贝叶斯滤波器，现在则实现了这一维卡尔曼滤波器。我试图向大家说明：这些滤波器其实都使用着相同的算法和原理。我们稍后要学习的卡尔曼滤波器的数学原理相当复杂，因此很难理解其背后的简单性。不过，这种复杂性也带来了巨大的优势：这种通用型滤波器的性能将远远优于本章所介绍的那些滤波器。

这一章的内容比较难以理解。要想真正掌握它，你可能需要反复阅读几遍。我建议你尝试更改代码中的各种参数，然后观察相应的结果。请相信：高斯分布确实是一种很好的方式，用来描述关于狗在走廊中的位置、飞机在空中的位置或化学反应室内的温度等信息的“单峰性信念”。同时，请相信：将多个高斯分布相加，确实能够根据先前的信念和新的测量结果，得出新的信念值。最后，请相信：如果你正在测量物体的运动情况，那么将各个高斯分布相加，就能更新你的信念值。

最重要的是，务必花足够的时间仔细阅读“算法的详细描述”部分，确保自己理解了该算法的原理，以及它与 g-h 滤波器和离散贝叶斯滤波器之间的关系。这里有个“技巧”：需要选择一个介于预测值和测量值之间的数值。虽然各种算法在具体计算方式上有所不同，但所遵循的逻辑其实是一样的。

第 5 章：多元高斯分布

多维度下的不确定性建模

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

上一章中介绍的各种技术虽然非常有效，但它们只能处理单个变量或维度的数据。这些技术无法用来表示多维数据，比如一只狗在田野中的位置和速度。位置和速度之间存在着相互关联的关系。正如我们在 g-h 章节中学到的，我们绝不能丢弃任何有用的信息。在这一章中，我们将学习如何用概率论的方式来描述这种相互关系。通过这一重要思路，我们将能够显著提升滤波器的性能。

多元正态分布

对于标量随机变量，我们一直使用高斯分布来描述，其数学表达式为 $N(\mu, \sigma^2)$ 。更正式的称谓是“单变量正态分布”，其中“单变量”指的是只有一个变量。高斯分布所对应的概率分布，就被称为单变量正态分布。

什么是多元正态分布呢？所谓“多元”，指的是多个变量。我们的目标是能够用多维的方式来表示正态分布。这里所说的“维度”并不一定指空间维度——比如，如果我们记录一架飞机在(x, y, z)坐标系中的位置、速度和加速度，那就构成了一个九维的模型。以二维情况为例：可以是机器人的 x 和 y 坐标，也可以是狗在 x 轴上的位置和速度，又或者是奶牛场的牛奶产量和饲料消耗量。其实具体是什么变量并不重要。我们可以看出，对于 n 维的情况，我们需要 n 个均值，这些均值可以被排列成一个列矩阵或向量，如下所示：

$$\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_n \end{bmatrix}$$

假设我们相信 $x = 2$ 和 $y = 17$ 的说法。那么我们就会……

$$\mu = \begin{bmatrix} 2 \\ 17 \end{bmatrix}$$

下一步是表示各个变量的方差。乍看之下，我们可能会认为，对于 N 个维度来说，需要 N 个方差值。比如，我们可以说 x 的方差是 10，y 的方差是 4。

$$\sigma^2 = \begin{bmatrix} 10 \\ 4 \end{bmatrix}$$

这个分析并不完整，因为它没有考虑到更一般的情况。在高斯分布的相关内容中，我们计算了学生身高的方差。方差实际上反映了学生们身高之间的差异程度。如果所有学生的身高都相同，那么方差为 0；而如果学生们的身高差异很大，那么方差就会很大。

身高和体重之间也存在一定的关系。一般来说，身高较高的人，体重也相对较重。身高和体重是相互关联的。我们希望找到一种方式来描述身高和体重的差异程度，以及它们之间的关联强度。换句话说，我们想知道体重是如何随着身高的变化而变化的。我们把这种关系称为协方差。

在理解多元正态分布之前，我们有必要先掌握与相关系数和协方差相关的数学知识。

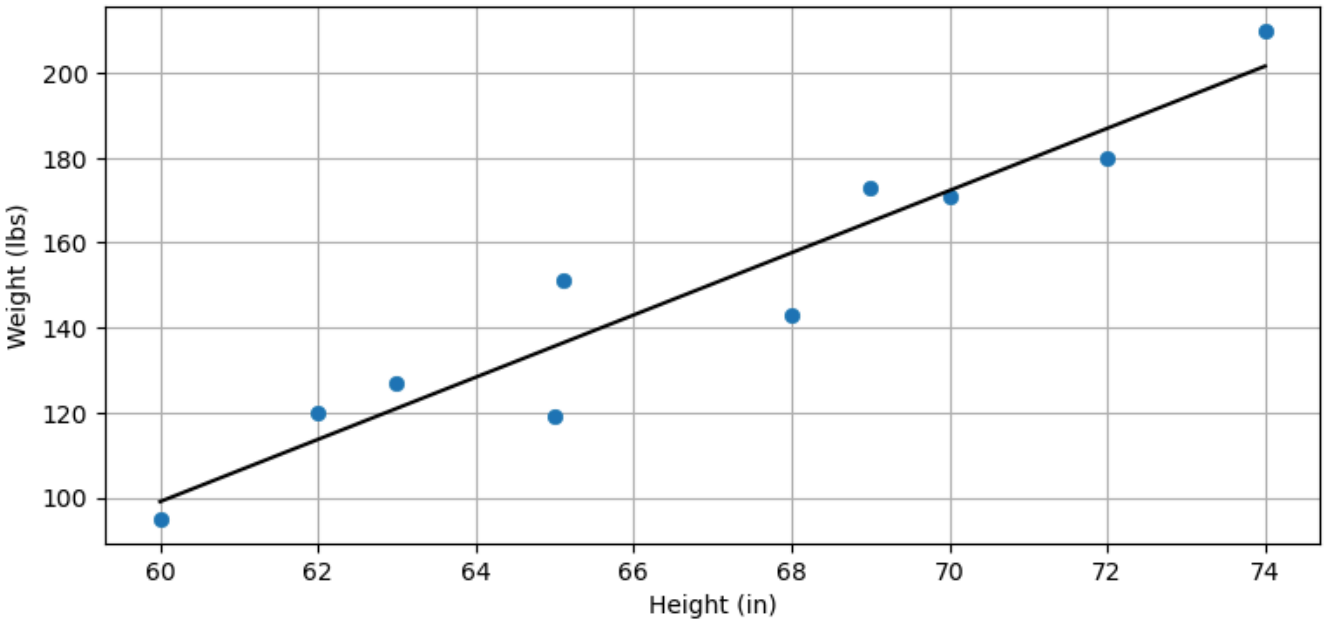
相关性与协方差

协方差用来描述两个变量相互变化的程度。所谓协方差，其实就是“相关方差”的简称。换句话说，方差衡量的是整个数据群体内部的变动情况，而协方差则衡量的是两个变量之间相互变化的程度。例如，身高增加时，体重通常也会增加。这两个变量是相关的，且属于正相关关系——因为其中一个变量增大时，另一个变量也会随之增大。再比如，室外温度下降时，家庭取暖费用会上升。这两个变量则是负相关关系——因为其中一个变量增大时，另一个变量则减小。至于茶的价格和我的狗摇尾巴的次数，它们之间没有任何关联，我们可以说它们是不相关的或独立的——每个变量的变化都不受另一个变量的影响。

相关性有助于进行预测。如果你比我高很多，那么我可以推测你的体重也比我重。随着冬天的到来，我预计自己需要花费更多钱来供暖。不过，如果我的狗摇尾巴的频率增加了，我并不会因此就认为茶叶的价格会发生变化。

例如，下图显示了该校田径队中各学生的身高和体重情况。如果某学生的身高为 68 英寸，那么可以推测他的体重约为 160 磅。不过，由于两者之间的关联并非完全精确，因此我的预测也未必完全准确。

```
from kf_book.gaussian_internal import plot_correlated_data

height = [60, 62, 63, 65, 65.1, 68, 69, 70, 72, 74]
weight = [95, 120, 127, 119, 151, 143, 173, 171, 180, 210]
plot_correlated_data(height, weight, 'Height (in)', 'Weight (lbs)', False)
```



在这本书中，我们只考虑线性相关性。我们假设各变量之间的关系是线性的。也就是说，用直线来描述这些数据是最合适的。我在上图中用直线来拟合了这些数据。虽然也存在非线性相关性的情况，但我们在本书中不会涉及这一概念。

X 与 Y 之间的协方差公式为

$$COV(X,Y) = \sigma_{xy} = E\left[(X-\mu_x)(Y-\mu_y)\right]$$

其中， $E[X]$ 表示 X 的期望值，其定义如下：

$$E[X] = \begin{cases} \sum_{i=1}^{n} p_i x_i & \text{discrete} \\ \int_{-\infty}^{\infty} f(x)xdx & \text{continuous} \end{cases}$$

我们假设每个数据点出现的可能性都是相等的，因此每个数据点被选中的概率都是 $\frac{1}{N}$ 。

$$E[X] = \frac{1}{N}\sum_{i=1}^{n} x_i$$

对于我们即将考虑的离散情况而言。

将协方差公式与方差公式进行比较。可以看出，两者非常相似：

$$\begin{aligned} VAR(X) = \sigma_x^2 &= E[(X-\mu)^2] \\ COV(X,Y) = \sigma_{xy} &= E\left[(X-\mu_x)(Y-\mu_y)\right] \end{aligned}$$

具体来说，如果你计算 $COV(X,X)$ ，就能得到 $VAR(X)$ 的公式。这一结果进一步印证了我的观点：方差实际上反映了随机变量自身的变动情况。

我们用协方差矩阵来表示多元正态分布中的各变量之间的协方差关系。其形式如下：

对角线上的数值表示各个变量的方差，而对角线以外的数值则表示第 i^{th} 个变量与第 j^{th} 个变量之间的协方差。因此， σ_3^2 表示第三个变量的方差，而 σ_{13} 则表示第一个变量与第三个变量之间的协方差。

协方差为 0 表示两者之间没有相关性。如果 x 的方差为 10， y 的方差为 4，且 x 与 y 之间不存在线性相关性，那么我们可以这样表示：

$$\Sigma = \begin{bmatrix} 10 & 0 \\ 0 & 4 \end{bmatrix}$$

如果 x 和 y 之间存在轻微的正相关关系的话，那或许……

$$\Sigma = \begin{bmatrix} 10 & 1.2 \\ 1.2 & 4 \end{bmatrix}$$

其中，1.2 表示 x 与 y 之间的协方差。我认为这种相关性“较弱”，因为 1.2 这个数值相对于 10 来说相当小。

如果 x 与 y 之间存在较大的负相关关系，那么可能会出现以下情况：

协方差矩阵是对称的。毕竟， x 与 y 之间的协方差始终等于 y 与 x 之间的协方差。也就是说，对于任意的 x 和 y ，都有 $\sigma_{xy} = \sigma_{yx}$ 成立。

恐怕我可能会失去你，所以让我们通过一个例子来说明吧。在高斯分布的例子中，我们有一组学生的身高分别为 H=[1.8, 2.0, 1.7, 1.9, 1.6]米。我们进行了如下计算：

$$\begin{aligned} VAR(H) &= E[(H - \mu_H)^2] \\ &= \frac{1}{N}\sum_{i=1}^{n}(H_i - \mu_H)^2 \\ &= \frac{1}{5}\left[(1.8-1.8)^2 + (2-1.8)^2 + (1.7-1.8)^2 + (1.9-1.8)^2 + (1.6-1.8)^2\right] \\ &= 0.02 \end{aligned}$$

很简单，对吧？如果我们称量这些学生的体重，可能会得到如下数值：W = [70.1, 91.2, 59.5, 93.2, 53.5]。那么，我们能否利用协方差公式来计算出协方差矩阵呢？当然可以。计算出来的协方差矩阵如下：

$$\Sigma = \begin{bmatrix} \sigma_H^2 & \sigma_{H,W} \\ \sigma_{W,H} & \sigma_W^2 \end{bmatrix}$$

我们刚刚计算出了身高的方差，该数值会出现在矩阵的左上角。右下角则包含体重的方差。利用同样的公式，我们可以得到：

$$\begin{aligned} \mu_W &= \frac{1}{5}(70.1 + 91.2 + 59.5 + 93.2 + 53.5) = 73.5 \\ \sigma_W^2 &= \frac{1}{5}\left[(70.1-73.5)^2 + (91.2-73.5)^2 + (59.5-73.5)^2 + (93.2-73.5)^2 + (53.5-73.5)^2\right] \\ &= 261.8 \end{aligned}$$

现在来计算协方差。利用上述公式，我们可以得到：

$$
\begin{aligned}
\sigma_{H,W} &= E\left[(H-\mu_H)(W-\mu_W)\right] \\
&= \frac{1}{N}\sum_{i=1}^{n}(H_i-\mu_H)(W_i-\mu_W) \\
&= \frac{1}{5}[(1.8-1.8)(70.1-73.5)+(2-1.8)(91.2-73.5)+(1.7-1.8)(59.5-73.5)+ \\
&\quad (1.9-1.8)(93.2-73.5)+(1.6-1.8)(53.5-73.5)] \\
&= 2.18
\end{aligned}
$$

那虽然很繁琐，但其实也挺简单的。我们再也不会这么做啦，因为 NumPy 当然会帮我们完成这项计算工作。

```
import numpy as np

W = [70.1, 91.2, 59.5, 93.2, 53.5]
H = [1.8, 2.0, 1.7, 1.9, 1.6]
np.cov(H, W)
```

```
array([[  0.025,   2.727],
       [  2.727, 327.235]])
```

这与我们的计算结果不符！到底哪里出错了？其实没什么问题。NumPy 在处理小样本量时会有相应的修正措施：它使用 $\frac{1}{N-1}$ 作为标准化参数，而不是 $\frac{1}{N}$ 。

这超出了本书的讨论范围。简单来说，假设某个班级的实际学生人数为 200 人，但由于无法对所有 200 名学生进行测量，我们只抽取了 5 名学生作为样本来进行计算。可以肯定的是，由于样本无法完全代表整个班级的情况，我们的估算结果必然会存在误差。当样本大小接近 200 时，误差会趋近于 0。此时，我们说该估算结果是无偏的。相反，如果样本规模较小，那么估算结果就会存在偏差，即误差不为零，此时该估算结果就是有偏的。

如果误差为零，那么用 N 来除以该数值是合理的。我不打算详细解释原因，但对于有偏差的估计值，我们需要用 $\frac{1}{N-1}$ 来修正因样本量过小而导致的误差。NumPy 默认就会这么做，因为在实际应用中，我们处理的几乎都是来自更大数据集的样本。如果你想要上面计算出的无偏估计值，那么在调用`np.cov`时，应使用 `bias=1` 。

```
np.cov(H, W, bias=1)
```

```
array([[  0.02 ,   2.182],
       [  2.182, 261.788]])
```

这与我们的计算结果一致。在本书中，我们不会再使用 `bias=1` 了。因为我们所使用的随机变量，其实是从我们所追踪的各个物体的所有可能位置中抽取的。在这里，我们计算的是整个样本群体的方差和协方差，因此 `bias=1` 的数值是正确的。

这个矩阵告诉我们什么信息呢？它表明，身高的方差为 0.02 m^2 ，而体重的方差则为 261.788 kg^2 。此外，该矩阵还显示，体重和身高之间存在正相关关系——也就是说，随着身高的增加，体重也会相应增加。

让我们来创建完全相关的数据吧。所谓“完全相关”，指的是数据点完全沿着一条直线分布——没有任何偏离这条直线的现象。

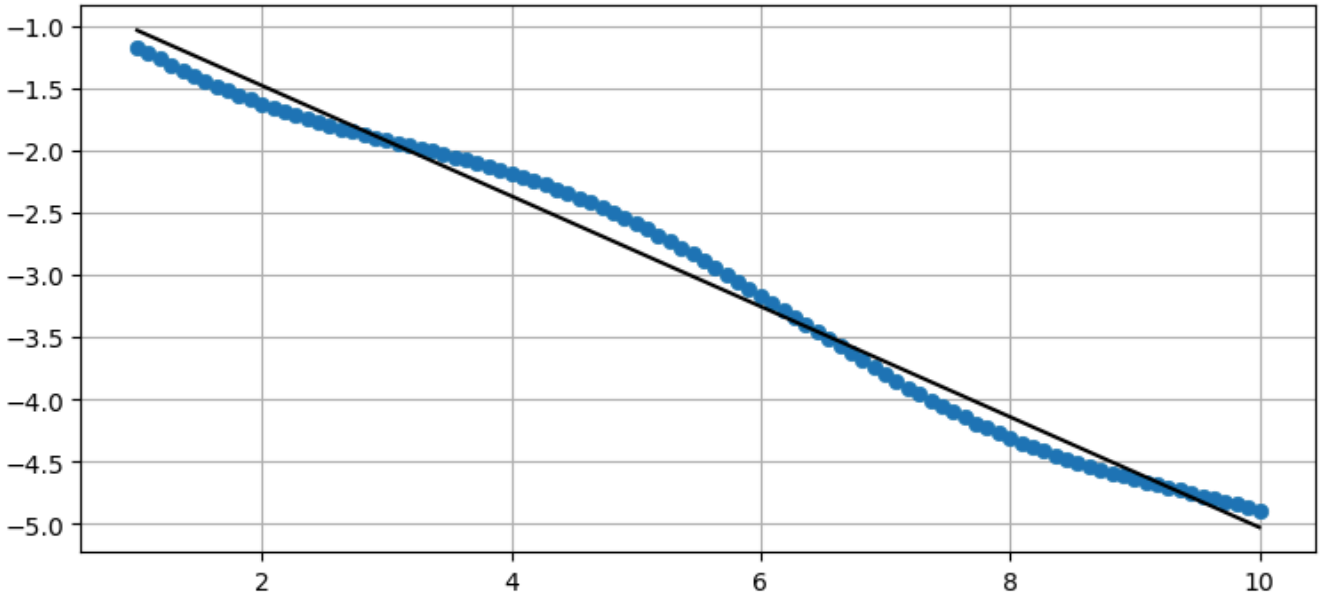
```
X = np.linspace(1, 10, 100)
Y = np.linspace(1, 10, 100)
np.cov(X, Y)
```

```
array([[6.956, 6.956],
       [6.956, 6.956]])
```

从协方差矩阵可以看出，x 和 y 的协方差实际上等于各自的方差。

现在，让我们对其中一个变量添加一些“噪声”，这样它们就不再完全相关了。我会把 Y 的值设为负数，从而产生负相关关系。

```
X = np.linspace(1, 10, 100)
Y = -(np.linspace(1, 5, 100) + np.sin(X)*.2)
plot_correlated_data(X, Y)
print(np.cov(X, Y))
```

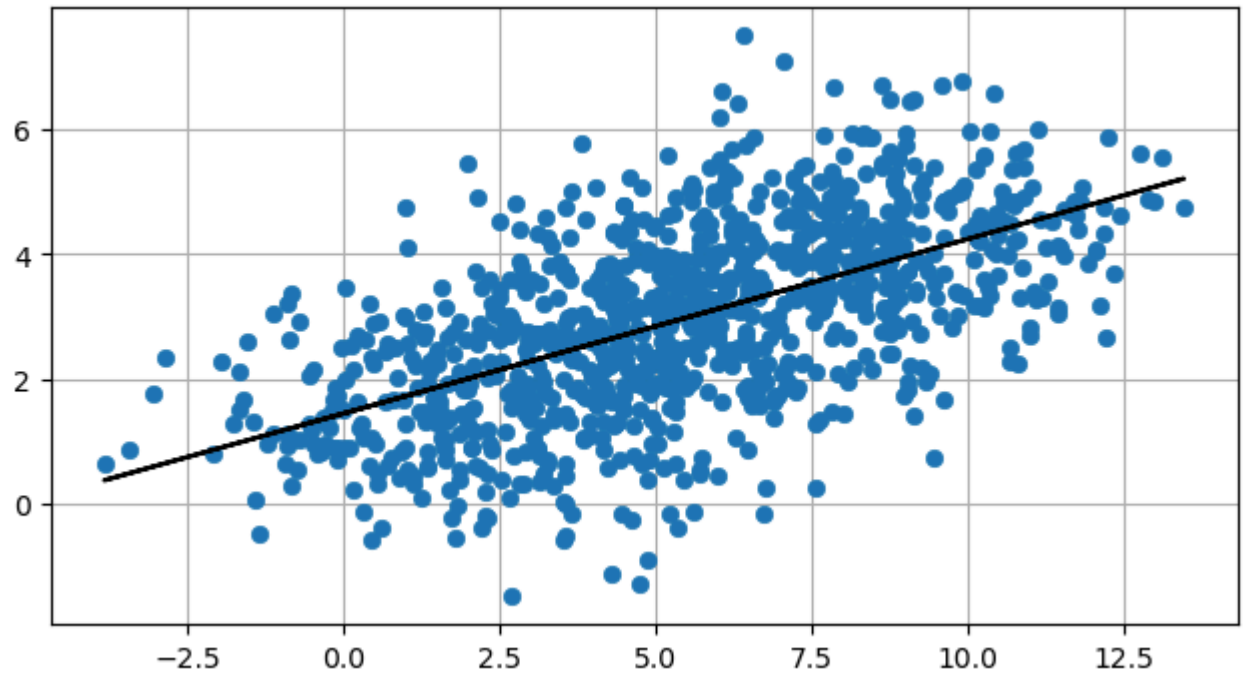



```
[[ 6.956 -3.084]
 [-3.084  1.387]]
```

这些数据不再呈直线形态。协方差值为 $\sigma_{xy} = -3.08$。与 σ_x^2 和 σ_y^2 的数值相比，该值远非零，因此可以确定两者之间仍然存在高度相关性。我们可以通过查看图表来验证这一点：数据点大致呈直线分布。

现在，我会在一条直线上添加随机噪声。

```
from numpy.random import randn
X = np.linspace(1, 10, 1000) + randn(1000)*2
Y = np.linspace(1, 5, 1000) + randn(1000)
plot_correlated_data(X, Y)
print(np.cov(X, Y))
```

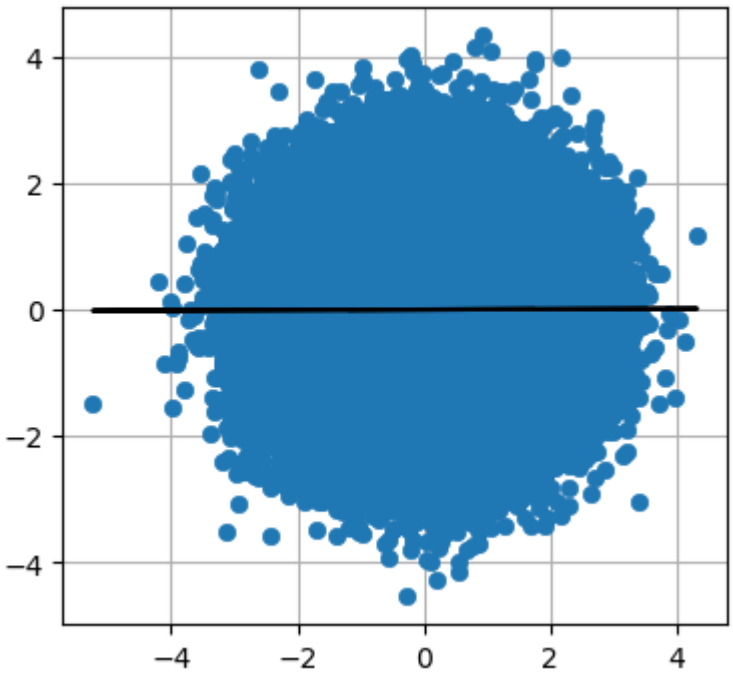



```
[[10.295  2.875]
 [ 2.875  2.299]]
```

我们发现，协方差相对于方差来说要小，这反映出 X 与 Y 之间的相关性较低。我们仍然可以用直线来拟合这些数据，但数据的波动幅度要大得多。

最后，这是完全随机数据之间的协方差。

```
X = randn(100000)
Y = randn(100000)
plot_correlated_data(X, Y)
print(np.cov(X, Y))
```




```
[[1.003 0.003]
 [0.003 1.001]]
```

这里的协方差值几乎为零。从图表中可以看出，根本无法画出一条能够准确描述数据的直线。无论是垂直线还是水平线，都同样无法有效描述这些数据。

多元正态分布方程

请回想一下高斯分布相关章节中关于正态分布的公式：

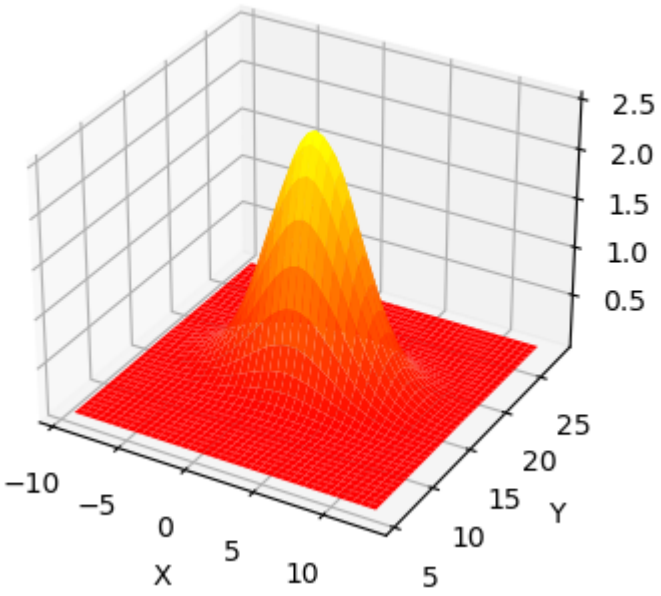
$$f(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{1}{2}(x-\mu)^2 \,/\, \sigma^2\right]$$

以下是 n 维空间中的多元正态分布。

$$f(x, \mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^n \mid \Sigma \mid}} \exp\left[-\frac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu)\right]$$

多元版本只不过是用矩阵来替代了单变量方程中的标量而已。如果你对线性代数有一定的了解，那么这个方程应该不难理解。如果你不太熟悉线性代数，也不用担心，FilterPy 和 SciPy 都提供了相应的函数来帮你计算。暂且先不考虑具体的计算过程，我们先来绘制一下这个方程的图像，看看它到底是什么样子的。

```
import kf_book.mkf_internal as mkf_internal

mean = [2., 17.]
cov = [[10., 0.],
       [0., 4.]]

mkf_internal.plot_3d_covariance(mean, cov)
```



这是一幅多元高斯分布的图示。其均值为 $\mu = \begin{bmatrix} 2 \\ 17 \end{bmatrix}$，协方差为 $\Sigma = \begin{bmatrix} 10 & 0 \\ 0 & 4 \end{bmatrix}$。三维图形展示了当 z 轴上的 (X, Y) 取不同值时，相应的概率密度情况。我将 x 轴和 y 轴的方差值投影到了图表的各个面上——可以看到，这些投影都呈现出高斯曲线形状。 X 对应的曲线比 Y 对应的曲线更宽，这一现象可以用 $\sigma_x^2 = 10$ 和 $\sigma_y^2 = 4$ 来解释。该三维曲面的最高点位于 X 和 Y 的均值所在的位置。

所有的多元高斯分布都具有这种形状。如果把这种分布看作是描述狗的位置的模型，那么在每个点上的 z 值就代表了狗处于该位置的概率密度。严格来说，这其实是一个联合概率密度函数，我稍后会详细解释。因此，狗出现在(2, 17)附近的概率最高；出现在(5, 14)附近的概率适中；而出现在(10, 10)附近的概率则非常低。与单变量情况类似，这里描述的是概率密度，而非具体的概率值。连续型分布的范围是无限的，因此，狗恰好出现在(2, 17)或任何其他特定点的概率都是 0%。我们可以通过计算曲面下的积分来得到狗处于某个特定范围内的概率。

FilterPy [2]在 `filterpy.stats.` 模块中利用 `multivariate_gaussian()` 函数来实现相关计算。SciPy 的 `stats` 模块则利用 `multivariate_normal()` 函数来实现多元正态分布的相关计算。该模块采用了“固定参数”的方式：只需一次性设定均值和协方差，之后就可以对任意数量的 x 值进行概率密度计算，无论调用多少次都是如此。我将自己的函数命名为 `multivariate_gaussian()`，以避免与 SciPy 的函数混淆。

> `scipy.stats` 模块的教程[1]详细介绍了如何“冻结”数据分布，以及该模块的其他实用功能。

```
from filterpy.stats import gaussian, multivariate_gaussian
```

我会先演示如何使用它，然后再介绍一些更有趣的内容。

首先，假设我们的狗位于点(2, 7)，其中 x 的方差为 8， y 的方差为 3。那么，我们就可以计算出狗位于点(2.5, 7.3)时的概率密度。

首先，将 x 设置为(2.5, 7.3)。你可以使用元组、列表或 NumPy 数组来表示这个数值。

```
x = [2.5, 7.3]
```

接下来，我们确定信念的均值：

```
mu = [2.0, 7.0]
```

最后，我们需要确定协方差矩阵的数值。在问题描述中，我们没有提到 x 和 y 之间的任何相关性，因此我们假设它们之间没有关联。这是合乎逻辑的：狗可以独立地选择向 x 方向或 y 方向移动，而不会影响到另一只狗。我会用 `P` 来表示这个变量。卡尔曼滤波器则用 P 来表示协方差矩阵，我们需要熟悉这种命名方式。

```
P = [[8., 0.],
     [0., 3.]]
```

现在调用该函数吧。

```
%precision 4
multivariate_gaussian(x, mu, P)
```

```
0.0315
```

我们也可以通过 `scipy.stats` 模块得到同样的结果。

```
import scipy
from scipy.stats import multivariate_normal
print(f'{multivariate_normal(mu, P).pdf(x):.4f}')
```

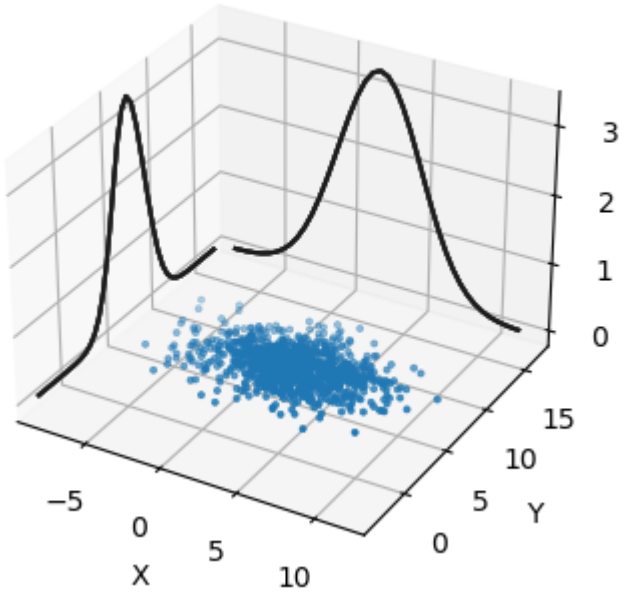
```
0.0315
```

是时候定义一些术语了。联合概率，用 $P(x, y)$ 表示，指的是 x 和 y 同时发生的概率。例如，如果你掷两个骰子，$P(2, 5)$ 就是第一个骰子掷出 2、第二个骰子掷出 5 的概率。假设这两个骰子都是六面体且公平无偏，那么该概率为 $P(2, 5) = \frac{1}{6} \times \frac{1}{6} = \frac{1}{36}$。上方的三维图表展示了联合概率密度函数。

边际概率是指在不考虑其他任何事件的情况下，某个事件发生的概率。在上图中，左侧的高斯曲线就是 Y 的边际概率。也就是说，这就是狗位于 Y 的任意位置时的概率，而与 X 的值无关。我之前说过：“我已经把 x 和 y 的方差分别标在了图的各个位置上”；这些其实就是 x 和 y 的边际概率。高斯分布的另一个优点是：多元高斯分布的边际分布仍然是一个高斯分布！

让我们从稍微不同的角度来考虑这个问题。我不会绘制出表示概率分布的图形，而是会生成 1,000 个点，这些点的分布符合 $\left[\begin{smallmatrix} 8 & 0 \\ 0 & 3 \end{smallmatrix}\right]$ 的分布规律。

```
mkf_internal.plot_3d_sampled_covariance(mu, P)
```



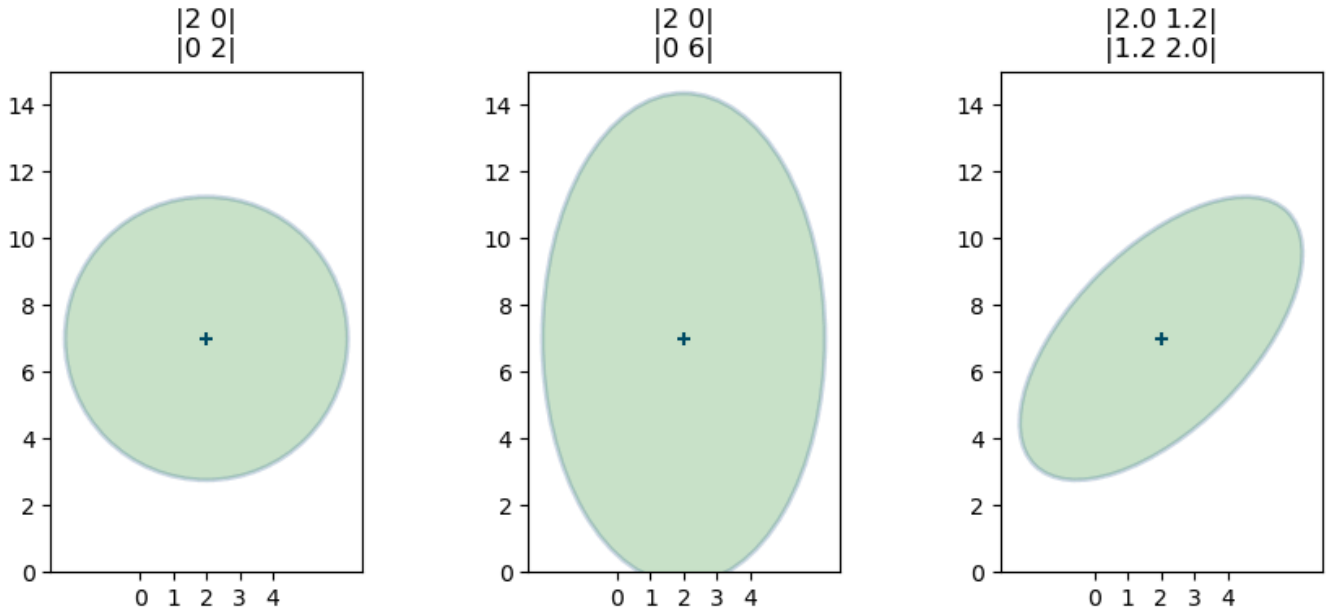
我们可以把这些采样点视为：在给定的均值和协方差条件下，狗狗可能出现的位置。旁边的等高线则代表了 X 和 Y 的边际概率。可以看出，狗狗出现在(2, 7)这个位置的可能性要大得多，因为该位置有大量的采样点；而出现在(-5, 5)这个位置的可能性则小得多，因为该位置只有很少的采样点。

虽然这些图表看起来很美观，但很难从中获取有用的信息。例如，很难判断 X 和 Y 是否具有相同的方差，以及它们之间的相关程度如何。在本书的大部分内容中，我都会用等高线图来表示高斯分布。

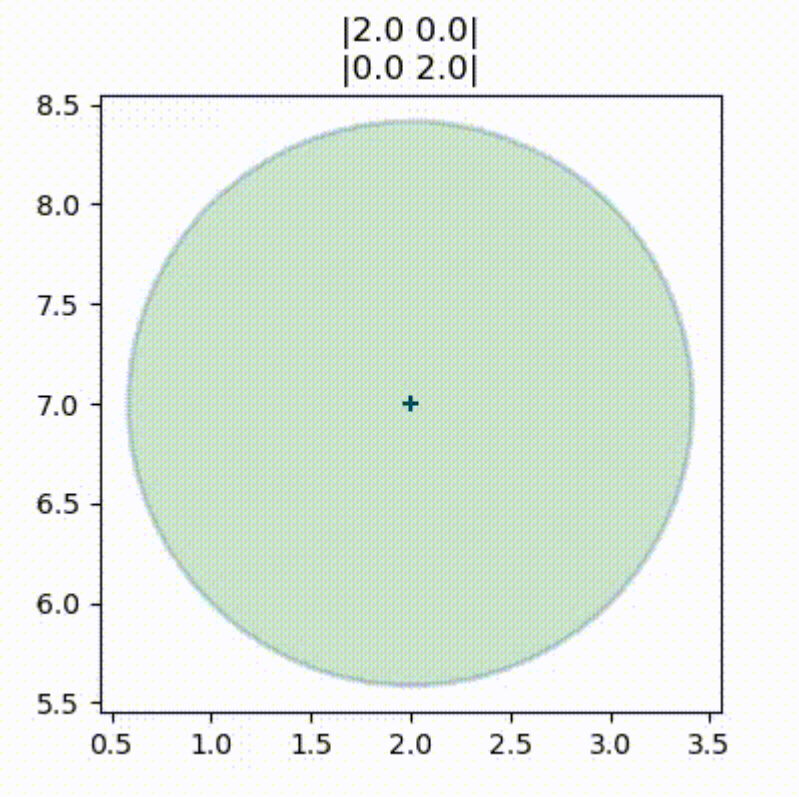
等高线图展示了在特定的标准差下，多元高斯分布所取值的范围。这就好比从三维图中切下水平的一层而已。

这些图表展示了在 3 个标准差范围内的数据分布情况。

```
mkf_internal.plot_3_covariances()
```



对于那些通过在线方式或使用电脑上的 Jupyter Notebook 来观看此内容的人士，这里有一个动画，展示了在方差保持不变的情况下，协方差如何发生变化。

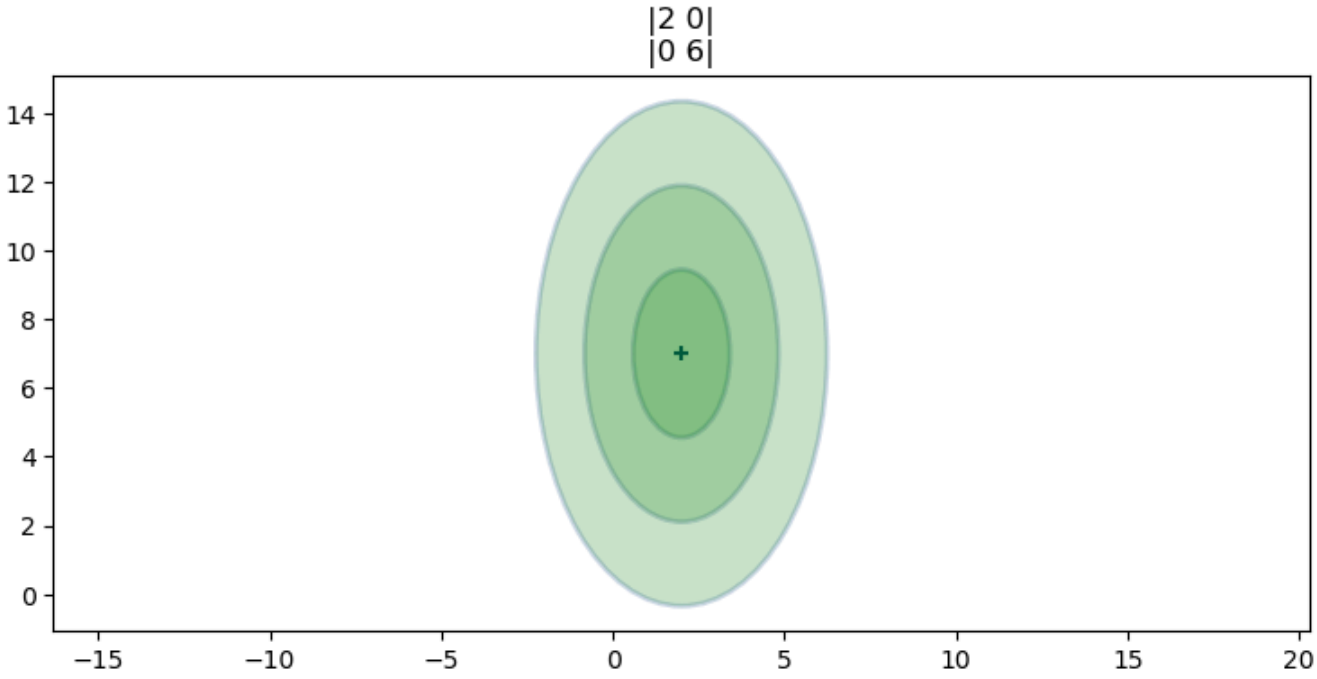


(来源：http://git.io/vqxLS)

这些图形看起来像圆形或椭圆形。事实上，无论从多变量高斯分布中截取哪个截面，得到的都是一条椭圆形。因此，在统计学中，我们不把这些图形称为“等高线图”，而是称之为“误差椭圆”或“置信椭圆”；这两个术语可以互换使用。

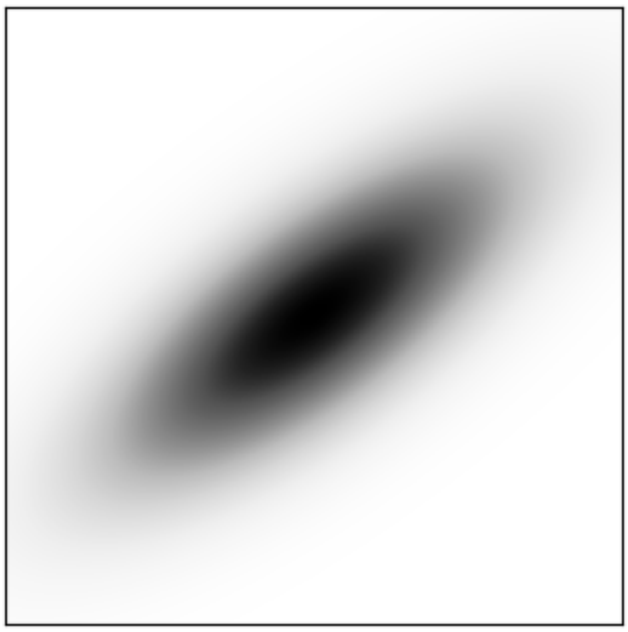
该代码使用了 `filterpy.stats` 中的 `plot_covariance_ellipse()` 函数。默认情况下，该函数会显示一个标准差数值。不过，你也可以使用 `variance` 或 `std` 参数来指定要显示的内容。例如，使用 `variance=3**2` 或 `std=3` 可以显示第三个标准差；而使用 `variance=[1,4,9]` 或 `std=[1,2,3]` 则可以同时显示第一个、第二个和第三个标准差。

```
from filterpy.stats import plot_covariance_ellipse
import matplotlib.pyplot as plt

P = [[2, 0], [0, 6]]
plot_covariance_ellipse((2, 7), P, fc='g', alpha=0.2,
                        std=[1, 2, 3],
                        title='|2 0|\n|0 6|')
plt.gca().grid(b=False);
```



这些纯色区域可能会让你误以为，概率分布在整个标准差范围内是恒定的。但实际上并非如此，从高斯函数的 3D 图上就可以看出来。下图是协方差概率分布的 2D 示意图：颜色越深，表示概率密度越高。

```
from kf_book.nonlinear_plots import plot_cov_ellipse_colormap
plot_cov_ellipse_colormap(cov=[[2, 1.2], [1.2, 1.3]]);
```

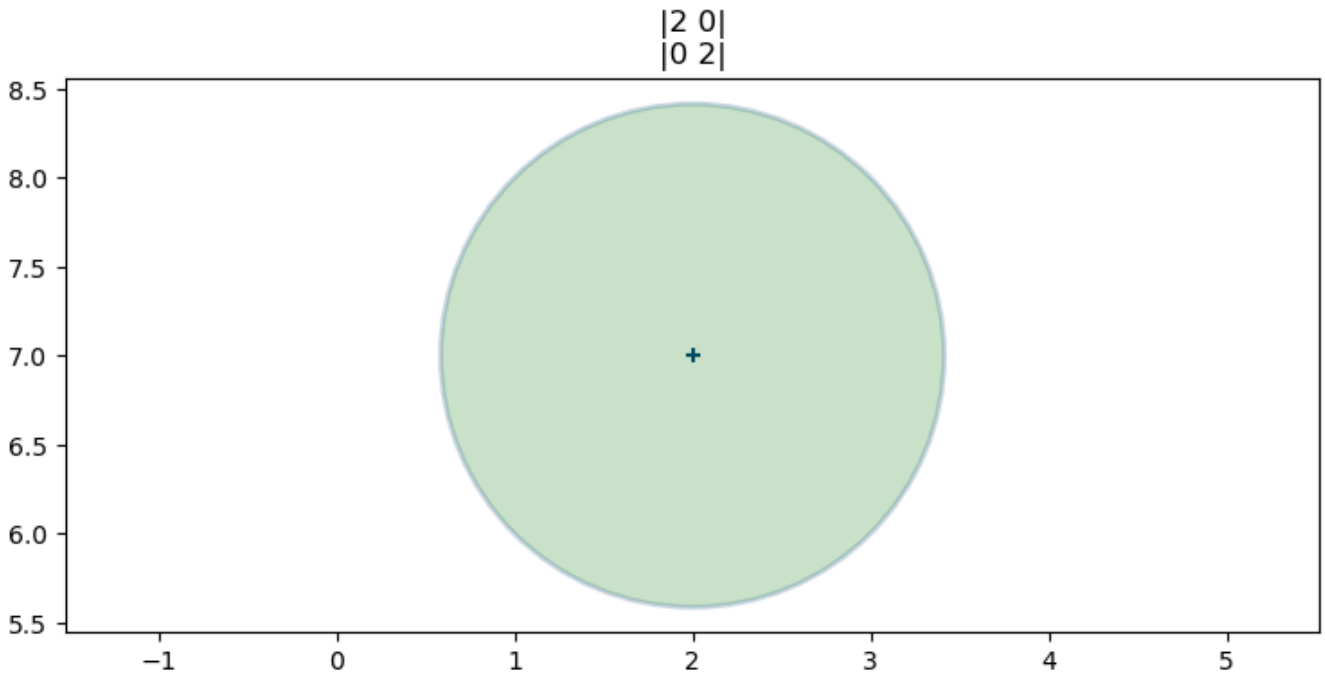


仔细思考这些图表的物理意义后，就能更清楚地理解它们的含义了。第一个图表的均值和协方差分别为……

$$\mu = \begin{bmatrix} 2 \\ 7 \end{bmatrix}, \Sigma = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

```
x = [2, 7]
P = [[2, 0], [0, 2]]
```

```
plot_covariance_ellipse(x, P, fc='g', alpha=0.2,
                        title='|2 0|\n|0 2|')
plt.gca().grid(b=False)
```

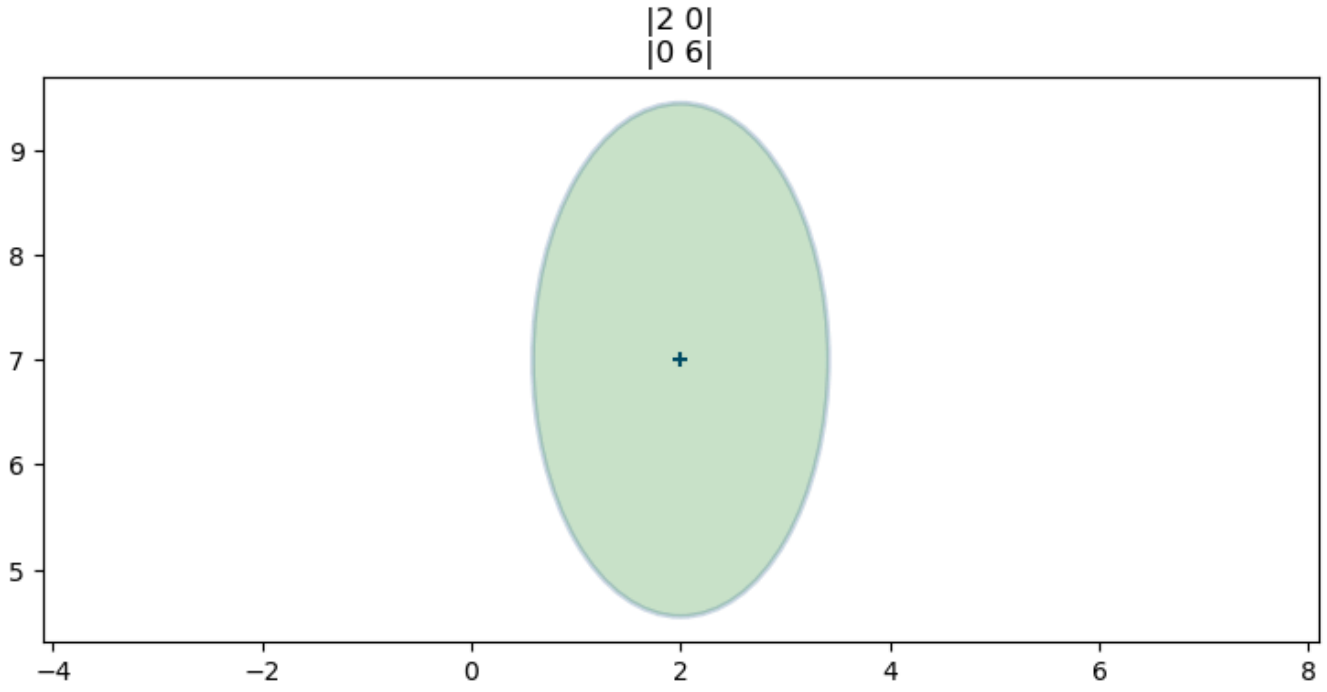


从贝叶斯的角度来看，椭圆的形状反映了我们信念中的误差大小。如果椭圆非常接近圆形，那就意味着我们的信念中的误差很小；而如果椭圆很扁，那就意味着我们的信念中存在较大的误差。椭圆的形状还体现了 X 和 Y 之间的误差比例关系。在这个例子中，由于椭圆接近圆形，因此 X 和 Y 的误差概率是相等的。

第二个图形的均值和协方差分别为：

$$\mu = \begin{bmatrix} 2 \\ 7 \end{bmatrix}, \Sigma = \begin{bmatrix} 2 & 0 \\ 0 & 6 \end{bmatrix}$$

```
x = [2, 7]
P = [[2, 0], [0, 6]]
plot_covariance_ellipse(x, P, fc='g', alpha=0.2,
                    title='|2 0|\n|0 6|')
plt.gca().grid(b=False)
```

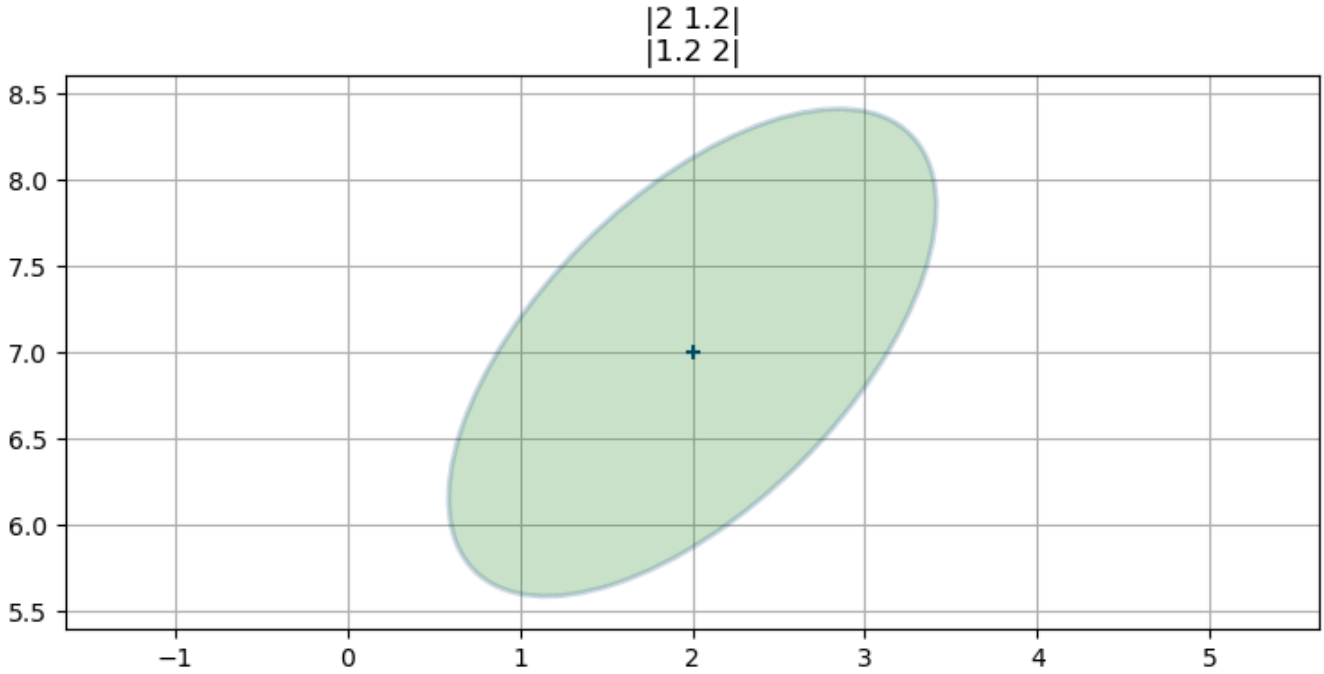


这一次，我们对 X ($\sigma_x^2 = 2$)和 Y ($\sigma_y^2 = 6$)使用了不同的方差值。结果得到的是一个又高又窄的椭圆形。可以看出， Y的不确定性要远远大于 X 的不确定性。在这两种情况下，我们都认为狗的位置在(2, 7)，但两者之间的不确定性程度不同。

第三幅图表展示了平均值和协方差。

$$\mu = \begin{bmatrix} 2 \\ 7 \end{bmatrix}, \Sigma = \begin{bmatrix} 2 & 1.2 \\ 1.2 & 2 \end{bmatrix}$$

```
x = [2, 7]
P = [[2, 1.2], [1.2, 2]]
plot_covariance_ellipse(x, P, fc='g', alpha=0.2,
                    title='|2 1.2|\n|1.2 2|')
```



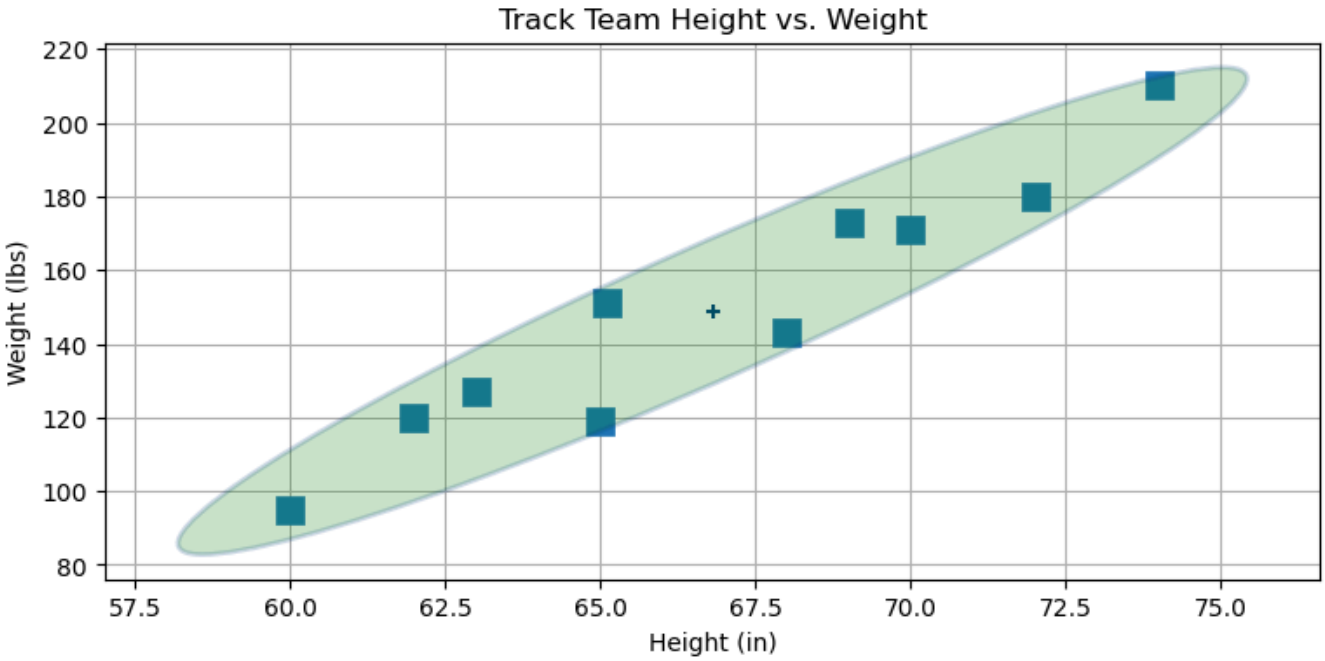
这是第一个在协方差矩阵的非对角线元素上也有数值的等高线图。同时，这也是第一个包含倾斜椭圆的等高线图。这并非巧合。这两个现象其实都在告诉我们同样的道理：倾斜的椭圆表明， x 和 y 这两个数值之间存在某种关联。协方差矩阵中的非对角线元素不为零，这也进一步说明了两者之间存在相关性。

回想一下关于身高与体重的散点图。那些点呈现出一种倾斜的分布态势。我们可以利用 NumPy 的函数来计算两个或多个变量之间的协方差，具体方法是把这些变量放入一个二维数组中。完成后，我们可以在数据上方绘制出表示协方差的椭圆。由于这些数据代表了整个总体，而非样本数据，因此我们需要使用相应的函数来进行计算。

```
cov_hw = np.cov(np.vstack((height, weight)), bias=1)
cov_hw
```

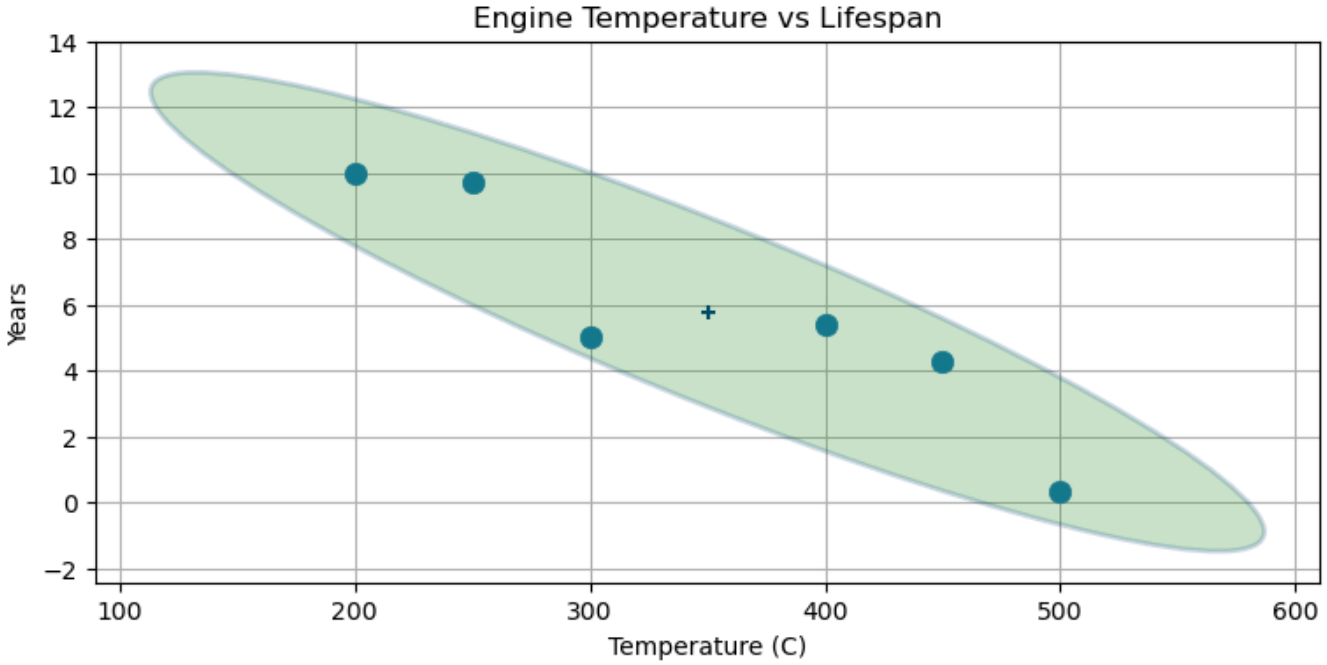
```
array([[  18.5249,  135.701 ],
       [ 135.701 , 1092.29  ]])
```

```
plt.scatter(height, weight, s=120, marker='s')
plt.title('Track Team Height vs. Weight')
plt.xlabel('Height (in)'); plt.ylabel('Weight (lbs)')
plot_covariance_ellipse((np.mean(height), np.mean(weight)), cov_hw, fc='g',
                        alpha=0.2, axis_equal=False, std=2)
```



这应该有助于你更好地理解协方差的含义和用途。协方差椭圆图能够显示数据之间的相互分布情况。如果椭圆很窄，那就说明数据之间的相关性很强。对于任何给定的身高值，其对应的体重值范围都很窄。如果椭圆向右倾斜，那就说明存在正相关关系——即随着 x 的增加，y 也会增加。如果椭圆向左倾斜，则说明存在负相关关系——即随着 x 的增加，y 会减少。我们可以在下图中看到这一点：

```
max_temp = [200, 250, 300, 400, 450, 500]
lifespan = [10, 9.7, 5, 5.4, 4.3, 0.3]

plt.scatter(max_temp, lifespan, s=80)
cov = np.cov(np.vstack((max_temp, lifespan)))
plot_covariance_ellipse((np.mean(max_temp), np.mean(lifespan)), cov, fc='g',
            alpha=0.2, axis_equal=False, std=2)
plt.title('Engine Temperature vs Lifespan')
plt.xlabel('Temperature (C)'); plt.ylabel('Years');
```



通过直观观察，很难理解方差和协方差之间的关系。因此，这里有一个交互式图表来帮助理解。如果您是以静态文本形式阅读此内容，请点击此处了解如何在线查看该图表：https://git.io/vza7b

```
from ipywidgets import interact
from kf_book.book_plots import figsize, FloatSlider

fig = None
def plot_covariance(var_x, var_y, cov_xy):
    global fig
    if fig: plt.close(fig)
    fig = plt.figure(figsize=(4,4))
    P1 = [[var_x, cov_xy], [cov_xy, var_y]]

    plot_covariance_ellipse((10, 10), P1, axis_equal=False,
                            show_semiaxis=True)

    plt.xlim(4, 16)
    plt.gca().set_aspect('equal')
    plt.ylim(4, 16)


with figsize(y=6):
    interact (plot_covariance,
          var_x=FloatSlider(5, min=0, max=20),
          var_y=FloatSlider(5, min=0, max=20),
          cov_xy=FloatSlider(1.5, min=0, max=50, step=.2));
```

```
interactive(children=(FloatSlider(value=5.0, continuous_update=False, description='var_x', max=20.0),
FloatSli…
```

皮尔逊相关系数

在本书中，我们不会使用这个系数。不过，你可能会在其他地方看到它。如果你不感兴趣的话，可以直接跳过这一部分。

两个变量之间的相关性可以用皮尔逊相关系数来表示为一个数值。其定义如下：

$$\rho_{xy} = \frac{COV(X,Y)}{\sigma_x \sigma_y}$$

该数值的取值范围在-1 到 1 之间。如果协方差为 0，则表示两者之间没有相关性。数值大于 0 表示正相关，数值小于 0 则表示负相关。接近-1 或 1 的数值表示相关性很强，而接近 0 的数值则表示相关性很弱。

相关性和协方差之间有着密切的联系。协方差具有具体的单位，而相关性则是一个没有单位的比率。例如，对于我们的狗 σ_{xy} 来说，其相关性的计算结果单位是平方米。

我们可以使用 `scipy.stats.pearsonr` 函数来计算皮尔逊系数。该函数会返回一个包含皮尔逊系数和双尾 p 值的元组。不过，在本书中，我们不会使用双尾 p 值。这里，我们以学生运动员的身高与体重为例来计算 ρ 值：

```
from scipy.stats import pearsonr
pearsonr(height, weight)[0]
```

```
0.9539731096080194
```

在这里，我们计算了发动机温度与使用寿命之间的相关性。

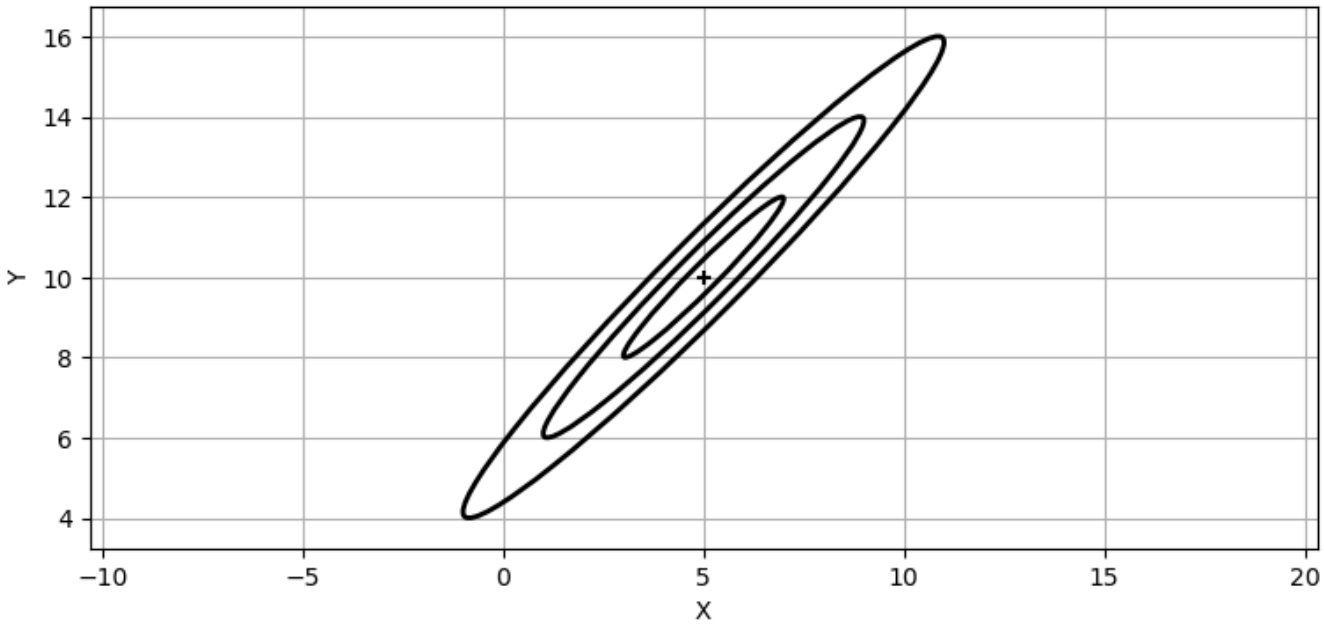
```
pearsonr(max_temp, lifespan)[0]
```

```
-0.9178223453527254
```

利用相关性来提高估算精度

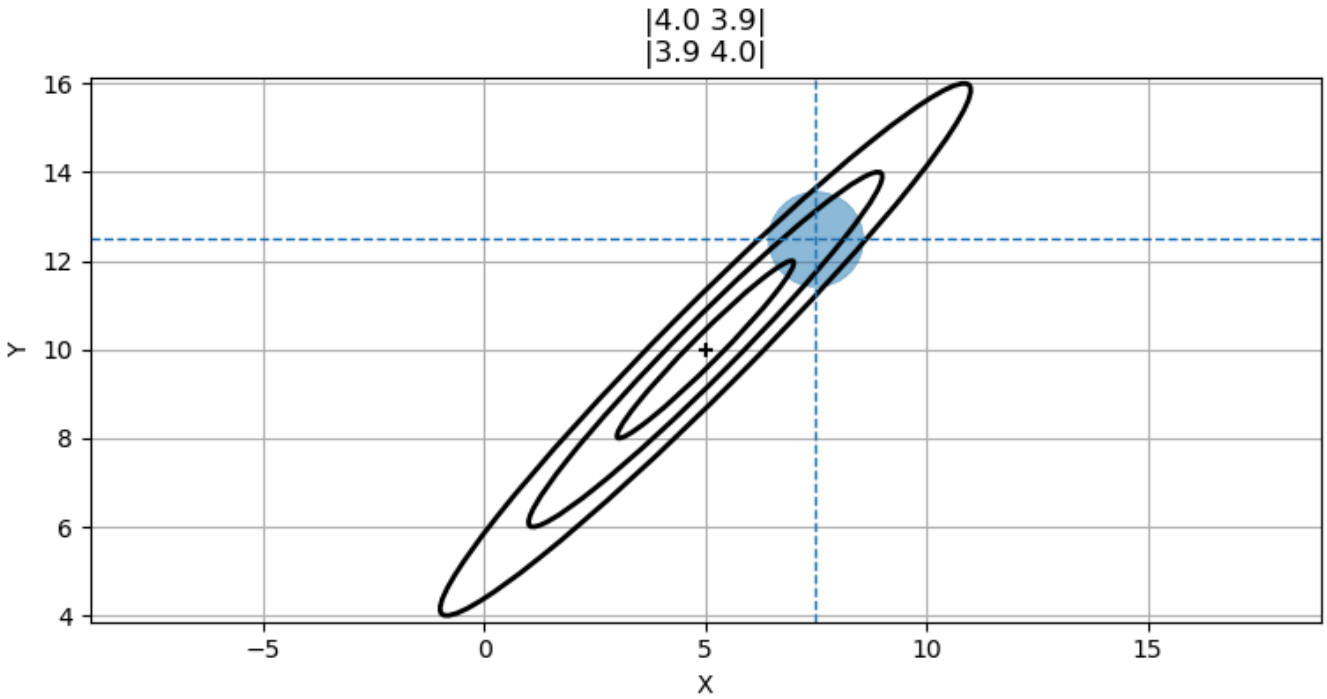
假设我们认为，我们的狗位于坐标点(5, 10)处，且该位置的协方差值已知。如果 x 轴和 y 轴上的标准差均为 2 米，但两者之间存在很强的相关性，那么协方差等高线就会呈现类似这样的形状。

```
P = [[4, 3.9], [3.9, 4]]

plot_covariance_ellipse((5, 10), P, ec='k', std=[1, 2, 3])
plt.xlabel('X')
plt.ylabel('Y');
```



现在，假设我告诉您，我们知道 $x = 7.5$ 的值。那么，关于 y 的值，我们能推断出什么呢？ y 的值极有可能位于 3 个 σ 所构成的协方差椭圆范围内。由于 x 和 y 之间存在相关性，我们可以利用协方差矩阵来推断 y 的值。我用蓝色实心圆标出了 y 的可能取值范围。

```
mkf_internal.plot_correlation_covariance()
```



这个圆在数学上并不完全准确，但大致能表达出我们的意图。我们将在下一节中详细探讨其中的数学问题。目前可以确定的是， y 的值很可能接近 12。而 $y = -10$ 的值则几乎不可能出现。

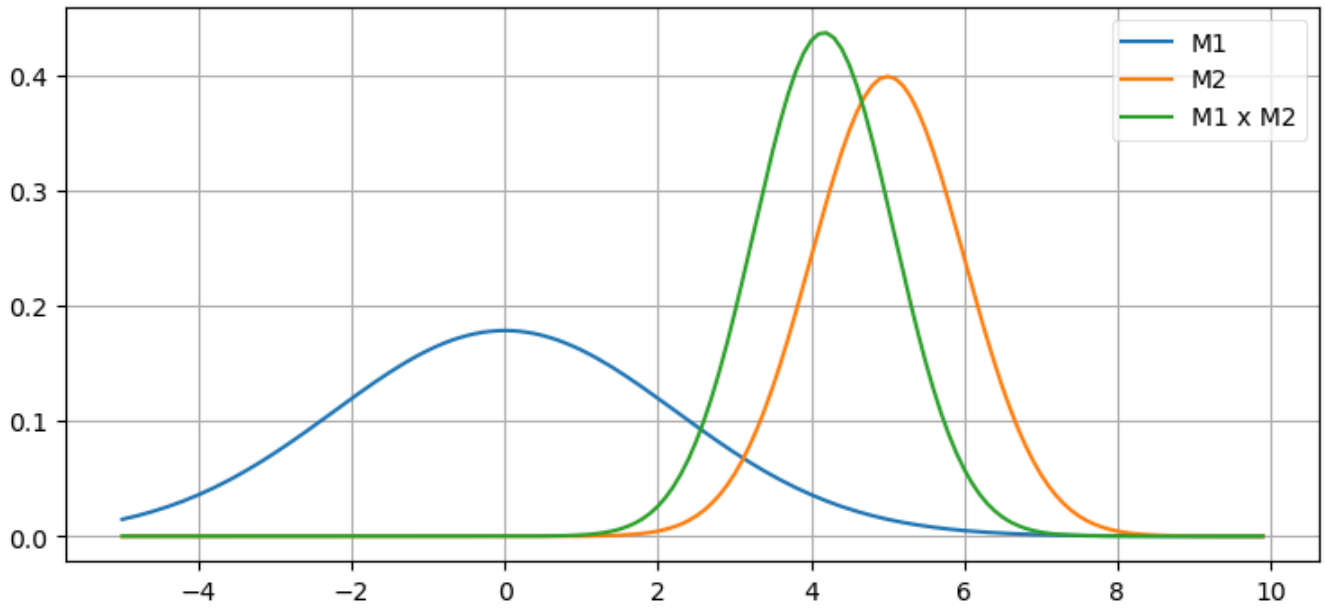
关于相关性和独立性需要说明的是：如果各个变量是独立的，那么它们可以各自独立地变化。比如，当你在开阔的田野中行走时，你可以朝东西方向移动，也可以朝南北方向移动，或者以任何组合的方式移动。独立的变

量之间必然也是不相关的。不过，特殊情况除外，反之则不成立。变量之间可能不相关，但却有可能是相互依赖的。例如，以 $y = x^2$ 为例。相关性是一种线性度量，因此 x 和 y 之间是不相关的。但是， y 却依赖于 x 。

多维高斯函数的乘法运算

在上一章中，我们通过将具有不确定性的测量结果和估计值的高斯分布相乘，来处理这些不确定性。这样得到的结果仍然是高斯分布，只不过其方差更小了。如果两段不确定的信息能够相互印证，那么我们得出的结论就会更可靠。相关图表如下所示：

```
mkf_internal.plot_gaussian_multiply()
```



将测量结果 1 和 2 结合起来后，结果的确定性会更高。因此，新的高斯分布曲线会更高、更窄——方差也变小了。在多维情况下，多元高斯分布也是如此。

以下是多元高斯分布相乘时的相关公式。大写的西格玛符号表示这些是矩阵，而非标量。具体来说，它们就是协方差矩阵。

$$\begin{aligned}\mu &= \Sigma_2(\Sigma_1 + \Sigma_2)^{-1}\mu_1 + \Sigma_1(\Sigma_1 + \Sigma_2)^{-1}\mu_2 \\ \Sigma &= \Sigma_1(\Sigma_1 + \Sigma_2)^{-1}\Sigma_2\end{aligned}$$

它们是通过将先验分布和估计值所对应的多元高斯分布代入贝叶斯定理中计算得出的。在关于高斯分布的那一章中，我已经给出了单变量情况下的数学推导过程。

你无需记住这些方程式，因为它们会由卡尔曼滤波公式来计算，这些公式稍后会详细介绍。在 FilterPy 中，也可以使用 `multivariate_multiply()` 方法来进行这些计算，你可以通过 `filterpy.stats` 来导入该函数。

为了让你更直观地理解这一点，不妨回想一下关于单变量高斯分布相乘的公式：

$$
\begin{aligned}
\mu &= \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2}, \\
\sigma^2 &= \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}
\end{aligned}
$$

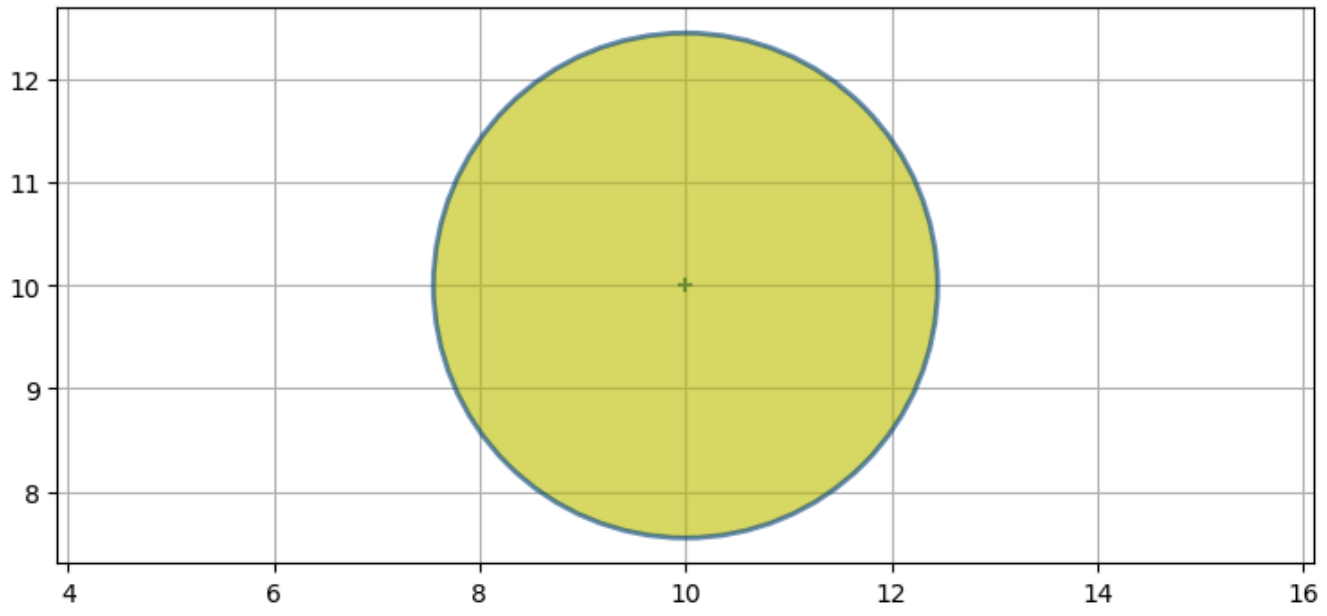
这看起来与多元方程的公式很相似。如果你明白矩阵求逆运算——用-1 的幂来表示——其实相当于求倒数的话，那么这一点就更加明显了。我会把矩阵求逆运算改写为除法形式。不过从数学上来说，这种做法并不完全正确，因为矩阵的除法并没有明确定义。不过，这样做有助于我们更好地理解这些方程式。

$$
\begin{aligned}
\mu &\approx \frac{\Sigma_2\mu_1 + \Sigma_1\mu_2}{\Sigma_1 + \Sigma_2} \\
\Sigma &\approx \frac{\Sigma_1\Sigma_2}{(\Sigma_1 + \Sigma_2)}
\end{aligned}
$$

在这种形式下，单变量方程与多变量方程之间的关系就一目了然了。

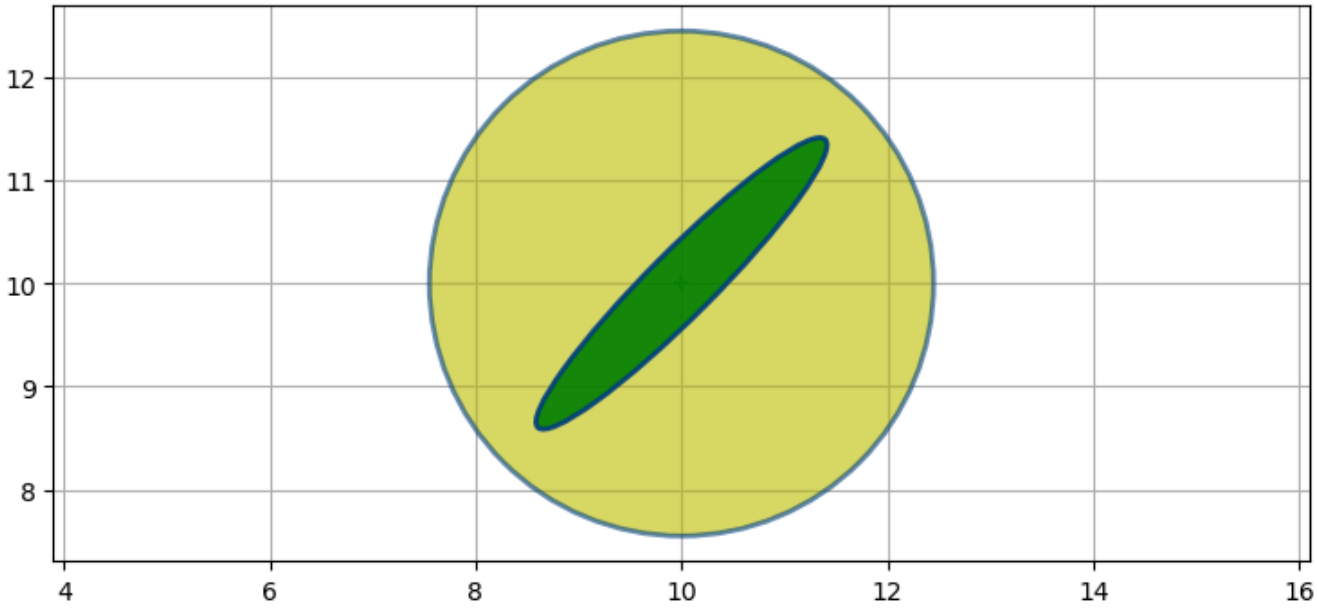
现在，让我们通过一个具体的例子来了解多元高斯分布。假设我们使用两个雷达系统来追踪一架飞机的位置。为了简化问题，我们不考虑飞机的飞行高度，因此可以使用二维坐标图来表示飞机的位置。雷达可以提供目标到探测器的距离和方位角。由于我们最初并不清楚飞机的确切位置，因此表示位置不确定性的协方差可能如下所示。在贝叶斯统计学中，这被称为“先验概率”。

```
P0 = [[6, 0], [0, 6]]
plot_covariance_ellipse((10, 10), P0, fc='y', alpha=0.6)
```



现在，假设在飞机的左下方有一个雷达。进一步假设，该雷达的方位测量是准确的，但距离测量则不准确。那么，这些测量误差的协方差可能如下所示（以绿色线条显示在黄色的基准线上方）：

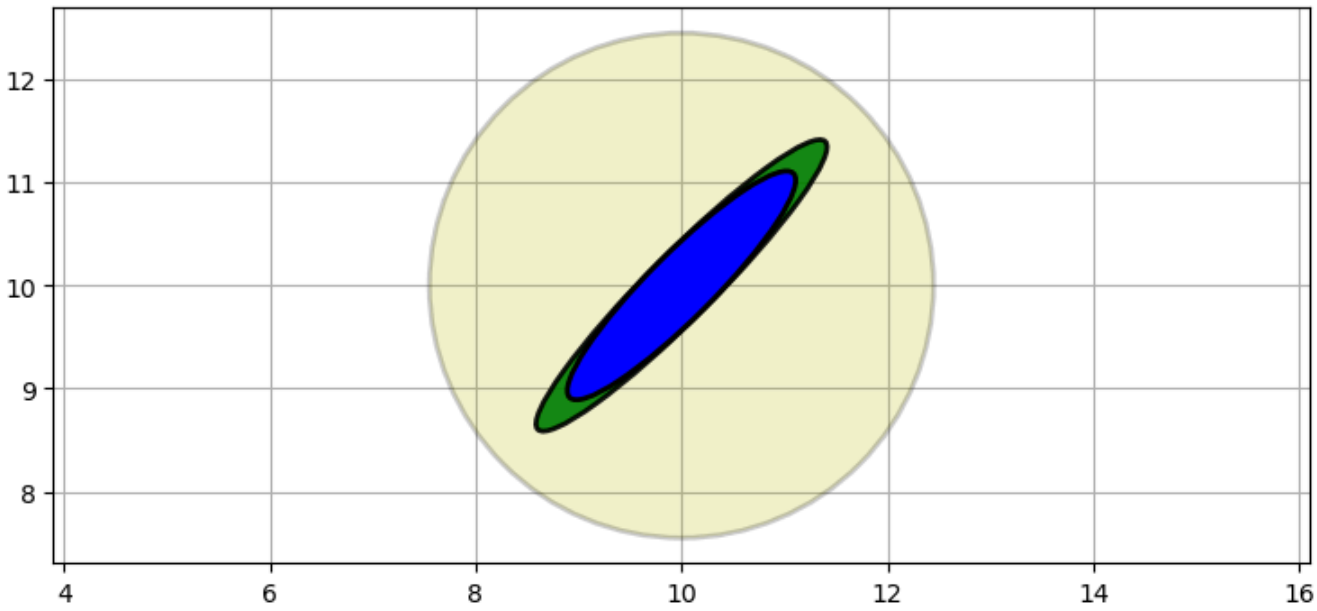
```
P1 = [[2, 1.9], [1.9, 2]]
plot_covariance_ellipse((10, 10), P0, fc='y', alpha=0.6)
plot_covariance_ellipse((10, 10), P1, fc='g', alpha=0.9)
```



请记住，贝叶斯统计学将此称为“证据”。椭圆的中心指向雷达所探测到的位置。由于测距不准确，椭圆的长度相当大；实际上，飞机可能与测得的距离有相当大的偏差。不过，由于方位角的测量非常精确，因此椭圆的宽度很小，这意味着飞机很可能就在该方位角所指示的位置附近。

我们希望找到后验概率——也就是在将各种证据纳入先验概率之后所得到的均值和协方差。与其他章节的做法一样，我们通过将各种证据相乘来合并它们。

```
from filterpy.stats import multivariate_multiply

P2 = multivariate_multiply((10, 10), P0, (10, 10), P1)[1]
plot_covariance_ellipse((10, 10), P0, ec='k', fc='y', alpha=0.2)
plot_covariance_ellipse((10, 10), P1, ec='k', fc='g', alpha=0.9)
plot_covariance_ellipse((10, 10), P2, ec='k', fc='b')
```

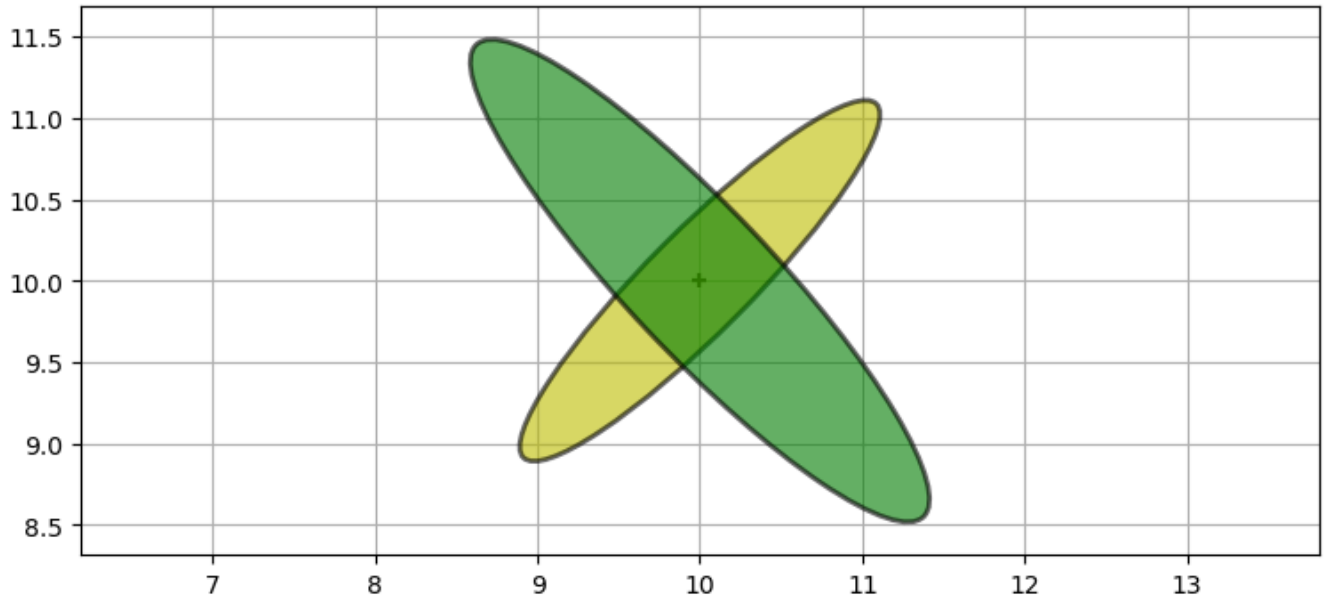


我用透明的黄色标出了最初的估算值（先验值），用绿色标出了雷达测得的数值（证据），再用蓝色标出了最终的估算值（后验值）。

后部的形状和位置与雷达测量结果一致，不过尺寸要小一些。我们在一维高斯分布中也观察到了这种现象。当将两个高斯分布相乘时，方差会减小，因为这样处理能整合更多信息，从而降低不确定性。另外需要注意的是，协方差的形状反映了飞机与雷达系统的物理布局。这一点在接下来的步骤中会变得更加清晰。

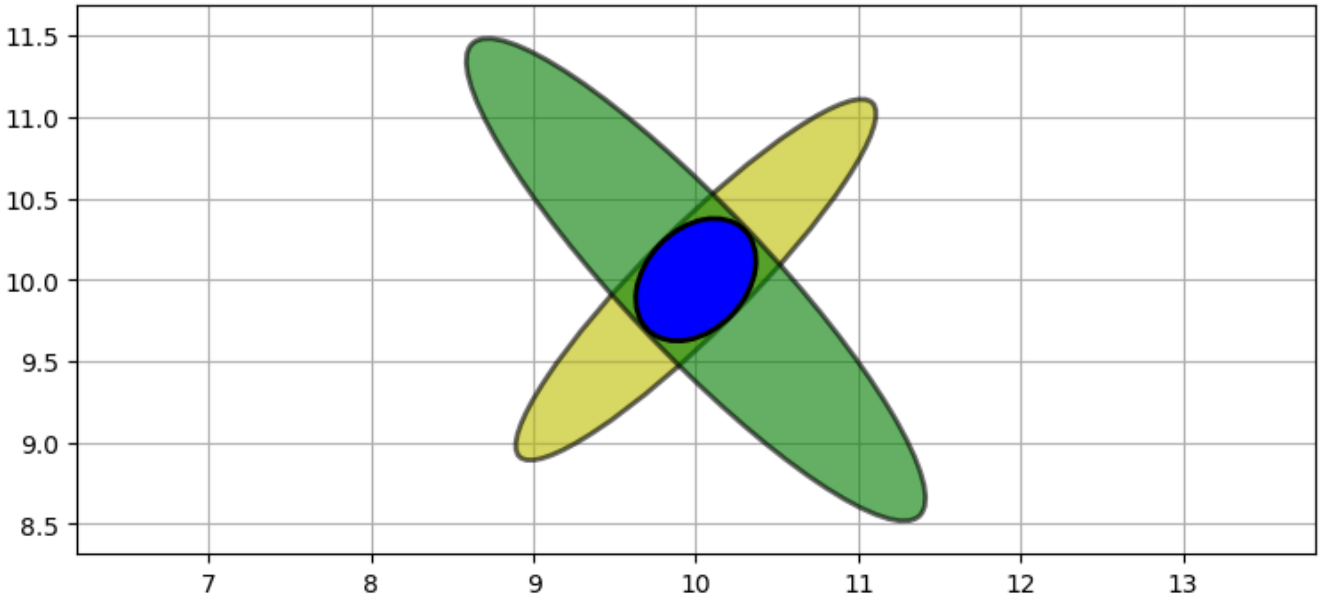
现在，假设我们从另一个雷达那里获得了测量数据，也就是右下角的那个雷达。上一步得到的后验概率值就变成了新的先验概率值，我用黄色来表示这个新的先验概率值。新的测量数据则用绿色来表示。

```
P3 = [[2, -1.9], [-1.9, 2.2]]
plot_covariance_ellipse((10, 10), P2, ec='k', fc='y', alpha=0.6)
plot_covariance_ellipse((10, 10), P3, ec='k', fc='g', alpha=0.6)
```



我们通过将高斯函数相乘来整合这些信息：

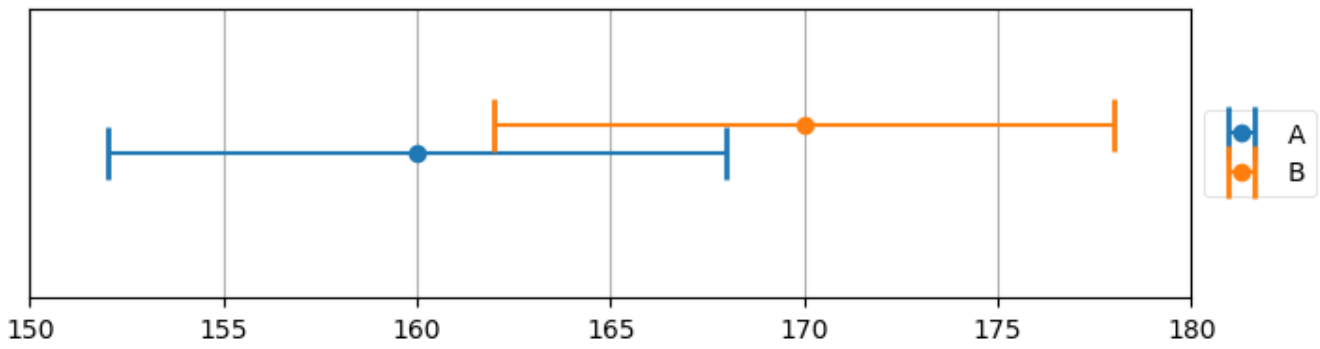
```
P4 = multivariate_multiply((10, 10), P2, (10, 10), P3)[1]
plot_covariance_ellipse((10, 10), P2, ec='k', fc='y', alpha=0.6)
plot_covariance_ellipse((10, 10), P3, ec='k', fc='g', alpha=0.6)
plot_covariance_ellipse((10, 10), P4, ec='k', fc='b')
```



飞机最可能所在的位置，就是那两个椭圆的交点处。这个交点是通过将先前的数据与测量结果相乘而得到的，其结果是一个新的高斯分布。这些几何形状反映了问题的本质特征。这样一来，我们就可以对飞机的位置进行精确的估算。实际上，我们并没有专门编写代码来进行三角测量；这只是将各项测量结果对应的高斯分布相乘后的自然结果而已。

回想一下关于 g-h 滤波器的那一章内容：我们当时展示了用秤称量两次所得结果的误差范围。估计值应该位于这两个误差范围所覆盖的区域内。在这里，估计值应该在 161 磅到 163 磅之间。

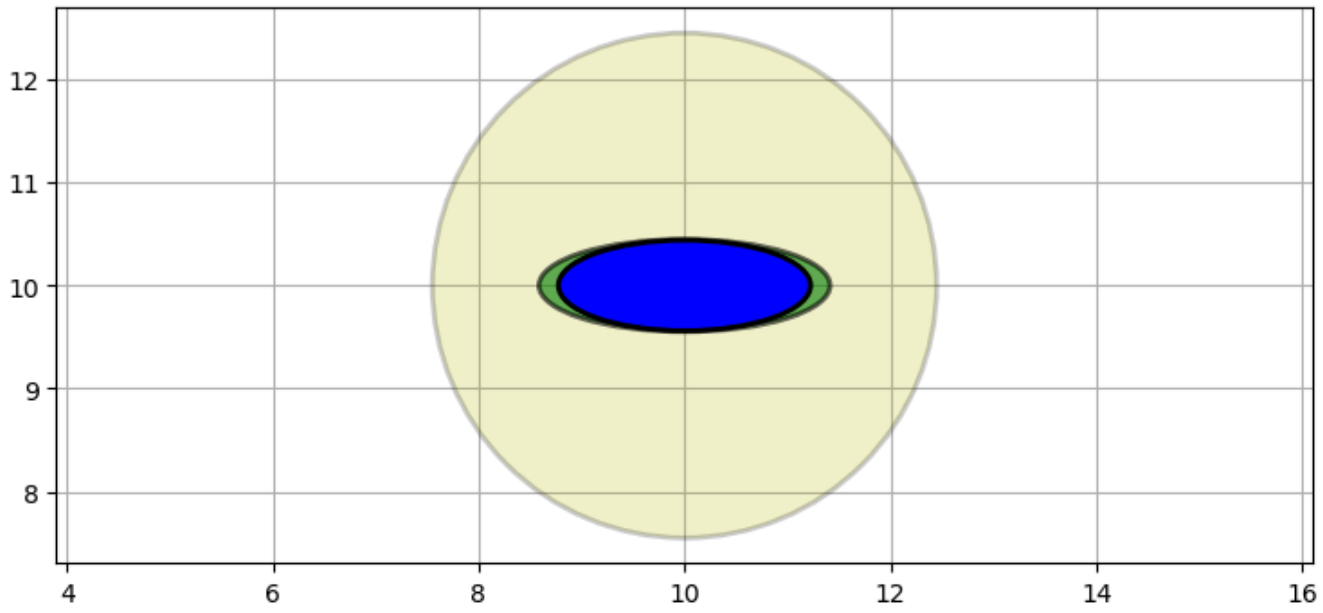
```
import kf_book.book_plots as book_plots
book_plots.plot_errorbars([(160, 8, 'A'), (170, 8, 'B')], xlims=(150, 180))
```



让我们考虑另一种布局。假设第一台雷达位于飞机的正左侧。我可以用某种方式来模拟这种测量误差。

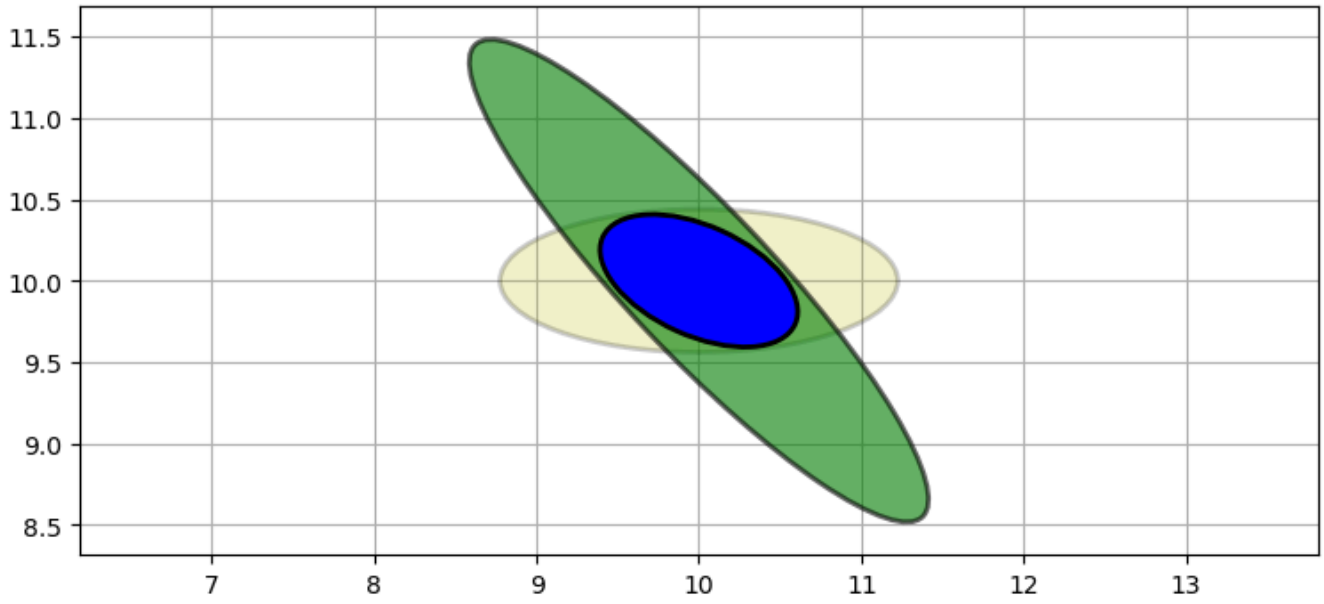
这里我们看到了将先验概率与测量结果相乘后的结果。

```
P1 = [[2, 0], [0, .2]]
P2 = multivariate_multiply((10, 10), P0, (10, 10), P1)[1]
plot_covariance_ellipse((10, 10), P0, ec='k', fc='y', alpha=0.2)
plot_covariance_ellipse((10, 10), P1, ec='k', fc='g', alpha=0.6)
plot_covariance_ellipse((10, 10), P2, ec='k', fc='b')
```



现在，我们可以把第二个雷达系统的测量数据也纳入考虑范围了。该雷达系统仍保持与之前相同的位置。

```
P3 = [[2, -1.9], [-1.9, 2.2]]
P4 = multivariate_multiply((10, 10), P2, (10, 10), P3)[1]
plot_covariance_ellipse((10, 10), P2, ec='k', fc='y', alpha=0.2)
plot_covariance_ellipse((10, 10), P3, ec='k', fc='g', alpha=0.6)
plot_covariance_ellipse((10, 10), P4, ec='k', fc='b')
```

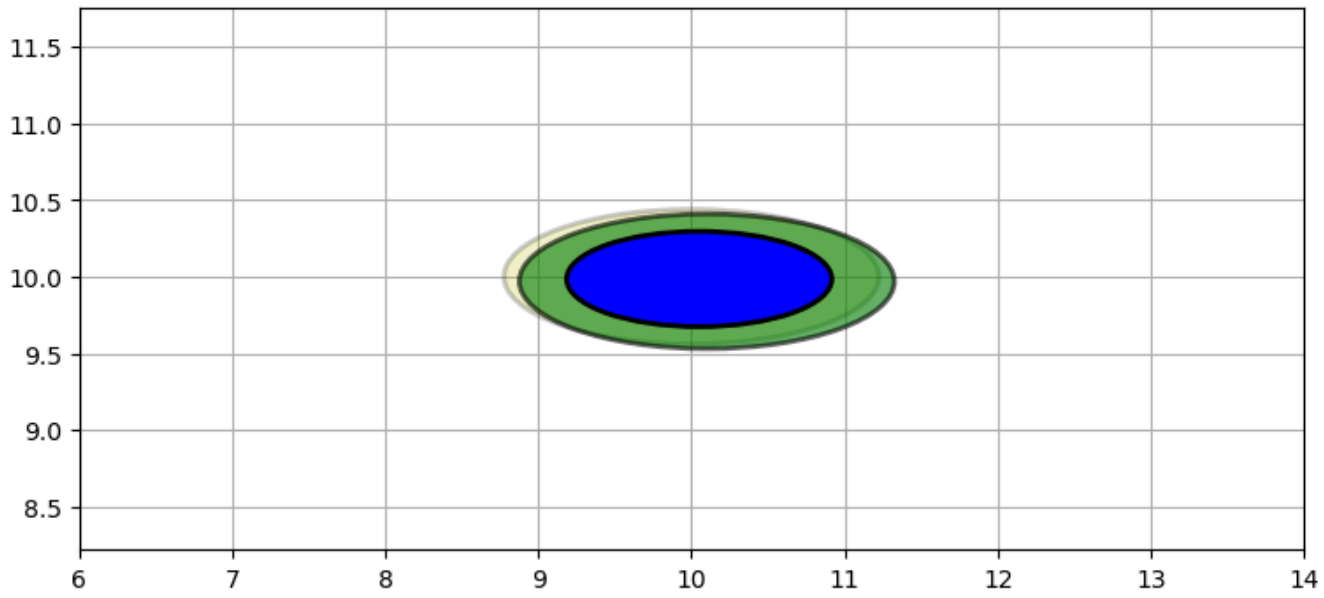


我们的估算结果不如前面的例子准确。由于这两个雷达站相对于飞机的位置不再处于正交状态，因此三角测量结果的准确性也就大打折扣了。

最后一个例子：假设我们在短时间内从同一台雷达设备上获取了两次测量数据。此时，两个协方差椭圆几乎会重叠在一起，从而导致我们的新估算结果出现很大的误差。

```
P5 = multivariate_multiply((10,10), P2, (10.1, 9.97), P2)
plot_covariance_ellipse((10, 10), P2, ec='k', fc='y', alpha=0.2)
```

```
plot_covariance_ellipse((10.1, 9.97), P2, ec='k', fc='g', alpha=0.6)
plot_covariance_ellipse(P5[0], P5[1], ec='k', fc='b')
plt.xlim(6, 14);
```

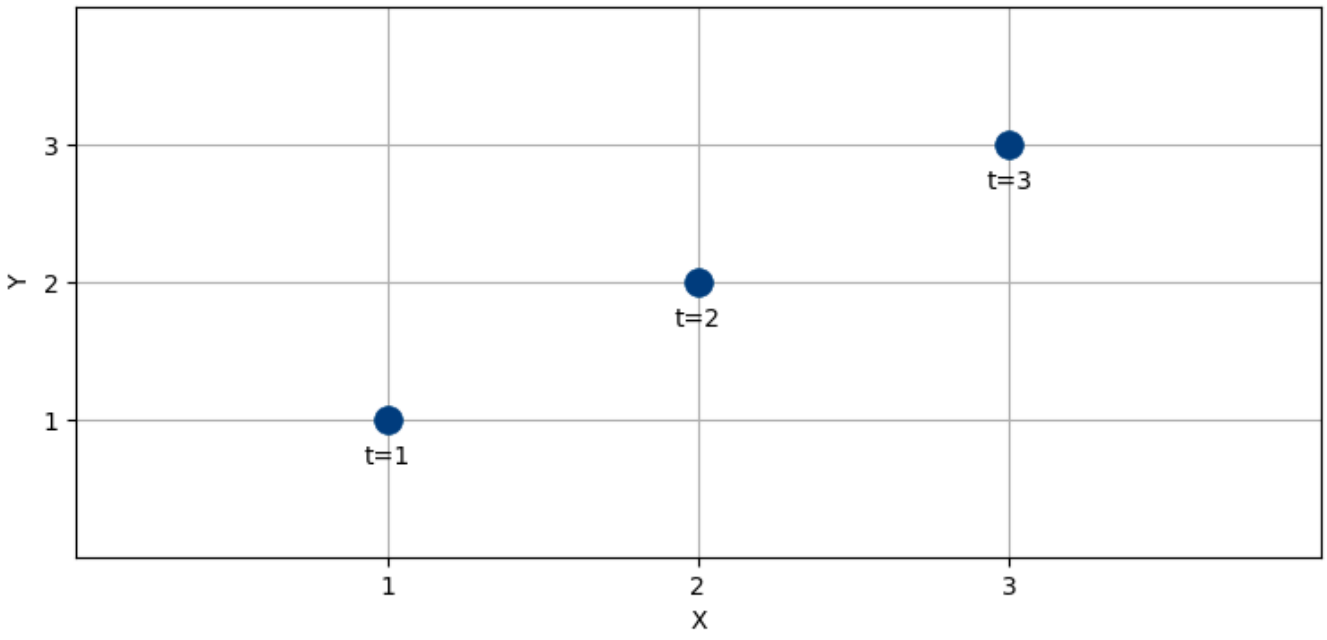


隐藏变量

你现在应该明白，为什么多元卡尔曼滤波器的性能会优于单变量卡尔曼滤波器了。各变量之间的相关性能够显著提升我们的估算精度。我们还可以进一步探讨这个问题。本节包含了本章的核心思想，请仔细阅读。

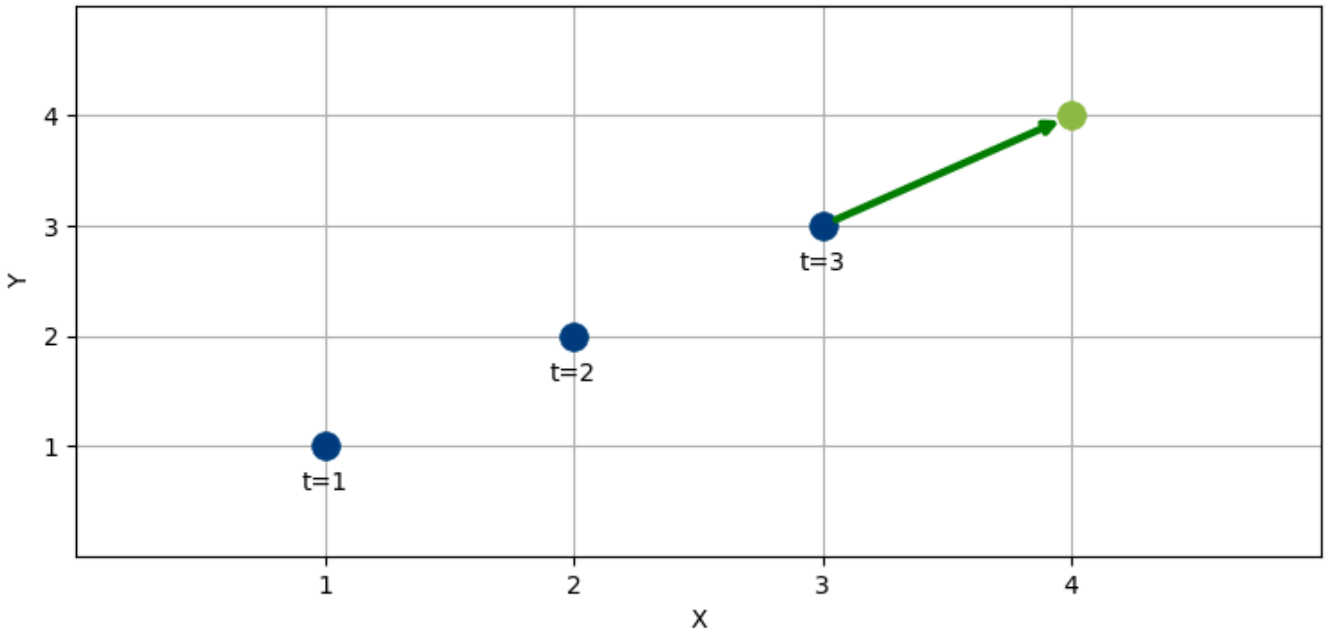
假设我们正在追踪一架飞机，我们在时间点 t 为 1 秒、2 秒和 3 秒时，分别得到了 x 和 y 坐标的数据。那么，凭直觉来看，当时间点 t 为 4 秒时， x 的数值会是多少呢?

```
mkf_internal.show_position_chart()
```



看起来，该飞机是沿直线飞行的。我们知道，飞机不可能瞬间改变飞行方向。最合理的推测是：在 t =4 时，飞机的位置应该在(4,4)处。我会用绿色箭头来标示这个位置。

```
mkf_internal.show_position_prediction_chart()
```



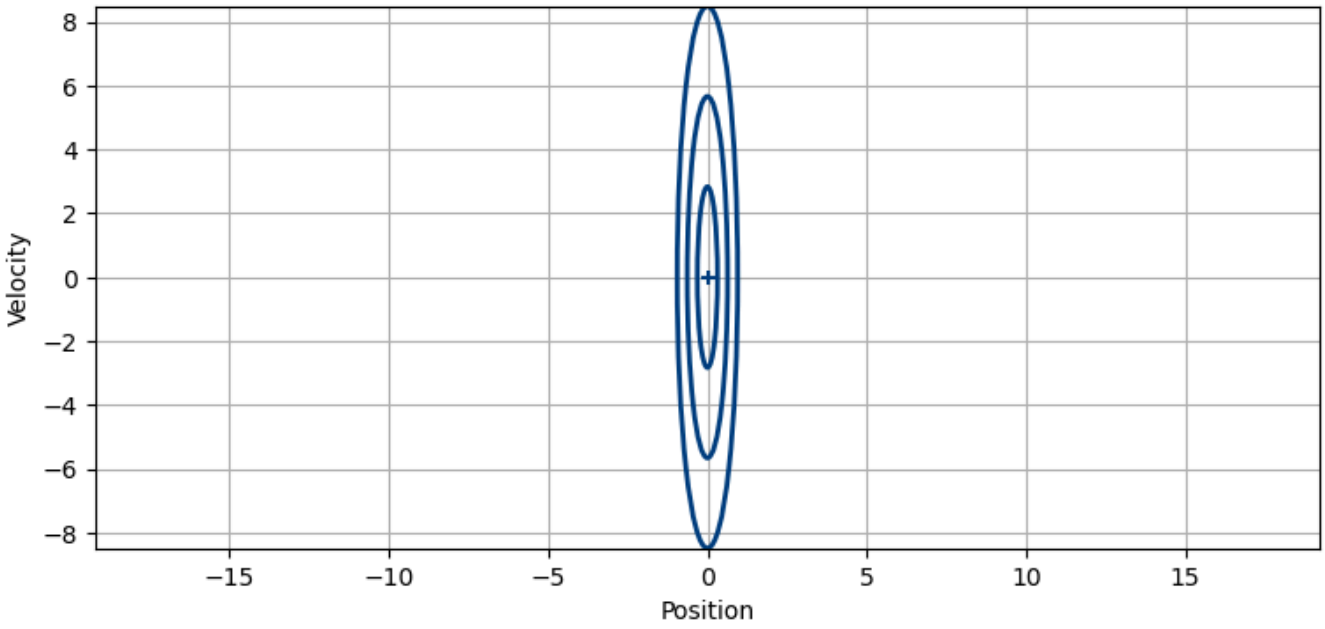
你得出这个结论，是因为你假设飞机的速度是恒定的。合理的假设应该是：在每个时间单位内，飞机在 x 轴和 y 轴上各移动一个单位长度。

回想一下在 g-h 滤波器那一章的内容吧。当时，我们试图改进那些受噪声干扰的体重测量结果。我们将体重变化纳入了计算公式中，因为这样能更准确地预测第二天的体重数值。g-h 滤波器利用 g 参数来调整对当前体重测量的重视程度，而 h 参数则用于调整对体重变化的重视程度。

我们对卡尔曼滤波器也会采用同样的处理方式。毕竟，卡尔曼滤波器本质上也是一种 G-H 滤波器。在这里，我们需要追踪的是飞机的位置和速度，而不是重量或重量的变化。重量的变化其实就是重量的导数，而速度自然就是位置的导数。要绘制出表示 x、y 以及它们对应速度的 4 维图表是不可能的，因此我们只需针对 x 的情况来处理即可。当然，这种数学原理同样适用于更多维度的情况。

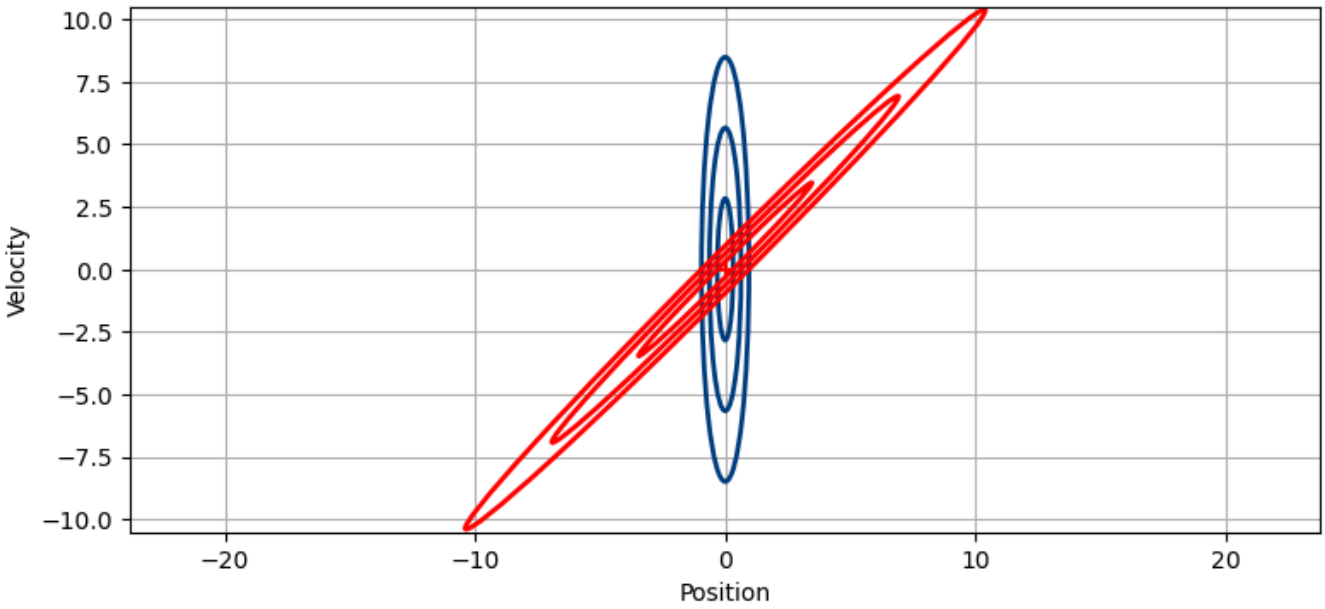
在时间点 1，我们或许能够相当确定物体的位置（x=0），但完全不知道其速度。我们可以用协方差矩阵来表示这一情况。矩阵的宽度越窄，说明我们对物体位置的确定性越高；而矩阵的高度则反映了我们对物体速度的了解程度有多低。

```
mkf_internal.show_x_error_chart(1)
```



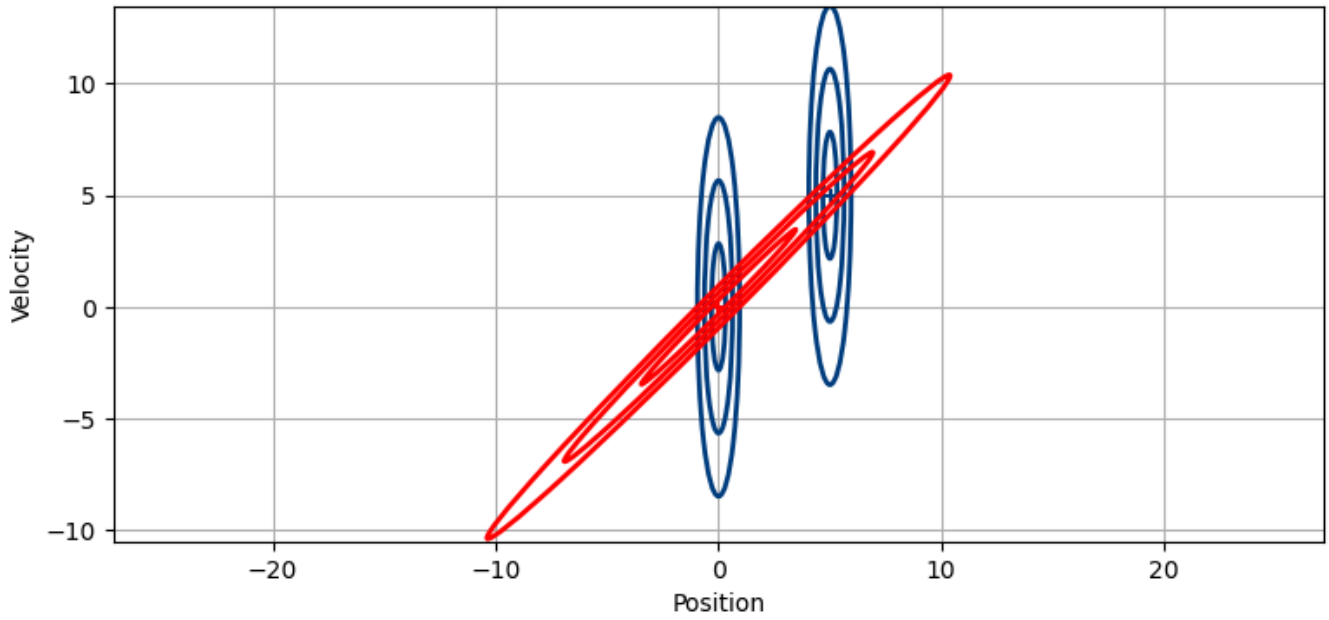
不过，位置和速度是相互关联的。如果速度为 5 米/秒，那么 1 秒钟后，位置就是 5 米。如果速度为-10 米/秒，那么 1 秒钟后，位置就是-10 米。我们可以通过在坐标系中对角线上绘制速度相关图来直观地理解这一点。

```
mkf_internal.show_x_error_chart(2)
```



此时，速度的协方差对我们没有任何帮助。因为我们不知道物体的速度是多少，所以无法预测其新的位置。不过，一秒钟后，我们得到了新的位置信息：x=5。

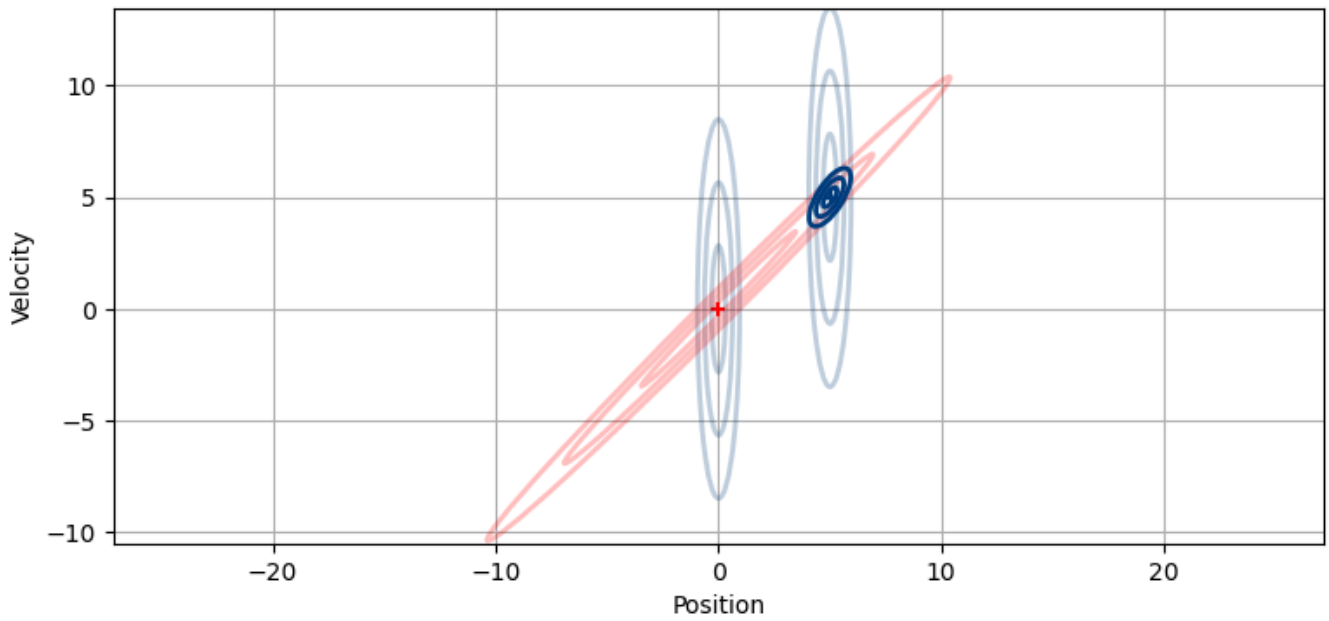
```
mkf_internal.show_x_error_chart(3)
```



直到下一章，我们才能明白我究竟是如何计算出上述结果的。先不必考虑具体的计算过程，只需思考其背后的含义。由于我们对物体速度的测量存在很大不确定性，因此我们无法准确确定物体的实际位置。正因如此，椭圆在 x 轴方向上会延伸得很远。当然，由于速度的不确定性，椭圆在 y 轴方向上也会分布得很广。但正如我在上一段中所说的，物体的位置与速度是相关的。如果速度为 5 米/秒，那么下一个位置就是 5；如果速度为 10 米/秒，下一个位置就是 10。由于两者高度相关，因此椭圆在 y 轴方向上应该相当狭窄。

这两种协方差的叠加效应，正是产生“神奇效果”的关键所在。在 t=1 时刻（此时位置为 5），最合理的估计值大致就是速度协方差与测量协方差的乘积！更准确地说，我们可以运用上一节中的数学公式，将这两个协方差相乘。从贝叶斯的角度来看，我们需要将先验概率与证据的概率相乘，从而得到后验概率。如果我们按照贝叶斯公式将位置协方差与速度协方差相乘，就能得到这样的结果：

```
mkf_internal.show_x_error_chart(4)
```



新的协方差值位于位置协方差和速度协方差的交点上。该值略微倾斜，表明位置和速度之间存在一定的相关性。更重要的是，这个新协方差值远远小于位置协方差和速度协方差中的任何一个。在上一章中，每次进行测量后，方差都会变小，因为之前的估计值会与新的测量结果相乘。这里的情况也是如此。不过，这里的改进效果更为显著。这是因为我们利用了两个相互关联的信息源。只要能大致了解速度及其与位置的相关性，就能得出非常准确的估计值。从图中不太容易看出来，但位置的不确定性确实变小了。我们可以确定这一点，因为新的协方差值在 x 轴上的范围比单独的测量结果要小。在 t=1 时刻的协方差值。这样一来，我们不仅对速度的测量更准确，对位置的测量也更为精确。如果只考虑位置信息而不考虑速度的话，所得到的结果当然会不那么准确。

这是一个关键点，请仔细阅读！雷达只能检测到飞机的位置。这一数值被称为“可观测变量”。根据这些位置数据，我们可以计算出飞机的速度。这个速度则被称为“隐藏变量”。所谓“隐藏变量”，顾名思义，就是没有专门的传感器来测量这个数值，因此我们无法直接得知它的大小。不过，我们可以利用位置和速度之间的关联关系，非常精确地推断出速度的值。

再补充一点术语说明：还有那些无法被直接观测到的变量。例如，飞机的各种状态参数包括飞行方向、发动机转速、重量、颜色、飞行员的姓名等等。我们无法通过位置传感器直接获取这些信息，因此它们属于无法被观测到的变量。也无法通过传感器的数据和相关性来推断出这些变量（比如，红色飞机不会比白色飞机飞得更快）。所以，它们并非“被隐藏了”的变量，而只是根本无法被观测到的变量而已。如果在模型中加入了这种无法被观测到的变量，那么对该变量的估算结果将毫无意义。

是什么让这一点成为可能呢？试想一下，如果我们把另一架飞机的速度数据叠加在位置图上会怎样。显然，这两者之间没有任何关联，将它们结合起来也无法获得任何有用的信息。而实际上，这架飞机的速度数据却能告诉我们非常重要的信息——飞机的行进方向和速度。只要飞机的速度保持不变，我们就能根据其当前速度来预测它下一步会出现在哪里。如果速度的测量误差较大，那么该速度数据与实际位置的匹配度就会很低。想想看，如果你突然改变方向，那么你的位置也会发生很大变化。如果位置数据的测量方向与速度变化的方向不一致，那么这个数据就很可能是不准确的。这两者是相互关联的：只要速度发生变化，位置也必然会随之变化，而且这种变化是可预测的。

重要的是要明白：我们利用了速度与位置之间存在关联这一事实。我们通过两次测量之间的距离和时间来大致估算速度，然后再利用贝叶斯定理，在进行少量观测后就能得到非常精确的估计值。如果您有任何疑问，请重新阅读这一部分。如果不理解这一点，那么您将很难理解后续章节的内容。

在我看来，如果只关心位置的话，考虑速度所带来的影响其实微乎其微。不过这只是在只进行一次更新之后的情况。在下一章中，我们将看到：经过多次更新后，我们的预测精度会有多么显著的提升。虽然测量结果的方差会很大，但估计出的位置方差则会很小。每次将速度的协方差与位置数据相结合后，x 轴上的方差都会变小，因此每次更新后的方差也会随之减小。

更高维度

到目前为止，我向你们展示了二维高斯分布，但实际上，数学原理并不局限于二维空间。在后面的章节中，我们将研究九维或十二维的空间。如果你从事天气预报等领域的工作，那么你可能会面对成千上万的维度。

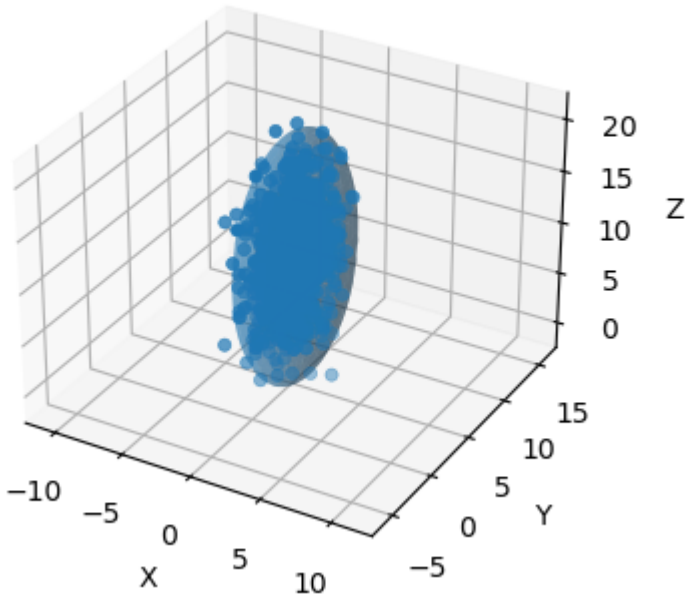
这些高维空间究竟“长什么样”呢？其实，二维高斯分布可以用误差椭圆来表示，因此，三维高斯分布自然可以用三维误差椭球来表示。这里就不详细讨论数学原理了，但事实证明确实如此。 `FilterPy` 提供了一个用于绘制这种椭球的函数。

首先，让我们生成一些具有特定协方差的随机数据，这样我们就可以把它们绘制在椭球体内部了。

```
from filterpy.stats import plot_3d_covariance

mu = [0.3, 5., 10.]
C = np.array([[1.0, .03, .2],
              [.03,  4.0, .0],
              [.2,  .0, 16.1]])

sample = np.random.multivariate_normal(mu, C, size=1000)
```

现在，我们使用 `FilterPy` 函数 `plot_3d_covariance` 来绘制椭球体，然后再将各个样本点以散点图的形式展示出来。

```
ax = plot_3d_covariance(mu, C, alpha=.4, std=3, limit_xyz=True)
ax.scatter(sample[:, 0], sample[:, 1], zs=sample[:, 2],);
```



根据理论，大约 99%的数据点会落在 3 个标准差范围内，实际情况似乎也确实如此。

九个维度？我还不清楚该如何在二维屏幕上绘制出九维椭球体，所以无法展示相关的图表。不过其原理是一样的：该分布的标准差误差可以用九维椭球体来描述。

摘要/总结

我们充分利用了该系统的几何特性和各种相关性，从而得到了非常精确的估算结果。在数学上，我们不必关心所处理的究竟是两个数值、一个数值与一个相关联的速度，还是其他类型的空间参数。如果房屋面积与房价之

间存在相关性，那么就可以用卡尔曼滤波器来追踪房价变化。同样地，如果年龄与疾病发病率相关，也可以用卡尔曼滤波器来监测疾病趋势。如果僵尸数量与猎枪的数量呈负相关，那么同样可以用卡尔曼滤波器来追踪僵尸数量的变化。我之前用几何概念来解释这一点，还提到了三角测量法。这么做只是为了帮助大家更好地理解这个原理。其实，即使某些状态变量没有明确的几何表现形式，也可以用卡尔曼滤波器来处理，比如股票价格或牛奶产量等数据（我甚至收到过有人用卡尔曼滤波器来追踪牛奶产量的邮件！）试着把这些看作是具有相关性的高斯分布吧。如果我们能把各种不确定性表示为多维高斯分布，那么就可以将先验概率与似然值相乘，从而得到更精确的结果。

参考文献

- [1] http://docs.scipy.org.cn/doc/scipy/reference/tutorial/stats.html
- [2] `FilterPy` 库。Roger Labbe 开发。https://github.com/rlabbe/filterpy

第 6 章：多元卡尔曼滤波器

多个随机变量的滤波处理

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

我们现在已准备好来研究并应用卡尔曼滤波器的完整多变量形式。在上一章中，我们了解了如何用多变量高斯分布来描述多个随机变量之间的相关性，比如飞机的位置和速度之间的关系。我们还了解到，变量之间的相关性能够显著提升后验概率的准确性。如果我们只能大致知道飞机的位置和速度，但这两个量之间存在相关性，那么我们的估算结果将会非常精确。

我建议你们通过几个具体的例子来了解这些滤波器的运作原理。我会略过许多细节。我所展示的某些方法只适用于特定情况；而另一些方法则显得“神奇”不已——你可能不明白我是怎么得出某个结果的。如果我从严格的、普遍适用的数学公式入手，你们肯定会搞不清楚这些符号的含义，也不明白如何将它们应用到自己的问题中。在后面的章节中，我会提供更严谨的数学基础。到那时，我就需要要么修正本章中的某些近似处理，要么补充这里没有提到的信息。

为了实现这一目标，我们将把研究范围限制在那些可以用牛顿运动方程来描述的问题的子集上。这类滤波器被称为离散化连续时间运动学滤波器。在卡尔曼滤波器的数学原理部分，我们将探讨如何处理非牛顿系统的相关数学问题。

牛顿运动定律

牛顿运动定律告诉我们：只要知道系统在某一时刻的恒定速度 v，我们就可以计算出经过时间 t 后该系统的位置 x。具体计算公式为：

$$x = vt + x_0$$

例如，如果我们从位置 13 开始移动，速度为 10 米/秒，经过 12 秒后，我们的最终位置将是 133（$10 \times 12 + 13$）。

我们可以用这个公式来表示匀加速运动。

$$x = \frac{1}{2}at^2 + v_0t + x_0$$

如果我们假设加速度保持恒定，那么就会得到……

$$x = \frac{1}{6}jt^3 + \frac{1}{2}a_0t^2 + v_0t + x_0$$

这些方程是通过积分微分方程得到的。只要给定一个恒定的速度 v，我们就可以利用该方程来计算物体随时间所行驶的距离。

$$x = vt + x_0$$

我们可以通过以下方式推导出来：

$$\begin{aligned}
v &= \frac{dx}{dt} \\
dx &= vdt \\
\int_{x_0}^{x} dx &= \int_0^t vdt \\
x - x_0 &= vt - 0 \\
x &= vt + x_0
\end{aligned}$$

在设计卡尔曼滤波器时，首先需要建立一组描述系统动态的微分方程。不过，大多数微分方程并不容易用这种方式来求解。我们选择使用牛顿方程，因为其可以很容易地被积分，从而得到封闭形式的解。这样一来，卡尔曼滤波器的设计就变得简单多了。此外，牛顿方程正是用于追踪移动物体的理想方程，而这也是卡尔曼滤波器的主要应用之一。

卡尔曼滤波算法

该算法与我们在前几章中使用的贝叶斯滤波算法相同。更新步骤稍微复杂一些，不过我们会在具体讲解时详细说明原因。

初始化

```
1. Initialize the state of the filter
2. Initialize our belief in the state
```

预测

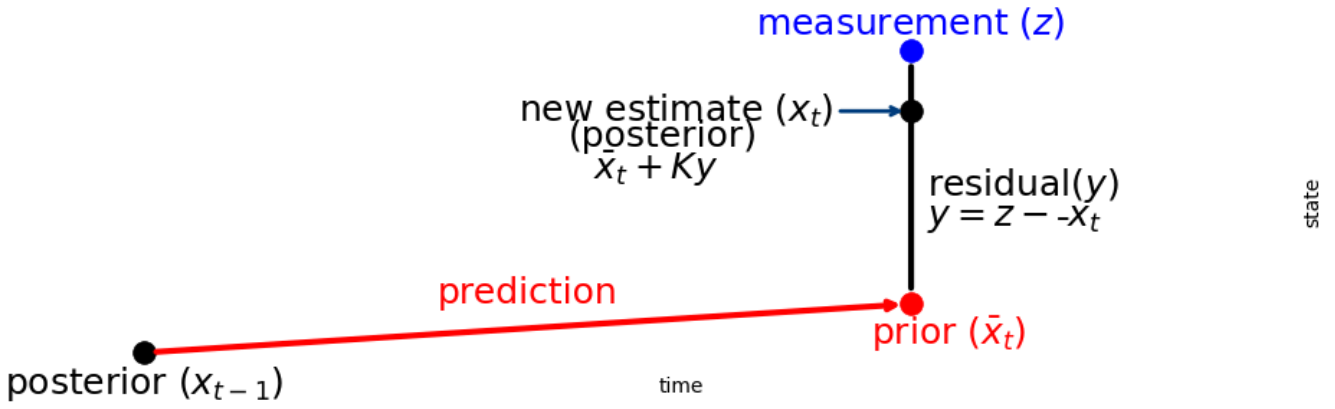
```
1. Use process model to predict state at the next time step
2. Adjust belief to account for the uncertainty in prediction
```

更新

```
1. Get a measurement and associated belief about its accuracy
2. Compute residual between estimated state and measurement
3. Compute scaling factor based on whether the measurement
or prediction is more accurate
4. set state between the prediction and measurement based
on scaling factor
5. update belief in the state based on how certain we are
in the measurement
```

作为提醒，以下是该算法的图形化展示：

```
import kf_book.book_plots as book_plots
book_plots.show_residual_chart()
```



单变量卡尔曼滤波器用单变量高斯分布来描述系统的状态。显然，多变量卡尔曼滤波器则使用多变量高斯分布来描述系统的状态。在上一章中我们了解到，多变量高斯分布用向量来表示均值，用矩阵来表示协方差。这意味着，卡尔曼滤波器在进行状态估计时需要运用线性代数的知识。

我不希望你们死记硬背这些方程式，不过我把单变量和多变量方程式都列在下面了。它们其实很相似。

预测

Univariate	Univariate (Kalman form)	Multivariate
$\bar{\mu} = \mu + \mu_{f_x}$	$\bar{x} = x + dx$	$\bar{x} = Fx + Bu$
$\bar{\sigma}^2 = \sigma_x^2 + \sigma_{f_x}^2$	$\bar{P} = P + Q$	$\bar{P} = FPF^T + Q$

无需过于纠结于线性代数的具体细节，我们可以看出：

x, P 分别代表该状态的均值和协方差。它们对应于 x 和 σ^2 。

F 是状态转换函数。当与 x 相乘时，就可以得到先验概率。

Q 表示的是过程的协方差。它与 $\sigma_{f_x}^2$ 相对应。

B 和 u 对我们来说都是新的概念。它们让我们能够对系统的控制输入进行建模。

更新

Univariate	Univariate (Kalman form)	Multivariate
	$y = z - \bar{x}$	$y = z - H\bar{x}$
	$K = \frac{\bar{P}}{\bar{P}+R}$	$K = \bar{P}H^T(H\bar{P}H^T + R)^{-1}$
$\mu = \frac{\bar{\sigma}^2\mu_z + \sigma_z^2\bar{\mu}}{\bar{\sigma}^2 + \sigma_z^2}$	$x = \bar{x} + Ky$	$x = \bar{x} + Ky$
$\sigma^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}$	$P = (1-K)\bar{P}$	$P = (I-KH)\bar{P}$

H 是一种测量函数。我们在本书中还没有见过这种函数，稍后我会详细解释。如果你在心中把 H 从这些方程式中去掉，就会发现这些方程式其实是非常相似的。

z, R 表示测量值的均值，σ_z^2 则表示噪声的协方差。它们分别对应于单变量滤波器中的 z 和 σ_z^2。为了使符号表示尽可能一致，我在单变量方程中用 x 替换了 μ。

y 和 K 分别代表残差和卡尔曼增益。

与单变量滤波器相比，其具体实现方式会有所不同，因为这里处理的是向量和矩阵。不过，其核心概念是完全相同的。

- 用高斯分布来表示我们对状态和误差的估计。
- 用高斯分布来表示测量值及其误差。
- 使用高斯分布来表示该过程模型。
- 利用该过程模型来预测下一个状态。
- 估算值应介于实际测量结果和之前的数值之间。

作为设计师，你的任务是设计状态（x, P）、过程（F, Q）、测量方式（z, R）以及测量函数 H。如果该系统包含各种控制输入装置，比如机器人，那你还需要设计 B 和 u。

我已经将卡尔曼滤波器的计算公式编写进了 FilterPy 中的 `predict` 和 `update` 函数中。你可以通过以下方式来导入这些函数：

```
from filterpy.kalman import predict, update
```

追踪一只狗

让我们回到那个经过验证的、用于追踪狗的算法上来。这一次，我们会运用上一章中提到的基本原理，利用“隐藏变量”来提升我们的估算精度。虽然数学公式看起来有所不同，可能也比前面的章节更复杂，但核心思想其实是一样的——我们只不过是在对高斯函数进行各种运算而已。

我们首先为这只狗编写模拟程序。该模拟会进行 `count` 步操作，每步狗会向前移动大约 1 米。在每一步中，狗的速度会根据过程方差 `process_var` 而发生变化。在更新了狗的位置之后，我们会根据假设的传感器方差 `z_var` 来计算相应的测量值。该函数会返回一个包含狗位置的 NumPy 数组，以及另一个包含测量值的数组。

```
import math
import numpy as np
from numpy.random import randn

def compute_dog_data(z_var, process_var, count=1, dt=1.):
    "returns track, measurements 1D ndarrays"
    x, vel = 0., 1.
    z_std = math.sqrt(z_var)
    p_std = math.sqrt(process_var)
    xs, zs = [], []
    for _ in range(count):
        v = vel + (randn() * p_std)
        x += v*dt
        xs.append(x)
```

```
        zs.append(x + randn() * z_std)
    return np.array(xs), np.array(zs)
```

预测步骤

在进行预测时，我们需要确定状态和协方差、过程模型以及过程噪声，必要时还要确定控制输入。我们将依次来处理这些问题。

设计状态变量

我们之前使用高斯函数来描述一只狗在一维空间中的位置。均值 (μ) 代表了该位置最可能出现的值，而方差 σ^2 则代表了该位置的概率分布。位置可以被视为系统的状态，我们把 μ 称为状态变量。

在这个问题中，我们需要同时追踪狗的位置和速度。为此，我们需要使用多元高斯分布来描述这些数据，该分布由状态向量 x 及其对应的协方差矩阵 P 来表示。

状态变量可以是可观测变量——即由传感器直接测量的数值；也可以是隐藏变量——需要从可观测变量中推断出来。在狗的跟踪问题中，传感器只能测量位置信息，因此位置属于可观测变量，而速度则属于隐藏变量。我们很快就会学习如何处理这些隐藏变量。

需要明白的是，对位置和速度的追踪其实是一种设计上的选择，而这种选择背后涉及各种假设和影响，我们目前还来不及深入探讨。例如，我们也可以追踪加速度，甚至是加速度的变化率。先回想一下，上一章中我们提到，如果在协方差矩阵中加入速度这一项，那么位置的方差会大大减小。在本章的后面部分，我们将了解卡尔曼滤波器是如何计算隐藏变量值的。

在单变量分析中，我们用一个标量值来表示狗的位置（例如 $\mu = 3.27$ ）。在上一章中，我们学习了如何使用多元高斯分布来处理多个变量。例如，如果我们想指定狗的位置为 10.0 米，速度为 4.5 米/秒，那么我们可以这样表示：

$$\mu = \begin{bmatrix} 10.0 \\ 4.5 \end{bmatrix}$$

卡尔曼滤波器是通过线性代数来实现的。我们使用矩阵来存储各种状态变量。在狗的跟踪问题中，我们用某个矩阵来表示狗的位置；而该矩阵的第一个导数则用来表示狗的速度。在计算导数时，我使用了牛顿点符号。例如，$\dot{x}$ 表示 x 关于 t 的第一个导数。卡尔曼滤波器的公式中，我们用 x 来表示状态变量，因此我们定义 x 为：

$$x = \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$

我们使用 x 来代替 μ，但明白这其实只是多元高斯分布的均值而已。

另一种表示方法是 $x = \begin{bmatrix} x & \dot{x} \end{bmatrix}^T$，因为行向量的转置就是列向量。这种表示法在文本中更易于使用，因为它占用的垂直空间更少。

x 和 x 这两个位置的名字恰好相同。如果我们是在 y 轴上追踪那只狗的移动轨迹，那么应该使用 $x = \begin{bmatrix} y & \dot{y} \end{bmatrix}^T$ 来表示，而不是 $y = \begin{bmatrix} y & \dot{y} \end{bmatrix}^T$。$x$ 是卡尔曼滤波算法中用于表示状态变量的标准名称，我们不会更改它的名称以使其更易于理解。这种命名上的统一性有助于我们与同行进行有效的沟通。

让我们来编写代码吧。 x 的初始化过程非常简单。

```
x = np.array([[10.0],
              [4.5]])
x
```

```
array([[10. ],
       [ 4.5]])
```

在我的代码中，我经常使用转置操作将行矩阵转换为列向量。我觉得这样处理起来既方便输入，也便于阅读。

```
x = np.array([[10., 4.5]]).T
x
```

```
array([[10. ],
       [ 4.5]])
```

不过，NumPy 将一维数组视为向量处理，因此我可以简化这行代码，直接使用一维数组即可。

```
x = np.array([10.0, 4.5])
x
```

```
array([10. ,  4.5])
```

数组中的所有元素都具有相同的类型，通常为 `float` 或 `int` 。如果列表中全部都是 `int` 元素，那么所创建的数组的数据类型也为 `int` ； 否则，数据类型为 `float` 。我经常利用这一特性，只需指定一个浮点数即可。

```
np.array([1., 0, 0, 0, 0, 0])
```

```
array([1., 0., 0., 0., 0., 0.])
```

以下是一些例子。

```
A = np.array([[1, 2], [3, 4]])
x = np.array([[10.0], [4.5]])

# matrix multiply
print(np.dot(A, x))
print()
```

```
[[19.]
 [48.]]
```

在 Python 3.5 及更高版本中，引入了矩阵乘法运算符“@”。不过，它的实用性其实没那么高，因为它要求参与运算的 `A` 和 `B` 都必须是数组形式。不过，在本书的数学计算中，有些变量可以是标量值，因此“@”运算符的实用性就大打折扣了。

```
# alternative matrix multiply)
print(A @ x)
print()

x = np.array([[10.0, 4.5]]).T
print(A @ x)
print()

x = np.array([10.0, 4.5])
print(A @ x)
```

```
[[19.]
 [48.]]

[[19.]
 [48.]]

[19. 48.]
```

最后一个函数会返回一维数组，不过我设计的卡尔曼滤波器类能够处理这种情况。不过，从后看来，这种设计可能会造成一些混淆，但实际上它还是可以正常工作的。

设计状态协方差

状态高斯分布的另一半则对应于协方差矩阵 P。在单变量卡尔曼滤波器中，我们需要为 σ^2 指定一个初始值，之后随着新测量数据的加入，滤波器会自动更新该数值。多变量卡尔曼滤波器也是如此：我们为 P 指定初始值，然后滤波器会在每个时间步长中不断更新该数值。

我们需要将各种数值设定在合理的范围内。例如，如果我们对物体的初始位置不太确定，可以选择 $\sigma^2_{pos} = 500m^2$ 作为数值。狗的最高奔跑速度约为 21 米/秒，因此，在没有其他关于速度的信息时，我们可以将 $3\sigma_{vel} = 21$ 或 $\sigma^2_{vel} = 7^2 = 49$ 设为这个数值。

在上一章中，我们了解到物体的位置和速度之间存在着相关性。不过，对于狗来说，这种相关性到底有多大呢？我不得而知。不过，正如我们将会看到的，滤波器会自动计算出这种相关性。因此，我先将各种协方差值初始化为零。当然，如果你知道这些协方差值的话，当然应该直接使用它们。

请记住，协方差矩阵的对角线元素分别代表各个变量的方差，而非对角线元素则代表各变量之间的协方差。因此，我们有：

$$P = \begin{bmatrix} 500 & 0 \\ 0 & 49 \end{bmatrix}$$

我们可以使用 `numpy.diag` 来创建对角矩阵。对角矩阵的特点是：非对角线上的元素都为零。这一点在线性代数中有所介绍。

```
P = np.diag([500., 49.])
P
```

```
array([[500.,   0.],
       [  0.,  49.]])
```

我本可以这样写：

```
P = np.array([[500., 0.],
              [0., 49.]])
P
```

```
array([[500.,   0.],
       [  0.,  49.]])
```

我们完成了。我们已经将滤波器的状态表示为多元高斯分布，并在代码中实现了这一模型。

设计流程模型

下一步是设计过程模型。这是一种用于描述系统行为的数学模型。该模型被滤波器用来预测在经过一个离散时间步长之后的系统状态。我们通过一组描述系统动态的方程来实现这一目标。

在单变量分析部分，我们用……来模拟狗的运动轨迹。

$$x = v\Delta t + x_0$$

我们的实施方式如下：

```
def predict(pos, movement):
    return gaussian(pos.mean + movement.mean,
                    pos.var + movement.var)
```

在本章中，我们将采用同样的方法，只不过是用多元高斯分布来代替单变量高斯分布。你可以这样想象这种实现方式：

$$x = \begin{bmatrix} 5.4 \\ 4.2 \end{bmatrix}, \dot{x} = \begin{bmatrix} 1.1 \\ 0. \end{bmatrix} x = \dot{x}t + x$$

不过，我们需要将这一原理推广到更一般的情况。卡尔曼滤波方程适用于任何线性系统，而不仅仅是牛顿系统。也许，你想要进行滤波处理的系统是化工厂中的管道系统：某根管道中的流体流量，实际上是由各个阀门的不同调节方式所共同决定的。

$$pipe_1 = 0.134(valve_1) + 0.41(valve_2 - valve_3) + 1.34$$

$$pipe_2 = 0.210(valve_2) - 0.62(valve_1 - valve_5) + 1.86$$

线性代数提供了一种处理方程组的有效方法。以这个方程组为例吧。

$$\begin{cases} 2x + 3y = 8 \\ 4x - y = 2 \end{cases}$$

我们可以将其表示为矩阵形式，即：

$$\begin{bmatrix} 2 & 3 \\ 4 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$$

如果按照这个方程式进行矩阵乘法运算，得到的结果就是上面那两个方程式。在线性代数中，我们会用 $Ax = B$ 来表示这个运算。

$$A = \begin{bmatrix} 2 & 3 \\ 4 & -1 \end{bmatrix}, x = \begin{bmatrix} x \\ y \end{bmatrix}, B = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$$

然后，我们可以利用 SciPy 的 `linalg` 包来求解 x 的值：

```
from scipy.linalg import solve
A = np.array([[2, 3],[4, -1]])
b = np.array([[8], [2]])
x = solve(A, b)
x
```

```
array([[1.],
       [2.]])
```

我们利用该过程模型来进行创新，因为这些方程式能够告诉我们：在给定当前状态的情况下，下一个状态会是什么。卡尔曼滤波器正是利用这些线性方程式来实现这一功能的，其中 $\bar{x}$ 代表先验状态或预测状态。

$$\bar{x} = Fx$$

我们可以将其明确表示为：

$$\begin{bmatrix} \bar{x} \\ \dot{\bar{x}} \end{bmatrix} = \begin{bmatrix} ? & ? \\ ? & ? \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$

作为卡尔曼滤波器的设计者，我们的任务是确定相关的参数 F，以便让 $\bar{x} = Fx$ 能够为我们的系统进行预测。为此，我们需要为每个状态变量都建立一个方程式。在我们的问题中，有 $x = \begin{bmatrix} x & \dot{x} \end{bmatrix}^T$ 个状态变量，因此我们需要一个方程来计算位置 x，另一个方程来计算速度 $\dot{x}$。我们已经知道了用于计算位置的方程式：

$$\bar{x} = x + \dot{x}\Delta t$$

那么，关于速度的公式是什么呢？我们没有任何模型可以预测狗狗的速度会如何随时间变化。在这种情况下，我们假设在每次“创新”发生时，狗狗的速度保持不变。当然，实际情况并非完全如此。不过，只要每次“创新”之后狗狗的速度变化不大，那么该滤波器的效果就会相当好。所以我们说：

$$\dot{\bar{x}} = \dot{x}$$

这就构成了我们系统的流程模型。

$$\begin{cases} \bar{x} &= x + \dot{x}\Delta t \\ \dot{\bar{x}} &= \dot{x} \end{cases}$$

该方程式正确地体现了状态中每个变量都对应一个方程式，且这些方程式都单独位于等式的左侧。我们需要将这组方程式表示为 $\bar{x} = Fx$ 的形式。重新排列各项后，就能更清楚地知道该如何处理了。

$$\begin{cases} \bar{x} &= 1x + \Delta t\dot{x} \\ \dot{\bar{x}} &= 0x + 1\dot{x} \end{cases}$$

我们可以将其以矩阵形式重新表示为：

$$\begin{bmatrix} \bar{x} \\ \bar{\dot{x}} \end{bmatrix} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$
$$\bar{x} = Fx$$

F 被称为状态转移函数或状态转移矩阵。在后面的章节中，它实际上是一个普通的函数，而非矩阵。因此，称其为“函数”更为准确。

```
dt = 0.1
F = np.array([[1, dt],
              [0, 1]])
F
```

```
array([[1. , 0.1],
       [0. , 1. ]])
```

让我们来测试一下吧！FilterPy 有一个名为 `predict` 的方法，该方法通过计算 $\bar{x} = Fx$ 来进行预测。我们先调用这个方法，看看会发生什么。我们已经将位置设置为 10.0 米，速度设置为 4.5 米/秒。同时，我们还定义了 `dt = 0.1`，这意味着时间步长为 0.1 秒。因此，我们预计在经过一次计算后，新的位置应该是 10.45 米。而速度应该保持不变。

```
from filterpy.kalman import predict

x = np.array([10.0, 4.5])
P = np.diag([500, 49])
F = np.array([[1, dt], [0, 1]])

# Q is the process noise
x, P = predict(x=x, P=P, F=F, Q=0)
print('x =', x)
```

```
x = [10.45  4.5 ]
```

这个方法有效。如果我们连续多次调用 `predict()`，其值会每次都被更新。

```
for _ in range(4):
    x, P = predict(x=x, P=P, F=F, Q=0)
    print('x =', x)
```

```
x = [10.9  4.5]
x = [11.35  4.5 ]
x = [11.8  4.5]
x = [12.25  4.5 ]
```

`predict()` 同时计算了该“创新值”的均值和协方差。这就是在进行了五次“创新预测”之后 P 的值，我们在卡尔曼滤波方程中将其记为 $\bar{P}$。

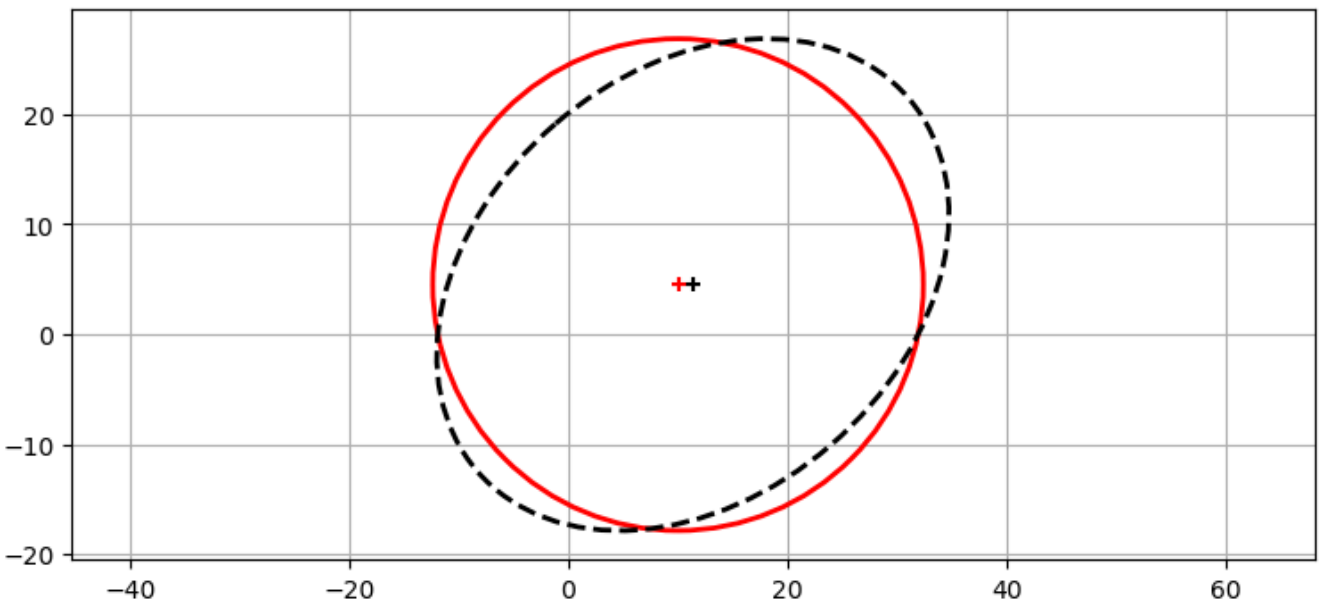
```
print(P)
```

```
[[512.25  24.5 ]
 [ 24.5   49.  ]]
```

通过观察对角线上的数值，我们可以发现位置方差有所增大。我们进行了五次预测，但并未进行任何测量，因此不确定性也增加了。非对角线上的元素不再为零——卡尔曼滤波器检测到了位置和速度之间的相关性！不过，速度的方差则没有发生变化。

在这里，我绘制了预测前后协方差的变化情况。初始值用实线红色表示，而预测结果则用虚线黑色表示。为了更清楚地展示这种变化，我还调整了协方差和时间步长。

```
from filterpy.stats import plot_covariance_ellipse

dt = 0.3
F = np.array([[1, dt], [0, 1]])
x = np.array([10.0, 4.5])
P = np.diag([500, 500])
plot_covariance_ellipse(x, P, edgecolor='r')
x, P = predict(x, P, F, Q=0)
plot_covariance_ellipse(x, P, edgecolor='k', ls='dashed')
```



可以看到，由于位置发生了变化，椭圆的中心也发生了轻微的移动（从 10 变为 11.35）。同时，椭圆的形状也发生了变化，这体现了位置与速度之间的关联。那么，该滤波器是如何计算出 $\bar{P}$ 的新值的呢？其计算依据又是什么？需要说明的是，我每次都会将过程噪声 Q 设置为零，因此这种变化并非由我人为添加的噪声所导致的。现在讨论这个问题还为时过早，但请记住，在迄今为止所讨论的每一个滤波器中，预测过程都伴随着信息的丢失。这里也是如此。等我们进一步探讨之后，我会详细解释这一点。

设计过程中的各种干扰/噪声

简单回顾一下与系统噪声相关的问题。假设一辆汽车处于定速巡航状态在道路上行驶，那么它的速度应该保持不变。我们可以用 $\bar{x}_k = \dot{x}_k \Delta t + x_{k-1}$ 来模拟这一过程。不过，实际行驶过程中会受到许多不可知因素的影响。

定速巡航系统无法完美地保持恒定速度。风速、道路上的坡度以及坑洼都会对汽车的行驶产生影响。此外，乘客如果摇下窗户，也会改变汽车的空气阻力。

我们可以用微分方程来描述这个系统。

$$\dot{x} = f(x) + w$$

其中， $f(x)$ 表示状态转换过程， w 则代表白色过程噪声。

在“卡尔曼滤波器数学原理”这一章中，我们将学习如何将一组微分方程转化为卡尔曼滤波器中的矩阵。在这一章中，我们利用了牛顿已经为我们推导出的运动方程这一事实。目前你只需知道：我们通过将过程噪声协方差矩阵 Q 加到协方差矩阵 P 中来考虑系统中的噪声因素。至于 x ，我们不需要做任何修改，因为该处的噪声属于“白噪声”——也就是说，噪声的均值为 0。如果均值为 0，那么 x 的值就不会发生变化。

单变量卡尔曼滤波器使用 `variance = variance + process_noise` 来计算预测结果的方差。多变量卡尔曼滤波器的处理方式类似，本质上也是 `P = P + Q` 。我之所以说“本质上类似”，是因为在协方差方程中，还有其他与噪声无关的项，我们稍后会详细讨论。

推导过程噪声矩阵相当复杂，我们暂不处理这个问题，留到卡尔曼滤波的数学部分再讨论。目前只需知道： Q 实际上就是白噪声 w 的期望值，其计算方式如 $Q = E[ww^T]$ 所示。在本章中，我们将重点理解：通过调整这个矩阵，滤波器的性能会如何发生变化。

FilterPy 提供了用于计算本章中各种运动学问题相关参数的函数。 `Q_discrete_white_noise` 函数需要 3 个参数： `dim` 用于指定矩阵的维度； `dt` 表示以秒为单位的时间步长； `var` 则代表噪声的方差。简而言之，该函数在给定的时间范围内对噪声进行离散化处理，具体的假设条件我们稍后会详细讨论。以下代码用于计算方差为 2.35、时间步长为 1 秒的白噪声相关参数：

```
from filterpy.common import Q_discrete_white_noise
Q = Q_discrete_white_noise(dim=2, dt=1., var=2.35)
print(Q)
```

```
[[0.588 1.175]
 [1.175 2.35 ]]
```

设计控制函数

卡尔曼滤波器不仅仅用于数据的滤波处理，它还能帮助我们把机器人、飞机等系统的控制指令也纳入滤波过程之中。假设我们正在控制一个机器人。在每个时间点上，我们都会根据机器人当前的位置与期望位置之间的关系，向机器人发送相应的控制信号。卡尔曼滤波器的算法会将这些信息纳入计算公式中，从而根据当前的速度以及驱动电机所接收到的控制指令来预测机器人的位置。请记住，我们绝不会丢弃任何有用的信息。

对于线性系统而言，控制输入所产生的效果可以用一组线性方程来描述。用线性代数的方式来表达，就是：

$$\Delta x = Bu$$

这里， u 是控制输入，而 B 则是控制输入的数学模型或控制函数。例如， u 可能是一个电压值，它决定了轮毂电机转动的速度；将 B 乘以 u 后，得到的结果就是 $\Delta\left[\begin{smallmatrix} x \\ \dot{x} \end{smallmatrix}\right]$ 。换句话说，需要计算出由于控制输入而使得 x 发生

多少变化。

因此，关于先验均值的完整卡尔曼滤波方程为：

$$\bar{x} = Fx + Bu$$

而这就是当你调用 `KalmanFilter.predict()` 时所计算出的公式。

你的狗可能已经被训练成能够对语音指令做出反应。不过，所有现有证据都表明，我的狗完全无法对任何指令做出反应。因此，我将 B 的值设为零。在 Python 中，我们可以这样编写代码：

```
B = 0.  # my dog doesn't listen to me!
u = 0
x, P = predict(x, P, F, Q, B, u)
print('x =', x)
print('P =', P)
```

```
x = [12.7  4.5]
P = [[680.587 301.175]
 [301.175 502.35 ]]
```

将 B 和 u 设置为零是没有必要的，因为 `predict` 的默认值本身就是 0。

```
predict(x, P, F, Q)[0] == predict(x, P, F, Q, B, u)[0]
```

```
array([ True,  True])
```

```
predict(x, P, F, Q)[1] == predict(x, P, F, Q, B, u)[1]
```

```
array([[ True,  True],
       [ True,  True]])
```

预测：总结

作为设计师，你的职责就是确定相关的矩阵参数。

- x 、 P ：状态与协方差
- F 、 Q ： 过程模型与噪声协方差
- B, u ： 可选：控制输入和功能设置

更新步骤

现在我们可以开始执行该滤波器的更新步骤了。你只需要再提供两个矩阵即可，这两个矩阵的含义相当简单易懂。

设计测量函数

卡尔曼滤波器在我们所说的“测量空间”中计算出所需的更新值。在讨论单变量情况的章节中，我们基本上忽略了这个问题，因为它会增加计算的复杂性。我们通过传感器来获取狗狗的位置信息。计算残差其实很简单：只需用测量得到的实际位置减去滤波器预测的位置即可。

$$residual = measuredposition - predictedposition$$

我们需要计算残差值，因为要将其乘以卡尔曼增益，从而得到新的估计值。

如果我们试图使用那种将温度数值转换为电压信号的体温计来测量温度，会怎么样呢？在这种情况下，用于计算残差的公式就毫无意义了——因为无法从电压中减去“温度”这个数值。

$$residual = voltage - temperature(NONSENSE!)$$

我们需要将温度转换为电压，这样才能进行减法运算。对于温度计来说，我们可以这样表示：

```
CELSIUS_TO_VOLTS = 0.21475
residual = voltage - (CELSIUS_TO_VOLTS * predicted_temperature)
```

卡尔曼滤波器通过让用户提供一个将状态转换为测量值的函数，从而解决了这个问题。

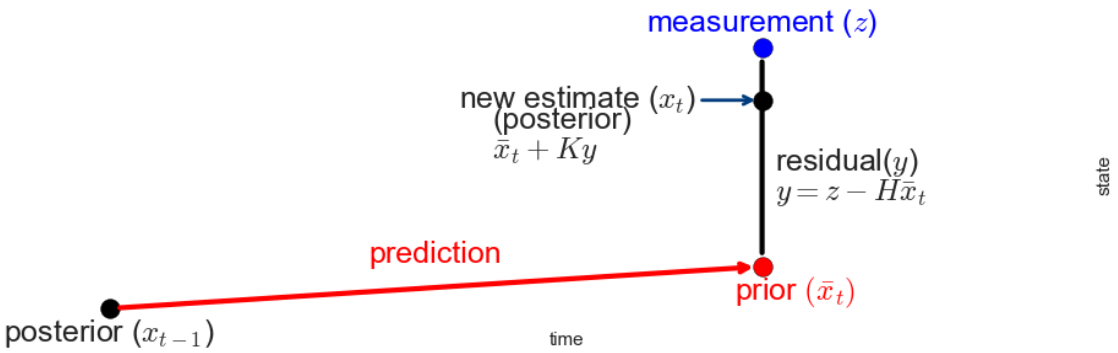
我们为什么要在“测量空间”中进行操作呢？为什么不将电压转换为温度，从而在“状态空间”中进行处理呢？这样一来，剩余的数值就可以表示为温度差了。

我们无法这么做，因为大多数测量结果都是不可逆的。在跟踪问题中，状态包含着隐藏变量 $\dot{x}$ 。我们无法将位置测量结果转换成包含速度的状态。另一方面，将包含位置和速度的状态转换成仅包含位置的“测量结果”则非常简单。为了能够计算残差值，我们必须在测量空间中进行计算。

测量值 z 和状态值 x 都是向量，因此我们需要使用矩阵来进行转换。执行这一操作的卡尔曼滤波方程如下：

$$y = z - H\bar{x}$$

其中， y 表示残差， $\bar{x}$ 表示先验值， z 表示测量值， H 则代表测量函数。我们先取先验值，再将其与 H 相乘，从而得到一个“测量值”。接着，用这个“测量值”减去原始的测量值。这样，我们就得到了预测值与实际测量值之间的差异！



我们需要对 H 进行设计，以便 $H\bar{x}$ 能够产生可测量的数值。对于这个问题，我们有一个用于测量位置的传感器，因此 z 将是一个一维向量。

$$z = \begin{bmatrix} z \end{bmatrix}$$

残差方程的形式如下：

$$\begin{aligned} \mathrm{y} &= z - H\bar{x} \\ \begin{bmatrix} y \end{bmatrix} &= \begin{bmatrix} z \end{bmatrix} - \begin{bmatrix} ? & ? \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \end{bmatrix} \end{aligned}$$

为了让 Hx 成为 1x1 矩阵， H 必须是一个 1x2 矩阵。请记住，将矩阵 $m \times n$ 与 $n \times p$ 相乘后，会得到一个 $m \times p$ 矩阵。

我们需要将位置 x 乘以 1，从而得到该位置的对应数值。由于计算该数值时不需要用到速度，因此我们将 $\dot{x}$ 乘以 0。

$$\begin{aligned} \mathrm{y} &= z - \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \end{bmatrix} \\ &= [z] - [x] \end{aligned}$$

因此，对于我们的卡尔曼滤波器，我们进行了如下设置：

$$H = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

```
H = np.array([[1., 0.]])
```

我们已经设计了卡尔曼滤波器的大部分组件。现在只需对传感器中的噪声进行建模即可。

设计测量方案/确定测量方式

该测量是通过 z 来表示的测量均值，以及 R 来表示的测量方差来实现的。

z 很简单。它以向量的形式来表示各项测量数值。因为我们只有一项测量数据，所以表达方式如下：

$$z = \begin{bmatrix} z \end{bmatrix}$$

如果我们有两个传感器或测量数据，那么就会有：

$$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$$

测量噪声矩阵将传感器中的噪声表示为一个协方差矩阵。不过，在实际应用中，这并不容易实现。复杂的系统中可能包含许多传感器，这些传感器之间的相关性往往不明确，而且它们的噪声通常也不是纯粹的高斯分布。例如，当温度较高时，某个传感器的测量值可能会偏高，因此噪声在均值的两侧分布并不均匀。我们稍后会学习如何处理这些问题。

卡尔曼滤波方程中，测量噪声用协方差矩阵 R 来表示。该矩阵的维度为 $m \times m$ ，其中 m 表示传感器的数量。这个协方差矩阵用于描述各个传感器之间的相关性。由于我们只有一个传感器，因此 R 的值为：

$$R = \begin{bmatrix} \sigma_z^2 \end{bmatrix}$$

如果 σ_z^2 的面积为 5 平方米，那么 $R = \begin{bmatrix} 5 \end{bmatrix}$ 的数值就是……

如果我们有两个位置传感器，其中一个的误差为 5 米 ^{2} ，另一个的误差为 3 米 ^{2} ，那么我们可以这样表示：

$$R = \begin{bmatrix} 5 & 0 \\ 0 & 3 \end{bmatrix}$$

我们将方差放在对角线上，因为这是一个协方差矩阵。在协方差矩阵中，方差位于对角线上，而协方差（如果有的话）则位于对角线以外的元素位置。这里我们假设两个传感器产生的噪声之间不存在相关性，因此协方差值为 0。

对于我们的问题来说，我们只有一个传感器，因此可以这样来实现。

```
R = np.array([[5.]])
```

我们通过调用 `update` 来执行更新操作。

```
from filterpy.kalman import update
z = 1.
x, P = update(x, P, z, R, H)
print('x =', x)
```

```
x = [ 1.085 -0.64 ]
```

要同时跟踪所有这些变量相当麻烦，因此 FilterPy 还提供了 `KalmanFilter` 这个类来帮助实现滤波功能。在本书的后续内容中，我将会继续使用这个类。不过，我觉得还是有必要让你们了解这些函数的实现方式，因为我知道，有些人并不喜欢面向对象编程。

实现卡尔曼滤波器

我已经把与滤波器相关的所有代码都提供给了你们，现在让我们把它们集中到一处吧。首先，我们需要创建一个 `KalmanFilter` 对象。我们必须使用 `dim_x` 参数来指定状态中变量的数量，用 `dim_z` 参数来指定测量值的数量。在这个例子中，状态中有两个随机变量和一项测量值，因此我们这样编写代码：

```
from filterpy.kalman import KalmanFilter
dog_filter = KalmanFilter(dim_x=2, dim_z=1)
```

这样就创建了一个对象，其中包含了卡尔曼滤波器各矩阵的默认值。

```
from filterpy.kalman import KalmanFilter
dog_filter = KalmanFilter(dim_x=2, dim_z=1)
print('x = ', dog_filter.x.T)
print('R = ', dog_filter.R)
print('Q = \n', dog_filter.Q)
# etc...
```

```
x =  [[0. 0.]]
R =  [[1.]]
Q =
 [[1. 0.]
 [0. 1.]]
```

现在，我们用适合本问题的数值来初始化滤波器的各种矩阵和向量。我把这个过程放入了一个函数中，这样用户就可以为 `R` 、 `P` 和 `Q` 指定不同的初始值。我们会创建并使用许多这样的滤波器，这样做可以大大节省我们的工作量。

```
from filterpy.kalman import KalmanFilter
from filterpy.common import Q_discrete_white_noise

def pos_vel_filter(x, P, R, Q=0., dt=1.0):
    """ Returns a KalmanFilter which implements a
    constant velocity model for a state [x dx].T
    """

    kf = KalmanFilter(dim_x=2, dim_z=1)
    kf.x = np.array([x[0], x[1]]) # location and velocity
    kf.F = np.array([[1., dt],
                     [0.,  1.]])  # state transition matrix
    kf.H = np.array([[1., 0]])    # Measurement function
    kf.R *= R                     # measurement uncertainty
    if np.isscalar(P):
        kf.P *= P                 # covariance matrix
    else:
        kf.P[:] = P               # [:] makes deep copy
    if np.isscalar(Q):
        kf.Q = Q_discrete_white_noise(dim=2, dt=dt, var=Q)
    else:
        kf.Q[:] = Q
    return kf
```

`KalmanFilter` 将 `R` 、 `P` 和 `Q` 初始化为单位矩阵。因此， `kf.P *= P` 是一种将所有对角线元素都设置为相同标量值的简便方法。现在我们来创建这个滤波器：

```
dt = .1
x = np.array([0., 0.])
kf = pos_vel_filter(x, P=500, R=5, Q=0.1, dt=dt)
```

你可以在命令行中输入相应的变量，从而查看该滤波器所有属性的当前值。

```
kf
```

```
KalmanFilter object
dim_x = 2
dim_z = 1
dim_u = 0
x = [0. 0.]
P = [[500.   0.]
     [  0. 500.]]
x_prior = [[0. 0.]].T
P_prior = [[1. 0.]
           [0. 1.]]
x_post = [[0. 0.]].T
P_post = [[1. 0.]
          [0. 1.]]
F = [[1.  0.1]
     [0.  1. ]]
Q = [[0.    0.   ]
     [0.    0.001]]
```

```
R = [[5.]]
H = [[1. 0.]]
K = [[0. 0.]].T
y = [[0.]]
S = [[0.]]
SI = [[0.]]
M = [[0.]]
B = None
z = [[None]]
log-likelihood = -708.3964185322641
likelihood = 2.2250738585072014e-308
mahalanobis = 0.0
alpha = 1.0
inv = <function inv at 0x00000268CA22DB80>
```

现在要做的，就是编写代码来运行卡尔曼滤波器了。

```
from kf_book.mkf_internal import plot_track

def run(x0=(0.,0.), P=500, R=0, Q=0, dt=1.0,
        track=None, zs=None,
        count=0, do_plot=True, **kwargs):
    """
    track is the actual position of the dog, zs are the
    corresponding measurements.
    """

    # Simulate dog if no data provided.
    if zs is None:
        track, zs = compute_dog_data(R, Q, count)

    # create the Kalman filter
    kf = pos_vel_filter(x0, R=R, P=P, Q=Q, dt=dt)

    # run the kalman filter and store the results
    xs, cov = [], []
    for z in zs:
        kf.predict()
        kf.update(z)
        xs.append(kf.x)
        cov.append(kf.P)

    xs, cov = np.array(xs), np.array(cov)
    if do_plot:
        plot_track(xs[:, 0], track, zs, cov, **kwargs)
    return xs, cov
```

这是该滤波器的完整代码。其中大部分内容都是些重复性的、没什么实际意义的代码。为了让该滤波器能适用于本章中的多种用途，我将其设计得相当灵活。不过，代码看起来有点冗长。我们一行一行地来分析吧。

首先，程序会检查你是否在 `zs` 中提供了必要的测量数据。如果没有，程序会使用我们之前编写的 `compute_dog_data` 函数来生成这些数据。

接下来的几行代码使用了我们的辅助函数来构建卡尔曼滤波器。

```
# create the Kalman filter
 kf = pos_vel_filter(x0, R=R, P=P, Q=Q, dt=dt)
```

我们所需要做的，就是对每次测量结果应用卡尔曼滤波器的更新和预测步骤。 `KalmanFilter` 类提供了 `update()` 和 `predict()` 这两个方法来实现这一功能。 `update()` 负责执行卡尔曼滤波器的测量更新步骤，因此它需要一个包含传感器测量数据的变量作为输入。

如果不进行结果存储的操作，该循环将执行如下操作：

```
for z in zs:
    kf.predict()
    kf.update(z)
```

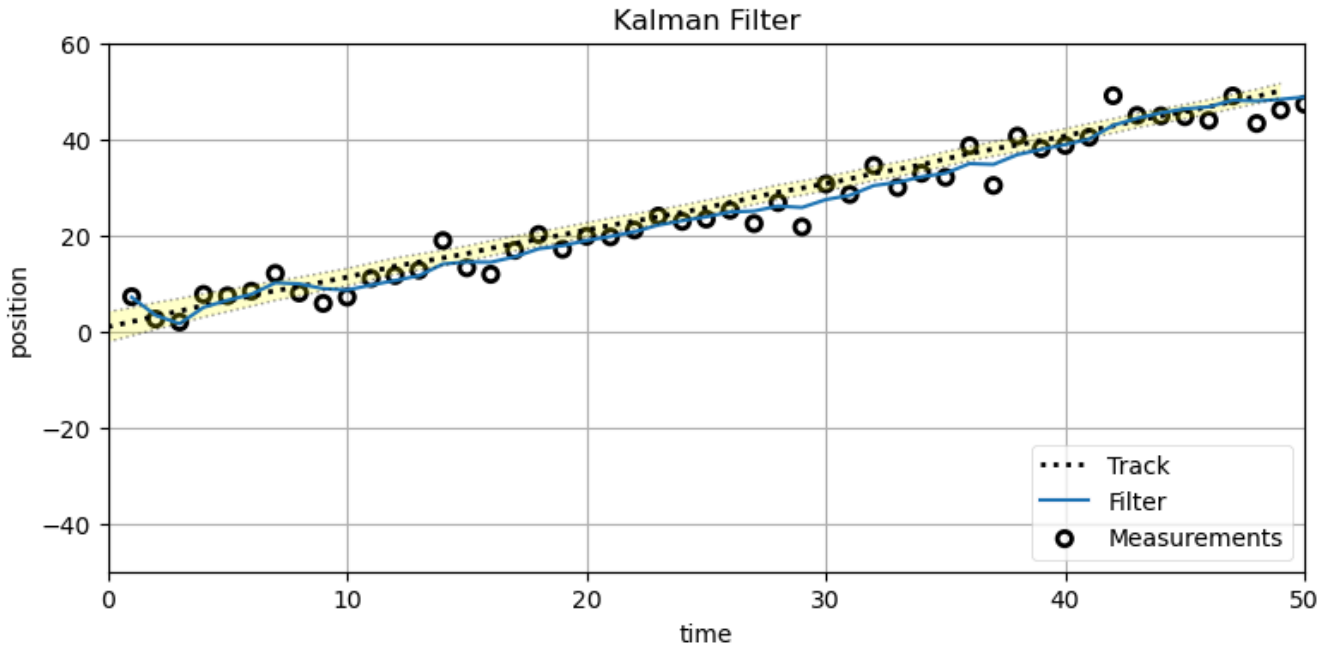
每次对 `predict` 和 `update` 的调用都会改变状态变量 `x` 和 `P` 的值。因此，在调用 `predict` 之后， `kf.x` 中保存的是先验值。在调用更新函数之后， `kf.x` 中则保存了后验值。这些状态值和协方差则分别保存在 `xs` 和 `cov` 中。

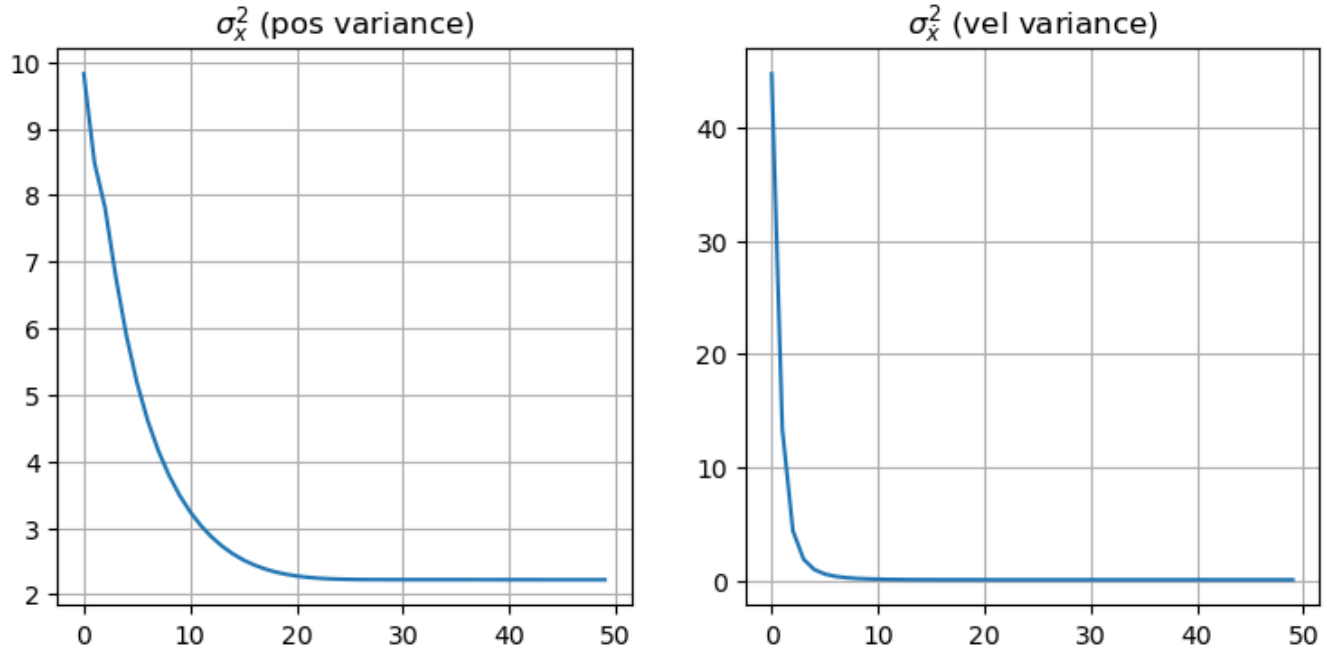
真的没有比这更简单的了。在处理更复杂的问题时，这段代码的基本结构依然保持不变；所有的工作都集中在对 `KalmanFilter` 矩阵的设置上；而执行滤波操作则非常简单。

其余的代码会根据需要绘制出结果图，然后返回已保存的状态值和协方差矩阵。

让我们来运行一下吧。我们有 50 组测量数据，其中噪声的方差为 10，而过程本身的方差则为 0.01。

```
P = np.diag([500., 49.])
Ms, Ps = run(count=50, R=10, Q=0.01, P=P)
```





虽然还有很多需要学习的地方，但我们已经成功实现了卡尔曼滤波器，所使用的理论和公式都与鲁道夫·卡尔曼所提出的完全一致！类似这样的代码被广泛应用于 GPS、客机、机器人等各种设备中。

第一张图表展示了卡尔曼滤波器的输出结果，以及狗狗的测量值和实际位置（标记为“Track”）。在经过最初的稳定期后，该滤波器应该能够非常精确地追踪狗狗的位置。黑色虚线之间的黄色阴影区域代表了滤波器误差的 1 个标准差，具体解释请见下一段。

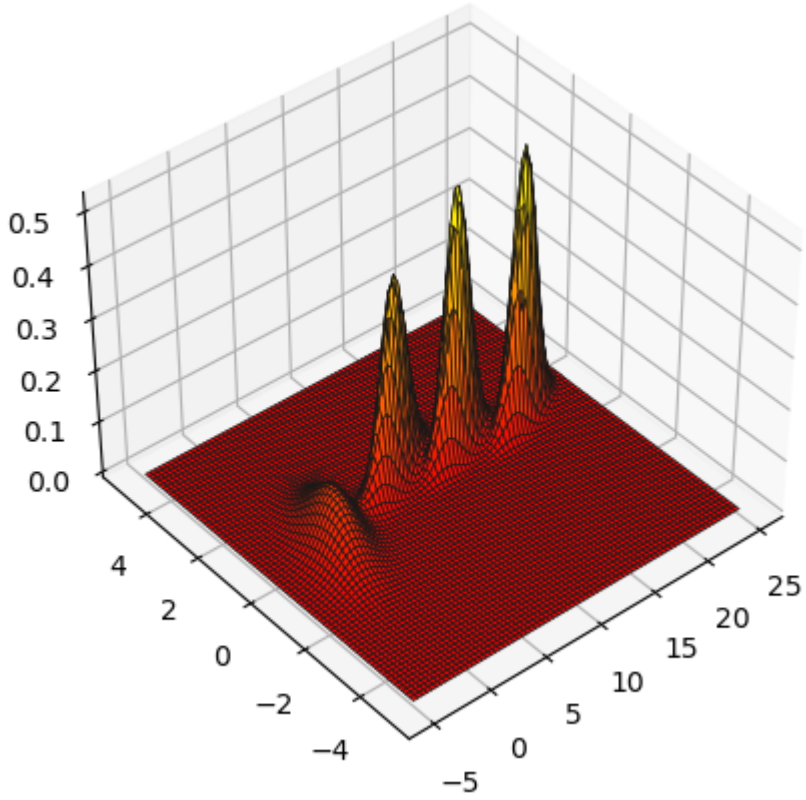
接下来的两个图表分别展示了 x 和 $\dot{x}$ 的方差。我还绘制了 P 的对角线值随时间的变化情况。需要记住的是，协方差矩阵的对角线元素分别代表各个状态变量的方差。因此， $P[0,0]$ 实际上就是 x 的方差，而 $P[1,1]$ 则是 $\dot{x}$ 的方差。可以看出，这两种情况的方差都很快趋近于较小的数值。

协方差矩阵 P 反映了在所有输入数据都准确的条件下，该滤波器的理论性能表现。需要记住的是，标准差是方差的平方根；在正态分布中，大约 68%的数据位于一个标准差范围内。如果滤波器的输出有至少 68%位于一个标准差范围内，那么该滤波器的性能就可以认为是不错的。在上面的图表中，我用黄色阴影区域标出了一个标准差的范围。从图表来看，该滤波器的输出似乎略微超出了这个范围，因此可能需要对其进行一些调整。

在单变量处理的章节中，我们使用了比上面代码简单得多的方式来处理那些包含大量噪声的信号。不过，请注意，我们现在所处理的只是一个非常简单的例子——即一个在一维空间中移动的物体，以及一个传感器而已。这已经算是利用上一章中的代码所能实现的极限了。相比之下，只要稍微调整一下代码中各变量的赋值方式，我们就可以用同样的代码来实现非常复杂的多维滤波处理。比如，在金融模型中，我们可能需要处理 100 个维度的数据。或者，对于一架飞机来说，它可能配备了 GPS、惯性导航系统、TACAN 雷达、雷达高度计、气压高度计和空速计。如果我们想把所有这些传感器的数据整合起来，从而预测飞机在三维空间中的位置、速度和加速度，那么我们完全可以利用本章中的代码来实现这一目标。

我希望你能更清楚地了解高斯函数随时间的变化情况。下图是一个 3D 图表，展示了每隔 7 个时间单位时高斯函数的状态。每隔 7 个时间单位，各个高斯函数之间的间距就足够大，从而可以单独观察每一个高斯函数。最左边的那个高斯函数对应的是时间点 $t = 0$ 。

```
from kf_book.book_plots import set_figsize, figsize
from kf_book.nonlinear_plots import plot_gaussians

P = np.diag([3., 1.])
np.random.seed(3)
Ms, Ps = run(count=25, R=10, Q=0.01, P=P, do_plot=False)
with figsize(x=9, y=5):
    plot_gaussians(Ms[::7], Ps[::7], (-5,25), (-5, 5), 75)
```



“节省型”类别

在 `run()` 方法中，我编写了用于保存滤波结果的样板代码。

```
    xs, cov = [], []
    for z in zs:
        kf.predict()
        kf.update(z)
        xs.append(kf.x)
        cov.append(kf.P)

    xs, cov = np.array(xs), np.array(cov)
```

有一个简单的办法可以避免这种情况。 `filterpy.common` 提供了 `Saver` 类，该类会在每次调用 `Saver.save()` 时，将所有属性保存到卡尔曼滤波器类中。我们先来实际操作一下，然后再详细讨论这个问题。

```
from filterpy.common import Saver
kf = pos_vel_filter([0, .1], R=R, P=P, Q=Q, dt=1.)
s = Saver(kf)
```

```
for i in range(1, 6):
    kf.predict()
    kf.update([i])
    s.save()  # save the current state
```

Saver 对象现在包含了 KalmanFilter 对象的所有属性信息。 kf.x 表示该滤波器当前的状态估计值。而 s.x 则保存了在循环过程中计算得到的状态估计值。

```
s.x
```

```
[array([0.531, 0.304]),
 array([1.555, 0.763]),
 array([2.784, 1.036]),
 array([3.944, 1.105]),
 array([5.015, 1.086])]
```

你可以通过 keys 属性查看所有可用的属性。

```
s.keys
```

```
['alpha',
 'likelihood',
 'log_likelihood',
 'mahalanobis',
 'dim_x',
 'dim_z',
 'dim_u',
 'x',
 'P',
 'Q',
 'B',
 'F',
 'H',
 'R',
 '_alpha_sq',
 'M',
 'z',
 'K',
 'y',
 'S',
 'SI',
 '_I',
 'x_prior',
 'P_prior',
 'x_post',
 'P_post',
 '_log_likelihood',
 '_likelihood',
 '_mahalanobis',
 'inv']
```

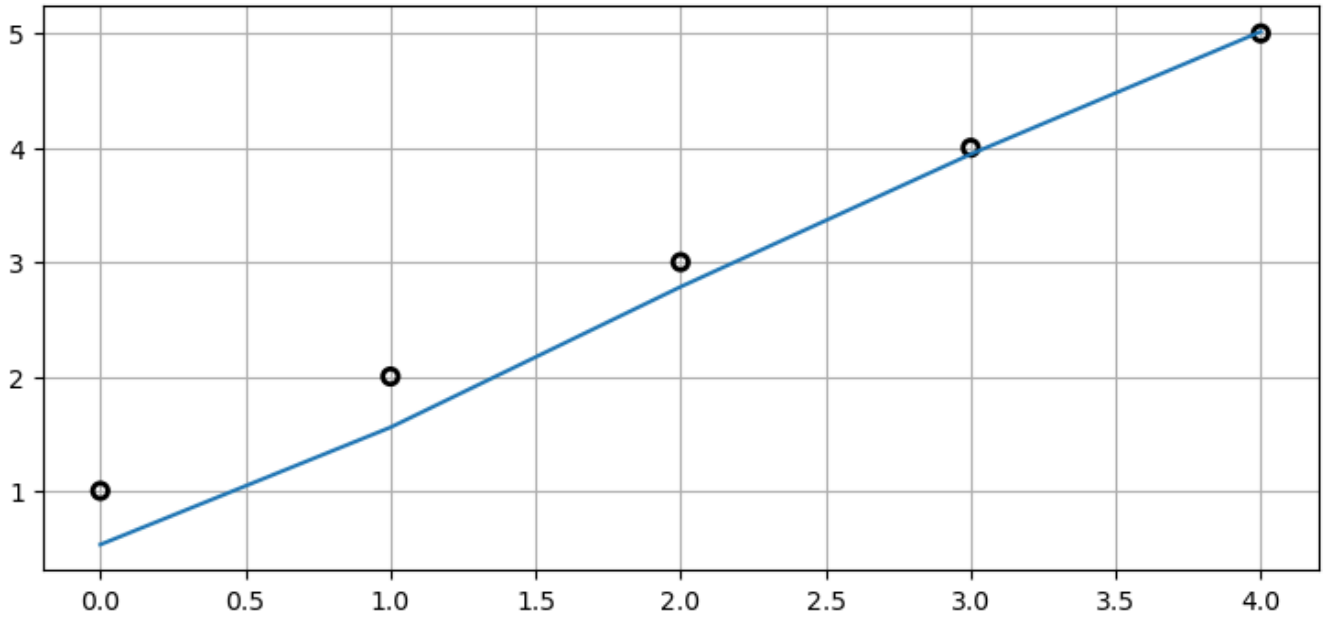
其中还有许多属性我们尚未讨论，不过大多数应该都是大家所熟悉的。

此时，你可以编写代码来绘制这些变量中的任何一个。不过，通常使用 np.array 会更为方便，不必使用列表。调用 Saver.to_array() 后，列表会被转换为 np.array 格式。不过有一点需要注意：如果在程序运行过

程中，其中某个属性的格式发生了变化，那么 to_array 会抛出异常，因为 np.array 要求所有元素的类型和大小都必须一致。

如果你再仔细看看这些按键，就会发现 z 也是可选选项之一。这很令人欣慰：显然， z 的测量结果已经被保存下来了。让我们把这一数值与估算结果进行对比吧。

```
import matplotlib.pyplot as plt

s.to_array()
book_plots.plot_measurements(s.z);
plt.plot(s.x[:, 0]);
```



虽然我是以 KalmanFilter 类来演示这一点的，但实际上，只要是由 FilterPy 实现的任何滤波类，都可以使用这种方法。当然，你自己编写的任何类也应该也能适用，因为该方法是通过检查对象来获取其属性名的。在整本书中，我们都会使用这个类，这样才能保持代码的简洁性和可读性。不过，使用 Saver 会降低代码的执行效率，因为背后会有很多复杂的操作。不过，就学习和探索的便利性而言， Saver 无疑是最优的选择。

卡尔曼滤波方程

我们现在准备了解 predict() 和 update() 是如何进行计算的。

关于符号的说明：我是一名程序员，我习惯于阅读那些易于理解的代码。

```
x = x + 1
```

那不是一个等式，因为等式的两边并不相等。那其实只是一个赋值操作而已。如果要用数学符号来表示的话，可以这样写：

卡尔曼滤波方程中到处都是下标和上标，这些符号的存在是为了确保方程在数学上的一致性。不过，我觉得这反而让方程变得极其难以理解。在本书中，我尽量不使用下标来表示各种变量。作为一名程序员，你应该明白：我所展示的这些代码其实代表了某种算法的实现方式，而这种算法需要一步步来执行。等我们有了具体的例子后，我会再详细解释这一点。

预测方程式

卡尔曼滤波器利用这些方程来计算系统的“先验值”——也就是对系统下一状态的预测值。具体来说，就是计算系统的先验均值（ $\bar{x}$ ）和协方差（ $\bar{P}$ ）。

$$\begin{aligned}\bar{x} &= Fx + Bu \\ \bar{P} &= FPF^T + Q\end{aligned}$$

Mean

$\bar{x} = Fx + Bu$

需要提醒的是，线性方程 $Ax = b$ 实际上代表了一组方程式。其中， A 表示这些方程式的系数； x 则是各个变量的向量。通过执行乘法运算 Ax ，可以得出该组方程式的右侧数值，这些数值由 b 表示。

如果 F 包含了某个特定时间步的状态转换信息，那么 Fx 则计算出该转换之后的状态。很简单吧！同样地， B 代表控制函数， u 代表控制输入，因此 Bu 则计算了各种控制因素对转换后状态的贡献。最后， $\bar{x}$ 的值则是 Fx 与 Bu 之和。

相应的单变量方程为

$$\bar{\mu} = \mu + \mu_{move}$$

如果你执行矩阵乘法 Fx ，就会得到关于 x 的这个方程。

让我们把这一点说清楚。回想一下上一章中 F 的值：

$$F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$$

因此， $\bar{x} = Fx$ 对应于以下线性方程组：

$$\begin{cases} \bar{x} &= 1x + \Delta t\dot{x} \\ \bar{\dot{x}} &= 0x + 1\dot{x} \end{cases}$$

Covariance

$\bar{P} = FPF^T + Q$

这个方程式不太容易理解，因此我们需要花更多时间来研究它。

该方程的单变量形式为：

$$\bar{\sigma}^2 = \sigma^2 + \sigma^2_{move}$$

我们将这种“运动过程中的不确定性”所导致的方差，加到我们的估计值方差上，以此来反映知识上的损失。在这里，我们也需要做同样的处理，只不过，对于多元高斯分布来说，这并不那么容易实现。

我们不能简单地写成 $\bar{P} = P + Q$ 。在多元高斯模型中，各个状态变量是相互关联的。这意味着什么呢？虽然我们对速度的认知并不完整，但我们仍然将其与位置信息一起加以考虑。

$$\bar{x} = \dot{x}\Delta t + x$$

由于我们无法准确知道 $\dot{x}$ 的值，因此 $\bar{x} = \dot{x}\Delta t + x$ 的值也就具有不确定性。由于位置和速度之间存在相关性，我们无法简单地将它们的协方差矩阵相加。例如，如果 P 和 Q 都是对角矩阵，那么它们的和也仍是对角矩阵。但由于位置与速度相关，对角线以外的元素必然不为零。

正确的方程式是

$$\bar{P} = FPF^T + Q$$

ABA^T 这样的表达式在线性代数中很常见。可以将其理解为用外部项来“投影”中间项。在本书的后续内容中，我们会多次使用这种表达式。不过，对我来说，这可能看起来像是个“神奇的公式”。让我们一起来探讨一下吧。

当我们用……来初始化 P 时

$$P = \begin{bmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_v^2 \end{bmatrix}$$

FPF^T 的值为：

$$\begin{aligned} FPF^T &= \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_v^2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix} \\ &= \begin{bmatrix} \sigma_x^2 & \sigma_v^2\Delta t \\ 0 & \sigma_v^2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix} \\ &= \begin{bmatrix} \sigma_x^2 + \sigma_v^2\Delta t^2 & \sigma_v^2\Delta t \\ \sigma_v^2\Delta t & \sigma_v^2 \end{bmatrix} \end{aligned}$$

P 的初始值中，位置和速度之间不存在协方差。位置是通过 $\dot{x}\Delta t + x$ 来计算的，因此位置和速度之间存在相关性。通过 FPF^T 进行运算后，可以得到 $\sigma_v^2\Delta t$ 这个协方差值。具体的数值并不重要；只要明白 FPF^T 利用了该模型来自动计算位置和速度之间的协方差即可！

另一种理解方式是，可以考虑 Fx 的乘法运算。该运算实际上是将 x 向时间上推进。 FP 看起来似乎是类似的运算，但实际上 P 是一个矩阵，而 x 则是一个向量。最后的 F^T 项确保了我们会同时与 F 的各行和各列相乘。在计算 FPF^T 的第二行中，我们得到了 FP 的值。可以看出， FP 实际上是一个上三角矩阵，因为我们还没有将 F 完全纳入到乘法运算中。

如果你具备一些线性代数和统计学方面的知识，那会很有帮助。由于预测而产生的协方差，可以被看作是预测过程中误差的期望值，其计算公式如下。

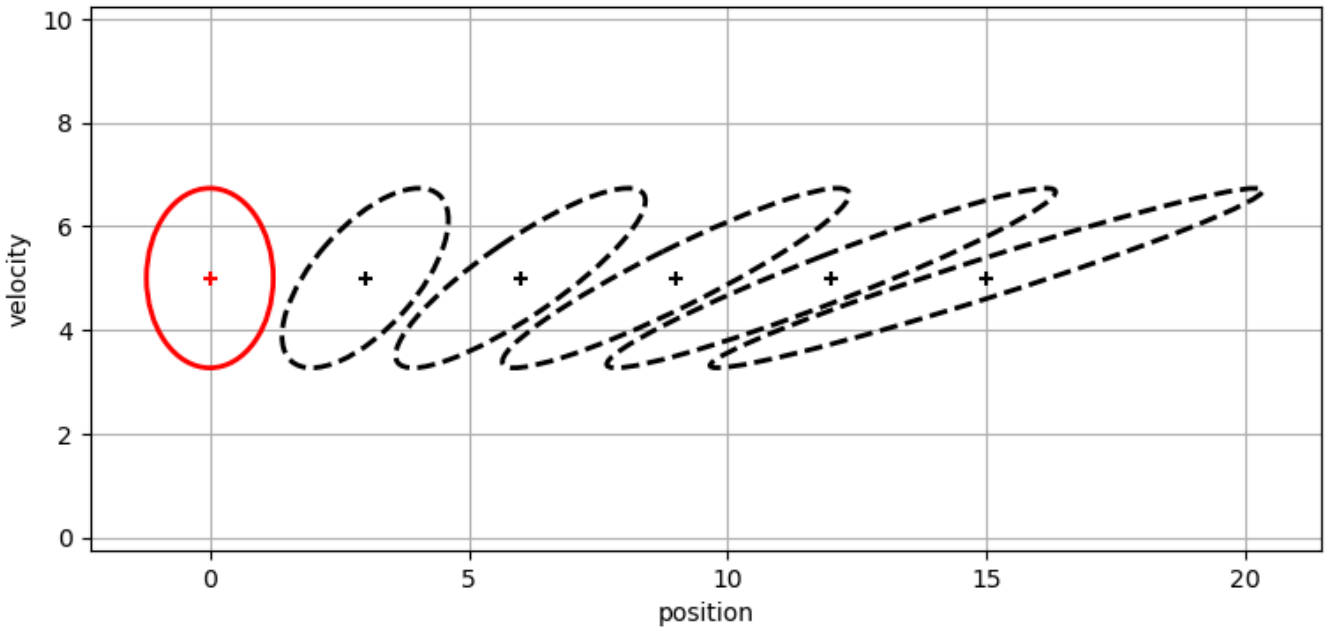
$$\begin{aligned}\bar{P} &= E[(Fx - F\bar{\mu})(Fx - F\bar{\mu})^T] \\ &= FE[(x - \bar{\mu})(x - \bar{\mu})^T]F^T\end{aligned}$$

当然， $E[(x - \bar{\mu})(x - \bar{\mu})^T]$ 其实也就是 P 而已，这没什么好奇怪的。

$$\bar{P} = FPF^T$$

让我们来看看它的效果。这里，我使用了我们滤波器中的 F，并将状态向前推进了 6/10 秒。我重复了这个操作五次，这样你就能看到 $\bar{P}$ 是如何持续变化的了。

```
dt = 0.6
x = np.array([0., 5.])
F = np.array([[1., dt], [0, 1.]])
P = np.array([[1.5, 0], [0, 3.]])
plot_covariance_ellipse(x, P, edgecolor='r')

for _ in range(5):
    x = F @ x
    P = F @ P @ F.T
    plot_covariance_ellipse(x, P, edgecolor='k', ls='dashed')
book_plots.set_labels(x='position', y='velocity')
```



可以看到，当速度为 5 时，物体在每 6/10 秒的时间步长内会准确移动 3 个单位。在每个时间步长中，椭圆的宽度都会增大，这说明由于在每个时间步长中都在 x 值上加上了 $\dot{x}\Delta t$，我们丢失了有关物体位置的信息。而椭圆的高度则没有变化——根据系统模型，速度是恒定的，因此我们对速度的判断也不会改变。随着时间的推移，椭圆会变得越来越倾斜。需要记住，倾斜程度表示了各种量之间的相关性。 F 与 x 、 $\dot{x}$ 之间存在线性相关性，这一关系可以用公式 $\bar{x} = \dot{x}\Delta t + x$ 来表示。 FPF^T 的计算方式正确地将这种相关性纳入了协方差矩阵中。

这是一个关于该方程的动画演示。您可以通过改变 F 的设置来观察其对 P 形状的影响。 `F00` 滑块用于调整 F[0, 0]的值。 `covar` 则用于设定位置和速度之间的初始协方差值($\sigma_x\sigma_{\dot{x}}$)。建议您至少回答这些问题。

- 如果 x 与 $\dot{x}$ 之间没有相关性会怎样呢？（将 F01 设置为 0，其余参数保持默认值。）

- 如果 $x = 2\dot{x}\Delta t + x_0$ 的值是多少呢？（将 F01 设置为 2，其余参数保持默认值）
- 如果 $x = \dot{x}\Delta t + 2x_0$ 的值是多少呢？（将 F00 设置为 2，其余参数保持默认值）
- 如果 $x = \dot{x}\Delta t$ 的值是多少呢？（将 F00 设置为 0，其余参数保持默认值）

```
from ipywidgets import interact
from kf_book.book_plots import IntSlider, FloatSlider

def plot_FPFT(F00, F01, F10, F11, covar):
    plt.figure()
    dt = 1.
    x = np.array((0, 0.))
    P = np.array(((1, covar), (covar, 2)))
    F = np.array(((F00, F01), (F10, F11)))
    plot_covariance_ellipse(x, P)
    plot_covariance_ellipse(x, F @ P @ F.T, ec='r')
    plt.gca().set_aspect('equal')
    plt.xlim(-4, 4)
    plt.ylim(-4, 4)
    #plt.title(str(F))
    plt.xlabel('position')
    plt.ylabel('velocity')
    plt.show()

interact(plot_FPFT,
         F00=IntSlider(value=1, min=0, max=2),
         F01=FloatSlider(value=1, min=0, max=2, description='F01(dt)'),
         F10=FloatSlider(value=0, min=0, max=2),
         F11=FloatSlider(value=1, min=0, max=2),
         covar=FloatSlider(value=0, min=0, max=1));
```

```
interactive(children=(IntSlider(value=1, continuous_update=False, description='F00', max=2),
FloatSlider(value…
```

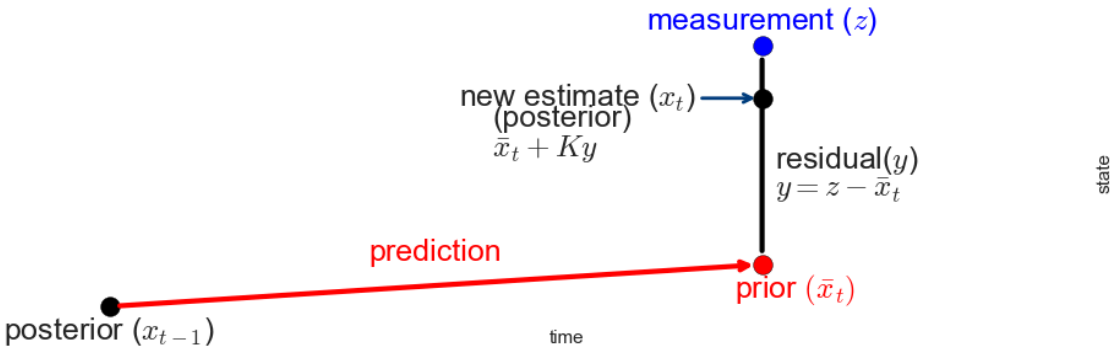
（如果您是以静态文本形式阅读此内容：在线运行该程序的说明请点击此处：https://git.io/vza7b）。或者，使用下面的链接进入 Binder 平台，再从那里打开该笔记本。

http://mybinder.org/repo/rlabbe/Kalman-and-Bayesian-Filters-in-Python

更新方程式

这些更新方程看起来比预测方程更复杂，但这主要是因为卡尔曼滤波器是在测量空间中进行计算。因为测量结果本身是不可逆的。例如，假设某个传感器能够测量出到目标的距离。那么，我们无法将这个距离直接转换为目标的位置——因为在圆周上，有无数个点对应的距离都是相同的。不过，如果我们知道目标的位置，那么我们总可以计算出相应的距离。

在继续之前，请记住，我们试图做的其实是一件非常简单的事情：在测量值和预测值之间选择一个合适的数值作为新的估计值，就像这张图表所展示的那样：



由于该系统具有多个维度，因此相关方程会相当复杂。但这就是我们需要做的操作。不要让这些复杂的方程式掩盖了这个概念本身的简单性。

System Uncertainty

$\mathrm{S} = H\bar{P}H^T + R$

为了在测量空间中进行运算，卡尔曼滤波器需要将协方差矩阵投影到测量空间中。相关的数学公式为 $H\bar{P}H^T$，其中 $\bar{P}$ 表示先验协方差，H 则代表测量函数。

你应该能认出这种 ABA^T 形式——它其实就是 FPF^T 中用来通过状态转换函数来更新 P 的预测步骤。在这里，我们同样使用这种形式，只不过是用测量函数来更新 P 而已。线性代数实际上起到了为我们调整坐标系的作用。

一旦协方差被确定下来，我们就需要考虑传感器带来的噪声。这其实很简单——只需将各个矩阵相加即可。所得的结果可以被称作“系统不确定性”或“创新协方差”。

如果忽略 H 项，那么这个方程就相当于卡尔曼增益的单变量方程中的分母部分。

$$K = \frac{\bar{\sigma}^2}{\bar{\sigma}^2 + \sigma_z^2}$$

比较该系统不确定性及其协方差的计算公式。

$$\begin{aligned} S &= H\bar{P}H^T + R \\ \bar{P} &= FPF^T + Q \end{aligned}$$

在每个方程中，P 被置于不同的空间中，这些空间中分别包含函数 H 或 F。接着，我们再把与该空间相关的噪声矩阵加进去。

Kalman Gain

$K = \bar{P}H^T S^{-1}$

再来看看残差图。一旦我们得到了预测值和测量值，就需要在这两个数值之间选择一个估计值。如果我们对测量值的把握更大，那么估计值就会更接近测量值；相反，如果我们对预测值的把握更大，那么估计值就会更接近预测值。

在单变量分析部分，我们使用该公式来对均值进行标准化处理。

$$\mu = \frac{\bar{\sigma}^2\mu_z + \sigma_z^2\bar{\mu}}{\bar{\sigma}^2 + \sigma_z^2}$$

我们将其简化为：

$$\mu = (1-K)\bar{\mu} + K\mu_z$$

这让我们得到了……

$$K = \frac{\bar{\sigma}^2}{\bar{\sigma}^2 + \sigma_z^2}$$

K 即为卡尔曼增益，它是一个介于 0 和 1 之间的实数。请确保你明白：卡尔曼增益是如何在预测值和测量值之间确定一个合适的数值的。卡尔曼增益实际上是一个百分比或比例——如果 K 的值为 0.9，那么就意味着会取测量值的 90%，以及预测值的 10%。

在多元卡尔曼滤波中， K 是一个向量，而非标量。以下是该方程的再次表述： $K = \bar{P}H^T S^{-1}$ 。这算是一种比率吗？我们可以把矩阵的逆理解为线性代数中求倒数的一种方式。虽然矩阵没有除法运算，但用这种方式来理解会很有帮助。因此，我们可以将 K 对应的方程理解为……

$$\begin{aligned} K &\approx \frac{\bar{P}H^T}{S} \\ K &\approx \frac{uncertainty_{prediction}}{uncertainty_{measurement}} H^T \end{aligned}$$

卡尔曼增益公式是根据我们对预测结果和测量结果的信任程度来计算出一个比例值。在之前的每一章中，我们都采用了同样的方法。虽然该公式因为需要用矩阵来处理多维数据而显得比较复杂，但其核心概念其实很简单。 H^T 这个项的含义不太清楚，我稍后会解释。如果忽略这个项，卡尔曼增益的公式就与单变量情况下的公式相同：只需将先验信息的不确定性除以先验信息和测量结果不确定性的总和即可。

Residual

$y = z - H\bar{x}$

这很简单，因为在设计测量函数 H 时，我们已经处理过这个方程式了。请记住，测量函数的作用是将某种状态转换为可测量的数值。因此， Hx 会将 x 转换为等效的测量值。完成这一步后，我们只需用测量值 z 减去这个转换后的数值，就能得到残差——也就是测量值与预测值之间的差异。

该单变量方程为：

$$y = z - \bar{x}$$

显然，它计算出的结果也是一样的，只不过是在一维空间中进行的计算而已。

State Update

$x = \bar{x} + Ky$

我们选择的新状态位于残差值上，该残差值经过了卡尔曼增益的缩放处理。这种缩放操作由 Ky 来完成：它既对残差值进行缩放，又利用 H^T 项将其转换回状态空间中的表示形式。该转换结果会被加到先验值上，从而得

到方程： $x = \bar{x} + Ky$。让我把 K 也写出来，这样我们就能完整地了解整个计算过程了：

$$\begin{aligned} x &= \bar{x} + Ky \\ &= \bar{x} + \bar{P}H^T S^{-1} y \\ &\approx \bar{x} + \frac{uncertainty_{prediction}}{uncertainty_{measurement}} H^T y \end{aligned}$$

或许，理解这一比例的更好方式是重新表述一下那个估算公式：

$$\begin{aligned} x &= \bar{x} + Ky \\ &= \bar{x} + K(z - H\bar{x}) \\ &= (I - KH)\bar{x} + Kz \end{aligned}$$

这与单变量形式的相似性应该是显而易见的：

Covariance Update

$P = (I - KH)\bar{P}$

I 是单位矩阵，它代表了我们在多维空间中表示 1 的方式。 H 是我们的测量函数，其实是一个常数。我们可以将这个方程视为 $P = (1 - cK)P$。 K 表示我们在预测结果和测量结果之间所采用的权重比例。如果 K 的值较大，那么 $(1 - cK)$ 的值就会较小，进而 P 的值也会被缩小。相反，如果 K 的值较小，那么 $(1 - cK)$ 的值就会较大， P 的值也会相应地增大。这意味着，我们可以通过卡尔曼增益来调整不确定性的大小。

这个方程在数值计算上可能不稳定，因此我在 FilterPy 中不使用它。减法操作会破坏方程的对称性，久而久之会导致浮点数误差。稍后，我会介绍一些更复杂但数值上更稳定的方程形式。

一个不使用 FilterPy 的示例

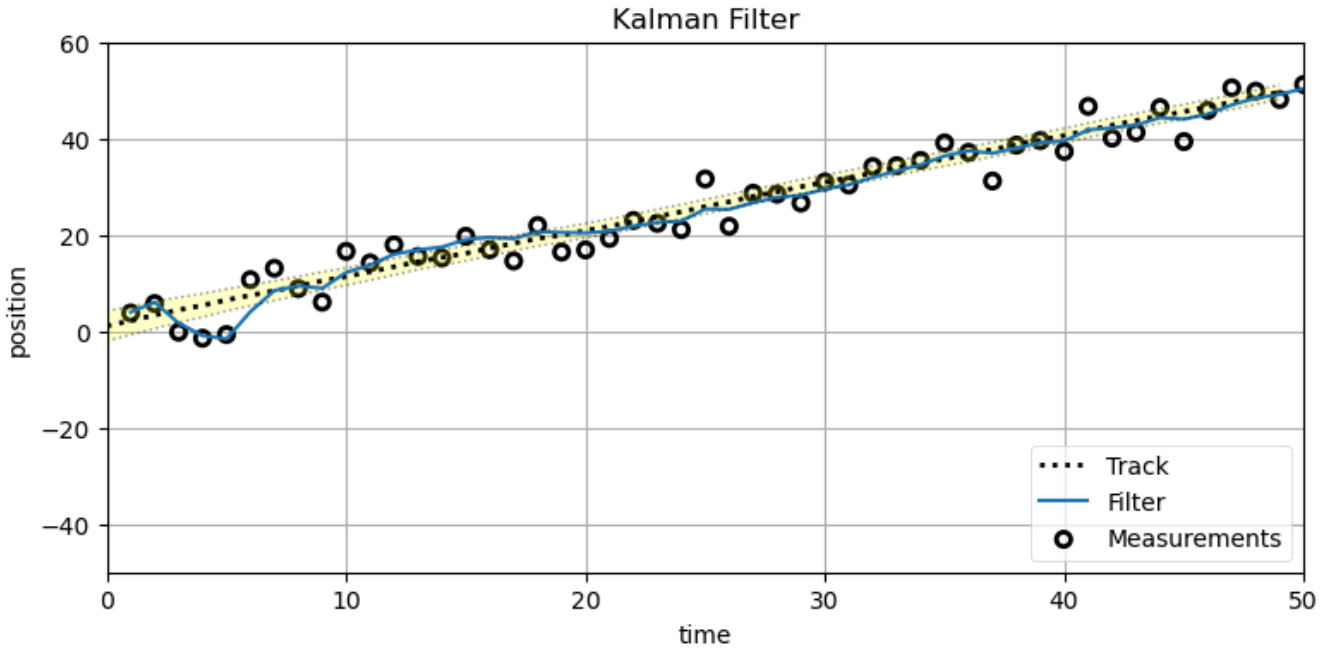
FilterPy 将实现细节隐藏了起来。通常情况下，这种处理方式很方便。不过，让我们试着不使用 FilterPy 来实现最后一个滤波器吧。为此，我们需要将相关的矩阵定义为变量，然后手动计算卡尔曼滤波所需的方程式。

在这里，我们对矩阵进行初始化：

```
dt = 1.
R_var = 10
Q_var = 0.01
x = np.array([[10.0, 4.5]]).T
P = np.diag([500, 49])
F = np.array([[1, dt],
              [0,  1]])
H = np.array([[1., 0.]])
R = np.array([[R_var]])
Q = Q_discrete_white_noise(dim=2, dt=dt, var=Q_var)
```

```
from scipy.linalg import inv

count = 50
track, zs = compute_dog_data(R_var, Q_var, count)
xs, cov = [], []
for z in zs:
    # predict
```

```
    x = F @ x
    P = F @ P @ F.T + Q

    #update
    S = H @ P @ H.T + R
    K = P @ H.T @ inv(S)
    y = z - H @ x
    x += K @ y
    P = P - K @ H @ P

    xs.append(x)
    cov.append(P)

xs, cov = np.array(xs), np.array(cov)
plot_track(xs[:, 0], track, zs, cov, plot_P=False)
```



其结果与 FilterPy 版本完全一致。您可以选择自己更喜欢的版本。我个人不喜欢在命名空间中使用 `x` 、 `P` 之类的变量；对我来说， `dog_filter.x` 的格式更易读。

更重要的是，这个例子要求你牢记并正确编写卡尔曼滤波器的相关公式。迟早你会犯错。而使用 FilterPy 的话，就能确保代码的准确性。另一方面，如果你在定义时出错，比如把 H 设为列向量而非行向量，那么使用 FilterPy 时出现的错误信息将比使用这种显式编码方式时更难调试。

FilterPy 的 KalmanFilter 类还提供了许多其他功能，比如数据平滑处理、批量处理、基于衰减因子的记忆滤波、最大似然函数的计算等等。无需自行编写代码，即可使用这些功能。

摘要/总结

我们已经学习了卡尔曼滤波器的各种方程式。以下是所有方程式的汇总，供您参考。虽然需要学习的内容很多，但我希望您在仔细研究每个方程式后，能意识到它们与单变量滤波器中的方程式其实有着相似之处。在关于卡尔曼滤波的数学原理部分，我会向您展示：如果我们把 x 的维度设为 1，那么这些方程式就又变回了单变

量滤波器的方程式。其实，卡尔曼滤波器并不是单变量滤波器的“简单扩展”——它实际上是单变量滤波器的多维实现形式而已。

$$
\begin{aligned}
&\text{Predict Step} \\
&\bar{x} &&= Fx + Bu \\
&\bar{P} &&= FPF^T + Q \\
&\text{Update Step} \\
&\text{S} &&= H\bar{P}H^T + R \\
&K &&= \bar{P}H^T S^{-1} \\
&\text{y} &&= z - H\bar{x} \\
&x &&= \bar{x} + K\text{y} \\
&P &&= (I - KH)\bar{P}
\end{aligned}
$$

我想分享一下在文献中常见的方程式表示方式。虽然存在多种不同的符号体系，但这种方式能让你大致了解方程式的表现形式。

$$
\begin{aligned}
\hat{x}_{k\mid k-1} &= F_k\hat{x}_{k-1\mid k-1} + B_k u_k \\
P_{k\mid k-1} &= F_k P_{k-1\mid k-1} F_k^T + Q_k \\
\tilde{y}_k &= z_k - H_k\hat{x}_{k\mid k-1} \\
S_k &= H_k P_{k\mid k-1} H_k^T + R_k \\
K_k &= P_{k\mid k-1} H_k^T S_k^{-1} \\
\hat{x}_{k\mid k} &= \hat{x}_{k\mid k-1} + K_k\tilde{y}_k \\
P_{k\mid k} &= (I - K_k H_k) P_{k\mid k-1}
\end{aligned}
$$

这种表示法采用了贝叶斯符号 $a \mid b$，其含义是：在给定 b 的证据的情况下，对 a 的估计值。符号“帽子”则表示估计值。因此， $\hat{x}_{k\mid k}$ 表示在给定步骤 k 的证据的情况下，对步骤 k 时状态 x 的估计值。换句话说，这就是后验概率。 $\hat{x}_{k\mid k-1}$ 则表示在给定步骤 $k-1$ 的估计值的情况下，对步骤 k 时状态 x 的估计值。换句话说，这就是先验概率。

这种记号法是从维基百科[1]上借鉴来的。它能让数学家更精确地表达自己的想法。在发表新研究成果的正式文献中，这种精确性是不可或缺的。不过，作为程序员，我觉得这种记号法很难理解。我习惯于将变量视为在程序运行过程中会不断变化的值，不会为每次计算都使用不同的变量名。由于学术界没有统一的记号规范，所以每位作者都会采用不同的方式来表示变量。我觉得在阅读不同的书籍和论文时，很难快速切换这种记号方式。因此，我选择了自己认为更简单的记号法。虽然数学家们可能会给我发愤怒的邮件，但我希望程序员和学生能喜欢我这种简化的记号法。

“符号说明附录”列出了不同作者所使用的符号表示法。但这又带来了另一个问题：不同的作者会使用不同的变量名称。 x 这个符号比较通用，但除此之外，就很难确定其他符号的含义了。例如，我通常用 F 来表示某个量，而其他作者则可能用 A 来表示同一个量。因此，读者必须仔细阅读文本，并希望作者能明确说明各个变量的含义（不过，很多时候作者并不会这么做）。

如果你是一名程序员，试图理解论文中的数学公式，我建议先去掉所有的上标、下标和特殊符号，把它们都替换为一个字母。对于那些每天都要处理这类公式的人来说，这个建议可能没什么用处。但对我来说，记住“ F

”代表上一步计算出的 P 的更新值，要比试图记住 $P_{k-1}(+)$ 的含义、它与 $P_k(-)$ 的关系，以及这些符号与 5 分钟前读到的论文中的符号之间的联系容易得多。

练习：展示隐藏变量的影响

在我们的模型中，速度是一个隐藏变量。如果我们不在模型中考虑速度这一因素，那么滤波器的性能会如何呢?

编写一个使用状态量 $x = \begin{bmatrix} x \end{bmatrix}$ 的卡尔曼滤波器，并将其与使用状态量 $x = \begin{bmatrix} x & \dot{x} \end{bmatrix}^T$ 的滤波器进行比较。

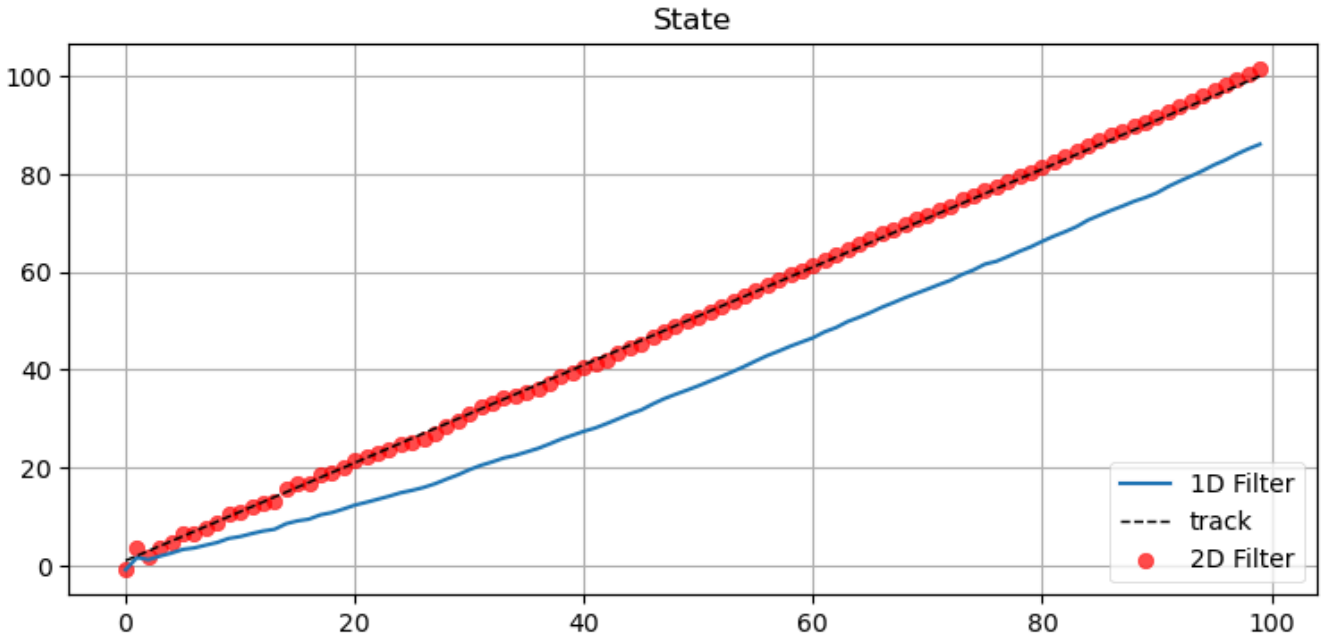
```
# your code here
```

解决方案

我们已经为位置和速度的计算实现了卡尔曼滤波器。因此，我会直接给出代码，不做太多解释，然后再绘制出相应的图表。

```
from math import sqrt
from numpy.random import randn

def univariate_filter(x0, P, R, Q):
    f = KalmanFilter(dim_x=1, dim_z=1, dim_u=1)
    f.x = np.array([[x0]])
    f.P *= P
    f.H = np.array([[1.]])
    f.F = np.array([[1.]])
    f.B = np.array([[1.]])
    f.Q *= Q
    f.R *= R
    return f

def plot_1d_2d(xs, xs1d, xs2d):
    plt.plot(xs1d, label='1D Filter')
    plt.scatter(range(len(xs2d)), xs2d, c='r', alpha=0.7, label='2D Filter')
    plt.plot(xs, ls='--', color='k', lw=1, label='track')
    plt.title('State')
    plt.legend(loc=4)
    plt.show()

def compare_1D_2D(x0, P, R, Q, vel, u=None):
    # storage for filter output
    xs, xs1, xs2 = [], [], []

    # 1d KalmanFilter
    f1D = univariate_filter(x0, P, R, Q)

    #2D Kalman filter
    f2D = pos_vel_filter(x=(x0, vel), P=P, R=R, Q=0)
    if np.isscalar(u):
        u = [u]
    pos = 0 # true position
    for i in range(100):
        pos += vel
        xs.append(pos)
```

```
        # control input u - discussed below
        f1D.predict(u=u)
        f2D.predict()

        z = pos + randn()*sqrt(R) # measurement
        f1D.update(z)
        f2D.update(z)

        xs1.append(f1D.x[0])
        xs2.append(f2D.x[0])
    plt.figure()
    plot_1d_2d(xs, xs1, xs2)

compare_1D_2D(x0=0, P=50., R=5., Q=.02, vel=1.)
```



讨论

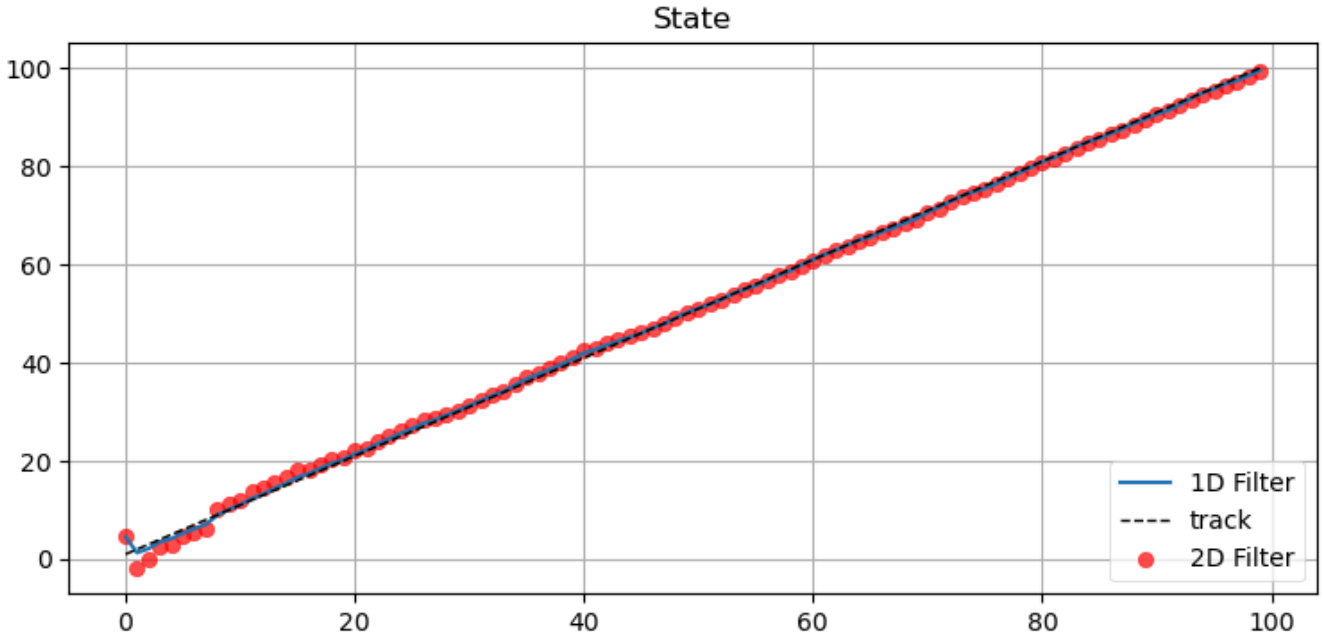
那种将速度信息也纳入状态参数的滤波器，其预测效果要比只追踪位置的滤波器好得多。单变量滤波器无法估算速度或位置的变化，因此它无法准确追踪目标物体的运动情况。

在单变量卡尔曼滤波器的相关章节中，我们在预测方程中加入了控制输入 u 。

```
    def predict(self, u=0.0):
        self.x += u
        self.P += self.Q
```

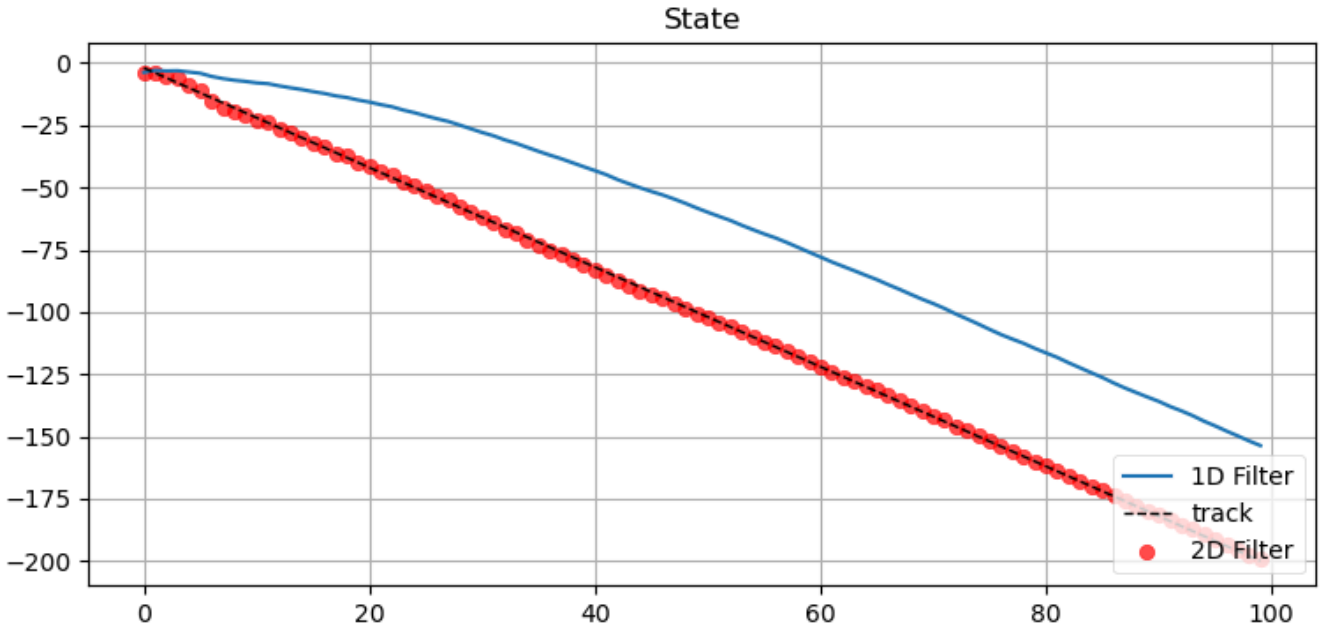
让我们试着指定一下控制输入吧：

```
compare_1D_2D(x0=0, P=50., R=5., Q=.02, vel=1., u=1.)
```



在这里，两种滤波器的性能相当，也许单变量滤波器的跟踪效果更佳。不过，让我们看看当实际速度 `vel` 与控制输入 `u` 不同时会发生什么：

```
compare_1D_2D(x0=0, P=50., R=5., Q=.02, vel=-2., u=1.)
```



如果我们正在控制某个机器人，那么使用单变量滤波器就能很好地完成跟踪任务，因为控制输入有助于滤波器做出准确的预测。但如果只是被动地跟踪机器人，那么控制输入就没什么用处了，除非我们能够事先准确预测出机器人的速度。不过，这种情况很少发生。

速度是如何计算出来的

我还没有解释该滤波器是如何计算速度的，也没有说明其中所涉及的任何隐藏变量。如果我们把为滤波器的各个矩阵计算出的数值代入其中，就能明白具体发生了什么。

首先，我们需要计算系统的不确定性。

$$\begin{aligned} \mathrm{S} &= H\bar{P}H^T + R \\ &= \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \sigma_x^2 & \sigma_{xv} \\ \sigma_{xv} & \sigma_v^2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} \sigma_z^2 \end{bmatrix} \\ &= \begin{bmatrix} \sigma_x^2 & \sigma_{xv} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} \sigma_z^2 \end{bmatrix} \\ &= \begin{bmatrix} \sigma_x^2 + \sigma_z^2 \end{bmatrix} \end{aligned}$$

现在我们有了 S，就可以计算出卡尔曼增益的值了：

换句话说， x 的卡尔曼增益为

$$K_x = \frac{VAR(x)}{VAR(x) + VAR(z)}$$

这一点在单变量情况下应该对你来说很熟悉吧。

速度 $\dot{x}$ 的卡尔曼增益为

这会带来什么影响呢？请记住，我们是这样来计算状态的：

$$\begin{aligned} x &= \bar{x} + K(z - Hx) \\ &= \bar{x} + Ky \end{aligned}$$

在这里，残差 y 是一个标量。因此，它需要与 K 中的每个元素相乘。这样一来，我们就得到了……

$$\begin{bmatrix} x \\ \dot{x} \end{bmatrix} = \begin{bmatrix} \bar{x} \\ \bar{\dot{x}} \end{bmatrix} + \begin{bmatrix} K_x \\ K_{\dot{x}} \end{bmatrix} y$$

这就得到了如下方程组：

$$\begin{aligned} x &= \bar{x} + yK_x \\ \dot{x} &= \bar{\dot{x}} + yK_{\dot{x}} \end{aligned}$$

预测值 $\bar{x}$ 的计算结果为 $x + \bar{x}\Delta t$ 。如果预测完全准确，那么残差将为 $y = 0$ （忽略测量中的噪声），此时速度估计值保持不变。相反，如果速度估计值有较大误差，那么预测值也会很差，残差会很大： $y >> 0$ 。在这种情况下，我们会用 $yK_{\dot{x}}$ 来更新速度估计值。 $K_{\dot{x}}$ 与 $COV(x, \dot{x})$ 成正比。因此，速度的更新值等于位置误差乘以与位置和速度之间的协方差成正比的数值。两者之间的相关性越高，所需的修正幅度就越大。

为了使这个过程完整起来， $COV(x, \dot{x})$ 实际上就是 P 中的非对角线元素。请记住，这些数值是通过 FPF^T 计算得出的。因此，位置和速度的协方差是在预测阶段计算出来的。速度的卡尔曼增益与这个协方差成正比，我们会根据上一个时间点上速度估计的准确性来调整速度值，调整的幅度则与这个协方差成正比。

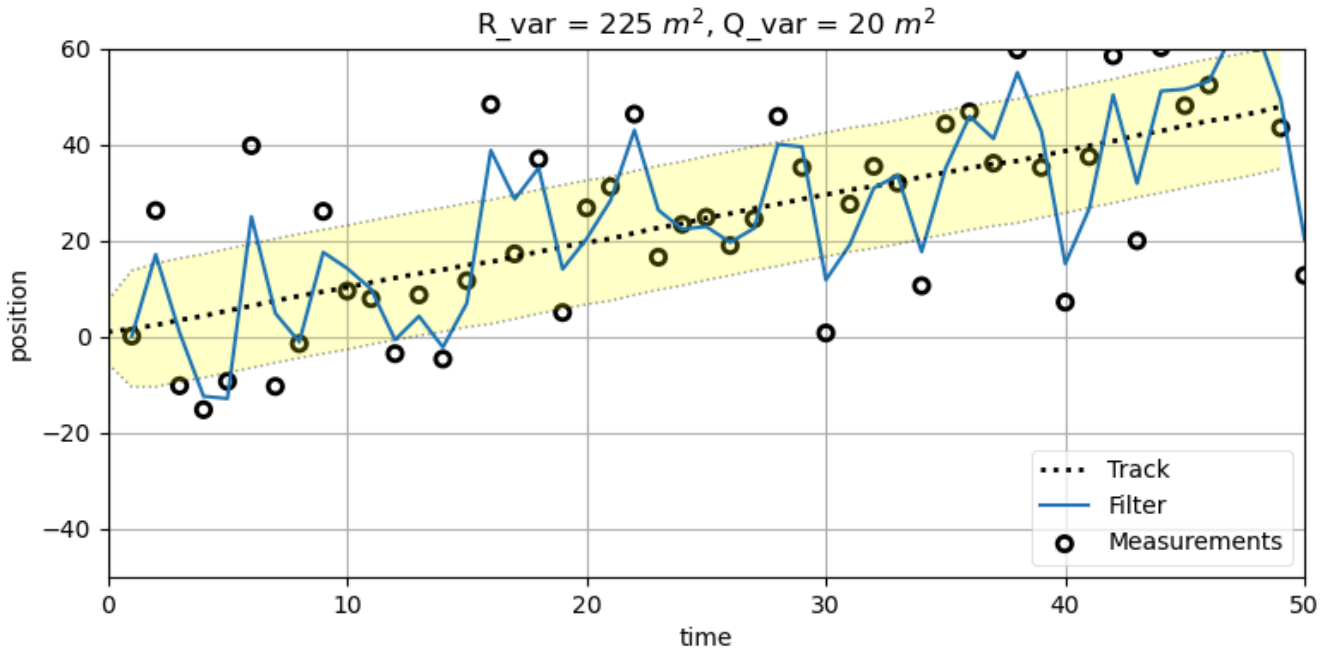
总的来说，这些线性代数方程可能让你觉得有些陌生，但实际上计算过程非常简单。本质上，这与我们在 g-h 滤波器中使用的计算方法是一样的。在本章中，由于我们需要考虑过程模型和传感器中的噪声因素，所以所使用的常数有所不同，但数学原理其实是一样的。

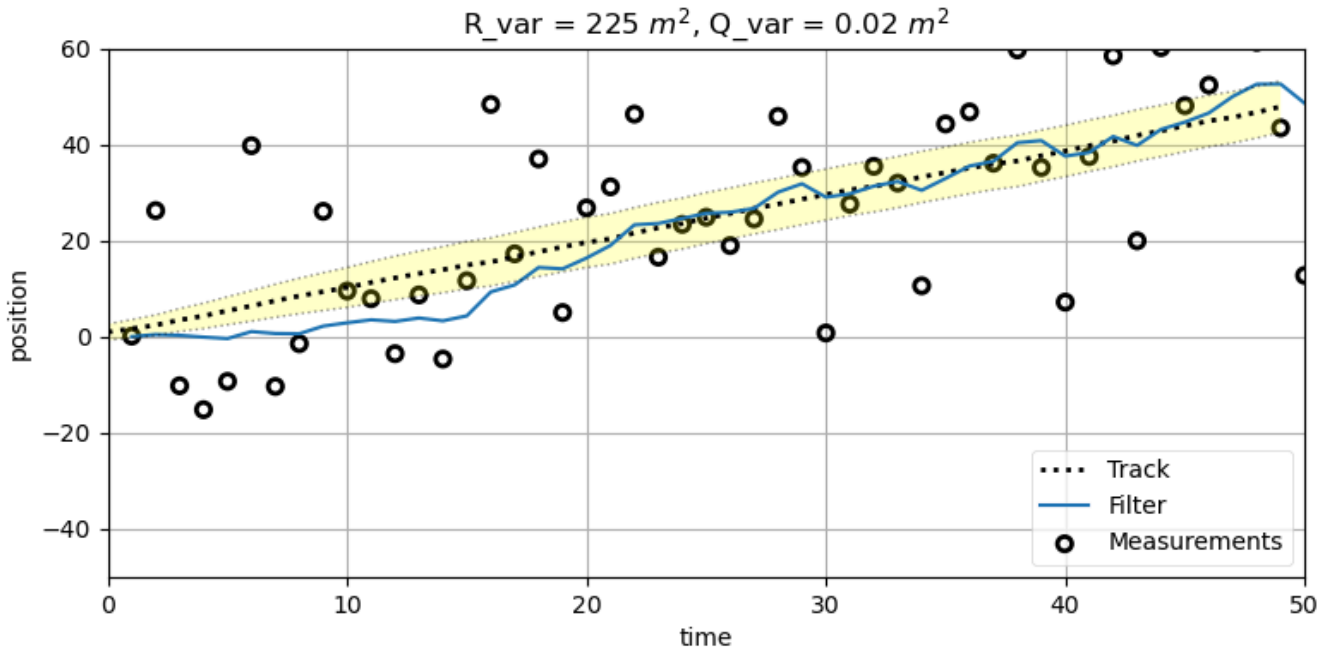
调整滤波器

让我们开始调整各种参数，看看不同调整所带来的效果。对于卡尔曼滤波器来说，这是很常规的操作。实际上，要精确地模拟传感器的性能是非常困难的，甚至是不可能的。模型不完善，那么滤波器的输出也会不准确。工程师们需要花费大量时间来调整卡尔曼滤波器，使其能够更好地适应现实世界中的传感器。现在，我们就来仔细研究这些调整所带来的效果。通过了解每项调整的效果，你就能逐渐掌握如何设计卡尔曼滤波器。设计卡尔曼滤波器既需要科学的方法，也需要艺术性的思维。我们用数学来模拟物理系统，但模型本身永远不可能做到完美。

让我们来看看测量噪声 R 和工艺噪声 Q 的影响。我们还想了解 R 和 Q 的不同设定所带来的影响。因此，我将测量结果的方差设为 225 平方米。这个数值虽然很大，但它能更明显地体现各种设计选择对结果的影响，从而便于我们理解其中的变化规律。在第一个实验中，我们保持 R 不变，只改变 Q 的值。

```
from numpy.random import seed
seed(2)
trk, zs = compute_dog_data(z_var=225, process_var=.02, count=50)

run(track=trk, zs=zs, R=225, Q=200, P=P, plot_P=False,
    title='R_var = 225 $m^2$, Q_var = 20 $m^2$')
run(track=trk, zs=zs, R=225, Q=.02, P=P, plot_P=False,
    title='R_var = 225 $m^2$, Q_var = 0.02 $m^2$');
```





在第一个图表中，滤波器的处理结果应尽可能贴近原始的、带有噪声的测量数据。而在第二个图表中，滤波器的处理结果则应与测量数据有较大的差异，同时更接近于一条直线。为什么 Q 会对图表产生这样的影响呢?

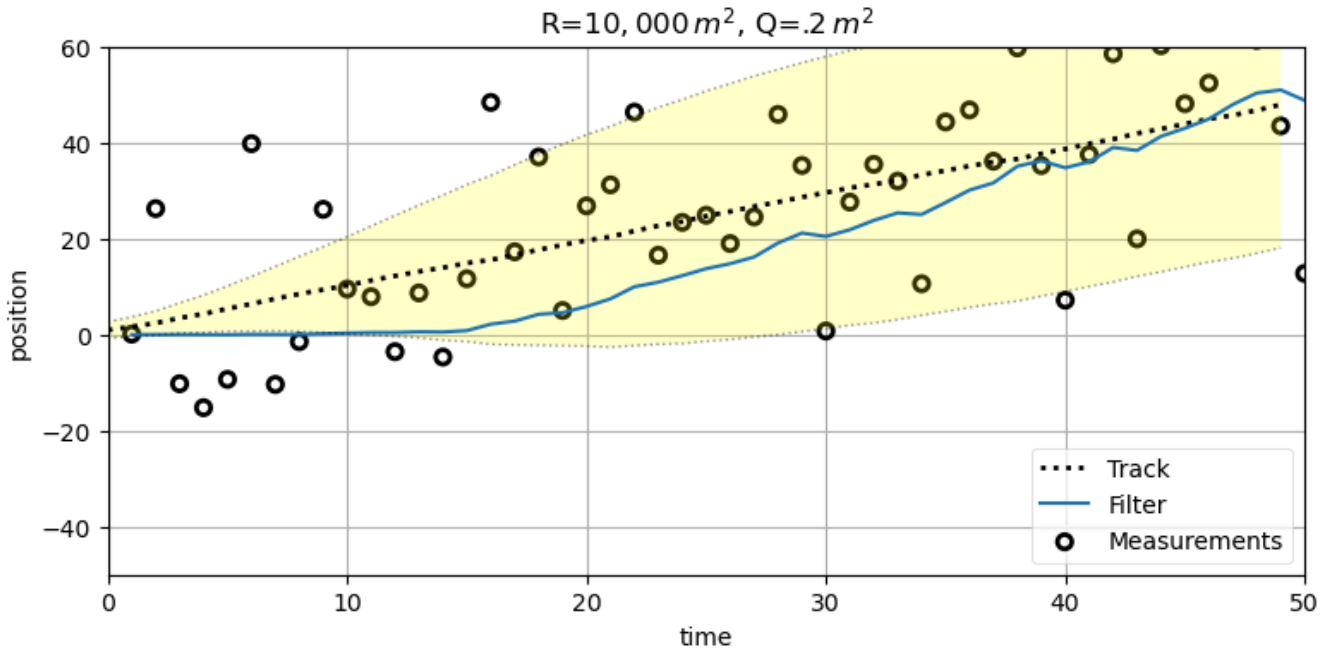
让我们先回顾一下“过程不确定性”这个概念的含义。以追踪球体的运动为例：在真空中，我们可以用数学模型来精确描述球体的运动轨迹。但是，一旦有风、空气密度变化、温度差异，或者球体表面不光滑，我们的数学模型就无法准确反映现实情况了。

在第一种情况下，我们将 `Q_var=20 m^2` 的值设置得相当大。从物理意义上讲，这相当于在告诉滤波器：“我不信任自己的运动预测结果。”因为我们指出，速度的方差为 20。严格来说，我们是在告诉滤波器：存在许多外部噪声，而这些噪声并未被 F 所体现出来。其结果就是，滤波器无法信任运动预测结果。滤波器会计算出速度值（ $\dot{x}$ ），但基本上会忽略这些数值，因为我们告诉滤波器，这些计算结果并不可信。因此，滤波器只能信任测量数据，从而会严格遵循这些测量结果来行事。

在第二种情况下，我们将 `Q_var=0.02 m^2` 的值设置得相当小。从物理意义上讲，这相当于告诉滤波器：“请相信这个预测结果，它非常准确！”更严格地说，这意味着系统中的随机噪声非常小（方差为 $0.02\ m^2$ ），因此系统模型具有很高的精确度。这样一来，滤波器就会忽略掉一些测量数据，因为这些数据的波动与我们预测的速度不符。

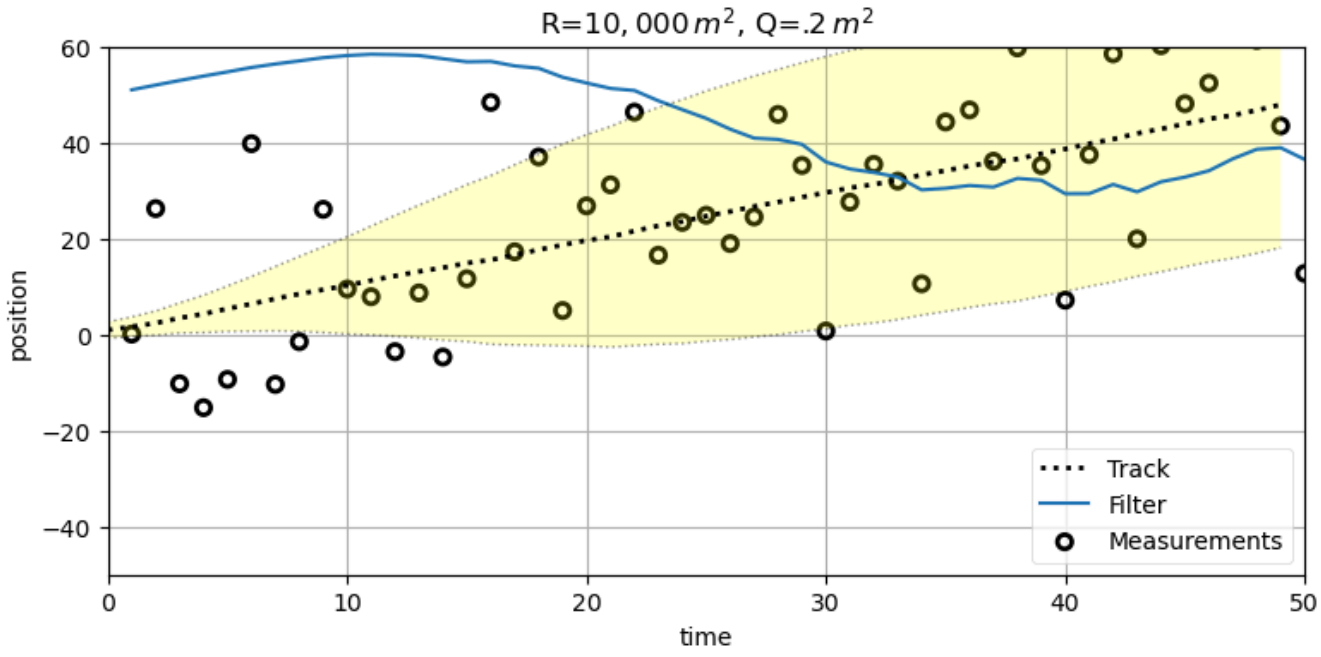
现在，让我们把 `Q_var` 的值设置为 $0.2m^2$ ，同时把 `R_var` 的值提高到 $10,000m^2$ 。这样做的目的是告诉滤波器：测量过程中的噪声非常大。

```
run(track=trk, zs=zs, R=10000, Q=.2, P=P, plot_P=False,
    title='R=$10,000\, m^2$, Q=$.2\, m^2$');
```



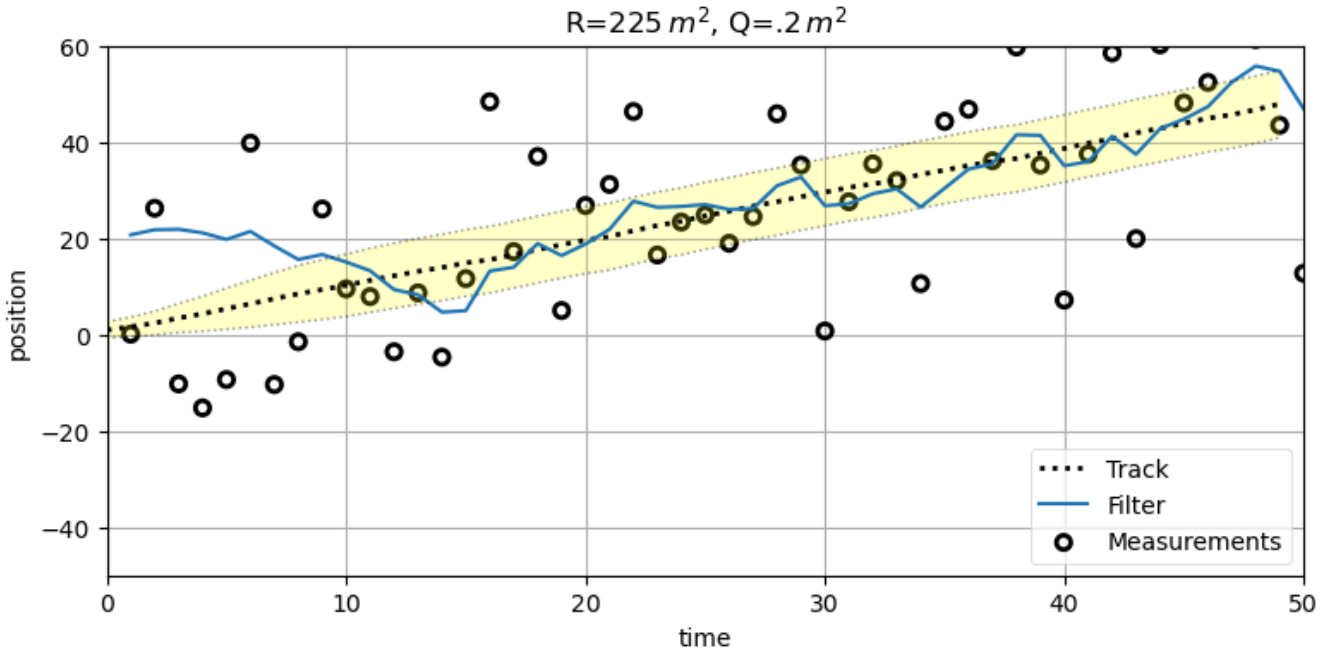
这种效应可能不太明显。我们之所以使用了效果不佳的滤波器，是因为实际的测量噪声方差为 225 m^2 ，而非 10,000 m^2 。将滤波器的噪声方差设置得如此之高，会迫使滤波器更倾向于采用预测值而非实际测量值。这样一来，虽然得到的结果看起来很平滑、很理想，但实际上并不可靠。在上图中，这条轨迹看起来非常完美，因为它与理想路径极为接近。不过，刚开始时的“完美表现”其实值得怀疑——因为滤波器尚未收敛（ P的值仍然很大），所以它不可能如此精确地反映实际位置。从图中可以看出， P的值仍未收敛，因为整个图表都用黄色背景来表示 P的大小。让我们试着用错误的初始值来测试一下：将初始位置设为 50 米，初始速度设为 1 米/秒，看看会得到什么结果。

```
run(track=trk, zs=zs, R=10000, Q=.2, P=P, plot_P=False,
    x0=np.array([50., 1.]),
    title='R=$10,000\, m^2$, Q=$.2\, m^2$');
```



我们可以看到，该滤波器无法正确获取轨迹信息。出现这种情况的原因是：尽管滤波器得到的测量结果还算不错，但它却认为这些测量结果不可靠，因此会在每一步都从错误的位置开始进行预测。如果你认为，如果测量噪声更小的话，那个错误的初始位置可能也会产生类似的结果，那我们就把初始值重新设置为 225 m^2 吧。

```
run(track=trk, zs=zs, R=225, Q=.2, P=P, plot_P=False,
    x0=np.array([20., 1.]),
    title='R=$225\, m^2$, Q=$.2\, m^2$');
```



我们可以看到，该滤波器在最初的几次迭代中难以准确追踪目标的轨迹，但之后就能准确地跟踪到我们的狗的移动轨迹。实际上，这样的效果已经相当好了——我们并没有将 Q 的设计做到最优，但 R 的设计已经是最优的。对于 Q 的设定，一般的经验法则是将其值定在 $\frac{1}{2}\Delta a$ 和 Δa 之间，其中 Δa 表示加速度在各个采样周期之间的最大变化幅度。当然，这一设定前提是我们假设加速度是恒定的，并且在不同时间周期之间彼此无关。在卡尔曼滤波的数学原理部分，我们将探讨多种设计 Q 的方法。

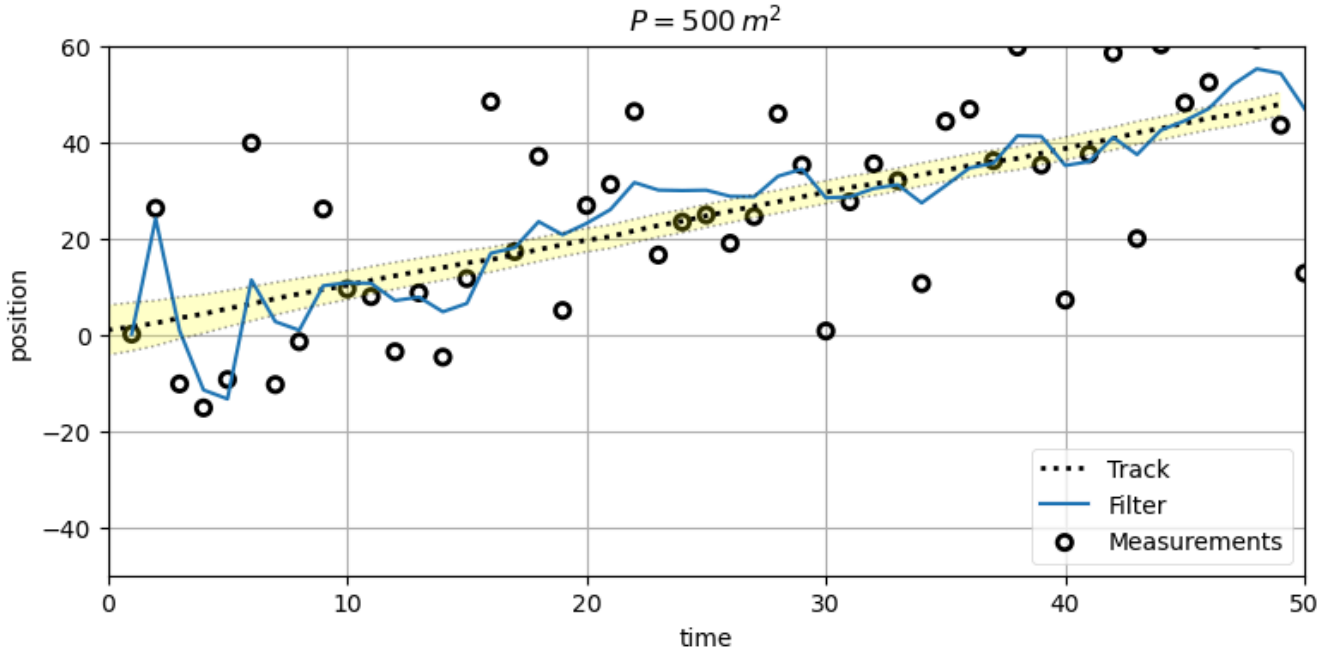
在某种程度上，通过调整 R 或 Q 的值，也可以得到类似的结果。但我建议不要随意更改这些数值，直到得到自己满意的输出为止。务必考虑到这些参数调整所带来的实际影响，并根据你对所处理系统的了解来调整 R 和/或 Q 的值。同时，还需要通过大量的模拟测试或使用真实数据来进行验证。

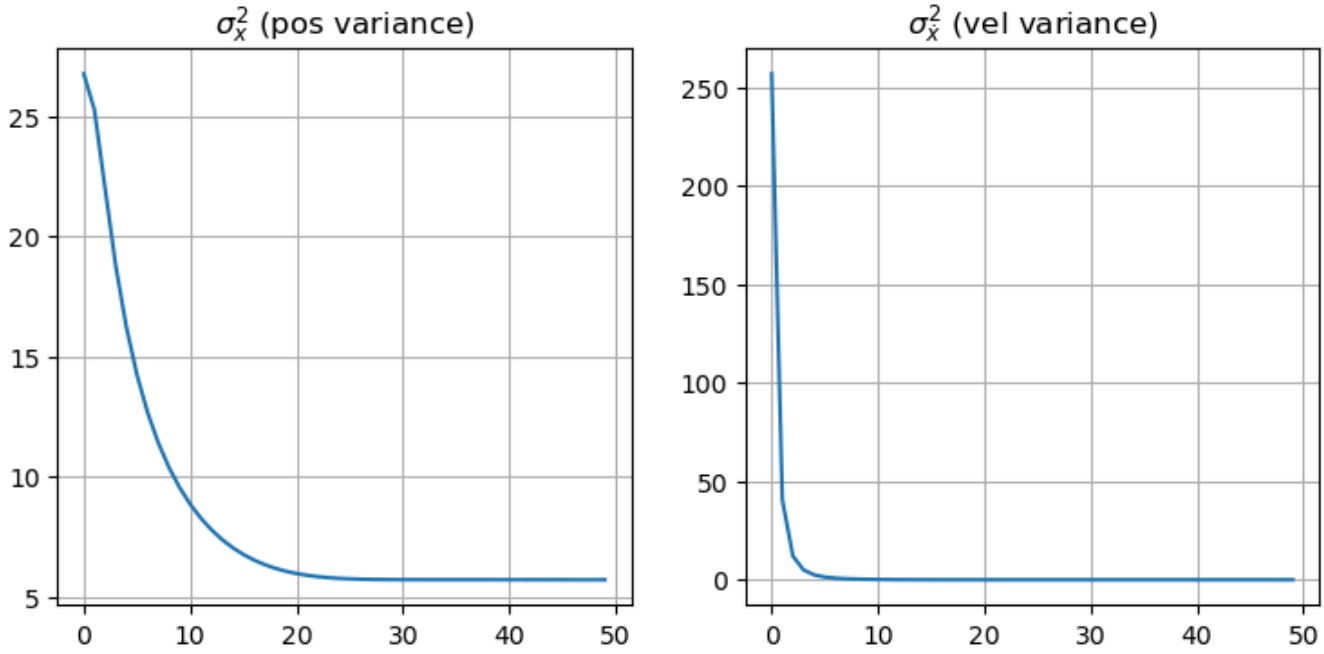
协方差矩阵的详细分析/对协方差矩阵的深入研究

我们先来重新回顾一下如何绘制曲线图。我把数据和噪声都硬编码到了 `zs_var_275` 中，这样就不受随机数生成器的影响了。因为随机数生成器可能会产生无法准确反映我想要表达的内容的数据。我会从 `P=500` 开始。

```
import kf_book.mkf_internal as mkf_internal

var = 27.5
data = mkf_internal.zs_var_275()
```

```
run(track=trk, zs=zs, R=var, Q=.02, P=500., plot_P=True,
    title='$P=500\, m^2$');
```





从输出结果来看，滤波器的输出在开始时出现了极大的波动。我们设置了 $\mathrm{P} = 500I_2$（这是一种简写方式，指的是一个对角线元素为 500 的 2x2 对角矩阵）。现在，我们有足够的信息来理解这一现象的含义，以及卡尔曼滤波器是如何处理这一情况的。左上角的 500 对应的是 σ_x^2；因此， x 的标准差为 $\sqrt{500}$ ，大约为 22.36 米。大约 99%的样本值都落在 3σ 的范围内。因此， $\sigma_x^2 = 500$ 告诉卡尔曼滤波器：预测值可能与实际值相差多达 67 米。这是一个相当大的误差。当测量值出现剧烈波动时，卡尔曼滤波器会不信任自己的估算结果，从而尝试通过调整来纳入这个新的测量值。随着滤波器的不断运行， P 会迅速收敛到更合理的数值。

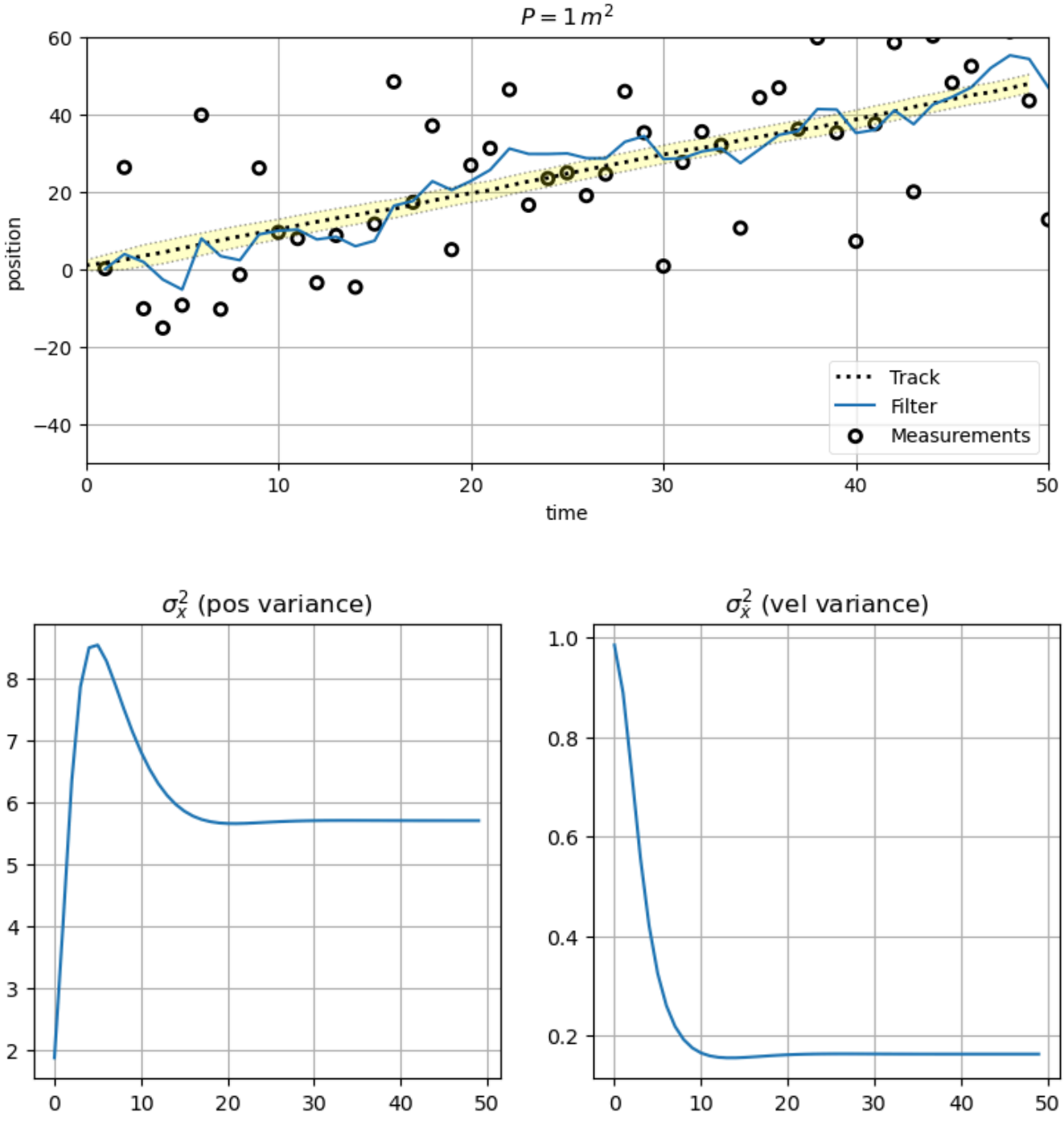
让我们来看看其中的数学原理。卡尔曼增益的公式为：

$$K = \bar{P}H^T S^{-1} \approx \frac{\bar{P}H^T}{S} \approx \frac{uncertainty_{prediction}}{uncertainty_{measurement}} H^T$$

这是一个关于预测值与测量值之间不确定性的比率。在这里，预测值的不确定性较大，因此 K 的数值也较大（如果这是一个标量的话，其值接近 1）。 K 与残差 $\mathrm{y} = z - H\bar{x}$ 相乘，而残差就是测量值减去预测值。因此，当 K 的数值较大时，就意味着测量值的可靠性更高。所以，如果 P 相对于传感器本身的不确定性 R 来说较大，那么滤波器就会主要依据测量值来生成最终的估计结果。

现在，让我们来看看 $P = 1.0I_2$ 的初始值较小时会产生怎样的效果。

```
run(track=trk, zs=zs, R=var, Q=.02, P=1., plot_P=True,
    title='$P=1\, m^2$');
```

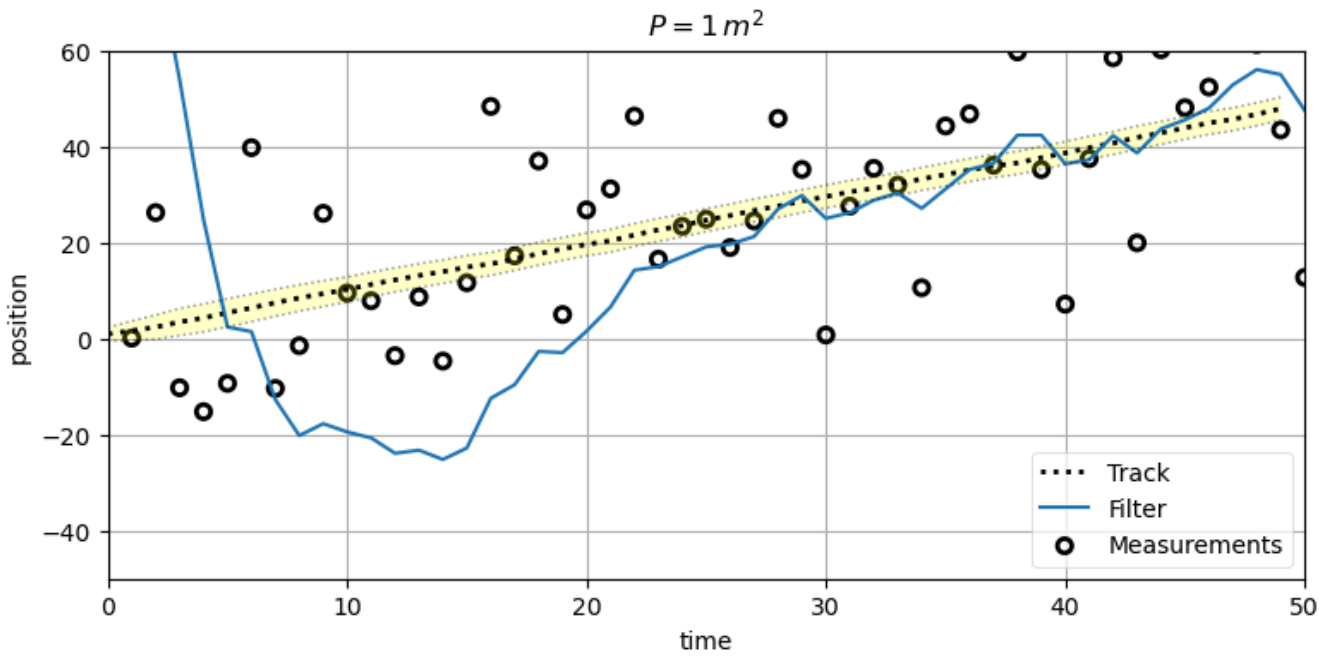


乍看之下，这个设计似乎不错。与之前的设计相比，该方案的曲线没有出现尖锐的峰值；滤波器能够很快开始对测量数据进行处理，无需很长时间就能稳定下来。不过，如果我们观察 P 相关的曲线，会发现位置变化的方差在开始时会有一个峰值，而且这个峰值始终无法真正稳定下来。糟糕的设计会导致较长的稳定时间，从而使得整体性能不佳。

因此，尽管滤波器的跟踪效果与实际信号非常接近，但我们仍不能断定，使用较小的 P 值就是解决问题的“秘诀”。没错，这样做确实可以避免卡尔曼滤波器需要较长时间才能准确跟踪信号。但如果我们对初始测量值本身就不确定，那么这种做法反而会导致滤波器产生糟糕的结果。如果我们要跟踪某个移动中的物体，那么在开始跟踪之前，我们很可能无法准确知道它的位置。而如果我们只是对温度计的测量结果进行滤波处理，那么第一次测量得到的数值与第 1000 次测量得到的数值具有同样的可靠性。为了让卡尔曼滤波器能够正常工作，必须将 P 的值设定为能够真实反映我们对数据的了解的数值。

让我们来看看，当初始估计值错误，且 P 的值又非常小的时候，会得到怎样的结果。我们将初始估计值设为 x = 100 米（而实际上，狗的起始位置应该是 0 米）。同时，我们将 P=1 米设为 ^{2} 。显然， P 的这个数值是错误的，因为实际误差为 100 米。但我们却告诉滤波器， 3σ 的误差仅为 3 米。

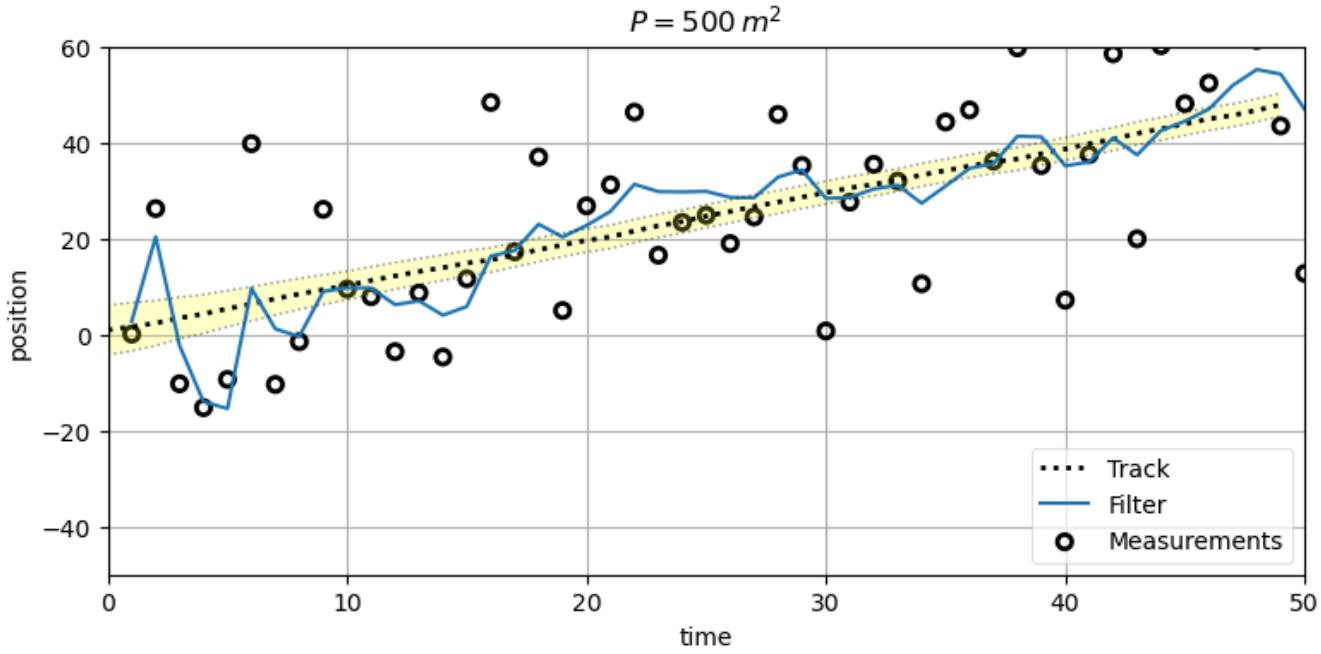
```
x = np.array([100., 0.])
run(track=trk, zs=zs, R=var, Q=.02, P=1., x0=x,
    plot_P=False, title='$P=1\, m^2$');
```



我们可以看到，最初的估计结果相当糟糕，而且卡尔曼滤波器需要很长时间才能开始收敛到正确的信号值。这是因为我们告诉卡尔曼滤波器，我们非常相信 100 米的初始估计值，但实际上这种判断是错误的。

现在，让我们给 P 一个更合理的数值，然后看看效果有何不同。

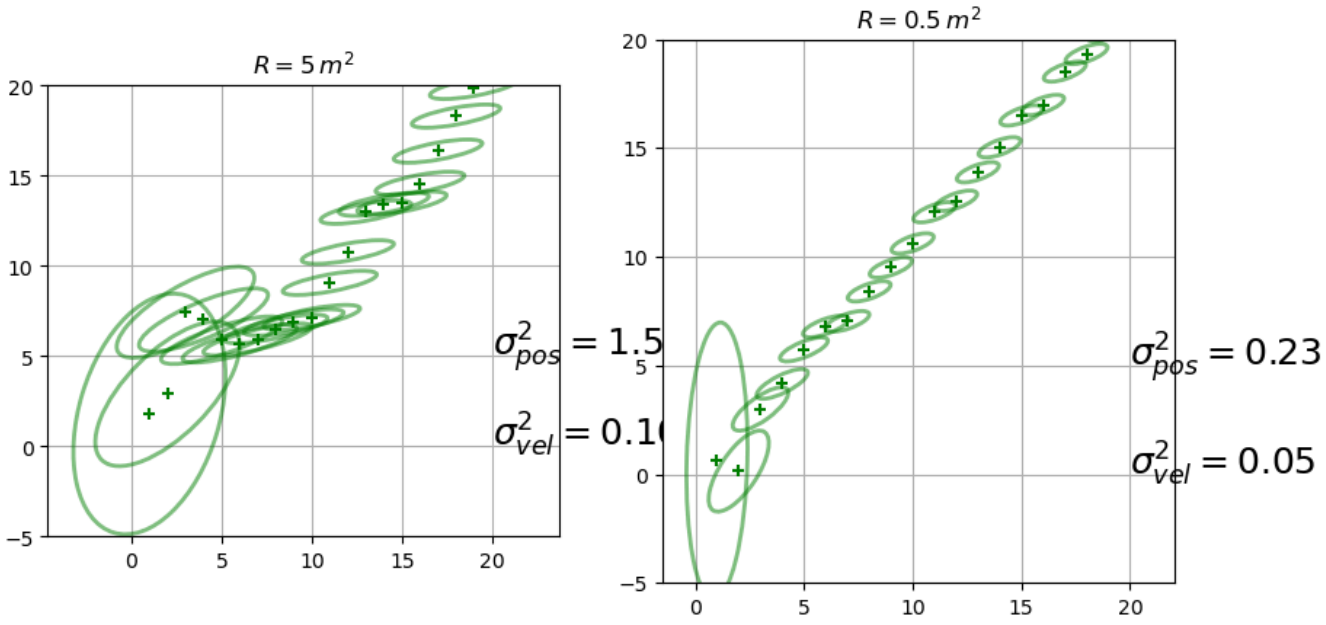
```
x = np.array([100., 0.])
run(track=trk, zs=zs, R=var, Q=.02, P=500., x0=x,
   plot_P=False, title='$P=500\, m^2$');
```



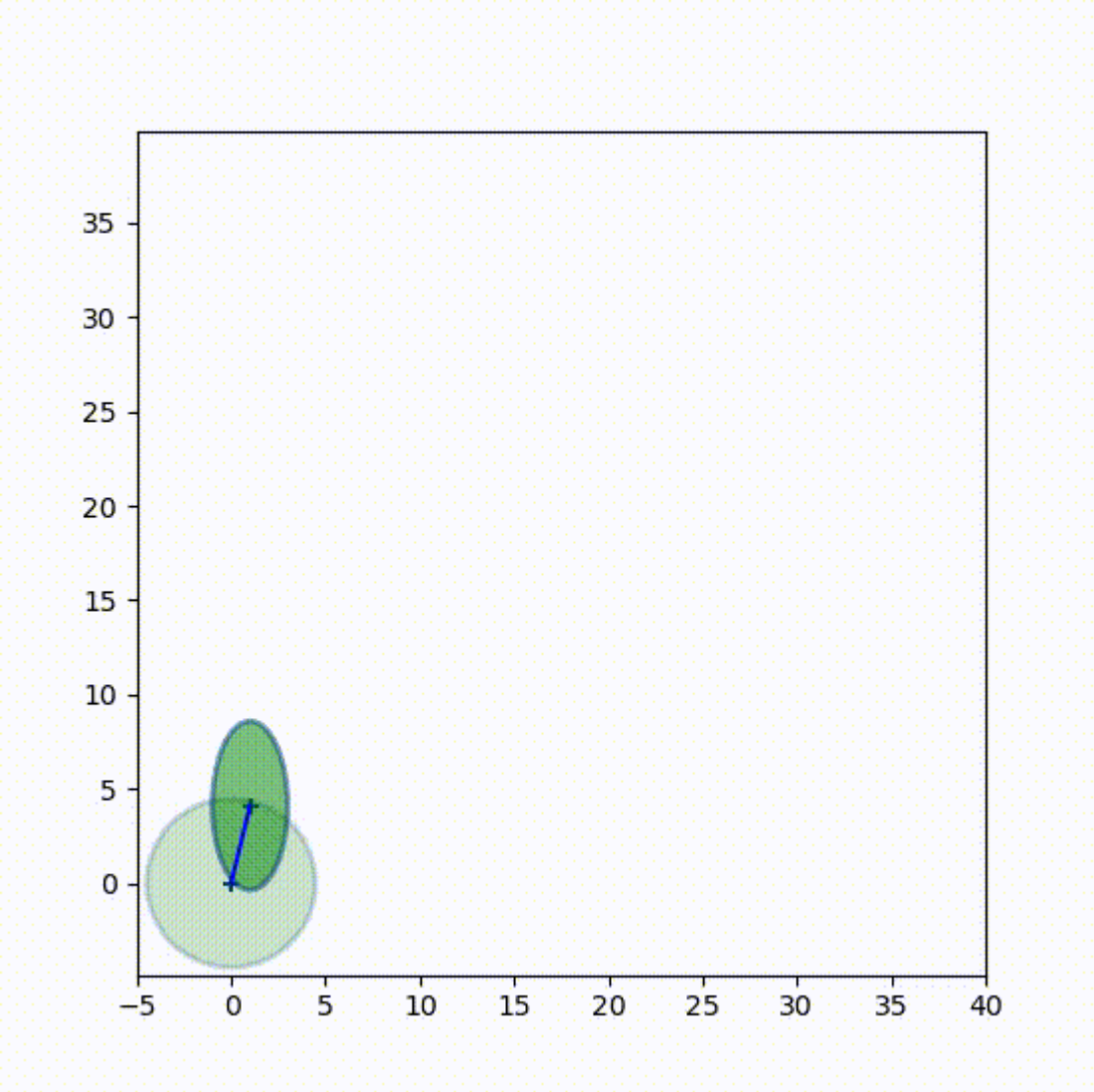
在这种情况下，卡尔曼滤波器对初始状态的估计相当不确定，因此它能够更快地收敛到正确的信号值。只需 5 到 6 个周期后，该滤波器就能输出较为准确的数值。根据我们目前所发展的理论，这已经算是最好的结果了。不过，这种情形其实有点人为制造出来的；如果我们不知道在开始追踪时物体位于何处，就不应该将滤波器的初始值设定为某个任意数值，比如 0 米或 100 米。我将在下文的“滤波器初始化”部分详细阐述这一点。

让我们再为我们的“狗”应用一次卡尔曼滤波算法。这次，把协方差椭圆与狗的位置一起绘制在同一个图表上吧。

```
from kf_book.mkf_internal import plot_track_ellipses

def plot_covariances(count, R, Q=0, P=20., title=''):
    track, zs = compute_dog_data(R, Q, count)
    f = pos_vel_filter(x=(0., 0.), R=R, Q=Q, P=P)

    xs, cov = [], []
    for z in zs:
        f.predict()
        f.update(z)

        xs.append(f.x[0])
        cov.append(f.P)
    plot_track_ellipses(count, zs, xs, cov, title)

plt.figure(figsize=(10,6))
plt.subplot(121)
plot_covariances(R=5, Q=.02, count=20, title='$R = 5\, m^2$')
plt.subplot(122)
plot_covariances(R=.5, Q=.02, count=20, title='$R = 0.5\, m^2$')
```



如果您是在 Jupyter Notebook 或网页上查看此内容，那么这里有一个展示该滤波器如何处理数据的动画。我已经调整了滤波器的参数，这样就能清楚地看到随着滤波器不断运行， P 的值是如何变化的。



这些图表的显示效果有点混乱，但你应该能够理解其中所反映的情况。在两个图表中，我们都绘制了每个点的协方差矩阵。首先，我们看到的是协方差 $P = \begin{pmatrix} 20 & 0 \\ 0 & 20 \end{pmatrix}$，这表明我们对初始状态的估计存在很大的不确定性。在接收到第一组测量数据后，卡尔曼滤波器会更新这一估计值，因此方差也就不再那么大了。在上面的图表中，最左边的那个椭圆应该稍微有些变形。随着滤波器不断处理更多的测量数据，协方差椭圆的形状会逐渐稳定下来，最终变成一个细长的椭圆，且其方向与物体的运动方向一致。

试想一下，这在物理上意味着什么。椭圆的 x 轴代表位置上的不确定性，y 轴则代表速度上的不确定性。因此，如果椭圆在垂直方向上的尺寸大于水平方向上的尺寸，那就意味着我们对速度的不确定性大于对位置的不确定性。相反，如果椭圆在水平方向上的尺寸较大，那么就意味着位置上的不确定性较大，而速度上的不确定性较小。最后，椭圆的倾斜程度则反映了这两个变量之间的关联程度。

第一个图表中，`R=5` m^2 所对应的图形最终呈现为一个椭圆形，其宽度大于高度。如果还不清楚的话，我在右下角列出了最后一个椭圆形的各项数值。

相比之下，第二个图表中，以 `R=0.5` m^2 表示的椭圆，其高度大于宽度。第二个图表中的所有椭圆都比第一个图表中的椭圆小得多。这是合乎逻辑的，因为较小的 R 意味着测量中的噪声较小。噪声越小，预测就越准确，因此我们对所处位置的判断也就越有信心。

问题：请解释椭圆的种种差异。

为什么 $R = 5m^2$ 所对应的椭圆比 $R = 0.5m^2$ 所对应的椭圆更接近水平方向呢？提示：请从这些椭圆在物理意义上的含义来思考这个问题，而不仅仅是从数学角度来分析。如果你不确定答案，可以试着将 R 替换为极大或极小的数值，比如 $100\ m^2$ 和 $0.1\ m^2$，观察由此产生的变化，再思考其中的含义。

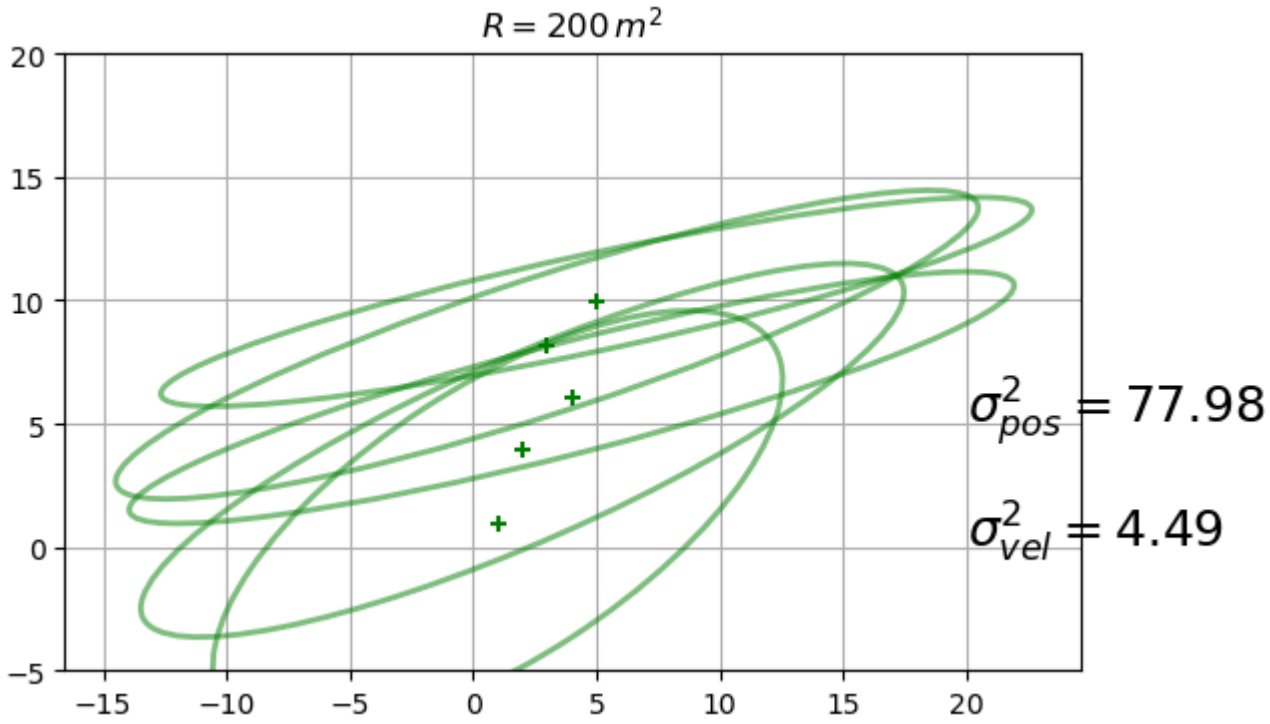
解决方案

x 轴代表位置，y 轴代表速度。如果椭圆几乎是垂直的，那就说明位置和速度之间没有关联；而如果椭圆是斜的，则说明两者之间存在很强的关联。不过，这样的结果听起来不太合理。虽然椭圆的倾斜角度会发生变化，但两者之间的关联程度不应该随时间而改变。不过，这其实只是对滤波器输出结果的描述，而非对现实物理世界的真实描述。当 R 的数值很大时，就意味着测量数据中包含大量噪声。在这种情况下，卡尔曼增益 K 会被设置为更重视预测结果而非测量数据，而预测结果则来自速度状态变量。因此，x 和 $\dot{x}$ 之间会有很强的关联。相反，如果 R 的数值较小，那就意味着测量数据相当可靠，此时 K 会被设置为更重视测量数据而非预测结果。如果测量结果已经非常精确了，那为什么滤波器还要使用预测结果呢？如果滤波器几乎不利用预测结果，那么所报告的相关性也会非常低。

这是需要重点理解的一点。卡尔曼滤波器是一种用于描述现实世界系统的数学模型。如果测量结果与预测结果之间的相关性较低，这并不意味着在物理系统中就不存在相关性，而只是说明在数学模型中不存在线性相关性而已。这一指标实际上反映了在模型中，测量数据与预测结果所占的比重。

让我们用一个极大的测量误差来来具体说明这一点吧。我们设定 $R = 200m^2$ 为某个数值。在查看图表之前，先试着想象一下图表会是什么样子吧。

```
plot_covariances(R=200., Q=.2, count=5, title='$R = 200\, m^2$')
```



希望得到的结果符合您的预期。该椭圆很快变得非常宽，而高度则很低。出现这种情况的原因是：卡尔曼滤波器在生成滤波结果时，主要依据的是预测值而非实际测量值。我们还可以看出，该滤波器在获取轨迹信息方面反应较慢。卡尔曼滤波器认为测量值中包含大量噪声，因此它需要很长时间才能更新对 $\dot{x}$ 的估计值。

请持续研究这些图表，直到你明白如何解读协方差矩阵 P。当处理 9×9 矩阵时，可能会觉得有些难以理解——毕竟有 81 个数值需要分析。不过，可以将其拆解开来理解：对角线上的数值代表各个状态变量的方差；而所有非对角线上的数值，则是两个方差与某个比例因子 p 的乘积。由于无法在屏幕上直接绘制 9×9 矩阵，因此你需要在這个简单的二维案例中培养自己的直觉和理解能力。

> 在绘制协方差椭圆时，务必在代码中使用 ax.set_aspect('equal')或 plt.axis('equal')。前者可以用来设置 x 轴和 y 轴的取值范围。如果坐标轴的刻度不同，那么绘制出的椭圆就会变形。例如，椭圆可能看起来比实际更窄，但实际上它可能比看上去要宽。

滤波器初始化

有多种初始化滤波器的方法。在大多数情况下，以下方法效果不错。按照这种方法，你需要等到获得第一个测量值 z_0 之后，才开始对滤波器进行初始化。根据这个测量值，你可以利用 $x_0 = z_0$ 来计算 x 的初始值。如果 z 的尺寸、类型和单位与 x 不一致——这种情况很常见——我们可以按照以下方式来使用我们的测量函数。

我们知道。

$$z = Hx$$

因此，

$$\begin{aligned} H^{-1}Hx &= H^{-1}z \\ x &= H^{-1}z \end{aligned}$$

矩阵求逆需要一个方阵，但 H 很少是方阵。SciPy 会计算 `scipy.linalg.pinv` 所表示的矩阵的摩尔-彭若斯伪逆，因此你的代码可能如下所示：

```
from scipy.linalg import pinv

H = np.array([[1, 0.]])
z0 = 3.2
x = np.dot(pinv(H), z0)
print(x)
```

```
[[3.2]
 [0. ]]
```

如果你对所处理的问题领域有深入的了解，可能会采用不同的计算方法。不过，这也是一种可行的处理方式。例如，如果系统中包含速度这一参数，你可以取前两次位置测量的数值，计算出两者之间的差值，再把这个差值作为初始速度。

现在，我们需要计算 P 的值。这个数值会因具体问题而有所不同。一般来说，对于相同的项，应使用测量误差 R 作为其值；而对于其他项，则应使用其最大值。也许这一点不太容易理解。在本章中，我们一直以位置和速度作为物体的状态参数来进行跟踪，而测量结果也是位置数据。在这种情况下，我们应该将 P 初始化为……

$$P = \begin{bmatrix} R_0 & 0 \\ 0 & vel^2_{max} \end{bmatrix}$$

P 的对角线元素代表了各个状态变量的方差，因此我们需要为这些元素赋予合理的数值。对于位置而言， R_0 是一个合理的方差值；而对于速度而言，最大速度的平方则是一个合理的方差值。之所以要取平方，是因为方差的定义本身就是如此： σ^2 。

你确实需要充分了解自己所工作的领域，并根据最可靠的信息来初始化滤波器。例如，假设我们试图追踪赛马比赛中的马匹位置。最初的测量结果可能相当不准确，导致得到的马匹位置与实际起点相去甚远。我们知道，马匹必须从起点开始奔跑；如果以最初的测量结果来初始化滤波器，那么得到的结果肯定不会理想。在这种情况下，我们应该始终以马匹的起点位置来初始化卡尔曼滤波器。

批量处理

卡尔曼滤波器是一种递归算法——每当有新的测量数据进来时，系统会立即更新估算结果。不过，通常情况下，我们手头还有许多已经收集到的数据也需要进行滤波处理。卡尔曼滤波器也可以以批量模式运行，即一次性处理所有测量数据。我们在 `KalmanFilter.batch_filter()` 中实现了这种功能。在内部，该函数会遍历所有的测量数据，将计算得到的状态值和协方差值存储在数组中。这种方式简化了代码逻辑，同时便于将所有处理结果汇总起来。我经常使用这个函数，但为了让大家能充分理解预测/更新循环的运作原理，我决定把相关内容放到本章的末尾再讲解。

首先，把各项测量数据收集到一个数组或列表中。这些数据可能保存在 CSV 文件里。

```
zs = read_altitude_from_csv('altitude_data.csv')
```

或者，你也可以使用生成器来生成它：

```
zs = [some_func(i) for i in range(1000)]
```

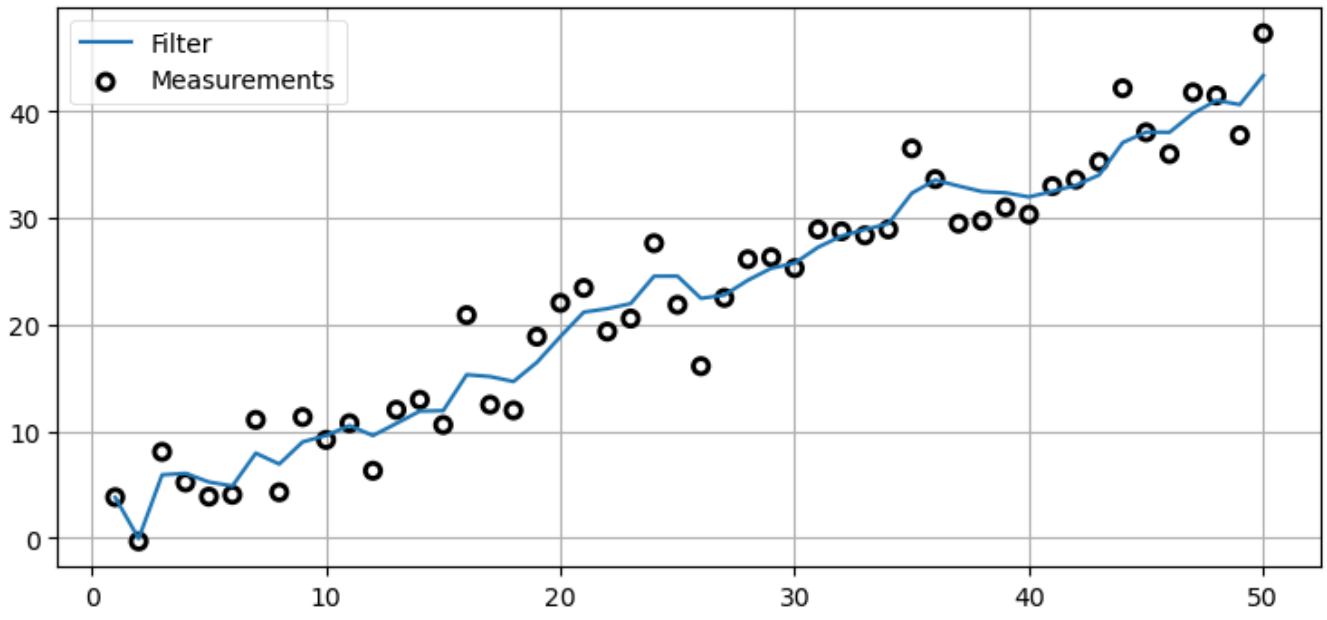
然后调用 `batch_filter()` 方法。

```
Xs, Ps, Xs_prior, Ps_prior = kfilter.batch_filter(zs)
```

该函数接收测量值列表，对其进行处理后，返回一个 NumPy 数组，其中包含状态估计值(Xs)、协方差矩阵(Ps)，以及这些值的先验概率(Xs_prior, Ps_prior)。

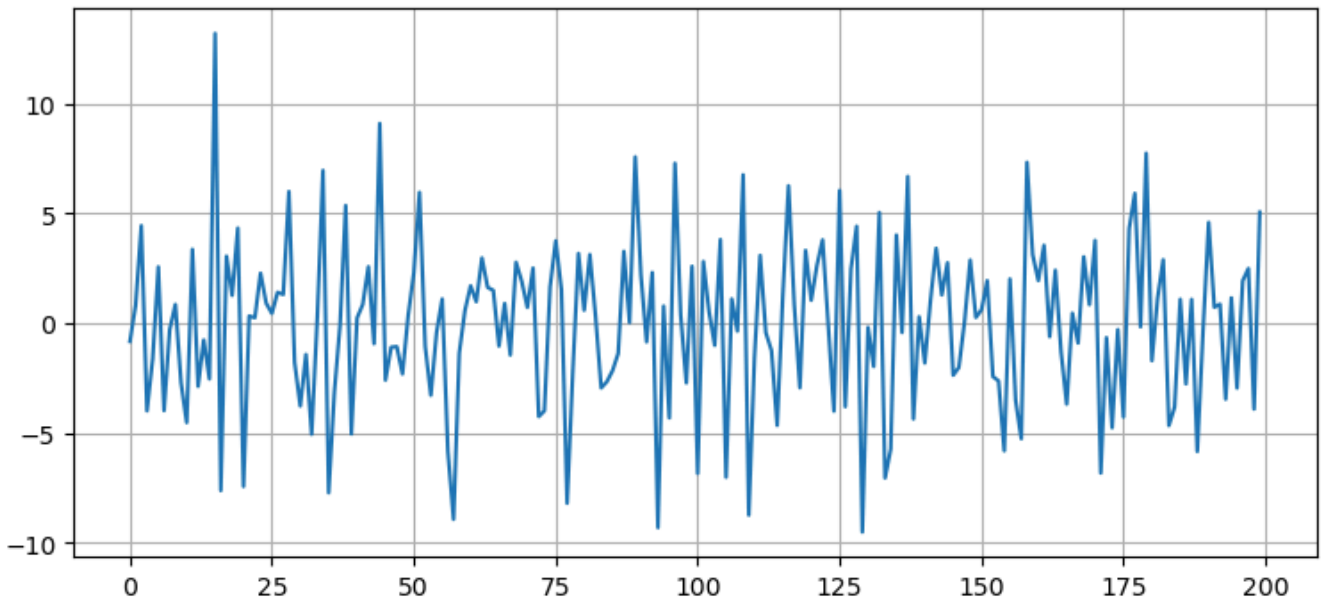
以下是一个完整的示例。

```
count = 50
track, zs = compute_dog_data(10, .2, count)
P = np.diag([500., 49.])
f = pos_vel_filter(x=(0., 0.), R=3., Q=.02, P=P)
xs, _, _, _ = f.batch_filter(zs)

book_plots.plot_measurements(range(1, count + 1), zs)
book_plots.plot_filter(range(1, count + 1), xs[:, 0])
plt.legend(loc='best');
```



该批量滤波器接受一个可选的 `filterpy.common.Saver` 对象作为参数。如果提供了该对象，那么滤波器的所有属性都会被一并保存下来。如果你想查看除状态和协方差之外的其他数值，这种方式非常有用。在这里，我通过绘制残差图来判断这些残差是否表现为以 0 为中心的噪声。这是一种简单的视觉检查方法，有助于判断滤波器的设计是否合理。如果残差偏离了 0，或者看起来不像是噪声，那就说明滤波器的设计有缺陷，或者相关过程并非高斯分布。我们将在后面的章节中详细讨论这个问题。目前，可以将这视为对 `Saver` 类的一个演示而已。

```
track, zs = compute_dog_data(10, .2, 200)
P = np.diag([500., 49.])
f = pos_vel_filter(x=(0., 0.), R=3., Q=.02, P=P)
s = Saver(f)
xs, _, _, _ = f.batch_filter(zs, saver=s)
s.to_array()
plt.plot(s.y);
```



对结果进行平滑处理

这本书中有一章专门介绍如何对处理结果进行平滑处理；这里就不再重复这些内容了。不过，这个工具的使用非常简单，而且能显著提升处理效果。我会用几个例子来简单演示一下。关于平滑处理的那一章并不难理解，你现在应该已经有足够的准备来阅读它了。

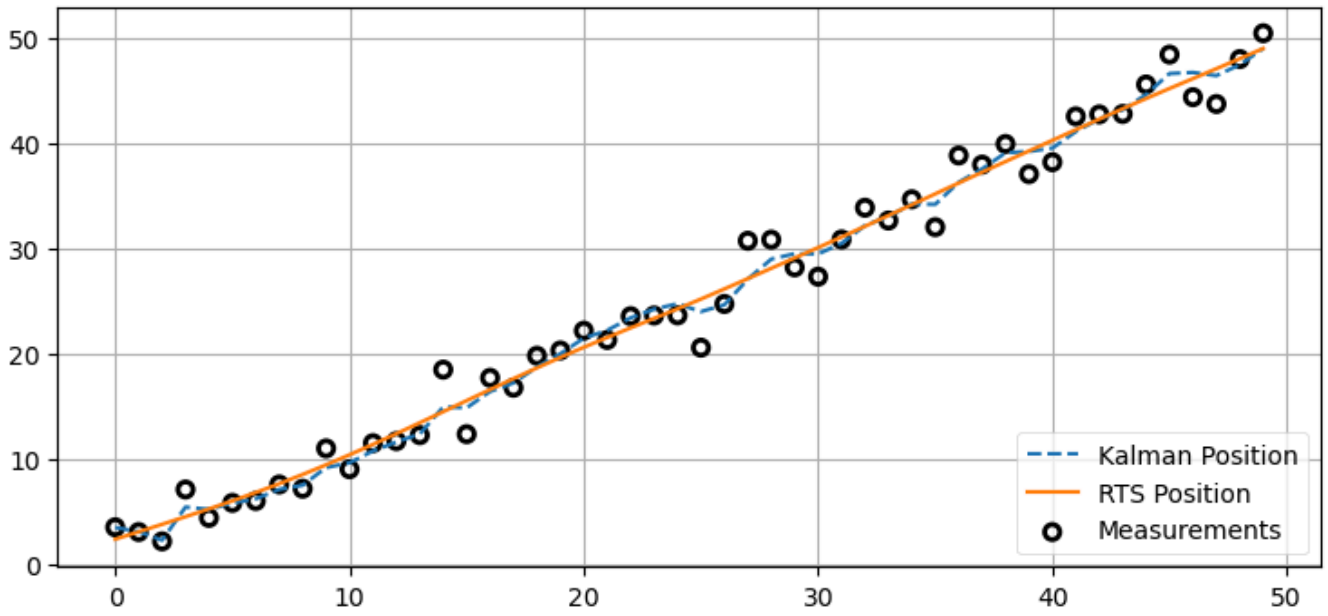
假设我们正在追踪一辆沿直线行驶的汽车。我们得到的测量结果显示，该汽车开始向左转弯。卡尔曼滤波器会调整状态估计值，使其更接近该测量结果。不过，卡尔曼滤波器无法判断：这一测量结果是由于噪声干扰产生的，还是汽车确实开始转弯了。

不过，如果我们有后续的测量数据，就能判断物体是否真的发生了转向。假设后续的所有测量结果都表明物体在继续向左转，那么我们就可以确定物体确实发生了转向。相反，如果后续的测量结果显示物体一直沿直线运动，那么就可以认为这些测量结果存在误差，应该被忽略。这样一来，我们就不需要在测量结果和预测结果之间进行折中处理了。根据后续测量结果所反映的物体运动情况，我们可以要么完全采纳这些测量结果，要么直接忽略它们。

`KalmanFilter` 实现了该算法的一种形式，这种形式被称为“RTS 平滑器”。该名称源自该算法的发明者：Rauch、Tung 和 Striebel。使用该方法时，需要输入在 `batch_filter` 步骤中计算得到的均值和协方差，之后就能得到经过平滑处理后的均值、协方差以及卡尔曼增益。

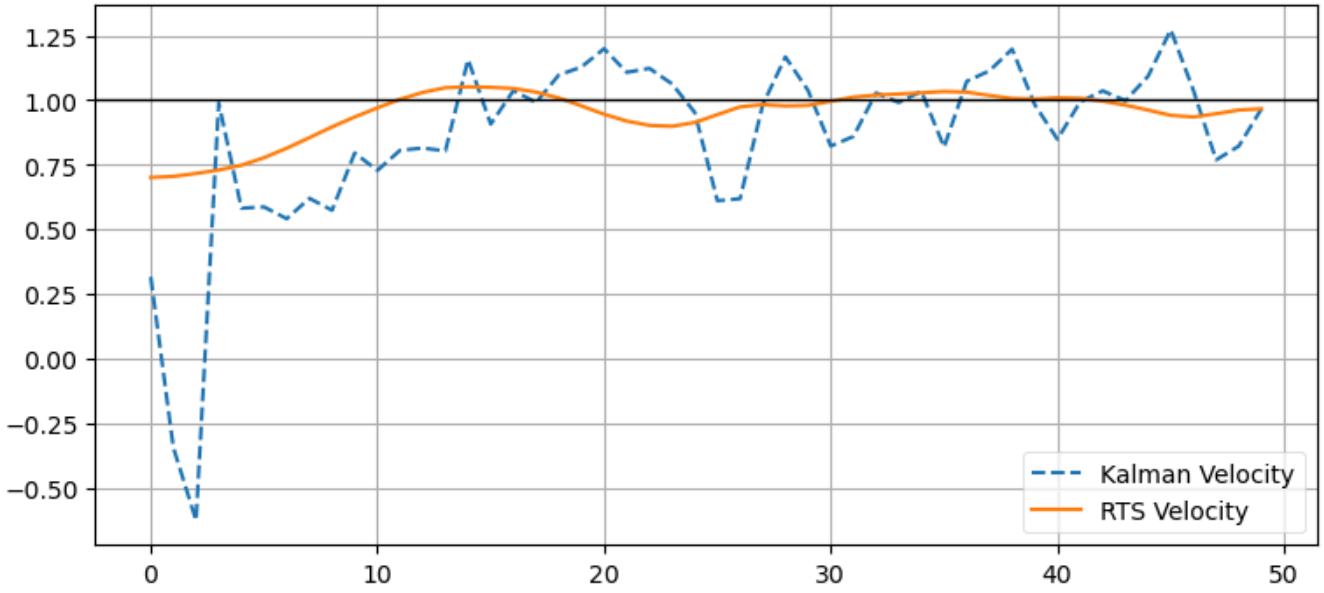
```
from numpy.random import seed
count = 50
```

```
seed(8923)

P = np.diag([500., 49.])
f = pos_vel_filter(x=(0., 0.), R=3., Q=.02, P=P)
track, zs = compute_dog_data(3., .02, count)
Xs, Covs, _, _ = f.batch_filter(zs)
Ms, Ps, _, _ = f.rts_smoother(Xs, Covs)

book_plots.plot_measurements(zs)
plt.plot(Xs[:, 0], ls='--', label='Kalman Position')
plt.plot(Ms[:, 0], label='RTS Position')
plt.legend(loc=4);
```



这个输出效果真是棒极了！从这张图表中可以明显看出两点。首先，RTS 平滑处理后的输出结果比 KF 方法的输出结果要平滑得多。其次，RTS 方法的输出结果几乎总是比 KF 方法的更准确（我们将在“平滑处理”这一章节中详细探讨这一点）。至于作为“隐藏变量”的速度值，其改善程度则更为显著：

```
plt.plot(Xs[:, 1], ls='--', label='Kalman Velocity')
plt.plot(Ms[:, 1], label='RTS Velocity')
plt.legend(loc=4)
plt.gca().axhline(1, lw=1, c='k');
```



我们将在接下来的练习中探讨其中的原因。

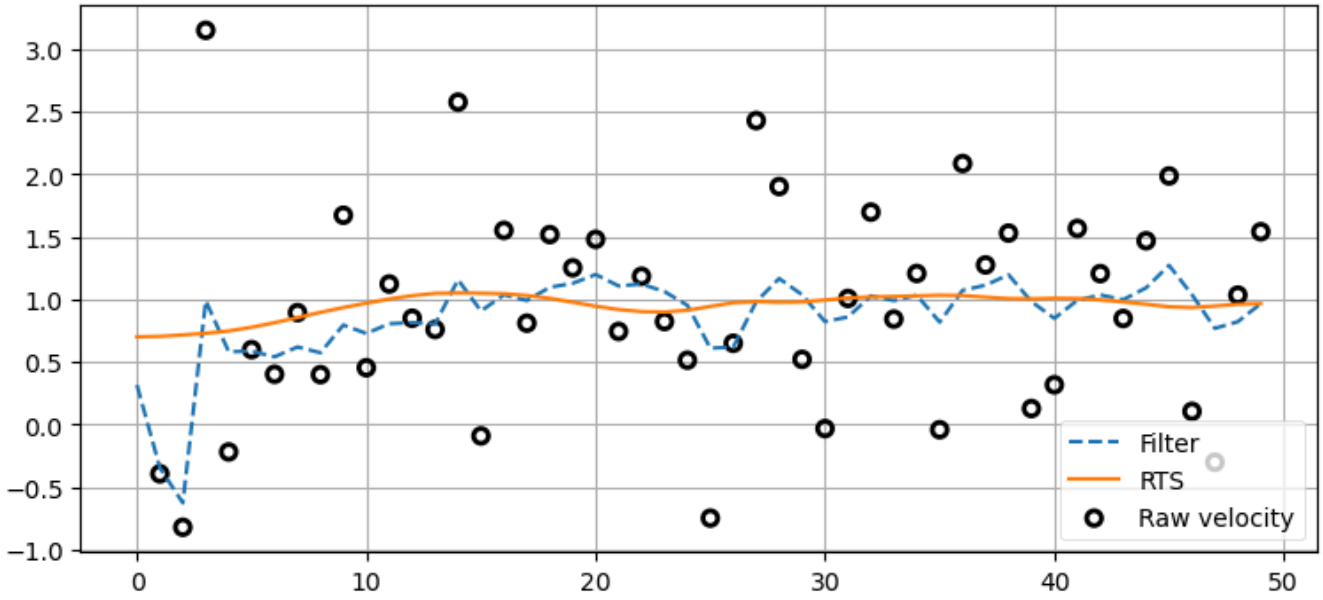
练习：比较速度大小

既然我们正在绘制速度数据，那我们就来了解一下所谓的“原始速度”究竟是什么。原始速度可以通过连续几次测量结果相减来计算得出。也就是说，第 1 时刻的速度可以近似为 `xs[1] - xs[0]`。请将原始速度值与卡尔曼滤波器和 RTS 滤波器所估算出的速度值进行比较。然后讨论一下你所观察到的结果。

```
# your code here
```

解决方案

```
dx = np.diff(Xs[:, 0], axis=0)
plt.scatter(range(1, len(dx) + 1), dx, facecolor='none',
            edgecolor='k', lw=2, label='Raw velocity')
plt.plot(Xs[:, 1], ls='--', label='Filter')
plt.plot(Ms[:, 1], label='RTS')
plt.legend(loc=4);
```



我们看到，噪声严重干扰了信号，导致原始数据几乎毫无用处。滤波器则单独对速度进行估算。卡尔曼增益是一个多维数值。例如，它可能具有某个具体的数值。第一个数值用于调整位置误差的幅度，第二个数值则用于调整速度误差的幅度。协方差矩阵则告诉滤波器，位置和速度之间的相关性如何，从而让滤波器能够更有效地处理这些数据。

我展示这一点，是为了强调使用卡尔曼滤波器来计算速度、加速度以及更高阶的物理量是多么重要。即便我的测量结果已经非常精确，我仍然会使用卡尔曼滤波器——因为它能让我得到更准确的数值估计。

讨论与总结

多元高斯模型使我们能够同时处理多个维度的数据，包括空间维度以及速度等其他维度。我们有一个重要的发现：隐藏变量有助于显著提高滤波器的准确性。之所以如此，是因为隐藏变量与那些可观测的变量之间存在相关性。

我对“可观测性”给出了一个直观的定义。可观测性这一概念是由卡尔曼博士为线性系统所提出的，其背后有着相当复杂的理论体系。可观测性用来判断：通过观察系统的输出，是否能够确定系统的状态。对于我们处理的问题来说，判断可观测性相对简单；但对于更复杂的系统，则需要复杂的分析才能得出结论。维基百科上的“可观测性”条目提供了相关概述；如果你想深入了解这个主题，[2]是一个不错的参考资料。

关于隐藏变量，有一个重要的注意事项。虽然很容易构建出能够估算隐藏变量的算法，但我可以编写一个算法来估算某辆汽车的颜色。不过，仅凭汽车的位置信息是无法确定其颜色的，因此这样的估算结果毫无意义。设计者必须确保这些变量的估算结果是正确的。如果你没有速度传感器，却仍然试图估算速度，那么你就需要验证这些估算结果是否准确——千万不要盲目相信这些结果。例如，假设速度具有周期性特征，即呈正弦波形态。如果你的采样时间小于频率的两倍，那么根据奈奎斯特定理，你将无法准确估算出速度。再假设采样周期恰好等于速度的频率，情况也是如此。该滤波器会显示速度是恒定的，因为它是在正弦波的同一位置对系统进行采样的。

对于隐藏变量来说，初始化过程是个特别棘手的问题。如果初始化方式不当，虽然该滤波器通常能够恢复出可观测的变量，但在处理隐藏变量时则可能会遇到困难，甚至无法完成处理。估计隐藏变量虽然是一种强大的工具，但也可能带来风险。

我制定了一系列用于设计卡尔曼滤波器的步骤。这些步骤并非卡尔曼滤波器相关文献中常见的内容，仅供参考之用，而非固定不变的准则。针对复杂问题的设计过程是一个迭代的过程：先对状态向量进行初步估计，确定测量模型和状态模型，进行相关测试，然后再根据需要调整设计方案。

R 和 Q 的设计往往相当具有挑战性。我试图让它们的设计看起来很“科学”。你的传感器存在 $N(0, \sigma^2)$ 的高斯噪声，因此请将 $R = \sigma^2$ 设置为合适的数值。其实，这种说法完全是谎言。传感器并不遵循高斯分布。我们在本书的开头以浴室秤作为例子进行说明。假设 $\sigma = 1$ 公斤，如果你试图称量重量为 0.5 公斤的物体，按照理论，测量结果应该是负数。但实际上，秤永远不可能显示小于零的数值。在现实世界中，传感器的数值分布通常具有“厚尾现象”（即峰度较高）和偏态特性。在某些情况下，比如浴室秤，数值分布的两端会被截断。

Q 的情况则更为糟糕。希望当我轻率地用“噪声矩阵”来描述对狗的行踪的预测时，你们能保持怀疑态度。谁又能知道狗接下来会做什么呢？我 GPS 中的卡尔曼滤波器根本不知道有山丘、外部风速，也不知道我糟糕的驾驶技术。然而，该滤波器却需要一个精确的数值来反映所有这些信息。无论是在沙漠中的越野驾驶，还是在一级方程式赛车赛道上，该滤波器都必须能够正常工作。

这些问题使得一些研究人员和工程师们贬称卡尔曼滤波器为“一团糟”。换句话说，卡尔曼滤波器的性能并不总是那么可靠。另一个需要了解的是：卡尔曼滤波器容易过于自信。其估算结果完全取决于用户所输入的噪声数据。这些数据可能导致过于乐观的估算结果。随着 P 的值越来越小，滤波器的准确性反而会越来越差！在最糟糕的情况下，滤波器甚至会失去其功能。在研究非线性滤波器时，我们会经常遇到这种情况。

卡尔曼滤波器是一种描述世界的数学模型。其输出结果的准确性完全取决于该模型的精确度。为了使数学模型易于处理，我们必须做出一些假设。我们假设传感器和运动模型产生的噪声都符合高斯分布。同时，我们还假设所有过程都是线性的。如果这些假设成立，那么卡尔曼滤波器就达到了最小二乘意义上的最优解。这意味着，不可能有比卡尔曼滤波器更准确的估计结果了。不过，这些假设几乎从来都不成立，因此该模型的有效性必然有限，实际应用中的卡尔曼滤波器也很难达到最优效果。

在后面的章节中，我们将探讨非线性问题。目前，你需要明白：线性滤波器的矩阵设计更像是一种实验性操作，而非纯粹的数学计算。你可以用数学方法来确定初始值，但之后还需要通过实验来调整参数。如果环境中存在大量难以控制的噪声（比如风声等），你可能需要增大某个参数的值。不过，如果该参数值过大，滤波器将无法快速响应各种变化。在“自适应滤波器”这一章中，你将学习一些替代性技术，这些技术允许你根据输入信号和系统性能实时调整滤波器参数。不过目前，你需要先找到一组适合滤波器所处环境条件的参数值。对于执行特技飞行的飞机来说，如果飞行员是新手，那么所需的噪声处理参数可能与经验丰富的飞行员不同，因为飞机的动态特性有很大差异。

参考文献

- [1] “卡尔曼滤波器”。维基百科：https://en.wikipedia.org/wiki/Kalman_filter#Details

- [2] Grewal, Mohinder S., Andrews, Angus P. 《卡尔曼滤波：基于 MATLAB 的理论与实践》。第三版。约翰·威利父子出版社。2008 年。

第 7 章：卡尔曼滤波的数学基础

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

如果你已经读到了这里，那么我希望你能够意识到：卡尔曼滤波器的“可怕名声”其实有些名不实。当然，我确实对一些公式进行了简化处理，但希望你的实现过程相当简单。其基本原理其实很简单——只需获取两次测量数据，或者一次测量数据加上一次预测值，然后选择输出值介于这两个数值之间的某个值。如果你更相信测量数据，那么你的预测值就会更接近实际值；而如果你认为预测值更准确，那么你的预测值就会更接近该预测值。这根本不是什么复杂的数学问题嘛（开玩笑的——正是这种数学方法让阿波罗飞船成功登月并返回）。

说实话，我一直在仔细挑选自己要解决的问题。对于某些复杂的难题来说，设计卡尔曼滤波器所需的矩阵会极其困难。不过，我并没有选择太复杂的题目。像牛顿运动定律这样的方程，在卡尔曼滤波器的应用中很容易计算出来。而这些方程，正是我们想要解决的大部分问题的核心内容。

我用代码和逻辑来阐释这些概念，而非数学公式。不过，也有一些主题需要用到比我所使用的更多的数学知识。本章将介绍你在阅读本书其余内容时所需要的数学知识。

动态系统的建模

动态系统是一种其状态（位置、温度等）会随时间而变化的物理系统。微积分正是研究数值变化的数学工具，因此我们用微分方程来描述动态系统。不过，有些系统无法用微分方程来描述，但在本书中我们不会遇到这类系统。

动态系统的建模其实是许多大学课程的内容。在某种程度上，没有任何东西可以替代几学期关于常微分方程和偏微分方程的学习，以及之后在控制系统理论方面的深入学习。如果你只是个业余爱好者，或者只是想解决工作中某个特定的滤波问题，那你很可能没有时间和精力去投入一年或更长时间来学习这些知识。

幸运的是，我能够阐述足够的理论内容，从而帮助我们为各种不同的卡尔曼滤波器建立相应的系统方程。我的目标是让你们能够阅读相关文献，并充分理解其内容，进而能够实际应用这些算法。虽然相关的数学原理相当复杂，但实际上我们只需要运用一些简单的技巧即可。

这是本书中涉及纯数学内容最长的部分。如果你想理解最常用的非线性滤波器——扩展卡尔曼滤波器，就必须掌握这一部分的所有内容。当然，我也介绍了些更现代的滤波器，这些滤波器不需要那么复杂的数学知识。你现在可以先跳过这一部分，如果以后决定要学习扩展卡尔曼滤波器，再回来阅读这部分内容。

首先，我们需要了解卡尔曼滤波所依据的数学方程和各种假设。既然我们试图用数学模型来描述现实世界中的现象，那么究竟有哪些因素需要考虑呢？

每一个物理系统都遵循一定的规律。例如，以某种速度行驶的汽车，在固定的时间内会行驶一定的距离；而汽车的速度则取决于其加速度。我们用高中时学过的牛顿定律来描述这种行为。

$$\begin{aligned} v &= at \\ x &= \frac{1}{2}at^2 + v_0 t + x_0 \end{aligned}$$

一旦我们学会了微积分，就会以这种形式来理解它们：

$$v = \frac{dx}{dt}, \quad a = \frac{dv}{dt} = \frac{d^2x}{dt^2}$$

在典型的汽车跟踪问题中，我们需要根据恒定的速度或加速度来计算汽车行驶的距离，就像我们在前面几章中所做的那样。不过，实际上情况并非如此简单。没有哪辆汽车能在完全平坦的道路上行驶。道路上会有各种凹凸不平之处，还有风阻以及起伏不平的路面，这些都会影响汽车的速度。此外，汽车的悬挂系统也属于机械装置，其中存在摩擦力，而且弹簧也并非完美无缺。

除了那些极其简单的问题之外，要完美地模拟一个系统是不可能的。我们不得不进行简化处理。在任何时候，我们可以认为，系统的真实状态（比如汽车的位置）其实是由不完美的模型所预测出的数值，再加上一些无法确定的随机噪声而已。

$$x(t) = x_{pred}(t) + noise(t)$$

这并不意味着 $noise(t)$ 是一个可以通过分析方法推导出来的函数。这仅仅是一个事实陈述而已：我们总可以把真实值表示为预测值加上各种过程噪声。所谓“噪声”并不一定指随机事件。如果我们正在追踪在大气中飞行的球，而模型却假设球处于真空中，那么空气阻力的影响就属于这种“过程噪声”。

在下一节中，我们将学习如何将一组高阶微分方程转化为一组一阶微分方程。转换完成后，无噪声系统的相关模型如下：

$$\dot{x} = Ax$$

A 被称为系统动态矩阵，因为它能够描述系统的动态变化过程。现在，我们需要对噪声进行建模。我们将把噪声表示为 w，并将其加入方程式中。

$$\dot{x} = Ax + w$$

w 这个名字看起来可能不太合适，但你会很快明白：卡尔曼滤波器其实是以白噪声作为假设条件的。

最后，我们需要考虑输入系统中的各种因素。我们假设存在某个输入值 u，并且存在一个线性模型来描述该输入值如何影响系统。例如，踩下汽车的油门会导致汽车加速；重力则使物体向下坠落。这些都是属于控制输入的例子。我们需要用矩阵 B 来将输入值 u 转化为对系统的影响。我们将这个矩阵加入我们的方程式中。

$$\dot{x} = Ax + Bu + w$$

就是这样。这就是卡尔曼博士试图解决的方程之一。如果我们假设 w 具有某些特定的性质，那么他就能找到一个最优的估算方法。

动态系统的状态空间表示

我们已经推导出了这个方程。

$$\dot{x} = Ax + Bu + w$$

不过，我们并不关心 x 的导数，而只关心 x 本身。暂且忽略那些干扰因素，我们希望找到一个方程，能够根据在时间 t_{k-1} 时的 x 值，递归地计算出在时间 t_k 时的 x 值。

$$x(t_k) = F(\Delta t)x(t_{k-1}) + B(t_k)u(t_k)$$

该约定允许我们将 $x(t_k)$ 表示为 x_k ，也就是说， x 的值相当于 t 在 k^{th} 时的值。

$$x_k = Fx_{k-1} + B_k u_k$$

F 是一种常见的状态转换矩阵。它之所以被如此命名，是因为它能够使状态在离散的时间步长之间进行转换。它与系统动态矩阵 A 非常相似。两者的区别在于： A 描述的是一组线性微分方程，属于连续型模型；而 F 则是离散型模型，它描述的是一组线性方程（而非微分方程），这些方程使得状态在离散的时间步长 Δt 内从 x_{k-1} 状态转换为 x_k 状态。

找到这个矩阵通常相当困难。方程 $\dot{x} = v$ 是最简单的微分方程，我们很容易就能将其积分出来：

$$\int_{x_{k-1}}^{x_k} \mathrm{d}x = \int_0^{\Delta t} v\mathrm{d}t$$

$$x_k - x_{k-1} = v\Delta t$$

$$x_k = v\Delta t + x_{k-1}$$

这个方程具有递归性质：我们根据 $k-1$ 时刻 x 的值来计算 k 时刻 x 的值。这种递归结构使得我们能够以卡尔曼滤波器所要求的形式来表示该系统（即过程模型）。

$$\begin{aligned} x_k &= Fx_{k-1} \\ &= \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_{k-1} \\ \dot{x}_{k-1} \end{bmatrix} \end{aligned}$$

我们能够做到这一点，是因为 $\dot{x} = v$ 是最简单的微分方程。在物理系统中，几乎所有其他微分方程都更为复杂，因此无法用这种方法来求解。

状态空间方法在阿波罗计划期间开始流行起来，这主要要归功于卡尔曼博士的贡献。其基本原理很简单：用一组 n 阶微分方程来描述系统。将这些方程转化为等价的一阶微分方程组。然后将其转化为前一节中提到的向量-矩阵形式。一旦转换成这种形式后，就可以利用各种技术将这些线性微分方程转化为递推方程。

$$x_k = Fx_{k-1} + B_k u_k$$

有些书籍将状态转移矩阵称为“基本矩阵”。许多人使用 Φ 来表示状态转移矩阵，而不是 F 。那些以控制理论为基础的资料，通常也使用这两种表示方式。

这些方法被称为“状态空间法”，因为我们是用系统状态来表示微分方程的解的。

从高阶方程推导出一阶方程

许多物理系统的模型都需要包含控制输入项的二阶或更高阶微分方程。

$$a_n\frac{d^n y}{dt^n} + a_{n-1}\frac{d^{n-1}y}{dt^{n-1}} + \cdots + a_2\frac{d^2y}{dt^2} + a_1\frac{dy}{dt} + a_0 = u$$

状态空间方法需要一阶方程。任何更高阶的方程组都可以通过为导数引入额外的变量，进而将其转化为一阶方程来处理。

我们来看一个例子。给定系统 $\ddot{x} - 6\dot{x} + 9x = u$ ，请找出其等效的一阶方程。为便于理解，我使用了点号来表示时间导数。

第一步是将方程中最高次项单独放在等式的一边。

$$\ddot{x} = 6\dot{x} - 9x + u$$

我们定义了两个新的变量：

$$\begin{aligned} x_1(t) &= x \\ x_2(t) &= \dot{x} \end{aligned}$$

现在，我们将这些值代入原方程中并求解。求解后会得到一组以这些新变量为一元的一阶方程。为了方便表示，通常会省略 (t) 符号。

我们知道 $\dot{x}_1 = x_2$ 的情况，也知道 $\dot{x}_2 = \ddot{x}$ 的情况。因此

$$\begin{aligned} \dot{x}_2 &= \ddot{x} \\ &= 6\dot{x} - 9x + u \\ &= 6x_2 - 9x_1 + u \end{aligned}$$

因此，我们的一阶方程组为：

$$\begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= 6x_2 - 9x_1 + u \end{aligned}$$

只要稍加练习，你就能熟练掌握这种方法。首先找出最高次项，定义一个新的变量及其导数，然后再进行代入计算。

状态空间形式下的一阶微分方程

将上一节中新定义的变量代入：

$$\frac{dx_1}{dt} = x_2, \frac{dx_2}{dt} = x_3, \ldots, \frac{dx_{n-1}}{dt} = x_n$$

代入一阶方程后得到：

$$\frac{dx_n}{dt} = \frac{1}{a_n}\sum_{i=0}^{n-1} a_i x_{i+1} + \frac{1}{a_n}u$$

使用向量-矩阵表示法，我们有：

$$\begin{bmatrix} \frac{dx_1}{dt} \\ \frac{dx_2}{dt} \\ \vdots \\ \frac{dx_n}{dt} \end{bmatrix} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_n \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -\frac{a_0}{a_n} & -\frac{a_1}{a_n} & -\frac{a_2}{a_n} & \cdots & -\frac{a_{n-1}}{a_n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ \frac{1}{a_n} \end{bmatrix} u$$

我们将其记作 $\dot{x} = Ax + Bu$ 。

求取时不变系统的基本矩阵

我们将系统方程表示为状态空间形式。

$$\dot{x} = Ax$$

其中， A 表示系统动态矩阵。我们需要找到那个能够使状态 x 在区间 Δt 内按照特定方程进行传播的基矩阵 F 。

$$x(t_k) = F(\Delta t)x(t_{k-1})$$

换句话说， A 是一组连续的微分方程；而 F 则是一组离散的线性方程，其作用是计算在离散时间步长内 A 的变化量。

按照惯例，通常会省略 t_k 和 (Δt) ，直接使用该符号来表示。

$$x_k = Fx_{k-1}$$

一般来说，有三种常见的方法可以用来确定卡尔曼滤波器中的该矩阵。最常用的方法是矩阵指数法。另一种方法是线性时不变系统理论。此外，还有数值计算方法。你可能还知道其他方法，但在卡尔曼滤波器的相关文献和实际应用中，你最常遇到的就是这三种方法。

矩阵指数

方程 $\frac{dx}{dt} = kx$ 的解可以通过以下方法求得：

$$\begin{aligned} \frac{dx}{dt} &= kx \\ \frac{dx}{x} &= kdt \\ \int \frac{1}{x} dx &= \int kdt \\ \log x &= kt + c \\ x &= e^{kt+c} \\ x &= e^c e^{kt} \\ x &= c_0 e^{kt} \end{aligned}$$

当 $t = 0$ 时， $x = x_0$ 。将这些值代入上述方程中。

$$\begin{aligned} x_0 &= c_0 e^{k(0)} \\ x_0 &= c_0 1 \\ x_0 &= c_0 \\ x &= x_0 e^{kt} \end{aligned}$$

运用类似的数学方法，一阶方程的解也可以得到。

$$\dot{x} = Ax, x(0) = x_0$$

其中， A 是一个常数矩阵。

$$x = e^{At}x_0$$

将 $F = e^{At}$ 代入后，我们可以这样表示：

$$x_k = Fx_{k-1}$$

这就是我们正在寻找的形式！我们把求出该矩阵的问题，简化为求出 e^{At} 的值的问题。

e^{At} 被称为矩阵指数。它可以通过以下幂级数来计算：

$$e^{At} = I + At + \frac{(At)^2}{2!} + \frac{(At)^3}{3!} + \dots$$

该级数是通过对方程 e^{At} 进行泰勒级数展开得到的，具体过程这里就不赘述了。

让我们用这个方法来求解牛顿方程。用 v 来代替 $\dot{x}$ ，并假设速度为常数，那么就可以得到线性矩阵-向量形式。

$$\begin{bmatrix} \dot{x} \\ \dot{v} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ v \end{bmatrix}$$

这是一个一阶微分方程，因此我们可以设定 $A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ 的值，然后解这个方程。我将 t 替换为 Δt ，以此来强调该矩阵是离散的。

$$F = e^{A\Delta t} = I + A\Delta t + \frac{(A\Delta t)^2}{2!} + \frac{(A\Delta t)^3}{3!} + \dots$$

如果你进行乘法运算，就会发现 $A^2 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$ 。这意味着 A 的所有高次幂也都等于 0 。这样，我们就能得到一个确切的答案，而无需处理无穷多个项。

$$\begin{aligned} F &= I + A\Delta t + 0 \\ &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \Delta t \\ &= \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \end{aligned}$$

我们将其代入 $x_k = Fx_{k-1}$ 中，就能得到结果。

$$x_k = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} x_{k-1}$$

你会认出，这就是我们在“多元卡尔曼滤波”一章中，通过分析方法推导出的、用于恒速卡尔曼滤波的矩阵。

SciPy 的 linalg 模块中包含了一个用于计算矩阵指数的函数。该函数没有使用泰勒级数法，而是采用了帕德近似法来计算矩阵指数。实际上，有多种方法可以用来计算矩阵指数，至少有 19 种方法。不过，所有这些方法都存在一定的数值计算难题[1]。如果你需要处理较大的矩阵指数，就必须注意这些问题。如果你搜索“帕德近似法与矩阵指数”的相关内容，会找到很多关于这个主题的学术论文。

实际上，你可能不必太担心这个问题。因为对于卡尔曼滤波器来说，我们通常只需要使用泰勒级数的前两项即可。不过，不要以为我对这个问题的处理已经完整了，就直接把这种技术应用到其他问题上，而不先对这种技术的性能进行数值分析。有趣的是，解决 e^{At} 问题的常用方法之一就是使用广义常微分方程求解器。换句话说，他们的做法与我们相反：他们把 A 转化为一组微分方程，然后再用数值方法来求解这些方程！

以下是使用 `expm()` 来解决 e^{At} 的示例。

```
import numpy as np
from scipy.linalg import expm

dt = 0.1
A = np.array([[0, 1],
              [0, 0]])
expm(A*dt)
```

```
array([[1. , 0.1],
       [0. , 1. ]])
```

时间不变性

如果系统的行为随时间而变化，那么可以说，该动态系统可以用一阶微分方程来描述。

$$g(t) = \dot{x}$$

不过，如果该系统是时不变的，那么该方程的形式如下：

$$f(x) = \dot{x}$$

“时间不变”是什么意思呢？以家用音响为例。如果你在时间点 t 向音响输入信号 x，那么音响会输出信号 $f(x)$。而如果你在时间点 $t + \Delta t$ 进行输入操作，输出信号仍然会是 $f(x)$，只不过时间上有所延迟而已。

一个反例就是 $x(t) = \sin(t)$ 系统在 $f(x) = tx(t) = t\sin(t)$ 环境下的情况。该系统不具备时间不变性：由于存在与 t 的乘法运算，不同时间点的系统状态是不同的。飞机同样不具备时间不变性。如果你在稍后的时间对飞机进行控制操作，其行为将会有所不同，因为飞机在此期间会消耗燃料，从而导致重量减轻。重量减轻自然会影响到飞机的行为表现。

我们可以通过对等式的每一侧进行积分来求解这些方程。上面我已经演示了如何对时间不变系统 $v = \dot{x}$ 进行积分。不过，对时间不变方程 $\dot{x} = f(x)$ 进行积分则没那么简单。我们可以利用变量分离法，将等式两边同时除以 $f(x)$，然后把 dt 项移到等式的右边，这样就可以对等式的每一侧分别进行积分了。

$$\frac{dx}{dt} = f(x)$$

$$\int_{x_0}^{x} \frac{1}{f(x)} dx = \int_{t_0}^{t} dt$$

如果我们让 $F(x) = \int \frac{1}{f(x)} dx$ 这样做的话，就会得到……

$$F(x) - F(x_0) = t - t_0$$

然后我们用该公式来求解 x 的值。

$$F(x) = t - t_0 + F(x_0)$$
$$x = F^{-1}[t - t_0 + F(x_0)]$$

换句话说，我们需要找到 F 的逆函数。这并非易事，因此在 STEM 教育中，有相当多的教学内容都是用来探讨如何解决这个棘手的分析问题的。

不过，这些其实都是些技巧而已。对于许多简单的 $f(x)$ 问题来说，要么没有明确的解析解，要么解决起来极其困难。因此，实际工作中的工程师们通常会采用状态空间法来寻求近似解。

矩阵指数的优点在于：我们可以将其用于任何满足时间不变性的微分方程组。不过，即便方程组不满足时间不变性，我们也常常会使用这种技术。以飞机为例，飞机在飞行过程中会消耗燃料，从而导致重量减轻。不过，一秒钟内的重量变化可以忽略不计，因此在这一时间尺度上，该系统可视为线性系统。只要时间步长足够短，我们的计算结果仍然会相当准确。

示例：质量-弹簧-阻尼器模型

假设我们想要追踪一个连接在弹簧和减震器上的物体的运动情况，比如汽车悬架中的部件。在某种输入信号的作用下，该物体的运动方程为：其中， m 表示质量， k 表示弹簧常数， c 表示减震力。

$$m\frac{d^2x}{dt^2} + c\frac{dx}{dt} + kx = u$$

为了方便表示，我会将其写成这样。

$$m\ddot{x} + c\dot{x} + kx = u$$

我可以通过令 $x_1(t) = x(t)$ 为某个值，然后再进行相应的代入，从而将此问题转化为一个一阶方程组。

$$\begin{aligned} x_1 &= x \\ x_2 &= \dot{x}_1 \\ \dot{x}_2 &= \ddot{x}_1 = \ddot{x} \end{aligned}$$

和往常一样，为了方便表示，我省去了 (t) 符号。这样就能得到这个方程式了。

$$m\dot{x}_2 + cx_2 + kx_1 = u$$

求解 $\dot{x}_2$ 后，我们得到一个一阶方程：

$$\dot{x}_2 = -\frac{c}{m}x_2 - \frac{k}{m}x_1 + \frac{1}{m}u$$

我们将其转化为矩阵形式：

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k\,/\,m & -c\,/\,m \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1\,/\,m \end{bmatrix} u$$

现在，我们利用矩阵指数来求解状态转移矩阵：

$$\Phi(t) = e^{At} = I + At + \frac{(At)^2}{2!} + \frac{(At)^3}{3!} + \ldots$$

前两项的结果是：

$$F = \begin{bmatrix} 1 & t \\ -(k/m)t & 1-(c/m)t \end{bmatrix}$$

这样做可能无法确保足够的精确度。你可以通过计算各个常量的 $\frac{(At)^2}{2!}$ 值来验证这一点，从而了解该矩阵对最终结果的贡献程度。

线性时不变理论

线性时不变理论，也被称为 LTI 系统理论，提供了一种利用拉普拉斯逆变换来求解 Φ 的方法。现在，你可能已经明白了，或者仍然完全搞不懂。在这本书中，我不会使用拉普拉斯变换。LTI 系统理论告诉我们：

$$\Phi(t) = L^{-1}[(sI - A)^{-1}]$$

我无意深入探讨这个话题，只想说：拉普拉斯变换将信号转换到一个与时间无关的域中。不过，要解出上述方程并非易事。如果您感兴趣，维基百科上关于线性时不变系统理论的条目可以提供一些入门信息。我提到线性时不变系统，是因为在某些文献中，人们会利用这一概念来设计卡尔曼滤波器矩阵，以解决各种复杂问题。

数值解

最后，还有一些数值方法可以用来求解 F。随着滤波器尺寸的增大，寻求解析解变得极为繁琐（不过像 SymPy 这样的工具可以简化这一过程）。C. F. van Loan[2]提出了一种数值方法，能够同时求解 Φ 和 Q。当然，这一切都是基于连续模型来进行的。

$$\dot{x} = Ax + Gw$$

其中，w 表示单位白噪声。根据 van Loan 的方法，可以同时计算出 F_k 和 Q_k 的值。

我已经在 `FilterPy` 中实现了 van Loan 的方法。你可以按照以下方式来使用它：

```
from filterpy.common import van_loan_discretization

A = np.array([[0., 1.], [-1., 0.]])
G = np.array([[0.], [2.]]) # white noise scaling
F, Q = van_loan_discretization(A, G, dt=0.1)
```

在“微分方程的数值积分”这一部分中，我介绍了几种在卡尔曼滤波中经常被使用的替代方法。

过程噪声矩阵的设计

一般来说，Q 矩阵的设计是卡尔曼滤波器设计中最困难的环节之一。造成这一困难的原因是多方面的。首先，相关的数学计算需要扎实的信号理论基础。其次，我们试图对那些我们知之甚少的现象进行建模。以投掷的棒球为例，我们可以将其视为在空气中运动的球体，但这一模型仍然存在许多未知因素：球的旋转和自旋衰减、有缝球的空气阻力系数、风和空气密度的影响等等。虽然我们可以为特定的过程模型建立精确的数学方程，但由于过程模型本身并不完整，Q 的计算结果也必然不完整。这对卡尔曼滤波器的性能有着重要影响：如果 Q

的值太小，滤波器会过度依赖其预测模型，从而导致预测结果与实际值相差甚远。而如果 Q 的值……如果滤波器的尺寸过大，那么测量结果就会受到噪声的过度影响，从而导致性能不佳。在实际操作中，我们需要花费大量时间来进行模拟计算和数据分析，以确定合适的 Q 值。不过，我们还是先从数学角度来分析这个问题吧。

假设我们有一个符合运动学规律的系统——也就是那种可以用牛顿运动定律来描述的系统。对于这个系统，我们可以做出几种不同的假设。

我们一直使用的是这样的流程模型。

$$\dot{x} = Ax + Bu + w$$

其中， w 代表过程噪声。运动系统是连续的——其输入和输出可以在任意时间点发生变化。不过，我们所使用的卡尔曼滤波器是离散的（虽然卡尔曼滤波器也有连续形式，但本书不涉及这些内容）。我们需要以固定的时间间隔对系统进行采样。因此，我们必须为上述方程中的噪声项找到相应的离散表示形式。这取决于我们对噪声行为的假设。我们将考虑两种不同的噪声模型。

连续白噪声模型

我们利用牛顿定律来模拟运动系统。在建立模型时，我们可以使用位置和速度作为参数，或者同时使用位置、速度和加速度。其实，我们完全可以进一步深入：还可以模拟系统的瞬时加速度、冲击力等因素。不过，通常我们不会这么做，因为引入超出系统实际运动规律的额外参数，只会降低模型的准确性。

假设我们需要对物体的位置、速度和加速度进行建模。我们可以假设，在每个离散的时间步长内，加速度都是恒定的。当然，系统中存在各种随机干扰，因此实际上加速度并非恒定不变。由于外部不可预测的力的作用，被追踪物体的加速度会随时间而发生变化。在本节中，我们假设加速度的变化是由连续的、均值为零的白噪声所引起的。换句话说，我们认为，速度的微小变化在时间上会相互抵消，其平均值仍为零。

由于噪声是持续变化的，因此我们需要对其进行积分处理，从而得到所选离散时间间隔内的离散噪声值。这里就不详细证明这个过程了，不过，噪声离散化的公式如下：

$$Q = \int_0^{\Delta t} F(t) Q_c F^T(t) dt$$

其中， Q_c 表示连续噪声。其原理其实很直观。 $F(t)Q_cF^T(t)$ 实际上是根据我们的过程模型 $F(t)$ 在时刻 t 时对连续噪声的投影。我们想要知道在某个离散的时间间隔 Δt 内，系统里增加了多少噪声。因此，我们需要在这个时间间隔 $[0, \Delta t]$ 内对这一表达式进行积分。

我们知道，牛顿系统的基本矩阵为

$$F = \begin{bmatrix} 1 & \Delta t & \Delta t^2 \,/\, 2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}$$

我们将连续噪声定义为：

$$Q_c = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \Phi_s$$

其中， Φ_s 表示白噪声的频谱密度。虽然这个数值可以通过计算得出，但超出了本书的讨论范围。详细内容可参考关于随机过程的标准教材。在实际应用中，我们往往不知道噪声的频谱密度，因此这实际上是一个“工程参数”——我们需要通过实验来调整这个数值，以确保滤波器的性能符合预期。可以看出， Φ_s 所乘的矩阵实际上负责将功率谱密度分配给各个加速度分量。这是合理的：我们假设系统具有恒定的加速度，当然，噪声会对此造成影响。

我们可以自己来进行这些计算，但我更愿意使用 SymPy 来解这些方程。

$$Q_c = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \Phi_s$$

```
import sympy
from sympy import (init_printing, Matrix, MatMul,
                   integrate, symbols)

init_printing(use_latex='mathjax')
dt, phi = symbols('\Delta{t} \Phi_s')
F_k = Matrix([[1, dt, dt**2/2],
              [0,  1,      dt],
              [0,  0,       1]])
Q_c = Matrix([[0, 0, 0],
              [0, 0, 0],
              [0, 0, 1]])*phi

Q = integrate(F_k * Q_c * F_k.T, (dt, 0, dt))

# factor phi out of the matrix to make it more readable
Q = Q / phi
MatMul(Q, phi)
```

$$\begin{bmatrix} \frac{\Delta t^5}{20} & \frac{\Delta t^4}{8} & \frac{\Delta t^3}{6} \\ \frac{\Delta t^4}{8} & \frac{\Delta t^3}{3} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^3}{6} & \frac{\Delta t^2}{2} & \Delta t \end{bmatrix} \Phi_s$$

为了完整性起见，让我们来计算一下 0 阶和 1 阶方程的解。

```
F_k = Matrix([[1]])
Q_c = Matrix([[phi]])

print('0th order discrete process noise')
integrate(F_k*Q_c*F_k.T,(dt, 0, dt))
```

```
0th order discrete process noise
```

$$\begin{bmatrix} \Delta t \Phi_s \end{bmatrix}$$

```
F_k = Matrix([[1, dt],
              [0, 1]])
Q_c = Matrix([[0, 0],
              [0, 1]]) * phi

Q = integrate(F_k * Q_c * F_k.T, (dt, 0, dt))
```

```
print('1st order discrete process noise')
# factor phi out of the matrix to make it more readable
Q = Q / phi
MatMul(Q, phi)
```

```
1st order discrete process noise
```

$$\begin{bmatrix} \frac{\Delta t^3}{3} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^2}{2} & \Delta t \end{bmatrix} \Phi_s$$

分段白噪声模型

另一种关于噪声的模型认为：在每个时间周期内，最高阶的项（比如加速度）是恒定的，但不同时间周期内的该值各不相同，而且各个时间周期内的数值之间没有相关性。换句话说，每个时间点上，加速度都会发生突变。这与之前的模型有所不同，因为在之前的模型中，我们假设最后一个项受到的是连续变化的噪声信号的影响。

我们将把这个问题建模为……

$$f(x) = Fx + \Gamma w$$

其中， Γ 表示系统的噪声增益，而 w 则表示那种恒定的分段式加速度（或速度、或急动度等）。

我们先来研究一阶系统。在这种情况下，我们有状态转移函数。

$$F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$$

在某个时间段内，速度的变化量为 $w(t)\Delta t$ ，位置的变化量为 $w(t)\Delta t^2 \ / \ 2$ 。

$$\Gamma = \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \end{bmatrix}$$

该过程噪声的协方差则为

$$Q = E[\Gamma w(t) w(t) \Gamma^T] = \Gamma \sigma_v^2 \Gamma^T$$

.

我们可以用 SymPy 来计算，具体步骤如下：

```
var = symbols('sigma^2_v')
v = Matrix([[dt**2 / 2], [dt]])

Q = v * var * v.T

# factor variance out of the matrix to make it more readable
Q = Q / var
MatMul(Q, var)
```

$$\begin{bmatrix} \frac{\Delta t^4}{4} & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & \Delta t^2 \end{bmatrix} \sigma_v^2$$

二阶系统也遵循同样的数学运算规则。

$$F = \begin{bmatrix} 1 & \Delta t & \Delta t^2 \, / \, 2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}$$

在这里，我们假设白噪声是一个离散时间的维纳过程。这样一来，我们就可以得到……

$$\Gamma = \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \\ 1 \end{bmatrix}$$

这种模型并没有什么“真理”可言，它只是使用起来方便，而且能得到不错的结果而已。例如，我们可以假设噪声是加在加速度上的，不过这样一来，相应的方程就会变得更复杂。

该过程噪声的协方差则为

$$Q = E[\Gamma w(t) w(t) \Gamma^T] = \Gamma \sigma_v^2 \Gamma^T$$

.

我们可以用 SymPy 来计算，具体步骤如下：

```
var = symbols('sigma^2_v')
v = Matrix([[dt**2 / 2], [dt], [1]])

Q = v * var * v.T

# factor variance out of the matrix to make it more readable
Q = Q / var
MatMul(Q, var)
```

$$\begin{bmatrix} \frac{\Delta t^4}{4} & \frac{\Delta t^3}{2} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^3}{2} & \Delta t^2 & \Delta t \\ \frac{\Delta t^2}{2} & \Delta t & 1 \end{bmatrix} \sigma_v^2$$

我们无法断言这种模型比连续模型更准确或更不准确——两者都只是对实际现象的近似描述而已。只有通过实践和实验，才能确定哪种模型更合适。在实际应用中，通常会发现两种模型都能给出合理的结果，但一般来说，其中一种模型的表现会更好一些。

第二种模型的优点在于，我们可以用 σ^2 来描述噪声，而 σ^2 又可以与物体的运动情况以及我们预期的误差大小相关联。第一种模型则要求我们指定光谱密度，这种方式不太直观。不过，第一种模型在处理不同时间点的样本时更为方便，因为噪声是在整个时间范围内进行积分处理的。不过，这些并非固定不变的规则——你可以根据测试结果或对物理模型行为的了解，选择使用其中任何一种模型，或者自行设计一种模型。

一个不错的经验法则是：将 σ 的值设定在 $\frac{1}{2}\Delta a$ 到 Δa 之间。其中， Δa 表示加速度在各个采样周期之间的最大变化幅度。在实际操作中，我们会先选定一个数值，然后利用数据来进行模拟测试，最终选择那个效果最佳的

数值。

使用 FilterPy 来计算 Q 值

FilterPy 提供了多种算法来计算 Q 矩阵。 `Q_continuous_white_noise()` 函数可以根据给定的 Δt 值以及光谱密度来计算 Q 。

```
from filterpy.common import Q_continuous_white_noise
from filterpy.common import Q_discrete_white_noise

Q = Q_continuous_white_noise(dim=2, dt=1, spectral_density=1)
print(Q)
```

```
[[0.333 0.5  ]
 [0.5   1.   ]]
```

```
Q = Q_continuous_white_noise(dim=3, dt=1, spectral_density=1)
print(Q)
```

```
[[0.05  0.125 0.167]
 [0.125 0.333 0.5  ]
 [0.167 0.5   1.   ]]
```

函数 `Q_discrete_white_noise()` 在假设噪声具有分段模型的前提下，计算出 Q 的值。

```
Q = Q_discrete_white_noise(2, var=1.)
print(Q)
```

```
[[0.25 0.5 ]
 [0.5  1.  ]]
```

```
Q = Q_discrete_white_noise(3, var=1.)
print(Q)
```

```
[[0.25 0.5  0.5 ]
 [0.5  1.   1.  ]
 [0.5  1.   1.  ]]
```

Q 的简化处理

许多处理方法对 Q 采用了一种更为简单的处理方式：将其设置为零，仅在右下角的元素中保留一个噪声项。这种处理方式合理吗？那么，让我们来考虑一下，当 Δt 较小时， Q 的值会是多少。

```
import numpy as np

np.set_printoptions(precision=8)
Q = Q_continuous_white_noise(
    dim=3, dt=0.05, spectral_density=1)
```

```
print(Q)
np.set_printoptions(precision=3)
```

```
[[0.00000002 0.00000078 0.00002083]
 [0.00000078 0.00004167 0.00125   ]
 [0.00002083 0.00125    0.05      ]]
```

我们可以看到，大多数项的数值都非常小。请记住，唯一使用这个矩阵的方程就是……

$$P = FPF^T + Q$$

如果 Q 的值相对于 P 来说很小，那么它对 P 的计算几乎没有任何贡献。可以将 Q 设置为零矩阵，右下角那个元素除外。

$$Q = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \sigma^2 \end{bmatrix}$$

虽然这种做法并不完全准确，但通常可以作为一种有效的近似方法。如果你要将这种方法应用于重要的场景中，那么就需要进行大量的测试，以确保该滤波器在各种情况下都能正常发挥作用。

如果你这样做的话，“右下角项”指的是每个变量中变化最快的那个数值。如果当前状态为 $x = \begin{bmatrix} x & \dot{x} & \ddot{x} & y & \dot{y} & \ddot{y} \end{bmatrix}^T$，那么 Q 的值将为 6x6；而 $\ddot{x}$ 和 $\ddot{y}$ 中的各个元素都必须在 Q 中被设置为非零值。

后验协方差的稳定计算

我已经给出了用于计算后验协方差的公式。

$$P = (I - KH)\bar{P}$$

严格来说，这种计算方式是正确的。但在 `FilterPy` 中，我并不采用这种方式来计算，而是使用约瑟夫方程来进行计算。

$$P = (I - KH)\bar{P}(I - KH)^T + KRK^T$$

我经常收到邮件或 GitHub 上的反馈，声称这种实现方式存在错误。但实际上并非如此。我采用这种方式是有几个原因的。首先，$(I - KH)$ 运算可能会导致结果矩阵不对称，这是由于浮点数运算的误差所导致的。而协方差矩阵必须是对称的，因此如果矩阵不对称，卡尔曼滤波器就会失效，或者因为 `NumPy` 中的检查机制而引发异常。

保持对称性的传统方法是使用以下公式：

$$P = (P + P^T) / 2$$

这是安全的，因为矩阵中所有协方差都为 $\sigma_{ij} = \sigma_{ji}$。因此，如果由于浮点误差导致两个数值出现偏差，该操作会计算这两个数值之差的平均值，从而消除误差。

如果你仔细观察上述方程中与 Joseph 相关的部分，会发现两项中都存在着类似的 ABA^T 模式。因此，这两项都保持了对称性。不过，这个方程究竟从何而来呢？我为什么要使用它，而不是别的公式呢？

$$P = (I - KH)\bar{P}P = (P + P^T) / 2$$

让我们从最基本的原则出发来推导这个方程吧。其实并不难。理解这个推导过程有助于我们明白该方程的用途。更重要的是，如果滤波器因为数值不稳定而出现异常，我们也能通过这个推导过程来找出问题所在。这个推导过程源自 Brown[4]的研究。

首先，先来解释一下各种符号的含义。 x 代表系统的真实状态。 $\hat{x}$ 则代表系统的估计状态，也就是后验概率。而 $\bar{x}$ 则代表系统的先验概率的估计值。

鉴于此，我们可以将模型定义为：

$$x_{k+1} = F_k x_k + w_k z_k = H_k x_k + v_k$$

换句话说，系统的下一个状态 x_{k+1} ，其实是当前状态 k 经过某种过程 F_k 处理后，再叠加上一些噪声 w_k 所得到的结果。

请注意，这些只是定义而已。没有任何系统能够完全遵循数学模型，因此我们用噪声项 w_k 来模拟这一情况。同样，由于传感器存在的误差，没有任何测量结果是完美的，所以我们用 v_k 来模拟这一误差。

我省略下标 k 吧，因为在后续的推导过程中，我们只考虑步骤 k 处的数值，而完全不考虑步骤 $k+1$ 处的数值。

现在，我们将估计误差定义为真实状态与估计状态之间的差异。

$$e = x - \hat{x}$$

这同样只是一种定义而已；我们不知道该如何计算 e 的值，它只不过是真实状态与估计状态之间的差异罢了。

这使我们能够确定估计值的协方差。协方差被定义为 ee^T 的期望值：

$$\begin{aligned} P &= E[ee^T] \\ &= E[(x - \hat{x})(x - \hat{x})^T] \end{aligned}$$

接下来，我们将后验估计定义为

$$\hat{x} = \bar{x} + K(z - H\bar{x})$$

那看起来像是卡尔曼滤波器中的公式，这也合情合理。不过，和之前遇到的其他数学公式一样，这其实只是一个定义而已。具体来说，我们还没有给 K 下定义；你也不应将其视为卡尔曼增益，因为我们在这里解决的是各种问题，而不仅仅是线性卡尔曼滤波器相关的问题。在这里， K 只不过是介于 0 和 1 之间的某个未具体指定的数值罢了。

现在我们已经有了相关的定义，接下来就来进行一些代入和代数运算吧。

术语 $(x - \hat{x})$ 可以通过用上述定义来替换 $\hat{x}$ 来进行展开，从而得到结果。

$$(x - \hat{x}) = x - (\bar{x} + K(z - H\bar{x}))$$

现在我们用 $Hx + v$ 来替换 z ：

$$
\begin{aligned}
(x - \hat{x}) \quad &= x - (\bar{x} + K(z - H\bar{x})) \\
&= x - (\bar{x} + K(Hx + v - H\bar{x})) \\
&= (x - \bar{x}) - K(Hx + v - H\bar{x}) \\
&= (x - \bar{x}) - KH(x - \bar{x}) - Kv \\
&= (I - KH)(x - \bar{x}) - Kv
\end{aligned}
$$

现在，如果我们注意到 $(x - \bar{x})$ 的期望值就是先验协方差 $\bar{P}$，而 v 的期望值则是 $E[vv^T] = R$，那么我们就可以求出 P 的值了：

$$
\begin{aligned}
P &= E\left[[(I - KH)(x - \bar{x}) - Kv)][(I - KH)(x - \bar{x}) - Kv]^T\right] \\
&= (I - KH)\bar{P}(I - KH)^T + KRK^T
\end{aligned}
$$

这正是我们来到这里想要证明的。

请注意，这个公式适用于所有的 K 值，而不仅仅是卡尔曼滤波器计算出的最优值 K。这也是我使用这个公式的理由。在现实中，卡尔曼滤波器计算出的增益并非最优值：因为现实世界从来都不是完全线性的或符合高斯分布的；同时，计算过程中还会出现浮点数误差。相比而言，这个公式更不容易导致卡尔曼滤波器在现实条件下出现异常行为。

那么，$P = (I - KH)\bar{P}$ 是从哪里来的呢？让我们来完成这个简单的推导过程吧。记得，卡尔曼滤波器的最优增益是由以下公式给出的：

$$
K = \bar{P}H^T(H\bar{P}H^T + R)^{-1}
$$

现在，我们将这个结果代入我们刚刚推导出的方程中：

$$
\begin{aligned}
&= (I - KH)\bar{P}(I - KH)^T + KRK^T \\
&= \bar{P} - KH\bar{P} - \bar{P}H^TK^T + K(H\bar{P}H^T + R)K^T \\
&= \bar{P} - KH\bar{P} - \bar{P}H^TK^T + \bar{P}H^T(H\bar{P}H^T + R)^{-1}(H\bar{P}H^T + R)K^T \\
&= \bar{P} - KH\bar{P} - \bar{P}H^TK^T + \bar{P}H^TK^T \\
&= \bar{P} - KH\bar{P} \\
&= (I - KH)\bar{P}
\end{aligned}
$$

因此，当增益处于最优值时，$P = (I - KH)\bar{P}$ 在数学上是正确的；不过 $(I - KH)\bar{P}(I - KH)^T + KRK^T$ 同样也是正确的。正如我们之前所讨论的，即使增益不是最优值，$(I - KH)\bar{P}(I - KH)^T + KRK^T$ 依然正确，而且其数值稳定性也更高。因此，我在 FilterPy 中使用了这种计算方式。

很有可能，你的滤波器仍然会出现发散现象，尤其是当运行周期达到数百或数千次时。你需要仔细检查这些方程式。文献中还提供了其他多种计算方法，也许其中有一些更适用于你的问题。当然，如果你正在解决那些一旦出错就会导致设备损坏或人员伤亡的实际工程问题，那么你就需要参考工程领域的专业文献，而不仅仅是这本书的内容。如果你处理的只是些“简单”的问题，且出错不会造成严重后果，那么一旦发现发散现象，你可以将 P 的值重置为某个“合理”的数值，然后继续计算。例如，你可以将矩阵中的非对角线元素设为零，这样矩阵就只包含方差了；之后，你可以乘以一个略大于一的常数，以弥补因操作而造成的信息损失。发挥你的想象力，多做些测试吧。

卡尔曼增益方程的推导

如果你已经读了上一节的内容，那不妨也把这一节读一读。通过这一节的讲解，我们就已经推导出了卡尔曼滤波器的方程式。

请注意，这种推导方式并没有使用贝叶斯定理。据我所知，至少有四种不同的方法可以用来推导卡尔曼滤波器的方程式。这种推导方式在文献中很常见，其逻辑其实源自上一节的论述。该推导方法的来源依然是 Brown[4]。

在上一节中，我们使用了某个未具体指定的缩放因子 K 来推导协方差方程的约瑟夫形式。如果我们想要得到最优的滤波器，就必须利用微积分来最小化方程中的误差。你应该对这个概念很熟悉吧。如果你想找到某个函数 $f(x)$ 的最小值，只需对该函数求导数，然后令其等于零即可：$\frac{x}{dx}f(x) = 0$。

在我们的问题中，误差是通过协方差矩阵 P 来表示的。具体来说，对角线上的元素分别代表了状态向量中各个元素的误差（即方差）。因此，为了找到最优增益，我们需要计算对角线上各元素之和的导数。

布朗提到了两个与迹的导数相关的公式：

$$\frac{dtrace(AB)}{dA} = B^T$$

$$\frac{dtrace(ACA^T)}{dA} = 2AC$$

其中，AB 为正方形，C 则具有对称性。

我们将 Joseph 方程展开为：

$$P = \bar{P} - K\bar{H}\bar{P} - \bar{P}\bar{H}^TK^T + K(\bar{H}\bar{P}\bar{H}^T + R)K^T$$

现在，我们需要求出 P 的迹关于 K 的导数：$\frac{dtrace(P)}{dK}$。

第一项关于 K 的迹的导数是 0，因为在表达式中不包含 K。

第二项的迹的导数为 $(\bar{H}\bar{P})^T$。

我们可以通过观察 $\bar{P}\bar{H}^TK^T$ 实际上是 $K\bar{H}\bar{P}$ 的转置这一事实，来求出第三项的迹的导数。矩阵的迹等于其转置的迹，因此，第三项的迹的导数与第二项的导数相同。

最后，第四项的迹的导数为 $2K(\bar{H}\bar{P}\bar{H}^T + R)$。

这样我们就得到了最终的数值。

$$\frac{dtrace(P)}{dK} = -2(\bar{H}\bar{P})^T + 2K(\bar{H}\bar{P}\bar{H}^T + R)$$

我们将该值设为零，然后求解方程，从而找到能够最小化误差的 K 值。

$$-2(\bar{H}\bar{P})^T + 2K(\bar{H}\bar{P}\bar{H}^T + R) = 0K(\bar{H}\bar{P}\bar{H}^T + R) = (\bar{H}\bar{P})^TK(\bar{H}\bar{P}\bar{H}^T + R) = \bar{P}\bar{H}^TK = \bar{P}\bar{H}^T(\bar{H}\bar{P}\bar{H}^T + R)^{-1}$$

这个推导并不完全无懈可击，因为我省略了关于“为何最小化迹值就能最小化总误差”的解释。不过，我觉得对于本书而言，这样的推导已经足够了。如果需要更详细的解释，可以参考各种标准教材。

微分方程的数值积分

我们接触过多种用于求解线性微分方程的数值方法。这些方法包括状态空间法、拉普拉斯变换以及范洛恩法。

这些方法适用于线性常微分方程的求解，但不适用于非线性方程。例如，试着预测一辆正在急转弯的汽车的位置吧。汽车是通过转动前轮来改变方向的。这样一来，汽车在前进的同时，其车体也会围绕后轴旋转。因此，汽车的行驶路径会不断变化，而线性预测方法必然会得到错误的数值。如果系统中变化的大小相对于某个阈值来说足够小，那么线性预测方法或许还能给出不错的结果。不过，对于我们将在后续章节中研究的非线性卡尔曼滤波器来说，这种情况很少见。

基于这些原因，我们必须掌握如何对常微分方程进行数值积分。这其实是一个相当复杂的主题，需要用好几本书来详细阐述。不过，我在这里会介绍几种简单的技巧，这些技巧足以应对你遇到的大多数问题。

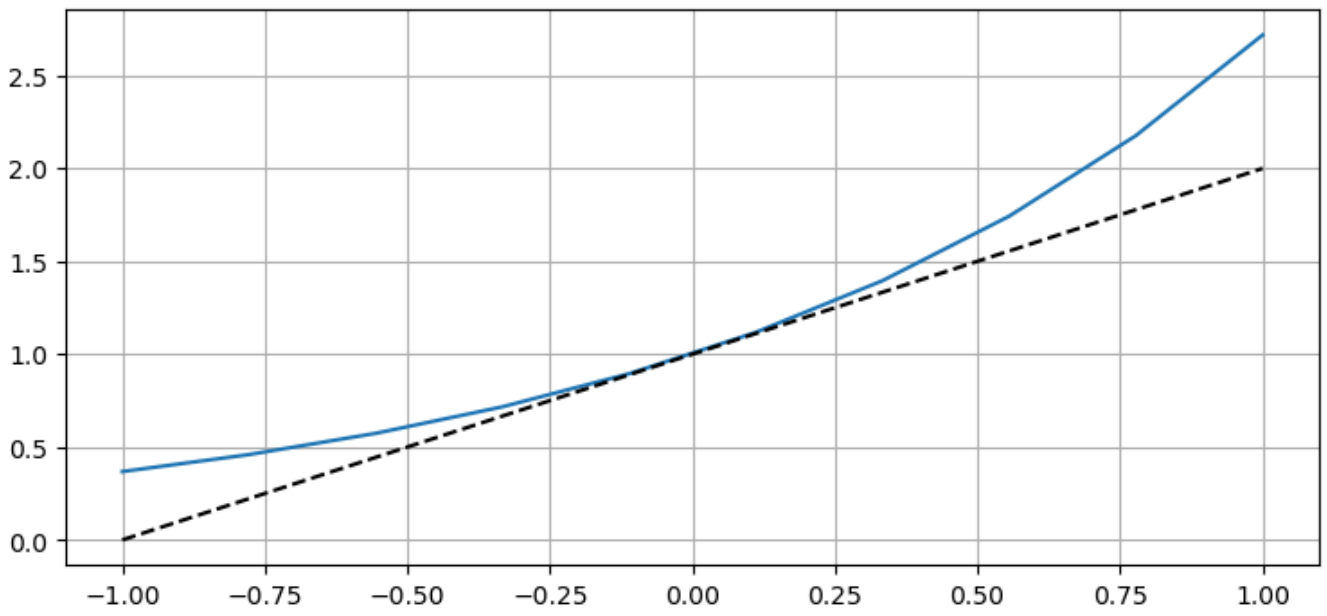
欧拉法

假设我们面临的是这样的初始条件问题：

$$
\begin{aligned}
y^{'} &= y, \\
y(0) &= 1
\end{aligned}
$$

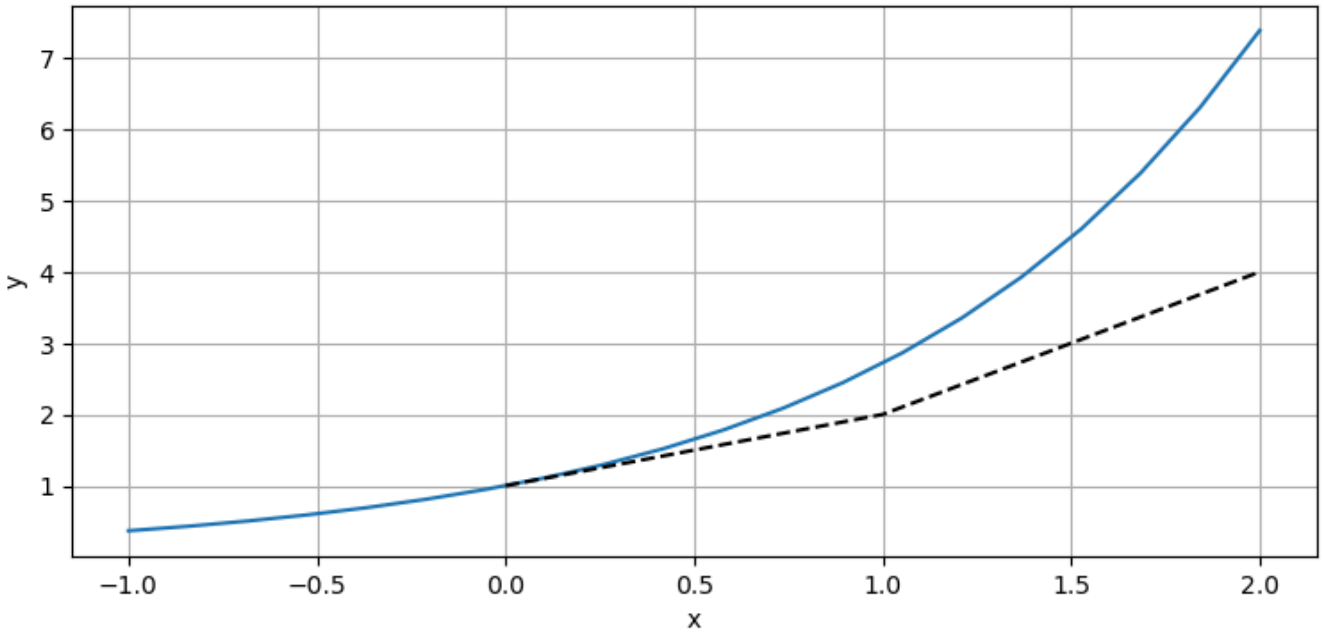
我们恰好知道确切的答案是 $y = e^t$ ，因为我们之前已经解出了这个方程。但对于一般的常微分方程来说，我们无法知道其确切的解。通常，我们只知道该方程的导数，而导数就相当于函数的斜率。我们还知道初始条件：在 $t = 0$ 和 $y = 1$ 时的函数值。只要知道了这两项信息，我们就可以利用 $t = 0$ 处的斜率以及 $y(0)$ 处的函数值来预测 $y(t = 1)$ 处的函数值。我在下图中绘制了这一关系。

```
import matplotlib.pyplot as plt

t = np.linspace(-1, 1, 10)
plt.plot(t, np.exp(t))
t = np.linspace(-1, 1, 2)
plt.plot(t,t+1, ls='--', c='k');
```



可以看到，在 $t = 0.1$ 处，曲线的斜率与估计值非常接近；但在 $t = 1$ 处，两者相差较大。不过，让我们暂时保持 1 的步长继续计算。在 $t = 1$ 处， y 的估计值为 2。接下来，我们可以通过计算 $t = 1$ 处曲线的斜率，再将其加到最初的估计值上，从而得到 $t = 2$ 处的值。由于斜率是在 $y' = y$ 处计算得出的，因此斜率为 2。

```
import kf_book.book_plots as book_plots

t = np.linspace(-1, 2, 20)
plt.plot(t, np.exp(t))
t = np.linspace(0, 1, 2)
plt.plot([1, 2, 4], ls='--', c='k')
book_plots.set_labels(x='x', y='y');
```



在这里，我们看到 y 的下一個估计值为 4。不过，误差值增长得相当快，这可能会让人觉得不太理想。不过，1 这个步长其实相当大。让我们把这一算法用代码实现出来，然后通过使用较小的步长来验证其是否能够正常工作。

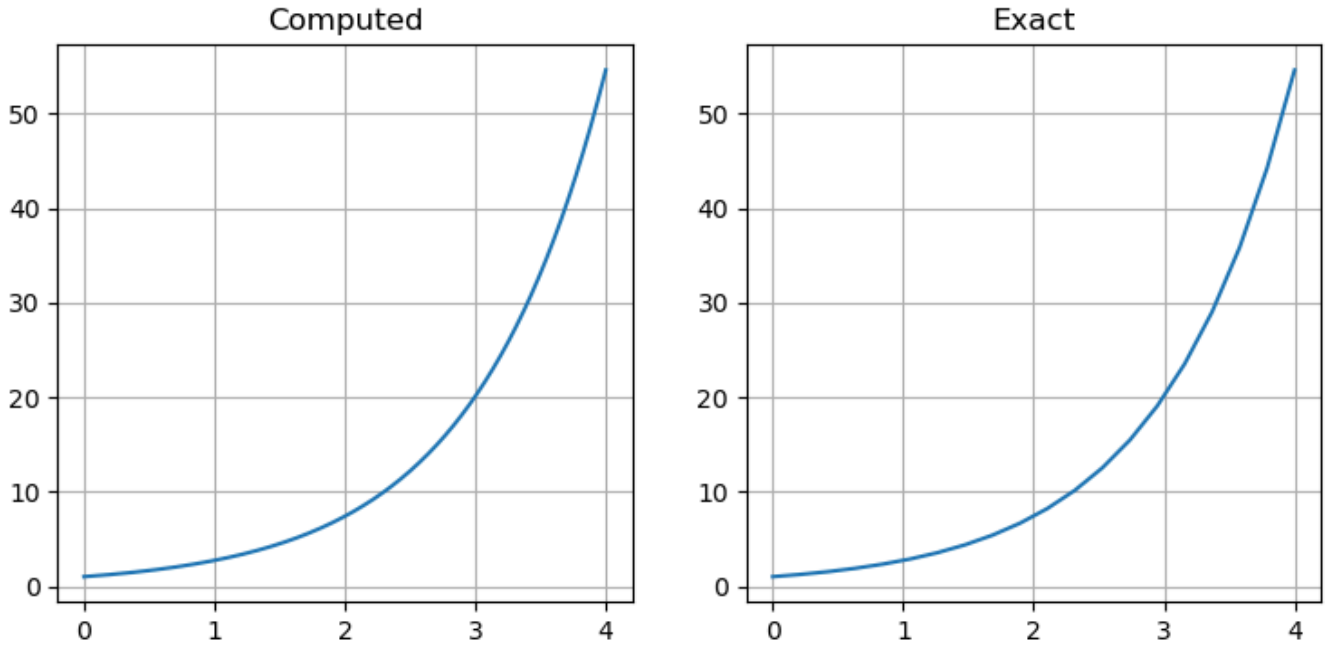
```
def euler(t, tmax, y, dx, step=1.):
    ys = []
    while t < tmax:
        y = y + step*dx(t, y)
        ys.append(y)
        t +=step
    return ys
```

```
def dx(t, y): return y

print(euler(0, 1, 1, dx, step=1.)[-1])
print(euler(0, 2, 1, dx, step=1.)[-1])
```

```
2.0
4.0
```

看起来没问题。那么现在，让我们试着用更小的步长来绘制结果吧。

```
ys = euler(0, 4, 1, dx, step=0.00001)
plt.subplot(1,2,1)
plt.title('Computed')
plt.plot(np.linspace(0, 4, len(ys)),ys)
plt.subplot(1,2,2)
t = np.linspace(0, 4, 20)
plt.title('Exact')
plt.plot(t, np.exp(t));
```




```
print('exact answer=', np.exp(4))
print('euler answer=', ys[-1])
print('difference =', np.exp(4) - ys[-1])
print('iterations =', len(ys))
```

```
exact answer= 54.598150033144236
euler answer= 54.59705808834125
difference = 0.0010919448029866885
iterations = 400000
```

可以看出，这里的误差相当小，但为了获得三位数的精确度，需要大量的迭代次数。实际上，对于大多数问题来说，欧拉方法的计算速度太慢了，因此我们通常会使用更复杂的方法来解决问题。

在继续之前，让我们先正式推导一下欧拉方法。因为欧拉方法是后文将要介绍的更复杂的龙格-库塔方法的基础。实际上，欧拉方法就是龙格-库塔方法中最简单的形式。

以下是 y 的泰勒展开式的前 3 项。如果进行无限项展开，就能得到精确的答案；而 $O(h^4)$ 则代表了由于有限项展开而产生的误差。

$$y(t_0 + h) = y(t_0) + hy'(t_0) + \frac{1}{2!}h^2y''(t_0) + \frac{1}{3!}h^3y'''(t_0) + O(h^4)$$

可以看出，欧拉法实际上运用了泰勒展开式的前两项。后续的各项都比前面的项要小，因此我们可以确信，该估算值与真实值的偏差不会太大。

龙格-库塔法

龙格-库塔法是数值积分领域中最常用的方法。文献中介绍了无数种数值积分方法。在实际应用中，使用我在这里介绍的龙格-库塔算法，几乎可以解决所有遇到的问题。该算法在速度、精度和稳定性方面都取得了很好的平衡。除非有充分的理由选择其他方法，否则龙格-库塔法都是最理想的数值积分方法。

让我们开始吧。首先从一些微分方程入手。

$$\dot{y} = \frac{d}{dt}y$$

.

我们可以用函数 f 来表示 y 的导数，具体如下：

$$\dot{y} = \frac{d}{dt}f(y,t)$$

.

推导这些方程超出了本书的讨论范围，不过龙格-库塔 RK4 方法正是基于这些方程而定义的。

$$y(t+\Delta t) = y(t) + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) + O(\Delta t^4)$$

$$\begin{aligned} k_1 &= f(y,t)\Delta t \\ k_2 &= f(y+\frac{1}{2}k_1, t+\frac{1}{2}\Delta t)\Delta t \\ k_3 &= f(y+\frac{1}{2}k_2, t+\frac{1}{2}\Delta t)\Delta t \\ k_4 &= f(y+k_3, t+\Delta t)\Delta t \end{aligned}$$

以下是相应的代码：

```
def runge_kutta4(y, x, dx, f):
    """computes 4th order Runge-Kutta for dy/dx.
    y is the initial value for y
    x is the initial value for x
    dx is the difference in x (e.g. the time step)
    f is a callable function (y, x) that you supply
    to compute dy/dx for the specified values.
    """

    k1 = dx * f(y, x)
    k2 = dx * f(y + 0.5*k1, x + 0.5*dx)
    k3 = dx * f(y + 0.5*k2, x + 0.5*dx)
    k4 = dx * f(y + k3, x + dx)

    return y + (k1 + 2*k2 + 2*k3 + k4) / 6.
```

我们用这个来举个简单的例子吧。假设……

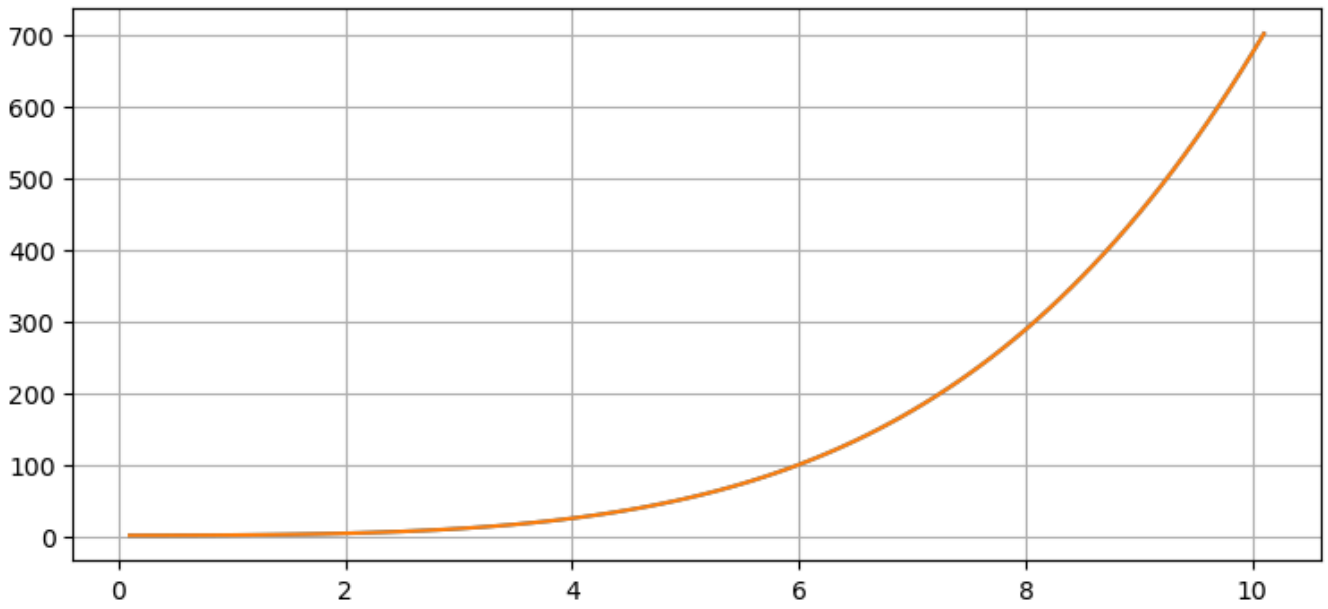
$$\dot{y} = t\sqrt{y(t)}$$

以初始值为准/以初始条件开始

$$\begin{aligned} t_0 &= 0 \\ y_0 &= y(t_0) = 1 \end{aligned}$$

```
import math
import numpy as np
t = 0.
y = 1.
dt = .1

ys, ts = [], []

def func(y,t):
    return t*math.sqrt(y)

while t <= 10:
    y = runge_kutta4(y, t, dt, func)
    t += dt
    ys.append(y)
    ts.append(t)

exact = [(t**2 + 4)**2 / 16. for t in ts]
plt.plot(ts, ys)
plt.plot(ts, exact)

error = np.array(exact) - np.array(ys)
print(f"max error {max(error):.5f}")
```

```
max error 0.00005
```



贝叶斯滤波

从“离散贝叶斯”这一章开始，我采用了贝叶斯方法来进行数据处理。假设我们正在追踪某个物体。我们可以将该物体在某一特定时刻的状态定义为它的位置、速度等信息。例如，我们可以将时间点 t 时的物体状态表示为 $x_t = \begin{bmatrix} x_t & \dot{x}_t \end{bmatrix}^T$。

当我们测量某个物体时，实际上是在测量该物体的状态或其中的一部分。由于传感器的存在，测量结果会受到噪声的干扰。不过，测量结果终究是由物体的状态所决定的。也就是说，物体状态的变化会影响到测量结果，但测量结果的变化则不会改变物体的状态。

在滤波处理过程中，我们的目标是计算从时间 0 到时间 t 期间，各状态 $x_{0:t}$ 的最优估计值。如果我们知道 $x_{0:t}$ 的值，那么计算出与这些状态相对应的测量值 $z_{0:t}$ 就很简单了。但实际上，我们只能得到一组测量值 $z_{0:t}$，而需要计算出与之对应的状态 $x_{0:t}$。这一过程被称为“统计反演”，因为我们试图从输出结果来反推输入值。

反演是一个棘手的问题，因为通常不存在唯一的解决方案。对于给定的一组状态 $x_{0:t}$，只有一组可能的测量结果（再加上噪声），但对于给定的一组测量结果来说，却有可能有多种不同的状态组合导致了这些测量结果。

回顾贝叶斯定理：

$$P(x \mid z) = \frac{P(z \mid x)P(x)}{P(z)}$$

其中，$P(z \mid x)$ 表示对测量结果 z 的置信度；$P(x)$ 则基于我们的过程模型所得出的先验概率；$P(z)$ 是一个归一化常数。$P(x \mid z)$ 表示后验概率，也就是在考虑了测量结果 z 之后的概率分布，这一结果也被称为“证据”。

这是一种统计上的反演过程，即从 $P(z \mid x)$ 状态转变为 $P(x \mid z)$ 状态。我们这个滤波问题的解决方案可以表示为：

$$P(x_{0:t} \mid z_{0:t}) = \frac{P(z_{0:t} \mid x_{0:t})P(x_{0:t})}{P(z_{0:t})}$$

一切都很好，直到下一次测量结果 z_{t+1} 出来为止。一旦有了新的测量数据，我们就需要重新计算整个区间 $0{:}\,t+1$ 内的数值。

实际上，这很难实现，因为我们试图计算整个时间范围内状态的后验概率分布。但是，当我们刚刚获得第十次测量结果时，我们真的需要知道第三步的状态概率分布吗？通常情况下并不需要。因此，我们放宽了要求，只计算当前时间步的状态概率分布即可。

第一个简化步骤是：我们将所描述的过程（比如，运动物体的运动模型）视为一个马尔可夫链。也就是说，当前状态完全取决于上一个状态以及一个转移概率 $P(x_k \mid x_{k-1})$，该转移概率指的是从上一个状态转移到当前状态的概率。我们将其表示为：

$$x_k \sim P(x_k \mid x_{k-1})$$

实际上，这种做法是完全合理的，因为很多事物都具备马尔可夫性质。比如，当你在停车场驾驶时，下一秒你的位置是否取决于你一分钟前是在高速公路上行驶，还是在土路上行驶呢？并不然。下一秒你的位置完全取决于你当前的位置、速度以及操控方式，而与一分钟前发生的事情无关。因此，汽车也具备马尔可夫性质，我们可以进行这样的简化处理，而不会损失任何精度或普遍性。

我们接下来的简化步骤是：将测量模型定义为取决于当前状态 x_k，同时考虑在给定当前状态的情况下进行测量的条件概率 $P(z_k \mid x_k)$。我们表示为：

$$z_k \sim P(z_k \mid x_k)$$

现在出现了循环现象，因此我们需要一个初始条件来终止这种循环。所以，我们可以说，初始分布其实就是状态 x_0 出现的概率。

$$x_0 \sim P(x_0)$$

这些参数被代入贝叶斯方程中。如果我们知道初始状态 x_0 以及第一次测量结果，就可以估算出状态 $P(x_1 \mid z_1)$。运动模型则用于确定先验概率 $P(x_2 \mid x_1)$。我们将这些数值代入贝叶斯定理中，从而计算出状态 $P(x_2 \mid z_2)$。我们不断重复这个预测-修正过程，仅根据时间点 $t-1$ 的状态和分布以及时间点 t 的测量结果，来递归地计算出时间点 t 的状态和分布。

这种计算所涉及的数学细节因具体问题而异。在“离散贝叶斯滤波”和“单变量卡尔曼滤波”这两章中，分别给出了两种不同的计算公式，你应该能够理解其中的原理。单变量卡尔曼滤波假设：对于标量状态而言，无论是噪声还是系统本身的行为，都受到均值为零、相互不相关的高斯噪声的影响。

多元卡尔曼滤波器也基于同样的假设，只不过这里的“状态”和“测量值”都是向量，而非标量。卡尔曼博士证明了：只要这些假设成立，卡尔曼滤波器就具有最小二乘意义上的最优性。通俗来说，这意味着我们无法从这些有噪声的测量数据中获取更多信息。在本书的后续部分，我将介绍那些不那么严格限制线性条件和高斯噪声假设的滤波器。

在继续之前，我想再谈谈关于统计反演的问题。正如 Calvetti 和 Somersalo 在《贝叶斯科学计算导论》中所写的那样：“我们采用贝叶斯视角：所谓‘随机性’，其实只是意味着信息的缺失而已”[3]。状态参数用来描述那些原则上可以被测量或计算出来的物理现象：比如速度、空气阻力等等。由于我们缺乏足够的信息来精确计算或测量这些数值，因此我们把它们视为随机变量。严格来说，它们并非真正的随机变量，所以这种看法其实是一种主观判断。

他们专门用了一整章的篇幅来讨论这个主题。我则只需用一段文字来说明即可。贝叶斯滤波之所以可行，是因为我们将统计特性赋予了那些未知的参数。在卡尔曼滤波中，我们可以得到用于确定最优估计的精确解。而其他滤波方法，比如离散贝叶斯滤波或粒子滤波（我们将在后面的章节中详细介绍），则是以一种更为复杂的、非最优的方式来处理概率问题。我们这种方法的优点在于：它把信息的缺失视为一个随机变量，用概率分布来描述这个随机变量，然后再利用贝叶斯定理来解决统计推断问题。

将卡尔曼滤波器转换为 g-h 滤波器

我之前说过，卡尔曼滤波其实是一种 g-h 滤波的特例。只需通过一些代数运算就能证明这一点。在一维情况下，这种证明更为简单，所以我将用一维情况来举例说明。请回忆一下。

$$\mu_x = \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2}$$

我会让它们看起来更美观一些，具体如下：

$$\mu_x = \frac{ya + xb}{a + b}$$

我们可以通过以下代数运算，轻松将其转化为 g-h 形式。

$$\begin{aligned}
\mu_x &= (x - x) + \frac{ya + xb}{a + b} \\
\mu_x &= x - \frac{a + b}{a + b}x + \frac{ya + xb}{a + b} \\
\mu_x &= x + \frac{-x(a + b) + xb + ya}{a + b} \\
\mu_x &= x + \frac{-xa + ya}{a + b} \\
\mu_x &= x + \frac{a}{a + b}(y - x)
\end{aligned}$$

我们差不多完成了，但请记住，估计值的方差是由以下公式给出的。

$$\begin{aligned}
\sigma_x^2 &= \frac{1}{\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}} \\
&= \frac{1}{\frac{1}{a} + \frac{1}{b}}
\end{aligned}$$

我们可以通过观察发现，可以将那个术语纳入到上面的方程式中。

$$\begin{aligned}
\frac{a}{a + b} &= \frac{a \,/\, a}{(a + b) \,/\, a} = \frac{1}{(a + b) \,/\, a} \\
&= \frac{1}{1 + \frac{b}{a}} = \frac{1}{\frac{b}{b} + \frac{b}{a}} \\
&= \frac{1}{b}\frac{1}{\frac{1}{b} + \frac{1}{a}} \\
&= \frac{\sigma_x^2}{b}
\end{aligned}$$

我们可以把这一切联系起来。

$$\begin{aligned}
\mu_x &= x + \frac{a}{a + b}(y - x) \\
&= x + \frac{\sigma_x^2}{b}(y - x) \\
&= x + g_n(y - x)
\end{aligned}$$

在……的地方/在何处

$$g_n = \frac{\sigma_x^2}{\sigma_y^2}$$

最终的计算方法是：将两次测量所得的残差乘以一个常数，然后将其加到之前的数值上。这就是 g-h 滤波器的 g 公式。 g 则是指新估计值的方差除以测量值的方差。当然，在这种情况下， g 并非恒定值，因为它会随着每个时间步长的变化而变化。我们也可以用同样的方法推导出 h 的公式。不过，这个推导过程并不太复杂，我就不详细介绍了。最终的结果就是……

$$h_n = \frac{COV(x, \dot{x})}{\sigma_y^2}$$

关键在于： g 和 h 的值完全由在时间点 n 时的测量值和预测值的方差及协方差来决定。换句话说，我们是通过一个由这两个输入数据的质量所决定的比例因子，来在测量值和预测值之间选择一个点。

参考文献

- [1] C.B. Molwer 和 C.F. Van Loan：“二十五年后，计算矩阵指数的十九种可疑方法”，《SIAM 评论》第 45 卷，第 3-49 页，2003 年。
- [2] C.F. van Loan，《涉及矩阵指数的积分计算》，《IEEE 自动控制汇刊》，1978 年 6 月。
- [3] Calvetti, D.和 Somersalo, E.，《贝叶斯科学计算导论：关于主观计算的十讲》，施普林格出版社，2007 年。
- [4] Brown, R. G.和 Hwang, P. Y.C.著，《随机信号与卡尔曼滤波应用导论》，Wiley and Sons 出版社，第四版，第 143-147 页，2012 年。

第 8 章：卡尔曼滤波器的设计

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

在上一章中，我们研究了那些“典型的”问题。这类问题的描述很简单，用几行代码就能实现相应的程序，也便于讲解。但现实世界中的问题很少如此简单。在这一章中，我们将研究更贴近实际的例子，学习如何评估滤波器的性能。

我们将首先在二维空间中追踪机器人的运动轨迹，比如在田野或仓库里。我们会从使用简单的传感器开始，这些传感器会输出有噪声的坐标数据。我们需要对这些数据进行处理，才能得到准确的二维轨迹。一旦掌握了这一原理，我们就会引入更多的传感器，并添加控制机制，从而进一步拓展这个模型的应用范围。

接下来，我们将探讨非线性问题。虽然现实世界是非线性的，但卡尔曼滤波器本身是线性的。在处理那些非线性程度不强的问题时，使用卡尔曼滤波器是可行的；但在处理非线性程度较高的问题时，则不可行。我会分别举例说明这两种情况。这将为本书的后续内容奠定基础，我们将在后续内容中学习处理非线性问题的各种技术。

追踪机器人

这次对机器人的跟踪尝试，其实与前面几章中讨论的一维空间中的物体跟踪问题非常相似。只不过，以前是使用能够检测走廊内位置的传感器，而现在则使用的是能够提供二维空间中位置信息的传感器。在每个时间点上，该传感器都会给出该点在二维空间中的坐标值。不过，这些坐标值都带有噪声。

编写与真实传感器进行交互的代码超出了本书的讨论范围。因此，我们仍然会像之前那样，对传感器进行简单的模拟。在开发过程中，我们会逐步创建出更复杂的传感器模型。在编写代码时，我会在函数名称后加上相应的数字来区分不同的传感器。

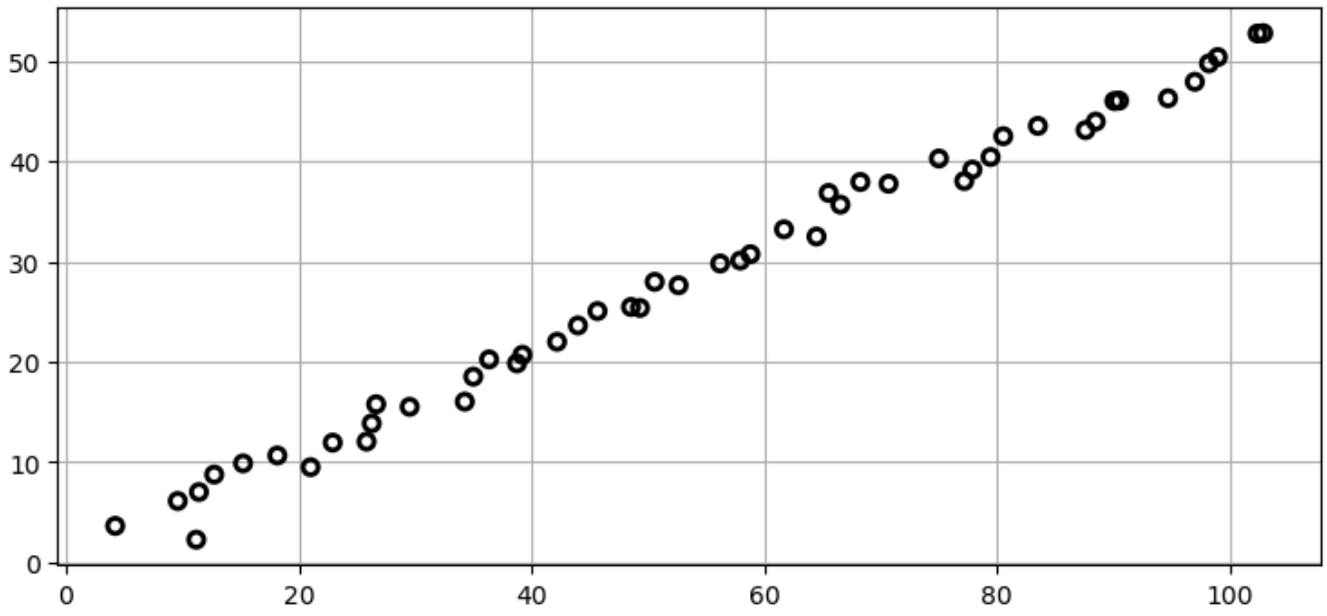
那么，让我们从一种非常简单的传感器开始吧。这种传感器用于模拟对沿直线运动的物体的跟踪。在初始化时，需要指定物体的初始位置、速度以及噪声的标准差。每次调用 `read()` 时，该传感器都会将物体的位置更新一个时间步长，并返回新的测量结果。

```
from numpy.random import randn

class PosSensor(object):
    def __init__(self, pos=(0, 0), vel=(0, 0), noise_std=1.):
        self.vel = vel
```

```
        self.noise_std = noise_std
        self.pos = [pos[0], pos[1]]

    def read(self):
        self.pos[0] += self.vel[0]
        self.pos[1] += self.vel[1]

        return [self.pos[0] + randn() * self.noise_std,
                self.pos[1] + randn() * self.noise_std]
```

进行一次快速测试，以确认其功能符合我们的预期。

```
import matplotlib.pyplot as plt
import numpy as np
from kf_book.book_plots import plot_measurements

pos, vel = (4, 3), (2, 1)
sensor = PosSensor(pos, vel, noise_std=1)
ps = np.array([sensor.read() for _ in range(50)])
plot_measurements(ps[:, 0], ps[:, 1]);
```



看起来没错。斜率确实是 1/2，这与速度为(2, 1)的情况相符。数据似乎是从接近(6, 4)的位置开始的。不过，这种描述看起来不太符合现实情况。这仍然只是“教科书式”的描述而已。在后续的分析中，我们会加入一些复杂的因素，以更真实地反映实际情况。

请选择状态变量。

和往常一样，第一步是确定状态变量。我们在两个维度上进行测量，每个维度上都有一个传感器来提供相应的数值。因此，我们拥有两个可观测变量： x 和 y 。如果只使用这两个变量来构建卡尔曼滤波器，其性能不会很好，因为我们忽略了速度这一重要信息。我们必须将速度信息也纳入方程式中。我会这样表示：

$$x=[x \cdot xy \cdot y]T$$

这个矩阵并没有什么特别之处。我本可以用 、 [xy˙x˙y]T 之类的符号来代替，反正这些符号也没什么逻辑意义。只要在矩阵的其他部分保持一致性即可。我喜欢将位置和速度放在一起，这样就能确保协方差矩阵中，位置和速度对应的子块中的协方差保持一致。在我的公式中， `P[1,0]` 代表了 、 x 以及 、 ˙x 之间的协方差。而在另一种表述方式中，该协方差则位于 `P[2, 0]` 位置。随着维度数的增加，这种情况会变得更加复杂。

让我们暂停一下，来探讨一下如何识别那些隐藏的变量。这个例子比较容易理解，因为我们已经处理过一维情况了。但其他问题则没有那么简单。对于这个问题，没有简单的答案。首先，你需要思考的是：从传感器获取的数据的一阶和二阶导数究竟代表什么含义。如果你以固定的时间间隔从传感器中读取数据，那么计算一阶和二阶导数在数学上其实很简单。一阶导数其实就是两次连续读数之间的差值。在我们的跟踪应用中，一阶导数有明确的物理意义：两次连续位置之间的差值就是速度。

除此之外，你还可以考虑如何将来自两个或更多不同传感器的数据结合起来，从而获得更多有用的信息。这就引出了“传感器融合”的概念，我们将在后面的章节中详细介绍相关案例。目前需要明白的是，选择合适的状态变量是确保滤波器能够发挥最佳性能的关键。一旦确定了隐藏变量，就必须进行大量测试，以确保所获得的结果是可靠的。卡尔曼滤波器会按照你指定的模型来运行；如果你的模型无法为隐藏变量提供准确的信息，那么卡尔曼滤波器的输出结果将毫无意义。

设计状态转换函数

我们的下一步是设计状态转换函数。请记住，状态转换函数实际上是一个矩阵 F 。我们将该矩阵与系统的当前状态相乘，从而得到下一个状态。

¯x=Fx

我就不再详细阐述了，因为这与我们在前一章中讨论的一维情况非常相似。状态转换方程如下：

x=1x+Δt˙x+0y+0˙yvx=0x+1˙x+0y+0˙yy=0x+0˙x+1y+Δt˙yvy=0x+0˙x+0y+1˙y

这样的表述方式让我们清楚地了解了 F 所要求的数值结构以及行与列的排列方式。我们可以将其转化为矩阵-向量形式来表示：

⎡⎢⎢⎢⎣x˙xy˙y⎤⎥⎥⎥⎦=⎡⎢⎢⎣1Δt000100001Δt0001⎤⎥⎥⎦⎡⎢⎢⎢⎣x˙xy˙y⎤⎥⎥⎥⎦

那么，我们就用 Python 来实现吧。其实非常简单：唯一需要做的就是将 `dim_z` 的值设置为 2。至于为什么要把它设置为 2，我们会在第 4 步中详细解释。

```
from filterpy.kalman import KalmanFilter

tracker = KalmanFilter(dim_x=4, dim_z=2)
dt = 1.   # time step 1 second

tracker.F = np.array([[1, dt, 0,  0],
                      [0,  1, 0,  0],
                      [0,  0, 1, dt],
                      [0,  0, 0,  1]])
```

设计过程噪声矩阵

FilterPy 可以为我们计算出 Q 矩阵。为简化问题，我假设噪声属于离散时间维纳过程——即每个时间点上的噪声值都是恒定的。这一假设使得我可以用方差来描述模型在不同步骤之间的变化程度。如果这一点还不清楚，请重新阅读卡尔曼滤波的数学原理部分。

```
from scipy.linalg import block_diag
from filterpy.common import Q_discrete_white_noise

q = Q_discrete_white_noise(dim=2, dt=dt, var=0.001)
tracker.Q = block_diag(q, q)
print(tracker.Q)
```

```
[[0.    0.001 0.    0.   ]
 [0.001 0.001 0.    0.   ]
 [0.    0.    0.    0.001]
 [0.    0.    0.001 0.001]]
```

在这里，我假设 x 和 y 中的噪声是相互独立的。因此，任意 x 和 y 变量之间的协方差都应该为零。这样一来，我就可以先计算出一维情况下的 Q 值，然后再利用 `block_diag` 将该值分别应用到 x 轴和 y 轴上。

设计控制函数

我们还没有为机器人添加任何控制功能，因此在这一步无需做任何操作。 `KalmanFilter` 类会将 `B` 的值初始化为 0，因为默认情况下不存在任何控制输入。所以，不需要编写任何代码。如果你愿意，也可以手动将 `tracker.B` 的值设置为 0，不过如你所见，它的值本来就是 0。

```
tracker.B
```

设计测量函数

测量函数 H 决定了如何利用方程 z=Hx 将状态变量转化为测量结果。在这种情况下，我们需要测量(x, y)这两个数值，因此我们会将 z 设计为 [xy]T 的形式，其维度为 2x1。而我们的状态变量则具有 4x1 的维度。通过回忆一下：将一个 MxN 尺寸的矩阵与一个 NxP 尺寸的矩阵相乘，得到的结果是一个 MxP 尺寸的矩阵。由此，我们可以确定 H 所需的尺寸。

$(2\times1)=(a\times b)(4\times1)=(2\times4)(4\times1)$

所以， H 的结果是 2 乘以 4。

填写 H 的值其实很简单，因为所测量的数值其实就是机器人的位置，而这些位置信息正好对应于状态 x 中的 x 和 y 变量。为了让这个过程更有趣一点，我们决定改变计量单位。测量结果是以英尺为单位，而我们希望以米为单位来表示这些数值。由于 H 的值会随着状态的改变而变化，因此转换公式为 feet=meters/0.3048 。这样处理之后，就能得到正确的结果了。

H=[10.30480000010.30480]

这对应于这些线性方程。

zx=(x0.3048)+(0∗vx)+(0∗y)+(0∗vy)=x0.3048zy=(0∗x)+(0∗vx)+(y0.3048)+(0∗vy)=y0.3048

这是个简单的问题，我们本可以直接得出方程式，而不必进行我上面所进行的那些维度分析。不过，记住卡尔曼滤波器的方程式决定了所有矩阵都必须具有特定的维度这一点是很重要的。当我在设计某个系统时遇到困难时，查看一下各矩阵的维度往往能提供解题的线索。

以下是我的实现方式：

```
tracker.H = np.array([[1/0.3048, 0, 0,        0],
                      [0,        0, 1/0.3048, 0]])
```

设计测量噪声矩阵

我们假设 x 和 y 这些变量都是独立的高斯过程。也就是说，x 中的噪声与 y 中的噪声没有任何关联，且这些噪声都服从均值为 0 的正态分布。目前，我们将 x 和 y 的方差都设为 5 米 2 。由于这些变量是独立的，因此它们之间没有协方差，对角线以外的元素都为 0。这样就得到了：

R=[σ2xσyσxσxσyσ2y]=[5005]

这是一个 2×2 矩阵，因为我们有两个传感器输入。对于 n 个变量来说，协方差矩阵的大小始终是 n×n 。在 Python 中，我们这样表示：

```
tracker.R = np.array([[5., 0],
                      [0, 5]])
tracker.R
```

```
array([[5., 0.],
       [0., 5.]])
```

初始条件

对于这个简单的问题，我们将把初始位置设定为(0,0)，初始速度也设为(0,0)。由于这只是一个初步的估计，我们会将协方差矩阵 P 的值设得很大。

x=⎡⎢⎣0000⎤⎥⎦,P=⎡⎢⎣500000050000005000000500⎤⎥⎦

Python 版本的实现方式是

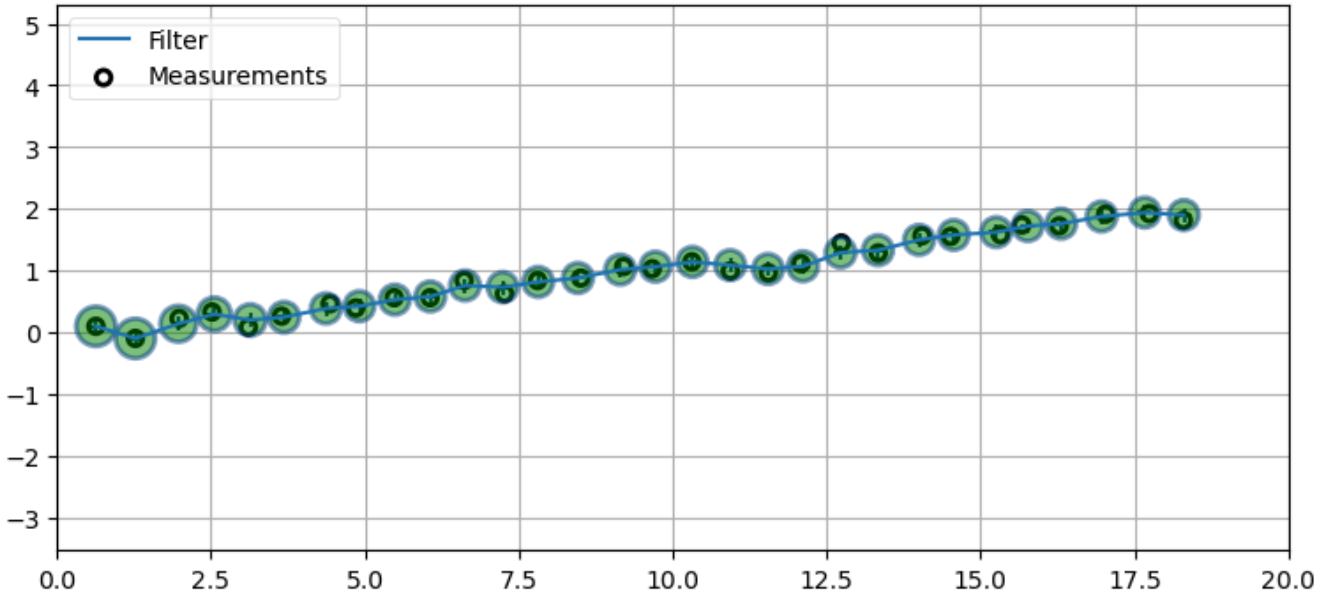
```
tracker.x = np.array([[0, 0, 0, 0]]).T
tracker.P = np.eye(4) * 500.
```

实现滤波功能

设计已经完成，现在只需编写代码来运行该滤波器，并将处理后的数据以我们希望的格式输出出来。我们将让代码运行 30 次。

```
from filterpy.stats import plot_covariance_ellipse
from kf_book.book_plots import plot_filter
```

```
R_std = 0.35
Q_std = 0.04

def tracker1():
    tracker = KalmanFilter(dim_x=4, dim_z=2)
    dt = 1.0   # time step

    tracker.F = np.array([[1, dt, 0,  0],
                          [0,  1, 0,  0],
                          [0,  0, 1, dt],
                          [0,  0, 0,  1]])
    tracker.u = 0.
    tracker.H = np.array([[1/0.3048, 0, 0, 0],
                          [0, 0, 1/0.3048, 0]])

    tracker.R = np.eye(2) * R_std**2
    q = Q_discrete_white_noise(dim=2, dt=dt, var=Q_std**2)
    tracker.Q = block_diag(q, q)
    tracker.x = np.array([[0, 0, 0, 0]]).T
    tracker.P = np.eye(4) * 500.
    return tracker

# simulate robot movement
N = 30
sensor = PosSensor((0, 0), (2, .2), noise_std=R_std)

zs = np.array([sensor.read() for _ in range(N)])

# run filter
robot_tracker = tracker1()
mu, cov, _, _ = robot_tracker.batch_filter(zs)

for x, P in zip(mu, cov):
    # covariance of x and y
    cov = np.array([[P[0, 0], P[2, 0]],
                    [P[0, 2], P[2, 2]]])
    mean = (x[0, 0], x[2, 0])
    plot_covariance_ellipse(mean, cov=cov, fc='g', std=3, alpha=0.5)

#plot results
zs *= .3048 # convert to meters
plot_filter(mu[:, 0], mu[:, 2])
plot_measurements(zs[:, 0], zs[:, 1])
plt.legend(loc=2)
plt.xlim(0, 20);
```



我鼓励你们试着去操作一下：将 、 Q 和 、 R 设置为不同的数值。不过，我们在前面的章节中已经做了很多类似的练习了，而且还有许多内容需要讲解。所以，接下来我会介绍一些更复杂的案例，在这些案例中，我们还将有机会亲自尝试调整这些数值。

我用绿色标出了 、 3σ 与 、 x 以及 、 y 之间的协方差椭圆。你能解释一下这些椭圆的形状吗？也许你原本以为它们会是倾斜的椭圆，就像在前面几章中所看到的那样。不过请记住，在那些章节中，我们并不是将 、 x 与 、 y 进行比较，而是将 、 x 与 、 ·x 进行比较。 、 x 与 、 ·x 之间存在相关性，而 、 x 则与 、 y 没有相关性或依赖性。因此，我们所绘制的椭圆都是垂直的。此外， 、 x 和 、 y 的噪声标准差也被设定为相同。如果我们把 R 设为某个具体的数值的话……

R=[100.5]

我们会告诉卡尔曼滤波器：在 x 处，噪声比在 y 处要大；同时，我们所绘制的椭圆形状也会更长，高度则相对较低。

P 的最终数值能告诉我们关于各状态变量之间相关性的所有必要信息。如果我们只看对角线上的数值，就能得知每个变量的方差。换句话说， P0,0 表示变量 x 的方差， P1,1 表示 ·x 的方差， P2,2 表示变量 y 的方差，而 P3,3 则表示 ·y 的方差。我们可以使用 `numpy.diag()` 来提取矩阵的对角线数值。

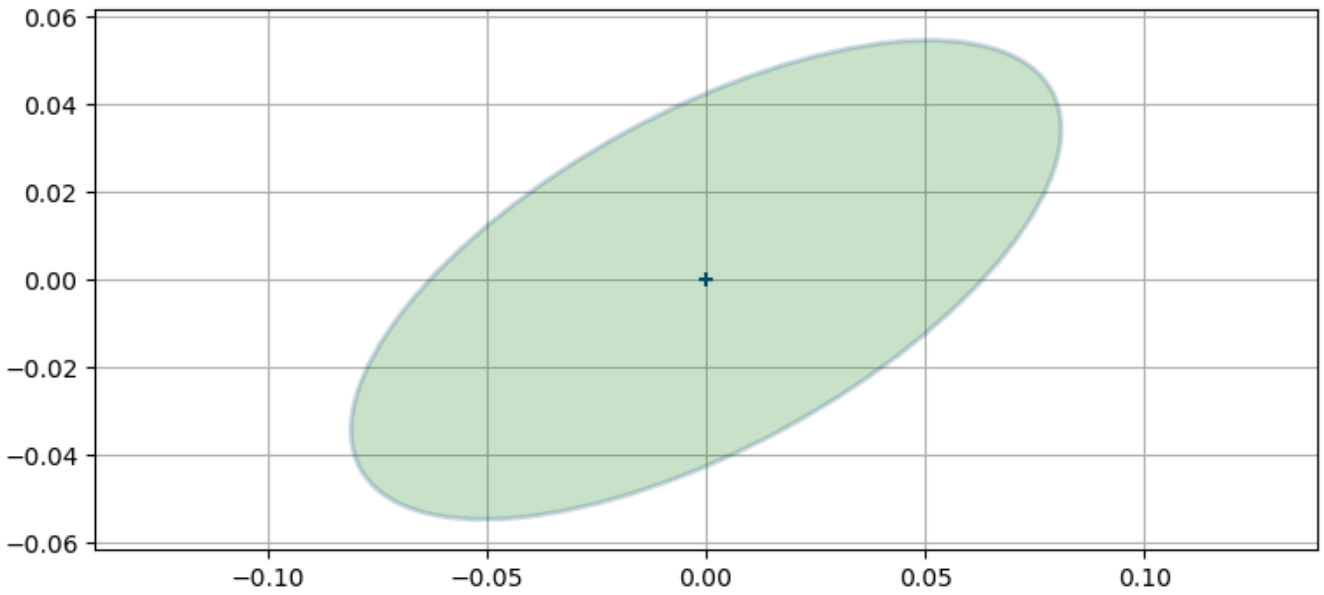
```
print(np.diag(robot_tracker.P))
```

```
[0.007 0.003 0.007 0.003]
```

该协方差矩阵包含四个 2×2 矩阵，你应该能够轻松识别出它们。这是因为 x 与 ·x 之间存在相关性，同时 y 与 ·y 之间也存在相关性。矩阵的左上角部分表示的是 x 与 ·x 之间的协方差。

```
c = robot_tracker.P[0:2, 0:2]
print(c)
plot_covariance_ellipse((0, 0), cov=c, fc='g', alpha=0.2)
```

```
[[0.007 0.003]
 [0.003 0.003]]
```



由于数据的结构安排，协方差矩阵的左上角部分包含了与 x 以及 ·x 相关的数据。请记住， Pi,j 和 Pj,i 这些单元格中存储的正是 σiσj 的数据。

最后，让我们看看 P 的左下角部分，那一块全是 0。为什么是 0 呢？请看 P3,0 。它存储的是 σ3σ0 这个数值，而 σ3σ0 实际上就是 ·y 与 x 之间的协方差。由于 ·y 和 x 是相互独立的，因此该协方差值为 0。其余的数值也是类似的情况，都是独立变量之间的协方差，所以也都为 0。

```
robot_tracker.P[2:4, 0:2]
```

```
array([[0., 0.],
       [0., 0.]])
```

滤波顺序

我们 bisher 只研究了位置和速度的跟踪问题。这种方法效果不错，但这只是因为我所选择的问题都适合用这种方法来处理。现在，你们已经对卡尔曼滤波有了足够的了解，可以更一般性地来考虑这个问题了。

我所说的“阶数”是什么意思呢？在这些系统模型的背景下，所谓“阶数”，指的是为准确描述某个系统而必需的导数数量。以一座不会发生变化的建筑为例：由于高度始终不变，因此不需要任何导数。此时，该系统的阶数为零。我们可以用方程来表示这一点： x=312.5 。

一阶系统具有其一阶导数。例如，位置的变化就是速度，我们可以将其表示为：

v=dxdt

我们将其整合进牛顿方程中。

x=vt+x0.

这也被称为“恒速模型”，因为该模型假设速度是恒定的。

二阶系统具有二阶导数。位置的二阶导数就是加速度，其计算公式为：

a=d2xdt2

我们将其整合进……之中。

x=12at2+v0t+x0.

这也被称为恒定加速度模型。

另一种理解方式是考虑多项式的阶数。在恒定加速度模型中，函数的二阶导数存在，因此该模型属于二阶模型。同样地，多项式 x=12at2+v0t+x0 也属于二阶多项式。

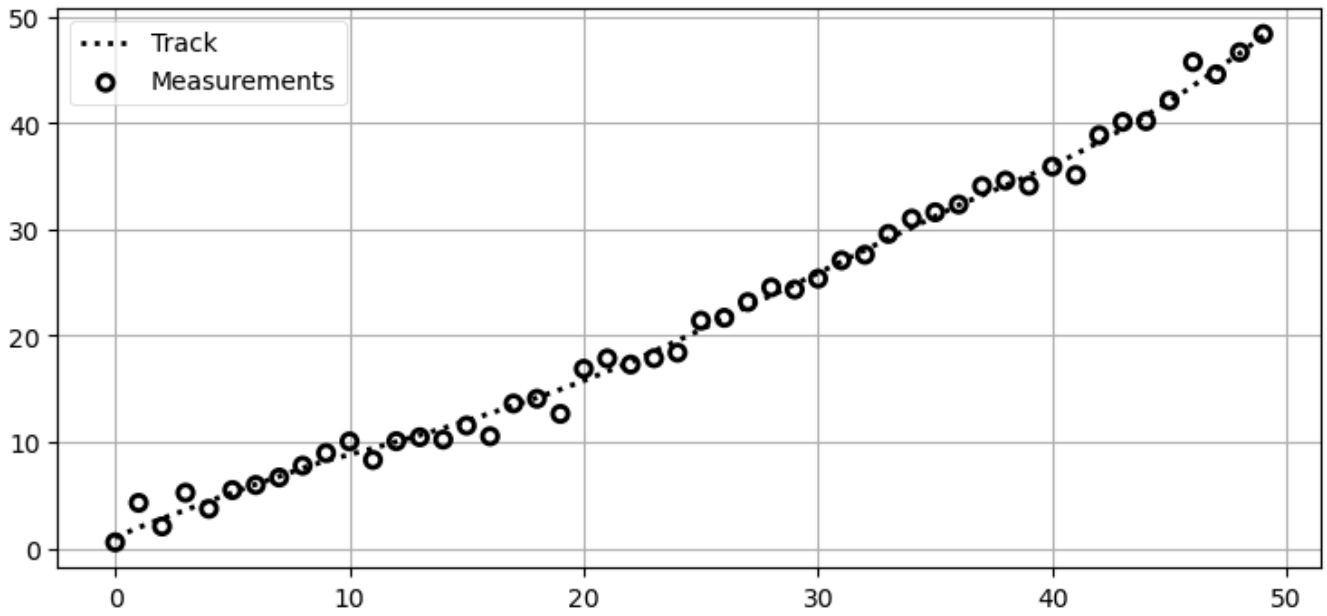
在设计状态变量和过程模型时，我们必须确定所要建模的系统的阶数。比如，如果我们想要追踪某个以恒定速度运动的物体。不过，现实世界中的任何过程都不可能是完美的，因此，在短时间内，物体的速度会存在轻微的波动。在这种情况下，最合适的做法是使用二阶滤波器，让加速度项来处理这些微小的速度波动。

实际上，这种做法并不奏效。为了彻底理解这个问题，让我们来看看使用与被滤波系统的顺序不匹配的流程模型会带来怎样的后果。

首先，我们需要一个用于滤波数据的系统。我会编写一个类来模拟那些具有恒定速度的物体。实际上，没有任何物理系统能够真正保持恒定速度，因此在每次更新时，我们都会让物体的速度发生微小的变化。此外，我还编写了一个“传感器”类，用来模拟传感器产生的高斯噪声。代码如下所示。我还绘制了测试结果，以确认该系统的运行是否正常。

```
from kf_book.book_plots import plot_track

class ConstantVelocityObject(object):
    def __init__(self, x0=0, vel=1., noise_scale=0.06):
        self.x = x0
        self.vel = vel
        self.noise_scale = noise_scale

    def update(self):
        self.vel += randn() * self.noise_scale
        self.x += self.vel
        return (self.x, self.vel)

def sense(x, noise_scale=1.):
    return x[0] + randn()*noise_scale

np.random.seed(124)
obj = ConstantVelocityObject()

xs, zs = [], []
for i in range(50):
    x = obj.update()
    z = sense(x)
    xs.append(x)
    zs.append(z)

xs = np.asarray(xs)
```

```
plot_track(xs[:, 0])
plot_measurements(range(len(zs)), zs)
plt.legend(loc='best');
```



我对这个模型很满意。由于我们在系统中加入了随机噪声，因此这条轨迹并不完全呈直线。这可能相当于一个人在街道上行走的轨迹，或者是一架飞机在多变的风力作用下飞行的轨迹。这里不存在有意的加速度，因此我们可以将其视为一个匀速运动系统。你可能会问：既然实际上存在微小的加速度，那为什么不用二阶卡尔曼滤波器来处理这些变化呢？让我们来了解一下吧。

如何设计零阶、一阶或二阶卡尔曼滤波器呢？其实我们一直在这么做，只不过没有使用这些专门的术语而已。虽然过程可能有点繁琐，但我还是会详细解释每一种情况。如果你已经理解了这些概念，那就可以直接跳过不读。

零阶卡尔曼滤波器

零阶卡尔曼滤波器本质上是一种不涉及任何导数的滤波器。由于我们要追踪的是位置信息，因此只需要一个与位置相关的状态变量即可（无需考虑速度或加速度）。状态转换函数也仅与位置有关。用矩阵形式表示的话，状态变量就是……

x=[x]

状态转换函数非常简单。由于位置不会发生变化，因此我们只需对 x=x 进行建模即可；换句话说，时间 t+1 时的 x 值与时间 t 时的 x 值相同。用矩阵形式表示，状态转换函数为：

F=[1]

测量功能其实非常简单。我们需要确定如何将状态变量 x 转换为可测量的数值。我们假设所测量的数值为位置信息。由于状态变量中只包含位置信息，因此测量过程就变得很简单了。

H=[1]

让我们编写一个函数，用于构建并返回一个零阶卡尔曼滤波器。

```
def ZeroOrderKF(R, Q, P=20):
    """ Create zero order Kalman filter.
    Specify R and Q as floats."""

    kf = KalmanFilter(dim_x=1, dim_z=1)
    kf.x = np.array([0.])
    kf.R *= R
    kf.Q *= Q
    kf.P *= P
    kf.F = np.eye(1)
    kf.H = np.eye(1)
    return kf
```

一阶卡尔曼滤波器

一阶卡尔曼滤波器用于追踪一阶系统，比如位置和速度等参数。在上面关于狗的追踪问题中，我们已经应用了这一方法，所以应该很容易理解。不过，我们还是再演示一遍吧。

一阶系统包含位置和速度这两个参数，因此状态变量必须同时包含这两个量。其矩阵表示形式可以如下：

x=[x˙x]

现在，我们必须来设计状态转换的规则了。在某个时间步长内，牛顿方程如下：

xt=xt−1+vΔtvt=vt−1

请记住，我们需要将其转化为线性方程。

[x˙x]=F[x˙x]

设置

F=[1Δt01]

从而得到了上述方程式。

最后，我们设计了测量函数。该测量函数需要能够实现相应的功能。

z=Hx

我们的传感器目前仍然只能检测位置信息。因此，应该从状态数据中获取位置信息，而将速度和加速度的值设为 0。具体来说就是：

H=[10]

该函数用于构建并返回一阶卡尔曼滤波器。

```
def FirstOrderKF(R, Q, dt):
    """ Create first order Kalman filter.
    Specify R and Q as floats."""

    kf = KalmanFilter(dim_x=2, dim_z=1)
    kf.x = np.zeros(2)
```

```
    kf.P *= np.array([[100, 0], [0, 1]])
    kf.R *= R
    kf.Q = Q_discrete_white_noise(2, dt, Q)
    kf.F = np.array([[1., dt],
                     [0., 1]])
    kf.H = np.array([[1., 0]])
    return kf
```

二阶卡尔曼滤波器

二阶卡尔曼滤波器用于跟踪二阶系统，比如位置、速度和加速度等参数。其状态变量即为上述参数。

$$x=\begin{bmatrix}x\\\dot{x}\\\ddot{x}\end{bmatrix}$$

现在，我们必须来设计状态转换的规则了。在某个时间步长内，牛顿方程如下：

$$x_t=x_{t-1}+v_{t-1}\Delta t+0.5a_{t-1}\Delta t^2 v_t=v_{t-1}+a_{t-1}\Delta t a_t=a_{t-1}$$

请记住，我们需要将其转化为线性方程。

$$\begin{bmatrix}x\\\dot{x}\\\ddot{x}\end{bmatrix}=F\begin{bmatrix}x\\\dot{x}\\\ddot{x}\end{bmatrix}$$

设置

$$F=\begin{bmatrix}1&\Delta t&.5\Delta t^2\\0&1&\Delta t\\0&0&1\end{bmatrix}$$

从而得到了上述方程式。

最后，我们设计了测量函数。该测量函数需要能够实现相应的功能。

$$z=Hx$$

我们的传感器目前仍然只能检测位置信息。因此，应该从状态数据中获取位置信息，而将速度值设为 0。具体来说就是：

$$H=[1\,0\,0]$$

该函数用于构建并返回一个二阶卡尔曼滤波器。

```
def SecondOrderKF(R_std, Q, dt, P=100):
    """ Create second order Kalman filter.
    Specify R and Q as floats."""

    kf = KalmanFilter(dim_x=3, dim_z=1)
    kf.x = np.zeros(3)
    kf.P[0, 0] = P
    kf.P[1, 1] = 1
    kf.P[2, 2] = 1
    kf.R *= R_std**2
    kf.Q = Q_discrete_white_noise(3, dt, Q)
    kf.F = np.array([[1., dt, .5*dt*dt],
                     [0., 1.,       dt],
                     [0., 0.,       1.]])
    kf.H = np.array([[1., 0., 0.]])
    return kf
```

评估滤波器的阶数

现在，我们可以将每个卡尔曼滤波器应用于模拟结果中，然后对得到的结果进行评估。

我们该如何评估这些结果呢？我们可以通过绘制轨迹图和卡尔曼滤波器的输出结果来进行定性分析。不过，更严谨的方法则是运用数学公式来分析。需要记住的是，系统协方差矩阵包含了各个状态变量的方差和协方差。对角线上的数值就是各个状态的方差。如果噪声呈高斯分布的话，那么大约 99%的测量结果都会落在某个范围内。如果这一点不太清楚，请先复习一下高斯分布的相关内容，因为这是一个非常重要的概念。

因此，我们可以通过比较估计状态与实际状态之间的偏差，再将其与从 P 中得出的标准差进行对比，从而评估该滤波器的性能。如果滤波器运行正常，那么 99%的偏差应该落在 3σ 的范围内。这一规律适用于所有的状态变量，而不仅仅是位置信息。

必须指出的是，这一结论仅适用于模拟系统。真实的传感器并不遵循完美的高斯分布，因此你可能需要将评估标准调整为适用于真实传感器数据的参数值，比如 5σ 。

那么，让我们用一阶卡尔曼滤波器来处理这个一阶系统，并评估其性能如何。你或许能猜到，它的表现应该不错。不过，我们还是用标准差来具体分析一下吧。

首先，让我们编写一个程序来生成这些带有噪声的测量数据吧。

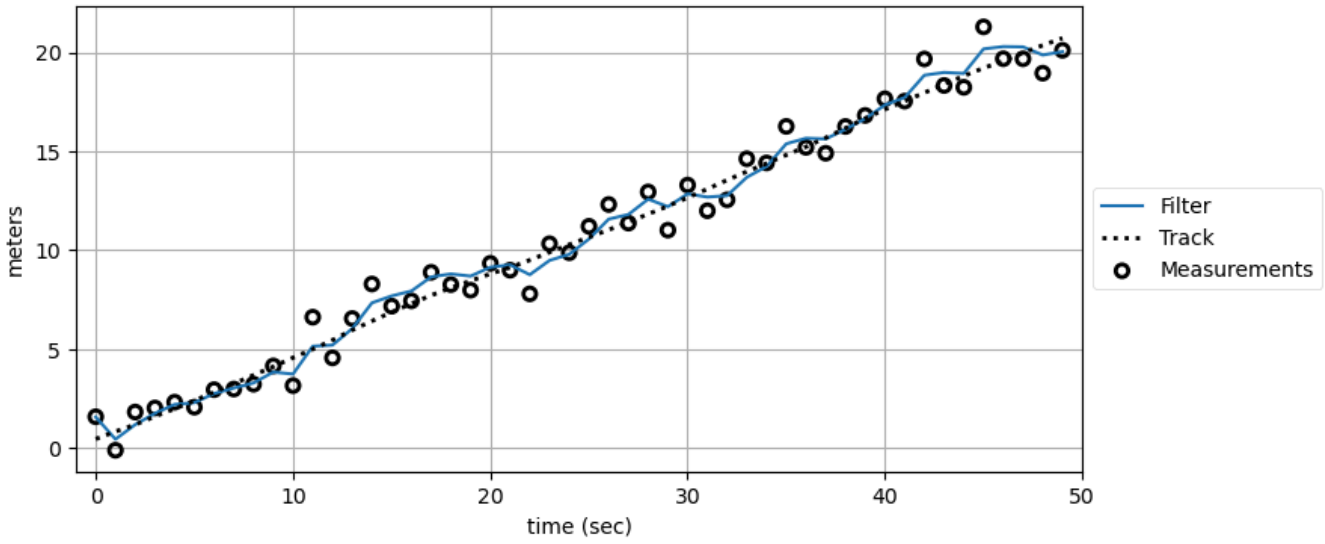
```
def simulate_system(Q, count):
    obj = ConstantVelocityObject(x0=.0, vel=0.5, noise_scale=Q)
    xs, zs = [], []
    for i in range(count):
        x = obj.update()
        z = sense(x)
        xs.append(x)
        zs.append(z)
    return np.array(xs), np.array(zs)
```

现在，我们来编写一个用于执行滤波操作并将结果保存到 `Saver` 对象中的程序。

```
from filterpy.common import Saver

def filter_data(kf, zs):
    s = Saver(kf)
    kf.batch_filter(zs, saver=s)
    s.to_array()
    return s
```

现在，我们已经准备好运行该滤波器并查看结果了。

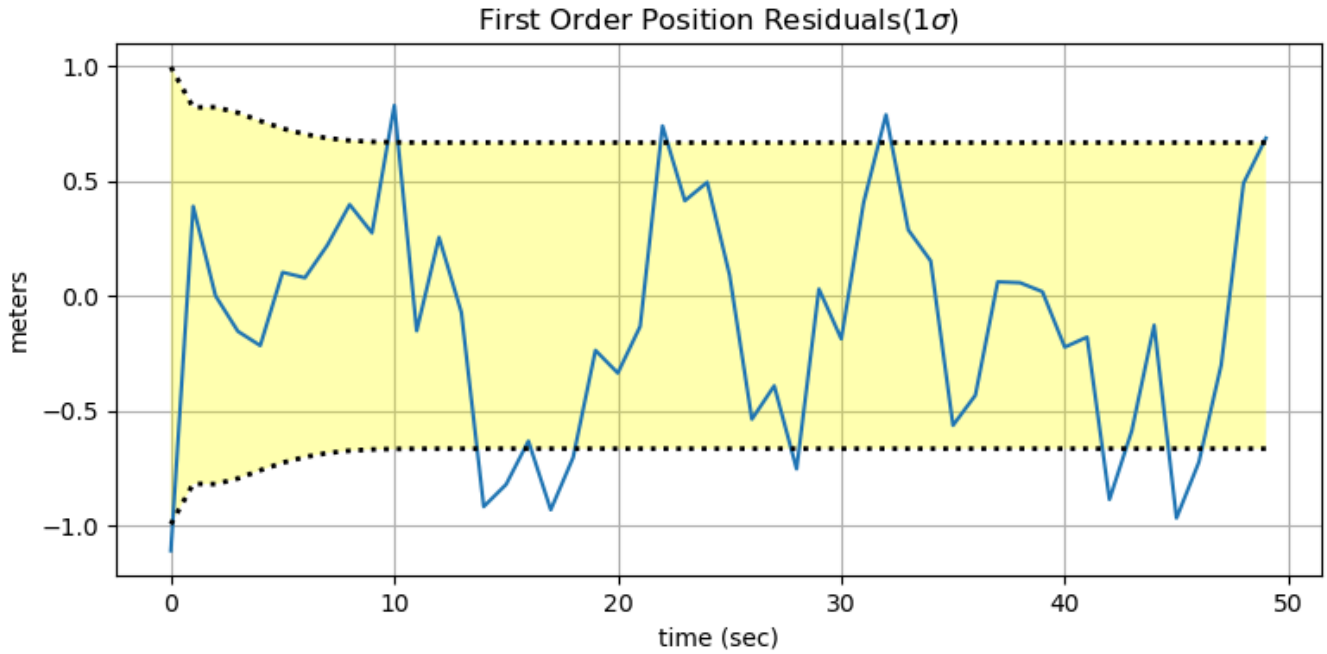
```
from kf_book.book_plots import plot_kf_output

R, Q = 1, 0.03
xs, zs = simulate_system(Q=Q, count=50)

kf = FirstOrderKF(R, Q, dt=1)
data1 = filter_data(kf, zs)

plot_kf_output(xs, data1.x, data1.z)
```



看起来，这个滤波器的性能相当不错，不过很难准确判断其具体的性能水平。让我们来看看残差数据，看看它们能否提供一些有用的信息。我们会经常这么做，所以我打算编写一个函数来绘制这些残差数据。

```
from kf_book.book_plots import plot_residual_limits, set_labels

def plot_residuals(xs, data, col, title, y_label, stds=1):
    res = xs - data.x[:, col]
    plt.plot(res)
    plot_residual_limits(data.P[:, col, col], stds)
    set_labels(title, 'time (sec)', y_label)
```

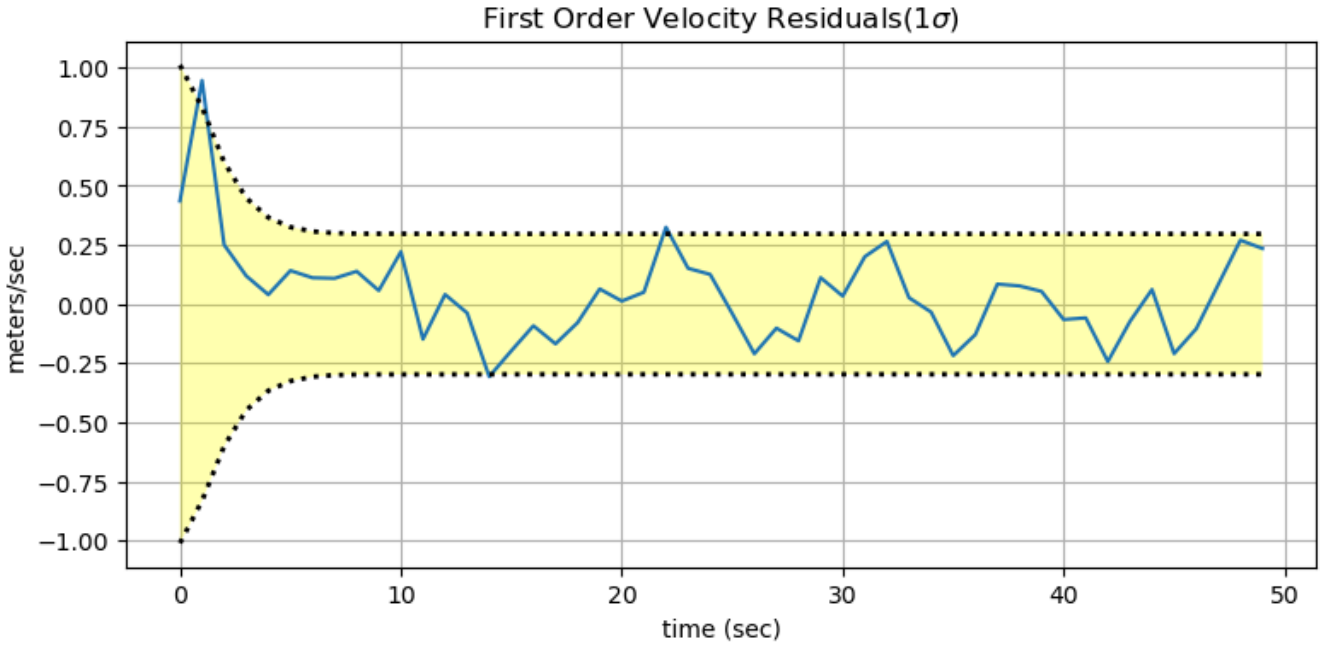
```
plot_residuals(xs[:, 0], data1, 0,
               title='First Order Position Residuals(1$\sigma$)',
               y_label='meters')
```



我们该如何解读这个图表呢？残差用锯齿状线条表示——也就是实际测量值与预测值之间的差异。如果不存在测量误差，且卡尔曼滤波器的预测始终准确无误，那么残差将始终为零。因此，理想的输出应该是一条位于 0 点的水平线。从图中可以看出，残差大致集中在 0 点附近，这表明误差呈高斯分布（因为误差在 0 点上下分布得比较均匀）。虚线之间的黄色区域代表了滤波器在 1 个标准差范围内的理论性能。换句话说，大约 68%的误差应该落在这一范围内。由于残差确实落在这一范围内，我们可以确认该滤波器的性能良好，没有出现偏差。

让我们来看看关于速度的残差情况。

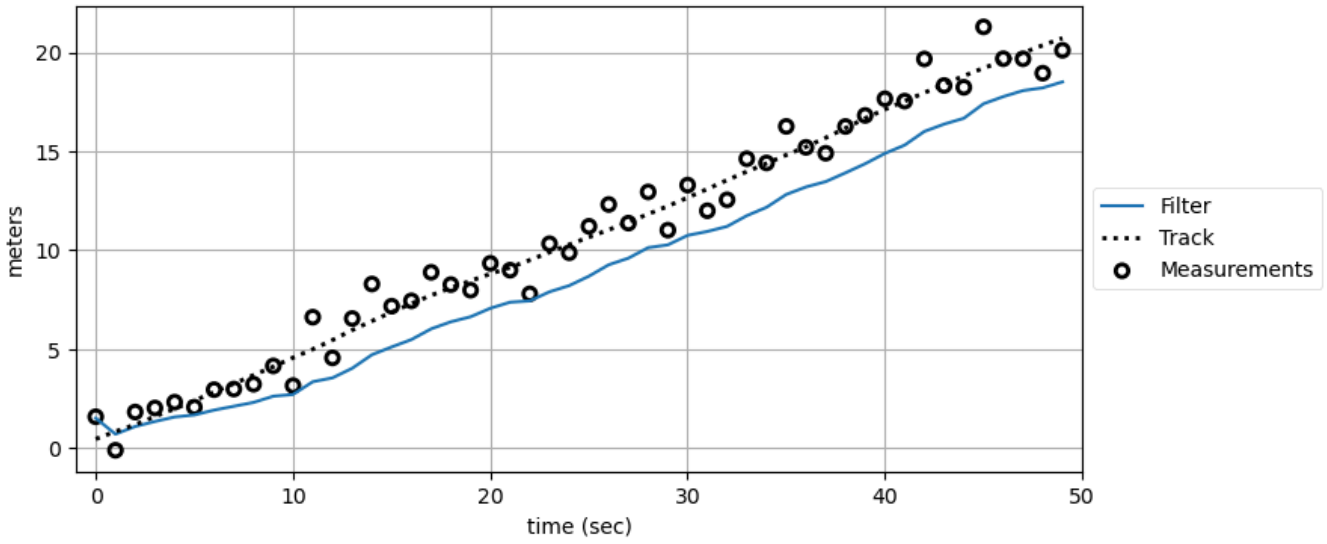
```
plot_residuals(xs[:, 1], data1, 1,
               title='First Order Velocity Residuals(1$\sigma$)',
               y_label='meters/sec')
```



同样，正如预期的那样，残差值都在该滤波器的理论性能范围内。因此，我们有信心认为，该滤波器非常适合用于该系统。

现在，让我们用零阶卡尔曼滤波器来做同样的事情。所有的代码和数学计算过程都差不多，所以我们只需查看结果即可，无需过多讨论实现细节。

```
kf0 = ZeroOrderKF(R, Q)
data0 = filter_data(kf0, zs)
plot_kf_output(xs, data0.x, data0.z)
```

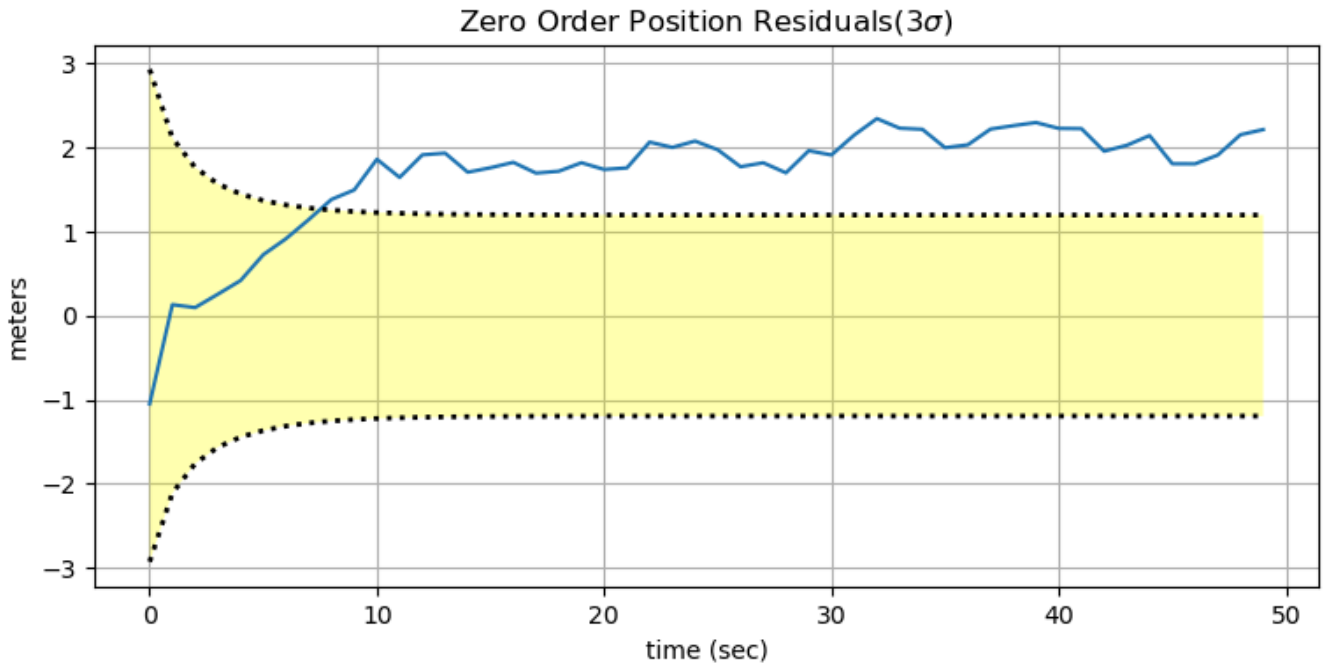


正如我们所预料的，该滤波器存在缺陷。回想一下 g-h 滤波器的情况：我们当时将加速度因素也纳入了滤波器的计算中。但由于缺乏足够的参数，g-h 滤波器始终无法及时响应速度的变化。在每一个时间步长上，卡尔曼滤波器都假设位置没有发生变化——如果当前位置是 4.3，那么它就会预测下一个时间点的位置仍然是 4.3。当然，实际位置应该更接近 5.3。由于测量值存在噪声，实际数值可能为 5.4。因此，滤波器会选取 4.3 和 5.4 之间的某个值作为估计结果，这就导致其数值与实际值 5.3 之间存在较大偏差。同样的情况会持续出现在后续的时间步长中。该滤波器永远无法跟上实际情况。

这引出了一个非常重要的问题。“恒定”这一假设，实际上只适用于离散样本之间的情况。滤波器的输出则可能会随着时间的推移而发生变化。

现在让我们来看看残差值。由于我们没有测量速度，因此只能查看与位置相关的残差值。

```
plot_residuals(xs[:, 0], data0, 0,
               title='Zero Order Position Residuals(3$\sigma$)',
               y_label='meters',
               stds=3)
```

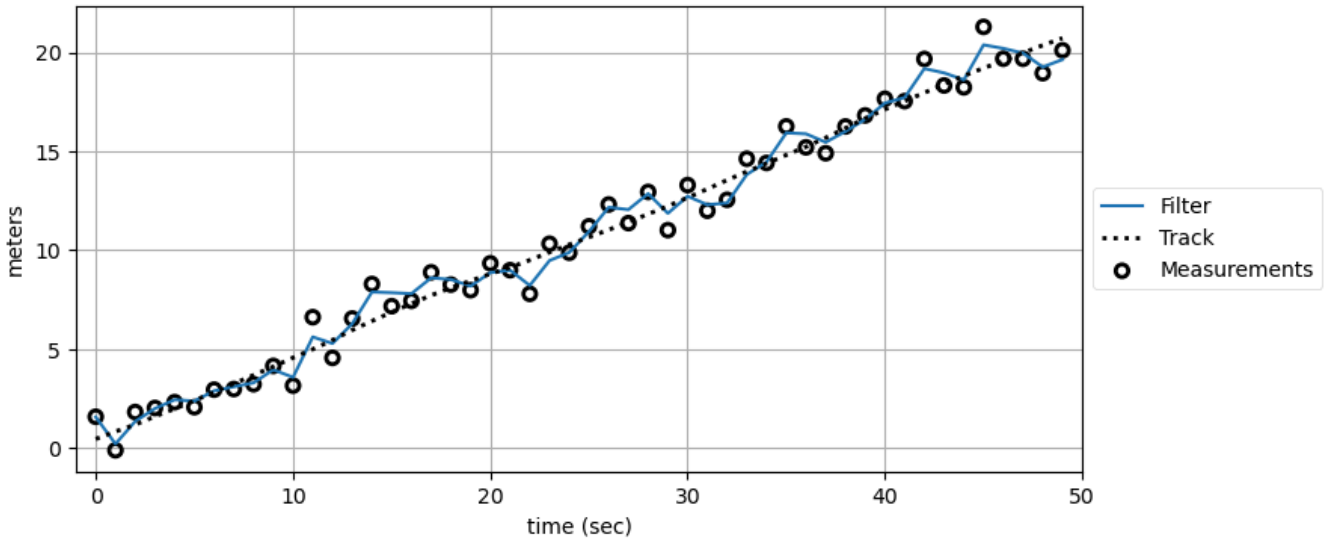


我们可以看到，该滤波器的性能几乎立刻就开始恶化了。几秒钟后，残差就超出了三个标准差的范围内。需要明白的是，协方差矩阵 P 所反映的只是假设所有输入数据都正确的情况下，该滤波器的理论性能。换句话说，虽然该卡尔曼滤波器的性能正在恶化，但 P 表明，随着时间的推移，其估计值其实会越来越准确，因为方差在逐渐减小。该滤波器无法察觉到你提供了关于系统的虚假信息。这种情况有时被称为“自大型滤波器”——因为它对自己的性能过于自信了。

在这个系统中，性能的差异是立竿见影且相当明显的。而在许多其他系统中，性能的差异则可能是渐进的，或者相当微小。为了确保滤波器的性能符合其理论上的预期，仔细分析这类图表是非常重要的。

现在，让我们来试试二阶模型。你可能会觉得这么做很合理。毕竟，我们知道，模拟物体的运动过程中存在一定的噪声，这意味着物体肯定有加速度。那为何不用二阶模型来描述这种加速度呢？如果没有加速度的话，那么加速度就应该被估计为 0，对吧？但实际情况真的如此吗？在继续之前，请先好好思考一下。

```
kf2 = SecondOrderKF(R, Q, dt=1)
data2 = filter_data(kf2, zs)
plot_kf_output(xs, data2.x, data2.z)
```

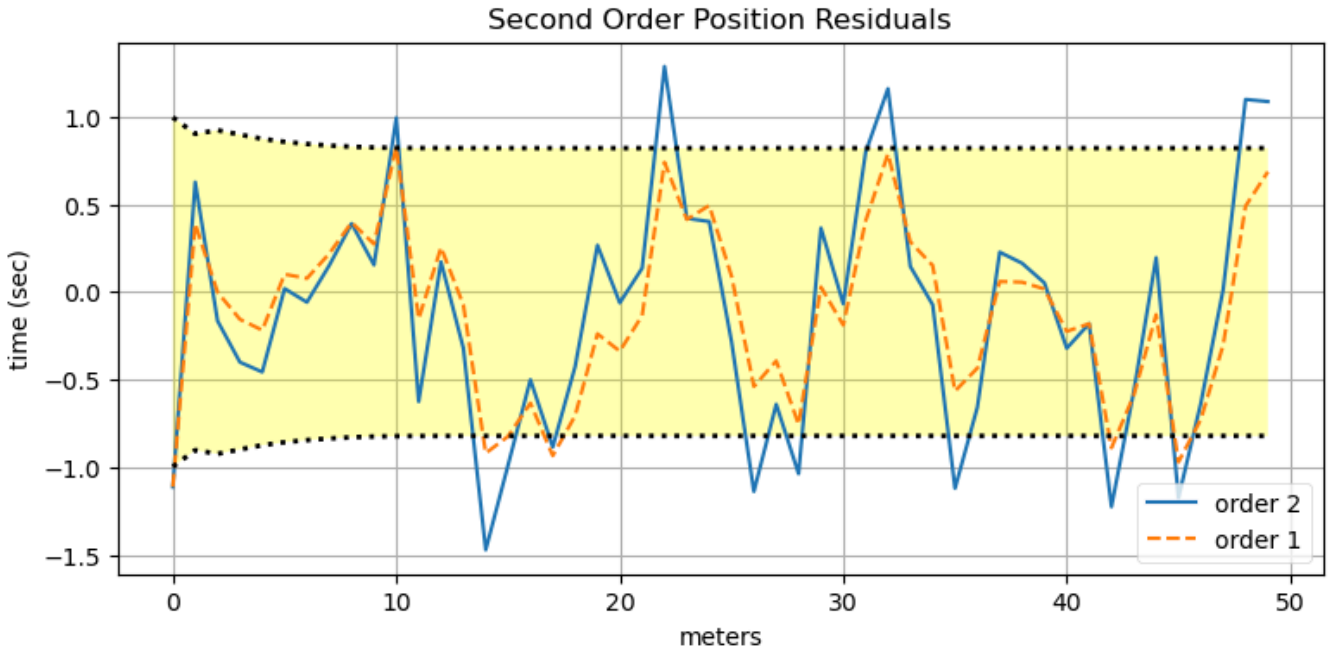


效果是否如你所预期?

可以看出，与一阶滤波器相比，二阶滤波器的性能要差得多。为什么呢？这种滤波器用于模拟加速度的变化，因此，当测量值出现较大波动时，系统会将其视为加速度而非噪声。这样一来，滤波器的输出结果就会与噪声的波动十分接近。更糟糕的是，如果噪声持续高于或低于某个水平，滤波器会过度响应这些噪声。因为滤波器错误地认为存在加速度，所以其预测结果会越来越偏离实际数值。这种情况显然不可取。

不过，这条曲线看起来还不算太糟糕。让我们来看看残差数据能告诉我们什么。这里有个需要注意的地方：二阶系统的残差情况还算不错，因为它们没有出现发散的情况，也没有超过三个标准差。不过，将一阶滤波器和二阶滤波器的残差数据进行比较会更有意义，所以我把两者都画在了同一张图上。

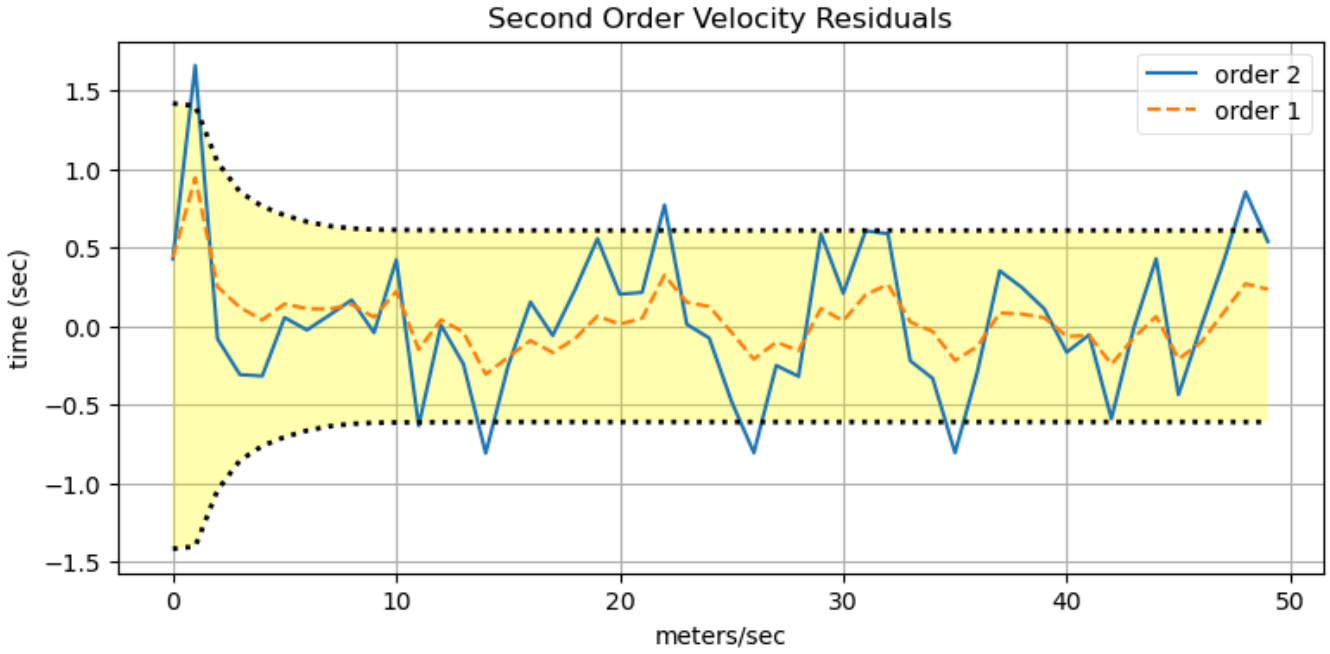
```
res2 = xs[:, 0] - data2.x[:, 0]
res1 = xs[:, 0] - data1.x[:, 0]

plt.plot(res2, label='order 2')
plt.plot(res1, ls="--", label='order 1')
plot_residual_limits(data2.P[:, 0, 0])
set_labels('Second Order Position Residuals',
           'meters', 'time (sec)')
plt.legend();
```



二阶滤波后的残差略高于一阶滤波后的残差，但仍然处于该滤波器的理论允许范围内。这并不值得过分担忧。

现在，让我们来看看速度的残差情况。

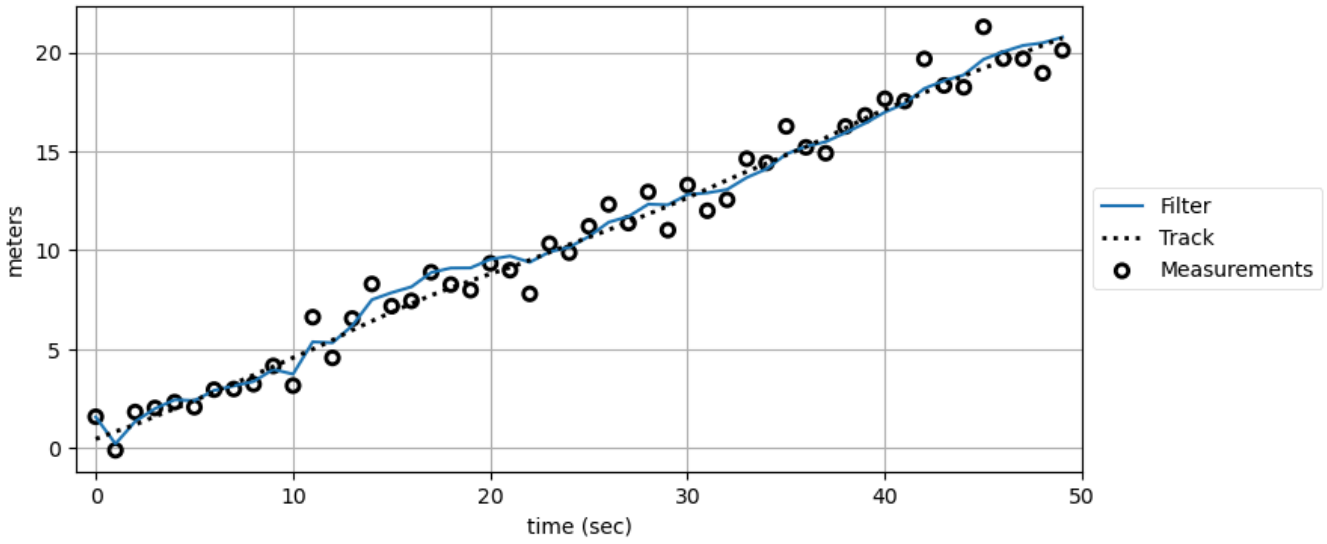
```
res2 = xs[:, 1] - data2.x[:, 1]
res1 = xs[:, 1] - data1.x[:, 1]

plt.plot(res2, label='order 2')
plt.plot(res1, ls='--', label='order 1')
plot_residual_limits(data2.P[:, 1, 1])
set_labels('Second Order Velocity Residuals',
                      'meters/sec', 'time (sec)')
plt.legend();
```



这里的情况则完全不同。虽然二阶系统的残差仍在滤波器性能的理论范围内，但我们可以看到，这些残差远远高于一阶滤波器的残差水平。这种情况其实是很常见的。滤波器错误地认为存在某种加速度，而实际上并不存在这种加速度。滤波器将测量中的噪声误认为是加速度，并将其纳入每次速度预测的结果中。当然，由于实际上并不存在加速度，因此速度的残差会远远大于其理想值。

我还有一个小窍门。我们所处理的系统是一阶系统，也就是说，物体的速度基本上是恒定的。不过，现实世界中的系统从来都不完美，因此，不同时间点上的速度肯定不会完全相同。当我们使用一阶滤波器时，我们会把这种微小的速度变化视为“过程噪声”来处理。矩阵 Q 就是用来处理这种微小变化的。如果我们改用二阶滤波器，那么就能更准确地反映速度的变化了。也许到那时，就不存在所谓的“过程噪声”了，我们就可以把 Q 的值设为零！

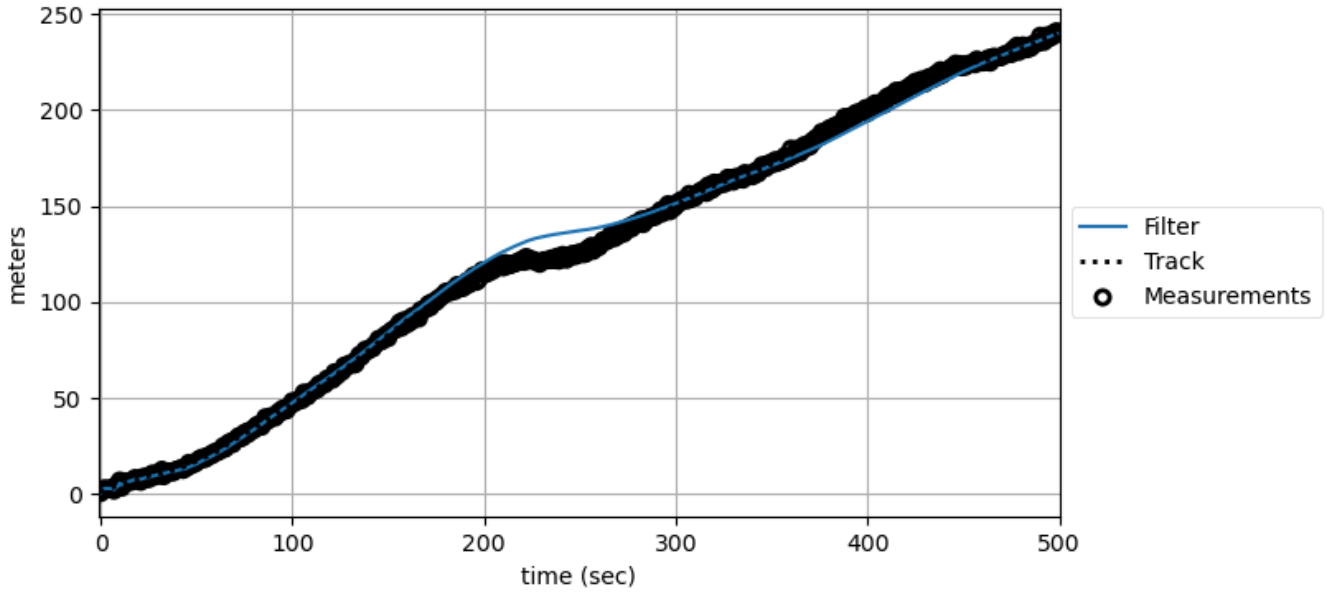
```
kf2 = SecondOrderKF(R, 0, dt=1)
data2 = filter_data(kf2, zs)
plot_kf_output(xs, data2.x, data2.z)
```

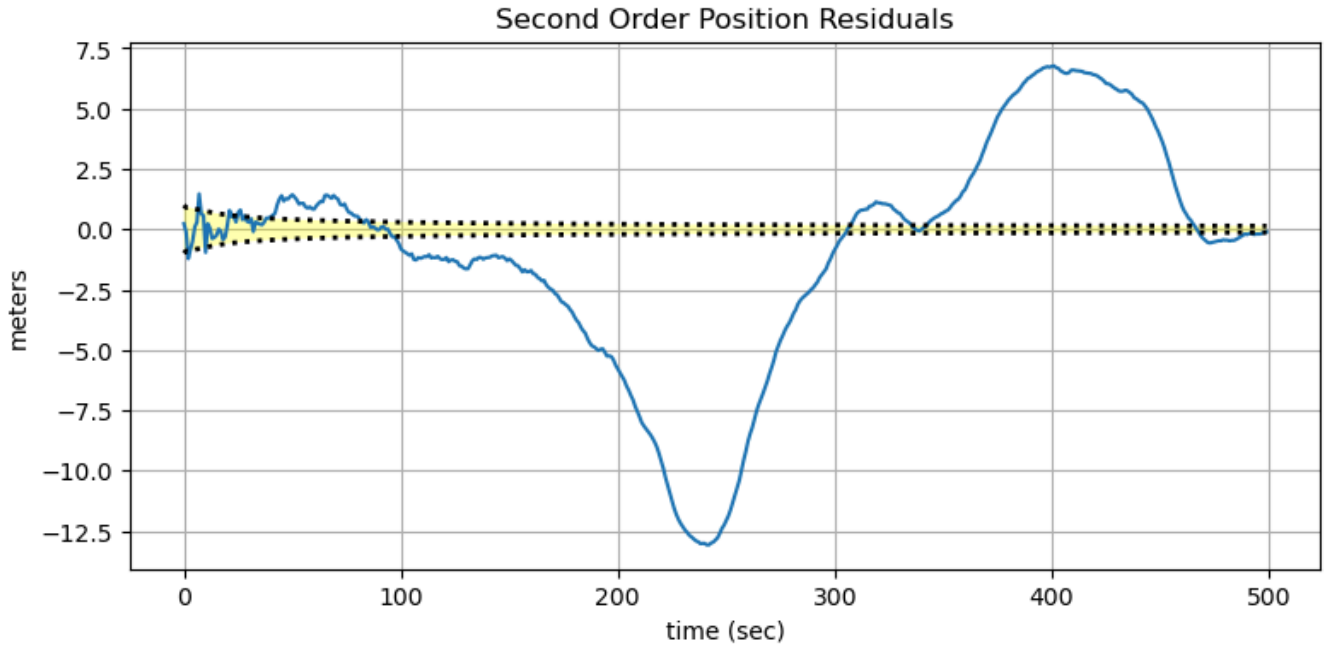


在我看来，该滤波器能够很快地与实际轨迹相吻合。成功了！

或者，也未必如此。将过程噪声设置为 0，就相当于告诉滤波器：该过程模型是完美的。让我们来观察一下该滤波器在更长时间内的表现吧。

```
np.random.seed(25944)
xs500, zs500 = simulate_system(Q=Q, count=500)

kf2 = SecondOrderKF(R, 0, dt=1)
data500 = filter_data(kf2, zs500)

plot_kf_output(xs500, data500.x, data500.z)
plot_residuals(xs500[:, 0], data500, 0,
               'Second Order Position Residuals',
               'meters')
```





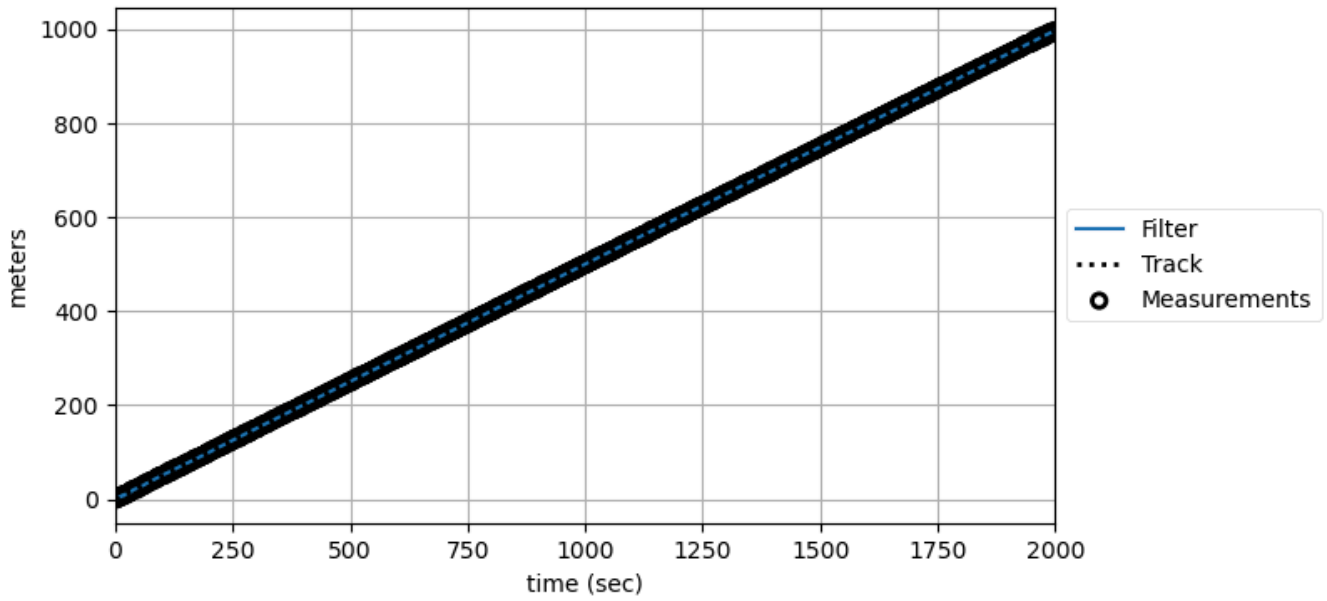
可以看出，该滤波器的性能糟糕透顶。从轨迹图中可以看出，滤波器的输出与理论轨迹有很长一段时间都存在较大偏差。残差图则更清楚地反映了这个问题。在第 100 次更新之前，滤波器的输出与理论值出现了巨大偏差。虽然最后滤波器似乎有所收敛，但我对此表示怀疑。在整个过程中，滤波器所报告的方差值越来越小。千万不要相信滤波器的协方差矩阵来判断其性能如何！

为什么会这样呢？请回想一下：如果我们把过程噪声设置为零，那就相当于告诉滤波器只使用过程模型来进行处理。这样一来，测量数据就被忽略了。物理系统本身并不完美，因此滤波器无法适应这种不完美的状况。

也许只是工艺上的噪音太大了而已？那就试试看吧。

```
np.random.seed(32594)
xs2000, zs2000 = simulate_system(Q=0.0001, count=2000)

kf2 = SecondOrderKF(R, 0, dt=1)
data2000 = filter_data(kf2, zs2000)
```

```
plot_kf_output(xs2000, data2000.x, data2000.z)
plot_residuals(xs2000[:, 0], data2000, 0,
               'Second Order Position Residuals',
               'meters')
```

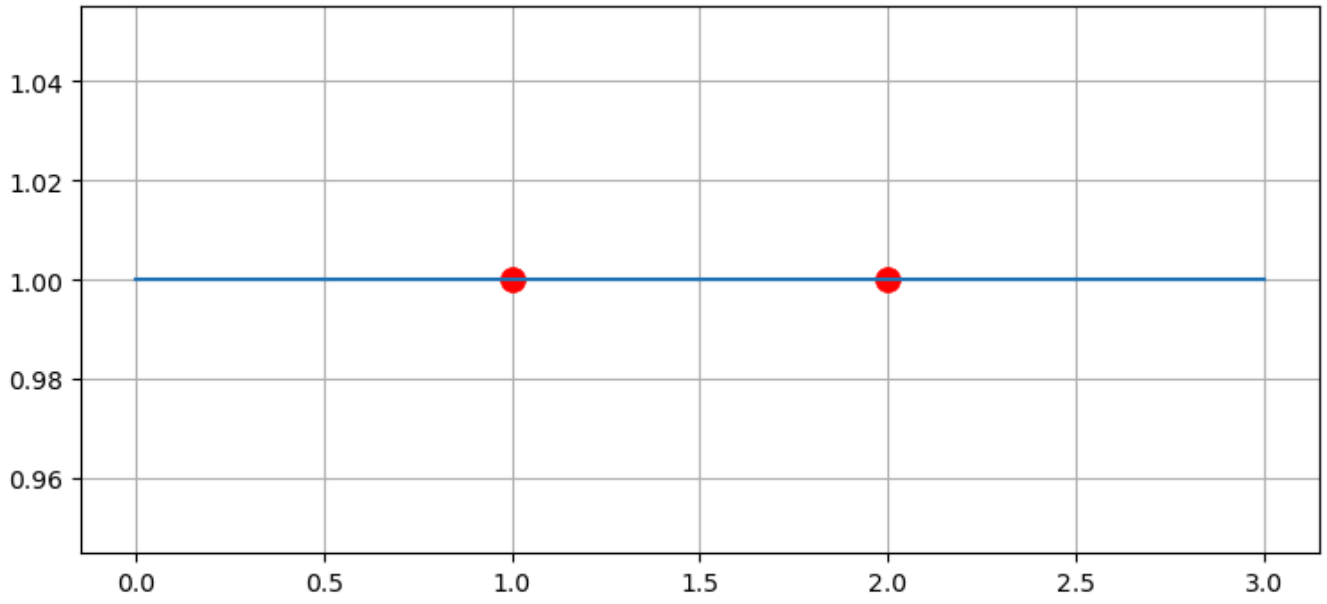


同样，残差图也揭示了问题所在。从曲线来看，效果相当不错，但残差图显示，滤波器在相当长的时间内都处于不稳定状态。

你该如何看待这一切呢？你可能会认为，最后的那个滤波器效果已经“足够好了”，也许确实如此。不过我得提醒你：发散型的滤波器并不一定会收敛。如果使用的是不同的数据集，或者面对的是行为表现不同的物理系统，那么最终得到的滤波器可能会越来越偏离实际的测量结果。

另外，让我们从数据拟合的角度来思考这个问题。假设我给你两个点，要求你用一条直线来拟合这两个点。

```
plt.scatter([1, 2], [1, 1], s=100, c='r')
plt.plot([0, 3], [1, 1]);
```



直线是唯一的正确答案。此外，这个答案也是最优解。如果我给你更多的点数，你可以用最小二乘法来找出最合适的直线，但从最小二乘法的意义上来说，得到的结果依然是最优的。

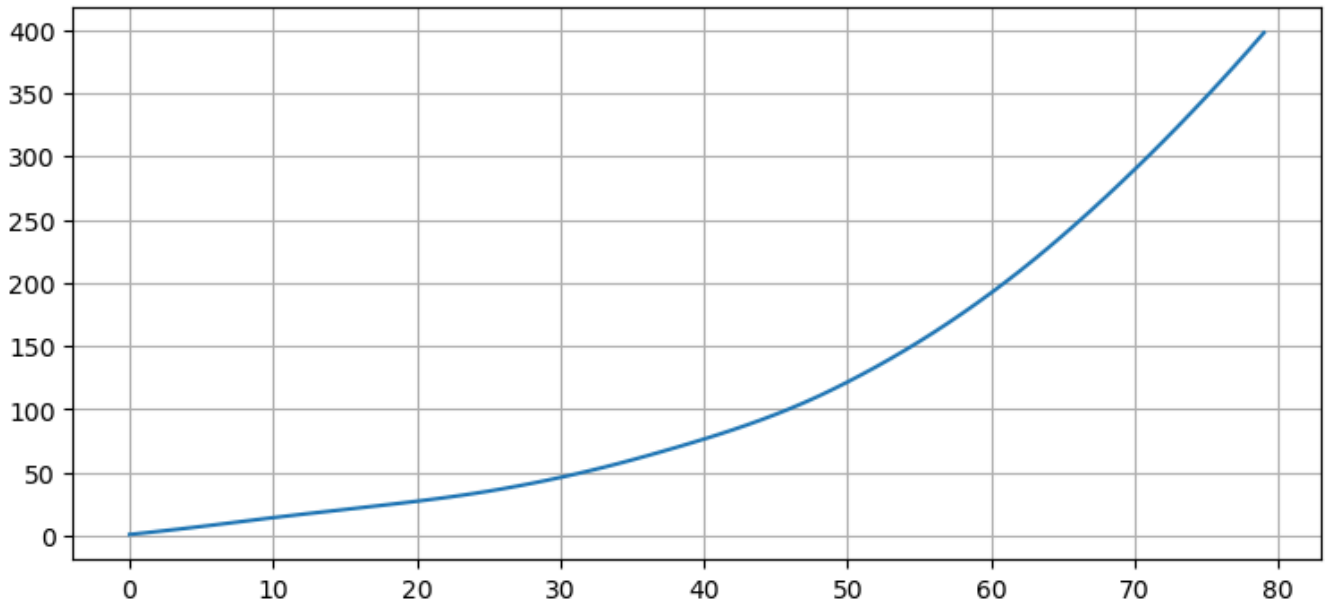
但是，假如我要求你用更高阶的多项式来拟合那两个点呢？对于这个问题，有无数种可能的解。例如，有无数条二次抛物线可以穿过那两个点。当卡尔曼滤波器的阶数高于实际物理过程的阶数时，同样也有无数种解可供选择。这样的解不仅不是最优解，而且往往会导致计算结果发散，永远无法得到稳定的结果。

为了获得最佳性能，所使用的滤波器的阶数必须与被监测对象的动态特性相匹配。在很多情况下，这并不难实现。例如，如果要用卡尔曼滤波器来检测冷冻柜内的温度，那么显然，零阶滤波器就是最佳选择。但是，如果我们想要监测汽车的运行情况呢？当汽车以恒定速度直线行驶时，一阶滤波器就足够了。不过，汽车还会转弯、加速或减速，这种情况下，二阶滤波器的性能会更好。这就是“自适应滤波器”这一章节要探讨的问题。在这一章中，我们将学习如何设计出能够适应被监测对象行为变化的滤波器。

也就是说，只要加入足够的“过程噪声”，并保持足够的离散化频率（通常来说，每秒 100 个样本就足以保证系统的局部线性特性），低阶滤波器仍然能够有效追踪高阶过程。虽然得到的结果不会是最理想的，但仍然相当不错。在尝试使用自适应滤波器之前，我总会先使用这种滤波器。下面以加速度为例来说明。首先来看模拟结果。

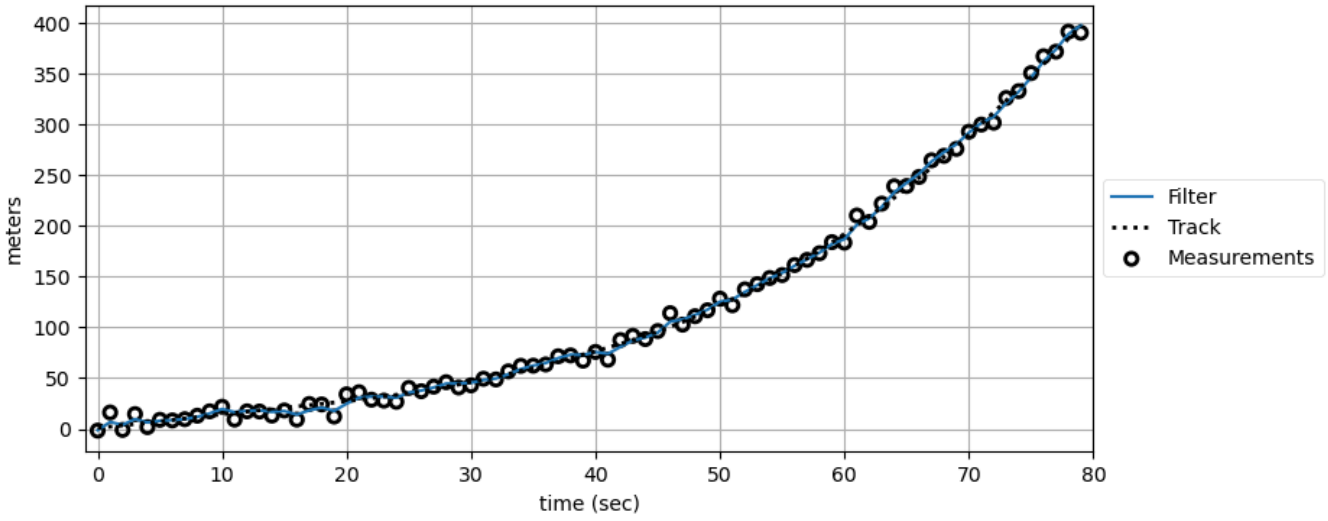
```
class ConstantAccelerationObject(object):
    def __init__(self, x0=0, vel=1., acc=0.1, acc_noise=.1):
        self.x = x0
        self.vel = vel
        self.acc = acc
        self.acc_noise_scale = acc_noise

    def update(self):
        self.acc += randn() * self.acc_noise_scale
        self.vel += self.acc
        self.x += self.vel
        return (self.x, self.vel, self.acc)
```

```
R, Q = 6., 0.02
def simulate_acc_system(R, Q, count):
    obj = ConstantAccelerationObject(acc_noise=Q)
    zs = []
    xs = []
    for i in range(count):
        x = obj.update()
        z = sense(x, R)
        xs.append(x)
        zs.append(z)
    return np.asarray(xs), zs

np.random.seed(124)
xs, zs = simulate_acc_system(R=R, Q=Q, count=80)
plt.plot(xs[:, 0]);
```



现在，我们将使用二阶滤波器来对数据进行滤波。

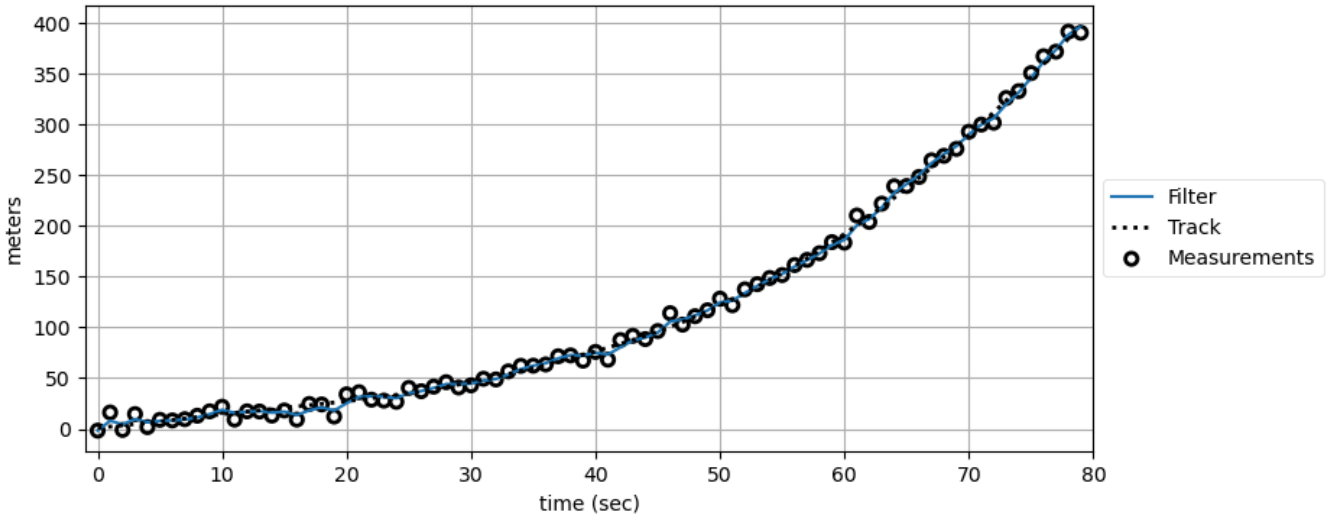
```
np.random.seed(124)
xs, zs = simulate_acc_system(R=R, Q=Q, count=80)

kf2 = SecondOrderKF(R, Q, dt=1)
data2 = filter_data(kf2, zs)

plot_kf_output(xs, data2.x, data2.z, aspect_equal=False)
plot_residuals(xs[:, 0], data2, 0,
               'Second Order Position Residuals',
               'meters')
```



可以看出，该滤波器的性能符合其理论上的极限值。

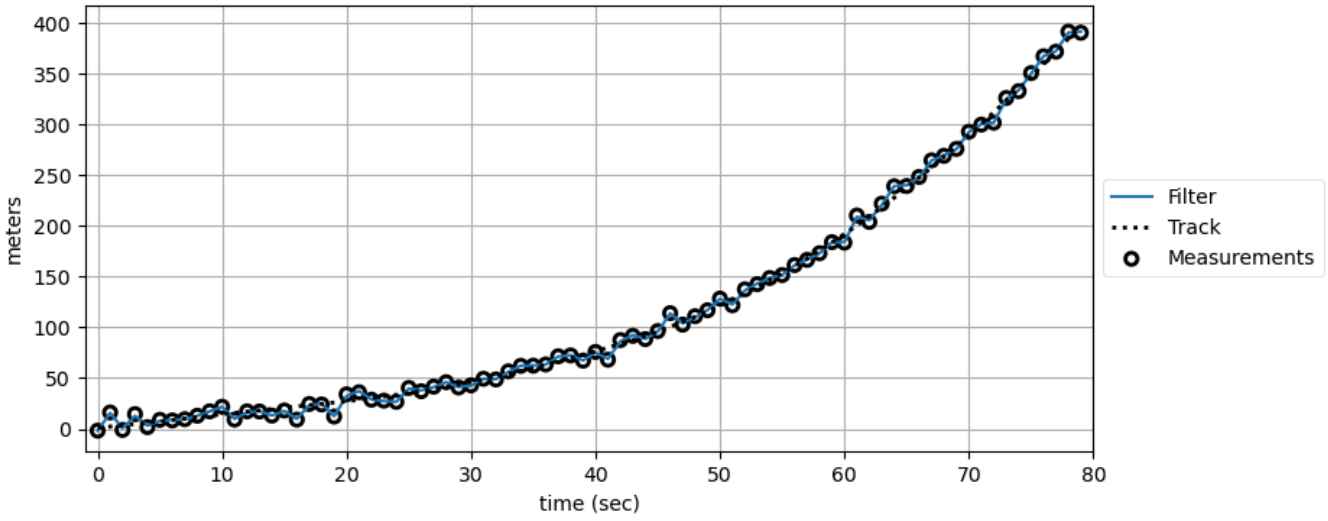
现在，让我们使用低阶滤波器。正如之前所演示的，低阶滤波器会导致信号出现延迟，因为它无法准确模拟加速度的变化。不过，我们可以通过增加“过程噪声”的大小来在一定程度上弥补这一缺陷。滤波器会将加速度视为过程模型中的噪声。虽然这样处理得到的结果并非最优，但如果设计得当，系统也不会出现异常行为。确定“过程噪声”的具体数值并非易事，需要通过实验来找到合适的数值。在这里，我将“过程噪声”增加了 10 倍，效果相当不错。

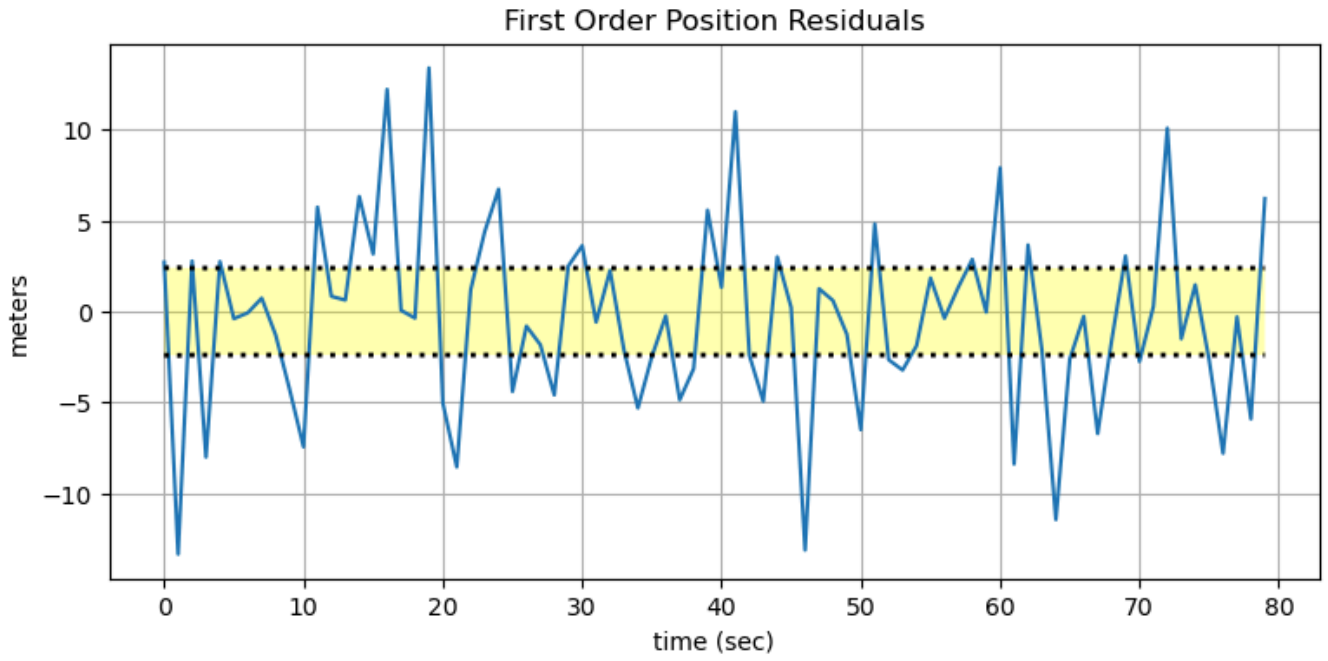
```
kf3 = FirstOrderKF(R, Q * 10, dt=1)
data3= filter_data(kf3, zs)

plot_kf_output(xs, data3.x, data3.z, aspect_equal=False)
plot_residuals(xs[:, 0], data3, 0,
               'First Order Position Residuals',
               'meters')
```



试想一下，如果将处理过程中的噪声水平提高到远高于实际所需的程度，会发生什么。较高的噪声水平会促使滤波器优先选择那些与噪声相似的测量结果。因此，我们可以预期，滤波器处理后的结果会与测量过程中的噪声非常相似。让我们来验证一下吧。

```
kf4 = FirstOrderKF(R, Q * 10000, dt=1)
data4 = filter_data(kf4, zs)

plot_kf_output(xs, data4.x, data4.z, aspect_equal=False)
plot_residuals(xs[:, 0], data4, 0,
               'First Order Position Residuals',
               'meters')
```





练习：状态变量设计

正如我所说，你可以按照自己喜欢的任何顺序来排列 、 x 中的变量。例如，你可以将一维恒定加速度定义为 x=[¨xx˙x]T 。虽然我不明白为什么要那样排列，但这种做法是可行的。

让我们来设计一些更合理的方案吧。为在二维空间中移动的机器人设计一个二阶滤波器吧。在这一章中，我们一直使用的是 x=[x˙xy˙y]T 这种参数设置。

为什么要选择不同的排序方式呢？正如您马上会看到的，改变 x 的顺序，会影响到滤波器中其他大部分矩阵的顺序。根据您想要分析的数据类型，比如 P 中的相关性，不同的 x 排序方式可能会让分析变得更简单或更复杂。

仔细想想该怎么做。需要做出哪些改变呢？显然，你只需要调整卡尔曼滤波器中的矩阵，以反映这一新的设计要求即可。

请尝试使用下面的模板代码来操作：

```
N = 30 # number of iterations
dt = 1.0 # time step
R_std = 0.35
Q_std = 0.04

sensor = PosSensor((0, 0), (2, .5), noise_std=R_std)
zs = np.array([sensor.read() for _ in range(N)])

tracker = KalmanFilter(dim_x=4, dim_z=2)
# assign state variables here

xs, ys = [], []
for z in zs:
    tracker.predict()
    tracker.update(z)
    xs.append(tracker.x[0])
    ys.append(tracker.x[1])
plt.plot(xs, ys);
```

解决方案

我们先从 F 开始吧。稍加练习后，你应该就能直接写出该矩阵了。如果你觉得有困难，那就试着把与 F 相关的方程式写出来，同时要按照状态变量所使用的顺序来排列各个变量。

x=1x+0y+1·xΔt+0·yΔty=0x+1y+0·xΔt+1·yΔt·x=0x+0y+1·xΔt+0·yΔt·y=0x+0y+0·xΔt+1·yΔt

我们可以把系数抄录下来。

F=⎡⎢⎣10Δt0010Δt00100001⎤⎥⎦

状态噪声也要求对各种元素进行重新排列。首先，需要确定状态变量的排列方式。如果将状态变量以垂直或水平的方式排列在矩阵中，就能更容易地理解它们之间的对应关系。不过，在 Jupyter 笔记本中很难做到这一点，所以我在这里就不详细介绍了。

Q=⎡⎢⎢⎣σ2xσxyσx·xσx·yσyxσ2yσy·xσy·yσ·xxσ·xyσ2·xσ·x·yσ·yxσ·yyσ·y·xσ2·y⎤⎥⎥⎦

x 与 y 之间没有关联，因此我们可以将涉及这两者的各项都设为零。

Q=⎡⎢⎢⎣σ2x0σx·x00σ2y0σy·yσ·xx0σ2·x00σ·yy0σ2·y⎤⎥⎥⎦

现在你看到这个之后，就能更快速地理解其中的规律，从而更轻松地设计出 Q 这样的作品了。

`Q_discrete_white_noise` 生成的矩阵中的元素排列方式不同，但我们只需复制其中的元素即可，具体操作会在下面的代码中展示。

现在让我们来设计 H 。它能够将状态 [xy·x·y]T 转换为测量结果 z=[zxzy]T 。

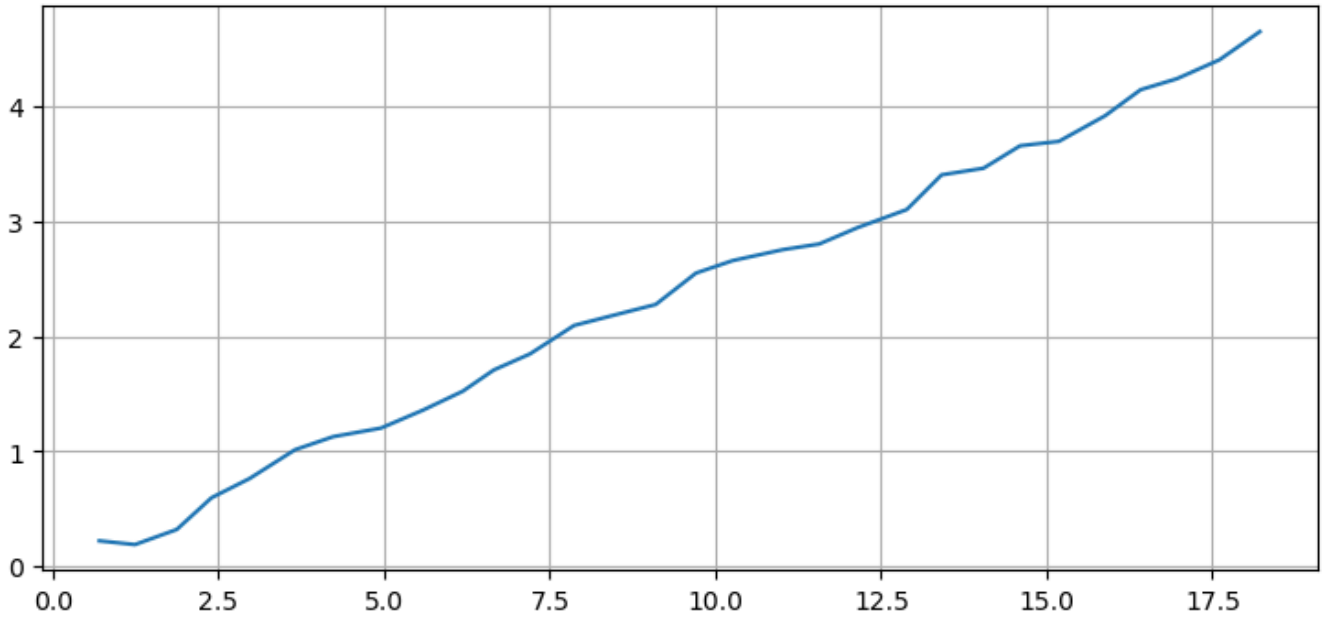
Hx=z[????????]⎡⎢⎢⎣xy·x·y⎤⎥⎥⎦=[zxzy]

现在，填写这个矩阵应该很容易了：

z=[zxzy]T 的数值没有变化，因此 R 也保持不变。

最后是 P 。它的排序方式与 Q 相同，所以其实本来就是为我们而设计的。

```
N = 30 # number of iterations
dt = 1.0 # time step
R_std = 0.35
Q_std = 0.04

M_TO_FT = 1 / 0.3048

sensor = PosSensor((0, 0), (2, .5), noise_std=R_std)
zs = np.array([sensor.read() for _ in range(N)])

tracker = KalmanFilter(dim_x=4, dim_z=2)

tracker.F = np.array([[1, 0, dt,  0],
                      [0, 1,  0, dt],
                      [0, 0,  1,  0],
                      [0, 0,  0,  1]])

tracker.H = np.array([[M_TO_FT, 0, 0, 0],
                      [0, M_TO_FT, 0, 0]])

tracker.R = np.eye(2) * R_std**2
q = Q_discrete_white_noise(dim=2, dt=dt, var=Q_std**2)
tracker.Q[0,0] = q[0,0]
tracker.Q[1,1] = q[0,0]
tracker.Q[2,2] = q[1,1]
tracker.Q[3,3] = q[1,1]
tracker.Q[0,2] = q[0,1]
tracker.Q[2,0] = q[0,1]
tracker.Q[1,3] = q[0,1]
tracker.Q[3,1] = q[0,1]

tracker.x = np.array([[0, 0, 0, 0]]).T
tracker.P = np.eye(4) * 500.

xs, ys = [], []
for z in zs:
    tracker.predict()
    tracker.update(z)
    xs.append(tracker.x[0])
    ys.append(tracker.x[1])
plt.plot(xs, ys);
```



检测并剔除不良测量结果

卡尔曼滤波器无法识别和剔除那些错误的测量数据。假设你正在追踪某架飞机，但你收到的测量数据实际上来自距离该飞机当前位置 100 公里以外的地方。如果你用这个错误的测量值来更新位置估计，那么新的估计值将会严重偏离实际位置。

我会进行一次模拟，以便给出一个具体的例子。在 100 个时间步之后，我会进行一次更新，更新后的数值为当前位置的两倍。 `filterpy.common` 会为 `kinematic_kf` 提供数据，从而形成一个任意维度和阶数的线性运动滤波器。在这里我使用这个滤波器是为了让代码更简洁；在本书的其余部分，我没有使用它，因为我希望你们能通过大量练习来掌握如何编写滤波器。

```
from filterpy.common import kinematic_kf

kf = kinematic_kf(dim=2, order=1, dt=1.0, order_by_dim=False)
kf.Q = np.diag([0, 0, .003, .003])
kf.x = np.array([[1., 1., 0., 0.]]).T
kf.R = np.diag([0.03, 0.21]) # use different errors

for i in range(101):
    kf.predict()
    kf.update(np.array([[i*.05, i*.05]])) # around 200 kph

p0 = kf.x[0:2]

kf.predict()
prior = kf.x
z = kf.x[0:2]*2
kf.update(z)
p1 = kf.x[0:2]

# compute error of measurement from prior
y = np.abs(z - kf.H @ prior)
dist = np.linalg.norm(y)

np.set_printoptions(precision=2, suppress=True)
```

```
print(f'bad measurement       : {z.T} km')
print(f'before bad measurement: {p0.T} km')
print(f'after bad measurement : {p1.T} km')
print(f'estimate shift        : {np.linalg.norm(p1 - prior[:2]):.1f} km')
print(f'distance from prior   : {dist:.1f} km')
```

```
bad measurement       : [[10.1 10.1]] km
before bad measurement: [[5. 5.]] km
after bad measurement : [[7.84 7.01]] km
estimate shift        : 3.4 km
distance from prior   : 7.1 km
```

kinematic_kf？那是什么？ filterpy.common 提供了 kinematic_kf 功能，使得用户能够创建任意维度和阶数的线性运动学滤波器。在本书中，我没有使用这个功能，因为我希望你们能通过实践来积累创建卡尔曼滤波器的经验。我在这里使用它，只是为了简化示例内容，同时让你们了解该库的这一功能而已。

回到正题。如您所见，估算值上升了 3.4 公里，而预测值与实际测量值之间的误差则超过了 7 公里。

我们能做些什么来避免这种情况呢？首先，我们可以检查一下：先前的估计值与实际测量值之间的差距是否过大。为什么要检查先前的估计值，而不是当前的估计值呢？因为经过更新后，当前的估计值可能会非常接近那个有误的测量值，不过在这个例子中并非如此。

请注意，虽然我本可以直接使用 prior[0:2] - z 来获取误差值，但我选择了在数学上更正确的表达方式 z−Hx 来计算误差。这样做只是为了便于说明而已。实际上，卡尔曼滤波器类会将误差值存储在 KalmanFilter.y 中。我使用 KalmanFilter.y 中的数值来代替上面计算出的结果，以便更清楚地说明这个问题。

```
print(f'error = {np.linalg.norm(kf.y):.1f} km, at a speed of {dist*3600:.0f} kph')
```

```
error = 7.1 km, at a speed of 25710 kph
```

在这个例子中，实际测量值与预测位置的偏差接近 7 公里。这听起来似乎“很远”。但真的如此吗？如果以公里为单位，更新频率为每秒 1 次的话，那确实很远——因为没有任何飞机能够以超过 25000 公里/小时的速率飞行。不过，如果以厘米为单位，更新频率为每分钟 1 次的话，那么这个偏差其实就微不足道了。

我们可以添加一项检查机制，该机制会考虑到飞机的性能限制因素。

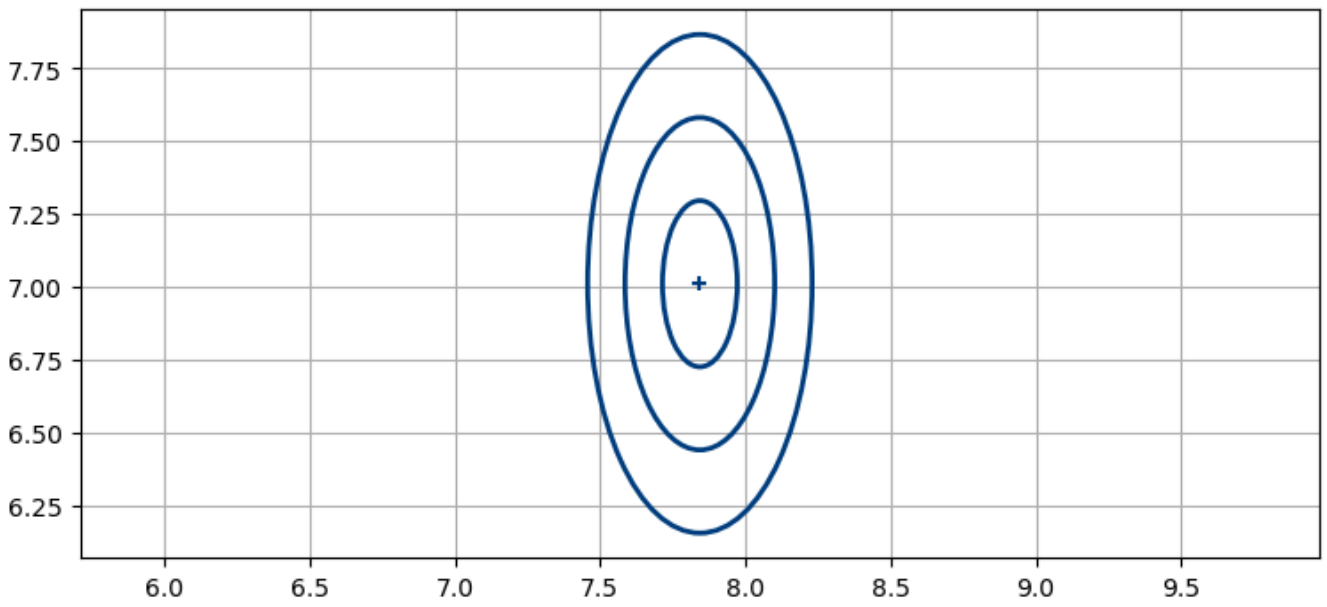
```
vel = y / dt
if vel >= MIN_AC_VELOCITY and vel <= MAX_AC_VELOCITY:
    kf.update()
```

你认为这是一个合理且可行的解决方案吗？在继续阅读之前，请尽可能多地想出各种反对意见吧。

这让我很不满意。假设我们只是用某个猜测的位置来初始化滤波器的话，那么所有有效的测量数据都会被丢弃，滤波过程也就无法开始。其次，这种方法完全忽略了我们对传感器及测量过程中可能出现的误差的了解。卡尔曼滤波器能够保持其当前的精确度。如果 P 表明 σx=10 米，而实际测量结果却是 1 公里，那么这个测量结果显然是不可信的，因为它偏离了之前的测量结果 100 个标准差。

让我们来绘制 P 的图表吧。我会分别绘制出第一、第二和第三标准差。

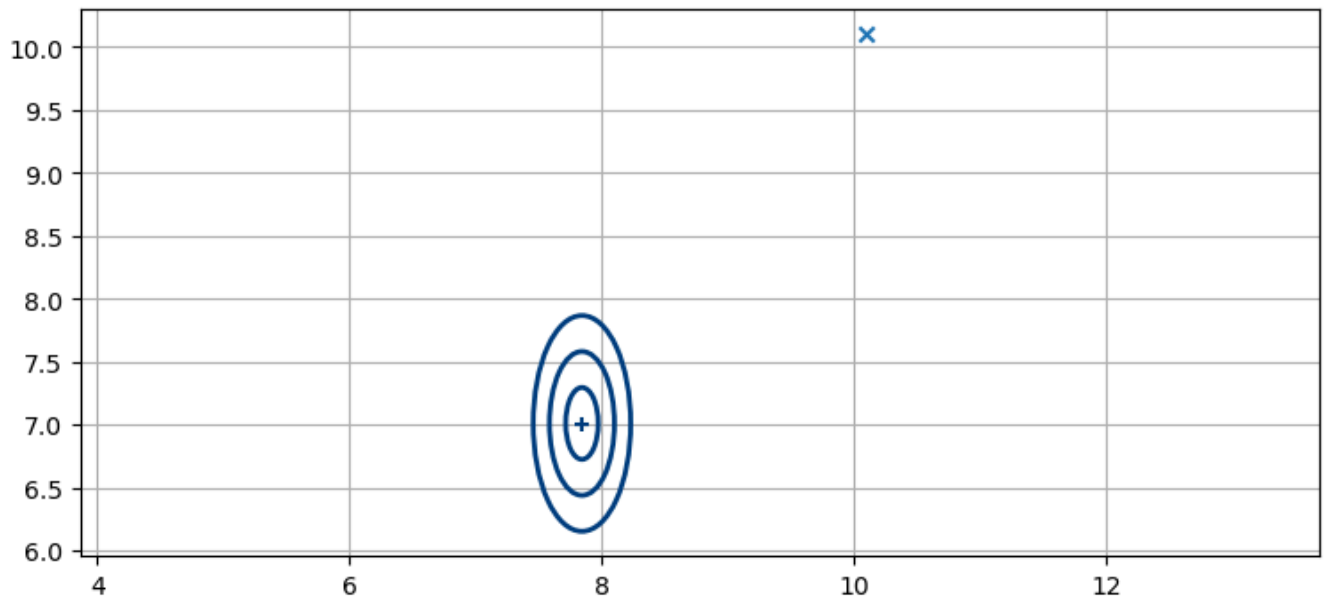
```
x, P = kf.x[0:2], kf.P[0:2, 0:2]
plot_covariance_ellipse(x, P, std=[1,2,3])
```



在之前的章节中， P 实际上是一个圆，而非椭圆。在我的代码中，我将 R=[0.03000.15] 的值设定为某个特定数值，这样一来， y 的测量结果就会包含 5 倍于 x 的误差。这种处理方式有些人为性，但在后续的章节中，几乎所有的协方差都会是椭圆形的，这是有充分理由的。

仔细想想这意味着什么。统计数据显示，99%的测量结果都落在 3 个标准差范围内；也就是说，99%的测量值都应该位于这个椭圆之内。让我们把测量结果与这个椭圆一起绘制出来吧。

```
plot_covariance_ellipse(x, P, std=[1,2,3])
plt.scatter(z[0], z[1], marker='x');
```



显然，这个测量结果远远超出了先验值所对应的协方差范围。我们或许应该将这个测量结果视为无效数据，不予使用。那我们该怎么做呢?

首先，我们需要计算 、 x 和 、 y 的标准差，然后编写相应的 if 语句来处理这些数据。在这里，我会利用 `KalmanFilter` 类的另一个功能。 `residual_of` 方法用于计算与先前数值的偏差。不过在当前情况下，我不需要使用这个方法，因为 `kf.y` 的值已经通过调用 `update()` 得到了。但如果我们决定放弃某些测量结果，那么 `update()` 就不会被调用，而 `kf.y` 则仍会包含上一个时期的数据。

首先，让我们先介绍两个术语。我们在这里讨论的是“门控机制”。所谓“门”，指的是一种用于判断某个测量结果是否合格的公式或算法。只有那些合格的测量结果才能通过这个“门”。这个过程就被称为“门控机制”。

实际上，测量结果并不完全符合高斯分布。因此，如果以 3 个标准差作为判断标准，那么一些有效的测量结果可能会被忽略。我稍后会详细解释这一点。目前，我们暂且使用 4 个标准差作为判断标准。

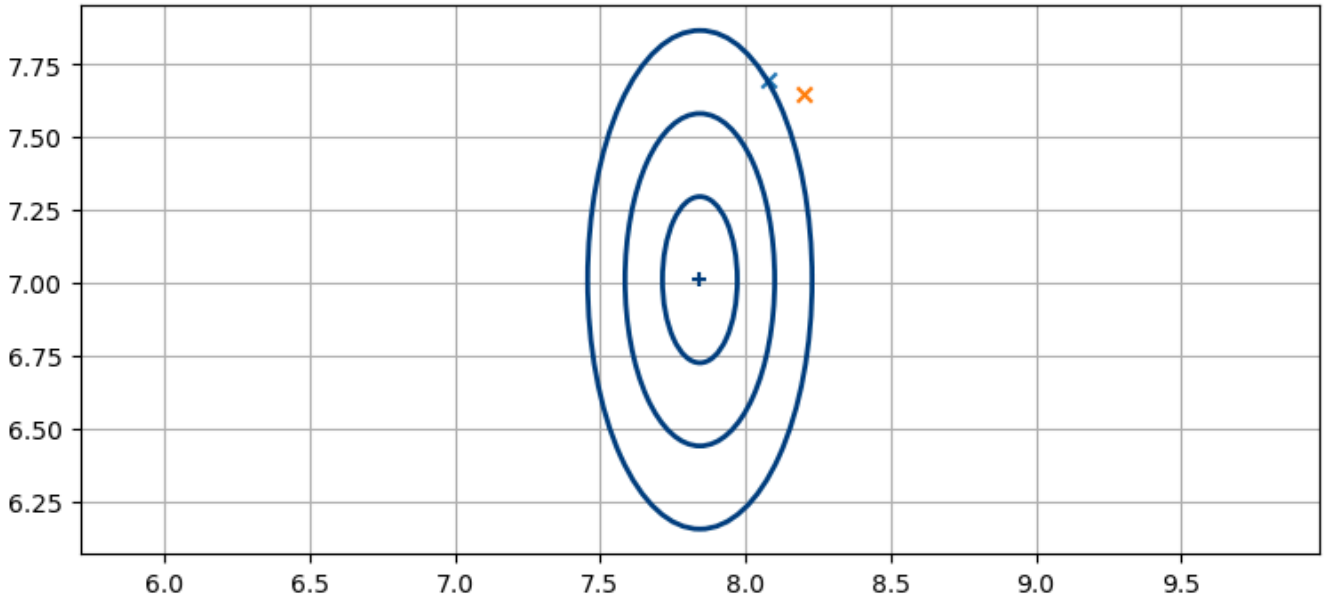
```
GATE_LIMIT = 4.
std_x = np.sqrt(P[0,0])
std_y = np.sqrt(P[1,1])
y = kf.residual_of(z)[:,0]

if y[0] > GATE_LIMIT * std_x or y[1] > GATE_LIMIT * std_y:
    print(f'discarding measurement, error is {y[0]/std_x:.0f} std, {y[1]/std_y:.0f} std')

print('y   is', y)
print(f'std is {std_x:.2f} {std_y:.2f}')
```

```
discarding measurement, error is 39 std, 18 std
y   is [5.05 5.05]
std is 0.13 0.29
```

我们看到，这些误差分别相当于 39 个和 18 个标准差。这样的结果够好吗?

也许吧。不过，请注意：if 语句所构成的区域实际上是一个围绕椭圆的矩形区域。在下图中，我画出了两个数值：其中一个明显超出了 3 个标准差的椭圆范围，但仍然会被系统接受；另一个数值则正好位于 3 个标准差的边界上。

```
plot_covariance_ellipse(x, P, std=[1,2,3])
plt.scatter(8.08, 7.7, marker='x')
plt.scatter(8.2, 7.65, marker='x');
```



定义这个“门”的方法其实还有很多种。马氏距离是一种用于衡量某一点与某个概率分布之间距离的统计量。在详细介绍定义和数学公式之前，我们先来计算几个点的马氏距离吧。 `filterpy.stats` 实现了 `mahalanobis()` 的功能。

```
from filterpy.stats import mahalanobis
m = mahalanobis(x=z, mean=x, cov=P)
print(f'mahalanobis distance = {m:.1f}')
```

```
mahalanobis distance = 20.6
```

由于不知道具体的单位，我们可以将其与在 、 x 和 、 y 中分别计算出的标准偏差误差进行比较：分别为 39 和 18。可以看出，我们的计算结果与这些标准偏差误差相当接近。让我们来看看我上面所画的那个点的具体数值吧。

```
print(f'mahalanobis distance = {mahalanobis(x=[8.08, 7.7], mean=x, cov=P):.1f}')
print(f'mahalanobis distance = {mahalanobis(x=[8.2, 7.65], mean=x, cov=P):.1f}')
```

```
mahalanobis distance = 3.0
mahalanobis distance = 3.6
```

正如我们将看到的，马氏距离实际上是一种衡量某一点与整个分布之间的“标量标准差距离”的方法，这与欧几里得距离类似——欧几里得距离则是用来衡量两点之间的标量距离的。

上面的表格正好证明了这一点。位于 3 个标准差边界上的那个点的马氏距离为 3.0，而位于椭圆之外的那个点的马氏距离则达到了 3.6 个标准差。

我们该如何计算马氏距离呢？其定义如下：

$$Dm = \sqrt{(x-\mu)TS^{-1}(x-\mu)}$$

请注意，这与欧几里得距离非常相似。欧几里得距离可以表示为：

De=√(x−y)T(x−y)

事实上，如果协方差矩阵为单位矩阵，那么马氏距离就等同于欧几里得距离。从线性代数的角度来看，这一点很容易理解：单位矩阵的逆还是单位矩阵，因此实际上我们相当于把所有数值都乘以了 1。直观地想吧：如果每个维度上的标准差都是 1，那么位于以原点为中心、半径为 1 的圆上的任意一点，其欧几里得距离也恰好为 1。

这提供了另一种解释。如果协方差矩阵是对角的，那么我们可以将马氏距离视为经过缩放的欧几里得距离。具体来说，每个数值都是根据对角线上的协方差值进行缩放后的结果。

Dm=□□ □□N∑i−1(xi−μi)2σi

在二维空间中，情况就是如此。

Dm=√1σ2x(x0−x1)2+1σ2y(y0−y1)2

这应该有助于你理解马氏距离的计算公式。虽然不能直接用矩阵来相除，但实际上，用矩阵的逆来相乘效果是一样的。在等式的每一侧都乘以 y=x−μ 后，我们得到的就是经过协方差调整后的平方范数： yTS−1yT 。由于协方差项都是平方形式，最后取平方根后，我们就得到了一个标量值，这个值实际上就是经过协方差调整后的欧几里得距离。

门控与数据关联策略

上述两种门状结构，根据其形状，被某些文献称为“矩形门”和“椭圆形门”。当然，还有其他多种形状的门状结构，这里就不再赘述了。例如，所谓“机动门”，是指用来界定物体可以机动运动的区域的一种结构。在确定机动门时，会考虑到物体的当前速度以及其机动能力。比如，战斗机的机动门形状类似一个锥体，位于飞机当前行进方向的前方；而汽车的机动门则呈较小的二维扇形，也位于汽车前方。至于轮船，由于其转向或加速的能力有限，因此其机动门的形状则更为狭窄。

应该使用哪种门控技术呢？并没有统一的答案。这主要取决于问题的维度以及可用的计算资源。矩形门控的计算成本很低，操纵门控的成本也差不多；但在高维度情况下，椭球形门控的计算成本会显著上升。不过，随着维度的增加，椭球形门控与矩形门控之间的面积差异也会越来越大。

这一点的重要性可能超乎你的想象。所有的测量结果都存在一定的误差。那些有误的测量结果也有可能被系统接受。椭球体之外的区域越大，系统接受错误测量结果的概率就越高。这里我就不进行具体的计算了，但在五维空间中，矩形区域接受错误测量结果的概率是椭球体的两倍。

如果计算时间是个问题，且存在大量无效的测量结果，可以采用两阶段处理方式。第一阶段使用一个较大的矩形结构来滤除那些明显错误的测量结果。只有通过第一阶段滤波的测量结果，才会被进一步处理，即进行复杂的马氏距离计算。如果在现代台式处理器上运行，这种矩阵运算所花费的时间可以忽略不计；但如果使用的是浮点运算能力有限的嵌入式芯片，那么这一过程可能会影响整体性能。

数据关联是一个极其复杂的课题，单凭一本书是无法完全阐述清楚的。最典型的例子就是利用雷达进行目标跟踪。雷达在每次扫描过程中都能检测到多个信号。我们需要从这些信号中识别出飞机的轨迹，同时排除那些有干扰的测量结果。这是个非常困难的任务。假设第一次扫描得到了 5 个测量数据，那么我们就能得到 5 条潜在的飞行轨迹。在第二次扫描中，如果我们得到 6 个测量数据，那么可以将第一次扫描中的任意一个数据与第二

次扫描中的任意一个数据结合起来，从而得到 30 条潜在的飞行轨迹。不过，也有可能这些都是新出现的飞机，因为在第一次扫描时并未被检测到。这样一来，又会多出 6 条潜在的飞行轨迹。仅仅经过几次扫描之后，潜在的飞行轨迹数量就会达到数百万，甚至数十亿条。

有了这数十亿条轨迹数据后，我们就可以为每条轨迹计算出一个得分。具体的计算方法我会在后面详细说明。不过，可以想象一下：如果某条轨迹在三个时间点上呈现出“Z”形，那这种轨迹是不可能存在的，因为没有任何飞机能够以这种方式移动。另一条轨迹虽然呈直线形状，但其速度却高达 10,000 公里/小时，这也显然不合常理。而另一条轨迹以 200 公里/小时的速度移动，这种可能性则相对较高。

因此，轨迹的追踪实际上涉及到数据的筛选、关联和剔除等步骤。例如，假设第二次雷达扫描刚刚完成。那么，是否应该把所有可能的数据组合起来形成一条轨迹呢？其实不必如此。如果第一次扫描中的点 1 和点 3 所对应的速度分别为 200 公里/小时，那么我们可以据此形成一条轨迹。但如果速度高达 5000 公里/小时，那就不必再继续处理了——因为这样的轨迹显然不太可能真实存在。随着轨迹的不断完善，我们就可以为它们设定明确的边界条件，从而更精确地筛选出与这些轨迹相关的数据。

对于各种轨迹，我们有不同的处理方式。我们可以选择将某次测量结果仅与某一条轨迹关联起来。或者，我们也可以选择将同一测量结果与多条轨迹关联起来，以此来反映我们无法确定该测量结果究竟属于哪条轨迹的事实。例如，从雷达的角度来看，飞机的轨迹可能会相互交叉。当飞机靠近时，就很难确定某次测量结果究竟属于哪架飞机。这时，我们可以暂时将该测量结果与两条轨迹都关联起来。随着更多测量数据的收集，我们可以根据新获得的情报来重新确定该测量结果应归属于哪条轨迹。

“数十亿”这个数字远远无法体现其中所涉及的复杂组合情况。只需几秒钟，计算机的内存就将被耗尽；再过一会儿，就需要利用宇宙中的所有原子来存储所有可能的路径了。因此，实用的算法必须对各种路径进行有效的筛选。而这种筛选过程本身又会消耗额外的计算资源。

本书的后面章节提出了解决这一问题的现代方法——粒子滤波法。该方法通过统计抽样来克服组合爆炸问题。这是我处理这类问题的首选方法，因此我就不再详细阐述本节中提到的各种技术了。由于我并不完全了解该领域的最新研究成果，如果你需要解决涉及多个物体的跟踪或处理大量错误测量数据的问题，建议你自行进行研究。当然，粒子滤波法也有其自身的局限性和难点。

我向您推荐几本相关书籍和几位研究学者。塞缪尔·S·布莱克曼所著的《雷达应用中的多目标跟踪》是我读过的关于该主题最清晰的著作，不过这本书的出版时间较久，为 1986 年。雅各布·巴尔-沙洛姆也发表了大量关于该主题的论文。苏巴什·查拉等人所著的《目标跟踪基础》则是一部较为现代的著作，涵盖了各种目标跟踪方法。不过，这本书在数学上相当复杂：各种滤波器被表述为一系列积分形式，这些积分实际上代表了不同的贝叶斯公式。你需要自己将这些公式转化为可行的算法。如果你能理解书中的所有数学内容，那么这本书还是可以读懂的，不过并不容易。劳伦斯·D·斯通所著的《贝叶斯多目标跟踪》将这一问题视为一个贝叶斯推理问题来处理，不过这本书也相当理论化。书中要求你求解复杂的积分，但实际上，你很可能需要使用粒子滤波器来解决问题。

回到我们最初的问题——如何追踪一个物体，同时应对那些偶尔出现的错误测量结果。其实解决方法很简单：如果某次测量结果有误，就直接忽略它，不要进行更新操作。这样一来，`predict()` 会被调用两次，但这没什么问题。虽然这样会导致不确定性增加，但偶尔的几次错误测量通常不会造成什么麻烦。

你应该为阈值设定多少呢？我不确定。理论上应该是 3 个标准差，但实际操作中并非如此。你需要通过实验来确定合适的数值。收集数据，使用不同的阈值进行筛选，看看哪个数值能带来最佳效果。在下一节中，我会给出一些用于评估滤波器性能的数学公式。也许你会发现，其实应该接受所有小于 4.5 个标准差的测量结果。我看过 NASA 的视频，他们使用的阈值大约在 5 到 6 个标准差之间。最终的选择取决于你的具体问题和所拥有的数据。

评估滤波器的性能

在模拟环境中设计卡尔曼滤波器相当简单。由于你知道自己在过程模型中加入了多少噪声，因此可以将 和 Q 的值设为相同。同样，由于你也清楚测量过程中存在的噪声大小，因此测量噪声矩阵 和 R 也很容易确定。

在实践中，设计往往更为复杂。实际的传感器很少能完全符合技术规格要求，其性能也很难符合高斯分布规律。此外，传感器还很容易受到环境噪声的干扰。例如，电路中的噪声会导致电压波动，从而影响传感器的输出信号。要建立准确的系统模型并考虑噪声因素则更加困难。以汽车为例，其运行过程极其复杂：转向操作会导致非线性行为，轮胎容易打滑，驾驶员的急刹车或加速也会导致轮胎打滑，风力则可能使汽车偏离行驶轨迹。最终，卡尔曼滤波器实际上只是对系统的一种近似描述而已。这种不准确性会导致系统性能不佳，最糟糕的情况下，甚至会导致滤波器完全失效。

由于存在许多未知因素，你无法通过分析来精确计算出滤波器矩阵的正确数值。你需要先给出最合理的估算值，然后再用各种模拟数据和真实数据来测试你的滤波器。通过对性能的评估，你就能知道需要对矩阵做哪些调整了。我们已经做了一些这方面的工作——我已经向你展示了当 和 Q 的数值过大或过小时会产生什么效果。

现在，让我们来探讨一些更有效的性能分析方法。如果卡尔曼滤波器的运行效果理想，那么其估计误差（即真实状态与估计状态之间的差异）应具备以下特性：

1. The mean of the estimation error is zero
2. Its covariance is described by the Kalman filter's covariance matrix

标准化估计误差平方值（NEES）

第一种方法最为有效，但只能通过模拟来实现。如果你正在模拟某个系统，那么你当然知道该系统的真实值，也就是所谓的“真实值”。此时，计算系统中任意时刻的误差就非常简单了：只需将真实值与滤波器所估计的状态值相减即可。

~x=x−^x

接下来，我们可以将标准化的估计误差平方值定义为：

ϵ=~xTP−1~x

为了理解这个方程，我们先假设该状态的维度为 1。在这种情况下，x 和 P 都是标量，因此……

ϵ=x2P

如果这一点还不清楚的话，请回想一下：如果 a 是标量，那么 aT=a 也是标量；同样地， a−1=1a 也是标量。

因此，当协方差矩阵的值越小时，为了保持相同的误差水平，NEES 的值就需要越大。协方差矩阵实际上反映了滤波器对自身误差的估计。如果协方差矩阵的数值相对于估计误差来说过小，那么滤波器的性能就会较差。

这个计算结果是一个标量值。如果 x 的维度为(n×1)，那么该计算的维度就是(1×n) × (n×n) × (n×1) = (1×1)。那我们该如何处理这个数值呢?

数学内容超出了本书的讨论范围。不过，形如 ~xTP−1~x 的随机变量被称为具有 n 个自由度的卡方分布。因此，该随机变量的期望值应为 n 。Bar-Shalom[1]对这一主题有非常精彩的阐述。

简单来说，就是取所有 NEES 值的平均值，这个平均值应该小于 x 的数值。我们用本章前面提到的例子来说明一下吧:

```
from scipy.linalg import inv

def NEES(xs, est_xs, Ps):
    est_err = xs - est_xs
    err = []
    for x, p in zip(est_err, Ps):
        err.append(x.T @ inv(p) @ x)
    return err
```

```
R, Q = 6., 0.02
xs, zs = simulate_acc_system(R=R, Q=Q, count=80)
kf2 = SecondOrderKF(R, Q, dt=1)
est_xs, ps, _, _ = kf2.batch_filter(zs)

nees = NEES (xs, est_xs, ps)
eps = np.mean(nees)

print(f'mean NEES is: {eps:.4f}')
if eps < kf2.dim_x:
    print('passed')
else:
    print('failed')
```

```
mean NEES is: 0.8893
passed
```

NEES 是通过 FilterPy 来实现的；可以通过相应的方式来访问它。

```
from filterpy.stats import NEES
```

这是衡量滤波器性能的绝佳方式，应尽可能加以使用，尤其是在生产环境中，当需要实时评估滤波器的性能时。在设计滤波器时，我仍然更喜欢通过图表来展示结果，这样能让我更直观地了解实际情况。

不过，如果你的模拟精度有限，那就需要采用另一种方法了。

似然函数

在统计学中，“似然”与“概率”非常相似，不过两者之间存在着一个重要的细微差别。概率指的是某件事发生的几率——比如，一个公平的骰子在五次投掷中三次掷出 6 点的概率是多少？而“似然”则反过来说：如果

一个骰子在五次投掷中三次掷出 6 点，那么这个骰子是公平骰子的概率是多少？

我们最早在“离散贝叶斯”这一章节中讨论了似然函数。在这些滤波器的背景下，似然函数用来衡量在当前状态下，获得某一测量结果的概率有多大。

这对我们来说很重要，因为我们掌握了滤波器的输出结果，同时希望了解：在假设高斯噪声和线性行为的前提下，该滤波器是否能够以最佳状态运行。如果这种可能性较低，那就说明我们的某个假设是错误的。在“自适应滤波”这一章中，我们将学习如何利用这些信息来改进滤波器；在这里，我们只需学会如何进行测量即可。

该滤波器的残差和系统不确定性定义如下：

$$y=z-H\bar{}xS=H\bar{}PHT+R$$

根据这些信息，我们可以计算出似然函数。

$$L=1\sqrt{}2\pi Sexp[-12yTS-1y]$$

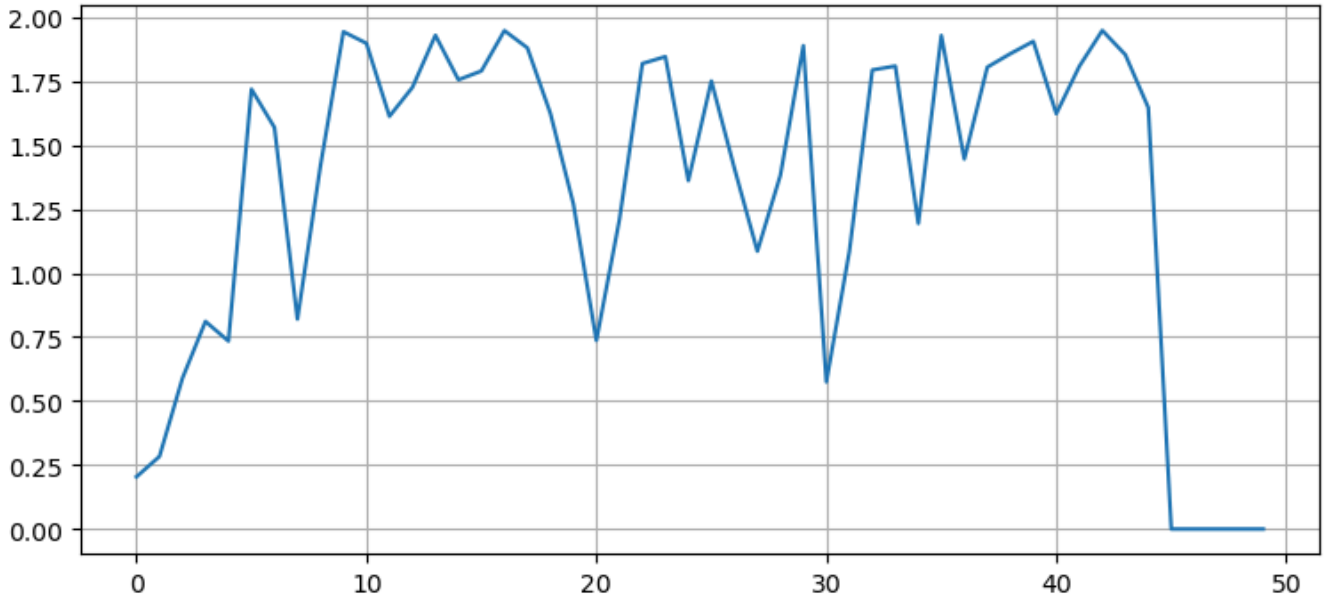
虽然看起来很复杂，但请注意，其中的指数函数实际上就是高斯分布的数学表达式。这就为该算法的实现提供了依据。

```
from scipy.stats import multivariate_normal
hx = np.dot(H, x).flatten()
S = np.dot(H, P).dot(H.T) + R
likelihood = multivariate_normal.pdf(z.flatten(), mean=hx, cov=S)
```

在实践中，情况略有不同。从数学角度处理概率值往往比较困难。因此，人们通常计算并使用对数概率，也就是概率值的自然对数。这样做有几个好处。首先，对数函数是严格递增的，并且在其定义域内，函数值达到最大值时的点也是相同的。如果想要找到某个函数的最大值，通常需要对该函数求导数；不过，对于某些复杂的函数来说，求导数可能比较困难。而计算对数函数的导数则很简单，其结果与直接求导得到的结果相同。在本书中，我们没有使用这一性质，但在对滤波器进行分析时，这一性质非常重要。

当调用 `update()` 时，系统的概率值和对数概率值会自动计算出来，这些数值可以通过‘log_likelihood’和 `likelihood` 数据属性来获取。我们来具体看看：我会先使用若干处于预期范围内的数值来运行该滤波器，然后再输入一些远远偏离预期值的数值来测试系统反应。

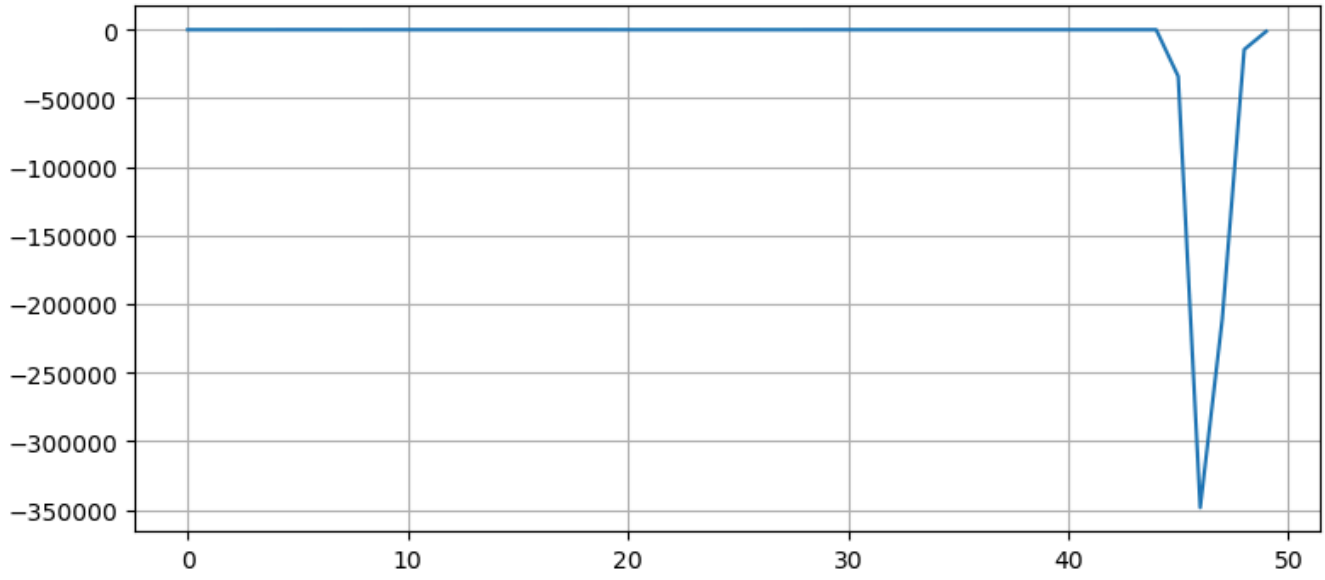
```
R, Q = .05, 0.02
xs, zs = simulate_acc_system(R=R, Q=Q, count=50)
zs[-5:-1] = [100, 200, 200, 200] # bad measurements, bad!

kf = SecondOrderKF(R, Q, dt=1, P=1)
s = Saver(kf)
kf.batch_filter(zs, saver=s)
plt.plot(s.likelihood);
```



在最初的几次迭代过程中，随着滤波器的不断收敛，这种可能性会越来越大。之后，该概率值会波动不定，直到遇到错误的测量结果时，概率值会降为零。这说明，如果测量结果是正确的，那么该滤波器不太可能是最优的。

可以看出，对数似然值清楚地表明了滤波器在哪些地方出现了“异常行为”。

```
plt.plot(s.log_likelihood);
```



为什么在最后数值会回到零呢？在阅读答案之前，请先思考一下这个问题。该滤波器通过将状态调整到接近实际测量值的位置，从而逐渐适应新的测量数据。这样一来，残差值就会变小，最终状态值与残差值趋于一致。

控制输入

在“离散贝叶斯”这一章中，我介绍了利用控制信号来提升滤波器性能的方法。我们不再假设物体会像之前那样持续运动，而是利用对控制输入的了解来预测物体的位置。在“单变量卡尔曼滤波器”这一章中，我们也采用了同样的思路。卡尔曼滤波器的预测功能就是基于这一原理来实现的。

```
def predict(pos, movement):
    return (pos[0] + movement[0], pos[1] + movement[1])
```

在上一章中，我们了解到状态预测的方程式为：

$$\bar{x} = Fx + Bu$$

我们的状态是一个向量，因此我们需要将控制输入也表示为向量形式。这里， u 代表控制输入，而 B 则是一个矩阵，该矩阵的作用是将控制输入转换为 x 的变化。我们来看一个简单的例子。假设我们所控制的机器人的状态为 $x = [x \cdot x]$ ，而控制输入则是指令速度。这样一来，我们的控制输入就为

$$u = [\cdot xcmd]$$

为简化问题，我们假设机器人能够立即对输入参数的变动做出反应。也就是说，经过 Δt 秒后，机器人的新位置和速度将会是……

$$x = x + \cdot xcmd\Delta t \cdot x = \cdot xcmd$$

我们需要将这组方程表示为 $\bar{x} = Fx + Bu$ 的形式。

我将使用 Fx 这个项来提取出最上方那个方程中的 x ；而对于其他方程，则使用 Bu 这个项。具体如下：

$$[x \cdot x] = \begin{bmatrix}1 & 0\\0 & 0\end{bmatrix}[x \cdot x] + [\Delta t1][\cdot xcmd]$$

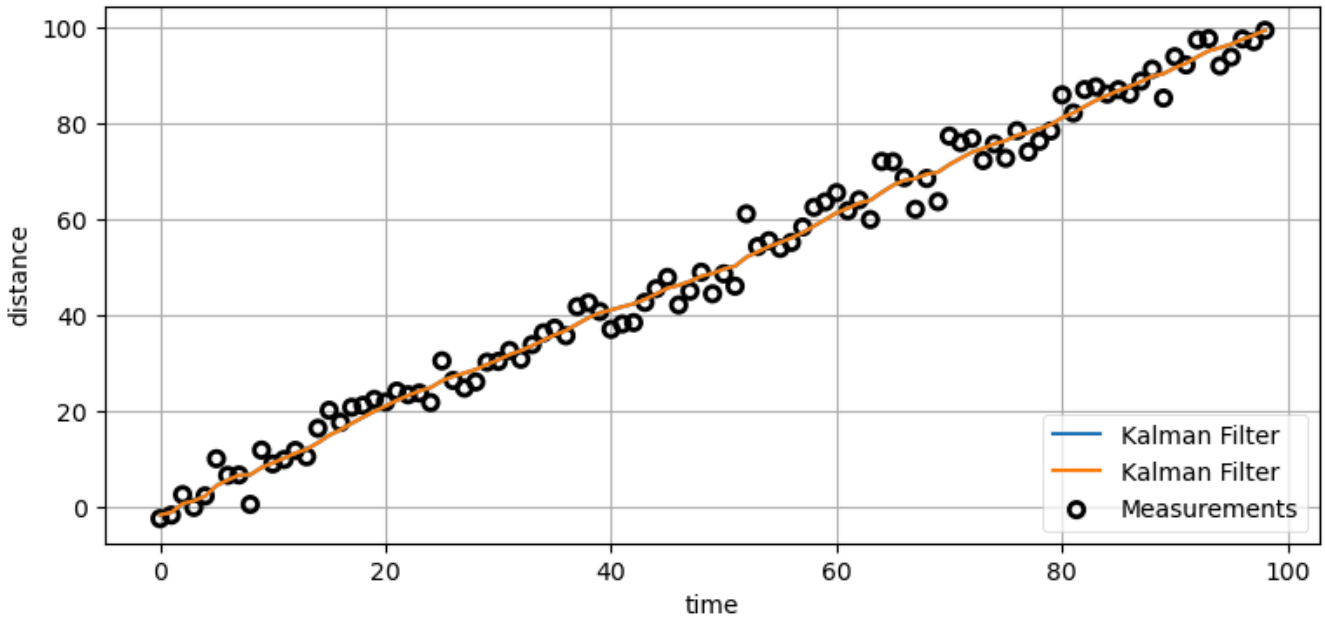
这是一种简化的描述；实际上，典型的控制输入包括转向角的调整以及加速度的变化。这些因素都会引入非线性效应，我们将在后面的章节中学习如何处理这些非线性问题。

卡尔曼滤波器的其余部分将按照常规方式来设计。你们已经多次见过这样的设计了，所以这里就不赘述了。下面是一个示例。

```
dt = 1.
R = 3.
kf = KalmanFilter(dim_x=2, dim_z=1, dim_u = 1)
kf.P *= 10
kf.R *= R
kf.Q = Q_discrete_white_noise(2, dt, 0.1)
kf.F = np.array([[1., 0], [0., 0.]])
kf.B = np.array([[dt], [ 1.]])
kf.H = np.array([[1., 0]])
print(kf.P)

zs = [i + randn()*R for i in range(1, 100)]
xs = []
cmd_velocity = np.array([1.])
for z in zs:
    kf.predict(u=cmd_velocity)
    kf.update(z)
    xs.append(kf.x[0])
```

```
plt.plot(xs, label='Kalman Filter')
plot_measurements(zs)
plt.xlabel('time')
plt.legend(loc=4)
plt.ylabel('distance');
```

```
[[10.  0.]
 [ 0. 10.]]
```



传感器融合

在关于 G-H 滤波器的章节中，我们讨论了如何为两种不同的尺度来设计滤波器：一种是精确的测量结果，另一种则是不准确的测量结果。我们得出结论：必须始终保留不准确滤波器所提供的信息——绝不能丢弃任何信息。那么，当有两个传感器来测量系统参数时，我们该如何将这些信息纳入卡尔曼滤波器中呢?

假设有一列火车或一辆车厢在铁轨上行驶。火车的轮子上装有传感器，可以记录轮子的转动次数，进而换算出火车在铁轨上的行驶距离。此外，火车上还装有类似 GPS 的传感器，我将其称为“位置传感器”，该传感器能够反馈火车的位置信息。我会在下一节中解释为什么不直接使用 GPS。这样，我们就有了两种能够反映火车在铁轨上位置的测量数据。假设轮子传感器的精度为 1 米，而位置传感器的精度为 10 米。那么，我们该如何将这两种测量数据结合起来呢？虽然这看起来有些复杂，但实际上，飞机也是通过融合来自 GPS、惯性导航系统、多普勒雷达、VOR、空速计等多种传感器的信息来获取精确位置的。

对于惯性系统而言，使用卡尔曼滤波器来处理数据相当困难。不过，将两个或更多传感器所测得的同一状态变量（比如位置）的数据进行融合则相对容易。相关的矩阵就是测量矩阵 H 。需要记住的是，该矩阵告诉我们如何将卡尔曼滤波器所计算出的状态 x 转换为可测量的数值 z 。假设我们决定让卡尔曼滤波器的状态包含火车的位置和速度信息，那么……

x=[x⋆x]

我们有两种用于确定位置的测量方式，因此，我们将把测量向量定义为来自车轮和位置传感器的各项测量数值所构成的向量。

z=[zwheelzps]

因此，我们必须设计矩阵 H，以便将 x 转换为 z。由于两者都属于位置信息，所以这种转换实际上就是乘以 1 而已。

[zwheelzps]=[1010][x⋅x]

为了更清楚地说明，假设轮子不报告其具体位置，而是报告其旋转的圈数。其中，每旋转一圈，轮子就会前进 2 米。在这种情况下，我们可以这样表示：

[zrotzps]=[0.5010][x⋅x]

现在，我们必须来设计测量噪声矩阵 R。假设位置的测量方差是车轮方差的两倍，而车轮的标准差为 1.5 米。据此，我们可以得出……

σwheel=1.5σ2wheel=2.25σps=1.5∗2=3σ2ps=9.

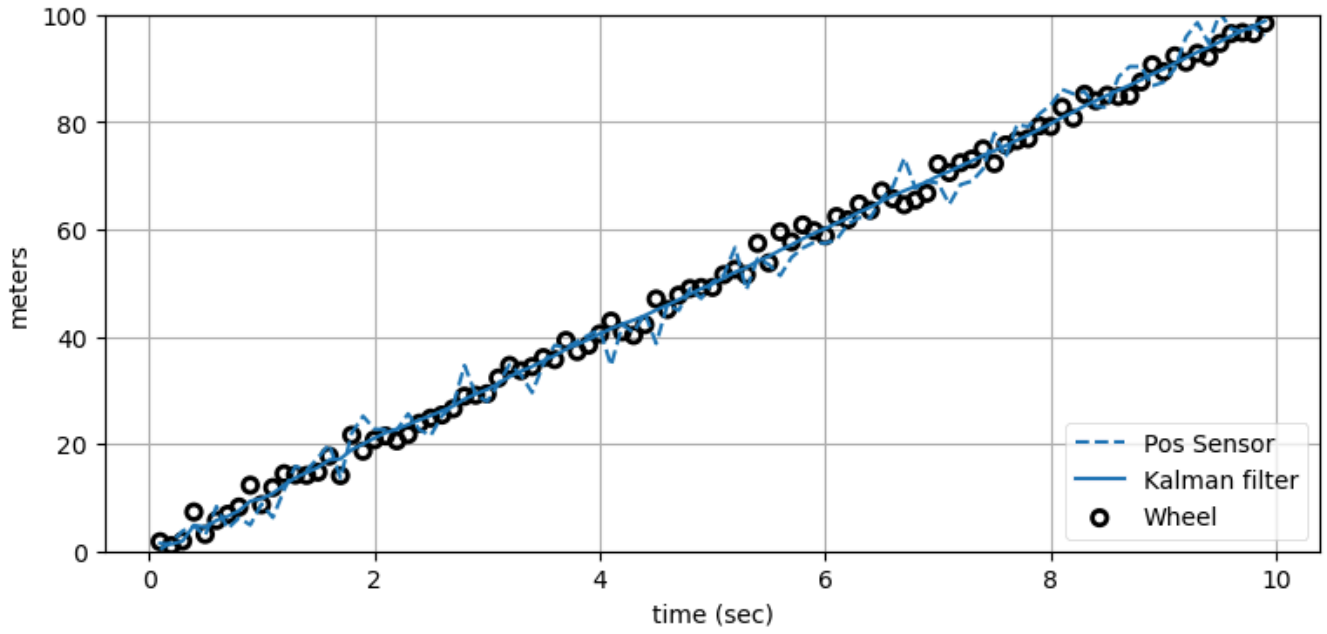
这就是我们的卡尔曼滤波器设计方案。我们需要为 和 Q 进行参数设定，不过这个设定与我们是否进行传感器数据融合无关。因此，我可以选择一些任意的数值来作为这些参数的值。

那么，让我们来模拟一下这个设计吧。我假设速度为 10 米/秒，更新频率为 0.1 秒。

```
from numpy import array, asarray
import numpy.random as random

def fusion_test(wheel_sigma, ps_sigma, do_plot=True):
    dt = 0.1
    kf = KalmanFilter(dim_x=2, dim_z=2)

    kf.F = array([[1., dt], [0., 1.]])
    kf.H = array([[1., 0.], [1., 0.]])
    kf.x = array([[0.], [1.]])
    kf.Q *= array([[(dt**3)/3, (dt**2)/2],
                   [(dt**2)/2,  dt      ]]) * 0.02
    kf.P *= 100
    kf.R[0, 0] = wheel_sigma**2
    kf.R[1, 1] = ps_sigma**2
    s = Saver(kf)

    random.seed(1123)
    for i in range(1, 100):
        m0 = i + randn()*wheel_sigma
        m1 = i + randn()*ps_sigma
        kf.predict()
        kf.update(array([[m0], [m1]]))
        s.save()
    s.to_array()
    print(f'fusion std: {np.std(s.y[:, 0]):.3f}')
    if do_plot:
        ts = np.arange(0.1, 10, .1)
        plot_measurements(ts, s.z[:, 0], label='Wheel')
        plt.plot(ts, s.z[:, 1], ls='--', label='Pos Sensor')
        plot_filter(ts, s.x[:, 0], label='Kalman filter')
        plt.legend(loc=4)
```

```
        plt.ylim(0, 100)
        set_labels(x='time (sec)', y='meters')

fusion_test(1.5, 3.0)
```

```
fusion std: 1.647
```



我们可以用蓝色来表示卡尔曼滤波的处理结果。

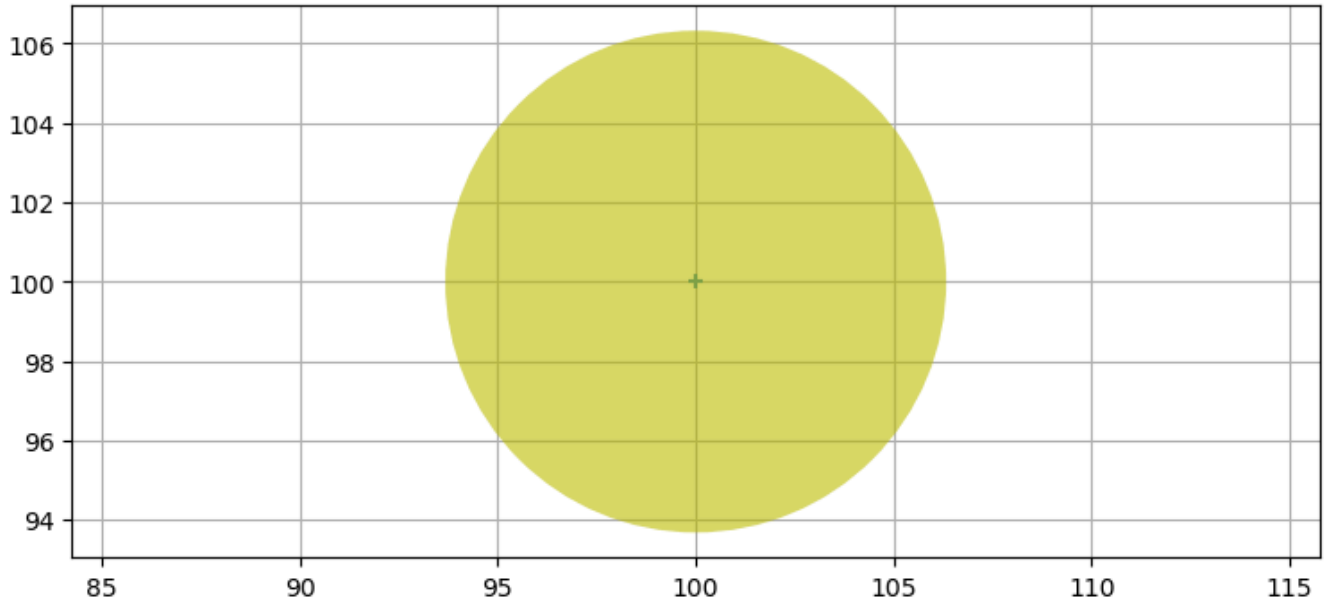
从直观的角度来看，可能有点难以理解上面的例子。让我们来看另一个例子吧。假设我们正在二维空间中追踪某个物体，同时有两组雷达系统分别位于不同的位置。每组雷达系统都能提供关于该目标的距离和方位信息。那么，来自这两组数据的信息会如何影响最终的测量结果呢?

这是一个非线性问题，因为我们需要利用三角函数来根据距离和方位角来计算坐标。而我们目前还不知道如何使用卡尔曼滤波器来求解非线性问题。因此，在解决这个问题时，请先忽略我所使用的代码，只需关注代码输出的结果即可。我们将在后面的章节中再回到这个问题上，学习如何编写这样的代码。

我会将目标定位在(100, 100)处。第一个雷达将位于(50, 50)处，第二个雷达则位于(150, 50)处。这样一来，第一个雷达测得的方位角为 45 度，而第二个雷达测得的方位角则为 135 度。

我先会构建卡尔曼滤波器，然后再绘制其初始协方差矩阵。我使用的是无迹卡尔曼滤波器，这一内容将在后面的章节中详细讲解。

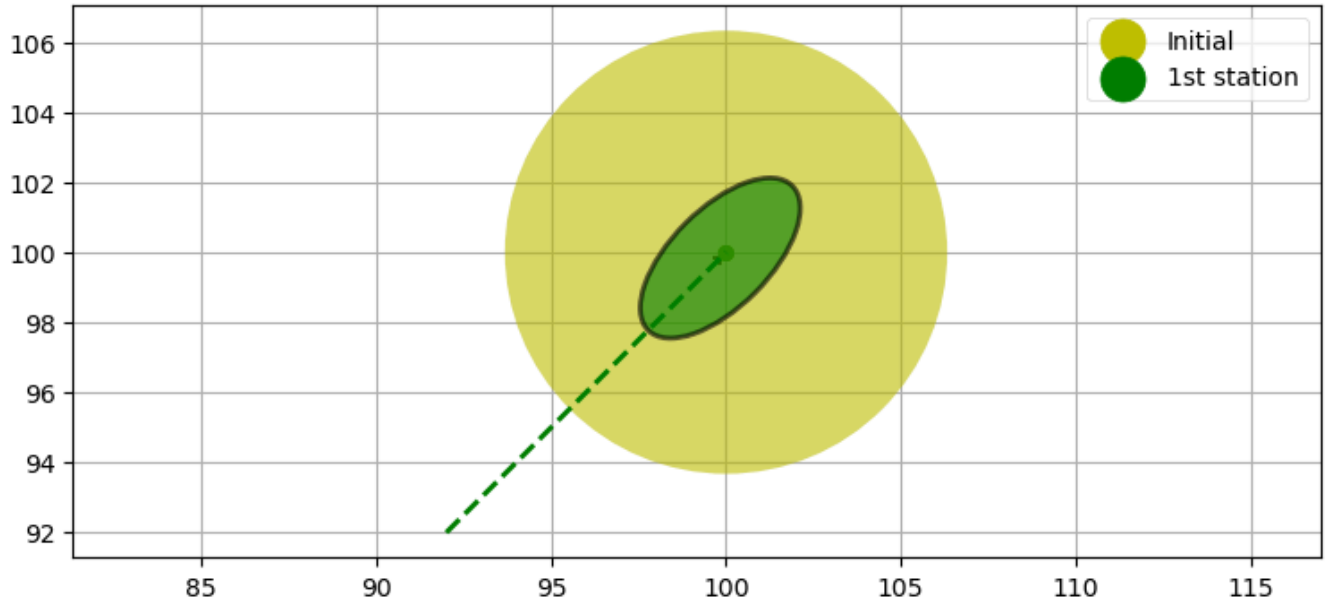
```
from kf_book.kf_design_internal import sensor_fusion_kf

kf = sensor_fusion_kf()
x0, p0 = kf.x.copy(), kf.P.copy()
plot_covariance_ellipse(x0, p0, fc='y', ec=None, alpha=0.6)
```



我们对物体在 x 轴和 y 轴上的位置同样不确定，因此协方差呈圆形分布。

现在，我们将用第一个雷达的测量数据来更新卡尔曼滤波器。我将方位角误差的标准差设置为 0.5，距离误差的标准差则设置为 3。

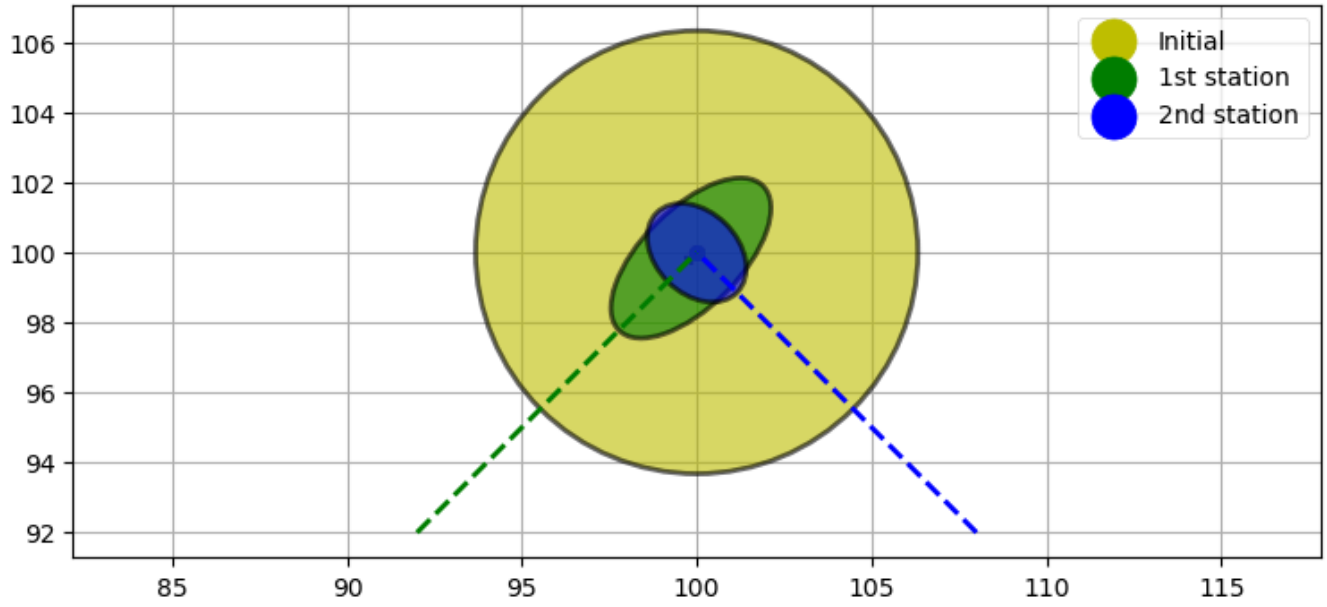
```
from math import radians
from kf_book.kf_design_internal import sensor_fusion_kf, set_radar_pos

# set the error of the radar's bearing and distance
kf.R[0, 0] = radians (.5)**2
kf.R[1, 1] = 3.**2

# compute position and covariance from first radar station
set_radar_pos((50, 50))
dist = (50**2 + 50**2) ** 0.5
kf.predict()
kf.update([radians(45), dist])

# plot the results
x1, p1 = kf.x.copy(), kf.P.copy()

plot_covariance_ellipse(x0, p0, fc='y', ec=None, alpha=0.6)
plot_covariance_ellipse(x1, p1, fc='g', ec='k', alpha=0.6)

plt.scatter([100], [100], c='y', label='Initial')
plt.scatter([100], [100], c='g', label='1st station')
plt.legend(scatterpoints=1, markerscale=3)
plt.plot([92, 100], [92, 100], c='g', lw=2, ls='--');
```



我们可以看到这些误差对问题几何形态的影响。雷达站位于目标的左下方。在 σ=0.5∘ 这个位置，方位角的测量非常精确；但在 σ=3 这个位置，距离的测量则不够准确。我用绿色的虚线标出了雷达的测量结果。从协方差椭圆的形状上，我们可以清楚地看到精确的方位角与不准确的距离所带来的影响。

现在，我们可以把第二个雷达站的测量数据也考虑进来了。第二个雷达位于(150,50)这个点上，也就是目标的正下方和右侧。在继续之前，请先思考一下：当我们将这个新的测量数据纳入考虑范围后，协方差会如何变化呢?

```
# compute position and covariance from second radar station
set_radar_pos((150, 50))
kf.predict()
kf.update([radians(135), dist])

plot_covariance_ellipse(x0, p0, fc='y', ec='k', alpha=0.6)
plot_covariance_ellipse(x1, p1, fc='g', ec='k', alpha=0.6)
plot_covariance_ellipse(kf.x, kf.P, fc='b', ec='k', alpha=0.6)

plt.scatter([100], [100], c='y', label='Initial')
plt.scatter([100], [100], c='g', label='1st station')
plt.scatter([100], [100], c='b', label='2nd station')
plt.legend(scatterpoints=1, markerscale=3)
plt.plot([92, 100], [92, 100], c='g', lw=2, ls='--')
plt.plot([108, 100], [92, 100], c='b', lw=2, ls='--');
```



我们可以看到，第二次雷达测量如何改变了协方差矩阵。由于目标相对于第一个雷达站的角度是垂直的，因此方位角和距离测量中的误差所造成的影响也发生了互换。这样一来，协方差矩阵的方向也就随之发生了变化，以与第二个雷达站的方向相一致。需要指出的是，不仅仅是方向发生了变化，协方差矩阵的大小也变小了很多。

协方差始终会包含所有可用的信息，包括与问题所涉及的几何结构相关的各种因素。这种处理方式使得理解其运作原理变得十分容易。不过，即便是一个传感器提供位置信息，另一个传感器提供速度信息，或者两个传感器都提供位置测量数据，情况也是一样的。

在继续之前，还有最后一件事需要说明：传感器融合是一个非常复杂的主题，我的阐述过于简单化，甚至可能产生误导。例如，GPS 在确定位置时，使用的是迭代最小二乘法，而不是卡尔曼滤波器。我在配套的笔记《用于传感器融合的迭代最小二乘法》中详细介绍了这一内容。

这其实是 GPS 接收机进行此类计算的常用方法，但并非唯一的方法。如果你只是个业余爱好者，我的介绍应该能帮你入门。而商用级别的滤波器则需要对各种处理流程进行精心设计。这一主题已经有很多相关书籍进行了探讨，你需要找一本与你的领域相关的书来进一步学习。

练习：你能对 GPS 的测量结果进行筛选吗？

在上面的例子中，我让你将卡尔曼滤波器应用于一种“类似 GPS 的”传感器。那么，你能否将卡尔曼滤波器应用于商用卡尔曼滤波器的输出结果呢？换句话说，你设计的滤波器所输出的数值，会优于、劣于还是等于 GPS 的输出结果呢？

解决方案

商用 GPS 设备内部都内置了卡尔曼滤波器，其输出结果就是经过该滤波器处理后的数据。那么，假设 GPS 持续输出位置信息以及位置误差数据，你难道不能将这些数据输入到自己的滤波器中吗？

那么，该数据流具有哪些特征呢？更重要的是，卡尔曼滤波器的输入数据应满足哪些基本要求呢？

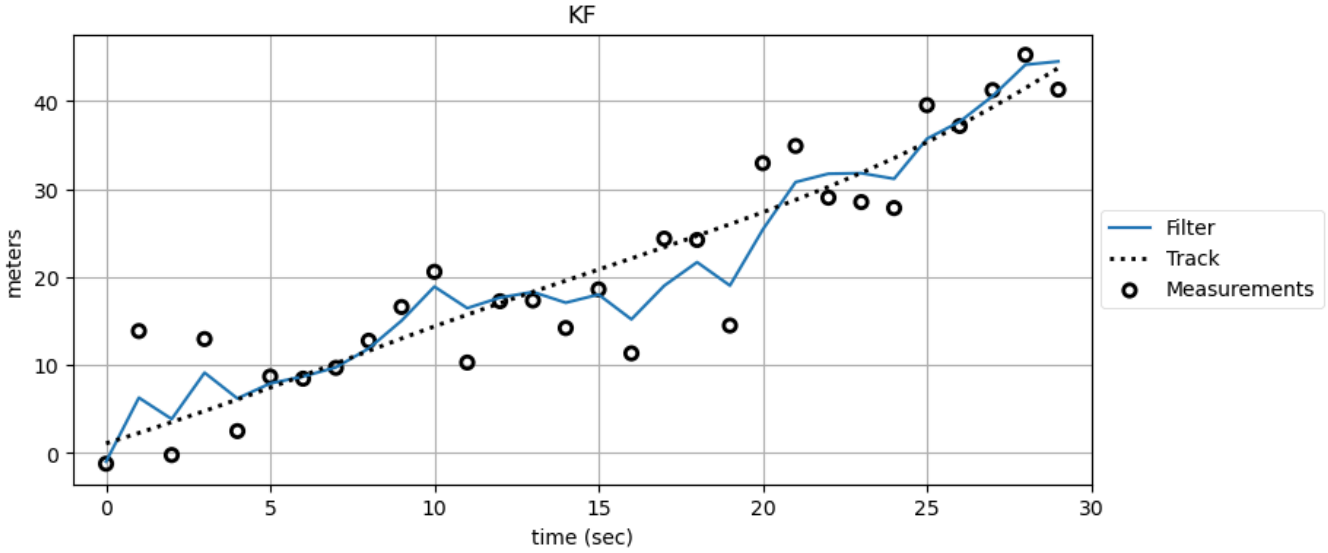
卡尔曼滤波器的输入数据必须符合高斯分布且具有时间独立性。这是因为我们遵循了马尔可夫性原则：当前状态仅取决于之前的状态和当前的输入数据。这一原则使得卡尔曼滤波器的递归处理成为可能。而 GPS 的输出数据则具有时间依赖性，因为滤波器会利用所有先前的测量结果来更新当前估计值。因此，GPS 输出的信号并不满足白噪声条件，也不具有时间独立性。如果将这样的数据输入卡尔曼滤波器，就违背了该滤波器的数学要求。所以，答案是否定的：无法通过对商业 GPS 的输出数据应用卡尔曼滤波器来获得更准确的估计结果。

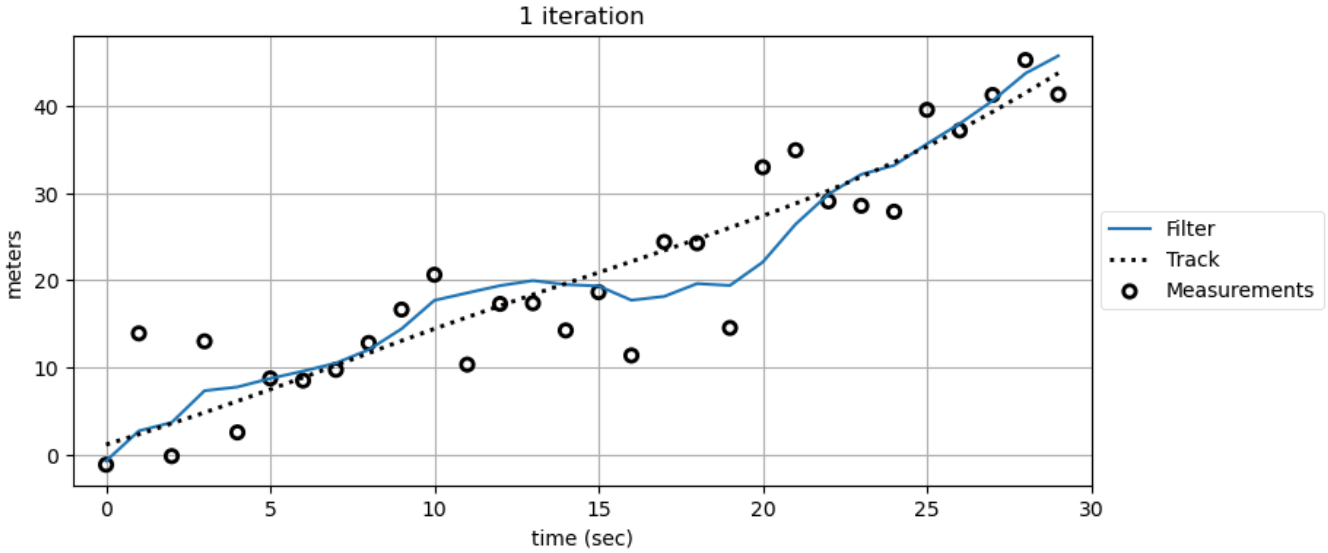
另一种理解方式是：卡尔曼滤波器在最小二乘意义上属于最优解。不可能找到一个最优解，然后将其通过某个滤波器处理后得到一个“更优”的结果，因为这在逻辑上是不可能的。最多，信号保持不变，这种情况下它仍然属于最优解；或者信号发生改变，从而不再是最优解。

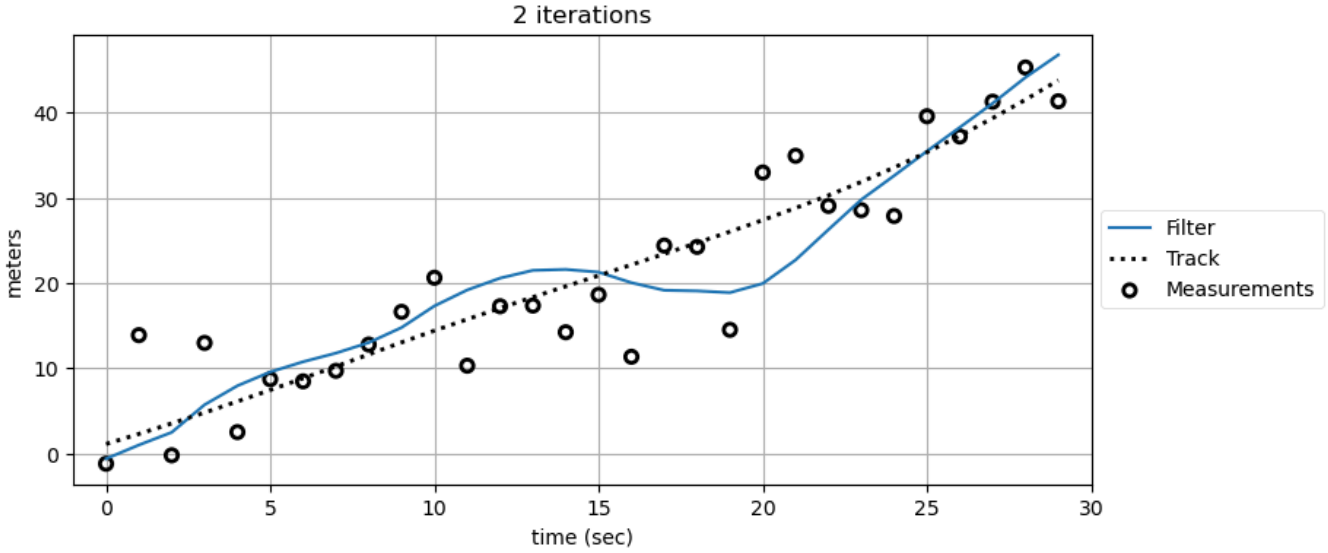
这是业余爱好者在尝试将 GPS、惯性测量单元以及其他现成传感器整合在一起时所面临的难题。

让我们来看看其效果如何。商用 GPS 设备会报告当前位置以及估计的误差范围。这个估计误差实际上来自卡尔曼滤波器中的相关矩阵。那么，我们就试着对一些有噪声的数据进行滤波处理，再把滤波后的结果作为新的输入数据送入滤波器中，看看会产生什么效果。换句话说， x 会提供 z 作为输入数据，而 P 则提供测量结果的协方差 R 。为了更清楚地展示这种效果，我会分别绘制两次处理后的结果。第二次处理其实没什么实际意义（没人会真的这么做），我只是用它来说明一个观点而已。首先，是相关的代码和图表。

```
np.random.seed(124)
R = 5.
xs, zs = simulate_acc_system(R=R, Q=Q, count=30)

kf0 = SecondOrderKF(R, Q, dt=1)
kf1 = SecondOrderKF(R, Q, dt=1)
kf2 = SecondOrderKF(R, Q, dt=1)

# Filter measurements
fxs0, ps0, _, _ = kf0.batch_filter(zs)

# filter twice more, using the state as the input
fxs1, ps1, _, _ = kf1.batch_filter(fxs0[:, 0])
fxs2, _, _, _ = kf2.batch_filter(fxs1[:, 0])

plot_kf_output(xs, fxs0, zs, 'KF', False)
plot_kf_output(xs, fxs1, zs, '1 iteration', False)
plot_kf_output(xs, fxs2, zs, '2 iterations', False)
R,Q
```


KF
meters
time (sec)
Filter
Track
Measurements


1 iteration
meters
time (sec)
Filter
Track
Measurements

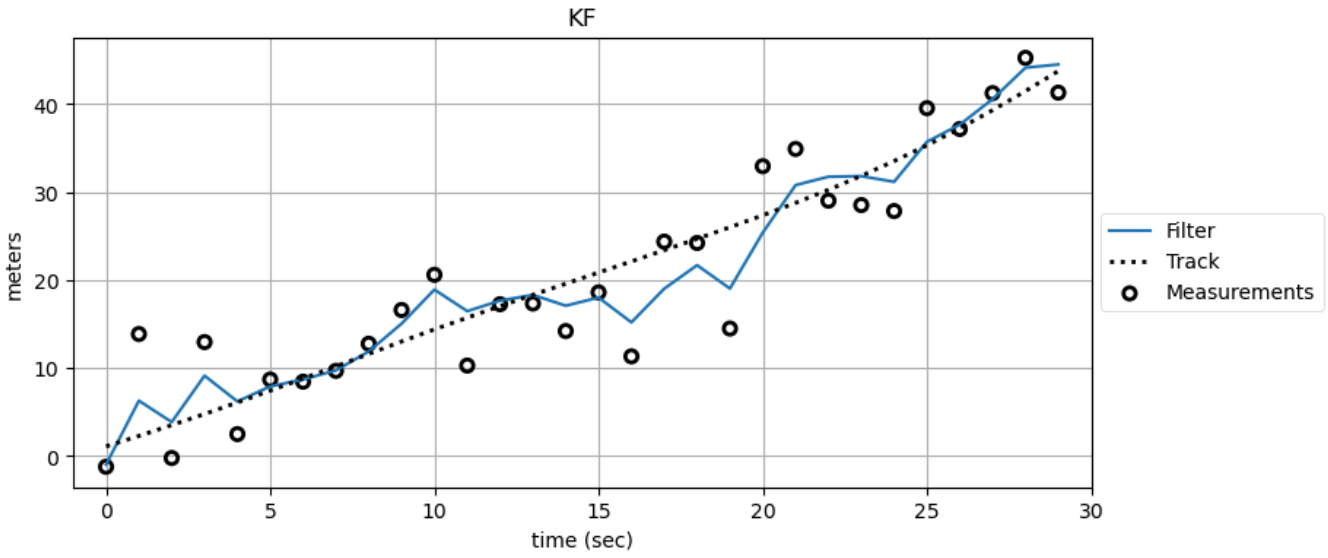

2 iterations
meters
time (sec)
Filter
Track
Measurements

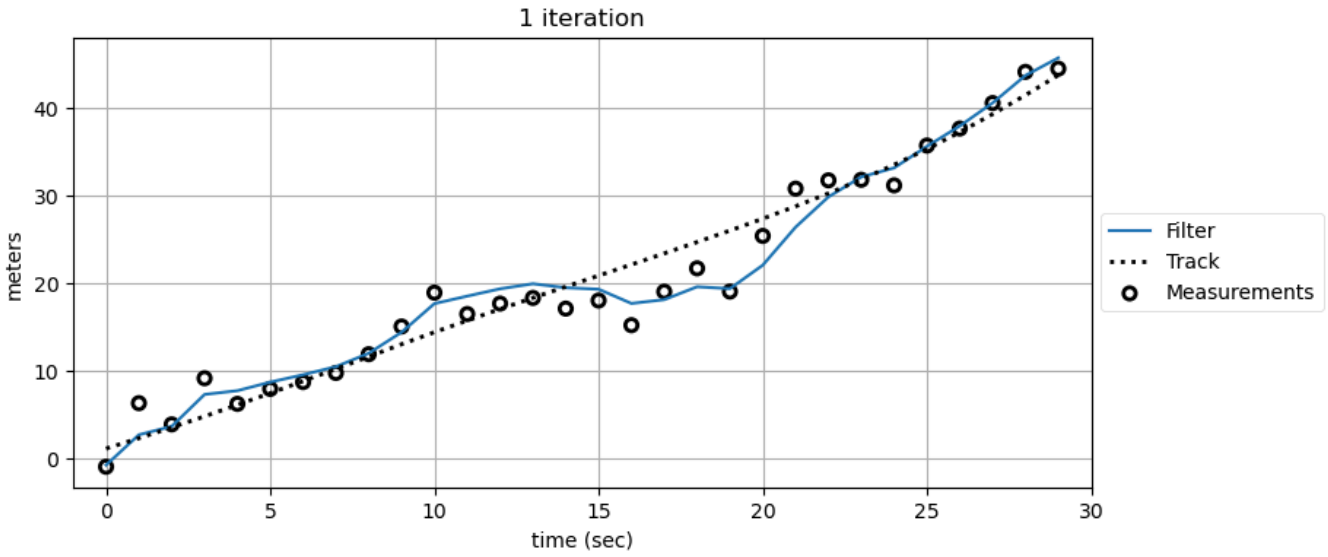
```
(5.0, 0.02)
```

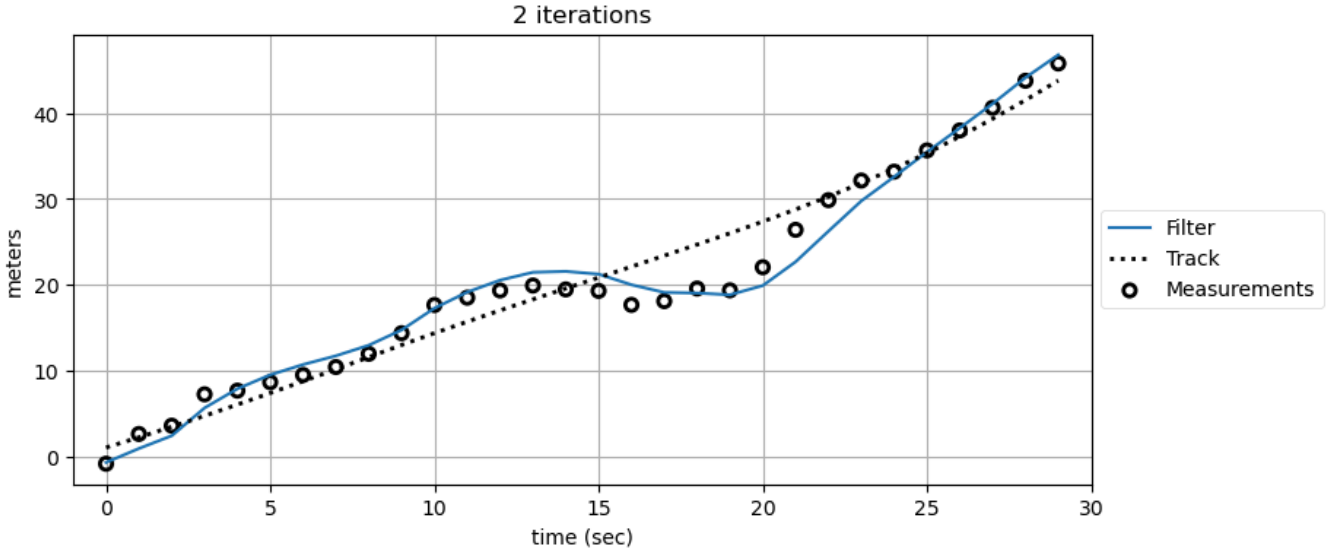
我们可以看到，经过重新处理的信号经滤波波处理后，其波形更加平滑了。不过，其数值也偏离了原始轨迹。这是为什么呢？请记住，卡尔曼滤波器要求信号在时间上是不相关的。但实际上，卡尔曼滤波器的输出在时间上是相关的，因为它会将所有之前的测量结果都纳入到对当前时刻的估计中。请看最后一个图表，其中展示了两次迭代后的结果。在开始阶段，测量结果中有一些峰值高于原始轨迹。这些数值被滤波器“记住了”（这个说法有点模糊，但我尽量不用复杂的数学公式来解释）。因此，滤波器认为该物体位于原始轨迹的上方。后来，在 13 秒左右的时间段内，所有测量结果都低于原始轨迹。这些数值也被滤波器“记住了”，结果，经过多次迭代后，输出结果远远低于原始轨迹。

现在，让我们从另一个角度来考虑这个问题。该迭代算法并不以 z 作为衡量标准，而是以上一个卡尔曼滤波器的输出作为基准。因此，我会将当前滤波器的输出与上一个滤波器的输出进行对比绘制。

```
plot_kf_output(xs, fxs0, zs, title='KF', aspect_equal=False)
plot_kf_output(xs, fxs1, fxs0[:, 0], '1 iteration', False)
plot_kf_output(xs, fxs2, fxs1[:, 0], '2 iterations', False)
```







希望现在大家都能明白这种方法的缺陷了。从下图可以看出，KF 算法实际上是在追踪上一个滤波器给出的不完美估计值。同时，由于上一次的测量结果也被纳入了当前的计算中，因此该算法还会将延迟效应也体现在信号中。

练习：证明位置传感器能够提升滤波效果。

想办法证明：将位置传感器的测量数据与车轮的测量数据相结合，所得到的结果会比单独使用车轮的测量数据更准确。

解决方案 1

通过将位置传感器的测量噪声设置为接近无穷大的值，迫使卡尔曼滤波器忽略该传感器的测量结果。重新运行滤波器后，观察残差的标准差。

```
fusion_test(1.5, 3.0, do_plot=False)
fusion_test(1.5, 1.e40, do_plot=False)
```

```
fusion std: 1.647
fusion std: 1.647
```

在这里，我们可以看到滤波器中的错误：在那些几乎完全忽略位置传感器测量值的情况下，误差要大于那些会利用位置传感器测量值的情况。

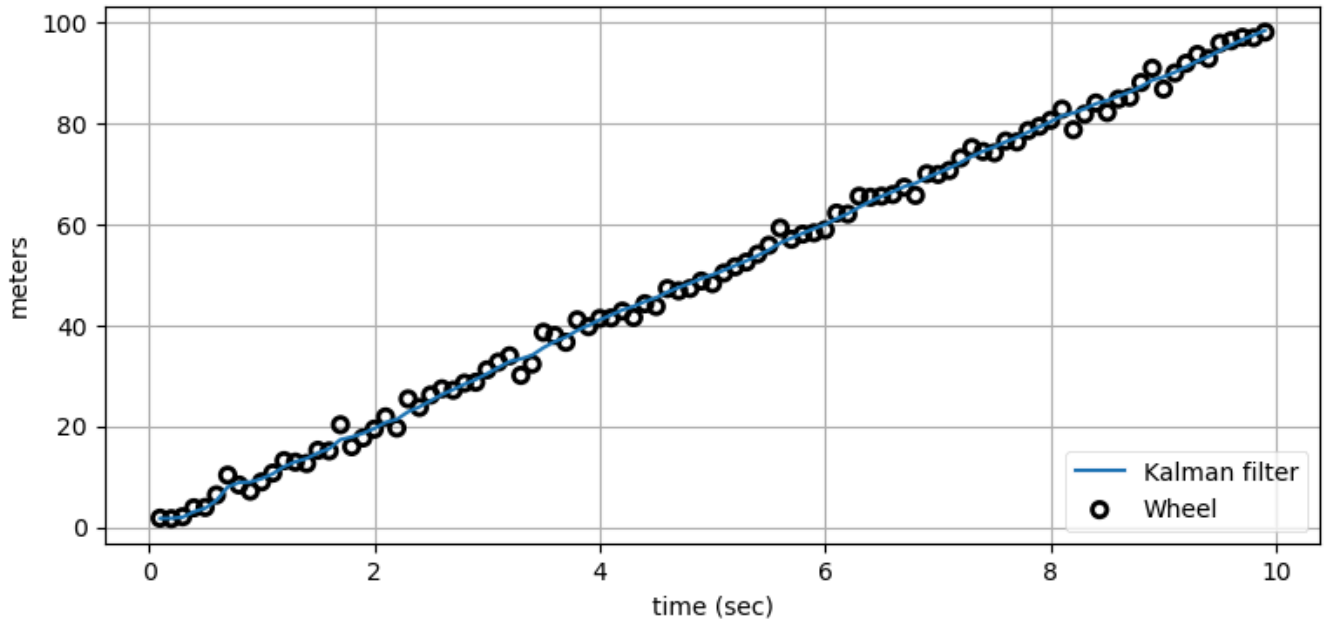
解决方案 2

这虽然需要更多的工作量，但我们完全可以编写出一个只需使用一次测量数据就能运行的卡尔曼滤波器。

```
dt = 0.1
wheel_sigma = 1.5
kf = KalmanFilter(dim_x=2, dim_z=1)
kf.F = array([[1., dt], [0., 1.]])
kf.H = array([[1., 0.]])
kf.x = array([[0.], [1.]])
```

```
kf.Q *= 0.01
kf.P *= 100
kf.R[0, 0] = wheel_sigma**2

random.seed(1123)
nom = range(1, 100)
zs = np.array([i + randn()*wheel_sigma for i in nom])
xs, _, _, _ = kf.batch_filter(zs)
ts = np.arange(0.1, 10, .1)

res = nom - xs[:, 0, 0]
print(f'std: {np.std(res):.3f}')

plot_filter(ts, xs[:, 0], label='Kalman filter')
plot_measurements(ts, zs, label='Wheel')
set_labels(x='time (sec)', y='meters')
plt.legend(loc=4);
```

```
std: 0.523
```



在这次测量中，我的标准差为 0.523，而合并后的测量结果的标准差则为 0.391。

非平稳过程

到目前为止，我们一直假设卡尔曼滤波器中的各个矩阵都是静态的——即不会随时间而发生变化。例如，在机器人跟踪的相关内容中，我们假设在 Δt=1.0 秒的时间段内，各矩阵保持不变，并据此设计了状态转移矩阵。

F=⎡⎣⎢1Δt000100001Δt0001⎤⎦⎥=⎡⎣⎢1100010000110001⎤⎦⎥

但如果我们的数据传输速率以某种不可预测的方式发生变化呢？或者，如果我们有两个传感器，而它们的工作速率各不相同，那又会怎样呢？再者，如果测量的误差也发生了变化，那又会如何呢？

处理起来很简单：只需调整卡尔曼滤波器中的矩阵，以反映当前的情况即可。让我们回到狗狗跟踪的例子上来。假设输入的数据比较零散不规律。针对这个问题，我们设计了相应的解决方案。

¯x=[x˙x]−F=[1Δt01]

并在初始化过程中按如下方式设置卡尔曼滤波器的变量 F：

```
dt = 0.1
kf.F = np.array([[1, dt],
                 [0, 1]])
```

我们该如何处理每次测量时 和 Δt 的变化呢？其实很简单——只需修改相应的矩阵即可。在这种情况下， F 是一个会变化的参数，因此我们需要在更新/预测的循环中不断更新其值。 Q 也随时间而变化，因此也必须在每次循环中重新赋值。以下是实现这一功能的代码示例：

```
kf = KalmanFilter(dim_x=2, dim_z=1)
kf.x = array([0., 1.])
kf.H = array([[1, 0]])
kf.P = np.eye(2) * 50
kf.R = np.eye(1)
q_var = 0.02

# measurement tuple: (value, time)
zs = [(1., 1.),  (2., 1.1), (3., 0.9), (4.1, 1.23), (5.01, 0.97)]
for z, dt in zs:
    kf.F = array([[1, dt],
                  [0, 1]])
    kf.Q = Q_discrete_white_noise(dim=2, dt=dt, var=q_var)
    kf.predict()
    kf.update(z)
    print(kf.x)
```

```
[1. 1.]
[2.   0.92]
[2.96 1.  ]
[4.12 0.97]
[5.03 0.96]
```

传感器融合：不同的数据传输速率

很少有两种不同的传感器能以相同的频率输出数据。假设位置传感器的更新频率为 3 赫兹，而车轮传感器的更新频率为 7 赫兹。另外，假设这些传感器的计时并不精确，会存在一定的误差，导致测量结果可能比预期时间稍早或稍晚一些。再复杂一点来说，假设车轮传感器提供的是速度估计值，而非位置估计值。

我们可以通过等待任一传感器发送的数据包来实现这一点。收到数据包后，我们可以计算出自上次更新以来经过的时间。之后，就需要对相关的矩阵进行相应的调整。 F 和 Q 中都包含时间相关的参数 Δt ，因此我们需要在每次数据更新时都对这些参数进行调整。

每次测量结果都有所不同，因此我们必须调整 、 H 以及 、 R 的值。位置传感器会改变 、 x 中的位置参数，因此我们需要进行相应的调整。

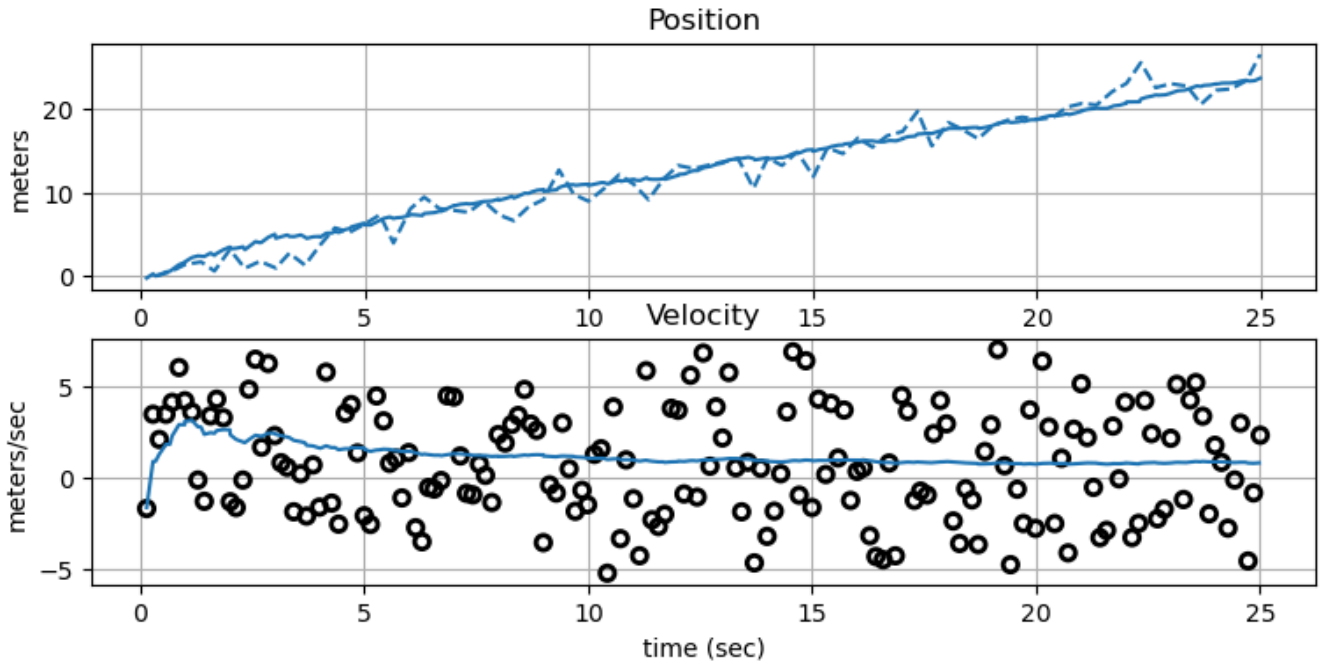
H=[10]R=σ2ps

轮速传感器会改变 x 中的速度参数，因此我们规定如下：

H=[01]R=σ2wheel

```
def gen_sensor_data(t, ps_std, wheel_std):
    # generate simulated sensor data
    pos_data, vel_data = [], []
    dt = 0.
    for i in range(t*3):
        dt += 1/3.
        t_i = dt + randn() * .01 # time jitter
        pos_data.append([t_i, t_i + randn()*ps_std])

    dt = 0.
    for i in range(t*7):
        dt += 1/7.
        t_i = dt + randn() * .006 # time jitter
        vel_data.append([t_i, 1. + randn()*wheel_std])
    return pos_data, vel_data


def plot_fusion(xs, ts, zs_ps, zs_wheel):
    xs = np.array(xs)
    plt.subplot(211)
    plt.plot(zs_ps[:, 0], zs_ps[:, 1], ls='--', label='Pos Sensor')
    plot_filter(xs=ts, ys=xs[:, 0], label='Kalman filter')
    set_labels(title='Position', y='meters',)

    plt.subplot(212)
    plot_measurements(zs_wheel[:, 0], zs_wheel[:, 1],  label='Wheel')
    plot_filter(xs=ts, ys=xs[:, 1], label='Kalman filter')
    set_labels('Velocity', 'time (sec)', 'meters/sec')


def fusion_test(pos_data, vel_data, wheel_std, ps_std):
    kf = KalmanFilter(dim_x=2, dim_z=1)
    kf.F = array([[1., 1.], [0., 1.]])
    kf.H = array([[1., 0.], [1., 0.]])
    kf.x = array([[0.], [1.]])
    kf.P *= 100

    xs, ts = [],  []

    # copy data for plotting
    zs_wheel = np.array(vel_data)
    zs_ps = np.array(pos_data)

    last_t = 0
    while len(pos_data) > 0 and len(vel_data) > 0:
        if pos_data[0][0] < vel_data[0][0]:
            t, z = pos_data.pop(0)
            dt = t - last_t
            last_t = t

            kf.H = np.array([[1., 0.]])
            kf.R[0,0] = ps_std**2
        else:
            t, z = vel_data.pop(0)
            dt = t - last_t
            last_t = t

            kf.H = np.array([[0., 1.]])
            kf.R[0,0] = wheel_std**2
```

```
        kf.F[0,1] = dt
        kf.Q = Q_discrete_white_noise(2, dt=dt, var=.02)
        kf.predict()
        kf.update(np.array([z]))

        xs.append(kf.x.T[0])
        ts.append(t)
    plot_fusion(xs, ts, zs_ps, zs_wheel)

random.seed(1123)
pos_data, vel_data = gen_sensor_data(25, 1.5, 3.0)
fusion_test(pos_data, vel_data, 1.5, 3.0);
```



追踪球体运动轨迹

现在，让我们来考虑这样一种情况：我们所追踪的物体的运动规律受到某种限制。在真空中抛出的球必须遵循牛顿定律。在恒定的重力场中，球会沿抛物线轨迹运动。当然，我假设你已经了解这个公式的推导过程了。

y=g2t2+vy0t+y0x=vx0t+x0

其中， g 表示万有引力常数， t 表示时间， vx0 和 vy0 分别表示物体在 x 轴和 y 轴方向的初始速度。如果球以 v 的初速度、与地平线成 θ 的夹角被抛出，那么我们可以计算出 vx0 和 vy0 的值。

vx0=vcosθvy0=vsinθ

由于我们没有真实的数据，因此我们将先编写一个用于模拟球的程序。和往常一样，我们会加入一个与时间无关的噪声因素，这样就能模拟出有噪声的传感器输出了。

```
from math import radians, sin, cos
import math

def rk4(y, x, dx, f):
    """computes 4th order Runge-Kutta for dy/dx.
```

```
    y is the initial value for y
    x is the initial value for x
    dx is the difference in x (e.g. the time step)
    f is a callable function (y, x) that you supply to
      compute dy/dx for the specified values.
    """

    k1 = dx * f(y, x)
    k2 = dx * f(y + 0.5*k1, x + 0.5*dx)
    k3 = dx * f(y + 0.5*k2, x + 0.5*dx)
    k4 = dx * f(y + k3, x + dx)

    return y + (k1 + 2*k2 + 2*k3 + k4) / 6.

def fx(x,t):
    return fx.vel

def fy(y,t):
    return fy.vel - 9.8*t


class BallTrajectory2D(object):
    def __init__(self, x0, y0, velocity,
                 theta_deg=0.,
                 g=9.8,
                 noise=[0.0, 0.0]):
        self.x = x0
        self.y = y0
        self.t = 0
        theta = math.radians(theta_deg)
        fx.vel = math.cos(theta) * velocity
        fy.vel = math.sin(theta) * velocity
        self.g = g
        self.noise = noise


    def step(self, dt):
        self.x = rk4(self.x, self.t, dt, fx)
        self.y = rk4(self.y, self.t, dt, fy)
        self.t += dt
        return (self.x + randn()*self.noise[0],
                self.y + randn()*self.noise[1])
```

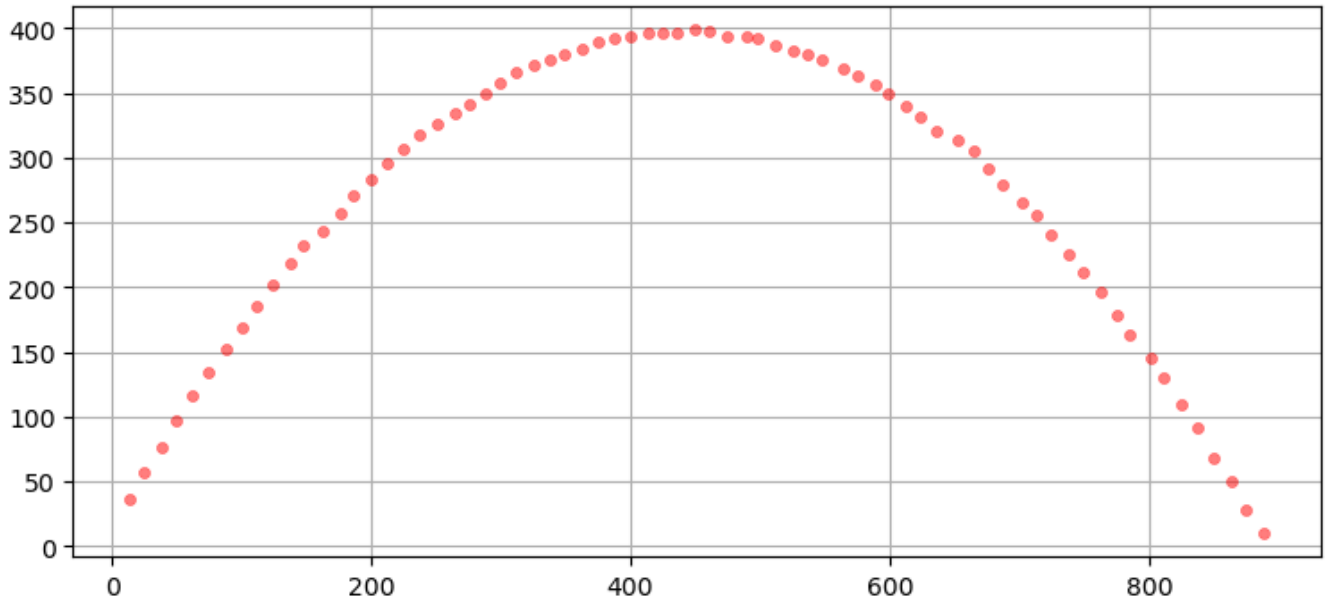
因此，要创建一条以点(0, 15)为起点、速度为 100 米/秒、角度为 60°的轨迹，我们可以这样表示：

```
traj = BallTrajectory2D(x0=0, y0=15, velocity=100, theta_deg=60)
```

然后在每个时间步长上调用 `traj.step(t)` 。我们来测试一下吧。

```
def test_ball_vacuum(noise):
    y = 15
    x = 0
    ball = BallTrajectory2D(x0=x, y0=y,
                            theta_deg=60., velocity=100.,
                            noise=noise)
    t = 0
    dt = 0.25
    while y >= 0:
        x, y = ball.step(dt)
        t += dt
```

```
        if y >= 0:
            plt.scatter(x, y, color='r', marker='.', s=75, alpha=0.5)

    plt.axis('equal');

#test_ball_vacuum([0, 0]) # plot ideal ball position
test_ball_vacuum([1, 1]) # plot with noise
```



这看起来挺合理的，那我们就继续吧（给读者的练习：请更全面地验证这个模拟结果）。

请选择状态变量。

我们可能会想使用与追踪狗的进程中所使用的相同的状态变量。不过，这样做是行不通的。需要记住的是，卡尔曼滤波的状态转换必须按照 ¯x=Fx+Bu 的形式来表示，这意味着我们必须根据先前的状态来计算当前状态。我们假设球是在真空中运动的，因此 x 方向上的速度是恒定的，而 y 方向上的加速度则完全由重力常数 g 决定。我们可以利用著名的欧拉方法来对牛顿方程进行离散化处理，其表达式为 Δt 。

xt=xt−1+vx(t−1)Δtvxt=vx(t−1)yt=yt−1+vy(t−1)Δtvyt=−gΔt+vy(t−1)

> 侧边栏：欧拉法是一种通过逐步积分来求解微分方程的方法。该方法假设在某个时间点上，函数的斜率（即导数）是恒定的。在这里，位置的导数就是速度。在每个时间点上，我们都假设速度保持不变，从而计算出新的位置值，再更新下一个时间点上的速度值。当然，还有更精确的方法，比如龙格-库塔法。但由于欧拉法在每一步都会根据实际测量值来更新状态，因此其精度已经相当高了。如果你想使用龙格-库塔法，就需要自己编写相应的函数来计算状态转换，然后再利用卡尔曼滤波公式来更新协方差矩阵。

这意味着，我们需要在卡尔曼滤波器中考虑 y 的加速度因素，而无需考虑 x 的加速度。由此可确定以下状态变量。

x=[x⋅xy⋅y¨y]T

不过，这种加速度实际上是由重力产生的，而重力是一个恒定不变的物理量。我们不必让卡尔曼滤波器去追踪这个恒定值，而可以将重力视为一种“控制输入”。换句话说，重力是一种以固定方式影响系统行为的力，它在球的整个飞行过程中都持续发挥作用。

状态预测的方程为 $\bar{x}=Fx+Bu$ 。 F 就是我们常用的状态转换函数，我们将利用它来模拟球的位置和速度。向量 u 用于指定输入到滤波器中的控制信号。对于汽车而言，这些控制信号包括油门和刹车的踩踏程度、方向盘的转动角度等等。而对于我们的球来说，控制信号就是重力。矩阵 B 则描述了这些控制信号如何影响系统的行为。同样地，对于汽车而言， B 会将油门和刹车的输入转化为速度的变化，将方向盘的输入转化为车辆的位置和行驶方向的变化。对于我们的球跟踪问题，该矩阵则用于计算由于重力而产生的速度变化。我们稍后会详细讨论这一点。目前，我们先确定状态变量的具体形式吧。

$x=[x\cdot xy\cdot y]T$

设计状态转换函数

我们的下一步是设计状态转换函数。需要记住的是，状态转换函数实际上是一个矩阵 F 。我们将该矩阵与系统的当前状态相乘，从而得到下一个状态，也就是先前的状态 $\bar{x}=Fx$ 。

我就不再详细阐述了，因为这与我们在前一章中讨论的一维情况非常相似。关于位置和速度的状态方程如下：

$\bar{x}=(1*x)+(\Delta t*vx)+(0*y)+(0*vy)\bar{v}x=(0*x)+(1*vx)+(0*y)+(0*vy)\bar{y}=(0*x)+(0*vx)+(1*y)+(\Delta t*vy)\bar{v}y=(0*x)+(0*vx)+(0*y)+(1*vy)$

请注意，所有公式中都不包含引力常数 g 。正如我在之前的解释中提到的，我们将利用卡尔曼滤波器的控制输入来处理重力效应。用矩阵形式表示就是：

$F=\begin{bmatrix}1&\Delta t&0&0\\0&1&0&0\\0&0&1&\Delta t\\0&0&0&1\end{bmatrix}$

设计控制输入函数

我们将利用控制输入来抵消重力的影响。在 $\bar{x}$ 中加入 Bu 这一项，是为了反映由于重力作用， $\bar{x}$ 发生了多大的变化。可以说， Bu 实际上包含了 $[\Delta xg\Delta\cdot xg\Delta yg\Delta\cdot yg]T$ 的内容。

如果我们观察这些离散化的方程，就会发现：重力只会对 y 的速度产生影响。

$xt=xt-1+vx(t-1)\Delta tvxt=vxt-1yt=yt-1+vy(t-1)\Delta tvyt=-g\Delta t+vy(t-1)$

因此，我们希望 Bu 的乘积等于 $[000-g\Delta t]T$ 。从某种意义上说，如何定义 B 和 u 并不重要，只要将它们相乘后能得到这个结果即可。例如，我们可以这样定义 $B=1$ 和 $u=[000-g\Delta t]T$ 。不过，这种定义方式与我们对 B 和 u 的定义并不相符。在那些定义中， u 是控制输入，而 B 则是控制函数。对于 y 的速度来说，控制输入就是 $-g$ 。所以，这只是其中一种可能的定义方式而已。

$B=\begin{bmatrix}0&0&0\\0&0&0\\0&0&0\\0&0&\Delta t\end{bmatrix},u=\begin{bmatrix}0\\0\\-g\end{bmatrix}$

在我看来，这似乎有些过头了。我建议，或许可以让 u 来负责控制 x 和 y 这两个维度。

$B=\begin{bmatrix}0&0\\0&0\\0&0\\0&\Delta t\end{bmatrix},u=[0-g].$

你可能更愿意只提供那些确实存在的控制输入。由于 和 x 并没有对应的控制输入，因此我们只能放弃对它们的操作。

B=⎡⎣⎢⎢000Δt⎤⎦⎥⎥,u=[−g].

我见过有人使用 。

虽然这样做确实能得到正确的结果，但我不想在 u 上花费太多时间。因为时间并非一种控制输入参数，它只是我们用来将控制输入转化为状态变化的媒介而已。而将控制输入转化为状态变化，才是 B 的职责所在。

设计测量函数

该测量函数定义了如何利用方程 z=Hx ，将状态变量转化为可测量的数值。我们假设有一个传感器能够提供球在(x,y)坐标系中的位置信息，但无法测量球的速度或加速度。因此，我们的测量函数必须满足以下条件：

[zxzy]=[10000010]⎡⎣⎢⎢⎢x˙xy˙y⎤⎦⎥⎥⎥

在……的地方/在何处

H=[10000010]

设计测量噪声矩阵

与机器人的情况类似，我们假设在 、 x 以及 、 y 这些参数上，误差是相互独立的。在这种情况下，我们先假设 x 和 y 方向的测量误差均为 0.5 平方米。因此，

R=[0.5000.5]

设计过程噪声矩阵

我们假设球体在真空中运动，因此不存在任何过程噪声。我们有 4 个状态变量，因此需要一个 4×4 协方差矩阵来描述这些状态变量之间的关系。

Q=⎡⎣⎢⎢0000000000000000⎤⎦⎥⎥

确定初始条件

在测试状态转换函数时，我们已经执行过这一步骤。请回想一下，我们是利用三角函数来计算 x 和 y 的初始速度的，并且确定了 x 的值。

```
omega = radians(omega)
vx = cos(omega) * v0
vy = sin(omega) * v0

f1.x = np.array([[x, vx, y, vy]]).T
```

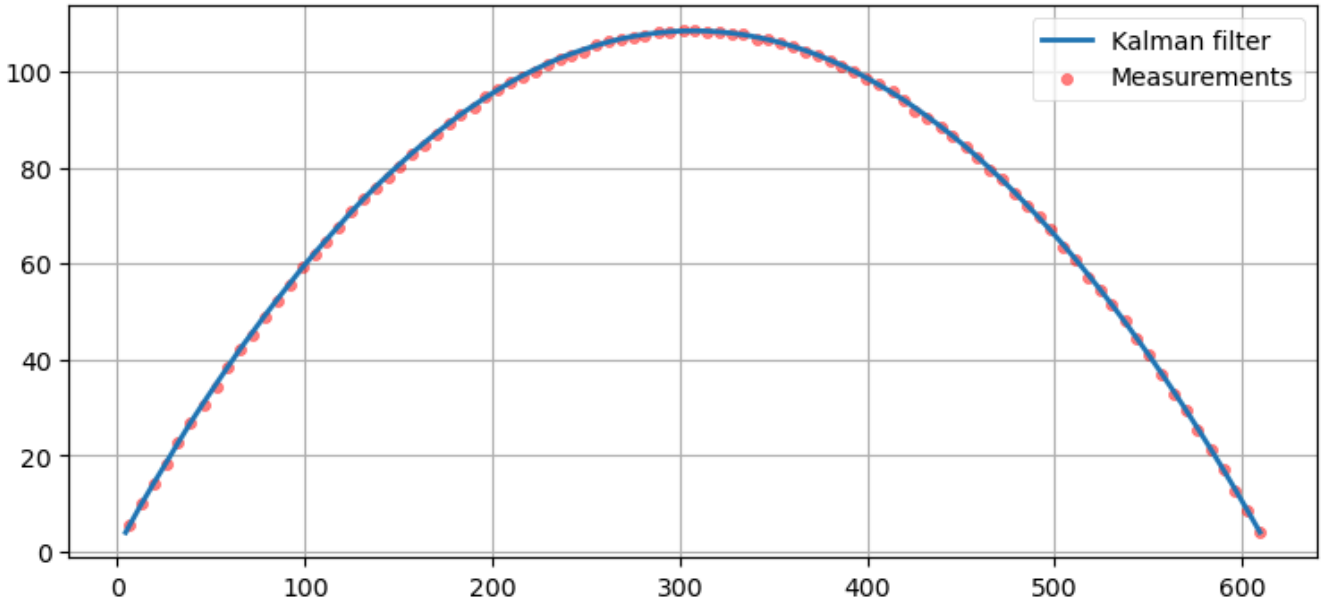
所有步骤都完成后，我们就准备好了来实现这个滤波器并进行测试。首先，是具体的实现过程：

```
from math import sin, cos, radians
```

```
def ball_kf(x, y, omega, v0, dt, r=0.5, q=0.):
    kf = KalmanFilter(dim_x=4, dim_z=2, dim_u=1)

    kf.F = np.array([[1., dt, 0., 0.],   # x   = x0 + dx*dt
                     [0., 1., 0., 0.],   # dx  = dx0
                     [0., 0., 1., dt],   # y   = y0 + dy*dt
                     [0., 0., 0., 1.]])  # dy  = dy0

    kf.H = np.array([[1., 0., 0., 0.],
                     [0., 0., 1., 0.]])

    kf.B = np.array([[0., 0., 0., dt]]).T
    kf.R *= r
    kf.Q *= q

    omega = radians(omega)
    vx = cos(omega) * v0
    vy = sin(omega) * v0
    kf.x = np.array([[x, vx, y, vy]]).T
    return kf
```

现在，我们将通过使用球体模拟类来生成球的各项测量数据，从而测试该滤波器的性能。

```
def track_ball_vacuum(dt):
    global kf
    x, y = 0., 1.
    theta = 35.  # launch angle
    v0 = 80.
    g = np.array([[-9.8]])  # gravitational constant
    ball = BallTrajectory2D(x0=x, y0=y, theta_deg=theta, velocity=v0,
                            noise=[.2, .2])
    kf = ball_kf(x, y, theta, v0, dt)

    t = 0
    xs, ys = [], []
    while kf.x[2] > 0:
        t += dt
        x, y = ball.step(dt)
        z = np.array([[x, y]]).T

        kf.update(z)
        xs.append(kf.x[0])
        ys.append(kf.x[2])
        kf.predict(u=g)
        p1 = plt.scatter(x, y, color='r', marker='.', s=75, alpha=0.5)
    p2, = plt.plot(xs, ys, lw=2)
    plt.legend([p2, p1], ['Kalman filter', 'Measurements'],
               scatterpoints=1)

track_ball_vacuum(dt=1./10)
```



我们可以看到，卡尔曼滤波器在追踪球的位置方面表现得相当不错。不过，正如之前所解释的，这其实是个过于简单的例子——因为我们不存在任何过程噪声。在真空中，我们可以以任意精确度来预测物体的轨迹；在这个例子中，使用卡尔曼滤波器其实毫无必要。采用最小二乘法进行曲线拟合，也会得到完全相同的结果。

追踪空中的球体

对于这个问题，我们假设自己正在追踪一个在地球大气层中运动的球体。球的运动轨迹会受到风、空气阻力以及球体自身旋转的影响。我们假设所使用的传感器是一台摄像机；虽然我们不会实际编写相关的代码，但可以想象，该摄像机会通过某种图像处理方式来确定球体的位置。在计算机视觉领域，这种处理方式被称为“目标检测”。不过，图像处理算法并不完美：在每一帧图像中，要么无法检测到球体，要么会检测到与球体无关的虚假目标。最后，我们不假设已知球的起始位置、角度或旋转状态；跟踪算法必须根据所获得的测量数据来开始球的跟踪过程。在这里，我们所做的最主要简化就是将问题简化为二维空间中的运动情况；我们假设球体始终沿着与摄像机传感器平面垂直的方向运动。在现阶段，我们不得不进行这样的简化处理，因为我们还没有探讨如何从只能提供二维数据的相机中提取三维信息。

实现空气阻力控制

我们的第一步是建立描述球在空气中运动的数学模型。有多种处理方法可供选择。一种较为完善的解决方案会考虑到各种因素，比如球的表面粗糙度（这会以非线性的方式影响阻力）、马格努斯效应（球的旋转会导致球的一侧相对于空气的速度高于另一侧，因此两侧的阻力系数也不同）、升力、湿度、空气密度等因素。不过，我认为读者应该对球体物理学的细节不感兴趣，因此我仅讨论空气阻力对不旋转的棒球的影响。我会使用 Nicholas Giordano 和 Hisao Nakanishi 在《计算物理学》[1997]一文中提出的数学模型。当然，这种模型并没有考虑所有因素。最详细的分析则可以在 Alan Nathan 的网站上找到：http://baseball.physics.illinois.edu/index.html。我在自己的计算机视觉研究中会用到他的数学模型，但我不想被那些过于复杂的模型所困扰。

重要提示：在继续之前，请注意，您无需理解接下来的物理原理，就能继续使用卡尔曼滤波器。我的目标是尽可能真实地模拟棒球在现实世界中的运动行为，这样我们就能测试卡尔曼滤波器在实际应用中的表现如何。在现实世界中，我们通常无法完全模拟某个系统的物理特性，因此只能使用能够反映系统整体行为的模型来替代。接着，我们需要调整测量噪声和系统噪声，直到滤波器能够很好地处理我们的数据。不过，这样做也有风险：虽然可以很容易地调整卡尔曼滤波器，使其在测试数据上表现完美，但一旦面对略有不同的数据，其性能就会大幅下降。这或许正是设计卡尔曼滤波器最困难的部分，也是为什么它被称为“黑科技”的原因。

我不喜欢那些不加解释就直接实施某种机制的书籍。因此，现在我来详细解释一下球在空气中运动的物理原理。如果您对具体的模拟过程不感兴趣，可以跳过这部分内容。

在空气中运动的球会遇到风阻。这种风阻会对球施加一种力，这种力被称为“阻力”，它会影响球的运动轨迹。在 Giordano 的理论中，这一现象被如此描述：

Fdrag=−B2v2

其中， B2 是通过实验确定的系数，而 v 则代表物体的速度。 Fdrag 可以分解为 x 和 y 这两个部分。

Fdrag,x=−B2vvxFdrag,y=−B2vvy

如果 m 表示球的质量，那么我们可以利用 F=ma 来计算球的加速度。

ax=−B2mvvxay=−B2mvvy

Giordano 为 B2m 提供了以下计算公式。该公式考虑了空气密度、棒球的横截面尺寸以及其表面粗糙度等因素。需要明白，这是一个基于风洞测试和各种简化假设得出的近似值。所有数值均以国际单位制表示：速度以米/秒为单位，时间以秒为单位。

B2m=0.0039+0.00581+exp[(v−35)/5]

首先，让我们从对真空中球体运动轨迹的欧拉离散化处理开始：

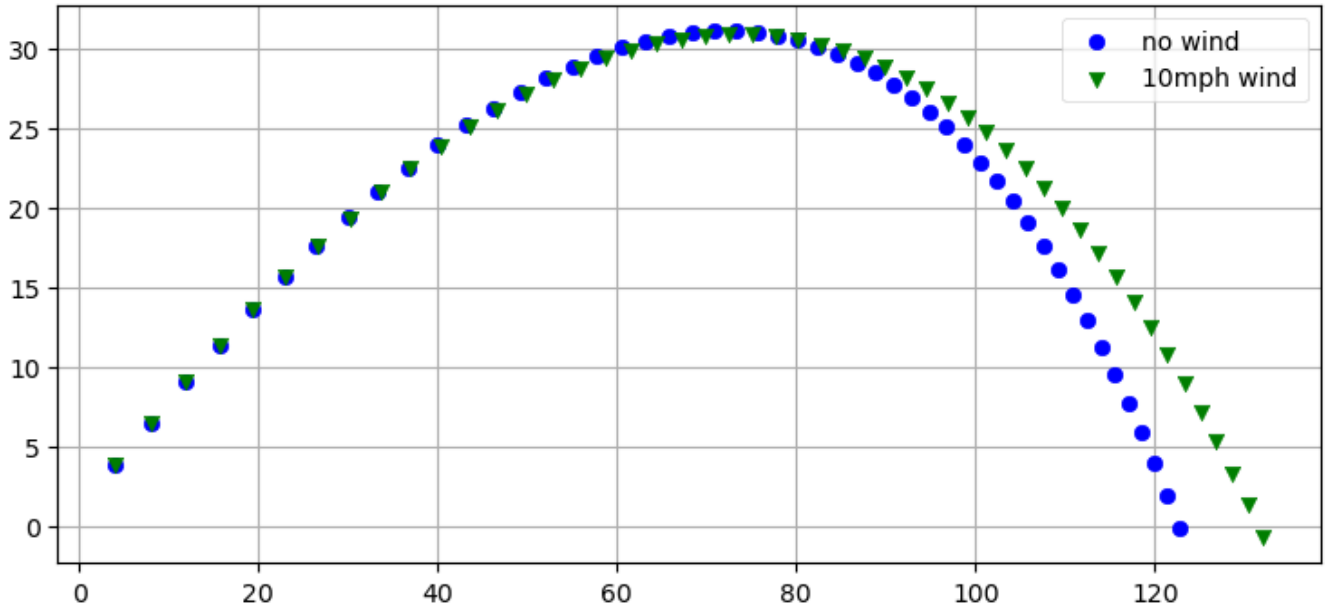
我们可以通过将 accel∗Δt 纳入速度更新公式中，从而将这种力（加速度）纳入我们的计算公式中。不过，我们需要减去这个分量，因为阻力会降低物体的速度。实现这一点的代码相当简单，我们只需把该力分解为 x 和 y 这几个分量即可。

我不想再详细讨论这个问题了，因为计算物理学超出了本书的讨论范围。需要指出的是，要想获得更精确的模拟结果，就必须考虑高度、温度、球的旋转速度等众多因素。如果你感兴趣的话，艾伦·内森的相关研究可以为你提供参考。我的目的是让模拟结果更贴近现实世界的情况，从而测试卡尔曼滤波器所使用的简单预测模型在这种现实情况下的表现如何。你的模型永远无法完全再现现实世界中的实际情况。因此，设计出优秀的卡尔曼滤波器的关键在于仔细测试其在真实数据下的表现。

下面的代码模拟了棒球在海上平面上、有风的情况下的飞行轨迹。我分别绘制了在没有风的情况下，以及有 10 英里/小时顺风的情况下，棒球的飞行轨迹。棒球相关的统计数据通常以美国单位来表示，这里也采用同样的单位制（http://en.wikipedia.org/wiki/United_States_customary_units）。需要说明的是，110 英里/小时的速度是棒球打出本垒打时的典型出口速度。

```
from math import sqrt, exp

def mph_to_mps(x):
```

```
    return x * .447

def drag_force(velocity):
    """ Returns the force on a baseball due to air drag at
    the specified velocity. Units are SI"""

    return velocity * (0.0039 + 0.0058 /
            (1. + exp((velocity-35.)/5.)))

v = mph_to_mps(110.)
x, y = 0., 1.
dt = .1
theta = radians(35)

def solve(x, y, vel, v_wind, launch_angle):
    xs = []
    ys = []
    v_x = vel*cos(launch_angle)
    v_y = vel*sin(launch_angle)
    while y >= 0:
        # Euler equations for x and y
        x += v_x*dt
        y += v_y*dt

        # force due to air drag
        velocity = sqrt((v_x-v_wind)**2 + v_y**2)
        F = drag_force(velocity)

        # euler's equations for vx and vy
        v_x = v_x - F*(v_x-v_wind)*dt
        v_y = v_y - 9.8*dt - F*v_y*dt

        xs.append(x)
        ys.append(y)

    return xs, ys

x, y = solve(x=0, y=1, vel=v, v_wind=0, launch_angle=theta)
p1 = plt.scatter(x, y, color='blue', label='no wind')

wind = mph_to_mps(10)
x, y = solve(x=0, y=1, vel=v, v_wind=wind, launch_angle=theta)
p2 = plt.scatter(x, y, color='green', marker="v",
                 label='10mph wind')
plt.legend(scatterpoints=1);
```



我们可以很容易地看出，球在真空中和空气中的运动轨迹是有所不同的。在上面的分析中，我对球在真空中的初始速度和发射角度都做了相同的设定。计算结果显示，球在真空中可以飞行 240 多米（接近 800 英尺）。而在空气中，球的飞行距离则略高于 120 米，大约 400 英尺。400 英尺这个距离对于一记击球精准的本垒打来说是很合理的，因此我们可以相信，我们的模拟结果相当准确。

不用多说，我们现在就来创建一个球体运动模拟。该模拟会利用上述数学公式来生成更符合实际情况的球体运动轨迹。需要指出的是，由于空气阻力的非线性特性，我们无法用解析方法来确定球体在任意时刻的位置。因此，我们必须通过逐步计算的方式来得到球体的位置。我在这里使用了欧拉方法来求解这个问题；至于使用更精确的算法，比如龙格-库塔法，则留给读者作为练习吧。对于我们的目的来说，这种稍微复杂的处理方式其实没必要，因为我们所使用的时间步长下，各种算法之间的精度差异其实很小。

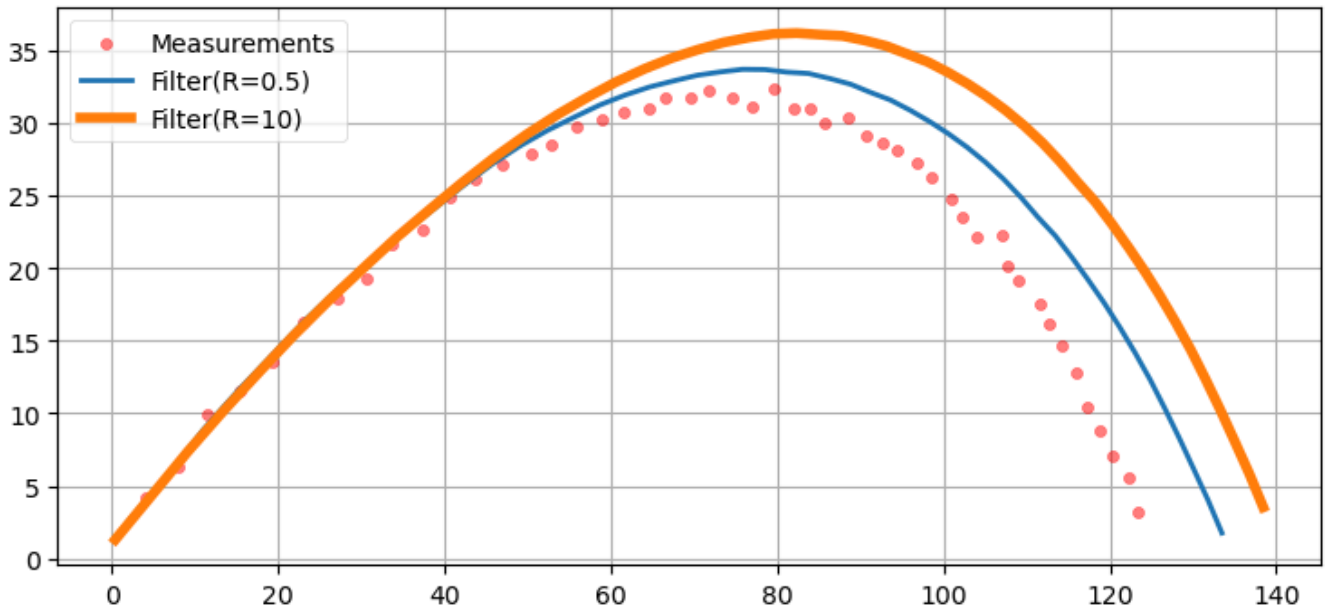
```
class BaseballPath:
    def __init__(self, x0, y0, launch_angle_deg, velocity_ms,
                 noise=(1.0, 1.0)):
        """ Create 2D baseball path object
           (x = distance from start point in ground plane,
            y=height above ground)

        x0,y0            initial position
        launch_angle_deg angle ball is travelling respective to
                         ground plane
        velocity_ms      speeed of ball in meters/second
        noise            amount of noise to add to each position
                         in (x, y)
        """

        omega = radians(launch_angle_deg)
        self.v_x = velocity_ms * cos(omega)
        self.v_y = velocity_ms * sin(omega)

        self.x = x0
        self.y = y0
        self.noise = noise


    def drag_force(self, velocity):
```

```
        """ Returns the force on a baseball due to air drag at
        the specified velocity. Units are SI
        """
        B_m = 0.0039 + 0.0058 / (1. + exp((velocity-35.)/5.))
        return B_m * velocity


    def update(self, dt, vel_wind=0.):
        """ compute the ball position based on the specified time
        step and wind velocity. Returns (x, y) position tuple.
        """

        # Euler equations for x and y
        self.x += self.v_x*dt
        self.y += self.v_y*dt

        # force due to air drag
        v_x_wind = self.v_x - vel_wind
        v = sqrt(v_x_wind**2 + self.v_y**2)
        F = self.drag_force(v)

        # Euler's equations for velocity
        self.v_x = self.v_x - F*v_x_wind*dt
        self.v_y = self.v_y - 9.81*dt - F*self.v_y*dt

        return (self.x + randn()*self.noise[0],
                self.y + randn()*self.noise[1])
```

现在，我们可以用该模型生成的测量数据来测试卡尔曼滤波器的性能了。

```
x, y = 0, 1.

theta = 35. # launch angle
v0 = 50.
dt = 1/10.   # time step
g = np.array([[-9.8]])

plt.figure()
ball = BaseballPath(x0=x, y0=y, launch_angle_deg=theta,
                    velocity_ms=v0, noise=[.3,.3])
f1 = ball_kf(x, y, theta, v0, dt, r=1.)
f2 = ball_kf(x, y, theta, v0, dt, r=10.)
t = 0
xs, ys = [], []
xs2, ys2 = [], []

while f1.x[2] > 0:
    t += dt
    x, y = ball.update(dt)
    z = np.array([[x, y]]).T

    f1.update(z)
    f2.update(z)
    xs.append(f1.x[0])
    ys.append(f1.x[2])
    xs2.append(f2.x[0])
    ys2.append(f2.x[2])
    f1.predict(u=g)
    f2.predict(u=g)

    p1 = plt.scatter(x, y, color='r', marker='.', s=75, alpha=0.5)
```

```
p2, = plt.plot(xs, ys, lw=2)
p3, = plt.plot(xs2, ys2, lw=4)
plt.legend([p1, p2, p3],
           ['Measurements', 'Filter(R=0.5)', 'Filter(R=10)'],
           loc='best', scatterpoints=1);
```



我绘制了两种不同卡尔曼滤波器设置下的输出结果。测量值用绿色圆点表示，R=0.5 时的卡尔曼滤波器结果用细绿色线表示，而 R=10 时的卡尔曼滤波器结果则用粗蓝色线表示。选择这些 R 值只是为了展示测量噪声对输出结果的影响而已，并不意味着这些参数就是最优的设计方案。

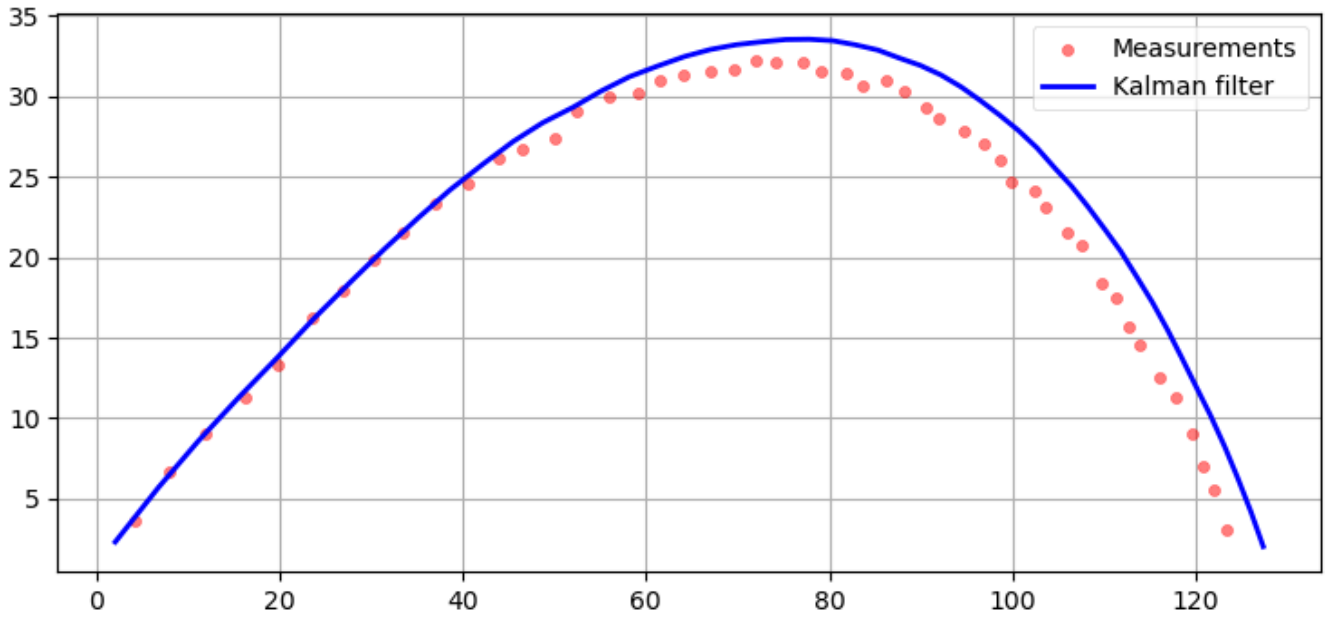
可以看出，这两种滤波器的表现都不理想。起初，两者都能很好地跟踪测量数据，但随着时间的推移，两者开始出现偏差。出现这种情况的原因是，空气阻力的状态模型是非线性的，而卡尔曼滤波器则假设该模型是线性的。回想一下我们在关于 g-h 滤波器的章节中讨论过的非线性问题，我们会明白为什么 g-h 滤波器总是会落后于系统的加速度。在这里也是如此：由于加速度为负值，卡尔曼滤波器总是会高估球的位置。只要加速度持续存在，滤波器就永远无法赶上实际情况，因此其偏差会持续扩大。

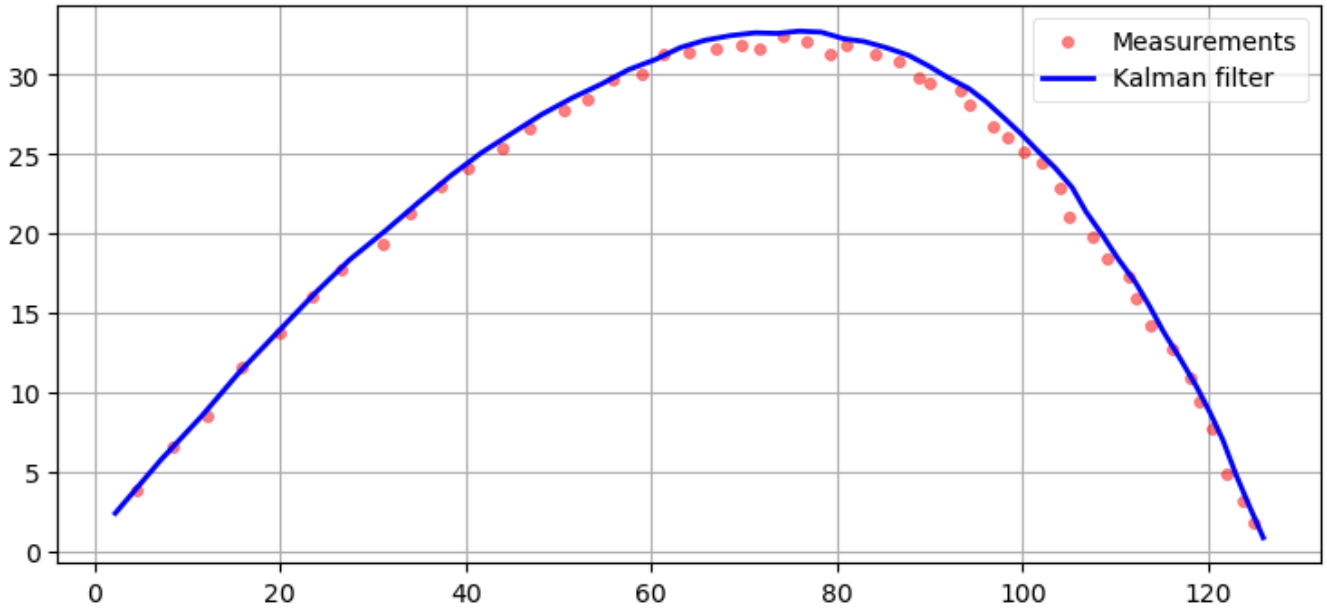
我们能做些什么来改善这个问题呢？最好的方法是使用非线性卡尔曼滤波器来进行处理，我们将在后面的章节中详细介绍这一方法。不过，对于这个问题，还有另一种所谓的“工程学上的解决方案”。卡尔曼滤波器假设球体处于真空环境中，因此不存在任何干扰因素。但实际上，球体处于空气中，大气会对球体施加力。我们可以把这种力视为一种干扰因素。不过，这种分析并不十分严谨：首先，这种力的分布并非高斯分布；其次，我们其实可以计算出这种力的大小，所以不能简单地说“这是随机的”就能解决问题。不过，让我们看看按照这种思路会得到怎样的结果吧。

以下代码实现了与之前相同的卡尔曼滤波器，只不过此时存在非零的过程噪声。我绘制了两个示例：一个是使用 `Q=.1` 的情况，另一个是使用 `Q=0.01` 的情况。

```
def plot_ball_with_q(q, r=1., noise=0.3):
    x, y = 0., 1.
    theta = 35. # launch angle
    v0 = 50.
    dt = 1/10.   # time step
    g = np.array([[-9.8]])
```

```
    ball = BaseballPath(x0=x,
                        y0=y,
                        launch_angle_deg=theta,
                        velocity_ms=v0,
                        noise=[noise,noise])
    f1 = ball_kf(x, y, theta, v0, dt, r=r, q=q)
    t = 0
    xs, ys = [], []

    while f1.x[2] > 0:
        t += dt
        x, y = ball.update(dt)
        z = np.array([[x, y]]).T

        f1.update(z)
        xs.append(f1.x[0])
        ys.append(f1.x[2])
        f1.predict(u=g)

        p1 = plt.scatter(x, y, c='r', marker='.', s=75, alpha=0.5)

    p2, = plt.plot(xs, ys, lw=2, color='b')
    plt.legend([p1, p2], ['Measurements', 'Kalman filter'])
    plt.show()

plot_ball_with_q(0.01)
plot_ball_with_q(0.1)
```

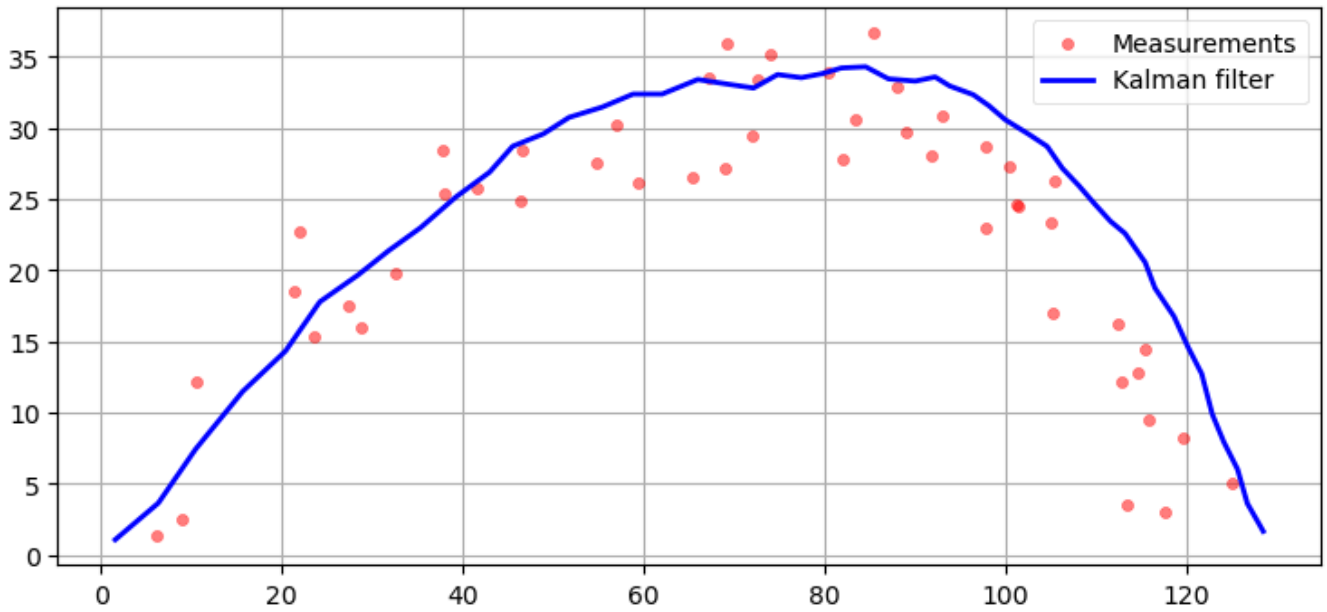


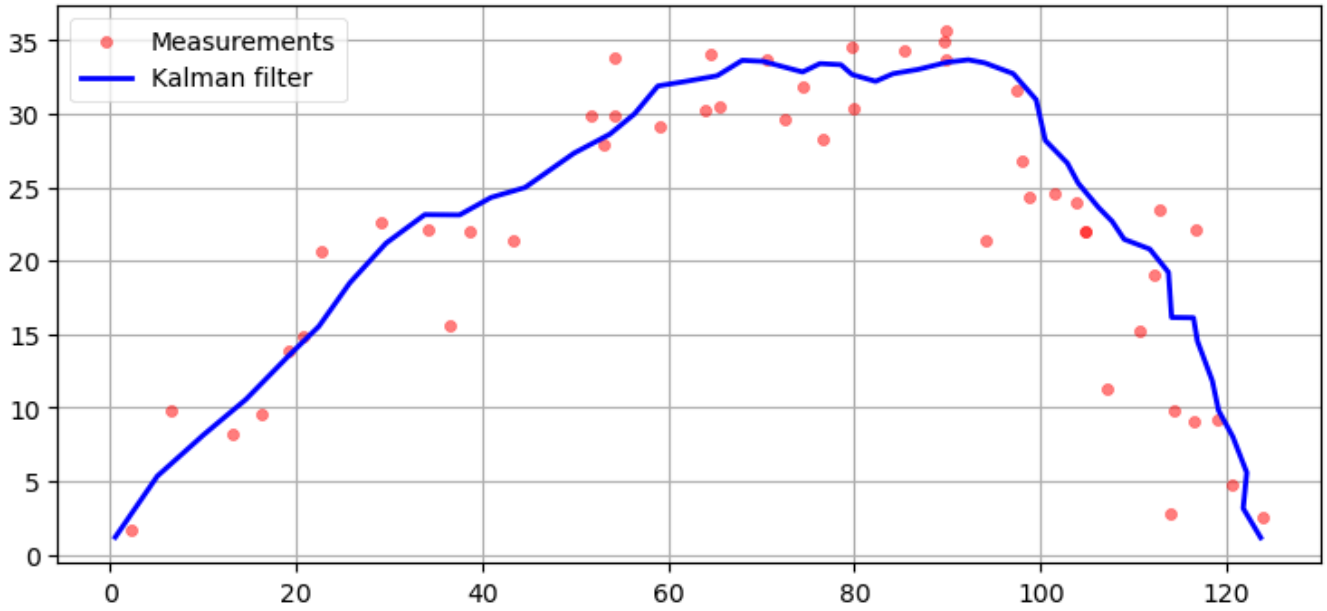


第二个滤波器在数据测量方面表现相当不错。虽然存在一点延迟，但幅度很小。

这是一种好的技术吗？通常来说并非如此，不过这也因具体情况而异。在这里，作用在球上的力的非线性特性相当稳定且规律。假设我们试图追踪一辆汽车——当汽车改变速度或转弯时，其加速度也会随之变化。如果我们把测量过程中产生的噪声幅度设置得比系统本身的噪声更大，那么滤波器就会更重视这些测量数据。如果你测量的数据中噪声不大，那么这种做法可能会有效。不过，请看下图：我增加了测量数据中的噪声幅度，效果如何呢？

```
plot_ball_with_q(0.01, r=3, noise=3.)
plot_ball_with_q(0.1, r=3, noise=3.)
```





这种输出效果糟糕透顶。线性卡尔曼滤波器别无选择，只能给予测量数据比预测结果更高的权重。但当测量数据存在噪声时，滤波器的输出就会完全受噪声的影响。正是线性卡尔曼滤波器的这一固有缺陷，促使人们开发出了非线性版本的滤波器。

也就是说，利用过程噪声来处理系统中的微小非线性误差是完全可行的。这也是卡尔曼滤波器所体现的“巧妙技巧”之一。我们对传感器和系统的建模从来都不可能做到完美。传感器的工作特性并非高斯分布，我们的过程模型也永远不可能完美无缺。我们可以通过将测量误差和过程误差设定得高于其理论上的正确值来弥补这些缺陷，但这样做的代价就是得到的解决方案并非最优解。当然，与其让卡尔曼滤波器无法正常工作，不如接受这种非最优的解决方案。不过，正如上面的图表所示，滤波器的输出结果很容易变得很糟糕。通常情况下，经过多次模拟和测试后，我们得到的滤波器在某些条件下表现不错。但当把该滤波器应用于实际数据时，由于条件略有不同，滤波器的性能就会变得很差。

目前，我们先暂且不讨论这个问题，因为在这个例子中，我们显然没有正确使用卡尔曼滤波器。我们将在后面的章节中重新探讨这个问题，看看使用各种非线性技术会带来怎样的效果。在某些领域，对于非线性问题，使用线性卡尔曼滤波器也是可行的。但通常情况下，你还是需要运用本书后面介绍的各种非线性技术。

参考文献

[1] Bar-Shalom, Yaakov 等人。《估计方法及其在跟踪与导航中的应用》。约翰·威利父子出版社，2001 年。

第 9 章：非线性滤波

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

我们所开发的卡尔曼滤波器依赖于线性方程，因此它只能处理线性问题。然而，现实世界是非线性的，所以迄今为止我们所研究的这种经典滤波器的作用其实非常有限。

在过程模型中可能存在非线性现象。比如，如果我们想追踪一个在大气中下落的物体，该物体的加速度取决于其所受到的阻力。而阻力又与空气密度有关，空气密度会随着高度的增加而降低。在一维情况下，这种现象可以用非线性微分方程来描述。

$$\ddot{x} = \frac{0.0034ge^{-x/22000}\dot{x}^2}{2\beta} - g$$

非线性的第二个来源在于测量过程本身。例如，雷达测量的是到目标的斜距，而我们实际上关心的是飞机在地面上的位置。根据勾股定理，我们可以得到以下非线性方程：

$$x = \sqrt{slant^2 - altitude^2}$$

卡尔曼滤波器的早期采用者们当然注意到了这些事实。在卡尔曼博士发表论文后不久，人们就开始研究如何将卡尔曼滤波器应用于非线性问题。

可以说，人们唯一会解的方程，其实就是 $Ax = b$ 这个方程。实际上，我们只会解线性方程组而已。无论给我什么样的线性方程组，我要么能解出来，要么能证明它无解。

凡是接受过数学或物理正规教育的人，都花了很多年时间学习各种解析方法来求解积分、微分方程等等。然而，即便是最简单的物理系统，其对应的方程也往往无法用解析方法来求解。我可以取一个原本可以用解析方法求解的方程，再在其中加入某个项，就能让该方程变得无法求解。这就引出了这样的笑话：物理学家们会说“假设有一头球形奶牛处于真空中的无摩擦表面上……”。如果不进行极端的简化处理，大多数物理问题都是无法用解析方法来求解的。

我们究竟是如何在计算机上模拟飞机周围的气流、预测天气或利用卡尔曼滤波器来追踪导弹的呢？我们依靠自己所熟悉的方法来解决问题：首先，将问题转化为线性方程组；然后，利用线性代数软件来求解近似解。

将非线性问题线性化后得到的结果往往不够精确。在卡尔曼滤波器或天气预报系统这类递归算法中，这些微小的误差可能会在每一步中相互叠加，从而迅速导致算法输出无意义的结果。

我们即将面临的，是一个极其棘手的难题。已经不存在那种显而易见、正确无误、在数学上最优的解决方案了。我们不得不使用各种近似方法来进行计算，这些方法必然会引入误差。同时，我们还将不得不不断与那些导致计算结果出现严重误差的滤波器作斗争。

在本章的剩余部分，我将详细阐述非线性卡尔曼滤波器所面临的具体问题。只有先了解非线性因素所带来的各种问题，才能设计出有效的滤波器。在后面的章节中，我将教你如何设计和实现各种类型的非线性滤波器。

非线性的问题所在

卡尔曼滤波的数学原理非常精妙，这很大程度上要归功于高斯方程的独特性质。高斯方程属于非线性方程，但当我们对高斯函数进行加减乘法运算时，得到的结果依然是高斯函数。这种情况其实非常罕见。 $\sin x * \sin y$ 运算后不会得到 sin 作为结果。

我所说的“线性”概念可能很容易理解，但实际上其中有一些细微之处需要留意。从数学角度来看，有两个关键要求：

- 可加性： $f(x+y) = f(x) + f(y)$
- 同质性： $f(ax) = af(x)$

由此可见，线性系统指的是这样一种系统：其输出与所有输入之和成线性比例关系。由此可推导出：如果输入为零，那么输出也必须为零，系统才能保持线性特性。以音频放大器为例：当我对着麦克风唱歌，而你同时说话时，输出应该是我们两个人声音的总和，再经过放大器的放大处理。但如果在输入为零的情况下，放大器仍然输出非零信号，那么这种线性关系就不复存在了。因为线性系统的要求就是：当输入为零时，输出也必须为零。显然，这种情况下输出应该保持不变，但如果放大器在输入为零时仍输出非零信号，那就不符合线性系统的要求了。

$$\begin{aligned} amp(voice) &= amp(voice + 0) \\ &= amp(voice) + amp(0) \\ &= amp(voice) + non_zero_value \end{aligned}$$

这显然是毫无道理的。因此，像这样的看似线性的方程……

$$L(f(t)) = f(t) + 1$$

因为 $L(0) = 1$ 的存在，所以该过程并非线性的。请小心行事！

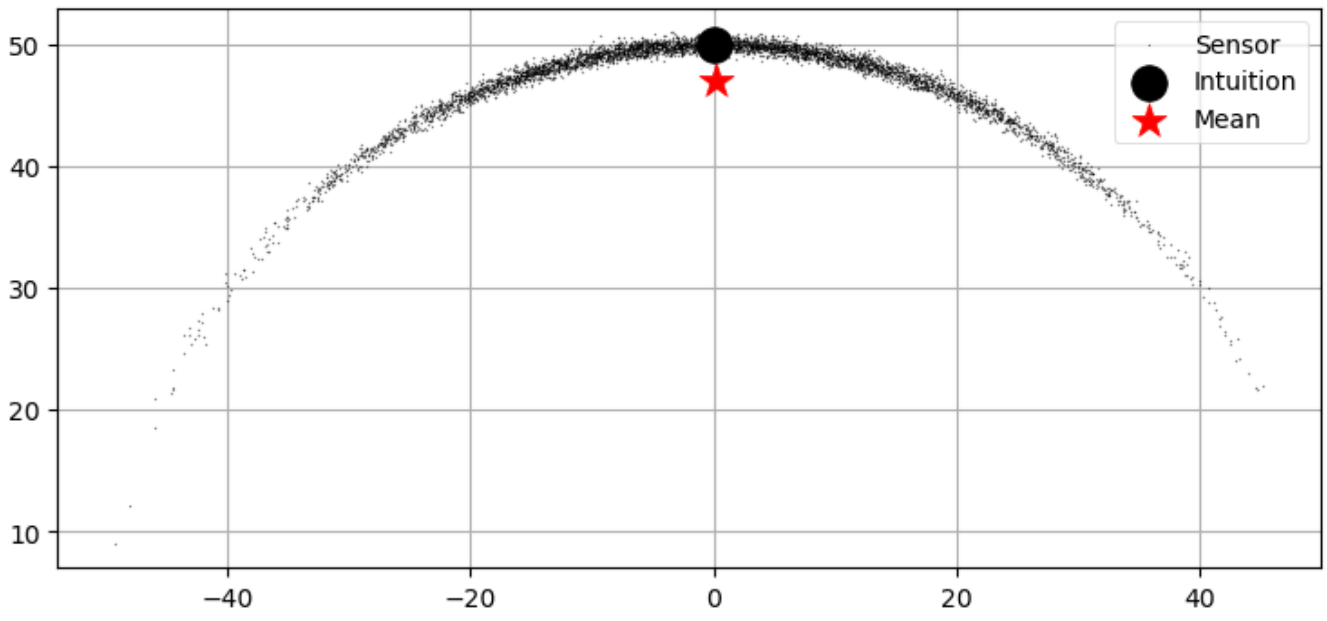
对这一问题的直观理解/对这个问题简单明了的看待方式

我特别喜欢以下这种解决问题的方式，这一方式借鉴了 Dan Simon 在《最优状态估计》[1]中的观点。假设我们面临一个跟踪问题：我们知道目标到我们的距离和方向角，但想要确定目标的确切位置。假设测得的距离为 50 公里，方向角为 90 度。假设距离和方向角的误差都遵循高斯分布。那么，在进行无限次测量后，目标位置的期望值是多少呢?

我一直建议利用直觉来获得洞察力，那么让我们看看这种方法在这个问题上的效果如何。我们可以这样推理：由于该范围的平均值是 50 公里，而角度的平均值是 90 度，因此答案应该是 x=0 公里，y=50 公里。

让我们把这些数据绘制成图表来查看吧。这里有 3000 个数据点：距离的数据服从 0.4 公里的正态分布，角度的数据则服从 0.35 弧度的正态分布。我们计算了所有这些点的平均值，并用星星来表示这个平均值。而我们的直观判断则用一个大圆圈来表示。

```
import numpy as np
from numpy.random import randn
import matplotlib.pyplot as plt

N = 5000
a = np.pi/2. + (randn(N) * 0.35)
r = 50.0     + (randn(N) * 0.4)
xs = r * np.cos(a)
ys = r * np.sin(a)

plt.scatter(xs, ys, label='Sensor', color='k',
            alpha=0.4, marker='.', s=1)
xmean, ymean = sum(xs) / N, sum(ys) / N
plt.scatter(0, 50, c='k', marker='o', s=200, label='Intuition')
plt.scatter(xmean, ymean, c='r', marker='*', s=200, label='Mean')
plt.axis('equal')
plt.legend();
```



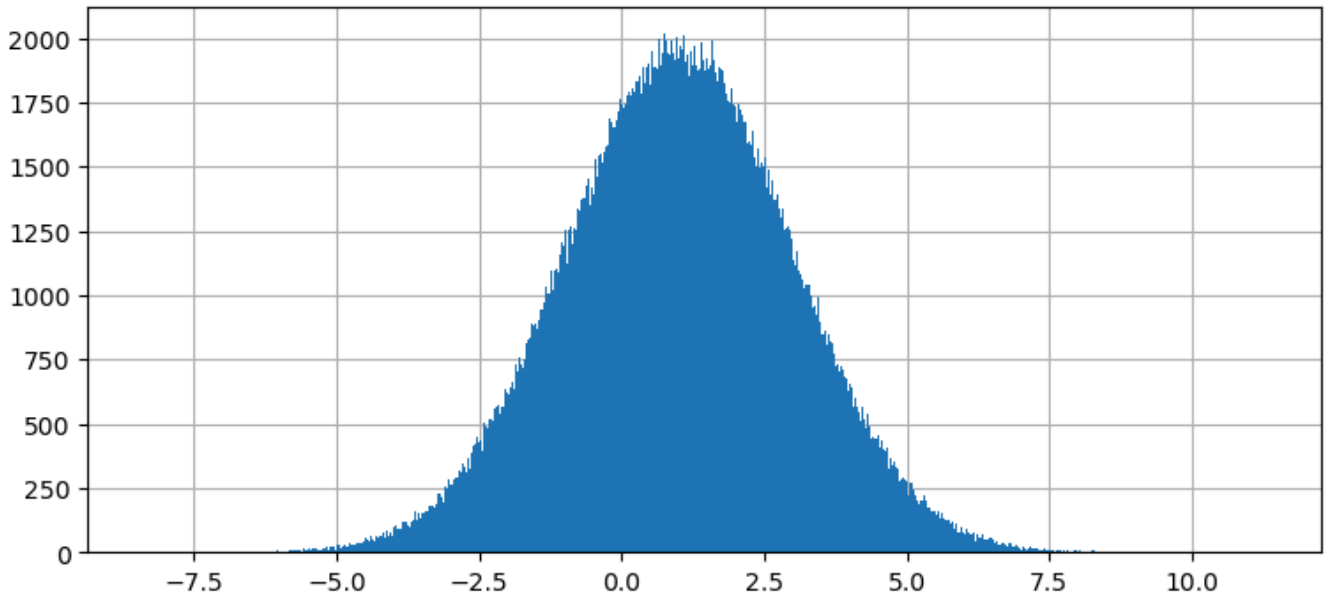
我们可以看出，直觉在这里不起作用了。由于该问题的非线性特性，所有的误差都倾向于朝某个方向偏移。这种偏差在多次迭代后会导致卡尔曼滤波器失去稳定性。即便滤波器没有失去稳定性，得到的结果也未必是最优的。将线性近似方法应用于非线性问题，必然会导致不准确的结果。

非线性函数对高斯函数的影响

高斯分布并不满足任意非线性函数的封闭性。回想一下卡尔曼滤波的公式：在每次迭代中，我们都会将表示状态的高斯分布通过相应的处理函数，从而得到下一个时间点上的高斯分布。由于我们的处理函数始终是线性

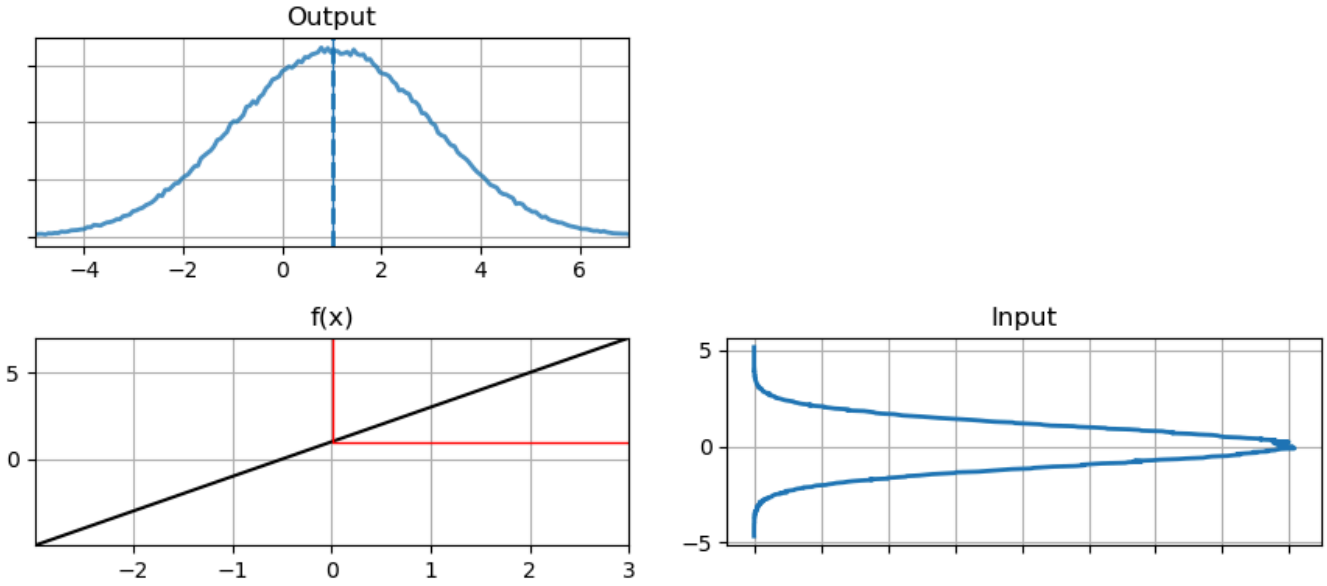
的，因此输出结果也始终是高斯分布。我们可以通过图表来直观地理解这一点。我会选取一个任意的高斯分布，将其通过某个函数处理后，再绘制出结果。虽然我们可以用解析方法来计算，但这里我选择用采样法来演示。我会生成 500,000 个服从正态分布的随机点，将它们通过该函数处理后，再绘制出结果。我选择这种方式，是因为下一个例子中的处理函数是非线性的，此时我们就无法用解析方法来计算了。

```
from numpy.random import normal

data = normal(loc=0., scale=1., size=500000)
plt.hist(2*data + 1, 1000);
```



这个结果并不意外。将高斯函数通过 $f(x) = 2x + 1$ 处理后，得到的结果依然是以 1 为中心的高斯函数。让我们同时来看看输入信号、非线性函数以及输出结果吧。

```
from kf_book.book_plots import set_figsize, figsize
from kf_book.nonlinear_plots import plot_nonlinear_func

def g1(x):
    return 2*x+1

plot_nonlinear_func(data, g1)
```

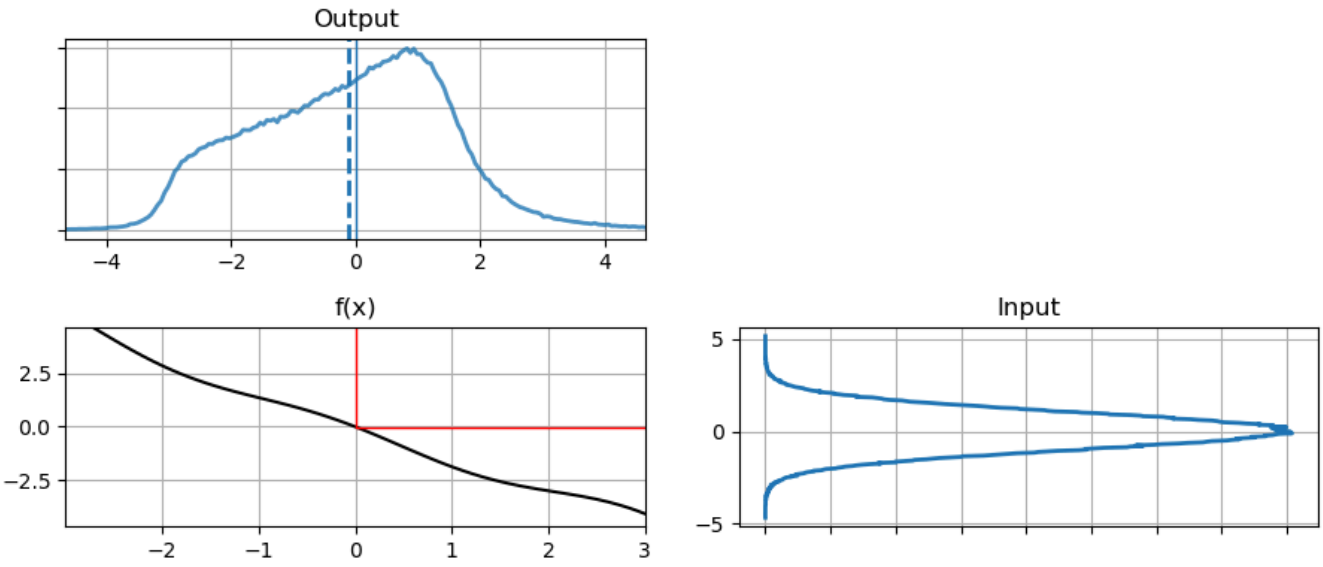


> 我在“Supporting_Notebooks”文件夹中的“Computing_and_Plotting_PDFs”文档里详细介绍了如何绘制高斯曲线图以及更多相关内容。您也可以在这里在线阅读该文档[1]。

标有“输入”的曲线图是原始数据的直方图。这些数据会被传递给函数 $f(x) = 2x + 1$，其处理结果则显示在图表的左下角。红线展示了某个具体数值 $x = 0$ 经过该函数处理后的结果。输入数据中的每一个数值都会以同样的方式被传递给右侧的输出函数。在计算输出结果时，我通过取所有数值的平均值来得到均值，并用蓝色虚线将结果标出。蓝色实线则代表了数值 $x = 0$ 的实际均值。输出结果呈现出高斯分布的特征，实际上确实如此。可以看出，输出数据的方差大于输入数据的方差，同时均值也从 0 偏移到了 1，这符合函数 $f(x) = 2x + 1$ 的处理方式。参数 $2x$ 会影响方差，而参数 $+1$ 则会影响均值。用蓝色虚线表示的计算均值与实际均值非常接近。如果我们在进行计算时使用更多的数值，就能无限接近真实值了。

现在，让我们来看看一个非线性函数，看看它如何影响概率分布。

```
def g2(x):
    return (np.cos(3*(x/2 + 0.7))) * np.sin(0.3*x) - 1.6*x

plot_nonlinear_func(data, g2)
```



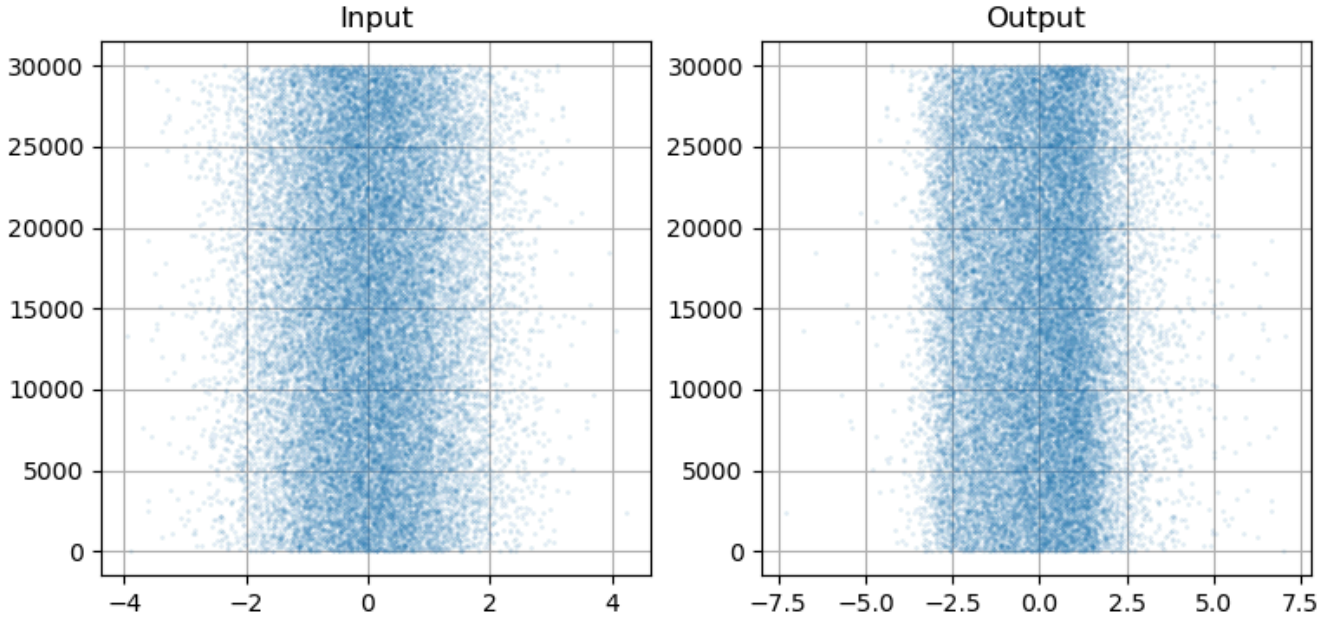
这个结果可能会让你有些惊讶。该函数看起来“相当”呈线性特征，但实际上，其输出的概率分布与高斯分布完全不同。请回想一下两个单变量高斯函数相乘时的公式：

$$\begin{aligned}\mu &= \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2} \\ \sigma &= \frac{1}{\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}}\end{aligned}$$

这些方程不适用于非高斯分布的情况，当然也不适用于上图“输出”中所显示的概率分布。

另一种查看相同数据的方式就是用散点图来表示。

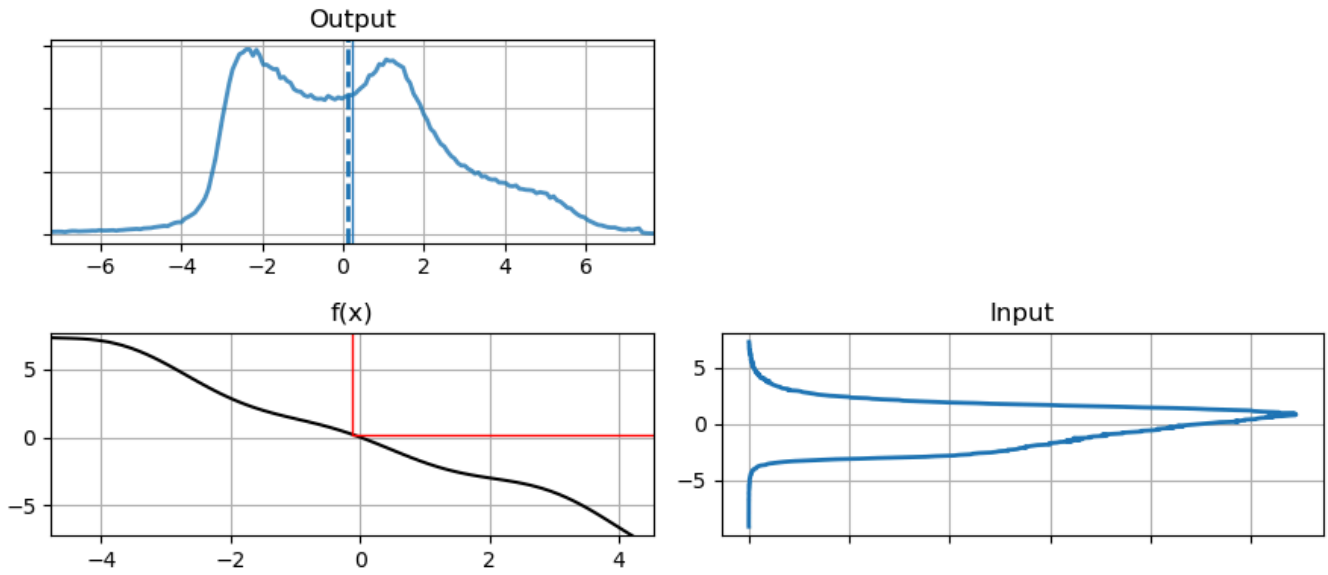
```
N = 30000
plt.subplot(121)
plt.scatter(data[:N], range(N), alpha=.1, s=1.5)
plt.title('Input')
plt.subplot(122)
plt.title('Output')
plt.scatter(g2(data[:N]), range(N), alpha=.1, s=1.5);
```



原始数据显然呈高斯分布，但经过 `g2(x)` 处理后的数据则不再符合正态分布。在-3 附近有一个明显的峰值区域，而数据点在该区域两侧的分布并不均匀。如果将此结果与上图中标记为“output”的概率密度函数进行比较，就能明白该概率密度函数的形状是如何与 `g(data)` 的数据分布相对应的了。

试想一下，这对上一章中提到的卡尔曼滤波算法意味着什么。所有的方程都假设：经过该处理函数后，高斯分布仍然保持为高斯分布。如果这一假设不成立，那么卡尔曼滤波的所有前提和保证条件就都不复存在了。让我们来看看，当再次将处理后的结果输入该函数时会发生什么，这实际上模拟了卡尔曼滤波的下一个时间步的处理过程。

```
y = g2(data)
plot_nonlinear_func(y, g2)
```



如图所示，概率函数已经严重偏离了原始的高斯分布形态。不过，该图表在 x=0 这一点上仍然具有某种对称性。让我们来计算一下平均值吧。

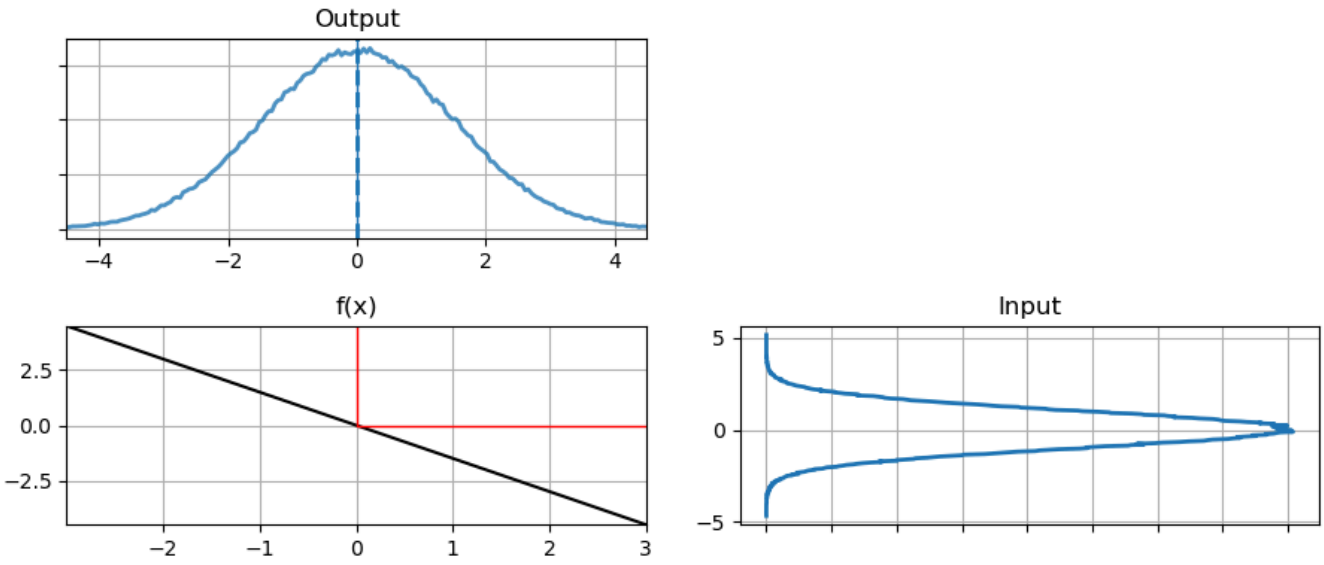
```
print('input  mean, variance: %.4f, %.4f' %
      (np.mean(data), np.var(data)))
print('output mean, variance: %.4f, %.4f' %
      (np.mean(y), np.var(y)))
```

```
input  mean, variance: 0.0022, 1.0000
output mean, variance: -0.1282, 2.4029
```

让我们将其与经过点(-2,3)和(2,-3)的直线函数进行比较。这条直线函数与我们所绘制的非线性函数非常接近。根据直线方程，我们有：

$$m = \frac{-3-3}{2-(-2)} = -1.5$$

```
def g3(x):
    return -1.5 * x

plot_nonlinear_func(data, g3)
out = g3(data)
print('output mean, variance: %.4f, %.4f' %
      (np.mean(out), np.var(out)))
```




```
output mean, variance: -0.0033, 2.2500
```

虽然各输出结果的形态差异很大，但它们的均值和方差却几乎相同。这让我们觉得，如果非线性方程“接近”线性方程的话，或许可以忽略这个问题。为了验证这一点，我们可以重复多次计算，然后比较结果。

```
out = g3(data)
out2 = g2(data)

for i in range(10):
    out = g3(out)
    out2 = g2(out2)
print('linear    output mean, variance: %.4f, %.4f' %
      (np.average(out), np.std(out)**2))
```

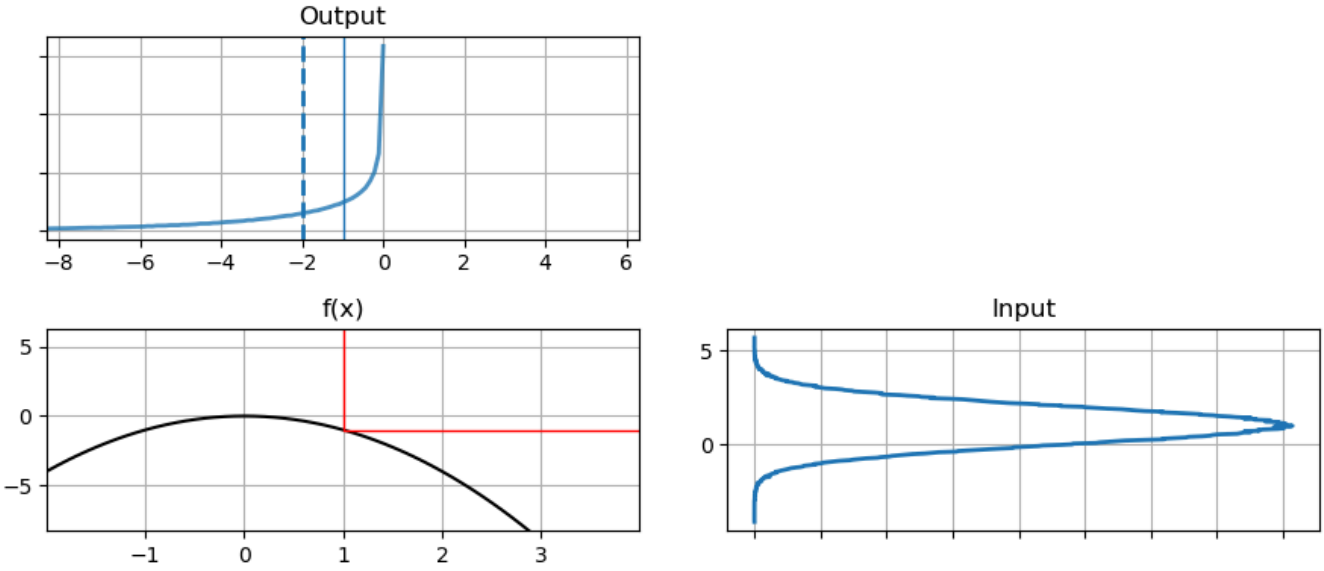
```
print('nonlinear output mean, variance: %.4f, %.4f' %
      (np.average(out2), np.std(out2)**2))
```

```
linear    output mean, variance: -0.1915, 7481.9567
nonlinear output mean, variance: -9.5578, 30512.3474
```

不幸的是，非线性模型并不稳定。其数值偏离了 0 这个平均值，偏差幅度相当大；同时，其方差也大了半个数量级。

我通过使用一个近似于直线的函数来最小化这个问题。如果该函数为 $y(x) = -x^2$ 的话，会怎么样呢?

```
def g3(x):
    return -x*x

data = normal(loc=1, scale=1, size=500000)
plot_nonlinear_func(data, g3)
```

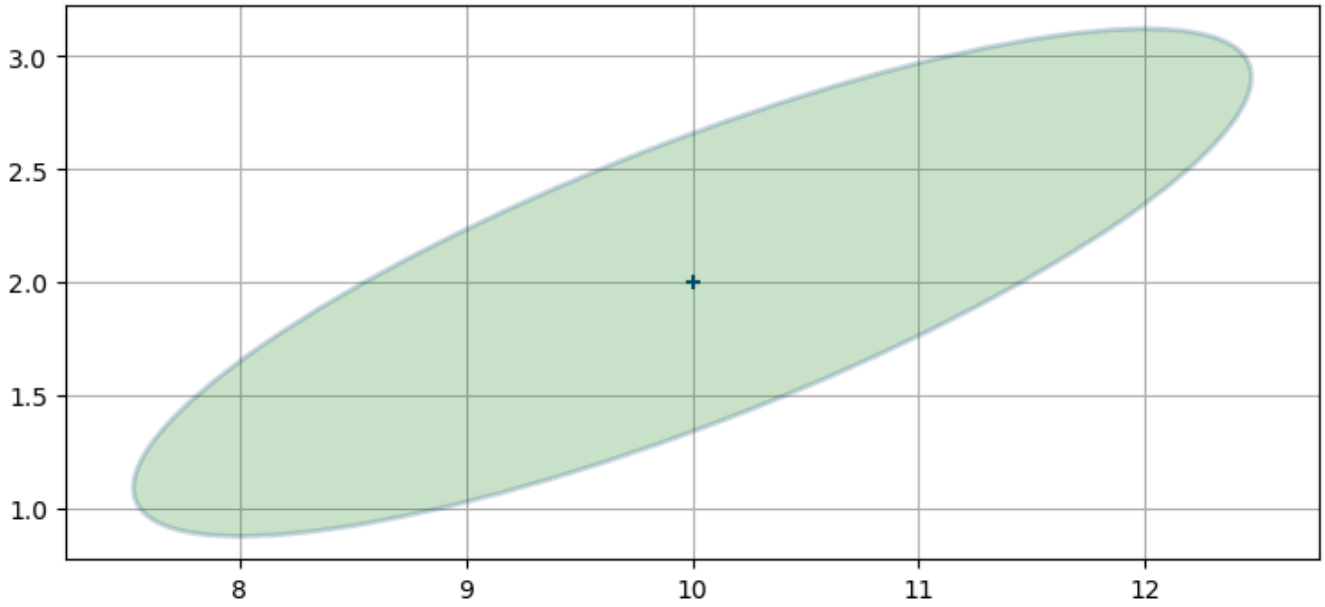


尽管在 $x = 1$ 处，曲线相当平滑且近乎直线，但输出值的概率分布却完全不符合高斯分布的形态。此外，计算出的输出均值也与直接测量得到的值有很大差异。这种现象并不罕见——抛射物运动轨迹呈抛物线形，而这种非线性特性正是滤波器需要处理的。如果你还记得的话，我们曾经试图追踪一个球的运动轨迹，但结果惨不忍睹。通过这张图表，你应该能明白为什么滤波器的性能会如此糟糕了。

一个 2D 示例

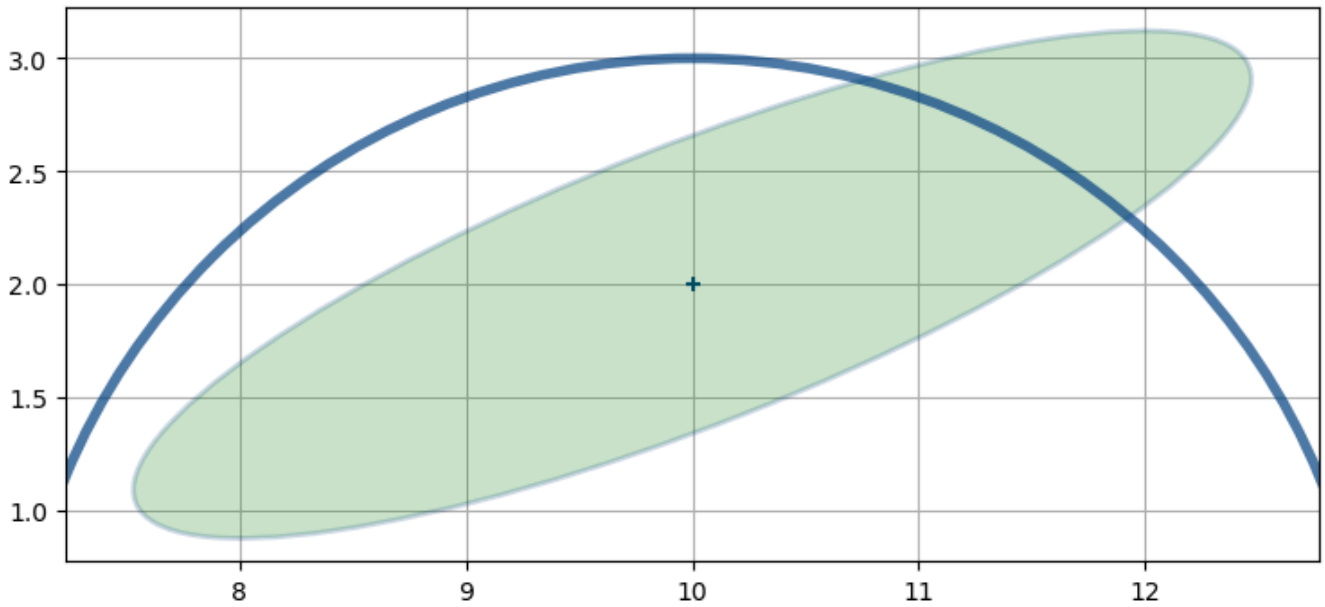
很难仅通过概率分布来推断滤波器中将会发生什么。那么，让我们以用雷达追踪飞机为例来思考吧。其估计值的相关性可能如下所示:

```
import kf_book.nonlinear_internal as nonlinear_internal

nonlinear_internal.plot1()
```



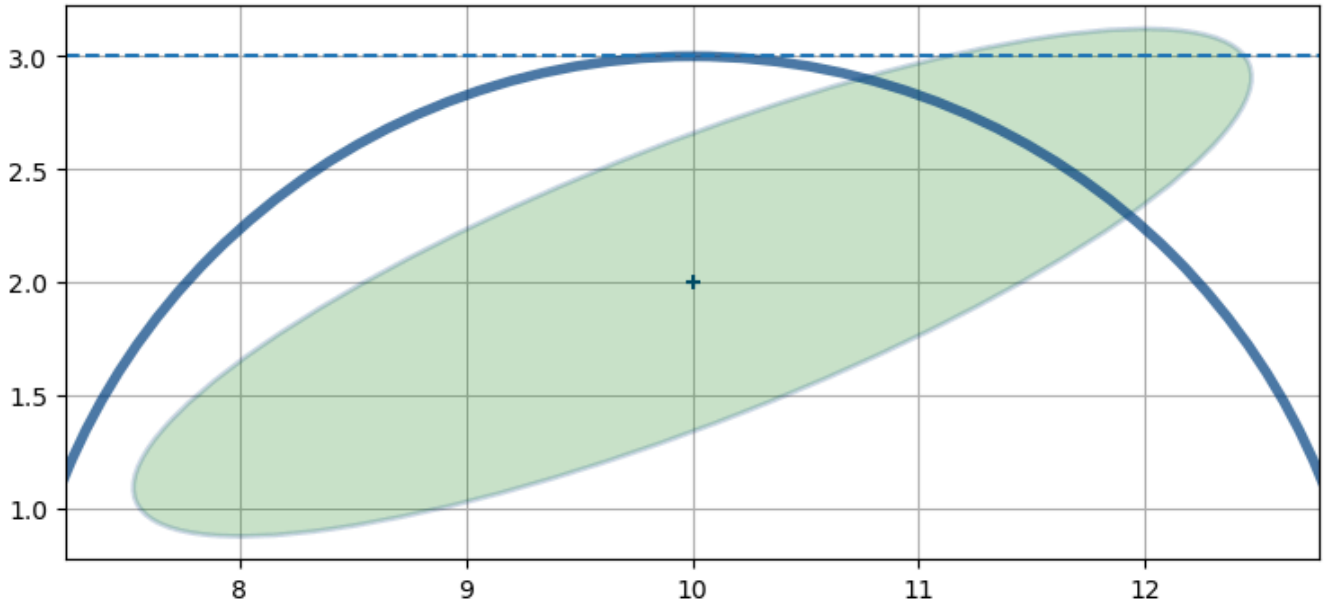
如果我们试图将这个问题线性化，会怎么样呢？雷达可以告诉我们飞机与雷达之间的距离。假设雷达正好位于飞机的正下方（x=10），而下一次测量结果显示飞机距离雷达 3 英里远（y=3）。与这一测量结果相符的所有点，会形成一个半径为 3 英里的圆。

```
nonlinear_internal.plot2()
```



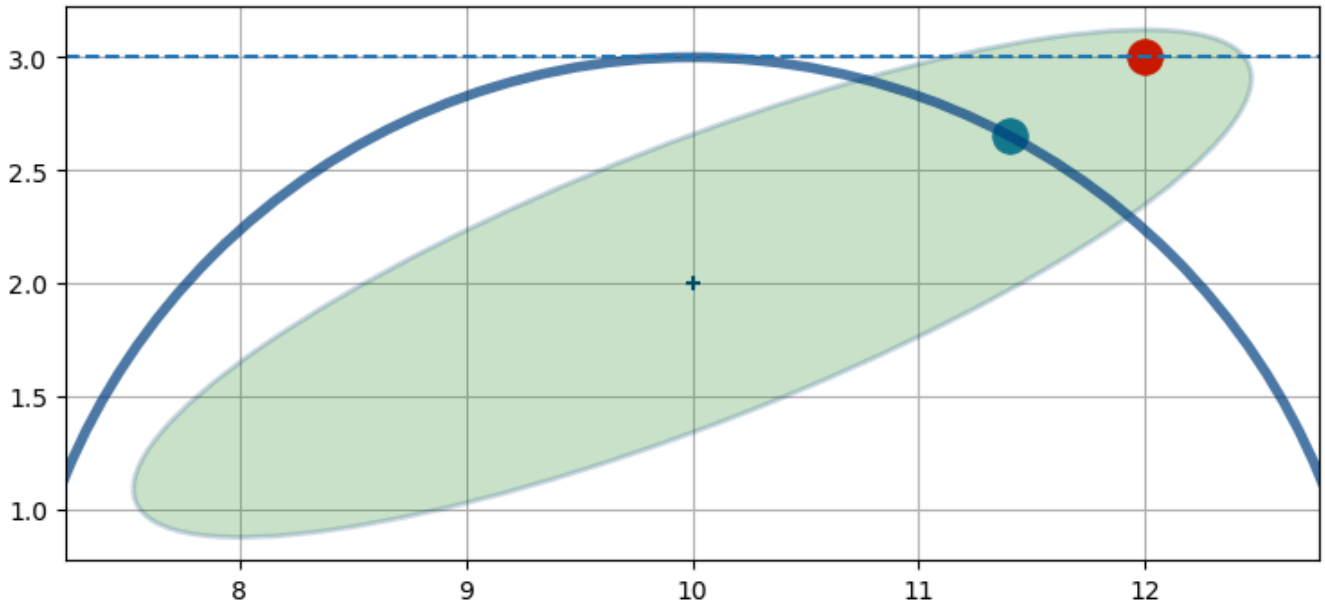
通过观察可以发现，飞机的可能位置应该在 x=11.4、y=2.7 附近，因为正是在该点上，协方差椭圆与测距结果相交。不过，测距结果属于非线性数据，因此我们需要对其进行线性化处理。我们还没有详细讲解过这一内容，但扩展卡尔曼滤波器会在飞机的最后位置——即(10,2)处进行线性化处理。在 x=10 时，测距结果对应的 y 值为 3，因此我们就在该点进行线性化处理。

```
nonlinear_internal.plot3()
```



现在，我们已经得到了该问题的线性表示形式（实际上就是一条直线），因此可以对其进行求解。不过，可以看到，这条直线与协方差椭圆的交点，距离飞机的实际位置还有相当大的距离。

```
nonlinear_internal.plot4()
```



这种错误往往会导致灾难性的后果。这次的估算误差相当大。但在滤波器的后续处理中，这个错误的估算值会被用来对下一个雷达测量结果进行线性化处理。这样一来，下一个估算结果很可能会比现在这个更糟糕。只要经过几次迭代，卡尔曼滤波器就会失去稳定性，产生的结果将完全不符合现实情况。

这个协方差椭圆的范围非常大。我故意夸大了其尺寸，以便说明高度非线性系统所带来的挑战。在实际的雷达跟踪系统中，非线性程度通常没那么严重，但误差仍然会不断累积。你可能会遇到的其他系统也可能存在类似的非线性问题——我这么做并非为了夸大其词。在处理非线性系统时，你始终要面对各种偏差问题。

这些算法/这些运算法则

你可能急于解决某个具体问题，不知道该使用哪种滤波方式。我会简要介绍一下各种选项。后面的章节彼此之间有一定的独立性，你可以随意跳读。不过，如果你真的想掌握所有内容，我建议按顺序阅读。

在非线性滤波器中，最常用的两种方法是线性化卡尔曼滤波器和扩展卡尔曼滤波器。这两种技术都是在卡尔曼发表其论文后不久被发明的，从那以后，它们就一直被广泛使用。飞机上的飞行控制系统、汽车或手机中的 GPS 系统，几乎都采用了这两种技术中的某一种。

不过，这些技术的要求极其苛刻。EKF 方法需要将微分方程在某一点处进行线性化处理，而这需要求解一个由偏导数构成的矩阵（即雅可比矩阵）。从理论上来说，这可能很难或根本无法实现。如果无法用解析方法求解，那就只能通过数值方法来计算雅可比矩阵。不过，这种做法计算成本很高，而且还会给系统带来更多误差。此外，当问题具有很强的非线性时，线性化处理会在每个计算步骤中引入大量误差，导致滤波器无法正常工作。你不能简单地把一些方程式输入到某个求解器中，就期望得到理想的结果。这对专业人士来说是个相当棘手的领域。值得注意的是，尽管 EKF 是实际应用中最常用的技术，但大多数卡尔曼滤波教材都只是简单提及了它，而没有深入讲解。

最近，该领域正在发生着令人振奋的变化。首先，计算能力的提升已经达到了这样的程度：我们现在可以使用那些曾经只有超级计算机才能处理的技术。这些技术属于蒙特卡洛法——计算机会生成成千上万个随机点，然后逐一将这些点与实际测量数据进行比对。根据各点与测量数据的匹配程度，计算机会决定是保留这些点还是丢弃它们。那些与测量数据相差较远的点很可能会被丢弃，而那些与测量数据非常接近的点则很可能被保留下来。经过几次迭代后，就会形成一组紧密围绕目标物体的点群，而在没有目标物体的地方则只留下稀疏的点分布。

这种方法有两个优点。首先，该算法即使面对极其非线性的问题也能保持良好的性能。其次，该算法能够同时追踪任意数量的对象——有些粒子会模拟某个对象的行为，而另一些粒子则负责模拟其他对象的行为。因此，这种技术常被用于追踪汽车流量、人群中的个体等等。

各项成本应该是明确的。在滤波器的每一步中，都需要对成千上万个点进行计算，这会带来很大的计算成本。不过，现代 CPU 的速度非常快，而对于 GPU 来说则是个不错的挑战，因为该算法的某些部分是可以并行处理的。另一个问题是，粒子滤波器无法提供像卡尔曼滤波器那样的精确数学结果。卡尔曼滤波器通过协方差矩阵来告诉我估计结果的误差大小，而粒子滤波器则无法提供这样的信息。最后，滤波器的输出是一组点，我需要自行判断如何解读这些点。通常，人们会计算这些点的平均值和标准差，但这其实是个复杂的问题。此外，还有许多点并不属于某个被追踪的物体，因此必须先使用聚类算法来识别出那些似乎在追踪某个物体的点。然后，还需要另一种算法来根据这些点来估算出系统的状态。这些步骤本身并不难实现，但计算成本相当高。

最后，我们还有一种名为“无迹卡尔曼滤波器”的新算法。该算法无需求解非线性方程的解析解，但其性能几乎总是优于 EKF。在处理非线性问题时，该算法表现尤为出色——而这些正是 EKF 难以处理的领域。该算法的设计也非常简单。虽然有些人认为 EKF 在某些方面仍更优，但在我看来，UKF 在几乎所有方面都优于 EKF。我建议，对于那些并非控制理论专业、没有相关学位的人来说，UKF 应该是首选的算法。不过，UKF 的缺点是运算速度可能稍慢于 EKF，但这取决于 EKF 是采用解析方法还是数值方法来计算雅可比矩阵。如果采用数值方法，那么 UKF 的速度肯定更快。目前还没有证据表明 UKF 总能给出更精确的结果（或许也永远无

法证明这一点）。比 EKF 更有效。在实际应用中，它的性能几乎总是优于 EKF，而且优势往往相当明显。这种滤波器的理解和实现都非常简单，我强烈建议将其作为起点来使用。

摘要/总结

世界本质上是非线性的，但我们只知道如何解决线性问题。这就给卡尔曼滤波器带来了很大的挑战。我们研究了非线性因素以三种不同但等效的方式影响滤波过程。下面我简要总结一下各种主要方法：线性化卡尔曼滤波器、扩展卡尔曼滤波器、无迹卡尔曼滤波器以及粒子滤波器。

直到最近，线性卡尔曼滤波器和 EKF 一直是解决这些问题的标准方法。不过，这些方法既难以理解和使用，而且可能还存在稳定性问题。

最近的发展成果带来了我认为更为出色的解决方案。UKF 无需求解偏微分方程，而且通常比 EKF 更精确。它的使用和理解都很简单。利用 FilterPy，我只需几分钟就能实现 UKF 的基本功能。粒子滤波则完全不需要数学建模，而是采用蒙特卡洛方法来生成数千个随机点。虽然其运行速度较慢，但能够相对轻松地解决那些原本难以处理的问题。

我收到的关于 EKF 的邮件比关于其他任何方法的邮件都要多。我觉得，这是因为在各种书籍、论文和网络资料中，EKF 都是最常用的方法。如果你想真正掌握这一领域的相关知识，那么当然有必要了解 EKF。不过，如果你只是想获得不错的处理结果，我建议你先学习 UKF 和粒子滤波器。这两种方法的实现、理解和使用都相对简单，而且通常比 EKF 更稳定。

有些人会反对这个建议。最近有很多研究论文都在比较 EKF、UKF 以及其他几种算法在解决特定问题时的优劣。对于您所面临的问题，难道不需要进行类似的比较吗？如果您要发射火箭前往火星，那当然必须这么做。您需要权衡精度、舍入误差、算法的稳定性、数学上的正确性以及所需的计算成本等因素。我无法想象有人会不了解 EKF 的原理。

另一方面，UKF 的表现实在出色！我在工作中实际应用了它。对于这些应用场景，我甚至没有尝试过使用 EKF，因为我已经确认 UKF 能够很好地发挥作用。当然，在某些情况下，或许还能从 EKF 中再提升 0.2%的性能吧？不过，我并不在乎这点。我完全理解 UKF 的实现原理，它很容易测试和验证。我可以把相关代码交给别人，而且有信心他们能够理解并修改代码。反正，既然已经有有效的解决方案了，我可不想再去处理那些复杂的方程式。如果 UKF 或粒子滤波器在处理某些问题时表现不佳，我才会考虑使用其他技术。不过，通常情况下，UKF 在各种问题和条件下的表现都要优于 EKF。如果“相当不错”就已经足够好了，那我就把时间花在解决其他问题上吧。

我之所以反复强调这一点，是因为在大多数教科书中，EKF 都是被重点介绍的内容，而 UKF 要么根本不被提及，要么只是被简单介绍两页而已，这显然无法让人真正掌握如何使用 UKF。UKF 毕竟还是相对较新的算法，因此需要时间来更新相关教材。在许多教材编写时，UKF 要么还未被发明，要么只是被认为是一种有潜力但尚未得到验证的算法。不过，现在 UKF 已经取得了巨大的成功，因此它绝对应该成为你工具箱中的重要工具。这也是我接下来要重点教授你的内容。

参考文献

[1] https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/Supporting_Notebooks/Computing_and_plotting_PDFs.ipynb

第 10 章：无迹卡尔曼滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

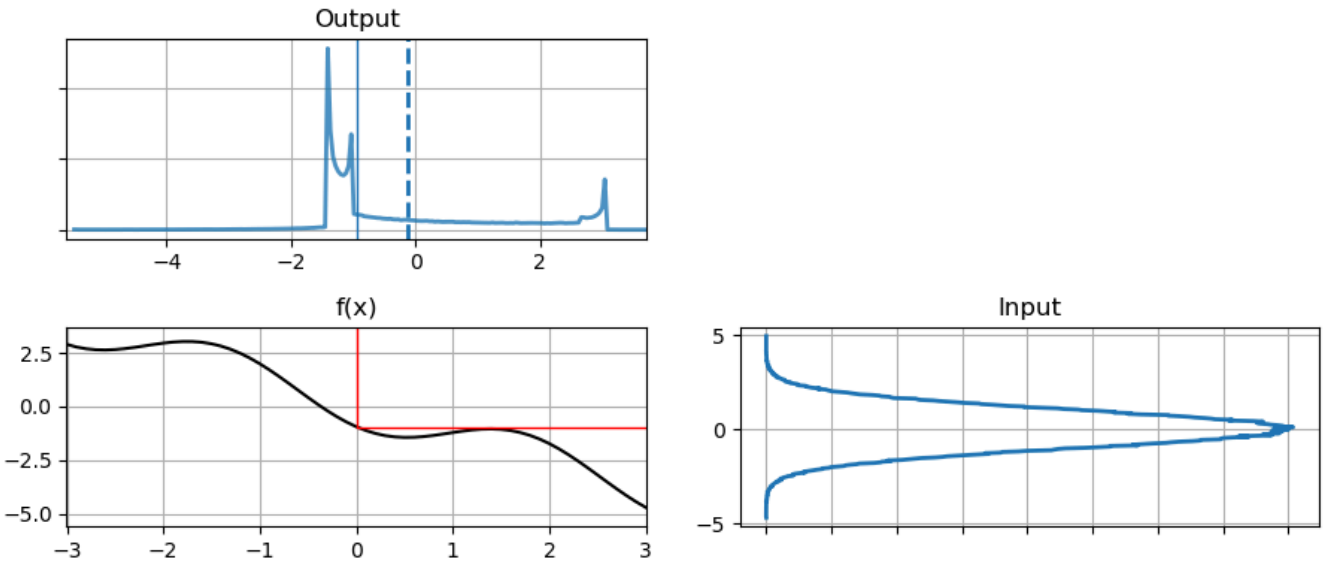
在上一章中，我们探讨了非线性系统所带来的各种难题。这种非线性现象可能出现在两个方面：其一，可能体现在我们的测量过程中。例如，雷达在测量到目标的斜距时，就需要进行开方运算才能得出目标的 x、y 坐标。

$$x = \sqrt{\text{slant}^2 - \text{altitude}^2}$$

这种非线性现象也可能出现在过程模型中——比如，当我们追踪一个在空气中飞行的球体时，空气阻力的影响会导致非线性行为。对于这类问题，标准的卡尔曼滤波器要么无法有效处理，要么效果很差。

在上一章中，我向你们展示过类似的图表。我对公式做了一些调整，以便更清楚地体现非线性效应。

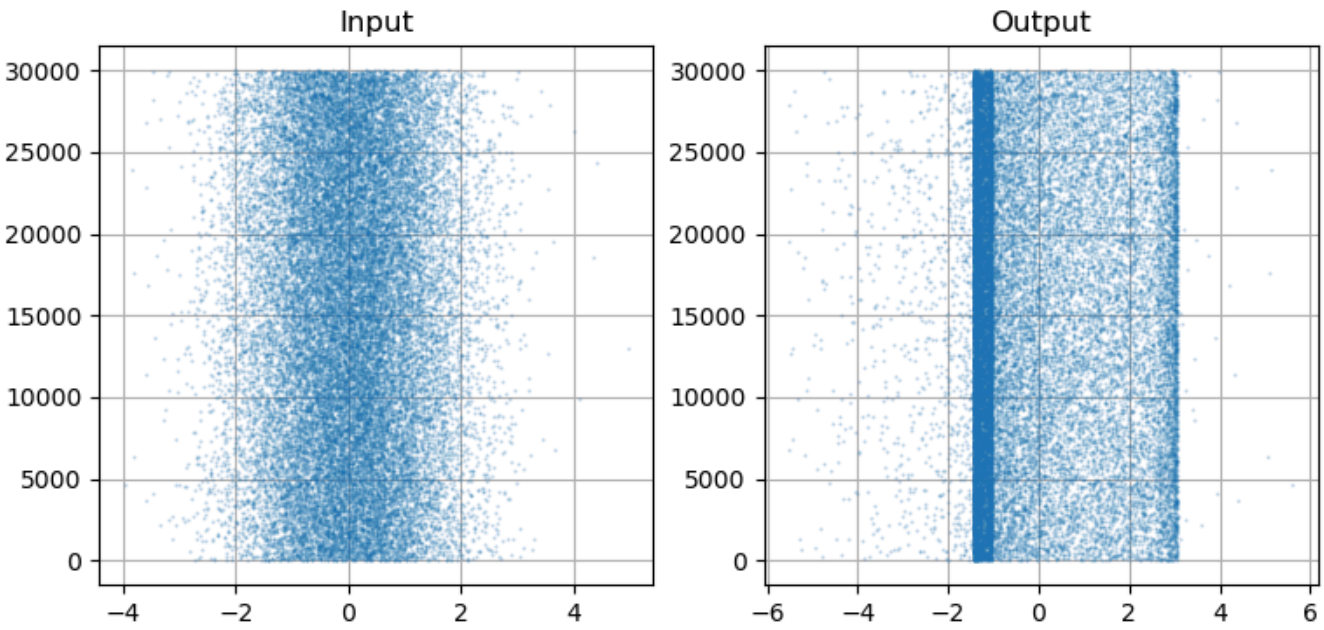
```
from kf_book.book_plots import set_figsize, figsize
import matplotlib.pyplot as plt
from kf_book.nonlinear_plots import plot_nonlinear_func
from numpy.random import normal
import numpy as np

# create 500,000 samples with mean 0, std 1
gaussian = (0., 1.)
data = normal(loc=gaussian[0], scale=gaussian[1], size=500000)

def f(x):
    return (np.cos(4*(x/2 + 0.7))) - 1.3*x

plot_nonlinear_func(data, f)
```



我是通过从输入数据中抽取 500,000 个样本，将这些样本经过非线性变换处理后，再绘制出相应的直方图来得到这个结果的。我们将这些点称为“西格玛点”。通过分析直方图，我们可以计算出平均值和标准差，从而得到一个近似的高斯分布曲线。

让我来向您展示一下数据在经过 `f(x)` 处理前后的散点图。

```
N = 30000
plt.subplot(121)
plt.scatter(data[:N], range(N), alpha=.2, s=1)
plt.title('Input')
plt.subplot(122)
plt.title('Output')
plt.scatter(f(data[:N]), range(N), alpha=.2, s=1);
```



这些数据本身似乎符合高斯分布，事实也确实如此。也就是说，这些数据看起来就像是在平均值零点周围分布的白噪声。而 `g(data)` 的数据则具有明显的结构：有两个明显的频带，频带之间还有大量的数据点。在频带的

外侧，也有零星的数据点，不过负数区域的数据点数量更多。

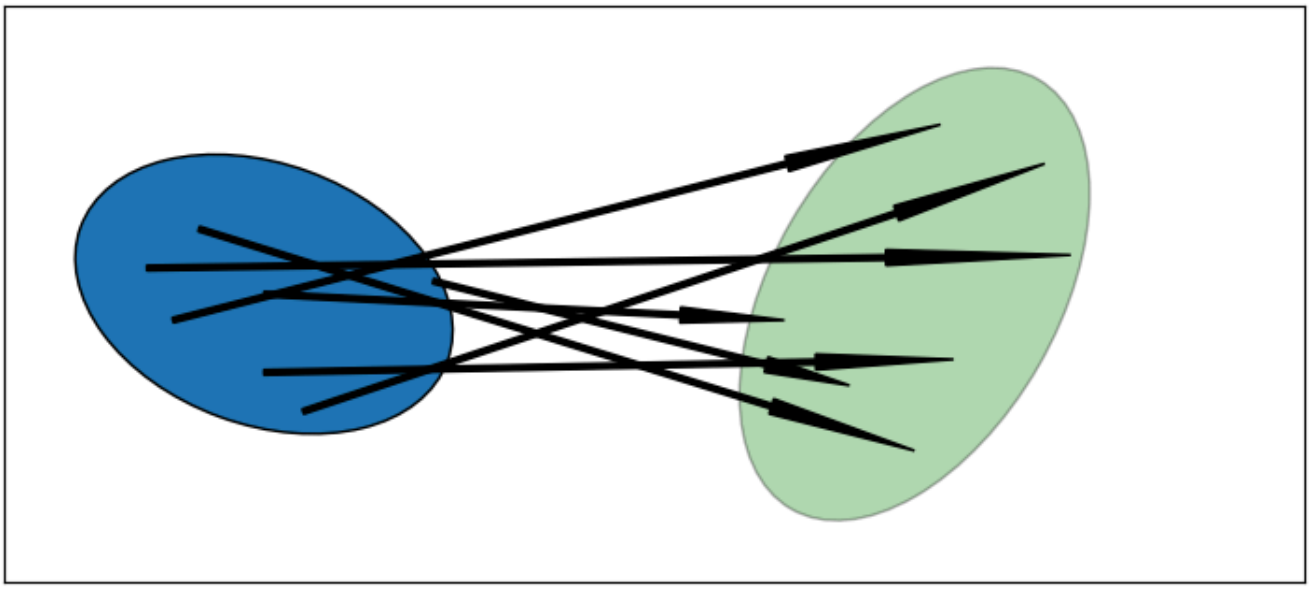
你或许已经意识到，这种采样方法其实可以解决我们的问题。假设在每次更新时，我们生成 500,000 个样本点，将这些样本点代入相应的函数中，然后再计算出结果的均值和方差。这种方法被称为蒙特卡洛方法。许多卡尔曼滤波器设计都采用了这种方法，比如集成滤波器和粒子滤波器。采样过程不需要任何特殊知识，也不需要复杂的数学公式来计算。无论该函数的复杂性如何，只要我们抽取足够多的样本点，就能得到准确的输出分布。

“点数不足”正是问题所在。上图是用 500,000 个西格玛点生成的，但结果仍然不够平滑。更糟糕的是，这还只是针对一维情况而言。所需点数会随着维数的增加而呈指数级增长：如果一维只需要 500 个点，那么二维就需要 500 的平方，即 250,000 个点；三维则需要 500 的立方，即 125,000,000 个点。因此，虽然这种方法确实有效，但计算成本极高。集成滤波器和粒子滤波器通过巧妙的方法大大降低了计算复杂度，但依然需要大量的计算资源。无迹卡尔曼滤波器也使用了西格玛点，但它通过确定性方法来选择这些点，从而大幅降低了计算量。

西格玛点——从概率分布中抽取样本

让我们从二维协方差椭圆的角度来分析这个问题。我选择二维形式，只是因为这样更便于绘图；当然，这个方法同样适用于任意维度的情况。假设存在某个非线性函数，我们可以从第一个协方差椭圆中随机选取点，将这些点代入非线性函数中，从而得到它们的新位置。接着，我们可以计算这些变换后点的均值和协方差，以此作为均值和概率分布的估计值。

```
import kf_book.ukf_internal as ukf_internal
ukf_internal.show_2d_transform()
```



左侧的椭圆代表了两个状态变量的 1σ 分布情况。箭头则展示了：通过某个任意的非线性函数，几个随机抽取的点如何被转换成新的分布。右侧的椭圆是半透明的，以此表示它只是对这些点的均值和方差的估算值而已。

让我们编写一个函数，该函数能随机选取 10,000 个来自高斯分布的点。

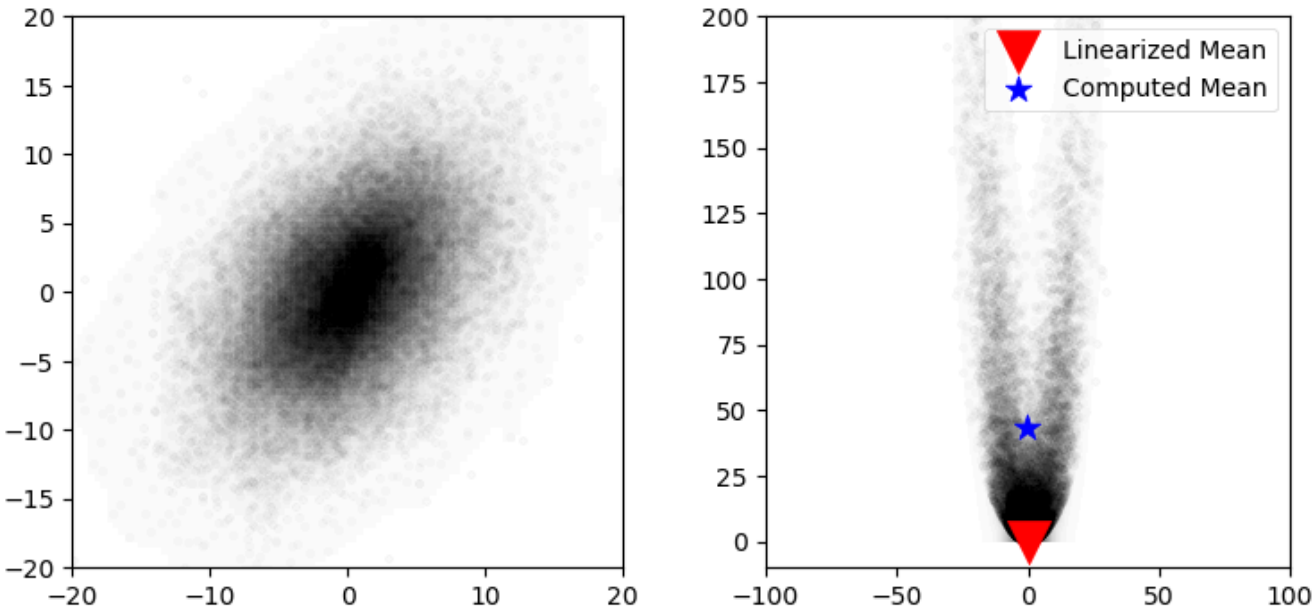
$$\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 32 & 15 \\ 15 & 40 \end{bmatrix}$$

通过非线性系统：

$$\begin{cases} \bar{x} & = x + y \\ \bar{y} & = 0.1x^2 + y^2 \end{cases}$$

```
import numpy as np
from numpy.random import multivariate_normal
from kf_book.nonlinear_plots import plot_monte_carlo_mean

def f_nonlinear_xy(x, y):
    return np.array([x + y, .1*x**2 + y*y])

mean = (0., 0.)
p = np.array([[32., 15.], [15., 40.]])
# Compute linearized mean
mean_fx = f_nonlinear_xy(*mean)

#generate random points
xs, ys = multivariate_normal(mean=mean, cov=p, size=10000).T
plot_monte_carlo_mean(xs, ys, f_nonlinear_xy, mean_fx, 'Linearized Mean');
```

```
Difference in mean x=-0.055, y=43.117
```



该图表清楚地显示了该函数所具有的强烈非线性特征。如果我们按照扩展卡尔曼滤波的方法来对其实行线性化处理，将会产生很大的误差（我们将在下一章中详细学习这一内容）。

一个简单的例子

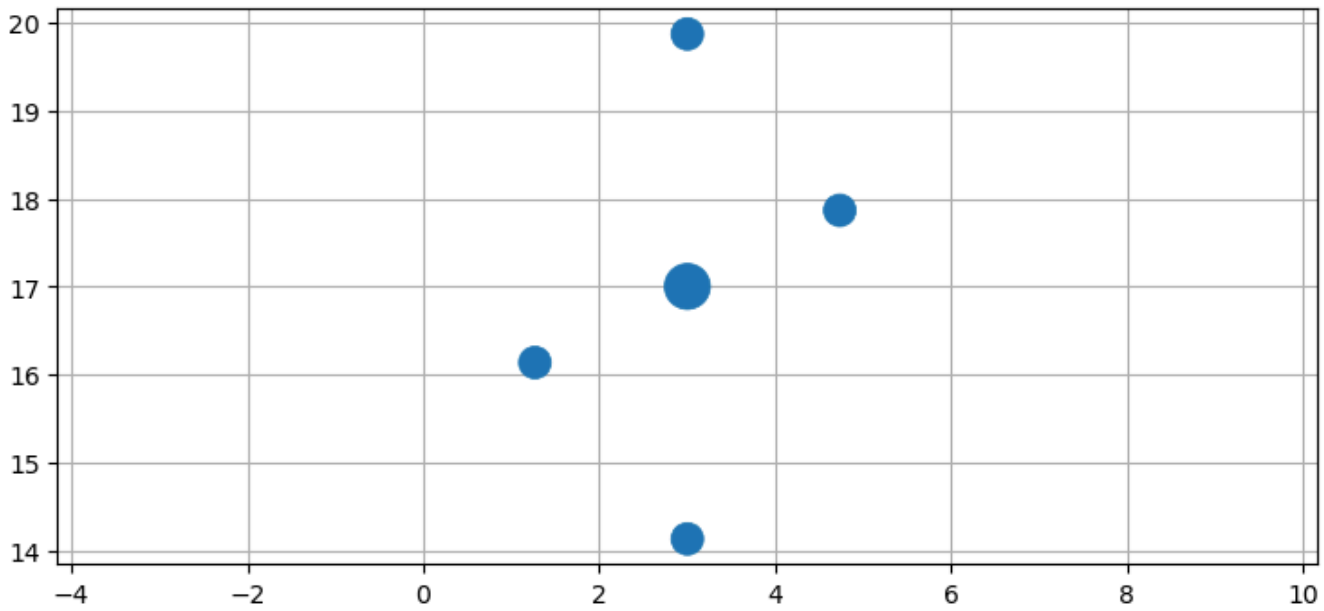
我马上就要开始讲解无迹卡尔曼滤波器在确定西格玛点以及进行后续计算时所使用的具体方法了。不过，我们先从一个例子开始吧，这样你就能明白整个过程的运作方式了。

我们将了解到，UKF 可以使用多种不同的算法来生成西格玛点。FilterPy 提供了几种这样的算法。以下是其中一种算法：

```
from filterpy.kalman import JulierSigmaPoints

sigmas = JulierSigmaPoints(n=2, kappa=1)
```

稍后就会更清楚了。不过，该对象会为任意给定的均值和协方差生成相应的加权西格玛点。我们来看一个例子吧：点的大小代表了该点的权重大小。

```
from kf_book.ukf_internal import plot_sigmas
plot_sigmas(sigmas, x=[3, 17], cov=[[1, .5], [.5, 3]])
```



可以看到，有 5 个点位于均值点(3, 17)的周围，呈一种特殊的排列方式。虽然看起来不可思议，但这些点的效果竟然可以与 500,000 个随机生成的点相媲美，甚至更佳！

好了，现在让我们来实现这个滤波器吧。我们将实现一维空间中的标准线性滤波器；目前我们还无法处理非线性滤波器。该滤波器的设计原理与我们之前所学的内容没什么不同，不过有一点例外。KalmanFilter 类使用矩阵 F 来计算状态转换函数。矩阵适用于线性问题，但不适用于非线性问题。因此，我们没有使用矩阵，而是像之前那样使用函数来替代。KalmanFilter 类还使用另一个矩阵 H 来实现测量函数，该函数负责将状态转换为相应的测量值。同样，矩阵意味着线性关系，所以这里也使用函数来替代矩阵。也许现在大家都明白为什么 H 被称为“测量函数”了吧：在线性 Kalman 滤波器中，它确实是一个矩阵，但实际上，那只是计算某个线性函数的快捷方式而已。

无需多言，以下是一维跟踪问题中的状态转移函数和测量函数。其中，状态为 $x = [x\dot{x}]^T$：

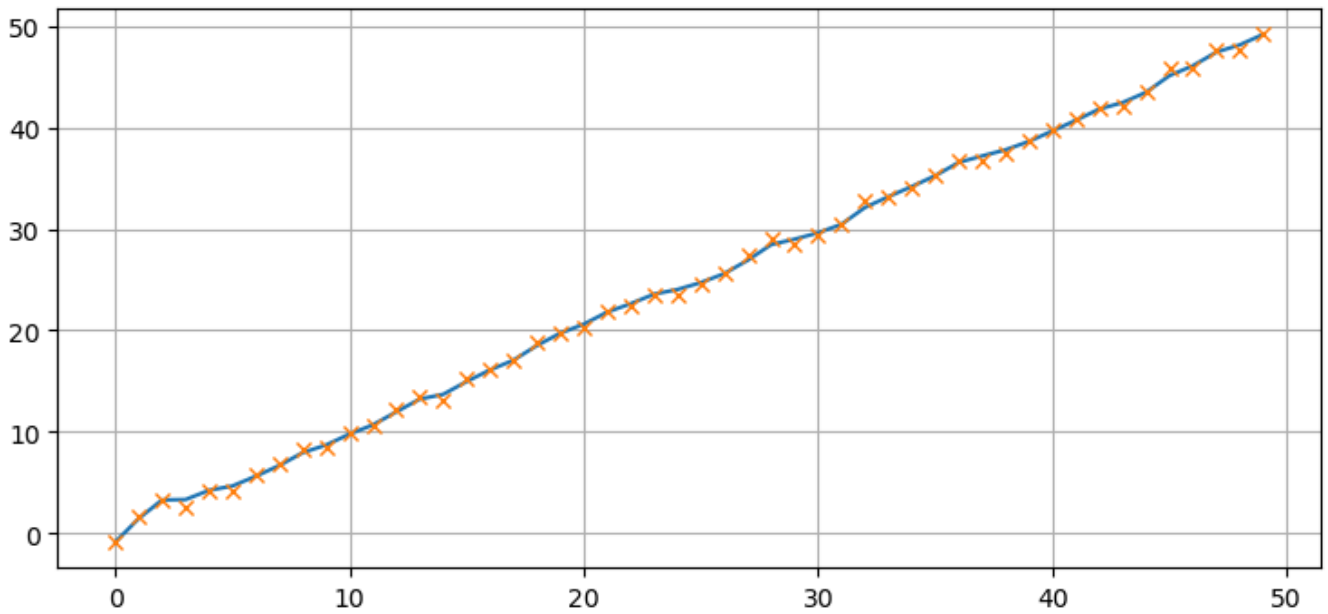
```
def fx(x, dt):
    xout = np.empty_like(x)
    xout[0] = x[1] * dt + x[0]
    xout[1] = x[1]
    return xout
```

```
def hx(x):
    return x[:1] # return position [x]
```

需要明确的是，这只是一个线性示例而已。对于线性问题来说，其实没必要使用 UKF 算法。不过，我还是从最简单的例子开始讲解。请注意，我使用 `fx()` 来计算 $\bar{x}$，是通过一组方程来实现的，而不是通过矩阵乘法。这样做是为了说明：在这里，我可以任意定义非线性函数；我们并不一定非得使用线性方程来表示数学关系。

其余的设计部分都是一样的。就是 P、 R 和 Q 这些设计。你们知道该如何操作，所以我们就把滤波器调试好并运行起来吧。

```
from numpy.random import randn
from filterpy.kalman import UnscentedKalmanFilter
from filterpy.common import Q_discrete_white_noise

ukf = UnscentedKalmanFilter(dim_x=2, dim_z=1, dt=1., hx=hx, fx=fx, points=sigmas)
ukf.P *= 10
ukf.R *= .5
ukf.Q = Q_discrete_white_noise(2, dt=1., var=0.03)

zs, xs = [], []
for i in range(50):
    z = i + randn()*.5
    ukf.predict()
    ukf.update(z)
    xs.append(ukf.x[0])
    zs.append(z)

plt.plot(xs);
plt.plot(zs, marker='x', ls='');
```



其实并没有什么新鲜的内容。你需要创建一个能够为你生成“西格玛点”的对象，同时用函数来代替 F 和 H 处的矩阵表达式。除此之外，其余的步骤都和以前一样。只要你能耐心地理解这些数学公式和算法，就应该能明白 UKF 到底是如何运作的了。

选择西格玛点

在本章的开头，我使用了 500,000 个随机生成的数值来计算高斯函数经过非线性函数处理后的概率分布。虽然这样计算出的均值相当准确，但每次更新都需要处理 500,000 个数值，这会使得滤波器的运行速度变得极慢。那么，我们最少需要使用多少个样本点呢？在这个问题中，又有哪些约束条件呢？我们假设自己对非线性函数一无所知，因为我们希望找到一种适用于所有函数的通用算法。

让我们先考虑最简单的情况，看看是否能从中得到一些启示。最简单的系统就是恒等函数： $f(x) = x$ 。如果我们的算法不适用于恒等函数，那么该滤波器就无法实现收敛。换句话说，如果输入为 1（对于一维系统而言），那么输出也必须为 1。如果输出不是 1，比如是 1.1，那么在下一个时间步中，当我们将 1.1 输入到该滤波器时，得到的输出又会是一个不同的数值，比如 1.23。这样的滤波器是无法实现收敛的。

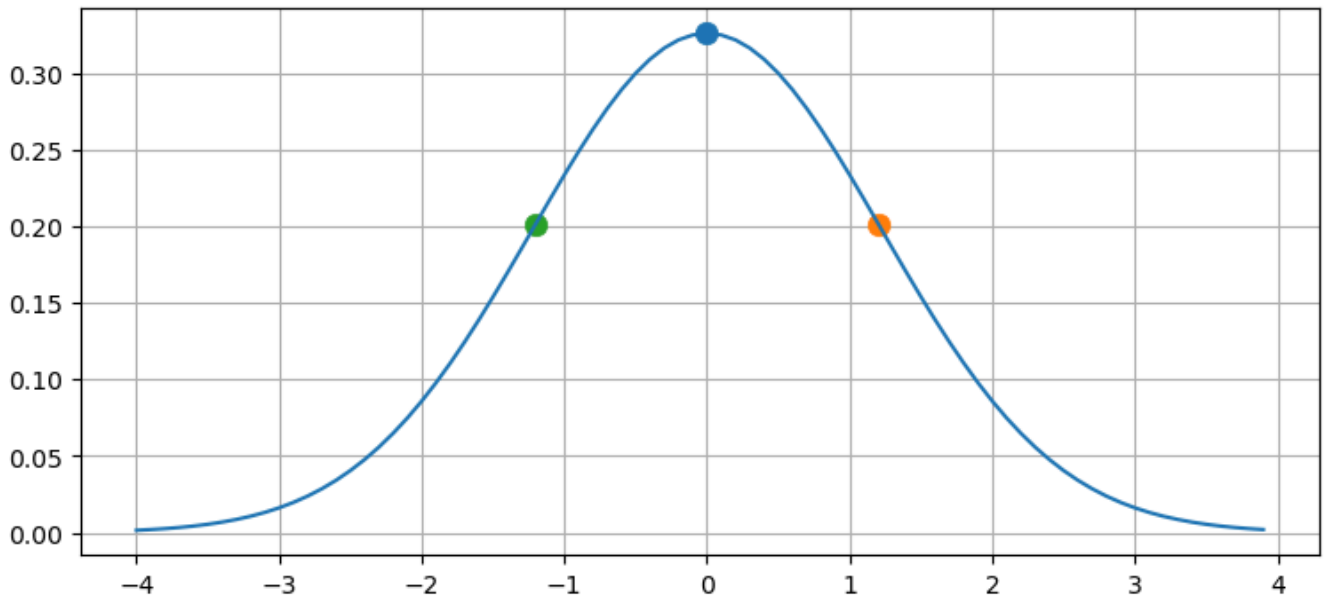
我们能够使用的最小点数是每个维度一个点。这也是线性卡尔曼滤波器所使用的数值。对于分布 $N(\mu, \sigma^2)$ 而言，卡尔曼滤波器的输入其实就是 μ 本身。不过，虽然这种做法在线性情况下有效，但对于非线性情况来说则并不适用。

也许我们可以为每个维度使用一个数值，但需要对这些数值进行某种处理。不过，如果我们把某个值 $\mu + \Delta$ 代入到恒等函数 $f(x) = x$ 中，该函数将无法收敛，因此这种方法行不通。如果我们不對 μ 进行任何处理，那么这就相当于标准的卡尔曼滤波器。综上所述，使用单个样本是无法实现该算法的。

那我们能选择的次低数字是多少呢？是 2。考虑到高斯分布具有对称性，而且为了使恒等函数能够正常工作，我们通常希望其中一个样本点恰好是输入数据的均值。如果选择 2 个点，那么我们需要先确定均值，然后再选择另一个点。但这个额外的点会引入输入数据中的不对称性，而这显然是我们不希望出现的。因此，要让恒等函数在处理 2 个点时能够正常工作，是非常困难的。

次低的数值是 3 分。3 分意味着我们可以选择平均值，然后再在平均值的两侧各选择 1 个点，如下图所示。

```
ukf_internal.show_3_sigma_points()
```



我们可以将这些点代入非线性函数 f(x)中，然后计算得到的平均值和方差。平均值可以是这三个点的算术平均值，但这种做法并不具有普遍性。例如，在处理高度非线性的问题时，我们可能希望给中心点的权重更大，而给边缘点的权重更小；或者反过来，给边缘点的权重更大。

一种更通用的方法是计算加权平均值 $\mu = \sum_i w_i f(X_i)$ ，其中，那些被标记为 X 的点就是所谓的“西格玛点”。我们需要确保所有权重的总和等于 1。在满足这一条件的情况下，我们的任务就是选择合适的 X 及其对应的权重，使得计算出的结果能够反映经过转换后的西格玛点的平均值和方差。

如果我们给均值赋予了权重，那么给协方差赋予权重也是合乎逻辑的。我们可以为均值（ w^m ）和协方差（ w^c ）使用不同的权重。在下面的公式中，我使用上标来表示索引。我们可以这样表示：

$$
\begin{aligned}
&Constraints: \\
&1 = \sum_i w_i^m \\
&1 = \sum_i w_i^c \\
&\mu = \sum_i w_i^m f(X_i) \\
&\Sigma = \sum_i w_i^c (f(X)_i - \mu)(f(X)_i - \mu)^T
\end{aligned}
$$

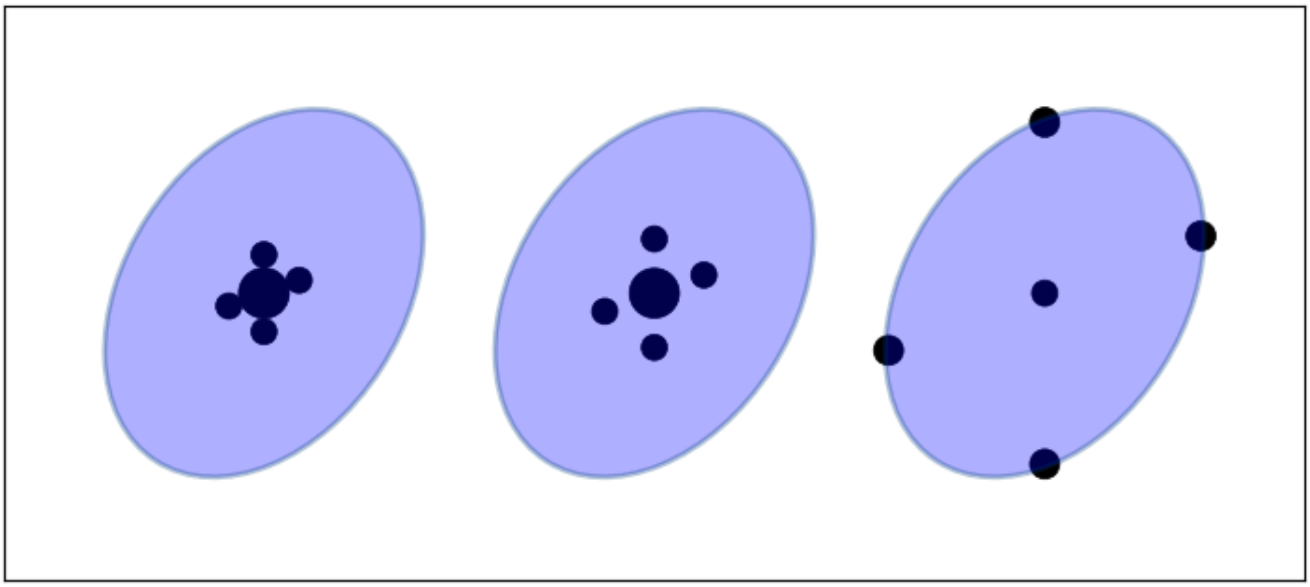
前两个方程体现了权重之和必须等于 1 这一约束条件。第三个方程则说明了如何计算权重的平均值。第四个方程可能不太常见，但请记住，两个随机变量之间的协方差公式就是如此。

$$
COV(x, y) = \frac{\sum (x - \bar{x})(y - \bar{y})}{n}
$$

这些约束条件并不唯一确定一个解。例如，如果你减小 w_0^m 的值，就可以通过增大 w_1^m 和 w_2^m 的值来加以弥补。你可以为均值和协方差赋予不同的权重，也可以使用相同的权重。实际上，这些方程并不要求其中的任何一个点必须等于输入数据的均值，不过，从某种意义上说，让某个点成为均值会更好一些。

我们希望找到一种能够满足这些约束条件的算法，理想情况下，每个维度上只需要 3 个点即可。在继续之前，我想确保大家都能明白这个概念。下面是同一个协方差椭圆在不同σ点下的三种表现形式。σ点的大小与赋予每个点的权重成正比。

```
ukf_internal.show_sigma_selections()
```



这些点并不位于椭圆的长轴和短轴上；各项约束条件也并不要求我让点们位于这些轴上。虽然我把这些点标示为等间距分布的，但约束条件其实并不要求如此。

西格玛点的排列和权重设置会影响到我们对概率分布的采样方式。那些彼此靠近的点能够捕捉到局部效应，因此更适合用于处理高度非线性的问题。而那些相距较远、或位于椭圆轴线之外的点，则能捕捉到非局部效应以及非高斯分布特性。不过，通过调整每个点的权重，我们可以缓解这一问题。如果这些点虽然远离平均值，但其权重很小，那么我们仍然可以获取到有关该分布的一些信息，同时避免因问题的非线性而导致估计结果出现偏差。

请理解，选择西格玛点的方法有无数种。我所选择的这些约束条件，只不过是其中一种方式而已。例如，并非所有用于生成西格玛点的算法都要求各权重之和为一。事实上，我在本书中推荐的算法就不具备这一特性。

无迹变换

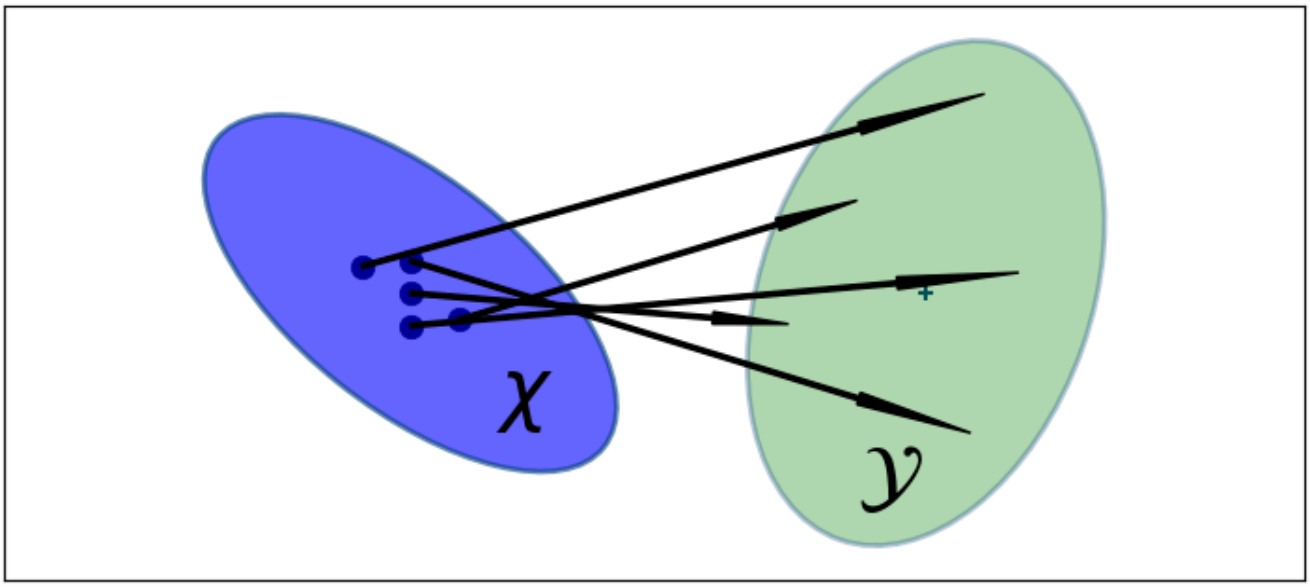
暂且假设，存在一种用于选择西格玛点与权重的算法。那么，这些西格玛点是如何被用来实现滤波功能的呢?

无迹变换是该算法的核心部分，但其实现方式却极为简单。该算法将σ点 χ 通过一个非线性函数进行处理，从而得到一组经过变换后的点。

$$Y = f(\chi)$$

接着，该算法会计算这些经过变换后的点的均值和协方差。这些均值和协方差便构成了新的估计值。下图展示了无迹变换的运作过程。右侧的绿色椭圆表示经过变换后的σ点的均值和协方差。

```
ukf_internal.show_sigma_transform(with_text=True)
```



西格玛点的均值和协方差计算如下：

$$\begin{aligned}\mu &= \sum_{i=0}^{2n} w_i^m Y_i \\ \Sigma &= \sum_{i=0}^{2n} w_i^c (Y_i - \mu)(Y_i - \mu)^T\end{aligned}$$

这些方程应该很熟悉——它们就是我们上面所推导出的约束方程。

简而言之，无迹变换就是将从任意概率分布中抽取的点，经过某个任意的非线性函数处理后，得到每个点对应的高斯分布。希望你能明白，我们可以利用这一原理来构建非线性卡尔曼滤波器。一旦得到了高斯分布，我们之前所开发的全部数学工具就可以派上用场了！

“无香味”这个名称可能会让人困惑。其实它并没有什么实际意义。这是该算法的发明者开的玩笑——他的意思是自己的算法“没有任何异味”。后来，这个名称就一直沿用下来了。这个词在数学上没有任何含义。

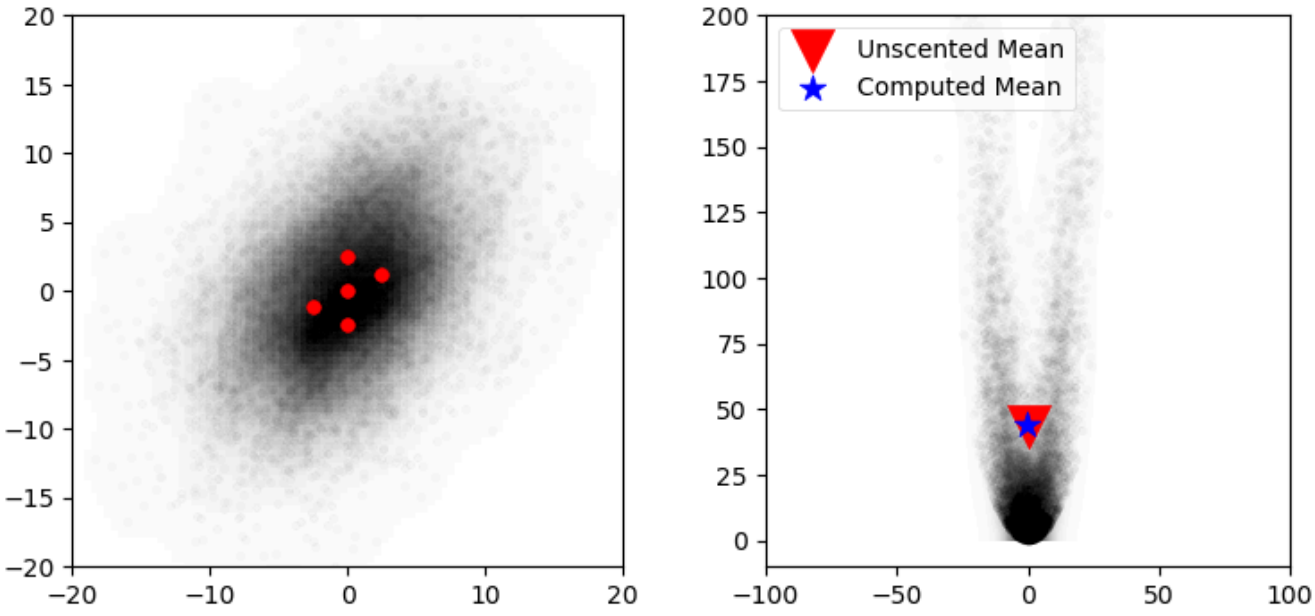
无中心变换的精度

之前我们编写了一个函数，该函数通过让 50,000 个数据点经过非线性函数来计算分布的平均值。现在，让我们让 5 个标准差的点也经过同一个函数处理，然后使用无中心变换来计算它们的平均值。我们将使用 FilterPy 函数的 MerweScaledSigmaPoints()函数来生成这些标准差点，而 `unscented_transform` 函数则用于执行变换操作；关于这些函数，我们将在后面详细讲解。在本章的第一个例子中，我使用了 `JulierSigmaPoints` 函数；它们都能生成标准差点，但实现方式有所不同，我会稍后解释。

```
from filterpy.kalman import unscented_transform, MerweScaledSigmaPoints
import scipy.stats as stats


#initial mean and covariance
mean = (0., 0.)
p = np.array([[32., 15], [15., 40.]])
```

```
# create sigma points and weights
points = MerweScaledSigmaPoints(n=2, alpha=.3, beta=2., kappa=.1)
sigmas = points.sigma_points(mean, p)

### pass through nonlinear function
sigmas_f = np.empty((5, 2))
for i in range(5):
    sigmas_f[i] = f_nonlinear_xy(sigmas[i, 0], sigmas[i ,1])

### use unscented transform to get new mean and covariance
ukf_mean, ukf_cov = unscented_transform(sigmas_f, points.Wm, points.Wc)

#generate random points
np.random.seed(100)
xs, ys = multivariate_normal(mean=mean, cov=p, size=5000).T

plot_monte_carlo_mean(xs, ys, f_nonlinear_xy, ukf_mean, 'Unscented Mean')
ax = plt.gcf().axes[0]
ax.scatter(sigmas[:,0], sigmas[:,1], c='r', s=30);
```

```
Difference in mean x=-0.097, y=0.549
```



我觉得这个结果相当了不起。仅使用 5 个点，我们就能够以极高的精度计算出平均值。x 方向的误差仅为-0.097，y 方向的误差为 0.549。相比之下，线性化方法（即 EKF 所使用的方法，我们将在下一章中详细介绍）在 y 方向的误差则超过了 43。如果你仔细查看生成这些西格玛点的代码，就会发现：该代码并不了解非线性函数的细节，它只知道初始分布的均值和协方差。即使我们面对的是完全不同的非线性函数，生成的 5 个西格玛点也是一样的。

我承认，我选择了一个非线性函数，这使得无迹变换的效果远远优于 EKF。不过，现实世界中到处都是非线性现象，而 UKF 能够很好地应对这些情况。我并没有刻意去寻找某个能让无迹变换效果良好的函数。在下一章中，你会看到，面对强烈的非线性现象时，传统技术会遇到很大困难。这张图表正好说明了为什么我建议尽可能使用 UKF 或类似的现代技术。

无迹卡尔曼滤波器

现在，我们可以来介绍 UKF 算法了。

预测步骤

UKF 的预测步骤是根据过程模型 $f()$ 来计算先验概率的。由于 $f()$ 被假设为非线性函数，因此我们需要根据某个特定函数来生成西格玛点 X 以及它们对应的权重 W^m, W^c 。

$$\begin{aligned} \chi &= \text{sigma-function}(x, P) \\ W^m, W^c &= \text{weight-function}(n, parameters) \end{aligned}$$

我们将每个西格玛点都通过 $f(x, \Delta t)$ 进行处理。这样，根据该过程模型，这些西格玛点会被“向前推进”到未来的某个时间点，从而形成新的先验分布。这套新的西格玛点被我们称为 Y。

$$Y = f(\chi, \Delta t)$$

我们利用无迹变换来处理经过变换后的西格玛点，从而计算出先验分布的均值和协方差。

$$\bar{x}, \bar{P} = UT(Y, w_m, w_c, Q)$$

这些就是无迹变换所对应的方程式：

$$\begin{aligned} \bar{x} &= \sum_{i=0}^{2n} w_i^m Y_i \\ \bar{P} &= \sum_{i=0}^{2n} w_i^c (Y_i - \bar{x})(Y_i - \bar{x})^T + Q \end{aligned}$$

该表格对比了线性卡尔曼滤波器与无迹卡尔曼滤波器的算法公式。为便于阅读，我省略了下标 i 。

Kalman	Unscented
	$Y = f(\chi)$
$\bar{x} = Fx$	$\bar{x} = \sum w^m Y$
$\bar{P} = FPF^T + Q$	$\bar{P} = \sum w^c (Y - \bar{x})(Y - \bar{x})^T + Q$

更新步骤

卡尔曼滤波器在测量空间中进行数据更新。因此，我们必须利用用户所定义的测量函数 $h(x)$ ，将先验状态中的西格玛点转换为可测量的数值。

$$Z = h(Y)$$

我们利用无迹变换来计算这些点的均值和协方差。下标 z 表示这些数值分别是测量所得的 sigma 点的均值和协方差。

$$
\begin{aligned}
\mu_z, P_z &= UT(Z, w_m, w_c, R) \\
\mu_z &= \sum_{i=0}^{2n} w_i^m Z_i \\
P_z &= \sum_{i=0}^{2n} w_i^c (Z_i - \mu_z)(Z_i - \mu_z)^T + R
\end{aligned}
$$

接下来，我们计算残差和卡尔曼增益。测量值 z 的残差很容易计算：

$$y = z - \mu_z$$

为了计算卡尔曼增益，我们首先需要计算状态量与测量值之间的交叉协方差。其定义如下：

$$P_{xz} = \sum_{i=0}^{2n} w_i^c (Y_i - \bar{x})(Z_i - \mu_z)^T$$

然后，卡尔曼增益的定义如下：

$$K = P_{xz} P_z^{-1}$$

如果你把“逆矩阵”视为某种矩阵的倒数，就会发现，卡尔曼增益其实是一个简单的比率，其计算方式如下：

$$K \approx \frac{P_{xz}}{P_z} \approx \frac{\text{belief in state}}{\text{belief in measurement}}$$

最后，我们利用残差和卡尔曼增益来计算新的状态估计值：

$$x = \bar{x} + Ky$$

新的协方差则按照以下方式计算：

$$P = \bar{P} - KP_zK^T$$

这一步中包含了一些需要直接接受的方程式，不过你应该能够理解这些方程式与线性卡尔曼滤波器方程式之间的联系。虽然线性代数的处理方式与线性卡尔曼滤波器略有不同，但所使用的算法其实还是我们在本书中一直使用的那种贝叶斯算法。

该表对比了线性 KF 算法和 UKF 算法的方程式。

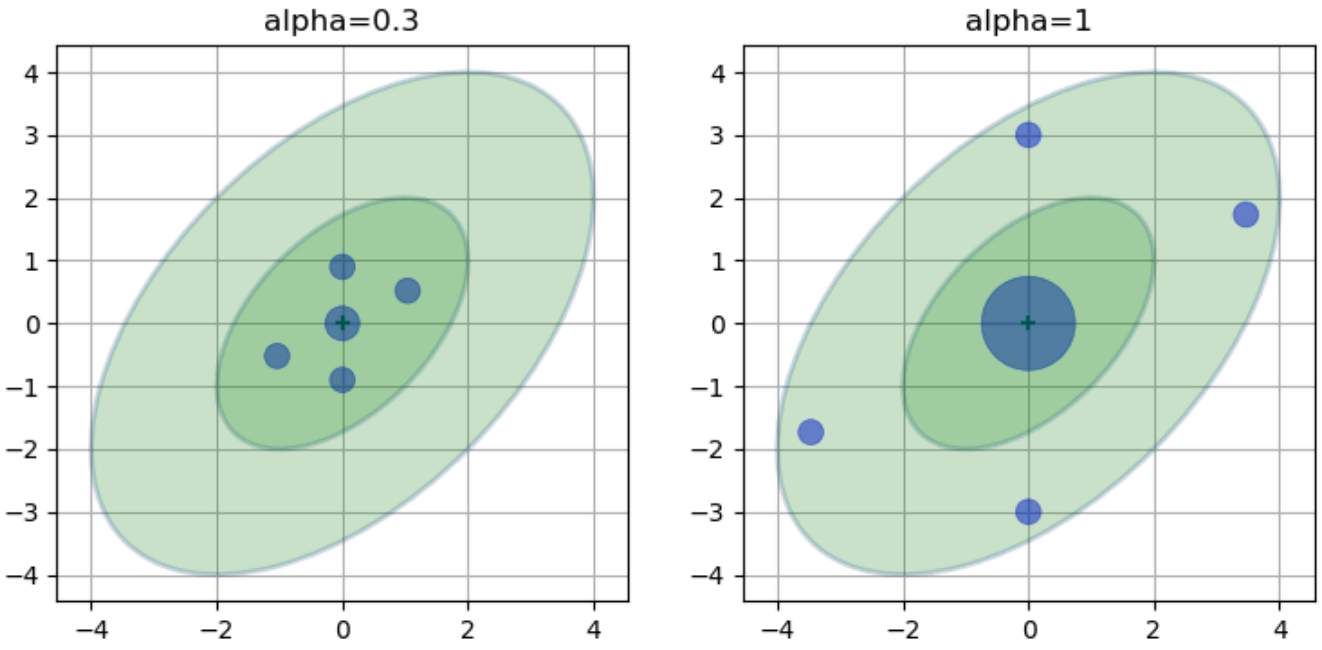
Kalman Filter	Unscented Kalman Filter
	$Y = f(\chi)$
$\bar{x} = Fx$	$\bar{x} = \sum w^m Y$
$\bar{P} = FPF^T + Q$	$\bar{P} = \sum w^c (Y - \bar{x})(Y - \bar{x})^T + Q$
	$Z = h(Y)$
	$\mu_z = \sum w^m Z$
$y = z - Hx$	$y = z - \mu_z$
$S = H\bar{P}H^T + R$	$P_z = \sum w^c (Z - \mu_z)(Z - \mu_z)^T + R$
$K = \bar{P}H^T S^{-1}$	$K = \left[\sum w^c (Y - \bar{x})(Z - \mu_z)^T \right] P_z^{-1}$
$x = \bar{x} + Ky$	$x = \bar{x} + Ky$
$P = (I - KH)\bar{P}$	$P = \bar{P} - KP_zK^T$

范德梅尔韦的标度西格玛点算法

用于选择西格玛点的算法有很多种。自 2005 年左右以来，无论是学术界还是工业界，都普遍采用 Rudolph Van der Merwe 在 2004 年的博士论文中提出的算法[1]。该算法在处理各种问题时都能表现出色，而且在性能与精度之间取得了良好的平衡。实际上，该算法是对 Simon J. Julier 提出的“缩放无迹变换”算法的略微改进而已[2]。

该公式使用 3 个参数来控制西格玛点的分布和权重： α 、 β 和 κ 。在开始计算之前，我们先来看一个例子。我会在表示第一和第二标准差的协方差椭圆上标出西格玛点，并根据各点的权重来调整它们的大小。

```
ukf_internal.plot_sigma_points()
```



我们可以看到，西格玛点位于第一和第二标准差之间。数值较大的 α 会使得这些点分布得更广。此外，数值较大的 α 会赋予均值（中心点）更高的权重，而数值较小的 α 则会使其他点的权重降低。这符合我们的直觉：一个点离均值越远，其权重就应该越低。我们还不清楚这些权重和西格玛点是如何确定的，但这些设定看起来是合理的。

西格玛点计算

第一个西格玛点就是输入数据的平均值。这就是上图中椭圆中心所对应的西格玛点。我们将其记为 χ_0 。

$$X_0 = \mu$$

为便于表示，我们定义 $\lambda = \alpha^2(n + \kappa) - n$ ，其中 n 表示 x 的维度。其余的西格玛点则通过相应公式计算得出。

i 下标表示选取矩阵中的第 i 个行向量。

换句话说，我们先用一个常数来调整协方差矩阵的值，然后取其平方根。为了确保矩阵的对称性，我们还会用该常数减去或加上矩阵的均值。关于如何对矩阵取平方根，我们稍后会详细讨论。

重量计算

该公式中，均值使用了一组权重，而协方差则使用了另一组权重。 X_0 的均值所对应的权重是这样计算得出的：

$$W_0^m = \frac{\lambda}{n+\lambda}$$

X_0 的协方差值为

$$W_0^c = \frac{\lambda}{n+\lambda} + 1 - \alpha^2 + \beta$$

其余的西格玛点的权重 $\chi_1 \ldots \chi_{2n}$，无论是对于均值还是协方差来说，都是相同的。这些权重值如下：

$$W_i^m = W_i^c = \frac{1}{2(n+\lambda)} i = 1..2n$$

虽然不太容易理解为什么这种做法是“正确的”，而且也无法证明它确实适用于所有的非线性问题。不过可以看出，我们所选择的西格玛点，其数值与协方差矩阵的平方根成正比，而方差的平方根就是标准差。因此，西格玛点的分布大致上遵循 $\pm 1\sigma$ 乘以某个比例因子的规律。分母中还有 n 这一项，所以当维度增加时，这些点的分布范围会更广，其权重则相应减小。

重要提示：通常情况下，这些权重之和并不等于 1。关于这一点，我收到了很多疑问。权重之和大于 1，或者出现负值的情况也是正常的。我将在下文中详细说明这一点。

参数的合理选择

$\beta = 2$ 是处理高斯问题的理想选择；当 n 为 x 的维度时， $\kappa = 3 - n$ 也是处理 κ 问题的好选择。而 $0 \leq \alpha \leq 1$ 则适合用于 α 的情况。此外，当 α 的数值较大时，各 sigma 点会更远离均值分布。

使用 UKF 算法

让我们来解几道题目，这样你就能明白使用 UKF 其实有多简单了。我们先从一个线性问题开始，你肯定已经知道如何用线性卡尔曼滤波器来解这个问题。虽然 UKF 最初是为了解决非线性问题而设计的，但对于线性问题来说，它得到的最优解与线性卡尔曼滤波器是一样的。我们将编写一个滤波器，利用恒定速度模型来追踪二维空间中的物体。这样，我们就能清楚地了解 UKF 与线性卡尔曼滤波器之间的相同之处（其实大部分都是相同的！），以及不同之处。

设计卡尔曼滤波器时，需要指定 x 、 F 、 H 、 R 和 Q 这些矩阵。我们已经做过很多次这样的设计了，所以这里就不再详细解释了。由于我们希望模型中的物体保持恒定速度运动，因此我们将 x 定义为……

$$x = \begin{bmatrix} x & \dot{x} & y & \dot{y} \end{bmatrix}^T$$

按照这种状态变量的排列方式，状态转移矩阵为：

$$F = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

这些系统都遵循牛顿方程来运作。

$$\begin{aligned} x_k &= x_{k-1} + \dot{x}_{k-1}\Delta t \\ y_k &= y_{k-1} + \dot{y}_{k-1}\Delta t \end{aligned}$$

我们的传感器能够提供位置信息，但无法提供速度信息。因此，该传感器的测量功能仅限于定位而已。

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

传感器的测量结果以米为单位，其在 x 轴和 y 轴上的误差均为 $\sigma = 0.3$ 米。由此我们可以得到一个测量噪声矩阵。

$$R = \begin{bmatrix} 0.3^2 & 0 \\ 0 & 0.3^2 \end{bmatrix}$$

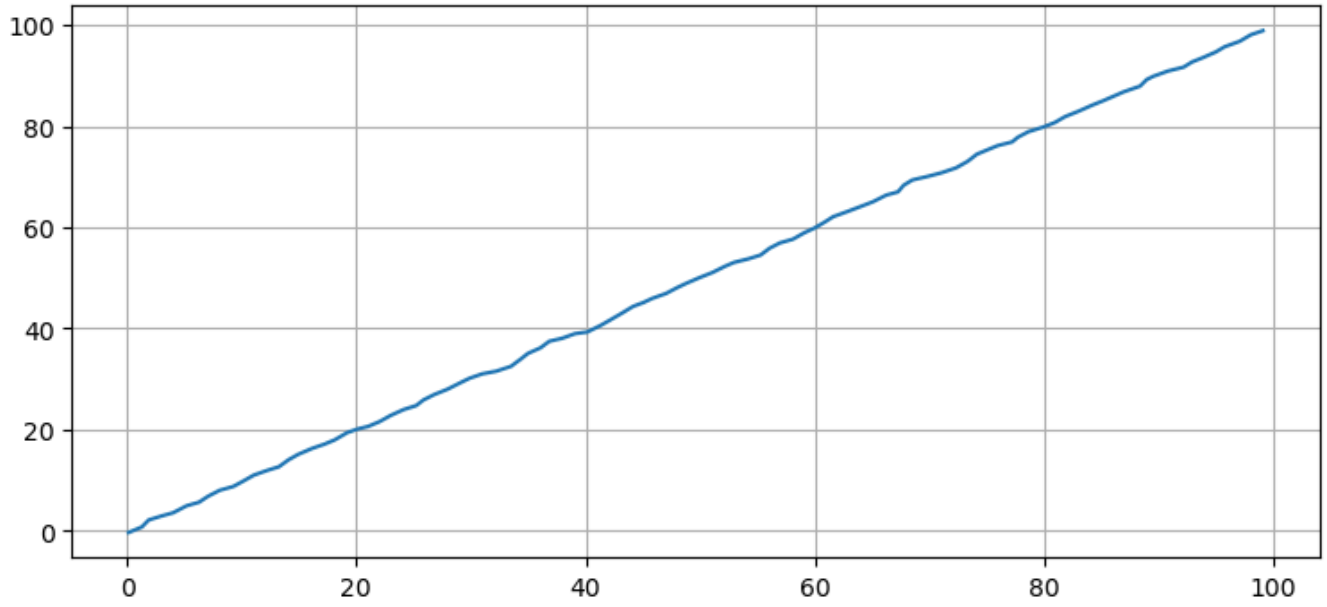
最后，假设过程噪声可以由离散白噪声模型来描述——也就是说，在每个时间点上，加速度都是恒定的。我们可以利用 `FilterPy` 的 `Q_discrete_white_noise()` 来生成这个矩阵。不过，为了便于理解，这里先不详细说明这个矩阵的具体内容。

$$Q = \begin{bmatrix} \frac{1}{4}\Delta t^4 & \frac{1}{2}\Delta t^3 \\ \frac{1}{2}\Delta t^3 & \Delta t^2 \end{bmatrix} \sigma^2$$

我对这个滤波器的实现方式如下：

```
from filterpy.kalman import KalmanFilter
from filterpy.common import Q_discrete_white_noise
from numpy.random import randn

std_x, std_y = .3, .3
dt = 1.0

np.random.seed(1234)
kf = KalmanFilter(4, 2)
kf.x = np.array([0., 0., 0., 0.])
kf.R = np.diag([std_x**2, std_y**2])
kf.F = np.array([[1, dt, 0, 0],
                 [0, 1, 0, 0],
                 [0, 0, 1, dt],
                 [0, 0, 0, 1]])
kf.H = np.array([[1, 0, 0, 0],
                 [0, 0, 1, 0]])

kf.Q[0:2, 0:2] = Q_discrete_white_noise(2, dt=1, var=0.02)
kf.Q[2:4, 2:4] = Q_discrete_white_noise(2, dt=1, var=0.02)

zs = [np.array([i + randn()*std_x,
                i + randn()*std_y]) for i in range(100)]
```

```
xs, _, _, _ = kf.batch_filter(zs)
plt.plot(xs[:, 0], xs[:, 2]);
```



这对你来说应该不算什么意外吧。现在，让我们来实现 UKF 算法。需要再次强调的是，这样做纯粹是出于教学目的而已；对于线性问题来说，使用 UKF 算法并没有任何好处。 `FilterPy` 通过 `UnscentedKalmanFilter` 这个类来实现 UKF 算法。

首先要实现 `f(x, dt)` 和 `h(x)` 这两个函数。 `f(x, dt)` 负责实现状态转换功能，而 `h(x)` 则负责实现测量功能。这些功能分别对应于线性滤波器中的 F 和 H 矩阵。

以下是这两个函数的合理实现方式。每个函数都应该返回一个包含结果的 1D NumPy 数组或列表。你可以给这些函数起一些更易理解的名称，而不是使用 `f` 和 `h` 这样的符号。

```
def f_cv(x, dt):
    """ state transition function for a
    constant velocity aircraft"""

    F = np.array([[1, dt, 0,  0],
                  [0,  1, 0,  0],
                  [0,  0, 1, dt],
                  [0,  0, 0,  1]])
    return F @ x

def h_cv(x):
    return x[[0, 2]]
```

接下来，你需要指定如何计算西格玛点与权重。上面我们介绍了 Van der Merwe 的方法，但实际上还有许多其他不同的实现方式。FilterPy 使用了一个名为 `SigmaPoints` 的类，该类必须实现相应的方法。

```
def sigma_points(self, x, P)
```

并且包含属性 `Wm` 和 `Wc` ，它们分别用于存储用于计算均值和协方差的权重值。

FilterPy 从 `SigmaPoints` 派生出 `MerweScaledSigmaPoints` 类，并实现了上述各种方法。

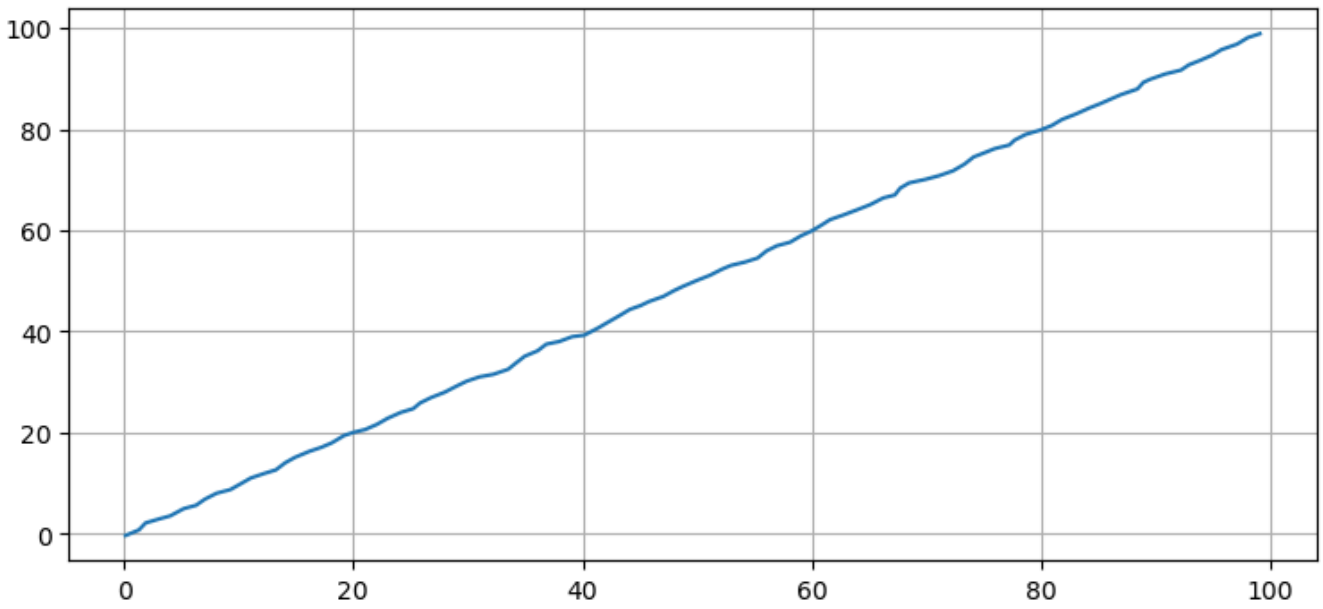
在创建 UKF 时，需要传入 $f()$ 和 $h()$ 函数以及 sigma 点对象，就像这个例子所示：

```
from filterpy.kalman import MerweScaledSigmaPoints
from filterpy.kalman import UnscentedKalmanFilter as UKF

points = MerweScaledSigmaPoints(n=4, alpha=.1, beta=2., kappa=-1)
ukf = UKF(dim_x=4, dim_z=2, fx=f_cv, hx=h_cv, dt=dt, points=points)
```

其余的代码与线性卡尔曼滤波器的代码相同。我会使用相同的测量数据，然后计算两种算法所得结果之间差异的标准差。

```
from filterpy.kalman import UnscentedKalmanFilter as UKF

import numpy as np

sigmas = MerweScaledSigmaPoints(4, alpha=.1, beta=2., kappa=1.)
ukf = UKF(dim_x=4, dim_z=2, fx=f_cv,
          hx=h_cv, dt=dt, points=sigmas)
ukf.x = np.array([0., 0., 0., 0.])
ukf.R = np.diag([0.09, 0.09])
ukf.Q[0:2, 0:2] = Q_discrete_white_noise(2, dt=1, var=0.02)
ukf.Q[2:4, 2:4] = Q_discrete_white_noise(2, dt=1, var=0.02)

uxs = []
for z in zs:
    ukf.predict()
    ukf.update(z)
    uxs.append(ukf.x.copy())
uxs = np.array(uxs)

plt.plot(uxs[:, 0], uxs[:, 2])
print(f'UKF standard deviation {np.std(uxs - xs):.3f} meters')
```

```
UKF standard deviation 0.013 meters
```



由此得出的标准差为 0.013 米，这个数值相当小。

UKF 的实现方式与线性卡尔曼滤波器并没有太大区别。只不过，线性卡尔曼滤波器中，状态转移函数和测量函数是以矩阵的形式来表示的，而 UKF 中则使用非线性函数来表示这些函数。除此之外，其余的理论和实现方式都是一样的。虽然用于实现 `predict()` 和 `update()` 的代码有所不同，但从设计者的角度来看，问题的描述方式以及滤波器的设计思路其实非常相似。

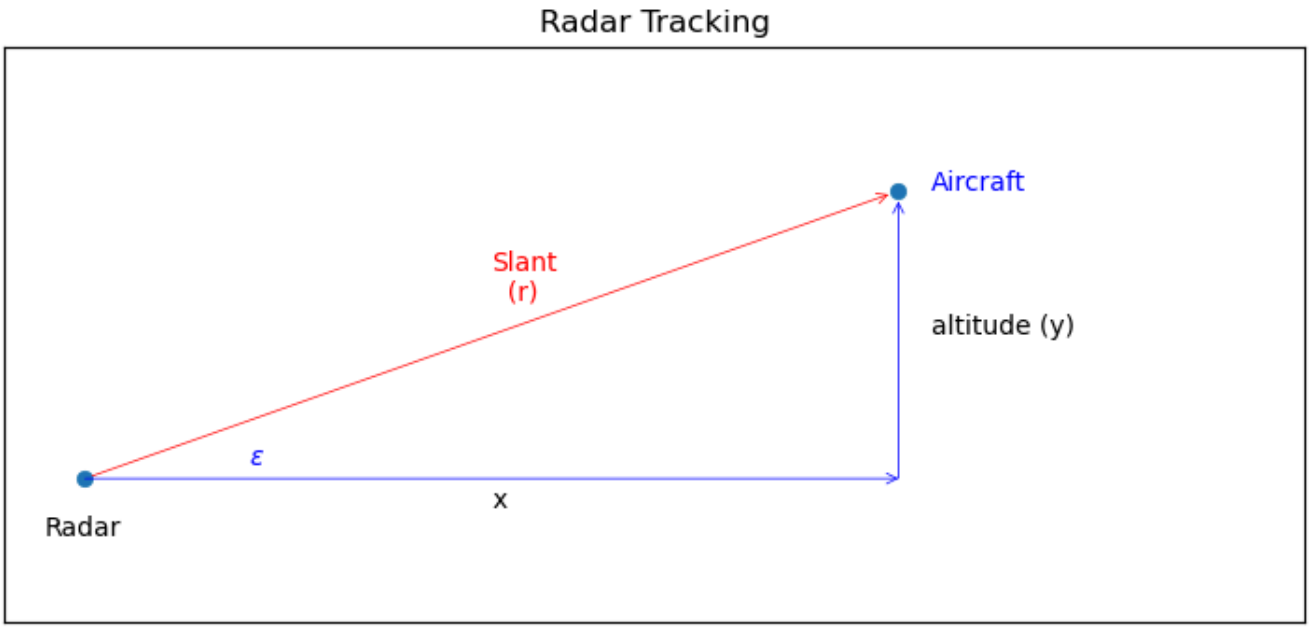
追踪飞机位置

让我们来处理第一个非线性问题吧。我们将编写一个滤波器，利用雷达作为传感器来追踪飞机的位置。为了尽可能保持与上一个问题的相似性，我们将在二维空间中进行追踪。具体来说，我们会追踪飞机在地面上的位置以及其飞行高度。由于每个维度都是独立的，因此我们可以这样处理，而不会失去一般性。

雷达的工作原理是发射无线电波或微波。只要位于波束路径上的任何物体，都会将部分信号反射回雷达。通过测量反射信号返回雷达所需的时间，就可以计算出该物体到雷达的斜距。斜距指的是从雷达到该物体的直线距离。而方位角则是通过天线的定向增益来确定的。

我们根据斜距和高程角来计算飞机的(x, y)坐标，具体方法如图所示：

```
import kf_book.ekf_internal as ekf_internal
ekf_internal.show_radar_chart()
```



仰角 ϵ 是指地面所形成的视线以上的角度。

我们假设飞机以恒定高度飞行。因此，我们需要一个包含三个变量的状态向量来描述飞机的状态。

$$x = \begin{bmatrix} distance \\ velocity \\ altitude \end{bmatrix} = \begin{bmatrix} x \\ \dot{x} \\ y \end{bmatrix}$$

状态转换函数是线性的。

$$\bar{x} = \begin{bmatrix} 1 & \Delta t & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ y \end{bmatrix}$$

其计算方式如下：

```
def f_radar(x, dt):
    """ state transition function for a constant velocity
    aircraft with state vector [x, velocity, altitude]'"""

    F = np.array([[1, dt, 0],
                  [0,  1, 0],
                  [0,  0, 1]], dtype=float)
    return F @ x
```

接下来，我们设计测量函数。与线性卡尔曼滤波器类似，测量函数的作用是将滤波器先前的估计值转化为可测量的数值。我们需要将飞机的位置和速度信息转换为雷达站能够识别的仰角和距离值。

距离是通过勾股定理计算得出的：

$$\text{range} = \sqrt{(x_{\text{ac}} - x_{\text{radar}})^2 + (y_{\text{ac}} - y_{radar})^2}$$

仰角 ϵ 实际上就是 y / x 的反正切值：

$$\epsilon = \tan^{-1} \frac{y_{ac} - y_{\text{radar}}}{x_{\text{ac}} - x_{\text{radar}}}$$

我们需要定义一个 Python 函数来计算这个值。我会利用函数可以拥有变量来存储雷达位置这一特性。虽然对于这个问题来说，这并非必需的（我们也可以直接将数值硬编码进去，或者使用全局变量），但这样做能让函数具有更高的灵活性。

```
def h_radar(x):
    dx = x[0] - h_radar.radar_pos[0]
    dy = x[2] - h_radar.radar_pos[1]
    slant_range = math.sqrt(dx**2 + dy**2)
    elevation_angle = math.atan2(dy, dx)
    return [slant_range, elevation_angle]

h_radar.radar_pos = (0, 0)
```

我们忽略了一个非线性因素，那就是角度具有“模块性”这一特性。残差指的是实际测量值与经过转换后的预测值之间的差异。359°和 1°之间的角度差应该是 2°，但实际上 359°-1°=358°。UKF 算法在处理数据时，会计算各种加权值的总和，这进一步加剧了这种非线性问题。目前，我们会把传感器和目标放置在这些非线性区域之外。稍后我会告诉你们如何解决这个问题。

我们需要对雷达和飞机进行模拟。对此你应该已经很熟悉了，所以我直接把代码提供给你，无需多言。

```
from numpy.linalg import norm
from math import atan2

class RadarStation:
```

```
    def __init__(self, pos, range_std, elev_angle_std):
        self.pos = np.asarray(pos)
        self.range_std = range_std
        self.elev_angle_std = elev_angle_std


    def reading_of(self, ac_pos):
        """ Returns (range, elevation angle) to aircraft.
        Elevation angle is in radians.
        """

        diff = np.subtract(ac_pos, self.pos)
        rng = norm(diff)
        brg = atan2(diff[1], diff[0])
        return rng, brg


    def noisy_reading(self, ac_pos):
        """ Compute range and elevation angle to aircraft with
        simulated noise"""

        rng, brg = self.reading_of(ac_pos)
        rng += randn() * self.range_std
        brg += randn() * self.elev_angle_std
        return rng, brg

class ACSim:
    def __init__(self, pos, vel, vel_std):
        self.pos = np.asarray(pos, dtype=float)
        self.vel = np.asarray(vel, dtype=float)
        self.vel_std = vel_std

    def update(self, dt):
        """ Compute and returns next position. Incorporates
        random variation in velocity. """

        dx = self.vel*dt + (randn() * self.vel_std) * dt
        self.pos += dx
        return self.pos
```

军用级雷达的测距精度可达到 1 米的均方根值，仰角测量的精度则可达到 1 毫弧度的均方根值[3]。不过，我们这里假设测距精度为 5 米，仰角精度为 0.5 度。这样的设定能给滤波器带来更大的挑战。

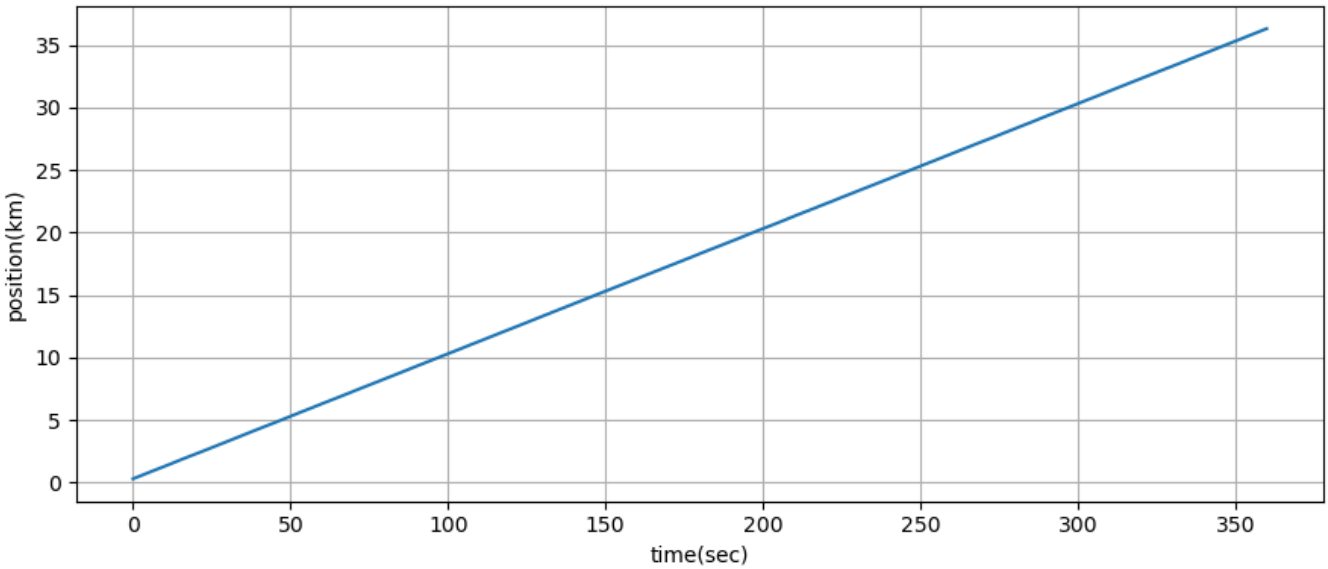
Q 的设计需要进一步讨论。当前的状态为 $\begin{bmatrix} x & \dot{x} & y \end{bmatrix}^T$。前两个参数分别是下降距离和速度，因此我们可以利用 `Q_discrete_white_noise` 噪声来计算 Q 矩阵左上角的数值。第三个参数是高度，我们假设它与 x 无关。这样一来， Q 的设计就可以用块设计来表示了：

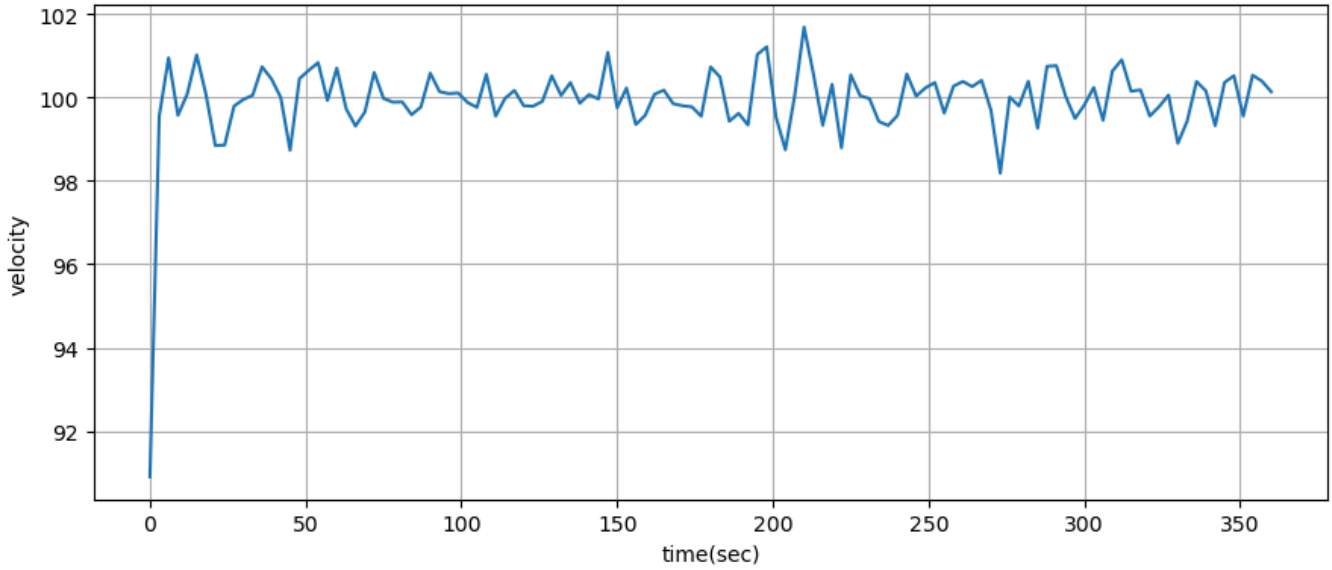
$$Q = \begin{bmatrix} Q_x & 0 \\ 0 & Q_y \end{bmatrix}$$

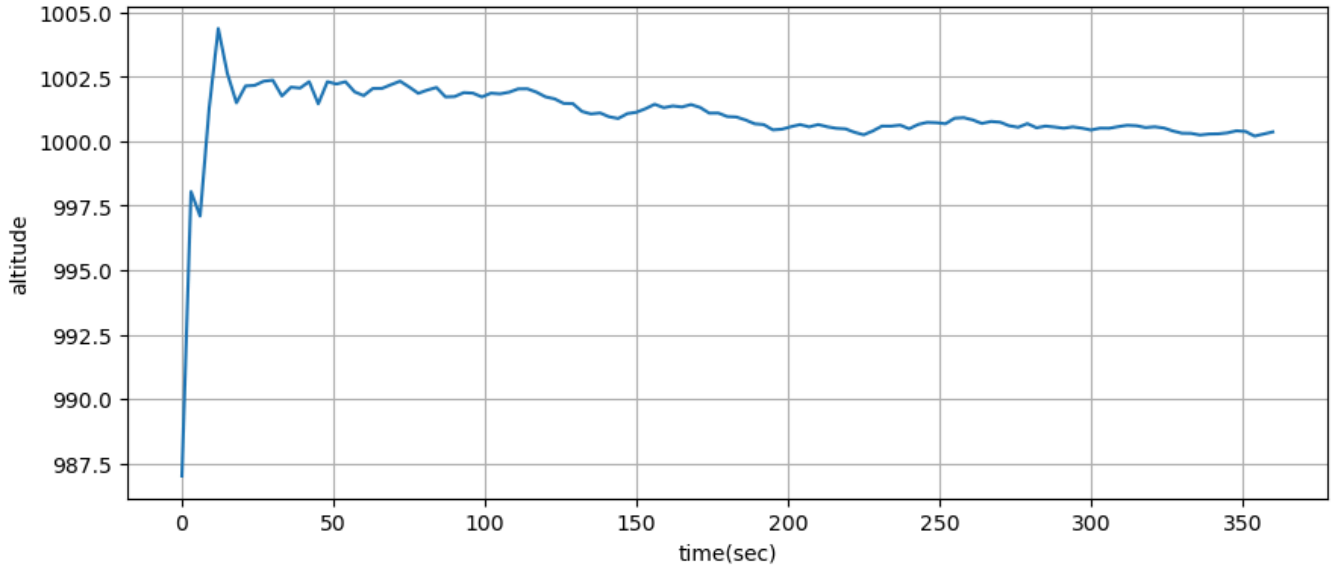
我先从那些位于雷达站正上方的飞机开始分析。这些飞机的飞行速度为 100 米/秒。典型的高度测量雷达可能每 3 秒才更新一次数据，因此我们就以 3 秒作为时间间隔来进行分析。

```
import math
from kf_book.ukf_internal import plot_radar

dt = 3. # 12 seconds between readings
```

```
range_std = 5 # meters
elevation_angle_std = math.radians(0.5)
ac_pos = (0., 1000.)
ac_vel = (100., 0.)
radar_pos = (0., 0.)
h_radar.radar_pos = radar_pos

points = MerweScaledSigmaPoints(n=3, alpha=.1, beta=2., kappa=0.)
kf = UKF(3, 2, dt, fx=f_radar, hx=h_radar, points=points)

kf.Q[0:2, 0:2] = Q_discrete_white_noise(2, dt=dt, var=0.1)
kf.Q[2,2] = 0.1

kf.R = np.diag([range_std**2, elevation_angle_std**2])
kf.x = np.array([0., 90., 1100.])
kf.P = np.diag([300**2, 30**2, 150**2])

np.random.seed(200)
pos = (0, 0)
radar = RadarStation(pos, range_std, elevation_angle_std)
ac = ACSim(ac_pos, (100, 0), 0.02)

time = np.arange(0, 360 + dt, dt)
xs = []
for _ in time:
    ac.update(dt)
    r = radar.noisy_reading(ac.pos)
    kf.predict()
    kf.update([r[0], r[1]])
    xs.append(kf.x)
plot_radar(xs, time)
```







这或许不会让你觉得有什么了不起，但我却觉得相当了不起！在“扩展卡尔曼滤波”这一章中，我们将解决同样的问题，不过需要用到大量的数学公式。

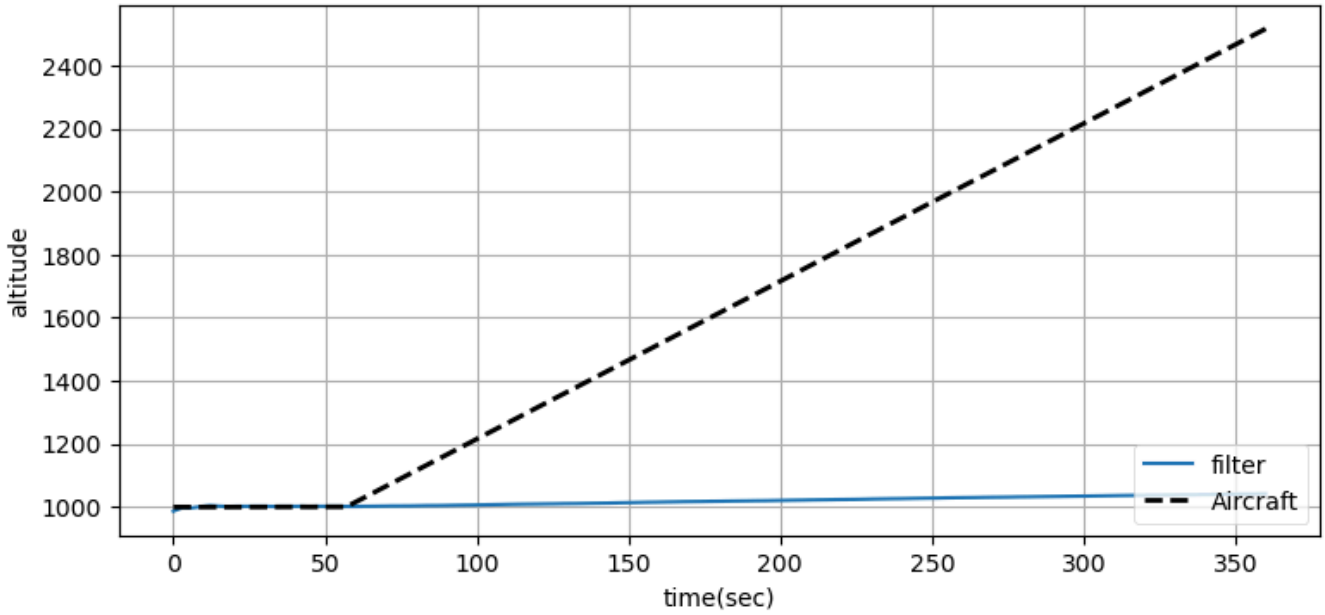
对机动飞机的跟踪监测

之前的示例取得了不错的效果，但该示例假设飞机的飞行高度保持不变。以下是当飞机在 1 分钟后开始上升时的滤波结果。

```
from kf_book.ukf_internal import plot_altitude

# reset aircraft position
kf.x = np.array([0., 90., 1100.])
kf.P = np.diag([300**2, 30**2, 150**2])
ac = ACSim(ac_pos, (100, 0), 0.02)

np.random.seed(200)
time = np.arange(0, 360 + dt, dt)
xs, ys = [], []
for t in time:
```

```
    if t >= 60:
        ac.vel[1] = 300/60 # 300 meters/minute climb
    ac.update(dt)
    r = radar.noisy_reading(ac.pos)
    ys.append(ac.pos[1])
    kf.predict()
    kf.update([r[0], r[1]])
    xs.append(kf.x)

plot_altitude(xs, time, ys)
print(f'Actual altitude: {ac.pos[1]:.1f}')
print(f'UKF altitude   : {xs[-1][2]:.1f}')
```

```
Actual altitude: 2515.6
UKF altitude   : 1042.1
```



该滤波器无法追踪高度的变化。我们在设计上需要做哪些调整呢?

我希望你的回答是“将爬升率添加到状态参数中”，也就是像这样:

$$x = \begin{bmatrix} distance \\ velocity \\ altitude \\ climbrate \end{bmatrix} = \begin{bmatrix} x \\ \dot{x} \\ y \\ \dot{y} \end{bmatrix}$$

为此，需要对状态转换函数进行如下修改，该函数仍然保持线性特性。

$$F = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ y \\ \dot{y} \end{bmatrix}$$

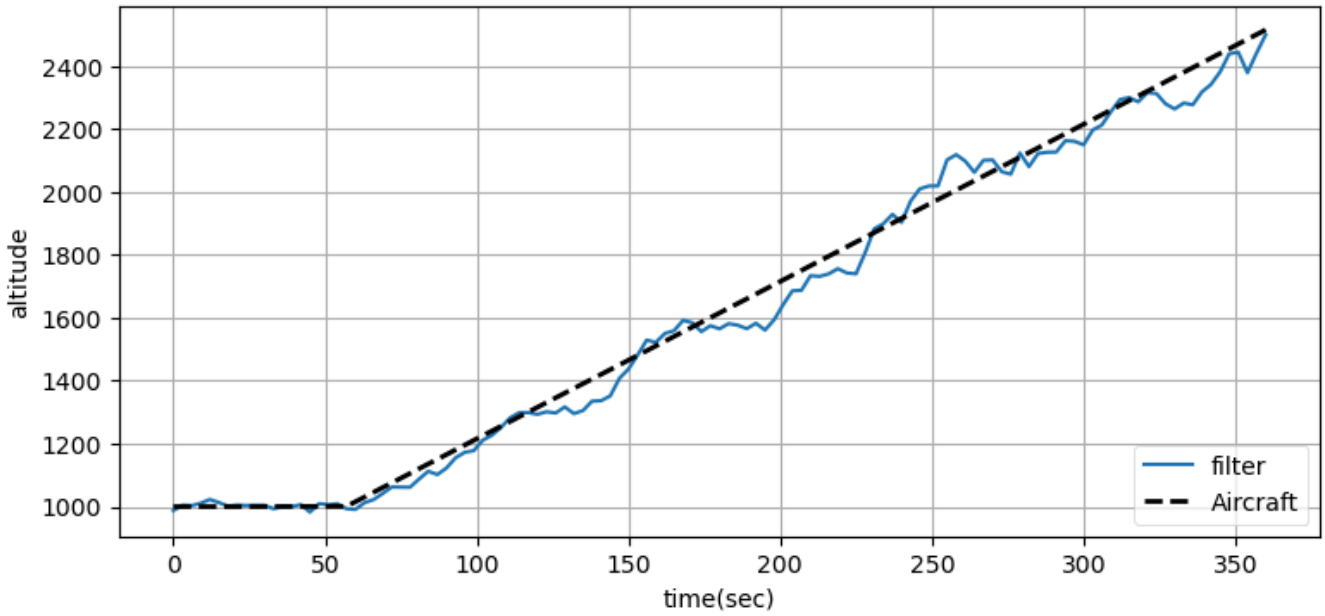
测量功能保持不变，但我们必须调整 Q 的值，以适应 x 的维度变化。

```
def f_cv_radar(x, dt):
    """ state transition function for a constant velocity
    aircraft"""
```

```
    F = np.array([[1, dt, 0, 0],
                  [0,  1, 0, 0],
                  [0,  0, 1, dt],
                  [0,  0, 0, 1]], dtype=float)
    return F @ x

def cv_UKF(fx, hx, R_std):
    points = MerweScaledSigmaPoints(n=4, alpha=.1, beta=2., kappa=-1.)
    kf = UKF(4, len(R_std), dt, fx=fx, hx=hx, points=points)

    kf.Q[0:2, 0:2] = Q_discrete_white_noise(2, dt=dt, var=0.1)
    kf.Q[2:4, 2:4] = Q_discrete_white_noise(2, dt=dt, var=0.1)
    kf.R = np.diag(R_std)
    kf.R = kf.R @ kf.R  # square to get variance
    kf.x = np.array([0., 90., 1100., 0.])
    kf.P = np.diag([300**2, 3**2, 150**2, 3**2])
    return kf
```

```
np.random.seed(200)
ac = ACSim(ac_pos, (100, 0), 0.02)

kf_cv = cv_UKF(f_cv_radar, h_radar, R_std=[range_std, elevation_angle_std])
time = np.arange(0, 360 + dt, dt)
xs, ys = [], []
for t in time:
    if t >= 60:
        ac.vel[1] = 300/60 # 300 meters/minute climb
    ac.update(dt)
    r = radar.noisy_reading(ac.pos)
    ys.append(ac.pos[1])
    kf_cv.predict()
    kf_cv.update([r[0], r[1]])
    xs.append(kf_cv.x)

plot_altitude(xs, time, ys)
print(f'Actual altitude: {ac.pos[1]:.1f}')
print(f'UKF altitude   : {xs[-1][2]:.1f}')
```

```
Actual altitude: 2515.6
UKF altitude   : 2500.1
```



在高度的测量过程中，确实存在相当大的误差，不过我们现在已经能够准确地测量高度了。

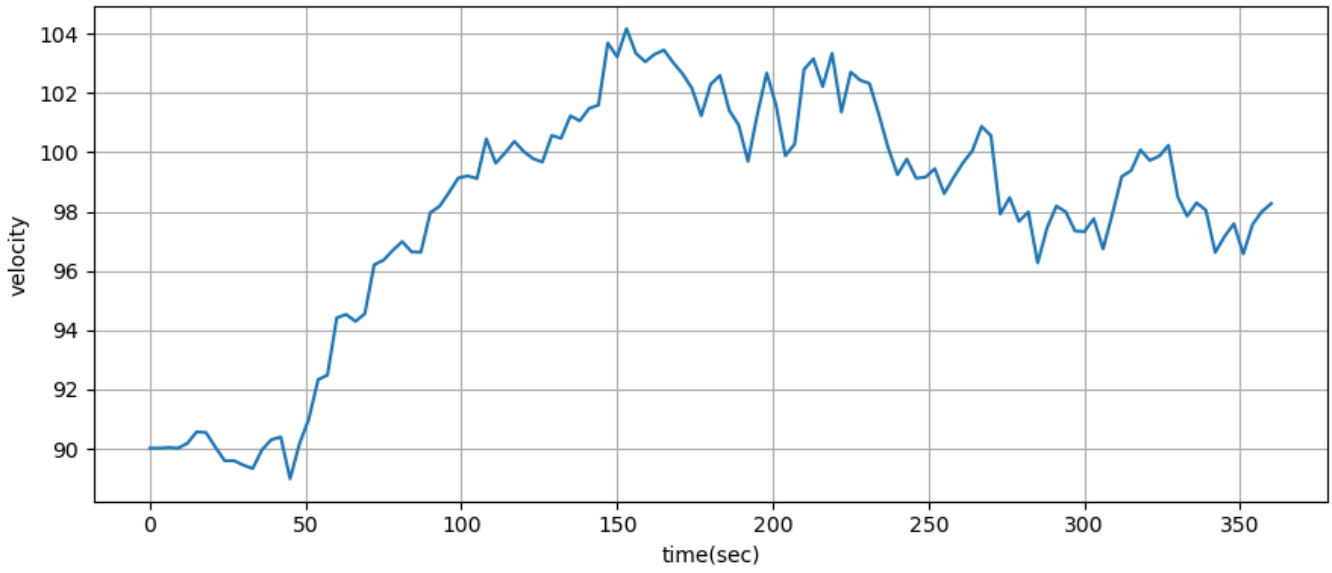
传感器融合

现在，让我们来看一个传感器融合的例子。我们有一种多普勒系统，它能够以 2 米/秒的均方根误差来估算物体的速度。我之所以说“某种多普勒系统”，是因为和雷达的情况类似，我并不打算在这里教你如何为多普勒系统设计出精确的滤波器。在实际应用中，必须考虑到信噪比、大气效应、系统的几何结构等因素。

在之前的例子中，该雷达的精度非常高，我们能够将速度测量的误差控制在 1 米/秒左右。现在，我会刻意降低其精度，以此来展示传感器融合的效果。我们将距离测量误差设为 $\sigma = 500$ 米，然后计算出速度测量的标准偏差。由于在测量初期，滤波器仍在调整中，因此会产生较大的误差。我会忽略前几组的测量结果。

不使用多普勒效应时的标准差为：

```
range_std = 500.
elevation_angle_std = math.degrees(0.5)
np.random.seed(200)
pos = (0, 0)
radar = RadarStation(pos, range_std, elevation_angle_std)
ac = ACSim(ac_pos, (100, 0), 0.02)

kf_sf = cv_UKF(f_cv_radar, h_radar, R_std=[range_std, elevation_angle_std])
time = np.arange(0, 360 + dt, dt)
xs = []
for _ in time:
    ac.update(dt)
    r = radar.noisy_reading(ac.pos)
    kf_sf.predict()
    kf_sf.update([r[0], r[1]])
    xs.append(kf_sf.x)

xs = np.asarray(xs)
plot_radar(xs, time, plot_x=False, plot_vel=True, plot_alt=False)
print(f'Velocity std {np.std(xs[10:, 1]):.1f} m/s')
```




```
Velocity std 3.4 m/s
```

在多普勒测量中，我们需要将 x 和 y 中的速度信息也纳入测量结果中。 `ACSim` 类将速度信息存储在数据成员 `vel` 中。为了执行卡尔曼滤波更新，我们只需调用 `update` ，并传入包含斜距、仰角以及 x 和 y 中的速度信息的列表即可。

$$z = [slant_range, \text{elevation angle}, \dot{x}, \dot{y}]$$

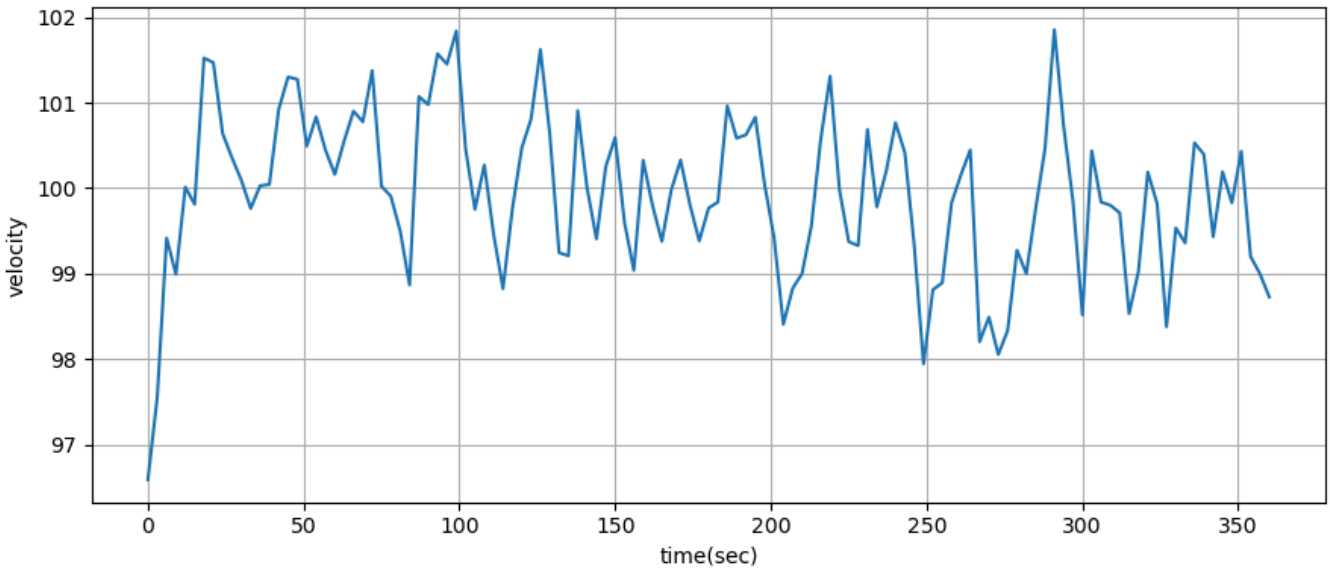
该测量结果包含四个数值，因此测量函数也需要返回四个数值。斜距和高程角的计算方式与以往相同。至于 x 和 y 处的速度值，则无需再单独计算，因为这些数值已经由状态估计结果给出。

```
def h_vel(x):
    dx = x[0] - h_vel.radar_pos[0]
    dz = x[2] - h_vel.radar_pos[1]
    slant_range = math.sqrt(dx**2 + dz**2)
    elevation_angle = math.atan2(dz, dx)
    return slant_range, elevation_angle, x[1], x[3]
```

现在我们可以实现我们的滤波功能了。

```
h_radar.radar_pos = (0, 0)
h_vel.radar_pos = (0, 0)

range_std = 500.
elevation_angle_std = math.degrees(0.5)
vel_std = 2.

np.random.seed(200)
ac = ACSim(ac_pos, (100, 0), 0.02)
radar = RadarStation((0, 0), range_std, elevation_angle_std)

kf_sf2 = cv_UKF(f_cv_radar, h_vel,
            R_std=[range_std, elevation_angle_std, vel_std, vel_std])

time = np.arange(0, 360 + dt, dt)
xs = []
for t in time:
```

```
    ac.update(dt)
    r = radar.noisy_reading(ac.pos)
    # simulate the doppler velocity reading
    vx = ac.vel[0] + randn()*vel_std
    vz = ac.vel[1] + randn()*vel_std
    kf_sf2.predict()
    kf_sf2.update([r[0], r[1], vx, vz])
    xs.append(kf_sf2.x)
xs = np.asarray(xs)
plot_radar(xs, time, plot_x=False, plot_vel=True, plot_alt=False)
print(f'Velocity std {np.std(xs[10:,1]):.1f} m/s')
```




```
Velocity std 0.9 m/s
```

通过引入速度传感器，我们成功将标准偏差从 3.5 米/秒降低到了 1.3 米/秒。

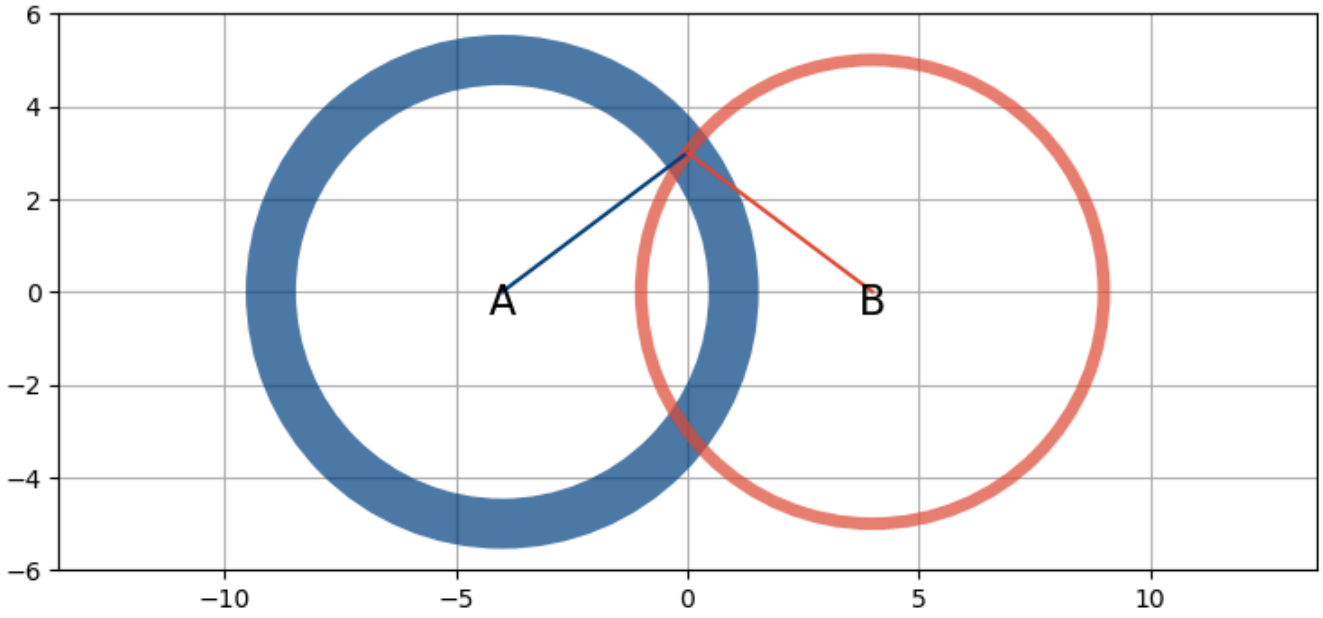
传感器融合是一个复杂的课题，而这里的实现方式相当简单。在典型的导航系统中，各种传感器会提供互补的信息。例如，GPS 能够以每秒一次的频率提供相当精确的位置信息，但速度测量的精度较低；而惯性测量单元则能以 50 赫兹的频率提供非常精确的速度信息，但位置测量的精度较差。各种传感器的优点和缺点是相互补充的。因此，人们采用了“互补滤波器”这种技术：将惯性测量单元提供的高频率速度数据与 GPS 提供的高精度但更新速度较慢的位置信息相结合，从而得到既准确又高频率的位置和速度数据。这些高频率的速度数据会被整合到 GPS 提供的位置数据中，进而得到更加精确的位置信息。

多个位置传感器

上一个关于传感器数据融合的例子其实相当简单。现在，让我们来处理一个没那么简单的问题吧。在 GPS 出现之前，飞机依靠各种测距和测向系统来导航，比如 VOR、LORAN、TACAN、DME 等等。这些系统会以无线电波的形式发出信号。传感器则从这些信号中获取到到信号源的距离和/或方向。例如，一架飞机可能装有两个 VOR 接收器。飞行员会将每个接收器调谐到不同的 VOR 信号源上。每个 VOR 接收器都会显示从地面上的 VOR 信号源到飞机的方向。飞行员再利用图表来确定这些方向的交点，从而确定飞机的位置。

那是一种精度较低的手动测量方法。而卡尔曼滤波器则能提供精度高得多的位置估计结果。假设我们有两个传感器，每个传感器都只能提供目标的方向信息，如下图所示。各条边的宽度与传感器噪声的大小成正比。因此，飞机很可能位于这两条边所围成的区域之内。

```
ukf_internal.show_two_sensor_bearing()
```



我们按照以下方式来计算传感器与目标之间的方位角：

```
def bearing(sensor, target_pos):
    return math.atan2(target_pos[1] - sensor[1],
                      target_pos[0] - sensor[0])
```

该滤波器以向量形式接收来自两个传感器的测量数据。该代码可以接受任何可迭代的数据结构，为了提高效率，我使用了 Python 列表来存储数据。具体实现方式如下：

```
def measurement(A_pos, B_pos, pos):
    angle_a = bearing(A_pos, pos)
    angle_b = bearing(B_pos, pos)
    return [angle_a, angle_b]
```

假设飞机的运动速度是恒定的。为了换个计算方式，我选择直接计算出飞机的新位置，而不是使用矩阵-向量乘法来计算。

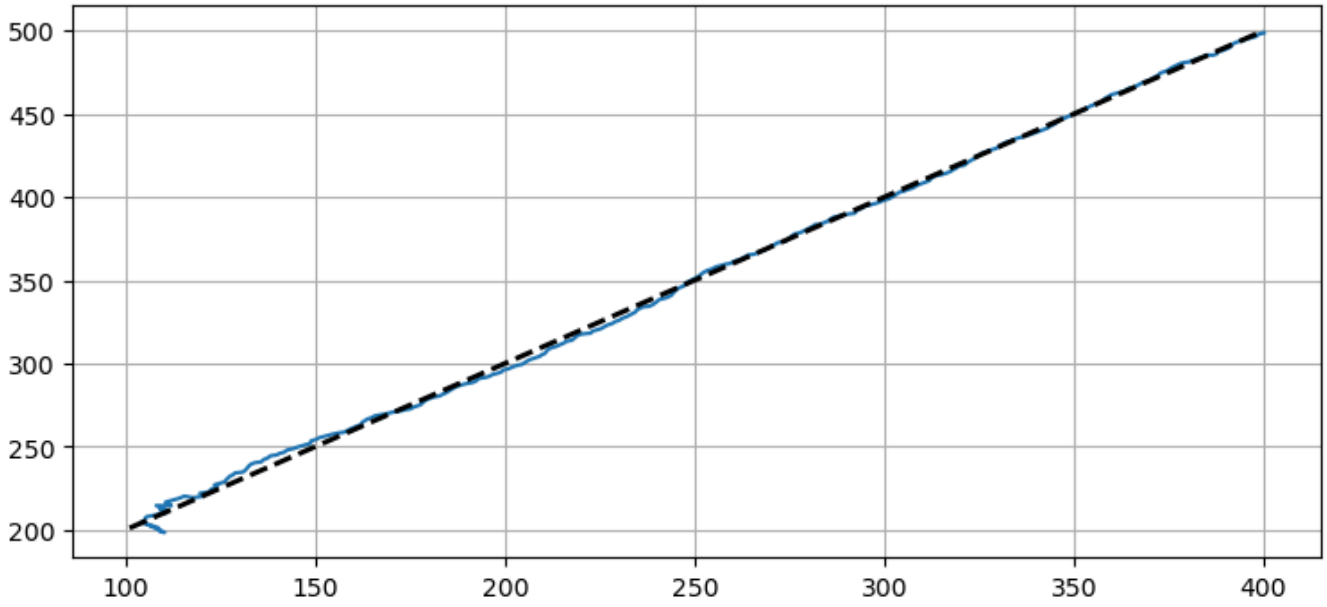
```
def fx_VOR(x, dt):
    x[0] += x[1] * dt
    x[2] += x[3] * dt
    return x
```

接下来，我们来实现测量功能。该功能会将先验信息转换为包含两个站点测量数据的数组。虽然我不喜欢使用全局变量，但为了演示如何与 $h()$ 共享数据，我还是将两个站点的位置分别存储在全局变量 `sa_pos` 和 `sb_pos` 中。

```
sa_pos = [-400, 0]
sb_pos = [400, 0]

def hx_VOR(x):
    # measurement to A
    pos = (x[0], x[2])
    return measurement(sa_pos, sb_pos, pos)
```

现在我们来编写一些通用代码，用于构建滤波器、运行该滤波器并绘制结果图表：

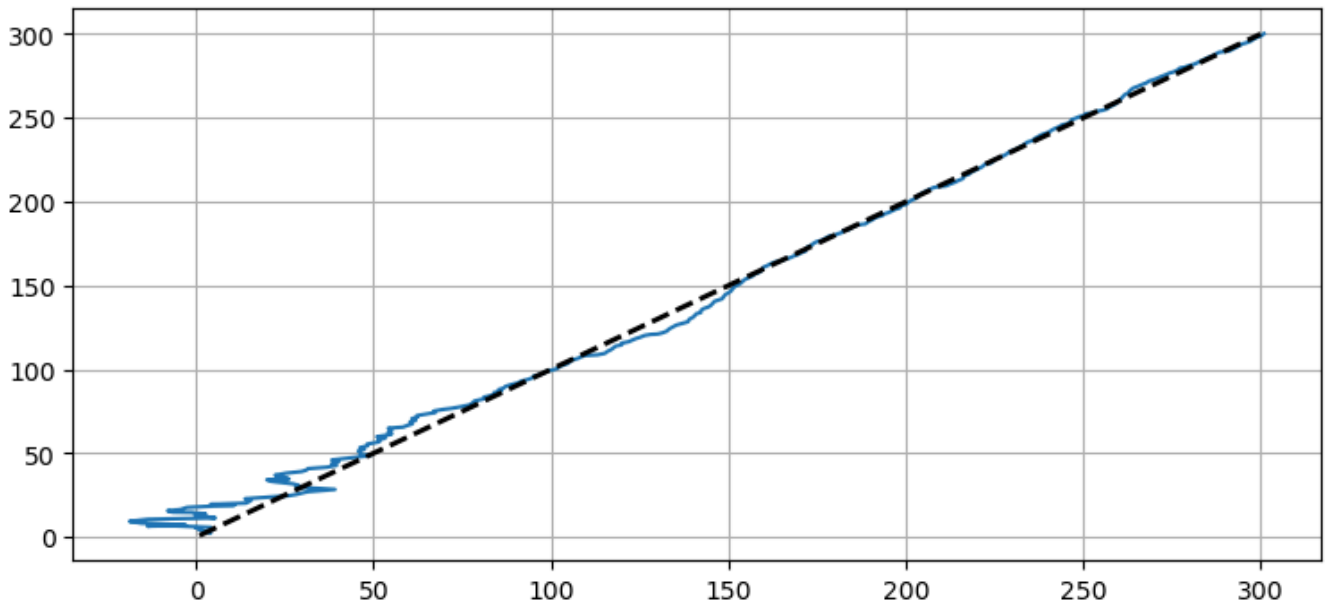
```
def moving_target_filter(pos, std_noise, Q, dt=0.1, kappa=0.0):
    points = MerweScaledSigmaPoints(n=4, alpha=.1, beta=2., kappa=kappa)
    f = UKF(dim_x=4, dim_z=2, dt=dt,
            hx=hx_VOR, fx=fx_VOR, points=points)
    f.x = np.array([pos[0], 1., pos[1], 1.])

    q = Q_discrete_white_noise(2, dt, Q)
    f.Q[0:2, 0:2] = q
    f.Q[2:4, 2:4] = q
    f.R *= std_noise**2
    f.P *= 1000
    return f

def plot_straight_line_target(f, std_noise):
    xs, txs = [], []
    for i in range(300):
        target_pos[0] += 1 + randn()*0.0001
        target_pos[1] += 1 + randn()*0.0001
        txs.append((target_pos[0], target_pos[1]))

        z = measurement(sa_pos, sb_pos, target_pos)
        z[0] += randn() * std_noise
        z[1] += randn() * std_noise

        f.predict()
        f.update(z)
        xs.append(f.x)

    xs = np.asarray(xs)
    txs = np.asarray(txs)

    plt.plot(xs[:, 0], xs[:, 2])
    plt.plot(txs[:, 0], txs[:, 1], ls='--', lw=2, c='k')
    plt.show()

np.random.seed(123)
target_pos = [100, 200]

std_noise = math.radians(0.5)
f = moving_target_filter(target_pos, std_noise, Q=1.0)
plot_straight_line_target(f, std_noise)
```



在我看来，效果相当不错。在处理的初期，误差较大，但经过滤波处理后，得到的结果相当准确。

让我们再来看看角度的非线性问题。我会把目标点置于两个传感器之间的(0,0)位置。这样一来，残差的计算结果就会出现非线性：因为平均角度会接近零。当角度小于 0 时，测量结果会显示出接近 2π 的巨大正数值。因此，预测值与测量值之间的残差会非常大，接近 2π，而不是接近 0。这就使得滤波器无法准确地进行计算，如下例所示。

```
target_pos = [0, 0]
f = moving_target_filter(target_pos, std_noise, Q=1.0)
plot_straight_line_target(f, std_noise)
```

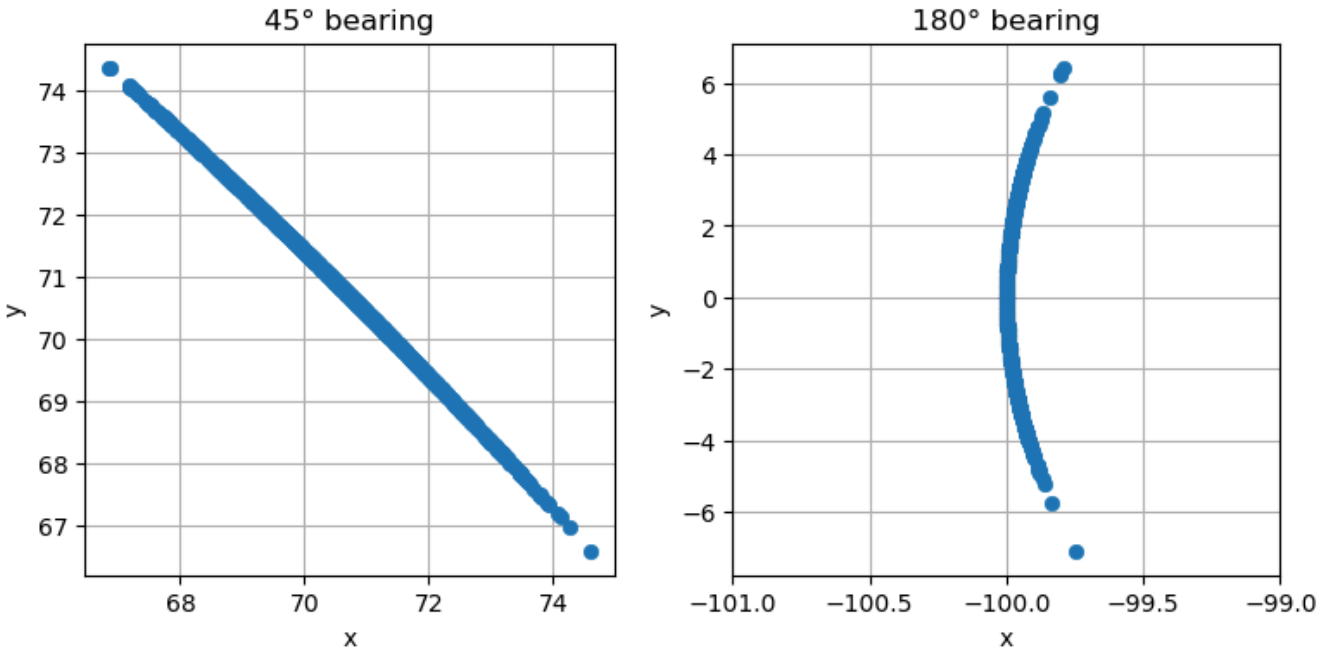


这种性能表现是不可接受的。 `FilterPy` 的 UKF 算法允许用户指定一个函数，用于计算在出现此类非线性行为时的残差值。本章的最后一个示例就展示了该算法的用法。

传感器误差与几何结构的影响

传感器相对于被追踪物体的几何结构会带来一些物理上的限制，这些限制在设计滤波器时很难克服。如果 VOR 站的辐射方向几乎相互平行，那么微小的角度误差就会导致巨大的距离误差。更糟糕的是，这种误差是非线性的——x 轴和 y 轴上的误差会随着实际方位角的不同而有所变化。这些散点图展示了在 1°的误差范围内，两种不同方位角下的误差分布情况。

```
ukf_internal.plot_scatter_of_bearing_error()
```



练习：解释滤波器的性能表现

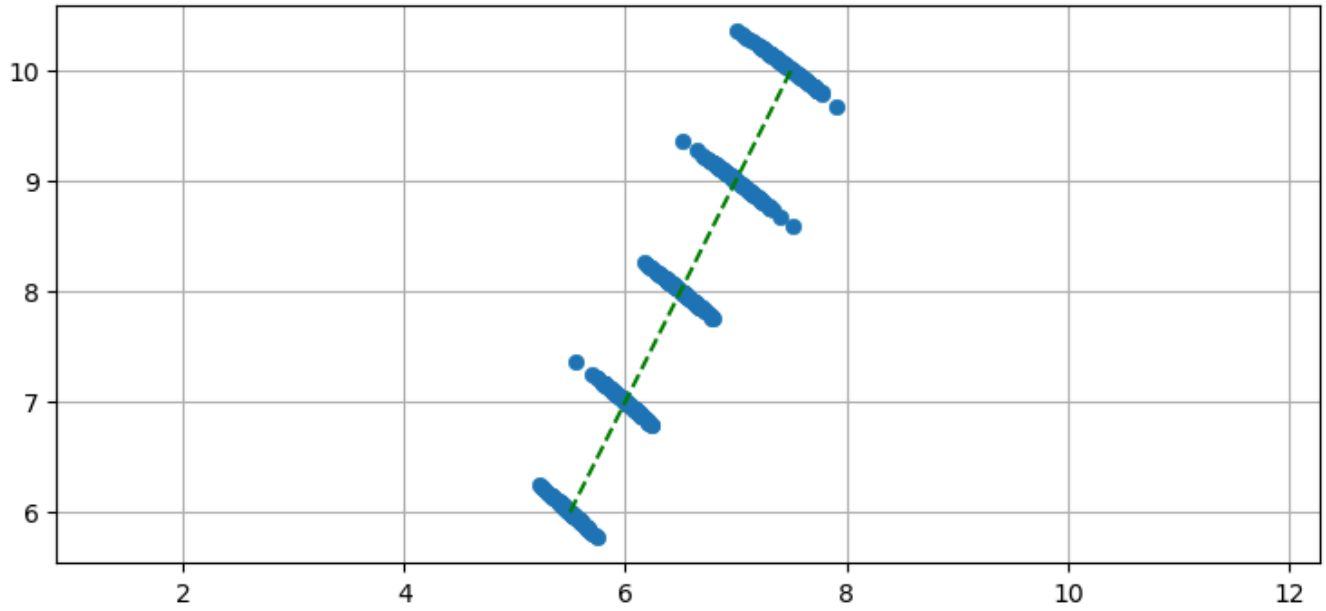
我们可以看到，当角度误差较小时，位置误差会非常大。请解释一下：在上述目标跟踪任务中，为什么 UKF 算法能够取得相对不错的性能。请同时针对单传感器和多传感器的情况进行说明。

解决方案

理解这一点非常重要。请尽量在阅读下面的答案之前自己试着回答这个问题。如果你无法回答，那么你可能需要重新复习一下“多维卡尔曼滤波器”这一章中的相关内容。

我们的成功得益于多个因素。首先，以仅使用一个传感器的情况为例。单次测量所得的结果范围可能非常大。不过，我们的目标物是在移动的，而 UKF 算法正好考虑到了这一点。让我们把对移动目标进行连续多次测量所得的结果绘制出来吧。

```
ukf_internal.plot_scatter_moving_target()
```



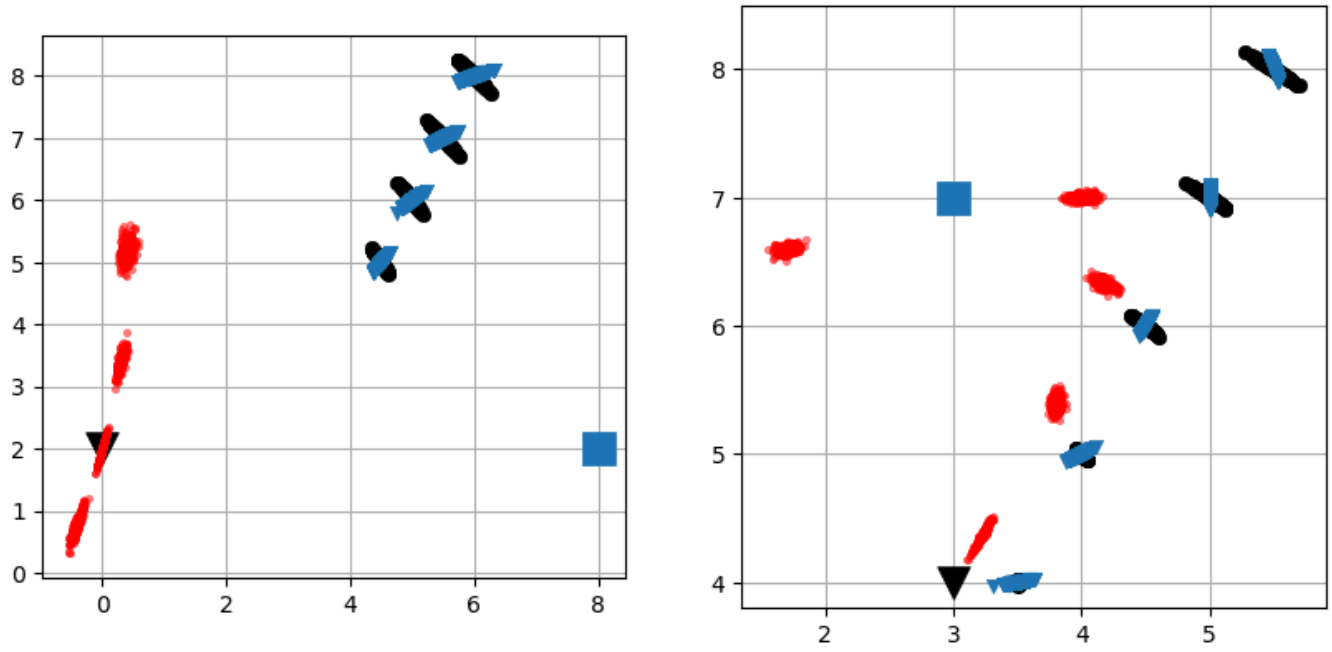
每次单独的测量结果都存在较大的误差。不过，将连续的测量结果绘制成图表后，可以清晰地看出一个趋势：目标显然正在向右上方移动。当卡尔曼滤波器计算卡尔曼增益时，它会利用测量结果来考虑误差的分布情况。在这个例子中，误差大致沿着 45°线分布，因此滤波器会降低该方向上的误差权重。而与这条线垂直方向的误差则几乎可以忽略不计，卡尔曼增益也会将这一点纳入考虑范围。

从这张图表来看，一切似乎都很简单：因为我们为每个位置更新都记录了 100 次测量数据。飞机的运动轨迹一目了然。相比之下，卡尔曼滤波器每次只进行一次测量。因此，其预测结果自然无法像那条绿色的虚线所显示的那样精确。

现在假设该传感器无法提供距离信息。如果我们把初始估计值设定为距离传感器 1,000 公里处（而实际距离仅为 7.07 公里），同时将 P 的值设得非常小。在这样远的距离上，1°的误差会转化为 17.5 公里的定位误差。由于滤波器对自身位置的估计过于精确，且测量数据中不包含距离信息，因此 KF 算法永远无法准确确定目标位置。

现在，让我们来考虑增加第二个传感器所带来的影响。下图展示了不同传感器位置所造成的影响。我用正方形和三角形分别代表这两个传感器，并用相同的符号形状和颜色来表示每个传感器所导致的误差分布。接着，我计算出了与这两个有误差的测量值相对应的坐标，用红点标出这些点，从而显示出在 x 和 y 区域内的误差分布情况。

```
with figsize(10,5):
    ukf_internal.plot_iscts_two_sensors()
```



在第一个图中，我将传感器放置得几乎与目标的初始位置垂直，这样就能得到漂亮的“X”形交叉点。从那些散布的红色点可以看出，随着目标的移动， x 和 y 的数值会如何变化。当目标远离传感器，而更靠近传感器 B 的 y 坐标点时，这些点的分布会呈现出明显的椭圆形。

在第二幅图中，飞机最初位于距离某个传感器很近的位置，然后飞过第二个传感器。由于误差的交点并不呈垂直状态，因此产生的位置误差会变得相当分散。

UKF 算法的实现

FilterPy 实现了 UKF 算法，但学习如何将数学公式转化为代码的过程也很有意义。实现 UKF 算法其实相当简单。首先，我们来编写代码来计算各 sigma 点的均值和协方差吧。

我们会将西格玛点和权重存储在矩阵中，具体如下：

$$\begin{aligned}\text{weights} &= \begin{bmatrix} w_0 & w_1 & \dots & w_{2n} \end{bmatrix} \\ \text{sigmas} &= \begin{bmatrix} X_{0,0} & X_{0,1} & \dots & X_{0,n-1} \\ X_{1,0} & X_{1,1} & \dots & X_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ X_{2n,0} & X_{2n,1} & \dots & X_{2n,n-1} \end{bmatrix}\end{aligned}$$

为了描述这么简单的事情，竟然需要使用这么多下标。下面以二维问题的例子来说明吧（ n =2）：

```
points = MerweScaledSigmaPoints(n=2, alpha=.1, beta=2., kappa=1.)
points.sigma_points(x=[0.,0], P=[[1.,.1],[.1, 1]])
```

```
array([[ 0.   ,  0.   ],
       [ 0.173,  0.017],
       [ 0.   ,  0.172],
```

```
        [-0.173, -0.017],
        [ 0.   , -0.172]])
```

均值的第一个σ点位于第一行，其坐标为(0, 0)，这与均值本身的坐标相同。第二个σ点的坐标为(0.173, 0.017)，依此类推。总共有 $2n+1=5$ 个σ点，每个σ点对应一行。如果 $n=3$ 个σ点，那么就会有 3 列和 7 行。

决定是将西格玛值以行列格式还是列行格式来存储，其实是个相对随意的选择。不过，我选择的方式让代码更易于理解：因为这样，我就可以将 i th 位置的西格玛值称为 `sigmas[i]`，而不是 `sigmas[:, i]`。

权重/比重

使用 NumPy 来计算权重非常简单。请记住，Van der Merwe 提出的方法是基于缩放后的 sigma 点来计算状态的。

$$\begin{aligned}
\lambda &= \alpha^2(n+\kappa) - n \\
W_0^m &= \frac{\lambda}{n+\lambda} \\
W_0^c &= \frac{\lambda}{n+\lambda} + 1 - \alpha^2 + \beta \\
W_i^m = W_i^c &= \frac{1}{2(n+\lambda)} i = 1..2n
\end{aligned}$$

这些的代码如下：

```
lambda_ = alpha**2 * (n + kappa) - n
Wc = np.full(2*n + 1,  1. / (2*(n + lambda_))
Wm = np.full(2*n + 1,  1. / (2*(n + lambda_))
Wc[0] = lambda_ / (n + lambda_) + (1. - alpha**2 + beta)
Wm[0] = lambda_ / (n + lambda_)
```

我在 `lambda_` 中使用下划线，是因为 `lambda` 在 Python 中是保留字。在字符串末尾加上下划线，是符合 Python 编程习惯的解决方法。

西格玛点数

西格玛点的计算公式如下：

$$\begin{cases}
X_0 = \mu \\
X_i = \mu + \left[\sqrt{(n+\lambda)\Sigma}\right]_i, & i = 1..n \\
X_i = \mu - \left[\sqrt{(n+\lambda)\Sigma}\right]_{i-n} & i = (n+1)\,..2n
\end{cases}$$

只要我们理解了 $\left[\sqrt{(n+\lambda)\Sigma}\right]_i$ 这个概念，使用 Python 就并不难了。

符号 $\sqrt{(n+\lambda)\Sigma}$ 实际上是一个矩阵，因为 Σ 本身也是矩阵。在 $[\sqrt{(n+\lambda)\Sigma}]_i$ 中，下标 i 表示选取该矩阵的第 i 个行向量。那么，什么是矩阵的平方根呢？其实并没有唯一的定义。一种定义是：矩阵 Σ 的平方根，就是那个满足以下条件的矩阵 S：当 S 与自身相乘时，结果恰好为 Σ，即：如果 $\Sigma = SS$，那么 $S = \sqrt{\Sigma}$。

我们将选择另一种定义方式，这种定义方式所具有的数值特性使得计算更为简便。我们可以将平方根定义为矩阵 S，当该矩阵与自身的转置相乘时，结果即为 Σ 。

$$\Sigma = SS^T$$

这一定义更受青睐，因为 S 是通过乔列斯基分解来计算的[5]。该算法将一个厄米特正定矩阵分解为一个三角矩阵及其共轭转置。这个三角矩阵可以是上三角矩阵，也可以是下三角矩阵，如下所示：

$$A = LL^*A = U^*U$$

星号表示共轭转置；由于我们处理的都是实数，因此可以这样表示：

$$A = LL^TA = U^TU$$

P 具有这些特性，因此我们可以将 $S = \text{cholesky}(P)$ 视为 P 的平方根。

SciPy 在 `scipy.linalg` 中提供了 `cholesky()` 方法。如果你使用的是 Fortran、C 或 C++等语言，那么 LAPACK 等库就提供了该功能。Matlab 则提供了 `chol()` 功能。

默认情况下， `scipy.linalg.cholesky()` 返回的是一个上三角矩阵。因此，我决定编写代码来指定期望得到一个上三角矩阵。出于这个原因，我以行为单位访问结果，这样第一个 sigma 点，也就是中心点，就会受到一整行非零值的影响。如果你使用的是自己的平方根计算算法，就需要考虑这一点。在文献中，你会发现有些算法会先处理列值。如果 Cholesky 分解后的矩阵是下三角矩阵，或者你使用的是其他能够计算对称矩阵的算法，那么行与列的顺序并不重要。

```
import scipy
a = np.array([[2., .1], [.1, 3]])
s = scipy.linalg.cholesky(a)
print("cholesky:")
print(s)
print("\nsquare of cholesky:")
print(s @ s.T)
```

```
cholesky:
[[1.414 0.071]
 [0.    1.731]]

square of cholesky:
[[2.005 0.122]
 [0.122 2.995]]
```

因此，我们可以用这段代码来实现西格玛点算法：

```
sigmas = np.zeros((2*n+1, n))
U = scipy.linalg.cholesky((n+lambda_)*P) # sqrt

sigmas[0] = X
for k in range (n):
    sigmas[k+1]   = X + U[k]
    sigmas[n+k+1] = X - U[k]
```

现在让我们来实现无迹变换吧。请回顾一下相关的公式。

$$\mu = \sum_i w_i^m X_i$$
$$\Sigma = \sum_i w_i^c (X_i - \mu)(X_i - \mu)^T$$

我们采用均值之和的方法来进行计算。

```
x = np.dot(Wm, sigmas)
```

如果你不经常使用 NumPy，那么这些内容可能会让你觉得陌生。NumPy 不仅仅是一个用于实现线性代数运算的库；实际上，它是用 C 和 Fortran 语言编写的，因此其运行速度远远高于 Python。通常，其速度提升可达 20 到 100 倍。为了获得这样的性能提升，我们必须避免使用 for 循环，而应使用 NumPy 内置的功能来执行计算。例如，与其用 for 循环来计算向量乘积的和，不如直接使用 NumPy 内置的 `numpy.dot(x, y)` 方法。两个向量的点积，其实就是各元素相乘后的总和。如果输入的是一维数组和二维数组，NumPy 会自动计算它们之间的内积。

```
a = np.array([10, 100])
b = np.array([[1, 2, 3],
              [4, 5, 6]])
np.dot(a, b)
```

```
array([410, 520, 630])
```

现在只需计算 $P = \sum_i w_i (X_i - \mu)(X_i - \mu)^T + Q$ 即可：

```
kmax, n = sigmas.shape
P = zeros((n, n))
for k in range(kmax):
    y = sigmas[k] - x
    P += Wc[k] * np.outer(y, y)
P += Q
```

这又体现了 NumPy 的另一个特性。状态变量 `x` 是一维的， `sigmas[k]` 也是一维的，因此差值 `sigmas[k]-X` 同样是一维的。NumPy 不会对一维数组进行转置操作；它认为 `[1,2,3]` 的转置就是 `[1,2,3]` 。因此，我们调用函数 `np.outer(y,y)` 来计算一维数组 y 的 yy^T 值。当然，也可以采用另一种实现方式：

```
y = (sigmas[k] - x).reshape(kmax, 1) # convert into 2D array
P += Wc[K] * np.dot(y, y.T)
```

这段代码运行速度较慢，而且不符合编程规范，因此我们不会使用它。

预测步骤

在预测阶段，我们将按照上述方式来生成权重和西格玛点。然后，我们将每个西格玛点代入函数 f 中进行计算。

$$Y = f(\chi)$$

接着，我们利用无迹变换来计算预测的均值和协方差。在下面的代码中可以看到，我假设这是一种属于某个类的方法，而该类负责存储滤波器所需要的各种矩阵和向量。

```
def predict(self, sigma_points_fn):
    """ Performs the predict step of the UKF. On return,
    self.xp and self.Pp contain the predicted state (xp)
    and covariance (Pp). 'p' stands for prediction.
    """

    # calculate sigma points for given mean and covariance
    sigmas = sigma_points_fn(self.x, self.Pp)

    for i in range(self._num_sigmas):
        self.sigmas_f[i] = self.fx(sigmas[i], self._dt)

    self.xp, self.Pp = unscented_transform(
                      self.sigmas_f, self.Wm, self.Wc, self.Q)
```

更新步骤

更新步骤通过函数 `h(x)` 将西格玛值转换为测量空间中的数值。

$$Z = h(Y)$$

这些点的均值和协方差是通过无迹变换来计算的。之后，再计算残差和卡尔曼增益。交叉方差则按以下方式计算：

$$P_{xz} = \sum_{i=0}^{2n} w_i^c (Y_i - \mu)(Z_i - \mu_z)^T$$

最后，我们利用残差和卡尔曼增益来计算新的状态估计值：

$$\begin{aligned} K &= P_{xz} P_z^{-1} \\ x &= \bar{x} + Ky \end{aligned}$$

新的协方差则按照以下方式计算：

$$P = \bar{P} - KP_zK^T$$

假设该函数属于某个类的方法，而该类负责存储各种必要的矩阵和数据，那么该函数的实现方式如下。

```
def update(self, z):
    # rename for readability
    sigmas_f = self.sigmas_f
    sigmas_h = self.sigmas_h

    # transform sigma points into measurement space
    for i in range(self._num_sigmas):
        sigmas_h[i] = self.hx(sigmas_f[i])

    # mean and covariance of prediction passed through UT
    zp, Pz = unscented_transform(sigmas_h, self.Wm, self.Wc, self.R)

    # compute cross variance of the state and the measurements
    Pxz = np.zeros((self._dim_x, self._dim_z))
```

```
    for i in range(self._num_sigmas):
        Pxz += self.Wc[i] * np.outer(sigmas_f[i] - self.xp,
                                     sigmas_h[i] - zp)

    K = np.dot(Pxz, inv(Pz)) # Kalman gain

    self.x = self.xp + np.dot(K, z - zp)
    self.P = self.Pp - np.dot(K, Pz).dot(K.T)
```

FilterPy 的实现方式

FilterPy 在某种程度上对相关代码进行了优化。你可以指定不同的西格玛点算法，决定如何计算状态变量的残差值（由于角度具有周期性，因此无法直接用减法来处理角度值）。此外，还可以指定矩阵求根函数等。详情请参阅帮助文档。

https://filterpy.readthedocs.org/#unscented-kalman-filter

批量处理

卡尔曼滤波器是一种递归式算法——其预测结果是基于当前的测量数据以及之前的预测结果得出的。不过，通常情况下，我们手头还有一批已经收集到的数据也需要进行滤波处理。在这种情况下，滤波器可以以批量处理的方式运行，也就是一次性对所有测量数据进行处理。

把测量得到的数值收集到一个数组或列表中。

```
zs = read_altitude_from_csv()
```

然后调用 `batch_filter()` 方法。

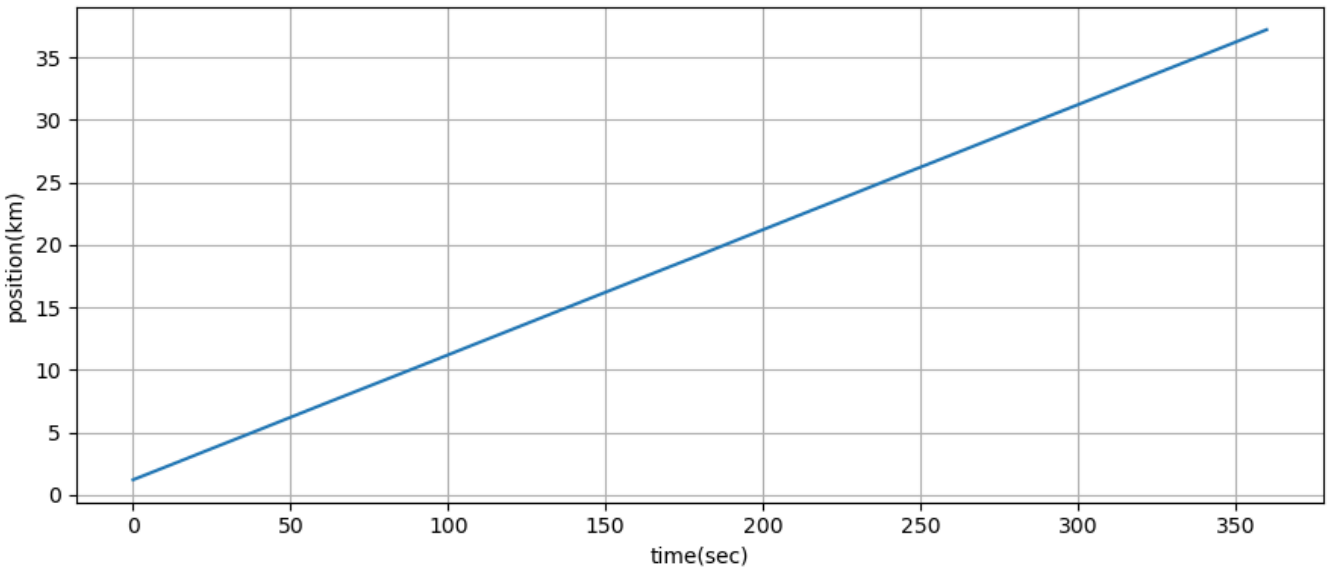
```
Xs, Ps = ukf.batch_filter(zs)
```

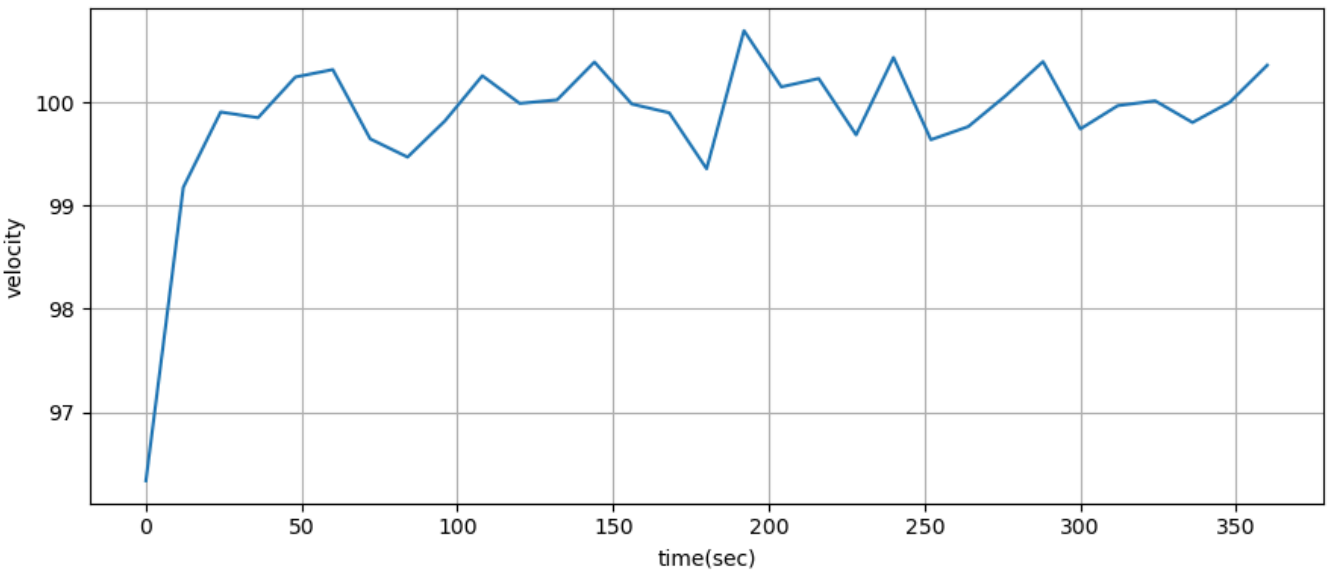
该函数接收测量值列表/数组，对其进行处理后，返回整个数据集的状态估计值数组（ `Xs` ）以及协方差矩阵数组（ `Ps` ）。

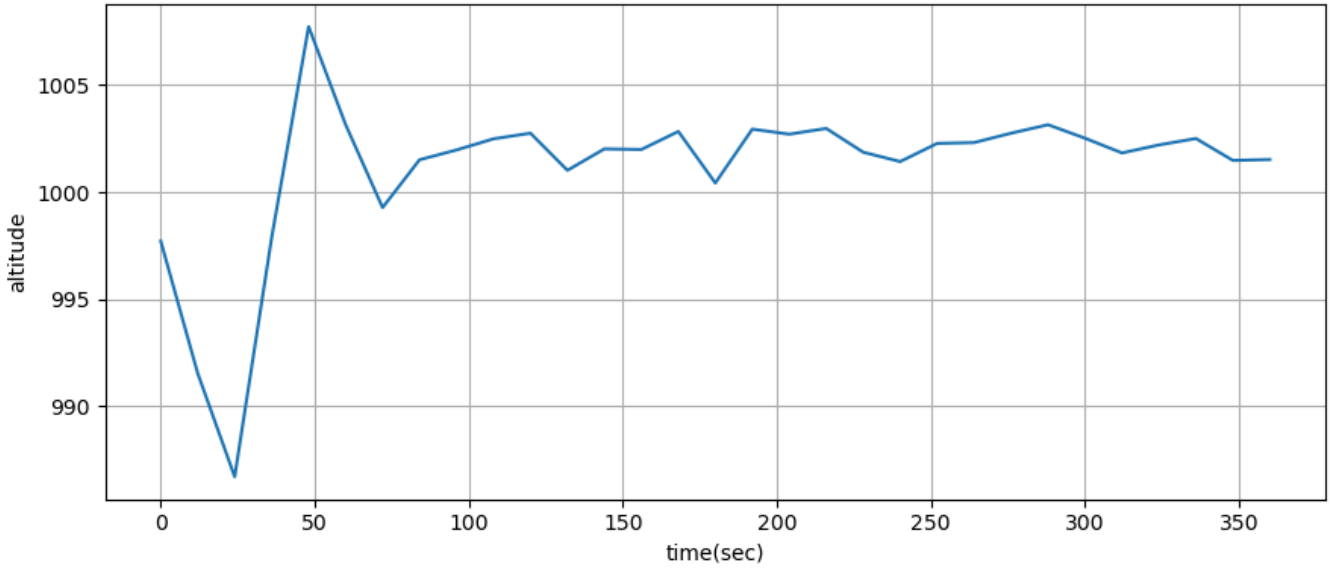
以下是针对上述雷达跟踪问题所给出的完整示例。

```
dt = 12. # 12 seconds between readings
range_std = 5 # meters
bearing_std = math.radians(0.5)

ac_pos = (0., 1000.)
ac_vel = (100., 0.)
radar_pos = (0., 0.)
h_radar.radar_pos = radar_pos

points = MerweScaledSigmaPoints(n=3, alpha=.1, beta=2., kappa=0.)
kf = UKF(3, 2, dt, fx=f_radar, hx=h_radar, points=points)

kf.Q[0:2 ,0:2] = Q_discrete_white_noise(2, dt=dt, var=0.1)
kf.Q[2, 2] = 0.1
```

```
kf.R = np.diag([range_std**2, bearing_std**2])
kf.x = np.array([0., 90., 1100.])
kf.P = np.diag([300**2, 30**2, 150**2])

radar = RadarStation((0, 0), range_std, bearing_std)
ac = ACSim(ac_pos, (100, 0), 0.02)

np.random.seed(200)

t = np.arange(0, 360 + dt, dt)
n = len(t)

zs = []
for i in range(len(t)):
    ac.update(dt)
    r = radar.noisy_reading(ac.pos)
    zs.append([r[0], r[1]])

xs, covs = kf.batch_filter(zs)
ukf_internal.plot_radar(xs, t)
```







对结果进行平滑处理

假设我们正在追踪一辆汽车。如果我们得到的测量数据有较大误差，表明汽车似乎要向左转弯，但状态函数却预测汽车会继续直线行驶。在这种情况下，卡尔曼滤波器别无选择，只能将状态估计值稍微向那个有误差的测量结果方向调整。因为它无法判断：这究竟只是因为测量数据有误差，还是汽车真的开始转弯了。

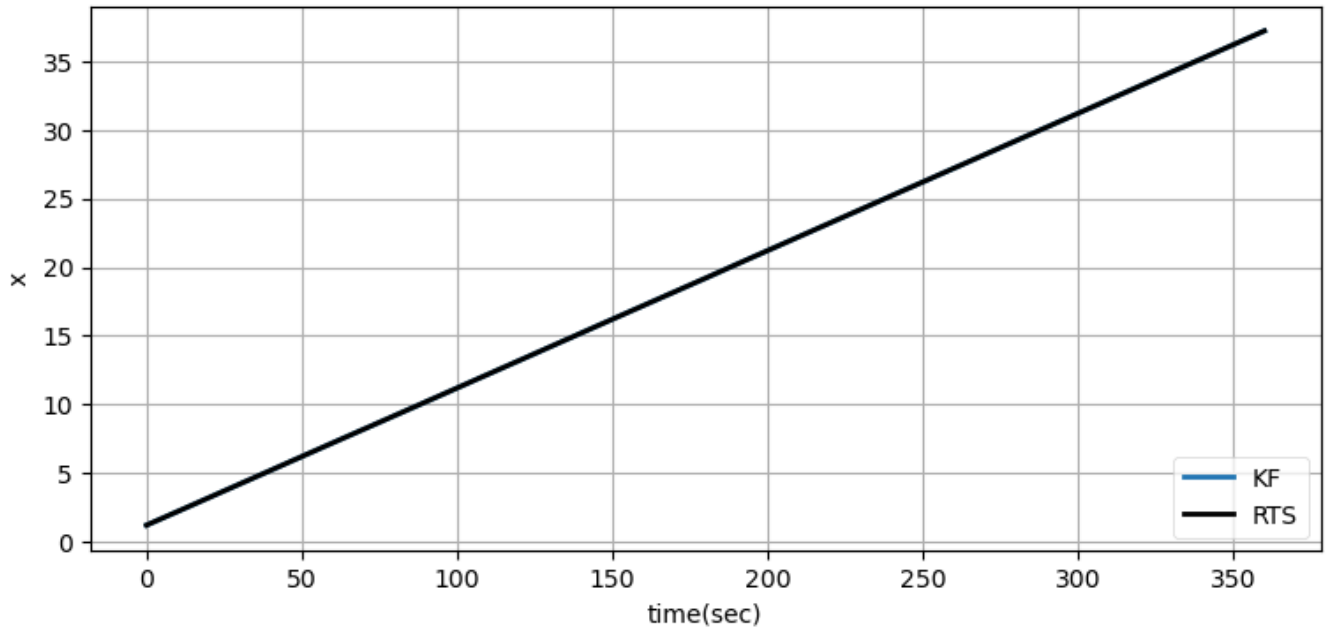
如果我们收集数据并进行后处理，那么在那些有疑问的测量结果之后，我们就能得到确切的测量结果，从而判断是否真的发生了转向。假设后续的所有测量结果都表明车辆在继续向左转弯，那么我们就可以确定，之前的测量结果并不可信，实际上车辆确实发生了转向。

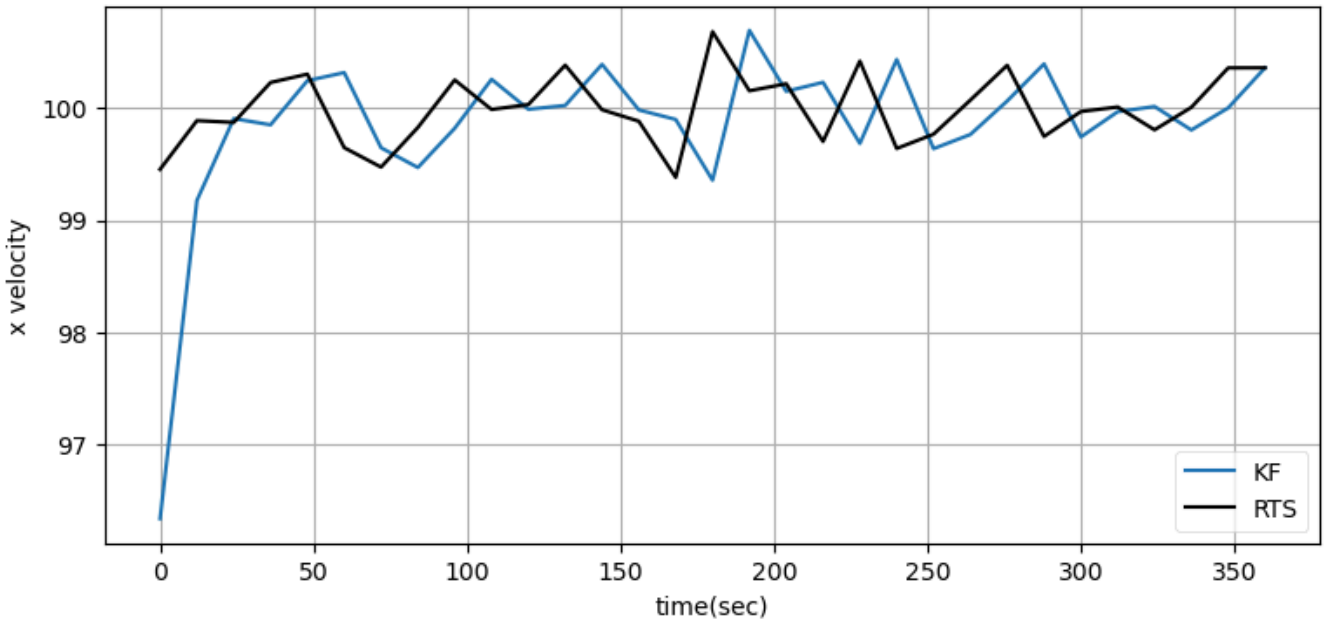
我们不会在这里详细阐述该数学公式或算法的实现细节。我只会向您展示如何在 `FilterPy` 中调用该算法。我们所实现的这个算法被称为“RTS 平滑器”，其名称源自该算法的三位发明者：Rauch、Tung 和 Striebel。

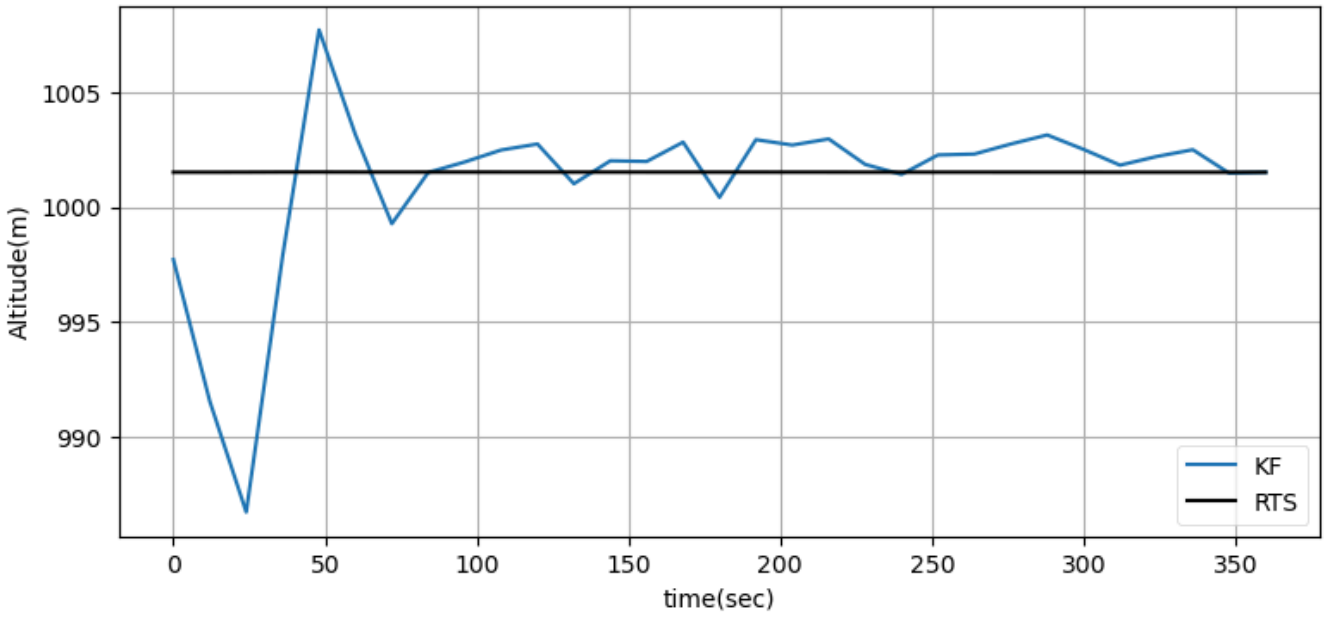
该程序的代码为 `UnscentedKalmanFilter.rts_smoother()` 。使用起来非常简单：我们只需将 `batch_filter` 步骤中计算得到的均值和协方差作为输入，就能得到经过处理的均值、协方差以及卡尔曼增益。

```
Ms, P, K = kf.rts_smoother(xs, covs)
ukf_internal.plot_rts_output(xs, Ms, t)
```

```
Difference in position in meters:
     [-1.4166 -0.2815  1.2679 -1.2405 -2.1863]
```


x
time(sec)
KF
RTS


x velocity
time(sec)
KF
RTS



从这些图表可以看出，位置的改善幅度很小，但速度的改善相当明显，尤其是在高度方面。位置上的差异非常微小，因此我打印出了过去 5 个数据点上 UKF 算法与平滑处理后结果之间的差异。如果您能够对数据进行后处理，我建议您始终使用 RTS 平滑算法。

选择西格玛参数

我发现，关于如何为 α 、 β 和 κ 确定合适的数值，现有的文献相当有限。范德梅尔韦的论文中包含了最多的相关信息，但仍然不够全面。那么，让我们来仔细研究一下这些数值应该怎么确定吧。

范德梅尔韦建议，在处理高斯问题时使用 $\beta = 2$ ，而使用 $\kappa = 3 - n$ 则适用于其他情况。那我们就从这里开始吧，然后试着改变 α 的值。我会让 $n = 1$ 来帮助我们尽量减少需要处理的数组大小，从而避免计算矩阵的平方根。

```
from kf_book.ukf_internal import print_sigmas
print_sigmas(mean=0, cov=3, alpha=1)
```

```
sigmas:  [ 0.  3. -3.]
mean weights: [0.6667 0.1667 0.1667]
cov weights: [2.6667 0.1667 0.1667]
lambda: 2
sum cov 2.9999999999999996
```

那么，这里到底发生了什么？我们可以看到，当平均值为 0 时，该算法选择的西格玛点分别为 0、3 和-3。但为什么会这样呢？请回想一下计算西格玛点的公式：

$$
\begin{aligned}
X_0 &= \mu \\
X_i &= \mu \pm \sqrt{(n+\lambda)\Sigma}
\end{aligned}
$$

我选择使用 $n = 1$ 的方式可以将所有数值都转化为标量形式，从而避免了对矩阵求平方根的计算。因此，对于我们所处理的数值来说，该方程可以简化为：

$$\begin{aligned} X_0 &= 0 \\ X_i &= 0 \pm \sqrt{(1+2) \times 3} \\ &= \pm 3 \end{aligned}$$

因此，当 α 的数值越大时，那些“西格玛点”就会分布得越广。我们不妨把它设置为一个极大的数值吧。

```
print_sigmas(mean=0, cov=3, alpha=200)
```

```
sigmas:  [   0.  600. -600.]
mean weights: [1. 0. 0.]
cov weights: [-39996.      0.      0.]
lambda: 119999
sum cov -39996.00000000001
```

我们可以看到，西格玛点分布在 100 个标准差范围内。如果我们的数据呈高斯分布，那么我们就会把那些远离平均值很多标准差的数据也纳入考虑范围；但对于非线性问题来说，这样做很可能无法得到准确的结果。不过，假设我们的数据并非高斯分布，而是具有“厚尾”特征呢？在这种情况下，我们可能有必要从这些“厚尾”区域中抽取样本才能得到准确的估计。因此，将 κ 的值设得更大些是合理的（当然，200 这个数值实在过大，因为那样做只会让西格玛点的变化显得过于明显）。

用类似的逻辑来推理，假设我们的概率分布几乎没有“尾部”——也就是说，其形状更像是一个倒置的抛物线。在这种情况下，我们可能会希望将那些“西格玛点”调整得更靠近均值，从而避免在那些永远不会有真实数据出现的区域进行采样。

现在让我们来看看权重的变化情况。当使用 $k + n = 3$ 时，均值的权重为 0.6667，而两个异常值的权重则分别为 0.1667。而当使用 $\alpha = 200$ 时，均值的权重上升到了 0.99999，而两个异常值的权重则降到了 0.000004。请记住，权重的计算公式如下：

$$\begin{aligned} W_0 &= \frac{\lambda}{n + \lambda} \\ W_i &= \frac{1}{2(n + \lambda)} \end{aligned}$$

我们可以看到，当 λ 的值越大时，均值所占的权重比例（ $\lambda / (n + \lambda)$ ）就越接近 1，而其他所有σ点的权重比例则越接近 0。这一规律与协方差的大小无关。因此，当我们从均值较远的位置进行采样时，这些样本所占据的权重就会变小；而如果我们从非常接近均值的位置进行采样，那么所有样本所占据的权重就会大致相等。

不过，范德梅尔韦的建议是将 α 的值限制在 $0 > \alpha \geq 1$ 的范围内。他认为 10^{-3} 是一个合适的数值。我们就试试看吧。

```
print_sigmas(mean=0, cov=13, alpha=.001, kappa=0)
```

```
sigmas:  [ 0.      0.0036 -0.0036]
mean weights: [-999999.  500000.  500000.]
```

```
cov weights: [-999996.  500000.  500000.]
lambda: -0.999999
sum cov 3.9999989999923855
```

机器人定位——一个完整的实现案例

是时候来处理一些棘手的问题了。大多数书籍都选择那些简单易懂、有标准答案的例子来讲解。这样一来，读者就会不知道该如何应对现实世界中的复杂问题。这个例子虽然无法教会你如何解决各种问题，但可以让你了解在设计和实现滤波器时需要考虑的各种因素。

我们将考虑机器人定位的问题。在这种情况下，有一个机器人正在环境中移动，它通过传感器来检测地标。这可能是一辆使用计算机视觉来识别树木、建筑物和其他地标的自动驾驶汽车。也可能是那种可以打扫你房子的小型机器人，或者是在仓库中工作的机器人。

这款机器人的结构与汽车类似，也有四个轮子。它通过转动前轮来改变方向。这样一来，机器人在前进的同时，也会以后轴为中心进行旋转。这是一种非线性运动方式，我们需要对这种行为进行建模。

该机器人配备了传感器，能够确定其与周围已知目标之间的大致距离和方位。这一过程属于非线性运算，因为要根据距离和方位来确定具体位置，就需要进行开方运算和三角函数计算。

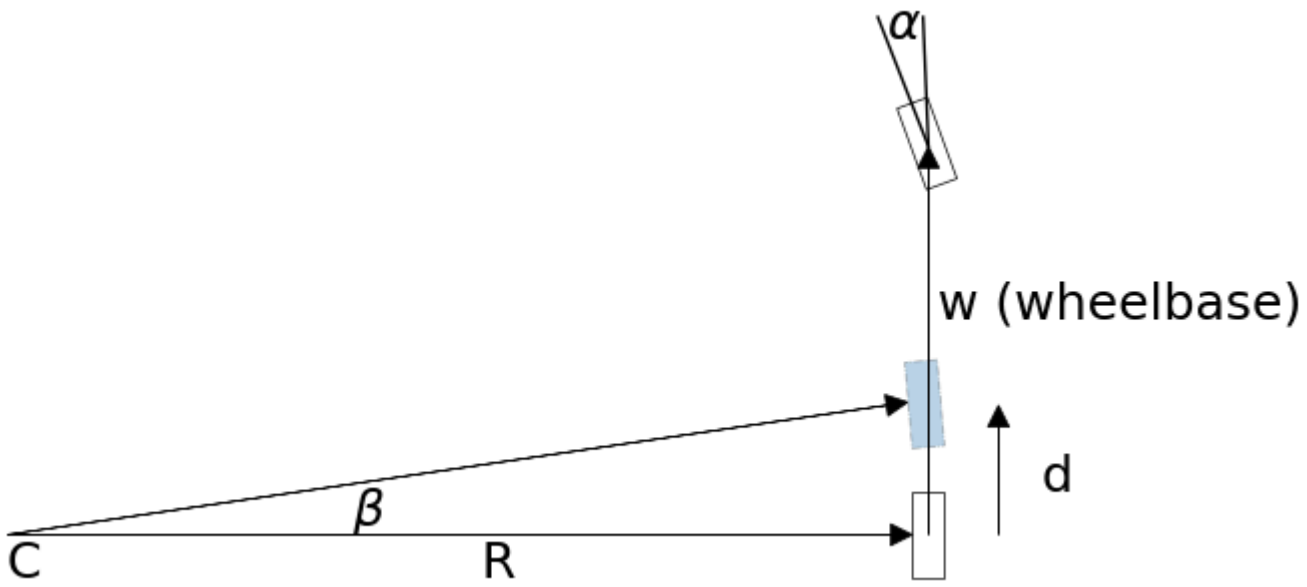
无论是过程模型还是测量模型，都属于非线性模型。UKF 算法能够同时处理这两种非线性情况。因此，我们可以初步得出结论：UKF 算法是解决该问题的合适选择。

机器人运动模型

简单来说，汽车在行驶过程中是通过转动前轮来实现转向的。汽车的前部会朝着车轮所指的方向移动，而整个车身则围绕后轮进行转动。不过，这一简单的机制实际上相当复杂：摩擦力会导致打滑现象；橡胶轮胎在不同速度下的表现也有所不同；此外，外侧轮胎和内侧轮胎的转动半径也必须有所不同。要准确描述汽车的转向过程，就需要用到复杂的微分方程。

在卡尔曼滤波方面，尤其是对于低速运行的机器人应用而言，一种较为简单的自行车模型被证明具有出色的性能。以下是该模型的示意图：

```
ekf_internal.plot_bicycle()
```



我们可以看到，前轮相对于轮距来说，其方向为 α 。在短时间内，汽车会向前移动，后轮则会移动到更靠前的位置，并稍微向内转动，正如蓝色阴影部分所显示的那样。在这么短的时间范围内，我们可以将这一过程视为以半径 R 为圆心进行的转弯。我们可以计算出转弯角度 β 。

$$\beta = \frac{d}{w}\tan(\alpha)$$

而转弯半径 R 则由以下公式给出：

$$R = \frac{d}{\beta}$$

当前进速度为 v 时，后轮行驶的距离为 $d = v\Delta t$ 。

以 θ 表示机器人的当前朝向，我们在转弯开始之前，先计算出位置 C 。

$$\begin{aligned} C_x &= x - R\sin(\theta) \\ C_y &= y + R\cos(\theta) \end{aligned}$$

在时间向前推进 Δt 之后，机器人的新位置和方向如下。

$$\begin{aligned} \bar{x} &= C_x + R\sin(\theta + \beta) \\ \bar{y} &= C_y - R\cos(\theta + \beta) \\ \bar{\theta} &= \theta + \beta \end{aligned}$$

一旦我们将 C 代入其中，就能得到

$$\begin{aligned} \bar{x} &= x - R\sin(\theta) + R\sin(\theta + \beta) \\ \bar{y} &= y + R\cos(\theta) - R\cos(\theta + \beta) \\ \bar{\theta} &= \theta + \beta \end{aligned}$$

如果你对运动模型的原理不感兴趣，那么就不必详细了解这些数学知识。需要记住的是，我们的运动模型是非线性的，因此我们必须利用卡尔曼滤波器来处理这个问题。

设计状态变量

对于我们的机器人来说，我们需要保持其位置和方向的稳定性：

$$x = \begin{bmatrix} x & y & \theta \end{bmatrix}^T$$

我可以将速度因素纳入这个模型中，但正如您所看到的，这样一来，数学计算会变得相当复杂。

控制输入 u 代表所需的行驶速度和转向角度。

$$u = \begin{bmatrix} v & \alpha \end{bmatrix}^T$$

设计系统模型

我们将该系统建模为非线性运动模型加上白噪声。

$$\bar{x} = x + f(x, u) + N(0, Q)$$

利用我们上面所创建的机器人运动模型，我们可以写出如下公式：

```
from math import tan, sin, cos, sqrt

def move(x, dt, u, wheelbase):
    hdg = x[2]
    vel = u[0]
    steering_angle = u[1]
    dist = vel * dt

    if abs(steering_angle) > 0.001: # is robot turning?
        beta = (dist / wheelbase) * tan(steering_angle)
        r = wheelbase / tan(steering_angle) # radius

        sinh, sinhb = sin(hdg), sin(hdg + beta)
        cosh, coshb = cos(hdg), cos(hdg + beta)
        return x + np.array([-r*sinh + r*sinhb,
                             r*cosh - r*coshb, beta])
    else: # moving in straight line
        return x + np.array([dist*cos(hdg), dist*sin(hdg), 0])
```

我们将使用这个函数来实现状态转换功能 `f(x)` 。

我会设计 UKF 算法，以确保 Δt 的数值尽可能小。如果机器人的运动速度较慢，那么该算法应该能够给出相当准确的预测结果。不过，如果 Δt 的数值较大，或者系统的动态特性非常非线性，那么这种算法就无效了。在这种情况下，就需要使用更复杂的数值积分技术，比如龙格-库塔法来处理问题。关于数值积分的更多内容，可以在卡尔曼滤波的数学原理部分找到。

设计测量模型

该传感器能够提供有关地形中多个已知位置的方位和距离信息。测量模型需要将相关数据转换为这些地标点的方位和距离值。如果 p 代表某个地标点的位置，那么距离 r 就是该地标点与传感器之间的距离。

$$r = \sqrt{(p_x - x)^2 + (p_y - y)^2}$$

我们假设，传感器所检测到的方位是相对于机器人的姿态而言的。因此，为了得到传感器的实际读数，必须先从检测到的方位中减去机器人的姿态，具体计算方式如下：

$$\phi = \tan^{-1}(\frac{p_y - y}{p_x - x}) - \theta$$

因此，我们的测量函数为

$$\begin{aligned} z &= h(x, P) &+ N(0, R) \\ &= \begin{bmatrix} \sqrt{(p_x - x)^2 + (p_y - y)^2} \\ \tan^{-1}(\frac{p_y - y}{p_x - x}) - \theta \end{bmatrix} &+ N(0, R) \end{aligned}$$

我暂时不会实施这个方案，因为存在一些需要在下文的“实施部分”中详细讨论的难题。

设计测量噪声

可以合理地假设，距离和方向的测量误差是相互独立的。因此，

$$R = \begin{bmatrix} \sigma^2_{range} & 0 \\ 0 & \sigma^2_{bearing} \end{bmatrix}$$

实施/执行

在开始编码之前，我们还有另一个问题需要处理。残差值为 $y = z - h(x)$ 。假设 z 的方位角为 1° ，而 $h(x)$ 的方位角为 359° 。将两者相减后得到的结果是 -358° 。这一数值会影响到卡尔曼增益的计算结果，因为正确的角度差应该是 2° 。因此，我们必须编写相应的代码来正确计算出方位角的残差值。

```
def normalize_angle(x):
    x = x % (2 * np.pi)    # force in range [0, 2 pi)
    if x > np.pi:          # move to [-pi, pi)
        x -= 2 * np.pi
    return x
```

```
print(np.degrees(normalize_angle(np.radians(1-359))))
```

```
1.9999999999999774
```

状态向量的该参数位于索引 2 的位置，而测量向量的同一参数则位于索引 1 的位置。因此，我们需要分别编写函数来处理这两种情况。另一个需要解决的问题是：当机器人移动时，会有不同的地标变得可见，所以我们必

须能够处理数量不固定的测量结果。用于处理测量误差的函数会接收一个包含多个测量值的数组，每个地标对应一个测量值。

```
def residual_h(a, b):
    y = a - b
    # data in format [dist_1, bearing_1, dist_2, bearing_2,...]
    for i in range(0, len(y), 2):
        y[i + 1] = normalize_angle(y[i + 1])
    return y

def residual_x(a, b):
    y = a - b
    y[2] = normalize_angle(y[2])
    return y
```

我们现在可以开始实施这个测量模型了。其公式如下：

表达式 $\tan^{-1}(\frac{p_y - y}{p_x - x}) - \theta$ 可能产生超出范围 $[-\pi, \pi)$ 的结果，因此我们需要将角度值调整到该范围内。

该函数会接收一个包含各地标信息的数组作为输入，需要输出一个格式为 `[dist_to_1, bearing_to_1, dist_to_2, bearing_to_2, ...]` 的测量结果数组。

```
def Hx(x, landmarks):
    """ takes a state variable and returns the measurement
    that would correspond to that state. """
    hx = []
    for lmark in landmarks:
        px, py = lmark
        dist = sqrt((px - x[0])**2 + (py - x[1])**2)
        angle = atan2(py - x[1], px - x[0])
        hx.extend([dist, normalize_angle(angle - x[2])])
    return np.array(hx)
```

我们的困难还没有结束。无迹变换会计算状态向量和测量向量的平均值，但这两个向量都包含角度信息。对于一组角度来说，并没有唯一的计算平均值的方法。例如，359° 和 3° 的平均值是多少呢？凭直觉，答案应该是 1°，但用简单的方法计算的话，结果却是 181°。

一种常见的方法是计算正弦值与余弦值之和的反正切值。

$$\bar{\theta} = atan2\left(\frac{\sum_{i=1}^{n} \sin\theta_i}{n}, \frac{\sum_{i=1}^{n} \cos\theta_i}{n}\right)$$

`UnscentedKalmanFilter.__init__()` 为用于计算状态均值的函数提供了参数 `x_mean_fn`，同时为用于计算测量值均值的函数提供了参数 `z_mean_fn`。我们将把这些函数分别编码为：

```
def state_mean(sigmas, Wm):
    x = np.zeros(3)

    sum_sin = np.sum(np.dot(np.sin(sigmas[:, 2]), Wm))
    sum_cos = np.sum(np.dot(np.cos(sigmas[:, 2]), Wm))
    x[0] = np.sum(np.dot(sigmas[:, 0], Wm))
    x[1] = np.sum(np.dot(sigmas[:, 1], Wm))
    x[2] = atan2(sum_sin, sum_cos)
    return x
```

```
def z_mean(sigmas, Wm):
    z_count = sigmas.shape[1]
    x = np.zeros(z_count)

    for z in range(0, z_count, 2):
        sum_sin = np.sum(np.dot(np.sin(sigmas[:, z+1]), Wm))
        sum_cos = np.sum(np.dot(np.cos(sigmas[:, z+1]), Wm))

        x[z] = np.sum(np.dot(sigmas[:,z], Wm))
        x[z+1] = atan2(sum_sin, sum_cos)
    return x
```

这些函数充分利用了 NumPy 中的三角函数能够对数组进行操作这一特点，同时 `dot` 还能实现元素级的乘法运算。由于 NumPy 是用 C 和 Fortran 语言实现的，因此 `sum(dot(sin(x), w))` 的运行速度远远快于用 Python 编写相同功能的代码。

现在，我们已经准备好实施 UKF 了。需要指出的是，在设计这个滤波器时，我并没有一次性完成所有功能的开发。我先创建了一个基本的 UKF 模型，包含了一些预定义的标志点，确认其有效性后，才开始逐步完善其余部分。“如果看到不同的标志点怎么办？”这个想法让我改变了测量函数的设计，使其能够接受多个标志点的输入。“如何计算角度的残差呢？”这个问题促使我编写了角度归一化的代码。“一组角度的平均值是多少？”我在网上搜索后，找到了一篇关于这个主题的维基百科文章，并实现了相应的算法。不要气馁。尽可能多地设计功能，然后逐一解决问题。

您已经看过 UKF 的实现方式了，所以我就不详细介绍了。这里有两个新的要点需要说明。在构建西格玛点和滤波器时，我们必须提供自己编写的用于计算残差和均值的函数。

```
points = SigmaPoints(n=3, alpha=.00001, beta=2, kappa=0,
                     subtract=residual_x)

ukf = UKF(dim_x=3, dim_z=2, fx=fx, hx=Hx, dt=dt, points=points,
          x_mean_fn=state_mean, z_mean_fn=z_mean,
          residual_x=residual_x, residual_z=residual_h)
```

接下来，我们需要将额外的数据传递给 `f(x, dt)` 和 `h(x)` 函数。我们希望使用 `move(x, dt, u, wheelbase)` 来处理 `f(x, dt)`，用 `Hx(x, landmarks)` 来处理 `h(x)`。要做到这一点，只需将额外的参数作为关键字参数传递给 `predict()` 和 `update()` 即可：

```
            ukf.predict(u=u, wheelbase=wheelbase)
            ukf.update(z, landmarks=landmarks)
```

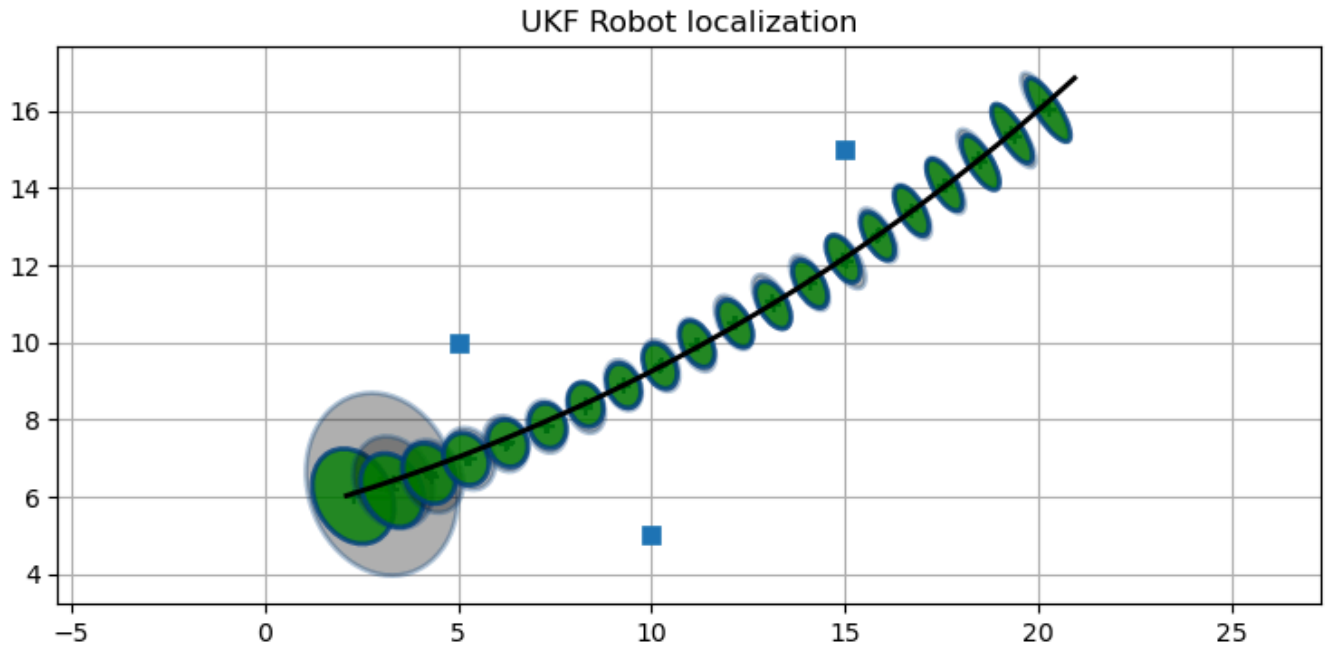
其余的代码负责执行模拟运算并绘制结果。我创建了一个名为 `landmarks` 的变量，用于存储各个参考点的坐标。模拟中，机器人的位置每秒更新 10 次，但 UKF 算法则每秒只计算一次。由于我们没有使用龙格-库塔法来求解运动方程，因此较小的时间步长有助于提高模拟的准确性。

```
from filterpy.stats import plot_covariance_ellipse

dt = 1.0
wheelbase = 0.5

def run_localization(
```

```
    cmds, landmarks, sigma_vel, sigma_steer, sigma_range,
    sigma_bearing, ellipse_step=1, step=10):

    plt.figure()
    points = MerweScaledSigmaPoints(n=3, alpha=.00001, beta=2, kappa=0,
                                    subtract=residual_x)
    ukf = UKF(dim_x=3, dim_z=2*len(landmarks), fx=move, hx=Hx,
              dt=dt, points=points, x_mean_fn=state_mean,
              z_mean_fn=z_mean, residual_x=residual_x,
              residual_z=residual_h)

    ukf.x = np.array([2, 6, .3])
    ukf.P = np.diag([.1, .1, .05])
    ukf.R = np.diag([sigma_range**2,
                     sigma_bearing**2]*len(landmarks))
    ukf.Q = np.eye(3)*0.0001

    sim_pos = ukf.x.copy()

    # plot landmarks
    if len(landmarks) > 0:
        plt.scatter(landmarks[:, 0], landmarks[:, 1],
                    marker='s', s=60)

    track = []
    for i, u in enumerate(cmds):
        sim_pos = move(sim_pos, dt/step, u, wheelbase)
        track.append(sim_pos)

        if i % step == 0:
            ukf.predict(u=u, wheelbase=wheelbase)

            if i % ellipse_step == 0:
                plot_covariance_ellipse(
                    (ukf.x[0], ukf.x[1]), ukf.P[0:2, 0:2], std=6,
                     facecolor='k', alpha=0.3)

            x, y = sim_pos[0], sim_pos[1]
            z = []
            for lmark in landmarks:
                dx, dy = lmark[0] - x, lmark[1] - y
                d = sqrt(dx**2 + dy**2) + randn()*sigma_range
                bearing = atan2(lmark[1] - y, lmark[0] - x)
                a = (normalize_angle(bearing - sim_pos[2] +
                     randn()*sigma_bearing))
                z.extend([d, a])
            ukf.update(z, landmarks=landmarks)

            if i % ellipse_step == 0:
                plot_covariance_ellipse(
                    (ukf.x[0], ukf.x[1]), ukf.P[0:2, 0:2], std=6,
                     facecolor='g', alpha=0.8)
    track = np.array(track)
    plt.plot(track[:, 0], track[:,1], color='k', lw=2)
    plt.axis('equal')
    plt.title("UKF Robot localization")
    plt.show()
    return ukf
```

```
landmarks = np.array([[5, 10], [10, 5], [15, 15]])
cmds = [np.array([1.1, .01])] * 200
ukf = run_localization(
```

```
    cmds, landmarks, sigma_vel=0.1, sigma_steer=np.radians(1),
    sigma_range=0.3, sigma_bearing=0.1)
print('Final P:', ukf.P.diagonal())
```




```
Final P: [0.0092 0.0187 0.0007]
```

其余的代码负责执行模拟运算并绘制结果。我创建了一个名为 `landmarks` 的变量，用于存储各个参考点的坐标。模拟过程中，机器人的位置每秒更新 10 次，但 UKF 滤波器则只运行一次。这样做有两个原因：首先，我们没有使用龙格-库塔法来求解运动方程，因此较小的时间步长有助于提高模拟的准确性。其次，在嵌入式系统中，处理速度往往有限。因此，必须仅在确实必要时才运行卡尔曼滤波器。

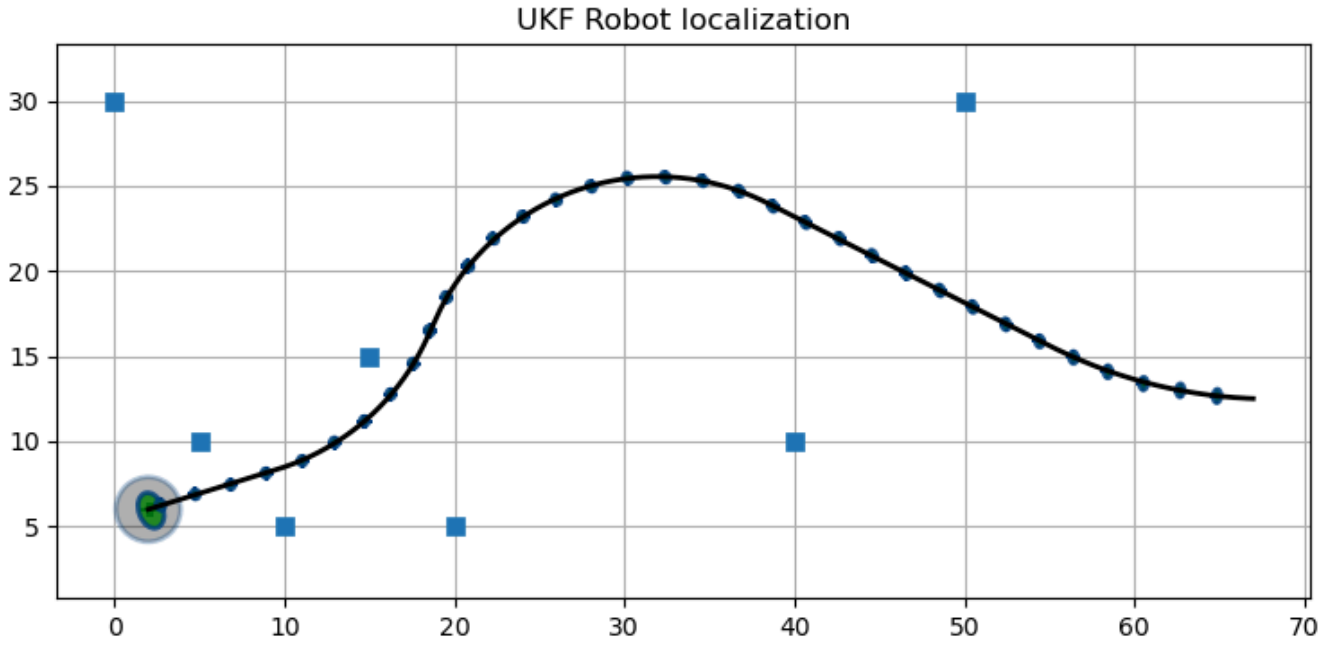
操控机器人

在上文的模拟中，车辆的转向操作并不真实。因为车辆的速度和转向角度始终没有变化，这对卡尔曼滤波器来说并不是什么大问题。我们本可以实现更复杂的 PID 控制算法来模拟车辆的转向行为，但我决定直接使用 NumPy 的 `linspace` 方法来生成不同的转向指令。此外，由于这次车辆的行驶距离会比第一个例子中的要长得多，我还会添加更多的参考点。

```
landmarks = np.array([[5, 10], [10, 5], [15, 15], [20, 5],
                      [0, 30], [50, 30], [40, 10]])
dt = 0.1
wheelbase = 0.5
sigma_range=0.3
sigma_bearing=0.1

def turn(v, t0, t1, steps):
  return [[v, a] for a in np.linspace(
                np.radians(t0), np.radians(t1), steps)]

# accelerate from a stop
cmds = [[v, .0] for v in np.linspace(0.001, 1.1, 30)]
cmds.extend([cmds[-1]]*50)
```

```
# turn left
v = cmds[-1][0]
cmds.extend(turn(v, 0, 2, 15))
cmds.extend([cmds[-1]]*100)

#turn right
cmds.extend(turn(v, 2, -2, 15))
cmds.extend([cmds[-1]]*200)

cmds.extend(turn(v, -2, 0, 15))
cmds.extend([cmds[-1]]*150)

cmds.extend(turn(v, 0, 1, 25))
cmds.extend([cmds[-1]]*100)
```

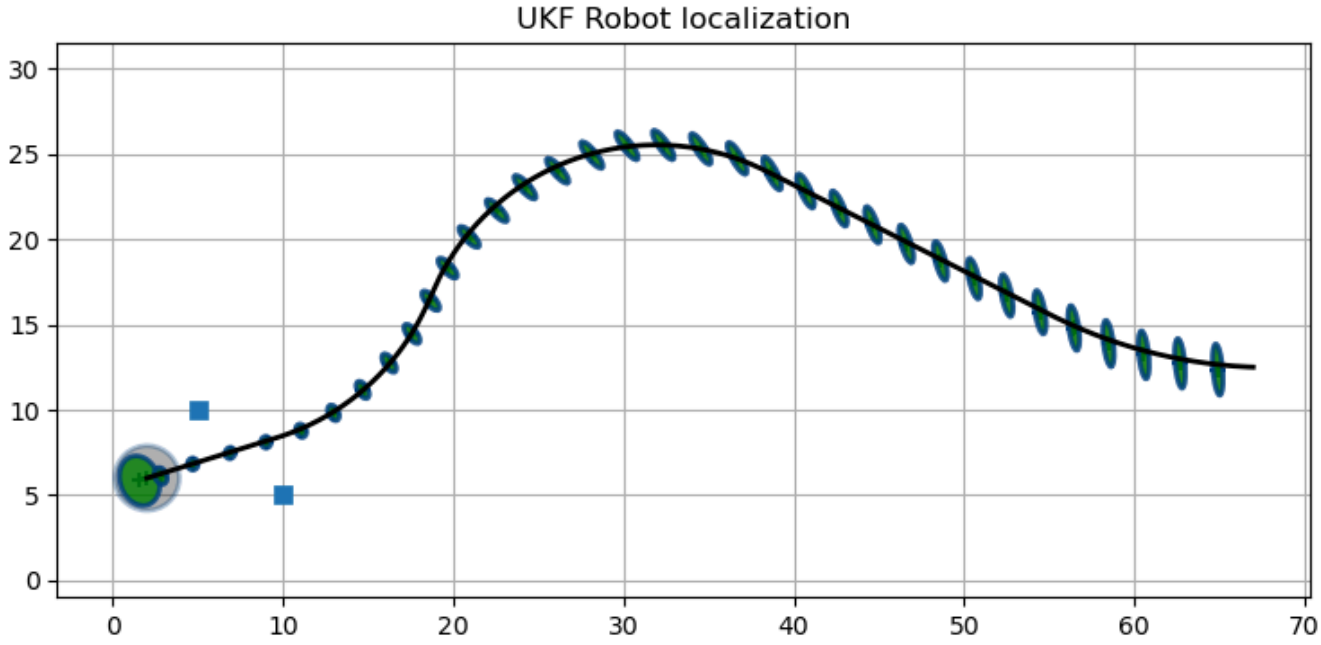
```
ukf = run_localization(
    cmds, landmarks, sigma_vel=0.1, sigma_steer=np.radians(1),
    sigma_range=0.3, sigma_bearing=0.1, step=1,
    ellipse_step=20)
print('final covariance', ukf.P.diagonal())
```




```
final covariance [0.0013 0.0043 0.0004]
```

这种不确定性会很快变得非常小。协方差椭圆虽然能够反映 6σ 的协方差情况，但由于椭圆太小，几乎难以被察觉。我们可以通过在起点附近仅设置两个参考点来增加计算结果的误差。当机器人偏离这些参考点越远时，计算结果中的误差就会越大。

```
ukf = run_localization(
    cmds, landmarks[0:2], sigma_vel=0.1, sigma_steer=np.radians(1),
    sigma_range=0.3, sigma_bearing=0.1, step=1,
    ellipse_step=20)
print('final covariance', ukf.P.diagonal())
```




```
final covariance [0.0026 0.0657 0.0008]
```

讨论

你对这一章的印象，很可能取决于你过去曾经实现过多少个非线性卡尔曼滤波器。如果你是第一次接触这类算法，那么计算 $2n+1$ 个σ点以及后续的 $f(x)$ 和 $h(x)$ 函数的实现过程可能会让你觉得有些繁琐。的确，由于需要处理与角度相关的数学运算，我花费的时间比预期的要多得多。不过，如果你曾经实现过扩展卡尔曼滤波器（EKF），那你可能会觉得这部分内容相当简单。虽然实现 UKF 时需要编写一些繁琐的代码，但其基本概念其实很简单。而针对同一问题，EKF 则涉及到相当复杂的数学运算。在很多情况下，我们无法直接求解 EKF 方程，只能采用迭代法来求解。

UKF 相比 EKF 的优势不仅在于其实现起来相对简单。不过现在讨论这一点还为时过早，因为你还没有了解 EKF 的原理。EKF 是通过将问题线性化处理后，再利用线性卡尔曼滤波器来处理数据的。而 UKF 则需要处理 $2n+1$ 个样本。因此，UKF 的精度通常高于 EKF，尤其是在问题具有高度非线性时。虽然不能保证 UKF 在所有情况下都比 EKF 更优，但实际上，UKF 的性能至少与 EKF 相当，通常还要好得多。

因此，我的建议是：始终先从使用 UKF 算法开始。如果滤波器的误差会导致严重的后果——比如人员伤亡、巨大经济损失、发电厂爆炸等——那么你当然需要进行复杂的分析和实验，以选择最合适的滤波器。不过，这些内容超出了本书的讨论范围，你最好去读研究生才能深入学习这些理论。

最后，我提到过使用 UKF 来实现西格玛点滤波器。但实际上并非如此。我所选择的其实是 Julier 提出的缩放无迹滤波器，这一模型的具体参数化方式是由 Van der Merwe 在他的 2004 年博士论文中提出的。如果你搜索 Julier、Van der Merwe、Uhlmann 和 Wan 的相关研究，你会发现他们还提出了许多类似的西格玛点滤波器。每种方法在确定西格玛点的选取和权重时都有所不同。不过，选择的方式还不止这些。例如，SVD 卡尔曼滤波器则利用奇异值分解来计算概率分布的近似均值和协方差。可以把这一章看作是对西格玛点滤波器的简单介绍，而非对其工作原理的详尽阐述。

参考文献

- [1] 鲁道夫·范德梅尔韦。《用于动态状态空间模型中概率推断的西格玛点卡尔曼滤波器》，博士论文（2004 年）。
- [2] 西蒙·J·朱利尔。“尺度不变的无迹变换”。《美国控制会议论文集》第 6 卷。IEEE 出版社。（2002 年）
- [3] http://www.esdradar.com/brochures/Compact%20Tracking%2037250X.pdf
- [4] Julier, Simon J.; Uhlmann, Jeffrey. “卡尔曼滤波在非线性系统中的新应用”。SPIE 会议论文集 3068，《信号处理、传感器融合与目标识别 VI》，第 182 页（1997 年 7 月 28 日）
- [5] 乔列斯基分解。维基百科。http://en.wikipedia.org/wiki/Cholesky_decomposition

第 11 章：扩展卡尔曼滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

我们首先建立了线性卡尔曼滤波器的理论。在接下来的两章中，我们探讨了将卡尔曼滤波器应用于非线性问题的方法。在这一章中，我们将学习扩展卡尔曼滤波器。扩展卡尔曼滤波器通过在当前估计点对系统进行线性化处理，然后再利用线性卡尔曼滤波器来处理这个线性化后的系统。这是一种最早被用于解决非线性问题的技术，至今仍是最常用的方法之一。

EKF 为滤波器的设计者带来了巨大的数学挑战；这是本书中最具有挑战性的章节。我尽一切努力避免使用 EKF，而倾向于采用其他用于解决非线性问题的技术。然而，这个主题却是不可避免的；所有经典的论文以及该领域当前的大部分研究都使用了 EKF。即使你在自己的工作中不使用 EKF，你也必须熟悉这个主题，才能理解相关文献。

线性化卡尔曼滤波器

卡尔曼滤波器使用线性方程来进行计算，因此无法处理非线性问题。非线性问题可能源于两个方面：首先，系统模型本身可能是非线性的。例如，物体在大气中下落时，会受到空气阻力的影响，从而降低其加速度。而空气阻力系数又取决于物体的运动速度。这种复杂的动态过程无法用线性方程来描述。其次，测量结果本身也可能具有非线性特征。例如，雷达可以提供目标的距离和方位信息。我们需要利用三角函数来计算目标的位置，而三角函数本身就是非线性的函数。

对于线性滤波器而言，其过程模型和测量模型对应的方程如下：

$$\begin{aligned}\dot{x} &= Ax + w_x \\ z &= Hx + w_z\end{aligned}$$

其中， A 表示系统的动态矩阵。利用卡尔曼滤波数学章节中介绍的状态空间方法，这些方程可以被转换成……其中， 代表系统的动态矩阵。利用卡尔曼滤波数学章节中介绍的态空间方法，这些方程可以被转化为 的形式。

其中， 为基准矩阵。噪声项 和 则被分别纳入矩阵 和 中。通过这种方程形式，我们就可以根据步骤 的测量结果以及步骤 的状态估计值，来计算步骤 的状态。在之前的章节中，我通过使用可以用牛顿方程来描述的例子，帮助大家理解相关概念，并尽量简化数学计算过程。根据高中物理知识，我们完全可以知道该如何设计 。

对于非线性模型，线性表达式 被一个非线性函数 替代，而线性表达式 则被另一个非线性函数 替代。

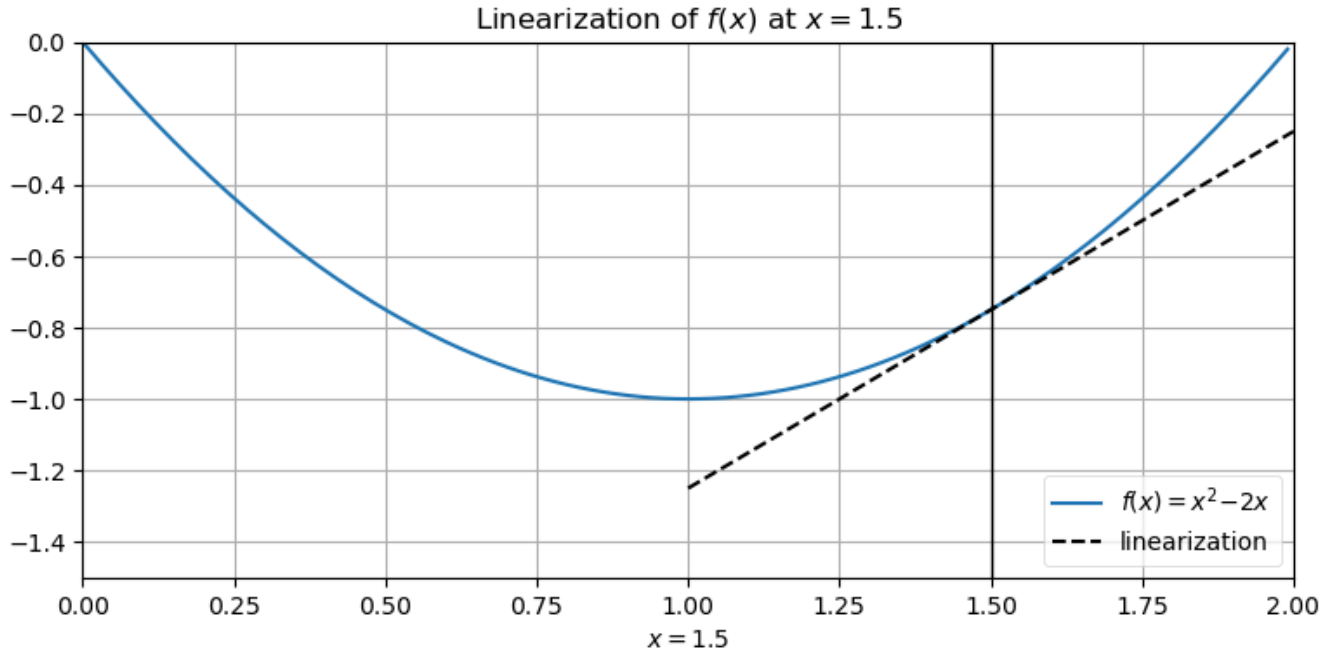
$$\begin{aligned}\dot{x} &= f(x, u) + w_x \\ z &= h(x) + w_z\end{aligned}$$

你可以想象，我们可能会尝试找到一组新的卡尔曼滤波方程，以最优地解决这些问题。但如果你还记得“非线性滤波”章节中的图表，就会明白：当将一个高斯分布通过非线性函数处理后，得到的概率分布就不再具有高斯特性了。因此，这种方法是行不通的。

EKF 并没有改变卡尔曼滤波器的线性方程。相反，它是在当前估计值处将非线性方程线性化，并将这种线性化应用于线性卡尔曼滤波器中。

线性化处理的意思就是找到一条直线与曲线在某个点最为吻合的直线。下面的图表展示了如何对抛物线 在 点进行线性化处理。

```
import kf_book.ekf_internal as ekf_internal
ekf_internal.show_linearization()
```



如果上面的曲线代表了该过程模型，那么虚线则代表了该曲线在 $x = 1.5$ 这一点的线性化表达。

我们通过对系统求导来使其线性化，求导的过程实际上就是确定曲线的斜率。

$$\begin{aligned} f(x) &= x^2 - 2x \\ \frac{df}{dx} &= 2x - 2 \end{aligned}$$

然后在 x 处对其进行评估：

$$\begin{aligned} m &= f'(x = 1.5) \\ &= 2(1.5) - 2 \\ &= 1 \end{aligned}$$

对微分方程系统进行线性化处理是类似的。我们通过计算各个点的偏导数来线性化处理 $f(x, u)$ 和 $h(x)$ ，从而求出在 x_t 和 u_t 点时的 F 和 H 。我们将矩阵的偏导数称为雅可比矩阵。这样我们就得到了离散状态转移矩阵和测量模型矩阵：

$$
\begin{aligned}
F &= \frac{\partial f(x_t, u_t)}{\partial x}\Big|_{x_t, u_t} \\
H &= \frac{\partial h(\bar{x}_t)}{\partial \bar{x}}\Big|_{\bar{x}_t}
\end{aligned}
$$

这就得到了 EKF 的相关方程式。我把那些与线性滤波器不同的部分用方框标了出来：

linear Kalman filter	EKF
	$F = \frac{\partial f(x_t, u_t)}{\partial x}\Big\|_{x_t, u_t}$
$\bar{x} = Fx + Bu$	$\bar{x} = f(x, u)$
$\bar{P} = FPF^T + Q$	$\bar{P} = FPF^T + Q$
	$H = \frac{\partial h(\bar{x}_t)}{\partial \bar{x}}\Big\|_{\bar{x}_t}$
$\mathrm{y} = z - H\bar{x}$	$\mathrm{y} = z - h(\bar{x})$
$K = \bar{P}H^T(H\bar{P}H^T + R)^{-1}$	$K = \bar{P}H^T(H\bar{P}H^T + R)^{-1}$
$x = \bar{x} + K\mathrm{y}$	$x = \bar{x} + K\mathrm{y}$
$P = (I - KH)\bar{P}$	$P = (I - KH)\bar{P}$

我们通常不使用 Fx 来传递 EKF 的状态信息，因为线性化处理会导致误差。通常，人们会使用欧拉法或龙格-库塔法等数值积分方法来计算 $\bar{x}$ 。因此，我使用了 $\bar{x} = f(x, u)$ 来进行计算。出于同样的原因，我们在计算残差时也不使用 $H\bar{x}$ ，而是选择更精确的 $h(\bar{x})$ 。

我认为，理解 EKF 最简单的方法就是先通过一个例子来了解。之后，你可能还想再回来阅读这一部分内容。

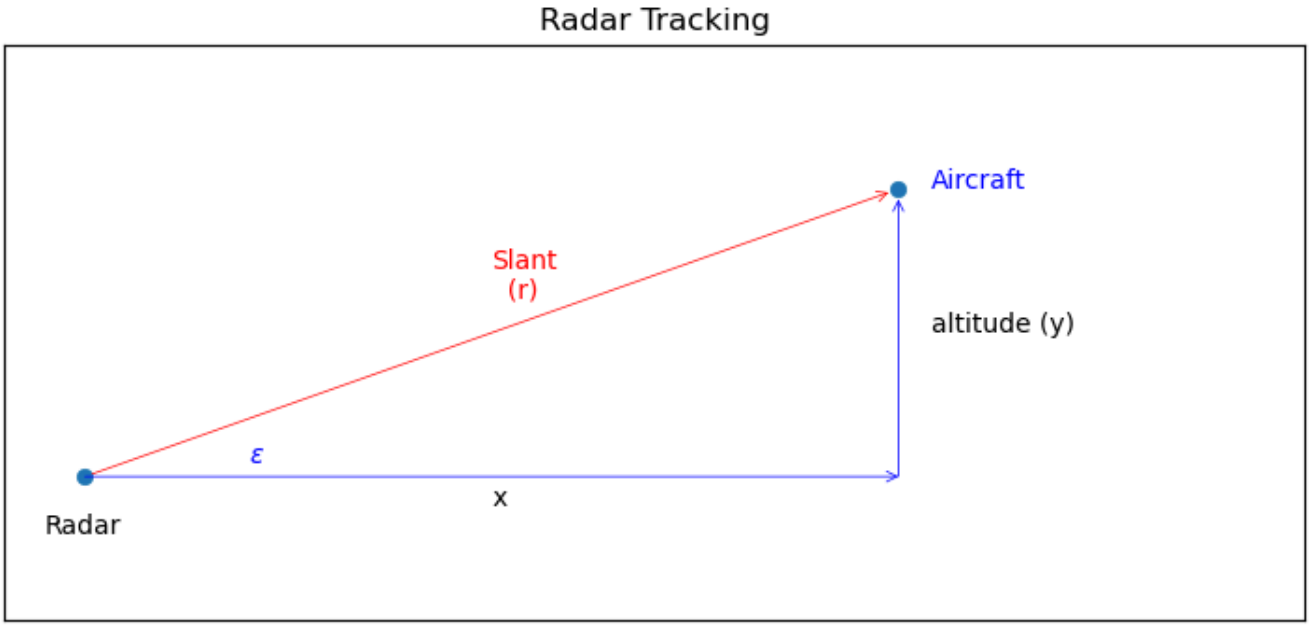
示例：追踪一架飞机

这个例子使用了基于地面的雷达来跟踪一架飞机。在上一章中，我们已经为这个问题实现了一个 UKF 滤波器。现在，我们将为同一个问题实现一个 EKF 滤波器，这样就能比较两种滤波器的性能以及实现它们所需的努力程度了。

雷达的工作原理是发射一束无线电波，然后扫描这些波的反射信号。任何位于该波路径上的物体都会将部分信号反射回雷达。通过测量反射信号返回雷达所需的时间，系统就可以计算出物体与雷达站之间的直线距离，也就是物体的实际距离。

下图展示了雷达的斜距 r 、仰角 ϵ 与飞机的水平位置 x 及高度 y 之间的关系：

```
ekf_internal.show_radar_chart()
```



这就得到了以下等式：

$$\begin{aligned}\epsilon &= \tan^{-1}\frac{y}{x} \\ r^2 &= x^2 + y^2\end{aligned}$$

设计状态变量

我们希望追踪在恒定速度和恒定高度下飞机的位置，同时还需要测量到该飞机的斜距。这意味着我们需要三个状态变量：水平距离、水平速度和高度。

$$x = \begin{bmatrix} distance \\ velocity \\ altitude \end{bmatrix} = \begin{bmatrix} x \\ \dot{x} \\ y \end{bmatrix}$$

设计流程模型

我们假设飞机的运动遵循牛顿运动定律。在之前的章节中，我们已经使用过这个模型。仔细观察的话，你应该能明白我们的意图是什么。

$$F = \begin{bmatrix} 1 & \Delta t & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

我将该矩阵划分成了几个区块。其中，左上角的区块对应的是 x 的恒速运动模型；右下角的区块则对应的是 y 的恒定位置模型。

不过，让我们来练习如何找到这些矩阵吧。我们用一组微分方程来描述各种系统。我们需要的是形如这样的方程：

其中， w 代表系统噪声。

变量 x 和 y 是相互独立的，因此我们可以分别对它们进行计算。一维运动中的微分方程如下：

$$
\begin{aligned}
v &= \dot{x} \\
a &= \ddot{x} = 0
\end{aligned}
$$

现在，我们将这些微分方程转化为状态空间形式。如果这是一个二阶或更高阶的微分系统，那么我们必须先将其转化为等价的一阶方程组。由于这些方程都是一阶的，因此我们可以直接将它们表示为状态空间矩阵形式。

在哪儿。

请记住， A 代表系统动态矩阵。它描述了一组线性微分方程。我们需要利用该矩阵来计算状态转移矩阵 F。 F 则描述了一组离散的线性方程，这些方程用于计算在离散时间步长 Δt 时的状态 x 。

计算 F 的常用方法是利用矩阵指数的幂级数展开式：

$$
F(\Delta t) = e^{A\Delta t} = I + A\Delta t + \frac{(A\Delta t)^2}{2!} + \frac{(A\Delta t)^3}{3!} + \ldots
$$

$A^2 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$ ，因此 A 的所有高次幂也都是 0 。所以，其幂级数展开形式为：

$$
\begin{aligned}
F &= I + At + 0 \\
&= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \Delta t \\
F &= \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}
\end{aligned}
$$

这与运动学方程得出的结果是一样的！除了用来演示如何从线性微分方程中求出状态转移矩阵之外，这个练习其实没有必要。在本章的最后，我们将通过一个例子来进一步应用这一技巧。

设计测量模型

该测量函数以先前的状态估计值 $\bar{x}$ 为输入，将其转换为斜距的测量值。我们利用勾股定理来推导这一关系：

$$
h(\bar{x}) = \sqrt{x^2 + y^2}
$$

由于涉及到平方根，斜距与地面上的位置之间的关系是非线性的。我们通过在某一点处计算其偏导数来将其线性化：

$$
H = \frac{\partial h(\bar{x})}{\partial \bar{x}} \Big|_{\bar{x}_t}
$$

矩阵的偏导数被称为雅可比矩阵，其形式如下：

$$
\frac{\partial H}{\partial \bar{x}} = \begin{bmatrix}
\frac{\partial h_1}{\partial x_1} & \frac{\partial h_1}{\partial x_2} & \cdots \\
\frac{\partial h_2}{\partial x_1} & \frac{\partial h_2}{\partial x_2} & \cdots \\
\vdots & \vdots &
\end{bmatrix}
$$

换句话说，矩阵中的每个元素都是函数 h 相对于 x 各变量的偏导数。对于我们的问题来说，情况正是如此。

$$
H = \begin{bmatrix} \partial h / \partial x & \partial h / \partial \dot{x} & \partial h / \partial y \end{bmatrix}
$$

依次解决每一个问题：

$$\begin{aligned}\frac{\partial h}{\partial x} &= \frac{\partial}{\partial x}\sqrt{x^2+y^2} \\ &= \frac{x}{\sqrt{x^2+y^2}}\end{aligned}$$

以及

$$\begin{aligned}\frac{\partial h}{\partial \dot{x}} &= \frac{\partial}{\partial \dot{x}}\sqrt{x^2+y^2} \\ &= 0\end{aligned}$$

还有

$$\begin{aligned}\frac{\partial h}{\partial y} &= \frac{\partial}{\partial y}\sqrt{x^2+y^2} \\ &= \frac{y}{\sqrt{x^2+y^2}}\end{aligned}$$

把它们给我们吧。

$$H = \begin{bmatrix} \frac{x}{\sqrt{x^2+y^2}} & 0 & \frac{y}{\sqrt{x^2+y^2}} \end{bmatrix}$$

这看起来可能有些令人生畏，但请先退一步思考：其实，所有这些数学运算都在处理一些非常简单的问题。我们有一个描述飞机斜距的方程，不过这个方程是非线性的。而卡尔曼滤波器只能处理线性方程，因此我们需要找到一个能近似表示该非线性方程的线性方程。正如我们上面所讨论的，求出非线性方程在某一点的斜率，就可以作为很好的近似值。对于卡尔曼滤波器来说，这个“某一点”就是状态变量 x，因此我们需要计算斜距关于 x 的导数。在线性卡尔曼滤波器中， H 是一个常数，我们可以在运行滤波器之前先计算出来。而在扩展卡尔曼滤波器中， H 的值会在每个时间步中随着评估点 $\bar{x}$ 的变化而更新。

为了更具体地说明这一点，现在让我们编写一个 Python 函数，用来计算该问题中 h 的雅可比矩阵。

```
from math import sqrt
def HJacobian_at(x):
    """ compute Jacobian of H matrix at x """

    horiz_dist = x[0]
    altitude   = x[2]
    denom = sqrt(horiz_dist**2 + altitude**2)
    return array ([[horiz_dist/denom, 0., altitude/denom]])
```

最后，让我们来看看 $h(\bar{x})$ 的代码实现：

```
def hx(x):
    """ compute measurement for slant range that
    would correspond to state x.
    """

    return (x[0]**2 + x[2]**2) ** 0.5
```

现在，让我们来编写一个用于我们的雷达的模拟程序吧。

```
from numpy.random import randn
import math

class RadarSim:
    """ Simulates the radar signal returns from an object
    flying at a constant altityude and velocity in 1D.
    """

    def __init__(self, dt, pos, vel, alt):
        self.pos = pos
        self.vel = vel
        self.alt = alt
        self.dt = dt

    def get_range(self):
        """ Returns slant range to the object. Call once
        for each new measurement at dt time from last call.
        """

        # add some process noise to the system
        self.vel = self.vel  + .1*randn()
        self.alt = self.alt + .1*randn()
        self.pos = self.pos + self.vel*self.dt

        # add measurement noise
        err = self.pos * 0.05*randn()
        slant_dist = math.sqrt(self.pos**2 + self.alt**2)

        return slant_dist + err
```

设计过程与测量噪声

雷达可以测量到目标的距离。我们用 $\sigma_{range} = 5$ 米来表示这种误差或噪声。这样我们就能得到……

$$R = \begin{bmatrix} \sigma^2_{range} \end{bmatrix} = \begin{bmatrix} 25 \end{bmatrix}$$

Q 的设计需要进一步讨论。状态 $x = \begin{bmatrix} x & \dot{x} & y \end{bmatrix}^T$ 中，前两个元素分别是位置（下降距离）和速度。因此，我们可以利用 `Q_discrete_white_noise` 噪声来计算 Q 左上角的数值。 x 的第三个元素是高度，我们假设高度与下降距离无关。基于这些假设， Q 的设计结构如下：

$$Q = \begin{bmatrix} Q_x & 0 \\ 0 & Q_y \end{bmatrix}$$

实施/执行

`FilterPy` 提供了 `ExtendedKalmanFilter` 这个类。它的用法与我们之前使用的 `KalmanFilter` 类类似，不同之处在于： `FilterPy` 允许用户指定一个函数来计算 H 的雅可比矩阵，同时还能指定 $h(x)$ 这个函数。

我们首先导入该滤波器并对其进行创建。 `x` 的维度为 3，而 `z` 的维度为 1。

```
from filterpy.kalman import ExtendedKalmanFilter

rk = ExtendedKalmanFilter(dim_x=3, dim_z=1)
```

我们开发了这款雷达模拟器：

```
radar = RadarSim(dt, pos=0., vel=100., alt=1000.)
```

我们将在飞机实际所在位置附近初始化该滤波器：

```
rk.x = array([radar.pos, radar.vel-10, radar.alt+100])
```

我们使用上面计算得到的泰勒级数展开式的首项来来确定系统矩阵：

```
dt = 0.05
rk.F = eye(3) + array([[0, 1, 0],
                       [0, 0, 0],
                       [0, 0, 0]])*dt
```

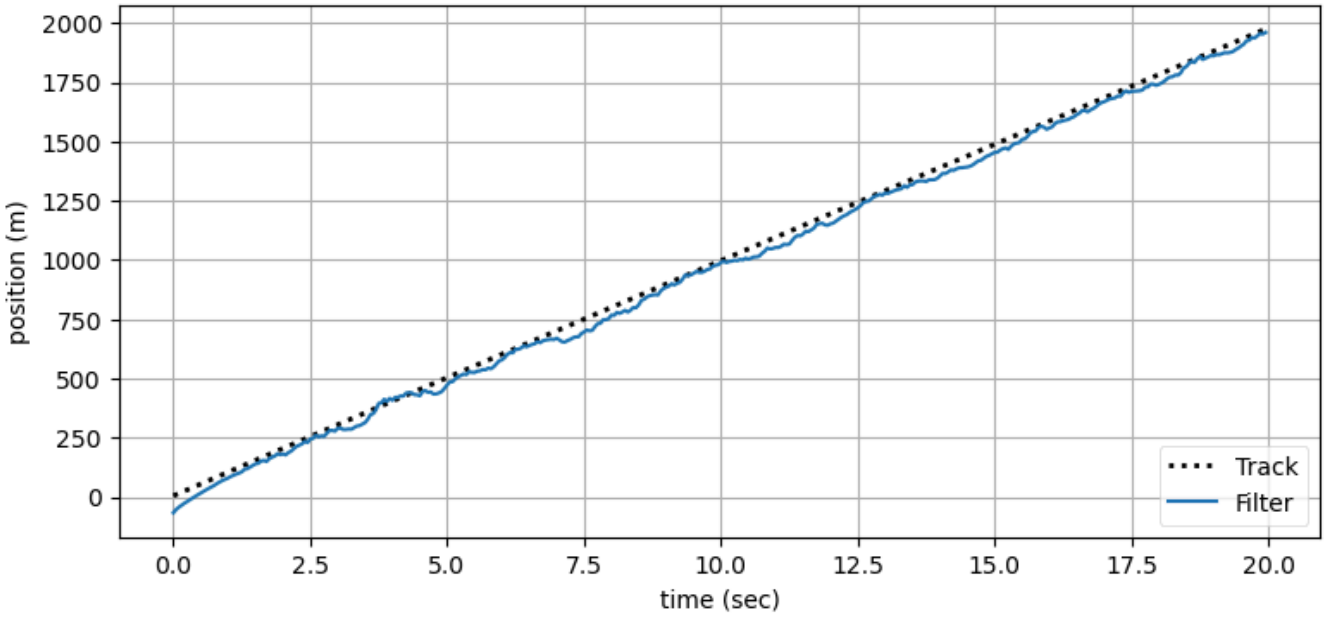
在为 R 、 Q 和 P 赋予合理的数值之后，我们就可以通过简单的循环来运行该滤波器了。我们将用于计算 H 和 $h(x)$ 的雅可比矩阵的函数，传递给 `update` 方法进行处理。

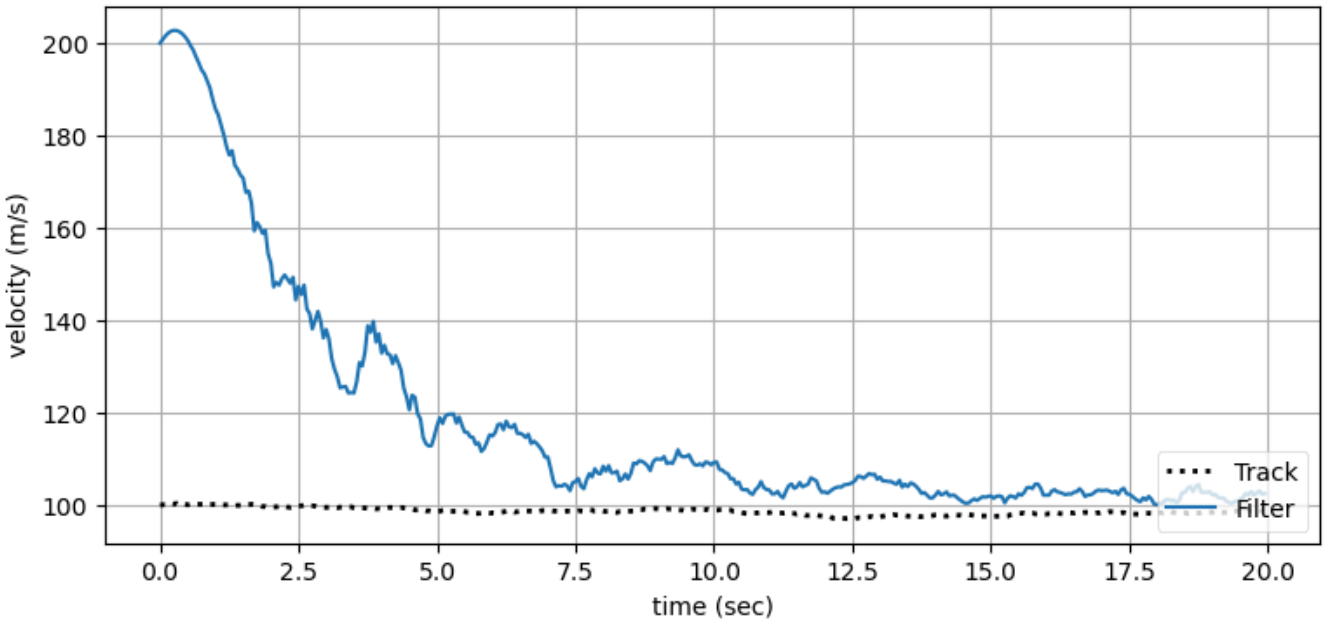
```
for i in range(int(20/dt)):
    z = radar.get_range()
    rk.update(array([z]), HJacobian_at, hx)
    rk.predict()
```

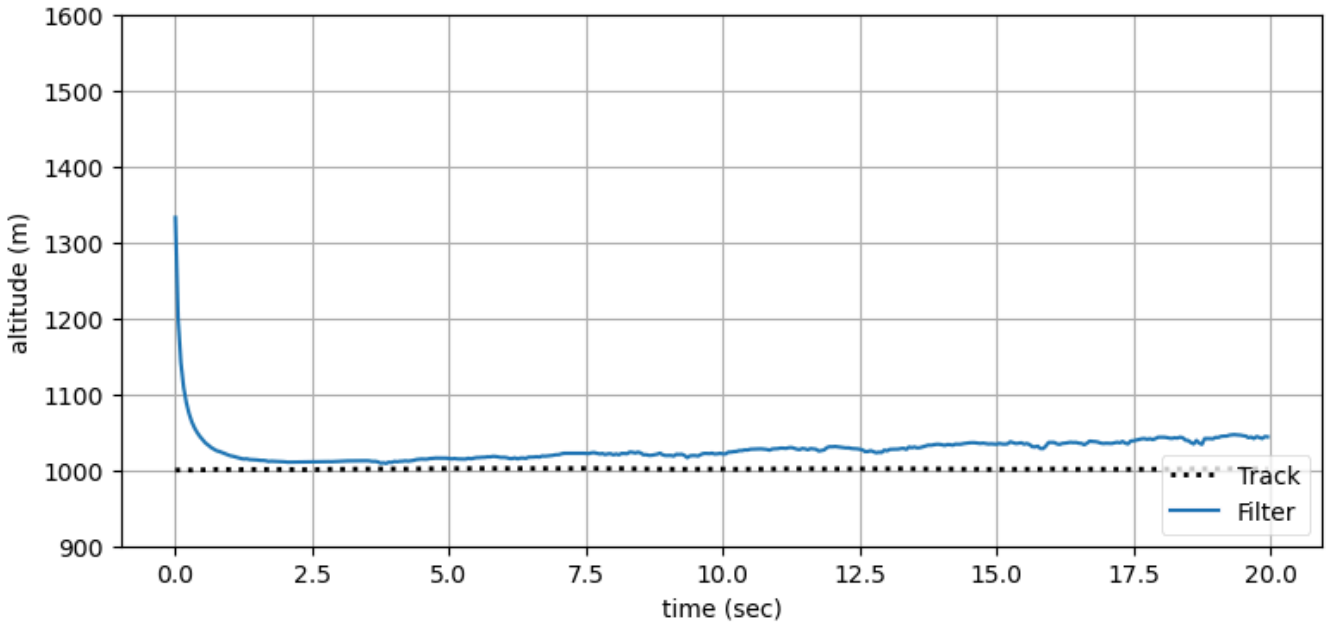
添加一些用于保存和绘制所得结果的样板代码：

```
from filterpy.common import Q_discrete_white_noise
from filterpy.kalman import ExtendedKalmanFilter
from numpy import eye, array, asarray
import numpy as np

dt = 0.05
rk = ExtendedKalmanFilter(dim_x=3, dim_z=1)
radar = RadarSim(dt, pos=0., vel=100., alt=1000.)

# make an imperfect starting guess
rk.x = array([radar.pos-100, radar.vel+100, radar.alt+1000])

rk.F = eye(3) + array([[0, 1, 0],
                       [0, 0, 0],
                       [0, 0, 0]]) * dt

range_std = 5. # meters
rk.R = np.diag([range_std**2])
rk.Q[0:2, 0:2] = Q_discrete_white_noise(2, dt=dt, var=0.1)
rk.Q[2,2] = 0.1
rk.P *= 50

xs, track = [], []
for i in range(int(20/dt)):
    z = radar.get_range()
    track.append((radar.pos, radar.vel, radar.alt))

    rk.update(array([z]), HJacobian_at, hx)
    xs.append(rk.x)
    rk.predict()
```

```
xs = asarray(xs)
track = asarray(track)
time = np.arange(0, len(xs)*dt, dt)
ekf_internal.plot_radar(xs, track, time)
```







使用 SymPy 计算雅可比矩阵

根据您对微分方程的掌握程度，您可能会觉得计算雅可比矩阵是一件很困难的事情。即使您觉得计算起来并不难，但稍微复杂一点的问题仍然会导致非常繁琐的计算过程。

如附录 A 中所解释的，我们可以使用 SymPy 软件包来计算雅可比矩阵。

```
import sympy
from IPython.display import display
sympy.init_printing(use_latex='mathjax')

x, x_vel, y = sympy.symbols('x, x_vel y')

H = sympy.Matrix([sympy.sqrt(x**2 + y**2)])

state = sympy.Matrix([x, x_vel, y])
J = H.jacobian(state)

display(state)
display(J)
```

$$\begin{bmatrix} x \\ x_{vel} \\ y \end{bmatrix}$$

$$\begin{bmatrix} \frac{x}{\sqrt{x^2+y^2}} & 0 & \frac{y}{\sqrt{x^2+y^2}} \end{bmatrix}$$

这个结果与我们上面计算出的结果相同，而且我们付出的努力要少得多！

机器人定位

是时候尝试解决一个真正的难题了。我提醒你，这一部分确实比较困难。不过，大多数书籍都选择简单易懂的问题来作为例子，并且给出的答案也很简单。而在这里，你就需要自己思考如何解决现实世界中的问题了。

我们将探讨机器人的定位问题。我们已经在“无迹卡尔曼滤波”那一章中实现了这一功能。如果您还没有阅读那一章，我建议您现在就去看看。在这种情况下，机器人通过传感器来识别周围的地标，从而确定自己的位置。这可以是利用计算机视觉来识别树木、建筑物等地标的自动驾驶汽车；也可以是那些用于打扫房子的微型机器人，或是仓库里的作业机器人。

这款机器人的结构与汽车类似，也有四个轮子。它通过转动前轮来改变方向。这样一来，机器人在前进的同时，也会以后轴为中心进行旋转。这是一种非线性运动方式，我们需要对这种行为进行建模。

这个机器人配备了一个传感器，可以测量与周围环境中已知目标之间的距离和方向。这种测量方式是非线性的，因为要从距离和方向信息计算出位置需要用到平方根和三角函数运算。

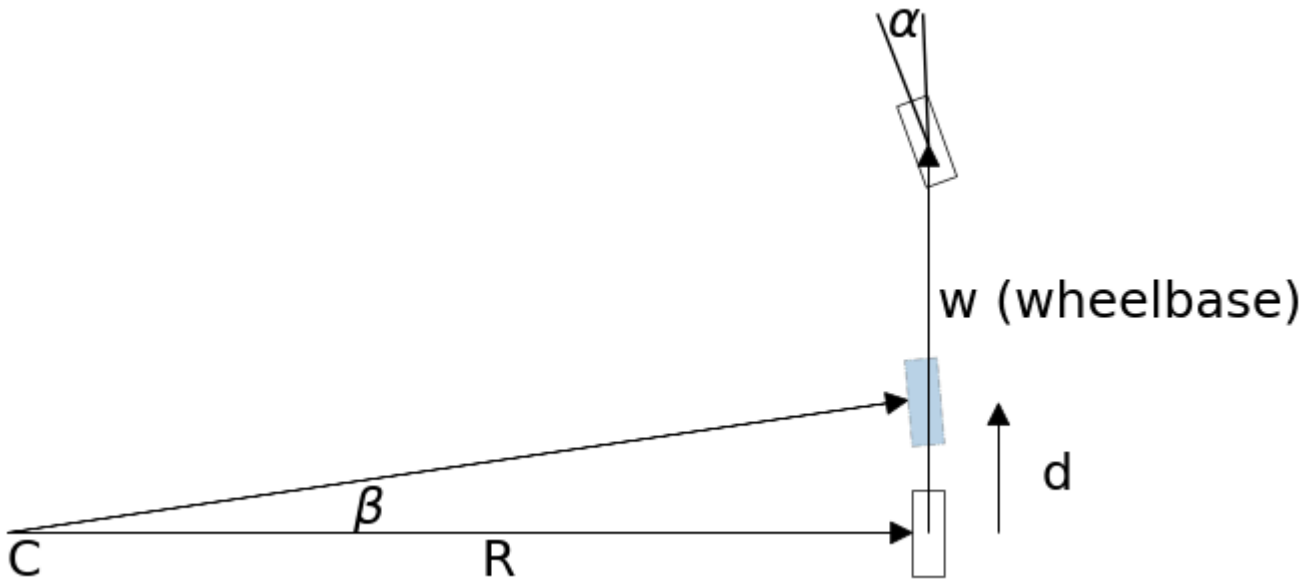
无论是过程模型还是测量模型，都属于非线性模型。而 EKF 能够同时处理这两种非线性问题，因此我们初步认为 EKF 是解决这个问题的一个可行方案。

机器人运动模型

简单来说，汽车在行驶过程中是通过转动前轮来实现转向的。汽车的前部会朝着车轮所指的方向移动，而整个车身则围绕后轮进行转动。不过，这一简单的机制实际上相当复杂：摩擦力会导致打滑现象；橡胶轮胎在不同速度下的表现也有所不同；此外，外侧轮胎和内侧轮胎的转动半径也必须有所不同。要准确描述汽车的转向过程，就需要用到复杂的微分方程。

对于速度较低的机器人应用来说，采用较为简单的自行车模型能够取得良好的效果。以下是该模型的示意图：

```
ekf_internal.plot_bicycle()
```



在无迹卡尔曼滤波这一章中，我们推导出了以下方程：

$$
\begin{aligned}
\beta &= \frac{d}{w}\tan(\alpha) \\
x &= x - R\sin(\theta) + R\sin(\theta+\beta) \\
y &= y + R\cos(\theta) - R\cos(\theta+\beta) \\
\theta &= \theta + \beta
\end{aligned}
$$

其中， θ 表示机器人的行进方向。

如果你对运动控制模型不感兴趣，那么无需详细了解该模型。需要明白的是，我们的运动模型是非线性的，因此我们必须利用卡尔曼滤波器来处理这个问题。

设计状态变量

在我们的滤波过程中，我们将保持机器人在位置 x, y 和方向 θ 上的状态不变。

$$
x = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}
$$

我们的控制输入 u 包括速度 v 和转向角 α 。

$$
u = \begin{bmatrix} v \\ \alpha \end{bmatrix}
$$

设计系统模型

我们将该系统建模为一个非线性运动模型，同时考虑了噪声的影响。

$$
\bar{x} = f(x, u) + N(0, Q)
$$

利用我们上面创建的机器人运动模型，我们可以将其扩展到……

$$
\begin{bmatrix} \bar{x} \\ y \\ \theta \end{bmatrix} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} + \begin{bmatrix} -R\sin(\theta) + R\sin(\theta+\beta) \\ R\cos(\theta) - R\cos(\theta+\beta) \\ \beta \end{bmatrix}
$$

我们通过计算 $f(x, u)$ 的雅可比矩阵来找到 F 的值。

$$
F = \frac{\partial f(x,u)}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} & \frac{\partial f_1}{\partial \theta} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} & \frac{\partial f_2}{\partial \theta} \\ \frac{\partial f_3}{\partial x} & \frac{\partial f_3}{\partial y} & \frac{\partial f_3}{\partial \theta} \end{bmatrix}
$$

当我们进行这些计算后，我们得到的结果是

$$
F = \begin{bmatrix} 1 & 0 & -R\cos(\theta) + R\cos(\theta+\beta) \\ 0 & 1 & -R\sin(\theta) + R\sin(\theta+\beta) \\ 0 & 0 & 1 \end{bmatrix}
$$

我们可以用 SymPy 来再次检查我们的计算结果。

```
import sympy
from sympy.abc import alpha, x, y, v, w, R, theta
from sympy import symbols, Matrix
sympy.init_printing(use_latex="mathjax", fontsize='16pt')
time = symbols('t')
d = v*time
beta = (d/w)*sympy.tan(alpha)
r = w/sympy.tan(alpha)

fxu = Matrix([[x-r*sympy.sin(theta) + r*sympy.sin(theta+beta)],
              [y+r*sympy.cos(theta)- r*sympy.cos(theta+beta)],
              [theta+beta]])
F = fxu.jacobian(Matrix([x, y, theta]))
F
```

$$\begin{bmatrix} 1 & 0 & -\frac{w\cos\left(\theta\right)}{\tan\left(\alpha\right)} + \frac{w\cos\left(\frac{tv\tan\left(\alpha\right)}{w}+\theta\right)}{\tan\left(\alpha\right)} \\ 0 & 1 & -\frac{w\sin\left(\theta\right)}{\tan\left(\alpha\right)} + \frac{w\sin\left(\frac{tv\tan\left(\alpha\right)}{w}+\theta\right)}{\tan\left(\alpha\right)} \\ 0 & 0 & 1 \end{bmatrix}$$

那看起来有点复杂。我们可以使用 SymPy 来替换其中的各项：

```
# reduce common expressions
B, R = symbols('beta, R')
F = F.subs((d/w)*sympy.tan(alpha), B)
F.subs(w/sympy.tan(alpha), R)
```

$$\begin{bmatrix} 1 & 0 & -R\cos\left(\theta\right) + R\cos\left(\beta+\theta\right) \\ 0 & 1 & -R\sin\left(\theta\right) + R\sin\left(\beta+\theta\right) \\ 0 & 0 & 1 \end{bmatrix}$$

该表格用于验证雅可比矩阵的计算是否正确。

现在，我们可以把注意力转向“噪声”问题。在这里，噪声存在于我们的控制输入中，也就是说，它属于控制空间中的因素。换句话说，我们指令机器人达到特定的速度和转向角度，但必须将这些指令转化为相应的误差值。在真实系统中，这些误差值会随着速度的变化而变化，因此需要针对每次预测重新计算这些误差值。我会选择这种模型来表示噪声；而对于真正的机器人来说，则需要选择能够准确反映系统中误差的模型。

$$M = \begin{bmatrix} \sigma_{vel}^2 & 0 \\ 0 & \sigma_\alpha^2 \end{bmatrix}$$

如果这是一个线性问题，我们就可以利用大家已经熟悉的 FMF^T 形式，将控制空间转换到状态空间。但由于我们的运动模型是非线性的，我们不会试图寻求其解析解，而是通过雅可比矩阵来对模型进行线性化处理，我们把这个雅可比矩阵称为 V。

$$V = \frac{\partial f(x,u)}{\partial u}\begin{bmatrix} \frac{\partial f_1}{\partial v} & \frac{\partial f_1}{\partial \alpha} \\ \frac{\partial f_2}{\partial v} & \frac{\partial f_2}{\partial \alpha} \\ \frac{\partial f_3}{\partial v} & \frac{\partial f_3}{\partial \alpha} \end{bmatrix}$$

这些偏导数很难处理。让我们用 SymPy 来计算它们吧。

```
V = fxu.jacobian(Matrix([v, alpha]))
V = V.subs(sympy.tan(alpha)/w, 1/R)
V = V.subs(time*v/R, B)
V = V.subs(time*v, 'd')
V
```

$$\begin{bmatrix} t\cos\left(\beta+\theta\right) & \frac{d\left(\tan^2\left(\alpha\right)+1\right)\cos\left(\beta+\theta\right)}{\tan\left(\alpha\right)} - \frac{w\left(-\tan^2\left(\alpha\right)-1\right)\sin\left(\theta\right)}{\tan^2\left(\alpha\right)} + \frac{w\left(-\tan^2\left(\alpha\right)-1\right)\sin\left(\beta+\theta\right)}{\tan^2\left(\alpha\right)} \\ t\sin\left(\beta+\theta\right) & \frac{d\left(\tan^2\left(\alpha\right)+1\right)\sin\left(\beta+\theta\right)}{\tan\left(\alpha\right)} + \frac{w\left(-\tan^2\left(\alpha\right)-1\right)\cos\left(\theta\right)}{\tan^2\left(\alpha\right)} - \frac{w\left(-\tan^2\left(\alpha\right)-1\right)\cos\left(\beta+\theta\right)}{\tan^2\left(\alpha\right)} \\ \frac{t}{R} & \frac{d\left(\tan^2\left(\alpha\right)+1\right)}{w} \end{bmatrix}$$

这应该能让你明白，EKF 在数学上变得难以处理的速度有多快。

这样，我们就得到了预测方程的最终形式：

$$\begin{aligned} \bar{x} &= x + \begin{bmatrix} -R\sin(\theta) + R\sin(\theta+\beta) \\ R\cos(\theta) - R\cos(\theta+\beta) \\ \beta \end{bmatrix} \\ \bar{P} &= FPF^T + VMV^T \end{aligned}$$

这种线性化处理方式并非预测 x 的唯一方法。例如，我们可以使用龙格-库塔之类的数值积分技术来计算机器人的运动轨迹。当时间步长较大时，就需要采用这种做法。与卡尔曼滤波器相比，EKF 的处理方式则没有那么简单明了。在解决实际问题时，必须先用微分方程来精确描述系统，然后再确定最合适的求解方法。究竟采用哪种方法，取决于所需的精度、方程的非线性程度、处理器的性能限制以及数值稳定性方面的要求。

设计测量模型

该机器人的传感器能够为地形中的多个已知位置提供方位和距离信息。测量模型需要将这些数据转换为相对于各参考点的距离和方位。如果 p 表示某个参考点的位置，那么距离 r 就是从该参考点到其他点的距离。

$$r = \sqrt{(p_x - x)^2 + (p_y - y)^2}$$

该传感器能够提供相对于机器人姿态的方位信息。因此，我们必须从该方位值中减去机器人的姿态信息，才能得到传感器的实际测量值。具体计算方式如下：

$$\phi = \arctan(\frac{p_y - y}{p_x - x}) - \theta$$

因此，我们的测量模型 h 就是如此。

$$\begin{aligned} z &= h(\bar{x}, p) + N(0, R) \\ &= \begin{bmatrix} \sqrt{(p_x - x)^2 + (p_y - y)^2} \\ \arctan(\frac{p_y - y}{p_x - x}) - \theta \end{bmatrix} + N(0, R) \end{aligned}$$

这显然是一个非线性系统，因此我们需要通过计算雅可比矩阵来在点 x 处对 h 进行线性化处理。我们使用 SymPy 来计算雅可比矩阵。

```
px, py = symbols('p_x, p_y')
z = Matrix([[sympy.sqrt((px-x)**2 + (py-y)**2)],
            [sympy.atan2(py-y, px-x) - theta]])
z.jacobian(Matrix([x, y, theta]))
```

$$\begin{bmatrix} \frac{-p_x + x}{\sqrt{(p_x - x)^2 + (p_y - y)^2}} & \frac{-p_y + y}{\sqrt{(p_x - x)^2 + (p_y - y)^2}} & 0 \\ -\frac{-p_y + y}{(p_x - x)^2 + (p_y - y)^2} & -\frac{p_x - x}{(p_x - x)^2 + (p_y - y)^2} & -1 \end{bmatrix}$$

现在，我们需要把这段代码写成一个 Python 函数。例如，我们可以这样写：

```
from math import sqrt

def H_of(x, landmark_pos):
    """ compute Jacobian of H matrix where h(x) computes
    the range and bearing to a landmark for state x """

    px = landmark_pos[0]
    py = landmark_pos[1]
    hyp = (px - x[0, 0])**2 + (py - x[1, 0])**2
    dist = sqrt(hyp)

    H = array(
        [[-(px - x[0, 0]) / dist, -(py - x[1, 0]) / dist, 0],
         [ (py - x[1, 0]) / hyp,  -(px - x[0, 0]) / hyp, -1]])
    return H
```

我们还需要定义一个函数，将该系统的状态转换为可测量的数值。

```
from math import atan2

def Hx(x, landmark_pos):
    """ takes a state variable and returns the measurement
    that would correspond to that state.
    """
    px = landmark_pos[0]
    py = landmark_pos[1]
    dist = sqrt((px - x[0, 0])**2 + (py - x[1, 0])**2)

    Hx = array([[dist],
                [atan2(py - x[1, 0], px - x[0, 0]) - x[2, 0]]])
    return Hx
```

设计测量噪声

可以合理地假设，距离和方位角的测量误差是相互独立的。因此，

$$R = \begin{bmatrix} \sigma^2_{range} & 0 \\ 0 & \sigma^2_{bearing} \end{bmatrix}$$

实施/执行

我们将使用 FilterPy 的 ExtendedKalmanFilter 类来实现该滤波器。该类的 predict() 方法采用了标准的线性方程来描述处理过程。不过，我们的系统属于非线性系统，因此我们必须用自己的实现来覆盖 predict() 方法。我还想利用这个类来模拟机器人的行为，因此会添加一个 move() 方法，用于计算机器人的位置。predict() 以及我的模拟程序都可以调用这个方法。

在预测步骤中，所涉及的矩阵规模相当大。在编写这段代码时，我犯了几个错误，最终才成功实现了功能。我是通过使用 SymPy 的 evalf 函数来发现这些错误的。 evalf 函数会用特定的变量值来计算 SymPy 表达式 Matrix 的结果。我决定向您展示这种技巧，因此在卡尔曼滤波器的代码中使用了 evalf 函数。您需要理解其中的几个要点。

首先， evalf 使用字典来指定各个数值。例如，如果你的矩阵中包含 x 和 y ，你可以这样表示：

```
M.evalf(subs={x:3, y:17})
```

正在评估 x=3 和 y=17 的矩阵情况。

其次， evalf 返回一个 sympy.Matrix 类型的对象。需要使用 numpy.array(M).astype(float) 将其转换为 NumPy 数组。而 numpy.array(M) 创建的是一个 object 类型的数组，这并不是我们想要的格式。

以下是 EKF 的编码：

```
from filterpy.kalman import ExtendedKalmanFilter as EKF
from numpy import array, sqrt
class RobotEKF(EKF):
    def __init__(self, dt, wheelbase, std_vel, std_steer):
        EKF.__init__(self, 3, 2, 2)
        self.dt = dt
        self.wheelbase = wheelbase
        self.std_vel = std_vel
        self.std_steer = std_steer

        a, x, y, v, w, theta, time = symbols(
            'a, x, y, v, w, theta, t')
        d = v*time
        beta = (d/w)*sympy.tan(a)
        r = w/sympy.tan(a)

        self.fxu = Matrix(
            [[x-r*sympy.sin(theta)+r*sympy.sin(theta+beta)],
             [y+r*sympy.cos(theta)-r*sympy.cos(theta+beta)],
             [theta+beta]])

        self.F_j = self.fxu.jacobian(Matrix([x, y, theta]))
        self.V_j = self.fxu.jacobian(Matrix([v, a]))

        # save dictionary and it's variables for later use
        self.subs = {x: 0, y: 0, v:0, a:0,
                     time:dt, w:wheelbase, theta:0}
        self.x_x, self.x_y, = x, y
        self.v, self.a, self.theta = v, a, theta

    def predict(self, u):
        self.x = self.move(self.x, u, self.dt)
```

```
        self.subs[self.x_x] = self.x[0, 0]
        self.subs[self.x_y] = self.x[1, 0]

        self.subs[self.theta] = self.x[2, 0]
        self.subs[self.v] = u[0]
        self.subs[self.a] = u[1]

        F = array(self.F_j.evalf(subs=self.subs)).astype(float)
        V = array(self.V_j.evalf(subs=self.subs)).astype(float)

        # covariance of motion noise in control space
        M = array([[self.std_vel**2, 0],
                   [0, self.std_steer**2]])

        self.P = F @ self.P @ F.T + V @ M @ V.T

    def move(self, x, u, dt):
        hdg = x[2, 0]
        vel = u[0]
        steering_angle = u[1]
        dist = vel * dt

        if abs(steering_angle) > 0.001: # is robot turning?
            beta = (dist / self.wheelbase) * tan(steering_angle)
            r = self.wheelbase / tan(steering_angle) # radius

            dx = np.array([[-r*sin(hdg) + r*sin(hdg + beta)],
                           [r*cos(hdg) - r*cos(hdg + beta)],
                           [beta]])
        else: # moving in straight line
            dx = np.array([[dist*cos(hdg)],
                           [dist*sin(hdg)],
                           [0]])
        return x + dx
```

现在我们还有另一个问题需要处理。从理论上讲，残差应该按照 $y = z - h(x)$ 的方式来计算。但实际上，由于我们的测量数据中包含了角度信息，这种计算方式并不适用。假设 z 的方位角为 1°，而 $h(x)$ 的方位角为 359°。如果直接将这两个数值相减，得到的角度差为 -358°，但实际上正确的数值应该是 2°。因此，我们必须编写相应的代码来正确计算出残差值。

```
def residual(a, b):
    """ compute residual (a-b) between measurements containing
    [range, bearing]. Bearing is normalized to [-pi, pi)"""
    y = a - b
    y[1] = y[1] % (2 * np.pi)    # force in range [0, 2 pi)
    if y[1] > np.pi:             # move to [-pi, pi)
        y[1] -= 2 * np.pi
    return y
```

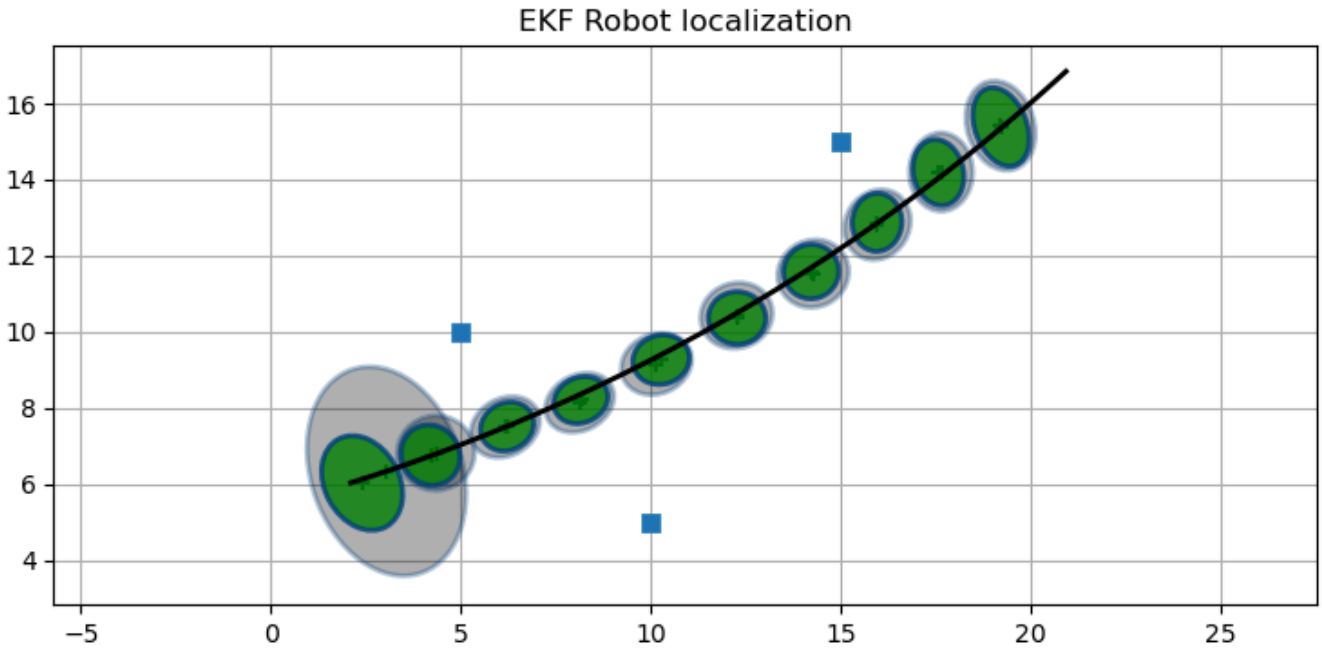
其余的代码负责执行模拟运算并绘制结果图，这些部分应该不需要太多解释。我创建了一个名为 `landmarks` 的变量，用于存储各个参考点的坐标。模拟中的机器人位置每秒更新 10 次，而卡尔曼滤波则每秒只计算一次。这样做有两个原因：首先，我们没有使用龙格-库塔法来求解运动方程，因此较小的时间步长有助于提高模拟的准确性。其次，在嵌入式系统中，处理速度往往有限。因此，必须仅在确实必要时才运行卡尔曼滤波器。

```
from filterpy.stats import plot_covariance_ellipse
from math import sqrt, tan, cos, sin, atan2
import matplotlib.pyplot as plt
```

```
dt = 1.0

def z_landmark(lmark, sim_pos, std_rng, std_brg):
    x, y = sim_pos[0, 0], sim_pos[1, 0]
    d = np.sqrt((lmark[0] - x)**2 + (lmark[1] - y)**2)
    a = atan2(lmark[1] - y, lmark[0] - x) - sim_pos[2, 0]
    z = np.array([[d + randn()*std_rng],
                  [a + randn()*std_brg]])
    return z

def ekf_update(ekf, z, landmark):
    ekf.update(z, HJacobian=H_of, Hx=Hx,
               residual=residual,
               args=(landmark), hx_args=(landmark))


def run_localization(landmarks, std_vel, std_steer,
                     std_range, std_bearing,
                     step=10, ellipse_step=20, ylim=None):
    ekf = RobotEKF(dt, wheelbase=0.5, std_vel=std_vel,
                   std_steer=std_steer)
    ekf.x = array([[2, 6, .3]]).T # x, y, steer angle
    ekf.P = np.diag([.1, .1, .1])
    ekf.R = np.diag([std_range**2, std_bearing**2])

    sim_pos = ekf.x.copy() # simulated position
    # steering command (vel, steering angle radians)
    u = array([1.1, .01])

    plt.figure()
    plt.scatter(landmarks[:, 0], landmarks[:, 1],
                marker='s', s=60)

    track = []
    for i in range(200):
        sim_pos = ekf.move(sim_pos, u, dt/10.) # simulate robot
        track.append(sim_pos)

        if i % step == 0:
            ekf.predict(u=u)

            if i % ellipse_step == 0:
                plot_covariance_ellipse(
                    (ekf.x[0,0], ekf.x[1,0]), ekf.P[0:2, 0:2],
                     std=6, facecolor='k', alpha=0.3)

            x, y = sim_pos[0, 0], sim_pos[1, 0]
            for lmark in landmarks:
                z = z_landmark(lmark, sim_pos,
                               std_range, std_bearing)
                ekf_update(ekf, z, lmark)

            if i % ellipse_step == 0:
                plot_covariance_ellipse(
                    (ekf.x[0,0], ekf.x[1,0]), ekf.P[0:2, 0:2],
                    std=6, facecolor='g', alpha=0.8)
    track = np.array(track)
    plt.plot(track[:, 0], track[:,1], color='k', lw=2)
    plt.axis('equal')
    plt.title("EKF Robot localization")
    if ylim is not None: plt.ylim(*ylim)
```

```
    plt.show()
    return ekf
```

```
landmarks = array([[5, 10], [10, 5], [15, 15]])

ekf = run_localization(
    landmarks, std_vel=0.1, std_steer=np.radians(1),
    std_range=0.3, std_bearing=0.1)
print('Final P:', ekf.P.diagonal())
```




```
Final P: [0.017 0.035 0.002]
```

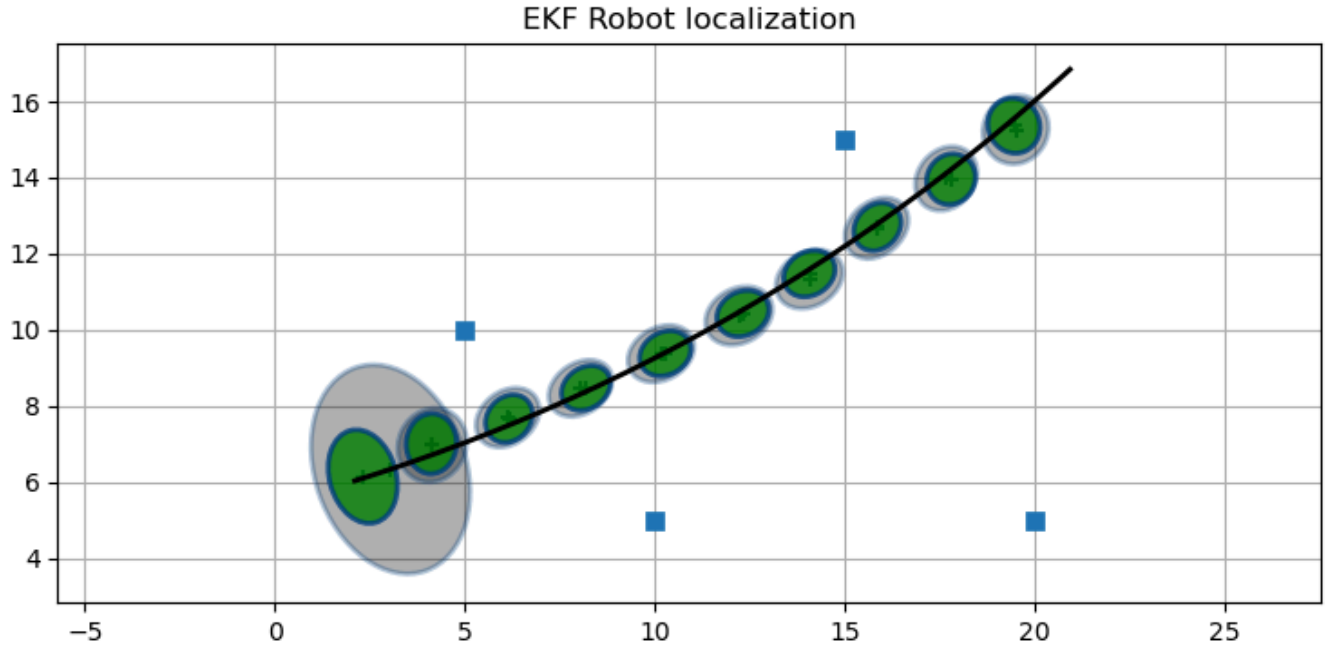
我将各个地标标记为实心方块。机器人的移动路径则用黑线表示。在预测阶段，协方差椭圆以浅灰色显示；而在更新阶段，协方差则用绿色表示。为了在这些比例下仍能看清这些椭圆，我将椭圆的边界宽度设置为 $6\,\sigma$。

我们可以看到，我们的运动模型引入了很多不确定性，而运动方向上的误差占据了主导地位。从蓝色椭圆曲线的形态可以判断出这一点。经过几次迭代后，我们发现滤波器已经能够整合了地标信息，从而减少了误差。

我使用了与英国飞行章节中相同的初始条件和地标位置。从误差椭圆的角度来看，英国飞行方式的精度要高很多。就 x 的估计结果而言，两者的表现都相当不错。

现在，让我们再添加一个标志性建筑吧。

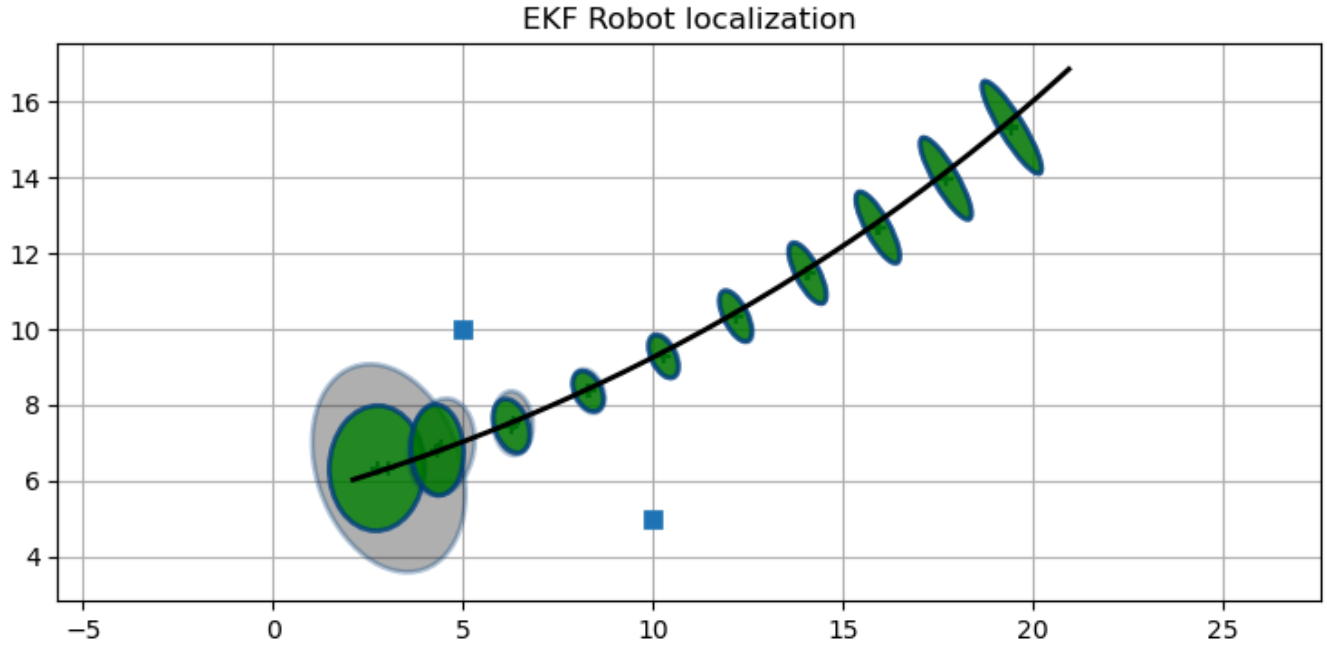
```
landmarks = array([[5, 10], [10, 5], [15, 15], [20, 5]])

ekf = run_localization(
    landmarks, std_vel=0.1, std_steer=np.radians(1),
    std_range=0.3, std_bearing=0.1)
plt.show()
print('Final P:', ekf.P.diagonal())
```




```
Final P: [0.014 0.016 0.002]
```

在轨道接近终点时，估算结果的不确定性较小。通过仅使用前两个地标，我们可以观察到多个地标对估算精度的影响。

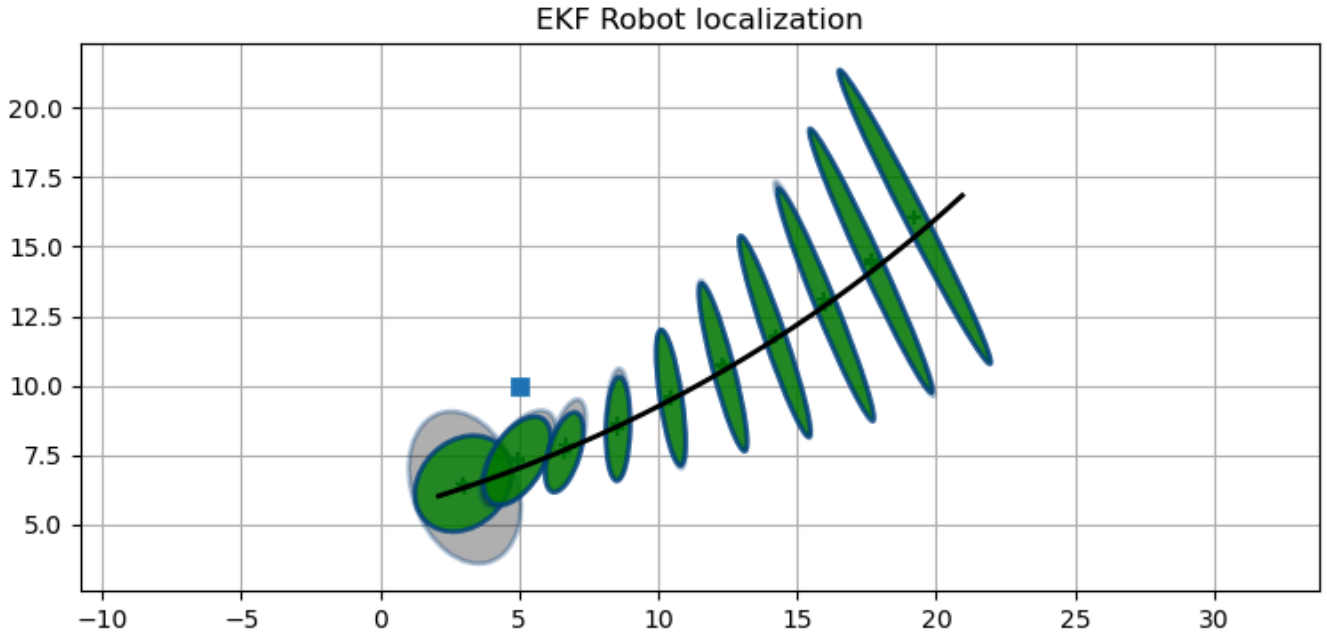
```
ekf = run_localization(
    landmarks[0:2], std_vel=1.e-10, std_steer=1.e-10,
    std_range=1.4, std_bearing=.05)
print('Final P:', ekf.P.diagonal())
```




```
Final P: [0.02  0.046 0.   ]
```

在经过各个地标之后，该估计值很快就会与机器人的实际路径产生偏差。同时，协方差也会迅速增大。让我们来看看，当只有一个地标时会发生什么：

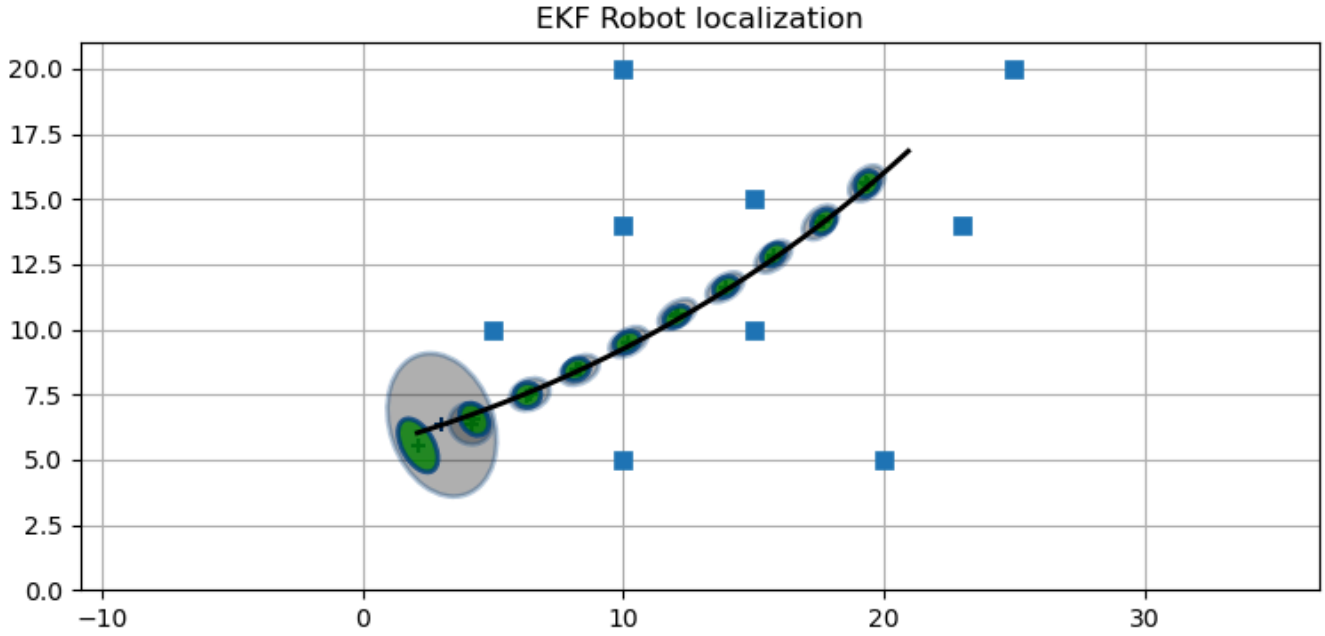
```
ekf = run_localization(
    landmarks[0:1], std_vel=1.e-10, std_steer=1.e-10,
    std_range=1.4, std_bearing=.05)
print('Final P:', ekf.P.diagonal())
```




```
Final P: [0.249 0.849 0.004]
```

正如你可能已经猜到的那样，如果只使用一个地标点，那么得到的结果会非常不准确。相反，如果使用大量的地标点，就能得到非常精确的估算结果。

```
landmarks = array([[5, 10], [10,  5], [15, 15], [20,  5], [15, 10],
                   [10,14], [23, 14], [25, 20], [10, 20]])

ekf = run_localization(
    landmarks, std_vel=0.1, std_steer=np.radians(1),
    std_range=0.3, std_bearing=0.1, ylim=(0, 21))
print('Final P:', ekf.P.diagonal())
```




```
Final P: [0.006 0.007 0.001]
```

讨论

我说过，这确实是个问题，从某种程度上来说确实如此。我见过一些采用机器人运动模型的替代方案，这些方案使得雅可比矩阵的计算更为简单。不过，我所建立的模型在多个方面也过于简化了。首先，该模型采用了自行车的运动模型来描述机器人的运动。而实际上，汽车有两组轮胎，每组轮胎的旋转半径都不同。此外，车轮与地面的接触也并非完美无缺。我还假设机器人能立即响应控制指令。塞巴斯蒂安·特伦在《概率机器人》一书中指出，这种简化模型是合理的，因为当用于跟踪真实车辆时，这种模型的效果相当不错。由此可见，虽然需要一个相当精确的非线性模型，但并不一定非得做到完美才能正常运行。作为设计师，你需要权衡模型的精确度与计算难度、以及执行线性代数运算所需的 CPU 时间之间的关系。

这个问题的另一个简化处理方式是：我们假设自己知道各个地标与相应测量数据之间的对应关系。但是，如果我们使用的是雷达技术，又该如何确定某个特定的信号回波对应于现场中的哪栋建筑物呢？这个问题其实涉及到 SLAM 算法——即同时定位与地图构建。不过，SLAM 并非本书的重点，因此我就不详细讨论这个话题了。

UKF 与 EKF 的比较

在上一章中，我使用了 UKF 方法来解决这个问题。两种方法的实现方式有着明显的区别。尽管采用了较为简单的运动模型，但计算状态模型和测量模型的雅可比矩阵并非易事。对于某些问题来说，雅可比矩阵可能难以或无法通过解析方法来求得。而 UKF 则只需要用户提供两个函数：一个用于计算系统运动模型，另一个用于计算测量模型。

在很多情况下，雅可比矩阵无法通过解析方法来求出。这些细节超出了本书的讨论范围，你不得不借助数值方法来计算雅可比矩阵。这项工作并非易事，因此在攻读 STEM 专业的硕士学位期间，你需要花费大量时间来学

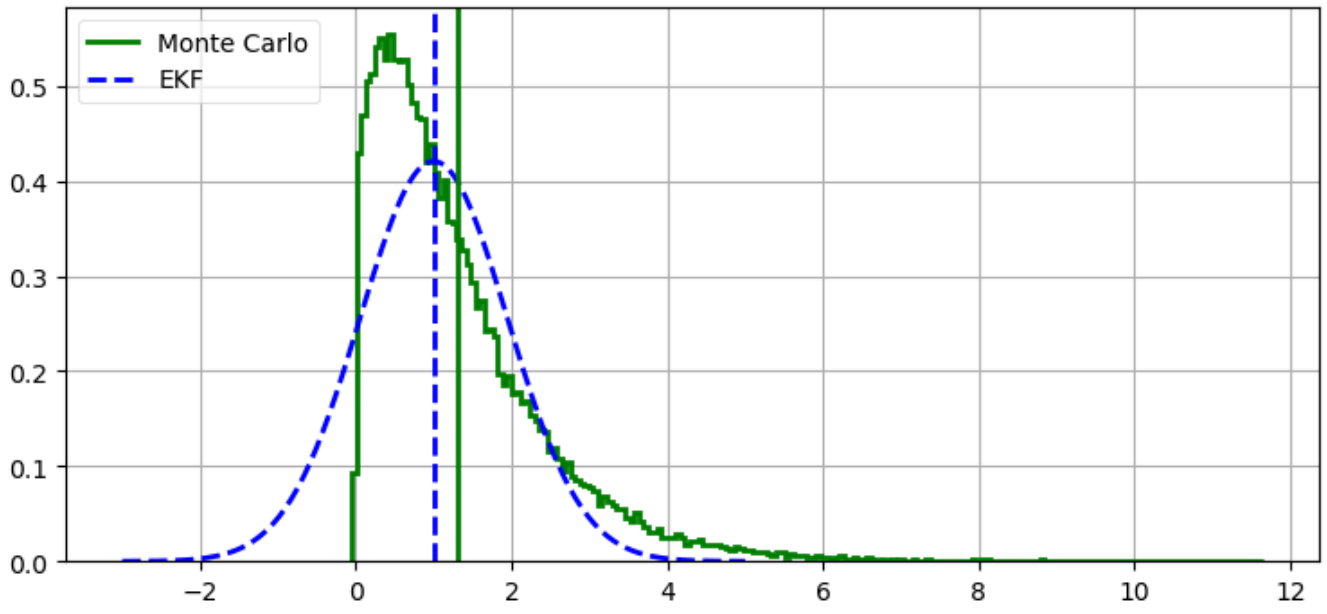
习处理这类问题的方法。即便如此，你通常也只能解决与自己所在领域相关的问题——比如，航空工程师会深入学习纳维-斯托克斯方程，但几乎不会涉及化学反应速率的建模。

所以，UKF 算法其实很简单。不过，它的精度如何呢？在实际应用中，UKF 的性能往往优于 EKF。有很多研究论文都证明了，在各种应用场景下，UKF 的表现都优于 EKF。这一点不难理解：EKF 是通过在某一点上对系统模型和测量模型进行线性化处理来工作的，而 UKF 则是在多个点上进行处理。

我们来看一个具体的例子。以 $f(x) = x^3$ 为例，让高斯分布函数作用于该值上。我会通过蒙特卡洛模拟来得到精确的结果。首先，我随机生成 50,000 个符合高斯分布的点，然后将每个点代入 $f(x)$ 中处理。最后，计算处理后的数据的平均值和方差。

EKF 通过求导来使该函数线性化，从而确定在评估点 x 处的斜率。这个斜率就构成了我们用来转换高斯分布的线性函数。下图展示了这一过程。

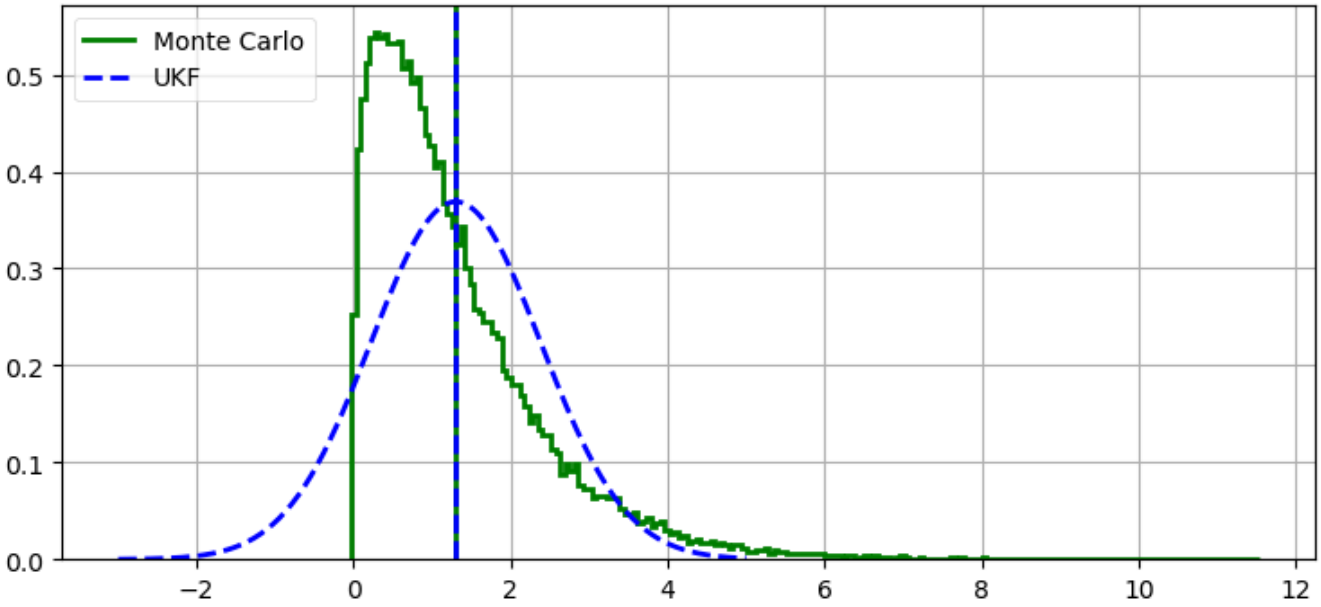
```
import kf_book.nonlinear_plots as nonlinear_plots
nonlinear_plots.plot_ekf_vs_mc()
```




```
actual mean=1.30, std=1.12
EKF    mean=1.00, std=0.95
```

EKF 算法的计算结果相当不准确。相比之下，UKF 算法的性能则要好得多：

```
nonlinear_plots.plot_ukf_vs_mc(alpha=0.001, beta=3., kappa=1.)
```




```
actual mean=1.30, std=1.13
UKF    mean=1.30, std=1.08
```

从这里可以看出，UKF 算法计算出的均值精度达到了两位小数。标准差的数值略有偏差，不过你可以通过调整 α 、 β 和 γ 这三个参数来微调 UKF 算法对概率分布的计算方式。我在这里使用了 $\alpha = 0.001$ 、 $\beta = 3$ 和 $\gamma = 1$ 这三个参数。你可以自行调整这些参数，看看效果如何。你应该能够得到比我更好的结果。不过，不要过度调整 UKF 算法的参数，因为那样虽然可能让你的特定测试案例表现得更好，但整体性能反而会下降。

第 12 章：粒子滤波

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

动机/动力

这就是我们面临的问题。我们需要追踪一些正在移动的物体。这些物体可能是战斗机或导弹，也可能是正在球场上打板球的人。其实并不重要。那么，我们所学过的各种滤波方法中，有哪种能够解决这个问题呢？遗憾的是，没有一种方法是完美的。让我们来仔细分析一下这个问题的特点吧。

- 多模态处理：我们希望能够同时追踪一个、两个或更多个物体。
- 遮挡现象：一个物体可以遮住另一个物体，从而导致对多个物体进行了一次性的测量。
- 非线性行为：飞机会受到风的冲击而颠簸不定，球会沿抛物线轨迹运动，人们也会相互碰撞。
- 非线性测量：雷达可以告诉我们物体与我们的距离。要将这个距离转换为(x,y,z)坐标，需要进行开方运算，而开方属于非线性运算。
- 非高斯噪声：当物体在背景上移动时，计算机视觉系统可能会将背景的某些部分误认为是物体本身。
- 连续性：物体的位置和速度（即状态空间）可以随时间平滑地变化。
- 多变量分析：我们需要同时追踪多个属性，比如位置、速度、转向速率等等。
- 未知的进程模型：我们可能不了解该系统的进程模型。

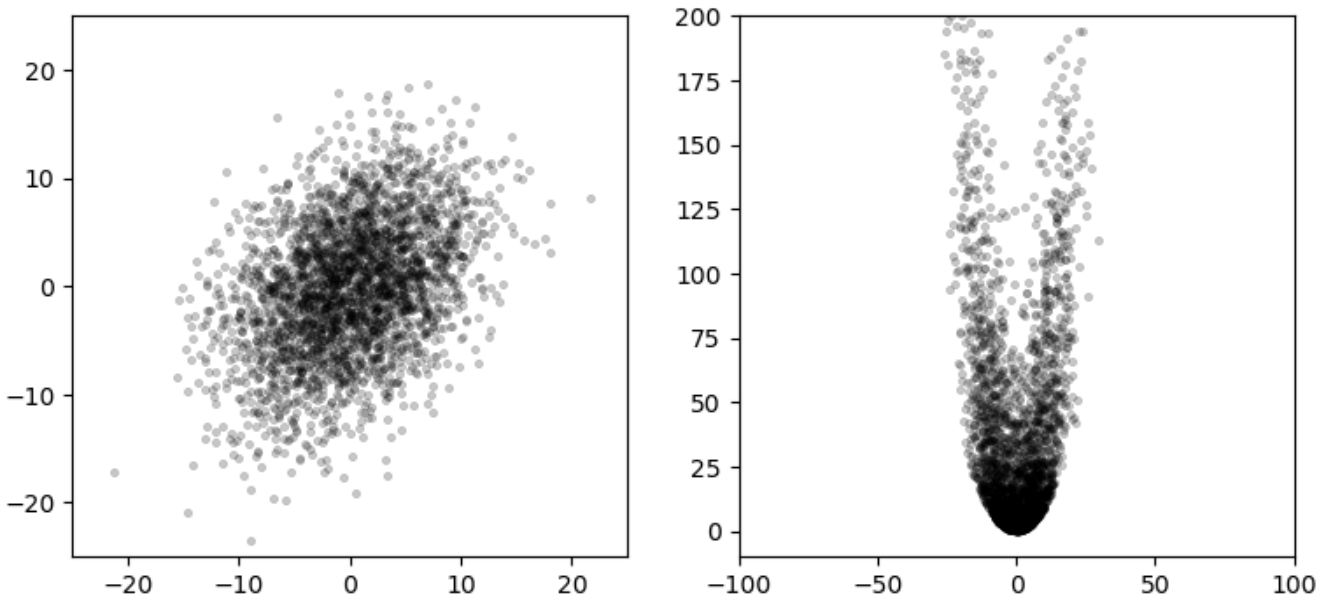
我们所了解的各类滤波方法，没有一种能在所有这些限制条件下都发挥良好的效果。

- 离散贝叶斯滤波器：它具备大多数所需的特性。该滤波器具有多模态处理能力，能够处理非线性测量数据，同时也可用于处理非线性行为。不过，它属于离散型且单变量处理的滤波器。
- 卡尔曼滤波器：卡尔曼滤波器能够为具有高斯噪声的单模线性系统提供最优的估计值。不过，我们的问题并不符合这些条件。
- 无迹卡尔曼滤波器：UKF 能够处理非线性、连续的多变量问题。不过，它无法处理多模态问题，也无法处理物体被遮挡的情况。它能够处理那些略带非高斯特性的噪声，但对于那些高度非高斯分布或高度非线性的问题，则处理能力较差。
- 扩展卡尔曼滤波器：与 UKF 相比，EKF 具有相同的优点和局限性。只不过，EKF 对强烈的非线性因素和非高斯噪声更为敏感。

蒙特卡洛抽样法

在 UKF 相关的内容中，我绘制了类似的图表，用以说明非线性系统对高斯分布的影响。

```
import kf_book.pf_internal as pf_internal
pf_internal.plot_monte_carlo_ukf()
```



左边的图表显示了 3,000 个遵循高斯分布的数值。

$$\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 32 & 15 \\ 15 & 40 \end{bmatrix}$$

右侧的图表显示了这些点如何满足这一组方程式：

$$\begin{aligned} x &= x + y \\ y &= 0.1x^2 + y^2 \end{aligned}$$

利用有限数量的随机采样点来计算结果的方法，被称为蒙特卡洛方法。其原理很简单：先生成足够多的样本点，以构成该问题的代表性样本；然后让这些样本点在所建模的系统中运行；最后根据这些样本点的运算结果来得出最终结果。

简而言之，这就是粒子滤波的运作原理。我们在本书中一直使用的贝叶斯滤波算法，实际上是应用于成千上万个“粒子”上的。每个粒子都代表着系统的一种可能状态。我们通过对这些粒子的加权统计结果，来得出系统的估计状态。

通用粒子滤波算法

1. **随机生成一群粒子**

这些粒子可以具有位置、方向以及任何其他需要估计的状态变量。每个变量都有一个权重（概率），表示该变量与系统实际状态匹配的可能性。初始化时，所有变量的权重应相同。

2. **预测粒子的下一个状态**

根据你对真实系统行为的预测来移动这些粒子。

3. **更新**

根据测量结果更新各粒子的权重。与测量结果最匹配的粒子，其权重会高于那些与测量结果不太符合的粒子。

4. **重新采样**

抛弃那些出现概率极低的粒子，用出现概率较高的粒子的“副本”来替代它们。

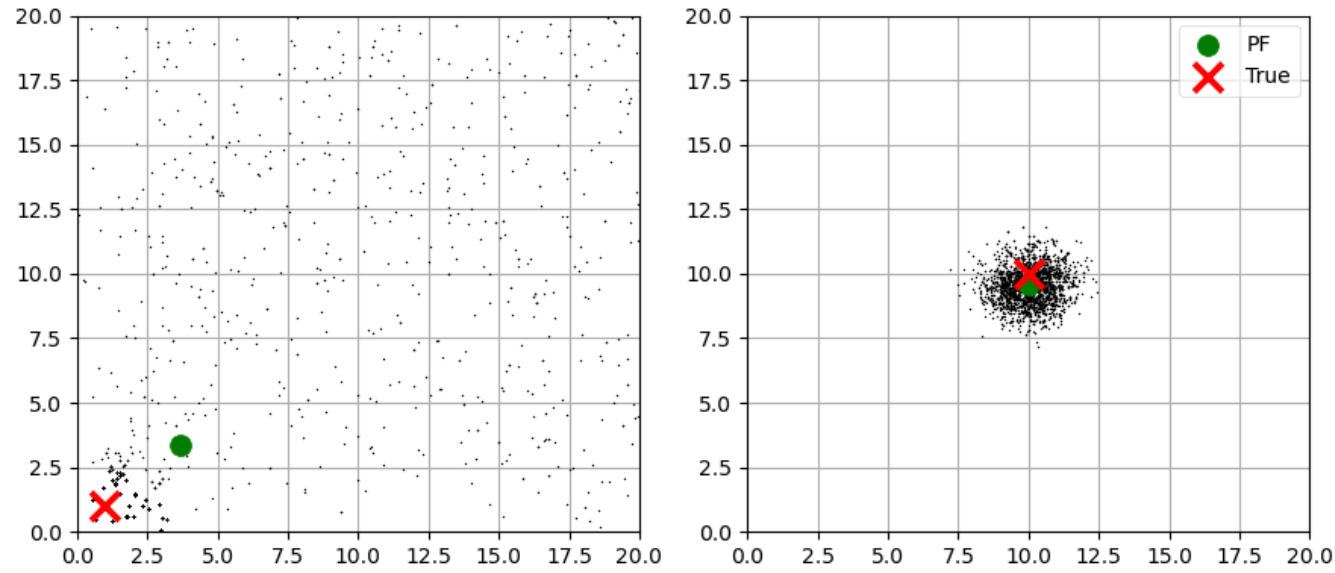
5. **计算估算值**

可选地，可以计算这些粒子的加权平均值和协方差，从而得到状态估计值。

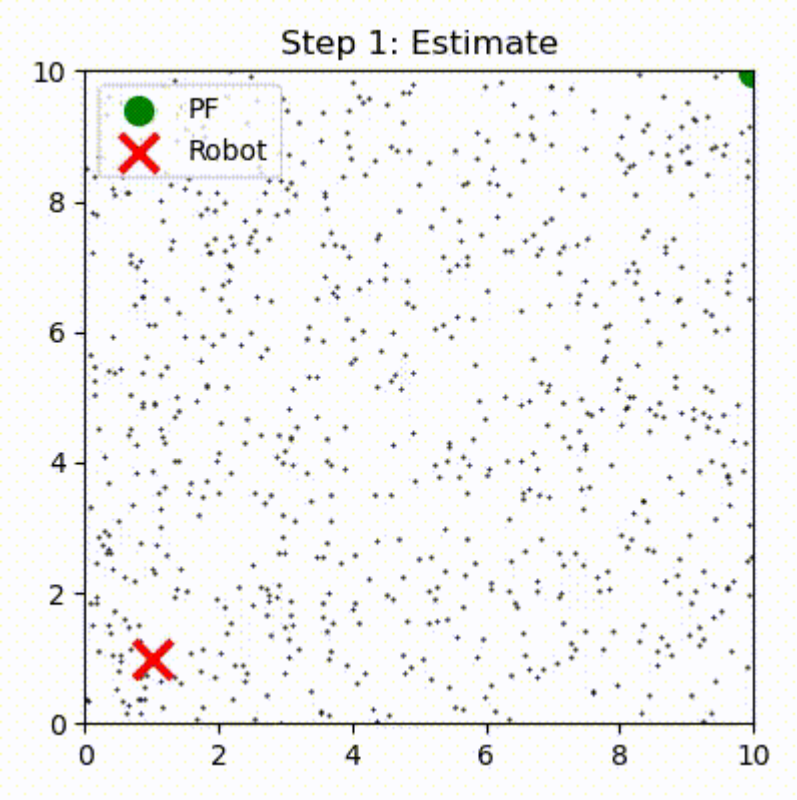
这种简单的算法在实际应用中存在诸多难题，我们需要设法克服这些困难。不过，这毕竟是一个基本的思路。我们来看个例子吧。我根据 UKF 和 EKF 的相关理论，为机器人定位问题编写了一个粒子滤波算法。该机器人具有转向和速度控制功能。它还配备了能够测量到可见地标距离的传感器。不过，无论是传感器还是控制机制都会产生噪声，而我们需要准确确定机器人的位置。

在这里，我运用了粒子滤波算法，并绘制出了各粒子的位置。左边的图显示的是经过一次迭代后的结果，右边的图则显示了经过 10 次迭代后的结果。红色的“X”标记代表了机器人的实际位置，而大圆圈则代表了经过计算得出的加权平均位置。

```
pf_internal.show_two_pf_plots()
```



如果您是通过浏览器查看此内容，那么该动画会完整地展示整个过程：



在第一次迭代之后，这些粒子仍然大致随机地分布在地图上。不过可以看出，有些粒子已经聚集在了机器人的附近。计算出的平均值与机器人的位置非常接近。这是因为每个粒子的权重取决于它与实际测量值的匹配程度。机器人位于(1,1)附近，因此那些位于(1,1)附近的粒子会因为与测量值高度匹配而拥有较高的权重。而那些远离机器人的粒子则无法与测量值匹配，因此其权重很低。最终的位置估计值就是各粒子位置的加权平均值。由于靠近机器人的粒子对计算结果的贡献更大，因此估计值相当准确。

经过几次迭代后，可以看到所有粒子都聚集在了机器人的周围。这是由于重采样步骤的作用。重采样会丢弃那些出现概率极低的粒子，然后用出现概率较高的粒子来替代它们。

我还没有完全解释清楚这一机制的运作原理，也没有详细阐述粒子权重分配和重采样的具体算法。不过，从直觉上来看，这个方法应该是可行的。我们可以生成大量随机粒子，让它们“大致”跟随机器人的移动轨迹；根据这些粒子与实际测量数据的匹配程度来为它们分配权重，只保留那些匹配度较高的粒子。看起来这种方法确实有效。

通过蒙特卡洛方法得到的概率分布

假设我们想知道曲线 $y = \mathrm{e}^{\sin(x)}$ 在区间$[0, \pi\ ]$内的面积。这个面积可以通过定积分 $\int_0^\pi \mathrm{e}^{\sin(x)}\mathrm{d}x$ 来计算。作为练习，请你自己试着算出答案吧；我会等待你的结果。

如果你够明智的话，就不会接受那个挑战吧； $\mathrm{e}^{\sin(x)}$ 是无法通过解析方法来计算的。世界上到处都是无法被解析计算的方程式。比如，试着计算某个物体的亮度吧。物体会反射部分照射到它身上的光线。其中一部分反射光又会照射到其他物体上，然后再反射回来照射到原来的物体上，从而增加了该物体的亮度。这就形成了一个递归积分问题。祝你好运吧。

不过，使用蒙特卡洛方法来计算积分其实相当简单。要计算曲线下的面积，可以先确定一个包含该曲线的矩形区域。然后在矩形区域内随机生成一些点，计算其中落在曲线下的点的比例。例如，如果 40%的点落在曲线下方，而矩形的面积为 1，那么曲线下的面积大约就是 0.4。当生成的点数趋于无穷大时，就能得到任意精确的结果。实际上，只需几千个点就能得到相当准确的结果。

你可以利用这种技术来对任何复杂程度的函数进行数值积分。这包括那些无法被积分或具有不连续性的函数。这项技术是由洛斯阿拉莫斯国家实验室的斯坦利·乌拉姆发明的，其目的是为了能够计算那些在纸上无法解决的核反应问题。

让我们通过计算圆的面积来求出 π 的值。我们先定义一个半径为 1 的圆，并将其置于一个边长为 2 的正方形内。那么，这个正方形的面积为 4。接下来，我们在这个正方形内随机生成一些点，然后统计其中有多少点落在圆内。圆的面积就可以通过正方形的面积乘以圆内点数与总点数之比来计算。最后，既然我们已经知道了 $A = \pi r^2$ 的值，那么就可以计算出 $\pi = A \, / \, r^2$ 了。

我们首先创建这些点。

```
N = 20000
pts = uniform(-1, 1, (N, 2))
```

如果一个点到圆心的距离小于或等于圆的半径，那么这个点就位于圆内。我们使用 `numpy.linalg.norm` 来计算距离，该函数用于计算向量的模长。由于向量的起点都是$(0, 0)$，因此调用 norm 函数就能得到该点到原点的距离。

```
dist = np.linalg.norm(pts, axis=1)
```

接下来，我们判断这些距离中哪些符合标准。该代码会返回一个布尔值数组：如果某个距离符合条件 `dist <= 1`，则该数组在该位置的值为 `True`。

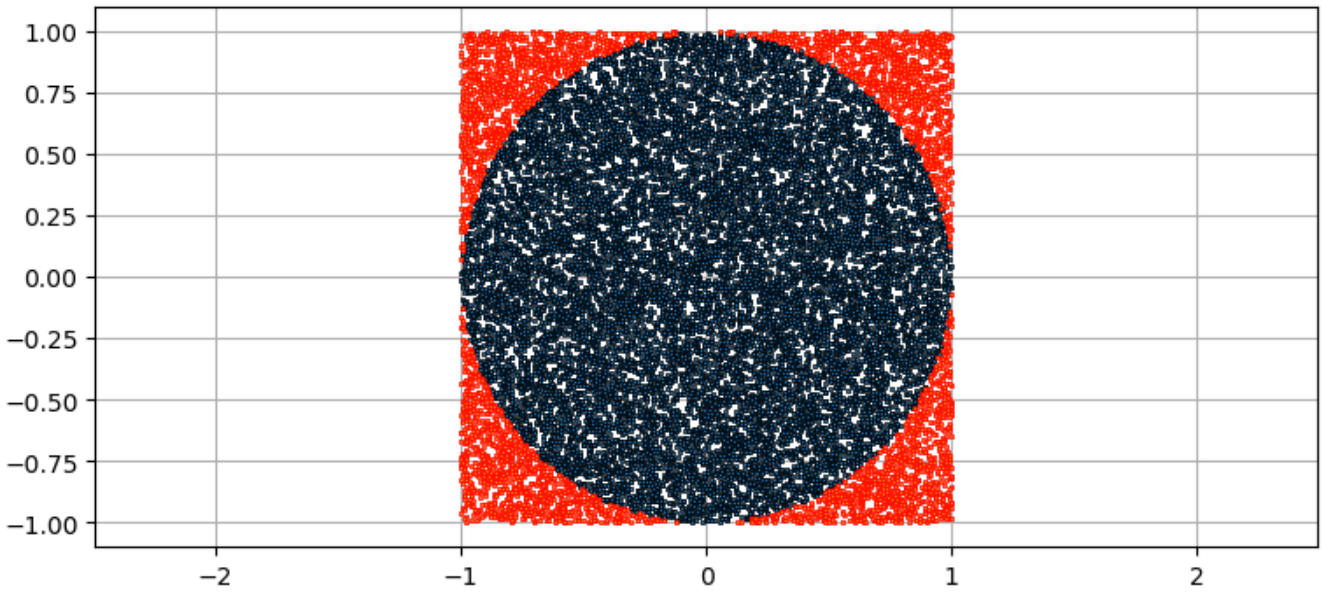
```
in_circle = dist <= 1
```

现在要做的就是计算圆内的点数、求出π的值，然后绘制出相应的图表。我把所有步骤都放在了一个单元格中，这样你就可以尝试使用不同的点数 `N` 来测试结果了。

```
import matplotlib.pyplot as plt
import numpy as np
from numpy.random import uniform

N = 20000  # number of points
radius = 1.
area = (2*radius)**2

pts = uniform(-1, 1, (N, 2))

# distance from (0,0)
dist = np.linalg.norm(pts, axis=1)
in_circle = dist <= 1

pts_in_circle = np.count_nonzero(in_circle)
pi = 4 * (pts_in_circle / N)
```

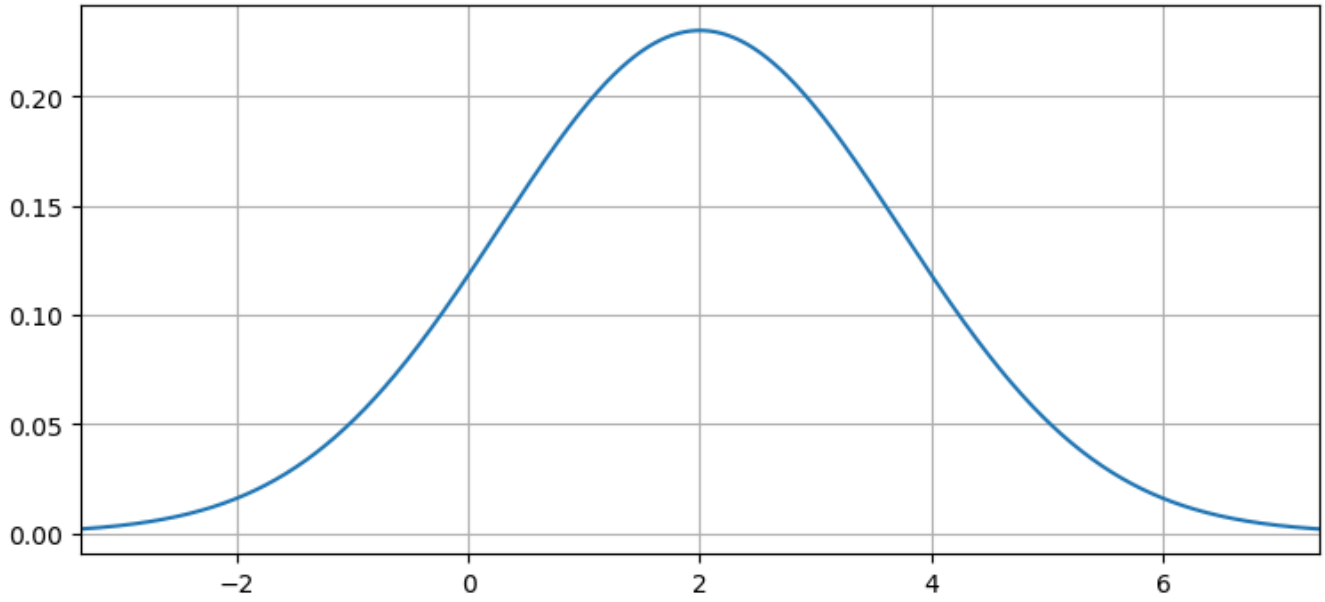
```
# plot results
plt.scatter(pts[in_circle,0], pts[in_circle,1],
            marker=',', edgecolor='k', s=1)
plt.scatter(pts[~in_circle,0], pts[~in_circle,1],
            marker=',', edgecolor='r', s=1)
plt.axis('equal')

print(f'mean pi(N={N})= {pi:.4f}')
print(f'err  pi(N={N})= {np.pi-pi:.4f}')
```

```
mean pi(N=20000)= 3.1684
err  pi(N=20000)= -0.0268
```



这一见解让我们意识到：我们可以利用蒙特卡洛方法来计算任意概率分布的概率密度。例如，假设我们有如下高斯分布：

```
from filterpy.stats import plot_gaussian_pdf
plot_gaussian_pdf(mean=2, variance=3);
```

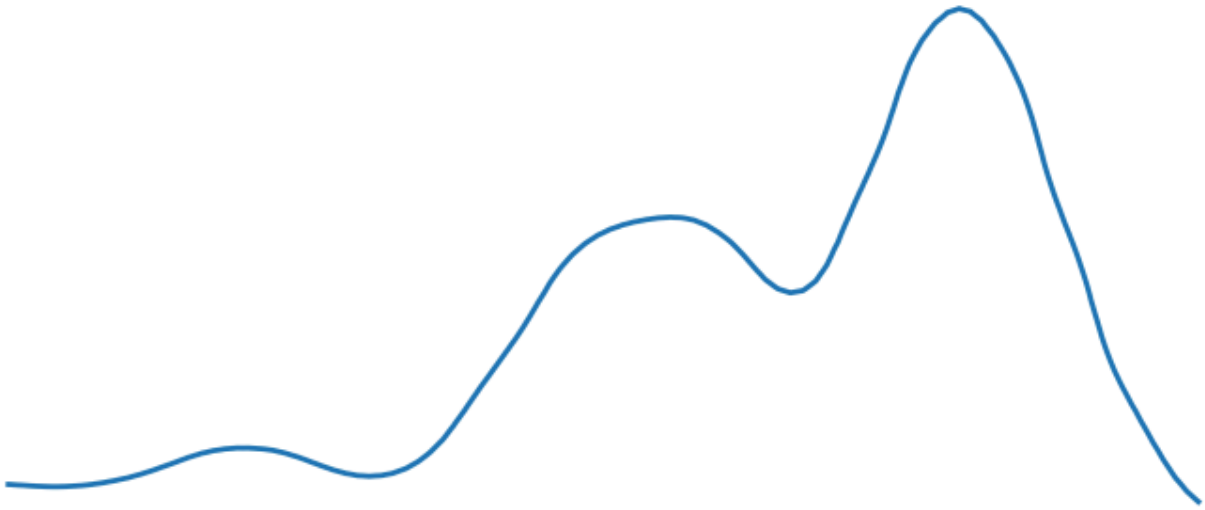


概率密度函数给出了随机变量落在两个特定数值之间的概率。例如，我们可能想知道在上图中，x 的数值位于 0 到 2 之间的概率。由于这是一个连续函数，我们需要通过积分来计算曲线下的面积。因为该面积恰好代表了该数值范围内事件发生的概率。

$$P[a \leq X \leq b] = \int_a^b f_X(x)dx$$

对于高斯函数来说，计算这个积分相当简单。但在现实生活中，情况并非如此简单。例如，下图展示了一个概率分布。我们无法用解析方法来描述任意的曲线，更不用说对其进行积分了。

```
pf_internal.plot_random_pd()
```



我们可以利用蒙特卡洛方法来计算任何积分。概率密度函数也是通过积分来计算的，因此，我们也可以用蒙特卡洛方法来计算这条曲线的概率密度函数。

粒子滤波器

所有这些内容都引出了粒子滤波器这一概念。以在城市环境中对机器人或汽车进行定位为例。为便于说明，我将继续使用在 EKF 和 UKF 章节中介绍的机器人定位问题作为示例。在这个问题中，我们需要追踪的机器人配备了能够测量到已知地标之间的距离和方向的传感器。

粒子滤波是一类算法。我在这里介绍的是一种易于理解的粒子滤波实现方式，这种实现方式与我们在这本书中研究的问题密切相关。不过，其中有一些步骤可能看起来有点“神奇”，因为我还没有给出完整的解释。这些解释会在本章的后面部分给出。

借鉴上一节的讨论，我们首先创建了数千个粒子。每个粒子的位置代表了机器人可能所处的位置；此外，每个粒子还可能具有方向和速度信息。假设我们完全不知道机器人的具体位置，那么我们应该将这些粒子均匀地分布在整个场景中。可以把所有的粒子看作是一种概率分布：粒子数量较多的地方，意味着我们对机器人位于该位置的置信度较高；粒子数量较少的地方，则意味着置信度较低。如果某个特定位置附近有大量的粒子，那就说明我们更确定机器人就位于那里。

每个粒子都需要一个权重——理想情况下，这个权重应代表该粒子所对应的位置是机器人真实位置的概率。不过，这个概率往往难以计算出来。因此，我们只需确保各个粒子的权重与那个可计算的概率成正比即可。在初始化阶段，我们没有理由优先选择某个粒子，所以我们会给所有的粒子都赋值为 $1 / N$。我们使用 $1 / N$ 这个数值，是为了确保所有概率之和等于 1。

粒子与权重的组合构成了我们问题中的概率分布。回想一下《离散贝叶斯》那一章的内容。在那一章中，我们将走廊中的各个位置视为离散的、等间距分布的点。这里的原理类似，只不过粒子是随机分布在连续空间中的，而不局限于离散的位置上。在这个问题中，机器人可以在任意维度的平面上移动，右下角的坐标为(0,0)。

为了追踪机器人的位置，我们需要记录下其 x、y 坐标以及行进方向。我们会将 `N` 个粒子存储在一个 `(N, 3)` 形状的数组中。该数组的前三列分别对应 x 坐标、y 坐标和行进方向。

如果你只是被动地跟踪某个对象（不进行任何控制操作），那么就需要将物体的速度也纳入状态参数中，并利用这一数值来进行预测。所需的维度越多，就需要相应地增加粒子的数量才能得到准确的预测结果。因此，我们总是尽量减少状态参数中随机变量的数量。

该代码会在某个区域内生成呈高斯分布的粒子群：

```
from numpy.random import uniform

def create_uniform_particles(x_range, y_range, hdg_range, N):
    particles = np.empty((N, 3))
    particles[:, 0] = uniform(x_range[0], x_range[1], size=N)
    particles[:, 1] = uniform(y_range[0], y_range[1], size=N)
    particles[:, 2] = uniform(hdg_range[0], hdg_range[1], size=N)
    particles[:, 2] %= 2 * np.pi
    return particles

def create_gaussian_particles(mean, std, N):
    particles = np.empty((N, 3))
    particles[:, 0] = mean[0] + (randn(N) * std[0])
    particles[:, 1] = mean[1] + (randn(N) * std[1])
    particles[:, 2] = mean[2] + (randn(N) * std[2])
```

```
    particles[:, 2] %= 2 * np.pi
    return particles
```

例如：

```
create_uniform_particles((0,1), (0,1), (0, np.pi*2), 4)
```

```
array([[0.772, 0.336, 4.171],
       [0.333, 0.34 , 4.319],
       [0.6  , 0.274, 5.02 ],
       [0.054, 0.022, 5.034]])
```

预测步骤

在贝叶斯算法中，预测步骤是利用系统模型来更新我们对系统状态的置信度。那么，我们该如何用粒子来实现这一过程呢？每个粒子都代表着机器人可能所处的位置。假设我们命令机器人移动 0.1 米，同时旋转 0.007 弧度。我们可以让每个粒子都按照这个量进行移动。不过，这样做很快就会遇到问题：机器人的控制并不完美，因此它的实际移动不会完全符合指令。因此，我们需要在粒子的移动过程中加入“噪声”，这样才能更准确地反映机器人的实际运动情况。如果不对系统中的不确定性进行建模，粒子滤波器就无法正确地描述我们对机器人位置的置信度分布。

```
def predict(particles, u, std, dt=1.):
    """ move according to control input u (heading change, velocity)
    with noise Q (std heading change, std velocity)`"""

    N = len(particles)
    # update heading
    particles[:, 2] += u[0] + (randn(N) * std[0])
    particles[:, 2] %= 2 * np.pi

    # move in the (noisy) commanded direction
    dist = (u[1] * dt) + (randn(N) * std[1])
    particles[:, 0] += np.cos(particles[:, 2]) * dist
    particles[:, 1] += np.sin(particles[:, 2]) * dist
```

更新步骤

接下来，我们会得到一组测量数据——每个当前可见的标志点都对应一组测量值。那么，该如何利用这些测量数据来调整由那些粒子所模拟出的概率分布呢?

回想一下关于离散贝叶斯的那一章。在那一章中，我们将走廊中的各个位置视为离散的、且间距均匀的点。我们为每个位置分配了一个概率值，称之为“先验概率”。当有新的测量数据出现时，我们会将该位置的当前先验概率与该位置符合测量结果的概率相乘。

```
def update(likelihood, prior):
    posterior = prior * likelihood
    return normalize(posterior)
```

这是该方程式的具体实现方式。

$$x = \|L\bar{x}\|$$

这其实体现了贝叶斯定理的原理：

$$\begin{aligned} P(x \mid z) &= \frac{P(z \mid x)P(x)}{P(z)} \\ &= \frac{likelihood \times prior}{normalization} \end{aligned}$$

我们对粒子也采用同样的处理方式。每个粒子都有一个位置值和一个权重值，该权重值用来衡量该粒子与实际测量结果的匹配程度。通过将所有粒子的权重值归一化，使其总和为 1，就可以将其转化为一个概率分布。通常来说，那些距离机器人最近的粒子的权重会更高，而那些距离机器人较远的粒子的权重则较低。

```
def update(particles, weights, z, R, landmarks):
    for i, landmark in enumerate(landmarks):
        distance = np.linalg.norm(particles[:, 0:2] - landmark, axis=1)
        weights *= scipy.stats.norm(distance, R).pdf(z[i])

    weights += 1.e-300      # avoid round-off to zero
    weights /= sum(weights) # normalize
```

在相关文献中，算法的这一部分被称为“顺序重要性抽样”，简称 SIS。用于确定各粒子权重的公式被称为“重要性密度”。我将在后面的章节中详细阐述这些理论基础。目前，希望大家能直观地理解这个概念。如果我们根据各粒子与测量结果的匹配程度来为它们赋予权重，那么这些粒子很可能是该系统在考虑了测量结果之后的概率分布的合理样本。理论也证明了这一点。在贝叶斯定理中，这些权重实际上代表了各种事件的“可能性”。当然，不同的问题可能需要以略微不同的方式来处理这一步骤，但总体思路是一样的。

计算状态估计值

在大多数应用中，你都希望能够在每次更新后得知估计的状态。不过，这个滤波器实际上只包含了一些粒子集合而已。假设我们正在追踪一个对象（即该状态是单峰的），那么我们可以把估计值的平均值计算为所有粒子的加权值之和。

$$\mu = \frac{1}{N}\sum_{i=1}^{N} w^i x^i$$

在这里，我使用符号 x^i 来表示 i^{th} 粒子。之所以使用上标，是因为我们通常需要用下标来表示时间步长。例如，表示 $k+1^{th}$ 时间步长时，如果使用下标的话，就会得到难以辨认的 x^i_{k+1} 这个符号。

该函数同时计算了各粒子的均值和方差：

```
def estimate(particles, weights):
    """returns mean and variance of the weighted particles"""

    pos = particles[:, 0:2]
    mean = np.average(pos, weights=weights, axis=0)
    var  = np.average((pos - mean)**2, weights=weights, axis=0)
    return mean, var
```

如果我们在 1x1 的正方形区域内以相等的权重来随机分布这些点，那么其平均位置会非常接近正方形的中心，即(0.5, 0.5)点。同时，该分布的方差也会很小。

```
particles = create_uniform_particles((0,1), (0,1), (0, 5), 1000)
weights = np.array([.25]*1000)
estimate(particles, weights)
```

```
(array([0.494, 0.514]), array([0.083, 0.085]))
```

粒子重采样

SIS 算法存在“退化问题”。该算法最初让所有粒子以相同的权重均匀分布。在机器人附近，可能只有少数几个粒子。随着算法的运行，那些与测量结果不匹配的粒子，其权重会变得极低。只有那些位于机器人附近的粒子，才能保持较高的权重。比如，可能有 5,000 个粒子，但其中只有 3 个粒子对状态估计有实际贡献！这种情况就被称为“算法退化”。为了解决这个问题，通常需要对粒子进行重新采样处理。

那些权重极小的粒子无法有效描述机器人的概率分布。重采样算法会丢弃那些概率极低的粒子，然后用概率较高的新粒子来替代它们。具体来说，就是复制那些概率较高的粒子。在预测过程中，这些被复制的粒子会因为噪声的影响而稍微发生偏移。这样一来，最终得到的粒子集合中，绝大多数粒子都能准确反映概率分布的情况。

有多种重采样算法。现在，我们先来看看其中最简单的一种。该算法会从当前的粒子集中随机抽取 N 次，从而得到一组新的粒子。在随机抽取过程中，每个粒子被选中的概率应与其权重成正比。

我们利用 NumPy 中的 `cumsum` 函数来实现这一目标。 `cumsum` 函数用于计算数组的累积和。也就是说，数组中的第一个元素是元素 0 和元素 1 的和，第二个元素是元素 0、1 和元素 2 的和，依此类推。接着，我们在 0.0 到 1.0 的范围内生成随机数，然后通过二分搜索来找到最接近该随机数的那个权重值。

```
def simple_resample(particles, weights):
    N = len(particles)
    cumulative_sum = np.cumsum(weights)
    cumulative_sum[-1] = 1. # avoid round-off error
    indexes = np.searchsorted(cumulative_sum, random(N))

    # resample according to indexes
    particles[:] = particles[indexes]
    weights.fill(1.0 / N)
```

我们并非在每个时间点都进行重采样。例如，如果你没有收到任何新的测量数据，那就意味着你没有任何可以用来进行重采样的信息。我们可以通过所谓的“有效 N 值”来决定何时进行重采样。所谓“有效 N 值”，其实就是指那些对概率分布有实际贡献的粒子数量。其计算公式为：

$$\hat{N}_{\text{eff}} = \frac{1}{\sum w^2}$$

我们可以在 Python 中用这种方式来实现它。

```
def neff(weights):
    return 1. / np.sum(np.square(weights))
```

如果 $\hat{N}_{\text{eff}}$ 的值低于某个阈值，那就需要重新采样了。一个合适的起始值是 $N/2$，不过这个数值因具体问题而异。也有可能出现 $\hat{N}_{\text{eff}} = N$ 的情况，这意味着所有粒子都聚集在同一个点上（此时每个粒子的权重都相等）。从理论上来说，这种情况并不理想。但如果真的发生了这种情况，我会重新生成一组粒子，希望能得到更多样化的粒子分布。如果你经常遇到这种情况，可能需要增加粒子数量或调整滤波器的设置。我们稍后会进一步讨论这个问题。

SIR 滤波器——一个完整的示例

虽然还有许多需要学习的内容，但就目前所掌握的知识而言，我们已经足以实现完整的粒子滤波算法了。我们将实现采样重要性重采样滤波器，也就是 SIR 算法。

我需要使用比上面所提到的更复杂的重采样方法。FilterPy 提供了多种重采样方式。我稍后会详细介绍这些方法。这些方法接收一个权重数组作为输入，然后返回那些被选中用于重抽样的粒子的索引。我们只需要编写一个函数，根据这些索引来执行重采样操作即可。

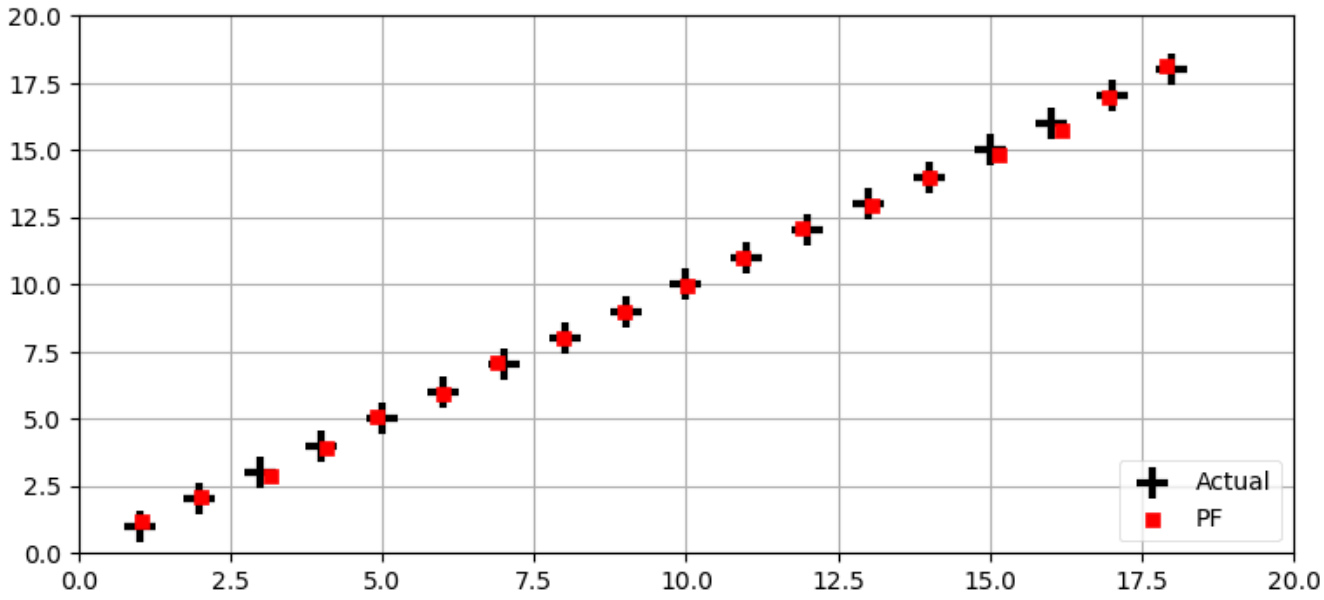
```
def resample_from_index(particles, weights, indexes):
    particles[:] = particles[indexes]
    weights.resize(len(particles))
    weights.fill (1.0 / len(weights))
```

为了实现该滤波器，我们需要先创建粒子以及各个参考点。接着，我们进入循环处理：依次调用 `predict`、`update`、重新采样函数，最后再利用 `estimate` 来计算新的状态估计值。

```
from filterpy.monte_carlo import systematic_resample
from numpy.linalg import norm
from numpy.random import randn
import scipy.stats

def run_pf1(N, iters=18, sensor_std_err=.1,
            do_plot=True, plot_particles=False,
            xlim=(0, 20), ylim=(0, 20),
            initial_x=None):
    landmarks = np.array([[-1, 2], [5, 10], [12,14], [18,21]])
    NL = len(landmarks)

    plt.figure()

    # create particles and weights
    if initial_x is not None:
        particles = create_gaussian_particles(
            mean=initial_x, std=(5, 5, np.pi/4), N=N)
    else:
        particles = create_uniform_particles((0,20), (0,20), (0, 6.28), N)
    weights = np.ones(N) / N

    if plot_particles:
        alpha = .20
        if N > 5000:
            alpha *= np.sqrt(5000)/np.sqrt(N)
```

```
        plt.scatter(particles[:, 0], particles[:, 1],
                    alpha=alpha, color='g')

    xs = []
    robot_pos = np.array([0., 0.])
    for x in range(iters):
        robot_pos += (1, 1)

        # distance from robot to each landmark
        zs = (norm(landmarks - robot_pos, axis=1) +
              (randn(NL) * sensor_std_err))

        # move diagonally forward to (x+1, x+1)
        predict(particles, u=(0.00, 1.414), std=(.2, .05))

        # incorporate measurements
        update(particles, weights, z=zs, R=sensor_std_err,
               landmarks=landmarks)

        # resample if too few effective particles
        if neff(weights) < N/2:
            indexes = systematic_resample(weights)
            resample_from_index(particles, weights, indexes)
            assert np.allclose(weights, 1/N)
        mu, var = estimate(particles, weights)
        xs.append(mu)

        if plot_particles:
            plt.scatter(particles[:, 0], particles[:, 1],
                        color='k', marker=',', s=1)
        p1 = plt.scatter(robot_pos[0], robot_pos[1], marker='+',
                         color='k', s=180, lw=3)
        p2 = plt.scatter(mu[0], mu[1], marker='s', color='r')

    xs = np.array(xs)
    #plt.plot(xs[:, 0], xs[:, 1])
    plt.legend([p1, p2], ['Actual', 'PF'], loc=4, numpoints=1)
    plt.xlim(*xlim)
    plt.ylim(*ylim)
    print('final position error, variance:\n\t', mu - np.array([iters, iters]), var)
    plt.show()

from numpy.random import seed
seed(2)
run_pf1(N=5000, plot_particles=False)
```

```
final position error, variance:
     [-0.106  0.106] [0.009 0.008]
```



这段代码的大部分内容都用于初始化和数据绘图。整个粒子滤波处理过程实际上就由这几行代码构成而已。

```
# move diagonally forward to (x+1, x+1)
predict(particles, u=(0.00, 1.414), std=(.2, .05))

 # incorporate measurements
update(particles, weights, z=zs, R=sensor_std_err,
       landmarks=landmarks)

# resample if too few effective particles
if neff(weights) < N/2:
    indexes = systematic_resample(weights)
    resample_from_index(particles, weights, indexes)

mu, var = estimate(particles, weights)
```

第一行代码预测了粒子的位置。在计算过程中，假设机器人沿直线运动（`u[0] == 0`），并且在 x 轴和 y 轴上各移动 1 个单位距离（`u[1]==1.414`）。转弯时的误差标准差为 0.2，而距离测量的误差标准差为 0.05。当该代码执行完毕时，所有粒子都会向前移动，但由于权重值没有得到更新，因此这些权重值已经不再准确了。

下一行代码将测量结果纳入了滤波过程中。这一操作不会改变粒子的位置，只会调整它们的权重。如果你还记得的话，粒子的权重实际上代表了该粒子与传感器误差模型所描述的高斯分布的匹配程度。粒子与实际测量值相差越远，它作为有效候选值的概率就越低。

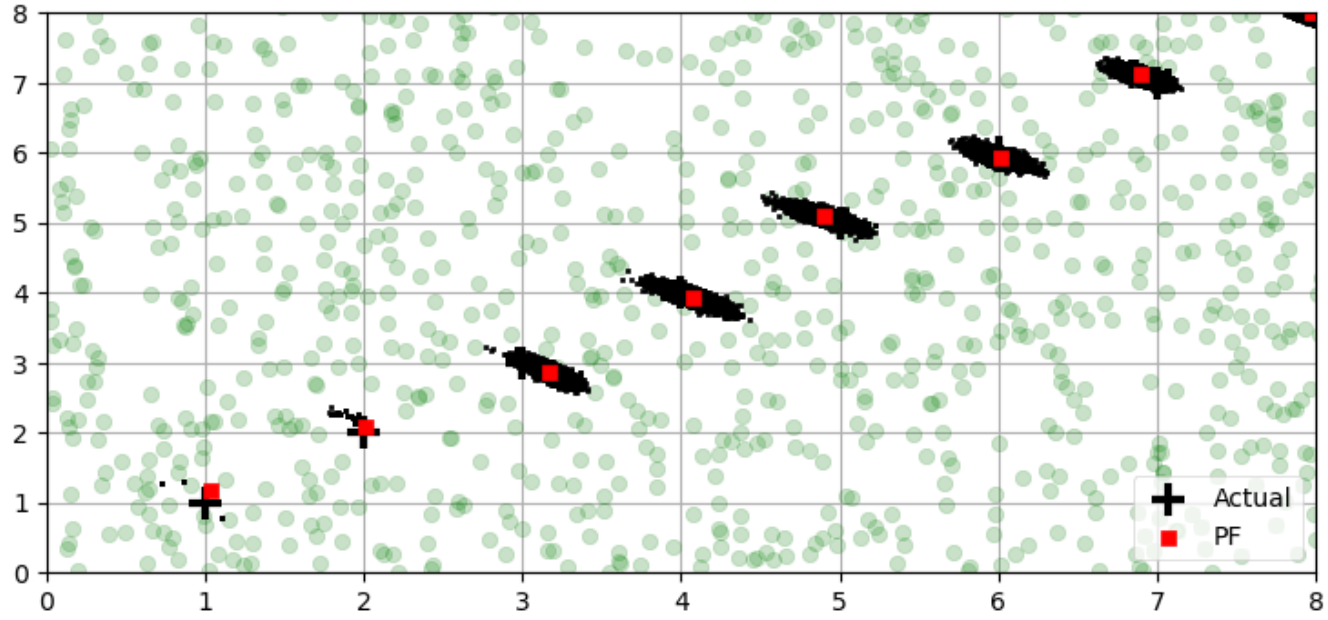
最后两行展示了有效的粒子数量（$\hat{N}_{\text{eff}}$）。如果该数值低于 $N/2$，我们将重新进行采样处理，以确保所生成的粒子能够准确反映实际的概率分布情况。

现在，让我们在绘制出所有粒子之后来观察这一现象。通过交互式的方式查看这一过程会更加直观明了，不过这种形式仍然能为我们提供有用的信息。我使用了非常浅绿色的圆圈来表示原始的随机分布点，以便与后续迭代中用黑色像素表示的粒子区分开来。由于粒子数量较多，很难看清细节，因此我将迭代次数限制在了 8 次，这样我们就可以放大视角，更仔细地观察了。

```
seed(2)
run_pf1(N=5000, iters=8, plot_particles=True,
```

```
        xlim=(0,8), ylim=(0,8))
```

```
final position error, variance:
     [-0.019 -0.005] [0.005 0.006]
```



从图表上看，前两个机器人位置处似乎只有少数几个粒子。但实际上并非如此：那里有 5,000 个粒子，但由于重采样处理，其中大部分都是重复的粒子。造成这种情况的原因是，传感器的高斯分布范围非常狭窄。这种现象被称为“样本贫乏”，可能导致滤波器失效。我将在下文中详细解释这一点。目前来看，当 x=2 时，粒子们已经有所分散。这种分散是由于运动模型中的噪声所导致的。所有粒子都根据控制输入向前移动，但由于机器人控制机制中的误差，每个粒子还会受到额外的噪声影响。到了第三步时，粒子们已经充分分散，从而在机器人周围形成了一个清晰的粒子云。

粒子云的形状是椭圆形的。这并非偶然。由于传感器和机器人控制机制都被视为高斯分布，因此整个系统的概率分布也是高斯分布。粒子滤波器实际上是对该概率分布的采样，所以粒子云自然呈现为椭圆形。

需要认识到的是，粒子滤波算法并不要求传感器或系统的行为符合高斯分布或线性规律。由于我们用粒子云来表示概率分布，因此该算法能够处理各种概率分布以及高度非线性的问题。在概率模型中，完全可以存在不连续性或硬性限制。

传感器误差对滤波器的影响

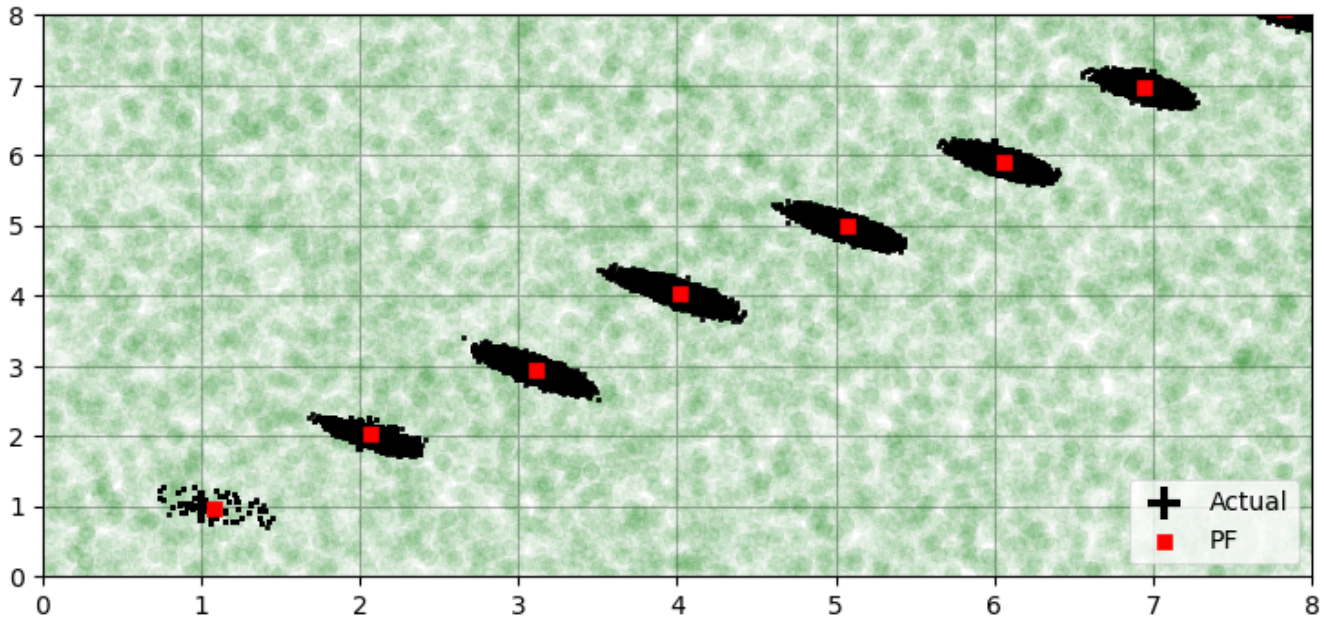
在滤波器的最初几次迭代过程中，产生了大量重复的粒子。出现这种情况的原因是：传感器的模型属于高斯模型，而我们给定了较小的标准差 $\sigma = 0.1$ 。乍看之下，这种设定似乎不合常理。通常情况下，当噪声较小时，卡尔曼滤波器的性能会更好；但粒子滤波器的性能反而可能更差。

我们可以推理出为什么会出现这种情况。如果 $\sigma = 0.1$ ，那么机器人位于(1, 1)点，而粒子位于(2, 2)点。此时，该粒子距离机器人有 14 个标准差之远。因此，该粒子对平均值估计几乎没有影响，而且在重新采样后，该粒子几乎不可能被保留下来。如果 $\sigma = 1.4$ ，那么该粒子与机器人的距离仅为 1σ ，因此它会对平均值估计有所贡献。在重新采样过程中，该粒子很可能会被复制一次或多次。

理解这一点非常重要：如果传感器的精度过高，那么滤波器的性能就会变差，因为只有极少数的粒子能够真正反映概率分布的情况。我们有一些解决办法。首先，可以人为增加传感器的噪声标准差，这样粒子滤波器就会接受更多符合机器人概率分布的点。不过，这种做法并非最佳方案，因为其中一些点其实并不符合概率分布。真正的问题在于，生成的点数量不足，导致没有足够的点位于机器人附近。增加粒子数量通常能解决这个问题。不过，这样做会带来一定的成本，因为粒子数量的增加会显著增加计算时间。不过，让我们来看看使用 100,000 个粒子时的效果吧。

```
seed(2)
run_pf1(N=100000, iters=8, plot_particles=True,
        xlim=(0,8), ylim=(0,8))
```

```
final position error, variance:
     [-0.17   0.084] [0.005 0.005]
```



在 x=1 的位置，有更多的粒子存在；而在 x=2 的位置，则形成了较为明显的“云状结构”。显然，这种滤波方式的效果更好，但代价是巨大的内存消耗和较长的处理时间。

另一种方法是更巧妙地生成初始粒子云。假设我们判断机器人位于(0, 0)附近。虽然这一判断并不完全准确，因为实际上机器人位于(1, 1)，但已经很接近了。如果在(0, 0)附近生成一个正态分布的粒子云，那么这些粒子就更有可能与机器人的实际位置相符。

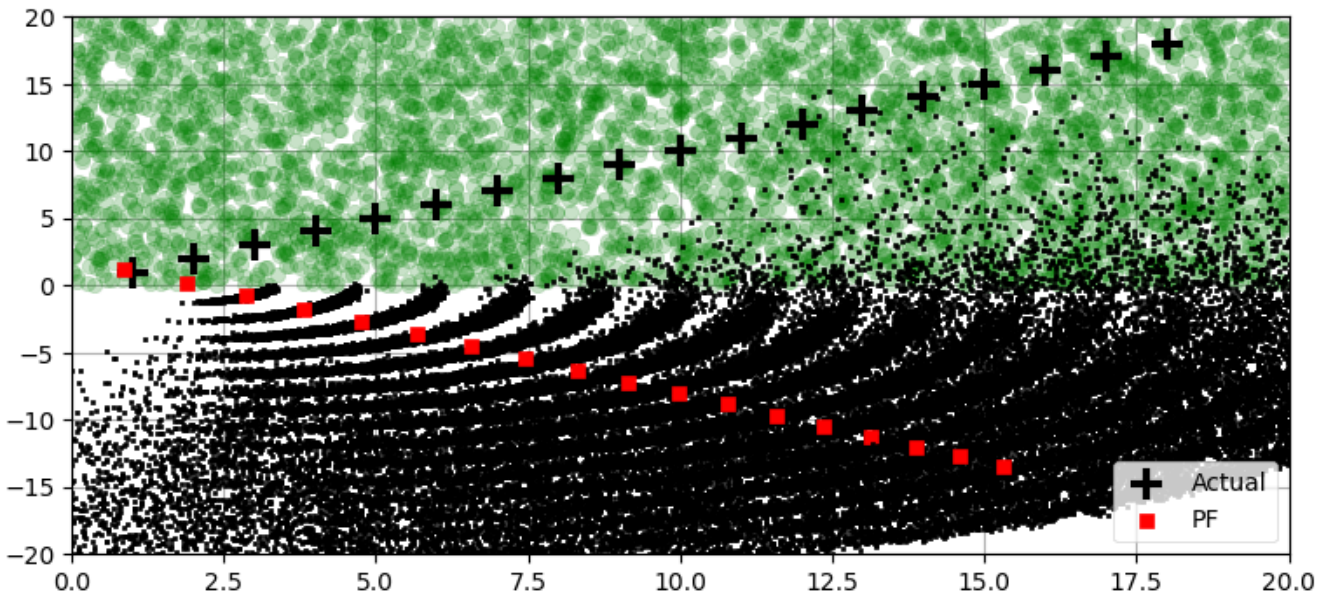
`run_pf1()` 具有一个可选参数 `initial_x` 。该参数用于指定机器人的初始位置估计值。之后，代码会利用 `create_gaussian_particles(mean, std, N)` 来在初始位置周围生成呈正态分布的粒子。我们将在下一节中使用这个功能。

因样本不足导致的滤波器性能下降问题

按照当前的实现方式，这个滤波器的效果远称不上完美。下面是使用不同随机种子时的测试结果。

```
seed(6)
run_pf1(N=5000, plot_particles=True, ylim=(-20, 20))
```

```
final position error, variance:
     [ -2.688 -31.47 ] [47.065 47.03 ]
```

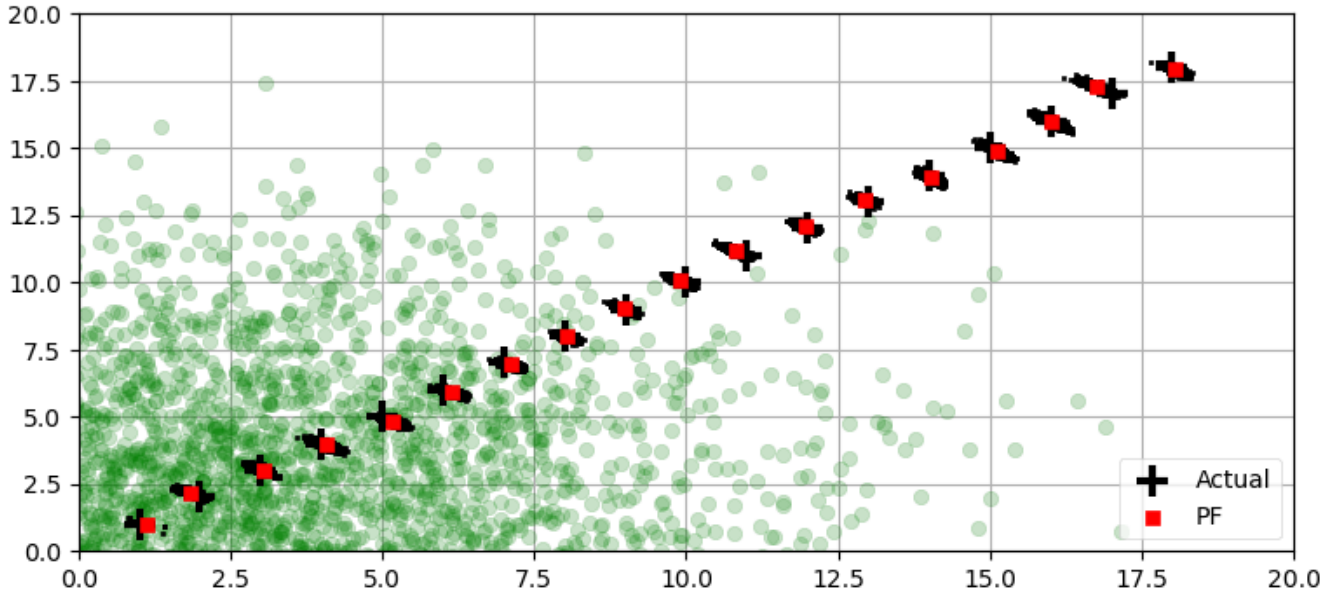


在这里，最初的样本点并没有在机器人附近生成任何新的点。在重采样过程中，粒子滤波器不会生成新的点，因此那些点会被重复使用。这样一来，这些点就无法真实反映概率分布的情况。正如前面所提到的，这种现象被称为“样本贫乏”。这个问题会迅速变得难以控制。这些粒子无法准确反映实际情况，因此它们会以一种高度非线性的方式分布开来，最终导致粒子滤波器失去对现实的准确描述。由于机器人附近没有可用的粒子，因此粒子滤波器永远无法达到收敛状态。

让我们利用 `create_gaussian_particles()` 方法来尝试在机器人附近生成更多的点。我们可以通过设置 `initial_x` 参数来指定生成粒子的位置。

```
seed(6)
run_pf1(N=5000, plot_particles=True, initial_x=(1,1, np.pi/4))
```

```
final position error, variance:
     [ 0.035 -0.077] [0.007 0.009]
```



这个方法效果很好。如果能够大致确定初始位置的话，最好尽量在那个位置附近生成粒子。不过也别过于谨慎——如果所有粒子都集中在某个位置附近，那么就无法充分捕捉系统中的非线性特征。毕竟，这是一个相当线性的系统，因此可以允许粒子分布的方差稍大一些。当然，这还要视具体问题而定。增加粒子数量确实能获得更好的样本，但相应的处理成本也可能超出你的承受范围。

重要性抽样

我试图用简单的方法来回避这个我们必须面对的难题。实际上，存在某种概率分布来描述机器人的位置和运动情况。我们希望从该概率分布中抽取一些样本点，然后利用蒙特卡洛方法来计算相关的积分值。

我们的难题在于：在很多情况下，我们并不知道数据的分布情况。例如，被追踪物体的运动方式可能与我们根据状态模型所预测的完全不同。既然不知道概率分布，那我们该如何从中抽取样本呢?

在统计学中，有一个被称为“重要性抽样”的定理[1]。有趣的是，这个定理为我们提供了一种从另一个已知概率分布中抽取样本的方法，进而利用这些样本来推断出那个未知概率分布的各个特性。这真是一个了不起的定理，让我感到非常高兴。

这个想法很简单，我们之前就已经用过这种方法了。我们从已知的概率分布中抽取样本，但会根据我们感兴趣的分布来给这些样本赋予不同的权重。这样，我们就可以通过计算这些样本的加权平均值和加权方差，从而得到诸如均值和方差之类的统计量。

在处理机器人的定位问题时，我们从状态模型预测结果所对应的概率分布中抽取了样本。换句话说，我们的推理是：“机器人当时就在那个位置，它可能正以这样的方向和速度移动，所以它现在很可能就在那儿。”不过，机器人也有可能做出完全不同的行为：它可能从悬崖上掉下来，或者被迫击炮击中。无论哪种情况，所对应的概率分布都是不准确的。看起来我们似乎遇到了难题，但实际上并非如此，因为我们还可以使用重要性抽样法。我们从那个可能有误的概率分布中抽取样本，然后根据这些样本与实际测量数据的匹配程度来给它们赋予权重。这种加权方式是基于真实的概率分布进行的，因此从理论上讲，最终计算出的均值、方差等数值应该是准确的!

这怎么可能呢？我来给你详细解释一下数学原理。如果你不想深入研究机器人定位问题，可以跳过这部分内容。不过，对于其他涉及粒子滤波的问题，就需要采用不同的重要性抽样方法，此时数学知识就非常重要了。此外，大多数文献和网络上的资料都采用数学公式来阐述这些概念，而不是我那种不太精确的“想象一下……”的说法。如果你想真正理解这些文献，就必须掌握以下公式。

我们有一个概率分布 $\pi(x)$ ，希望从中抽取样本。不过，我们不知道 $\pi(x)$ 的具体数值；我们只知道另一个概率分布 $q(x)$ 。在机器人定位的背景下， $\pi(x)$ 代表了机器人的概率分布，但我们并不知道它的具体数值。而 $q(x)$ 则是我们所能知道的、与测量结果相关的概率分布。

具有概率分布 $\pi(x)$ 的函数 $f(x)$ 的期望值是

$$E\left[f(x)\right] = \int f(x)\pi(x)dx$$

我们不知道 $\pi(x)$ 的值，因此无法计算这个积分。不过，我们知道另一种分布方式 $q(x)$ ，因此可以将它加到积分中，而不会改变积分的值。

$$E\left[f(x)\right] = \int f(x)\pi(x)\frac{q(x)}{q(x)}dx$$

现在我们来重新排列并整理各项。

$$E\left[f(x)\right] = \int f(x)q(x)\cdot\frac{\pi(x)}{q(x)}dx$$

我们已知 $q(x)$ 的值，因此可以利用 MC 积分来计算 $\int f(x)q(x)$ 。这样就只剩下 $\pi(x)\ /\ q(x)$ 了。 $\pi(x)\ /\ q(x)$ 其实是一个比率，我们将其定义为一种“权重”。这样我们就得到了所需的结果。

$$E\left[f(x)\right] = \sum_{i=1}^{N} f(x^i)q(x^i)w(x^i)$$

也许这听起来有点抽象。如果我们想计算这些粒子的平均值，那么我们需要进行相应的计算。

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x^i w^i$$

这就是我在本章前面给出的方程式。

要求各权重与比例 $\pi(x)\ /\ q(x)$ 成正比。由于我们通常无法知道该比例的具体数值，因此在实际操作中，我们会通过将各权重除以 $\sum w(x^i)$ 来进行归一化处理。

在构建粒子滤波算法时，你需要根据具体情境来实施这一步骤。在机器人定位任务中，用于表示 $q(x)$ 的最佳分布，就是滤波过程中所生成的粒子分布。让我们再看一下代码吧：

```
def update(particles, weights, z, R, landmarks):
    for i, landmark in enumerate(landmarks):
        dist = np.linalg.norm(particles[:, 0:2] - landmark, axis=1)
        weights *= scipy.stats.norm(dist, R).pdf(z[i])

    weights += 1.e-300      # avoid round-off to zero
    weights /= sum(weights) # normalize
```

在这里，我们根据贝叶斯计算方法来确定权重值。 $\|\text{likelihood} \times \text{prior}\|$

当然，如果你能够根据先验概率分布来计算后验概率分布，那就应该这么做。如果你无法做到这一点，那么重要性抽样就能帮助你解决这个问题。实际上，计算后验概率分布极其困难。卡尔曼滤波之所以能够取得巨大成功，是因为它利用了高斯分布的特性，从而找到了解析解。一旦我们放宽卡尔曼滤波所要求的条件（马尔可夫性、高斯测量值以及高斯过程），那么重要性抽样和蒙特卡洛方法就能让这个问题变得容易处理了。

重采样方法

重采样算法会影响到滤波器的性能。例如，如果我们随机选择粒子来进行重采样，那么就会选中许多权重极低的粒子。这样一来，最终得到的粒子集合就无法准确反映该问题的概率分布情况。

关于这个主题的研究仍在继续，不过有几种算法在各种情况下都表现良好。我们希望找到一种具备多种优良特性的算法：它应优先选择那些概率较高的粒子；同时，应选取足够多的高概率粒子，以避免样本的不足；此外，还应该包含一定数量的低概率粒子，以便让该算法能够检测到那些强烈的非线性行为。

FilterPy 实现了几种常用的算法。由于 FilterPy 并不知道你的粒子滤波器是如何实现的，因此它无法自行生成新的样本。相反，这些算法会生成一个包含被选中粒子索引的列表。你需要自行完成重采样操作。例如，我在开发机器人相关程序时就是这么做的。

```
def resample_from_index(particles, weights, indexes):
    particles[:] = particles[indexes]
    weights.resize(len(particles))
    weights.fill(1.0 / len(weights))
```

多项式重采样

多项式重采样是我在开发机器人定位相关功能时所使用的算法。其原理很简单：只需计算各权重值的累积和即可。这样就能得到一个从 0 到 1 的数值序列。下图展示了权重值是如何分布的。图中的颜色并无实际意义，只是为了便于区分各个数值段而已。

```
from kf_book.pf_internal import plot_cumsum
print('cumulative sume is', np.cumsum([.1, .2, .1, .6]))
plot_cumsum([.1, .2, .1, .6])
```

```
cumulative sume is [0.1 0.3 0.4 1. ]
```

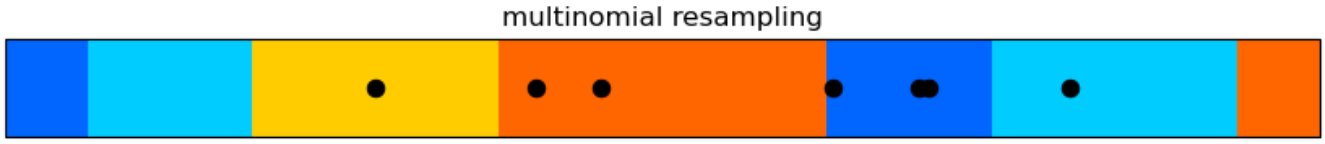
为了确定每个元素的权重，我们会随机生成一个介于 0 和 1 之间的数值，然后利用二分搜索法来确定该数值在累积和数组中的位置。权重较大的元素所占用的空间较大，因此被选中的概率也更高。

使用 NumPy 的 ufunc 功能，实现这一功能非常简单。Ufunc 功能可以将函数应用于数组中的每个元素，从而得到一个包含所有计算结果的数组。 `searchsorted` 就是 NumPy 中的二分搜索算法。如果你向该函数提供一个包含搜索值的数组，它将返回一个包含对应搜索结果的数组——每个搜索值都对应一个结果。

```
def multinomal_resample(weights):
    cumulative_sum = np.cumsum(weights)
    cumulative_sum[-1] = 1.  # avoid round-off errors
    return np.searchsorted(cumulative_sum, random(len(weights)))
```

以下是一个例子：

```
from kf_book.pf_internal import plot_multinomial_resample
plot_multinomial_resample([.1, .2, .3, .4, .2, .3, .1])
```



这是一种 $O(n\log(n))$ 算法。虽然不算太差，但还有 $O(n)$ 种重采样算法在样本的均匀性方面表现更好。我之所以展示这种算法，是为了让你能理解其他算法其实都是基于这种算法衍生而来的。其实，还有更高效的实现方式，那就是利用该分布的累积分布函数的倒数来进行重采样。如果你感兴趣的话，可以在网上搜索相关资料。

使用以下方式从 FilterPy 中导入该函数：

```
from filterpy.monte_carlo import multinomal_resample
```

残差重采样

残差重采样不仅有助于缩短多项式重抽样的运行时间，还能确保对所有粒子的抽样都是均匀的。这一方法的巧妙之处在于：首先将权重进行归一化处理，然后再用每个归一化后的权重来确定该粒子应被抽取的样本数量。例如，如果某个粒子的权重为 0.0012，而 N 的值为 3000，那么该粒子的归一化后权重为 3.6，因此该粒子将被抽取 3 次。这样一来，所有权重较高的粒子都至少会被抽取一次。该方法的运行时间为 $O(N)$ ，因此比多项式重抽样更快。

不过，这样并不能选出全部 N 个粒子。为了选出剩下的粒子，我们需要取剩余值：即权重减去其整数部分，这样就得到了小数部分。接着，我们使用更简单的抽样方法，比如多项式抽样，根据这个剩余值来选出剩下的粒子。在上面的例子中，加权后的数值为 3.6，因此剩余值为 0.6（3.6 - int(3.6)）。由于这个剩余值较大，该粒子很可能会被再次选中。这是合理的，因为剩余值越大，四舍五入时的误差就越大，这意味着该粒子在整数步长下被选中的概率相对较低。

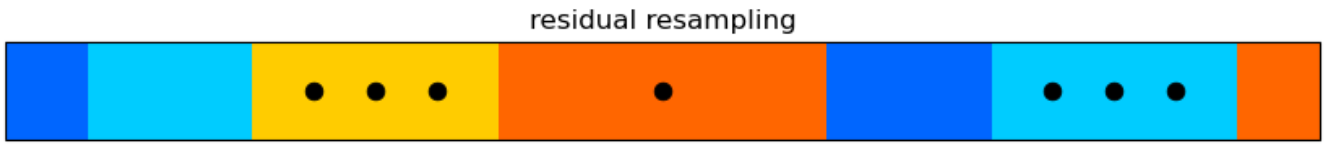
```
def residual_resample(weights):
    N = len(weights)
    indexes = np.zeros(N, 'i')

    # take int(N*w) copies of each weight
    num_copies = (N*np.asarray(weights)).astype(int)
```

```
    k = 0
    for i in range(N):
        for _ in range(num_copies[i]): # make n copies
            indexes[k] = i
            k += 1

    # use multinormial resample on the residual to fill up the rest.
    residual = w - num_copies     # get fractional part
    residual /= sum(residual)     # normalize
    cumulative_sum = np.cumsum(residual)
    cumulative_sum[-1] = 1. # ensures sum is exactly one
    indexes[k:N] = np.searchsorted(cumulative_sum, random(N-k))

    return indexes
```

你可能会想用切片 `indexes[k:k + num_copies[i]] = i` 来替代内部的 for 循环，但实际上，过短的切片处理起来比较慢，而 for 循环的运行速度通常更快。

我们来看一个例子：

```
from kf_book.pf_internal import plot_residual_resample
plot_residual_resample([.1, .2, .3, .4, .2, .3, .1])
```



你可以使用 FilterPy 来导入这个模块。

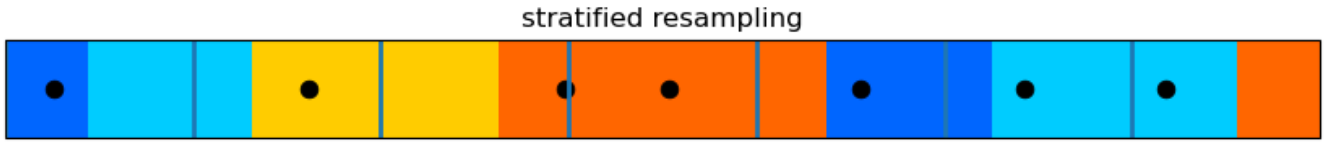
```
from filterpy.monte_carlo import residual_resample
```

分层重采样

该方案的目的是让各个粒子被选中的概率尽可能均匀。具体而言，先将累积和分成 N 个相等的部分，然后从每个部分中随机选择一个粒子。这样一来，就能确保每个样本之间的间隔都在 0 到 $\frac{2}{N}$ 之间。

下图对此进行了说明。彩色条形图表示了该数组的累积值，黑色线条则代表了各个等分的区域。那些用黑色圆圈表示的粒子，是随机分布在各个分区中的。

```
from kf_book.pf_internal import plot_stratified_resample
plot_stratified_resample([.1, .2, .3, .4, .2, .3, .1])
```



用于执行分层处理的代码相当简单易懂。

```
def stratified_resample(weights):
    N = len(weights)
    # make N subdivisions, chose a random position within each one
    positions = (random(N) + range(N)) / N

    indexes = np.zeros(N, 'i')
    cumulative_sum = np.cumsum(weights)
    i, j = 0, 0
    while i < N:
        if positions[i] < cumulative_sum[j]:
            indexes[i] = j
            i += 1
        else:
            j += 1
    return indexes
```

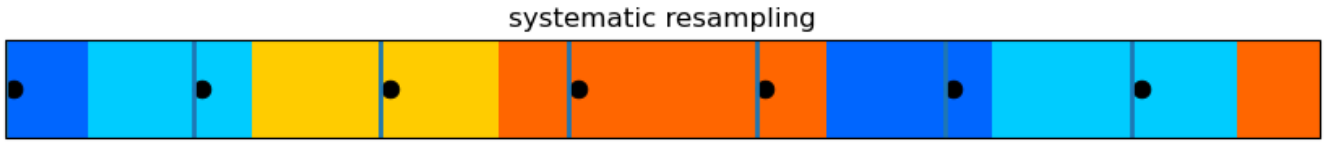
通过 FilterPy 将其导入。

```
from filterpy.monte_carlo import stratified_resample
```

系统化重采样

我们最后要介绍的算法是系统重采样。与分层重采样类似，该算法也是将整体空间划分为若干个区域。接着，我们为每个区域选择一个随机偏移量，确保所有样本之间的间距都恰好为 $\frac{1}{N}$。具体操作方式如下所示。

```
from kf_book.pf_internal import plot_systematic_resample
plot_systematic_resample([.1, .2, .3, .4, .2, .3, .1])
```



看了前面的例子后就会发现，这段代码简直再简单不过了。

```
def systematic_resample(weights):
    N = len(weights)

    # make N subdivisions, choose positions
    # with a consistent random offset
    positions = (np.arange(N) + random()) / N

    indexes = np.zeros(N, 'i')
    cumulative_sum = np.cumsum(weights)
    i, j = 0, 0
    while i < N:
        if positions[i] < cumulative_sum[j]:
            indexes[i] = j
            i += 1
        else:
            j += 1
    return indexes
```

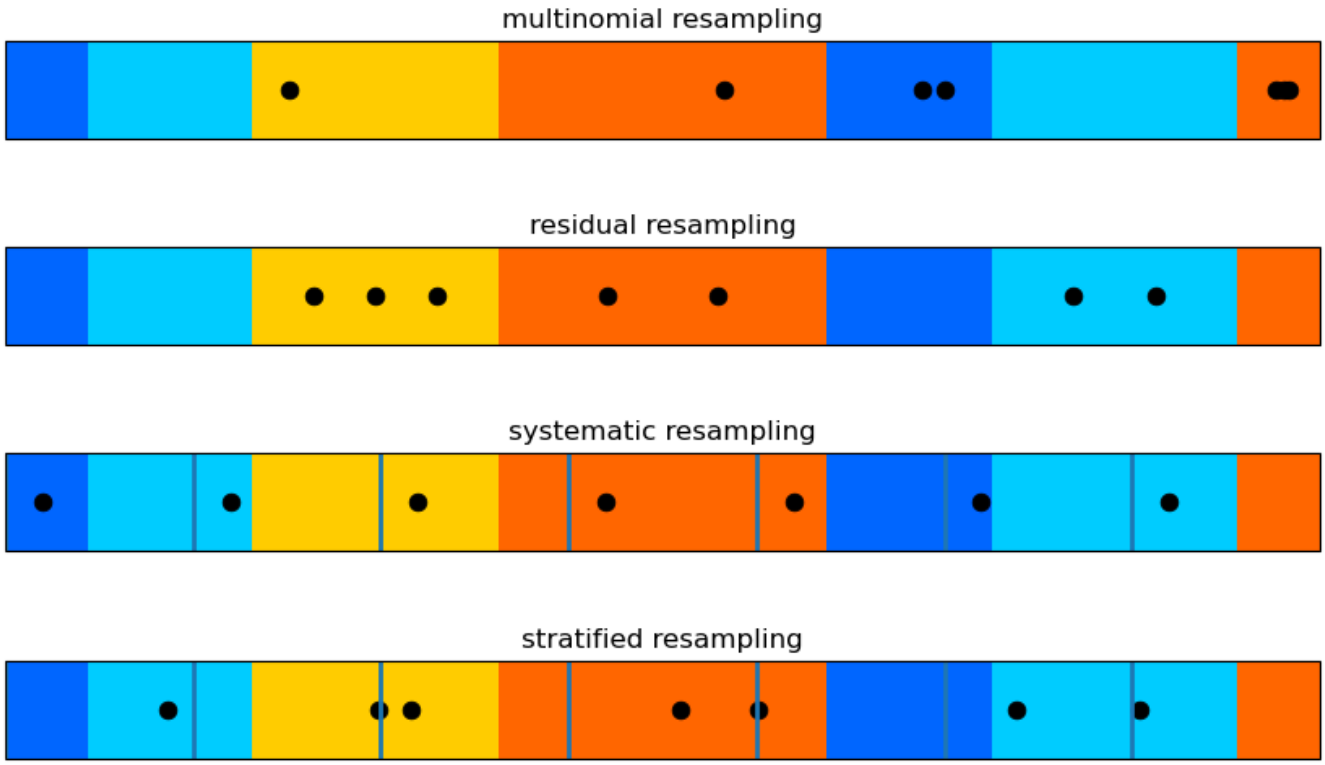
通过 FilterPy 进行导入

```
from filterpy.monte_carlo import systematic_resample
```

选择重采样算法

我们一起来看一下这四种算法吧，这样比较起来会更容易些。

```
a = [.1, .2, .3, .4, .2, .3, .1]
np.random.seed(4)
plot_multinomial_resample(a)
plot_residual_resample(a)
plot_systematic_resample(a)
plot_stratified_resample(a)
```



多项式重采样的效果相当糟糕。有一个权重值根本没有被采样到。最大的权重值只被采样了一次，而最小的权重值则被采样了两次。我读过的网上大多数教程都使用了多项式重采样方法，不过我不明白为什么。在学术文献或实际应用中，多项式重采样很少被使用。除非有充分的理由，否则建议不要使用这种方法。

这种残差重采样算法在实现其目标方面表现得相当出色：它确保了所有较大的权重都能被多次重采样。不过，该算法并没有将样本均匀地分配到各个粒子上——许多数值较大的权重根本就没有被重采样。

无论是系统抽样还是分层抽样，效果都相当不错。系统抽样能够确保我们从粒子空间的各个部分进行采样，同时还能确保那些权重较大的样本被更频繁地重新采样。虽然分层抽样的均匀性不如系统抽样，但在确保高权重样本被更频繁地重新采样方面，分层抽样的效果稍好一些。

关于这些算法的理论性能，已有大量研究文献，大家可以自行查阅。在实际应用中，我通常将粒子滤波算法用于那些难以用分析方法解决的问题。因此，我对针对这些问题的具体分析方法的可靠性持怀疑态度。实际上，无论是分层算法还是系统化算法，在处理各种问题时都能取得不错的效果。我的建议是：先尝试其中一种算法，如果有效果，那就继续使用它。如果算法的性能至关重要，那就两种都试试看。同时，也可以查阅一下关于你所处理问题的相关文献，以便获得更有效的指导。

摘要/总结

本章只是对这个庞大主题的浅尝辄止。我的目的并非要教授你们整个领域的知识，而是让你们了解用于数据滤波的实际贝叶斯蒙特卡洛技术。

粒子滤波是一种集成滤波方法。卡尔曼滤波用高斯分布来表示系统状态。根据贝叶斯定理，将测量结果应用于该高斯分布，然后利用状态空间方法进行预测。这些技术都是针对高斯分布这一概率分布形式来应用的。

相比之下，集成技术则是通过一组离散的点以及与这些点相关的概率来表示概率分布。各种测量结果都是针对这些点来进行的，而不是针对高斯分布。同样，系统模型也是应用于这些点，而非高斯分布。之后，我们再计算由这些点构成的整体所具有的统计特性。

这些选择都伴随着各种权衡。卡尔曼滤波器效率极高，只要满足线性和高斯噪声的假设条件，它就是一种最优的估计方法。但如果问题具有非线性特征，就必须先对问题进行线性化处理。另外，如果需要跟踪多个目标，卡尔曼滤波器也无法胜任这一任务。卡尔曼滤波器要求用户必须了解系统的状态模型。如果无法了解系统的行为方式，那么该滤波器的性能就会很差。

相比之下，粒子滤波器可以处理任何形式的、非解析的概率分布。只要粒子数量足够多，这些粒子就能很好地近似出真实的概率分布。即便存在严重的非线性情况，粒子滤波器也能表现出色。通过重要性抽样，即使我们不知道具体的概率分布，也能计算出相应的概率值。蒙特卡洛技术则取代了其他滤波器所必需的解析积分运算。

这种能力当然需要付出相应的代价。最明显的代价就是，该滤波器会给计算机带来沉重的计算和内存负担。另一个不太明显的代价是，这些滤波器的稳定性较差。必须小心避免粒子状态的退化或发散现象。要验证滤波器的正确性其实相当困难。如果处理的是多模态分布，还需要进一步对粒子进行分类，以确定各个物体的运动轨迹。当各个物体彼此靠近时，这一过程会变得极其复杂。

粒子滤波器有很多不同的类型；我在这里只介绍了最简单的 SIS 算法，随后又介绍了性能较好的 SIR 算法。实际上，粒子滤波器有很多种类，每种类型下又有许多具体的实现方式。要详细描述所有这些算法，恐怕需要写一本完整的书了。

在阅读有关粒子滤波的文献时，你会发现其中充斥着各种积分运算。我们在处理概率分布时也会使用积分，因此，积分这种表示方式让作者能够更简洁、高效地表达数学概念。你需要明白的是，当把这些数学公式转化为代码时，实际上是用粒子来表示各种概率分布，而积分则被替换为对粒子的求和操作。只要记住本章的核心思想，这些内容其实并不难理解。

参考文献

[1] 重要性抽样，维基百科。https://en.wikipedia.org/wiki/Importance_sampling

第 13 章：平滑

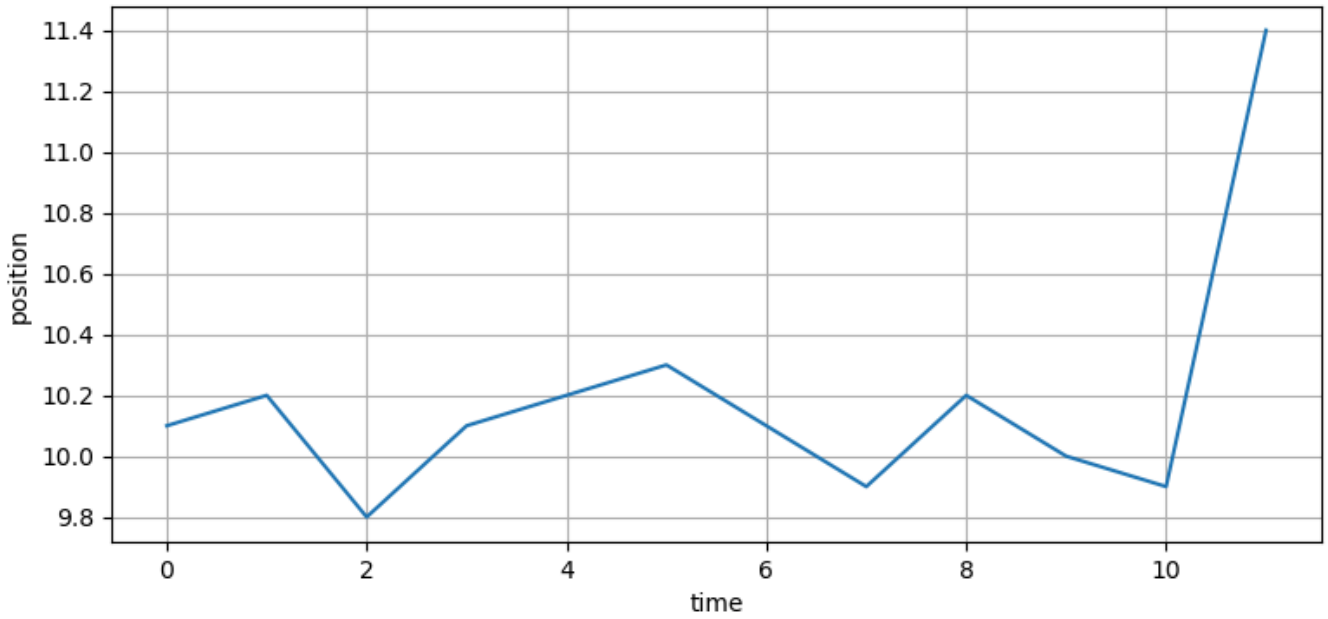
```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

当需要考虑未来数据时，卡尔曼滤波器的性能并不理想。例如，假设我们正在追踪一架飞机，而最新的测量结果与当前轨迹相差很大，如下图所示（为简化起见，这里只考虑一维情况）：

```
import matplotlib.pyplot as plt

data = [10.1, 10.2, 9.8, 10.1, 10.2, 10.3,
        10.1, 9.9, 10.2, 10.0, 9.9, 11.4]
plt.plot(data)
plt.xlabel('time')
plt.ylabel('position');
```



在经历了一段相对稳定的状态之后，出现了巨大的变化。假设这种变化已经超出了飞机的飞行极限范围。不过，卡尔曼滤波器仍然会根据当前的卡尔曼增益，将这一新测量结果纳入滤波处理中。由于该测量结果可能反映了飞机开始转弯的信号，因此卡尔曼滤波器无法将其视为噪声而予以忽略。当然，飞机如此突然地转弯的可能性不大，但我们也无法确定这一点。

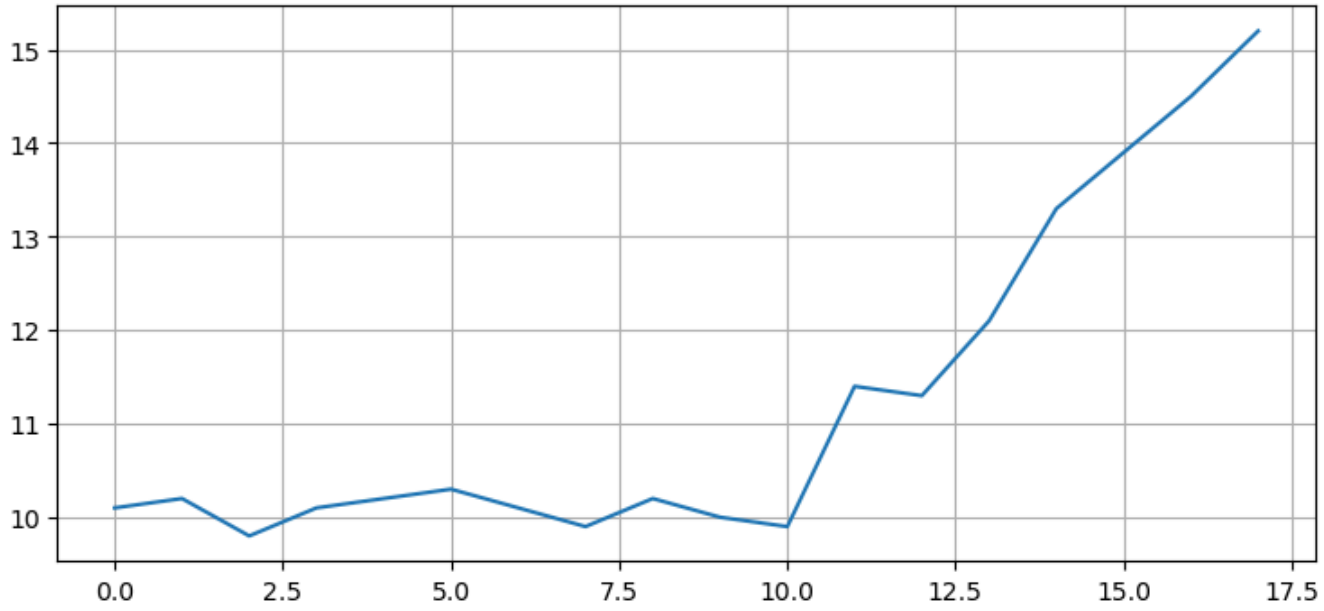
- 该飞机不久前开始转弯，但之前的测量数据存在较大误差，无法准确反映这一变化。

- 飞机正在转弯，因此这个测量结果存在很大的误差/数据不够准确。
- 测量结果存在很大误差，而且飞机并没有发生转向。
- 那架飞机正在朝相反的方向转弯，而测量数据非常不稳定。

现在，假设我们有以下测量数据：

11.3 12.1 13.3 13.9 14.5 15.2

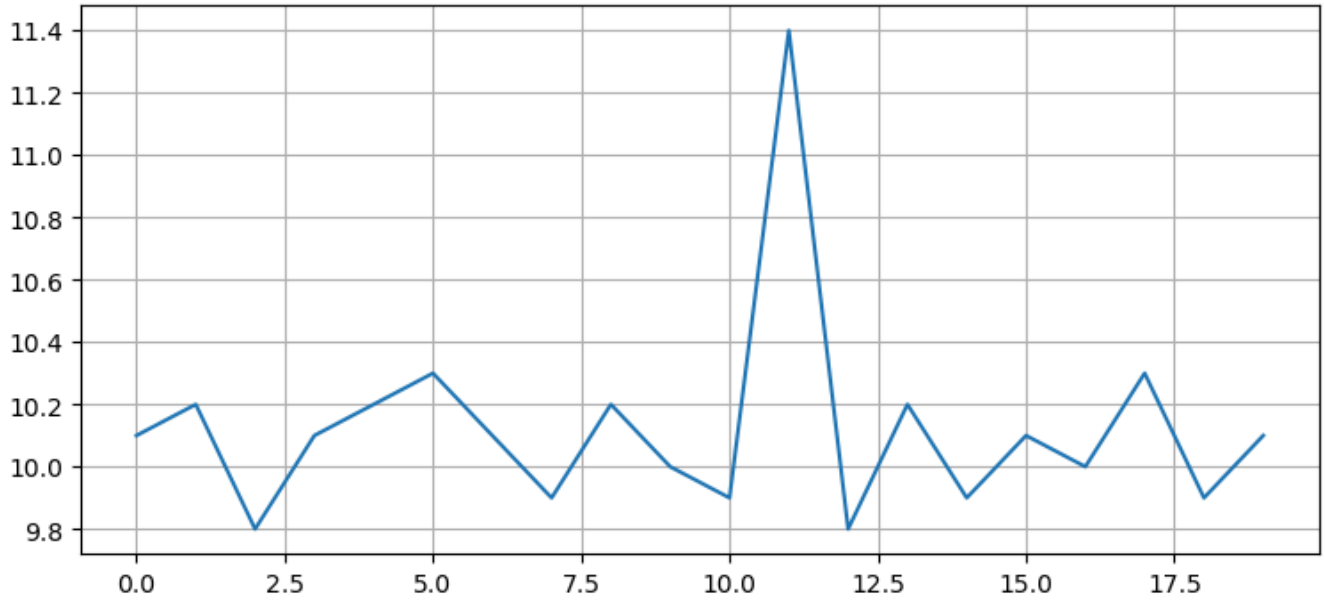
```
data2 = [11.3, 12.1, 13.3, 13.9, 14.5, 15.2]
plt.plot(data + data2);
```



根据这些未来的测量数据，我们可以推断，确实，那架飞机开始转向了。

另一方面，假设这些是以下的测量结果。

```
data3 = [9.8, 10.2, 9.9, 10.1, 10.0, 10.3, 9.9, 10.1]
plt.plot(data + data3);
```



在这种情况下，我们可以得出结论：该飞机并没有改变方向，而那些异常的测量数据其实只是因为噪声干扰造成的。

平滑处理功能的运作原理概述

卡尔曼滤波器是一种具有马尔可夫性质的递归滤波器。其在第 0 步的估计值仅基于第 1 步的估计值以及第 2 步的测量结果。不过，这意味着第 3 步的估计值又以第 4 步的估计值为基础，依此类推，直到第一个时间点。因此，第 5 步的估计值实际上取决于所有之前的测量结果，只不过影响程度各不相同。其中，第 6 步的测量结果影响最大，第 7 步次之，依此类推。

平滑滤波器会将未来的测量数据纳入到步骤 `k` 的估算中。其中，来自 `k+1` 的测量数据会产生最大的影响， `k+2` 的影响次之， `k+3` 的影响则更小，以此类推。

这个主题被称为“平滑处理”，但我认为这个名称有些误导。我可以通过使用低通滤波器来平滑上述数据。这样做可以得到平滑的结果，但准确性并不高，因为低通滤波器会同时消除真实的变化和噪声。相比之下，卡尔曼平滑器则属于最优解决方案——它们利用了所有可用的信息，从而得出在数学上可行的最佳估计结果。

研磨器的类型

在这种情况下，有三种类型的卡尔曼平滑器可以实现更好的跟踪效果。

- 固定间隔平滑法

这是一种基于批量处理的滤波器。这种滤波器会在收集完所有数据之后才会进行任何估计。例如，假设你是一名科学家，正在收集实验数据，那么在实验完成之前，你并不需要知道实验结果。固定间隔平滑器会收集所有数据，然后利用所有可用的过去和未来的测量数据来估计每个时刻的状态。如果你能够以批量模式运行卡尔曼滤波器，那么强烈建议使用这种滤波器，因为它会提供比前面章节中介绍的递归式滤波器更好的结果。

- 固定滞后平滑处理

固定延迟平滑处理方式会在输出结果中引入一定的延迟。假设我们选择 4 步的延迟时间。此时，滤波器会接收前 3 个测量值，但不会立即输出处理后的结果。当第 4 个测量值进入滤波器后，滤波器才会根据 1 到 4 个测量值来计算出第 1 个测量值的处理结果。当第 5 个测量值进入滤波器后，滤波器又会根据 2 到 5 个测量值来计算出第 2 个测量值的处理结果。当需要使用最新数据，但可以接受一定程度的延迟时，这种处理方式非常有用。例如，如果你使用机器视觉来监控生产过程，即使允许有几秒钟的延迟，使用固定延迟平滑处理方式也能得到非常准确且平滑的结果。

- 定点平滑处理

固定点滤波器的工作方式与普通的卡尔曼滤波器类似，但它还会在某个固定时间点 j 处给出该时刻的状态估计值。在时间 k 到达 j 之前，滤波器会像普通滤波器一样运作。一旦时间达到 $k > j$，滤波器会先估计出 x_k 的值，然后再利用从 $j \dots k$ 开始的所有测量数据来更新对 x_j 的估计值。这种方法有助于确定系统的初始参数，或者准确估计在特定时间发生的事件。例如，假设有一台机器人在时间 j 时拍摄了照片。那么，就可以利用固定点滤波器来获取机器人在时间 j 时相机的最佳姿态信息。

滤镜的选择

这些滤波器的选择取决于你的需求，以及你能够分配多少内存和处理时间。固定点平滑算法需要存储所有的测量数据，其计算成本很高，因为每个时间步长的输出都需要重新计算。不过，这种滤波器能够为当前测量值提供不错的结果，因此它可以用于实时应用。

固定滞后平滑法只需要存储一定时间范围内的数据即可。由于每次只需处理该时间窗口内的数据，因此其处理要求相对较低。不过，这种方法的缺点是：滤波器的输出结果总是滞后于输入数据，而且平滑效果也不如固定间隔平滑法那么显著。

固定间隔平滑法虽然能得到最平滑的处理结果，但缺点是需要进行批量处理。大多数算法都采用某种前向/后向算法来处理数据，其效率仅为递归卡尔曼滤波器的两倍而已。

固定间隔平滑法

在相关文献中，有许多种用于处理数据延迟问题的平滑算法。我选择使用 Rauch、Tung 和 Striebel 所提出的平滑算法，因为它易于实现且计算效率较高。实际上，这也是在实际应用中最常被使用的平滑算法。这种平滑算法通常被称为 RTS 平滑算法。

RTS 平滑算法的推导过程涉及好几页密密麻麻的数学公式。我就不把这些内容呈现给你们了。相反，我会简要介绍该算法和相关的数学公式，然后直接展示该平滑算法的实际应用和效果。

RTS 平滑算法的工作原理是：首先以批量处理的方式运行卡尔曼滤波器，计算出每一步的滤波结果。在得到每次测量的滤波结果以及与这些结果相对应的协方差矩阵后，RTS 会反向处理这些数据，将其对未来的预测结果纳入到对过去的测量数据中。当处理到第一次测量结果时，处理过程就完成了。此时，经过滤波后的结果已经包含了所有可能的信息，达到了最优状态。

RTS 平滑器的计算公式非常简单，易于实现。这里的推导是针对线性卡尔曼滤波器而言的。对于 EKF 和 UKF，也有类似的推导方法。这些计算是在批量处理的结果上进行的，从最新的数据开始，逐步回溯到最初的估计值。每次迭代都会将关于未来的信息纳入状态估计中。由于状态估计已经包含了所有的历史测量数据，因此每次估计都会包含关于过去和未来所有测量数据的信息。在这里，区分过去、现在和未来非常重要，所以我使用下标来标明数据是来自未来还是过去。

Predict Step

$$P \quad = FP_kF^T + Q$$

Update Step

$$\begin{aligned} K_k &= P_kF^TP^{-1} \\ x_k &= x_k + K_k(x_{k+1} - Fx_k) \\ P_k &= P_k + K_k(P_{k+1} - P)K_k^T \end{aligned}$$

和往常一样，实现过程中最困难的部分就是如何正确处理下标问题。如果不加注释也不进行错误检查，那么最简单的实现方式如下：

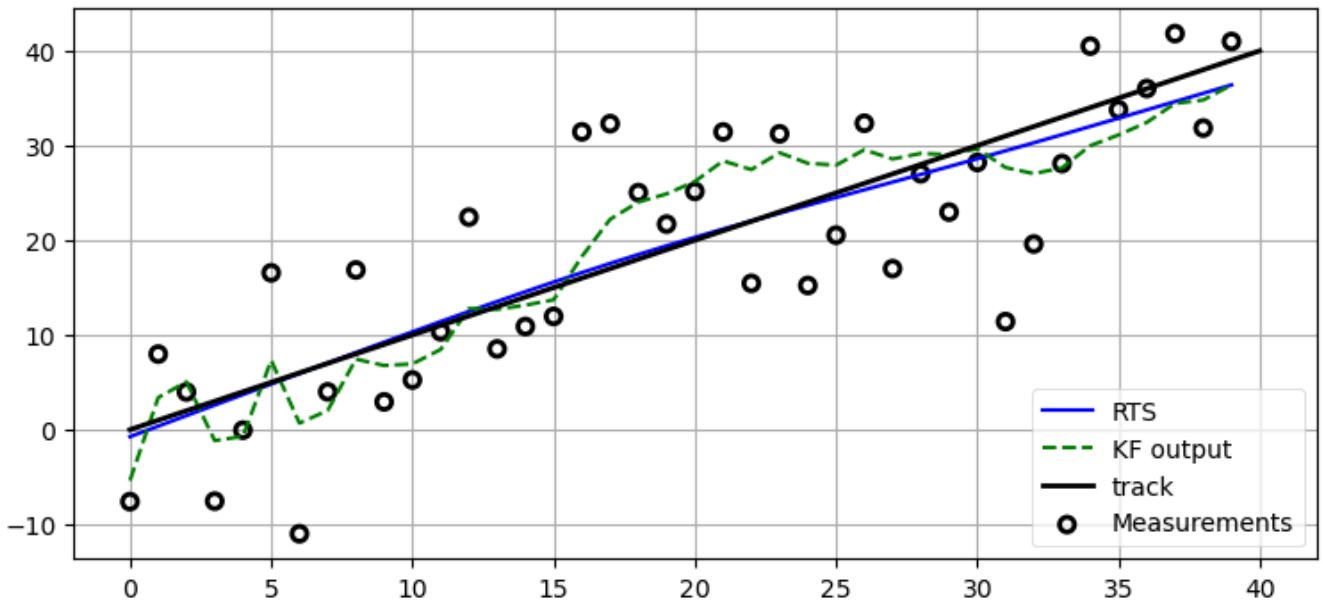
```
def rts_smoother(Xs, Ps, F, Q):
    n, dim_x, _ = Xs.shape

    # smoother gain
    K = zeros((n,dim_x, dim_x))
    x, P, Pp = Xs.copy(), Ps.copy(), Ps.copy

    for k in range(n-2,-1,-1):
        Pp[k] = F @ P[k] @ F.T + Q # predicted covariance

        K[k]  = P[k] @ F.T @ inv(Pp[k])
        x[k] += K[k] @ (x[k+1] - (F @ x[k]))
        P[k] += K[k] @ (P[k+1] - Pp[k]) @ K[k].T
    return (x, P, K, Pp)
```

该实现方式与 FilterPy 中的实现方式相同。它假定卡尔曼滤波器是在外部以批量模式运行的，状态值和协方差的结果则通过 `Xs` 和 `Ps` 变量传递进来。

以下是一个例子。

```
import numpy as np
from numpy import random
from numpy.random import randn
import matplotlib.pyplot as plt
from filterpy.kalman import KalmanFilter
from filterpy.common import Q_discrete_white_noise
import kf_book.book_plots as bp

def plot_rts(noise, Q=0.001, show_velocity=False):
    random.seed(123)
```

```
    fk = KalmanFilter(dim_x=2, dim_z=1)

    fk.x = np.array([0., 1.])      # state (x and dx)

    fk.F = np.array([[1., 1.],
                     [0., 1.]])    # state transition matrix

    fk.H = np.array([[1., 0.]])    # Measurement function
    fk.P*= 10.                     # covariance matrix
    fk.R = noise                   # state uncertainty
    fk.Q = Q_discrete_white_noise(dim=2, dt=1., var=Q)  # process uncertainty

    # create noisy data
    zs = np.asarray([t + randn()*noise for t in range (40)])

    # filter data with Kalman filter, than run smoother on it
    mu, cov, _, _ = fk.batch_filter(zs)
    M, P, C, _ = fk.rts_smoother(mu, cov)

    # plot data
    if show_velocity:
        index = 1
        print('gu')
    else:
        index = 0
    if not show_velocity:
        bp.plot_measurements(zs, lw=1)
    plt.plot(M[:, index], c='b', label='RTS')
    plt.plot(mu[:, index], c='g', ls='--', label='KF output')
    if not show_velocity:
        N = len(zs)
        plt.plot([0, N], [0, N], 'k', lw=2, label='track')
    plt.legend(loc=4)
    plt.show()

plot_rts(7.)
```

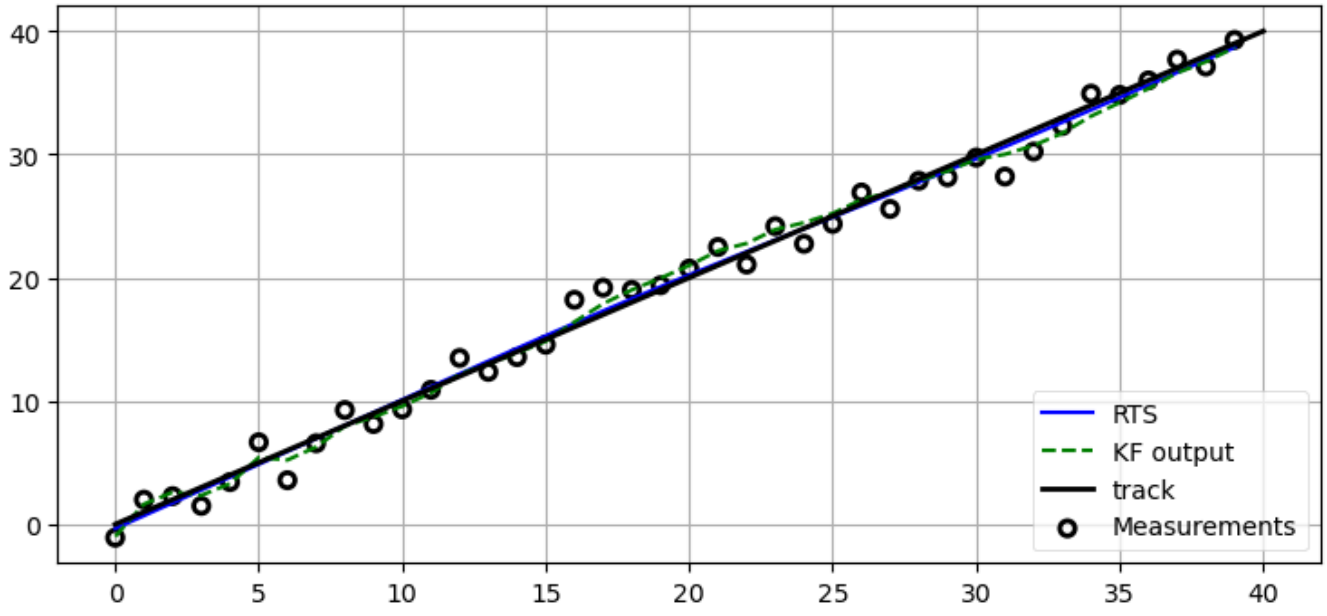


我在信号中加入了大量的噪声，以便您能够清晰地区分 RTS 输出与理想输出之间的差异。在上面的图表中，我们可以看到卡尔曼滤波器以绿色虚线表示，其曲线相对于输入信号来说相当平滑。但是，当连续几次测量结果

都偏向曲线的某一侧时，卡尔曼滤波器的曲线就会偏离理想线。相比之下，RTS 输出的曲线则非常平滑，并且非常接近理想输出值。

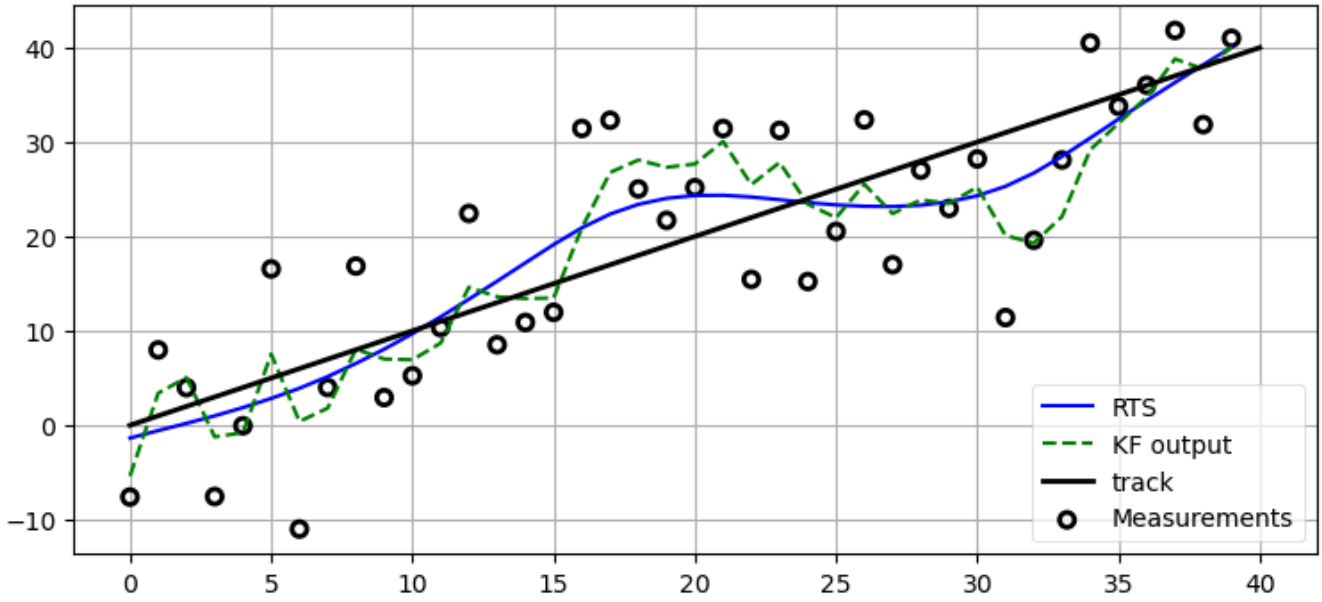
在噪声水平相对较低的情况下，我们可以看到，RTS 的输出几乎与理想输出一致。而卡尔曼滤波器的输出虽然有所改善，但仍然存在较大的偏差。

```
plot_rts(noise=1.)
```



不过，我们必须明白，这种平滑处理是基于系统模型来进行的。我们告诉滤波器：我们所追踪的对象遵循恒定速度的运动规律，且其运动过程中的误差极小。当滤波器预测未来趋势时，会发现未来的运动情况确实符合恒定速度的模式，因此它能够滤除信号中的大部分噪声。但如果系统的噪声较大呢？例如，如果我们试图在狂风中追踪一架轻型飞机，那么飞机的速度会频繁变化。此时，滤波器就很难区分噪声和由风引起的异常运动了。我们可以在下图中看到这一现象。

```
plot_rts(noise=7., Q=.1)
```

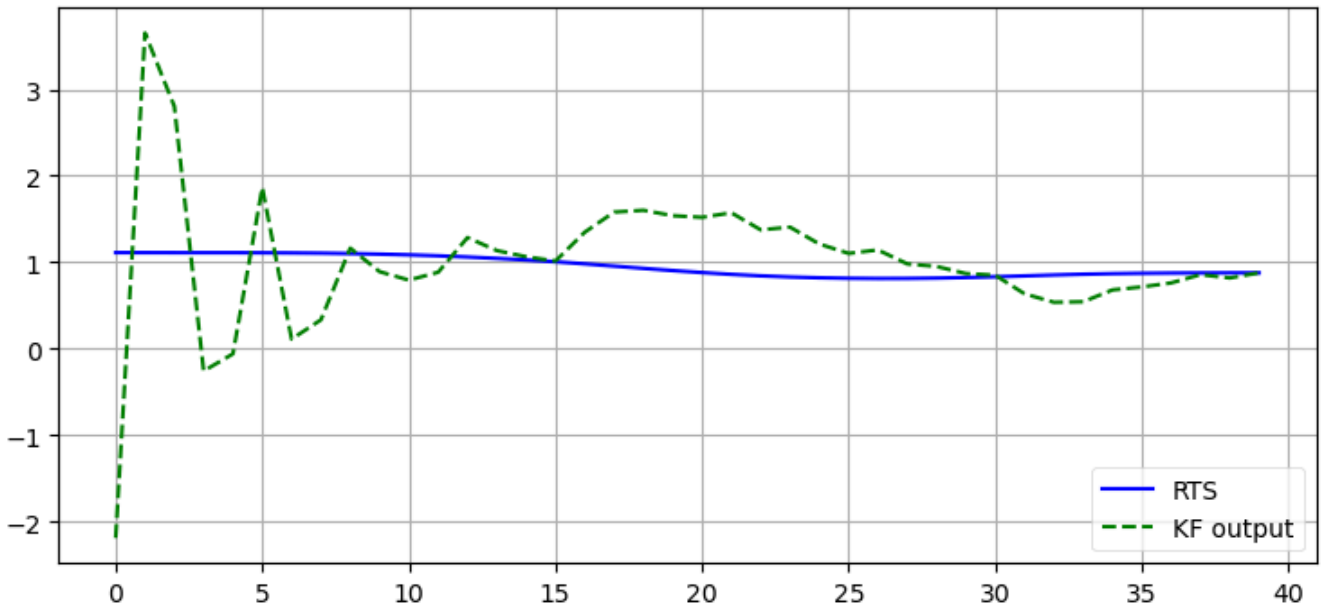


这表明，这些滤波器并非以人们通常所理解的那种方式来“平滑”数据。实际上，滤波器是根据之前的测量结果、未来的测量数据，以及用户所提供的关于系统行为、系统中的噪声以及测量过程中的各种因素的信息，来做出最优化的估算。

最后，我们来比较一下卡尔曼滤波器和 RTS 平滑算法在速度估算方面的表现。

```
plot_rts(7.,show_velocity=True)
```

```
gu
```

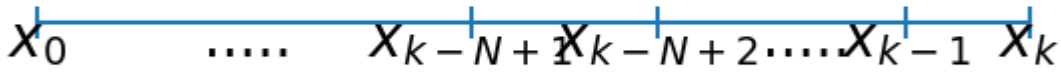


作为隐藏变量的速度方面的改善则更为显著。

固定滞后平滑法

如果您能够以批处理模式来运行程序，那么上述的 RTS 平滑算法无疑应该是您的最佳选择。因为该算法会将所有可用的数据都纳入到每次的估算结果中。不过，并非所有情况都允许这样做。不过，您可能仍然希望获得之前各次估算的平滑处理后的数值。下图有助于理解这一概念。

```
from kf_book.book_plots import figsize
from kf_book.smoothing_internal import *

with figsize(y=2):
    show_fixed_lag_numberline()
```



在步骤 k 时，我们可以利用标准的卡尔曼滤波公式来估算 x_k 的值。不过，如果我们利用与 x_k 相关的测量数据，就能更准确地估算出 x_{k-1} 的值。同样地，利用与 x_{k-1} 和 x_k 相关的测量数据，我们就能更准确地估算出 x_{k-2} 的值。我们可以将这种计算方式向后延伸，适用于任意数量的步骤 N。

这种数学推导超出了本书的讨论范围。如果你感兴趣的话，丹·西蒙所著的《最优状态估计》[2]中有非常详细的阐述。这个方法的精髓在于：我们不再使用状态向量 x，而是使用一个包含更多信息的扩展状态向量来表示系统状态。

$$x = \begin{bmatrix} x_k \\ x_{k-1} \\ \vdots \\ x_{k-N+1} \end{bmatrix}$$

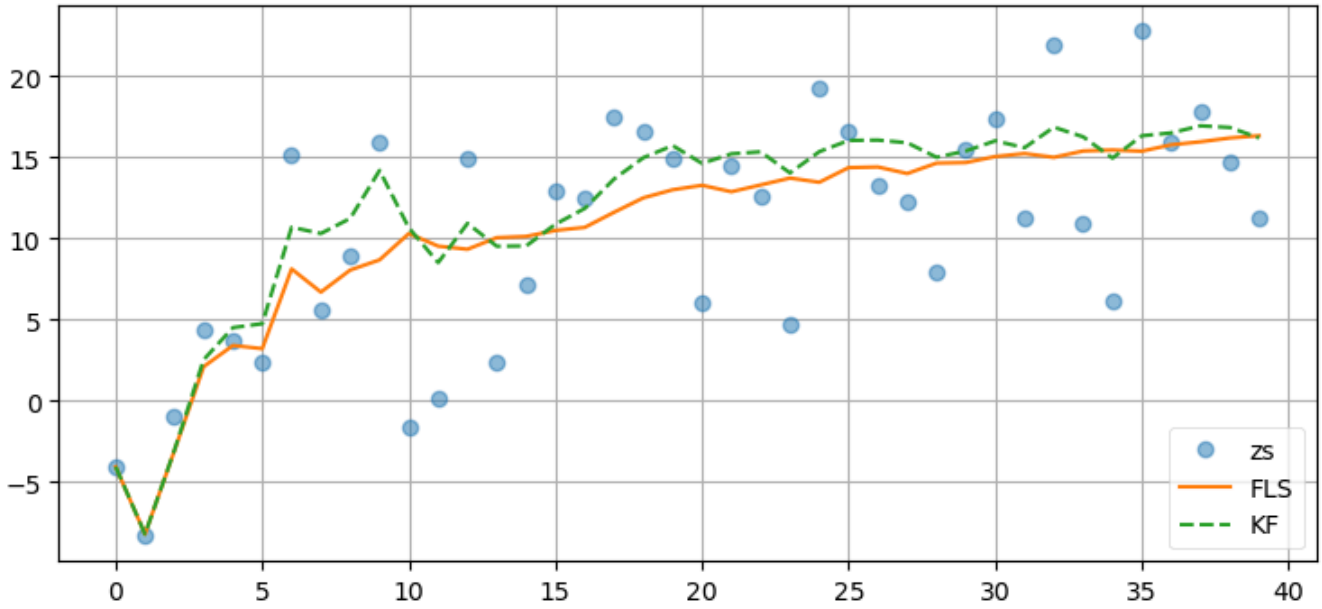
这样就会得到一个非常大的协方差矩阵，其中包含了不同时间点上各状态之间的协方差关系。FilterPy 中的 `FixedLagSmoother` 类会自动处理所有这些计算工作，包括生成各种所需的矩阵。你只需像使用 `KalmanFilter` 类那样来使用它，然后调用 `smooth()` 即可。 `smooth()` 负责执行算法中的预测和更新步骤。

每次调用 `smooth` 时，都会计算出当前值的估计值。同时，该过程还会重新调整之前的 `N-1` 个数值。经过平滑处理后的数值会存储在列表 `FixedLagSmoother.xSmooth` 中。如果使用 `FixedLagSmoother.x`，则只能得到最新的估计值。不过，该值未经平滑处理，因此与标准卡尔曼滤波器的输出结果并无区别。

```
from filterpy.kalman import FixedLagSmoother, KalmanFilter
import numpy.random as random

fls = FixedLagSmoother(dim_x=2, dim_z=1, N=8)

fls.x = np.array([0., .5])
fls.F = np.array([[1.,1.],
```

```
                  [0.,1.]])

fls.H = np.array([[1.,0.]])
fls.P *= 200
fls.R *= 5.
fls.Q *= 0.001

kf = KalmanFilter(dim_x=2, dim_z=1)
kf.x = np.array([0., .5])
kf.F = np.array([[1.,1.],
                 [0.,1.]])
kf.H = np.array([[1.,0.]])
kf.P *= 200
kf.R *= 5.
kf.Q = Q_discrete_white_noise(dim=2, dt=1., var=0.001)

N = 4 # size of lag

nom =  np.array([t/2. for t in range (0, 40)])
zs = np.array([t + random.randn()*5.1 for t in nom])

for z in zs:
    fls.smooth(z)

kf_x, _, _, _ = kf.batch_filter(zs)
x_smooth = np.array(fls.xSmooth)[:, 0]


fls_res = abs(x_smooth - nom)
kf_res = abs(kf_x[:, 0] - nom)

plt.plot(zs,'o', alpha=0.5, label='zs')
plt.plot(x_smooth, label='FLS')
plt.plot(kf_x[:, 0], label='KF', ls='--')
plt.legend(loc=4)

print(f'standard deviation fixed-lag: {np.mean(fls_res):.3f}')
print(f'standard deviation kalman: {np.mean(kf_res):.3f}')
```

```
standard deviation fixed-lag: 2.616
standard deviation kalman: 3.564
```



在这里，我设置了 `N=8` 的值。这意味着，我们会将未来 8 次的测量结果纳入到我们的估算中。这样一来，当滤波器收敛后，我们就能得到非常精确的估算结果。不过，这样的处理方式需要大约 8 倍于标准卡尔曼滤波器的计算量。你可以尝试使用不同数值的 `N` 来测试效果。我选择 8 这个数值纯属随机，并没有什么理论上的依据。

参考文献

- [1] H. Rauch、F. Tung 和 C. Striebel。“线性动态系统的最大似然估计”，《AIAA 杂志》，第 3 卷第 8 期，第 1445-1450 页（1965 年 8 月）。http://arc.aiaa.org/doi/abs/10.2514/3.3166
- [2] 丹·西蒙著。《最优状态估计》，约翰·威利父子出版社，2006 年。

第 14 章：自适应滤波

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

到目前为止，我们所考虑的都是那些符合我们处理模型要求的物体跟踪问题。例如，我们可以使用恒速滤波器来跟踪沿直线运动的物体。只要物体以相对恒定的速度沿直线运动，或者其运动轨迹和速度的变化非常缓慢，那么这种滤波器就能很好地发挥作用。不过，如果我们试图跟踪那些需要不断改变运动轨迹的物体，比如在路上行驶的汽车、正在飞行的飞机等等，那么这些滤波器的效果就会大打折扣。再比如，如果我们试图跟踪海上的帆船，即使我们能够模拟出帆船的操控输入，我们也无法模拟风力和海流的影响。

解决这个问题的一个初步方法是增大过程噪声 Q 的值，以此来弥补系统动态的不可预测性。虽然这样做可以确保滤波器不会发散，但所得到的结果通常并不理想。较大的 Q 值会导致滤波器过分强调测量数据中的噪声成分。我们稍后会看到一个具体的例子。

在本章中，我们将探讨自适应滤波器的概念。当系统检测到过程模型无法解释的动态变化时，自适应滤波器会自动调整自身的参数以适应这些变化。我会先通过一个具体的例子来说明这个问题，然后再详细介绍各种自适应滤波器的实现方式。

机动目标

我们需要一个能够模拟目标移动行为的模型。我会构建一个简单的二维模型，该模型能够根据输入的转向指令来调整自身的状态。你只需指定新的速度和/或方向，模型就会相应地调整自己的状态。

```
from math import sin, cos, radians

def angle_between(x, y):
  return min(y-x, y-x+360, y-x-360, key=abs)

class ManeuveringTarget(object):
    def __init__(self, x0, y0, v0, heading):
        self.x = x0
        self.y = y0
        self.vel = v0
        self.hdg = heading

        self.cmd_vel = v0
        self.cmd_hdg = heading
        self.vel_step = 0
        self.hdg_step = 0
```

```
        self.vel_delta = 0
        self.hdg_delta = 0

    def update(self):
        vx = self.vel * cos(radians(90-self.hdg))
        vy = self.vel * sin(radians(90-self.hdg))
        self.x += vx
        self.y += vy

        if self.hdg_step > 0:
            self.hdg_step -= 1
            self.hdg += self.hdg_delta

        if self.vel_step > 0:
            self.vel_step -= 1
            self.vel += self.vel_delta
        return (self.x, self.y)

    def set_commanded_heading(self, hdg_degrees, steps):
        self.cmd_hdg = hdg_degrees
        self.hdg_delta = angle_between(self.cmd_hdg,
                                       self.hdg) / steps
        if abs(self.hdg_delta) > 0:
            self.hdg_step = steps
        else:
            self.hdg_step = 0

    def set_commanded_speed(self, speed, steps):
        self.cmd_vel = speed
        self.vel_delta = (self.cmd_vel - self.vel) / steps
        if abs(self.vel_delta) > 0:
            self.vel_step = steps
        else:
            self.vel_step = 0
```

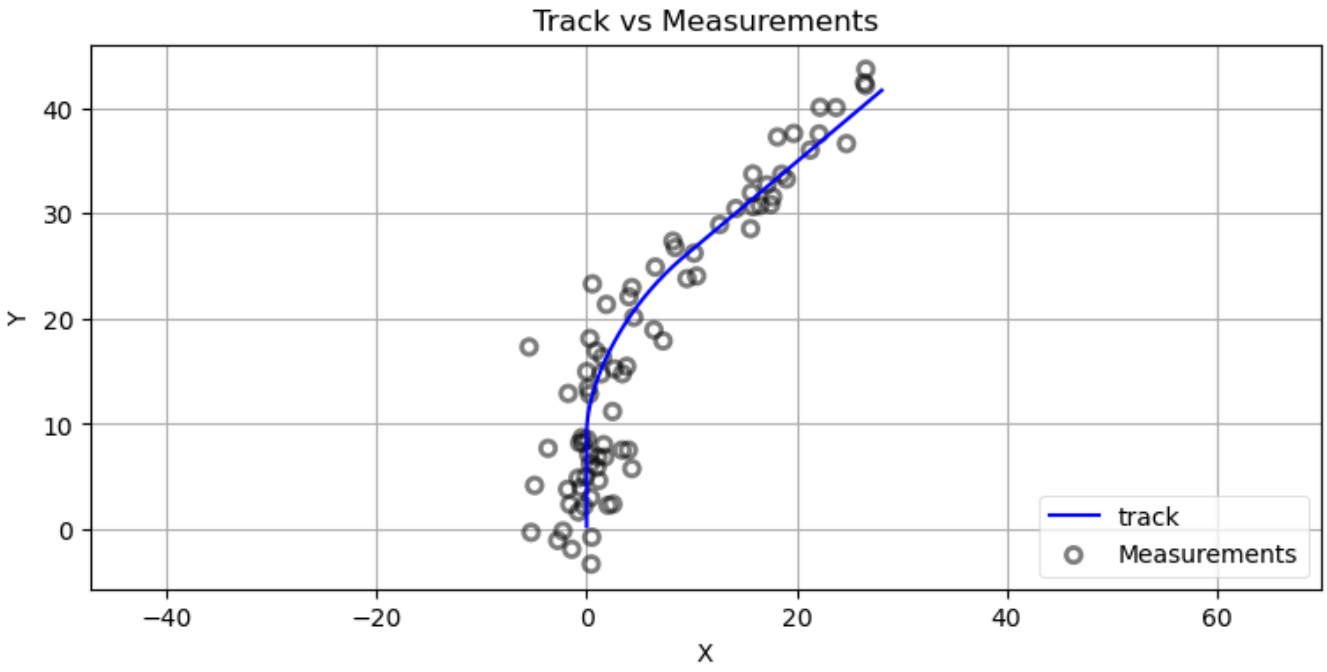
现在，让我们来实现一个带有噪声的模拟传感器吧。

```
from numpy.random import randn

class NoisySensor(object):
    def __init__(self, std_noise=1.):
        self.std = std_noise

    def sense(self, pos):
        """Pass in actual position as tuple (x, y).
        Returns position with noise added (x,y)"""

        return (pos[0] + (randn() * self.std),
                pos[1] + (randn() * self.std))
```

现在，让我们生成一条路径并将其绘制出来，以确认一切正常。我会把数据生成的过程封装在一个函数中，这样我们就可以创建出不同长度的路径了（具体原因稍后就会明白）。

```
import kf_book.book_plots as bp
import numpy as np
import matplotlib.pyplot as plt

def generate_data(steady_count, std):
    t = ManeuveringTarget(x0=0, y0=0, v0=0.3, heading=0)
    xs, ys = [], []
```

```
    for i in range(30):
        x, y = t.update()
        xs.append(x)
        ys.append(y)

    t.set_commanded_heading(310, 25)
    t.set_commanded_speed(1, 15)

    for i in range(steady_count):
        x, y = t.update()
        xs.append(x)
        ys.append(y)

    ns = NoisySensor(std)
    pos = np.array(list(zip(xs, ys)))
    zs = np.array([ns.sense(p) for p in pos])
    return pos, zs

sensor_std = 2.
track, zs = generate_data(50, sensor_std)
plt.figure()
bp.plot_measurements(*zip(*zs), alpha=0.5)
plt.plot(*zip(*track), color='b', label='track')
plt.axis('equal')
plt.legend(loc=4)
bp.set_labels(title='Track vs Measurements', x='X', y='Y')
```



如此大量的数据有助于我们更清楚地了解各种设计方案所带来的效果。

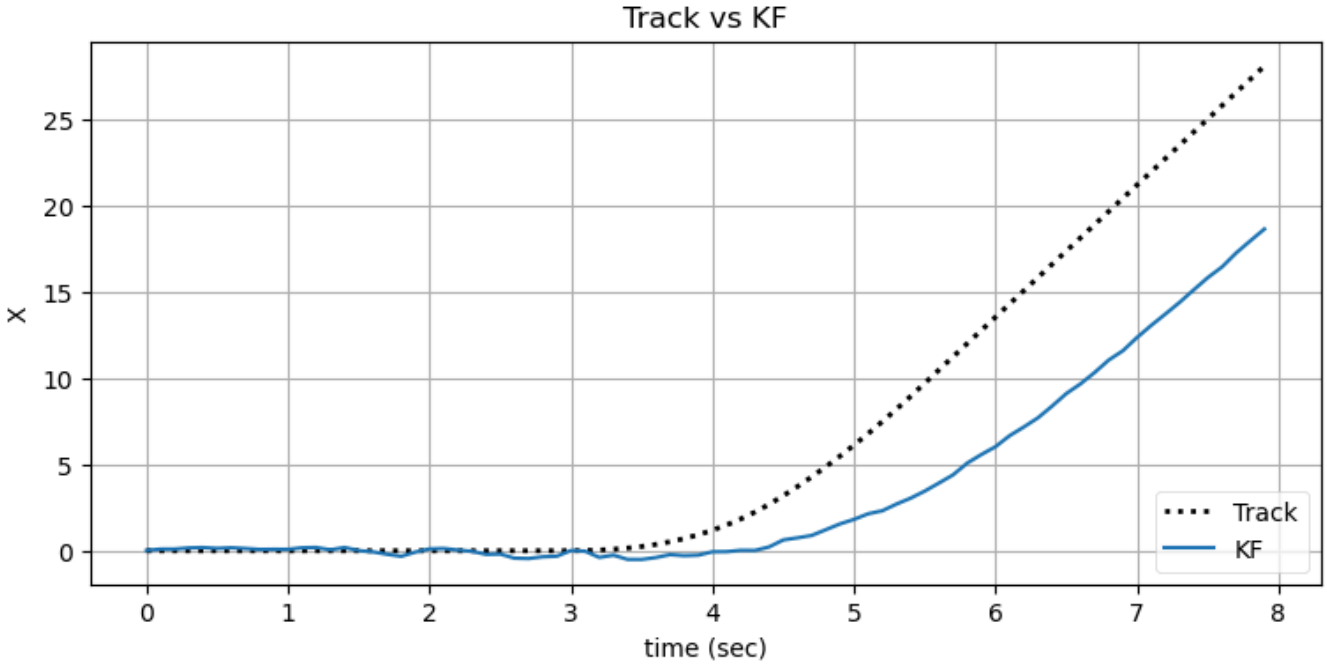
现在，我们可以利用卡尔曼滤波器来追踪这个物体了。不过，我们可以先进行简化处理：由于 x 坐标和 y 坐标是相互独立的，因此我们可以分别独立地追踪这两个坐标。在本章的后续内容中，我们将只追踪 x 坐标，这样可以尽量简化代码和相关的数学运算。

我们先从恒定速度滤波器开始。

```
from filterpy.kalman import KalmanFilter
from filterpy.common import Q_discrete_white_noise

def make_cv_filter(dt, std):
    cvfilter = KalmanFilter(dim_x = 2, dim_z=1)
    cvfilter.x = np.array([0., 0.])
    cvfilter.P *= 3
    cvfilter.R *= std**2
    cvfilter.F = np.array([[1, dt],
                           [0,  1]], dtype=float)
    cvfilter.H = np.array([[1, 0]], dtype=float)
    cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=0.02)
    return cvfilter

def initialize_filter(kf, std_R=None):
    """ helper function - we will be reinitialing the filter
    many times.
    """
    kf.x.fill(0.)
    kf.P = np.eye(kf.dim_x) * .1
    if std_R is not None:
        kf.R = np.eye(kf.dim_z) * std_R
```

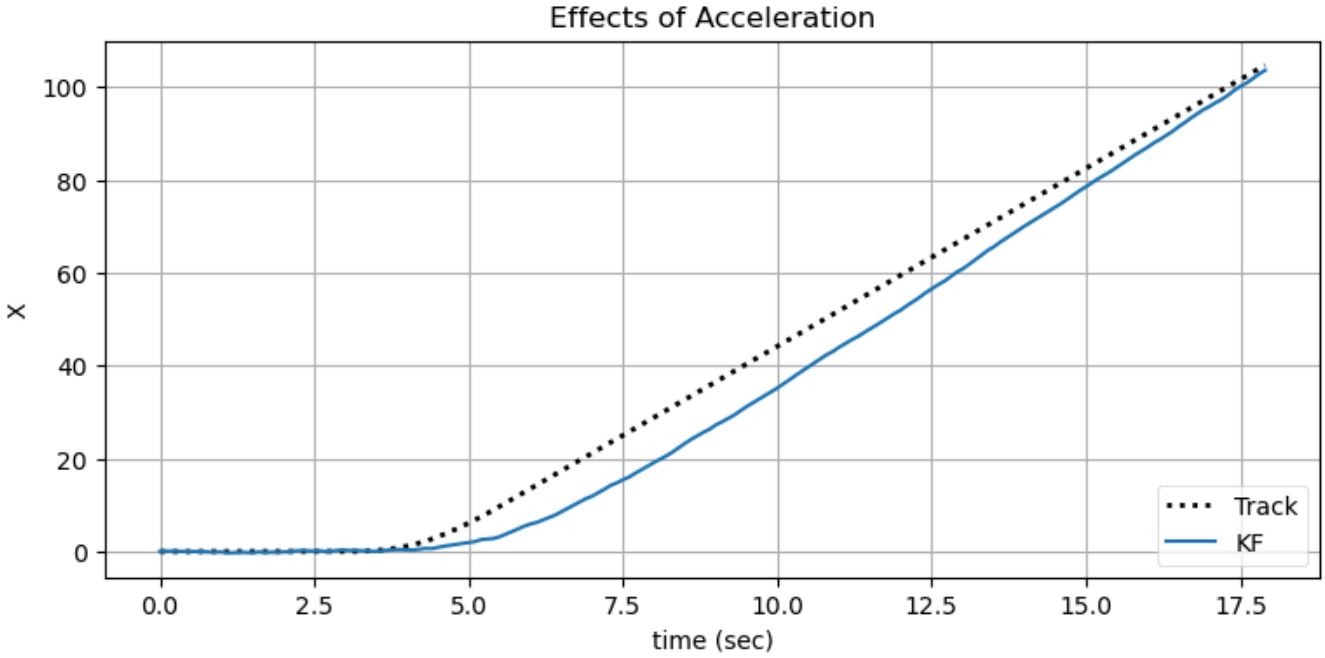
现在我们来运行它：

```
sensor_std = 2.
dt = 0.1

# initialize filter
cvfilter = make_cv_filter(dt, sensor_std)
initialize_filter(cvfilter)

track, zs = generate_data(50, sensor_std)

# run it
z_xs = zs[:, 0]
kxs, _, _, _ = cvfilter.batch_filter(z_xs)

# plot results
bp.plot_track(track[:, 0], dt=dt)
bp.plot_filter(kxs[:, 0], dt=dt, label='KF')
bp.set_labels(title='Track vs KF', x='time (sec)', y='X');
plt.legend(loc=4);
```



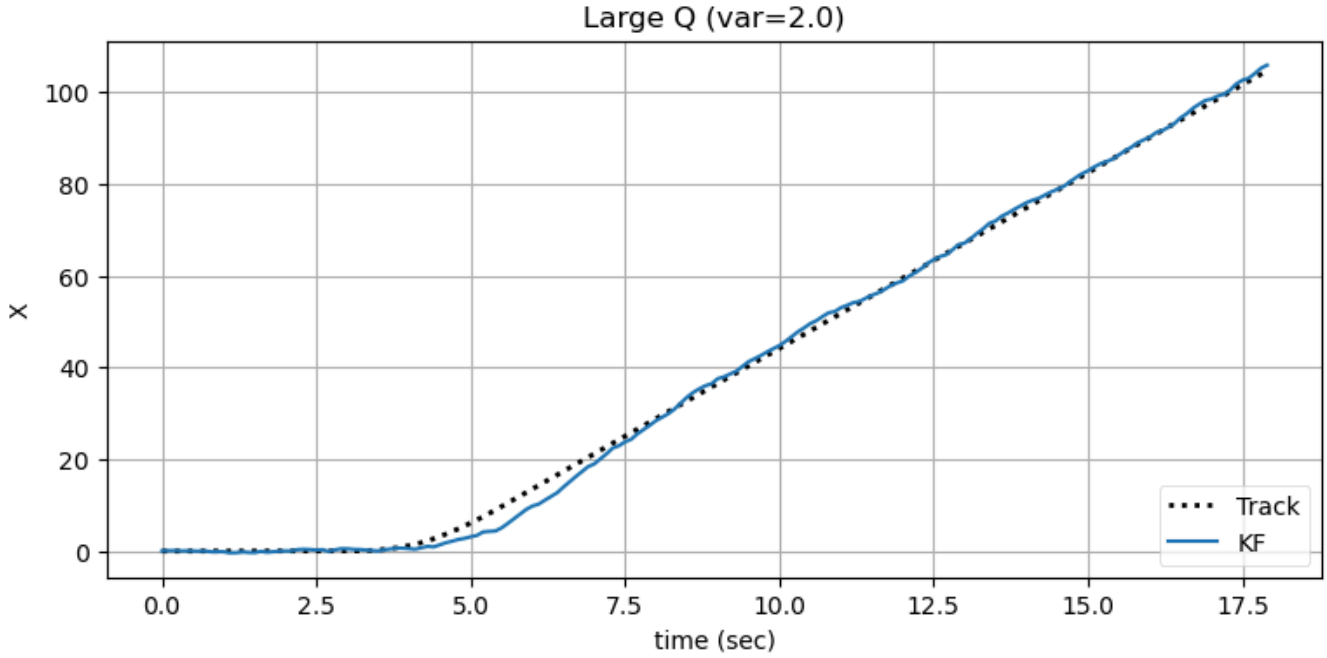
从图表中可以看出，卡尔曼滤波器无法准确追踪航向的变化。回想一下在 g-h 滤波器那一章的内容，这是因为该滤波器没有考虑加速度因素，因此它总是会滞后于输入信号。不过，如果信号处于稳定状态，滤波器最终会追上该信号。我们来看看具体情况吧。

```
# reinitialize filter
dt = 0.1
initialize_filter(cvfilter)

track2, zs2 = generate_data(150, sensor_std)
xs2 = track2[:, 0]
z_xs2 = zs2[:, 0]

kxs2, _, _, _ = cvfilter.batch_filter(z_xs2)

bp.plot_track(xs2, dt=dt)
bp.plot_filter(kxs2[:, 0], dt=dt, label='KF')
plt.legend(loc=4)
bp.set_labels(title='Effects of Acceleration',
              x='time (sec)', y='X')
```



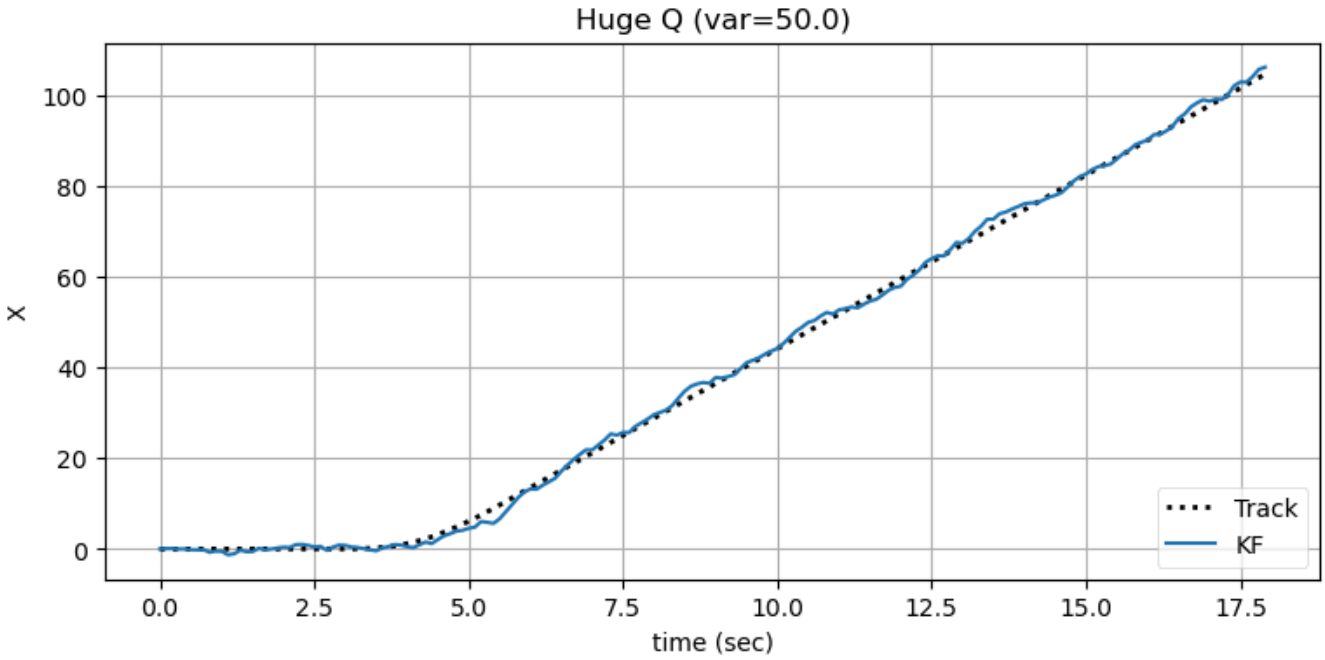
根本问题在于：我们的过程模型在系统处于稳态时是有效的，但当物体处于运动状态时则不再适用。我们可以通过增大 Q 的值来弥补这一缺陷。

```
# reinitialize filter
dt = 0.1
initialize_filter(cvfilter)
cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=2.0)
track, zs = generate_data(50, sensor_std)

# recompute track
kxs2, _, _, _ = cvfilter.batch_filter(z_xs2)
bp.plot_track(xs2, dt=dt)
bp.plot_filter(kxs2[:, 0], dt=dt, label='KF')
plt.legend(loc=4)
bp.set_labels(title='Large Q (var=2.0)', x='time (sec)', y='X')
```



我们可以看到，该滤波器能够更快地重新捕捉到目标轨迹，但代价是输出信号中包含了大量噪声。此外，在第 4 秒到第 8 秒之间，存在较长的延迟时间，这种延迟在许多跟踪场景中是无法接受的。我们虽然可以进一步减少延迟时间，但代价是输出信号会变得极其嘈杂，如下所示：

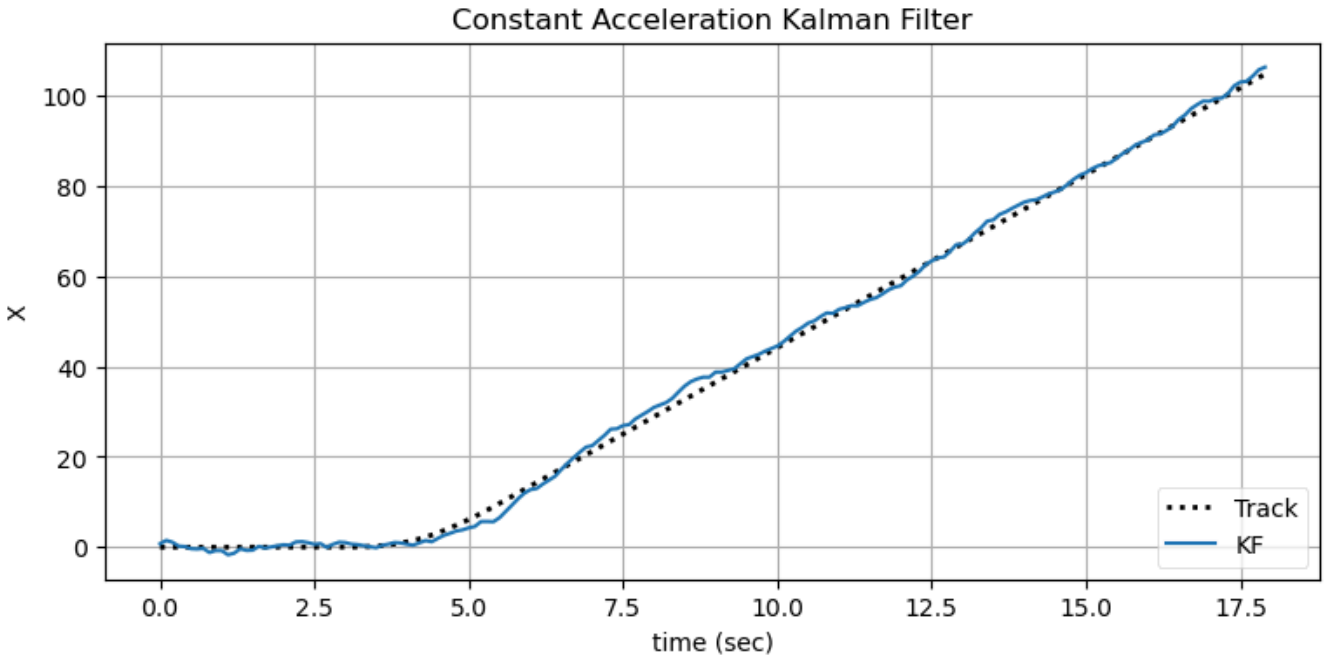
```
# reinitialize filter
dt = 0.1
initialize_filter(cvfilter)
cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=50.0)
track, zs = generate_data(50, sensor_std)

# recompute track
cvfilter.x.fill(0.)
kxs2, _, _, _ = cvfilter.batch_filter(z_xs2)

bp.plot_track(xs2, dt=dt)
bp.plot_filter(kxs2[:, 0], dt=dt, label='KF')
plt.legend(loc=4)
bp.set_labels(title='Huge Q (var=50.0)', x='time (sec)', y='X')
```



各种机动操作都意味着加速度的变化，因此，让我们使用恒定加速度卡尔曼滤波器来处理这些数据，看看其效果如何。

```
def make_ca_filter(dt, std):
    cafilter = KalmanFilter(dim_x=3, dim_z=1)
    cafilter.x = np.array([0., 0., 0.])
    cafilter.P *= 3
    cafilter.R *= std
    cafilter.Q = Q_discrete_white_noise(dim=3, dt=dt, var=0.02)
    cafilter.F = np.array([[1, dt, 0.5*dt*dt],
                           [0, 1,         dt],
                           [0, 0,          1]])
    cafilter.H = np.array([[1., 0, 0]])
    return cafilter

def initialize_const_accel(f):
    f.x = np.array([0., 0., 0.])
    f.P = np.eye(3) * 3
```

```
dt = 0.1
cafilter = make_ca_filter(dt, sensor_std)
initialize_const_accel(cafilter)

kxs2, _, _, _ = cafilter.batch_filter(z_xs2)

bp.plot_track(xs2, dt=dt)
bp.plot_filter(kxs2[:, 0], dt=dt, label='KF')
plt.legend(loc=4)
bp.set_labels(title='Constant Acceleration Kalman Filter',
              x='time (sec)', y='X')
```



该恒定加速度模型能够毫无延迟地跟踪各种机动动作。不过，其缺点是，在系统处于稳定状态时，输出信号会包含大量噪声。这种噪声产生的原因在于，滤波器无法区分机动动作的开始阶段与信号中的正常噪声。由于信号中的噪声也被视为一种加速度信号，因此滤波器会将其也计算在内。

看来我们无路可走了。当目标物体处于加速状态时，恒定速度滤波器无法做出快速反应；而恒定加速度滤波器则会在目标物体处于静止状态时，将噪声误认为是加速度信号。

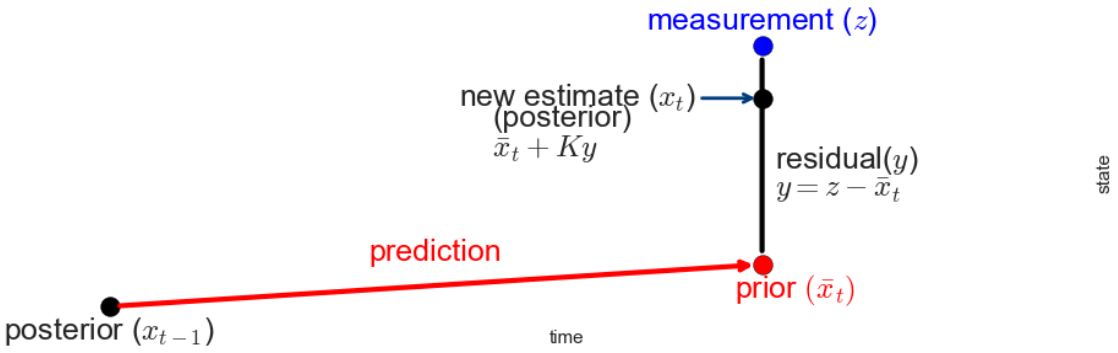
不过，这里有一个重要的见解，它能帮助我们找到解决方案。当目标物体没有进行任何运动时（即加速度为零），恒定速度滤波器能发挥最佳效果。而当目标物体在运动时，恒定加速度滤波器表现良好；另外，如果人为增加较大的处理噪声，恒定速度滤波器也能取得不错的效果。如果我们能设计出一种能够根据被跟踪物体的运动情况来自动调整自身的滤波器，那么就能兼得两种滤波器的优点了。

检测到某种机动动作/察觉到某种操控行为

在讨论如何创建自适应滤波器之前，我们首先需要弄清楚：“如何检测到某种机动行为呢？”如果我们不知道何时发生了机动行为，那就无法合理地调整滤波器以应对这种机动行为。

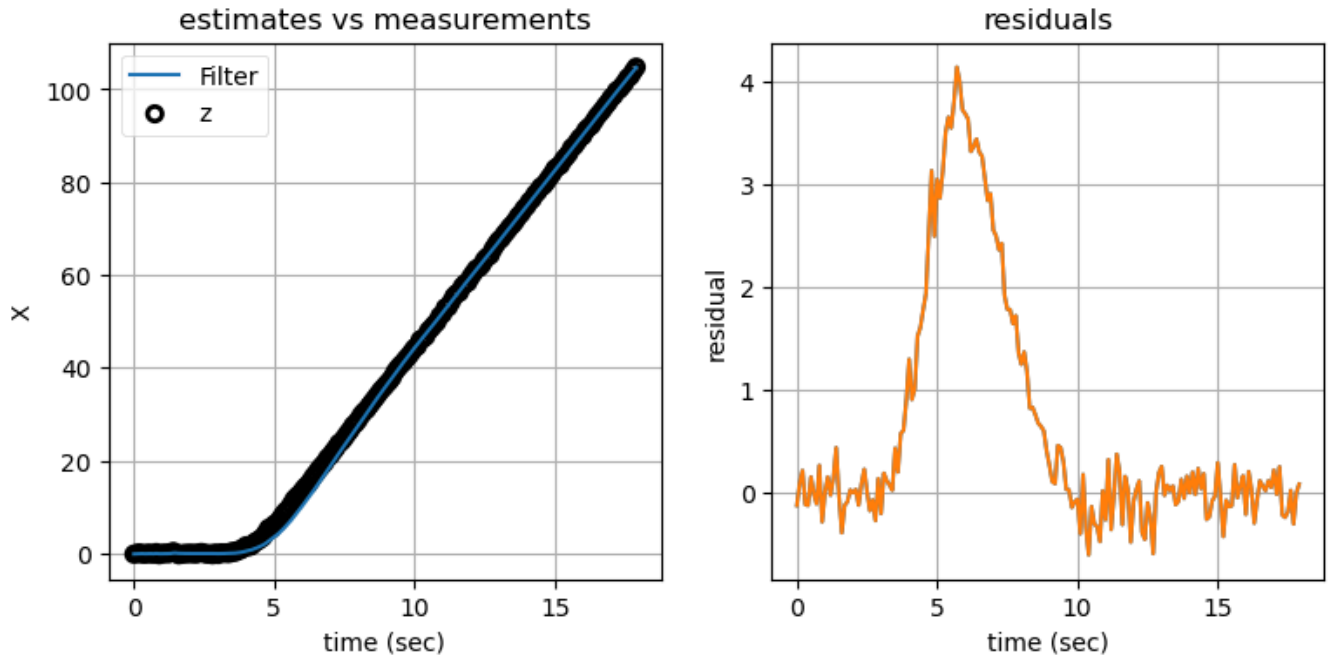
我们通常将“机动”定义为被跟踪物体处于加速状态的时刻。不过，一般来说，如果某个物体的行为与卡尔曼滤波器所使用的模型预测的结果不同，那么就可以认为该物体正在发生机动。

对于滤波器来说，一个处于运动状态的物体会产生怎样的数学影响呢？该物体的行为会与滤波器的预测结果不同，因此残差会很大。需要记住的是，残差指的是滤波器的当前预测值与实际测量值之间的差异。



为了验证这一点，让我们把该滤波器在操作过程中的残差值绘制出来。我会减少数据中的噪声，以便更清楚地看到残差值。

```
from kf_book.adaptive_internal import plot_track_and_residuals

def show_residual_chart():
    dt = 0.1
    sensor_std = 0.2

    # initialize filter
    cvfilter = make_cv_filter(dt, sensor_std)
    initialize_filter(cvfilter)
    pos2, zs2 = generate_data(150, sensor_std)
    xs2 = pos2[:, 0]
    z_xs2 = zs2[:, 0]

    cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=0.02)
    xs, res = [], []
    for z in z_xs2:
        cvfilter.predict()
        cvfilter.update([z])
        xs.append(cvfilter.x[0])
        res.append(cvfilter.y[0])

    xs = np.asarray(xs)
    plot_track_and_residuals(dt, xs, z_xs2, res)

show_residual_chart();
```



在左侧的图表中，我将各种有噪声的测量值与卡尔曼滤波器的输出结果进行了对比。在右侧的图表中，则显示了卡尔曼滤波器计算出的残差——也就是测量值与卡尔曼滤波器预测值之间的差异。需要特别强调的是，右侧的图表并非只是左侧图表中两条曲线之间的差异而已。左侧图表显示的是测量值与卡尔曼滤波器最终输出结果之间的差异，而右侧图表则显示的是测量值与过程模型预测值之间的差异。

这看起来可能只是微小的差别，但从图表上可以看出并非如此。在左侧的图表中，当操作开始时，偏差很小；但在右侧的图表中，情况则完全不同。如果被追踪物体的运动符合该模型，那么残差图应该围绕 0.0 波动。因为此时，所有测量数据都应符合该方程式。

$$measurement = process_model(t) + noise(t)$$

一旦目标开始移动，其对行为的预测就不再与实际行为相符了，因为相关方程式已经不再适用了。

$$measurement = process_model(t) + maneuver_delta(t) + noise(t)$$

因此，如果残差值偏离了 0.0 这个平均值，那就说明某种操作已经开始。

从残差图中可以看出，我们面临着相当大的挑战。虽然可以从残差图中清晰地看到该操作带来的效果，但信号中的噪声严重干扰了我们对操作起始点的判断。这正是我们长期以来在从噪声中提取有效信号时所面临的难题。

可调节的工艺噪声

我们首先考虑的方法是使用低阶模型，并根据是否正在进行某种操作来调整过程噪声。当残差“较大”时（当然，这里有个合理的“较大”标准），我们会增加过程噪声。这样一来，滤波器就会更倾向于使用实际测量值，而不是基于过程模型的预测值。当残差较小时，我们则会减少过程噪声。

在相关文献中，有多种实现这种方式的方法。我会介绍其中的几种。

持续调整

第一种方法（源自 Bar-Shalom [1]）是通过以下公式来对残差的平方进行标准化处理：

$$\epsilon = y^T S^{-1} y$$

其中， y 表示残差， S 表示系统不确定性（协方差）。其数学表达式如下：

$$S = HPH^T + R$$

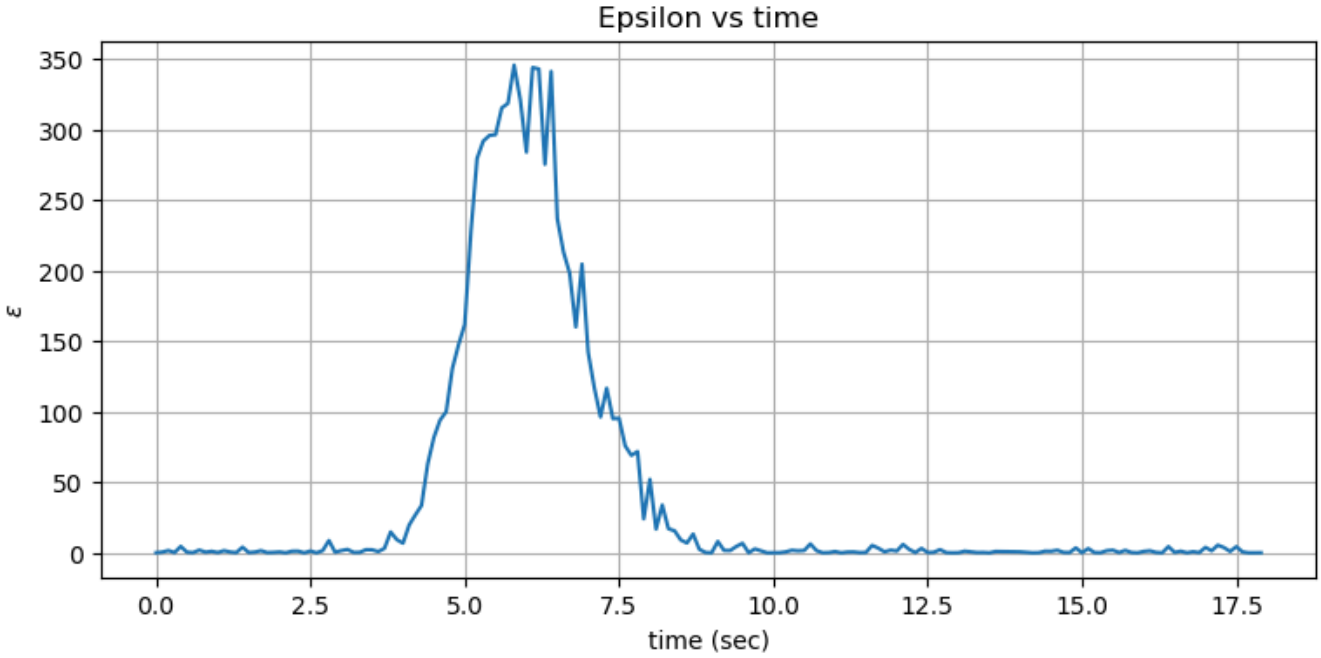
如果用于计算此结果的线性代数知识让你感到困惑，那么请记住：我们可以把矩阵的逆理解为一种除法运算。因此， $\epsilon = y^T S^{-1} y$ 可以被看作是一种除法运算的结果。

$$\epsilon \approx \frac{y^2}{S}$$

y 和 S 都是 `filterpy.KalmanFilter` 的属性，因此实现起来相当简单。

让我们来看看 ϵ 随时间变化的图表吧。

```
from numpy.linalg import inv
dt = 0.1
sensor_std = 0.2
cvfilter= make_cv_filter(dt, sensor_std)
_, zs2 = generate_data(150, sensor_std)

epss = []
for z in zs2[:, 0]:
    cvfilter.predict()
    cvfilter.update([z])
    y, S = cvfilter.y, cvfilter.S
    eps = y.T @ inv(S) @ y
    epss.append(eps)

t = np.arange(0, len(epss) * dt, dt)
plt.plot(t, epss)
bp.set_labels(title='Epsilon vs time',
              x='time (sec)', y='$\epsilon$')
```



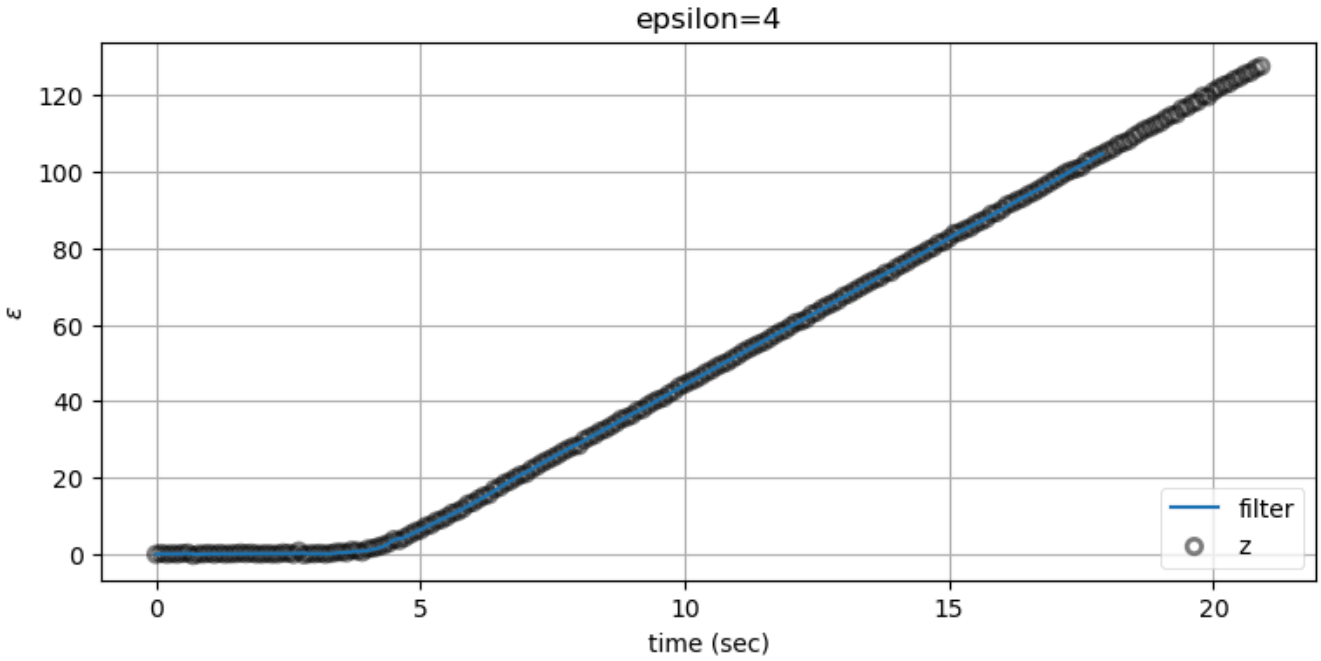
该图表清楚地展示了对残差进行标准化处理后的效果。通过对残差进行平方处理，可以确保信号的数值始终大于零。而通过用测量值的协方差来进行标准化处理后，我们就能更清楚地判断出残差相对于测量噪声而言发生了多大变化。这一处理过程从 t=3 秒时开始，可以看到，此后不久， ϵ 的值就开始迅速上升了。

当 ϵ 的值超过某个阈值时，我们就开始将 Q 的值放大；而当 ϵ 的值再次低于该阈值时，则将 Q 的值缩小。我们通过将 Q 乘以一个缩放因子来实现这一过程。或许有相关的文献可以指导我们如何确定这个缩放因子；不过我则是通过实验来确定的。对于 ϵ 的阈值（即 ϵ_{max} ），我们可以更精确地加以确定——一般来说，当残差大于 3 个标准差左右时，就可以认为这种差异是由于真实的变化引起的，而非噪声所致。不过，实际上传感器的数据往往并不完全符合高斯分布，因此实践中通常会使用 5-6 个标准差作为阈值。

我使用了合理的 ϵ_{max} 值和 Q 缩放因子来实施该算法。为了便于查看结果，我将模拟过程限制在了最初的 10 秒内。

```
# reinitialize filter
dt = 0.1
sensor_std = 0.2
cvfilter = make_cv_filter(dt, sensor_std)
_, zs2 = generate_data(180, sensor_std)

Q_scale_factor = 1000.
eps_max = 4.

xs, epss = [], []

count = 0
for i, z in zip(t, zs2[:, 0]):
    cvfilter.predict()
    cvfilter.update([z])
    y, S = cvfilter.y, cvfilter.S
    eps = y.T @ inv(S) @ y
    epss.append(eps)
    xs.append(cvfilter.x[0])

    if eps > eps_max:
```

```
        cvfilter.Q *= Q_scale_factor
        count += 1
    elif count > 0:
        cvfilter.Q /= Q_scale_factor
        count -= 1

bp.plot_measurements(zs2[:,0], dt=dt, label='z', alpha=0.5)
bp.plot_filter(t, xs, label='filter')
plt.legend(loc=4)
bp.set_labels(title='epsilon=4', x='time (sec)', y='$\epsilon$')
```



这种滤波器的性能明显优于恒速滤波器。在开始机动操作后，恒速滤波器需要大约 10 秒才能重新获取信号；而自适应滤波器则只需不到 1 秒即可完成同样的任务。

持续调整——标准差版本

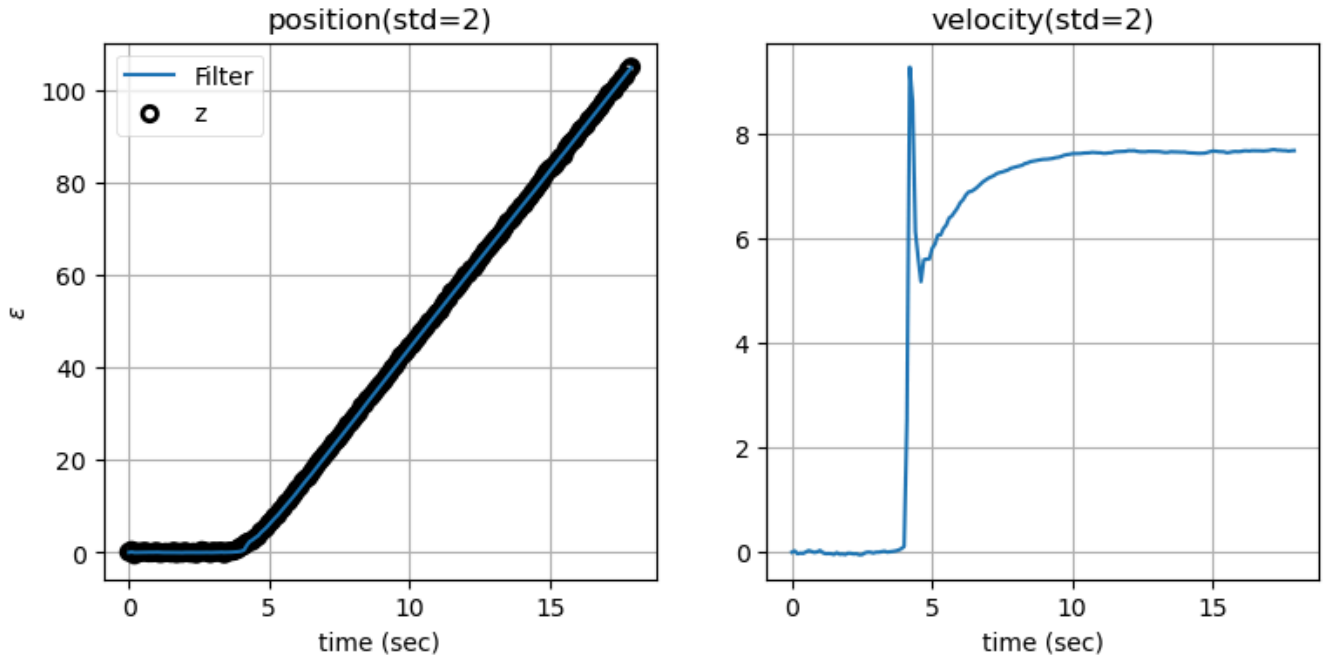
Zarchan[2]提出的另一种类似方法则是根据测量误差协方差的标准差来确定该极限值。相关的公式如下：

$$\begin{aligned} std &= \sqrt{HPH^T + R} \\ &= \sqrt{S} \end{aligned}$$

如果残差的绝对值大于上述标准差的若干倍，我们就将过程噪声增加一个固定数值，然后重新计算 Q 值，再继续下一步操作。

```
from math import sqrt

def zarchan_adaptive_filter(Q_scale_factor, std_scale,
                            std_title=False,
                            Q_title=False):
    cvfilter = make_cv_filter(dt, std=0.2)
    pos2, zs2 = generate_data(180-30, std=0.2)
    xs2 = pos2[:,0]
    z_xs2 = zs2[:,0]
```

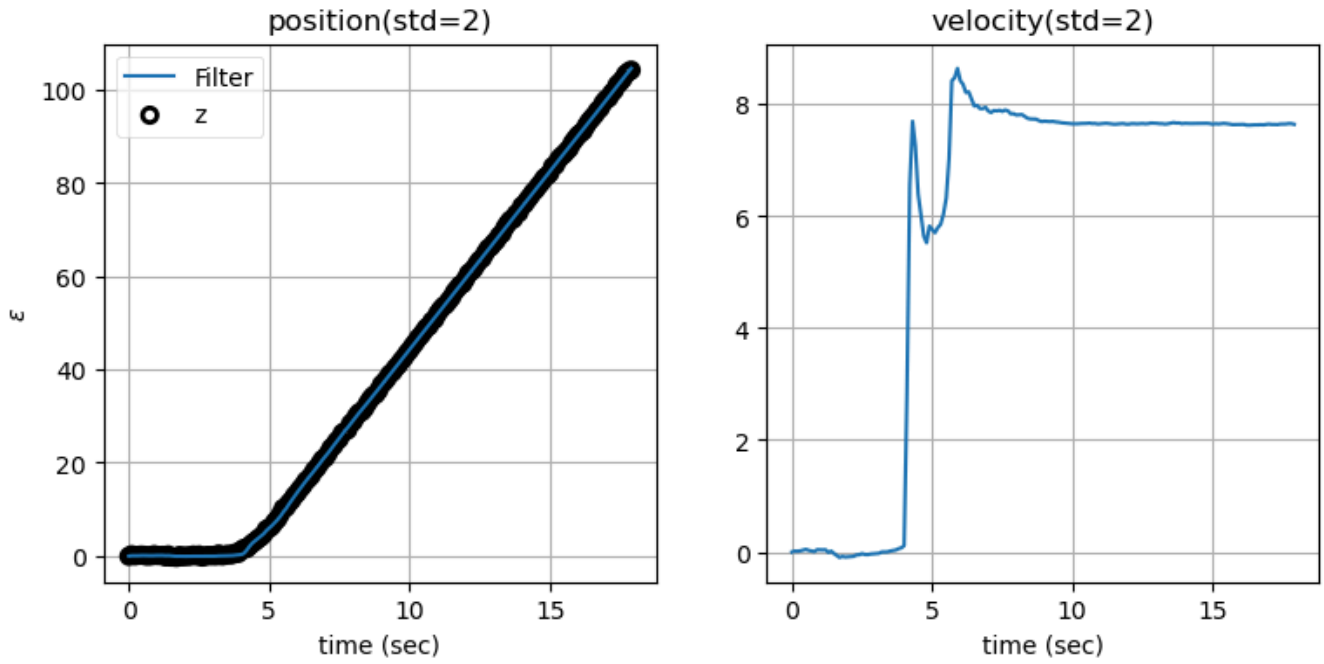
```
    # reinitialize filter
    initialize_filter(cvfilter)
    cvfilter.R = np.eye(1)*0.2

    phi = 0.02
    cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=phi)
    xs, ys = [], []
    count = 0
    for z in z_xs2:
        cvfilter.predict()
        cvfilter.update([z])
        y = cvfilter.y
        S = cvfilter.S
        std = sqrt(S)

        xs.append(cvfilter.x)
        ys.append(y)

        if abs(y[0]) > std_scale*std:
            phi += Q_scale_factor
            cvfilter.Q = Q_discrete_white_noise(2, dt, phi)
            count += 1
        elif count > 0:
            phi -= Q_scale_factor
            cvfilter.Q = Q_discrete_white_noise(2, dt, phi)
            count -= 1

    xs = np.asarray(xs)
    plt.subplot(121)
    bp.plot_measurements(z_xs2, dt=dt, label='z')
    bp.plot_filter(xs[:, 0], dt=dt, lw=1.5)
    bp.set_labels(x='time (sec)', y='$\epsilon$')
    plt.legend(loc=2)
    if std_title:
        plt.title(f'position(std={std_scale})')
    elif Q_title:
        plt.title(f'position(Q scale={Q_scale_factor})')
    else:
        plt.title('position')

    plt.subplot(122)
    plt.plot(np.arange(0, len(xs)*dt, dt), xs[:, 1], lw=1.5)
    plt.xlabel('time (sec)')
    if std_title:
        plt.title(f'velocity(std={std_scale})')
    elif Q_title:
        plt.title(f'velocity(Q scale={Q_scale_factor})')
    else:
        plt.title('velocity')
    plt.show()

zarchan_adaptive_filter(1000, 2, std_title=True)
```



所以我选择将 1000 作为噪声的缩放因子，2 则作为标准差的上限。为什么选择这些数值呢？首先，让我们来看看 2 个标准差和 3 个标准差之间的差异吧。

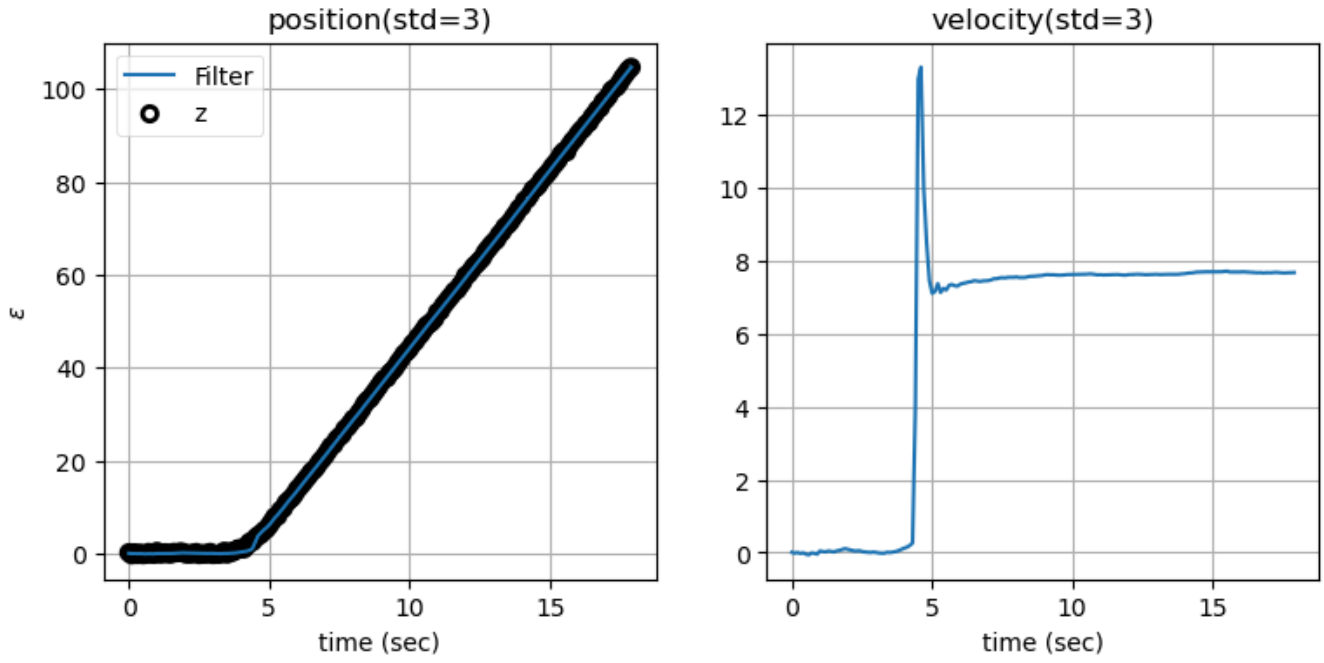
两个标准差

```
zarchan_adaptive_filter(1000, 2, std_title=True)
```



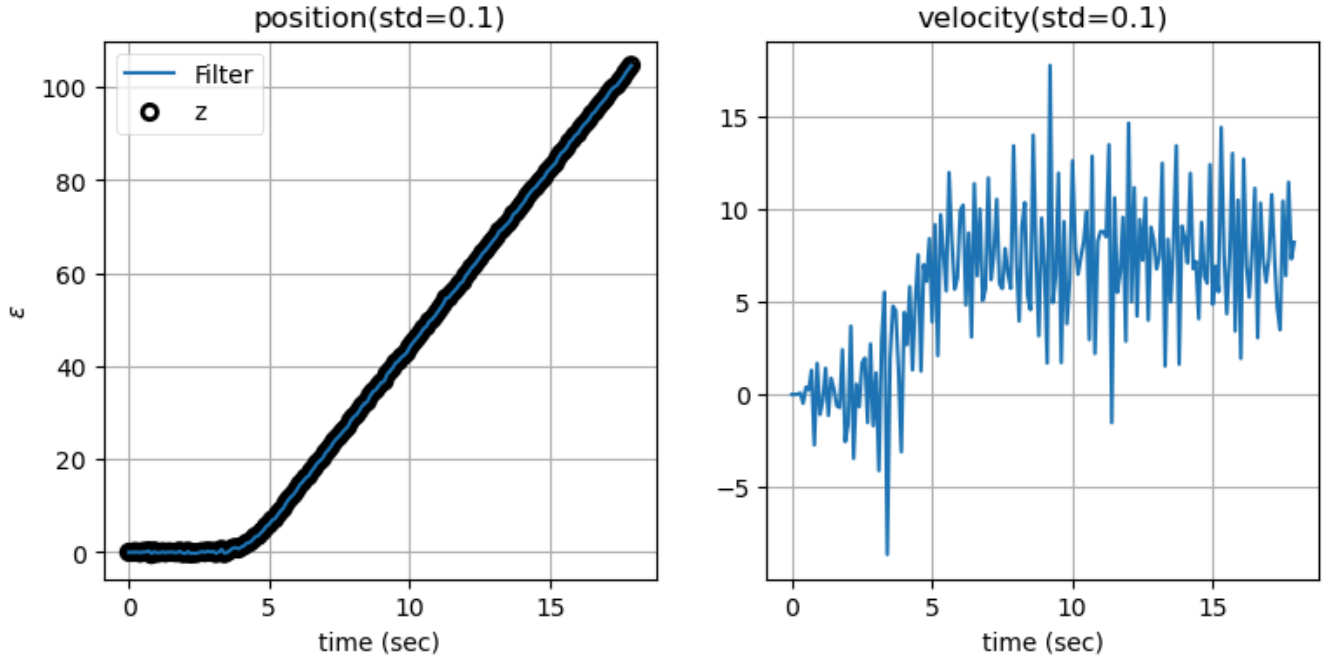
三个标准差

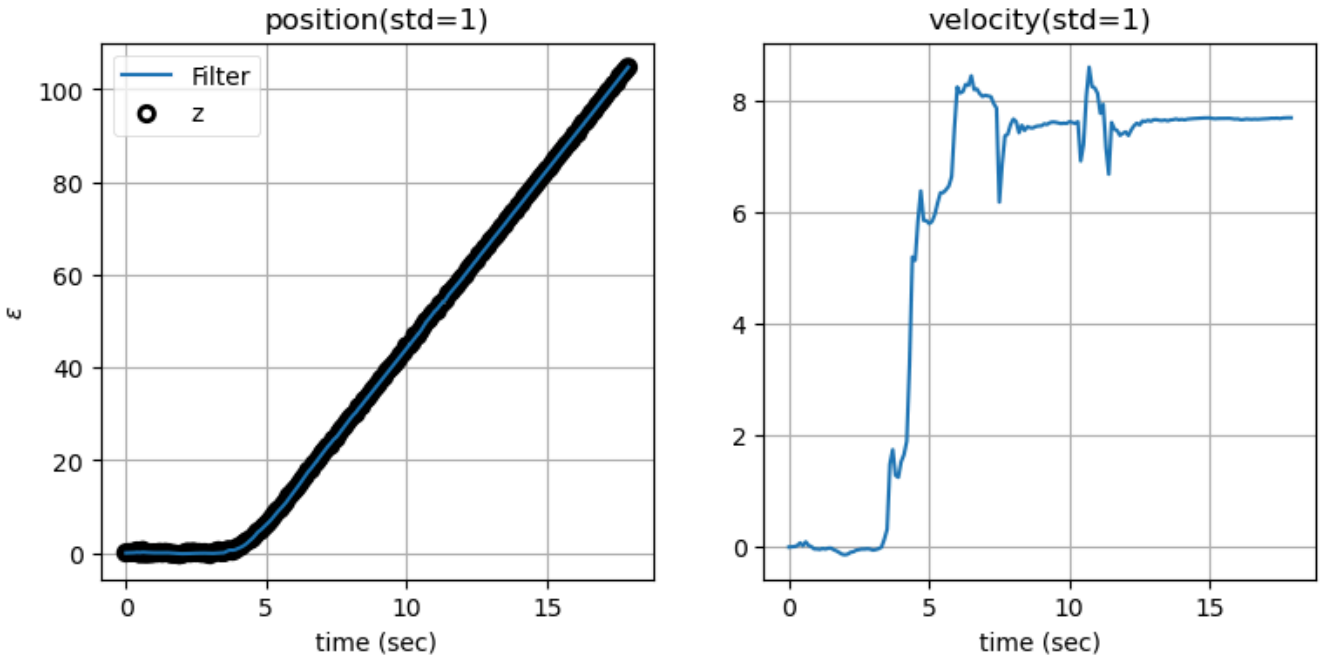
```
zarchan_adaptive_filter(1000, 3, std_title=True)
```



从图表中可以看出，无论我们使用 2 个标准差还是 3 个标准差来作为滤波条件，该位置的滤波结果其实非常相似。不过，速度的计算则有所不同。让我们进一步探讨这个问题。首先，我们将标准差设置得非常小。

```
zarchan_adaptive_filter(1000, .1, std_title=True)
zarchan_adaptive_filter(1000, 1, std_title=True)
```

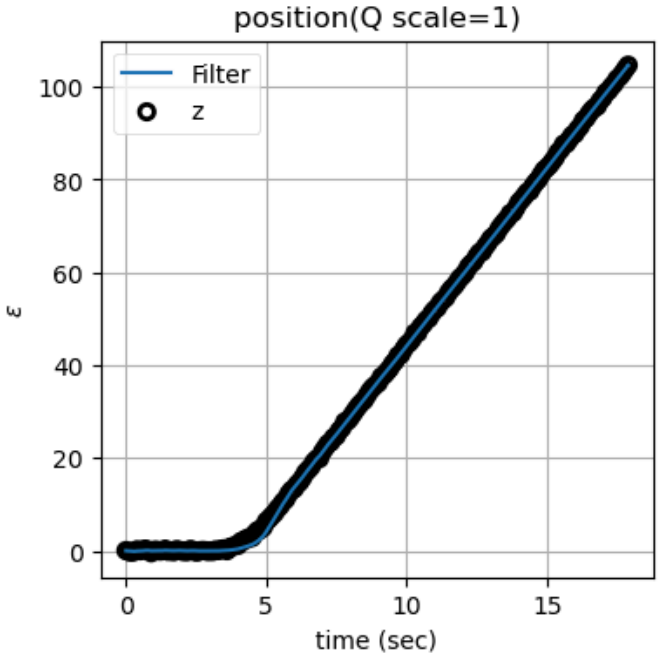


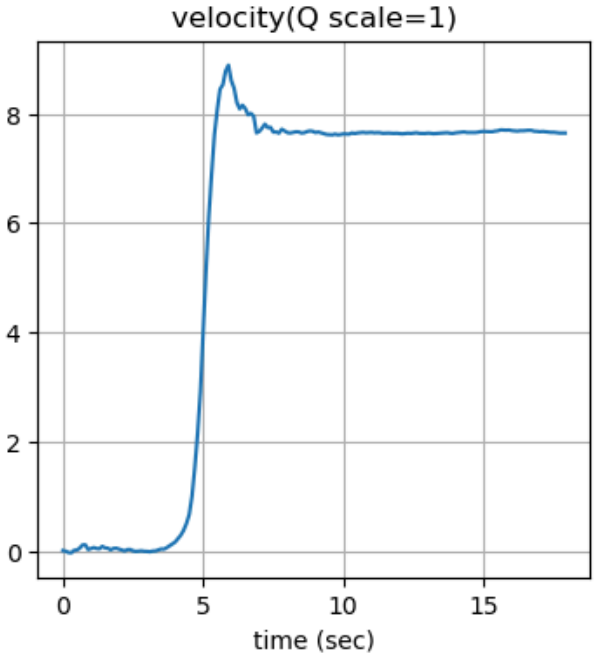


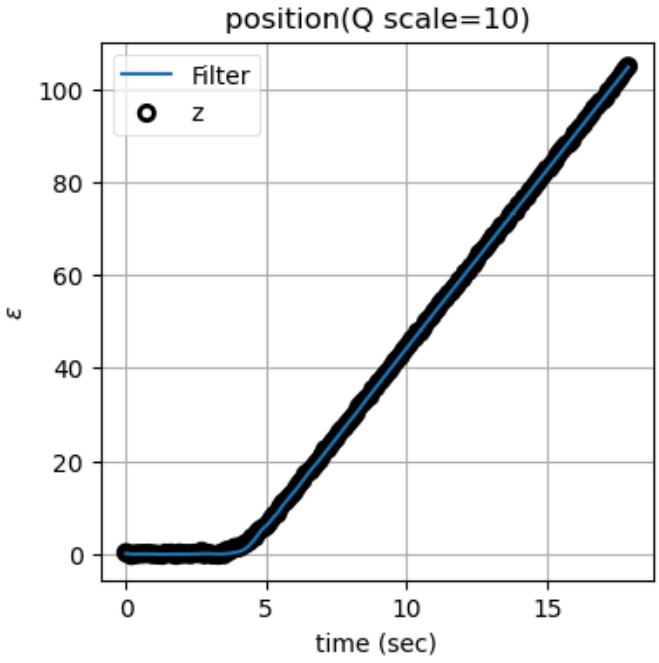
随着标准差的限制值越来越小，速度的计算结果就会变得越不准确。想想看，为什么会这样吧。如果我们调整滤波器的参数，使其更倾向于使用实际测量值而非预测值，那么一旦残差稍有偏离预测值，我们就会几乎完全依赖实际测量值来计算结果。由于完全不考虑预测值，我们就无法获得任何用于计算隐藏变量的信息。因此，当标准差限制值为 0.1 时，速度值就会被测量数据中的噪声所淹没。不过，由于我们更重视实际测量值，物体的位置就能几乎完美地跟随操控指令来移动。

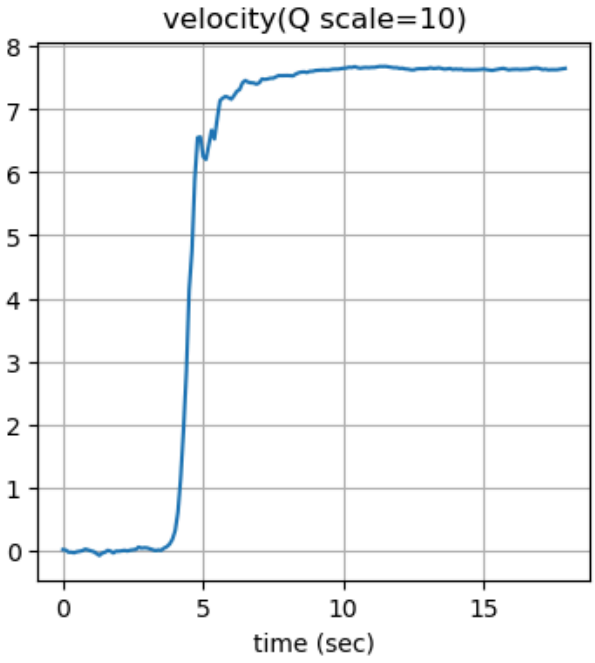
现在，让我们来看看工艺噪声的各个增量所带来的影响。在这里，我将标准差限制在了 2 个标准差以内，同时将增量从 1 变化到 10,000。

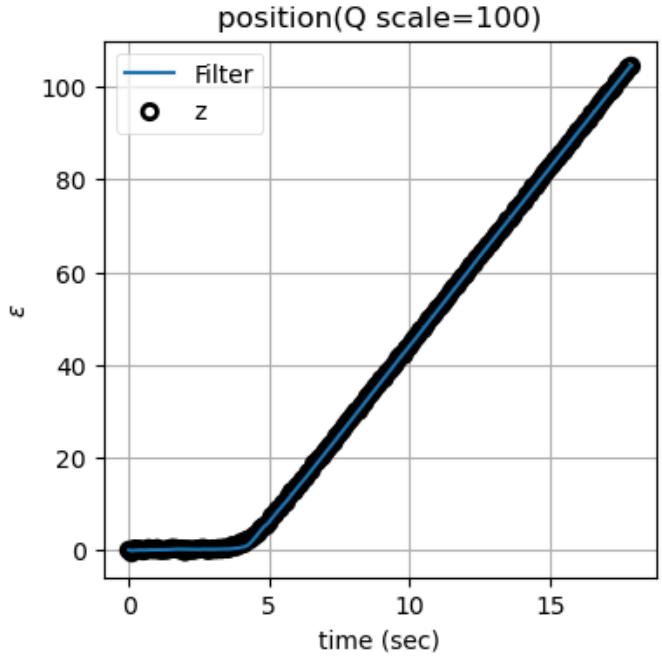
```
zarchan_adaptive_filter(1, 2, Q_title=True)
zarchan_adaptive_filter(10, 2, Q_title=True)
zarchan_adaptive_filter(100, 2, Q_title=True)
zarchan_adaptive_filter(1000, 2, Q_title=True)
zarchan_adaptive_filter(10000, 2, Q_title=True)
```

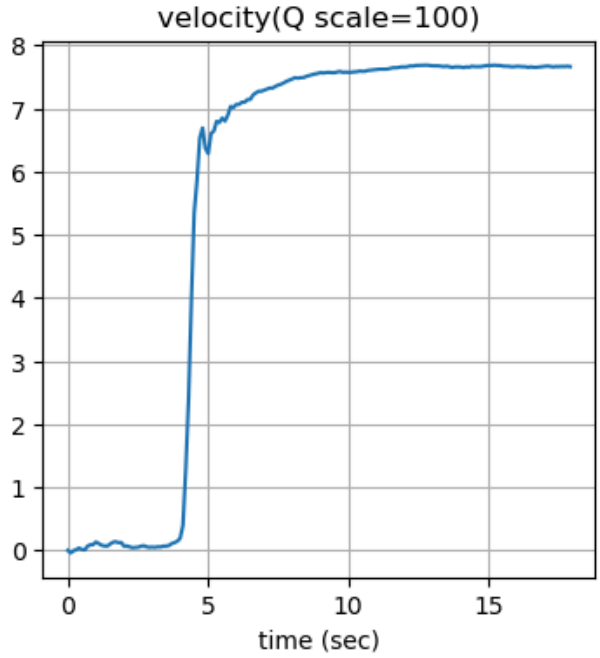

position(Q scale=1)
Filter
z
100
80
60
40
20
0
ε
0
5
10
15
time (sec)

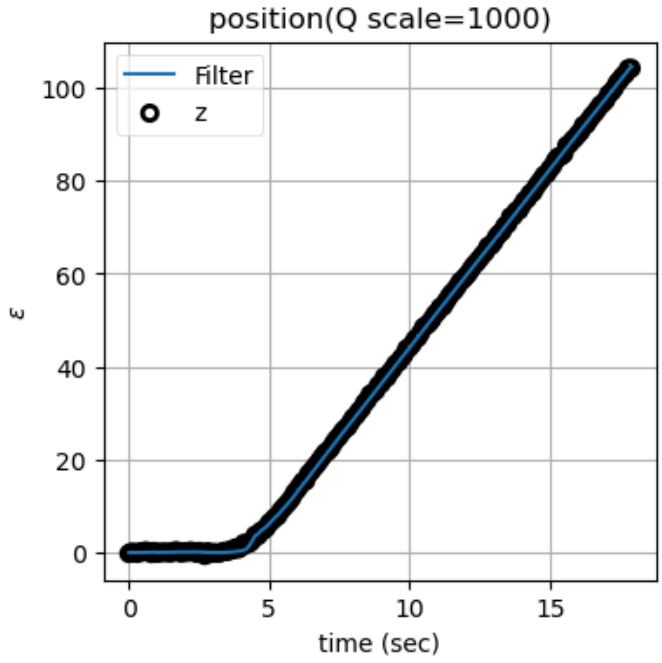

velocity(Q scale=1)
8
6
4
2
0
0
5
10
15
time (sec)

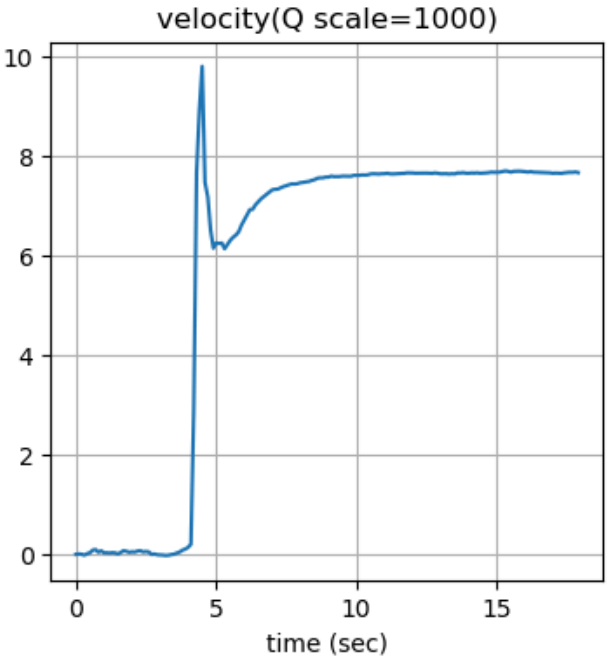

position(Q scale=10)
Filter
z
100
80
60
40
20
0
ε
0
5
10
15
time (sec)

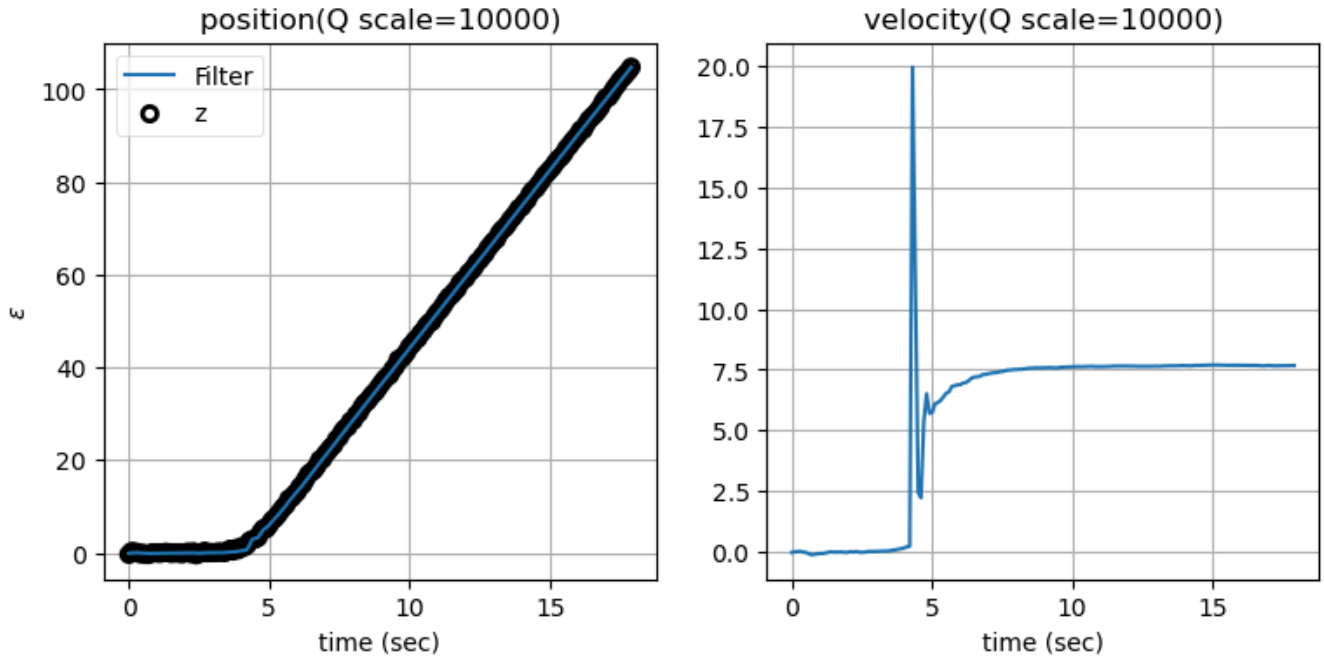

velocity(Q scale=10)
8
7
6
5
4
3
2
1
0
0
5
10
15
time (sec)


position(Q scale=100)
Filter
z
100
80
60
40
20
0
ε
0
5
10
15
time (sec)


velocity(Q scale=100)
8
7
6
5
4
3
2
1
0
0
5
10
15
time (sec)


position(Q scale=1000)
Filter
z
100
80
60
40
20
0
ε
0
5
10
15
time (sec)


velocity(Q scale=1000)
10
8
6
4
2
0
0
5
10
15
time (sec)



可以看出，随着增量因子的增加，位置估计的精度会有所提升，但速度估计则会出现较大的偏差。

我无法告诉你其中哪一种才是“正确的”方案。你需要用真实数据和模拟数据来测试各种滤波器的性能，然后选择那种最能满足你对各个状态变量所要求的性能指标的滤波器设计。

逐渐衰减的记忆滤波器

衰减记忆滤波器通常不被视为自适应滤波器，因为它们不会根据输入信号进行自我调整。不过，对于那些运动轨迹复杂的目标来说，这类滤波器依然能发挥不错的效果。此外，一阶、二阶和三阶运动滤波器都采用了非常简单的计算方式（比如本章中我们所使用的滤波器）。这种简单的计算方式无需使用 Ricatti 方程来计算卡尔曼滤波器的增益，从而大大减少了计算量。不过，也有一种形式是与标准卡尔曼滤波器兼容的。由于本章的重点在于自适应滤波器，因此我将在本章中主要介绍后者。这两种形式的衰减记忆滤波器都已在 `FilterPy` 中实现。

卡尔曼滤波器是一种递归算法，但它会将所有的历史测量数据都纳入到当前对滤波器参数的计算中。如果目标行为与模型预测一致，那么卡尔曼滤波器就能为每一次测量提供最准确的估计值。以飞行的球为例：如果我们把所有的历史测量数据都考虑进来，就能更准确地确定球在时间 t 时的位置。如果我们只使用部分测量数据，那么对当前位置的判断就会不够准确，从而更容易受到测量误差的影响。如果还是不明白的话，不妨想象最糟糕的情况：如果我们只保留了最后一次的测量数据，那么我们就无法确定球的位置和轨迹了，这时就只能给最新的测量数据更高的权重。如果测量数据有误差，那么估计结果自然也会有误差。每次对卡尔曼滤波器进行初始化时，都会出现这种现象。最初的估计值往往比较不稳定，但随着更多测量数据的获取，这些估计值会逐渐稳定下来。

不过，如果目标物体在移动过程中会改变方向，那么它的行为就不一定会符合过程模型的预测。在这种情况下，记住所有的历史测量数据和估算结果反而会带来负面影响。从上面的图表中可以看出这一点：当目标物体

开始转向时，卡尔曼滤波器仍然会继续预测其沿直线运动的情况。这是因为滤波器根据以往的测量数据得出了目标物体的运动轨迹，从而错误地认为目标物体会以恒定的方向和速度沿直线运动。

“记忆衰减滤波器”通过给予较旧的测量数据较小的权重，而给予较新的测量数据较大的权重，从而解决了这个问题。

关于这种衰减记忆滤波器，有多种不同的实现方式。我采用的是 Dan Simon 在《最优状态估计》[3]一书中提出的方法。这里就不详细阐述他的推导过程了，直接给出结果即可。

卡尔曼滤波器中，用于描述估计误差协方差的方程为：

$$\bar{P} = FPF^T + Q$$

我们可以通过乘以一个项 α，迫使滤波器忘记之前的测量结果。

$$\tilde{P} = \alpha^2 FPF^T + Q$$

当 $\alpha > 1.0$ 为某种特定值时，如果 $\alpha == 1$ 成立，那么滤波器的性能就会与普通的卡尔曼滤波器相同。 α 是 `KalmanFilter` 类的一个属性；其默认值为 1。因此，除非给 α 赋值为 1 以外的数值，否则滤波器会表现得像普通的卡尔曼滤波器。对于 α 的取值并没有固定的规则，但通常其值会非常接近 1，比如 1.01。你需要使用模拟数据或真实数据进行多次测试，才能找到一个合适的值：这个值既能有效应对各种操作情况，又不会因为过度依赖有噪声的测量数据而导致估计结果过于不稳定。

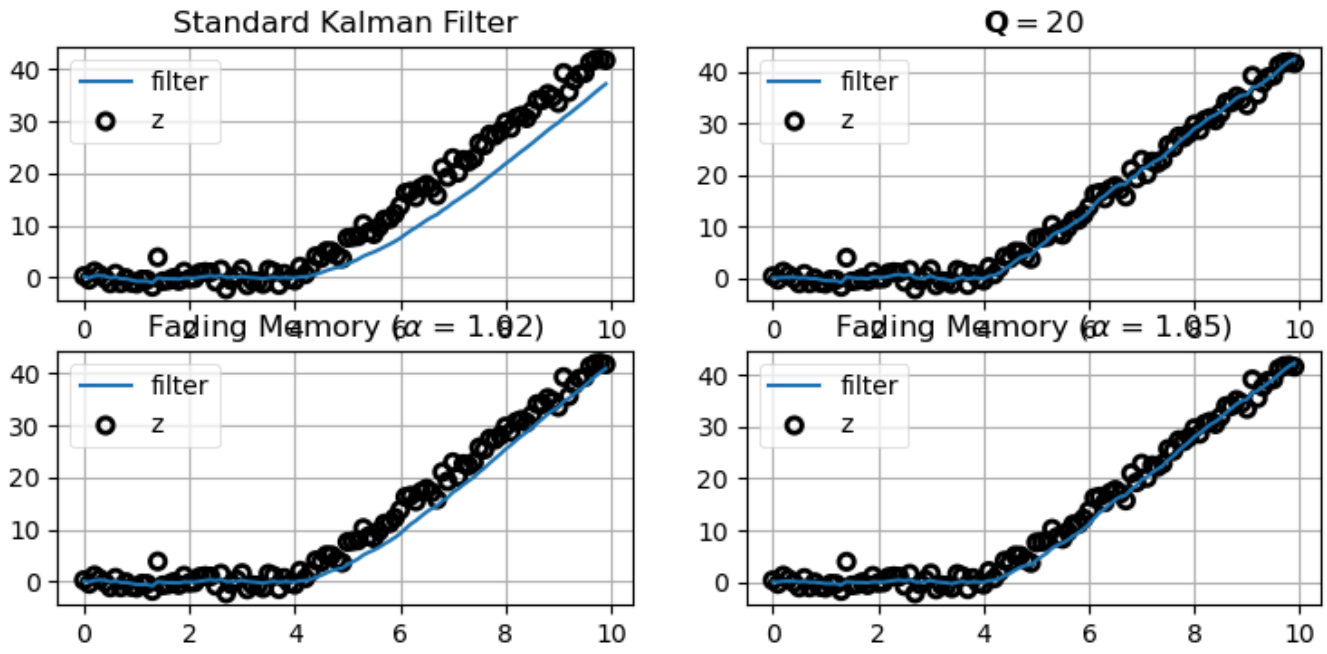
为什么这种方法有效呢？如果我们增加估计误差的方差，滤波器对自身估计结果的确定性就会降低。因此，滤波器会给予测量数据更高的权重。

需要注意一点：如果我们使用 α，那么计算出来的其实是 $\tilde{P}$，而不是 $\bar{P}$。换句话说， `KalmanFilter.P` 并不等于先验值的协方差，因此不要将其视为先验值的协方差。

让我们使用“衰减记忆滤波器”来处理数据，然后看看结果如何。我会在系统中引入大量误差，这样我们就可以比较不同的处理方法了。

```
pos2, zs2 = generate_data(70, std=1.2)
xs2 = pos2[:, 0]
z_xs2 = zs2[:, 0]

cvfilter = make_cv_filter(dt, std=1.2)
cvfilter.x.fill(0.)
cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=0.02)
cvfilter.alpha = 1.00

xs, res = [], []
for z in z_xs2:
    cvfilter.predict()
    cvfilter.update([z])
    xs.append(cvfilter.x[0])
    res.append(cvfilter.y[0])
xs = np.asarray(xs)
plt.subplot(221)
bp.plot_measurements(z_xs2, dt=dt, label='z')
plt.plot(t[0:100], xs, label='filter')
plt.legend(loc=2)
plt.title('Standard Kalman Filter')
```

```
cvfilter.x.fill(0.)
cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=20.)
cvfilter.alpha = 1.00

xs, res = [], []
for z in z_xs2:
    cvfilter.predict()
    cvfilter.update([z])
    xs.append(cvfilter.x[0])
    res.append(cvfilter.y[0])

xs = np.asarray(xs)

plt.subplot(222)
bp.plot_measurements(z_xs2, dt=dt, label='z')
plt.plot(t[0:100], xs, label='filter')
plt.legend(loc=2)
plt.title('$\mathbf{Q}=20$')

cvfilter.x.fill(0.)
cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=0.02)
cvfilter.alpha = 1.02

xs, res = [], []
for z in z_xs2:
    cvfilter.predict()
    cvfilter.update([z])
    xs.append(cvfilter.x[0])
    res.append(cvfilter.y[0])
xs = np.asarray(xs)
plt.subplot(223)
bp.plot_measurements(z_xs2, dt=dt, label='z')
plt.plot(t[0:100], xs, label='filter')
plt.legend(loc=2)
plt.title('Fading Memory ($\\alpha$ = 1.02)')

cvfilter.x.fill(0.)
cvfilter.Q = Q_discrete_white_noise(dim=2, dt=dt, var=0.02)
cvfilter.alpha = 1.05

xs, res = [], []
for z in z_xs2:
    cvfilter.predict()
    cvfilter.update([z])
    xs.append(cvfilter.x[0])
    res.append(cvfilter.y[0])
xs = np.asarray(xs)
plt.subplot(224)
bp.plot_measurements(z_xs2, dt=dt, label='z')
plt.plot(t[0:100], xs, label='filter')
plt.legend(loc=2)
plt.title('Fading Memory ($\\alpha$ = 1.05)');
```



第一幅图展示了卡尔曼滤波器的性能。在机动开始时，滤波器的性能会变得不稳定，需要大约 10 秒才能重新捕捉到信号。为了使滤波器能够更快地跟踪目标运动，我增加了过程噪声的幅度。不过，这样做会导致滤波器的估计值变得非常不稳定，因为过大的噪声会影响到滤波器的计算结果。随后，我采用了带有 $\alpha = 1.02$ 的衰减记忆滤波器。使用这种滤波器后，估计值会更加平滑。不过，当目标恢复到稳定状态时，仍需要几秒钟才能完成收敛。不过，与卡尔曼滤波器相比，其收敛时间要短得多，且滞后程度也小得多——衰减记忆滤波器的估计值比卡尔曼滤波器的估计值更接近真实轨迹。最后，我将 α 的值调整为 1.05。我们可以看到，该滤波器几乎能够立即对各种操作做出响应。但在稳态运行期间，其估计结果并不那么精确，因为滤波器会“忘记”之前的测量结果。

对于如此微小的代码改动来说，这样的性能已经相当不错了！需要注意的是，这里并没有所谓的“正确”选择。你需要根据自身的需求，以及测量噪声、过程噪声以及目标的运动特性来设计滤波器。

多重模型估计

在本章中，我所举的例子中，目标物体先以恒定速度运动，然后执行某种机动动作，之后又恢复到恒定速度状态。我们可以将这一过程视为两种模型：恒定速度模型和恒定加速度模型。只要能够用有限几种模型来描述某个系统，就可以采用“多模型估计”方法。具体来说，我们会使用多个滤波器，每个滤波器都采用不同的方式来描述该系统。根据被跟踪物体的运动情况，我们可以在这些滤波器之间切换或对它们的输出结果进行加权处理。

正如你所想象的，这是一个相当复杂的主题，设计和实现运动估计器的方法也多种多样。不过，对于本章中我们所要追踪的目标来说，可以采用一种简单的方法。一种思路是同时使用恒定速度滤波器和恒定加速度滤波器，当通过分析残差值发现物体有运动变化时，再在两种滤波器的输出结果之间进行切换。即便如此，我们仍然有多种选择。以物体转弯时的运动为例：汽车转弯时，前轮转动，而汽车则围绕后轮旋转。这是一个非线性过程，因此为了获得最佳效果，我们应该使用某种非线性滤波器（如 EKF、UKF 等）来模拟这一过程。另一方面，对于行驶过程中的稳定状态，线性恒定速度滤波器就足够了。因此，我们的滤波器组合可以包括一个线

性 KF 滤波器，以及一个用于处理转弯情况的 EKF 滤波器。不过，这两种方法都不太适合用来模拟物体加速或制动之类的行为。因此，一个性能出色的运动模型估计器可能需要包含多个滤波器，每个滤波器都针对被跟踪物体的特定运动特性来进行优化。

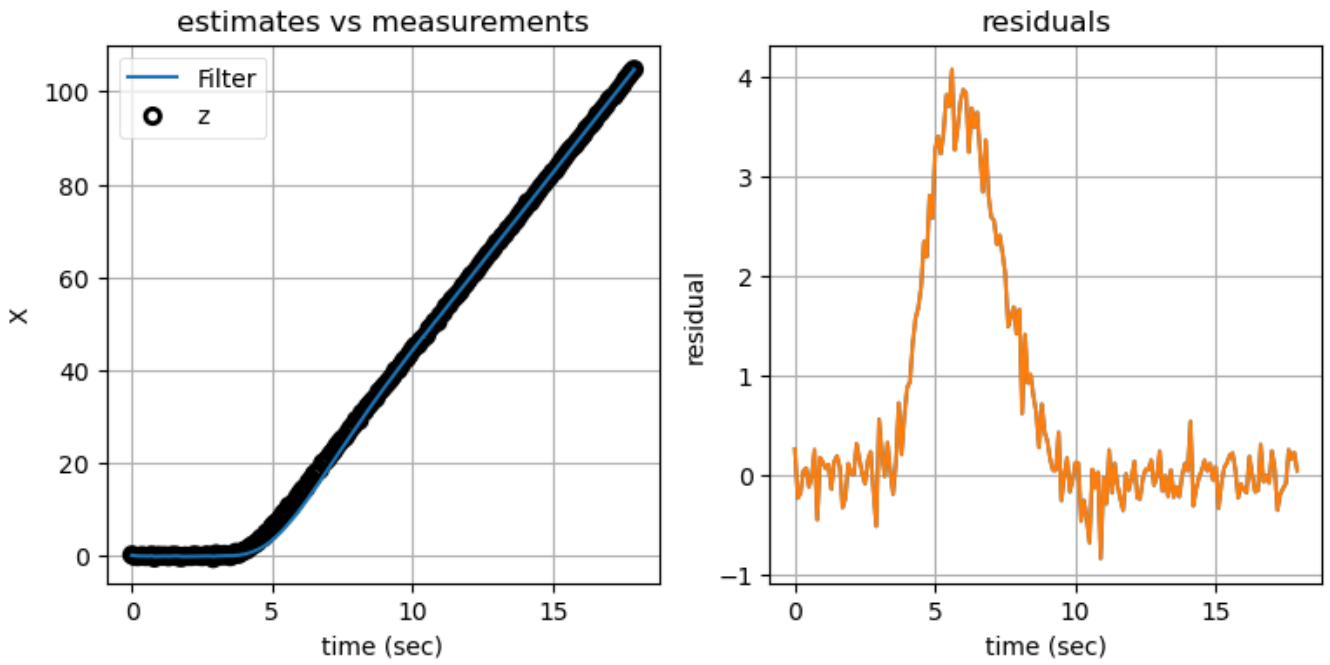
当然，你不必非得根据模型的顺序来选择滤波方式。你可以使用不同的噪声模型，或者为每个模型选择不同的适配器。例如，在上面的分析中，我展示了多个图表，这些图表展示了参数变化对速度和位置估计值的影响。也许，某种设置更适合用于位置估计，而另一种设置则更适合用于速度估计。你可以把这两种设置都纳入到滤波器的选择中。这样，你就可以从其中一个滤波器中获取位置的最佳估计值，从另一个滤波器中获取速度的最佳估计值。

一种双滤波器自适应滤波器

我相信，通过切换不同的滤波器来获得最佳性能这一理念是清晰的。但是，要实现这一理念，我们应该依据哪些数学原理呢？我们面临的问题是：如何从充满噪声的测量数据中判断出，何时应该调整模型。在卡尔曼滤波器中，是哪个参数用来衡量测量结果与预测值之间的偏差呢？没错，就是残差。

假设我们使用的是一阶卡尔曼滤波器（即恒速运动情况下的滤波器）。只要目标物体没有进行任何机动动作，该滤波器就能准确追踪其运动轨迹。大约 68%的测量值应该落在 $1\,\sigma$ 的范围内。此外，残差值应该围绕 0 波动，因为如果传感器输出的数据符合高斯分布，那么正误差和负误差出现的概率应该是相等的。如果残差值持续超出预期范围，那就说明目标物体的运动行为并不符合状态模型的预测。从图表中可以看出，当被追踪物体开始机动时，残差值从围绕 0 波动的状态突然变为持续高于 0 的值。

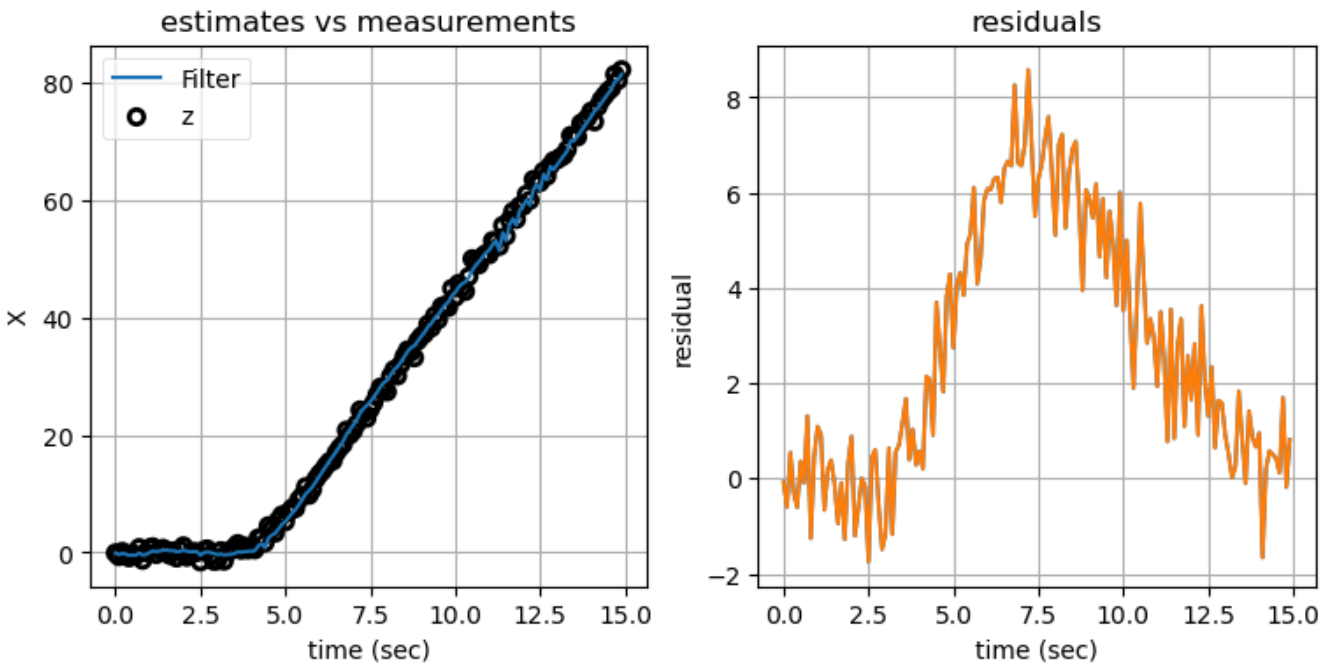
```
show_residual_chart()
```



对于这个问题，我们发现：当物体处于稳定状态时，恒定速度滤波器的性能优于恒定加速度滤波器；而当物体处于运动状态时，情况则相反。在上图中，这种转变发生在 4 秒时。

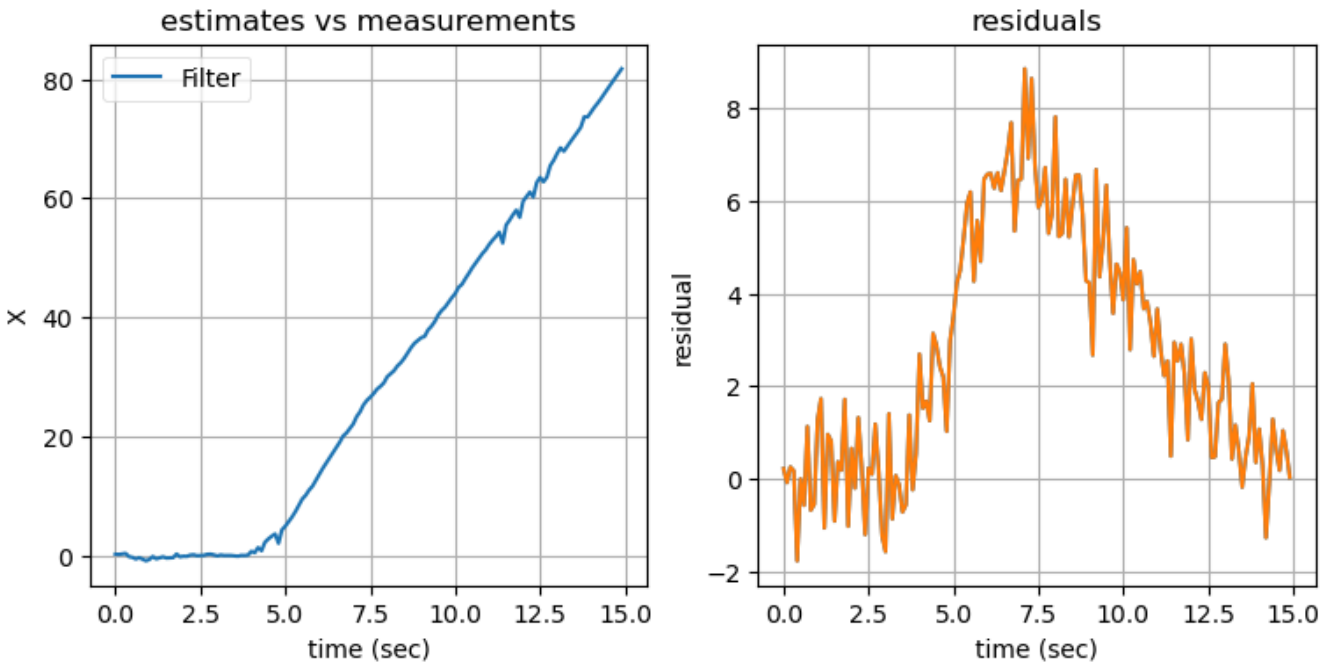
该算法其实很简单。首先初始化恒定速度滤波器和恒定加速度滤波器，然后让它们在预测/更新循环中同时运行。每次更新后，都需要检查恒定速度滤波器的残差值。如果该残差值在理论范围内，就使用恒定速度滤波器的估算结果；否则，就使用恒定加速度滤波器的估算结果。

```
def run_filter_bank(threshold, show_zs=True):
    dt = 0.1
    cvfilter= make_cv_filter(dt, std=0.8)
    cafilter = make_ca_filter(dt, std=0.8)
    pos, zs = generate_data(120, std=0.8)
    z_xs = zs[:, 0]
    xs, res = [], []

    for z in z_xs:
        cvfilter.predict()
        cafilter.predict()
        cvfilter.update([z])
        cafilter.update([z])

        std = np.sqrt(cvfilter.R[0,0])
        if abs(cvfilter.y[0]) < 2 * std:
            xs.append(cvfilter.x[0])
        else:
            xs.append(cafilter.x[0])
        res.append(cvfilter.y[0])
    xs = np.asarray(xs)
    if show_zs:
        plot_track_and_residuals(dt, xs, z_xs, res)
    else:
        plot_track_and_residuals(dt, xs, None, res)

run_filter_bank(threshold=1.4)
```



该滤波器能够精确地跟踪目标的运动轨迹。当目标没有进行任何运动时，我们的估算结果几乎不受任何噪声干扰；而一旦目标开始运动，我们就能立即察觉到这一变化，并立即切换到恒定加速度滤波器模式。不过，这种做法并不理想。以下是该滤波器的输出结果：

```
run_filter_bank(threshold=1.4, show_zs=False)
```



可以看到，当滤波器组从一种滤波器切换到另一种滤波器时，估算结果会有较大的波动。我不会在生产系统中使用这种算法。下一节将介绍一种能够解决这个问题的先进滤波器组实现方式。

MMAE

使用多个滤波器来检测某种操作或行为，这一思路本身是合理的。但是，当我们在不同的滤波器之间突然切换时，得到的估计结果会显得很不连贯。选择其中一个滤波器来处理数据，这与本书的整个理念相悖——本书主张利用概率来评估各种测量结果和模型的可靠性。我们不应简单地选择测量结果或预测结果中较有可能的那个，而应按照它们各自的概率来对两者进行加权处理。在这里，我们也应该采用同样的方法。这种方法被称为“多模型自适应估计器”，简称 MMAE。

在“卡尔曼滤波器的设计”这一章中，我们学习了似然函数。

$$L = \frac{1}{\sqrt{2\pi S}}\exp[-\frac{1}{2}y^T S^{-1} y]$$

这告诉我们：在给定输入条件下，某个滤波器实现最佳性能的可能性有多大。 y 代表残差， S 则代表系统不确定性（即测量空间中的协方差）。实际上，这只不过是残差和系统不确定性的高斯分布而已。如果残差较大，那么系统不确定性也会随之增大，进而导致该测量结果与滤波器当前状态相匹配的概率降低。我们可以利用这一信息来计算每个滤波器作为数据最佳匹配结果的概率。如果我们有 N 个滤波器，那么就可以计算出滤波器 i 相对于其他滤波器而言，是正确匹配数据的概率。

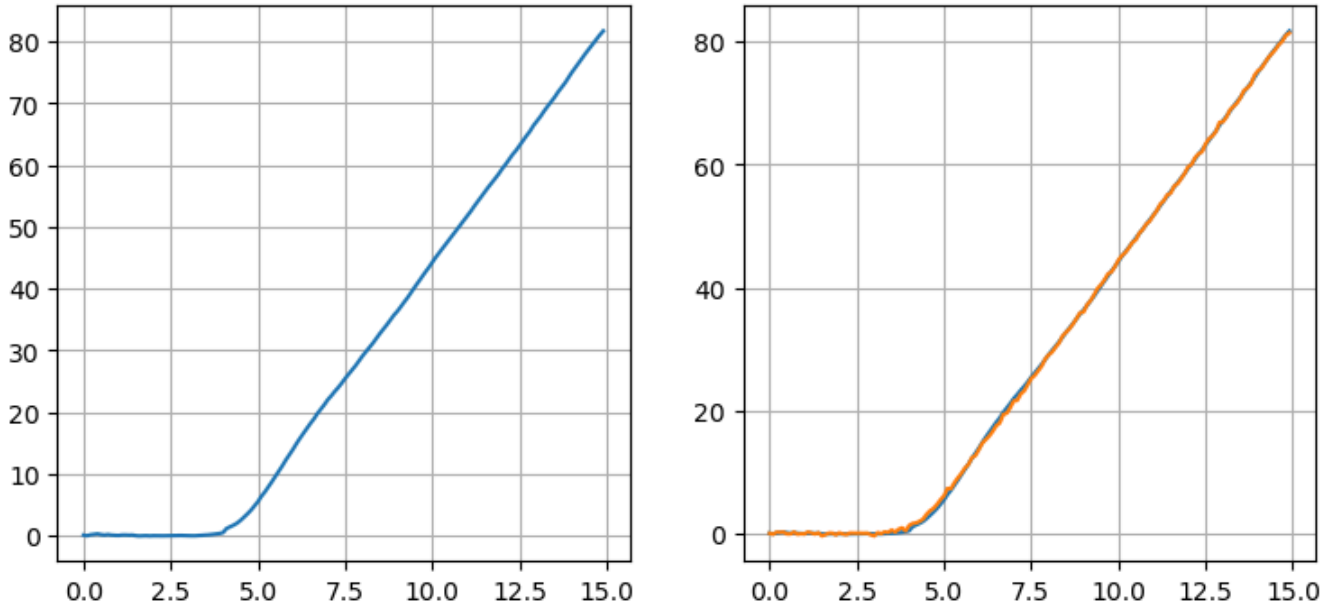
$$p_k^i = \frac{L_k^i p_{k-1}^i}{\sum_{j=1}^{N} L_k^j p_{k-1}^j}$$

看起来有点复杂，但实际上很简单。分子就是当前时间步的概率，再乘以上一个时间步中该滤波器判断正确的概率。为了确保所有滤波器的概率之和为 1，我们需要用其他所有滤波器的概率来对当前概率进行归一化处理。

这是一个递归定义，因此我们需要为每个滤波器指定一个初始概率。在缺乏更准确的信息时，可以给每个滤波器都赋值为 $\frac{1}{N}$。之后，我们可以通过将每个滤波器所预测的状态乘以该滤波器正确的概率，再把所有结果相加，从而得到最终的估计状态。

以下是完整的实现代码：

```
def run_filter_bank():
    dt = 0.1
    cvfilter = make_cv_filter(dt, std=0.2)
    cafilter = make_ca_filter(dt, std=0.2)

    _, zs = generate_data(120, std=0.2)
    z_xs = zs[:, 0]
    xs, probs = [], []

    pv, pa = 0.8, 0.2
    pvsum, pasum = 0., 0.

    for z in z_xs:
        cvfilter.predict()
        cafilter.predict()
        cvfilter.update([z])
        cafilter.update([z])

        cv_likelihood = cvfilter.likelihood * pv
        ca_likelihood = cafilter.likelihood * pa

        pv = (cv_likelihood) / (cv_likelihood + ca_likelihood)
        pa = (ca_likelihood) / (cv_likelihood + ca_likelihood)

        x = (pv * cvfilter.x[0]) + (pa*cafilter.x[0])
        xs.append(x)
        probs.append(pv / pa)

    xs = np.asarray(xs)
    t = np.arange(0, len(xs) * dt, dt)
    plt.subplot(121)
    plt.plot(t, xs)
    plt.subplot(122)
    plt.plot(t, xs)
    plt.plot(t, z_xs)
    return xs, probs

xs, probs = run_filter_bank()
```



我在左侧单独绘制了滤波器的估算结果，这样你可以看出其结果的平滑程度如何。在右侧，我同时展示了估算结果和实际测量值，以此证明滤波器确实能够准确反映物体的运动情况。

我再次强调，这其实只不过是我们在整个书中一直在使用的贝叶斯算法而已。我们有两个或更多的测量结果/估计值，每个结果都对应着一定的概率。我们会选择这些值的加权平均值作为最终结果，其中各值的权重与其对应的正确概率成正比。在每一步中，概率的计算方式都是如此。

$$\frac{\text{Prob}(\text{meas} \mid \text{state}) \times \text{prior}}{\text{normalization}}$$

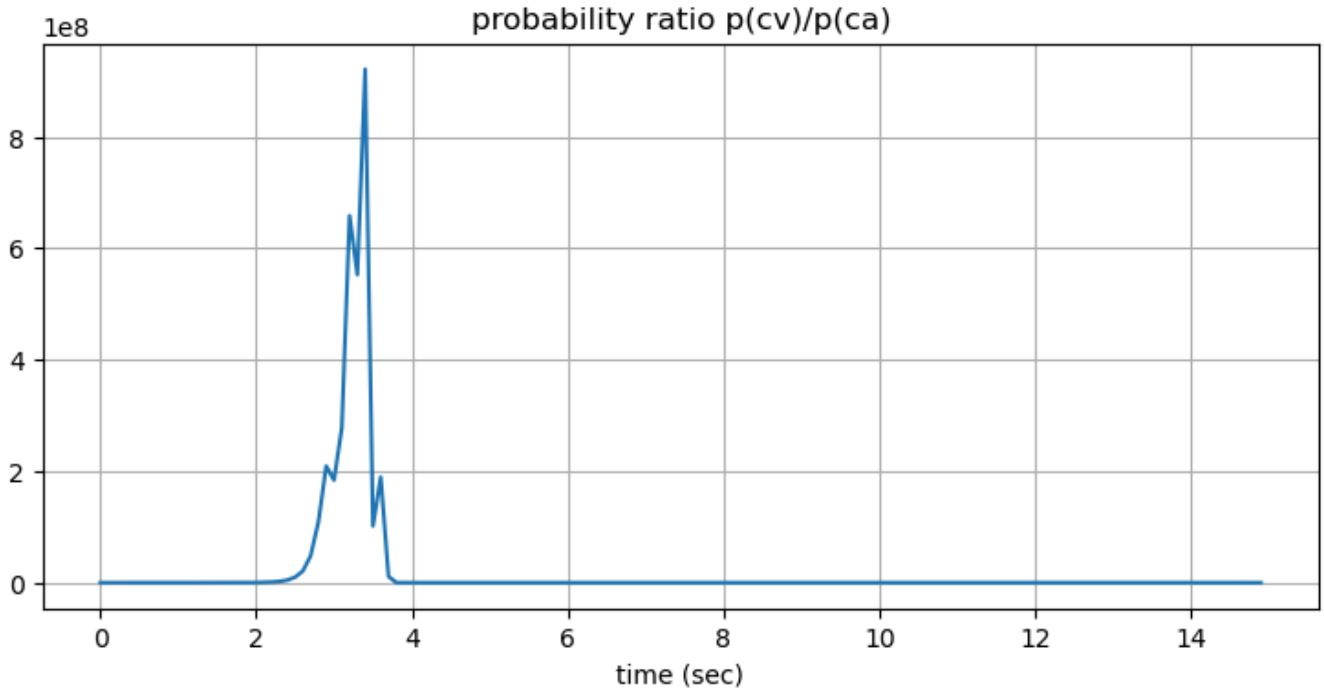
这就是贝叶斯定理。

在解决现实世界的问题时，你很可能需要使用两个以上的滤波器。在我的工作中，我利用计算机视觉来追踪各种物体。比如冰球——冰球会滑动、弹跳、滚动，还会被球员快速“运球”。我还需要追踪运动员的动向，而运动员的非线性行为几乎是无穷无尽的。在这种情况下，两个滤波器远远不够用。我需要建立多个处理模型，同时还要考虑与计算机视觉检测相关的各种噪声因素。不过，大概的思路你应该明白了吧。

MMAE 滤波器的局限性

正如我所阐述的，MMAE 存在一个严重的问题。请看这张图表，它展示了恒定速度滤波器和恒定加速度滤波器的概率比。

```
plt.plot(t[0:len(probs)], probs)
plt.title('probability ratio p(cv)/p(ca)')
plt.xlabel('time (sec)');
```



在最初的三秒钟里，由于被追踪物体沿直线运动，因此使用恒定速度模型来描述其运动情况更为合理。一旦物体开始改变运动方向，概率就会迅速转向支持恒定加速度模型的方向。不过，这种运动状态在第六秒时就会结束。人们可能会认为，此时恒定速度模型的概率应该会再次上升，但实际上，该概率仍然为零。

出现这种情况是因为概率值的递归计算方式所致。

$$p_k = \frac{Lp_{k-1}}{\sum \text{probabilities}}$$

一旦某个滤波器的概率变得极低，那么它就永远无法恢复到原来的状态。这样一来，滤波器组会迅速只保留那些概率最高的滤波器。因此，一种有效的方案是持续监控每个滤波器的概率，将那些概率极低的滤波器剔除，然后用那些更有可能发挥良好作用的滤波器来替代它们。可以将现有的滤波器进一步拆分，从而创造出能够更好地发挥各自功能的新型滤波器。在最坏的情况下，如果某个滤波器出现了异常，可以重新初始化该滤波器的状态，使其更符合当前的测量结果。

交互式多模型方法（IMM）

让我们换一种方式来思考各种模型。场景和之前一样——我们希望追踪某个正在移动的目标。我们可以设计出多种卡尔曼滤波器，这些滤波器基于不同的建模假设。它们可能在滤波器的阶数上有所不同，或者处理模型中噪声的量也有所差异。每当有新的测量数据进入时，每个滤波器都有机会成为最正确的模型。

这种简单的方法会导致组合爆炸的问题。在第 1 步中，我们会生成 N 个假设，也就是每个滤波器产生 1 个假设。第 2 步中，我们又会生成另外 N 个假设，这些假设需要与之前的 N 个假设结合起来，从而得到 N^2 个最终假设。人们尝试了多种方法来处理这个问题：要么淘汰那些不太可能的假设，要么将相似的假设合并起来。不过，这些算法仍然存在计算成本高和/或性能不佳的问题。在本书中，我不会详细讨论这些方法。不过，文献中比较著名的例子就是广义伪贝叶斯算法。

交互式多模型算法是由 Blom[5]提出的，旨在解决多模型带来的组合爆炸问题。Blom 和 Bar-Shalom 后来发表的论文被引用次数最多[6]。该算法的核心思想是：为系统的每种可能行为模式都设置一个滤波器。在每个时间点上，这些滤波器会相互协作。那些更有可能发生的事件对应的滤波器会调整那些不太可能发生的事件对应的滤波器的数值，从而让所有滤波器的输出能更准确地反映系统的当前状态。这种调整是按概率来进行的，因此那些不太可能发生的事件对应的滤波器也会对那些更有可能发生的事件对应的滤波器产生影响，不过调整的幅度要小得多。

例如，假设我们有两种选择：直接前进或转弯。每种选择都由一个卡尔曼滤波器来表示，这些滤波器可能是一阶或二阶滤波器。现在，假设目标物体正在转弯。那么，二阶滤波器会给出更准确的估计结果，而一阶滤波器则会产生较慢的响应。通过比较各种滤波器的输出结果，我们就能判断哪种滤波器的预测更有可能正确。由于一阶滤波器的预测准确性较低，我们需要用二阶滤波器的结果来大幅修正其估计值。而二阶滤波器的预测结果则相当可靠，因此一阶卡尔曼滤波器只需对其估计值进行轻微的调整即可。

现在，假设目标物体不再转动了。由于我们不断用二阶滤波器的估算结果来修正一阶滤波器的估算结果，因此一阶滤波器对信号的滞后现象会大大减少。只需几个时间周期后，一阶滤波器就能给出非常准确的估算结果，从而成为最可靠的滤波器。之后，它将为二阶滤波器的估算结果做出重要贡献。需要记住的是，二阶滤波器容易将测量噪声误认为是加速度信号。这种调整有助于大幅减少这种错误。

各模式的概率

我们为该系统定义了若干种工作模式，记为 m 。我们假设目标始终处于这些模式中的某一种。在上面的讨论中，我们提到了“直线行驶”和“转弯”这两种模式，因此可以记为 $m = \{\text{straight},\ \text{turn}\}$ 。

我们为目标处于各种不同状态的概率进行赋值。这样，我们就能得到一个包含所有可能状态的概率向量。 m 有两种状态，因此我们得到的概率向量将包含两个数值。如果我们认为目标直线前进的概率为 70%，那么就可以这样表示。

$$\mu = \begin{bmatrix} 0.7 & 0.3 \end{bmatrix}$$

由于所有概率之和必须等于 1，因此在这一轮中，我们得到的概率值为 0.3。 μ 通常被用来表示众数概率，不过并非所有情况下都如此。我会使用这个符号。请不要将其与平均值混淆。

在 Python 中，我们可以这样来实现。

```
mu = np.array([0.7, 0.3])
mu
```

```
array([0.7, 0.3])
```

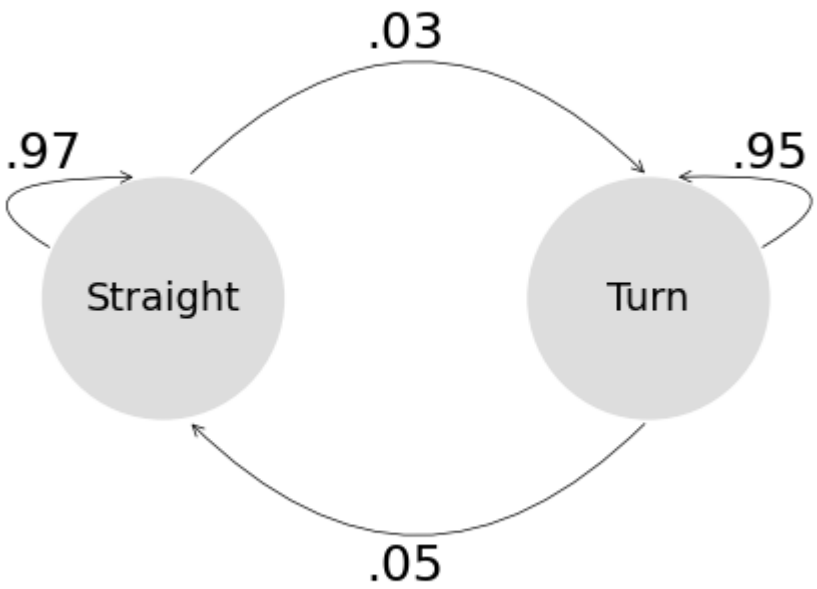
我们可以这样正式表述：在已知先前的测量结果 Z 的条件下， m_i 为正确的概率（即，被操控的物体处于模式 i ）为……

$$\mu_i = P(m_i \mid Z)$$

模式转换

接下来，我们必须考虑到，这是一个会不断改变运动轨迹的目标。它会先直线前进，然后转弯，再继续直线前进。我们可以把这种状态转换过程建模为马尔可夫链，就像下图所示：

```
import kf_book.adaptive_internal as adaptive_internal
adaptive_internal.plot_markov_chain()
```



这里展示了一个目标的两种运动模式的示例：直线前进和转弯。如果目标当前处于直线前进状态，那么我们有97%的概率认为它会继续直线前进，而只有 3%的概率认为它会开始转弯。一旦目标开始转弯，我们有 95%的概率认为它会继续沿转弯路径前进，而 5%的概率认为它会回到直线路径上。

该算法对具体数值并不敏感，通常需要通过模拟或试验来选择合适的数值。不过，这些数值还是具有相当的代表性。

我们用转移概率矩阵来表示马尔可夫链，我们将其记为 M。对于图中的马尔可夫链，我们可以这样表示：

$$M = \begin{bmatrix} .97 & .03 \\ .05 & .95 \end{bmatrix}$$

换句话说， $M[i,j]$ 表示在上一模式为 i 的条件下，当前模式为 j 的概率。在这个例子中，如果上一模式是转弯 $(i=1)$，那么当前模式为直线 $(j=0)$ 的概率就是 $M[1,\ 0] = 0.05$。在 Python 中，我们可以这样表示：

```
M = np.array([[.97, .03], [.05, .95]])
print(M)
print('From turn to straight probablility is', M[1, 0], 'percent')
```

```
[[0.97 0.03]
 [0.05 0.95]]
From turn to straight probablility is 0.05 percent
```

这样，我们就可以根据状态转换的概率来计算出各种状态的概率了。让我们来计算一下状态转换后直接处于某种状态的概率吧。有两种情况可能导致状态直接改变：要么是原本就处于该状态，然后继续保持该状态；要么是先发生了状态转换，然后再回到该状态。前一种情况的概率用 (0.7×0.97) 来计算，后一种情况的概率则用 (0.3×0.05) 来计算。最后，我们将各种状态的概率与马尔可夫链中相应的概率相乘。最终的概率就是这两者之和，即 $(0.7)(0.97) + (0.3)(0.05) = 0.694$ 。

请回想一下第二章中的总概率定理。该定理指出，几个互斥事件的概率之和为……

$$P(A) = \sum P(A \mid B)P(B)$$

这里， $P(A \mid B)$ 表示转移矩阵， M 和 $P(B)$ 则分别对应于 μ 。我们使用数组和矩阵来进行计算，这样一来，向量与矩阵相乘时，就可以得到各元素的乘积之和。

$$\begin{bmatrix} \mu_1 & \mu_2 \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix} = \begin{bmatrix} \mu_1 m_{11} + \mu_2 m_{21} & \mu_1 m_{12} + \mu_2 m_{22} \end{bmatrix}$$

IMM 文献将这一点表述为：

$$\bar{c}_j = \sum_{i=1}^{N} \mu_i M_{ij}$$

我们使用 NumPy 中的 `dot` 函数来计算这个值。当然，我们也可以使用矩阵乘法运算符 `@` ，不过我觉得，用点号来表示求和操作——也就是点积——更为直观。

```
cbar = np.dot(mu, M)
cbar
```

```
array([0.694, 0.306])
```

计算众数概率

我们将利用贝叶斯定理来计算新的模式概率。请记住，贝叶斯定理的内容如下：

$$\text{posterior} = \frac{\text{prior} \cdot \text{likelihood}}{\text{normalization factor}}$$

这里的“先验概率”指的是我们在上一节中计算出的总概率。卡尔曼滤波器则负责计算“似然值”，即在滤波器当前状态下，各项测量结果的合理性程度。为了便于理解，公式如下：

$$L = \frac{1}{\sqrt{2\pi S}} \exp[-\frac{1}{2} y^T S^{-1} y]$$

用数学符号表示，更新后的模式概率为：

$$\mu_i = \|L_i \bar{c}_i\|$$

换句话说，对于每一个卡尔曼滤波器/模式，我们都会计算该模式的概率。这个概率等于：该模式在考虑了所有可能的转换情况后的概率，再乘以该模式确实是正确模式的概率。之后，我们会将所有概率归一化，使得它们的总和为一。

在 Python 中，这个计算非常简单。我会使用变量 `L` 来存储各种概率值。概率值是通过 `KalmanFilter.update()` 步骤计算得出的。在下面的代码片段中，由于我们还没有创建卡尔曼滤波器，所以直接给 `L` 赋了固定的值。

```
# L = [kf0.L, kf1.L]  # get likelihoods from Kalman filters
L = [0.000134, 0.0000748]
mu = cbar * L
mu /= sum(mu) # normalize
mu
```

```
array([0.802, 0.198])
```

可以看出，由于“直线滤波器”的作用，选择“直线模式”的概率从 70%上升到了 80.2%。

概率的混合

此时，我们可以利用模式转换来计算所有可能情况的概率。如果当前模式为直线运动 $(\mu = 0.63)$，那么我们可以利用转移概率矩阵来计算所有可能的结果。也就是说，如果目标物体继续沿直线运动，或者发生转向，我们就可以分别计算出两种新的概率。对于转向模式 $(\mu = 0.27)$，我们也做同样的处理。这样，原本 2 种模式的概率就变成了 4 种。再下一步，4 种概率又会变成 8 种，依此类推。从计算上来说，这种方法是可行的，但实际上不可行。因为只需 30 个时间步长后，就需要 8GB 的内存来存储双精度格式下的模式概率。

我们需要一种更好、尽管只是近似性的解决方案。IMM 通过计算各种模式的混合概率来解决这个问题。其原理很简单：假设当前第一种模式（直线行驶）的概率很高，而第二种模式（转弯）的概率很低。此时，不需要再使用卡尔曼滤波器来计算直线行驶状态——只需将滤波器组中所有滤波器的输出按概率加权平均即可。那些更有可能匹配目标模式的滤波器，其权重会更大。这样一来，那些概率较低的滤波器所获取的信息也能得到改善，从而提升整个系统的准确性。这就是该算法的精髓所在。

我们需要做的其实很简单。每个卡尔曼滤波器都会执行更新操作，计算出新的均值和协方差。不过，我们还会根据所谓的“混合概率”来计算每个滤波器的新的均值和协方差——这些混合概率实际上就是各种滤波器对结果的影响程度。那些被认为不太可靠的滤波器，会稍微调整那些被认为可靠的滤波器的计算结果；而那些被认为可靠的滤波器，则会大幅调整那些不太可靠的滤波器的计算结果。在文献中，这些经过调整后的均值和协方差被称为“混合条件”或“混合初始条件”。我使用符号 x_j^m 来表示混合状态，用符号 P_j^m 来表示混合协方差。相关公式如下：

$$
\begin{aligned}
x_j^m &= \sum_{i=1}^{N} \omega_{ij} x_i \\
P_j^m &= \sum_{i=1}^{N} \omega_{ij} \left[(x^i - x_i^m)(x^i - x_i^m)^T + P_i \right]
\end{aligned}
$$

可以把下标看作是数组中的索引。用伪 Python 语言来表示的话，可以这样写：

```
for j in N:
    x0[j] = sum_over_i(w[i,j] * x[i])
    P0[j] = sum_over_i(w[i, j] * (P[i] + np.outer(x[i] - x0[j])))
```

不要让这些符号搞乱了一个简单的概念：只需将那些“可能发生”的情况的估计值纳入到那些“不太可能发生”的情况的估计值中，这样就能确保所有情况的估计都较为准确。

我们该如何计算这些混合概率呢？请先仔细思考一下，试着给出一个合理的答案，然后再继续阅读。我们有描述各模式当前概率的“模式概率”，还有描述我们切换模式的可能性的“转移概率”。那么，我们该如何计算出新的概率呢？

当然就是贝叶斯定理了！先验概率与似然概率相乘后再进行归一化处理。先验概率实际上就是各状态的概率值，而似然概率则来自马尔可夫链，这些数据都存储在矩阵 M 中。

$$\omega_{ij} = \|\mu_i \cdot M_{ij}\|$$

我们可以按以下方式来计算。我上面计算 μ 和 $\bar{c}$ 的值时顺序有误（实际上，计算 $\bar{c}$ 时必须将转移概率矩阵纳入 μ 的计算中）。因此，我需要在这里进行修正。

```
cbar = np.dot(mu, M) #compute total probability that target is in mode j

omega = np.zeros((2, 2))
for i in range(2):
    for j in range(2):
        omega[i, j] = (M[i, j] * mu[i]) / cbar[j]
omega
```

```
array([[0.987, 0.114],
       [0.013, 0.886]])
```

卡尔曼滤波器需要通过预测步骤来计算新的先验值。在计算过程中，它们会使用混合估计值。

$$\begin{aligned}\bar{x}_j &= F_j x_j^m \\ \bar{P}_j &= F_j P_j^m F_j^T + Q_j\end{aligned}$$

IMM 预估值

现在，我们需要从这些滤波器中得到最终的估计值。该怎么做呢？只需对每个卡尔曼滤波器给出的估计值进行加权处理即可。

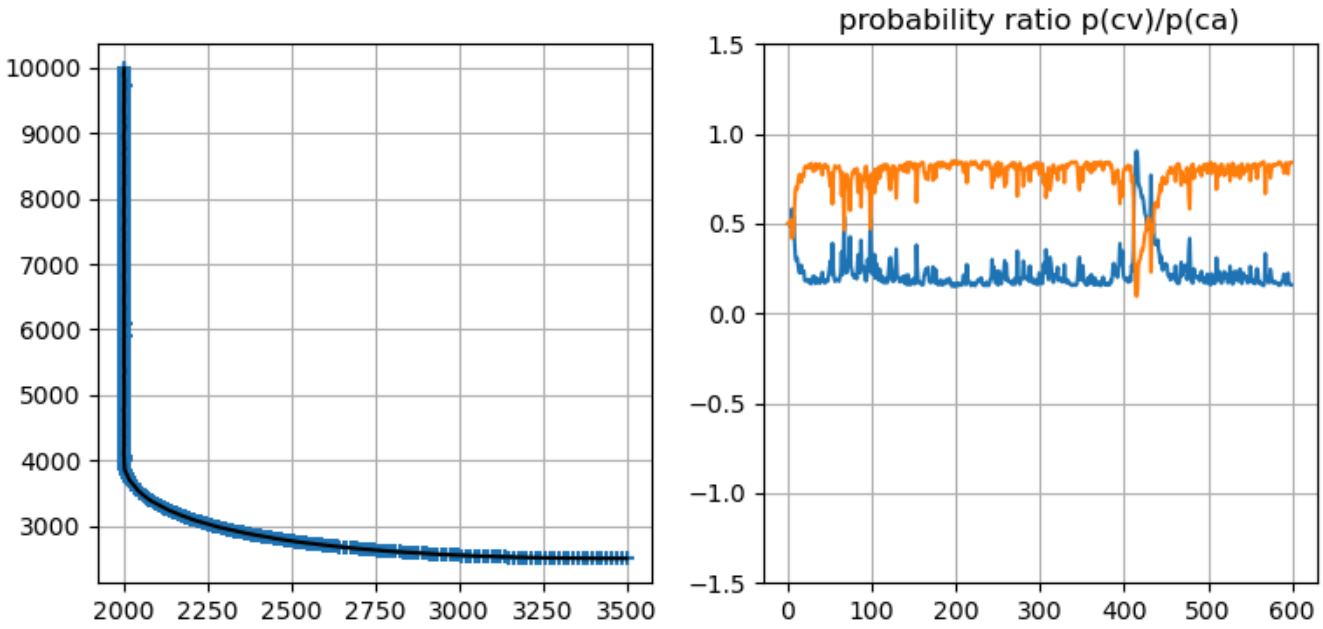
$$\begin{aligned}x &= \sum_{j=1}^{N} \mu_j \bar{x}_j \\ P &= \sum_{j=1}^{N} \mu_j \left[(\bar{x}_j - \bar{x})(\bar{x}_j - \bar{x})^T + \bar{P}_j \right]\end{aligned}$$

利用 IMM 技术追踪机动目标

我们来看一个具体的例子吧。Crassidis[4]的论文中提供了具体的计算示例，因此我选择了他的例子来说明。在该例子中，系统对一个移动目标进行了 600 秒的跟踪。该目标最初是直线运动的，但在 400 秒时开始受到外部控制力的作用，导致目标方向发生 90 度的转变。Crassidis 使用了两个具有恒定加速度特性的卡尔曼滤波器。其中一个滤波器假设不存在过程噪声，另一个则假设存在过程噪声，其谱密度为 $10^{-3}I$。他还假设了两个滤波器的初始值都设置得相当准确，即两个滤波器的 $P = 10^{-12}$ 值都相同。我的实现方式如下：

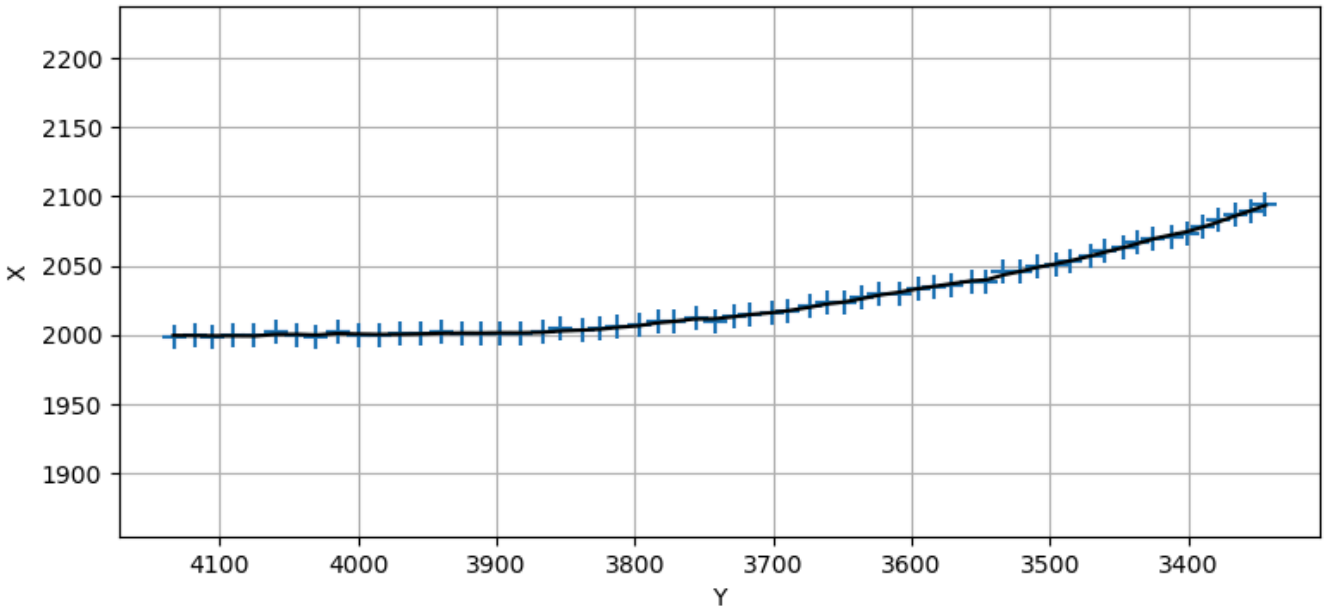
```
import copy
from scipy.linalg import block_diag
from filterpy.kalman import IMMEstimator
from filterpy.common import Saver

N = 600
dt = 1.
imm_track = adaptive_internal.turning_target(N)

# create noisy measurements
zs = np.zeros((N, 2))
r = 1
for i in range(N):
    zs[i, 0] = imm_track[i, 0] + randn()*r
    zs[i, 1] = imm_track[i, 2] + randn()*r

ca = KalmanFilter(6, 2)
dt2 = (dt**2)/2
F = np.array([[1, dt, dt2],
              [0,  1,  dt],
              [0,  0,   1]])

ca.F = block_diag(F, F)
ca.x = np.array([[2000., 0, 0, 10000, -15, 0]]).T
ca.P *= 1.e-12
ca.R *= r**2
q = np.array([[.05, .125, 1/6],
              [.125, 1/3, .5],
              [1/6, .5, 1]])*1.e-3
ca.Q = block_diag(q, q)
ca.H = np.array([[1, 0, 0, 0, 0, 0],
                 [0, 0, 0, 1, 0, 0]])

# create identical filter, but with no process error
cano = copy.deepcopy(ca)
cano.Q *= 0

filters = [ca, cano]

M = np.array([[0.97, 0.03],
              [0.03, 0.97]])
mu = np.array([0.5, 0.5])
bank = IMMEstimator(filters, mu, M)

xs, probs = [], []
for i, z in enumerate(zs):
    z = np.array([z]).T
    bank.predict()
    bank.update(z)

    xs.append(bank.x.copy())
    probs.append(bank.mu.copy())
```

```
xs = np.array(xs)
probs = np.array(probs)
plt.subplot(121)
plt.plot(xs[:, 0], xs[:, 3], 'k')
plt.scatter(zs[:, 0], zs[:, 1], marker='+')

plt.subplot(122)
plt.plot(probs[:, 0])
plt.plot(probs[:, 1])
plt.ylim(-1.5, 1.5)
plt.title('probability ratio p(cv)/p(ca)');
```



很难直接看出该滤波器的性能如何，那么让我们在转弯开始时来观察一下其表现吧。我调换了 x 和 y 轴的坐标，以便能更仔细地观察情况。下图中的转弯点位于 $Y = 4000$ 处。仔细观察可以发现，转弯开始后，估算值会有轻微的波动，但滤波器能够即时、准确地跟踪测量结果，很快就能恢复平稳的跟踪状态。

```
plt.plot(xs[390:450, 3], xs[390:450, 0], 'k')
plt.scatter(zs[390:450, 1], zs[390:450, 0], marker='+', s=100);
plt.xlabel('Y'); plt.ylabel('X')
plt.gca().invert_xaxis()
plt.axis('equal');
```



IMM 的局限性

我并没有过多地使用过 IMM 系统，因此无法像我希望的那样详细地阐述这个问题。不过，IMM 系统本来就是为帮助空中交通管制部门追踪飞机的移动而设计的。据各种报道，该系统在这一功能上表现得相当出色。

这种使用方式基于几个前提条件。其中最关键的一点是：该系统中所有的滤波器都必须具有相同的结构设计。通过数学计算可以理解其中的原因。为了得出综合估算结果，IMM 会进行如下计算：

$$x = \sum_{j=1}^{N} \mu_j \bar{x}_j$$

当且仅当每个滤波器中的状态 x 具有相同的维度时，该过程才是可计算的。此外，$x[i]$ 的含义在每个滤波器中必须保持一致。

例如，假设我们尝试使用基于恒定速度模型的滤波器，以及基于恒定加速度模型的滤波器来进行滤波。这样做是行不通的，因为这两种模型的维度不同。如果试图使用维度不同的滤波器，系统会报错。

```
ca = KalmanFilter(3, 1)
cv = KalmanFilter(2, 1)

trans = np.array([[0.97, 0.03],
                  [0.03, 0.97]])

imm = IMMEstimator([ca, cv], (0.5, 0.5), trans)
```

我偶尔会收到关于此问题的邮件或故障报告。过去，我建议将滤波器的设计参数设置为恒定速度，使其维度为 3；同时，使用 `F` 来忽略加速度因素。

```
F = np.array([[1, dt, 0],
              [0, 1,  0],
              [0, 0,  0]])
```

回顾起来，我不确定这是否是个合理的建议。虽然这种方法能让 IMM 正常工作，但显然，加速度的估算结果会是错误的。因为其中一个滤波器能准确估算出加速度，而另一个滤波器则估算出值为 0。而这个错误的加速度数据会被用来进行下一次预测。

再考虑一个极端的例子。假设其中一个滤波器将 `x[2]` 解释为加速度，而另一个滤波器则将其解释为角旋转速率。显然，这种混合后的结果是没有意义的，因为你无法将加速度与角旋转速率相加。

正如我所说，我对 IMM 并不太了解。也许相关的文献中会介绍如何处理这些情况。我只能说，FilterPy 所实现的 IMM 方式不适用于这些用例。

在用于空中交通控制的 IMM 系统中，采用了基于不同假设条件而设计的滤波器。飞机可能处于水平飞行状态，也可能处于下降或上升状态；它可能在进行协调转弯，也可能在进行非协调转弯。对于每种情况，都可以设计出相应的滤波器，这些滤波器使用不同的 `F` 和 `Q` 矩阵。不过，所有的情况下，状态估计值 `x` 都是一样的。

摘要/总结

本章包含了本书中一些较为复杂的内容。不过，它却是实现实用型卡尔曼滤波器的关键所在。如果我们正在控制某个机器人，那么我们就知道该机器人的过程模型，此时构建卡尔曼滤波器就相当简单。但更常见的情况是，我们只能得到一组时间序列数据，需要从中提取有用的信息。在这种情况下，该过程模型对我们来说几乎是未知的。我们需要利用本章中的技术来学习如何对模型进行参数化处理。由于目标状态会随时间而变化，因此我们的滤波器也必须具备自适应能力。

要找到最优解需要处理大量的组合情况，这在实际操作中是不切实际的。IMM 算法因其出色的性能和易于实现的特性，已成为标准算法。

一个真正的滤波器组通常包含两个以上的滤波器。通常情况下，滤波器的数量会更多。随着目标状态的变化，某些滤波器被使用的概率会变得极低。大多数自适应滤波器都采用某种算法来淘汰那些几乎不可能被使用的滤波器，然后用那些更符合当前目标状态的滤波器来替代它们。这种处理方式因具体问题而异，通常需要根据具体情况来设计相应的方案。你需要为滤波器的创建和淘汰制定相应的方案，并通过模拟数据或真实数据来测试这些方案的有效性。

尽管这些算法相当复杂，但我希望你们能明白，其背后的原理其实非常简单。我们仍然使用第二章中学到的两种工具：贝叶斯定理和全概率定理。我们利用贝叶斯定理来处理新信息，再利用全概率定理来分析各种过程模型所带来的影响。

对我来说，这一章充分体现了卡尔曼滤波器中贝叶斯方法的优点。我并不太在意你是否了解 IMM 算法的细节。但我希望你能明白：这些结果其实都是通过非常简单的概率推理得出的。卡尔曼博士所推导出的卡尔曼滤波器的线性代数方程，其实源自另一种称为“正交投影”的推理方式。这种推理方式本身也很美妙，我建议你阅读他的论文。不过，我觉得这些公式不太容易理解，也不清楚如何利用这些方法来设计出像 IMM 这样的非最优滤波器。相比之下，贝叶斯定理则能让我们轻松地解决这些问题。

参考文献

- [1] Bar-Shalom, Y., Xiao-Rong L., and Thiagalingam Kirubarajan. 《估计方法及其在跟踪与导航中的应用》。纽约：Wiley 出版社，第 424 页，2001 年。
- [2] Zarchan, P.和 Musoff, H.，《卡尔曼滤波基础：实用方法》第四版。弗吉尼亚州雷斯顿：美国航空航天学会，第 584-595 页，2015 年。
- [3] 西蒙，D.，《最优状态估计：卡尔曼滤波与非线性方法》。新泽西州霍博肯：威利-因特科学出版社，第 208-212 页，2006 年。
- [4] Crassidis, John L.和 John L. Junkins. 动态系统的最优估计. CRC 出版社, 2011.
- [5] Blom, H.A.P.，《一种适用于突发变化系统的有效滤波算法》，载于《第 23 届决策与控制会议论文集》，内华达州拉斯维加斯，1984 年 12 月。
- [6] Blom, H.A.P.和 Bar-Shalom, Y.，“具有马尔可夫切换系数的系统的交互式多模型算法”，《IEEE 自动控制汇刊》，第 AC-8 卷，第 8 期，1998 年 8 月，第 780-783 页。

附录 A：安装

本书使用 Jupyter Notebook 编写。这是一个基于浏览器的交互式 Python 环境，可以混合 Python、文字和数学。我选择它，是因为交互功能非常重要：在开始使用交互环境之前，我觉得卡尔曼滤波几乎无法学会。如果不能改变参数并立即看到输出，就很难对许多参数形成直觉。交互环境也让你能够试验各种“如果”：例如，“如果把 Q 设为零会怎样？”在 Jupyter Notebook 中，找到答案轻而易举。

另一个原因是，大多数教材让许多事情不透明。例如，某段伪代码旁边有一幅漂亮的图。图是软件生成的，但读者拿不到软件。我希望制作这本书用到的一切都对你开放。协方差椭圆是怎么画出来的？大多数书不会告诉你；在 Jupyter Notebook 中，只要查看源代码就知道了。

即使选择在线阅读，你仍然需要安装 Python 和 SciPy 软件栈，以便编写自己的卡尔曼滤波器。安装方法很多，无法一一介绍，但我会说明几种典型情况。

安装 SciPy 软件栈

本书需要 IPython、Jupyter、NumPy、SciPy、SymPy 和 Matplotlib。由 NumPy、SciPy、Matplotlib 组成的 SciPy 软件栈依赖第三方 Fortran 和 C 代码，从源代码安装并不简单。SciPy 网站强烈建议使用预编译发行版，我同意这一建议。

Jupyter Notebook 让你在浏览器内运行 Python，本书就是一组 Jupyter 笔记本。IPython 为 Jupyter 和数据可视化提供基础。NumPy、SciPy 提供滤波器所用的线性代数实现。SymPy 负责符号运算，我会用它对代数方程求导。Matplotlib 则提供绘图功能。

我使用 Continuum Analytics 的 Anaconda 发行版。它把上述全部软件包及许多其他包整合在一起，是很好的发行版；IPython 也推荐这种安装方式。安装过程很直接，可以与计算机上已有的其他 Python 安装并存，而且可以免费使用。下载地址：http://continuum.io/downloads 。我强烈建议选择它提供的最新 Python 3 版本；本书及 FilterPy 支持 Python 3.6 或更高版本。

也有其他安装 SciPy 软件栈的方法，见 http://scipy.org/install.html 。过程可能很繁琐，我不为此提供支持或详细说明。

许多 Linux 发行版预装了这些包，但通常版本较旧；本书依赖较新的版本，可能需要升级。如何更新特定 Linux 系统不在本书范围内。Anaconda 的优点是不修改系统原有的 Python，因此通常不会破坏 Linux 发行版。有些读者安装了 Anaconda，却仍在使用系统默认 Python，导致书中程序运行异常，需要留意。

我不会为旧版库运行回归测试。事实上，我知道代码无法在例如 2014—2015 年的旧版库上运行。我不想把全部时间都花在一本书的技术支持上，因此请读者自行安装较新的 Python 和 SciPy 软件栈。

需要 Python 3.6 或更高版本。

如果安装了最新 Anaconda 仍有问题，请在本书的 GitHub 仓库提交错误报告；我会继续保证本书适配新的 Anaconda 版本。对于旧环境则无法提供同等支持，我没有时间兼顾每个人不同的配置。`jupyter notebook`

等软件发展很快，我无法跟进所有变化，同时还保持对所有旧版的兼容。

如果其他项目需要旧版软件，Anaconda 允许多个版本并行安装，文档见：

https://conda.io/docs/user-guide/tasks/manage-python.html

整理注：以上版本要求与链接按原文保留，属于原书撰写时的说明，并非对所有当前版本组合的兼容性保证。

安装 FilterPy

FilterPy 是一个 Python 库，实现了本书使用的全部滤波器及许多其他滤波器。可以使用 `pip` 安装，在命令提示符下运行：

```
pip install filterpy
```

FilterPy 由我编写，最新开发版本始终可以在 https://github.com/rlabbe/filterpy 获取。

下载和运行本书

本书存储在 GitHub 仓库中。在命令行输入：

```
git clone --depth=1 https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python.git
```

这会创建名为 Kalman-and-Bayesian-Filters-in-Python 的目录。`depth` 参数只获取最新版本；除非需要完整提交历史，否则这样更快、更节省空间。

没有安装 Git 时，可以在浏览器访问 https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python 并下载本书。

接着，在命令行进入刚创建的目录，再运行 Jupyter Notebook：

```
cd Kalman-and-Bayesian-Filters-in-Python
jupyter notebook
```

浏览器窗口应当打开，显示本书所有章节。点击第一章，再打开相应的笔记本链接。

有关运行 Notebook 的更多信息见：

http://jupyter-notebook-beginner-guide.readthedocs.org/en/latest/execute.html

配套软件

本书专用代码存放在 *./kf_book* 子目录中。这些代码仍在变化，所以暂时不在此详细记录。使用该目录代码时，我会在书中说明，因此不必觉得神秘。

kf_book 里有形如 *xxx*_internal.py 的 Python 文件，保存对某章有用的函数。这样可以隐藏不太值得逐行阅读的辅助代码。例如，生成图表时，我希望你关注图表表达的内容，而非用 Python 绘图的步骤；若感兴趣，可以直接查看源代码。

有些章节引入的函数会在后文继续使用。它们最初在 Notebook 中定义，同时也存入 Python 文件，供后续章节导入。我会在首次定义时说明，但相关整理仍未完成。我尽量避免这种重复，以免目录代码与书中代码不同步。不过，IPython Notebook 没有提供跨笔记本引用代码单元的功能，这是我所知共享功能的唯一机制。

另有一个没有文档的 **experiments** 目录，是我把代码放入书中之前进行编写和测试的地方。里面有有趣的东西，可以自由查看。随着本书发展，我计划创建示例和项目，许多材料会移入其中，小型试验最终可能删除。若只想阅读本书，可以放心忽略它。

./kf_book 中有定义本书外观的 CSS 文件。IPython Notebook 默认外观较朴素，这方面仍在改进。我借鉴了 Probabilistic Programming and Bayesian Methods for Hackers 等书，也深受 Lorena Barba 教授的优秀工作影响：CFDPython。本书的外观得益于这些项目。

使用 Jupyter Notebook

完整的 Jupyter Notebook 教程超出了本书范围，网上有很多可用资料。简而言之，Python 代码放在单元中，前面通常有 `In [1]:` 这样的提示，代码本身位于方框中。当焦点在代码框内时按 CTRL-ENTER，代码会运行，结果显示在下方。例如：

```
print(3+7.2)
```

```
10.2
```

如果现在用 Jupyter Notebook 打开了本书，请修改上面的代码，改变 print 语句里的表达式，再按 CTRL+ENTER。输出应随你输入的代码而变化。

SymPy

SymPy 是执行符号数学运算的 Python 包。它的完整能力超出本书范围，但可以进行代数运算、积分、求导、求解微分方程等等。例如，我们会用它计算矩阵的雅可比，以及期望值的积分。

先看一个简单例子：导入 SymPy，初始化美化输出功能（使用 LaTeX 显示公式），再声明一个供 SymPy 使用的符号。

```
import sympy
sympy.init_printing(use_latex='mathjax')

phi, x = sympy.symbols('\phi, x')
phi
```

ϕ

注意，它如何用 LaTeX 显示符号 phi。现在做一些数学运算：$\sqrt{\phi}$ 的导数是什么?

```
sympy.diff('sqrt(phi)')
```

$\frac{1}{2\sqrt{\phi}}$

可以对表达式进行因式分解：

```
sympy.factor(phi**3 -phi**2 + phi - 1)
```

$(\phi - 1)\left(\phi^2 + 1\right)$

也可以展开表达式。

```
((phi+1)*(phi-4)).expand()
```

$\phi^2 - 3\phi - 4$

可以给变量赋具体数值，计算表达式的值：

```
w =x**2 -3*x +4
print(w.subs(x, 4))
print(w.subs(x, 12))
```

```
8
112
```

也可以使用字符串，表示包含尚未定义符号的表达式：

```
x = sympy.expand('(t+1)*2')
x
```

$2t + 2$

现在用 SymPy 计算矩阵的雅可比。给定函数

$$h = \sqrt{(x^2 + z^2)}$$

求它相对于 x、y、z 的雅可比。

```
x, y, z = sympy.symbols('x y z')

H = sympy.Matrix([sympy.sqrt(x**2 + z**2)])

state = sympy.Matrix([x, y, z])
H.jacobian(state)
```

$\begin{bmatrix} \frac{x}{\sqrt{x^2+z^2}} & 0 & \frac{z}{\sqrt{x^2+z^2}} \end{bmatrix}$

下面根据连续过程噪声矩阵，计算离散过程噪声矩阵 $\mathbf{Q}$。连续矩阵为

$$\mathbf{Q} = \Phi_s \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

积分为

$$\mathbf{Q} = \int_0^{\Delta t} \mathbf{F}\left(t\right)\mathbf{Q}\mathbf{F}^T\left(t\right)\, dt$$

其中

$$\mathbf{F}\left(\Delta t\right) = \begin{bmatrix} 1 & \Delta t & \Delta t^2/2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}$$

```
dt = sympy.symbols('\Delta{t}')
F_k = sympy.Matrix([[1, dt, dt**2/2],
                    [0,  1,      dt],
                    [0,  0,       1]])
Q = sympy.Matrix([[0,0,0],
                  [0,0,0],
                  [0,0,1]])

sympy.integrate(F_k*Q*F_k.T,(dt, 0, dt))
```

$$\begin{bmatrix} \frac{\Delta t^5}{20} & \frac{\Delta t^4}{8} & \frac{\Delta t^3}{6} \\ \frac{\Delta t^4}{8} & \frac{\Delta t^3}{3} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^3}{6} & \frac{\Delta t^2}{2} & \Delta t \end{bmatrix}$$

相关链接

https://ipython.org/

https://jupyter.org/

https://www.scipy.org/

附录 B：符号与记法

下面汇集了不同作者在表示线性卡尔曼滤波方程时所使用的记法。

Labbe

$$
\begin{aligned}
\bar{\mathbf{x}} &= \mathbf{F}\mathbf{x} + \mathbf{B}\mathbf{u} \\
\bar{\mathbf{P}} &= \mathbf{F}\mathbf{P}\mathbf{F}^{\mathsf{T}} + \mathbf{Q} \\
\mathbf{y} &= \mathbf{z} - \mathbf{H}\bar{\mathbf{x}} \\
\mathbf{S} &= \mathbf{H}\bar{\mathbf{P}}\mathbf{H}^{\mathsf{T}} + \mathbf{R} \\
\mathbf{K} &= \bar{\mathbf{P}}\mathbf{H}^{\mathsf{T}}\mathbf{S}^{-1} \\
\mathbf{x} &= \bar{\mathbf{x}} + \mathbf{K}\mathbf{y} \\
\mathbf{P} &= (\mathbf{I} - \mathbf{K}\mathbf{H})\,\bar{\mathbf{P}}
\end{aligned}
$$

维基百科的记法

$$
\begin{aligned}
\widehat{\mathbf{x}}_{k\mid k-1} &= \mathbf{F}_k\widehat{\mathbf{x}}_{k-1\mid k-1} + \mathbf{B}_k\mathbf{u}_k \\
\mathbf{P}_{k\mid k-1} &= \mathbf{F}_k\mathbf{P}_{k-1\mid k-1}\mathbf{F}_k^{\mathsf{T}} + \mathbf{Q}_k \\
\tilde{\mathbf{y}}_k &= \mathbf{z}_k - \mathbf{H}_k\widehat{\mathbf{x}}_{k\mid k-1} \\
\mathbf{S}_k &= \mathbf{H}_k\mathbf{P}_{k\mid k-1}\mathbf{H}_k^{\mathsf{T}} + \mathbf{R}_k \\
\mathbf{K}_k &= \mathbf{P}_{k\mid k-1}\mathbf{H}_k^{\mathsf{T}}\mathbf{S}_k^{-1} \\
\widehat{\mathbf{x}}_{k\mid k} &= \widehat{\mathbf{x}}_{k\mid k-1} + \mathbf{K}_k\tilde{\mathbf{y}}_k \\
\mathbf{P}_{k|k} &= (I - \mathbf{K}_k\mathbf{H}_k)\,\mathbf{P}_{k|k-1}
\end{aligned}
$$

Brookner

$$
\begin{aligned}
X^*_{n+1,n} &= \Phi X^*_{n,n} \\
X^*_{n,n} &= X^*_{n,n-1} + H_n\left(Y_n - MX^*_{n,n-1}\right) \\
H_n &= S^*_{n,n-1}M^{\mathsf{T}}\left[R_n + MS^*_{n,n-1}M^{\mathsf{T}}\right]^{-1} \\
S^*_{n,n-1} &= \Phi S^*_{n-1,n-1}\Phi^{\mathsf{T}} + Q_n \\
S^*_{n-1,n-1} &= (I - H_{n-1}M)\,S^*_{n-1,n-2}
\end{aligned}
$$

Gelb

$$
\begin{aligned}
\widehat{\underline{x}}_k\left(-\right) &= \Phi_{k-1}\widehat{\underline{x}}_{k-1}\left(+\right) \\
\widehat{\underline{x}}_k\left(+\right) &= \widehat{\underline{x}}_k\left(-\right) + K_k\left[Z_k - H_k\widehat{\underline{x}}_k\left(-\right)\right] \\
K_k &= P_k\left(-\right)H_k^{\mathsf{T}}\left[H_kP_k\left(-\right)H_k^{\mathsf{T}} + R_k\right]^{-1} \\
P_k\left(+\right) &= \Phi_{k-1}P_{k-1}\left(+\right)\Phi_{k-1}^{\mathsf{T}} + Q_{k-1} \\
P_k\left(-\right) &= \left(I - K_kH_k\right)P_k\left(-\right)
\end{aligned}
$$

Brown

$$
\begin{aligned}
\widehat{\mathbf{x}}_{k+1}^{-} &= \boldsymbol{\phi}_k\widehat{\mathbf{x}}_k \\
\widehat{\mathbf{x}}_k &= \widehat{\mathbf{x}}_k^{-} + \mathbf{K}_k\left[\mathbf{z}_k - \mathbf{H}_k\widehat{x}_k^{-}\right] \\
\mathbf{K}_k &= \mathbf{P}_k^{-}\mathbf{H}_k^{\mathsf{T}}\left[\mathbf{H}_k\mathbf{P}_k^{-}\mathbf{H}_k^{T} + \mathbf{R}_k\right]^{-1} \\
\mathbf{P}_{k+1}^{-} &= \boldsymbol{\phi}_k\mathbf{P}_k\boldsymbol{\phi}_k^{\mathsf{T}} + \mathbf{Q}_k \\
\mathbf{P}_k &= \left(\mathbf{I} - \mathbf{K}_k\mathbf{H}_k\right)\mathbf{P}_k^{-}
\end{aligned}
$$

Zarchan

$$
\begin{aligned}
\widehat{x}_k &= \Phi_k\widehat{x}_{k-1} + G_ku_{k-1} + K_k\left[z_k - H\Phi_k\widehat{x}_{k-1} - HG_ku_{k-1}\right] \\
M_k &= \Phi_kP_{k-1}\phi_k^{\mathsf{T}} + Q_k \\
K_k &= M_kH^{\mathsf{T}}\left[HM_kH^{\mathsf{T}} + R_k\right]^{-1} \\
P_k &= \left(I - K_kH\right)M_k
\end{aligned}
$$

附录 D：H∞ 滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

我仍在考虑该如何写这一章。在此期间，可以阅读克利夫兰州立大学 Dan Simon 教授的这篇易懂的介绍：

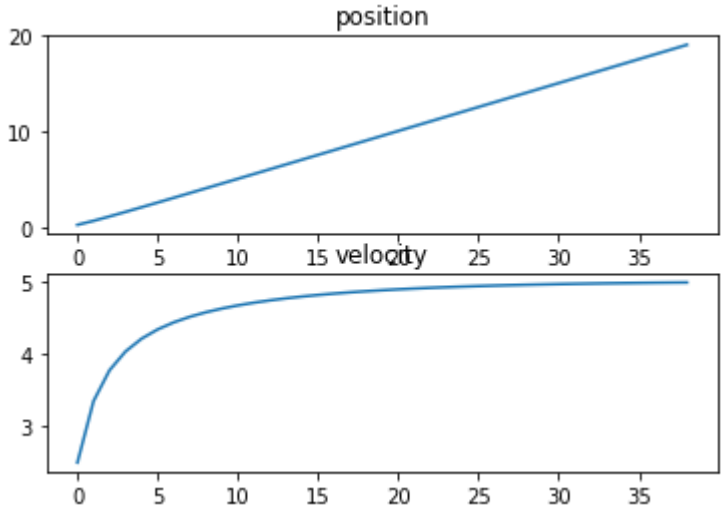
http://academic.csuohio.edu/simond/courses/eec641/hinfinity.pdf

用一句话说，H_∞（H 无穷）滤波器类似卡尔曼滤波器，但在面对非高斯、不可预测的输入时具有鲁棒性。

我的 FilterPy 库包含 H∞ 滤波器。下面粘贴了一些测试代码，实现了 Simon 在上述文章中设计的滤波器，希望对你有帮助。

```
import numpy as np
import matplotlib.pyplot as plt

from filterpy.hinfinity import HInfinityFilter

dt = 0.1
f = HInfinityFilter(2, 1, dim_u=1, gamma=.01)

f.F = np.array([[1., dt],
                [0., 1.]])

f.H = np.array([[0., 1.]])
f.G = np.array([[dt**2 / 2, dt]]).T

f.P = 0.01
f.W = np.array([[0.0003, 0.005],
                [0.0050, 0.100]])/ 1000  #process noise

f.V = 0.01
f.Q = 0.01
u = 1. #acceleration of 1 f/sec**2

xs = []
vs = []

for i in range(1,40):
    f.update (5)
    #print(f.x.T)
    xs.append(f.x[0,0])
    vs.append(f.x[1,0])
    f.predict(u=u)

plt.subplot(211)
plt.plot(xs)
plt.title('position')
plt.subplot(212)
```

```
plt.plot(vs)
plt.title('velocity');
```



附录 E：集合卡尔曼滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

> 我对“集合滤波器”这一概念并不十分了解。为了本书的需要，我尝试实现了一个这样的滤波器，也确实让它能够正常工作。不过，在实际应用中，我从未使用过这种滤波器。不同的资料中，这些滤波器的实现方式略有不同。如果我按照其他资料中的公式来实现滤波器，那么滤波器就无法正常工作。很可能是我哪里做错了。不过，我在网上看到很多人也提到，他们采用了与我相同的做法来让滤波器正常工作。总的来说，我对这个主题了解得不够深入，但我选择坦诚地承认自己的不足，而不是试图掩饰它。我希望将来能掌握这个主题，并写出更详细的介绍。在本章的末尾，我记录下了自己目前的困惑和疑问。无论如何，如果我都被这些资料搞糊涂了，那么你也可能遇到同样的问题。因此，记录下我的困惑有助于你避免犯同样的错误。

集合卡尔曼滤波器与上一章中介绍的无迹卡尔曼滤波器非常相似。如果你还记得的话，无迹卡尔曼滤波器使用了一组经过非线性状态和测量函数处理后的加权西格玛点。这些西格玛点经过处理后，我们会计算出它们的均值和协方差，然后以此作为滤波器的新均值和协方差。当然，这只是一个近似值，因此并非最优解。但实际上，该滤波器的精度相当高。它的优点在于，通常能比卡尔曼滤波器给出更准确的估计结果；同时，也不需要手动推导状态方程和测量方程的线性化表达式。

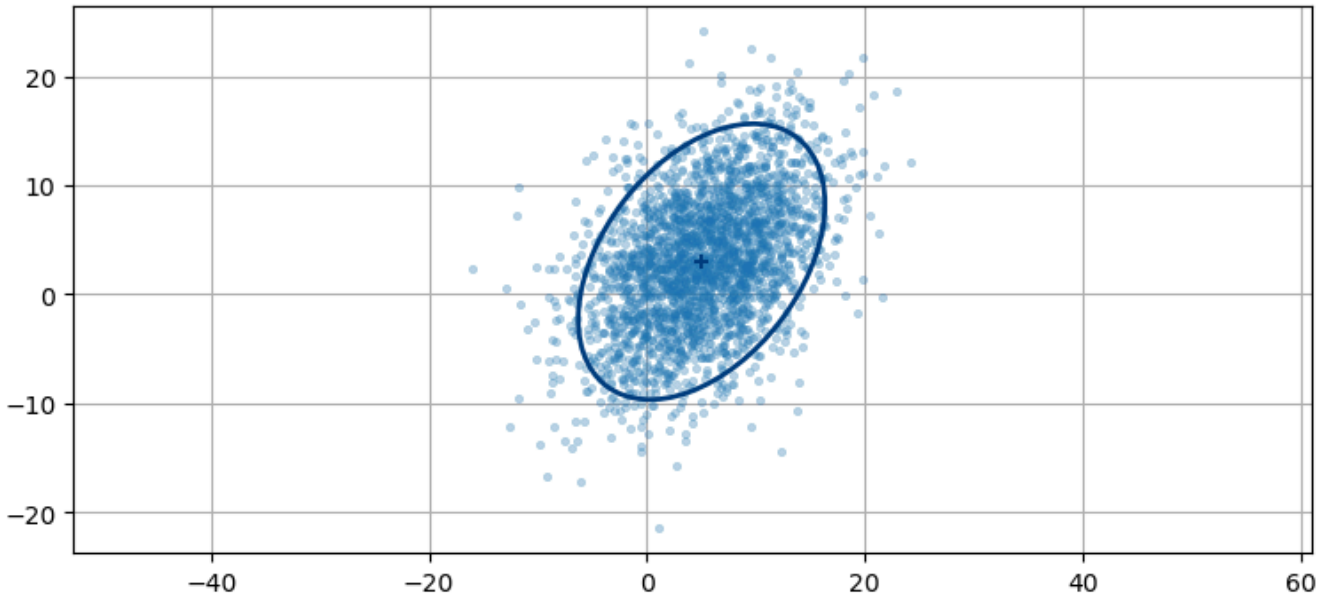
卡尔曼滤波器的工作原理类似，只不过它使用蒙特卡洛方法来选取大量的“西格玛点”。这一方法的起源可以追溯到地球物理学领域，为了解决模拟海洋、大气等系统时所面临的巨大状态和系统规模问题而发明。在《SIAM News》上有一篇关于该技术在天气预报中的应用的有趣文章[1]。该滤波器首先随机生成大量点，这些点分布在滤波器的初始状态周围。这些点的分布与滤波器的协方差有关。换句话说，68%的点会位于平均值的一个标准差范围内，95%的点会位于两个标准差范围内，依此类推。我们以二维情况来说明这一点。我们将使用 `numpy.random.multivariate_normal()` 函数，从均值为(5, 3)、协方差为……的多变量正态分布中随机生成点。

$$\begin{bmatrix} 32 & 15 \\ 15 & 40 \end{bmatrix}$$

我画出了表示两个标准差的协方差椭圆，以此来展示这些点的分布情况。

```
import matplotlib.pyplot as plt
import numpy as np
from numpy.random import multivariate_normal
from filterpy.stats import (covariance_ellipse,
                            plot_covariance_ellipse)
```

```
mean = (5, 3)
P = np.array([[32, 15],
              [15., 40.]])

x,y = multivariate_normal(mean=mean, cov=P, size=2500).T
plt.scatter(x, y, alpha=0.3, marker='.')
plt.axis('equal')

plot_covariance_ellipse(mean=mean, cov=P,
                        variance=2.**2,
                        facecolor='none')
```



该算法

正如我之前所说，当滤波器被初始化时，会从初始状态(x)和协方差矩阵(P)中随机选取大量西格玛点。之后的算法流程与 UKF 非常相似。在预测阶段，这些西格玛点会经过状态转换函数处理，同时会加入少量噪声以考虑过程中的随机误差。在更新阶段，西格玛点会经过测量函数转换到测量空间中，同样会加入少量噪声来弥补测量误差。卡尔曼增益则是根据相关公式计算得出的。

我们之前已经提到了 UKF 和 EnKF 之间的主要区别：UKF 是以确定性方式来选择西格玛点的。上述算法还揭示了另一个区别：在 UKF 中，我们在每次预测时都会生成新的西格玛点；这些点经过非线性函数处理后，再通过无迹变换被转换成均值和协方差。而 EnKF 则继续使用最初生成的西格玛点；我们只需要计算出滤波后的均值和协方差即可。

让我们来看看该滤波器的计算公式。像往常一样，我会省略那些下标和上标；我这里描述的是算法，而非数学函数。其中， N 表示西格玛点的数量， χ 则代表西格玛点的集合。

初始化步骤

$$\chi \sim N(x_0, P_0)$$

这意味着要从滤波器的初始均值和协方差中选取西格玛点。在代码中，可能表现为如下形式：

```
N = 1000
sigmas = multivariate_normal(mean=x, cov=P, size=N)
```

预测步骤

$$\begin{aligned}\chi &= f(\chi, u) + v_Q \\ x &= \frac{1}{N}\sum_1^N \chi\end{aligned}$$

这说法简短明了，但可能不太容易理解。第一行代码是将所有的西格玛点通过一个预先定义的状态转换函数进行处理，然后再根据 Q 矩阵来添加一些噪声。在 Python 中，我们可以这样编写：

```
for i, s in enumerate(sigmas):
    sigmas[i] = fx(x=s, dt=0.1, u=0.)

sigmas += multivariate_normal(x, Q, N)
```

第二行代码用于计算各个“sigma”值的平均值。在 Python 中，我们可以利用 `numpy.mean` 这一语法来简洁高效地完成这一操作。

```
x = np.mean(sigmas, axis=0)
```

我们现在可以选择性地计算均值的协方差。该算法并不一定非得计算这个值，但在进行分析时，这个数值往往很有用。其计算公式为：

$$P = \frac{1}{N-1}\sum_1^N [\chi - x^-][\chi - x^-]^T$$

$\chi - x^-$ 是一个一维向量，因此我们将使用 `numpy.outer` 来计算 $[\chi - x^-][\chi - x^-]^T$ 的值。在 Python 中，我们可以这样编写代码：

```
P = 0
for s in sigmas:
    P += outer(s-x, s-x)
P = P / (N-1)
```

更新步骤

在更新步骤中，我们将这些西格玛点代入测量函数中，计算出这些西格玛点的均值和协方差。接着，根据协方差来计算卡尔曼增益，最后通过用卡尔曼增益来调整残差值，从而更新卡尔曼状态。相关的公式如下：

$$
\begin{aligned}
\chi_h &= h(\chi, u) \\
z_{mean} &= \frac{1}{N}\sum_1^N \chi_h \\
P_{zz} &= \frac{1}{N-1}\sum_1^N [\chi_h - z_{mean}][\chi_h - z_{mean}]^T + R \\
P_{xz} &= \frac{1}{N-1}\sum_1^N [\chi - x^-][\chi_h - z_{mean}]^T \\
K &= P_{xz}P_{zz}^{-1} \\
\chi &= \chi + K[z - \chi_h + v_R] \\
x &= \frac{1}{N}\sum_1^N \chi \\
P &= P - KP_{zz}K^T
\end{aligned}
$$

这与线性 KF 和 UKF 非常相似。我们逐行来分析吧。

第一行，

$$\chi_h = h(\chi, u),$$

只是将西格玛点通过测量函数 h 进行处理而已。我们将处理后的点称为 χ_h，以便与西格玛点区分开来。在 Python 中，我们可以这样表示：

```
sigmas_h = h(sigmas, u)
```

下一行代码用于计算各个测量值σ的平均值。

$$z_{mean} = \frac{1}{N}\sum_1^N \chi_h$$

在 Python 中，我们这样编写：

```
z_mean = np.mean(sigmas_h, axis=0)
```

现在，既然我们已经知道了各个测量值σ的均值，那么就可以计算出每个测量点σ的协方差，以及各个测量点σ之间的交叉协方差。这些可以通过以下两个公式来表示。

$$
\begin{aligned}
P_{zz} &= \frac{1}{N-1}\sum_1^N [\chi_h - z_{mean}][\chi_h - z_{mean}]^T + R \\
P_{xz} &= \frac{1}{N-1}\sum_1^N [\chi - x^-][\chi_h - z_{mean}]^T
\end{aligned}
$$

我们可以用 Python 来表达这个公式。

```
P_zz = 0
for sigma in sigmas_h:
    s = sigma - z_mean
    P_zz += outer(s, s)
```

```
P_zz = P_zz / (N-1) + R

P_xz = 0
for i in range(N):
    P_xz += outer(self.sigmas[i] - self.x, sigmas_h[i] - z_mean)
P_xz /= N-1
```

卡尔曼增益的计算相当简单。

在 Python 中，是这样的。

```
K = P_xz @ inv(P_zz)
```

接下来，我们用新值来更新西格玛点。

$$\chi = \chi + K[z - \chi_h + v_R]$$

这里， v_R 表示我们加到 sigma 值上的扰动量。在 Python 中，我们可以用以下方式来实现这一点：

```
v_r = multivariate_normal([0]*dim_z, R, N)
for i in range(N):
    sigmas[i] += K @ (z + v_r[i] - sigmas_h[i])
```

最后一步是重新计算该滤波器的均值和协方差。

```
    x = np.mean(sigmas, axis=0)
    P = self.P - K @ P_zz @ K.T
```

实施与示例

我在 `FilterPy` 库中实现了 EnKF 算法。不过，从很多方面来看，这只是一个简单的实现而已。使用大量西格玛点进行滤波会导致处理速度极慢。此外，文献中还有许多该算法的细微变体。我编写这个代码主要是出于对这种滤波算法的好奇心。我并没有将其用于解决实际问题，因此也无法就如何将此算法应用于那些需要使用该算法的复杂问题给出建议。因此，我会把我的描述限制在实现一个非常简单的滤波器上。我会用它来跟踪一维空间中的物体，并将结果与线性卡尔曼滤波器的输出进行比较。这种滤波器是我们在这本书中已经多次使用过的，所以我会简单介绍一下其实现方式。我们的状态向量将是……

$$x = \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$$

状态转换函数为

$$F = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

而测量函数则是……

$$H = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

EnKF 算法是为了解决非线性问题而设计的。因此，不需要使用矩阵来表示状态转换和测量函数，而是需要用 Python 函数来实现这些功能。对于这个问题，相应的 Python 函数可以这样编写：

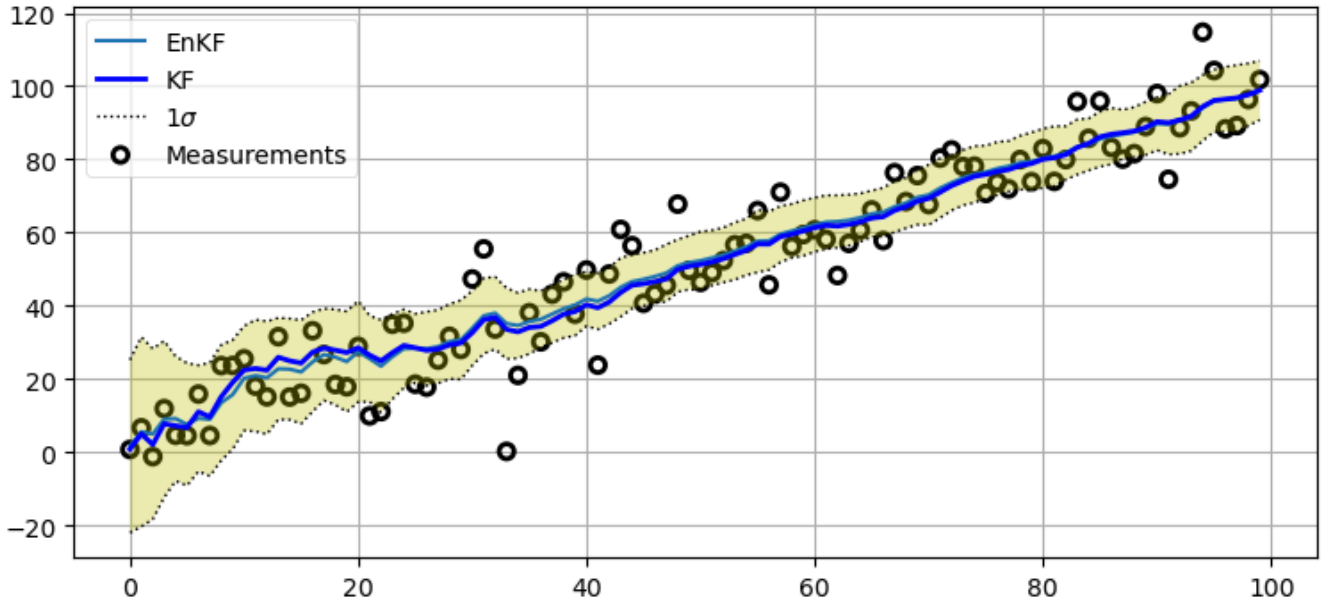
```
def hx(x):
    return np.array([x[0]])

def fx(x, dt):
    return F @ x
```

最后一点：与 UKF 算法类似，EnKF 算法也使用一维数组来表示 x，而不是像线性卡尔曼滤波算法那样使用二维矩阵来表示。

不用多说了，以下就是代码。

```
import kf_book.book_plots as bp
from numpy.random import randn
from filterpy.kalman import EnsembleKalmanFilter as EnKF
from filterpy.kalman import KalmanFilter
from filterpy.common import Q_discrete_white_noise

np.random.seed(1234)

def hx(x):
    return np.array([x[0]])

def fx(x, dt):
    return F @ x

F = np.array([[1., 1.],[0., 1.]])

x = np.array([0., 1.])
P = np.eye(2) * 100.
enf = EnKF(x=x, P=P, dim_z=1, dt=1., N=20, hx=hx, fx=fx)

std_noise = 10.
enf.R *= std_noise**2
enf.Q = Q_discrete_white_noise(2, 1., .001)

kf = KalmanFilter(dim_x=2, dim_z=1)
kf.x = np.array([x]).T
kf.F = F.copy()
kf.P = P.copy()
kf.R = enf.R.copy()
kf.Q = enf.Q.copy()
kf.H = np.array([[1., 0.]])

measurements = []
results = []
ps = []
kf_results = []

zs = []
for t in range (0,100):
    # create measurement = t plus white noise
    z = t + randn()*std_noise
    zs.append(z)

    enf.predict()
    enf.update(np.asarray([z]))
```

```
    kf.predict()
    kf.update(np.asarray([[z]]))

    # save data
    results.append (enf.x[0])
    kf_results.append (kf.x[0,0])
    measurements.append(z)
    ps.append(3*(enf.P[0,0]**.5))

results = np.asarray(results)
ps = np.asarray(ps)

plt.plot(results, label='EnKF')
plt.plot(kf_results, label='KF', c='b', lw=2)
bp.plot_measurements(measurements)
plt.plot (results - ps, c='k',linestyle=':', lw=1, label='1$\sigma$')
plt.plot(results + ps, c='k', linestyle=':', lw=1)
plt.fill_between(range(100), results - ps, results + ps, facecolor='y', alpha=.3)
plt.legend(loc='best');
```



虽然有点难以察觉，但 KF 和 EnKF 在初始阶段的数值略有不同，但很快就会趋近于相同的数值。EnKF 是一种次优的滤波器，因此无法像 KF 那样得到最优解。不过，我故意将 N 的值设得相当小（为 20），以确保 EnKF 的输出结果确实是次优的。如果我把这个数值设为 2000 左右，那么在这张图表上就很难分辨出两种滤波器输出结果之间的差异了。

尚未解决的问题/仍有疑问的问题

所有这些都应该被视为我自己的问题，而不是文献中已有的、悬而未决的问题。不过，我所引用的公式都是直接摘自该领域的权威来源的，而这些公式并没有解释那些不一致之处。

首先，在 Brown[2]中，所有的求和结果都乘以了 $\frac{1}{N}$，就像文中所述那样。

$$\widehat{x} = \frac{1}{N}\sum_{i=1}^{N} \chi_k^{(i)}$$

在 Crassidis 的著作[3]中，同一个方程的表述如下（我会使用与 Brown 相同的符号表示法，不过 Crassidis 使用的符号有所不同）。

$$\hat{x} = \frac{1}{N-1}\sum_{i=1}^{N} \chi_k^{(i)}$$

在计算协方差时，这两个来源中的数值都是如此。Crassidis 在讨论滤波器的协方差时指出，使用 $N-1$ 是为了确保得到的估计结果是无偏的。根据 Crassidis 所给出的关于均值和标准差的标准公式（见第 2 页），这种处理方式在计算协方差时是合理的。

$$\begin{aligned} \mu &= \frac{1}{N}\sum_{i=1}^{N}[\tilde{z}(t_i) - \hat{z}(t_i)] \\ \sigma^2 &= \frac{1}{N-1}\sum_{i=1}^{N}\{[\tilde{z}(t_i) - \hat{z}(t_i)] - \mu\}^2 \end{aligned}$$

不过，我觉得没有理由使用 $N-1$ 来计算平均值。如果在计算平均值的滤波器中使用 $N-1$，那么该滤波器将无法收敛，系统状态将完全取决于原始测量数据，而不会经过任何滤波处理。不过，我觉得可以像 Crassidis 所做的那样，将 $N-1$ 用于计算协方差，而 Brown 则持相反观点。我的决定是基于实证结果得出的：在滤波器的实现过程中，$N-1$ 是有效的，而 N 则无效。

我的第二个问题与 R 矩阵的使用有关。在 Brown 的著作中，R 被加到了 P_{zz} 上，但在 Crassidis 及其他资料中则没有这样的处理方式。我在网上看到其他实现者的笔记，他们指出，加入 R 元素有助于提升滤波效果。在我看来，这种做法确实合理且必要，所以我也是这么做的。

我的第三个问题与协方差 P 的计算方式有关。同样，Crassidis 和 Brown 给出的公式并不相同。我选择了 Brown 的公式，因为其计算结果符合我的预期（即 P 会随着时间的推移而趋于稳定），而且其形式与线性 KF 算法中的公式很相似。相比之下，Crassidis 的公式似乎很难让计算结果趋于稳定。

我的第四个问题与状态估计的更新有关。在 Brown 的模型中，我们有……

$$\chi = \chi + K[z - z_{mean} + v_R]$$

而在 Crassidis 中，情况则有所不同。

$$\chi = \chi + K[z - \chi_h + v_R]$$

在我看来，Crassidis 方程相当合理。它所生成的滤波器，其性能与用于线性问题的线性 KF 滤波器类似。因此，我选择了这种公式。

我不敢断言这两本书中的哪一本是错的；很有可能是我忽略了某些能让这些方程组成立的要点。可以说，按照书中的描述来实现这些方程组时，我并没有得到一个能够正常工作的滤波器。我所说的“正常工作”，指的是其功能应与处理线性问题的线性 KF 算法基本相同。在阅读了网上的实现说明并仔细分析了各种问题之后，我选择了本章中所介绍的实现方式，而这种方式确实似乎能够正常工作。不过，我还需要进一步研究大量的原始文献，这些文献或许能最终解释这些差异。即便我找到了能解释这些差异的解释，我还是想把这里的内容保留下来。因为如果连我都被这些书搞糊涂了，那么其他人很可能也会如此。

参考文献

- [1] 麦肯齐，达娜。《集合卡尔曼滤波器有助于使天气模型保持最新状态》，《Siam News》，第 36 卷，第 8 期，2003 年 10 月。http://www.siam.org/pdf/news/362.pdf
- [2] 罗伯特·格罗弗·布朗、黄培基。《随机信号与卡尔曼滤波应用导论》，附 MATLAB®练习题及解答。威利出版社，2012 年。
- [3] Crassidis, John L.和 John L. Junkins. 动态系统的最优估计方法. CRC 出版社, 2011 年.

附录 G：非线性卡尔曼滤波器的设计

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

作者注：我原本打算专门设一章来比较各种不同的设计方法。不过这一章目前还没有什么有用的内容，建议直接跳过。

我们可以看到，卡尔曼滤波器在追踪球的位置方面表现得相当不错。不过，正如之前所解释的，这其实是个不恰当的例子：在真空中，我们可以以任意高的精度来预测物体的轨迹。因此，在这个例子中使用卡尔曼滤波器其实是没有必要的。

考虑空气阻力的卡尔曼滤波器

我将跳过那些按步骤来处理的流程，采用一种更自然的方式来进行分析，就像在处理非玩具类工程问题时所使用的那种方式。我们已经开发出了一个卡尔曼滤波器，它在追踪真空环境中的球体时表现得相当出色。不过，该滤波器并未将空气阻力的影响纳入模型中。既然该过程模型是用 F 来实现的，那我们就先从这方面入手吧。

从理论上讲， F 所进行的计算其实是……

$$x^{'} = Fx$$

在没有空气阻力的情况下，我们取得了……

$$F = \begin{bmatrix} 1 & \Delta t & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & \Delta t & \frac{1}{2}\Delta t^2 \\ 0 & 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

这对应于相应的方程式。

$$\begin{aligned} x &= x + v_x\Delta t \\ v_x &= v_x \end{aligned}$$

$$\begin{aligned} y &= y + v_y\Delta t + \frac{a_y}{2}\Delta t^2 \\ v_y &= v_y + a_y\Delta t \\ a_y &= a_y \end{aligned}$$

从上面的分析可知，我们新的欧拉方程必定是……

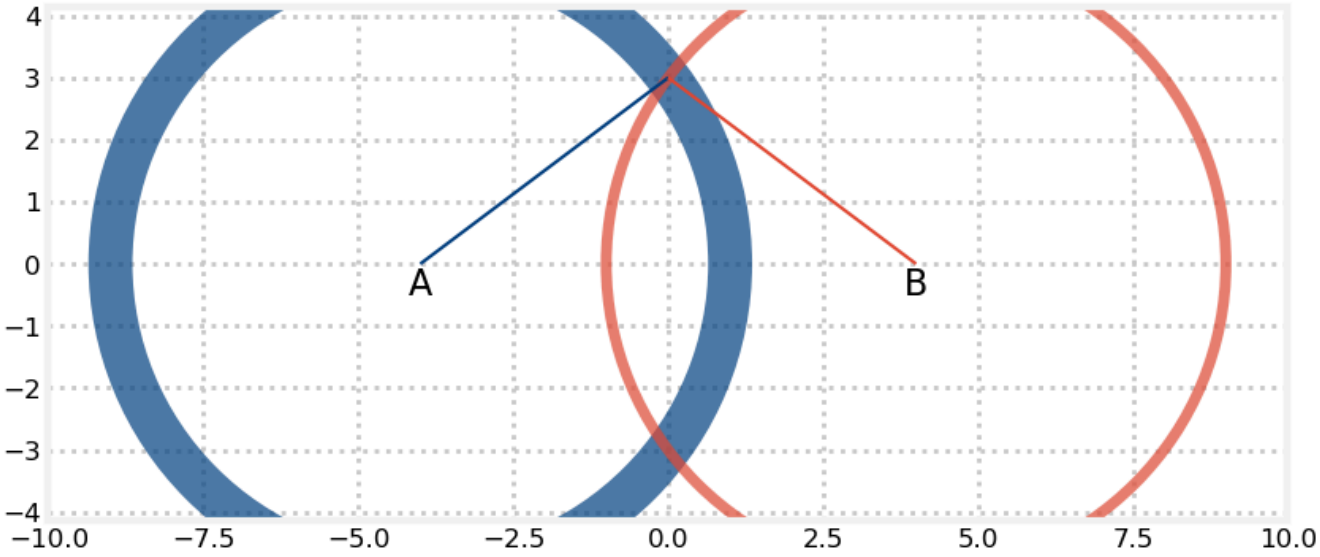
$$
\begin{aligned}
x &= x + v_x\Delta t \\
v_x &= v_x \\
y &= y + v_y\Delta t + \frac{a_y}{2}\Delta t^2 \\
v_y &= v_y + a_y\Delta t \\
a_y &= a_y
\end{aligned}
$$

实用的 2D 位置传感器

在最后一个例子中，所使用的位置传感器并不十分现实。通常情况下，不存在能够直接提供(x, y)坐标的“原始”传感器。虽然我们有 GPS，但 GPS 本身已经使用了卡尔曼滤波器来处理数据并生成输出结果。除非我们添加额外的传感器来提供更多信息，否则再通过另一个卡尔曼滤波器来处理信号是无法提升信号质量的。我们稍后会再讨论这个问题。

请想象以下场景：在一片开阔的场地上，我们把两个发射器放置在与已知位置相对应的地方。每个发射器都会发出可以被我们检测到的信号。我们对这些信号进行处理，从而确定自己与这些信号源之间的距离。当然，这一过程中会存在一定的误差。首先，让我们通过图表来直观地了解这一过程。

```
import matplotlib.pyplot as plt

circle1=plt.Circle((-4, 0), 5, color='#004080',
                   fill=False, linewidth=20, alpha=.7)
circle2=plt.Circle((4, 0), 5, color='#E24A33',
                   fill=False, linewidth=5, alpha=.7)

fig = plt.gcf()
ax = fig.gca()

plt.axis('equal')
plt.xlim((-10, 10))
plt.ylim((-10, 10))

plt.plot ([-4, 0], [0, 3], c='#004080')
plt.plot ([4, 0], [0, 3], c='#E24A33')
plt.text(-4, -.5, "A", fontsize=16, horizontalalignment='center')
plt.text(4, -.5, "B", fontsize=16, horizontalalignment='center')

ax.add_artist(circle1)
ax.add_artist(circle2)
plt.show()
```



在这里，我将发射器 A 用红色标出，其位置为(-4,0)；发射器 B 则用蓝色标出，其位置为(4,0)。红色和蓝色的圆圈分别表示了各发射器到机器人的距离，圆圈的半径则反映了每个发射器的角度误差对测量结果的影响。图中，蓝色发射器的角度误差比红色发射器更大。机器人的最可能位置就是这两个圆圈的交点，我用红色和蓝色的线条标出了该位置。你可能会觉得有两个交点，而不是一个，不过在设计测量函数时，我们会找到解决这个问题的方法。

这是一种非常常见的传感器配置。飞机至今仍使用这种系统来进行导航，这种系统被称为 DME（距离测量设备）。不过，如今 GPS 已经成为了更常见的导航系统。我曾经参与过一项项目，当时我们将这种传感器与 GPS、惯性导航系统、高度计等设备一起整合到了导航系统中。关于多传感器融合的技术，我们稍后会详细讨论；现在，我们就先来了解这种简单的传感器配置吧。

第一步是确定状态变量的具体数值。我们假设机器人以恒定速度沿直线运动。当然，这种假设在长时间内不太可能成立，但在短时间内是可以接受的。这与之前的问题没什么不同——我们仍然需要跟踪机器人的位置和速度信息。因此，

$$x = \begin{bmatrix} x \\ v_x \\ y \\ v_y \end{bmatrix}$$

下一步是设计状态转换函数。这与之前的问题类似，因此无需多言。

$$x^{'} = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} x$$

下一步是设计控制输入。由于我们目前没有可用的控制输入，因此将 $B = 0$ 设置为默认值。

下一步是设计测量函数 $z = Hx$ 。我们可以利用勾股定理来对这一测量过程进行建模。

$$z_a = \sqrt{(x - x_A)^2 + (y - y_A)^2} + v_a z_b = \sqrt{(x - x_B])^2 + (y - y_B)^2} + v_b$$

其中， v_a 和 v_b 属于白噪声。

我们面临着一个紧迫的问题：卡尔曼滤波器是为处理线性方程而设计的，而当前的问题显然属于非线性问题。在接下来的章节中，我们将探讨几种处理非线性问题的有效方法。不过目前，我们可以采取一种更简单的方法：如果我们知道机器人的大致位置，那么就可以以该点为基准，将相关方程线性化处理。虽然我现在可以详细阐述这种方法的数学原理，但在这里，我先通过一个具体的例子来说明这一方法的运用方式。

我们不会去计算 H 的值，而是会计算 H 关于机器人位置 x 的偏导数。你可能已经熟悉偏导数的概念了，如果不熟悉的话，简单来说，偏导数就是表示 H 如何随着机器人位置的变化而变化。其计算方式如下：对 H 进行偏导数运算即可。

$$\frac{\partial h}{\partial x} = \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \frac{\partial h_1}{\partial x_2} & \cdots \\ \frac{\partial h_2}{\partial x_1} & \frac{\partial h_2}{\partial x_2} & \cdots \\ \vdots & \vdots & \end{bmatrix}$$

让我们先来计算第一个偏导数。我们需要找出的就是这个偏导数。

$$\frac{\partial}{\partial x}\sqrt{(x-x_A)^2 + (y-y_A)^2}$$

我们将其计算为：

$$\begin{aligned} \frac{\partial h_1}{\partial x} &= ((x-x_A)^2 + (y-y_A)^2))^{\frac{1}{2}} \\ &= \frac{1}{2} \times 2(x-x_a) \times ((x-x_A)^2 + (y-y_A)^2))^{-\frac{1}{2}} \\ &= \frac{x_r - x_A}{\sqrt{(x_r-x_A)^2 + (y_r-y_A)^2}} \end{aligned}$$

我们继续对这两个距离方程关于 x 、 y 、 dx 和 dy 的偏导数进行计算，最终得到以下结果：

$$\frac{\partial h}{\partial x} = \begin{bmatrix} \frac{x_r - x_A}{\sqrt{(x_r-x_A)^2 + (y_r-y_A)^2}} & 0 & \frac{y_r - y_A}{\sqrt{(x_r-x_A)^2 + (y_r-y_A)^2}} & 0 \\ \frac{x_r - x_B}{\sqrt{(x_r-x_B)^2 + (y_r-y_B)^2}} & 0 & \frac{y_r - y_B}{\sqrt{(x_r-x_B)^2 + (y_r-y_B)^2}} & 0 \end{bmatrix}$$

那确实很麻烦，而这些其实都是非常简单的方程式而已。对于更复杂的系统来说，计算雅可比矩阵可能极其困难，甚至无法实现。不过，利用 SymPy 模块，我们可以轻松地让 Python 来完成这项工作[1]。SymPy 是一个用于符号数学的 Python 库。它的功能非常强大，超出了本书的讨论范围。不过，它能够进行代数运算、积分和微分计算、求解微分方程等等。我们将利用它来计算雅可比矩阵！

首先，来看一个简单的例子。我们将导入 SymPy 库，并启用其漂亮的输出格式功能（该功能会使用 LaTeX 来格式化方程式）。接着，我们会为 NumPy 声明一个符号，以便后续使用。

```
import sympy
from sympy import init_printing
init_printing(use_latex='mathjax')

phi, x = sympy.symbols('\phi, x')
phi
```

ϕ

请注意，我们使用了 LaTeX 公式来表示符号 `phi` 。其实这并非必要，但这样做的话，输出结果就会以 LaTeX 格式呈现。现在我们来算点数学题吧。 $\sqrt{\phi}$ 的导数是多少呢?

```
sympy.diff('sqrt(phi)')
```

$\frac{1}{2\sqrt{\phi}}$

我们可以对方程进行因式分解。

```
sympy.factor('phi**3 -phi**2 + phi - 1')
```

$(\phi - 1)\left(\phi^2 + 1\right)$

SymPy 拥有许多出色的功能。虽然我很乐意逐一试用这些功能，但在这里我们无法一一详述。不过，我们可以先来计算一下雅可比矩阵吧。

```
from sympy import symbols, Matrix
phi = symbols('\phi')
phi

x, y, xa, xb, ya, yb, dx, dy = symbols('x y x_a x_b y_a y_b dx dy')

H = Matrix([[sympy.sqrt((x-xa)**2 + (y-ya)**2)],
            [sympy.sqrt((x-xb)**2 + (y-yb)**2)]])

state = Matrix([x, dx, y, dy])
H.jacobian(state)
```

$$\begin{bmatrix} \frac{x - x_a}{\sqrt{(x-x_a)^2 + (y-y_a)^2}} & 0 & \frac{y - y_a}{\sqrt{(x-x_a)^2 + (y-y_a)^2}} & 0 \\ \frac{x - x_b}{\sqrt{(x-x_b)^2 + (y-y_b)^2}} & 0 & \frac{y - y_b}{\sqrt{(x-x_b)^2 + (y-y_b)^2}} & 0 \end{bmatrix}$$

简而言之，(0,0)这个坐标点所代表的数值，是机器人所在位置的 x 坐标与发射器 A 的 x 坐标之差，再除以机器人与发射器 A 之间的距离。对于(2,0)来说，其数值的计算方式相同，只不过涉及到的是机器人和发射器 B 的 y 坐标。最下面一行则同样按照上述方式计算，只不过此时涉及的是发射器 B 的数据。值为 0 的坐标点则代表了状态变量中的速度分量；当然，该测量结果本身并不直接反映机器人的实际速度。

随着机器人位置的改变，该矩阵中的数值也会随之变化。因此，这个数值并非恒定不变的；我们必须在滤波器的每个时间步长中重新计算该数值。

如果你仔细观察，就会发现这其实只是对 x/dist 和 y/dist 的计算而已。因此，我们可以将其转换为三角函数形式，而不会失去其普遍性。

$$\frac{\partial h}{\partial x} = \begin{bmatrix} -\cos\theta_A & 0 & -\sin\theta_A & 0 \\ -\cos\theta_B & 0 & -\sin\theta_B & 0 \end{bmatrix}$$

不过，这就引出了一个巨大的问题。我们现在计算的不是 H ，而是 ΔH ，也就是 H 的变化量。如果我们在不改变卡尔曼滤波器其他设计参数的情况下直接将这个变化量输入到滤波器中，那么输出结果将毫无意义。例

如，我们之前是通过将 Hx 与某个数值相乘来得到基于 x 的估计值所对应的测量结果。但现在，由于 H 已经根据我们的当前位置进行了线性化处理，它实际上包含了测量函数中的变化量。

因此，我们不得不使用 x 的变化来作为状态变量的数值。这样一来，我们就需要重新设计状态变量了。

> 请注意，这在卡尔曼滤波器的设计过程中是再正常不过的现象。教科书里通常把这类例子当作理所当然的事情来介绍，好像“状态变量应该是速度而非位置”这种结论是显而易见的。也许，当你做了足够多的这类题目后，这一点确实会变得显而易见。但到那时，你还要读教科书干什么呢？我自己经常反复阅读各种讲解资料，试图理解为什么作者会做出那样的选择。最终我发现，这些选择都是因为后文中的某些因素所导致的。我的讲解可能比较冗长，但它反映了实际设计滤波器时的真实情况：你会做出看似合理的决策，但随着设计的深入，你会发现一些新的情况，从而不得不重新考虑之前的决策。因此，我打算放弃那种按步骤来讲解的方法，因为现实中的问题往往没那么简单。

如果我们的状态变量只包含机器人的速度信息，而不包含位置信息，那我们该如何确定机器人的具体位置呢？根本无法确定。以这种方式进行线性化的卡尔曼滤波器，实际上依赖于所谓的“标称轨迹”——也就是说，先假设一个初始位置和运动方向，然后再根据速度和加速度的变化来计算轨迹的变动。还能有别的办法吗？回想一下那个展示两个测距圆相交情况的图形：有两个交点。试想一下，如果两个发射器之间的距离非常近，那么交点就会变成两条很长的月牙形。按照设计，这种卡尔曼滤波器无法仅通过测量到发射器的距离来确定机器人的真实位置。也许你已经在思考解决这个问题的方法了。如果是这样，请继续阅读，因为后面的章节会详细介绍这些解决方案。在我看来，试图一次性呈现完整的解决方案，只会带来更多的混乱，而无法带来真正的洞察力。

那么，让我们重新设计一下状态转换函数吧。我们假设物体处于恒定速度运动状态，没有加速度，因此状态方程可以表示为:

这就得到了该系统的状态转换函数。

$$F = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

最后一个需要考虑的复杂因素，与我们输入的测量数据有关。 Hx 目前正在计算从标称位置出发时的测量值变化。因此，我们输入的测量数据不应是 A 点和 B 点之间的距离，而应是从实际测量得到的距离到标称位置的距离变化。

这里有太多需要理解的内容了，所以我们还是逐行仔细分析代码吧。首先，我们需要定义一个函数，用来计算每个时间步长对应的 $\frac{\partial h}{\partial x}$ 值。

```
from math import sin, cos, atan2

def H_of(pos, pos_A, pos_B):
    """ Given the position of our object at 'pos' in 2D, and two
    transmitters A and B at positions 'pos_A' and 'pos_B', return
    the partial derivative of H
    """
```

```
    theta_a = atan2(pos_a[1] - pos[1], pos_a[0] - pos[0])
    theta_b = atan2(pos_b[1] - pos[1], pos_b[0] - pos[0])

    return np.array([[0, -cos(theta_a), 0, -sin(theta_a)],
                     [0, -cos(theta_b), 0, -sin(theta_b)]])
```

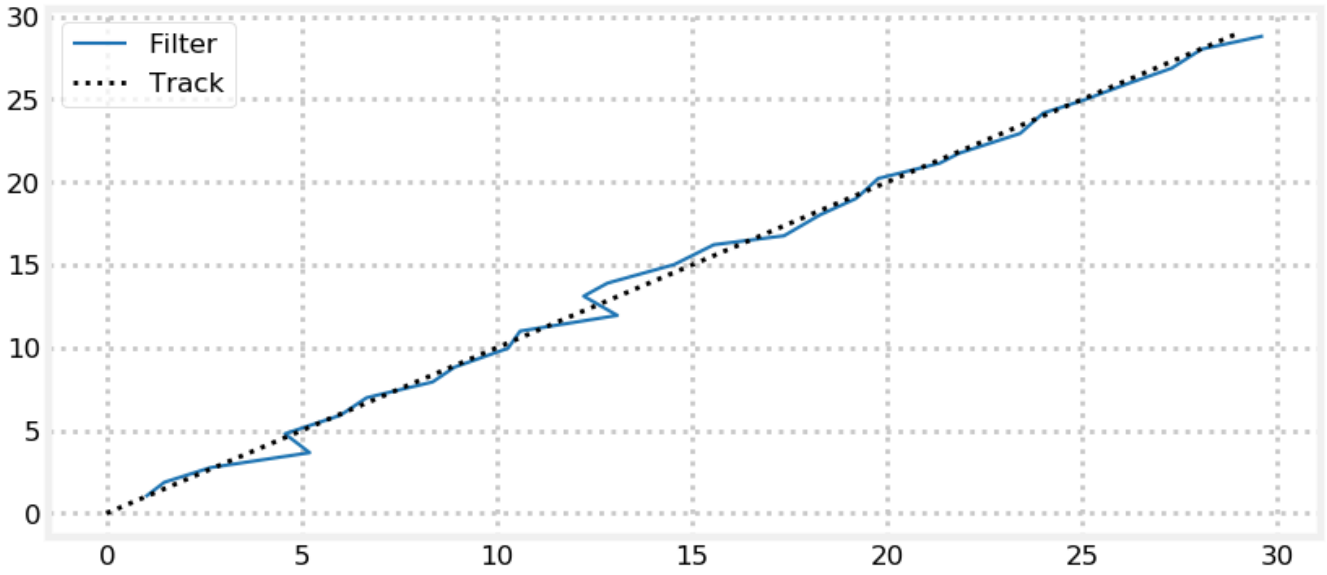
现在，我们需要创建这个模拟传感器。

```
from numpy.random import randn

class DMESensor(object):
    def __init__(self, pos_a, pos_b, noise_factor=1.0):
        self.A = pos_a
        self.B = pos_b
        self.noise_factor = noise_factor

    def range_of(self, pos):
        """ returns tuple containing noisy range data to A and B
        given a position 'pos'
        """

        ra = math.sqrt((self.A[0] - pos[0])**2 + (self.A[1] - pos[1])**2)
        rb = math.sqrt((self.B[0] - pos[0])**2 + (self.B[1] - pos[1])**2)

        return (ra + randn()*self.noise_factor,
                rb + randn()*self.noise_factor)
```

最后，我们终于可以开始编写卡尔曼滤波器的代码了。我会把发射器分别放置在 x=-100 和 x=100 的位置上，两者的 y 坐标都为-20。这样，当机器人移动时，就能从这两个发射器那里获得足够精确的定位信息。我会让机器人从(0,0)点开始移动，每个时间步长内，机器人会向右移动 1 个单位，向上移动 1 个单位。

```
import kf_book.book_plots as bp
from filterpy.kalman import KalmanFilter
import math
import numpy as np

pos_a = (100, -20)
pos_b = (-100, -20)

f1 = KalmanFilter(dim_x=4, dim_z=2)

f1.F = np.array ([[0, 1, 0, 0],
                  [0, 0, 0, 0],
                  [0, 0, 0, 1],
                  [0, 0, 0, 0]], dtype=float)

f1.R *= 1.
f1.Q *= .1

f1.x = np.array([[1, 0, 1, 0]], dtype=float).T
f1.P = np.eye(4) * 5.

# initialize storage and other variables for the run
count = 30
xs, ys = [], []
pxs, pys = [], []

# create the simulated sensor
d = DMESensor(pos_a, pos_b, noise_factor=3.)
```

```
# pos will contain our nominal position since the filter does not
# maintain position.
pos = [0, 0]

for i in range(count):
    # move (1,1) each step, so just use i
    pos = [i, i]

    # compute the difference in range between the nominal track
    # and measured ranges
    ra,rb = d.range_of(pos)
    rx,ry = d.range_of((pos[0] + f1.x[0, 0], pos[1] + f1.x[2, 0]))
    z = np.array([[ra - rx], [rb - ry]])

    # compute linearized H for this time step
    f1.H = H_of (pos, pos_a, pos_b)

    # store stuff so we can plot it later
    xs.append(f1.x[0, 0]+i)
    ys.append(f1.x[2, 0]+i)
    pxs.append(pos[0])
    pys.append(pos[1])

    # perform the Kalman filter steps
    f1.predict()
    f1.update(z)

bp.plot_filter(xs, ys)
bp.plot_track(pxs, pys)
plt.legend(loc=2)
plt.show()
```



线性化卡尔曼滤波器

既然我们已经了解了将卡尔曼滤波器线性化的具体示例，现在我们就能更深入地理解其中的数学原理了。

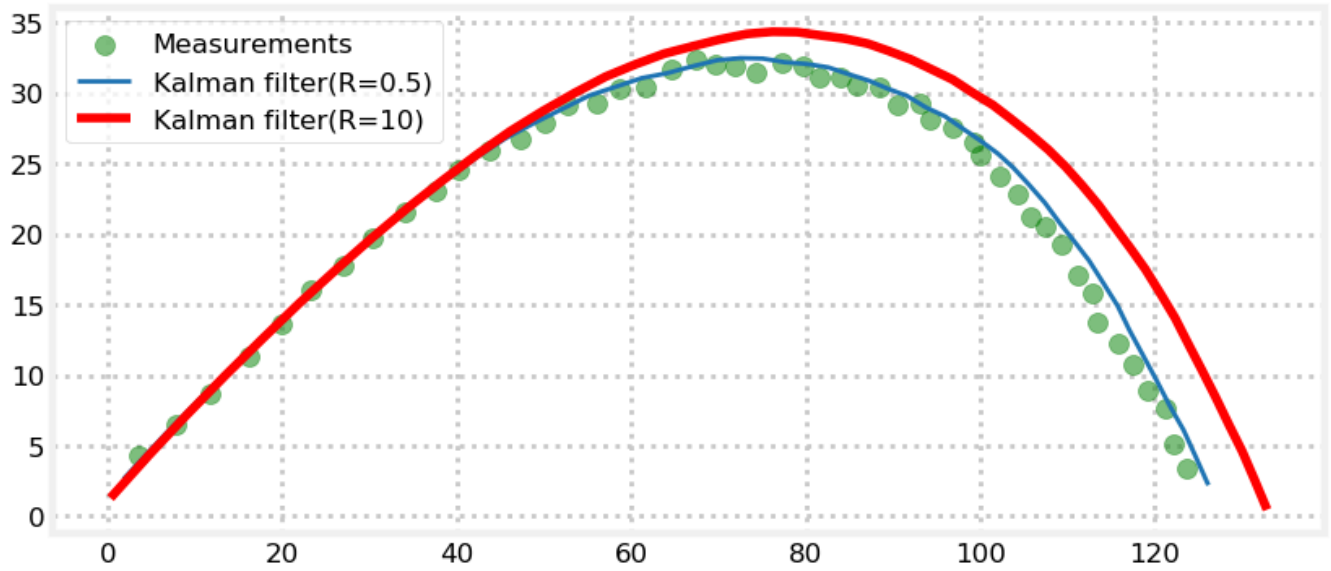
我们首先假设某个函数 f。

示例：下落的球体

作者注：暂时忽略此部分。

在“卡尔曼滤波器的设计”这一章中，我首先研究了在真空中追踪球体的问题，随后又研究了在空气中的情况。在真空中，卡尔曼滤波器的表现相当出色；但在空气中，其预测结果与球体的实际运动轨迹出现了偏差。为了避免让本章内容过于杂乱，我把与代码相关的内容都放在了`ekf_internal.py`文件中。

```
import kf_book.ekf_internal as ekf
ekf.plot_ball()
```



我们可以通过将 Q 的值设置得很大，从而强制卡尔曼滤波器去跟踪球的位置。这样一来，滤波器会怀疑自己的预测结果，进而调整卡尔曼增益 K 的数值，使其更倾向于信任实际的测量数据。不过，这种做法并不妥当。如果卡尔曼滤波器能够正确地预测过程状态，我们就没必要通过谎报“存在实际并不存在的误差”来“欺骗”滤波器。在某些情况下，这种做法或许可行，但总体而言，卡尔曼滤波器的性能将会受到影响。

回想一下在“卡尔曼滤波器的设计”这一章中提到的内容：加速度为

$$a_x = (0.0039 + \frac{0.0058}{1+\exp[(v-35)/5]})*v*v_x a_y = (0.0039 + \frac{0.0058}{1+\exp[(v-35)/5]})*v*v_y - g$$

在研究这个课题的过程中，这些方程式会让人觉得非常难以处理。因此，目前我先采用一个更简单的一维模型来进行分析。在这个模型中，我使用了经过简化的加速度公式，该公式没有考虑阻力系数的非线性特性。

$$\begin{aligned}\ddot{y} &= \frac{0.0034ge^{-y/20000}\dot{y}^2}{2\beta} - g \\ \ddot{x} &= \frac{0.0034ge^{-x/20000}\dot{x}^2}{2\beta}\end{aligned}$$

这里，β 表示弹道系数。数值越大，表示空气阻力越小。

这仍然是一个非线性方程，因此我们需要在当前状态点将这个方程线性化。如果我们的状态是位置和速度，那么我们需要一个描述 x 发生微小变化的方程，具体如下：

$$\begin{bmatrix} \Delta\dot{x} \\ \Delta\ddot{x} \\ \Delta\dot{y} \\ \Delta\ddot{y} \end{bmatrix} = \begin{bmatrix} \frac{\partial\dot{x}}{\partial x} & \frac{\partial\dot{x}}{\partial\dot{x}} & \frac{\partial\dot{x}}{\partial y} & \frac{\partial\dot{x}}{\partial\dot{y}} \\ \frac{\partial\ddot{x}}{\partial x} & \frac{\partial\ddot{x}}{\partial\dot{x}} & \frac{\partial\ddot{x}}{\partial y} & \frac{\partial\ddot{x}}{\partial\dot{y}} \\ \frac{\partial\dot{y}}{\partial x} & \frac{\partial\dot{y}}{\partial\dot{x}} & \frac{\partial\dot{y}}{\partial y} & \frac{\partial\dot{y}}{\partial\dot{y}} \\ \frac{\partial\ddot{y}}{\partial x} & \frac{\partial\ddot{y}}{\partial\dot{x}} & \frac{\partial\ddot{y}}{\partial y} & \frac{\partial\ddot{y}}{\partial\dot{y}} \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta\dot{x} \\ \Delta\dot{y} \\ \Delta\ddot{y} \end{bmatrix}$$

这些方程式中不可能同时包含 x 和 y 的项。因此，任何同时包含 x 和 y 的偏导数都必定等于零。我们还知道 $\frac{\partial\dot{x}}{\partial x} = 0$ 和 $\frac{\partial\dot{x}}{\partial\dot{x}} = 1$ 的值，所以最终的矩阵形式如下：

$$F = \begin{bmatrix} 0 & 1 & 0 & 0 \\ \frac{0.0034e^{-x/22000}\dot{x}^2 g}{44000\beta} & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{aligned} \ddot{x} &= -\frac{1}{2}C_d\rho A\dot{x} \\ \ddot{y} &= -\frac{1}{2}C_d\rho A\dot{y} - g \end{aligned}$$

```
from sympy.abc import *
from sympy import *

init_printing(pretty_print=True, use_latex='mathjax')

x1 = (0.0034*g*exp(-x/22000)*((x)**2))/(2*b) - g

x2 = (a*g*exp(-x/c)*(Derivative(x)**2))/(2*b) - g

#pprint(x1)
#pprint(Derivative(x)*Derivative(x,n=2))
#pprint(diff(x2, x))
```

这种方法的缺点很多。首先，线性化处理无法得到精确的结果。更重要的是，我们实际上并不是对真实的路径进行线性化处理，而是对滤波器所估算出的路径进行线性化处理。我们之所以对估算结果进行线性化处理，是因为从统计学角度来看，这种处理方式很可能是正确的；但实际上，这种处理方式并不一定正确。因此，如果滤波器的输出结果有误，那么我们得到的线性化结果自然也是错误的，进而会导致更糟糕的估算结果。在这种情况下，滤波器会很快失去稳定性。这就是卡尔曼滤波器所面临的“难题”所在。我们试图对估算结果进行线性化处理，但无法保证滤波器一定会保持稳定。关于卡尔曼滤波器的大量研究都集中在这个问题上。另一个问题是，我们必须用解析方法来对系统进行线性化处理。对于某些问题来说，可能很难或根本无法找到解析解。在其他情况下，我们或许能够找到相应的线性化表达式，但这样的计算过程极其复杂且耗时。**

参考文献

[1] http://sympy.org

附录 H：最小二乘滤波器

```
%matplotlib inline
```

```
#format the book
import book_format
book_format.set_style()
```

引言

作者注：这些内容从 g-h 滤波一章中移出，因为不适合放在那里。这一章还不适合阅读——我还没有把它写完。

在那一章开头附近，我用 `numpy.polyfit()` 对体重测量数据拟合了一条直线。它通过“最小二乘拟合”，用 n 次多项式拟合数据。这与 g-h 滤波器有什么不同？

这要视具体情况而定。我们最终会学到，在某些条件下，卡尔曼滤波从最小二乘拟合的角度看是最优的。不过，`polyfit()` 拟合的是多项式，而非任意曲线，方法是最小化下式：

$$E = \sum_{j=0}^{k} \left| p\left(x_j\right) - y_j \right|^2$$

我假定自己的体重每天恒定增加 1 磅。因此，使用 $n = 1$ 的多项式，也就是直线，拟合结果与真实增重非常接近。但当然，没有人会一直只增重或只减重，我们的体重会波动。对较长的数据序列使用 `polyfit()`，结果会很差。相反，g-h 滤波器能应对变化率的改变，其中 h 项控制滤波器响应这些变化的速度。如果我们先增重，稳定一段时间，再减重，滤波器会自动跟踪变化。除非这种增减能很好地用多项式表示，否则 `polyfit()` 做不到这一点。

即便数据能用 n 次多项式拟合，这类滤波器还有一个优点：它是*递归*的。也就是说，只需知道上一时刻的估计和变化率，就能计算本时刻的估计。相反，查看 `polyfit()` 的实现会发现，它需要取得全部数据后才能给出答案。因此，`polyfit()` 这样的算法不适合实时数据滤波。20 世纪 60 年代卡尔曼滤波问世时，计算机速度很慢，内存极其有限，根本无法存储例如飞机惯性导航系统的数千次读数，也不能在为导航提供准确、及时信息所要求的短时间内处理这些数据。

直到 20 世纪中叶，这类滤波主要采用各种形式的最小二乘估计。例如，美国国家航空航天局的阿波罗计划使用地面网络跟踪指令与服务舱（CSM）和登月舱（LM）。地面系统在许多分钟内收集测量值，将数据成批汇总，再缓慢计算答案。1960 年，NASA 艾姆斯研究中心的 Stanley Schmidt 认识到鲁道夫·卡尔曼那篇开创性论文的价值，邀请他到艾姆斯。Schmidt 把卡尔曼的工作应用于 CSM 和 LM 的机载导航系统，并将其称为“卡尔曼滤波器”。[1] 此后，人们很快转向这种更快的递归滤波器。

卡尔曼滤波只需存储上一次估计和少量相关参数，便能用相对较少的计算生成下一次估计。今天的内存和处理能力非常充裕，这个优势不像当时那么关键。但在那个年代，卡尔曼滤波不仅因其数学性质而成为重大突破，

还因为它能勉强在当时的硬件上运行。

这一主题远比这段简短讨论所呈现的深刻。全书后面还会多次讨论这些问题。

附录 I：性能的解析评估

整理注：所选版本的原书在此仅有标题，没有正文。本版如实保留，不补写作者尚未提供的内容。

```
#format the book
%matplotlib inline
from book_format import load_style
load_style()
```